

海思网络产品管理部

ASIC 工程师手册

文档版本 1.1

发布日期 2013-12-27

序 言

2007 年 2 月，ASIC 工程师手册编写组正式成立。编写组主要由网络产品管理部的设计、验证、后端、资料的骨干组成，他们一方面投身于项目开发中，一方面抽出时间，吸收各领域专家已经凝聚的理论知识，可谓实践与理论兼得，为工程师手册奠定了良好的基础。

编写组成立之初，就明确了《ASIC 工程师手册》的宗旨：

- 帮助初级工程师掌握基本概念和工作技巧
- 帮助工程师扩大知识面
- 通过搜集一部分案例，扩大共享，避免错误频发
- 通过搜集一部分工作方法、技巧、经典知识，促进项目组之间的经验共享
- 提供一个传播研发经验的平台

从这些宗旨来看，手册的基本定位在于“帮助”，而不是“指导”。手册既不最求大而全，也不追究权威性，而是追求实用性：能够帮助网络 ASIC 工程师在技术上有一些提高，能够把一些基本概念，基本技巧融入到我们的日常工作中，能够把一些小错误避免在萌芽状态。这些就是手册编写组追求的最终目标。

概括来说，《ASIC 工程师手册》要努力做到：新手上路捷径、工程师好帮手、经验共享平台。

ASIC 工程师手册编写组成员组成如下：

江潮	40206	海思网络产品管理部	组长
刘建强	39120	海思网络产品管理部	副组长
杨国新	14362	海思网络产品管理部	组员
陈华彬	30913	海思网络产品管理部	组员
林泗喜	36318	海思网络产品管理部	组员
梁创	38967	海思网络产品管理部	组员
黄天强	39770	海思网络产品管理部	组员
肖芹	41225	海思资料开发部	组员
李明月	48028	海思网络产品管理部	组员
董长城	49078	海思网络产品管理部	组员
李强	44749	海思网络产品管理部	组员

刘建强、黄天强、李明月、江潮、陈华彬、李强负责第 1 章、第 2 章；林泗喜、董长城、杨国新负责第 3 章；梁创负责第 4 章；肖芹负责第 5 章。

另外手册的编写工作也分布到网络产品管理部的各设计、验证、后端小组中。刘智勇、汤晓丹、黄丽、周冬宝、原飞、欧斯思、刘庆芳、范贵成、张红标、熊艳清、傅建新、施雪峰、郑玉婷、雷李云、关雪明、熊艳清等参加了设计章

节的编写工作；徐善锋、葛湘、杨小飞、程功宝、康毅等参加了验证章节的编写工作。陈会勇、王中伟、王建国等参加了后端章节的编写工作。

《ASIC 工程师手册》的开发是一个持续的过程，2007 年 7 月 19 日，手册推出 1.0 版本，这一次推出 1.1 版，虽然比 1.0 版有了一些进步，但我们仍然感到任重而道远。希望海思所有关心工程师手册发展的同事们向我们提出改进建议，我们一定仔细聆听、积极响应。

海思网络产品管理部《ASIC 工程师手册》编写组

2008 年 3 月 31 日

目 录

第 1 章 ASIC 开发流程 21
1.1 海思 IPD 开发流程简介 21
1.2 ASIC 开发总体过程描述 24
1.3 逻辑开发子过程描述 25
第 2 章 ASIC 设计知识 30
2.1 设计规范 30
2.1.1 海思 SOFT IP 设计规范 30
2.1.2 Verilog 2001 可综合语法介绍 85
2.1.3 后端对设计的建议 95
2.2 设计利器 105
2.2.1 MPI 模块级接口生成工具 105
2.2.2 代码顶层产生工具 108
2.2.3 代码行统计工具 109
2.2.4 代码模版产生工具 109
2.2.5 DOC_HTML 转换工具 110
2.2.6 GVIM 高级命令 111
2.3 通用接口 122
2.3.1 SPI4 接口概述 122
2.3.2 Interlaken 接口 129
2.3.3 SerDes 接口 160
2.3.4 PCI 接口 162
2.3.5 PCI Express 接口 165
2.3.6 QDR-II 187
2.3.7 RLDRAM-II 190
2.3.8 DDR 193
2.3.9 AMBA2.0 总线 198
2.4 常用工具 211
2.4.1 Debussy 入门 211
2.4.2 nLint 入门 213
2.4.3 Quartus 入门 218
2.4.4 Xilinx 工具入门 224
2.5 经典电路 238
2.5.1 MPI 接口设计 238
2.5.2 中断电路设计 248
2.5.3 复位电路设计 257
2.5.4 CRC 算法电路设计 263
2.5.5 时钟分频电路 274
2.5.6 异步电路 277
2.5.7 状态机 282

2.5.8 WRR 调度 288
2.5.9 PRBS 电路 290
2.5.10 性能计数器 292
2.5.11 环回测试电路 298
2.5.12 上下沿检测电路 301
2.5.13 FIFO 设计 305
2.5.14 缓存管理 318
2.5.15 8B/10B 编码 321
2.5.16 64B/66B 编码 327
2.5.17 IBM 的 memory 使用简介 330
2.5.18 ECC（Error Correction Code，纠错码） 333
2.5.19 FEC（Forward Error Correction，前向纠错） 336
2.6 经典案例 339
2.6.1 概述 339
2.6.2 反面案例 339
第 3 章 ASIC 验证知识 343
3.1 验证方法学 343
3.1.1 VMM 验证方法学 343
3.1.2 软硬件联合验证 372
3.2 验证语言 381
3.2.1 SystemC 382
3.2.2 SystemVerilog 414
3.3 SVA 简介 440
3.3.1 断言简介 440
3.3.2 断言的作用 442
3.3.3 断言的使用 443
3.3.4 断言的好处 443
3.3.5 总结 444
3.4 验证工具 444
3.4.1 VCS 使用入门 444
3.4.2 cadence 公司的 NC 系列仿真工具 454
3.4.3 仿真工具脚本 466
3.5 验证案例 485
3.5.1 后仿真指导 485
3.5.2 Debussy 在验证中的应用 508
3.5.3 提高验证效率的技巧 512
3.5.4 在 VMM 中如何不让 TC 里设定的低打印等级的 components 打印 log？ 513
3.5.5 cond coverage 有些条件没有统计？ 513
第 4 章 ASIC 后端知识 515
4.1 ASIC 后端基本概念 515
4.1.2 85%网表 516
4.1.3 95%网表 516
4.1.4 100%网表 516

4.1.5 Signoff....517
4.2 ASIC Vendor 后端流程概述....517
4.2.1 IBM 后端流程简介....517
4.2.2 LSI 后端流程简介....525
4.2.3 TI 后端流程简介....526
4.2.4 Faraday 后端流程简介....528
4.2.5 ST 后端流程简介....528
4.2.6 海思后端（COT）流程介绍....529
4.2.7 各厂家后端对比....532
4.3 逻辑综合 Logic Synthesis....533
4.3.1 逻辑综合基本概念....533
4.3.2 逻辑综合工具介绍....535
4.4 静态时序分析 Static Timing Analysis....579
4.4.1 STA 基本概念....579
4.4.2 STA 工具 Einstimer 与脚本介绍....583
4.4.3 STA 工具 Primetime 与脚本介绍....612
4.4.4 STA 案例分析....621
4.5 CDC 检查 Clock Domain Crossing Analysis....640
4.5.1 CDC 检查基本概念....640
4.5.2 CDC 检查工具与脚本....640
4.5.3 CDC 报告确认....644
4.5.4 CDC 案例分析....648
4.6 形式验证 Formal Verification....650
4.6.1 形式验证基本概念....650
4.6.2 Formality 的文件结构以及脚本书写方法....652
4.6.3 形式验证实现与脚本介绍....653
4.6.4 形式验证案例分析....656
4.7 可测性设计 Design For Test....661
4.7.1 DFT 基本概念....661
4.7.2 DFT 工具介绍与 DFT 实现....663
4.7.3 DFT 案例分析....663
4.8 其它....663
4.8.1 ECO 介绍....663
4.8.2 如何写 ECO 脚本....665
第 5 章 ASIC 文档编写....671
5.1 ASIC 文档体系....671
5.1.1 研发文档体系....671
5.1.2 客户文档体系....671
5.2 AISC 文档规范....672
5.2.1 IPR 要求....672
5.2.2 中文写作规范....672
5.2.3 绘图规范....674
5.3 文档编写工具....676

5.3.1 Visio 676
5.3.2 TimingDesigner 679
5.4 文档编写技巧 683
5.4.1 构建文档框架 683
5.4.2 组织信息结构 685
5.4.3 设计图、表 686
5.4.4 其他需要关注的元素 686
5.4.5 了解高质量文档的标准 689

插图目录

图 1-1 海思 IPD 开发流程图 21
图 1-2 生命周期模型端到端总图 24
图 1-3 生命周期模型逻辑开发子过程 25
图 2-1 顶层出现 glue logic 67
图 2-2 逻辑反馈环示意图 69
图 2-3 典型的异步设计电路示意图 69
图 2-4 典型的同步设计电路示意图 70
图 2-5 多时钟路径电路示意图 71
图 2-6 伪路径电路示意图 71
图 2-7 时钟作为数据采样示意图 71
图 2-8 时序 always 模块中使用非阻塞赋值(左)和阻塞赋值(右) 72
图 2-9 三态控制器件示意图 74
图 2-10 双向 net 器件示意图 75
图 2-11 透明法处理 latch 电路 75
图 2-12 使用双沿时钟电路示意图 76
图 2-13 门控时钟电路示意图 76
图 2-14 内部产生时钟电路示意图 77
图 2-15 77
图 2-16 时钟作为数据影响 DFT 77
图 2-17 组合逻辑环示意图 78
图 2-18 浮空的输入示意图 79
图 2-19 时钟直接输出到管脚示意图 79
图 2-20 时钟直接输出到管脚的 DFT 处理示意图 79
图 2-21 时钟作复位信号使用示意图 80
图 2-22 时钟作复位信号使用的 DFT 处理示意图 80
图 2-23 异步环路示意图 81
图 2-24 输出信号寄存示意图 82
图 2-25 输出信号寄存器时序分析图 82
图 2-26 输入信号与时钟之间、时钟与输出信号之间的关系示意图 83
图 2-27 时钟策略示意图 84

图 2-28 手工添加的门控时钟示意图.....84
图 2-29 generate loop 综合结果.....89
图 2-30 Conditional generate 综合结果（parameter＝ 1）.....91
图 2-31 Conditional generate 综合结果（parameter !＝ 1）.....91
图 2-32 比特选择语法综合结果.....94
图 2-33 行波时钟.....96
图 2-34 门控时钟改为非门控时钟的示意图.....96
图 2-35 门控时钟输出信号与控制门之间的关系示意图.....97
图 2-36 Multicycle 采样点的选取.....98
图 2-37 异步电路与组合逻辑时钟.....98
图 2-38 异步复位置位电路.....99
图 2-39 不同时钟的 MUX 选择电路.....99
图 2-40 同步时钟选择电路.....100
图 2-41 Multi-Cycles path 电路结构示意.....102
图 2-42 Multi-Cycles path 电路时序检查示意.....102
图 2-43 非 Multi-Cycle 的电路结构示意.....103
图 2-44 非 Multi-Cycle 的电路时序检查示意.....103
图 2-45 Multi-Cycles path 电路结构示意 2.....103
图 2-46 setup time 不满足时的亚稳态现象.....104
图 2-47 hold time 不满足时的亚稳态现象.....105
图 2-48 异步电路如何避免亚稳态.....105
图 2-49 MPI 模块结构.....106
图 2-50 SPI4 数据通道状态图.....124
图 2-51 SPI4 状态通道状态图.....125
图 2-52 SPI4 数据通道状态图.....**错误！未定义书签。**
图 2-53 通道条状发送示例（Lane Striping）.....133
图 2-54 字格式.....133
图 2-55 BurstShort 保证最小间隔.....134
图 2-56 控制字格式.....136
图 2-57 每条链路接收流程.....139
图 2-58 接收接口状态流程.....140
图 2-59 发送接口状态.....141
图 2-60 带外流控逻辑时序.....142

图 2-61 64B/67B 字边界锁定 147
图 2-62 元帧结构（单链路） 148
图 2-63 扰码同步状态机 150
图 2-64 Interlaken 链路对齐 (以 4 条链路为例) 151
图 2-65 诊断字 151
图 2-66 CRC32 计算说明 152
图 2-67 时钟补偿过程 153
图 2-68 跳字格式 154
图 2-69 速率匹配情景 155
图 2-70 设备参考 156
图 2-71 延迟说明 158
图 2-72 Interlaken diagram 160
图 2-73 预加重技术 162
PCIe 存储器读写请求结构示意图 175
PCIe IO 读写请求结构示意图 178
PCIe 配置读写请求示意图 181
PCIe 完成请求结构示意图 184
PCIe 消息请求结构示意图 187
图 2-74 典型的 AMBA2.0 总线的体系结构示意图 199
图 2-75 AMBA2.0 中的 multiplex 示意图 200
图 2-76 AMBA2.0 中的组件示意图 200
图 2-77 AHB 总线信号之间的基本互联示意图 202
图 2-78 AHB 总线基本传送周期示意图 202
图 2-79 AHB 带有等待周期的传输示意图 203
图 2-80 AHB 多个传输周期的传送流水示意图 203
图 2-81 AHB 总线中带有错误的响应示意图 205
图 2-82 AHB 中不带有等待态的 grant 的访问 206
图 2-83 AHB 带有等待态的 grant 的访问 206
图 2-84 AHB arbiter 对数据总线的所有权切换示意图 207
图 2-85 AHB arbiter 对数据总线的所有权切换示意图（突发传输之后的切换） 207
图 2-86 AHB arbiter 对 split 传输之后的切换示意图 208
图 2-87 APB 传输的状态机 209
图 2-88 APB 的基本写传送 210

图 2-89 APB 的基本读传送 210
图 2-90 Xilinx 设计流程图 225
图 2-91 时序逼近流程图 226
图 2-92 新项目窗口 227
图 2-93 添加已有源文件对话框 228
图 2-94 选择文件类型 228
图 2-95 结构向导选择对话框 229
图 2-96 DCM 向导－通用设置窗口 230
图 2-97 项目源文件窗口 232
图 2-98 当前源文件进程窗口 232
图 2-99 进程条 232
图 2-100 关联一个约束文件 233
图 2-101 器件结构窗口和封装引脚窗口 234
图 2-102 设计层次窗口 235
图 2-103 设计部件列表－I/O 引脚窗口 235
图 2-104 指定一个位置 236
图 2-105 指定一个位置 236
图 2-106 当前源文件进程窗口 237
图 2-107 项目源文件窗口 237
图 2-108 当前文件进程窗口 238
图 2-109 Intel 模式写时序图 239
图 2-110 Intel 模式读时序图 240
图 2-111 Motorola 模式写时序图 240
图 2-112 Motorola 模式写时序图 241
图 2-113 MPI 结构图 242
图 2-114 两级寄存器的结构 244
图 2-115 写时序 1 245
图 2-116 单级寄存器的结构 246
图 2-117 写时序 2 247
图 2-118 读时序 248
图 2-119 中断树结构示意图 250
图 2-120 中断树的基本结构图 253
图 2-121 寄存器延迟的同步处理复位结构 257

图 2-122 异步处理的异步复位结构 258
图 2-123 多时钟域的复位设计 259
图 2-124 专用延迟单元的纯异步结构 259
图 2-125 寄存器延迟的纯同步结构 260
图 2-126 长除法的 CRC-4 电路 266
图 2-127 多项式乘法电路 268
图 2-128 多项式相乘相除电路 269
图 2-129 CRC4_D8 接口信号接口时序 270
图 2-130 偶数分频电路波形图 275
图 2-131 四分频电路结构图 275
图 2-132 占空比 50%的五分频电路波形图 276
图 2-133 占空比 50%的五分频电路结构图 276
图 2-134 小数分频电路结构图 277
图 2-135 无毛刺异步时钟切换电路 278
图 2-136 低速到高速数据传输时序图 278
图 2-137 同步信号产生电路 279
图 2-138 切换电路 279
图 2-139 异步 FIFO 框图 282
图 2-140 状态机的典型结构 283
图 2-141 状态机的描述图 284
图 2-142 一段式状态机 284
图 2-143 两段式状态机 285
图 2-144 三段式状态机 286
图 2-145 WRR 算法流程图 289
图 2-146 PRBS23（$x^{23}+x^{18}+1$）产生电路原理图 290
图 2-147 PRBS 发送时序 291
图 2-148 PRBS23 接收机状态图 291
图 2-149 PRBS23 测试序列检测图 292
图 2-150 流水加法波形图 294
图 2-151 采用两块 Dual Port RAM 实现多通道计数示意图 295
图 2-152 采用三块 Two Port RAM 实现多通道计数示意图 297
图 2-153 GE MAC 内环回示意图 298
图 2-154 内环回测试示意图 299

图 2-155 GE MAC 外环回示意图 300
图 2-156 环回实现示意图 300
图 2-157 简单的上升沿检测电路 301
图 2-158 上升沿检测推荐电路 301
图 2-159 简单的下降沿检测电路 302
图 2-160 下降沿检测推荐电路 303
图 2-161 上升/下降沿检测推荐电路 304
图 2-162 线性反馈表 308
图 2-163 格雷码 FIFO 的基本原理图 309
图 2-164 格雷码的时序 310
图 2-165 空标志信号产生示意图 310
图 2-166 满标志信号产生示意图 310
图 2-167 FIFO 的自我保护机制 313
图 2-168 FIFO 的位宽的转换 314
图 2-169 FIFO 转换示意图 315
图 2-170 有数据包信号 Drpa 的产生状态机 316
图 2-171 不定态的传播和解决办法 317
图 2-172 双端口 BlockRAM 和分布式 Ram 的读写时序图 318
图 2-173 BM 电路结构图 319
图 2-174 fba_bitmap 和 fba_fifo 结构图 320
图 2-175 8B/10B 编码 322
图 2-176 D 分组说明 322
图 2-177 行程偏差状态机 325
图 2-178 8B/10B 编解码在某系统中的应用 327
图 3-1 分割设计 345
图 3-2 分割设计的验证环境 346
图 3-3 采用 Scoreboard 的验证环境 354
图 3-4 采用 RM 的验证环境 355
图 3-5 Pre_Simulation Offline Checking 356
图 3-6 ost-Simulation Offline Checking 356
图 3-7 验证环境 359
图 3-8 分层的验证环境 360
图 3-9 测试平台的具体实现 361

图 3-10 功能子层的划分 363
图 3-11 vmm_env base class 366
图 3-12 事务级接口的连接 367
图 3-13 通知接口 368
图 3-14 Seamless CVE 软硬件协同验证环境的组成 373
图 3-15 ISS 基本功能示意图 375
图 3-16 BIM 模型示意图 376
图 3-17 Seamless CVE memory 系统 378
图 3-18 未优化时的硬件总线行为示意图 379
图 3-19 优化后的硬件总线行为示意图 380
图 3-20 相对时钟异步的信号 394
图 3-21 systemC 语言结构 399
图 3-22 事件的发生示意图 400
图 3-23 直接用 netlist 连接的 top 415
图 3-24 采用 interface 连接的 top 416
图 3-25 SystemVerilog 与 Verilog 的对比 423
图 3-26 Object assignmet 425
图 3-27 Inheritance – extended a class with new fields 426
图 3-28 Inheritance – override existing fields to a class 426
图 3-29 Inheritance – overridden 427
图 3-30 polymorphism 427
图 3-31 handle assignment 428
图 3-32 约束与权重对仿真的作用 429
图 3-33 Constraint Block 429
图 3-34 分布权重 430
图 3-35 In-Line 约束和分布 430
图 3-36 Fork-join 431
图 3-37 Mailbox 432
图 3-38 Semaphore 432
图 3-39 Events 433
图 3-40 SVA 触发覆盖率采样 437
图 3-41 DVE 统计分析覆盖率 1 438
图 3-42 DVE 统计分析覆盖率 2 439

图 3-43 DVE 统计分析覆盖率 3 439
图 3-44 DVE 统计分析覆盖率 4 440
图 3-45 VCS 流程图 445
图 3-46 后仿真流程 486
图 3-47 SRAM cell 的文档注释 489
图 3-48 IBM 的库文件（1）与补丁库文件（2）的比较 490
图 3-49 SD598 的问题分析图 491
图 3-50 状态机典型案例分析 493
图 3-51 ncelab's force 的使用方法 493
图 3-52 窄脉冲现象的波形和告警文件 494
图 3-53 典型的寄存器异步复位网表设计 494
图 3-54 ncelab'force 问题的逻辑电路图 495
图 3-55 IBM's RAM 典型逻辑设计图 496
图 3-56 RAM 的 verilog 强制读操作代码 496
图 3-57 准备进行 force 的信号 497
图 3-58 采用 verilog 语言进行的 force 操作 497
图 3-59 进行 Force 时 NC 仿真器出现的出错信息 497
图 3-60 不同频异步电路出现不定态的典型报警信息 498
图 3-61 SD598 项目中跨时钟域电路的同步器的典型设计 498
图 3-62 syn1 寄存器时序不满足的典型波形 498
图 3-63 同步数据接口，异步复位接口的寄存器 499
图 3-64 逻辑设计图 500
图 3-65 对应的波形图 500
图 3-66 LPH_4001_LPC_E 的时序图 501
图 3-67 LPH_4001_LPC_E 的实际仿真波形 501
图 3-68 95%网表中 hold violation 的波形 501
图 3-69 DATA_CDR 的 RTL 代码 501
图 3-70 DATA_CDR_REG[0]DATA_CDR_REG[1]的网表图 502
图 3-71 两级寄存器之间的逻辑设计图 502
图 3-72 FIFO 的典型逻辑设计图 503
图 3-73 MACDDR's FIFO 典型分析 504
图 3-74 手册中对 PLLREGOVR 信号的描述 505
图 3-75 使用示例 506

图 3-76 SD598 的后仿真计划 GANT 图.....506
图 3-77 同步器的规范命名示意图.....507
图 3-78 MAC 模块的测试电路.....507
图 4-1 设计流程简图.....515
图 4-2 IBM 后端流程图.....518
图 4-3 IBM 后端流程 Milestones－IDR 阶段.....519
图 4-4 IBM 后端流程之 RTF 阶段.....520
图 4-5 IBM 后端流程之 RTP 阶段.....522
图 4-6 IBM DFT 流程图.....523
图 4-7 IBM 后端流程 Milestones－RTL 阶段.....524
图 4-8 IBM 后端流程 Milestones－RTC 阶段.....525
图 4-9 LSI 后端流程图.....526
图 4-10 TI 后端流程图.....527
图 4-11 Faraday 后端流程图.....528
图 4-12 ST 后端流程图.....529
图 4-13 Design Compiler 的输入和输出.....534
图 4-14 综合的基本流程/命令.....535
图 4-15 DC 中 Library 的调用关系.....539
图 4-16 三种线负载模型的区别.....547
图 4-17 Wire Load Mode 设置示意.....547
图 4-18 创建分频时钟电路、波型示意 1.....558
图 4-19 创建分频时钟电路、波型示意 2.....559
图 4-20 创建倍频时钟电路、波型示意.....559
图 4-21 综合和 CTS 后的时钟网络示意.....560
图 4-22 综合时 CTS 的模型化.....561
图 4-23 Input Delay 设置示意.....562
图 4-24 output Delay 设置示意.....563
图 4-25 multicycle 设计电路示意.....566
图 4-26 set_multicycle_path 的 path_multipler 示意.....567
图 4-27 Multiple Instance 问题示意.....569
图 4-28 uniquify 解决 Multiple Instance 问题示意.....569
图 4-29 Set_dont_touch 解决 Multiple Instance 问题示意.....570
图 4-30 Top-down Script.....577

图 4-31 Bottom-up Script 578
图 4-32 基本时序路径 580
图 4-33 到达时间传递 581
图 4-34 要求到达时间传递 581
图 4-35 触发器的建立时间和保持时间 582
图 4-36 STA 分析中建立时间检查 582
图 4-37 STA 分析中建立时间检查 583
图 4-38 Cycle-to-cycle jitter 抖动波形 583
图 4-39 Theguide 工作界面 584
图 4-40 Theguide console 界面 584
图 4-41 Theguide 中常用的菜单项 585
图 4-42 Theguide 工作路径设置 585
图 4-43 环境参数设置 586
图 4-44 设计相关参数 587
图 4-45 时序约束文件路径设置 587
图 4-46 PLL 信息设置 588
图 4-47 寄生参数文件路径设置 588
图 4-48 多电压文件路径设置 588
图 4-49 库文件路径设置 589
图 4-50 RTF、RTL 等 Methodologies 选择 589
图 4-51 Organzier 设置错误时提示信息 589
图 4-52 正常的 Data Organizer 设置 590
图 4-53 建立新的 Data Organizer 590
图 4-54 Theguide 中 ET 启动选项 590
图 4-55 ET 启动参数设置 591
图 4-56 ET console 窗口 591
图 4-57 ET 工作窗口 592
图 4-58 时序分析参数设置 592
图 4-59 输出报告设置 593
图 4-60 NetWork 工作界面 594
图 4-61 搜索界面设置 594
图 4-62 电路连接关系追踪 595
图 4-63 去掉窗口显示的某个器件 595

图 4-64 查看器件的时序信息 596
图 4-65 时钟域标签划分 597
图 4-66 数据标签 598
图 4-67 相位含义 598
图 4-68 单一数据相位标签传递 599
图 4-69 多数据相位标签传递 599
图 4-70 多个时钟相位标签传递 599
图 4-71 门控时钟相位标签传递 600
图 4-72 单时钟域相位标签传递 600
图 4-73 多时钟域相位标签传递 600
图 4-74 时钟相位标签经反相器传递 600
图 4-75 寄存器时钟反相影响输出相位标签 601
图 4-76 发射时钟和捕获时钟前沿同相位建立时间检查调整 601
图 4-77 发射时钟和捕获时钟前沿同相位保持时间检查调整 602
图 4-78 发射时钟的前沿晚于捕获时钟前沿建立时间检查调整 602
图 4-79 发射时钟的前沿晚于捕获时钟前沿保持时间检查调整 602
图 4-80 发射时钟的前沿早于捕获时钟前沿建立时间检查调整 603
图 4-81 发射时钟的前沿早于捕获时钟前沿保持时间检查调整 603
图 4-82 时钟路径延时不同时检查调整关系 603
图 4-83 异步时钟检查关系 604
图 4-84 FEP 之前网表时钟连接 604
图 4-85 输入管脚信号到达时间约束与延时关系 606
图 4-86 输出管脚延时约束 607
图 4-87 输出端口 Don’t Cares 约束 608
图 4-88 输入出端口 Don’t Cares 约束 609
图 4-89 多周期路径设置 609
图 4-90 分频时钟重冲命名 610
图 4-91 任意两个引脚之间的 setup/hold 检查 611
图 4-92 时序检查图 611
图 4-93 异步路径示例图 611
图 4-94 输出管脚 skew 约束 612
图 4-95 PrimeTime 基本流程 615
图 4-96 相位穿通示意图 622

图 4-97 udt 与 eta 冲突时，udt 约束无效的时序报告 623
图 4-98 udt 与 eta 冲突时，udt 约束有效的时序报告 623
图 4-99 反标 SDF 正确后仿的结果 624
图 4-100 反标 SDF 不正确后仿的结果 624
图 4-101 ncsim 的告警信息 624
图 4-102 ET 中生成 SDF 文件的选项 625
图 4-103 生成 SDF 的脚本 625
图 4-104 chopper 电路正相位输入 626
图 4-105 chopper 电路负相位输入 626
图 4-106 时钟定义示意图 627
图 4-107 时钟定义示意图 627
图 4-108 时钟定义示意图 628
图 4-109 CCLKA 和 CCLKB 相位关系 629
图 4-110 SRAM2DR 应用实例：原电路 629
图 4-111 SRAM2DR 应用实例：有 seperation 的电路 630
图 4-112 时钟树示意图 631
图 4-113 CLK_DIV2_REG 的 B 和 C 的 pulse width 检查 631
图 4-114 DATA_REG 的 B 和 C 的 pulse width 检查 631
图 4-115 INV1 的 pulse width 检查 631
图 4-116 M155+时钟定义在 INV1/Z 的情况 632
图 4-117 INV1 的 inactive period 检查 632
图 4-118 随路时钟和随路数据输出约束电路示意图 634
图 4-119 随路时钟和随路数据输出约束时序示意图 635
图 4-120 形式验证在后端流程中的位置 652
图 4-121 Formality 目录结构 653
图 4-122 Formality 流程 655
图 4-123 灰尘落入 wafer 导致芯片缺陷 661
图 4-124 金属被压断导致芯片缺陷 662
图 4-125 桥接导致的芯片缺陷 662
图 4-126 芯片缺陷的的代价 663
图 5-1 图形组成说明 674
图 5-2 文字倒置示意图 675
图 5-3 Visio 欢迎界面 676

图 5-4 线条端点设置.....677
图 5-5 运算工具说明.....678
图 5-6 流程图方案选择.....679
图 5-7 TimingDesigner 设置线条和字体示意图.....680
图 5-8 时钟信号的参考线设置示意图.....681
图 5-9 时钟信号参考线样例图.....681
图 5-10 约束属性对话框.....682
图 5-11 输入文本示意图.....682
图 5-12 导出选项示意图 1.....683
图 5-13 导出选项示意图 2.....683
图 5-14 自上而下的信息组织方法.....684
图 5-15 自下而上的信息组织方法.....684
图 5-16 高质量文档的八要素.....689

表格目录表 1-1 海思 IPD 开发流程活动描述.....21
表 1-2 研发活动描述.....26
表 2-1 命名规则总表.....30
表 2-2 文件头规则总表.....32
表 2-3 注释规则总表.....34
表 2-4 编码风格规则总表.....34
表 2-5 编码风格规则总表.....36
表 2-6 STA 相关规则总表.....37
表 2-7 仿真规则总表.....37
表 2-8 DFT 相关规则总表.....38
表 2-9 设计风格规则总表.....38
表 2-10 常见的命名缩略语.....45
表 2-11 SPI4 数据格式.....123
表 2-12 突发空闲控制字格式.....136
表 2-13 帧层功能总览.....144
表 2-14 Bit 66.....146
表 2-15 同步字编码.....146
表 2-16 元帧控制字块类型.....148
表 2-17 同步字和扰码状态字.....149
表 2-18 Skew Budget.....154
表 2-19 速率匹配参数.....156
表 2-20 统计值.....158
表 2-21 延迟参数.....159
表 2-22 性能分析.....159
表 2-23 AHB 总线信号列表.....200
表 2-24 AHB 总线响应 HRESP 说明表.....204
表 2-25 AMBA2.0 中 APB 总线的信号列表.....208
表 2-26 Intel 模式写时序参数表.....239
表 2-27 Intel 模式读时序参数表.....240
表 2-28 Motorola 模式写时序参数表.....240
表 2-29 Motorola 模式读时序参数表.....241
表 2-30 三种方式的对比.....287
表 2-31 3B 到 4B 的编码.....324

表 2-32 表 2 5B 到 6B 的编码.....324
表 2-33 8B/10B 编解码映射表.....325

版本记录

版本	说明	日期
1.0	Release	2007-7-19
1.1	新增内容： 2.2.4 代码模版产生工具 2.3.4 PCI 接口 2.3.6 QDR-II 2.3.7 RLDRAM-II 2.3.9 AMBA2.0 总线 2.4.4 Xilinx 工具入门 2.5.10 性能计数器 2.5.11 环回测试电路 2.5.12 上下沿检测电路 2.5.13 FIFO 设计 2.5.14 缓存管理 2.5.15 8B/10B 编码 3.1.2 软硬件联合验证 3.3 SVA 简介 3.4.1 VCS 使用入门	2008-3-31
1.2	新增内容： 2.3.2 Interlaken 接口 （作者：徐双武） 2.3.5 PCI Express 接口 （作者：冯海） 2.3.8 DDR （作者： 魏华） 2.5.16 64/66B 编码 （作者：原飞） 2.5.18 ECC （作者：聂世玮） 2.5.19 FEC （作者：聂世玮） 4.2.6 海思后端（COT）流程介绍 （作者：廖国柱） 4.4.4.13 如何约束异步电路延时 （作者：朱治宇） 4.5 CDC 检查 （作者：朱治宇） 删除内容： 5.3.1 WORD 工具	2013-12-27

第1章 ASIC 开发流程

概述：这里介绍海思的 ASIC 开发流程，帮助读者熟悉开发流程的各基本阶段。

1.1 海思 IPD 开发流程简介

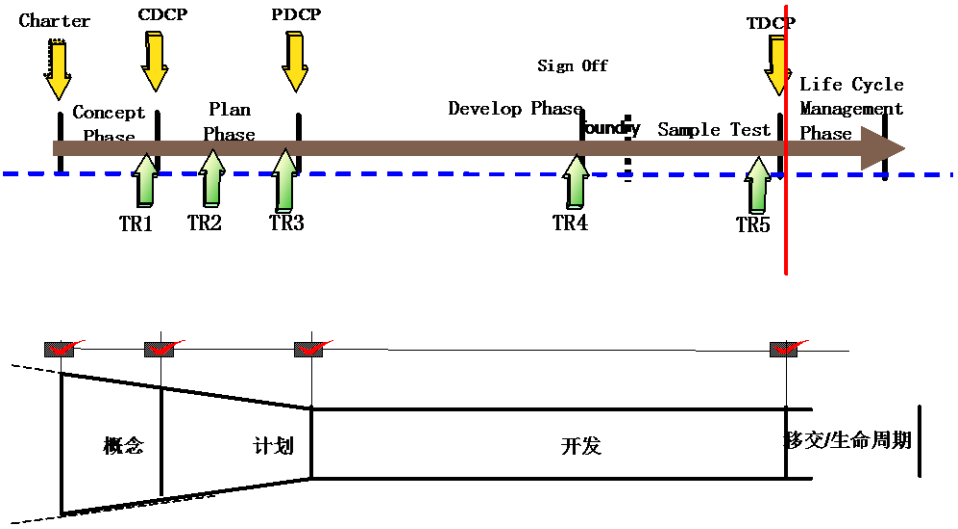


图1-1 海思 IPD 开发流程图

表1-1 海思 IPD 开发流程活动描述

活动	活动描述
概念启动（Charter DCP）	根据产品路标规划和规划基础活动的成果，MKT 对概念启动的可行性进行分析，拟制 Charter 任务书及准备项目背景材料，提交 PMT 预审，通过后提交 IPMT 进行立项评审。 芯片 IPMT 对 Charter 进行评审，关注是否符合公司业务路线与战略。评审通过，则项目正式立项，并给出是否独立组建 PDT 的建议。PDT 根据批准的概念阶段计划开展工作。
概念决策评审 (CDCP)	PDT 在概念阶段完成业务计划初稿，收集各种需求，通过需求验证，确定产品包需求，提出项目策略建议，开展项目的财务分析等工作，并将结果（做什么）和建议提交 IPMT 评审，IPMT 根据 PDT 交付，审查业务计划的可行性及投入、预期财务收益、风险与策略，以确定项目是否有必要进入计划阶段。 假如 IPMT 决定是“批准”，则要给出建议增补任命外围组或系统分析与设计组，给出外围小组成员的候选名单和任命，PDT 开始按照计划阶段计划开展工作。 假如芯片 IPMT 没有批准，IPMT 要给出是否解散 PDT 的建议。PDT 要就项目文件、数据库、项目组成员解散等工作作出周密安排并实施。不管哪种情况，PDT 核心组要进行经验教训总结，并归档到海思过程资产库（海思 PAL 库）。

活动	活动描述
计划决策评审 (PDCP)	PDT 在计划阶段要提交最终的业务计划。指定项目的 E2E 计划、开展项目资源的预算、市场分析与评估、财务分析与评估，并根据概念阶段确定的产品包需求，完成需求分解与分配、产品包的概要设计（各子系统的需求规格 LRS/SRS/HRS），确定一种被选方案，并做相应的 TR2、TR3 评审，准备 PDCP 评审材料，将阶段结果（怎么做，建议）提交 IPMT 评审。 IPMT 评审业务计划和建议，并且决定是否批准项目继续下去。假如决定是“批准”，IPMT 将给出必要的外围组成员的全员任命，PDT 与 IPMT 签署项目合同书，进入开发阶段。PDT 负责按照 IPMT 批准的计划管理项目，按照项目合同条款对交付负责，并按时提交 METRICS 状态报告或向 IPMT 作正式汇报。IPMT 要按时提供开发阶段到 GA 点的资金和资源支持，并对 PDT 进行绩效考核。 假如 IPMT 不批准，项目可能中止，可能转向。当决定项目中止时，PDT 要更新项目文件，数据库，立即解散并终止项目进一步的工作。不管哪种情况，PDT 核心组进行项目总结，并将经验教训归档到海思过程资产库(海思 PAL 库)。
迁移决策评审 (TDCP)	通常情况下，样片回片，海思提供样片给相关产品线进行产品测试，海思ASIC产品测试工程师加入到芯片测试中去，双方进行联合测试直至提供测试结论。条件满足，海思可以开展后续TR5活动。TDCP是迁移决策评审点，该决策评审点通过，标志着海思ASIC芯片成功迁移到华为产品线。
TR1（关注产品包需求）	在概念阶段，CDCP 评审前，PDT 要针对产品包需求和产品概念进行评审（TR1）。TR1 重点关注产品包需求的完备性以及选择的产品概念是否满足客户需求。 ● 评估产品包需求是否充分满足了客户需求； ● 确保产品包需求的技术可行性以及产品概念的有效性； ● 判断设计需求是否充分映射产品包需求，描述是否清晰而足以指导计划阶段的系统设计规格定义； ● 评估产品标准策略； ● 评估部件重用技术。 TR1 通过后应建立需求基线，需求纳入变更控制管理。
TR2（关注产品规格）	PDT 在计划阶段对产品设计规格的评审（TR2）。 PDT 根据概念阶段确定的产品包需求和设计需求，进行需求分解与分配，确定产品设计规格。TR2 的准备活动包括各功能领域完成专项评审，在 TR2 的评审会上将由 PDT 核心团队在综合层面进行讨论，确保设计规格包括并适应每单元和构建模块（譬如逻辑模块、软件模块）的设计。 TR2 的评审过程包括对关键设计规格和要点的综合讨论，也对产品架构（配置）进行细致审阅（走读），以确保后续的概要设计（HLD）或者各模块（逻辑、软件、硬件）的需求规格设计能有效进行。 TR2 通过后，产品设计规格应被置于更改控制之下。 ● 设计规格充分映射了设计需求，并足以指导后续的 IPD 流程及功能领域子流程的产品开发活动； ● 定位产品规格和配置中的缺陷和限制，评估风险，形成规避策略和应急计划； ● 形成有效的产品配置。
TR3（关注产品概要设计或者逻辑/软件/硬件需求规格）	TR3 是在计划阶段对产品概要设计或者产品各模块（逻辑/软件/硬件）需求规格的评审，确保产品设计规格已经完全、正确地在产品的逻辑/软件/硬件需求规格中得到体现。 TR3 的结果将作为开发阶段的后续各模块的详细设计活动是否继续投入资源的根据。 TR3 的准备活动包括各功能领域完成专项评审，在 TR3 的评审会上将由 PDT 核心团队在综合层面进行讨论，确保产品概要设计或者逻辑/软件/硬件需求规格足以指导产品项目计划制订、资源预算、技术可行性、以及后续的详细设计（LLD）活动。 TR3 的评审过程包括对概要设计或者需求规格各主要冲突点的讨论，也需要跟踪设计规格、共用模块重用计划、产品策略、产品配置的落实情况。 TR3 通过后产品概要设计或者各模块的需求规格应被置于更改控制之下。

活动	活动描述
TR4（TAPE OUT 评审）	TR4 保证系统的模块用于系统级构建之前是完整的。任何不符合规定的情况都应该在 TR4 问题记录中得到记录，并进行风险评估。TR4 要检查逻辑所包含模块的详细设计审查结果、单元测试（UT）结果、模块验证环境的准备情况，是否满足进入模块集成测试的条件。对于软件或者硬件项目，则需要检查软件或者硬件模块的单元测试（UT）、模块集成测试（MIT）、模块系统测试（MST）结果。保证项目可以进入 SDV。 在逻辑模块系统测试、软硬件联调、FPGA 验证、后端设计与检视完成后，对产品技术上的成熟度进行评估，确保所有存在的问题和风险都进行了评估，并生成了相应的改进计划。同时确保供应商的制造能力足以支撑样片生产活动。 TR4A 作为 IPD 流程中一个关键的技术评审点，其目的包括： ● 对模块集成测试和系统测试结果、与生产厂商交互设计的检视结果、遗留问题及风险、改进计划进行评审，判定是否可以 TAPE OUT； ● 对供应商能力进行评审，保证足以支撑样片生产。 ● 评估样片测试的准备情况；
TR5（关注样片测试结果）	TR5 主要通过对原型机（样片）测试结果的评价，以确定样片是否满足了功能、性能的设计需求，以及对样片的设计稳定性和技术成熟度方面的独立评估活动。TR5 也是开发阶段结束的标志： ● 检查样片的规格是否符合计划阶段产品规格的要求； ● 检查样片的制造过程是否影响芯片的功能、可靠性和性能规格。 ● TR5 是进入验证阶段的必要和充分条件。完成 TR5 就表明小批量生产和销售已经准备就绪。 ● 根据进行 TR5 评审的产品版本所具有的功能和性能规格，判断该版本是否适合启动客户应用测试；
TR6（关注客户应用测试结果）	TR6 是对小批量应用结果的系统级的评审，确保产品的客户小批量应用是否取得满意效果，产品生产与制造能力已经能适应全球范围内发货的需求。 TR6 关注于以下几方面： 系统总揽、计划评审、量产评审、供应商评审、制造计划周期、产品保证（遗留问题是否解决，产品质量目标是否符合要求）、可靠性、可维护性、可安装性、可服务性目标是否符合要求、所有特性（含功能、性能）的目标是否符合要求、客户应用测试中的问题是否已经被解决。 在客户应用测试后，TR6 是验证阶段唯一的技术评审。 TR6 是 ADCP 和 GA 的判断准则之一。 TR6 关注于产品生产问题的状态和解决情况。当决定产品是否能从小批量开始放量生产时，需要考虑 TR6 的结果。 TR6 的目标是评估产品级交付件的技术成熟度，分析继续进行发布阶段的后续活动（生产放量等）的风险。因此 TR6 有如下 5 个目的： ● 作为 ADCP 参考输入，TR6 检查产品是否可以推向市场并且确认进入发布阶段可能存在的风险。 ● 评审客户应用测试的产品规格和产品制造的输出、问题和解决方案的影响。 ● 确认最终产品配置已经文档化，所有 EC（变更请求）已经生效、已经执行并且测试完成。 ● 确保生产和产品支持系统已经被验证，并且适合进入批量生产。 ● 确保技术支持和销售、订单支撑系统已经准备好，并且进入发布阶段后必需的相关信息已经完成。

1.2 ASIC 开发总体过程描述

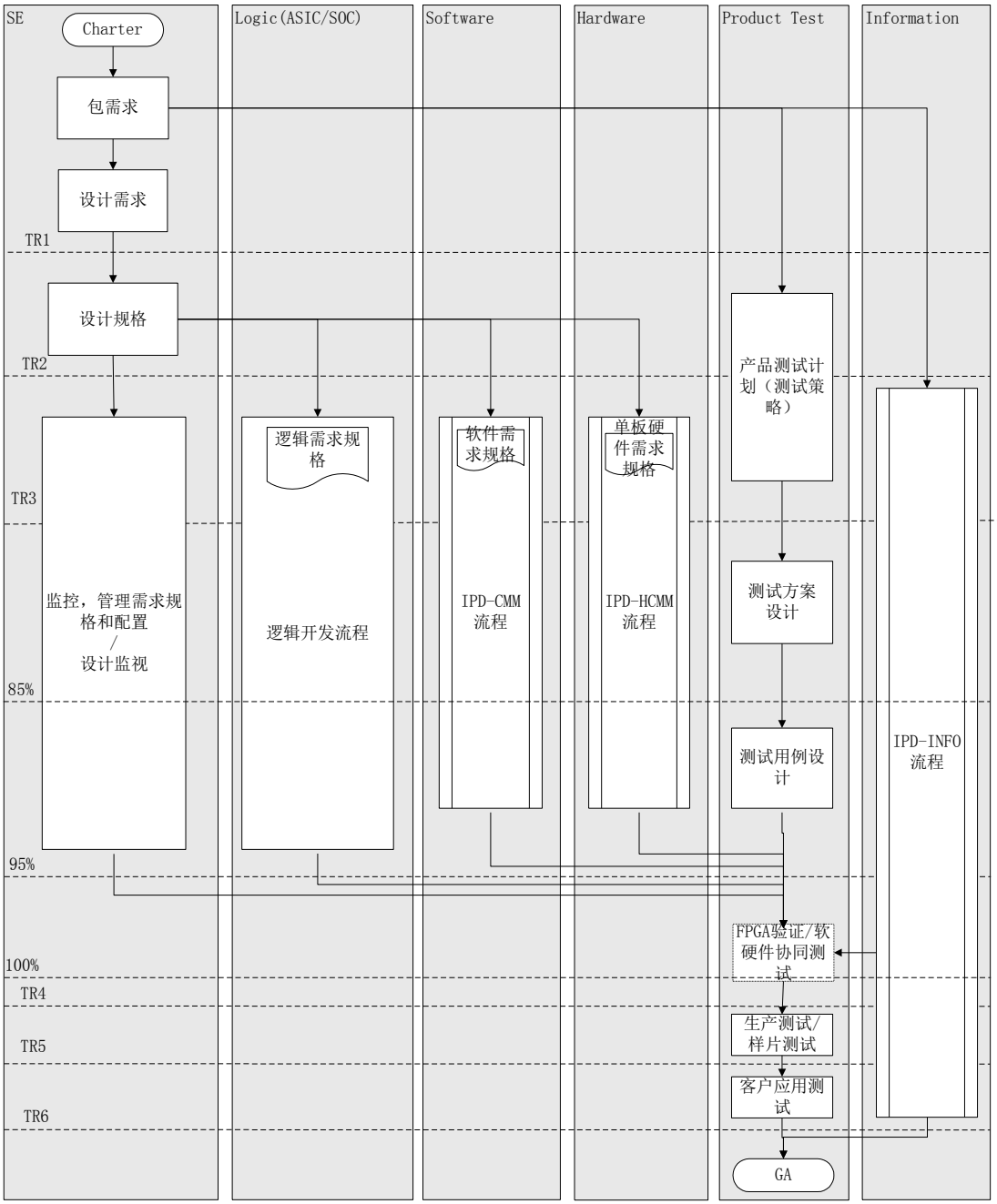


图1-2 生命周期模型端到端总图

1.3 逻辑开发子过程描述

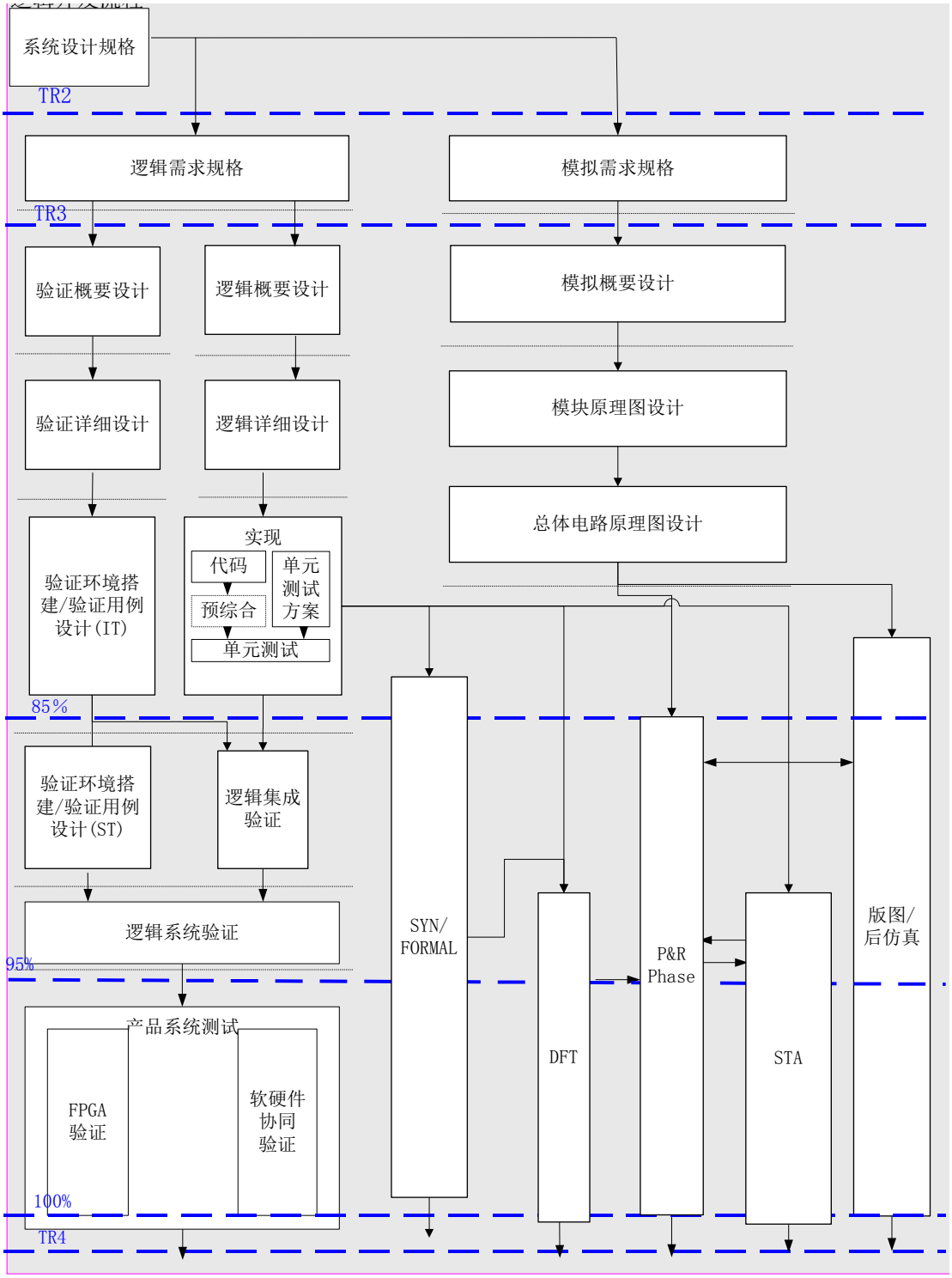


图1-3 生命周期模型逻辑开发子过程

表1-2 研发活动描述

活动	活动描述
系统工程	
提供技术备选方案	从满足市场需求的角度，提出多个产品包概念并检查每一个的优缺点；提出并评估融入概念的产品、元器件、制造工艺等的多种技术选择；评估技术风险；确定预备技术作为备用。
概念选择	检查市场需求；定义芯片产品包概念的评价标准和建议；定义技术生存能力、准备就绪评价标准；产生多个产品包概念；记录每个概念的优缺点；根据标准对每个概念评价；选取一个概念。
确定产品包需求	基于选择的产品包概念开展产品包需求分析，从内外部客户/使用者的角度清晰地描述芯片产品的需求，包需求包括内容有：市场需求、内部需求，兼容性、共用性、成本有效性、可靠性、可服务性、可测试性、地理市场、技术方面、可制造性。如果需要，进一步从操作概念和操作场景等方面对包需求进行功能、性能层面的细化分析，输出芯片产品包设计需求。
系统设计和规格制定	基于需求的分解和分配，生成芯片产品的系统、子系统、部件、模块及接口的技术规格，要求表达清晰以减少误解及重复；生成芯片系统架构图。
确定系统测试计划	基于系统设计概念框图，以及来自于数模设计、软件、硬件以及技术支持（FAE）、营销（Marketing）收集的用户需求和设计，形成包括了资源、成本、和时间进度估计的详细计划，阐述了系统测试（内部样片测试和外部客户应用测试）、品质保证，以及各国准入法规。
监控和管理需求、规格和配置	定期地对设计和开发活动进行检查以确保开发人员正确地按产品包技术规格进行开发；对产品的需求、规格和配置进行跟踪和变更管理。
制定对外合作策略/计划	根据业务计划和项目端到端计划，基于产品规格和系统架构设计，考虑项目开发过程中缺少的 IP，关键技术、人力资源获取等，制定对外合作策略和计划。 审阅各合作项目立项建议书及立项评审表，根据具体对外合作项目的需要，采购代表参与相关合作项目。优化细化各合作项目计划： • 各合作项目要求的启动时间和要求完成时间，及项目成果（含阶段成果）提交时间 • 工作量估算及人力需求计划（公司内部投入需求、对外部 人力需求） • 合作费用分析：投资分析、占产品研发预算比例 • 各合作项目成果验收标准 • 风险预测（内部风险如需求可能变动、技术方案变动；外部风险如达不到技术要求、时间延误等） 细化后的合作时间计划/人力投入计划/费用计划/验收标准，放入业务计划中。
对外技术合作	各合作项目经理（产品合作、资源合作、技术合作）依据对外合作合同/业务计划负责监控实施对外合作计划，保证合作计划与产品开发的同步，并将合作成果应用于产品开发中。 如遇需求变更或计划调整，应按产品开发要求走变更流程，并通过对外合作部/业务部合作分部与合作方达成一致或协调解决。
合作成果验收	按照既定的合作项目成果验收标准验收合作项目。
知识产权/标准分析	检查基于以前项目经验教训的智力资产以防重犯过去的错误；探索能使用在新项目上的内部技术（共用基础模块）；评估时间进度、成本和交付的风险；检查概念和技术的有关可申请专利的、专利权、法律问题。
制订标准/专利计划	制订知识产权（主要是专利申请）/标准活动的计划，包括交付内容、责任人、关键里程碑。

活动	活动描述
专利检索分析和专利申请	相关技术领域、标准、关键技术、竞争对手等专利检索及专利分析，知识产权风险分析及风险归避、跟踪，制订 PDT 知识产权策略。 挖掘创新点，完成专利技术交底书的写作、评审、修改，提交专利申请。 其他知识产权/标准计划实施。
制定企业标准和内控标准	《企业标准》、《企业内控标准》皆源于《产品规格书》，并在开发过程中参照国际、国内标准，运用技术与经济相结合原则，在不增加成本基础上，为达到优于其他制造厂家同类产品质量而选择合适的功能、性能指标编入到标准文本中，标准的各项指标定位要恰到好处，并要体现层次和梯度：根据国家规定，《企业标准》所列功能、性能指标可优于国际、国内同类标准中相应的部分，而《企业内控标准》在功能、性能指标上必须优于、严于《企业标准》的规定。
资料知识产权审查	外出资料（如用户手册、应用指导书等）的知识产权（版权、商标、技术秘密、专利等方面）审查、评审，确保产品能够规避 IPR 风险。
IPR 评估	知识产权工作完成总结；如有知识产权风险，需提出后续推广销售等工作中的 IPR 建议，风险及规避措施，注意的问题。
EC 管理	将所有技术信息放入数据库并通过正式的工程更改（EC）流程来管理更改；使用“交付件更改管理”使能流程来决定提议的更改，并使用 PDM 或其它相似的系统来自动管理工程更改（EC release）管理流程。EC release 管理并不是冻结数据，而是对更改进行控制，每个人都应该被告知有关的更改以便每个人都基于相同的信息进行工作。
数字/数模	
逻辑需求规格	根据系统设计规格分解，分配 ASIC/SOC 设计中的逻辑（硬件）部分规格。确定某些特殊需求如何采用子系统或模块的组合来实现，它们之间的接口关系也要定义清楚。
逻辑概要设计&详细设计	设计概念块图，进行模块级即需要多少个模块以及每个模块的环境及功能等的逻辑系统概要设计；与系统工程师一起来解决与其它功能开发小组之间的冲突和问题。必要的话，提出需求和/或规格更改方面的建议。 根据概要设计细化模块内部的设计，用状态机或计数器等实现需要的逻辑功能，设计好电路内部的接口及时序关系，保证设计满足需求。
电路实现	用合适的形式实现设计（数字电路的 RTL 代码，模拟电路的原理图设计等）；以及设计人员完成自己模块的单元仿真。
电路的维护与修改/样片测试用例	根据验证发现的缺陷，或者是后端时序的要求，设计人员对代码（数字）或者是原理图（模拟）进行的电路修改和维护；开发样片测试需要的用例，尽可能根据系统对芯片的每一个功能点进行遍历。
样片测试	从生产线生产出来的首批芯片样片进行功能/性能测试。验证产品是否符合原定规格。
支持客户样片试用	对客户的小批量芯片使用进行支持。参与制定和优化客户环境下应用测试用例，参与测试活动，解决测试中发现的问题。
验证	
逻辑验证方案设计	包含验证策略，验证方案以及 feature list 等设计。
逻辑验证环境	编写验证环境代码，为开展验证活动做准备。
逻辑集成验证	在功能模块单元仿真后将部分模块进行集成验证，确认集成功能模块的符合预定的功能。
逻辑系统验证	建立一个逻辑系统，开展验证活动，确认逻辑系统与已经定义的芯片逻辑规格的一致性。

活动	活动描述
FPGA/Quickturn 验证	利用 FPGA 测试板，将逻辑加载进 FPGA，对系统的各个组成部分进行验证/在 Quickturn 上进行验证活动。确认逻辑在真实的系统中的功能正确性。
软硬件联合验证	利用 EDA 工具将逻辑部分，软件、硬件部分联合进行验证。确认逻辑与驱动软件的功能正确性。
后端	
STA	针对网表进行的静态时序分析，验证逻辑设计的时序是否满足约束的要求。
DFT	可测试性设计。插入 SCAN 链，边界扫描电路等保证电路的可测试性。生成生产测试向量。
P&R	将设计（网表/电路图）转化为版图，包括 Floorplan /Powerplan，Try Layout，Routing 等子活动。
综合	将数字部分设计转换为原始的或者厂商的逻辑门。
SI 分析	
SI 分析	基于芯片规格和应用环境，进行 IO 模块性能仿真分析、测试，确定接口方案，进行封装电性能优化设计约束，描绘出接口应用的物理设计方案。
软件	
软件需求规格	根据系统设计规格进行分解分配，确定驱动软件、应用软件部分需求规格书。
驱动软件概要设计&详细设计	进行驱动软件模块级需要多少个模块以及每个模块的环境及功能等的软件系统概要设计；描绘出驱动软件详细的模块、要完成的功能、输入、输出和界面格式。
开发驱动软件	使用标准的设计工具（根据情形选用高层次语言，CASE）来进行驱动软件代码设计。
软硬件联合验证	利用 EDA 工具将逻辑部分，软件、硬件部分联合进行验证。确认逻辑与驱动软件的功能正确性。
应用软件概要设计&详细设计	进行应用软件模块级需要多少个模块以及每个模块的环境及功能等的软件系统概要设计。描绘出应用软件详细的模块、要完成的功能、输入、输出和界面格式。
开发&测试应用软件单元	使用标准的设计工具（根据情形选用高层次语言，CASE）来进行应用软件代码设计；进行测试和调试代码、并且在编程数据库中维护代码及其历史资料。
软件系统测试用例	开发应用软件在测试系统中能够执行的测试用例，包含 EDA 环境就能够执行的和需要实际环境才能够执行的用例部分。
软件系统测试	在 FPGA 验证环境或样片测试环境执行软件系统测试用例，对整个软件进行功能、性能测试的遍历。
硬件	
硬件需求规格	根据系统设计规格进行分解分配，确定硬件部分需求规格书。
验证板硬件概要设计&详细设计	基于概要的 BOM 结构树及系统的设计规格书，开发一个到板级的硬件概要设计；基于硬件概要设计，使用标准的设计工具（根据情形选用 RF/模拟/数字/芯片设计）来进行详细设计，描绘出明确的板、卡、元器件、要完成的功能和界面；对每一个板/卡/元器件，开发电路设计，原理图，零部件清单，网表等；按照测试计划合并测试点来支持测试；在数据库中维护设计信息。
开发&测试验证板硬件单元	硬件工程师准备好相应的物料，验证板单板 PCB 投板回来后，逐步加工好单板，进行单板软硬件、逻辑调试，测试单板上模块电路特性是否符合设计要求。

活动	活动描述
Demo 板硬件概要设计	基于概要的 BOM 结构树及系统的设计规格书，开发一个到板级的硬件概要设计；与系统工程师一起来解决与其它功能开发小组之间的冲突和问题。必要的话，提出需求和/或规格更改方面的建议。
Demo 板硬件详细设计	基于硬件概要设计，使用标准的设计工具（根据情形选用 RF/模拟/数字/芯片设计）来进行详细设计，描绘出明确的板、卡、元器件、要完成的功能和界面；对每一个板/卡/元器件，开发电路设计，原理图，零部件清单，网表等；按照测试计划合并测试点来支持测试；在数据库中维护设计信息。
开发&测试 demo 板硬件单元	硬件工程师准备好相应的物料，demo 板单板 PCB 投板回来后，逐步加工好单板，进行单板软硬件、逻辑调试，测试单板上模块电路特性是否符合设计要求。

第2章 ASIC 设计知识

概述：ASIC 设计知识章节是希望给员工的代码设计提供一个指导，提高规范性，设计能力，避免犯一般性错误。

2.1 设计规范

2.1.1 海思 SOFT IP 设计规范

本规范规定了海思 Soft IP Design Rules 的条款与细则。适用于海思内部自主开发的 IP 设计、与高校等外部单位合作开发的 IP 设计以及海思内部的代码编码规范。

本规范主要从命名规则、文件头规则、注释规则、编码风格规则、可综合规则、STA 相关规则、仿真规则、DFT 相关规则、设计风格规则共 9 个类别描述了对 IP 设计的规范要求。主要集中在阐述代码编写方面的要求，其中也附带提到了对某些重要内容的文档描述要求。

本规范根据设计流程的要求（RTL 编码、综合、仿真、STA、DFT 等）、可读性（命名、注释、文件头等）、可重用性（设计风格）等分了类。主要命名方法如下：Soft IP 简称 S，命名（Naming Convention）简称 NC，则 Soft IP 的命名相关规则第七条就简称为[S.NC.7]。便于检索与查阅。其它命名可依此缩写方式类推。

此外，本规范给设计规则分了级。便于让大家了解不同的设计规则的重要程度也不尽相同。

M1（Mandatory Level 1）——必须遵循的规则，如果没有遵循，设计无法完成预期功能和制作流程。

M2（Mandatory Level 2）——必须遵循的规则，如果没有遵循，必须提供相关的说明文档。

R（Recommended）——建议的规则。如果没有遵循，可能会给自己和他人带来不少额外的工作。

1. 设计规则总表

(1) Naming Convention Rules（命名规则）

表2-1 命名规则总表

	Naming Convention Rules 命名规则	Level 优先级
[S.NC.1]	A file must contain at most one module unit. 文件需要包含最多一个模块单元。	M2

	Naming Convention Rules 命名规则	Level 优先级
[S.NC.2]	File name should be identical to design unit name and in lowercase. 文件名称需要与设计模块名称保持一致且为小写。	R
[S.NC.3]	Only alphanumeric and underscore are allowed in naming. 命名中只允许出现字母、数字、下划线。	M1
[S.NC.4]	Names starting with a letter. 命名必须字母开头。	M2
[S.NC.5]	Names can not be distinguishable in letter case only. 命名不能只通过大小写来区分。	M1
[S.NC.6]	Use underscores to separate names composed of several words. 命名中不同单词使用下划线分隔。	R
[S.NC.7]	Use consistent signal names throughout the hierarchy 例化模块时，模块端口信号的命名和连接信号的命名不变。	M2
[S.NC.8]	Parameters and text macros for constant names are uppercase letters 参数和宏等常量大写。	M2
[S.NC.9]	Ports,signals,constructs names are lowercase letters. 端口、信号、结构名称小写。	M2
[S.NC.10]	Use meaningful names. 使用有具体含义的名称便于阅读。	R
[S.NC.11]	Consistent clock signal name. 时钟使用 clk 关键字命名。	M2
[S.NC.12]	Consistent reset signal name 复位使用 rst 关键字命名。	M2
[S.NC.13]	Consistent active-low signal name 低有效信号加_n 关键后缀命名。	M2
[S.NC.14]	For FSM variables, naming in <fsm_curr_st>, <fsm_next_st> 状态机当前状态以状态机名称+curr_st，状态机下一状态以状态机名称+next_st 表述。	R
[S.NC.15]	For latch variables, end in _lat. Latch 变量加_lat 后缀命名。	R
[S.NC.16]	For tri-state signals, end in _z. 三态信号使用_z 后缀命名。	R
[S.NC.17]	For asynchronous signals, end in _a 异步信号使用_a 后缀命名。	R
[S.NC.18]	Verilog and VHDL keywords are prohibited. 禁用 Verilog 和 VHDL 的关键字。	M1

	Naming Convention Rules 命名规则	Level 优先级
[S.NC.19]	The length of signal names does not exceed 20 characters. 信号的名称不超过 20 字符。	M2
[S.NC.20]	Descriptive and consistent abbreviation. 使用意义的缩写，如 Acknowledge 可缩写为 ack。	R
[S.NC.21]	Document abbreviations and additional naming conventions. 文件名使用 IP 的缩写并增加功能描述的命名。	R
[S.NC.22]	Instance names must be identical to the module name. 例化名称要与原模块名称一样，如果有多个，加后缀_num。	M2
[S.NC.23]	Consistent ordering of bits for describing multibit buses. 定义多 bit 总线使用降序。	M2
[S.NC.24]	For delayed registers, end in _ff1,_ff2,etc. 延迟的寄存器加_ff1、_ff2 后缀描述。	R
[S.NC.25]	Module and istance names are upper letters. 模块与模块例化名称使用大写。	R

(2) File Header Rules（文件头规则）

表2-2 文件头规则总表

	File Header Rules 文件头规则	Level 优先级
[S.FH.1]	Each file must include a file header. 文件必须包含文件头。	M1
[S.FH.2]	File header must start and end in file header boundary tags. 文件头必须使用文件头的边界符号包括。	M1
[S.FH.3]	Necessary fields for a file header. 必须要包含下面的域：	
[S.FH.3.1]	Include file name. 包含文件名。	M2
[S.FH.3.2]	Include author and contact information. 包含作者和联系方式	M2
[S.FH.3.3]	Include a release version and version description. 包含发布版本编号和版本描述。	M2
[S.FH.3.4]	Include a release date. 包含发布时间。	M2
[S.FH.3.5]	Include a purpose section. 包含文件的编写目的。	M2

	File Header Rules 文件头规则	Level 优先级
[S.FH.3.6]	Include a parameter description. 包含参数的描述。	M2
[S.FH.4]	Other header reuse issues. 其它可重用的信息，如包含的 function 等。	
[S.FH.4.1]	Include the reset strategy description. 包含复位策略描述。	R
[S.FH.4.2]	Include the clock strategy description. 包含时钟策略描述。	R
[S.FH.4.3]	Include the critical timing description. 包含关键路径描述。	R
[S.FH.4.4]	Include the test feature description. 包含测试特性描述。	R
[S.FH.4.5]	Include the asynchronous interface description if existed. 包含异步接口的描述，如果有的话。	R
[S.FH.4.6]	Include the scan methodology description. 包含扫描链策略描述。	R
[S.FH.5]	Additional construct should include a header. 额外的结构需要包含子文件头。	M2
[S.FH.6]	Additional construct header must start and end in construct header boundary tags. 额外的结构需要使用额外的边界符号包起来。	M2
[S.FH.7]	Necessary fields for additional construct header. 额外的结构需要包含下面的域：	
[S.FH.7.1]	Include construct name and data type of return values. 包含结构名称和数据类型的返回值。	M2
[S.FH.7.2]	Include construct type. 包含结构类型。	M2
[S.FH.7.3]	Include a purpose section. 包含结构的编写目的。	M2
[S.FH.7.4]	Include a parameter description. 包含参数的描述。	M2

(3) Comments Rules(注释规则)

表2-3 注释规则总表

	Comments Rules 注释规则	Level 优先级
[S.COM.1]	Use comments for port declarations. 端口定义加注释。	M1
[S.COM.2]	Use comments for internal signal declarations. 内部信号定义加注释。	R
[S.COM.3]	Use comments for functional sections. 独立功能的实现加注释。	R
[S.COM.4]	Use one-line comments instead of multiple-line comments. 注释使用单行。	R
[S.COM.5]	Use comments for cell instantiations. 例化 cell 加注释。	M2
[S.COM.6]	Delete unnecessary codes. 删除不必要的、冗余的代码。	R
[S.COM.7]	Use comments for synthesis directives. 综合指令加注释。	R
[S.COM.8]	Use comments for compiler directives. 编译指令加注释。	R

(4) Coding Style Rules(编码风格规则)

表2-4 编码风格规则总表

	Coding Style Rules 编码风格规则	Level 优先级
[S.CS.1]	Code should be aligned in a tabular format. 使用缩进的方式编写代码。	M2
[S.CS.2]	Use 4 space instead of tab stops for code indentation. 使用 4 个空格替代 Tab 键。	M2
[S.CS.3]	Use a separate line for each HDL statement. 禁止多个 HDL 语句写在同一行。	M1
[S.CS.4]	Use a separate line for each port declaration. 一个端口定义占用一行。	M1
[S.CS.5]	Preserve port order. 端口的定义与模块端口定义的次序一致。	M2
[S.CS.6]	Declare all internal nets in one section. 在一个地方定义所有的内部 net。	R
[S.CS.7]	Keep line length within 72 characters. 保持每行在 72 个字符。	R

	Coding Style Rules 编码风格规则	Level 优先级
[S.CS.8]	Use parameters over the 'define directives to specify configurable constants. 使用参数定义而不是宏定义来指定可配置的参数。	R
[S.CS.9]	Preserve relationships between constants. 参数间如果有关系(如倍数关系)，需要通过定义指明。	R
[S.CS.10]	Use explicit port mapping instead of positional association in module instantiation. 使用显式端口映射，禁止使用位置端口映射。	M1
[S.CS.11]	Avoid expression in port connections. 避免在端口连接时使用表达式(会产生 glue logic)。	M1
[S.CS.12]	Use parameters for FSM state encoding. 状态机的状态编码使用参数定义。	M1
[S.CS.13]	Use parentheses in complex equations. 复杂表达式要使用括号(不要测试编译工具的能力)。	M2
[S.CS.14]	Wire must be explicitly declared. 线型信号必须显式定义。	M1
[S.CS.15]	Operand size must match. 操作符左右的位宽必须匹配。	M1
[S.CS.16]	Expression in condition should be 1- bit value. 条件判断语句的表达必须是 1bit 的值。	R
[S.CS.17]	Finite State Machine (FSM) coding style. 状态机的编码风格使用组合逻辑与时序逻辑分开的方式。	R
[S.CS.18]	Port description first input , then output、inout; first clock and reset. 端口的描述，先 input 然后 output、inout；时钟和复位最先定义。	R
[S.CS.19]	“Assign” only used for changing name. Assign 表述只用作修改名。	M2
[S.CS.20]	Use “XXX_PROC” to label a process. 加入 XXX_PROC 标记一个进程。	M2
[S.CS.21]	Not use for loop. 不使用 for 循环。	R

(5) Synthesis Rules(可综合规则)

表2-5 编码风格规则总表

	Synthesis Rules 可综合规则	Level 优先级
[S.SYN.1]	Use only synthesizable statement. 只使用可综合的语句。	
[S.SYN.1.1]	Waveform statement are prohibited. 波形类型的语句禁止使用。	M1
[S.SYN.1.2]	System task and function for simulation (e.g. $display, $monitor, $printf, …) are prohibited 系统任务(如$display, $monitor, $printf, …)禁止使用。	M1
[S.SYN.1.3]	Wait statement and # delay statement are prohibited. wait 和# delay 禁止使用。	M1
[S.SYN.1.4]	Data type of real and event are prohibited. 实型数据类型和事件禁止使用。	M1
[S.SYN.1.5]	Only one clock per always sensitivity list. 每个 always 的敏感信号列表中时钟信号只使用 1 个。	M1
[S.SYN.1.6]	Loop must be in a static range. 循环语句 Loop 必须使用静态范围（循环次数必须是常量）。	M1
[S.SYN.2]	Avoid any embedded synthesis scripts. 避免内嵌的综合脚本。	M2
[S.SYN.3]	Avoid full_case and parallel_case synthesis directive statement. 避免 full_case 和 parallel_case 等直接综合的约束语句。	M2
[S.SYN.4]	Specify code fragment for combinational logic completely. 组合逻辑编码要完备(比如条件缺少、赋值缺少等都不容许)。	M2
[S.SYN.5]	Verilog primitives are prohibited. Verilog 保留字禁用。	M1
[S.SYN.6]	All unused module inputs must be driven. 不使用的模块输入必须保证有驱动。	M1
[S.SYN.7]	All unused module outputs should be connected. 所有不使用的输出必须保证有连接(如使用_nc 后缀线连接。)	R
[S.SYN.8]	Avoid top level glue logic. 避免顶层出现 glue logic。	M2
[S.SYN.9]	Incomplete case statements should have default case assignments. 条件不全的 case 语句必须有 default 赋值。	M2
[S.SYN.10]	Assign a default state to the state machines. 状态机必须有 1 个缺省状态。	R

(6) Static Timing Analysis Rules（STA 规则）

表2-6 STA 相关规则总表

	Static Timing Analysis Rules STA 规则	Level 优先级
[S.STA.1]	Avoid Combinational feedback loop. 避免出现组合逻辑反馈环。	M2
[S.STA.2]	Using synchronous design practices. 尽量使用同步设计。	M2
[S.STA.3]	Simplify register clock origin. 简化寄存器时钟域。	M2
[S.STA.4]	Avoid multicycle path and false path. 避免多时钟路径和伪路径。	M2
[S.STA.5]	Avoid clock as data. 避免时钟作为数据使用。	M2
[S.STA.6]	Avoid using latches. 避免使用 latch 锁存器。	M2
[S.STA.7]	If there is an asynchronous process.need a individual module to implement it. 如果有异步处理模块，需要用一个单独的模块去实现。	M2

(7) Simulation Rules（仿真规则）

表2-7 仿真规则总表

	Simulation Rules 仿真规则	Level 优先级
[S.SIM.1]	Use non-blocking assignments in sequential always block. 在时序 always 模块中使用非阻塞赋值语句。	M1
[S.SIM.2]	Use blocking assignments in combinational always block. 在组合 always 模块中使用阻塞赋值语句。	M1
[S.SIM.3]	Avoid missing sensitivity lists in combinational always block. 避免在组合 always 模块中敏感信号列表中缺少信号。	M1
[S.SIM.4]	Avoid redundant sensitivity lists. 避免敏感信号列表中出现冗余信号。	M1
[S.SIM.5]	Initialize control storage elements. 存储器内容需要初始化，避免仿真结果因仿真器而不同。	M2
[S.SIM.6]	Do not assign signals don't care (x) value. 不要使用 X 给信号赋值。	M1
[S.SIM.7]	Avoid using delay assignment. 不要使用 delay 延迟。	M2

(8) Design for Test Rules（DFT 相关规则）

表2-8 DFT 相关规则总表

	Design for Test DFT 规则	Level 优先级
[S.DFT.1]	Avoid tri-state devices. 避免使用三态的器件。	M1
[S.DFT.2]	Avoid bi-directional nets. 避免使用双向的 net。	M2
[S.DFT.3]	Avoid latches. 避免使用 latch。	M2
[S.DFT.4]	Avoid using both edges of clocks. 避免使用双沿时钟。	M2
[S.DFT.5]	Avoid gating clocks. 避免使用门控时钟。	M2
[S.DFT.6]	Avoid internal generated clocks. 避免使用内部产生的时钟。	M2
[S.DFT.7]	Avoid internal generated set/reset signals. 避免使用内部产生的置位/复位信号。	M2
[S.DFT.8]	Avoid using clocks and set/reset signals as data. 避免使用模块的时钟、置位/复位信号作为数据。	M2
[S.DFT.9]	Avoid combinational loop. 避免出现组合逻辑环。	M2
[S.DFT.10]	Avoid constant inputs or floating outputs. 避免常量输入或者浮空的输出。	R
[S.DFT.11]	Avoid clock or set/reset signals output directly. 避免直接的时钟或者置位/复位信号的输出。	R
[S.DFT.12]	Avoid using clock as set/reset signals. 避免使用时钟用作置位/复位信号。	R
[S.DFT.13]	Avoid asynchronous path loop. 避免出现异步路径环路。	R

(9) Design Style Rules（设计风格规则）

表2-9 设计风格规则总表

	Design Style 设计风格规则	Level 优先级
[S.D.1]	IP interface design issue. IP 接口设计相关要求。	

	Design Style 设计风格规则	Level 优先级
[S.D.1.1]	Output signals must be registered. 输出信号要寄存。	M2
[S.D.1.2]	Registers in an IP's interface should be clocked in a single edge of the same clock. IP 接口的寄存器必须是使用同一个时钟的同一个沿。	R
[S.D.1.3]	The interface of an IP should connect to a bus. IP 的接口必须连接到 Bus。	R
[S.D.1.4]	The detail information of the input to clock and clock to output signal delay must be planned and provided. 输入信号与时钟之间、时钟与输出信号之间的细节需要规划并具体给出。	M1
[S.D.2]	IP should be synchronous. IP 要求使用同步设计。	M2
[S.D.3]	Clock strategy must be documented. 时钟策略必须有成文的描述。	M1
[S.D.4]	Avoid handcrafted clock gating. 避免手工添加的门控时钟。	M2
[S.D.5]	Rules of reset in the IP. IP 的复位规则。	
[S.D.5.1]	IP must be able to reset. IP 必须能被复位。	M2
[S.D.5.2]	IP reset strategy must be documented. IP 的复位策略必须使用文档表述。	M1
[S.D.6]	Avoid any technology-dependent element. 避免任何依赖工艺的要素。否则必须文档说明。	M2

2. 设计规则细则（含 Case 库）

(1) Naming Convention Rules（命名规则）

I. [S.NC.1] A file must contain at most one module unit.

文件需要包含最多一个模块单元。

优先级：M2

细则：除了单独的宏定义文件不需要定义模块单元外，其它文件均定义一个且仅有一个模块单元。这样做的目的是保持文件结构的清晰。

II. [S.NC.2] File name should be identical to design unit name and in lowercase.

优先级：R

细则：文件的名称最好与设计模块名称保持一致且为小写。

不当的做法：

```
//////////////process.v///////////
module TEST(a,
        b,
        c);
. . .
endmodule
```

好的做法：

```
//////////////test.v///////////
module TEST(a,
        b,
        c);
. . .
endmodule
```

III. [S.NC.3] Only alphanumeric and underscore are allowed in naming.

命名中只允许出现字母、数字、下划线。

优先级：M1

细则：可以使用大写字母 A 到 Z，小写字母 a 到 z，数字 0 到 9，下划线。但是请注意，不要使用连续的下划线命名，不要大小写命名混用。

不当的做法：

```
01，GetSignal，wait___long___time;
```

好的做法：

```
addr，rd_addr，bank0，TIMER_PARA
```

IV. [S.NC.4] Names starting with a letter.

命名必须字母开头。

优先级：M2

细则：主要的原因：使用下划线开头的命名可能会导致语法错误；某些 EDA 工具也不允许这样的命名；以数字开头的命名可读性不好。

V. [S.NC.5] Names can not be distinguishable in letter case only.

命名不能只通过大小写来区分。

优先级：M1

细则：这不是好的编码习惯，代码可读性差，维护困难。

不当的做法：使用 ABC, abc, and aBc 来表示 3 个不同的命名。

VI. [S.NC.6] Use underscores to separate names composed of several words.

命名中不同单词使用下划线分隔。

优先级：R

细则：这样做是为了增强代码的可读性。例如表示一个 RAM 的地址。

不当的做法：

```
ramaddr
```

好的做法：

```
ram_addr
```

VII. [S.NC.7] Use consistent signal names throughout the hierarchy.

例化模块时，模块端口信号的命名和连接信号的命名不变。

优先级：M2

细则：例化模块时，模块端口信号的命名和连接信号的命名不变。便于 IP 的阅读理解。

不当的做法：

```
BLOCK1 U_BLOCK1 (
                .reset_n   (rst_n    ),
                .clock     (clk      ),
                .address   (addr     ),
                .write_data   (wr_data  ), // no good way
                . . .,
                );
```

好的做法：

```
BLOCK1 U_BLOCK1 (
                .rst_n    (rst_n     ),
                .clk       (clk      ),
                .addr     (addr      ),
                .wr_data  (wr_data   ),
                . . .,
```

```
);
```

VIII. [S.NC.8] Parameters and text macros for constant names are uppercase letters.

参数和宏等常量大写。

优先级：M2

细则：参数（parameter）多用来定义模块信号数据位宽、状态机状态命名等，需要大写。宏（define）多用来定义全局变量，也需大写。一般来讲，能够使用 parameter 定义的，尽量不要使用 define 来定义。因为 define 定义是全局变量，很容易与其它模块出现定义冲突，尽量慎重使用 define 语句。

好的做法：

```
`define DLY 1
`define AHB_TRANS_SEQ 2'b11
parameter BUS_WIDTH = 16;
```

IX. [S.NC.9] Ports,signals,constructs names are lowercase letters.

端口、信号、结构名称小写。

优先级：M2

细则：端口、信号、结构名称小写。

好的做法：

```
input       rst_n     ;
output      wr_data   ;
......
reg [7:0]   a_ff      ;
wire        we_n      ;
......
function    call_addr;
```

X. [S.NC.10] Use meaningful names.

使用有具体含义的名称便于阅读。

优先级：R

细则：尽量使用有具体含义的名称，通常受到信号长度的限制，一般会用到缩略语。

不好的做法：A1，XYZ 等。

好的做法：ram_addr 等。

XI. [S.NC.11] Consistent clock signal name.

时钟使用 clk 关键字命名。

优先级：M2

细则：时钟命名中包含 clk。

好的做法：hclk、clk_77m 等等。

XII. [S.NC.12] Consistent reset signal name.

复位使用 rst 关键字命名。

优先级：M2

细则：复位命名中包含 rst。

好的做法：rst_n、hrst_n、arst 等等。

XIII. [S.NC.13] Consistent active-low signal name.

低有效信号加_n 关键后缀命名。

优先级：M2

细则：低有效的信号使用_n 后缀命名。

好的做法：低有效复位定义为 rst_n。

XIV. [S.NC.14] For FSM variables, naming in <fsm_curr_st >, <fsm_next_st> .

状态机当前状态以状态机名称+curr_st，状态机下一状态以状态机名称+next_st 表述。

优先级：R

细则：状态机名称+curr_st 命名。状态机名称+next_st 命名。

好的做法：当前状态命名为 myfsm_curr_st、下一个状态命名为 myfsm_next_st。

XV. [S.NC.15] For latch variables, end in _lat.

Latch 信号名称加_lat 后缀命名。

优先级：R

细则：latch 信号的命名以_lat 命名。

好的做法：my_data_lat。

XVI. [S.NC.16] For tri-state signals, end in _z.

三态信号使用_z 后缀命名。

优先级：R

细则：三态信号的命名以_z 命名。三态信号均出现在外部端口上。

好的做法：data_z。

XVII. [S.NC.17] For asynchronous signals, end in _a.

异步信号使用_a 后缀命名。

优先级：R

细则：异步信号的命名以_a 命名。

XVIII. [S.NC.18] Verilog and VHDL keywords are prohibited.

禁用 Verilog 和 VHDL 的关键字。

优先级：M1

细则：由于 IP 应用的环境复杂，命名不当，很容易出现问题。初学者容易犯的一个错误：不能用"_reg"作为最后的后缀名，因为综合工具会给寄存器自动加上_reg，如果命名里就用_reg 作为后缀名则扰乱了网表的可读性。Verilog 的关键字：

```
always, and, assign, automatic, begin, buf, bufif0, bufif1,
case, casex, casez, cell, cmos, config, deassign, default,
defparam, design, disable, edge, else, end, endcase,
endconfig, endfunction, endgenerate, endmodule,
endprimitive, endspecify, endtable, endtask, event, for,
force, forever, fork, function, generate, genvar, highz0,
highz1, if, ifnone, incdir, include, initial, inout, input,
instance, integer, join, large, liblist, library, localparam,
macromodule, medium, module, nand, negedge, nmos, nor,
noshowcancelled, not, notif0, notif1, or, output, parameter,
pmos, posedge, primitive, pull0, pull1, pulldown, pullup,
pulsestyle_onevent, pulsestyle_ondetect, rcmos, real,
realtime, reg, release, repeat, rnmos, rpmos, rtran, rtranif0,
rtranif1, scalared, showcancelled, signed, small, specify,
specparam, strong0, strong1, supply0, supply1, table, task,
time, tran, tranif0, tranif1, tri, tri0, tri1, triand, trior,
trireg, unsigned, use, vectored, wait, wand, weak0, weak1,
while, wire, wor, xnor, xor
```

VHDL 的关键字：

```
Access, after, alias, all , assert, architecture, array,
attribute,begin , block, body, buffer, bus, case, component,
configuration, constant, disconnect , downto, elsif, end,
entity, exit,file, for, function,generate, generic, group,
guarded, impure, in, inertial, inout, is,label, library,
linkage, literal, loop, map, new, next, null,of, on, open,
others, out, package, port, postponed, procedure, process,
pure, range, record，register，reject，report，return，select，
severity，signal，shared，subtype，then，to，transport ,type，
```

unaffected，units，until，use，variable，wait，when，while，with，note，warning，error，failure

XIX. [S.NC.19] The length of signal names does not exceed 20 characters.

信号的名称不超过 20 字符。

优先级：M2

细则：信号的命名长度太长的话，ASIC 厂家定义的最大长度可能会无法满足要求，导致后端网表的信号会截短而找不到原来定义的长命名的信号。

XX. [S.NC.20] Descriptive and consistent abbreviation.

使用意义的缩写，如 Acknowledge 可缩写为 ack。.

优先级：R

细则：建议尽量使用有意义的缩写方法。

常用的命名缩略语见下表：

表2-10 常见的命名缩略语

全称	缩写	中文含义
acknowledge	ack	应答
Address	addr（ad）	地址
Arbiter	arb	仲裁
Check	chk	校验，如 CRC 校验
Clock	clk	时钟
Config	cfg	configuration，配置
Control	ctrl	控制
Count	cnt	计数
data in	din（di）	数据输入
data out	dout （do）	数据输出
Decode	de	译码
Decrease	dec	减一
Delay	dly	延迟
Disable	dis	不使能
Error	err	错误（指示）
Enable	en （e）	使能
Frame	frm	帧
Generate	gen	生成，如 CRC 生成
Grant	gnt	申请通过

全称	缩写	中文含义
Increase	inc	加一
Input	in （i ）	输入
Length	len	（帧、包）长
Nmport	nm	网管相关
Output	out（o）	输出
packet 不推荐 package	pkt	与帧相同
Priority	pri	优先级
Pointer	ptr	指针
rd enable	ren	读使能
Read	rd	读（操作）
Ready	rdy	应答信号或准备好
Receive	rx	（帧数据）接收
Request	req	（服务、仲裁）请求
Reset	rst	复位
Segment	seg	段
Source	scr	源（端口）
Statistics	stat	统计
Timer	tmr	定时器
switcher	sf	switch fabric
Temporary	tmp	临时
Transmit	tx	发送（帧数据）相关
Valid	vld（v）	有效、校验正确
wr enable	wen	写使能
Write	wr	写操作

XXI. [S.NC.21] Document abbreviations and additional naming conventions.

文件名使用 IP 的缩写并增加功能描述的命名。

优先级：R

细则：文件名均需要使用缩写，通常使用 IP 代号缩写＋”_”+功能描述缩写的方式。例如：

usb_clk_gen.v、ata_decoder.v 等。

XXII. [S.NC.22] Instance names must be identical to the module name.

例化名称要与原模块名称一样，如果有多个，加后缀_num。

优先级：M2

细则：如果例化一次，则使用”U_”+模块名称；如果例化多次，则使用“U_”+模块名称+num。

好的做法：

例化一次模块：

```
DECODER U_DECODER (…);
```

例如例化多次模块：

```
MUX4 U_MUX4_1 (…);
MUX4 U_MUX4_2 (…);
```

XXIII. [S.NC.23] Consistent ordering of bits for describing multibit buses.

定义多 bit 总线使用降序。

优先级：M2

细则：建议均使用降序排列。

好的做法：

32bit 的 sdram_addr 总线定义为 sdram_addr[31:0]。

XXIV.[S.NC.24] For delayed registers, end in _ff1,_ff2,etc.

延迟的寄存器加_ff1、_ff2 后缀描述。

优先级：R

细则：建议延迟的寄存器使用_ff1、_ff2 等后缀表述。

好的做法：

打一拍后的信号统一加后缀“_ff1”表示；

打二拍后的信号统一加后缀“_ff2”表示；

打三拍后的信号统一加后缀“_ff3”表示。

XXV. [S.NC.25] Module and instance names are upper letters.

模块与模块例化名称使用大写。

优先级：R

细则：模块例化大写。

好的做法：

```
module BLOCK1（…
BLOCK1 U_BLOCK1 (…);
```

(2) File Header Rules（文件头规则）

I. [S.FH.1] Each file must include a file header.

文件必须包含文件头。

优先级：M1

细则：详细做法见后面的文件头示例。

II. [S.FH.2] File header must start and end in file header boundary tags.

文件头必须使用文件头的边界符号包括。

优先级：M1

细则：文件头的边界符号使用“//**********”。详细做法见后面的文件头示例。

III. [S.FH.3] Necessary fields for a file header.

必须要包含下面的域：

优先级：M2

细则：详细做法见后面的文件头示例。

[S.FH.3.1] Include file name.

包含文件名。

[S.FH.3.2] Include author and contact information.

包含作者和联系方式。

[S.FH.3.3] Include a release version and version description.

包含发布版本编号和版本描述。

[S.FH.3.4] Include a release date.

包含发布时间。

[S.FH.3.5] Include a purpose section.

包含文件的编写目的。

[S.FH.3.6] Include a parameter description.

包含参数的描述。

IV. [S.FH.4] Other header reuse issues.

其它可重用的信息，如包含的 function 等。

优先级：R

细则：详细做法见后面的文件头示例。

[S.FH.4.1] Include the reset strategy description.

包含复位策略描述。

[S.FH.4.2] Include the clock strategy description.

包含时钟策略描述。

[S.FH.4.3] Include the critical timing description.

包含关键路径描述。

[S.FH.4.4] Include the test feature description.

包含测试特性描述。

[S.FH.4.5] Include the asynchronous interface description if existed.

包含异步接口的描述，如果有的话。

[S.FH.4.6] Include the scan methodology description.

包含扫描链策略描述。

V. [S.FH.5] Additional construct should include a header.

额外的结构需要包含子文件头。

优先级：M2

细则：详细做法见后面的文件头示例。

VI. [S.FH.6] Additional construct header must start and end in construct header boundary tags.

额外的结构需要使用额外的边界符号包起来。

优先级：M2

细则：详细做法见后面的文件头示例。

VII. [S.FH.7] Necessary fields for additional construct header.

额外的结构需要包含下面的域：

优先级：M2

细则：详细做法见后面的文件头示例。

[S.FH.7.1] Include construct name and data type of return values.

包含结构名称和数据类型的返回值。

[S.FH.7.2] Include construct type.

包含结构类型。

[S.FH.7.3] Include a purpose section.

包含结构的编写目的。

[S.FH.7.4] Include a parameter description.

包含参数的描述。

细则：见如下的文件头例子：

```
//*************************************************************
//
// Copyright(c) 2006, Hisilicon Technologies Co, Ltd
// All rights reserved.
//
// IP LIB INDEX     :   IP lib index just as UTOPIA_B
// IP Name          :   top module_name of this ip, usually, is same
//                  small ip classified name
// File Name        :   file_name of this file just as “tx_fifo.v”
// Module name      :   module_name of this file just as “TX_FIFO”
// Full name    :   complete English name of the abbreviated
//                      module_name
// Author       :   Author/ID
// Email            :   Author's email
// Data         :    2005/07/20
// Version      :    V1.0
//
// Abstract     :    *********
// Called by    :    Father module
//
// Modification history
// -------------------------------------------------------------
//
// $Log$
//*************************************************************
//*************************************************************
//+CHDR-------------------------------------------------------
--------------
// CLOCK and Reset Strategy
// CLOCK is clk , synchronous design.
// Reset is rst_n, which is low valid and is an asynchchous
signal.
//-CHDR-------------------------------------------------------
--------------
//+CHDR-------------------------------------------------------
--------------
// CONSTRUCT NAME & RETURN VALUES:
```

```
// task_write(haddr, hwrite, hwdata)
//-------------------------------------------------------------
--------------
// TYPE: task
//-------------------------------------------------------------
--------------
// PURPOSE: input address, write, data signal to execute
system write
//-------------------------------------------------------------
--------------
// PARAMETERS:
//PARAMETER NAME RANGE DESCRIPTION DEFAULT VALUE
//DATA_WIDTH [32,16] width of the data 32
//-CHDR---------------------------------------------------------
--------------
```

(3) Comments Rules(注释规则)

I. [S.COM.1] Use comments for port declarations.

端口定义加注释。

优先级：M1

细则：端口定义处一定要加入注释。

好的做法：

```
module  A       (
                ....
                out ,
                ....
                );
            ...
output  [n:0]  dout ;   // [n:0] , encode data out
            ...
reg     [n:0]   dout ;
            ...
```

II. [S.COM.2] Use comments for internal signal declarations.

内部信号定义加注释。

优先级：R

细则：内部定义和使用的信号如 Reg 和 Wire 类型要加注释。

好的做法：

```
            ...
reg     [n:0]   internal_register ;   // register definition
wire    [m:0]   internal_wire     ;   // wire definition
            ...
```

III. [S.COM.3] Use comments for functional sections.

独立功能的实现加注释。

优先级：R

细则：完成独立功能的 always 和 assign 等语句使用注释说明功能。

好的做法：

```
...
// my function comment here ...
always @(posedge clk or negedge rst_n)
...
// my function comment here ...
assign signal_A = register_B;
...
```

IV. [S.COM.4] Use one-line comments instead of multiple-line comments.

注释使用单行。

优先级：R

细则：使用单行注释//便于添加和删除注释。不要使用/**/。

好的做法：

```
// my function comment here ...
```

不当的做法：

```
/* my function comment here...*/
```

V. [S.COM.5] Use comments for cell instantiations.

例化 cell 加注释。

优先级：M2

细则：例化的 cell 如 RAM、IO 等必须有注释才容易被理解。

好的做法：

```
and U_AND_1 (a_out, sel_b, a); // here need to explain the
reason
```

VI. [S.COM.6] Delete unnecessary codes.

删除不必要的、冗余的代码。

优先级：R

细则：包括以前的代码、不必要的逻辑，保持代码的清晰整洁。最典型的一个例子，冗余的代码可能会被综合器报警告，淹没掉一些我们需要关注的综合警告。

VII. [S.COM.7] Use comments for synthesis directives.

综合指令加注释。

优先级：R

细则：综合指令，可以加入特别的注释。注意，这些语句对综合器是有依赖的，能不用尽量不用。

不建议的做法：

```
case (select) // synopsys full_case
    3'b000: y = 1'b1;
    3'b001: y = 1'b0;
    3'b101: y = 1'b0;
    3'b111: y = 1'b1;
endcase
```

VIII. [S.COM.8] Use comments for compiler directives.

编译指令加注释。

优先级：R

细则：例如想要设置一些仿真时需要加入的代码，保证满足仿真和实现的需要，加入注释注明。

好的做法：

```
`ifdef FOR_SIMULATION // show some test messages for debug
...
`else // NORNAL CONFIGURATION, i.e., for synthesis
...
`endif // end FOR_SIMULATION
```

(4) Coding Style Rules(编码风格规则)

I. [S.CS.1] Code should be aligned in a tabular format.

使用缩进的方式编写代码。

优先级：M2

细则：代码最高层次顶格写，子层次依次缩进 4 个空格。

好的做法：

```
if（rst_n==1'b1）
    bit_data <= 1'b0; //  从 if 的 i 字母起缩进 4 格书写此行代码。
```

II. [S.CS.2] Use 4 space instead of tab stops for code indentation.

使用 4 个空格替代 Tab 键。

优先级：M2

细则：代码中不允许出现 Tab 键。原因是不同系统下的编辑器对 Tab 键的显示并不一致。为了保证代码的清晰，统一使用 4 个空格替代 Tab 键。

III. [S.CS.3] Use a separate line for each HDL statement.

禁止多个 HDL 语句写在同一行。

优先级：M1

细则：禁止多个 HDL 语句写在同一行。不要试图测试编译器的能力，这会消耗不必要的工作量。例如：

好的做法：

```
out1 = a1 & b1; // AND operation
out2 = a2 | b2; // OR operation
```

不当的做法：

```
out1 = a1 & b1; out2 = a2 | b2;
```

IV. [S.CS.4] Use a separate line for each port declaration.

一个端口定义占用一行。

优先级：M1

细则：一个端口定义占用一行。不要将多个端口一起定义。例如：

好的做法：

```
input A; // port A description
input B; // port B description
```

不当的做法：

```
input A, B;
```

V. [S.CS.5] Preserve port order.

端口的定义与模块端口定义的次序一致。

优先级：M2

细则：端口的定义最好和模块端口定义的次序必须一致。保持比较好的可读性。

不当的做法：

```
module TEST (
              A,  // port A description
              B,  // port B description
              C   // port C description
);

input        B;  // port B description
input        A;  // port A description
output       C;  // port C description
```

好的做法：

```
module TEST  (
              A,   // port A description
              B,   // port B description
              C,   // port C description
);

input        A; // port B description
input        B; // port A description
output       C; // port C description
```

VI. [S.CS.6] Declare all internal nets in one section.

在一个地方定义所有的内部 net。

优先级：R

细则：需要在一个地方定义内部所有的 net。建议定义的位置在端口定义后面。

VII. [S.CS.7] Keep line length within 72 characters.

保持每行在 72 个字符。

优先级：R

细则：为了保证代码便于浏览和显示，每行不超过 72 个字符。原因是终端一行显示 80 个字符，如果显示行号，已经占用了 8 个字符的位置，所以只剩下 72 个字符来显示代码。对于超出长度的代码分多行来完成，使用回车分割，第一个非空格字符从原始行左侧第一个非空格字符起向右缩进 4 格开始，表示是该行的延续。

好的做法：

```
if (( ( (cnt == 3'b010 ) && ( cnt_num == 16'haa ))  ||
    (( sdramc_ready == 1'b1 ) && (sdramc_wen == 1'b1))    )
......
```

VIII. [S.CS.8] Use parameters over the 'define directives to specify configurable constants.

使用参数定义而不是宏定义来指定可配置的参数。

优先级：R

细则：如果使用`define 来指定可配置的参数。设计环境的其它模块很有可能定义了同样的参数而导致产生 redefinition（重复定义）的告警。参数是局部变量，则不会有此类的问题。

IX. [S.CS.9] Preserve relationships between constants.

参数间如果有关系(如倍数关系)，需要通过定义指明。

优先级：R

细则：好的做法:

```
parameter HALFWORD = 16;
parameter WORD = (2 * HALFWORD);
```

X. [S.CS.10] Use explicit port mapping instead of positional association in module instantiation.

使用显式端口映射，禁止使用位置端口映射。

优先级：M1

细则：模块的例化 Verilog 标准中准许使用 2 种映射。一种是显式端口映射，一种是位置端口映射。例如：

好的做法：

```
adder U_ADDER(
            .a (A),              // 隐含端口映射
            .b (B),
            .sum (SUM)
            );
```

不当的做法：

```
adder U_ADDER(
              A,               // 位置端口映射
              B,
              SUM
              );
```

XI. [S.CS.11] Avoid expression in port connections.

避免在端口连接时使用表达式(会产生 glue logic)。

优先级：M1

细则：端口连接时使用表达式是不允许的，这会产生 glue logic。例如：

好的做法和不当的做法：

```
foo U_FOO (
            .bad((m&n)^((p|q) > 4'hc)),  // bad which can
cause glue logic
            .good(good)                  // good
            );
```

XII. [S.CS.12] Use parameters for FSM state encoding.

状态机的状态编码使用参数定义。

优先级：M1

细则：使用参数定义状态，可以使状态机的可读性增强，便于理解设计。不要使用`define 的方式来定义状态，因为这是全局变量，很难确定其它模块是否使用同样的宏定义而导致错误。

好的做法：

```
parameter [1:0] ST_IDLE  = 2'b00; // symbolic state_vector
parameter [1:0] ST_IDLE  = 2'b00; // symbolic state_vector
parameter [1:0] ST_READ  = 2'b01; // symbolic state_vector
parameter [1:0] ST_WRITE = 2'b10; // symbolic state_vector
parameter [1:0] ST_WAIT  = 2'b11; // symbolic state_vector
```

XIII. [S.CS.13] Use parentheses in complex equations.

复杂表达式要使用括号(不要测试编译工具的能力)。

优先级：M2

细则：复杂表达式需要多使用括号来清晰的表述设计者的思路。虽然不使用括号也没有特别的问题，但是这样并没有带来什么好处，反而使得代码变得晦涩难懂，也考验了编译工具的能力。例如：

不当的做法：

```
if ( &a==1'b1 && !flag==1'b1 || b==1'b1)
```

好的做法：

```
if ((&a==1'b1) && (!flag==1'b1) || (b==1'b1))
```

XIV. [S.CS.14] Wire must be explicitly declared.

线型信号必须显式定义。

优先级：M1

细则：线型信号必须显式定义。位宽为 1bit 的线型信号不定义是可以被大多数编译器识别的，但是多 bit 的线型信号是必须要显式定义才行。所以还是那句老话，不要试图挑战编译器的能力。另外从语言的发展来看，类型的检查只能越来越严格。

不当的做法：

```
block u_block(
              .signal_a (int_signal_a), // int_signal_a 未定义
              .signal_b (int_signal_b)  // int_signal_b 未定义
              );
```

好的做法：

```
wire int_signal_a; // internal signal for block1 signal_a
wire int_signal_b; // internal signal for block1 signal_b
block u_block(
              .signal_a (int_signal_a),
              .signal_b (int_signal_b)
              );
```

XV. [S.CS.15] Operand size must match.

操作符左右的位宽必须匹配。

优先级：M1

细则：这个错误常会犯，赋值的时候要特别注意类型是否一致。例如：

不当的做法：

```
wire [32:0] signal_a; // signal_a with 33 bit width
wire [7:0] signal_b ; // signal_b with 8 bit width
…… // other codes
signal_x <= signal_a & signal_b;
```

好的做法：

```
wire [32:0] signal_a; // signal_a with 33 bit width
wire [7:0]  signal_b; // signal_b with 8 bit width
…… // other codes
signal_x <= signal_a[7:0] & signal_b;
```

XVI. [S.CS.16] Expression in condition should be 1- bit value.

条件判断语句的表达必须是 1bit 的值。

优先级：R

细则：条件判断语句的表达必须是 1bit 的值，否则会有多余的 bit 未引用。例如：

不当的做法：

```
reg [7:0] data; // 8bit data register
if (data) // data 为多比特值被用来作逻辑判断
data_enable = 1;
```

好的做法：

```
Reg [7:0] data; // 8bit data register
if (data > 0) // 此处为逻辑表达式
data_enable = 1;
```

XVII. [S.CS.17] Finite State Machine (FSM) coding style.

状态机的编码风格使用组合逻辑与时序逻辑分开的方式。

优先级：R

细则：对于 Mealy 状态机来说，一组合逻辑完成计算下一个状态的逻辑，二时序逻辑完成下一个状态的跳转。对于 Moore 状态机来说，一组合逻辑完成计算下一个状态的逻辑，一时序逻辑完成下一个状态的跳转。

好的做法：

```
......
module STATE4  (
        clock,
        reset,
        out
        );
input       reset;
input           clock;
output [1:0]   out;
......
parameter  [1:0] stateA = 2'b00;
parameter  [1:0]  stateB = 2'b01;
parameter  [1:0]  stateC = 2'b10;
parameter  [1:0]  stated = 2'b11;
......
reg     [1:0] curr_st;
```

```
reg     [1:0] next_st;
reg         [1:0] out;
......
// 时序逻辑描述 curr_st
always @ (posedge clock)
begin : CURR_ST_PROC
    if (reset == 1'b0)
        curr_st <= stateA;
    else
        curr_st <= next_st;
end
// 组合逻辑描述 next_st
always @(*)
begin : NEXT_ST_PROC
    case (curr_state)
        stateA: begin
            next_st = stateB;
        end
        stateB: begin
            next_st = stateC;
        end
        stateC: begin
             next_st = stateD;
        end
        stateD: begin
             next_st = stateA;
        end
default: next_st = stateA;
endcase
end
// 时序输出 curr_state 驱动其他逻辑
always@(postdge clock or negedge reset)
begin : OUT_PROC
   if(reset==1'b0)
       out <= 2'b0;
   else begin
```

```
        if(state==...)
            out<= ...;
        else
            out<=...;
    end
end
endmodule
```

XVIII. [S.CS.18] Port description first input , then output、inout; first clock and reset.

端口的描述，先 input 然后 output、inout；时钟和复位最先定义。

优先级：R

细则：端口的描述次序：input、output、inout。时钟和复位等影响全局的信号最好放在最前面。一个模块如果信号很多，最好按照与其它模块连接关系进行分组，便于读者理解和分析。

XIX. [S.CS.19] “Assign” only used for changing name.

Assign 表述只用作修改名。

优先级：M2

细则：建议在设计实现中除了使用 Assign 表述来修改信号名称以外，其它均使用 always 结构来表述，不提倡使用 assign 语句来实现组合逻辑。

XX. [S.CS.20] Use “XXX_PROC” to label a process.

加入 XXX_PROC 来标记一个处理进程。

优先级：M2

细则：见下面的示例。增加了 OUT_PROC，综合后的网表会生成这个标记，方便查找。

```
always@(postdge clock or negedge reset)
begin :OUT_PROC  // need a meaningful process name here
    if(reset==1'b0)
        out <= 2'b0;
    else begin
        if(state==...)
            out<= ...;
        else
            out<=...;
    end
```

```
end
endmodule
```

XXI. [S.CS.21] Not use for loop.

不要使用 for 循环。

优先级：R

细则：使用 For 循环，优点是代码简洁、参数化方便，缺点是综合工具、代码覆盖率等工具容易出错。建议禁止使用 For 循环。如可能需要使用循环的代码，建议使用脚本完成编写；参数配置的代码，建议使用脚本改写。

(5) Synthesis Rules(可综合规则)

I. [S.SYN.1] Use only synthesizable statement.

只使用可综合的语句。

优先级：M1

细则：只有可综合、可实现的设计才有被我们在这里讨论的价值。常见的不能综合的表达：

实数函数。

事件（尤其是关于时钟）。

系统函数（包括 OS 函数、PLI 函数、EDA 工具接口函数）。

用户自定义单元（UDP）。

宏单元。

disable 语句在 always block 中。

运算符＝＝＝和！＝＝。

整型、实型、无符号型、time、realtime 型数（其中定义单位延时和循环变量是个例外）

trireg (具有电荷保持特性连线） NET 型定义。

为 net、n_input、n_ouput、enable_gate 型变量定义驱动强度、电荷保持强度以及延时。（系统默认无效）

tri1、tri0、triand 和 trior 型连线（net）。

Initial、wait、force、release 、deassign 等表达。

过程语句中用 assign。

不要使用#delay 结构（在给寄存器赋值时例外），

[S.SYN.1.1] Waveform statement are prohibited.

波形类型的语句禁止使用。

优先级：M1

细则：与波形类型的语句都不能被综合。以 cadence 公司的 ncverilog 仿真器为例，要的与波形相关的语句有：

`$finish、$shm_open、$shm_probe、$shm_close`

[S.SYN.1.2] System task and function for simulation (e.g. $display, $monitor, $printf, …) are prohibited

系统任务(如$display, $monitor, $printf, …)禁止使用。

优先级：M1

细则：系统任务和函数包括了 OS 函数、PLI 函数、EDA 工具接口函数等。以 Solaris 8 系统为例，常用的系统任务有：

`$fopen、$fclose、$display、$monitor、$printf、$fwrite、$fread`

[S.SYN.1.3] Wait statement and # delay statement are prohibited.

wait 和# delay 禁止使用。

优先级：M1

细则：wait 一般是等待与非特定时钟相关的事件发生，无法使用带时钟端的标准寄存器器实现，因而也无法综合。＃delay 表达无法与综合单元建立联系，因而也无法综合。特别的，为了仿真的方便，有些人在寄存器的时序逻辑中加入了固定的延迟，但是由于 Synopsys、Mentor、ARM 等大的 IP 供应商并没有这样做（我们单独加入固定延迟可能会导致仿真问题），我们建议尽量不要加＃delay。

[S.SYN.1.4] Data type of real and event are prohibited.

实型数据类型和事件禁止使用。

优先级：M1

细则：实型数据类型和事件都无法用单元库表达，也就是无法综合。

[S.SYN.1.5] Only one clock per always sensitivity list.

每个 always 的敏感信号列表中时钟信号只使用 1 个。

优先级：M1

细则：因为触发器只有一个时钟端子，二个或者二个以上的时钟都是无法实现的。

[S.SYN.1.6] Loop must be in a static range

循环语句 Loop 必须使用静态范围（循环次数必须是常量）。

优先级：M1

细则：循环次数如果不是常量，综合器会迷惑究竟该实现几个同样的处理逻辑，只有常数值的循环才能让综合器做出正确的判断。循环语句建议尽量少使用，除非同样类型的逻辑很多才考虑使用 Loop 语句简化代码，增强代码的可读性。

II. [S.SYN.2] Avoid any embedded synthesis scripts.

避免内嵌的综合脚本。

优先级：M2

细则：避免一些隐含的综合脚本。如可执行的 Compile、Gen、Report 等直接可执行的脚本命令。这些脚本都会影响到综合脚本的正常执行。

III. [S.SYN.3] Avoid full_case and parallel_case synthesis directive statement.

尽量避免 full_case 和 parallel_case 等直接综合的约束语句。

优先级：M2

细则：这些语句对综合器是有依赖的，尽量少用。

```
case (select) // synopsys full_case
    3'b000: y = 1'b1;
    3'b001: y = 1'b0;
    3'b101: y = 1'b0;
    3'b111: y = 1'b1;
endcase  // not good
always @(signal_1 or signal_2 or signal_3)
begin
    case (1) //synopsys parallel_case
       signal_1: value = 2'b01;
       signal_2: value = 2'b10;
       signal_3: value = 2'b11;
     endcase // not good
end
```

IV. [S.SYN.4] Specify code fragment for combinational logic completely.

组合逻辑编码要完备(比如条件缺少、赋值缺少等都不容许)。

优先级：M2

细则：这些都容易产生 latch。

不好的做法：

```
//Missing condition:
always @ (a)
begin : A_PROC
     if (a == 1'b1)
          data_out = 1'b0;
```

```
end
// Missing assignment:
always @(condition)
begin : DATA_OUT_PROC
    case (condition)
        2'b00: data_out = 4'b0000;
        2'b01:
        2'b10: data_out = 4'b0011;
        2'b11:
     endcase
end
```

V. [S.SYN.5]Verilog primitives are prohibited.

Verilog 源语禁用。

优先级：M1

细则：Verilog 源语包括了用户自定义单元（UDP）和 Verilog 保留字。由于 IP 应用的环境可能涉及到非常多的内容。比如：不能用"_reg"作为最后的后缀名，因为综合工具会给寄存器自动加上_reg，如果命名里就用_reg 作为后缀名则扰乱了网表的可读性。

Verilog 的关键字：

```
always, and, assign, automatic, begin, buf, bufif0, bufif1,
case, casex, casez, cell, cmos, config, deassign, default,
defparam, design, disable, edge, else, end, endcase,
endconfig, endfunction, endgenerate, endmodule,
endprimitive, endspecify, endtable, endtask, event, for,
force, forever, fork, function, generate, genvar, highz0,
highz1, if, ifnone, incdir, include, initial, inout, input,
instance, integer, join, large, liblist, library, localparam,
macromodule, medium, module, nand, negedge, nmos, nor,
noshowcancelled, not, notif0, notif1, or, output, parameter,
pmos, posedge, primitive, pull0, pull1, pulldown, pullup,
pulsestyle_onevent, pulsestyle_ondetect, rcmos, real,
realtime, reg, release, repeat, rnmos, rpmos, rtran, rtranif0,
rtranif1, scalared, showcancelled, signed, small, specify,
specparam, strong0, strong1, supply0, supply1, table, task,
time, tran, tranif0, tranif1, tri, tri0, tri1, triand, trior,
trireg, unsigned, use, vectored, wait, wand, weak0, weak1,
while, wire, wor, xnor, xor
```

VI. [S.SYN.6] All unused module inputs must be driven.

不使用的模块输入必须保证有驱动。

优先级：M1

细则：不使用的模块输入必须保证有驱动，避免有浮空的输入，这会导致综合工具报告存在致命错误。

VII. [S.SYN.7]All unused module outputs should be connected.

所有不使用的输出必须保证有连接(如使用_nc 后缀)相连接。

优先级：R

细则：我们常常会碰到这种情况，一个模块的输出很有可能并没有全部使用。未连接的输出对实现一般不会有什么影响，但是很容易让 IP 的应用者感到迷惑！建议还是使用使用_nc 后缀线连接未连接的模块。例如：

```
// Sub_block description
module SUB_BLOCK(
                 rst_n          ,
                 clk            ,
                 multiplier_out,
                 divider_out
                 );
input            rst_n          ;
input            clk             ;
output [15:0]    multiplier_out ;
output [15:0]    divider_out     ;
```

不好的做法：

```
module top       (
                 rst_n          ,
                 clk             ,
                 control         ,
                 top_out
                 );
input            rst_n           ;
input            clk             ;
output [7:0]     top_out         ;
wire   [7:0]     temp            ;

SUB_BLOCK U_SUB_BLOCK (
                      .rst_n (rst_n),
                      .clk (clk),
                      .multiplier_out ({top_out, temp})
```

```
                        );
```

好的做法：

```
module TOP      (
                rst_n           ,
                clk             ,
                control         ,
                top_out
                );
input           rst_n           ;
input           clk             ;
output [7:0]    top_out         ;
wire   [7:0]    multiplier_out_nc;
wire   [15:0]   divider_out_nc  ;
SUB_BLOCK U_SUB_BLOCK (
                    .rst_n (rst_n),
                    .clk (clk),
                    .multiplier_out          ({top_out,
multiplier_out_nc}),
                    .divider_out (divider_out_nc)
                    );
```

VIII. [S.SYN.8]Avoid top level glue logic.

避免顶层出现 glue logic。

优先级：M2

细则：顶层出现 glue logic 的情况如下图：

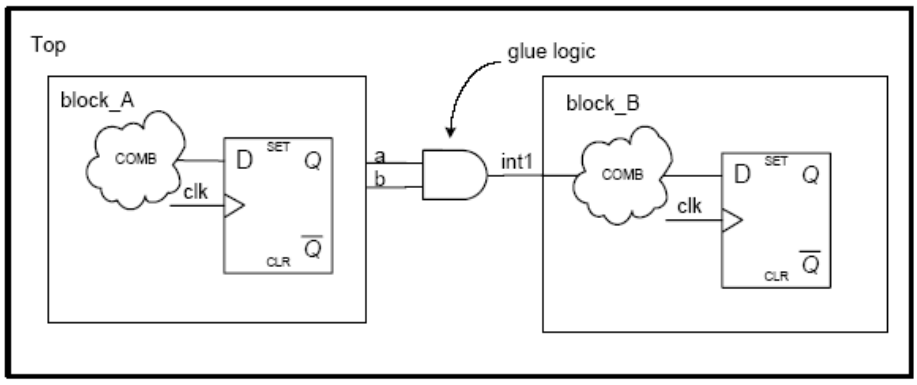


图2-1 顶层出现 glue logic

顶层出现的都应该是以模块例化的形式存在，不应该出现此类的 glue logic。glue logic 带来的不利后果是综合器无法将这些组合逻辑放入任何一个模块中进行优化。

IX. [S.SYN.9] Incomplete case statements should have default case assignments.

条件不全的 case 语句必须有 default 赋值。

优先级：M2

细则：条件不全的 case 语句必须有 default 赋值，否则会产生不必要的 latch。例如：

不当的做法：

```
always @(condition)
begin
    case (condition) // unexpected latch here
        2'b00: data_out = 4'b0000;
        2'b10: data_out = 4'b0010;
        2'b11: data_out = 4'b0011;
    endcase
end
```

好的做法：

```
always @(condition)
begin
    case (condition) // add default value
        2'b00: data_out = 4'b0000;
        2'b10: data_out = 4'b0010;
        2'b11: data_out = 4'b0011;
    default: data_out = 4'b0000;
endcase
end
```

X. [S.SYN.10]Assign a default state to the state machines.

状态机必须有 1 个缺省状态。

优先级：R

细则：状态机的状态设计中必须有一个缺省状态。状态机可能会被综合工具所优化，如果以前的状态在实际电路中不存在了，一旦状态寄存器进入该状态，

电路会导致意想不到的致命错误。在状态机中设定 1 个缺省状态，就确保了状态寄存器处于非法值时，仍可以进入正常工作。

(6) Static Timing Analysis Rules（STA 规则）

I. [S.STA.1] Avoid Combinational feedback loop.

避免出现组合逻辑反馈环。

优先级：M2

细则：逻辑反馈环如下图中间的组合逻辑：

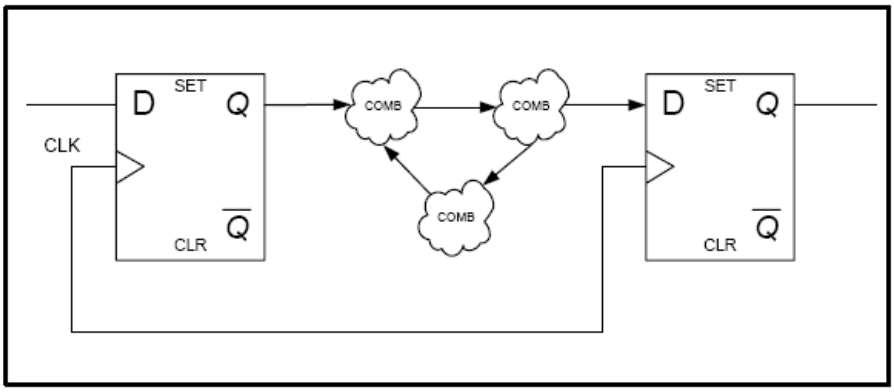


图2-2 逻辑反馈环示意图

逻辑反馈环会导致 STA 分析工具计算时序路径的延迟出现循环。如果因为出于设计需要有逻辑反馈环，可以使用 set_disable_timing 来避免逻辑反馈环时序检查出现的循环，另外 STA 工具也会自动停止该循环。

II. [S.STA.2] Using synchronous design practices.

尽量使用同步设计。

优先级：M2

细则：在无法避免异步设计的时候才使用异步设计，而且异步设计要尽量与同步设计分开；很多 STA 分析工具无法处理异步电路；异步电路设计在功能和时序两个方面均需要进行认真的验证。典型的异步设计电路如下图所显示：

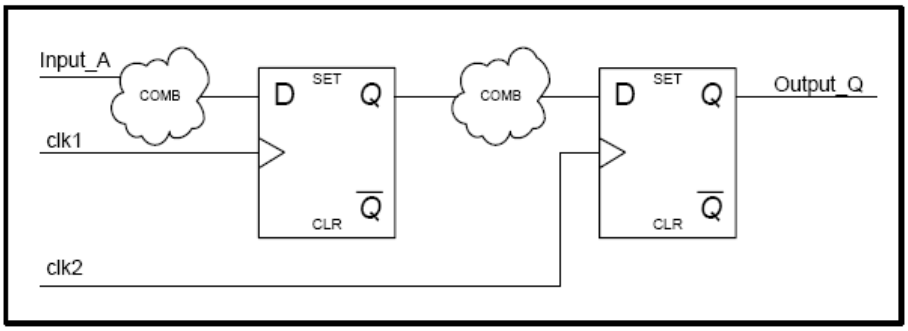


图2-3 典型的异步设计电路示意图

一般对于异步设计电路，STA 工具 PrimeTime 中会使用如下语句来定义异步时序路径：

```
sta_shell> set_false_path -from clk1 -to clk2
```

III. [S.STA.3] Simplify register clock origin.

简化寄存器时钟域。

优先级：M2

细则：典型的同步设计电路的寄存器时钟域示意图如下：

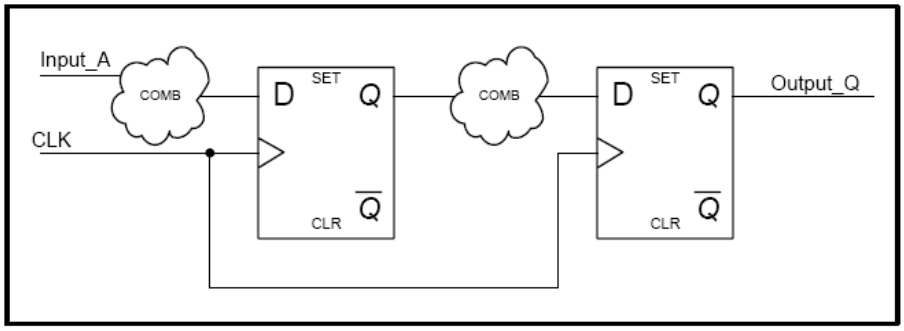


图2-4 典型的同步设计电路示意图

相关的提示和建议：

- 避免将内部产生的信号或者门控时钟连接到数据寄存器的时钟端上。
- 如果内部产生的时钟和门控时钟是出于低功耗等设计意图而无法避免，则尽量将该部分电路在电路顶层使用一个专门的时钟模块实现。
- 简化寄存器的时钟域，可以尽量避免在数据寄存时出现亚稳态。

IV. [S.STA.4] Avoid multicycle path and false path.

避免多时钟路径和伪路径。

优先级：M2

细则：

多时钟路径和伪路径属于 2 种异常的时序路径。

多时钟路径指的是从一级寄存器的输出到下一级寄存器的输入中间的这部分逻辑，需要 2 个时钟周期或者多个时钟周期才能完成该逻辑，而不是 1 个时钟周期内完成，例如多位乘法器、多位除法器等等处理逻辑。详见下图：

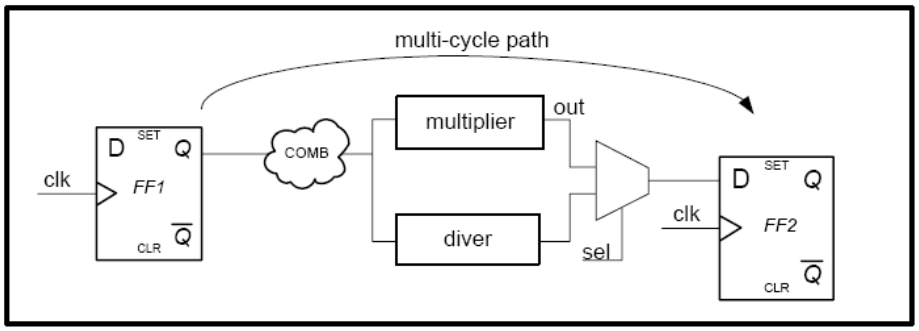


图2-5 多时钟路径电路示意图

伪路径指的是不需要时序检查工具检查的路径，数据在不同时钟域的转换电路通常需要设定伪路径，另外还有逻辑上不需要检查时序的路径也需要设定伪路径。详见下图：

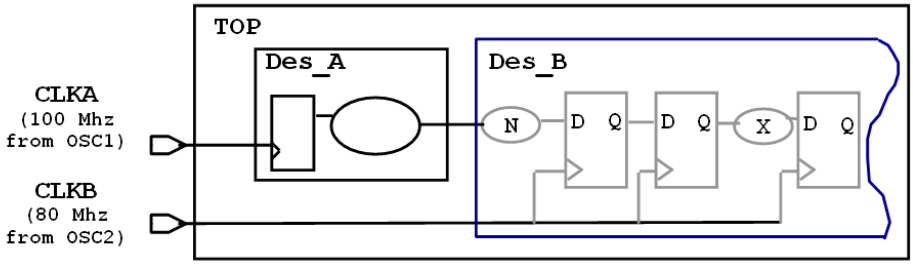


图2-6 伪路径电路示意图

V. [S.STA.5] Avoid clock as data.

避免时钟作为数据使用。

优先级：M2

细则：见下图。直接或者间接的将时钟作为数据输入会导致工具出现 Timing Violation 和竞争冒险的出现。一般的解决办法是插入 Buffer 来解决时序问题。

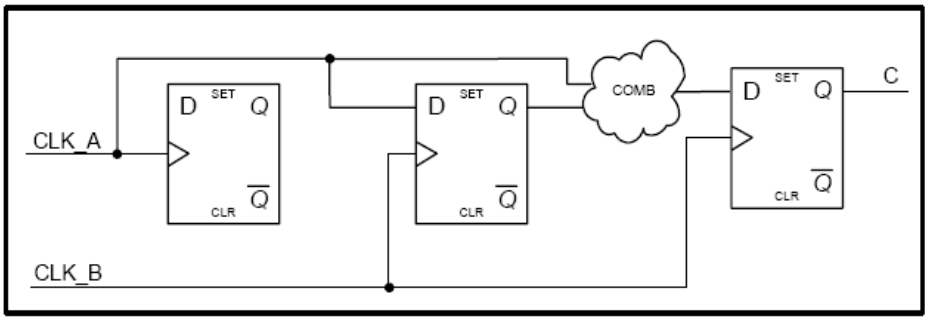


图2-7 时钟作为数据采样示意图

VI. [S.STA.6] Avoid using latches.

避免使用 latch 锁存器。

优先级：M2

细则：一般下面的几种情况会产生 Latch：

使用 if 选择语句的组合逻辑没有 else 部分表达。

使用 case 选择语句的组合逻辑没有 default 表达。

VII. [S.STA.7] If there is an asynchronous process.need a individual module to implement it.

如果有异步处理模块，需要用一个单独的模块去实现。

优先级：M2

细则：如果有异步处理模块，最好使用一个单独的模块去实现。这样做的好处是，简化 STA 的分析工作，避免在复杂的、大规模的电路中完成时序分析。

(7) Simulation Rules（仿真规则）

I. [S.SIM.1] Use non-blocking assignments in sequential always block.

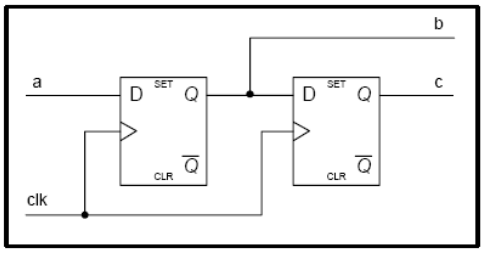
在时序 always 模块中使用非阻塞赋值语句。

优先级：M1

细则：如果在时序 always 模块中使用阻塞赋值语句，会导致时序紊乱。例如下图，一个信号 a 寄存一个时钟输出 b，b 寄存一个时钟周期输出 c，如果使用阻塞赋值，会导致 a 寄存一个时钟输出 b、c。

```
always @(posedge clk)
  begin
    b <= a;
    c <= b;
  end
```

```
always @(posedge clk)
  begin
    b = a;
    c = b;
  end
```



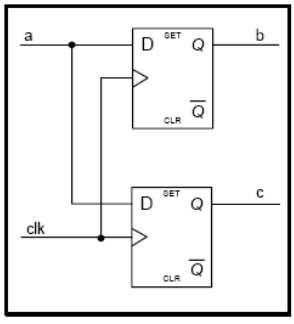


图2-8 时序 always 模块中使用非阻塞赋值(左)和阻塞赋值(右)

II. [S.SIM.2] Use blocking assignments in combinational always block.

在组合 always 模块中使用阻塞赋值语句。

优先级：M1

细则：在组合 always 模块中使用阻塞赋值会人为导致仿真器无缘无故的空运行。综合之后该代码并不会有什么问题。

III. [S.SIM.3] Avoid missing sensitivity lists in combinational always block.

避免在组合 always 模块中敏感信号列表中缺少信号。

优先级：M1

细则：这会导致综合前的仿真结果和综合以后的仿真结果不一致！

不好的做法：

```
always @ (a)
begin
    z = a or b;
end
```

IV. [S.SIM.4] Avoid redundant sensitivity lists.

避免敏感信号列表中出现冗余信号。

优先级：M1

细则：这会导致仿真器作多余的判断操作而延长仿真的时间。

不好的做法：

```
always @ (a or b or c)
begin
    z = a or b;
end
```

V. [S.SIM.5] Initialize control storage elements.

存储器内容需要初始化，避免仿真结果因仿真器而不同。

优先级：M2

细则：所有的存储单元，包括了 latch 和寄存器，它们很可能会处于控制电路的控制路径上，存储器内容的初始化就必须要被复位或者置位。每个没有被初始化的 latch 或者寄存器的初始值会因为仿真器的不同而不同，仿真结果依赖仿真器是我们不愿意看到的！

VI. [S.SIM.6] Do not assign signals don't care (x) value.

不要使用 X 值给信号赋值。

优先级：M1

细则：这个要求主要是为了防止 X 态的扩散。因为如果被赋值的信号是个一个控制信号，它参与的逻辑结果均为 X 态，这样导致很多信号出现 X 态。

VII. [S.SIM.7]Avoid using delay assignment.

不要使用 delay 延迟。

优先级：M2

细则：由于综合器是忽略这些延迟的，所以如果加入该延迟，会导致前仿真和后仿真的结果会不一致。

(8) Design for Test Rules（DFT 相关规则）

I. [S.DFT.1] Avoid tri-state devices.

避免使用三态的器件。

优先级：M1

细则：在测试过程中，为了满足可控性原则，电路所有节点的状态必须确定。总线冲突和总线漂移会导致电路的不确定状态。因此，在电路设计中要避免在电路内部出现总线漂移和总线冲突现象。对于三态总线如图所示，如果同时有两个三态驱动有效，会导致总线冲突，如果某一时刻没有任何三态驱动有效，会导致总线漂移。

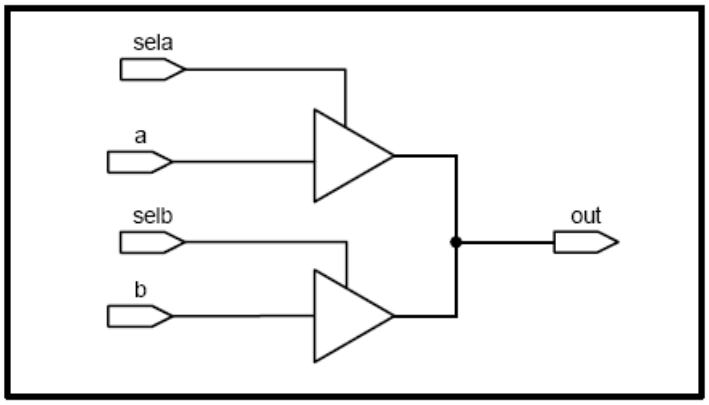


图2-9 三态控制器件示意图

为了避免出现总线漂移和总线冲突，内部三态总线每个时刻只能仅仅一个三态驱动有效。对于单个三态门，不会出现总线冲突，但是仍然有总线漂移问题。显然对于上图，如果 sela=1，a 的值可以通向到 OUT，但如果 sela=而且 b 无输入，在这种情况下，OUTA 的状态是高阻，也就是说，状态是漂移的，见下图。这也表明对于三态门，其使能端口 sela 是不可测的。我们建议，尽量避免在电路内部使用三态总线。

II. [S.DFT.2] Avoid bi-directional nets.

避免使用双向的 net。

优先级：M2

细则：避免使用双向的 net，主要原因是 ATPG 工具无法正确判断双向信号的方向而导致无法正确产生测试向量（Test Pattern）。如下图：

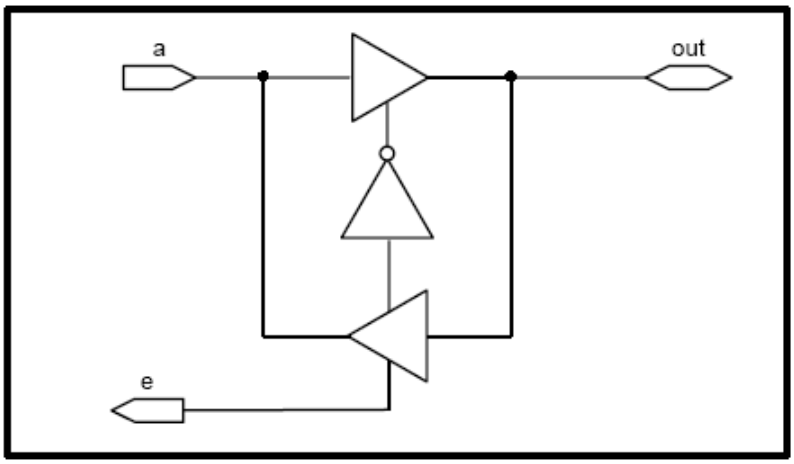


图2-10 双向 net 器件示意图

III. [S.DFT.3] Avoid latches.

避免使用 latch。

优先级：M2

细则：latch 会给 dft 产生很多不利的影响。例如：

DFT 中的扫描测试设计存在全扫描模式（Full scan）和部分扫描模式（Partial scan）这 2 种主要方式。全扫描模式要求在插 DFT 链过程中将触发器用带扫描功能的触发器替代，而 latch 是不能够做成可扫描的，而只能被当作黑盒处理。对于 latch 的处理，可以采用 bypass 方法，或采用透明法处理（如下图），latch 的存在显然增加了 DFT 的工作量，降低了测试覆盖率。

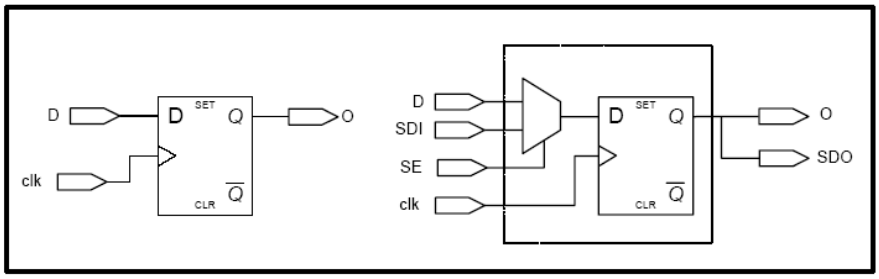


图2-11 透明法处理 latch 电路

IV. [S.DFT.4] Avoid using both edges of clocks.

避免使用双沿时钟。

优先级：M2

细则：使用双沿时钟（见下图）使得将这些 DFF 寄存器串成 DFT 链变得复杂。针对双沿时钟主要有 2 种主要的解决办法：

在上升沿与下降沿 DFF 寄存器之间插入保护电路。

将分为上沿与下沿的 DFF 寄存器分配到不同的测试链。

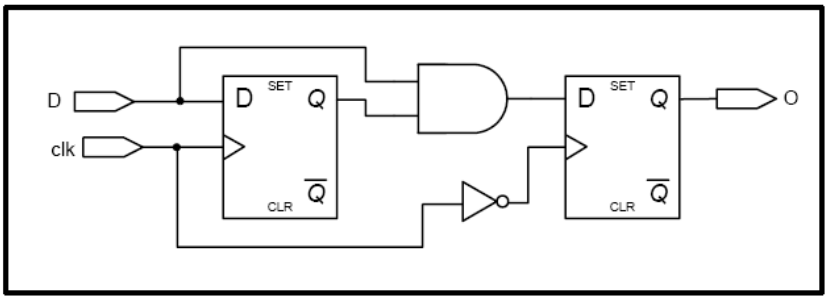


图2-12 使用双沿时钟电路示意图

V. [S.DFT.5] Avoid gating clocks.

避免使用门控时钟。

优先级：M2

细则：DFT 的一个基本原则就是时钟信号可控。基于 ATPG 的扫描测试要求被测电路的所有触发器都用特殊设计的具有扫描功能的触发器代替，使其在测试时链接成一个或几个移位寄存器链，这样，电路分成了可以分别进行测试的纯组合电路和移位寄存器，电路中的所有状态可以直接从原始输入和输出端得到控制和观察，很明显，所有触发器的时钟都是要求外部可控的。门控制时钟往往会切断扫描移位操作，对 DFT 明显不利！

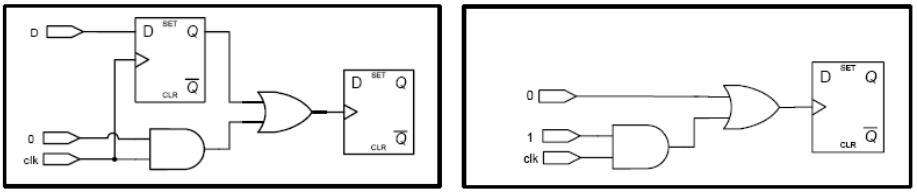


图2-13 门控时钟电路示意图

VI. [S.DFT.6] Avoid internal generated clocks.

避免使用内部产生的时钟。

优先级：M2

细则：避免内部产生的时钟。内部产生的时钟使得扫描链上的数据变得不同步，破坏了扫描操作，这样会降低失效覆盖率（fault coverage）。例如：

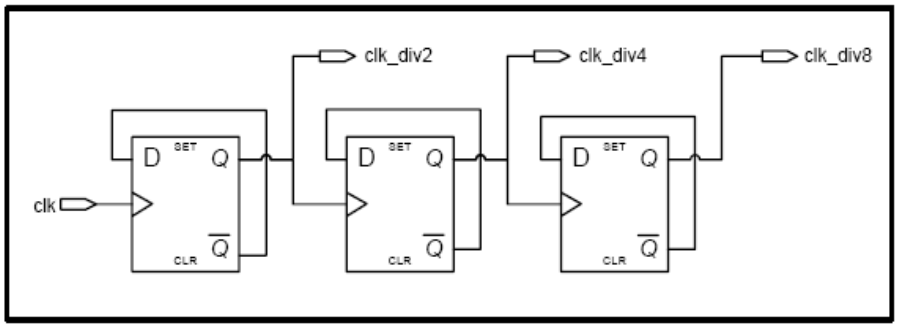


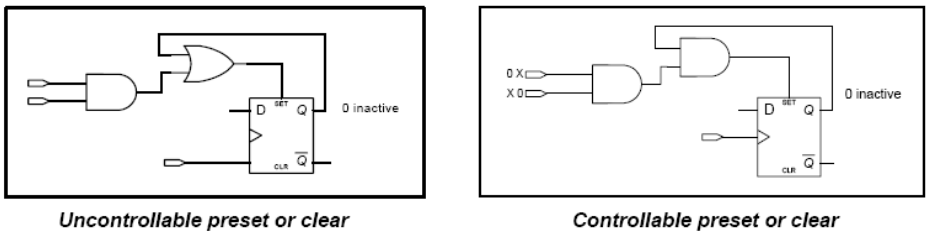
图2-14 内部产生时钟电路示意图

VII. [S.DFT.7] Avoid internal generated set/reset signals.

避免使用内部产生的置位/复位信号。

优先级：M2

细则：寄存器的置位和复位要被基本输入信号所直接控制。在扫描操作中，它们都要被置为无效。内部产生的置位和复位信号可能会在扫描移位操作中破坏移位数据的完整性。原因是扫描操作要求置位和复位端口保持在无效的状态下保证数据不受影响。，而内部产生的置位和复位信号无法直接控制。例如：



Uncontrollable preset or clear *Controllable preset or clear*

图2-15

VIII. [S.DFT.8] Avoid using clocks and set/reset signals as data.

避免使用模块的时钟、置位/复位信号作为数据。

优先级：M2

细则：避免模块的时钟、置位/复位信号作为数据。以时钟为例子，如下图。当时钟信号用作触发器的数据输入时，出现这种情况时，ATPG 工具在 capture 阶段无法得到有效数据，因此在 Design Rule Check 时会报 violation 的（详细的介绍见 synopsys 的在线文档）。对这样问题的处理是修改设计，增加 mux 逻辑，使得在测试模式下，数据路径同时钟分开。

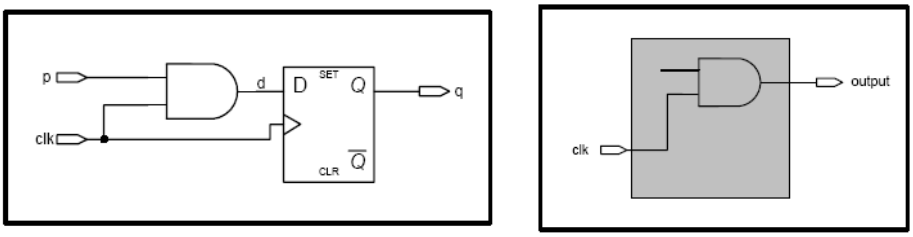


图2-16 时钟作为数据影响 DFT

IX. [S.DFT.9] Avoid combinational loop.

避免出现组合逻辑环。

优先级：M2

细则：组合反馈环是扫描测试技术所无法解决的。主要问题有：

- 使得组合逻辑电路会不停地产生连续动作，这样就很难使用组合 ATPG 工具；
- 可能会导致竞争和冒险，产生无法预计的电路动作。
- 环路通常是与延迟相关的，任何 ATPG 算法都无法对其进行测试。

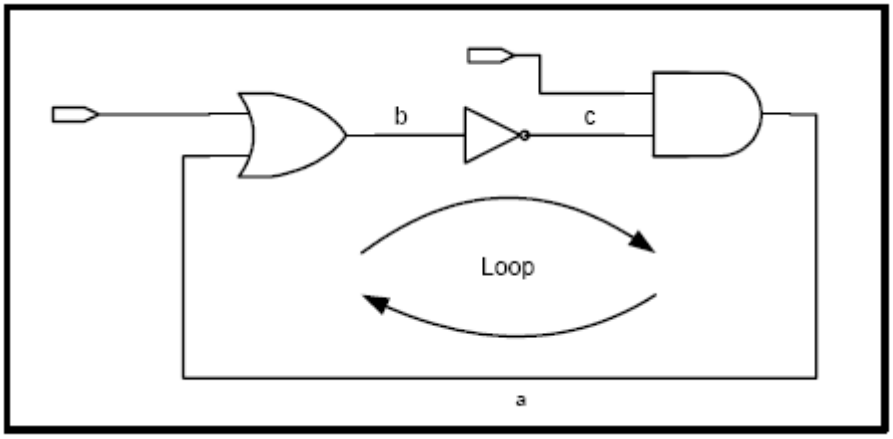


图2-17 组合逻辑环示意图

在一定情况下，这些环可能是无意中建立的。当多位设计者参与同一设计时，这种情况就可能出现。例如，当 IC 的总线很多时，可能有各种信号接到总线。每个设计者在其设计的部分都尽量避免产生组合反馈环；但即使如此，当两部分电路组合时还是有可能形成组合反馈环。避免出现跨模块的组合逻辑环较好的一种办法是保持寄存输入，输出寄存的良好编码习惯。

如果组合反馈环无法避免，DFT 会增加测试逻辑，在测试状态时强制性地阻断反馈环，但这样无疑会增加 DFT 的复杂程度。

X. [S.DFT.10] Avoid constant inputs or floating outputs.

避免常量输入或者浮空的输出。

优先级：R

细则：当我们使用已有的设计集成到新的设计中的时候，常常会碰到此问题。由于我们无法控制常量的输入和观察浮空的输出，这会降低电路的可控制性和可观测性，从而降低了失效覆盖率（fault coverage）。

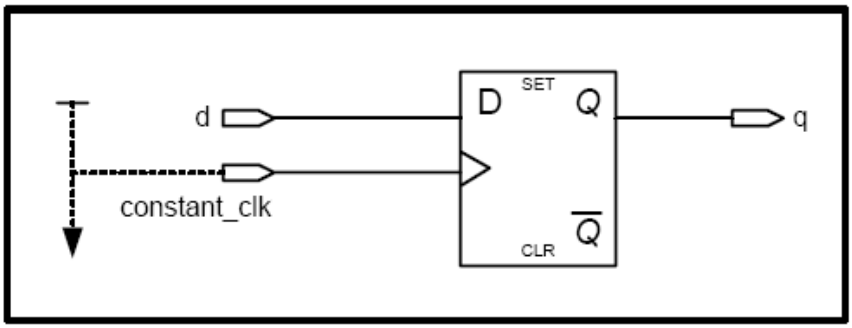


图2-18 浮空的输入示意图

XI. [S.DFT.11] Avoid clock or set/reset output directly.

避免直接的时钟或者置位/复位信号的输出。

优先级：R

细则：时钟直接输出到管脚上，由于时钟的不断变化，ATPG 工具会在 capture 阶段无法得到有效数据，因此在 Design Rule Check 时会报 violation 的（详细的分析见 synopsys 等公司的 ATPG 工具文档）。在设计中也会碰到复位/置位信号直接输出到芯片外部的情况，这和时钟信号直接输出到芯片外部一样。

但是设计还是会经常碰到时钟直接输出到芯片外部的情况，如下图：

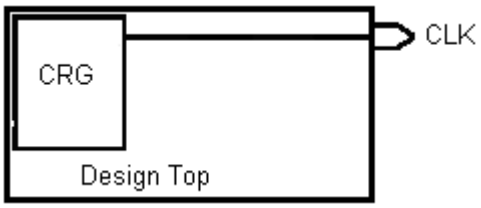


图2-19 时钟直接输出到管脚示意图

对这样问题的处理是修改设计，增加 mux 逻辑，使得在测试模式下，数据路径同时钟路径分开，如下图：

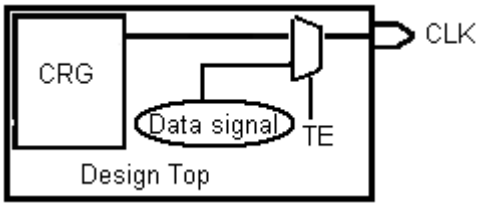


图2-20 时钟直接输出到管脚的 DFT 处理示意图

XII. [S.DFT.12] Avoid using clock as set/reset signals.

避免使用时钟用作置位/复位信号。

优先级：R

细则：时钟信号做复位，如果该复位信号不做 DFT 处理的话，由于时钟的不断变化，引起相应的寄存器被反复复位，因此 ATPG 工具在 capture 阶段无法得

到有效数据，因此在 Design Rule Check 时会报 violation 的（详细的介绍见 synopsys 的在线文档）。

在设计中有时可能会发现时钟信号被连接到寄存器的复位端口，如下图：

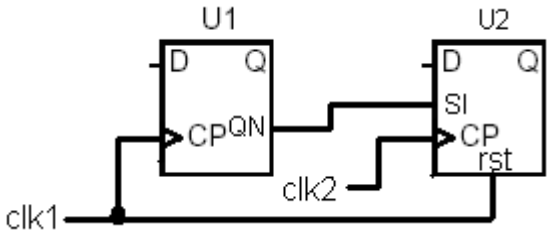


图2-21 时钟作复位信号使用示意图

对这样问题的处理是修改设计，增加 mux 逻辑，使得在测试模式下，复位信号连接到外部复位管脚上，如下图：

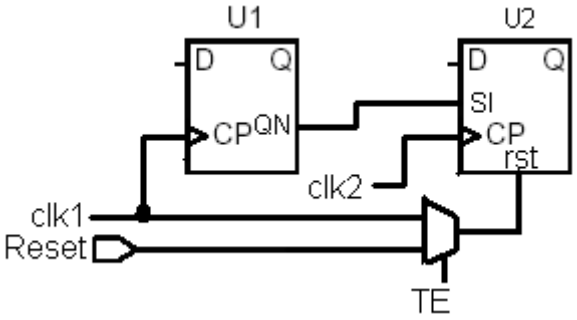


图2-22 时钟作复位信号使用的 DFT 处理示意图

XIII. [S.DFT.13] Avoid asynchronous loop.

避免出现异步环路。

优先级：R

DFT 的可控制性要求，在输入激励确定的情况下，大部分异步环路中的 nodes 的值是无法确定的，扫描测试技术（及 APTG 工具）无法解决这个问题。另外，异步环路可能会导致竞争和冒险，这也可能导致无法预计的电路动作。而且环路通常是与延迟有关的，这样用任何的 ATPG 算法都是无法进行测试的。

举个实例来说明这个问题。下面是一个关于异步环路及的 RTL 级代码及其综合后的一种门级电路。

如下面的电路：

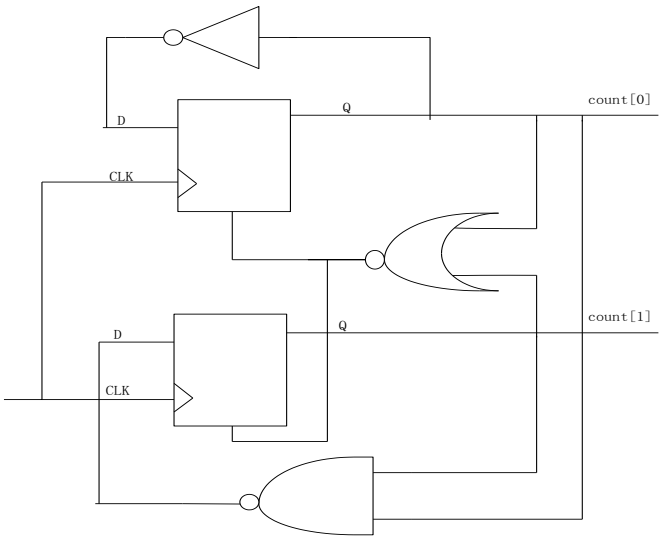


图2-23 异步环路示意图

在设计中应尽量避免异步环路，在一定情况下，这些环可能是无意中建立的。当多位设计者参与同一设计时，这种情况就可能出现。但象上图这样的异步环路，寄存器的 reset 端其实是不可控的，DFT 方面只需要增加 MUX 将外部 reset 信号连接起来就可以实现可控。

(9) Design Style Rules（设计风格规则）

I. [S.D.1] IP interface design issue.

IP 接口设计相关要求。

细则：见后面各条子规则。

II. [S.D.1.1] Output signals must be registered.

输出信号要寄存。

优先级：M2

细则：如下图。输出信号寄存可以改善 IP 的 input delay 和输出驱动能力。

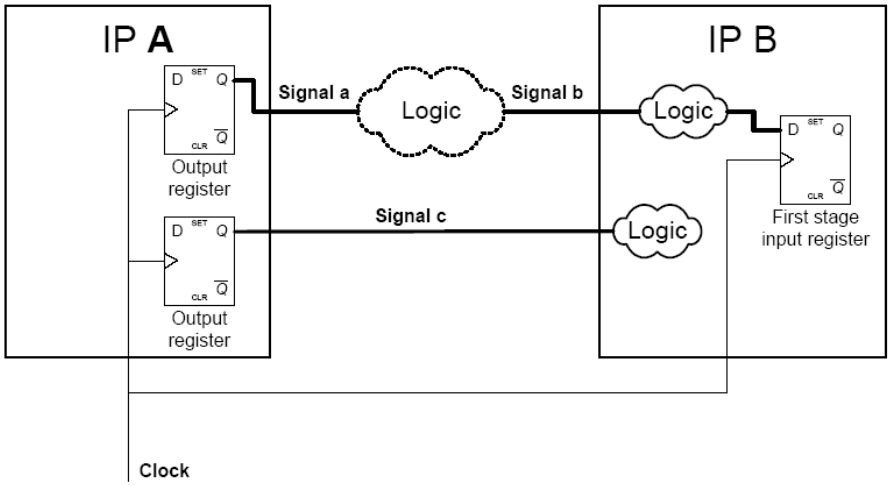


图2-24 输出信号寄存示意图

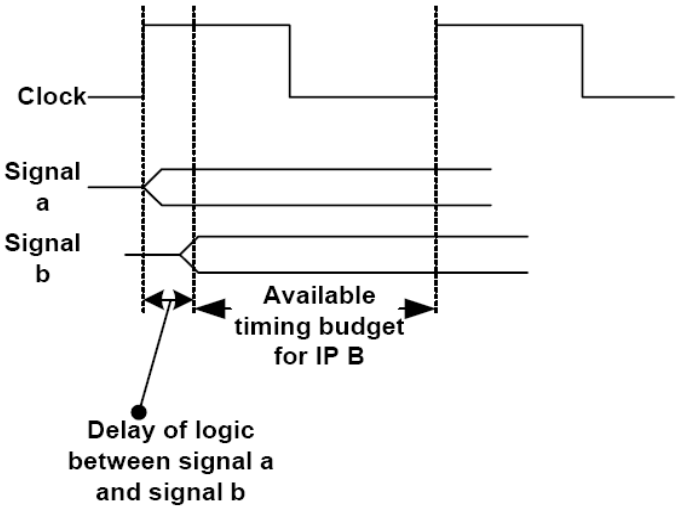


图2-25 输出信号寄存器时序分析图

III. [S.D.1.2] Registers in an IP's interface should be clocked in a single edge of the same clock.

IP 接口的寄存器必须是使用同一个时钟的同一个沿。

优先级：R

细则：这降低了访问 IP 接口的逻辑设计的复杂性，使得 IP 的集成环境设计变得容易。

IV. [S.D.1.3] The interface of an IP should connect to a bus.

IP 的接口必须连接到 Bus。

优先级：R

细则：IP 最好能通过共享的系统总线与系统的其它部分相连接。如果不能使用点到点的连接或者总线的连接，那么就必须遵循一定的工业标准。这是 IP 的通

用性方面的考虑。例如一个 USB1.1 Device IP，一边可提供 AMBA APB 外设总线接口，另外一边是遵循 USB 1.1 相关工业标准的 USB 接口。

V. [S.D.1.4] The detail information of the input to clock and clock to output signal delay must be planned and provided.

输入信号与时钟之间、时钟与输出信号之间的细节需要规划并具体给出。

优先级：M1

细则：特别是输入信号与时钟之间的时序关系，时钟与输出信号之间的时序关系，需要很具体的给出。最好使用图表的方式给出来。如图：

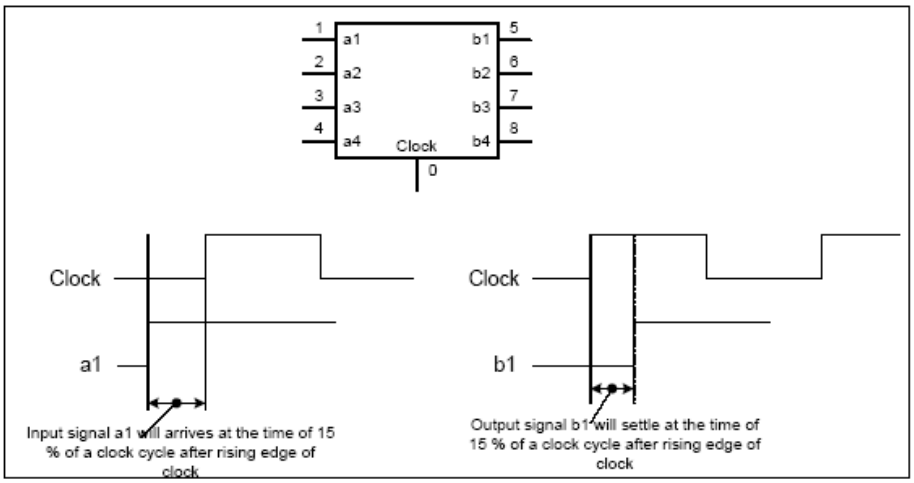


图2-26 输入信号与时钟之间、时钟与输出信号之间的关系示意图

VI. [S.D.2] IP should be synchronous.

IP 要求使用同步设计。

优先级：M2

细则：尽管异步设计在低功耗和高性能设计方面存在优势，但是这会导致亚稳态!异步设计将作为特殊的设计方法来讨论，一般的设计规则还是遵循同步设计的原则。

VII. [S.D.3] Clock strategy must be documented.

时钟策略必须有成文的描述。

优先级：M1

细则：时钟策略必须有成文的描述，放在设计方案等文档中，最好辅助使用图的方式来表述。例如下图：

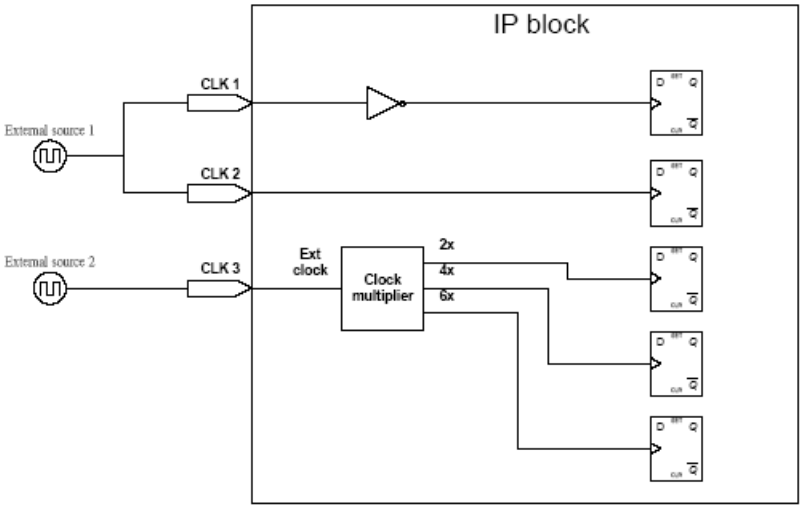


图2-27 时钟策略示意图

VIII. [S.D.4] Avoid handcrafted clock gating.

避免手工添加的门控时钟。

优先级：M2

细则：设计者不要手工添加门控时钟。如果出于低功耗设计的考虑，则尽量让低功耗优化工具（例如 Synopsys 公司出品的 Power Compiler）自动将不带门控时钟的寄存器更换为带门控时钟的寄存器。

下面是手工添加的门控制时钟代码：

```
// Do not use handcrafted clock gating
assign gate_clock= clock & gated_signal;
always @(posedge gate_clock)
{
    assign data_reg_1 <=data_in;
}
```

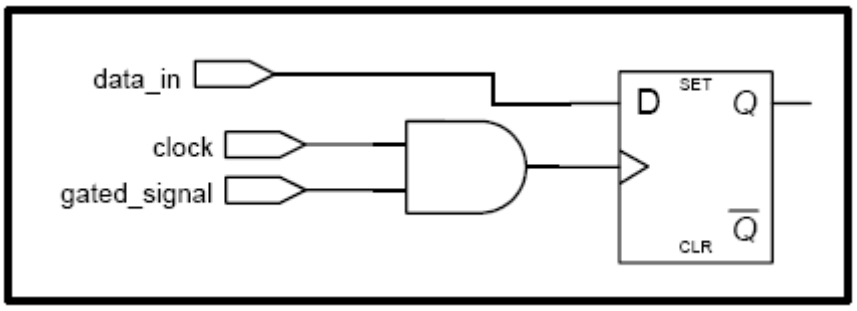


图2-28 手工添加的门控时钟示意图

IX. [S.D.5] Rules of reset in the IP.

IP 的复位规则。

细则：见后面的细则。

[S.D.5.1] IP must be able to reset.

IP 必须能被复位。

优先级：M2

细则：IP 必须能够被复位而保证 IP 能够被应用 IP 的系统所完全控制。

[S.D.5.2] IP reset strategy must be documented.

IP 的复位策略必须使用文档表述。

优先级：M1

细则：复位的策略必须表述在设计方案等文档中。

X. [S.D.6] Avoid any technology-dependent element.

避免任何依赖工艺的要素。否则必须文档说明。

优先级：M2

细则：可能存在的情况：

- 例化特殊工艺库单元，如 BUFFER、反相器、RAM 等。其中规模不大的 RAM 建议可直接使用寄存器搭建。
- 特殊的用户自定义单元，如特别的硬核 IP。

2.1.2 Verilog 2001 可综合语法介绍

概述：现在大家最常用的 verilog 语法都是基于 Verilog 95 的，推出的 verilog 2001 语法在公司还没有全面使用起来，本文通过对 verilog 2001 语法的介绍，让大家了解 2001 语法和 95 的不同之处，让大家能合理的选择 verilog 语法，方便设计，提高效率。

本章节介绍的 Verilog2001 语言用于扩展读者的视野，请大家在开发过程过仍然采用 Verilog 95 的标准语言。

1. 端口定义

在 Verilog2001 语法中，可以将端口的类型定义合并，以减小代码的冗长。如在 verilog95 中，定义一个端口，通常要先定义端口的输入输出类型，然后再定义端口的数据类型。

```
#verilog95
module encoder （clk, rst_n, a, b, c, sel, d, mux_out）;
input      clk, rst_n, a, b,c,d;
input [1:0]   sel;
output         mux_out;      //定义输入输出
reg            mux_out;      //定义数据类型
```

而在 verilog2001 中，端口定义在一条语句内就可以完成。

```
#verilog2001
module encoder （clk, rst_n, a, b, c, sel, d, mux_out）;
input              clk, rst_n, a, b,c,d;
input        [1:0]    sel;
output   reg          mux_out;
```

此外，verilog2001 还支持将端口类型的定义放在端口列表中。

```
module encoder （ input clk, rst_n, a, b, c, d,
        input       [1:0]   sel,
        output reg          mux_reg）;
```

2. 敏感列表

在 verilog95 中，在敏感列表中的多个信号是通过关键字 or 来区分的。

```
#verilog95
always @（a or b or cin）
   sum = a - b - cin;

always @（posedge clock or negedge reset）
   if （!reset）
      q <= 0;
   else
      q <= d;
```

而在 verilog2001 中，可以通过逗号来区分敏感列表中的信号。

```
#Verilog 2001
always @（a, b , cin）
   sum = a - b - cin;

always @（posedge clock, negedge reset）
   if （!reset）
      q <= 0;
   else
      q <= d;
```

此外，verilog2001 还支持敏感列表的两种表示方法：@（*）和@*。这两种表示方法意味着将该 always 语句下所有的信号默认为敏感列表成员。

Verilog 2001

```
// One style
always @（*）
    sum = a - b - cin;

// Another style
always @*
    sum = a - b - cin;
```

3. 有符号数据类型

在 verilog95 中，只有一种有符号数据类型，integer。Wire 和 reg 数据类型都被认为是无符号数据类型。而在 verilog2001 中，wire 和 reg 数据类型可以用关键字 signed 定义为有符号数据类型。

```
＃verilog95
integer a;
reg[63:0] b;
b = 6 ;   a = -2;
b / a = 0
```

因为 b 是无符号数，a 也被认为是无符号数，负数被看成无符号数时是个很大的数，所以 b / a = 0。

```
＃verilog2001
integer a;
reg signed [63:0] b;
b = 6 ;   a = -2;
b / a = -3
```

而在 verilog2001 中，b 可以被定义为有符号数，得到正确的结果－3。

4. 参数传递

在 verilog95 中，顶层对底层定义的 parameter 参数进行修改的方法是通过底层模块例化时在模块例化名前面加上需要重定义的 parameter 参数值，并以“#”打头。

```
＃verilog95
module top（ /* port list of top-level signals */ ）;
    dff #（10, 5, 1, 16 ） inst_dff（q, d, clk）;
endmodule
```

这种参数传递的缺点是：底层模块 inst_dff 中的所有参数都要按循序传递，否则就会参数传递出错。如果只修改 inst_dff 中的其中一个参数时，其他的参数（至少它前面的几个参数）也需要一起传递。

在 verilog2001 中，采用显示的参数传递方式，从而简化了参数传递个数，尤其是在参数比较多的情况下。如只修改 inst_dff 模块的第 2 个参数。

```
module top（ /* port list of top-level signals */ ）;
    dff #（.P1（5） ） inst_dff（q, d, clk）;
endmodule
```

5. Generate 语法

在 verilog95 中，没有语句支持对同一块逻辑的多次复制，只能进行多次例化。而在 verilog2001 中新增加了一种 generate 结构，能够在一个 for 循环中多次例化实例。Generate 有两种应用场景。一种是 generate loop，另一种是 conditonal generate。下面对这两种应用分别作介绍。

(1) Generate loop

generate loop 顾名思义，能够通过 for 循环对模块进行多次例化。Generate loop 需要定义一个 genvar 变量。genvar 变量可以被定义为 0 或者正整数。genvar 即可以在 genrate 块中定义，也可以在 generate 块外定义。当在 generate 块外定义时，genvar 变量可以被其他 generate 块使用。下面通过一个实例来看一下 generate 的用法。

```
#verilog2001
module addr_8（sum, c0, a, b, cin）;
input [7:0] a,b;
input       cin;
output [7:0] sum;
output      c0;
assign {c0, sum} = a+b+cin;
endmodule

module   addr_32  （a, b, sum, cin_in, c0_out）;
input[31:0] a,b;
input cin_in;
output [31:0] sum;
output      c0_out;
wire [4:0] c0;

assign c0[0] = cin_in;

generate
```

```
    genvar i;
    for （i=0; i<=3; i=i+1）
    begin :inst
        addr_8 add （sum [8*i+7 : 8*i], c0[i+1],  a[8*i+7 : 8*i], b[8*i+7 : 8*i],
c0[i]）;
    end
endgenerate

assign c0_out = c0[4];
endmodule
```

综合后的结果如下图，分别产生了 4 个实例：inst[0].add，inst[1].add，inst[2].add，inst[3].add。

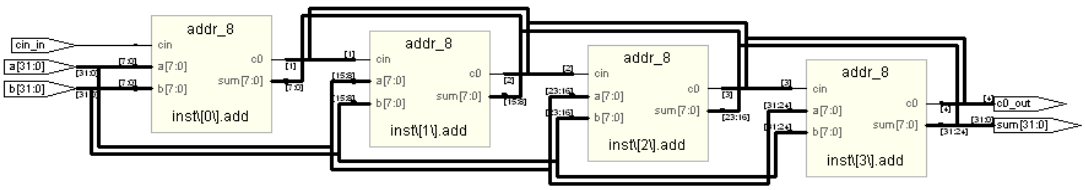


图2-29 generate loop 综合结果

需要注意的是：使用 generate loop，一个必须的条件就是要给 for 循环的 begin－end 块定义一个名字，如上面的例子中就给 begin－end 块命名为 inst，这样就可以为每个实例分配一个特定的名字——inst[*].add，方括号中的数为 genvar 的变量值。否则，综合会报错。

(2) Conditional Generate

Conditional generate 有两种格式

- If-else 语句
- case 语句

Conditional generate 通过参数的定义来决定实例的结构。

```
//if-else
module adder_8（sum, c0, a, b, cin）;
input [7:0] a,b;
input       cin;
output [7:0] sum;
output      c0;
assign {c0, sum} = a+b+cin;
```

```
endmodule

module adder_16（sum, c0, a, b, cin）;
input [15:0] a,b;
input       cin;
output [15:0] sum;
output     c0;
assign {c0, sum} = a+b+cin;
endmodule

module   add  （a, b, sum, cin_in, c0_out）;
input[15:0] a,b;
input cin_in;
output [15:0] sum;
output     c0_out;
parameter WIDTH = 1;
generate
    if （WIDTH == 1 ）
        adder_8 x1 （sum, c0_out, a, b, cin_in）;
    else
        adder_16 x1 （sum, c0_out, a, b, cin_in）;
endgenerate
endmodule
```

上例中如果 WIDTH 参数为 1，则 x1 实例为 8bit 位宽全加器，否则，x1 实例为 16bit 位宽全加器。

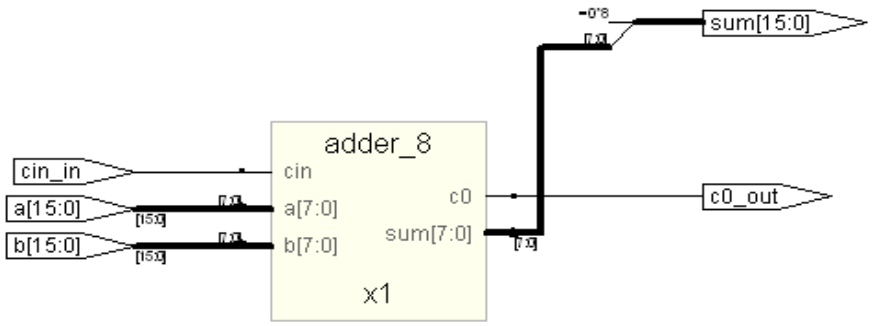


图2-30 Conditional generate 综合结果（parameter＝ 1）

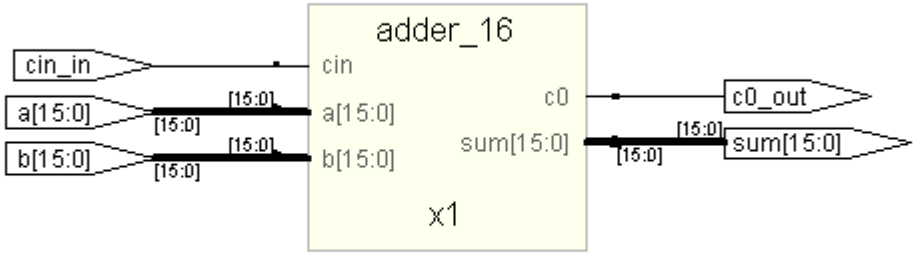


图2-31 Conditional generate 综合结果（parameter != 1）

```
//case
Parameter WIDTH = 1
generate
    case （WIDTH）
      1: adder1 x1 （c0, sum, a, b, ci）;
      2: adder2 x1 （c0, sum, a, b, ci）;
      default: adder # width （c0, sum, a, b, ci）;
  endcase
endgenerate
```

case 语句的含义与 if－else 语句类似。

需要注意的地方：

- if 语句和 case 语句中的变量必须是个静态变量值。通常情况下，是使用参数 parameter 来定义。
- 在 generate 块中合法的语句包括
  模块和原语实例
  Initial 和 always 块
  连续赋值
  线类型和变量定义
  参数再定义
  Task 和 function 定义 （不能在 generate loop 中使用）

6. Constant 函数

在 verilog95 中，向量的位宽一般都是通过参数直接定义。

```
#verilog95
Module ram（。。。）；
Parameter RAM_SIZE = 1024;
Parameter ADDRESS = 12;
Input [ADDRESS – 1 :0] address_bus;
```

在 verilog2001 中，可以通过一个 constant 函数来计算向量的位宽。

```
#verilog2001
Module ram（…）；
Parameter RAM_SIZE = 1024;
Input [clogb2（RAM_SIZE） – 1 :0] address_bus;

function integer clogb2;
input [31:0] value;
    for （clogb2=0; value>0; clogb2=clogb2+1）
    value = value>>1;
endfunction
```

使用 constant 函数来计算向量位宽，可以避免人为计算出错，同时，在参数变量修改时，如上例中 ram 模块被例化，RAM_SIZE 参数被修改为 2048，那么 clogb2（RAM_SIZE）的计算值也可以随之更改为 13。

7. Localparam 语法

在 verilog2001 中，localparam 和 parameter 的用法大体一致，唯一的一点区别是，localparam 不能被 defparam 语句和参数传递进行直接修改。不过，localparam 可以定义为一个包含 parameter 的表达式，通过修改 parameter 来修改 localparam。

```
#verilog2001
```

```
Module dft（…）；
parameter ONE = 1
localparam TWO=2*ONE
localparam [3:0] THREE=TWO+1;
```

注意，例化模块时 localparam 是不能进行参数传递的。

```
dft #（.ONE（5） ） inst_dft（。。。）;
```

8. 比特选择

在 verilog2001 中，可以使用表达式（+: 和-:）来对向量的比特位进行可索引的部分选择。

具体的语法格式如下：

my_vector [base_expression +: width_expression]

my_vector [base_expression -: width_expression]

my_vector 表示被选择的向量；

base_expression 表示比特选择的起点，它可以是任何合法的表达式或变量，可以在运行的时候变化，因而它是可索引的；

width_expression 表示选择的比特位宽，它只能是固定的常数；

例如

```
module addr_8（sum,  a, b）;
input [7:0] a;
input [2:0] b;
output [2:0] sum;
parameter index = 1;
assign sum = a[b+:3]; //根据输入的起点 b，选择 3 比特位
endmodule
```

综合的结果如下，逻辑与门用于判断输入的 b 值，b 值范围是 0～7，所以有 8 个与门。根据判断的 b 值，将已经选择好的比特位选择输出。如 b 值为 0，则将 a[2:0]选择输出。

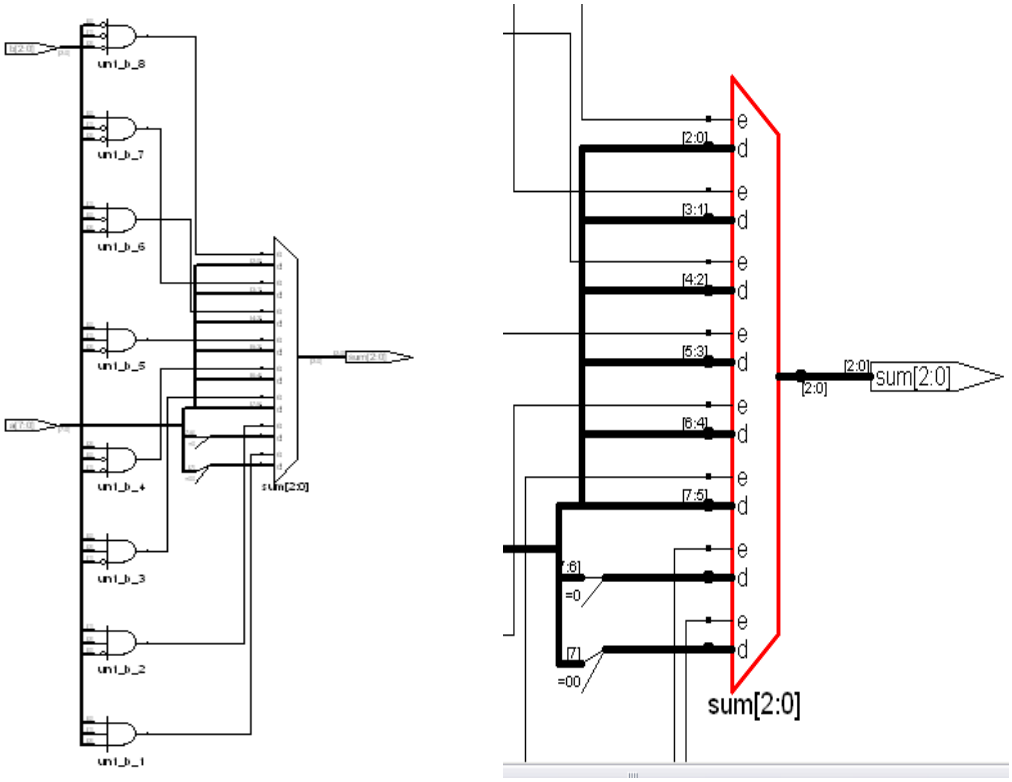


图2-32 比特选择语法综合结果

9. 条件编译指令

在 Verilog95 中，条件编译指令只有三个：`ifdef、`else、`endif。而在 Verilog2001 中，条件编译指令，增加了`ifndef、`elsif、`undef 指令。`ifndef 指令与 ifdef 指令含义正好相反，即如果没有定义宏，则编译 ifndef 下的指令。`undef 指令表示取消宏的定义。`elsif 指令用在多条件编译的情况下。如在 verilog95 中，多条件编译的写法是

```
#verilog95
`ifdef A1_OUT
        assign a= 1;
endif
`ifdef A2_OUT
        assign a = 2;
endif
`ifdef A3_OUT
        assign a = 3;
`endif
```

而在 verilog2001 中，可以使用`elsif 简化写法。

```
`ifdef A1_OUT
        assign a= 1;
```

```
`elsif A2_OUT
        assign a = 2;
`else
        assign a = 3;
`endif
```

2.1.3 后端对设计的建议

概述：设计人员在进行前端代码设计的时候，往往会忽视前端对后端的影响，有的时候前端工作稍微修改一点，往往能给后端带来很大的方便，因此后端对前端设计有哪些要求需要让代码设计人员清楚，本节主要是收集此需求，供前端设计人员学习。

由于 STA 主要是基于同步电路来做检查的，所以最重要的一点就是同步设计，因为STA 对于异步电路是无法做出正确的分析的(除非用户自己定义一些检查)，但这样既复杂且无法完全保证检查的正确性。

因此，代码设计人员在设计阶段，应当充分考虑以下几个方面。

1. 时钟

(1) 尽量避免采用多个时钟

在 IC 设计时，由于时钟建立应尽量避免采用多时钟网络，或者采用适当的措施减少时钟的个数，把使用频率低的时钟尽量简化消除。最理想的情况当然是整个电路只使用一个时钟，当然这一般不太可能。此时对时钟的处理尽量集中，集中处理完后的时钟在再分配到各个以时钟域（Clock Domain）划分的各模块使用。

(2) 应按照时钟域划分子模块，不同时钟域之间的接口，应做同步处理(参考)

当芯片中存在多时钟网络，应将同一时钟域的功能集中处理，即按时钟域划分子模块。不同时钟域之间的接口应尽量集中在 TOP 或大模块层，有 FIFO 隔离或专门的同步处理模块。不要将异步接口散的到处都是，不利于代码检视。

(3) 芯片异步接口（如 CPU 接口），应做同步处理

对芯片异步接口做同步处理是同步设计的基本规范，对从 input pin 进来信号要先用本芯片的采样时钟同步，这种在芯片层面上的异步接口建议也是先集中处理，然后将同步后的信号引入内部子模块。同时我们输出到 pin 上的信号也要经过寄存。

(4) 整个设计应尽量只用一个时钟沿

STA 主要是基于同步电路来做检查的，我们认为一个好的设计除了有同步的理念，还要尽可能只用一个采样沿。 在同一时钟域中一会儿用上升沿采样，一会

儿用下降沿采样，会人为的收紧 setup、hold 时间，增加时序分析的难度，对电路的可靠性、可实现性带来风险。

(5) 不建议使用行波时钟

行波时钟，即一个触发器的输出用作另一个触发器的时钟输入。行波时钟使得与电路有关的定时计算变得很复杂。行波时钟在行波链上各触发器的时钟之间产生较大的时间偏移，并且会超出最坏情况下的建立时间、保持时间和电路中时钟到输出的延时，使系统的实际速度下降。转换的方法是采用同步计数器，

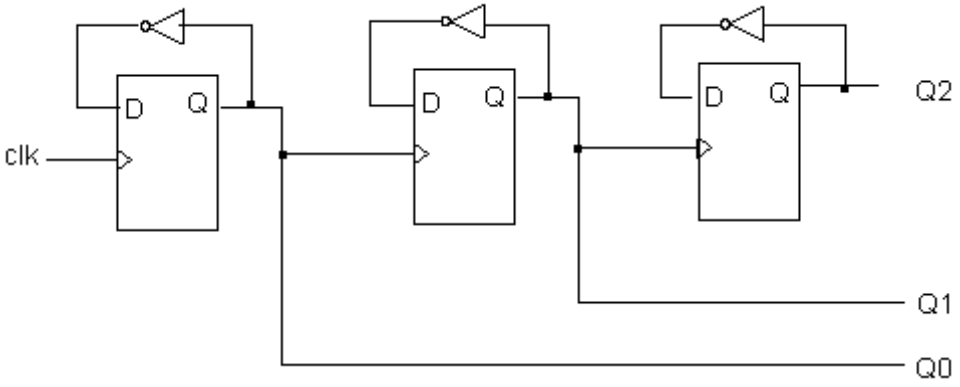


图2-33 行波时钟

(6) 禁止时钟在不同芯片中级连，尽量降低时钟到各个器件时钟偏差值

同一个时钟在不同芯片中使用时不允许级连，而且应该保证该时钟到达不同芯片输入引脚的时钟偏差足够小。因为经过多个芯片的延时后，难以估计该时钟在链上各个触发器时钟之间产生的大量时间偏移，如果这种时间偏移引起建立时间、保持时间难以满足的话，那么设计的逻辑就极有可能要失败了。

(7) 尽量避免使用组合逻辑时钟或门控时钟

组合逻辑很容易产生毛刺，用组合逻辑的输出作为时钟很容易使系统产生误动作。

应当尽量避免门控时钟的出现，此时尽可能的将需用作门控时钟的控制信号综合到数据端，比如时钟使能（ClockEnable ）端等。

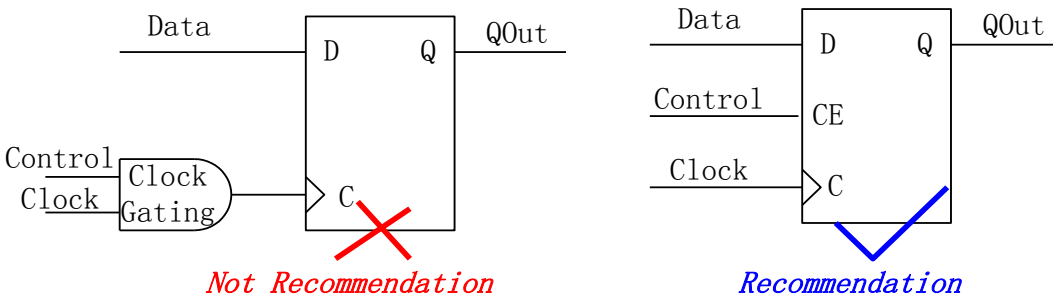


图2-34 门控时钟改为非门控时钟的示意图

在非使用门控时钟不可的时候，一定要遵循一个原则：**门控时钟的输出只能跟着时钟信号进行跳变，而不能跟着控制信号进行跳变**。也就是说对于用与门（AND Gate）或与非门（NAND Gate）实现的门控时钟，控制信号只能在时钟的低电平处跳变；对于用或门（OR Gate）或或非门（NOR Gate）实现的门控时钟，控制信号只能在时钟的高电平处跳变。如下图所示。

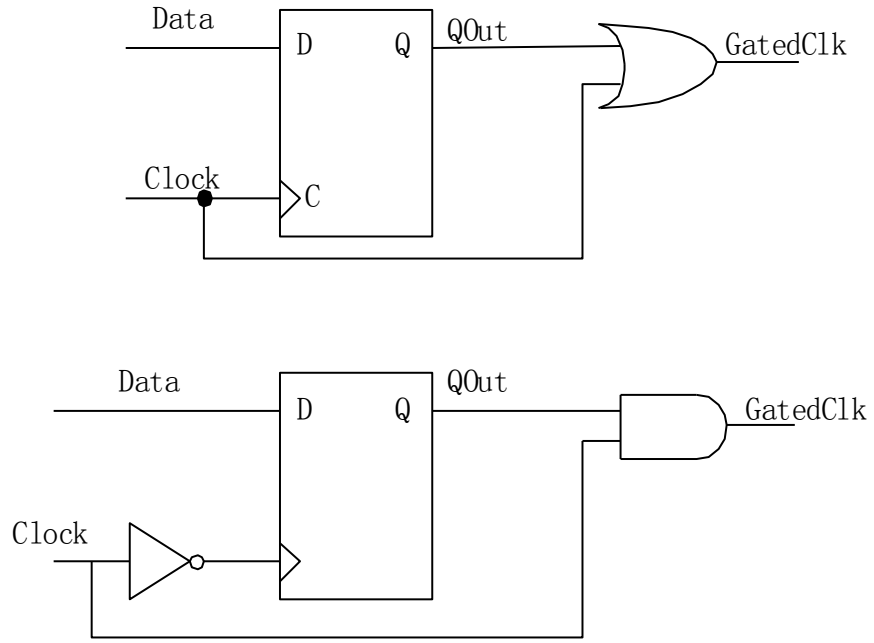


图2-35 门控时钟输出信号与控制门之间的关系示意图

2. 复位

(1) 整个设计所使用的复位信号数目应尽量少，最好只用一个全局异步复位信号

对于硬复位（异步复位）我们希望是独立于芯片的时钟，应直接使用 vendor 的 delay 器件来去毛刺，而不是通过芯片的主时钟打拍来去毛刺。要避免出现当芯片时钟丢失或异常时连硬复位都失效的情况。

(2) 触发器的置/复位端尽量避免出现毛刺，及自我复位电路等

(3) 设计中的同步复位信号在到达不同时钟域的寄存器之前，应使用各时钟域的时钟做同步处理

3. 裕量设计

应从方案阶段考虑时序问题，给 STA 留有裕量，而不要等 STA 结果出来后再改代码。因为有时的这种改动是架构性的变化，对项目进度影响很大。

(1) 通过拆分逻辑或加一级流水来缓解时序紧张

(2) 通过复制逻辑或寄存器来解决扇出过大问题

(3) 关于 Multicycle 采样点的选取

用低频时钟采样高频时钟输出的数据，通常我们的做法是将数据展宽。以下图为例，高频时钟在 A 端将数据打出，低频时钟在 E 端采样，这样给 setup/hold 留有很多余量。若低频时钟在 B 端采样，则 setup 紧张；若低频时钟在 H 端采样，则 hold 紧张。因此设计者对 Multicycle 采样点的选取要仔细考虑。

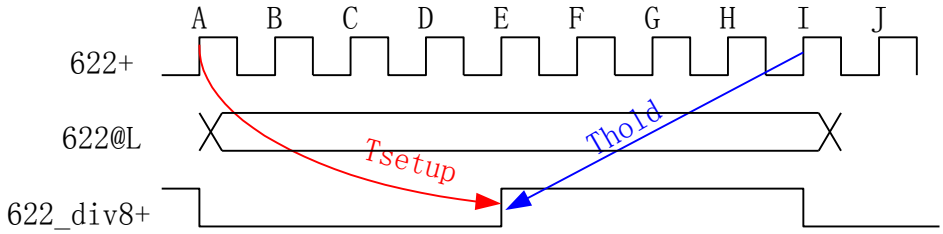


图2-36 Multicycle 采样点的选取

4. 几种不推荐的电路

(1) 异步时钟或组合逻辑时钟

应尽量将它们做一定的处理（比如沿检测）后去用作时钟使能等，而不要用作时钟。

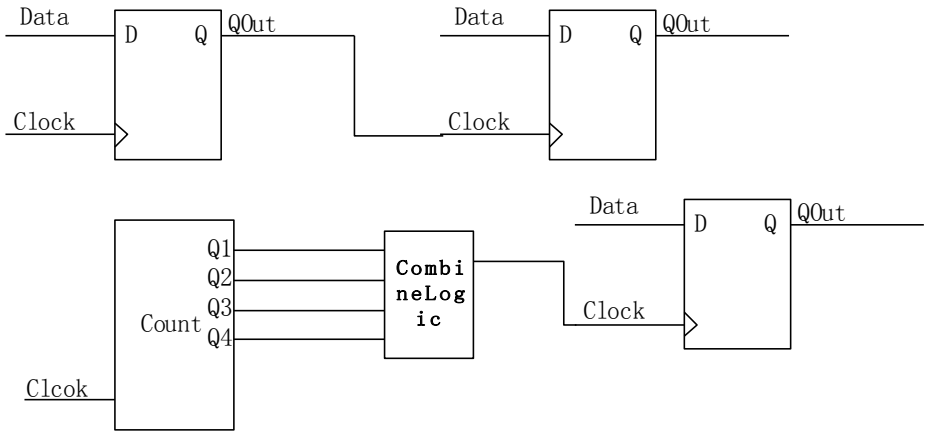


图2-37 异步电路与组合逻辑时钟

(2) 异步的复位、置位电路 h

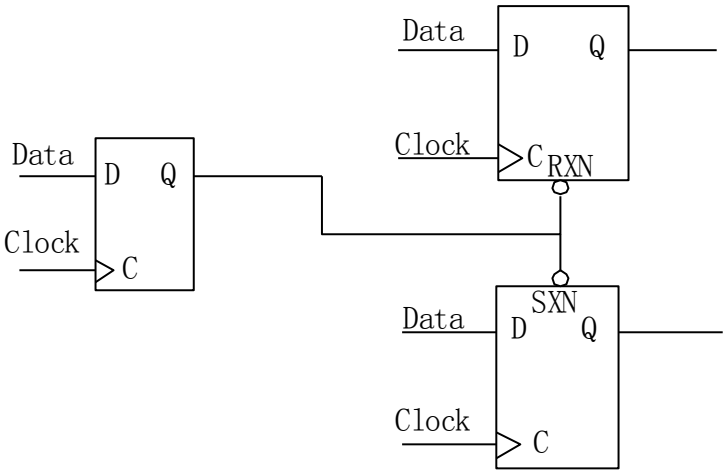


图2-38 异步复位置位电路

上述电路中将前一触发器的输出在后面用作异步的复位或置位信号。最好能象前面的第一个例子一样，作一定的处理后去作同步复位。最理想的情况应该是出现在触发器的异步复位或异步置位端的只有芯片外的硬复位。

(3) 异步的时钟选择电路

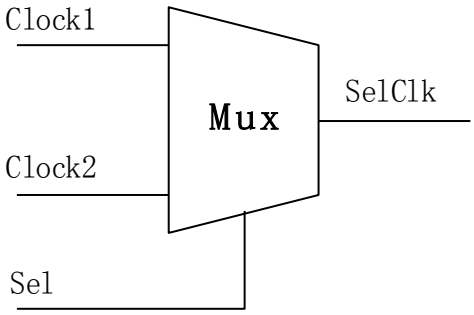


图2-39 不同时钟的 MUX 选择电路

如果选择信号 Sel 与时钟为异步的话，则选择出的时钟就可能出现展宽或变窄的情况，甚至有尖脉冲（Glitch）的出现。所以我们建议使用如下的同步时钟选择电路，特别是对时钟的要求比较高的电路中。

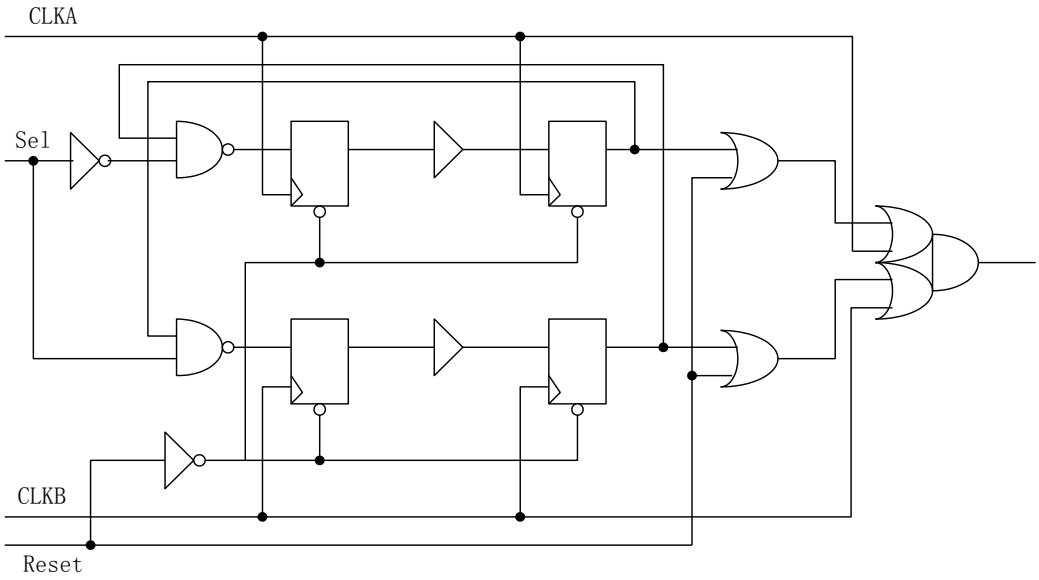


图2-40 同步时钟选择电路

在上面的电路中，特别要注意的后面的四输入的或与门的 ClockGating，若选择信号是用时钟的正沿打的，则可用“或与门”，若是用时钟的反沿打的，则可用“与或门”。当然，这两种方式下的逻辑可能不一致。

5. 异步接口的时序考虑

这里说的异步接口是指两个时钟域之间数据通讯的实现方式。由于异步接口的两个时钟域相位是任意的。而不管在仿真还是时序分析，我们必须给出一个确定的时钟相位关系。因此在进行异步接口的设计时必须要小心，因为进行功能仿真时，只是仿真了任意时钟相位关系中的一种。在时序检查时，这样的电路被设置为 False Path。由于仿真和时序检查都无法保证电路的正确性，所以设计时要特别小心，从多角度仔细分析。

常见的异步接口处理方式如下：

(1) 不进行任何的处理

这种方式常见的是从一个时钟域到另一个时钟域的控制信号。例如模式选择，使能控制等。通常这些信号是缓变的（即很长时间才可能变化一次）。对于这样的异步信号，通常在时序分析时可以设置为 False Path，由设计保证功能的正确。在这样的接口设计时要注意以下几点：

- 在信号变化时，可能出现亚稳态，因此在信号变化时，功能可能出现不正常。
- 如果是一组 Bus，由于各个 Bits 的延时不可能完全一致，用于时钟相位是任意的，因此会出现采样到其他的数值。
- 设计的强壮性，不能由于亚稳态，不期望的数据，导致死机，或者无法恢复正常工作等。

(2) 使用异步 FIFO 隔离

通常使用格雷码的异步 FIFO 隔离两个异步时钟域，进行数据通讯。

有一个问题需要注意，异步 Gray FIFO 是将地址转化为 Gray Code，比较地址产生空满信号。如果设置了 False path，这部分跨时钟域的 Path 将不检查。那么如果地址延时很大，那么可能出现产生的空满信号将不能反映真实的 FIFO 空满。因此建议如下：

- FIFO 的空满要有一定的裕量
- 检查地址到空满信号产生允许的最大 Delay，是否满足要求。

(3) 使用控制信号指示有效数据区间

由于使用 FIFO 可能消耗的资源相对多一些，因此在异步设计中，还有一种方法，降低异步接口数据的变化速率，使用快时钟采样慢数据。常见的方法就是送出数据信号，以及指示数据的有效信号。接收端使用鉴沿电路，产生采样使能脉冲，采样数据。通常使能脉冲选取在数据稳定的区间。在时序分析时将两个时钟域设置为 False Path。

使用这样的电路，如果有效指示信号，和数据信号之间的 Delay 差别很大，虽然理论上可以稳定采样，可能到了实际中，可能采样到不正确的数据。

因此建议设计人员在使用这样的电路时，注意一下几点：

- 有效指示信号在鉴沿时考虑到亚稳态问题（见附录）
- 采样使能脉冲和被采样数据的位置关系的确定，要留出一定的裕量。
- 检查有效信号和数据之间的 Skew 是否满足要求。

6. 附录

(1) Multicycle_path

什么是 Multicycle path?

Multi-Cycles path 意思是多周期的数据路径，即一个时钟周期不能完成数据传输的路径。一般是由于大的逻辑电路引起的。由于大的逻辑电路的存在导致数据的最大延迟可能大于一个时钟周期。这时就需要指明该路径为 multicycle path，因为一般的时序检查工具缺省是按单周期的约束对数据路径进行检查。从有效的 latch edge 到有效的 capture edge。

对于我们的设计，如果组合逻辑网络的最大延时确实大于一个周期，我们就需要修改我们的设计，减小组合逻辑的延时，或者修改电路如下图，例如：

源触发器每隔两拍打出一个数据，下游触发器也是每隔两拍接收源触发器打出的数据。

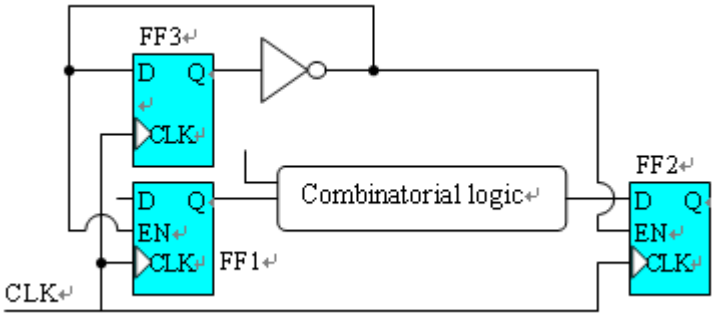


图2-41 Multi-Cycles path 电路结构示意

此时的波型，以及正确的时序检查关系应该如下图所示。

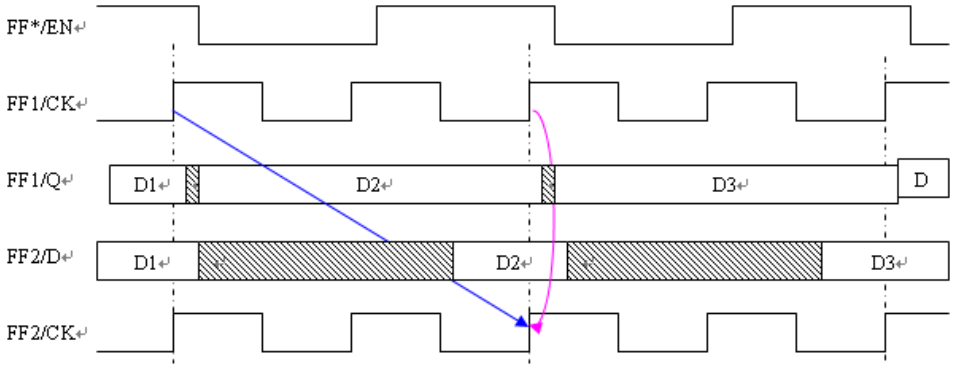


图2-42 Multi-Cycles path 电路时序检查示意

在 DC、PT 中，设置如下的约束，告诉工具这时一个 2 Multi-Cycles Path。

set_multicycle_path 2 –setup –from FF1/Ck –to FF2/D

set_multicycle_path 1 –hold –from FF1/Ck –to FF2/D

大家看看这次的 2 Multi-Cycles Path 的命令设置和前面有什么不同？也看看时序检查有什么不同？

Synopsys 的 DC、PT 工具在设置 Multi-Cycle 时，使用命令 set_multicycle_path –setup（或者你使用：set_multicycle_path，不指明选项 -setup、-hold），调整了 Setup check 的时钟沿关系，Hold 时钟沿关系也会随之移动。你必需在接着使用命令： set_multicycle_path –hold 将 Hold 检查关系在进行调整。

⚠ 注意：

上面的电路中，从触发器 FF3 的输出不能设计为 Multi-Cycles。

另外要注意，并不是触发器每隔两拍（或者以上）变化输出数据（不包括 Launch Edge 打出的变化数据），或者接收触发器每隔两拍接收数据，（或者二者的组合），就可以设计为 Multi_Cycles。而是应该从 Launch clock Edge 到有效的

Capture Clock Edge 这一段时间，（设计 Multi-Cycles 后），Launch 的数据不发生变化，才能设置为 Multi-Cycles。例如上面电路的变形：

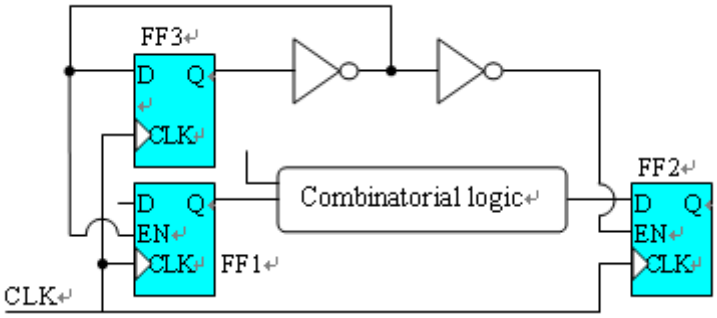


图2-43 非 Multi-Cycle 的电路结构示意

触发器 FF1 每隔两拍打出数据，FF2 每隔两拍接收 FF1 打出的数据，但是从 FF1 到 FF2 不能设置为 Multi-Cycles。我们画出设计 Multi_Cycle 后，时序检查和实际数据的变化关系如下：

在上面的电路中，源触发器在 0、4、等位置打出数据，那么接收时钟就在 2、6 等位置接收数据。这样的电路是不能设置为 Multi-Cycles 的。

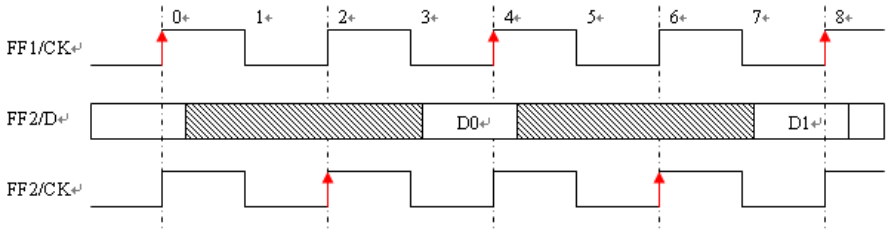


图2-44 非 Multi-Cycle 的电路时序检查示意

在上面说的 Capture Clock Edge 时，我们说是有效的 Capture Clock edge。有效的含义是只我关心该拍输出的数据，而其它沿打出的数据是不关心的。例如上面的电路在变化一下：

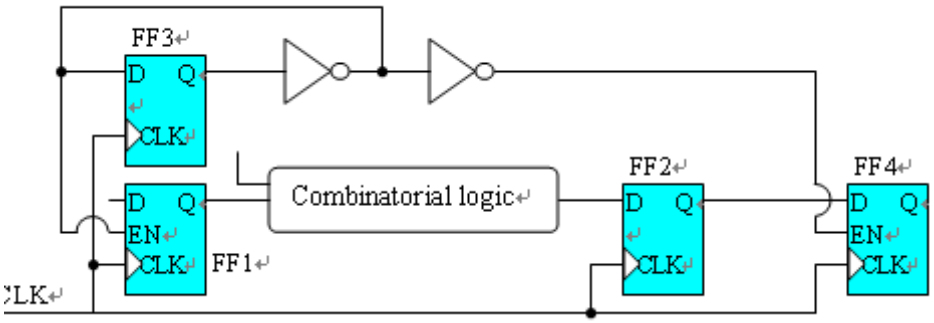


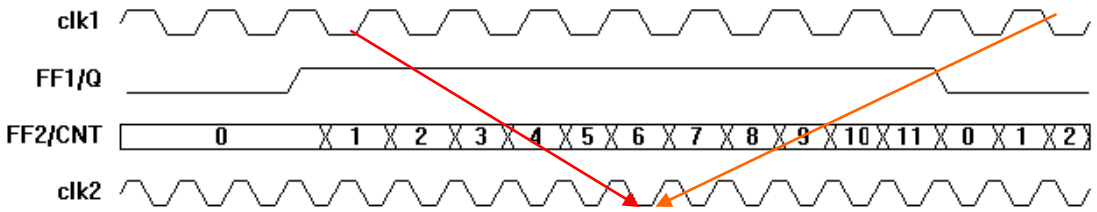
图2-45 Multi-Cycles path 电路结构示意 2

如果触发器 FF2 的输出只有到 FF4，尽管 FF2 每拍都采样数据，但是对于 FF4 来讲，只关心 FF1 输出变化数据那拍，FF2 输出的数据。因此，对于这样的电

路，也是满足 Multi_Cycles 的。我们可以设置从 FF1 到 FF2 两拍的 Multi-Cycles。

从本节可以看到，对于 Multi-Cycle 电路，电路设计人员要特别注意，Multicycles 电路是 RTL 代码人员设计出来的，并不是当最大延迟超过一个周期就可以设置为 MultiCycles。另外从 DFT 角度来看，不建议做 MultiCycles，否则会带来测试困难，降低测试覆盖率问题。对于较大的数学运算单元，尽量例化 DesignWare 中的多阶 Pipeline DesignWare 单元。对于我们自己设计的比较复杂的单元，尽量在中间插入触发器，进行 Pipeline 设计。

例子 1： 在异步电路设计的处理当中，由于时钟频率、相位的不同，目的时钟采样信号时，希望信号保持稳定，如下图所示，clk2 在 cnt 为 4 时采样 FF1 输出的数据，这样可以保证采样的数据是正确的。虽然路径延迟可能小于一个周期，但由于缺省的工具的检查方式都是按单周期。Setup 检查从有效的 latch edge 发出的数据能被有效的 capture edge 接受。Hold 检查从下一个 latch edge 发出的数据不能被该 capture edge 所接受。对于下图的电路可以设置 multicycle path，setup 检查是 5 拍，hold 检查是 7 拍，允许的数据路径延迟{-7＋t_{hold}，5－t_{setup}}，如果确认路径延迟不超过 5 拍，也可以设置 false path。



(2) 亚稳态

对于异步电路在进行同步化处理时，我们要求使用触发器连续打几拍。因为每打一拍，其后在出现亚稳态的几率将大大的降低，但不管打多少拍，理论上是不可能下降为 0。通常，根据经验，推荐打三拍（要求不是特别高时，打两拍也可以）。

现做图来举例触发器在亚稳态时的输出情况，再做图分析异步电路在同步化时为什么打两拍或三拍即可以较好地避免亚稳态的现象；

I. Setup time不满足时的亚稳态现象

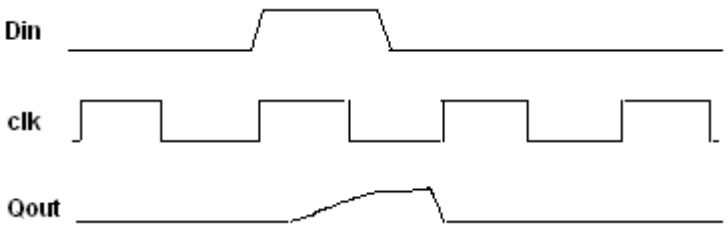


图2-46 setup time 不满足时的亚稳态现象

II. Hold time不满足时的亚稳态现象：

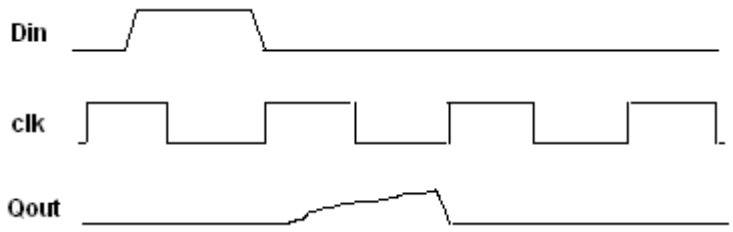


图2-47 hold time 不满足时的亚稳态现象

III. 异步电路在同步化时，通过打两拍避免了亚稳态的现象：clk2时钟上沿采到dat1时，dat1正在变化，不稳定，满足不了保持时间，dat2出现亚稳态。

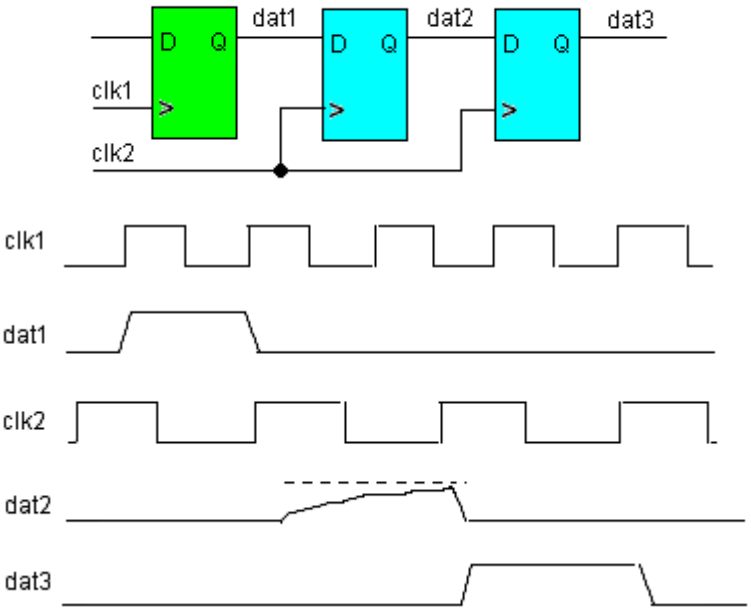


图2-48 异步电路如何避免亚稳态

2.2 设计利器

概述：海思的设计工程师在日常工作中，自行开发了不少小工具，运用这些工具有两大好处：大幅度提高工作效率；提高准确性，避免人为疏忽。当然，使用工具之前必须准确掌握使用方法，这样才能真正发挥工具的威力。在这里我们搜集了很多这类小工具的使用方法，请读者选用。

2.2.1 MPI 模块级接口生成工具

1. 获取途径

<在 WS 环境下，访问 CVSSRV3:/HISI_WS 的 HiSiEI 模块>

2. MPI 模块结构

MPI 模块有两个功能：产生芯片全部的寄存器，即寄存器在 MPI 模块；产生访问 RAM 的总线功能模块，即 RAM 在功能模块里。

MPI 模块的结构如下图所示。

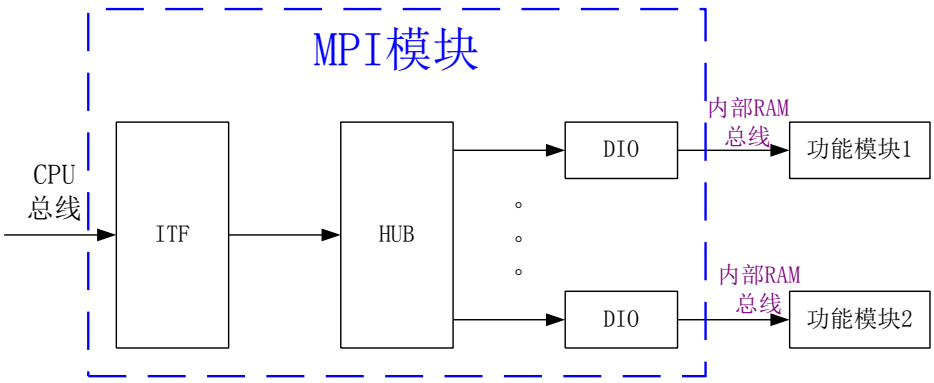


图2-49 MPI 模块结构

MPI 模块由 3 部分组成：

- ITF 模块：该模块实现 CPU 时钟域到逻辑系统时钟域的转换，文件名：mpi_itf.v
- HUB 模块：根据地址选择某一个 DIO 模块，文件名：mpi_hub.v
- DIO 模块：根据地址读写该 DIO 模块内的某个具体的寄存器，或产生 RAM 的总线信号输出到功能模块。

自动生成代码是指生成全部 DIO 模块的代码，HUB 模块需要做少量的译码条件修改（根据 DIO 分配的地址），ITF 模块一般不用修改。一般按逻辑功能来划分 DIO 模块，可以让一个功能模块对应一个 DIO 模块。若芯片的模块较多，也可以让 2～3 个功能模块对应一个 DIO 模块。

3. 寄存器和 RAM 的定义

寄存器和 RAM 都是通过脚本定义，定义寄存器脚本如下图所示。

```
// MACO's DIO BEGIN
// The first DIO
//--- MACO_GLB_DIO BEGIN
//### Registers

//1.version Register
0x8000  31:24       year                    roni    8'd05
        23:16       month                   roni    8'd12
        15:8        day                     roni    8'd11
        7:0         version                 roni    8'd0

//2.window Register
0x8004  31:15       addr_window             rw      0
        14:0        reserved                ro      0
```

说明：

(1) 以//开头，且不含－－－或＃＃＃的语句为注释。

(2) ”//－－－MAC0_GLB_DIO BEGIN”表示一个 DIO 模块的开始，注意这一行里除了 MAC0_GLB 需要改为实际的名称外，其他的内容固定不变。

(3) ### Registers 表示开始定义寄存器，寄存器可定义成 32bit 或者 16bit，以下面的脚本为例：

```
//5.MAC0_DBA advanced config Register
0xb010  31:28       reserved                    ro      0
        27:16       start_allocid               rw      0
        15          position                    rw      0
        14:12       state_set                   rw      0
        11:0        onu_id                      rw      0
```

- 第 1 行为寄存器的注释。
- 第 2 行从左到右分别为寄存器的地址、起止 bit、名称、类型（如只读、可读可写等）和复位值。其中[31：28]为保留未用的区域，[27：16]、[15]、[14：12]、[11：0]为有效的定义。使用时根据实际的域进行定义。注意：地址 0xb010 只写一次；若复位值为 5，可以写为 0x5 或 0d5。

(4) //### Rams 表示开始开始定义 RAM，以下面的脚本为例：

```
//### RAMs-5
0x0012_0000         RAM_for_HEC_ERR_CNT0        rw      32bits x 64         MAC0_GEM
0x0013_0000         RAM_for_DBA_ALLOC_INDEX0    rw      9bits x 4096        MAC0_DBA
0x0014_0000         RAM_for_DBA_INDEX_CFG0      rw      32bits x 256        MAC0_DBA
0x0015_0000         RAM_for_DBA_REPORT0         ro      32bits x 256        MAC0_DBA
0x0030_0000         RAM_for_DBA_INNER_TABLE0    rw      69bits x 256        MAC0_DBA
//--- MAC0_GEM_DBA_DIO END
```

上面定义了 5 个 RAM 的接口，每一行从左到右分别为 RAM 的起始地址、RAM 的名称、RAM 的类型、RAM 的大小（宽度×深度）及 RAM 所属的功能模块。

注意：程序通过 RAM 的大小可以算出 RAM 的结束地址，所以这里没有定义 RAM 的结束地址；RAM 的名称必须以 RAM_for 开头。

(5) 上图最后一行“//－－－MAC0_GEM_DBA_DIO END”表示一个模块定义的结束。注意：除了 MAC0_GEM_DBA 可以改为实际的模块名称外，其他内容不可更改。

(6) 寄存器的说明

常用的寄存器类型如下：

- RO：只读；
- RW：可读可写；
- RWS：软件置 1，逻辑清 0；
- RWC：软件写清零，逻辑置 1，和 RWS 型的寄存器有相似之处；
- RC：读清零寄存器，用于中断类型寄存器，逻辑置 1，软件读清零；
- RWCI：加 1 统计计数器；

- ROID：加 1、减 1 统计计数器；
- RONI：只读且无接口信号，一般用于逻辑日期和版本寄存器。

(7) RAM 的说明

支持 RAM 的位宽为 1～128bit。当 RAM 的位宽为 128bit 时，CPU 需要 4 次才能完成一个表项的访问。

4. 工具使用方法

(1) 获得自动化脚本 vreg.pl 文件，修改 ITF 和 HUB 模块，划分 DIO 模块，分配 DIO 模块的地址。

(2) 编写寄存器和 RAM 定义脚步文件“DIO”，此文件无需后缀名。

(3) 打开 vreg.pl 文件，更改生成文件的目录，见下图，需要把“../mpi/”改为实际生成 dio 文件的目录（＃号为 perl 代码的注释）。

```
25 #define the output directory
26 $dio_outdir = "../mpi/";
27 $adr_outdir = "../mpi/";
28 #$reg_outdir = "../../vrf/rm/";
```

(4) 运行脚本，在脚本所在目录下键入：perl vreg.pl DIO

(5) 获取文件，生成的文件包括：

- ×××_dio.v（和实际的 DIO 模块名一致）
- dio_addr_seg.v（宏定义，定义每个 DIO 模块的寄存器起止地址，每个 RAM 的起止地址）。

2.2.2 代码顶层产生工具

1. 获取途径

<在 WS 环境下，访问 CVSSRV3:/HISI_WS 的 HiSiEI 模块>

2. 工具使用方法

- 该文件为 perl 语言编写的 RTL 顶层 topbench 文件自动联线工具；
- 该脚本对 RTL 代码有如下要求：
- 需要联线的子模块名称应该以“moudulename_top.v”的格式命名；
- 文件内部所有 input，output 信号必须顶格写；
- inout 类型的输入输出由于其特殊属性，需要手工进行连接；
- 运行方式：
- 首先将需要连接在顶层的所有子模块的相对路径放在文件“rtl_list.pl”中；
- 运行“perl topbench_gen.pl”
- 脚本会自动生成两个文件：“topbench.v 和 newtopbench.v”；前者为中间过程文件，后者为脚本生成的最终 topbench 文件；

- 如果需要对某个模块进行多次例化，则只需将该子模块文件按序号命名即可，例如：“module1_top,module2_top...”

🕮 **说明：**

脚本毕竟只是脚本，虽然能够节省大量工作时间，提高工作效率，但并不能完全取代人的所有工作。

2.2.3 代码行统计工具

为了能有效的统一部门的代码统计，现在部门要求采用一个固定的代码行统计工具。工具路径：/nas/data2/share/design/codestat/code_line_cal.tcl

工具使用说明：

- 使用 TCL 脚本编写的小工具，可以通过文件列表统计代码行数。
- 代码行统计中，认为是有效代码行的标准有 3 个：.v 文件中的代码行；处在关键字“module”之后的代码行；存在数字或字母的代码行。同时满足上述 3 个条件的代码行就会被统计。
- 统计的内容包括逻辑代码行和有文字的注释行。
- 文件列表可以嵌套，但一定要符合这种格式：“-f PATH/filelist_name.f”。最多只能嵌套一层。
- 统计结果存放在一个文件中（和工具脚本在同一个目录），记录所统计的文件总数、统计的总代码行数以及每个文件的代码行数。

小工具的使用步骤如下：

- 把文件 code_line_cal.tcl 拷贝到任何目录下，运行它。
- 运行小工具后，会得到一个输入文件列表的提示。输入文件列表后回车，如图示：

```
hw160:/project6/SD8901/w46022/SD8901/digital_data/rtl/sd8901_top> code_line_cal.tcl
Input filelist please:
../lopp/lopp_filelist_syn.tcl
```

- 小工具运行完毕后会生成一个名为“code_line_sum.txt”的报告文件，统计结果都存储在这个报告中（包括总行数，统计文件数，每文件行数）。

2.2.4 代码模版产生工具

1. 获取途径

/home/lzy/reuse/release_tools/

2. 工具使用说明

工作站上必须装有所使用的工具的脚本语言的解释器；

要想运行脚本工具，脚本文件的属性必须是可执行文件；

工具运行完成后，输出 Verilog 代码文件，文件中包括文件头、输入输出信号定义和一些标准的注释。

3. 工具使用方法

调用并执行工具脚本可以有两种方法。其一，是将该脚本拷到当前目录再运行；其二是将脚本放在某个文件夹中，比如 ~/bin 下面，再把 ~/bin 添加到.cshrc 文件中的 path 变量中，则在任何目录下都将可以直接运行该脚本；

输入脚本文件名，不带任何参数，然后回车，屏幕上将打出该命令的详细使用说明；

工具的输出文件都有一个文件头里面包含一些作者名、作者的 Email 等信息，这些信息必须存储在用户根目录下的文件名为.profile 的文件中。如果用户根目录下没有这个文件，或该文件中没有相应信息，则第一次执行该脚本时，会以交互的方式提示用户输入相应信息，并自动生成该文件；

输入下面命令运行脚本：get_hdl.tcl object_filename.v port_define_file，其中 object_filename.v 为输出文件的文件名称，port_define_file 为输入输出信号定义文件，port_define_file 文件一定要按照下面的格式编写：

port_name direction bus_width(bists)

运行完脚本后，就可以输出 Verilog 代码文件。

2.2.5 DOC_HTML 转换工具

1. Features

- 生成 html 文件，可以方便的进行文档阅读
- 可以有效地发现人为的书写错误
- 更小巧实用，链接清晰，提高效率

2. perl 代码主要模块功能描述

(1) parse-rules.pl

定义了寄存器的操作类型、perl 脚本查找文件的标志及查找结束标志

(2) gen-rules.pl

定义代码的版本联系人及 mail

(3) gen-reg-pl.pl

输入：cp.txt（文件名可设定，由最初的 doc 文档而来）

输出：<chipname>_reg_map_complete.pl 和<chipname>_reg_map.pl 两个文件

这两个文件用以生成我们的 `html` 和 `vh` 文件，chipname 来源于 txt 文件本身，在文件中会有几处定义 chip（如：iTM-FI 和 iTM-FE）。

<chipname>_reg_map_complete.pl 文件格式是固定的，使用 perl 的数据格式表示:

```
$::evalRegInfoAlias = '*::regInfo = \%::itmfeRegInfo;';
```

```
$::evalTemplateInfoAlias = '*::templateInfo = \%::itmfeTemplateInfo;';
$::evalBaseAddrAlias = '*::baseAddr = \%::itmfeBaseAddr;';
$::evalChipNameAlias = '$::chipName = "iTMFE"';     //生成的 chip name
$::evalSpecVersionAlias = '$::specVersion = "1.0"'; //生成的版本
%::itmfeBaseAddr = (
...                                  //包含各个模块的基地址
)
%::itmfeRegInfo = (
...                                  //包含各个模块内寄存器的字段等信息
)
%::itmfeTemplateInfo = (
...                                  //包含各个模块内寄存器的说明等信息
)
```

(4) gen_reg_html.pl

输入：<chipname>_reg_map_complete.pl

输出：.html 文件

html 文件以 index.htm 为首页，根据模块之间的联系为链接，逐步到具体的寄存器说明，同时还提供根据地址查阅寄存器内容的功能

(5) gen_reg_vh.pl

输入: <chipname>_reg_map_complete.pl

输出：.vh 文件

.vh 文件可以用来做脚本的宏定义，从而避免人为的每个寄存器的去指定地址 perl 脚本处理流程

2.2.6 GVIM 高级命令

VIM 是一个非常强大的编辑工具，但是也是一种需要一定练习才能熟练掌握的工具。不要因为觉得忙就不去练习，花费少量的时间来练习 VIM 的操作将会为你节约更多的时间。如果你忘了如何快速移动到行尾，那么应该做的是找出这个命令，再花一点时间去记住这个命令。而不是干脆就用鼠标将光标定位过去。

当然，你也不必学习所有的 VIM 命令。那样只会浪费你的时间。只需要学习其中的小部分命令就足以应付日常的工作了。提高效率的关键在于熟练使用，对于一些常见的操作，应该练习这些操作对应的快捷命令直到能够不假思索的使用它们。而如果你进行了一次某项特殊的操作，并且没指望将来还要进行类似的操作，那就不用试着去琢磨它了。

VIM 编辑器有多种工作模式，主要的有 Normal、Command Line 和 Insert 三种模式。下文所介绍的各种命令，基本都是工作于 Normal 模式或者 Command Line 模式下的。

在本文中，用 xxx 字体来表示 Normal 模式下的命令，用 :xxx 来表示 Command Line 模式下的命令

本文中只是介绍了一些常用的编辑命令，如果希望了解更多使用技巧，可以参考 VIM 的使用手册，或者在 VIM 中键入：VIM 来阅读其帮助文档。

1. 移动

(1) 基本移动

实际上，编辑文件的多数时间往往都花费在浏览，检查错误或者找出你要进行编辑工作的正确位置。所以高效编辑的第一要义是学习如何能够在文本中快速移动，准确定位。

基本移动：

移动到下个单词的词首 : w

移动到上个单词的词首 ： b

移动到下个单词的词尾 ： e

移动到上个单词的词尾 ： ge

移动到当前行的第一个非空白字符上 ： ^

移动到当前行的最后一个非空白字符上 ： g_

移动到行首：0 或者 <Home>

移动到行尾：$ 或者 <End>

移动到指定的第 n 行 ： nG

📖 说明：

例如要跳到 100 行，则键入 100G。

跳转到第一行：gg

跳转到最后一行：G

跳转到当前屏幕中的首行 ： H
跳转到当前屏幕的中间一行：M
跳转到当前屏幕的尾行：L

将当前行放到屏幕中央：zz

单字符搜索：fx 在当前行上查找下一个字符 x

Fx 在当前行查找上一个字符 x

搜索字符串：/ 先键入/，再输入想搜索的字符串，然后回车。.*[ ] ^ %/ ?~$ 这些字符有着特殊的意义，如果想搜索这些字符本身，需要在前面加上反斜线\ 转译。搜索当前光标下的 word 可以用快捷键 ： * 或者 # （他们的区别在于*向后查找，而#向前查找）。

当 VIM 搜索到你想搜索的字符串时，一般它将会用高亮来显示该字符串，如果你想跳到下一个匹配的字符串，键入 n ，跳到上一个则是 N

移动到匹配的括号上：% （默认只匹配（）[ ]{ } 三种括号）。

📖 说明：

- 此功能非常有用，当层次多时可以迅速定位括号的匹配是否出现了问题。默认只支持三种括号的匹配，如果想加入新的配对符号 a、b，可以输入：
- ：set mps += a:b 这里的 a、b 都只能是单字符的符号。
- 例如 ：set mps += <:>

---

(2) 高级移动

跳到最后修改的那一行：'. （很有用 ^_^）

跳到最后修改的点：`. （不仅跳到最后修改的那一行，还要定位到修改点）

跳到当前变量在当前文件的定义处：gd

📖 说明：

其实是跳转到当前变量在此文件中第一次出现的地方，不过一般来说，第一次出现的地方也就是变量定义的地方。

在当前位置作标记：ma

跳转到标记 a 处：`a

📖 说明：

这里的 a 可以是任意字母。

跳转到上一次光标所在处： ``

📖 说明：

相当有用，比如看代码时不小心误按了 n，光标一下子不知道跳到哪了，就可以输入``迅速回到原来的位置。再比如在看代码时，如果看到了一个不太清楚含义的变量，则可以先键入 gd 跳到定义处，再键入``回到刚才浏览的地方。

---

2. 编辑

(1) 基本编辑

删除当前光标下的字符： x

替换当前光标下的字符：ra （用字母 a 代替当前光标下的字符，这里的 a 可以是任意字母）

VIM 中基本的编辑命令形式为：编辑命令+作用范围

这里的编辑命令可以为 ：

删除：d

复制：y

改变：c ，这里改变的含义为删除并进入插入模式。

作用范围就是上一节移动中介绍过的各种移动命令，例如

d$就是删除光标所在位置到该列的最后一个字母。dw 就是删除光标所在位置到下个字的第一个字母。以及 dfx ：删除光标所在位置到本行中下一个字符 x 出现的位置（这里的 x 可以为其他任意字母）。

dd 删除一整行

yy 复制一整行

cc 改变一整行

🕮 说明：

vim 中的命令大多可以在前面加上数字用来表示重复多少次，例如删除三个 word 就可以用 “3dw”，删除 3 行可以用“3dd”等。

在当前位置的后面粘贴： p

在当前位置的上面粘贴： P

🕮 说明：

以"d"或"x"这样的命令删除文本时，被删除的内容还是被保存了起来．你还可以用 p 命令把它取回来，这个特性可以用来实现“剪切”的功能，例如要移动某行至另一个位置，可以先 dd 删除该行，再移动到别处用 p 粘贴。

将下一行与本行合并 J

改变当前光标下字符的大小写 ： ~

删除一个单词（包括词尾空格）：daw

🕮 说明：

dw 是删除光标所在位置到下个字的第一个字母，因此如果我们要用 dw 删除整个单词，就需要先将光标移动到词首，这样很不方便。使用 daw 就可以比较方便的做到这一点。这里的 aw 意为 A Word。

删除一个单词（不包括词尾空格）:：diw

插入模式下进行单词补齐：Ctrl - p

> 📖 **说明：**
>
> 如果你有一个信号名叫 fifo_ovfw_int_dly ，那么每次都敲这么长的信号名无疑是一件很痛苦的事情，这时你就可以利用单词补齐功能来帮助你键入，键入部分信号名后再输入 Ctrl -p，vim 开始在文档中搜索开头与其匹配的单词并补齐，如果有多个类似的信号名，则可以继续输入 Ctrl -p 来继续查找。

对齐

写代码时，将代码对齐经常占用了不少的时间，使用对齐命令将省去不少时间。

- 左对齐：:{range} left [margin]
  range 是需要进行对齐的行范围，margin 则是对齐后的行首距离窗口最左边的距离。
- 右对齐：:{range} right [width]
  与左对齐类似，range 是需要进行对齐的行范围，width 则是对齐后的每行的长度。

替换

替换是一种常见的操作，对应的命令是 substitute，其一般形式为：:[range]substitute/from/to/[flags]

上面的命令表示将“from”替换成“to”命令中的 substitute 可以用一个 s 来代替。[range]代表作用范围，[flags]代表可以输入的参数。

如果不指定范围，那么将只替换本行中的对象。如果不加参数，那么将只替换一行中第一次被发现的目标字符串。

范围的一般形式是最简单形式是两个以数字表示的行号．如 a,b 意味对从第 a 行到第 b 行之间的区域进行替换操作。（包括第 a 行和第 b 行）

“.” 代表当前行，“$”则代表最后一行。“%”代表对所有的行进行操作。

还可以用＋－来表示相对的偏移，比如 “. , .+5”就表示从当前行直到当前行下的第 5 行。（要键入这样的内容，可以直接在当前行上键入数字，再键入“：”，比如，按下"5:"，你会看到 vim 的最下方出现了 ：. , .+4 ）

可以在后面附加的参数包括 g （替换一行中所有符合的字符串） p （列出所有被改变的行） 以及 c （在执行每个替换前都要求用户确认）

如果你在替换命令后加上了参数 c，那么每次 Vim 找到目标字符串时都会停下来要求你确认，这时你可以键入的答案包括：

y	好吧，替换吧
n	不，这个先留着
a	别问了，全部换掉吧
q	退出，剩下的也不要管了

I	把现在这个改完就退出吧
CTRL-E	向上滚屏一行
CTRL-Y	向下滚屏一行

这里的 CTRL-E 和 CTRL-Y 是为了让你看清上下文来决定是否替换而设置的。这样可以防止当前行正好位于屏幕底端导致无法看到下文的情况。

替换命令中的目标字符串也可以由正则表达式来代替，（关于正则表达式的用法，由于偶自己还不是很精通，就不献丑了，有兴趣的兄弟可以自己去研究研究），这里只举一个例子：

:%s/\s\+$//

这个命令可以消除所有行尾多余的空格，命令中的\s\+$搜索所有的行尾空格，//将空格替换为空。

(2) 高级编辑

上面介绍的都是些基本操作，下面我们将介绍几个真正强大的技巧：

缩写

在编程中，总有一些类似的语句是经常出现的，例如 Verilog 中的 Always 块，E 中的 sequense 定义、for 循环等。如果每次都重新键入这些，无疑是在浪费时间，因此我们可以定义一种缩写，帮助自己快速定义。

定义缩写的命令是： :iab xx yyyyy （定义 xx 为 yyyyy 的缩写，这里的 yyyyy 不一定是一个单词，可以是多个单词及空格等）。

📖 说明：

- 例如我们想用“ad”来代表“advertisement”，就可以键入命令：
- :iab ad advertisement 这样你再键入 ad 后，Vim 就会自动将其扩展为 advertisement。仅仅键入 ad 并不会触发扩展，只有键入了一个完整的单词 ad 才会进行扩展，因此键入 add 是不会触发扩展的。

如果需要在缩写所代表的内容中加入回车，可以用<Enter>来代表回车

例如 ： :iab fft for i from to do { <Enter> <Enter>};

这条命令可以让你在键入 fft 后自动扩展为：

```
for i from to do {

};
```

类似这样的缩写命令可以将他们都写在一个文件中，再在.gvimrc 文件中加入 source 此文件的命令，以后就可以一直使用这样的缩写啦。

宏操作

编程中，我们经常会需要做一些重复性的操作，这时候，采用宏操作可以帮助我们简化这一过程。

使用宏操作需要三个步骤：

- 使用"qa"命令开始，后续的动作将被记录进名为 a 的寄存器中.（这里的 a 可以为 a 到 z 之间的任意一个字母）
- 执行你要执行的操作
- 按下 q 以结束对命令的记录（注意仅仅是 q 一个字符，不要键入多余的字符）.

现在你可以通过"@a"命令来执行刚刚记录下来的记录宏。（如果第一步里你用了其他的寄存器名，比如 b，那么这里就应该执行@b）。

下面是一个使用宏的例子：

假如你有如下的文件名列

stdio.h

fcntl.h

unistd.h

而你实际想要的结果如下:

#include "stdio.h"

#include "fcntl.h"

#include "unistd.h"

将光标移动到第一行上. 接下来执行:

qa	开始将后续的命令记入寄存器 a
^	将光标移动到行首
i\#include "<Esc>	在该行之前插入#include "
$	移动到行尾
a"<Esc>	在行尾加上"字符
j	移到下一行
q	停止记录

现在第一行已经搞定了，其它三行只需要执行 3 次"@a"命令，此命令也可以加一个数字前辍，这会使该记录被回放由该数字指定的次数。例如要执行 3 次，则可以输入"3@a"

使用宏时应该注意让你输入的操作形成一个闭循环，在上面的例子中，就是要注意在操作完本行后移动到下一行，这样你才可以用"3@a"来完成多次操作。

如果录制宏的过程中出现了一些问题，比如敲错了命令，那么怎么办呢？

当然你可以重来一遍，不过如果是比较复杂的宏，重来一遍也是件很痛苦的事情，特别是当你只敲错了一点点的时候。这种情况下我们有更好的办法来进行补救：

G	到文件最后

o<Esc> 生成一个新行，再进入 Normal 模式

"np 将寄存器 n 的内容置于该行. 你会看到这些命令就象你键入的普通文本一样.

{edits} 修改错误的部分. 这与编辑普通文本无异

0 到行首

"ny$ 将正确的结果回存到寄存器 n 中

dd 删除这行草稿

现在就可以使用修改过的寄存器 n 了。

3. 键映射、定义新命令

如果你有一些经常执行的重复操作，那么每次都录制宏也是一件很不爽的事情，对于这类经常的操作，我们有更好的办法，那就是：键映射。

键映射的基本思想是把某个键或组合键映射为一系列键的组合，这使得我们可以定义属于自己的快捷键。

<F1>之外的功能键在 Vim 中并没有预定义内容，因此拿这些键来做映射对象是最合适不过了。但是并不仅限于 F2 之类的键，事实上，你可以用任意键或者任意组合键来进行映射，下面是一个例子：

如果你想将 mount 变为 “mount”

那么可以定义映射：**map <F2> wbi”<Esc>ea”<Esc>**

这里之所以用 wb，而不是用 b 来移动到当前单词的词首，是为了防止当前光标就停在词首时，b 将跳到上个单词的词首。

这里的 map 命令是定义在 Normal 模式下对键序的重新映射.，如果要定义在 Insert 模式下的映射，可以使用 imap 命令

如果要取消一个键映射，可以使用 unmap 命令

有时候我们的重复操作中可能需要加上参数，这时候仅仅定义键映射可能就不能很好的完成作用了，这时候我们可以采用定义新命令的方式来做到这一点。

:command 是一条可以创建新命令的命令。例如我们希望把当前行中的“output”替换成“reg”，那么我们可以键入

:command SS s/output/reg/

这里我们就定义了一个新的命令 SS，它就等于 s/output/reg/今后我们就可以键入 “：SS”来代替“：s/output/reg/”。（自定义的冒号命令必需以一个大写字母开头. 但不能使用":X", ":Next"和":Print"这些保留字，也不能使用下划线! 数字是允许的，但是不鼓励使用）

如果需要在命令中加入参数，那么可以在 **command** 后面加上-nargs 选项，-nargs 的参数包括：

-nargs=1 1 个参数

-nargs=* 任意个数的参数

-nargs=? 0 个或 1 个参数

-nargs=+ 1 个或多个参数

比如我们可以定义：

:command -nargs=+ SS s/output/<args>/

此命令将 output 替换为 SS 命令后跟的内容

用上面的方法定义出的命令无法指定作用范围，如果需要指定作用范围，那么需要使用 －range 选项，-range 的参数包括：

-range 允许使用行号范围，默认是当前行

-range=% 允许使用行号范围，默认是所有行

定义了-range 选项后，可以在自定义的命令中以<line1>和<line2> 这两个关键字来代表这个范围中的起始行和结束行。例如：

:command -range=% SS <line1>,<line2>s/output/reg/

此命令将指定范围的行中的 output 替换为 reg。

重新定义一个命令可以使用**:command!**

删除一个命令使用**:delcommand**

4. 块操作

假设我们现在有若干行代码需要注释掉，那么最方便的方法是什么呢？当然你可以定义一个宏来完成这一操作，但是对于这种简单的编辑，最方便的办法应该是块操作。

命令 CTRL-v 可以进入一种特殊的选择模式（Visual Block 模式），在此模式下你可以选择一个矩形的文本块。然后可以对这个文本进行一些特殊的操作。（如果在你的 Gvim 中按下 CTRL-v 的作用是粘贴，那么也许你的 Visual Block 模式命令被映射成了 CTRL-q，如果 CTRL-q 也不行，那你可以用

:unmap <C-v> 这个命令来取消掉对 CTRL-v 的映射）

回到刚才的例子，如果你有若干行代码需要注释掉，那么可以进行如下操作：

首先移动光标到需要注释掉代码的第一行行首，再键入 CTRL-v 进入 Visual Block 模式，这时你会看到 vim 的左下方出现了 VISUAL BLOCK 的字样。

用 j 键向下移动，你会发现 Vim 以高亮为你选择了一条矩形文本块。

移动到需要注释代码的最后一行，键入 I 进入插入模式

这是光标跳回到了文本块的第一行，键入 //，按下<Esc>结束插入，你会发现刚刚选择的文本块的所有行都被加上了//。是不是很爽？

选择文本块的过程中，可以象移动光标一样用 hjkl 键来选择文本块，还可以使用"$"命令让每一行的被选择区域扩展到该行的末尾，不管这些行的长短是否参差不齐。

选择好文本块后，可以执行的操作有：

I 在文本块的左边添加文本 （这里添加文本时，不能包括换行，否则添加的文本不会被扩展到所有的行上）

A 在文本块的右边添加文本

c 删除文本块并添加文本

r 整个文本块以 r 后键入的 1 个字符来进行填充

> 整个文本块右移 1 个 tab 的距离

< 整个文本块左移 1 个 tab 的距离

除了上面列出的操作外，还可以键入 y 来 Copy 整个文本块，用 p 来进行粘贴。当然也可以用 d 或者 x 来进行删除操作。

选择一个文本块是需要花一点时间的，假如选择好了之后操作却出现了失误，难道就又要重新去选择文本块？别急，Vim 是很体贴的，这时候你只要键入 gv 就可以重新选中刚才选中的文本块了。 重新进行操作就可以啦。

除了用 CTRL-v 选择出的文本块，还存在另一种块：语句块

语句块指用（[{}]）这几种括号括起来的块，比较常见的应该就是 E 语言中的{}语句块了，如果想把一个{}括起来的语句块整体缩进一个 tab，那么就可以将光标移动到{上后键入 >% ，类似的，还可以用 d％来删除整个语句块，y%来 Copy 整个语句块。

5. 多文件编辑

正在编写着一个文件，突然又想打开另一个文件看看，怎么办呢？也许你的做法是再打开一个 vim。不过这样比较耽误时间，更好的办法是直接在当前 Vim 里打开，下面的命令可以让 Vim 关闭当前文件并打开另一个文件 aaa.v

:e aaa.v（如果 aaa.v 不在当前目录下，那么你就需要指定相对或者绝对路径）

如果当前文件存在还没保存的改动，那么 vim 会显示错误信息，并且也不会打开你想打开的文件。

被关闭的当前文件并没有消失，而是被 VIM 放到了 1 个缓冲区（Buffer）中。在 Gvim 中，你可以点击菜单上的 Buffer 按钮来看到所有的 Buffer 的列表，你也可以点击 Buffer 列表中对应的 Buffer 来打开对应的文件。

除了点击菜单上的 Buffer 按钮，还可以键入 : buffers 命令来查看整个缓冲区列表。:ls 命令也能起到一样的效果。键入:buffers 或者:ls 后你能看到的结果如下：

```
1   #h          "help.txt"       line 62
2   %l+         “usr_21.txt"     line 1
3               "usr_toc.txt"    line 1
```

第一列是缓冲区编号. 你可以在编辑该文件时以此代替文件名，见下

文. 缓冲区编号之后是缓冲区类型标志字符. 然后是文件名和上次退出时光标所在的行号. 可能的缓冲区类型标志字符如下（从左到右）:

如果想跳到另一个文件，比如上例子中的 usr_toc.txt，你可以键入

: buffer3 这里的 buffer 可以简写为 b，也就是说你可以键入 :b3

这种跳转并不会关闭当前文件，也就是说，你不必非得保存你的改动，等你再跳回来的时候，你的改动会依然在那里。

当然你也可以以文件名来指定跳转，可以键入

: buffer usr_toc.txt 来执行跳转，并且和 VIM 中其他地方一样，这里也可以用 TAB 键来进行文件名的自动补齐，并且特殊的是：这里键入文件名并不一定非得从头键入，你完全可以随便键入文件名中的任意几个字符然后让 VIM 去寻找能匹配的文件，比如说，你可以键入 :buffer toc 然后按下 TAB，你会发现 VIM 自动匹配到了合适的文件名。

6. 多窗口编辑

编码中一定会经常遇到需要对比一下其他文件，或者需要对比一下本文件的两个地方的情况，这时候你怎么办呢？打开两个 VIM 然后把他们上行放到一起？Oh my GOD！这样做效率可真不是一般的低，这时候最好用的办法就是使用多窗口编辑。

打开一个新窗口的命令是： **:split** 其快捷键是 **CTRL-w + s** （先按 CTRL-w 再按 s）。这命令会把该命令将屏幕分为上下两个窗口并将光标定位在上面的窗口中。

ok，现在你等于同时拥有两个 VIM 窗口了，你可以在一个窗口打开一个文件，另一个窗口打开另一个，这在比较的时候是很好用的。

关闭当前窗口的命令是**:close** 其快捷键是 **CTRL-w + c**

如果你打开了一大堆窗口, 但现在你只想把重心放在其中一个上面。那么你可以使用命令 **:only** ，这会关闭所有其他的窗口而只保留当前窗口。其快捷键是 **CTRL-w + c**

如果你想打开一个新窗口并编辑另一个文件，那么当然不需先用 split 分裂处一个窗口再用 ：e 来打开文件，这两步工作可以用一个命令来完成：

: split xxxx.v 此命令可以打开第二个窗口同时在新打开的窗口中开始编辑文件 xxxx.v

如果你想打开一个新的空窗口以便开始编辑一个新文件，那么可以用命令

: new

上面提到的新窗口都是位于当前窗口之上的，如果你想把新窗口放在当前窗口的左边，也是说，如果你想从垂直方向上分隔窗口，那么可以用命令

: vsplit

在打开一大堆窗口的情况下要退出 Vim, 你当然可以一个一个地关闭这些

窗口. 但是你也可以一下搞定：

: qall 此命令会一下退出所有窗口，如果这些窗口中有被修改又没保存的,Vim 就不会退出. 光标也会自动被定位到该窗口中。

类似的，还有 **: wall** 保存所有窗口中的改动。

还有一个对":qall"和":wall"的组合: 保存并退出所有窗口： **: wqall**

最后再介绍两个非常有用的快捷键吧。

gf 此快捷键可以以当前光标所在的 word 为文件名，在当前窗口打开此文件。

例如你的文件中有这样一行：

import /home/l48758/xxx.v

那么现在把光标移动到 xxx.v 上再摁下 gf，VIM 就会找到并打开该文件。

如果你想在另一个窗口打开此文件，那么可以用 CTRL-w +f 。

: bd 此快捷键可以从当前的文件回到上一次的文件。

例如，你使用 gf 在当前窗口打开了一个文件，如果此时想退回原窗口即可以使用：bd。为了方便起见，你还可以给：bd 定义一个快捷键如 F6，参见前面的键映射有，map <F6> :bd<Enter>。如果一个模块下面有很多文件，你可以建一个文件列表，这样你只需要打开文件列表，然后使用 gf 和：bd 就可以在各文件之间快速切换，而不需要每次都新开一个 VIM。

2.3 通用接口

概述：在网络芯片开发过程中，我们经常要遇到一些接口电路，这里列举了几种接口路的基本概念，帮助设计/验证工程师了解这些电路接口的知识

2.3.1 SPI4 接口概述

1. 概述

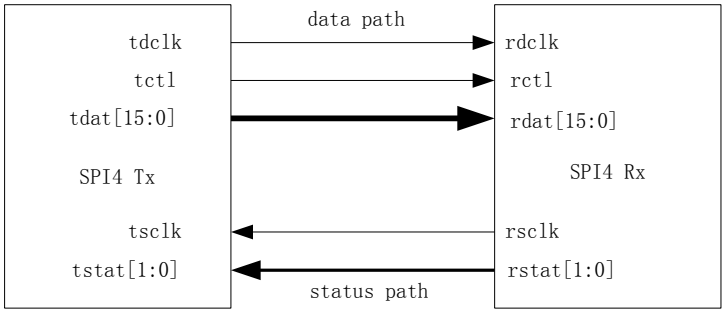
SPI4 接口是一种可以实现 10Gbps 带宽传输的高速接口，包含数据通道和 FIFO 状态通道。数据通道传输数据报文，状态通道传递各端口 FIFO 的流控信息，用来调整各端口的流量突发。SPI4 接口有以下特性：

- 最大支持 256 个端口；
- 数据通道：
  16bit 宽；
  控制字占用传输带宽；
  LVDS I/O；
  源同步双沿时钟，最小工作频率 311MHz；
  单根信号带宽最小为 622Mbps；
- FIFO 状态通道：
  LVTTL 或 LVDS I/O；
  LVTTL，状态通道时钟频率为数据通道时钟的 1/4，LVDS，状态通道和数

据通道频率相同；

2bit 宽的 FIFO 状态信息；

2. SPI4 接口信号



- rdclk（tdclk）：数据通道随路时钟
- rctl（tctl）：数据通道控制信号（数据传输）
- rdat（tdat）：数据通道数据总线（数据传输）
- rsclk（tsclk）：状态通道随路时钟（LVTTL，1/4 rdclk ）
- rstat（tstat）：FIFO 状态信息（传递反压）

3. SPI4 数据通道

(1) SPI4 数据格式

- SPI4 控制字：
  PCW：payload 控制字；
  ICW：idle 控制字；
  TCW：training 控制字
- SPI4 数据字：
  PDW：payload 数据字
  TDW：training 数据字

表2-11 SPI4 数据格式

信号域	名称	描述
rctl	control	控制字标志。0：rdat 为数据字；1：rdat 为控制字。
rdat[15]	type	控制字类型。0：PCW；1：ICW 或 TCW。
rdat[14:13]	EOPS	EOP 状态。00：非 EOP；01：EOP 丢弃；10：2byte EOP；11：1Byte EOP。
rdat[12]	SOP	SOP 标志。0：非 SOP；1：SOP。
rdat[11:14]	ADDR	端口地址。
rdat[3:0]	DIP4	DIP4 校验。

(2) 数据通道状态图

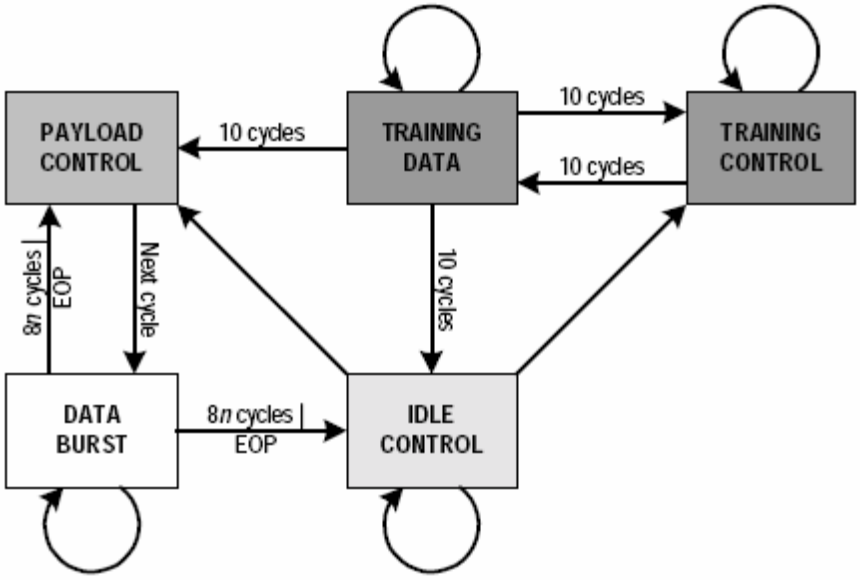


图2-50 SPI4 数据通道状态图

(3) 数据通道的重要概念

- Burst
  一个包可分成一个或若干个 burst 传输，非 EOP 的 burst 大小（burst_size）为 16Byte 的整数倍．EOP 的 burst 大小为包尾部分，小于等于 burst_size。
- 两种数据传输模式
  Interleave 模式：各个端口的包以 burst 为单位交织着传输。
  包模式：一个包的传输过程中不允许插入其它包的 burst。
- Training
  分为初始化 training 和周期性的 training。用于 Rx 的同步以及动态 deskew。一个 training 方式中包含若干个 training 序列，一个 training 序列由连续的 10 个 F000（rctl＝0）和 10 个连续的 0FFF（rctl＝１）组成。周期性 training 方式的间隔为 DATA_MAX_T，一次 training 方式中的 training 序列数为 alpha。

4. SPI4 状态通道

(1) 状态通道数据类型

- Framming ：11 帧
- FIFO 通道状态：
  00：starving
  01：hungry
  10：satisifed
  11：非法通道状态
- DIP2

(2) 日历（Calendar）

- Calendar_i：通道序列

- Calendar_len：通道序列长度
- Calendar_m：通道序列循环次数

(3) 状态通道状态图

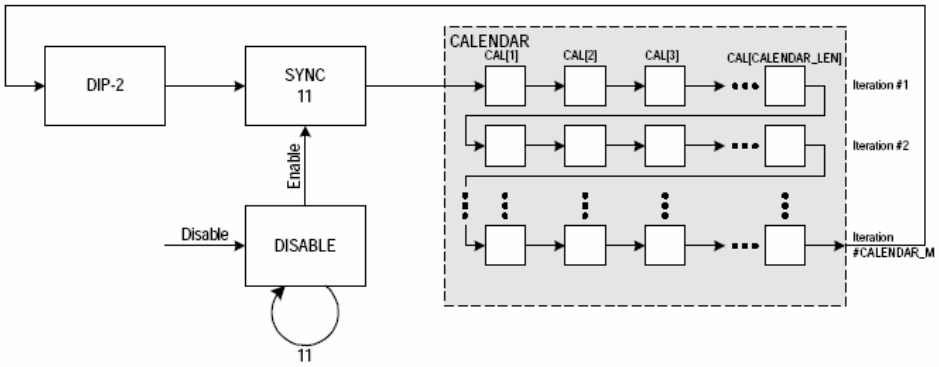


图2-51 SPI4 状态通道状态图

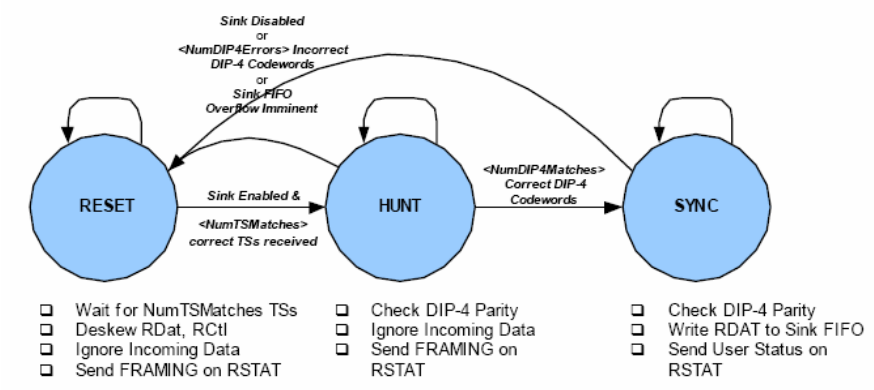
5. SPI4 初始化参数及同步过程

(1) 初始化参数

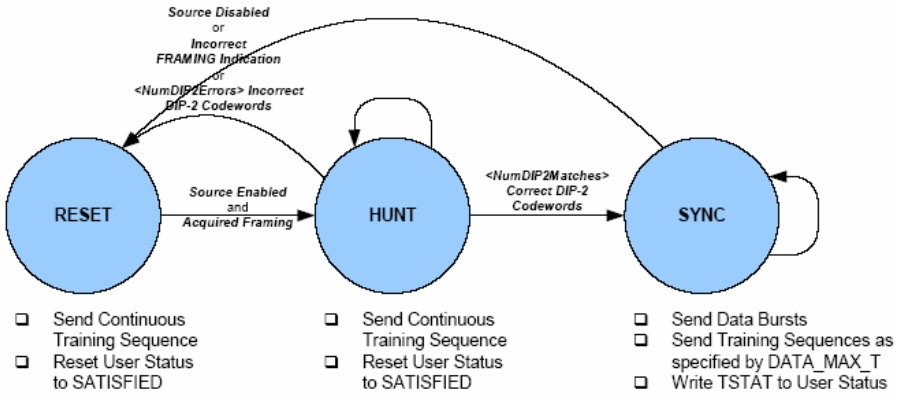
- SPI4 Rx：
  NumTSMatches：正确的 training 序列数.
  NumDip4Matches：连续正确 Dip4 的数目.
  NumDip4Errors：连续错误 Dip4 的数目.
  Calendar：Calendar_i，Calendar_len，Calendar_m.
- SPI4 Tx：
  NumDip2Matches：连续正确 Dip2 的数目.
  NumDip2Errors：连续错误 Dip2 的数目.
  DATA_MAX_T , alpha: 周期性 training.
  Calendar ： Calendar_i， Calendar_len， Calendar_m.

(2) 同步过程

I. SPI4 Rx的同步过程



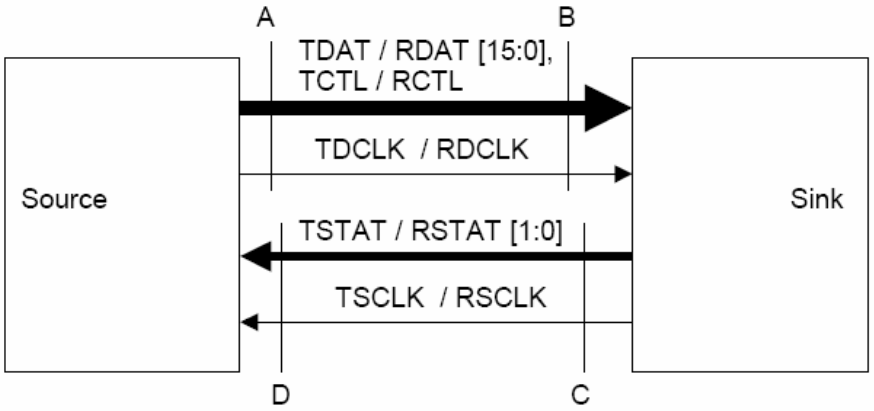
II. SPI4 Tx的同步过程



6. SPI4 接口时序

由于 SPI4 接口的数据采取 DDR 的方式传输，为了达到 10Gbps 的传输速率，数据通道随路时钟必须工作到 311MHz 以上（311M x 2 x 16 = 9.952Gbps）。由于控制字及 training 序列对带宽的消耗，实际应用中，工作频率一般都会高达 350MHz 左右，因此一个数据周期（UI）也就只有 1/（2x350M）= 1.43ns 左右。

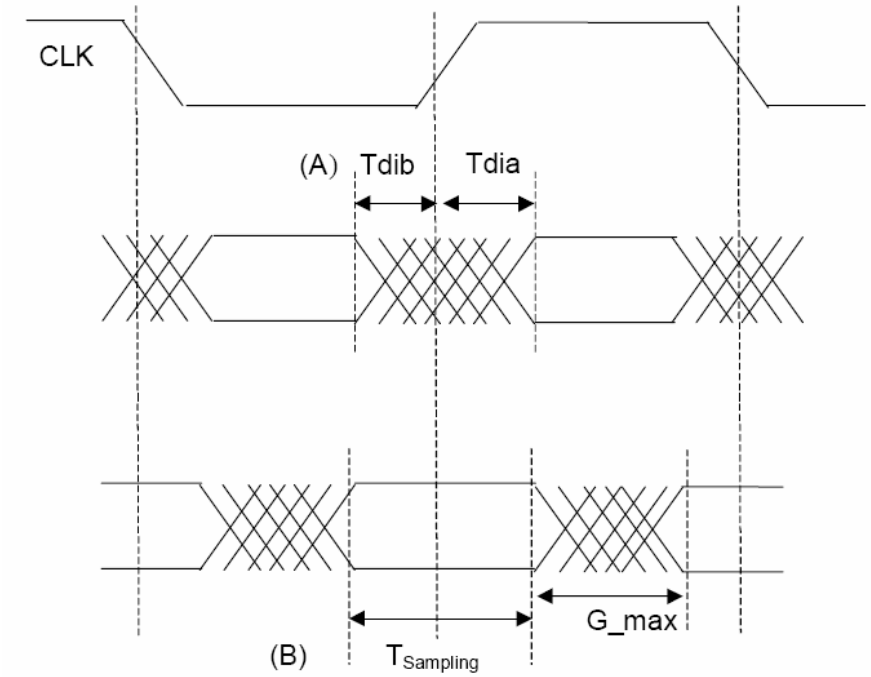
(1) SPI4 接口各时序采样点



(2) 数据通道（静态对齐）

静态对齐的方式，就是用一个时钟相位对 rctl 和 rdat 进行采样，这样要求总线的 skew 足够小，才能找到可靠的采样窗口。

Symbol	Description		Min	Max	Units
fD	TDCLK / RDCLK Frequency		311		MHz
	TDCLK / RDCLK Duty Cycle		45	55	%
T_dia, T_dib	Data invalid window with respect to clock edge. (Reference point A)			280	ps
G_max	Worst-case cumulative skew and jitter contribution. (Reference point B)			790	ps
Tsampling	Data valid window with respect to clock edge. (Reference point B)			$\frac{1}{2fD}$ - G_max	ps
	20% - 80% rise and fall times (UI = 1/2fD)	(Reference point A)	100 ps	0.30 UI	
		(Reference point B)	100 ps	0.36 UI	



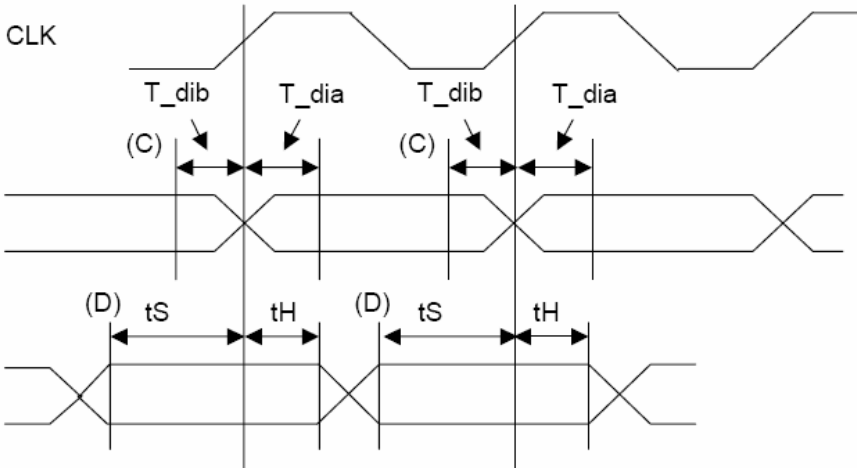
(3) 数据通道（动态对齐）

采用动态对齐的方式，可以对每个信号线用一个独立的时钟相位单独采样，各时钟相位频率相同。这样，对总线的 skew 要求就降低了，但是增加了实现的难度。

Symbol	Description		Min	Max	Units
fD	TDCLK / RDCLK Frequency		311		MHz
	TDCLK / RDCLK Jitter (at reference point A)			0.10	UI
	TDAT / RDAT / TCTL / RCTL Jitter (at reference point A)			0.24	UI
	20% - 80% rise and fall times (UI = 1/2fD)	(Reference point A)	100 ps	0.30 UI	
		(Reference point B)	100 ps	0.36 UI	

(4) 状态通道

Symbol	Description		Min	Max	Units
fS	TSCLK Frequency			fD / 4	
	TSCLK Duty Cycle		40	60	%
T_dia	Data invalid window with respect to clock edge.	(Reference point C)	2.5		ns
T_dib		(Reference point C)	1		
tS_{SCLK}	TSTAT Setup time to TSCLK , RSTAT Setup time to RSCLK.	(Reference point D)	2		ns
tH_{SCLK}	TSTAT Hold time to TSCLK, RSTAT Hold time to RSCLK.	(Reference point D)	0.5		ns



2.3.2 Interlaken 接口

1. 常用术语

BurstMax 最大数据突发长度 (64 字节的倍数)

BurstShort	突发控制字最小间隔 (最小为 32 字节，8 字节递增)
BurstMin	指定最小的尾包突发长度
MetaFrameLength	每条链路上传送的数据长度，包括一个同步字，一个扰码状态字，一个诊断字，一个或多个跳字，以及数据净荷
Word	8 字节，接口上传送的数据和控制字的基本单元
Block Type	块类型，每个控制字的前 6 个比特，用于区分不同的控制字：比特[63:58]区分同步字、跳字、扰码状态字和诊断字，比特[63]用于区分突发/空闲字节
Burst Control Word	突发控制字，比特 63 = ‘1’ 和类型 = ‘1’
Idle Control Word	空闲控制字，比特 63 = ‘1’ 和类型 = ‘0’
Synchronization Word	同步控制字，块类型 = 0b011110，以 MetaFrameLength 为周期同时在所有链路上发送，用于同步扰码器和链路对齐
Scrambler State Word A	扰码状态控制字，块类型 = 0b001010, 紧跟同步控制字发送，用于传送当前的扰码器状态给接收端
Skip Word	跳字，块类型 = 0b000111, 用于中继器（repeater）的时钟补偿
Diagnostic Word	诊断字，块类型 = 0b011001, 紧挨在同步字字之前发送, 用于传递每链路的错误信息诊断和（可选的）每链路状态信息
Lane Skew Tolerance	链路偏差容忍：107 UI

2. 简介

当前在网络应用中有两种主流的高速芯片互连接口协议：XAUI 和 SPI4.2。SPI4.2 优点在于通道化，可编程突发长度，支持每条链路的反压，但是过宽的接口限制了它的可测试性，以及协议本身的源同步特性降低了它的有效传输距离。相反的，XAUI 是一个 4 通道窄接口，长传输距离，并且适合各种介质：PCB 上的 FR4，背板和电缆。然而作为一种基于包的接口，协议缺少通道化和流控制限制了它的应用。并且这两种协议都支持固定的配置，限制了开发者适应应用的裁减。

Interlaken，它是一种窄的，高速的，通道化的包接口。

Interlaken 接口具有以下特性：

- 支持自动初始化和自恢复（定界/同步/对齐）；
- 支持 256 个 channel，最大可扩展至 8k；
- 支持大带宽，目前可支持单 core 20G~500G；
- 支持 lane 数目可配置，目前可支持单 core 4lane~24lane;

- 支持 SerDes 速率灵活，3.125Gbps~28Gbps；
- 支持 lane mapping；
- 支持 channel/link 级流控，支持带内/带外两种方式；
- 支持 packet 模式和 interleaved 模式；
- 支持可配置的 BurstMax, BurstMin, BurstShort 和 MetaFrame 等参数；
- 支持 CRC32/CRC24 校验：
- CRC24 多项式
  - X^24 + X^21 + X^20 + X^17 +X^15 + X^11 + X^9 + X^8 + X^6 + X^5 + X + 1
- CRC32 多项式
  - X^32+X^28+X^27+ X^26+ X^25+ X^23+ X^22+ X^20+ X^19+ X^18+ X^14+ X^13+ X^11+ X^10+ X^9+ X^8+ X^6+ 1
- 支持 64B/67B 编解码、加解扰；
- 加扰多项式
  - X^58＋X^39＋1
- 支持各 lane 之前的 skew，最大支持 226UI；
- 支持丰富的错误检测和上报

3. 应用场景

Interlaken 协议可以应用与多种场合：

o 帧/MAC 与网络处理器（NPUF）或者二层三层交换机(L2/L3 switch)接口

o 线卡与交换机光纤接口

它也能在多种介质传输：FR4（PCB），背板或者电缆。

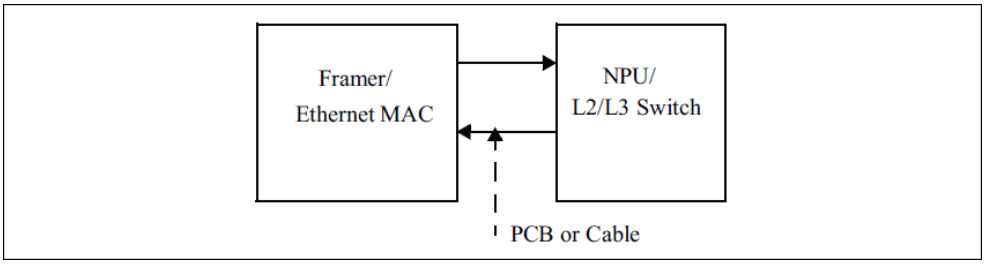


图2-52 Framer/MAC to NPU/L2 or L3 Switch

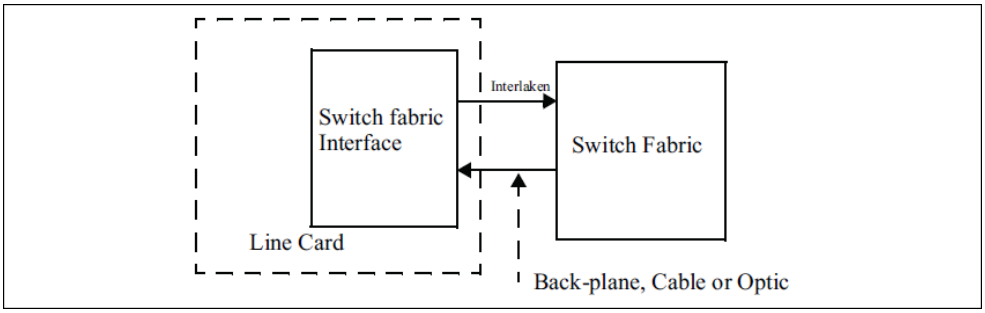


图2-53 线卡与交换机光纤接口

4. 基本概念

在 Interlaken 协议中定义了两种基本的数据结构：数据传输格式和元帧。数据通过接口发送的时候要将分解成突发段进行传送。每个突发段由一前一后两个控制字作为边界，其中一个在前，一个在后。在这两个控制字中定义了接下来是数据或者其它功能如：包开始，包结束，错误检测等等。每一个突发段都与一个逻辑通道相关，它可能是系统中的一个实际端口，也可能是其它相连的数据流。包数据分解为一个或者多个突发段在通道上按照顺序传输，其中突发段长度是个可配置参数。通过将数据分解成突发段，接口可以交替的在不同的通道上发送数据以实现低延迟。协议定义的元帧是用于支持数据在 SerDes 接口上传输。元帧包含了 4 个唯一的的控制字，通过它们实现链路对齐，扰码器同步，时钟补偿和链路诊断。元帧在带内与数据一起传输，通过控制字的特定格式来将其从数据中区分开。

数据传输格式以及元帧格式将在接下来的小节中阐述。

5. 协议层

1) 传输格式

数据在 Interlaken 接口上通过一系列可配置数量的 SerDes 通道进行传输。为了实现这个目的通道（或链路）被定义为两颗芯片之间的一条简单串行接口。协议被设计为可以运行在任意数量的通道上，包括一条，而且本身没有上限。实际项目中可以根据需要选择合适的通道数目。协议中并不要求支持可变数量的通道。在接口上传送的基本数据单元为 8 字节长的字。选择这个长度的原因是为了满足协议中采用的 64B/67B 编码需要，同时它也是描绘突发段的控制字的长度。将传送数据单元设计为与控制字同样的长度（64 字节）后就很容易的适应接口的宽度。数据和控制在接口上按照顺序条状的分布到各个通道，从通道 0 一直到通道 M，然后在下一个块重复此过程。在下面中说明了该过程

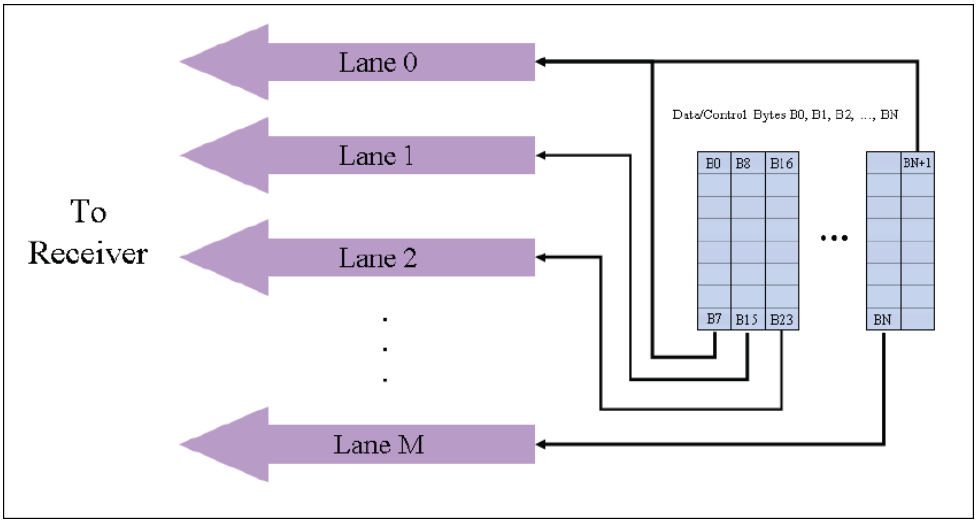


图2-54 通道条状发送示例（Lane Striping）

64B/67B 编码在每个通道上是完全独立进行的。传输过程通过两个基本类型字来完成：数据字和突发/空闲控制字；这两个字通过 64B/67B 的帧字段区别，它们的格式如图所示：

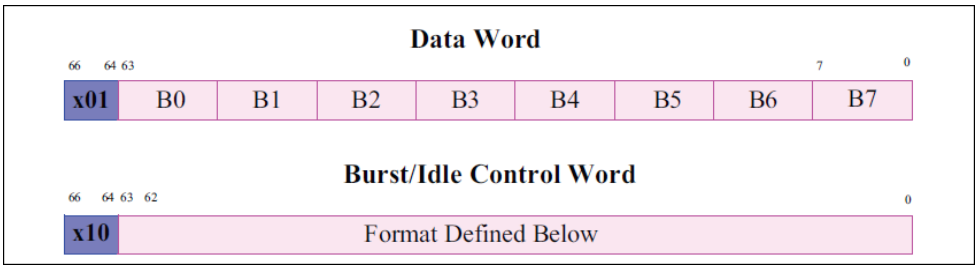


图2-55 字格式

数据字和控制字在发送的时候都是按比特顺序发送（每个字都是从最高位到最地位），从比特[66]到比特[0]；

在帧层介绍了另外 4 中控制字，具体细节见后续小节帧层。

2) 突发结构

数据传送过程

Interlaken 接口带宽从支持的数据通路被分解到数据突发中。数据包在接口发送的时候被拆分为一个或者多个突发段，这些突发段由一个或者多个控制字来描述。

为了避免将包拆分为任意长度的突发段，协议定义了两个参数：

- BurstMax: 最大数据突发长度 (64 字节的倍数)
- BurstShort: 突发控制字最小间隔 (最小为 32 字节，8 字节递增)

接口在通常情况下发送长度为 BurstMax 的数据和一个控制字。发送端可以选择可用的通道顺序在上面发送，而这些通道受到流量控制状态的限制。突发段

可以在每条通道上传输直到包被发送完成，此时在那条通道上也可能会有新的包发送。

由于接口是通道化的，因此当每条通道上都剩余非常少量的数据发送的时候，包发送结束可能会在多条通道上紧接着出现（back-to-back）。发送端和接收端的内部缓存都设计为非常大的宽度，这种情况下，它们必须工作在非常高的时钟频率下来处理。为了减少接收和发送端的这种负担，定义了 BurstShort 参数来保证连续的突发控制字之间具有一个最小的间隔。最小的 BurstShort 间隔为 32 字节，按 8 字节递增，无上限要求。

下图说明了如何用 BurstShort 保证最小间隔。 BurstShort 强制要求在下一个突发控制字之前插入额外的空闲控制字。在下面的例子中，控制字 1 的包尾（EOP_Format）指示了包尾（EOP）和下一个数据字的大小，并且控制字 1 的 CRC24 字段包括了最后一个字数据的空闲控制字 1。空闲控制字 2 和 3 的插入是为了满足 BurstShort 的要求，而接下来的突发控制字是为了后面发送的数据。

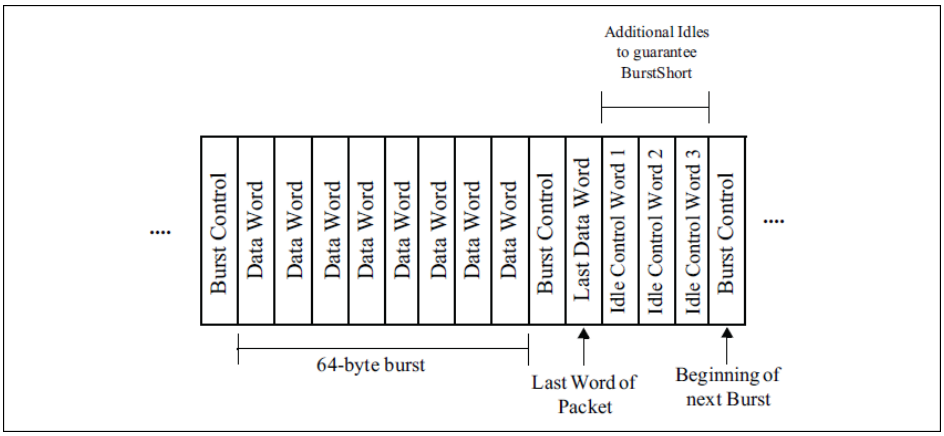


图2-56 BurstShort 保证最小间隔

可选的发送机制

对于一些特定的包长和 BurstMax，在上面这种简单的调度机制下可能会在包尾造成带宽的浪费。当包的长度用 BurstMax 取模(包长% BurstMax)后，如果得到的值比较小，那么在最后一个 BurstMax 长度的突发段发送以后，就必须在剩下的字中插入空闲字来满足 BurstShort 的长度要求，最坏的情况下，将在每个包中浪费（BurstShort - 1）个字节的带宽。但是，通过查找前面的数据包来寻找的包尾（EOP）的方式是可以提高调度机制的。下面就提供了这种机制的说明以及可选的优化接口性能的指导方针。

这个方针中引入了一个新的参数：

- BurstMin: 步长为 32 字节，服从 BurstMin <= BurstMax/2 和 BurstMin >= BurstShort; 其用法在下面阐述。

下面定义的额外的参数用来更好的说明问题：

- packet_length = 包的总长度

- packet_remainder = 数据包开始传送以后包中剩余未传送的数据
- data_transfer = 当前突发传送的数据长度
- i = 传送包所需要的突发块的次数

调节突发块长度的算法在下面的算法中描述：

```
packet_remainder = packet_length
for (x=1; x <= i; x++) {
if (packet_remainder >= BurstMax + BurstMin) then
data_transfer = BurstMax
else
if (packet_length MOD BurstMax < BurstMin) && (packet_remainder > BurstMax) then
data_transfer = BurstMax - BurstMin
else
data_transfer = packet_remainder
packet_remainder = packet_remainder - data_transfer
}
```

这个算法可以保证包的最后一个突发块长度介于 BurstMin 和 BurstMax 之间，避免了各种包尾短分段的问题。但是为了要让这个算法正确的运行，BurstMin 的值不能超过的 BurstMax 一半。举个例子，一个 513 字节长度的包需要在 Interlaken 接口上发送，并且 BurstMax = 256 字节 和 BurstMin = 64 字节，在这种情况下，发送的的 3 个突发段为：

- 突发段 1 = BurstMax = 256 字节
- 突发段 2 = BurstMax - BurstMin = 256 - 64 = 192 字节
- 突发段 3 = packet_remainder = 65 字节

如果上面的包长为 511 个字节，那么仅仅两个突发块需要被传送：

- 突发块 1 = BurstMax = 256 字节
- 突发块 2 = packet_remainder = 255 字节

只要遵循上面的规定，项目中可以根据需要自行调整 BurstMax 和 BurstMin 参数的值。

上述可选的算法是用于指导解决项目中传输突发块遇到的效率问题。只要发送端正确遵循 BurstShort 和 BurstMax 参数要求，即使采用不同的方法对包进行分段的时候，接收端就也不会增加额外的负担。有这样的例子，当接口类型需要转换成另外一种时，重新对突发长度的格式进行调整将会增加不必要的负担。主要遵循上面的规定限制，设计者也可以采用其它的调度算法。

控制字格式

突发块由 8 字节长的控制字描述。控制字的识别通过数据流中比特位[66:64]为'x10'，比特位[63]为'1'来实现。突发和空闲控制字格式在图中说明：

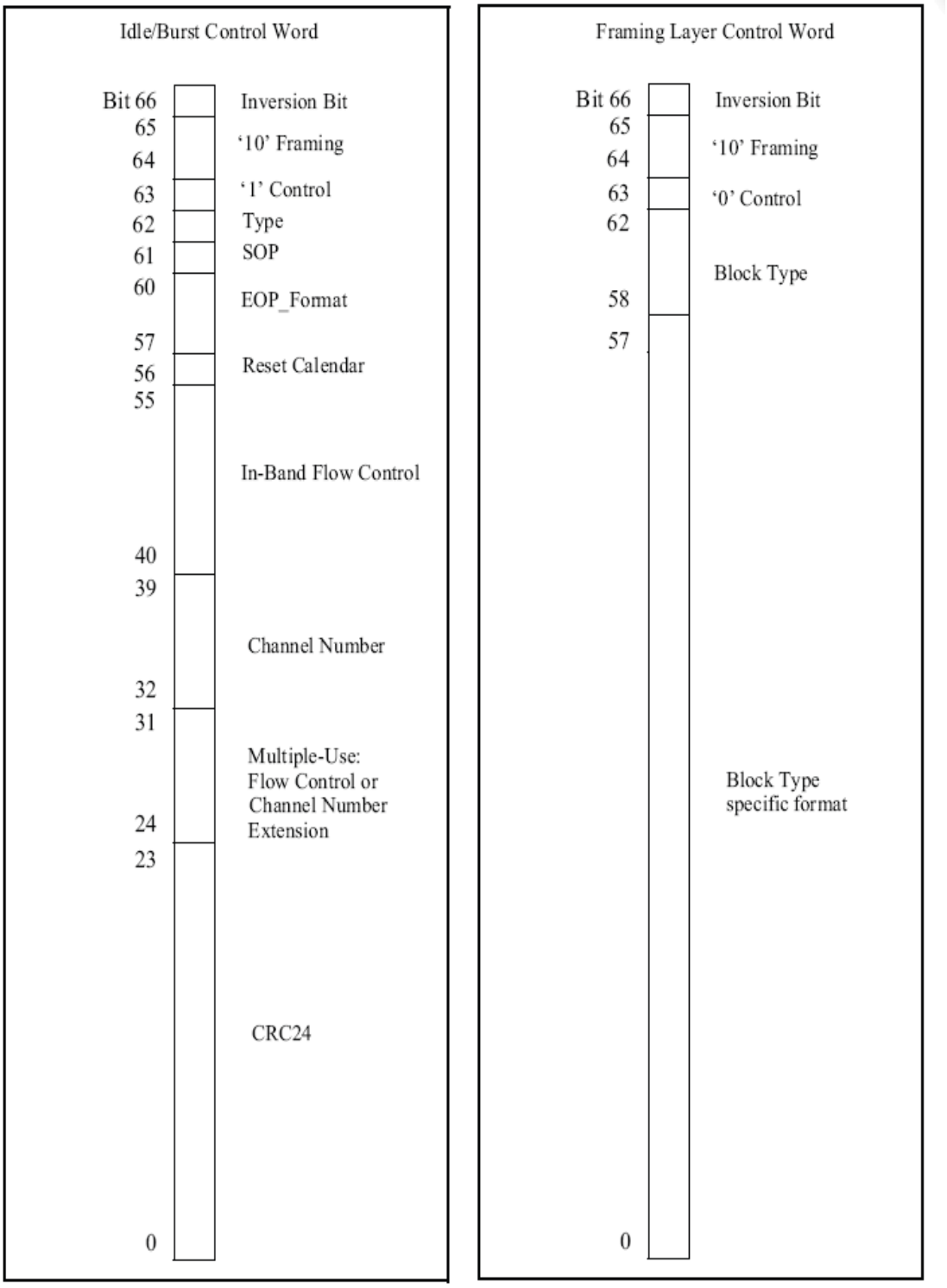


图2-57 控制字格式

表2-12 突发空闲控制字格式

段名	比特位置	功能

反转指示位（Inversion）	66	指示比特[63:0]是否需要反转，1=反转，0=不反转
帧指示位（Framing）	66:64	64B/67B 编码中区别控制字还是数据 01：数据 10：控制字
控制标志位（Control）	63	1：空闲/突发控制字(Burst/Idle Control Word) 0：帧层控制字(Framing Layer Control Word)
类型指示位（Type）	62	1：突发控制字(Burst Control Word) 0：空闲控制字(Idle Control Word)
包起始指示（SOP）	61	包起始指示 1：本控制字后面的数据突发字为包开始数据 0：本控制字后面的数据突发字为包中间或者尾部数据
包结束指示（EOP_Format）	60:57	1xxx：指示包结束，此时比特[59:57]指示突发段中最后 8 字节数据中有几个有效数据。如’000’表示 8 字节有效，’001’表示 1 个字节效，’111’表示 7 个字节有效；合法的字节是从比特位置[63:56]开始。 0000：不是包尾，无错误 0001：错误并且包结束 其它组合未定义
复位列表（Reset Calendar）	56	1：指示带内流控制状态表示通道列表（calendar）的开始
带内流控（In-Band Flow Control）	55:40	当前 16 个访问列表（calendar entries）中的单比特流控制状态。 1：访问列表代表的通道状态为开（XON） 0：访问列表代表的通道状态为关（XOFF）
通道数目（Channel Number）	39:32	当前控制字后面的数据所在通道号；对于空闲控制字，本段全部设置为 0
多用途段（Multiple-Use）	31:24	本段根据应用有多种用途。 1. 如果需要超过256个通道，那么这8比特可以用来扩展通道号，它将作为通道号的低8位。 2. 如果需要额外的带内流控，这些比特可以用来表示8个带内流控状态，紧跟[55:40]表示的16个带内流控状态。 如果需要，这些比特还可以用作别的用途，而不限于在此规范。
校验段（CRC24）	23:0	CRC 校验段，包括前一个数据突发段和本控制字

突发控制字（Type=’1’）指示了数据突发段的开始。每个突发数据传送必须由一个突发控制字作为开始，它指示了将要传送的数据的突发的开始和通道的数目。当突发控制字出现在数据突发块之间时，包尾指示(EOP_Format)和CRC24 字段应用于前一个突发数据，而包开始和通道号应用于紧跟着的后面数据（这样做的目的是与 SPI4.2 保持相似的突发控制机制）。

在没有新数据需要传送的时候，空闲控制字（Type=’0’）会被一直发送。这是因为流控制信息必须不断的发送到接收方，而在空闲控制字和突发控制字中，流控制字段都是有效的。发送端通常会在两种类型的控制字中都发送有效的流控制状态信息。

突发控制字中的包尾（EOP_Format）字段指示了在最后一个数据字中有多少字节是有效的。无效的字节会被接收端丢弃。这里约定，第一个有效的字节位于[63:56]，第二个位于[55:48]，以此类推。

数据和控制字的完整性由 24 位的 CRC 校验来保证。CRC24 校验的计算字段位突发中的所有数据以及控制字的所有字段。CRC24 多项式如下：

x24 + x21 + x20 + x17 +x15 + x11 + x9 + x8 + x6 + x5 + x + 1

具体的 CRC 计算规定见附录 B。

3) 状态图

下面的这些状态图说明了接口控制逻辑的主要步骤：

每条 SerDes 接收通道的工作过程如图:

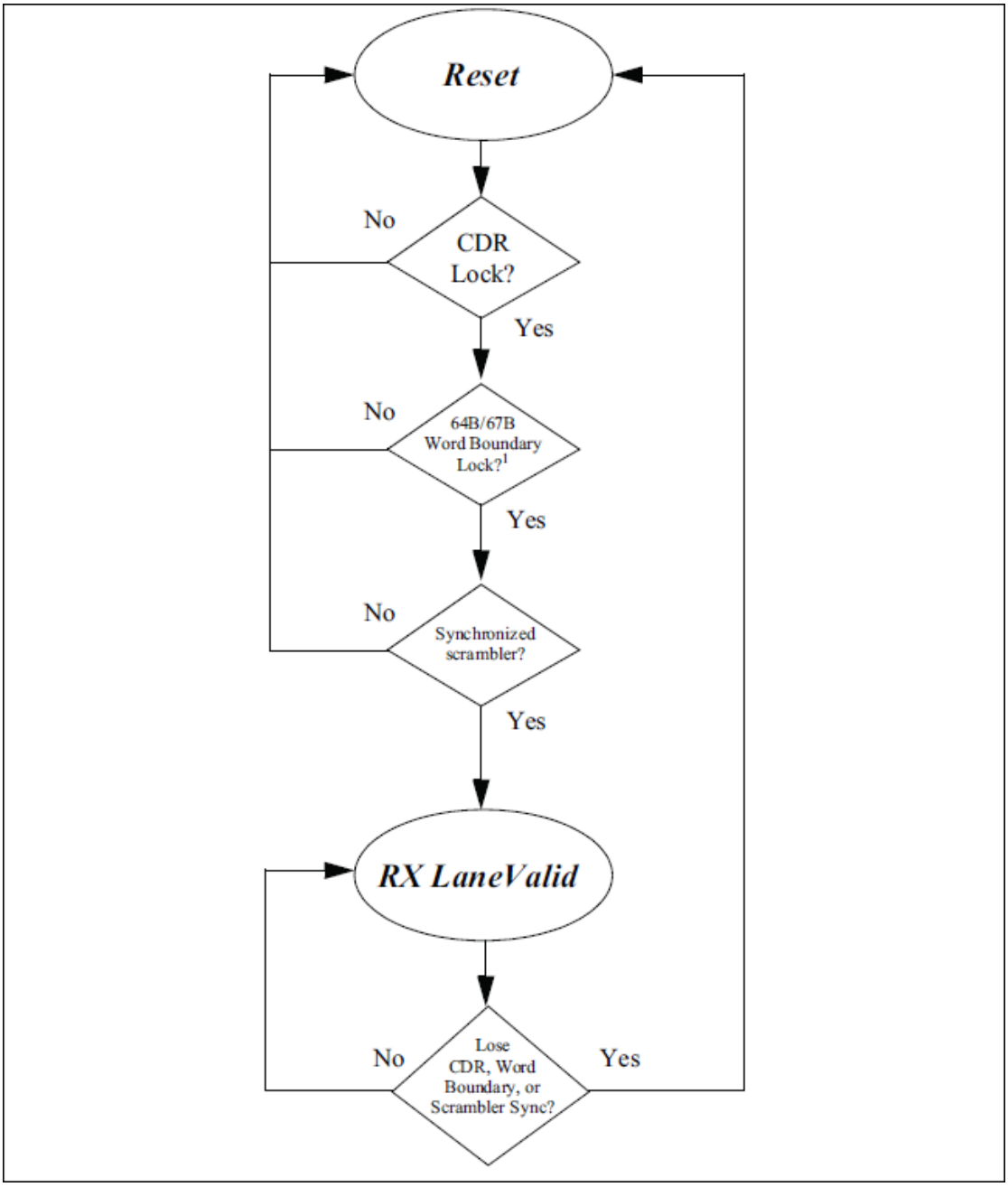


图2-58 每条链路接收流程

注意:

1. 64B/67B 字节边界锁定图。

接收端所有链路（作为一个逻辑整体）的工作流程如下：

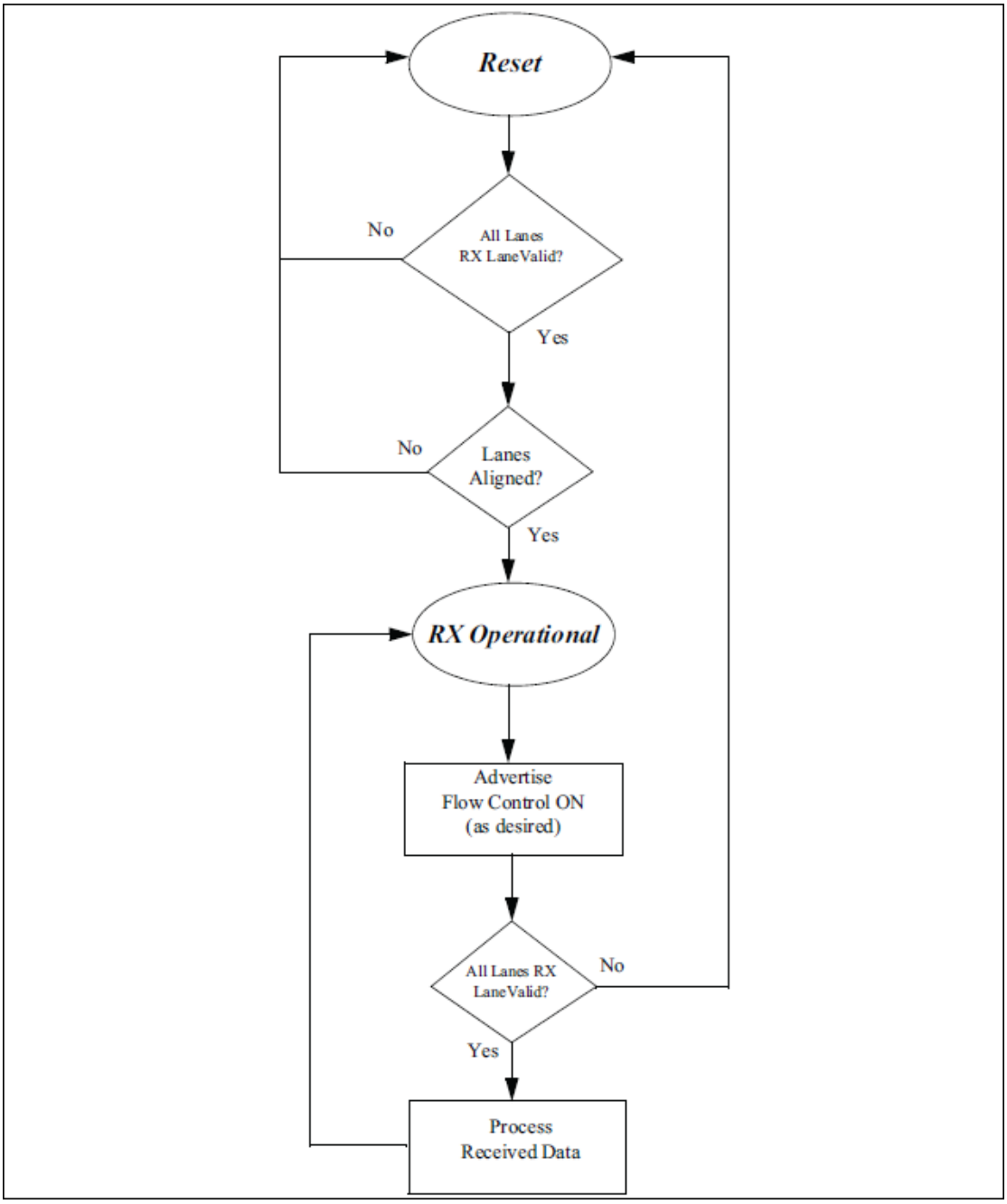


图2-59 接收接口状态流程

当接口的接收端进入接收状态（RX Operational）状态后，接口可自由的通过发送 ON 状态到所有通道上的方式，来告知发送端可以发送数据。

发送端（所有的通道作为一个逻辑整体）的运行状态流程如图:

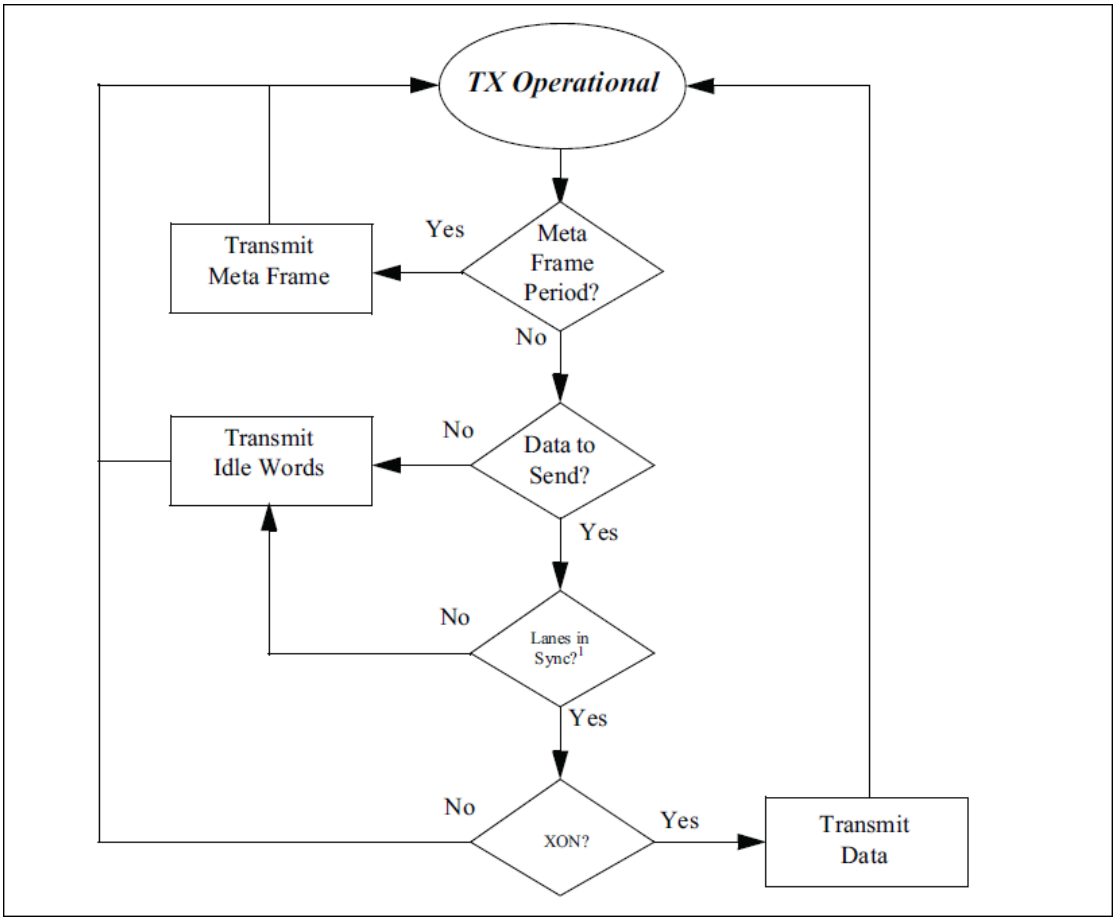


图2-60 发送接口状态

注意:

1. 可选的测试可以由硬件自动完成，也可由外部软件控制

4) 流控

Interlaken 协议的一个重要特性就是每个通道的反压。为了实现此功能，定义了两种规则：带外流控制接口（out-of-band flow control interface）和带内通道（in-band channel）。也就是说流控制机制采用了简单的开-关（on-off）机制来允许某条通道上传输。

每条通道上的开-关流控状态通过单 bite 状态位传递。约定俗称的，1 表示开启(XON)状态，允许数据在此通道上传输。0 表示关闭(XOFF)状态，停止在通道上的数据传输。

协议里面没有信任这个概念。一旦一个通道被指示为开（XON）状态，发送端就可以选择发送任意多数据，直到通道的流控制状态变为关闭（XOFF）状态。接收端决定在开（XON）状态和关（XOFF）状态之间转换的水线设置是可配置变量，取决于支持的通道数目，接收缓存的深度，以及给定环境中的流控制延迟。

流控制通道映射到列表中是可选的，因此流控制可以映射到任何一组访问列表中。比如，它们可以是一对一将通道映射到访问列表；一对多映射来增加特定通道的频率；或者插入空的字段来匹配不同通道的定义。

列表结构也可以用来提供连接层(link-level)的流控制，因为列表中的一个比特表示了接口上的整个数据发送的许可。连接状态与通道状态极性是一致的：1 表示允许发送，0 表示立即停止发送。为使用这个功能，每个访问列表都可以配置为通道(channel)信息或者连接(link)信息。为了降低连接状态延迟，接口需要提供足够的访问列表来编排连接状态到每个突发/空闲控制字的同一个比特位置。比如，采用超过 16 个通道，将按如下安排：

第一个控制字：

访问列表 0 = 连接状态（link status）

访问列表 1 = 通道 0 状态（channel 0 status）

访问列表 2 = 通道 1 状态（channel 1 status）

...等

第二个控制字:

访问列表 15 = 连接状态（link status）

访问列表 16 = 通道状态（channel 15 status）

...等.

采用这种方法，连接状态将总是出现在突发/空闲控制字的比特[55]位置。

带外流控

为了支持系统简单控制需求，定义了一个带外流控制选项。这个由源同步接口实现，并且由下面几个信号确定：

FC_CLK: 流控制数据的同步时钟

FC_DATA: 流控制状态信息（单比特）

FC_SYNC: 同步信号，用于指示流控制列表的开始

这些信号的焊盘可以采用 LVDS 或者 LVCMOS 工艺。这几个信号之间的时序关系如下图：

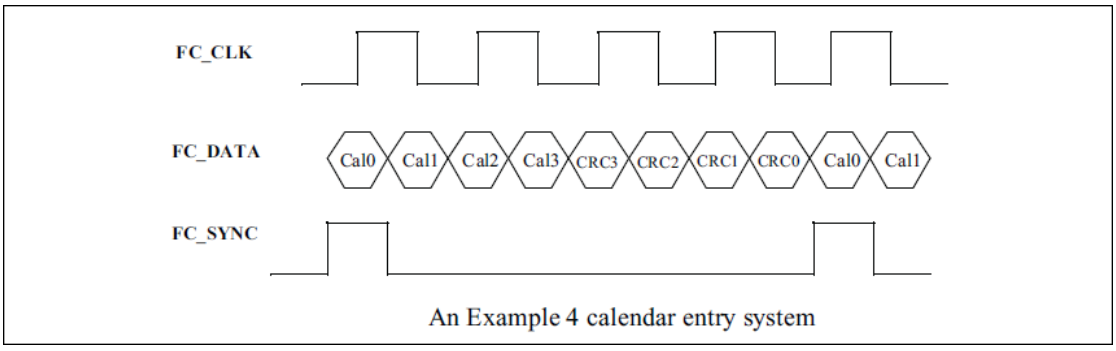


图2-61 带外流控逻辑时序

带外流控通道提供了 4 比特的 CRC 校验保护，它最多可以覆盖 64 比特的流控制数据。根据[3]中建议，CRC4 多项式为：

x4 + x + 1

CRC 计算的细节在附录 B 中。当通道数目为 64 或者更少，CRC4 的校验将会立即出现在最后一个列表时隙后，并且后面紧跟流控制流列表时隙 0。当列表时隙大于 64，CRC4 校验将会在流控制状态中每 64 个比特出现一次。对于最后一组列表时隙，CRC4 校验在最后一个支持的列表时隙后出现，其后再紧跟列表时隙 0。

如上图所示，FC_CLK 时钟的上升沿和下降沿作为 FC_DATA 的时钟。最大速率为 100MHZ 情况下，假设应用中支持 48 个通道和 24Gbps，最坏情况流失(in flight)数据为：

FC_CLKperiod = 10 ns

流失时间 = (10 ns) / (2 比特/周期) * (48 通道 + 4 CRC 比特) = 260 ns

流失数据 = (260 ns) * (24 Gbps) = 780 字节

对于支持 256 个通道和 24Gbps 的应用，最坏情况流失数据为：

流失时间 = (10 ns) / (2 比特/周期) * (256 通道 + 16 CRC4 比特) = 1.36 μ sec

流失数据 = (1.36 μ sec) * (24 Gbps) = 4.08 KB

带外流控制接口时序

这一小节描述了带外流控制接口的交流(AC)时序参数。为了提供最大的捕获窗口，FC_DATA 和 FC_SYNC 信号都以时钟的两倍速率发送，并且与时钟 FC_CLK 有 90 度相移。注意：时序关系还是一样的在时钟的上升沿和下降沿。

Symbol	Parameter	Min	Typ	Max	Units
f_{clk}	FC_CLK Clock Frequency	0		100	MHz
	FC_CLK Clock Duty Cycle	45		55	%
T_{isu}	Input Data/Sync setup time w.r.t clock edge.	0.75			ns
T_{ih}	Input Data/Sync hold time w.r.t clock edge.	0.75			ns
T_{on}	Next Output Data/Sync Invalid time w.r.t previous quadrature point of next clock edge.			0.75	ns
T_{op}	Previous Output Data/Sync Invalid time w.r.t. quadrature point of previous clock edge.			0.75	ns

带内流控制(In-Band Flow Control)

当采用这个功能时候，接收器可以将流控制状态放在控制字中当成正常的数据传送。这个功能需要由全双工应用提供，并且需要的外部信号管脚数量也是最少的。

如之前的图 7 中所示，流控制字段在控制字中为 16 个比特，位于比特位[55:40]。此外比特位[31:24]也可以为流控制提供额外的 8 个比特，这样一共就有 24 个可用比特作流控制。这些状态比特位代表每个 Interlaken 列表通道中的开-关

(ON-OFF)状态，其中的比特[55]为当前的列表 X 入口,比特[54]为列表 X＋1 入口，以此类推。为了同步列表的开始，在空闲/突发控制字中提供了一个比特来复位列表。当这个位设置位'1'，则列表入口 0 的状态出现在比特[55]；当这位设置为'0'，则列表将继续先前离开的控制字。一旦所有的通道状态被接收，发送端置位复位列表比特位后序列开始重复。最后一个列表的控制字中不需要的比特位（如当通道的数目不是状态位比特数目的整数倍）将被发送端设置为 0，并且接收端将忽略这些 bite。

由于控制字的 CRC24 包括流控制字段，因此没有必要再进行单独的错误检测，而且带外流控选项中的 CRC4 计算也不在此涉及。

流控制信息通常在空闲和突发控制字中都传送。因为控制字是在突发数据中传送，所以最差的情况下，流控制信息在每一个很大长度的突发中传递一次。在实际应用中为了流控制带宽的需要应该选择合适突发长度(BurstMax)。举一个性能计算的例子，对一个突发长度为 256 字节和 48 个通道的接口，在列表被传送过程中流失(the data in flight)的数据为：

流失数据 = (2 突发) * (256 字节/突发) +(2 控制字) * (8 字节/控制字) = 528 字节

完整包的流控制

因为 Interlaken 的优势是通过交替的在不同通道上传送数据，它也为提供了需要完整包传送的应用。在这些应用中，发送端设备应该仅仅需要避免数据发送从一个通道切换到另外一个通道即可，一直到当前通道的包被发送完成为止。

完整包的发送模式下由两种流量控制：一种是收到 XOFF 信息以后立即停止发送数据，另外一种是完成当前的包发送再停止。第一种方式的好处在于可以较少接收端在响应流量控制之前的缓存要求，不过代价是其队首将阻塞接口的别的通道；第二种方式则刚好相反。因为不同的应用有不同的需求，因此规范的两种流控制方式由设计者自行选择，可以为其中一种，也可以是两种都有。

流控制扩展

在一些应用场合可能会希望利用 Interlaken 协议提供的开-关(XON/XOFF)来实现不同的流量控制方式。比如它们可能需要在发送端和接收端直接交互信任。Interlaken 并不是直接去规范它，而是将其作为高层协议功能而用额外的通道来传递这些信息。通过将这些扩展信息作为数据净荷的一部分的方式，任何高层的协议都可以提取并可靠地在 Interlaken 接口种传输。

6. 帧层

1) 引言

Interlaken 协议为帧层定义了多种功能来实现简单、可靠的传输，这些功能包括：

表2-13 帧层功能总览

功能	目的

64B/67B 编码	区分 8-byte 字边界 区分控制和数据字
同步扰码	保证 bit 传输密度 去除错误乘法
链路对齐	对齐所有链路在一束
诊断	提供诊断和可选的每条链路状态信息
跳字	在电中继器补偿时钟差异
速率匹配	通过 Interlaken 与下游业务的数据速率匹配来优化接收器设计

上表中所有的功能都在接下来的小节中说明。帧层使用的控制字可以参考 16 页中的表 7。（控制字）中的[63:58]位用于区分这种类型的控制字，其中[63]比特位需置为 0 而[62:58]用于区分这个块的类型（同步字、扰码态、跳字或者诊断字）。

2) 编码

在串行接口中为了描述字节边界，提供电路传输过程程中产生的 EMI 的随机化处理，时钟恢复和维持直流均衡，需要采用编码和扰码技术。在 Interlaken 协议中对 IEEE 802.3ae 10G 以太网规范中的 64B/66B 进行了修改后将其作为其编码方案。

现有的802.3 64B/66B编码通过对64-bit的数据或者控制字净荷扰码以及在前面添加 2 个附加的未进行扰码的 bit 的方式来解决字节边界确认的问题。如果这两个同步比特为"01"，表明这是个数据字；如果是"10"，则表明这是个控制字。这两个比特的另外两种组合"00"和"11"没有采用。接收端搜索接收数据流中的同步字，一旦接收到 64 个正确的匹配就可以对字节边界锁定，并且持续搜索这两个比特来维持锁定状态。

这种方式的缺点在于会有极大的基线漂移。基线漂移和 DC 不均衡是由于在独立的 SerDes 链路上多于的 1 或者 0 的传输累积引起的。由于电信号传输过程是与时间常量相关的，因此在高速接口中通常不允许在下一个比特传输前的全电压摆动。因此持续的 1 或者 0 的数目的不均衡将导致差分对的中心电压在眼图中位置的移动。通过分析 64B/66B 扰码多项式可知，在 64K 的数据传送中将超过+/-1000 个比特的不匹配，这将导致眼图的极大移位，使整个接受电路的设计变得复杂，而且会增加比特错误的几率。

为了有效限制此问题，Interlaken 协议中将判断字在么个传输字中反转，使传送的不匹配限制在+/-96 个 bit 范围内。每条链路都维持一个运行不匹配数目：每个 1 增加 1 个不匹配，每个 0 减少一个不匹配。在传传送每个新字之前，其不匹配数目将进行计算，并与当前传送字的不匹配数目进行比较。如果它们都是同号(同正或者负)，则将新字中的比特进行反转。接收端通过接收到帧字的[66]位置的 bit 来判断该字是否需要进行反转，定义如下：

表2-14 Bit 66

Bit 66	含义
0	比特[63:0]没有反转，接收端可以不对该字进行修改
1	比特[63:0]反转，接收端必须在处理前对该字进行修改

这种编码在此文档中被称为 64B/67B 编码。每个字中包括比特 66 在内的所有比特都将包括在运行不匹配数目计算中。当比特 66 置为 1，比特[63:0]将会反转。合法的 3 比特同步字（bit[66:64]）取值如下：

表2-15 同步字编码

Bits[66:64]	含义
001	数据字，未反转
010	控制字，未反转
101	数据字，比特[63:0]反转
110	控制字，比特[63:0]反转
All others	非法

IEEE 的 64B/66B 编码定义了一种锁定同步比特的过程。接收端搜索高到低或者低到高的跳变（唯一的合法同步编码），并且将其作为假定的同步图案，接收器在下一个帧比特位置搜索同样的合法图案。如果搜索到同样的合法图案，则重复这个过程；如果没有，则复位搜索状态搜索另一个合法的图案。只有再连续检测到 64 个正确的同步图案以后才能声明进入锁定状态。

64B/67B 开销为 4.5%。

对于 64B/67B 编码，Interlaken 协议增加了一个同步比特（bit66），但是这三个同步比特与 64B/66B 编码一样，只有 50%的组合是合法的。因此为了确保锁定与之（64B/66B）具有相同低的概率错误，接收器也必须保证 64 个合法的同步图案（表 5 中定义）被检测到。

64B/67B 编码将会产生 4.5%（3bit/67bit）的冗余。

图中为 64B/67B 编码实现和维持字节边界锁定的过程:

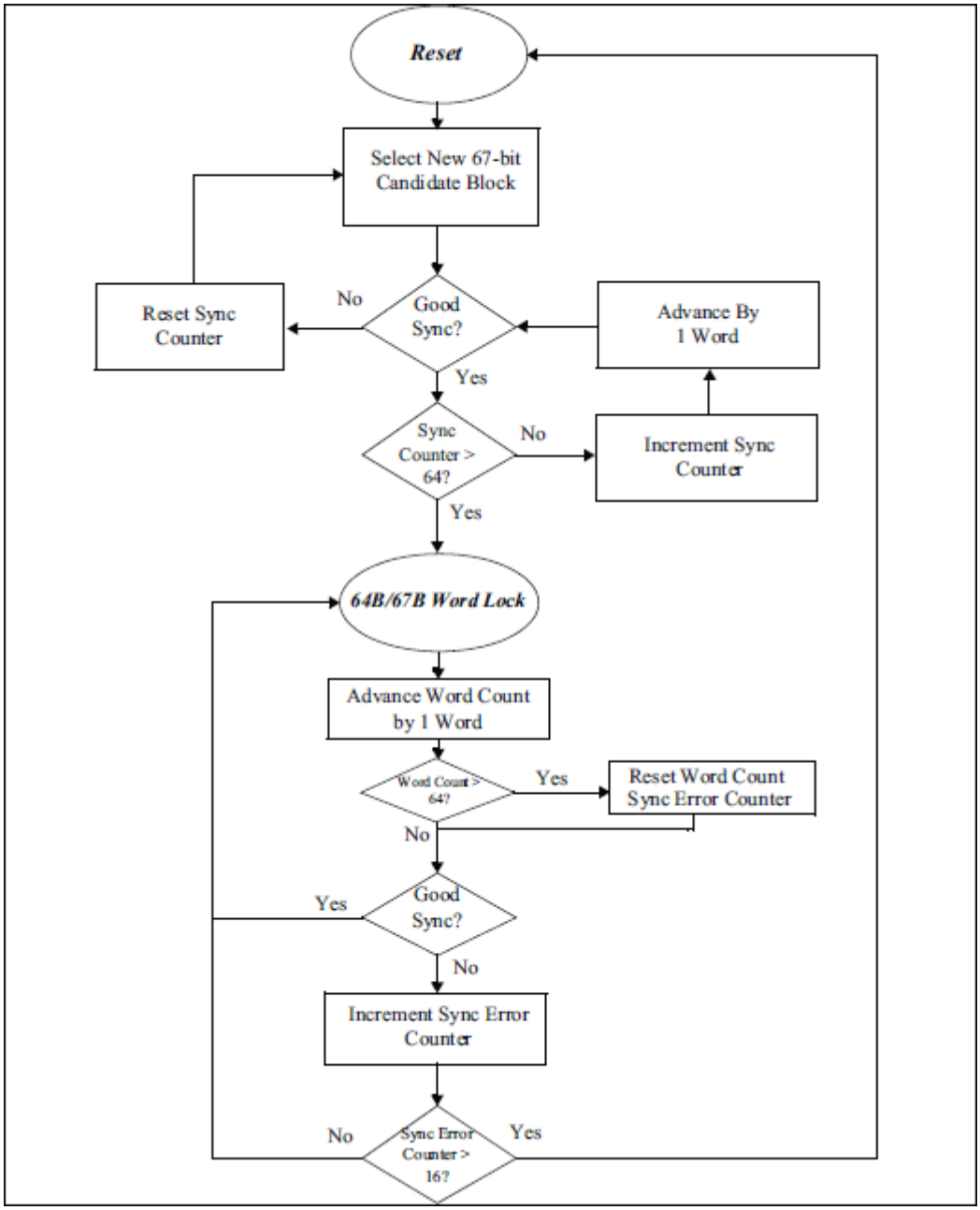


图2-62 64B/67B 字边界锁定

3) 元帧（Meta Frame）

Interlaken 帧方式引入了元帧（Meta Frame）的概念。元帧定义为每条链路上携带净荷（突发数据或者控制信息）的同步、扰码状态、跳字以及诊断字。图中说明了这种结构：

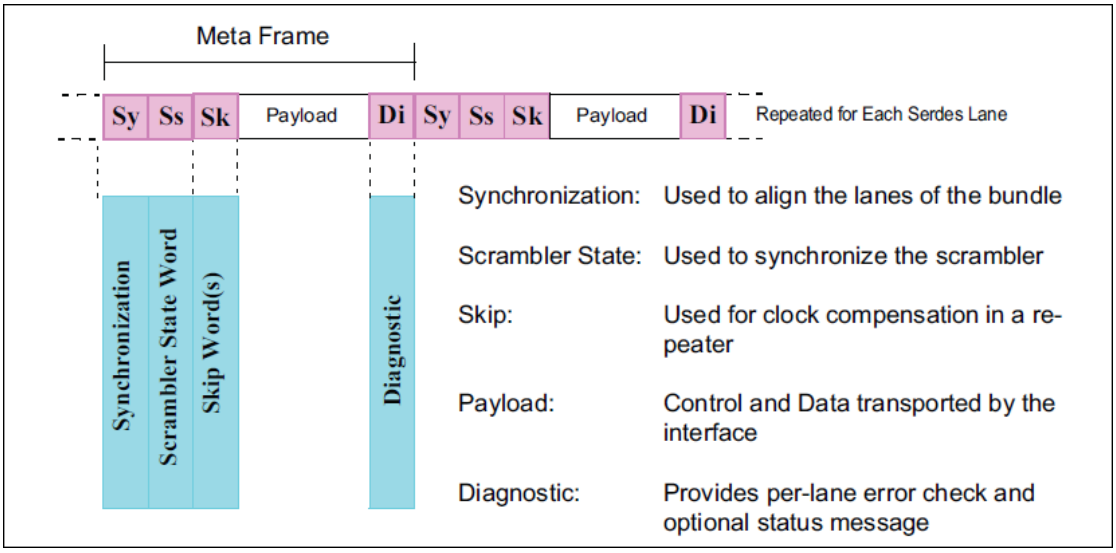


图2-63 元帧结构（单链路）

Sy（Synchronization）－同步字：用于链路同步及对齐

SS（Scrambler State）－扰码字：用于同步扰码状态

Sk（Skip－跳字：用于 repeater 的时钟补偿

Payload －净荷：Burst 控制字/IDLE 控制字和数据字

Di (Diagnostic)－诊断字：用于链路的错误诊断及传输链路状态信息。

元帧长度 MetaFrameLength 是一个单独可配置参数，应用于所有链路。这个值是净荷（payload）以及一套同步字、扰码字、跳字和诊断字的长度总和。元帧结构与数据传送为正交结构，这些元帧控制字可以出现在一个数据突发的任何位置。

除同步字、扰码字以及诊断字以外，跳字被定义为做 repeater 的时钟补偿。对于元帧控制字，唯一的块类型编码在表中说明：

表2-16 元帧控制字块类型

元帧控制字	块类型（正）	块类型（负）
Synchronization	011110	100001
Scrambler State	001010	110101
Skip	000111	111000
Diagnostic	011001	100110

详细的同步字、扰码状态字、诊断字和跳字将在接下来的小节中详细阐述

4) 同步扰码器（Synchronous Scrambler）

在 802.3 64B/66B 编码中，净荷部分采用了自同步扰码器。这样做的好处在于步需要任何的同步操作，而且扰码器的状态是接收数据流的函数，可以通过接收到整个扰码数据（58bits）以后进行恢复。但是这种扰码器采用 2 拍反馈，同

样的会把错误复制两次。因此，线路上的单 bit 错误将在接收端变成 3 比特错误。由于 Interlaken 将数据条状的分布到各条链路上，所以这种增加（错误 bit）会将推动错误 bit 跨越字边界。这样下个错误字可能不会出现在同一个突发里面，这就意味着错误定位将不在限制在同一个突发里面。对于多比特错误，这将降低 CRC24 的错误检测性能并且完全是非设计所需产物。

为消除此种情况，Interlaken 协议在接口的每条链路上采用了独立的同步扰码器。这种同步扰码器不在输入数据上反馈自身，而是将每个比特都与当前扰码器的状态进行异或操作，这样就没有任何错误被复制。扰码器多项式如下：

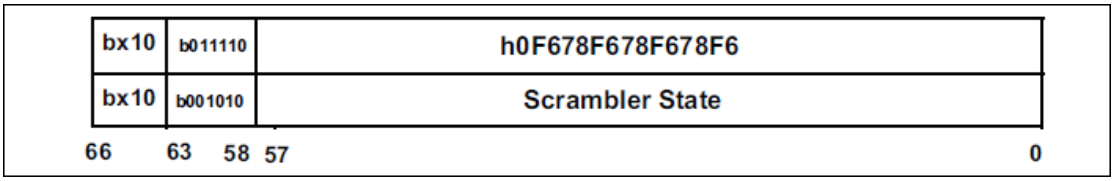
$$x58 + x39 + 1$$

扰码多项式在器件复位以后被激活，并且不再复位。同时当前的扰码器状态将被传送到接收端，以便恢复后面的数据。扰码器在接口运行中不确定的超前滚动，只有在传送未扰码同步字和扰码状态字的时候不会超前。

对每条链路并不要求采用相同的扰码状态字，而且为了减少链路之间的串扰，应该在各条链路复位时候设置不同的值。唯一的限制就是扰码器状态字不能设置为全 0，因为扰码状态字将在数据通路中明确的传送，接收端没有必要知道发送端扰码器的初时状态是何值。

为了正确的对接收数据解码，接收器必须要与扰码多项式状态同步。Interlaken 中采用元帧中连续传送的而且唯一的 64 比特的同步字和扰码状态字来进行同步：

表2-17 同步字和扰码状态字



为了在开始和出现错误以后能够同步，同步字和扰码状态字在传送的时候是未经扰码的。在复位状态，每条链路分别搜索唯一的同步字图案。当接收器接收到一个同步字（匹配所有的 64 比特）后，就开始计数，直到接收到 MetaFrameLength（以 8 byte 为一个字计算）个数据再开始检测下一个同步字。检测到同步字后再次重复此过程，直到检测到 4 个连续的同步字。检测状态如图所示：

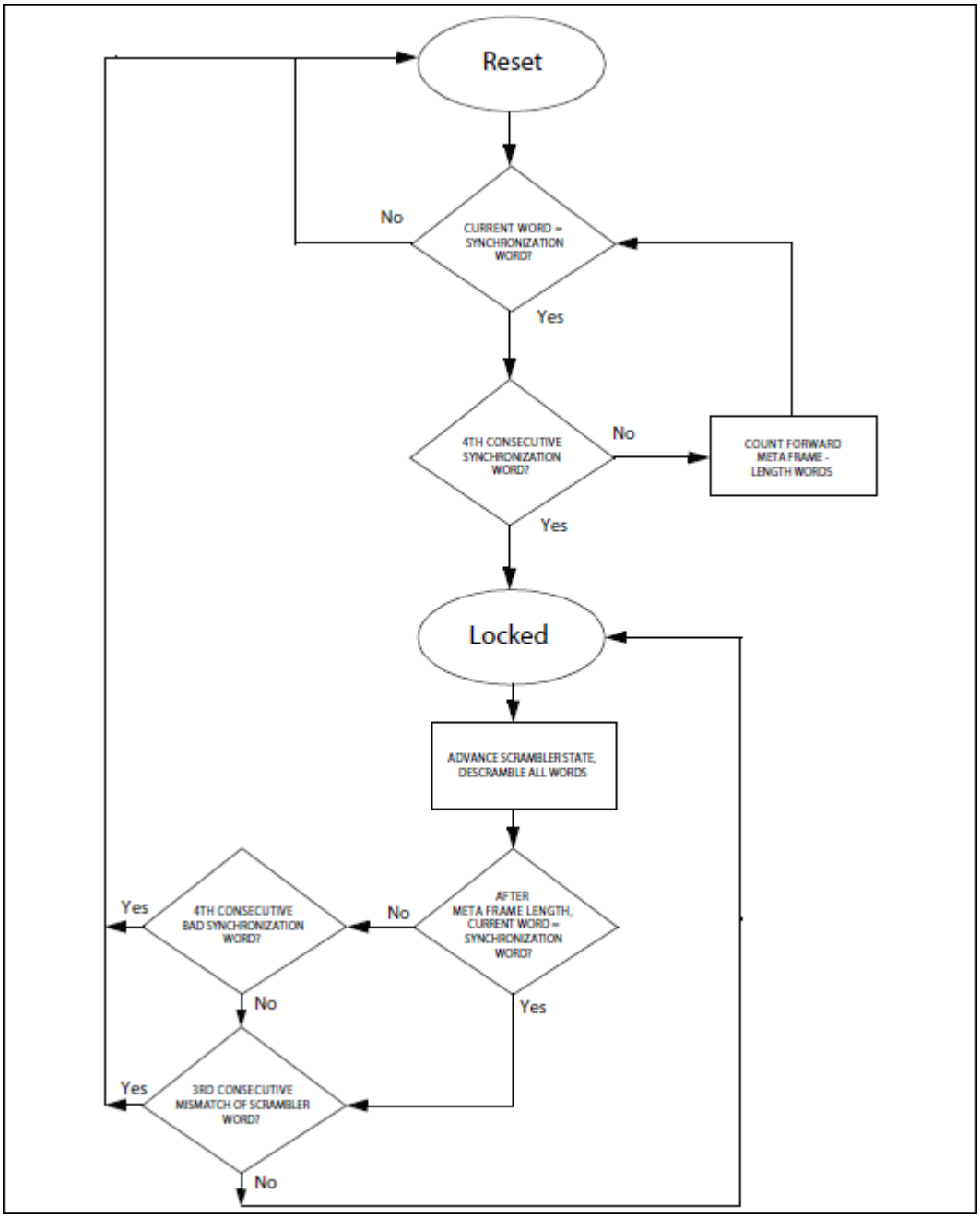


图2-64 扰码同步状态机

一旦达到同步状态，接收器就将从扰码状态字中恢复出来的扰码多项式作为种子传递给解码器。除同步字和扰码状态字以外，所有的数据和控制字都是从[63:0]做扰码处理；帧指示字[66:64]从不做扰码。每条链路都需要校验从接收到的扰码字中提取的扰码器状态是否与当前期望的扰码器状态一致，如果连续 3 次不一致，则给出错误信号，错误扰码器状态。

元帧的长度通常都是 MetaFrameLength，因为 Interlaken 提供了可添加和移除的跳字（Skip word）来对 repeater 的时钟补偿进行管理。Repeater 也可能需要调整同步字的相对于它最初发送时候的位置。这种调节经常出现，但是接收器再两个同步字之间会接收到相同数量的数据。

如果没有搜索到同步字，那么接收器将声明一个错误信号。如果连续 4 个同步字无法识别，接收器将返回复位状态并重新开始搜索同步字。如果出现连续 3 个扰码器状态与期望状态不匹配（都再一条链路上），那么接收器将声明错误并且开始重新同步扰码器.

5) 链路对齐

一旦字节边界确认并且扰码器被合适的复位，组里的各条链路就必须对齐。Interlaken 确保同步字以一个固定的频率发送到接口上，使 SerDes 接口能够有规律的对齐。为了达到对齐，同步字在所有的链路上同时发送。在接收端识别这些字，测量链路之间的偏差（skew），在内部通过补偿电路进行调节。具体的实现逻辑在取决于实现，Interlaken 只是定义了实现对齐的方法。

同步字的传送频率由 MetaFrameLength 决定。

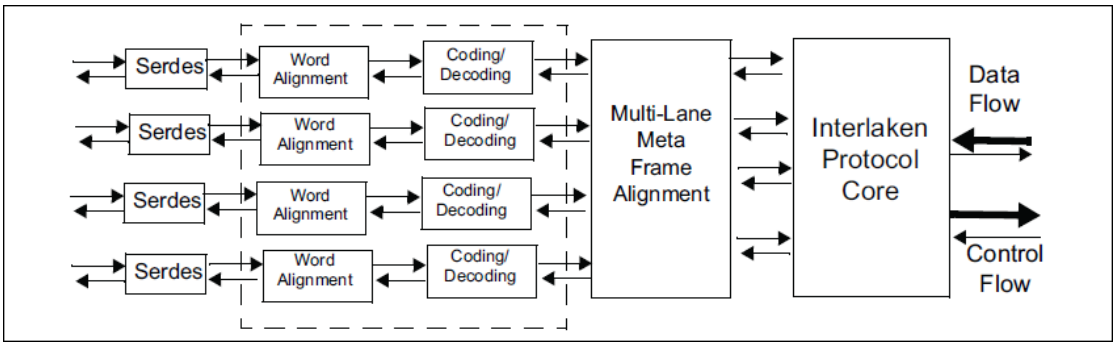


图2-65 Interlaken 链路对齐 (以 4 条链路为例)

6) 链路诊断字

链路诊断字用块类型中的 0b011001 来识别，具体格式如下图所示：

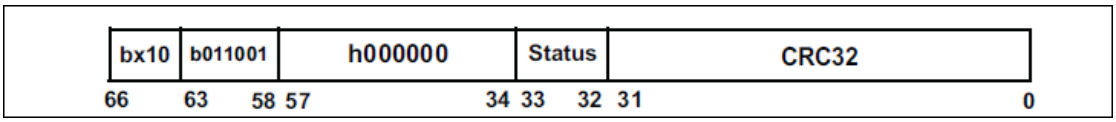


图2-66 诊断字

诊断字有两个功能：提供链路信息和每条链路的错误检测。其中的 2 比特状态字段定义了从接收器给发送器的单条链路信息，具体功能定义在附录 A 中说明。CRC32 字段是每条链路诊断的工具，使得接口出现的错误可以定位到单条链路上。在扰码和反转之前，CRC32 将计算元帧内所有字的除去 64B/67B 分割位（framing bits）的部分，包括诊断字自身的[63:0]部分（将 CRC32 部分填充位 0 计算）。为方便实现，在 CRC32 计算时可以将扰码状态字中的 58 比特扰码状态当成全 0 计算。CRC32 多项式如下：

x32+x28+x27+x26+x25+x23+x22+x20+x19+x18+x14+x13+x11+x10+x9+x8+x6+1

为了易于计算，CRC32 计算的字段如图所示：

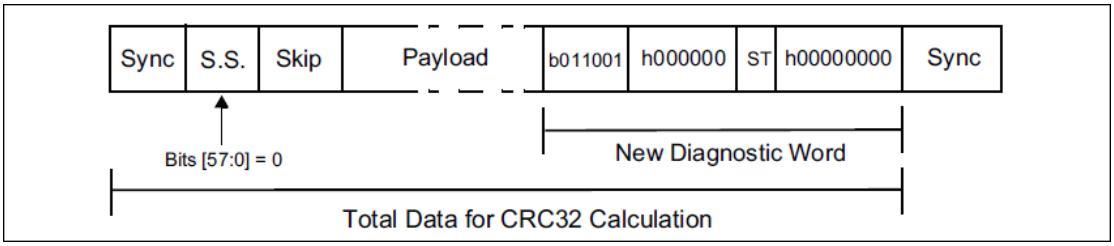


图2-67 CRC32 计算说明

诊断字与同步字、扰码状态字、跳字、数据和突发/空闲控制字一样，算作 MetaFrameLength 的一部分。

7) 时钟补偿

跳字的目的是提供给中继器（repeater）使之能够实现时钟补偿，因而协议能够在电路传播上实现中继。在每个中继器侧，时钟都会有微小的差异；为了弥补这个差异，如果第二个时钟比第一个时钟慢，则移除一个跳字，相对的如果第二个时钟比第一个时钟快，则增加一个跳字。单一的跳字为元帧中所必须的，但是对于额外增加的跳字则可以添加到元帧中除诊断字、同步字和扰码状态字之间的任意位置。接收器必须能够正确地从接收到的数据中识别和移除这些添加的跳字。

如果在源端和接收端存在中继器，如果转发时钟较慢，则可以自行丢弃这些跳字。如果出现这种情况，中继器必须维持两个同步字之间的恒定间隔。为实现此要求，可以将下一个元帧中的第一个净荷字移到当前的元帧中，并且在本元帧结束时候用正确的扰码状态对齐进行扰码。图说明了这个过程：

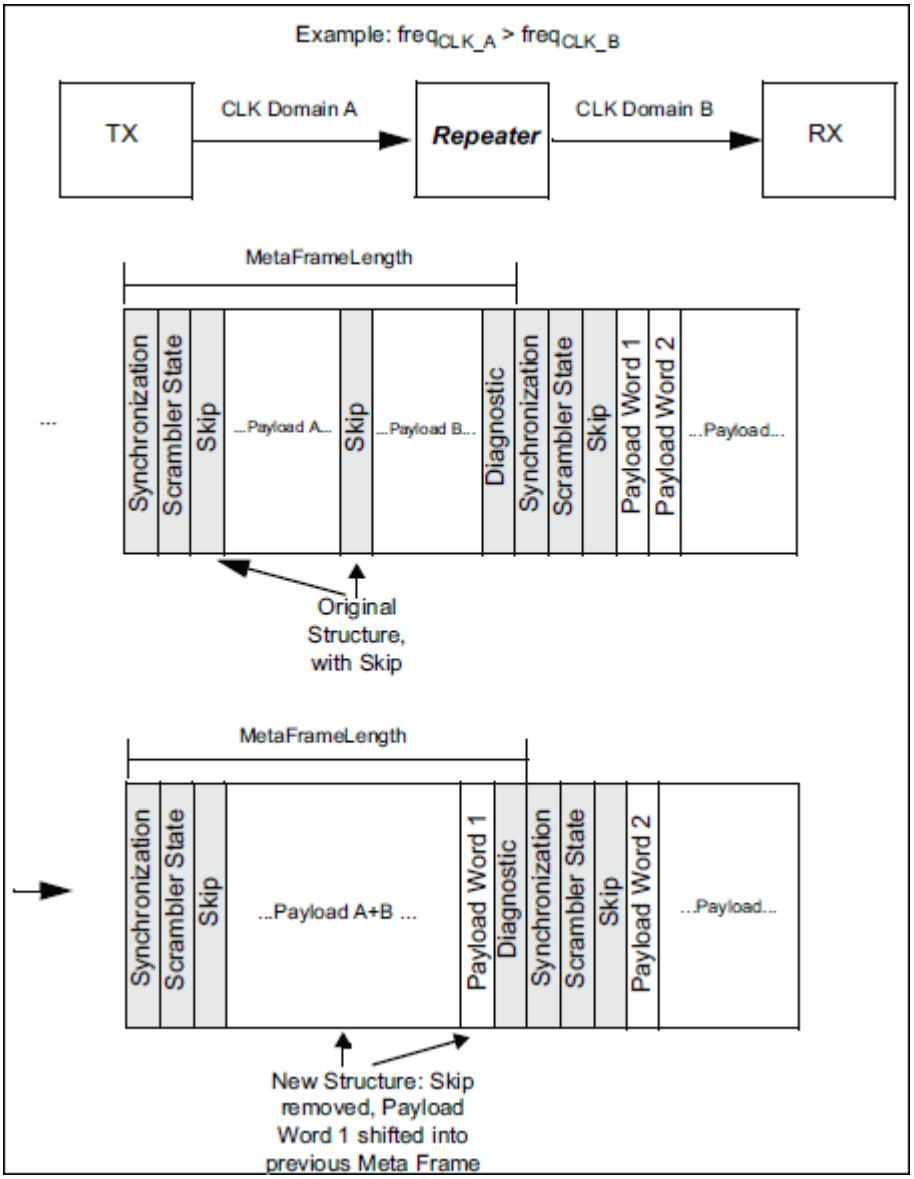


图2-68 时钟补偿过程

要知道删除一个跳字荷将净荷字从一个元帧转移到前一个元帧中将会导致诊断字、扰码状态字和同步字从一个元帧移位到前一个元帧中。为了维持元帧格式的统一，这种情况下诊断字、同步字、扰码状态字都必须删除。如果诊断字被删除，那么中继器应该通过低通滤波器发送状态信息来避免消除任何短期的状态信息。具体细节由实现决定。

如果上面例子中的 CLK_B 比 CLK_A 快，相反的就必须在当前元帧中插入一个跳字，而且净荷中的最后一个字将移到下一个元帧中去。最终这个过程需要一个诊断字被添加，在这种情况下，状态信息应该与紧跟着的前一个状态信息一致。

如果中继器发现某些链路存在时钟差异需要丢弃跳字，那么它应该在所有链路上都同时丢弃而不是仅仅在由差异的链路上。对于 MetaFrameLength 为 2k 的字，每 16KB 数据可以丢弃 16 个 bytes，换句话说为 1：1024。而 100ppm 的

时钟频率差异相当于 1：10000，因此这个元帧的频率满足时钟补偿的要求。需知 MetaFrameLength 可以设置更短来满足更快的链路对齐或者更少的诊断 CRC 计算数据。

跳字用快状态类型的值 0b000111 来区分，跳字格式如下图所示：

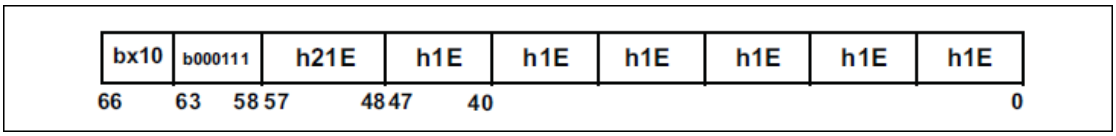


图2-69 跳字格式

8) 开销

因为同步字、诊断字和扰码状态字被传送，因此极少情况下它们消耗最小的接口带宽。开销取决于 MetaFrameLength 的大小，假设 2k 的字，在最差的情况下开销（只有一个跳字）为：

32/(16,384) = 0.20%

9) 偏差预算（Skew Budget）

Interlaken 规定最差情况下链路之间的的偏差可以为 107 单位间隔（UI:unit intervals），见下表：

表2-18 Skew Budget

Skew Source Budget (UI)	Budget(UI)
PMA Tx	67
PCB and Medium	40
Total	107

这些预算是从下面这些观测所得：

PMA Tx： SerDes 在驱动每条数据通路的时候并去要求共享发送锁相环（PLL），因此发送器输出由于采用不同的 PLL 导致 SerDes 的两个或者多个块出现偏斜。在保守的、最差的情况下是采用 67 比特位宽接口到 SerDes 链路，这时候采用不同 PLL 的 SerDes 的两个块之间的最大偏差将是 67 比特，或者 67UI 在每条串行链路上。

PCB & Medium：Interlaken 的应用环境与 802.3ae-2002 定义的 XAUI 应用环境类似，类似的这个参数也与 SerDes 的速率成比例。Interlaken 中采用的 SerDes 接口最高支持 6.375Gbps 速率，是 XAUI 速率的两倍，因此偏斜要求也是 XAUI 的要求 20UI 的 2 倍，即为 40UI。

Interlaken 控制器也必须为接收端 PMA 产生的偏差做补偿，但是不必指定这个值（偏差）来满足协同工作，需要具体的实现来计算和修正这个偏差。

10) 速率匹配

在一些应用中希望在 Interlaken 协议和一些现有协议如 SPI4.2 之间转换。对于这些应用，两个接口的带宽可能不匹配，那么桥接设备将会花费昂贵的缓冲功能来实现。

另外，如果数据速率能够保证限制在最大的可实现速率范围内，那么接收端缓冲容量就可以降低。为了提供这种优化，Interlaken 强制定义了速率匹配功能。

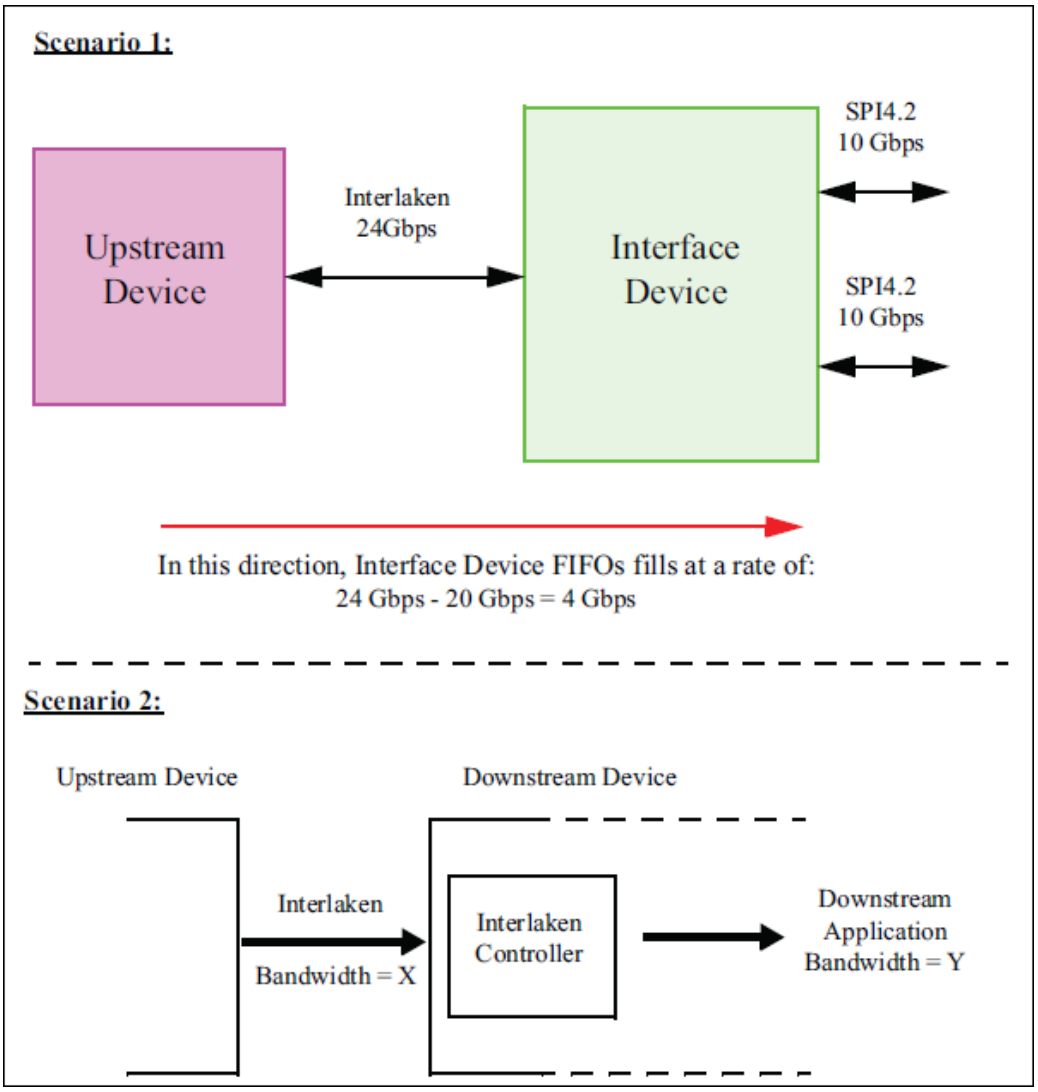


图2-70 速率匹配情景

Interlaken 通过在数据通路中以一个确定的频率插入空闲控制字的方式来限制接口数据发送带宽，来实现速率匹配。

速率匹配控制的是整个接口的流量，而不是针对单个通道。在具体实现中需要设计为单标记漏斗形式，以设定的速率的增加，而在数据传送以后减少。如果标记漏洞为空，则开始发送空闲字，一直到正的标记有效为止。由于空闲字必须在突发数据之间传送，因此在最差情况下，速率匹配逻辑将会有 BurstMax 的延迟。标记漏斗的间隔是可以配置的值，最小可以为 1 个字节，因此它可以匹配两个采用这种接口的内部数据通道的间隔的设备。

在速率匹配功能中提供了两个参数，见表：

表2-19 速率匹配参数

参数	功能
RateLimit	总的接口速率限制（字节/秒）
BurstLimit	调用速率限制逻辑之前在接口上传送突发数据的最大量（字节）

11) 错误情况

错误情况即可应用与整个接口也可应用与单个突发。Interlaken 包含了丰富的错误检测逻辑显著提高了像 SPI4.2 协议提供的错误保护。由于不同的应用要求不同的错误相应机制，因此硬性规定哪种错误应该如果处理已经超越了协议的范畴。不过在接下来的小节中将给出所有可能出现的错误类型并给出处理建议。参考对象如图所示：

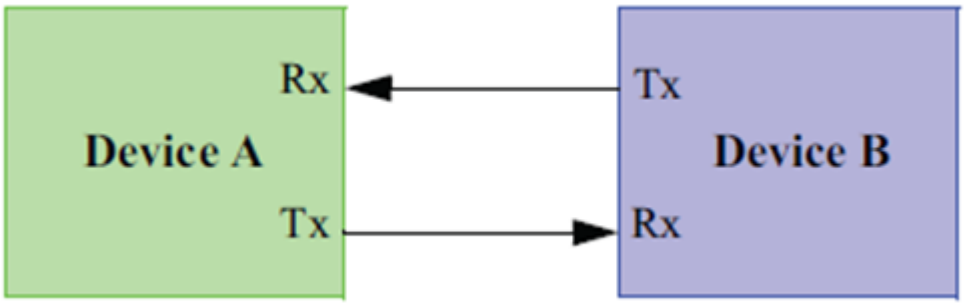


图2-71 设备参考

- 接收 SerDes 失锁

如果 A 设备的一条 SerDes 数据通路在恢复时钟上失去锁定，那么 A 设备将不能够再正确的从 B 设备接收数据或者突发控制信息。因为突发数据在各条链路上是条形分布的，如果其中一条链路损坏，那么突发通信中丢失部分将是随机的。在此情况下，A 设备将会在所有通道上对打开包进行错误标记。A 设备应该在所有的流控制状态通道向 B 设备发送 XOFF 信号来让 B 设备停止发送数据。B 设备一旦接收到该流控制状态信号，通常情况下就会开始发送空闲字。A 设备也可以选择发送同步信息指示接收失败（定义见附录 A）。

A 设备立即开始尝试获取 SerDes 时钟，一旦它获取，还需要字节对齐。对于数据丢失的数量取决于时钟恢复的时间和总的接口带宽。

一旦 A 设备在 SerDes 上重新获取时钟并且正确识别 64 个字边界，为表明已经准备好，就重新广播 XON 信号在它的所有流控制通道。B 设备随即重新开始正常发送数据。

- 接收端失去字节边界

接收 SerDes 如果不能从接受到字数据的[66:64]正确识别到合法的 3 比特同步图案，那么它将失去字边界同步。接收端在 64 字的窗口内如果不能找到 16 个正确的同步图案，它将声明自身失锁；此时只有在连续接收到 64 个合法的同步图案以后才能重新进入锁定状态。具体可以参考图 13 中字边界同步状态机转移算法。

- 错误扰码状态

在最初的同步以后，接收端扰码器的状态很可能与期望的当前扰码器状态不同。但是一旦扰码器同步以后，它就不会再变为非同步状态，因此这种错误应该只会出现在扰码器状态字内部出现比特错误或者接口出现对齐错误。只有连续出现 3 个扰码器状态不匹配后，接收端才会声明错误并且尝试重新同步扰码器。

- 链路对齐失败

如果在规定的时间间隔（107UI）中，SerDes 接口逻辑不能从所有的数据通道中找到同步字，那么接口将备认为是未对齐的。如果接口失去对齐，将无法可靠的确认正确的数据序列。这种情况下，接收端应该对所有开启的包标记错误，并且尝试在下一个可用的元帧中对齐。

为了避免单一的同步字错误破获整个对齐，接收端只有在连续出现 4 个对齐错误的时候才会声明失去对齐。为了重新获得对齐，接收端也需要重新接收到 4 个正确的连续的对齐。

- 突发 CRC24 错误

错误发生在检测到突发或者控制字的 CRC 错误不匹配。CRC24 覆盖了之前的所有的突发和空闲控制数据的[63:24]位。包中的最后一个字，在包长不是 8 的倍数的情况下为无效字节，为了易于计算，可以在计算 CRC 的时候把它们都设置为 0。基于同样的原因，控制字的 CRC24 字段也被当成全 0 来计算，当然它们也包含在 CRC24 的计算中。

CRC 错误指示了在当前数据或者控制信息中出现了错误。因为突发/空闲控制字包含了链路编号字段，因此不可能识别接下来的数据突发通道。因此一旦检测到 CRC 错误，将标记所有打开的通道。

- 流控制错误

由于流控制信息速率很高，唯一的有丢失问题的地方是流控制状态传输时的延迟。如果出现错误，接收端应该能够保存并且假设所有通道处于 XOFF 状态，一直到下一个列表复位和错误解除信息。

- 未知控制字类型

I 接口如果接收到一个非定义的控制字（如控制字段不是为"1"或者不是定义的块类型之一）或者位于一个错误的位置，将会声明一个错误。这种情况可能发生在帧类新字段突变为"10"，接口失去字同步，或者块类型字段错误。这些控制字应该被丢弃，并且因为它们可能是一个出现错误的控制字，保守的响应是错误标记所有打开的通道。

- 错误的 64B/67B 编码字

如果接口接收到帧字段为错误图形的 64B/67B 编码字，它将丢弃该字并且错误标记所有打开的链路。这种情况发生在数据字或者控制字的帧字段出现错误。

- 诊断 CRC32 错误

CRC32 是最先用于追踪错误发生在哪条特定链路的快速定位错误的工具。另外，接口也可以利用 CRC 错误检测来实时指示链路的错误并且将它们从当前的服务中移除。具体实现不在规范以内。

7. 电气规范

Interlaken 制定为一个多链路、全双工的接口，在每个终端采用了差分对与 SerDes 电路相连。由于 8 字节块编码字是按照条状分布到各条链路，因此在应用中采用多少条链路没有具体要求。这个协议可以分派 1 个到任意数目的链路给一颗芯片。协议的吞吐量与每条 SerDes 链路的速率比例相关。Interlaken 联盟已经发布了协同指导书，其中就包含了电气规范和相关建议。

8. 推荐统计量

在表中这些接口统计变量仅仅被定义为推荐所用。协议不要求这些在应用中与规范一致。

表2-20 统计值

统计值	功能
RX_Bad_Packets	错误包数量（如 CRC 错，FIFO 溢出，错误比特设置等等；每链路）
RX_FIFO_Overflow	因为接收 FIFO 溢出而丢弃的包数目（每链路）
RX_CRC_Error	CRC 错误的突发个数
RX_FC_Error	带外流控错误数目
RX_BurstMax_Erro	突发长度超过最大长度 BurstMax 的个数
RX_Alignment_Error	接收端对齐错误次数（如违背当前对齐）
RX_Alignment_Failure	对齐丢失次数（在 4 次对齐错误 RX_Alignment_Error 以后）
RX_Word_Sync_Error	链路丢失字边界同步次数（每链路）
RX_CDR_Error	链路丢失时钟－数据恢复次数（每链路）
RX_Lane_CRC_Error	链路 CRC 错误次数（每链路）
RX_Bad_Control_Error	接收到的控制字帧字段（‘x10’）与规定任何控制字帧字段不符合次数。

9. 延迟考虑

在设计接收逻辑的缓冲容量的时候必须要考虑流控制响应的延迟。下面的这些参数作为指导来帮助评估，而它们具体的值由设计决定。

为了更好的说明，定义了一个抽象的 buffer

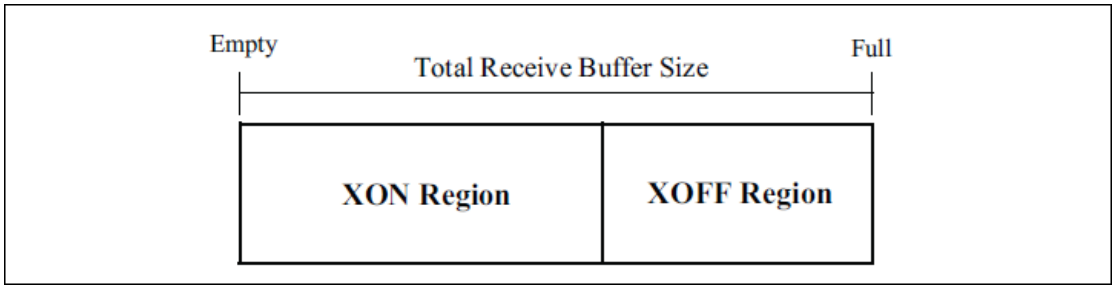


图2-72 延迟说明

表2-21 延迟参数

参数	符号	描述
状态转换延迟（Status Change Latency）	tSCL	检测到 XON 到 XOFF 后建立新的状态信息延迟，相反方向亦是
状态传送延迟（Status Transmit Latency）	tSTL	状态信息从接收端发送到发送端的延迟
传送控制延迟（Transmitter Control Latency）	tTCL	发送器处理新状态信息延迟
传送管道延迟（Transmitter Pipeline Latency）	tTPL	数据已经存在发送器管道导致延迟（从 XON->XOFF），或者从发送管段推出数据延迟（从 XOFF->XON）

打开和关闭时间是下面 4 步的函数：

1）接收器检测到接收 buffer 区域的变化和产生新状态的时间。

2）新状态从接收器发送到发送器的时间。

3）发送器处理新状态和调整它时序的时间。

4）数据进入发送器流水线管道处理到准备好的时间。

总的流控制延迟必须考虑上面所有的情况：

tL = tSCL + tSTL + tTCL + tTPL

因此，接收端缓冲器的大小是接口速率，流控制带宽，内部 pipeline 结构大小以及这个延迟时间的函数。

10. 性能

接口性能可以理解为传送净荷的带宽占总带宽的百分比。对于 Interlaken，常见类型数据性能如下表所示：

表2-22 性能分析

链路配置	POS 41 字节帧	Ethernet, 65 字节帧	Ethernet, 9601 字节帧
效率因子	69.8	77.5	92.3

其中：

效率因子 = （编码效率）×（成帧效率）×（对齐效率）×（元帧效率）×100%

编码效率 ： 采用 64B/67B 的情况下编码效率为 95.5%

成帧效率 ： 帧或者单元的 8 字节控制字开销影响（以 256 字节作为 BurstMax 为例）

对齐效率 ： 为对齐 8 字节边界而在帧尾田间的无用字符的影响

元帧效率 ： 对于包含同步字、扰码状态字、诊断字和跳字的元帧效率为 99.8％(假设元帧长度 MetaFrameLength 为 2K 字长，并且步考虑速率匹配而插入空闲控制字的情况).

11. Interlaken 整体实现框图

分为独立的 TX 和 RX 两个部分，LBUS 为应用侧接口，通常与芯片的内部数据通路相连接，通过并行总线接口与 Serdes 相连，提供丰富的配置、状态、告警上报信号，供 CPU 查询。

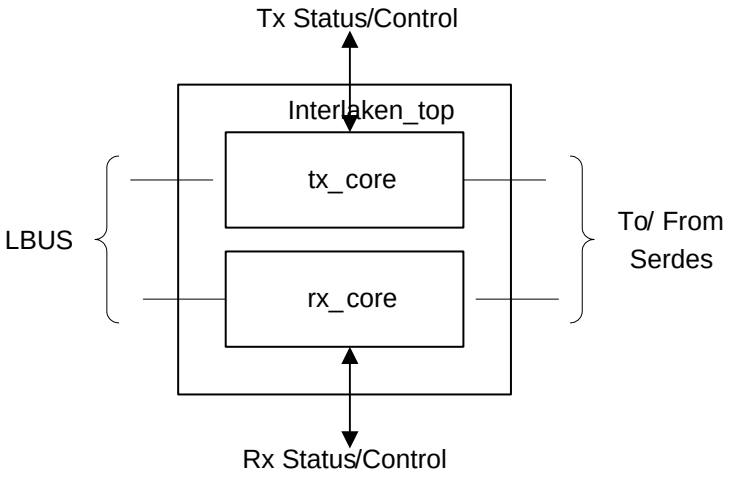


图2-73 Interlaken diagram

2.3.3 SerDes 接口

随着互连网的飞速发展，用户对带宽的需求每五个月就要翻番，这就要求对信息处理、存储和传输的设备必须跟上带宽发展的需求；这就要求芯片之间交换信息的带宽也越来越大。

由于封装的限制和总线间干扰的制约，不可能再象以前那样通过无限制地增加并行总线宽度来提高信号传输带宽。

高速 IO 接口技术就是在这种情况下应运而生并得到快速发展。通过高速接口可以有效地提高信号的传输带宽，并最终实现芯片及系统间大流量数据的交换。

SerDes（Serializer&Deserializer）的名称说明了高速 IO 的主要目的就是完成多路低速数据和串行高速数据的相互转换。

1. 分类

- 按接口类型可以分为光口和电口。
- 按速率分（如 OC12/OC48，GE）。
- 按应用场合分，LAN/WAN 类型。

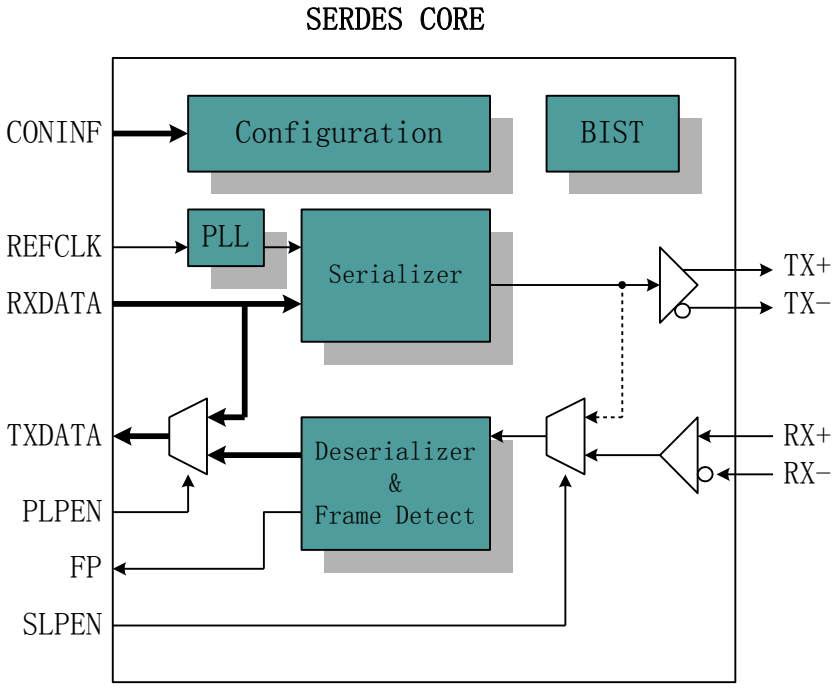
> 说明：
> 通常传统意义上的 PHY 芯片与我们这里提到的 SerDes 有一定的区别，前者往往特指可以跨接较长距离传输媒质（如 Category 5 UTP cable）的物理层收发器。但现在也开始出现一些技术上的融合。

2. 技术来源

目前业界有三类提供高速 SerDes 解决方案的公司：

- 专门提供高速 SerDes 器件，如 AMCC、Vitesse、Cypress 等。
- ASIC Verdor（提供高速 IO 的 Core 用于 ASIC 设计），如 IBM、TI、LSI、Toshiba、Fujitsu 等。
- 提供高速 IO IP 技术交付的公司。

3. 基本结构



SerDes 的主要功能模块包括：

- PLL 及时钟产生电路；
- 复接和分接电路；
- CDR（时钟数据恢复电路）；
- 驱动和接收电路
- 帧同步和编解码电路
- 配置和测试用电路（BIST 等）

4. 关键技术：预加重，接收均衡

当串口速率超过 Gbs 后，对驱动和接收电路的设计提出了很高的要求。特别是当速率达到 2.5Gbps 以上时，越来越多的商用 SerDes IP 采取了预加重和接收均衡技术，来改善信号质量。

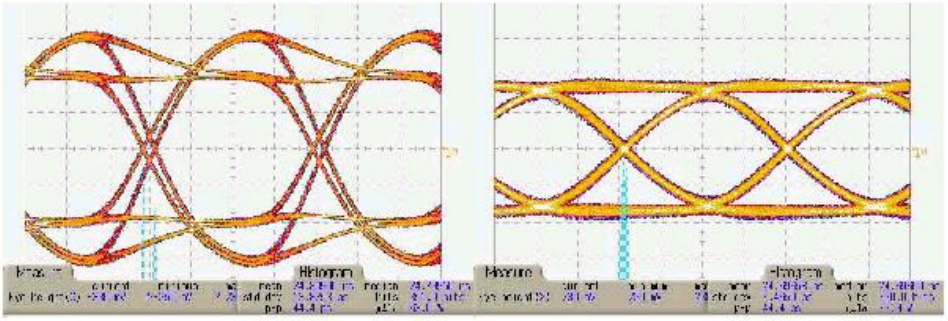


Figure 5: Transmitter Output – Max Pre-Emphasis Figure 6: Receiver Input – Max Pre-Emphasis

图2-74 预加重技术

2.3.4 PCI 接口

PCI 的含义为外围部件互连（Peripheral Component Interconnect）。由 Intel 公司提出。PCI 局部总线是一种具有多路地址线和数据线的高性能的 32/64 位总线。它在高集成度的外围控制器件、外围插件板和处理器/存储器之间作为互联机构应用。

目前应用的 PCI 局部总线规范 2.2 版本包括 PCI 局部总线部件和扩展板的协议、电气、机械和配置规范，并规定了 PCI 的硬件环境。

PCI 总线的主要特点如下：

- 独立于处理器，设备的设计独立于处理器的升级。
- 支持 Master 设备和 Slave 设备。
- 地址总线和数据总线分时复用
- 在全部的读写传送中可实现突发传送。
- 数据传输率高。在 32b@33MHz 时支持 132MB/s 的峰值传送速率。32b@66MHz 支持 264MB/s 的峰值传送速率。
- 隐式总线仲裁。总线仲裁能够在另一个主设备执行传输时进行。
- 交易完整性校验。在地址、命令和数据上都进行奇偶校验。
- 3 类地址空间。即存储器、IO 和配置地址空间。
- 自动配置。支持自动的设备检测与配置。

1. 接口信号

在一个 PCI 应用系统中，如果某设备取得了总线控制权，就称其为“主设备”，而被主设备选中进行通信的设备就是“从设备”。作为从设备，至少需要 47 根信号线，作为主设备至少需要 49 根。PCI 总线接口信号说明如下：

必选信号：

地址/数据线：AD[31:0]; C/BE[3:0]; PAR

接口控制信号：FLAME#, TRDY#, IRDY#, STOP#, IDSEL#, DEVSEL#,

错误报告信号：PERR#, SERR#

仲裁信号：REQ#，GNT#

系统信号：CLK，RST#

系统信号定义：

CLK： IN，频率范围：0～33M 或者 0～66M，除了 RST 格 INT（A～D）外，其余信号都在 CLK 的上升沿有效；

RST#： IN，异步复位，复位是 PCI 的全部输出应驱动到 3 态；

地址和数据信号：

AD[31:0]：地址、数据复用信号，一个总线交易有一个地址期和一个或多个数据期构成，FRAME#有效时，是地址期，IRDY#和 TRDY#有效时是数据期；

C/BE[3:0]：总线命令和字节使能多路复用信号线；在地址期中，传输的是总线命令；在数据期内，传输的是字节使能信号。[0]对应于最低的 8 个字节。

接口控制信号：

FLAME #：帧周期信号；由当前的主设备驱动，表示一次交易的开始和持续时间；FRAME 失效后，是交易的最后一个数据期；

IRDY #：主设备准备好信号；由当前主设备驱动；在读周期，表示主设备已作好接收数据的准备；在写周期，表明数据已提交到 AD 总线上；

TRDY#：目标设备准备好信号；由当前被寻址的目标设备驱动；在读周期，表明数据已提交到 AD 总线上；在写周期，表示从设备已作好接收数据的准备；

数据传输期间，TRDY#, IRDY#任一个无效都将插入等待周期；

STOP#：停止数据传送信号；由目标设备驱动；表示目标设备要求主设备中止当前的数据传送；

IDSEL#：初始化设备选择信号；在参数配置读和配置写期间，用作片选信号；

DEVSEL#：设备选择信号；由当前被寻址的目标设备驱动；

LOCK#：锁定信号（可选）；

仲裁信号：

REQ#：总线占用请求；

GNT#：总线允许信号；

PCI 的仲裁为“隐式”仲裁，即在一个主设备控制总线时，仲裁器仍然起作用。当主设备接受来自仲裁器的授权时，必须等待当前的主设备完成其传送，直到采样到 FRAME 和 IRDY 均无效时，它才认为自己取得总线授权。

错误报告信号：

PERR#：数据奇偶校验错误信号；由数据的接收端驱动，同时设置其状态寄存器中的奇偶校验错误位。一个交易的主设备负责给软件报告奇偶校验错误，为此在写数据期它必须检测 PERR 信号。

SERR#：系统错误报告信号；它的作用是报告地址奇偶错误，特殊周期命令的数据错误。SERR#是一个 OD（漏极开路）信号，它通常会引起一个 NMI 中断，Power PC 中会引起机器核查中断。

中断信号：

中断在 PCI 中是可选项，属于电平敏感型，低电平有效，OD，与时钟异步。其中 INTB ～INTD 只能用于多功能设备。中断线和功能之间的最终对应关系是由中断引脚寄存器来定义的。

2. 总线命令

总线命令是用来规定主、从设备之间的传输类型的，它出现于地址期的 C/BE[3:0]#总线上。常见的 PCI 总线命令如下：

C/BE[3：0]#	命令说明
0010	IO 读
0011	IO 写
0110	存储器读
0111	存储器写
1010	配置读
1011	配置写

3. 交易原则

对于实际 PCI 接口设计，重要的是记住以下 PCI 总线的交易原则，具体波形可参考 PCI2.2 协议。

(1) 主设备信号

- FRAME#信号初次有效标识一次传输的开始。
- FRAME#和 IRDY#信号定义了总线的忙/空闲状态。当其中一个有效时，总线处于忙状态；两个都无效时，总线处于空闲状态。
- 一旦 FRAME#信号被置为无效，在同一传输期间不能重新设置。
- 除非设置 IRDY#信号，一般情况下不能设置 FRAME#信号无效。
- 一旦主设备设置了 IRDY#信号，直到当前的数据期结束为止，主设备不能改变 IRDY#和 FRAME#信号的状态。
- 最后一个数据期完成后，主设备必须撤销 IRDY#信号。

(2) 从设备信号

- STOP#一旦建立，就必须保持到 FRAME#信号撤销为止，接着 STOP#也必须撤销。
- 只要 STOP#信号有效，主设备必须撤销 FRAME#并尽快发出 IRDY#。
- 一个设备通过 DEVSEL#信号表明它是被访问的目标。
- DEVSEL#信号的有效必须早于或同时于目标使能输出时所对应的时钟边沿。
- TRDY#、STOP#和 DEVSEL#信号必须在最后一个数据期完成后的时钟处撤销，并保持三态到下一个时钟处。
- 一个数据传输发生在每个 IRDY#和 TRDY#同时有效的时钟沿上，而与 STOP#的状态无关。

(3) 延迟

- 所有从设备应在 FRAME#有效后的 16 个时钟内完成传输的初始数据期。
- 从设备应在完成前一数据期后的 8 个时钟内完成后续数据期。
- 对于任何数据期，主设备应在 8 个时钟内使它的 IRDY#有效。

4. 配置空间

配置空间是一个容量为 256 字节的地址空间，用以记录设备供应商识别 ID、设备标识 ID、版本 ID、自身地址空间大小、支持的总线操作等特性。在实际设计中，需要重点关注命令寄存器和基地址寄存器。

2.3.5 PCI Express 接口

PCI Express®（又称 PCIe®）是一种高性能、高带宽串行通讯互连标准，由外设组件互连特别兴 趣组（PCI-SIG）制定，以取代基于总线的通信架构，如：PCI、PCI Extended (PCI-X) 以及加速图形端口（AGP）。

从 PCI 转向 PCIe 的主要动机是以更低的生产成本获得明显提高的系统吞吐量、可扩展性和灵活性，而上述这些是传统基于 PCI 总线的互连几乎根本无法达到的。

PCI Express 标准的制定是着眼未来的，它还在继续发展为系统提供更高的吞吐量。第一代 PCIe 约定的吞吐量是 2.5 千兆位/秒（Gbps），第二代则达到 5.0Gbps，而 PCIe3.0 标准则能支持 8.0Gbps 的速率。

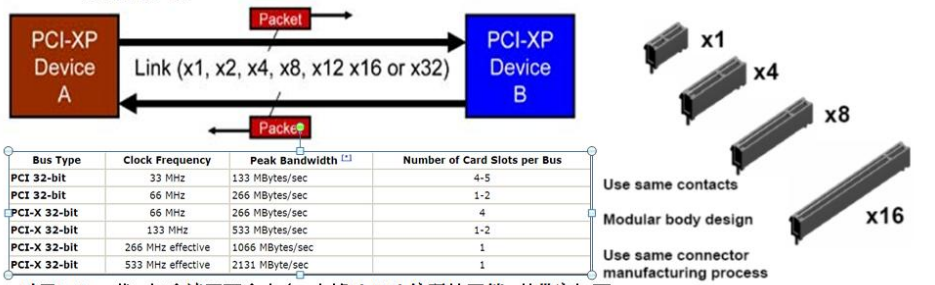
在 PCIe 标准继续利用最新的技术以提供不断增加的吞吐量的同时，利用分层协议、通过使驱动程序保持与现有 PCI 应用的软件兼容性将简化从 PCI 到 PCIe 的过渡。

虽然最初定位在电脑扩展卡和图形卡应用，但目前，PCIe 已在更广泛的领域得到应用，包括：网络、通信、存储、工业和消费类电子产品等。 本章节的目的是使读者充分了解 PCI Express 以及对成功实现 PCIe 来说，那些关键的设计挑战

1. 特点

(1) 全双工，速率可升级，链路数目可扩展，带宽大

如下图所示：



Bus Type	Clock Frequency	Peak Bandwidth	Number of Card Slots per Bus
PCI 32-bit	33 MHz	133 MBytes/sec	4-5
PCI 32-bit	66 MHz	266 MBytes/sec	1-2
PCI-X 32-bit	66 MHz	266 MBytes/sec	4
PCI-X 32-bit	133 MHz	533 MBytes/sec	1-2
PCI-X 32-bit	266 MHz effective	1066 MBytes/sec	1
PCI-X 32-bit	533 MHz effective	2131 MByte/sec	1

对于PCIe一代，包含读写两个方向，去掉8b/10b编码的开销，其带宽如下：

PCI Express Link Width	x1	x2	x4	x8	x12	x16	x32
Aggregate Bandwidth (GBytes/sec)	0.5	1	2	4	6	8	16

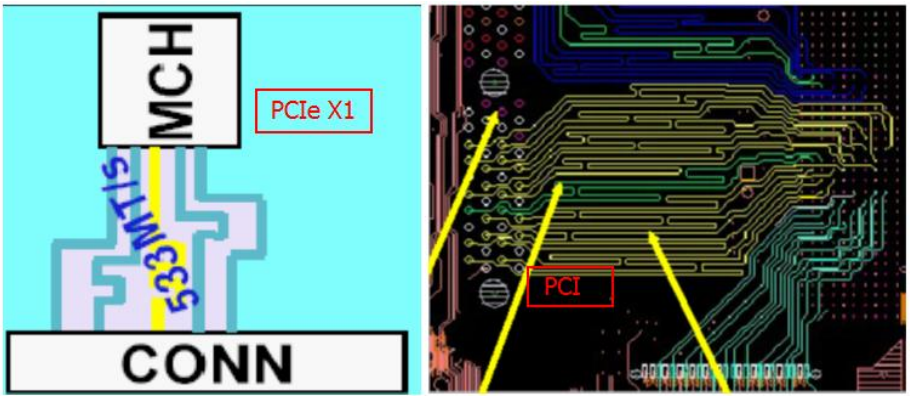
相对于传统的 PCI 总线，PCI Express 有如下优点：

双通道，高带宽，传输速率快：实现类似全双工传输模式（RX 和 TX 分开）；传输速率高，

第一代 PCIe X1 的带宽是 2.5 千兆位/秒（Gbps），第二代则达到 5.0Gbps，而最近发布的 PCIe3.0 标准则能支持 8.0Gbps 的速率；

对更大的带宽，则可以通过扩展链路数目（X1，X2，X4，X8，X16）来实现，而带宽则是其 N 条通路的 N 倍。

(2) 节省 IO，减少板级空间，降低串扰：例如，典型的 PCI 总线数据线至少达到 50 根 IO，而 PCIe XI 仅需要 4 根 IO。而 IO 减少，节省了板级空间，各个 IO 直接的距离可以更宽，从而减少串扰。



(3) 兼容性：在软件层保持与 PCI 兼容，版本升级，向下兼容 PCI 软件

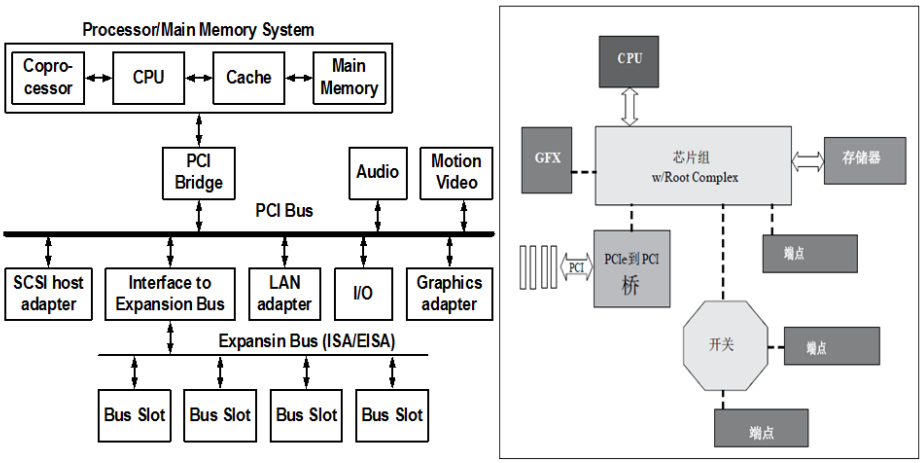
(4) 易用性：支持热插拔，PCIe 总线接口插槽中包括“热插拔检测信号”，可以像 USB 进行热拔插和热交换

(5) 具有错误处理和先进的错误报告功能：得益于 PCI Express 总线是分层结构，其软件层具有错误处理和提供错误报告能力

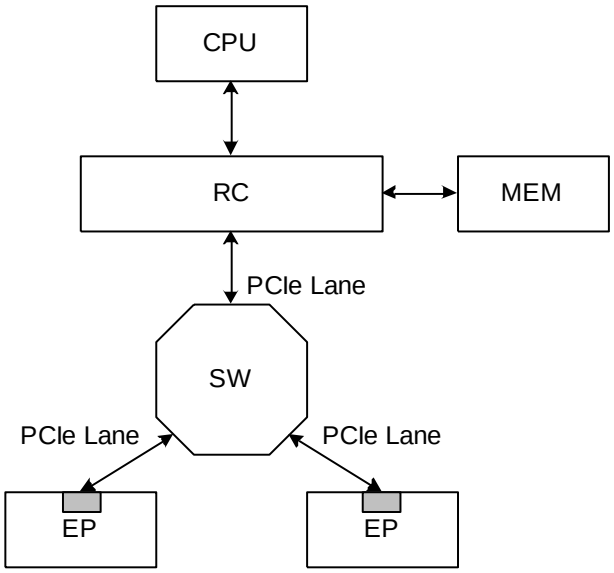
(6) 每个物理连接还有多个虚拟通道：在每个物理通道中支持多个虚拟通道（理论上允许 8 个虚拟通道进行独立的通讯控制），从而支持每个虚拟通道的 Qos，达到非常高的流量质量控制

2. 总线拓扑

如下图所示,PCI 是基于共享设备的拓扑，而 PCIe 基于点对点分离的拓扑。



对于总线拓扑的名词解释，如下图所示：



其中各个组成部分说明如下：

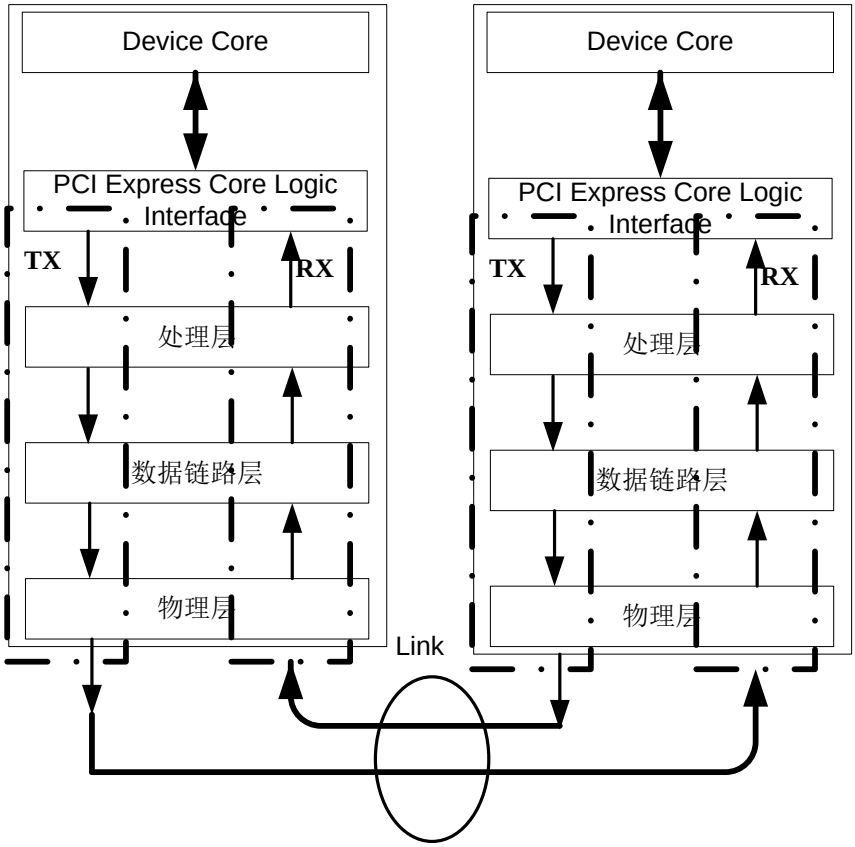
- RC：即根复合体（RootComplex）。是将 CPU 和主存储器连接到 PCIe 线路结构（Fabric）的设备，相当于 PCI 系统中的主桥（或称北桥）。
- SW：即交换开关（Switch）。是 PCIe 系统中用于多设备互连的设备，实现路由寻址转发、仲裁等功能。
- EP：即端点设备（Endpoint）。是只具有上游端口（即指向 RC 或 SW 的端口）的一个具体的设备。端点设备可分为两种类型：传统的

（Legacy）PCIe 端点（简称传统端点）和纯粹的（Native）PCIe 端点（简称纯 PCIe 端点）。它们的主要区别如下：

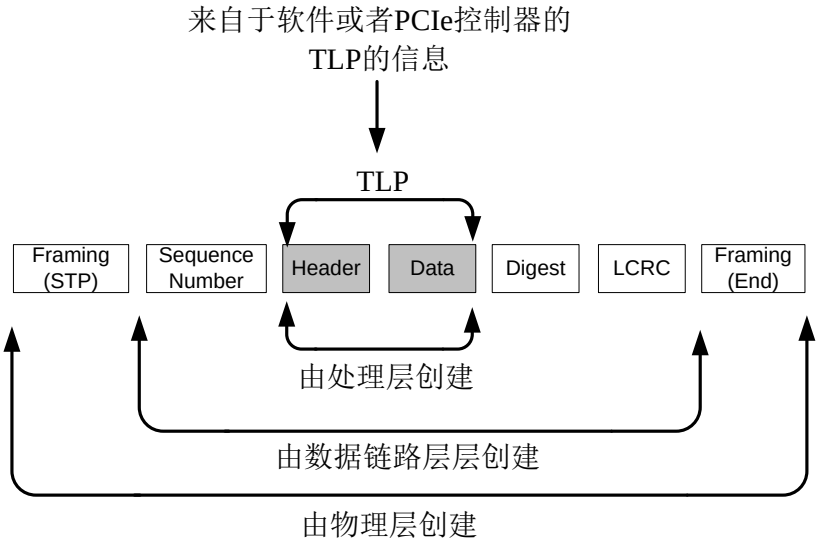
- 传统端点设备：支持 I/O 传送、锁定传送；不需要支持 64 位存储器寻址。
- 纯 PCIe 端点设备：不必支持 IO 或锁定事务，必须支持 64 位存储器寻址。
-

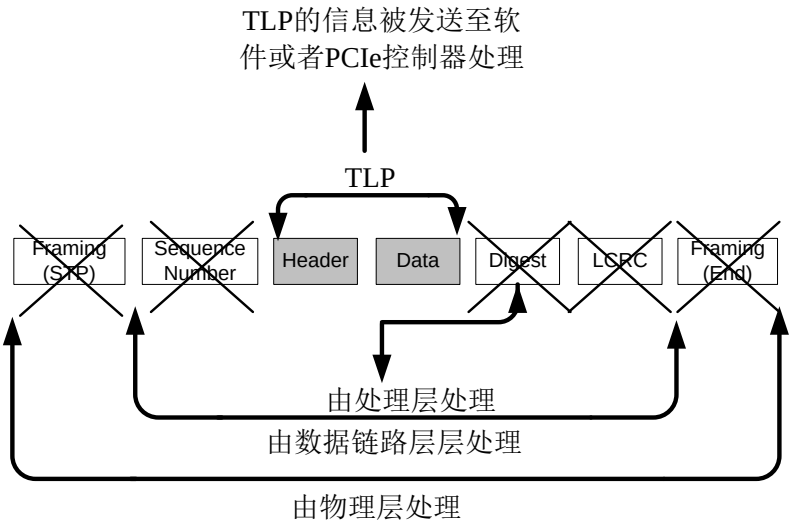
3. 数据结构

PCI Express 规范定义了四层结构。为了保持 PCI Express 和目前主要硬件驱动程序以及应用程序的完全兼容。PCI Express 仍然采用了兼容 PCI 总线寻址的方式，并使用了与定义 PCI 设备即插即用特性一样的标准规范来定义 PCI Express。PCI Express 规范定义的四层结构如下 PCIe 分层结构示意图所示。



在 PCI Express 结构中．首先由软件层产生数据的读写请求，然后数据将被传送到处理层(Tranction)，处理层对数据进行封装之后再传到数据链路层(Data Link)．而数据链路层会为已封装的数据增加时序号(Squence Number)和 CRC 纠错信号，以增加传输数据的正确性和可靠性，然后数据就到达最底层的物理层(Physical Layer)，并通过物理层传送至另一个 PCI Express 设备，并各层进行数据包逆向（拆解）过程（如下图所示）。



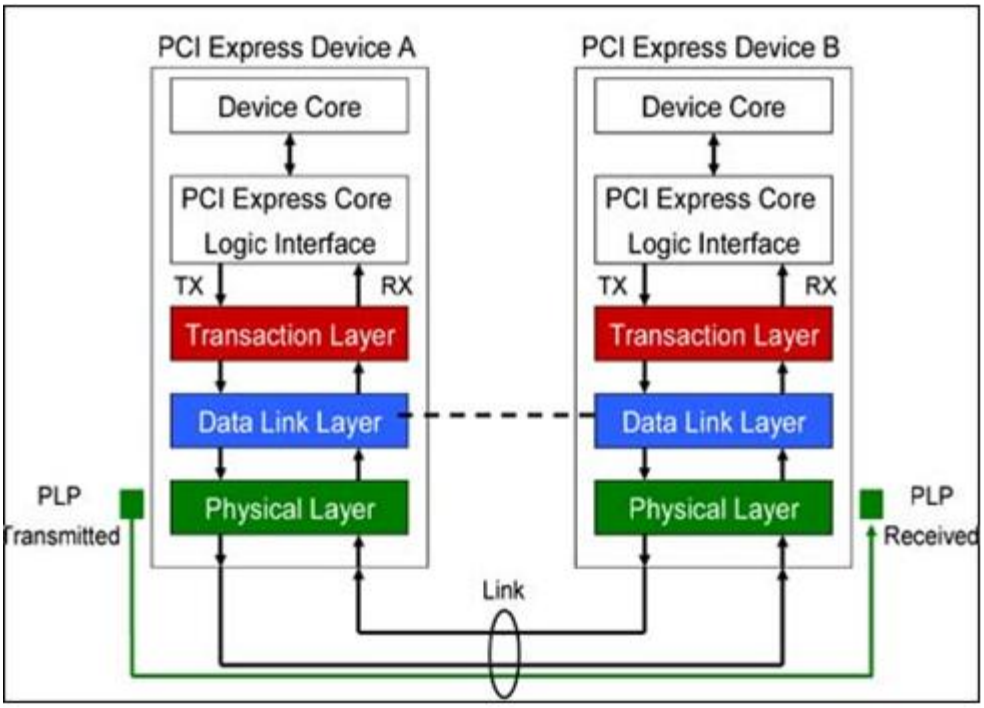


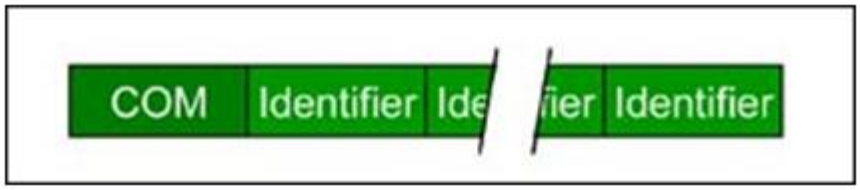
- 物理层(Physical Layer)

和局域网 OS 模型的物理层相似，PCI Express 总线架构中的物理层主要决定总线的物理特征。在未来的发展中，可以通过速度提升或更改编解码方式（PCI Express 1.0 和 2.0，是 8b/10b 编码，PCI Express 3.0 是 128b/130）来进一步提升 PCI Express 总线性能，而这些变更都只会作用于物理层，不会影响其它结构，为升级带来了便利。

PCIe SPEC	编码方式	速率
PCIe 1.0	8b/10b	2.5Gbps
PCIe 2.0	8b/10b	5.0Gbps
PCIe 3.0	128b/130b	8.0Gbps
待发布 PCIe 4.0	128b/130b	16.0Gbps

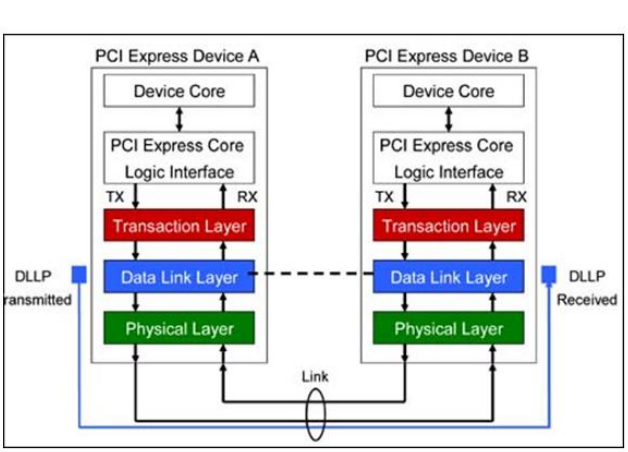
物理层报文主要：源自发送设备的物理层，终至接收设备的物理层，产生和接收有序集，用于链路训练，时钟偏差补偿，将链路置入电气空闲低功耗状态，或者将链路从低功耗状态唤醒等

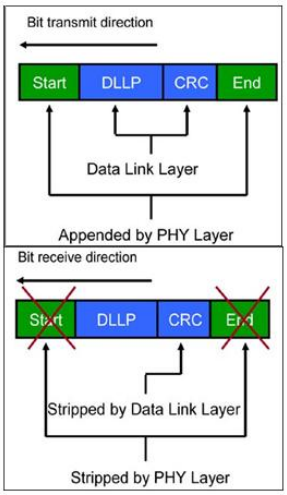




- 数据链路层(Data Link Layer)

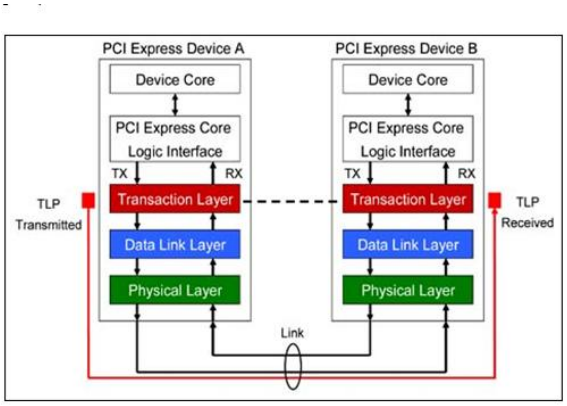
数据链路层的重要作用就是要保证 PCI Express 总线传输的数据包的正确性和可靠性。它会检查数据包封装是否完整正确，并将序列号和冗余校验代码（CRC）添加到数据中进行检测和纠错，并采用 ack/nack 握手协议进行检测纠错。

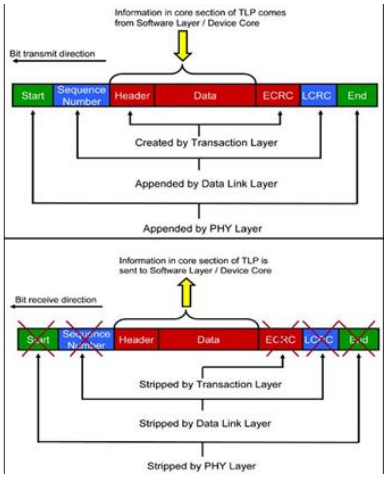




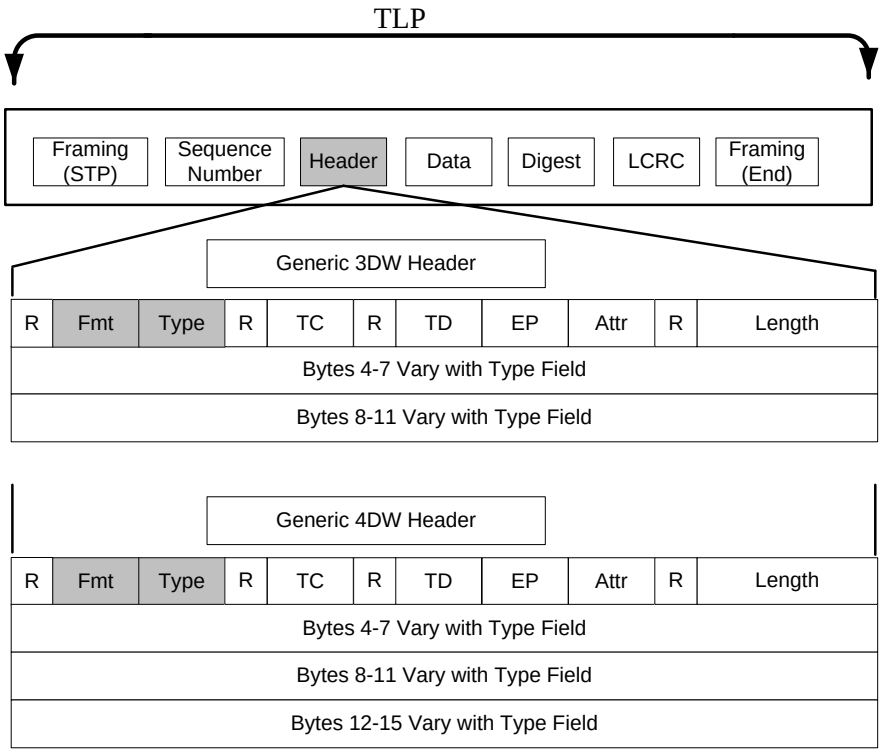
- 传输层(Transaction Layer)

处理层的主要作用有两个，一是接受来自软件层送来的读、写请求，或者本身创建一个请求封装包传到数据链路层，这种数据包称为“处理层数据包”





TransactionLayer Packet，TLP 另一个作用是接受从链路层传来的响应数据包(DataLink Layer Packet，DLL P，并将其与相关的软件请求相关联，传送至软件层处理（TLP 结构如下图 TLP 结构示意图所示）。



所有的数据包部包含有唯一的标识便于和相应的请求及响应相匹配(包括优先权的设置等)，并确保对点通信的正确性。处理层可处理来自 4 个地址空间的数据。包括内存、I/O、配置以及信息地址空间。其中前 3 个地址空间来自 PCI

总线结掏，最后的信息空间(MessageSpace)是 PCI Exp~m 规范新加的。上述地址空间类型如 PCI Express 地址空间类型表格所示。

地址空间	处理方式	处理对象
内存	读、写	来自于或者发送到内存中的数据； 最大可以访问地址范围为 2 的 64 次方
I/O	读、写	来自于或者发送到 I/O 的数据； 最大可以访问地址范围为 2 的 32 次方
配置	读、写	EP 相关设置；每个功能最大的配置空间为 4Kbyte
消息	中断，供应商信息，电源管理，热插拔等	事件信息，通用信息等

传输层还有一个重要作用就是处理系统中断请求，PCI Express 结构的物理层将兼容两种中断模式，即 INTx 模式和 MSI 模式。其中 INTx 束自 PCI 规范并予以保留，MSI(Message Sig~mled Interrupt，信号信息中断)中断方式将不必经由 CPU 请求而能够由各种外设直接发出，这样就比由主处理系统发出中断请求显得更为有效。

- 软件层(Software Layer)

对于 PCI Express 而言，软件的兼容性是一个异常重要的因素．而软件的兼容性主要体现在两个方面：对硬件的检测和支持其运行。PCI 接口的外设在操作系统中能被系统自动发现，由系统分配相应的资源，如内存、I/O 空间和中断等，构建一个优化的系统运行环境。PCI Express 结构继续保持了 PCI 兼容的配置空间和 I/O 设备连接的规范。在软件方面和 PCI 的完全兼容，包括支持本地存储模式 内存共享模式和在 PCI Express 平台中所有操作系统的引导程序等，所有的软件在 PCI Express 系统中执行都不会有所改变。

4. PCI Express 事务

(1) PCIe 存储器请求事务

PCI Express 存储器事务包括两大类：读请求、完成和写请求。下图描述了系统存储器映射及 3 DWords，4 DWords 存储器请求数据包格式。

3 DWords 和 4 DWords 存储器请求头各个字段的定义：表列出了在 4 DWords 存储器请求头中每个字段的位置和用途。



注意

3 DWords 和 4 DWords 头之间差别在于起始地址字段的位置和大小。

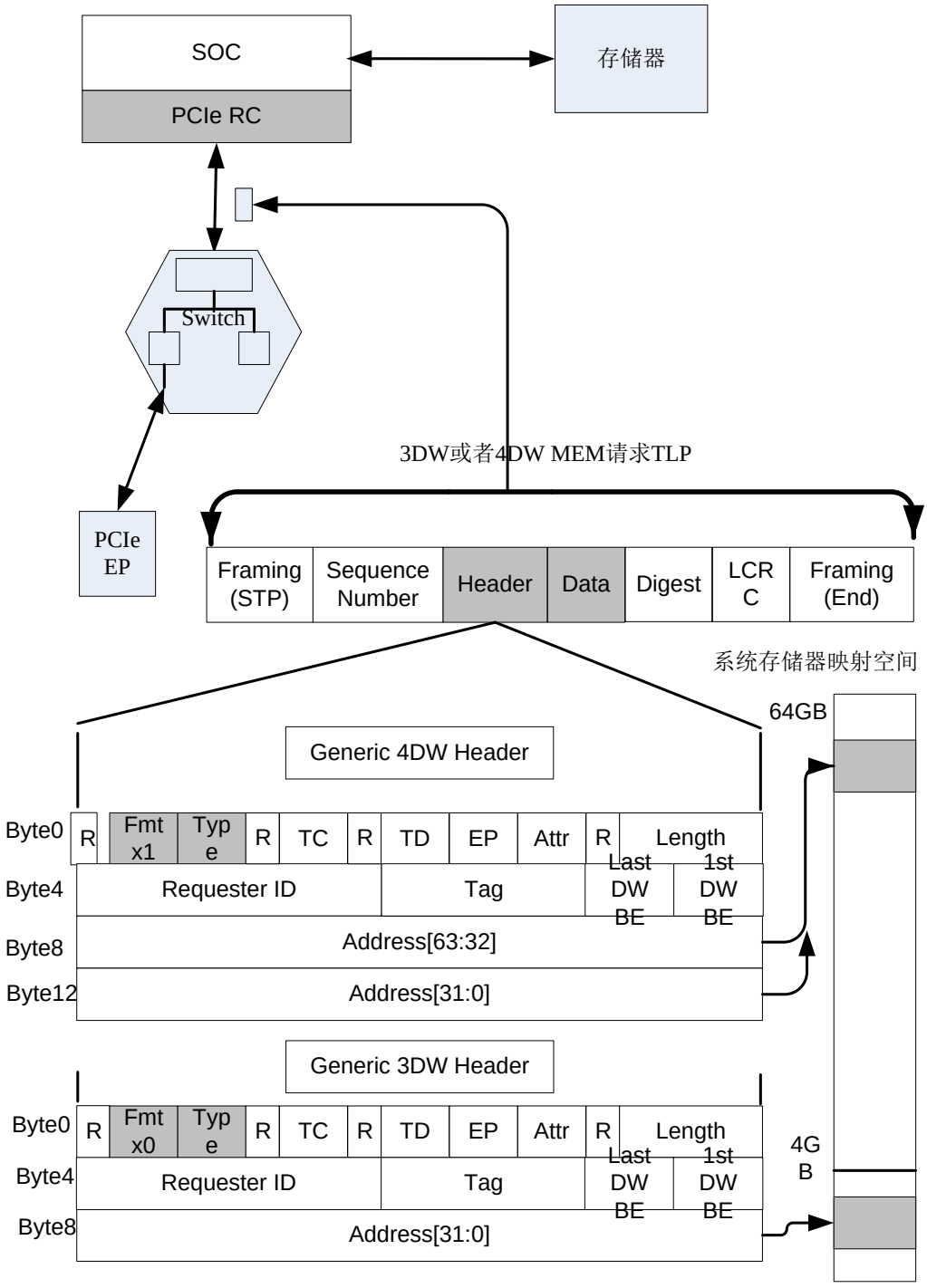
- 对于 3 DWords（32 比特寻址）：地址比特 31:2 在字节 8~11 中，字节 12~15 没有用。
- 对于 4 DWords（64 比特寻址）：地址比特 31:2 在字节 12~15 中，比特 63:32 在字节 8~11 中。
- 二者其他的头字段相同。

4 DWords 存储器请求头字段

字段名	头字节/比特	功能
Length[9:0]	字节 3 的比特 7:0 字节 2 的比特 1:0	TLP 数据有效载荷大小，以 DWord 为单位。最大为 10bit，2^10=1024 DWords(4KB)。编码： 00 0000 0001b=1DWord 00 0000 0010b=2DWord 11 1111 1111b=1023 DWords 00 0000 0000b=1024 DWords
ATTR[1:0]	字节 2 的比特 5:4	比特 5=灵活顺序。 设置为 1 时，对于此 TLP，允许 PCI-X 的灵活顺序。如果设置为 0，则使用 PCI 的严格顺序。 比特 4=不侦测。 当设置为 1 时，表明请求者对于此 TLP 不存在缓存一致性问题。无需要系统硬件引起处理器缓存侦测一致性。 当设置为 0 时，要求进行 PCI 类型的缓冲侦测保护。
EP （ Error Poisoned ）	字节 2 的比特 6	如果设置为 1，尽管允许正常完成事务，但伴随次数据的数据应该被认为是无效的。
TD	字节 2 的比特 7	如果设置为 1，可选一个双字的 TLP 摘要(Digest)字段包含在此 TLP 中。下面是一些规则： TD=1，但没有摘要字段的 TLP 作为 TLP 处理。 如果某设备在场 ECRC 校验并且 TD=1，则必须执行 ECRC 校验。 如果在最终目的地，某设备部不支持 ECRC 校验（可选），则该设备必须忽略此摘要字段
TC[2:0]	字节 1 的比特 6:4	这 3 个比特用例编码应用于此 TLP 以及其相关的完成数据包（如果有的话）的流量类别。 000=流量类别 0（默认） 111=流量类别 7 TC0 是默认的类型，TC1~7 用来提供不同的服务。

字段名	头字节/比特	功能
Type[4:0]	字节 0 的比特 4:0	TLP 数据包的类别字段： 00000b=存储器读或者写 00001b=存储器读锁定
Fmt[1:0]	字节 0 的比特 6:5	数据包的格式： 00b=存储器读（3 DWords 带数据/无数据） 01b=存储器写（3 DWords 带数据） 10b=存储器读（4 DWords 带数据/无数据） 11b=存储器写（4 DWords 带数据）
1st DWord BE[3:0]	字节 7 的比特 3:0	高电平比特对应的第一个 DWord 有效载荷中的字节。
Last DWord BE[3:0]	字节 7 的比特 7:4	高电平比特对应的最后一个 DWord 有效载荷中的字节。
Tag[7:0]	字节 6 的比特 7:0	用来标识请求者发出的每一个未完成的请求。如果发送的非报告请求，则分配下一个连续的标志。 默认请求：只使用比特 4:0（在某一时刻，有 32 个为完成事务）。 如果 PCI Express 控制器寄存器中的扩展标记比特设置为 1，则可以使用全部 8 个比特（256 标记）。
Requester ID	字节 5 的比特 7:0 字节 4 的比特 7:0	识别请求者，以便返回完成数据包等。 字节 4,7:0=总线号 字节 5,7:3=设备号 字节 5,2:0=功能号
Address[31:2]	字节 15 的比特 7:2 字节 14 的比特 7:0 字节 13 的比特 7:0 字节 12 的比特 7:0	存储器传送的 64 比特起始地址的低端 32 比特。注意 32 比特地址的最低端 2 比特（00b），强制起始地址为 DWord 对齐。
Address[63:32]	字节 11 的比特 7:2 字节 10 的比特 7:0 字节 9 的比特 7:0 字节 8 的比特 7:0	存储器传送的 64 比特高端 32 地址

PCIe 存储器读写请求结构示意图



PCI Express 存储器请求事务的特点：

- 存储器传送决不允许跨越 4KB 边界
- 所有的存储器映射写是要被报告的，因此可以产生更高的性能。（即是写后不管）
- 可以使用 32 比特或者 64 比特寻址。3 DWords 头格式支持 32 比特地址，而 4 DWords 头支持 64 比特地址。

- 可以利用突发传送的最大能力，其传送长度为 0~1024 DWords（0~4KByte）。

(2) PCIe IO 请求事务

为了兼容传统设备和可能依赖驻留在 IO 映射而不是存储器映射空间的兼容设备中的软件，PCIe 接口保留支持 IO 事务，并同时支持 32 比特地址 IO 范围，也支持低 16 比特（64KB）的 IO 范围。

注意

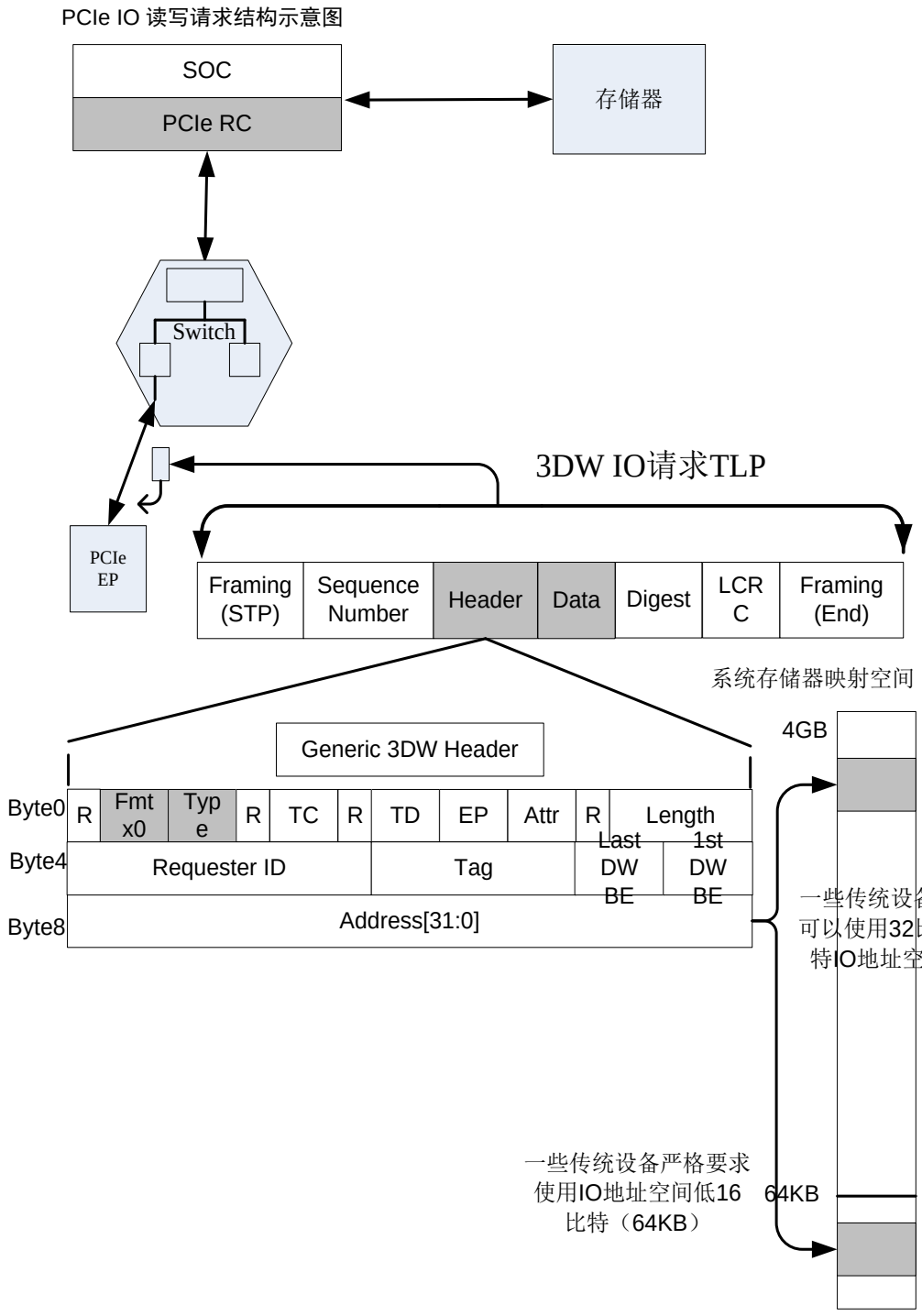
PCIe 接口 EP 模式时，可以配置传统的 PCI Express 设备，支持基址配置寄存器请求 IO 地址分配。

IO 请求头字段的定义。下表给出了一个 IO 请求头的每个字段的位置和用途。

IO 请求头字段

字段名	头字节/比特	功能
Length[9:0]	字节 3 的比特 7:0 字节 2 的比特 1:0	TLP 数据有效载荷大小，以 DWord 为单位。该字段总是为 1。
ATTR[1:0]	字节 2 的比特 5:4	比特 5=灵活顺序。 在配置请求中总是为 0。 比特 4=不侦测。 在 IO 请求中总是为 0。
EP （ Error Poisoned ）	字节 2 的比特 6	如果设置为 1，尽管允许正常完成事务，但伴随次数据的数据应该被认为是无效的。
TD	字节 2 的比特 7	如果设置为 1，可选一个双字的 TLP 摘要(Digest)字段包含在此 TLP 中。下面是一些规则： TD=1，但没有摘要字段的 TLP 作为 TLP 处理。 如果某设备在场 ECRC 校验并且 TD=1，则必须执行 ECRC 校验。 如果在最终目的地，某设备部不支持 ECRC 校验（可选），则该设备必须忽略此摘要字段
TC[2:0]	字节 1 的比特 6:4	这 3 个比特用例编码应用于此 TLP 以及其相关的完成数据包（如果有的话）的流量类别。 在 IO 请求中 TC=0。
Type[4:0]	字节 0 的比特 4:0	TLP 数据包的类别字段。对于 IO 请求，总是为 00001b.

字段名	头字节/比特	功能
Fmt[1:0]	字节 0 的比特 6:5	数据包的格式： 00b=IO 读（3 DWords 无数据） 10b=IO 写（3 DWords 带数据）
1st DWord BE[3:0]	字节 7 的比特 3:0	高电平比特对应的第一个 DWord 有效载荷中的字节
Last DWord BE[3:0]	字节 7 的比特 7:4	高电平比特对应的最后一个 DWord 有效载荷中的字节
Tag[7:0]	字节 6 的比特 7:0	用来标识请求者发出的每一个未完成的请求。如果发送的非报告请求，则分配下一个连续的标志。 默认请求：只使用比特 4:0（在某一时刻，有 32 个为完成事务） 如果 PCI Express 控制器寄存器中的扩展标记比特设置为 1，则可以使用全部 8 个比特（256 标记）
Requester ID[15:0]	字节 5 的比特 7:0 字节 4 的比特 7:0	识别请求者，以便返回完成数据包等。 字节 4,7:0=总线号 字节 5,7:3=设备号 字节 5,2:0=功能号
Address[31:2]	字节 11 的比特 7:2 字节 10 的比特 7:0 字节 9 的比特 7:0 字节 8 的比特 7:0	IO 传送的 32 比特起始地址高端 30 比特。注意 32 比特地址的最低端 2 比特（00b），强制起始地址为 DWord 对齐。



(3) PCIe 配置请求事务

为了保持与 PCI 兼容，PCI Express 同时支持类型 0 和类型 1 配置周期。类型 1 周期向下游传播，直到它到达目标所在的总线的桥接口。在目的链路上，桥将配置事务从类型 1 转换成类型 0。PCIe 配置读写请求示意图举例说明类型 1 配置周期向下游传播的过程。在目的链路上，它将被转换成类型 0，并由端点

设备声明。所有端点设备都在每个类型 0 配置写周期中捕获由上游设备提供的总线号和设备号信息，该信息包含在配置请求头的 8~9 字节中。

说明

配置请求总是使用 3 DWords 头格式，而且根据 ID 字段的内容来路由。

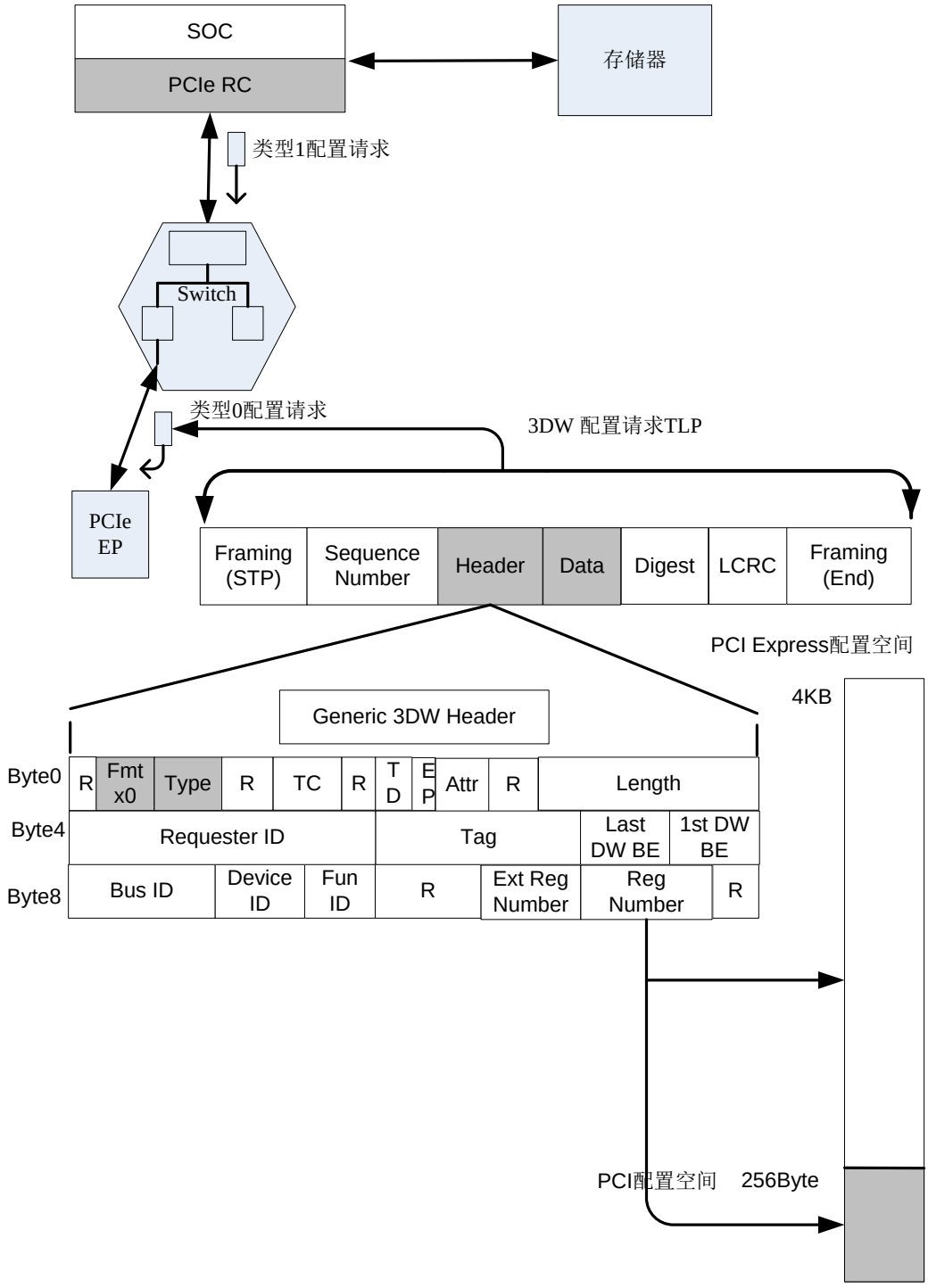
配置请求头各个字段定义。

配置请求头字段

字段名	头字节/比特	功能
Length[9:0]	字节 3 的比特 7:0 字节 2 的比特 1:0	TLP 数据有效载荷大小，以 DWord 为单位。该字段总是为 1。
ATTR[1:0]	字节 2 的比特 5:4	比特 5=灵活顺序。 在配置请求中总是为 0。 比特 4=不侦测。 在配置请求中总是为 0。
EP （ Error Poisoned ）	字节 2 的比特 6	如果设置为 1，尽管允许正常完成事务，但伴随次数据的数据应该被认为是无效的。
TD	字节 2 的比特 7	如果设置为 1，可选一个双字的 TLP 摘要(Digest)字段包含在此 TLP 中。下面是一些规则： TD=1，但没有摘要字段的 TLP 作为 TLP 处理。 如果某设备在场 ECRC 校验并且 TD=1，则必须执行 ECRC 校验。 如果在最终目的地，某设备部不支持 ECRC 校验（可选），则该设备必须忽略此摘要字段。
TC[2:0]	字节 1 的比特 6:4	这 3 个比特用例编码应用于此 TLP 以及其相关的完成数据包（如果有的话）的流量类别。 在配置请求中 TC=0。
Type[4:0]	字节 0 的比特 4:0	TLP 数据包的类别字段： 00100b=配置 0 请求 00101b=配置 1 请求
Fmt[1:0]	字节 0 的比特 6:5	数据包的格式： 00b=配置读（3 DWords 无数据） 10b=配置写（3 DWords 带数据）
1st DWord BE[3:0]	字节 7 的比特 3:0	高电平比特对应的第一个 DWord 有效载荷中的字节。
Last DWord BE[3:0]	字节 7 的比特 7:4	高电平比特对应的最后一个 DWord 有效载荷中的字节。

字段名	头字节/比特	功能
Tag[7:0]	字节 6 的比特 7:0	用来标识请求者发出的每一个未完成的请求。如果发送的非报告请求，则分配下一个连续的标志。 默认请求：只使用比特 4:0（在某一时刻，有 32 个为完成事务）。 如果 PCI Express 控制器寄存器中的扩展标记比特设置为 1，则可以使用全部 8 个比特（256 标记）。
Requester ID[15:0]	字节 5 的比特 7:0 字节 4 的比特 7:0	识别请求者，以便返回完成数据包等。 字节 4,7:0=总线号 字节 5,7:3=设备号 字节 5,2:0=功能号
Register Number[5:0]	字节 11 的比特 7:2	配置空间偏移量（DWord）的低 6 比特。寄存器好与扩展寄存器号一起用来表示 1024 DWords（4096 字节）PCI Express 配置空间的 10 比特偏移量。
Ext Register Number[3:0]	字节 10 的比特 3:0	配置空间偏移量（DWord）的高 4 比特。扩展寄存器好与扩展寄存器号一起用来表示 1024 DWords（4096 字节）PCI Express 配置空间的 10 比特偏移量。
Complete ID[15:0]	字节 9 的比特 7:0 字节 8 的比特 7:0	标识此配置周期要访问的完成者。设备可以在配置类型 0 写时，捕获这字段中的总线和设备。 字节 4,7:0=总线号 字节 5,7:3=设备号 字节 5,2:0=功能号

PCIe 配置读写请求示意图



(4) PCIe 完成请求事务

完成在以下每个非报告请求后返回：

- 存储器读请求可能返回带数据的完成（CplD）。
- IO 读请求可能导致带数据或者不带数据的完成（CplD）。

- IO 写请求可能导致没有数据的完成（Cpl）。
- 配置读请求可能导致带数据的完成（CplD）。
- 配置写请求可能导致没有数据的完成（CplD）。
- 配置读请求。

完成请求头中的许多字段与相应的请求必须要有相同的值，包括流量类别，属性比特和用于路由完成数据返回原请求者的原请求者 ID。下图描述了在非报告请求之后返回的一个完成，以及 3 DWords 的完成头格式。

完成头各字段的定义。列出了完成头各字段的位置和用途。

完成请求头字段

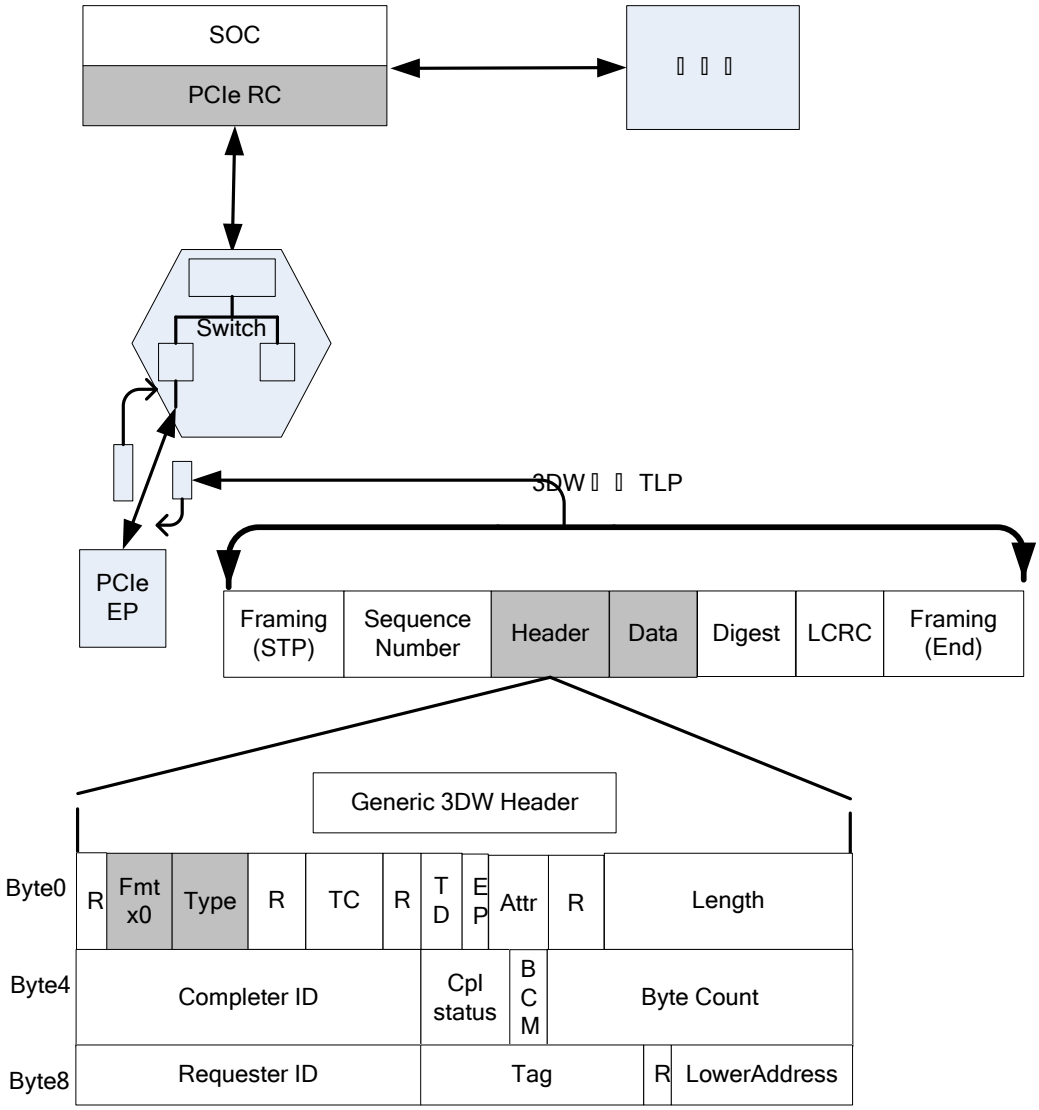
字段名	头字节/比特	功能
Length[9:0]	字节 3 的比特 7:0 字节 2 的比特 1:0	TLP 数据有效载荷大小，以 DWord 为单位。该字段反映与完成数据包相关的数据有效载荷的大小。
ATTR[1:0]	字节 2 的比特 5:4	属性 1：灵活顺序。 属性 0：不侦测比特 在完成数据包中，这些比特设置成和请求中的状态一样。
EP （ Error Poisoned ）	字节 2 的比特 6	如果设置为 1，表示数据有效载荷中毒了。
TD	字节 2 的比特 7	如果设置为 1，可选一个双字的 TLP 摘要(Digest)字段包含在此 TLP 中。下面是一些规则： TD=1，但没有摘要字段的 TLP 作为 TLP 处理。 如果某设备在场 ECRC 校验并且 TD=1，则必须执行 ECRC 校验。 如果在最终目的地，某设备部不支持 ECRC 校验（可选），则该设备必须忽略此摘要字段
TC[2:0]	字节 1 的比特 6:4	表示包传送类别，对于完成数据包，TC 设置成和请求相同的值。
Type[4:0]	字节 0 的比特 4:0	对于完成数据包，TLP 数据包的类别字段总是=01010b
Fmt[1:0]	字节 0 的比特 6:5	数据包的格式： 00b=无数据的完成数据包（Cpl） 10b=带数据的完成数据包（CplD）
Byte Counter	字节 7 的比特 7:0 字节 6 的比特 3:0	读请求结束前的剩余字节数。通常，可以由原请求的长度字段获得。
BCM	字节 6 的比特 4	修改的字节计数，只能有 PCI-X 完成者设置为 1，所以在应用中，PCIe 接口默认设置为 0。

字段名	头字节/比特	功能
CS [2:0]	字节 6 的比特 7:5	完成状况代码。 000b=成功完成(SC) 001b=不支持请求成功(UR) 010b=配置请求重试终止(CR) 100b=完成值终止(CA) 其他保留。
Complete ID[15:0]	字节 5 的比特 7:0 字节 4 的比特 7:0	标识完成者。尽管路由完成数据包时不需要这个信息，但在调试总线流量时，这一信息非常有用。 字节 4,7:0=总线号 字节 5,7:3=设备号 字节 5,2:0=功能号
Low Address[6:0]	字节 11 的比特 6:0	一次读操作返回数据的第一允许字节的地址低端 7 比特。可以由请求数据包中的长度和字节允许来计算。常用来确定下一合法读完成的边界。
Tag[7:0]	字节 6 的比特 7:0	设置这些比特以反映请求标记。请求者使用这一字段将入站的完成数据包与未完成的请求关联起来。
Requester ID[15:0]	字节 9 的比特 7:0 字节 8 的比特 7:0	从请求中复制到此字段，用来路由完成数据包返回原请求者。 字节 4,7:0=总线号 字节 5,7:3=设备号 字节 5,2:0=功能号

完成状况代码总结（参见上表中的完成状况字段）：

- 000b(SC)成功完成的代码，表示原请求在目标处已正确完成。
- 001b(UR)不支持的请求，表示原请求在目标处失败，因为它以不支持的地址为目标、携带不支持的地址或请求等，为不可修复的错误。
- 010（CRS）配置请求重试状况，表示目标暂时掉线，应该尝试重试（例如：复位后的初始化延时等）。
- 100b 完成中止代码，表示完成者由于某一错误而掉线，为不可修复的错误。

PCIe 完成请求结构示意图



(5) PCIe 消息请求事务

消息请求可以代替早期总线协议使用的许多中断，错误和电源管理边带信号。所有的消息请求都使用 4 DWords 头格式，并且处理消息请求的过程与处理存储器写事务基本相同。消息可以使用地址路由，ID 路由或者隐式路由方法来路由。下表描述了消息请求头的格式。

消息请求头各字段的定义。下 0 描述了头各字段的位置和用途。

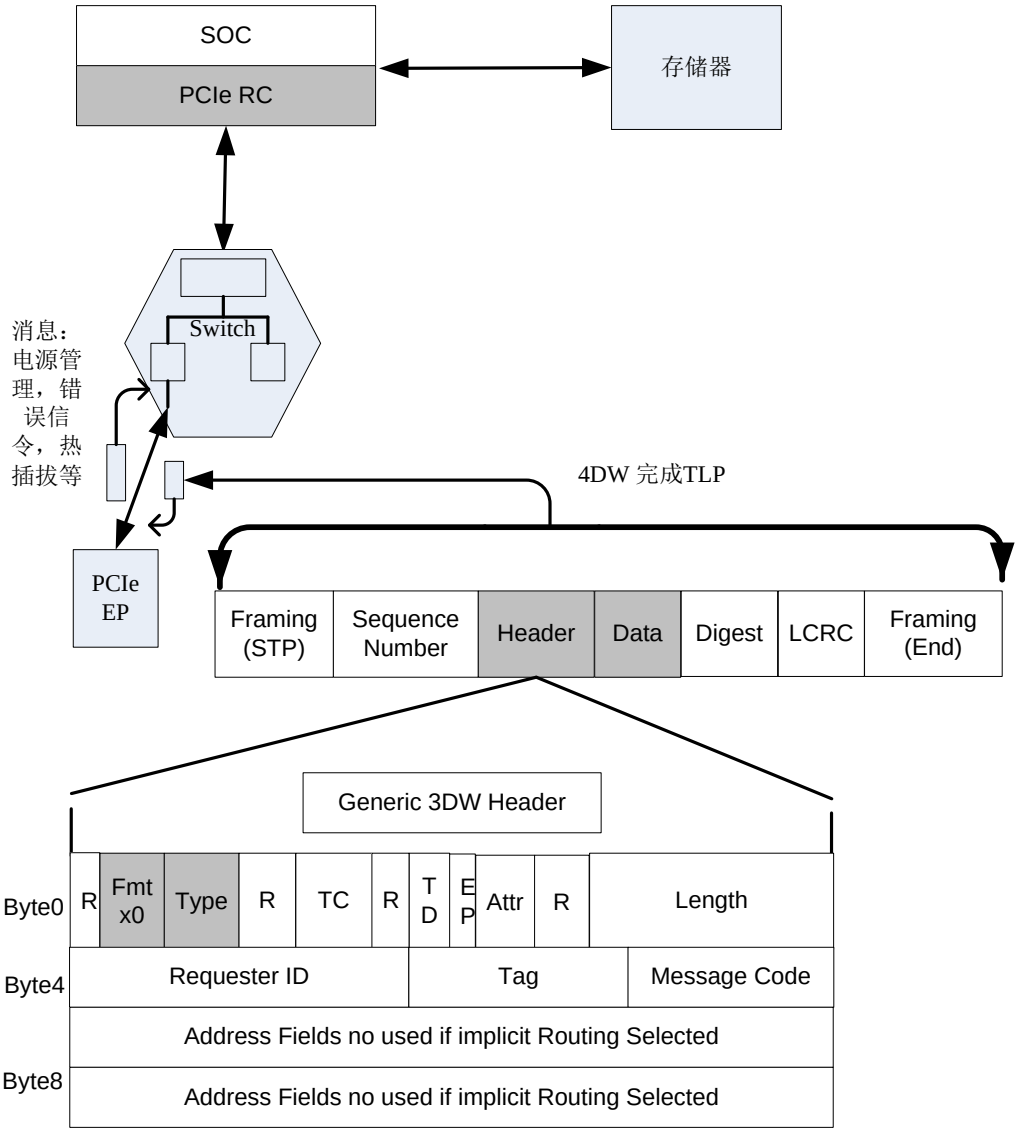
4 DWords 消息请求头字段

字段名	头字节/比特	功能
Length[9:0]	字节 3 的比特 7:0 字节 2 的比特 1:0	TLP 数据有效载荷大小，以 DWord 为单位。该字段总是为 1。

字段名	头字节/比特	功能
ATTR[1:0]	字节 2 的比特 5:4	属性 1：灵活顺序。 属性 0：不侦测比特。 在消息数据包中，这些比特总是设置 0。
EP （ Error Poisoned ）	字节 2 的比特 6	如果设置为 1，代表数据中毒了。
TD	字节 2 的比特 7	如果设置为 1，可选一个双字的 TLP 摘要(Digest)字段包含在此 TLP 中。下面是一些规则： TD=1，但没有摘要字段的 TLP 作为 TLP 处理。 如果某设备在场 ECRC 校验并且 TD=1，则必须执行 ECRC 校验。 如果在最终目的地，某设备部不支持 ECRC 校验（可选），则该设备必须忽略此摘要字段。
TC[2:0]	字节 1 的比特 6:4	这 3 个比特用例编码应用于此 TLP 以及其相关的完成数据包（如果有的话）的流量类别。 在消息请求中 TC=0。
Type[4:0]	字节 0 的比特 4:0	TLP 数据包的类别字段： 比特 4:3 10=消息请求 比特 2:0 000b=路由至根联合体 001b=根据地址路由 010b=根据 ID 路由 011b=根联合体广播 Msg 100b=本地，终止于接收器 101b=收集/路由至根联合体 其他保留。
Fmt[1:0]	字节 0 的比特 6:5	数据包的格式： 01b=4 DWords 无数据消息请求 11b=4 DWords 带数据消息请求

字段名	头字节/比特	功能
Msg Tag[7:0]	字节 6 的比特 7:0	该字段含有表明要发送的消息的类型的编码。 0000 0000b=解除锁定消息 0001 xxxxb=电源管理消息 0010 0xxxb=Intx 消息 0011 00xxb=出错消息 0100 0000b=热插拔消息 0101 0000b=插槽电源消息 0111 111xb=厂商类型 0 消息 0111 1111b=厂商类型 0 消息
Requester ID	字节 5 的比特 7:0 字节 4 的比特 7:0	识别请求者，以便返回完成数据包等。 字节 4,7:0=总线号 字节 5,7:3=设备号 字节 5,2:0=功能号
Address[31:2]	字节 15 的比特 7:2 字节 14 的比特 7:0 字节 13 的比特 7:0 字节 12 的比特 7:0	如果消息选择地址路由方法，则此字段有效。传送的 64 比特起始地址的低端 32 比特。注意 32 比特地址的最低端 2 比特（00b），强制起始地址为 DWord 对齐。
Address[63:32]	字节 11 的比特 7:2 字节 10 的比特 7:0 字节 9 的比特 7:0 字节 8 的比特 7:0	如果消息选择地址路由方法，则此字段有效。消息传送的 64 比特高端 32 地址。

PCIe 消息请求结构示意图



2.3.6 QDR-II

四倍数据速率(Quad Data Rate，QDR)SRAM 技术由赛普拉斯、瑞萨、IDT 等公司共同定义并开发，主要面向高性能通信应用。新型 QDR-II 架构是对该联合开发小组研制的 QDR 系列高性能网络和通信 SRAM 产品的一个补充。QDR/QDR-II SRAM 不仅极大地增加了系统存储器带宽，而且还可用作面向这些高性能网络系统中的查找表、链接列表和控制器缓冲存储器的存储器解决方案。

QDR-II 与 QDR 架构的主要差异包括增加了 DLL，以及额外的半个周期延迟(最初的 QDR 为 1 个周期，QDR-II 为 1.5 个周期)。这些变化的结果是使时钟至数据有效时间 T_{co} 在 167MHz 的频率条件下从 3.0ns 缩减至 0.45ns，使数据

有效窗口增大，从而提高系统时序性能。另一个结果是出现一个用于实现可靠数据获取的源同步回送时钟(Echo Clock)。

QDR-II SRAM 接口管脚定义：

Pins	Description
D	Write Data
Q	Read Data
K	Write Clock
Kn	Inverted Write Clock
C	Read Clock
Cn	Inverted Read Clock
CQ	Echo Clock
CQn	Inverted Echo Clock
All Other	Address and Command

1. 主要特性

(1) 独立的读写端口

QDR-II 架构包括两个用于对存储器阵列进行存取的独立端口，分别为一个读端口和一个写端口，利用一根公用地址总线来实现对每个端口的访问。QDR-II 器件可以同时执行读和写操作，而没有死周期(dead cycle)，对于读/写操作数均衡的应用，QDR-II 可在相同的频率和数据总线宽度条件下提供最高的带宽。

(2) 源同步回送时钟

CQ 和 CQ#是以帮助存储器控制器对来自 QDR-II SRAM 的读数据进行锁存为目的而生成的自由振荡输出时钟。CQ 和 CQ#是由把 C 和 C#(在单时钟模式中则为 K 和 K#)用作输入基准时钟的内部 DLL 生成的。这些回送时钟的上升沿与有效数据严格匹配。数据在回送时钟上升沿之后拥有 300ps 的最大保证时间(250MHz 器件)。

(3) 输出阻抗匹配电路

阻抗匹配电路使得用户能够设定 QDR-II SRAM 的输出驱动器的强度。阻抗匹配是通过在 ZQ 引脚和地之间连接一个电阻器的方法来实现的，ZQ 电阻器的阻值应为所需输出阻抗的 5 倍(在 25Ω 至 70Ω 之间)，阻抗匹配电路的精确度约为±15%。该特点使得用户能够对器件的驱动强度进行微调，以便与传输线阻抗相匹配。

2. 操作说明

QDR-II 采用了两个输入时钟(K 和 K#)，在两个时钟的上升沿锁存输入数据。一对可选的输入时钟(C 和 C#)负责控制输出数据寄存器，并决定何时从器件中读出数据。

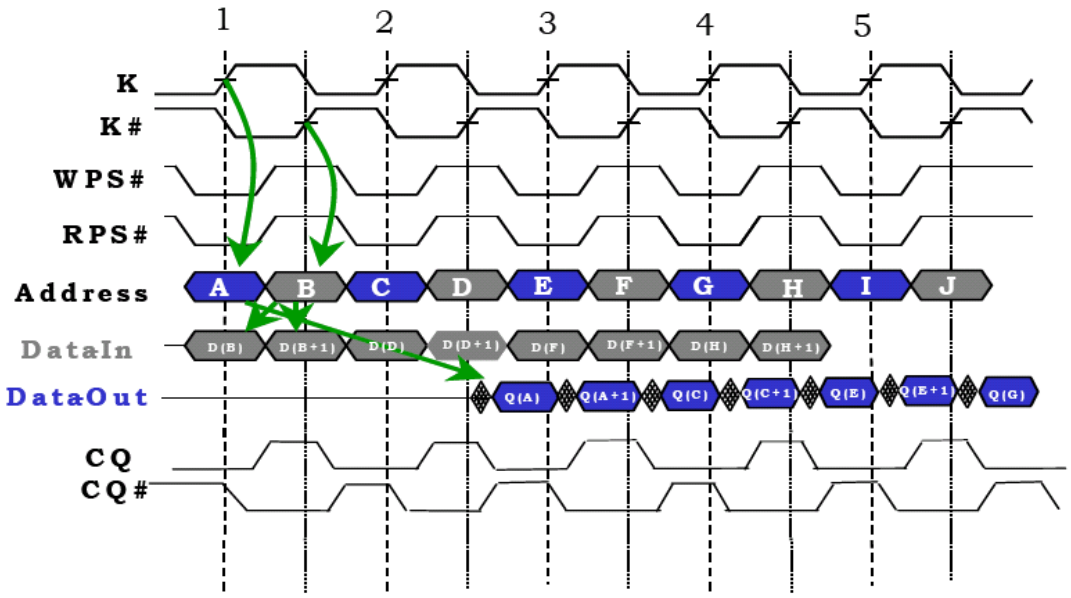
控制信号包括 WPS#(写端口选择)、RPS#(读端口选择)和 BWSx#(字节写选择)。置位(asserting)WPS#将启动一个写操作，置位 RPS#将启动一个读操作，BWSx#用来执行字节选择写操作。读写操作都是面向 Burst 的，支持 Burst

Length 为 2 和 4 两种脉类型。突发长度为 2 和 4 采用相同的控制信号，但这两种类型的地址速率和写数据是不同的。

(1) Burst Length 为 2

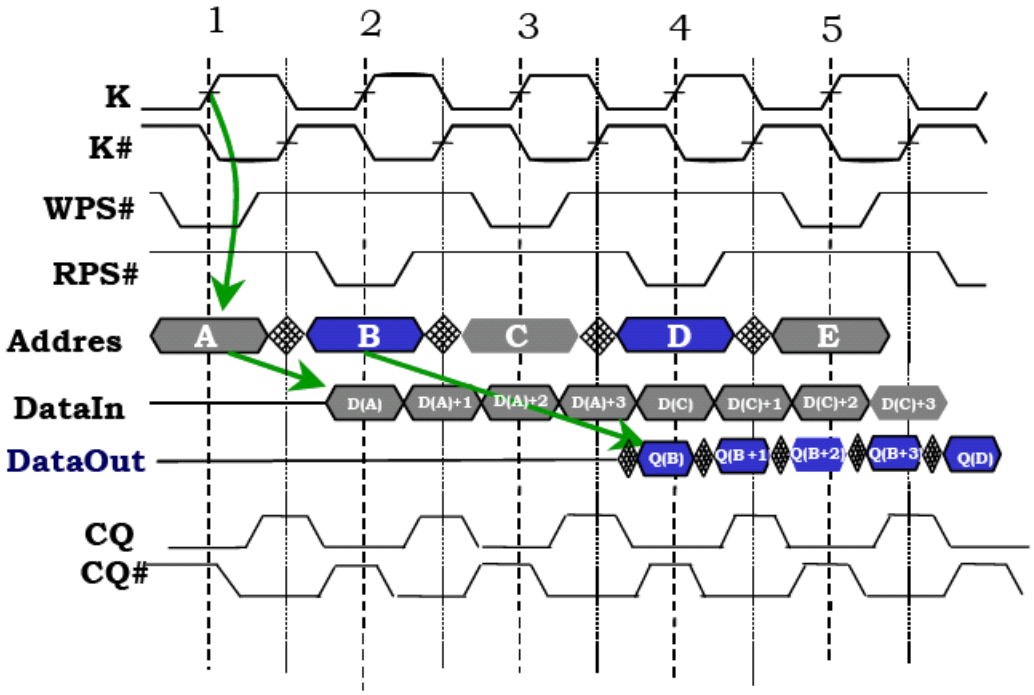
一个写操作和一个读操作可以在相同的时钟周期启动。由于写端口和读端口共用相同的地址总线，因此 BL=2 的 QDR-II 采用的是一种双倍地址速率(double-address-rate)操作。读存取和写存取分别通过在 K 脉冲的上升沿置位 RPS#和 WPS#的方法来启动。读地址被锁存于 K 脉冲的相同上升沿，而写地址被锁存于 K#脉冲的上升沿。两个写数据字在相同的 K 和 K#脉冲上升沿输入 SRAM。两个读数据字则在 K 脉冲上升沿之后的一个半周期从 SRAM 输出。

时序图如下：



(2) Burst Length 为 4

在 BL=4 的器件中，在 K 时钟脉冲的每个上升沿启动一个读存取或写存取。写存取是通过在 K 脉冲的上升沿置位 WPS#来完成的。写地址被锁存于相同的 K 时钟脉冲上升沿。从 K 脉冲的后一个上升沿开始，4 个连续的数据字被锁存于 K 和 K#脉冲的上升沿。读存取是通过在 K 脉冲的上升沿置位 RPS#来完成的。读地址被锁存于相同的 K 时钟脉冲上升沿。在下一个 K 时钟脉冲上升沿之后，通过将 C#脉冲(而在单时钟模式中则是 K#脉冲)的上升沿用作时钟基准的方法来在输出数据端口上输出 4 个数据字中的第一个。其余的 3 个数据字则在随后的 3 个 C 和 C#脉冲(在单时钟模式中则为 K 和 K#脉冲)的上升沿输出。数据信号的有效脉冲沿与 CQ 和 CQ#回送时钟脉冲的上升沿严格匹配。时序图如下：



对于单时钟模式中的操作，C 和 C#时钟输入需要从外部连接至 Vdd。选择单时钟模式还是双时钟模式应在启动任何存取操作之前确定。

2.3.7 RLDRAM-II

如今语音、数据和视频数据流的融合无疑给网络系统设计师带来了极大的挑战。随着今后应用要求的不断提高，下一代网络(NGN)设计和新的宽带平台必须能在严格的带宽和时延条件下提供更强的系统性能。为了满足设计师的要求，一种特别的存储器技术——RLDRAM(低延时 DRAM) 脱颖而出，已经成为适合网络应用的高性价比解决方案。

RLDRAM 是一种先进的存储器解决方案，由 Micron 和 Infineon 合作开发。目前主流的应用是第二代 RLDRAM-II。由于采用了一种独特的内部存储架构，允许快速的随机访问，其自由存取延时仅为 15-20ns，而传统的 SRAM 延时超过 40ns，这是它最大的特色。RLDRAM-II 的其它主要特点如下：

- 操作简单，类似 SRAM。
- 由 8 个专门针对高速数据传输优化的 Bank 构成。与典型的 4 个 Bank 的 DRAM 架构相比，这种架构的 Bank 尺寸小了许多。减小 Bank 尺寸可以使 RLDRAM II 器件获得较高的峰值带宽，同时减少随机访问冲突的可能性。
- 密度为 288Mb 和 576Mb 两种。
- 数据吞吐量大。在时钟频率为 400Mhz 时可提供 28.8Gbps 的吞吐量。
- 2、4 和 8 可编程突发长度。
- 可以配置为分离 I/O(SIO)和公共 I/O(CIO)。SIO 架构的地址和数据总线是分开的，这种架构支持读和写周期之间最高的总线周转时间，因此可以消除总线竞争；CIO 架构提供公共的地址和数据总线，这种架构被认为对于

读或者写占总线操作最高百分比的应用是最佳配置，CIO 架构还具有高性能存储器件中可能是最少的管脚数量。

- 9、18 和 36 位宽数据总线可以提供实现奇偶校验/ECC 所需的额外的位，奇偶校验/ECC 是网络和通信应用所要求的。
- 32ms 刷新时间。
- 写数据支持字节掩码（Data Mask）。
- 片上延时锁定环（DLL），消除时钟和数据之间存在的较大片上偏移，支持精确的输出数据安排。
- 智能片上终结(ODT)功能，可以在高速系统设计中产生更好的信号完整性。
- 25 欧到 60 欧可编程输出阻抗以匹配 HSTL 或 SSTL I/O。
- 内核需要 2.5V 和 1.8V 电压，I/O 电平是 1.8V 或 1.5V。
- 144-ball 11*18.5mm 大小的 FBGA 封装。

1. 操作命令

RLDRAM-II 的操作命令包括：NOP、MRS、READ、WRITE、AREF。RLDRAM 通过 CS#、WE#、REF#三根低有效的信号组合表示上述命令。如下表所示：

Operation	Code	CS#	WE#	REF#	A0–An^2	BA0–BA2	Notes
Device DESELECT/no operation	DSEL/NOP	H	X	X	X	X	
MRS	MRS	L	L	L	OPCODE	X	3
READ	READ	L	H	H	A	BA	4
WRITE	WRITE	L	L	H	A	BA	4
AUTO REFRESH	AREF	L	H	L	X	BA	

Notes: 1. X = "Don't Care;" H = logic HIGH; L = logic LOW; A = valid address; BA = valid bank address.
2. n = 21.
3. Only A0–A17 are used for the MRS command.
4. Address width varies with burst length; see Table 19 on page 34 for details.

RLDRAM-II 的操作特点如下：

- 不存在 Bank 的激活和关闭操作。
- 写/读操作中，操作单元只与地址总线 A[21:0]，Bank 总线 BA[2:0]和突发长度 BL 有关。
- 写/读操作中，时间参数除了 tRC 之外，再考虑 tWL/tRL（写/读数据到写/读指令间的延时）即可。
- 刷新操作中，刷新单元由 RLDRAM-II 内部自动分配。从接口上看，只需要关心刷新的 Bank（BA[2:0]）即可，不需要关心地址总线。
- 刷新操作中也需要满足 tRC 的时间要求。
- 在写到读的操作中，由于 tRL 值与 tWL 值极为接近，使得 RLDRAM-II 相对于 RLDRAM-I 来说 DQ 总线利用率大幅提升。

2. 初始化

RLDRAM-II 必须按照规定的程序进行上电和初始化。从实际经验上看，正确上电初始化对于对于 RLDRAM-II 能否正常工作十分重要。本文档简要介绍一下 RLRAM 上电和初始化的过程。因厂家的 Datasheet 会有更新，因此设计时以厂家最新的 Datasheet 为准。

(1) 按照 Vext、Vdd、Vddq、Verf、Vtt 的顺序上电，电压稳定之后提供时钟。在 Command 管脚上提供 NOP 命令。

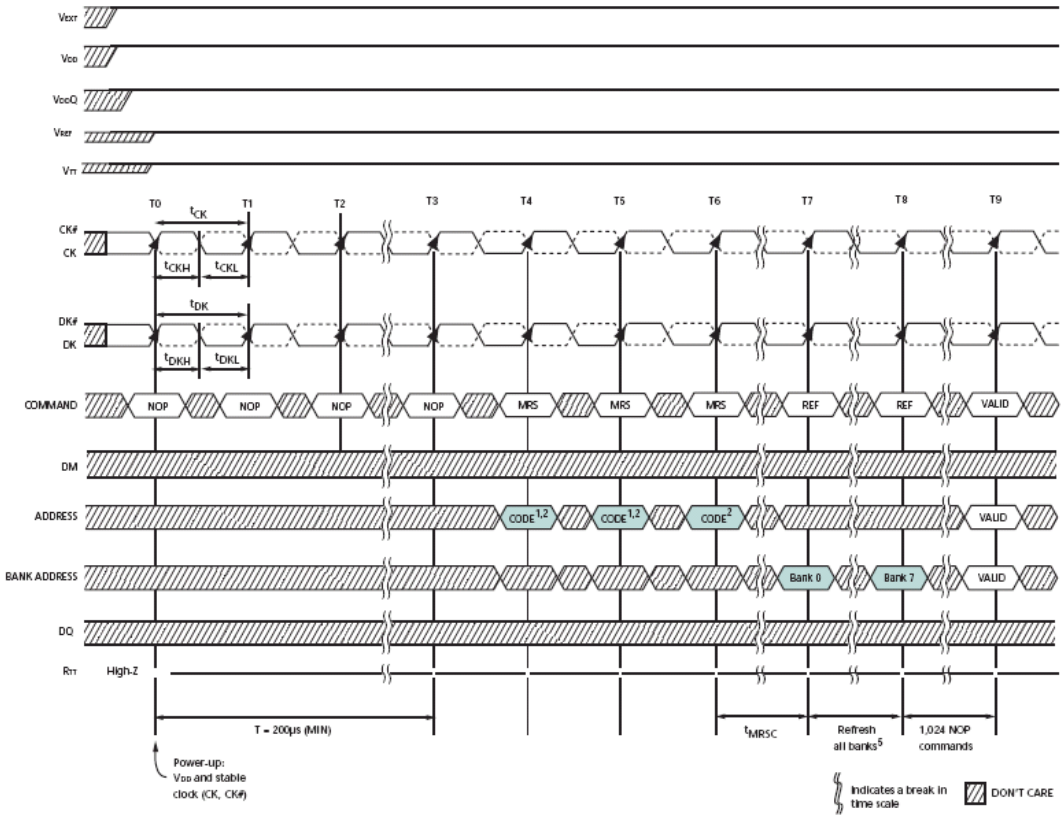
(2) CS 信号上电之后即保持高电平状态，避免任何不确定的指令写入 RLDRAM。

(3) 提供的 CK 时钟稳定后，保持稳定状态至少 200us。

(4) 提供至少三次连续的 MRS 指令：由两个或者多个 Dummy 加上一个有效的 MRS 组成。目的是对 RLDRAM 进行复位和模式配置。

(5) 过 tMRS 时间，对全部 8 个 Bank 进行自刷新，再过 1024 个 NOP 命令时间之后才能开始正常操作。

RLDRAM-II 上电和初始化图示如下：



3. RLDRAM II 的选择

如前所述，RLDRAM II 器件分两种类型，分别是分离 I/O(SIO)和公共 I/O(CIO)：SIO 架构的地址和数据总线是分开的，这种架构支持读和写周期之间最高的总线周转时间，因此可以消除总线竞争；CIO 架构提供公共的地址和数据总线，这种架构被认为对于读或者写占总线操作最高百分比的应用是最佳配置，CIO 架构还具有高性能存储器件中可能是最少的管脚数量。

设计师必须注意选择突发长度(2,4 或 8)和合适数量的 Bank 才能获得最优的总线使用率。tRC 定义了任意一个 Bank 被访问的频繁程度，而突发长度则用来确定新 Bank 访问所需的频繁程度。这两个变量都可以用模式寄存器编程设置。使用不同突发长度的 SIO 和 CIO 器件之间的性能差异如下表所示。

Part Number	Description
MT49H16M18CFM-xx	16 Meg x 18 RLDRAM II (SIO)
MT49H8M36FM-xx	8 Meg x 36 RLDRAM II (CIO)
MT49H16M18FM-xx	16 Meg x 18 RLDRAM II (CIO)
MT49H32M9FM-xx	32 Meg x 9 RLDRAM II (CIO)
MT49H8M36FM-IT	8 Meg x 36 RLDRAM II (CIO - DLP)

Note: "xx" denotes speed grade designation.

在 1:1 读写比例下，对于 4 和 8 字的突发长度，SIO RLDRAM II 可以达到 100%的总线使用率，这种情况一般出现在帧和数据包缓冲应用中。极端情况下，总线使用率会降至 50%。对于 CIO 器件，在 2 字突发长度和 1:1 读写比例时，总线使用率最低，为 67%，其他情况下的使用率都更高。4 字长度的使用率不低于 80%，8 字突发长度的使用率为 89%。16 字和 32 字数据连续操作可达到更高的总线利用率，分别是 94%和 97%。即使是使用 CIO 总线，总线周转周期也会因为最优数据时序安排而变得更小。使器件的写延时(WL)比读延时(RL+1)长一个周期就实现。

2.3.8 DDR

1. 概述

DDR SDRAM 是 Double Data Rate SDRAM 的缩写，是双倍速率同步动态随机存储器的意思。DDR 内存是在 SDRAM 内存基础上发展而来的，仍然沿用 SDRAM 生产体系，因此对于内存厂商而言，只需对制造普通 SDRAM 的设备稍加改进，即可实现 DDR 内存的生产，可有效的降低成本。

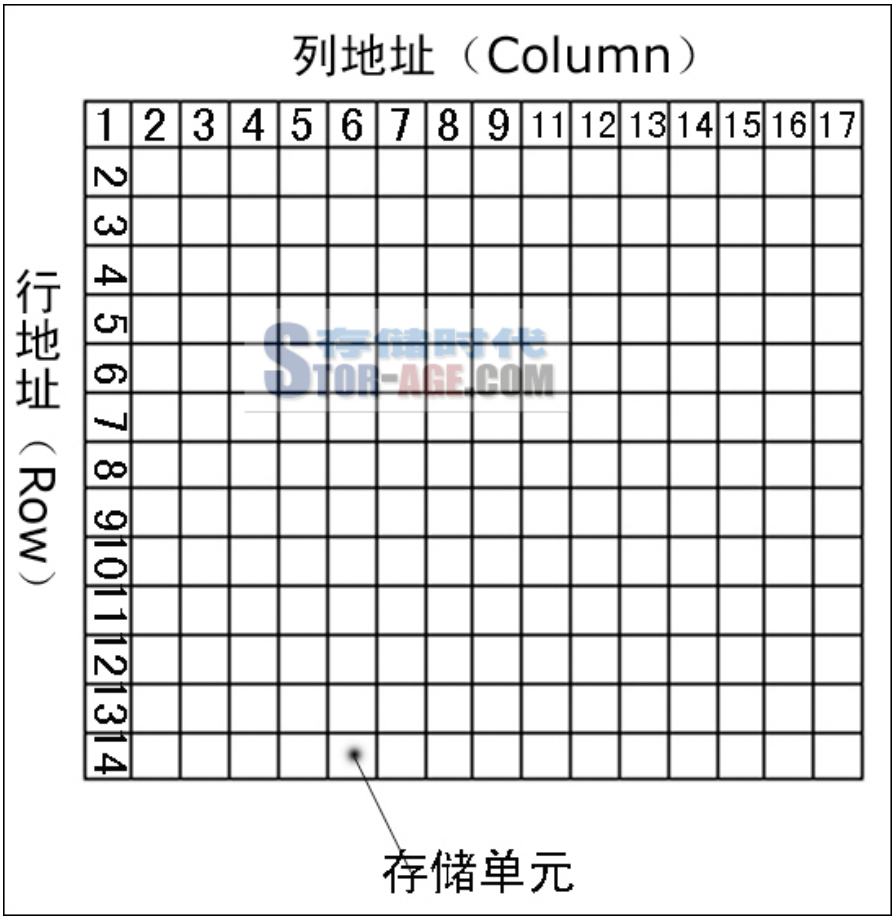
SDRAM在一个时钟周期内只传输一次数据，它是在时钟的上升期进行数据传输；而DDR内存则是一个时钟周期内传输两次次数据，它能够在时钟的上升期和下降期各传输一次数据，因此称为双倍速率同步动态随机存储器。DDR内存可以在与SDRAM相同的总线频率下达到更高的数据传输率。

与 SDRAM 相比：DDR 运用了更先进的同步电路，使指定地址、数据的输送和输出主要步骤既独立执行，又保持与 CPU 完全同步；DDR 使用了 DLL(Delay Locked Loop，延时锁定回路提供一个数据滤波信号)技术，当数据有效时，存储控制器可使用这个数据滤波信号来精确定位数据，每 16 次输出一次，并重新同步来自不同存储器模块的数据。DDL 本质上不需要提高时钟频率就能加倍提高 SDRAM 的速度，它允许在时钟脉冲的上升沿和下降沿读出数据，因而其速度是标准 SDRAM 的两倍。

2. 存储结构

要理解 DDR 的存储结构，需要首先理解三个关键概念：逻辑 BANK，行（row）和列（column）。

简单地说，SDRAM 的内部是一个存储阵列。阵列就如同表格一样，将数据“填”进去，你可以把它想象成一张表格。和表格的检索原理一样，先指定一个行（Row），再指定一个列（Column），我们就可以准确地找到所需要的单元格，这就是内存芯片寻址的基本原理。对于内存，这个单元格可称为存储单元,那么这个表格（存储阵列）叫什么呢？它就是逻辑 Bank.BANK 存储阵列示意图如下所示：



由于技术、成本等原因，不可能只做一个全容量的 L-Bank，而且最重要的是，由于 SDRAM 的工作原理限制，单一的 L-Bank 将会造成非常严重的寻址冲突，大幅降低内存效率。所以在 SDRAM 内部分割成多个 L-Bank，较早以前是 2 个/4 个，目前 DDR3 的 L-Bank 的数量已经提高到了 8 个。这样，在进行寻址时就要先确定是哪个 L-Bank，然后再在这个选定的 L-Bank 中选择相应的行与列进行寻址

3. 接口信号

重要接口信号描述如下（完整接口信号请参考 DDR datasheet）：

Name	Direction	Description
DDR_CKE	intput	DDR SDRAM clk enable signal, CKE HIGH activates and CKE LOW deactivates the DDR internal clock
DDR_CK DDR_CK_N	input	differential clock outputs for DDR SDRAM
DDR_CS_N	input	Chip select signal for DDR SDRAM
DDR_RAS_N DDR_CAS_N DDR_WE_N	input	DDR_RAS_N , DDR_CAS_N , DDR_WE_N (along with DDR_CS_N) define the command being entered to DDR SDRAM
DDR_BA[2:0]	input	Bank address inputs for DDR SDRAM
DDR_ADDR[15:0]	input	row or column address inputs for DDR SDRAM
DDR_DQ[15:0]	inout	Data input/output of DDRSDRAM
DDR_DQS[1:0]	inout	Data strobe of DDR SDRAM
DDR_DM[1:0]	input	output data mask of DDR SDRAM

4. 访问命令

对 DDR 的操作需要使用读,写,刷新等等各种命令,下面将常用的几个命令描述如下表:

Name (Function)	CS_N	RAS_N	CAS_N	WE_N	Address
不选择 DDR (NOP)	H	X	X	X	X
无操作 (NOP)	L	H	H	H	X
ACTIVE(打开行，ADDR 提供行地址 BA 提供 bank 地址)	L	L	H	H	Bank/row
READ (读 ADDR 提供列地址 BA 提供 bank 地址)	L	H	L	H	Bank/col
WRITE (写 ADDR 提供列地址 BA 提供 bank 地址)	L	H	L	L	Bank/col
BURST TERMINATE（突发终止）	L	H	H	L	X
PRECHARGE (关闭 bank)	L	L	H	L	Code
AUTO REFRESH or SELF REFRESH（自动刷新或自刷新）	L	L	L	H	X
LOAD MODE REGISTER （写模式寄存器）	L	L	L	L	Op-code

DDR 在读写某一个指定地址之前，需要“打开”相应的行，而且每个 bank 同时只能有一个行打开。形象的说，一个 4bank DDR 就像一个有着四个独立单元的楼房一样，楼房中有多个房间，而每个房间内依次存放着多列的物品。每个单元相当于一个 bank,每个房间相当于一个 DDR 行，这个楼房的每个单元只有一把钥匙，这把钥匙只能在所有房门都关闭时打开任意一个房间的门，如果已经有打开的房间了，在打开下一个房间时，必须先关闭已打开者，然后才能打开新的房间。

也就是说，如果我们想在第 i 单元的第 j 房间的第 k 列存放或者拿出物品，就必须先“打 开”第 i 单元的第 j 房间，然后再到第 k 列存放或者拿出物品；而且，在打开时，需要保证第 i 单元没有其他房间被打开，否则，需要先“关闭”其他已打开房间，再打开第 j 房间。这个“打开”就是 DDR 的 ACTIVE 命令，在 ACTIVE 命令的同时，需要由 BA 给出 bank 地址，并由 ADDR 给出行地址，而且需要保证打开的那个 bank 处于空闲状态（就是说所有的行都关闭）。一个 ACTIVE 命令如图 4.1 所示。而“拿出”或“存放”物品则对应 DDR 命令的“读”、“写”命令，这两个命令都只能对已打开行，在进行“读”、“写”时，由 BA 给出 bank 地址、由 ADDR 给出列地址，而在表 2 中可以看出，读和写由 WE_N 区别。读写完毕后，如果不再需要操作该行时，需要“关闭”该 bank，就是运行“PRECHARGE”命令，PRECHARGE 命令实施时，如果地址总线的第十一位 ADDR[10]为 0，则表示关闭 BA 指定的 bank，否则表示关闭所有 bank。

5. 重要时序参数

对 DDR 的访问需要遵循严格的时序要求，这些要求由一系列重要的时序参数来体现，下面对部分重要时序参数进行详细描述：

Parameter	Description
CL	CAS Latency Control ,CAS latency 是“内存读写操作前列地址控制器的潜伏时间”。CAS 控制从接受一个指令到执行指令之间的时间。
tRP	Row Precharge Timing，表示"内存行地址控制器预充电时间"，预充电参数越小则内存读写速度就越快。 tRP 用来设定在另一行能被激活之前 ,RAS 需要的充电时间。
tRCD	RAS to CAS Delay，表示"行寻址到列寻址延迟时间"，数值越小，性能越好。

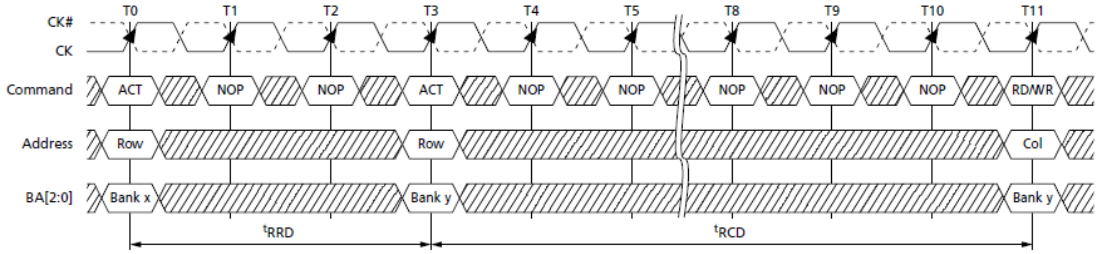
tRRD	Row to Row Delay，也被称为 RAS to RAS delay，表示"行单元到行单元的延时"。该值也表示向相同的 bank 中的同一个行单元两次发送激活指令(即：REF 指令)之间的时间间隔。tRRD 值越小越好。 延迟越低，表示下一个 bank 能更快地被激活，进行读写操作。
tRC	Row Cycle Time，表示“SDRAM 行周期时间”，它是包括行单元预充电到激活在内的整个过程所需要的最小的时钟周期数，也是同一个 bank 两次激活命令之间的最小间隔时间；
tRFC	Row Refresh Cycle Time，表示“SDRAM 行刷新周期时间”，它是行单元刷新所需要的时钟周期数。该值也表示向相同的 bank 中的另一个行单元两次发送刷新指令(即：REF 指令)之间的时间间隔。tRFC 值越小越好，它比 tRC 的值要稍高一些。
tWR	Write Recovery Time (tWD)，表示“写恢复延时”。该值说明在一个激活的 bank 中完成有效的写操作及预充电前，必须等待多少个时钟周期。
tWTR	Write to Read Delay，表示“读到写延时”。三星公司称其为“TCDLR (last data in to read command)”，即最后的数据进入读指令。它设定向 DDR 内存模块中的同一个单元中，在最后一次有效的写操作和下一次读操作之间必须等待的时钟周期
tREF	Refresh Period，表示“刷新周期”。它指内存模块的刷新周期,即最少相隔多少个 cycle 需要发送一次刷新命令；计算方式是用 DDR 颗粒的刷新周期（32ms/64ms）除以行数（8192）
tWCL	Write CAS Latency，表示“写指令到行地址控制器延时”
BL	Burst Length，突发长度。表示一次读/写命令访问的存储单元的个数，例如某个 ddr 颗粒位宽 16bit，BL8 表示突发长度为 8，则每次访问的数据大小是：16*8=128bit。对于如今市场上流行的 ddr3 来说 BL=8 是最常用的工作模式。

tFAW	DRAM Four Active Window 的缩写，DDR 内存的一个时序参数，定义了同一 Rank 中允许同时发送大于四个行激活命令的间隔时间，因此最小值应该不小于 tRRD(RAS to RAS Delay)的四倍

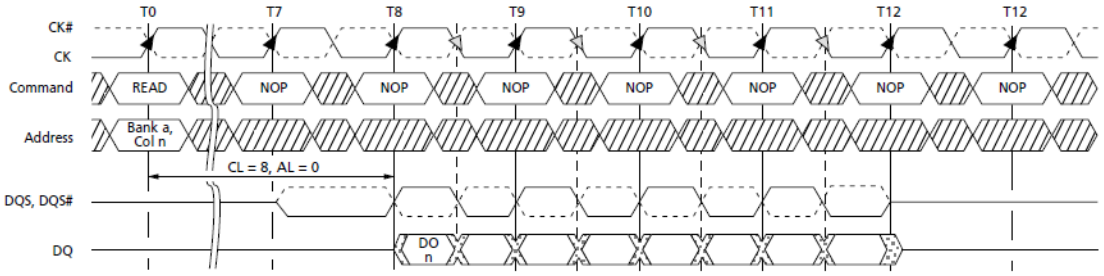
6. 主要命令时序

本节描述几个主要命令的时序图，从时序图中也可以看出某些关键时序参数的物理意义以及它们之间的关系。

ACTIVATE Operation（激活命令）

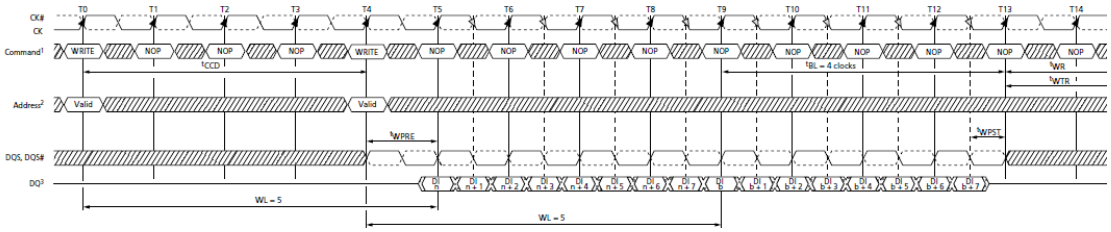


READ Operation（读命令）



WRITE Operation（写命令）

Figure 86: Consecutive WRITE (BL8) to WRITE (BL8)



2.3.9 AMBA2.0 总线

一个基于 AMBA2.0 架构的 SOC，一般来说包含了高性能的 AHB 系统总线与低带宽的 APB 外设总线。系统总线是负责连接例如 ARM 之类的嵌入式处理器、DMA 控制器（DMAC）、片上 memory 和片外存储器接口，或其它高带宽需求的组件；而外设总线则是用来连接系统的低速外设组件。其协议相对 AHB 来讲

较为简单，与 AHB 之间则通过 AHB2APB 桥相连，期望能减少系统总线的负载。一个典型的 AMBA2.0 总线架构如下图所示：

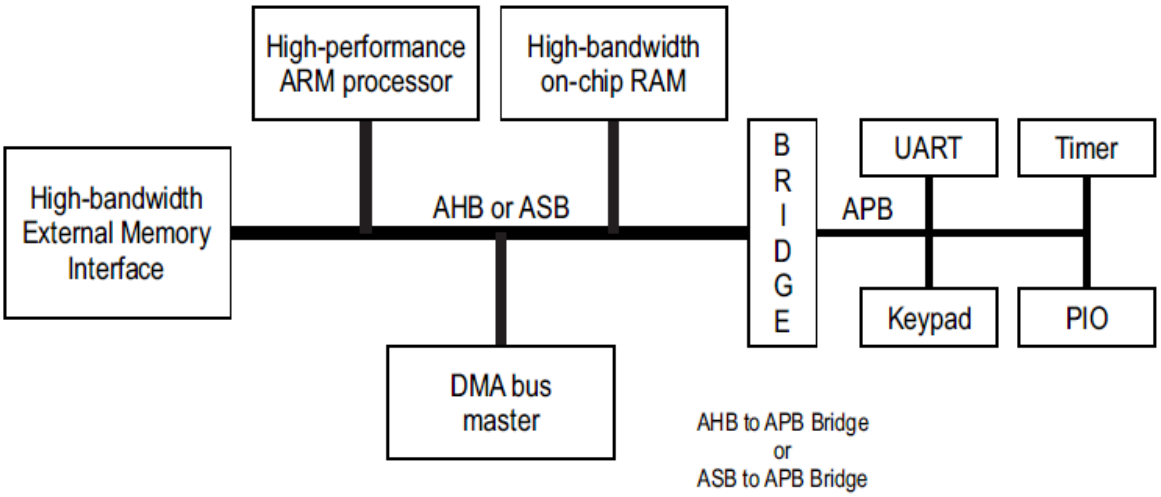


图2-75 典型的 AMBA2.0 总线的体系结构示意图

以下为 AMBA2.0 中基本常用的术语名词：

- Bus cycle：总线时钟周期的基本单位，用于方便描述 AHB、APB 总线协议。一个 clock 的上升沿到下一个上升沿称作 bus cycle。
- Bus transfer(transaction)：对数据目标的一次读写操作，可能会花费一个或者多个 bus cycle。Transfer 的结束由被寻址访问的 slave 的响应为标志。
- Burst transfer：有 bus master 发起的一个 or 多个 bus transaction，其对递增的地址空间范围有着一致宽度的 transaction。

2. AHB 总线

(1) AHB 总线概述

AMBA2.0 的 AHB 能满足高性能、高时钟频率的特性需求。以下为其一些基本特性：

- burst transfer
- split transactions
- single cycle bus master handover
- single clock edge operation
- wider data bus configuration

AHB 总线系统是由 Master、Slave、Infrastructure（俗称 Bus matrix）三部分所组成。整个 AHB 总线上的传输都是由 master 发起，slave 负责响应。而 infrastructure 则由仲裁器、master2slave、multiplexer、slave2master multiplexer、译码器、dummy slave、dummy master 组成。

AHB 总线系统之所以会需要仲裁器，是因为它支持单层总线上的 multiple master，因此需要仲裁器来仲裁总线使用权限。而译码器则是负责地址的译码，从 multiple slave 中选择要响应传送的 slave 。而两个 multiplexer 则是负责总

线的路由(这样可避免使用 tri-state 双向总线)，将总线上的信号在 master 和 slave 之间传送，下图说明了 multiplexer 与 master、slave 之间连结的情形：

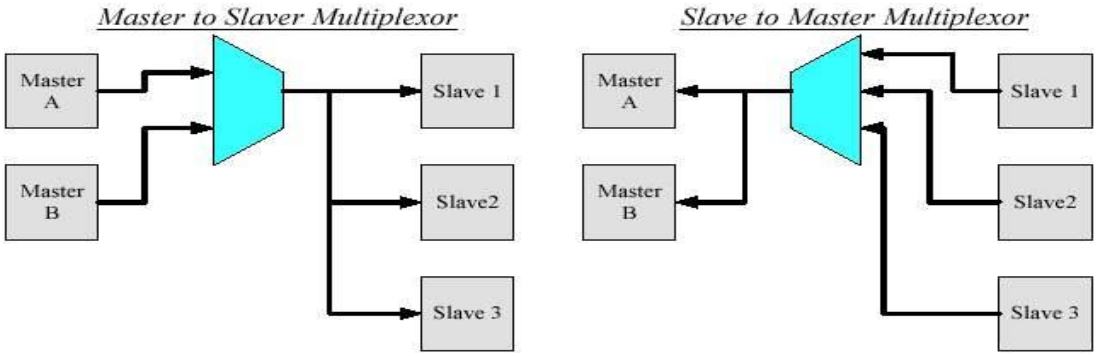


图2-76 AMBA2.0 中的 multiplex 示意图

AHB 总线上传输的信号，可以分成 clock 与 reset、仲裁、地址、控制信号、写数据、读数据、响应信号七种。除了 clock/reset 与仲裁信号之外，其余的信号皆会经过 multiplexer。其中经过 master2slave multiplexer 的信号有地址、控制信号、写数据；而经过 slave2master multiplexer 的则有读数据、响应信号。基本示意图如下：

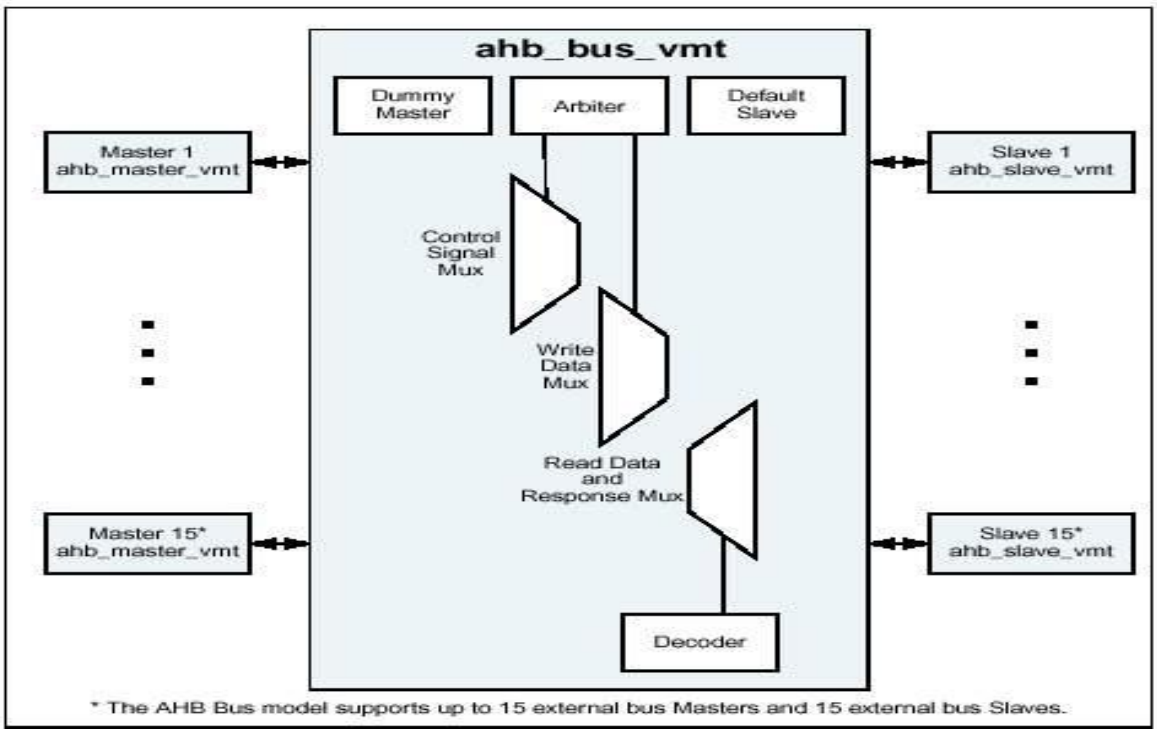


图2-77 AMBA2.0 中的组件示意图

AHB 总线中相关的信号列表如下：

表2-23 AHB 总线信号列表

Signal Name	Source	Description

HCLK	系统时钟复位产生电路	总线时钟信号，上升延触发
HRESETn	系统时钟复位产生电路	总线复位信号，低电平有效
HADDR[31:0]	Master	当前传送中的32bit地址总线
HTRANS[1:0]	Master	当前传送类型
HWRITE	Master	当前传送方向。1：write传送；0：read传送
HSIZE[2:0]	Master	当前传送位宽
HBURST[2:0]	Master	当前突发传送指示
HPROT[3:0]	Master	当前传送中的保护指示
HWDATA[31:0]	Master	写数据总线
HSELx	译码器	slave select信号
HRDATA[31:0]	Slave	读数据总线
HREADY	Slave	传送完成指示。1：传送done；0：extending传送
HRESP[1:0]	Slave	当前传送响应
HBUSREQx	Master	总线请求
HLOCKx	Master	锁定传送
HGRANTx	仲裁器	bus grant指示
HMASTER[3:0]	仲裁器	被授权的master指示
HMASTLOCK	仲裁器	锁定指示
HSPLITx[15:0]	Slave Split	split传输完成指示

AHB 总线系统中大概的总线互联示意图如下。在图中省略的部分有：

- 各种控制信号 (HBURST, HTRANS 等)的连接，其连接实际与 HADDR 一致；
- Master 与仲裁器之间的 request/grant 信号；
- 译码器与各个 Slave 间会有 Selection 的信号；
- 控制 multiplexer 的 output 会有部分控制信号除了接到 Slave 外也会接到仲裁器 (HTRANS/HBURST)；
- 响应信号(HREADY, HRESP)的多路器，另外仲裁器会输出 HMASTER 信号，这个信号会接到 master2slave multiplexer，以作为 selection 信号。

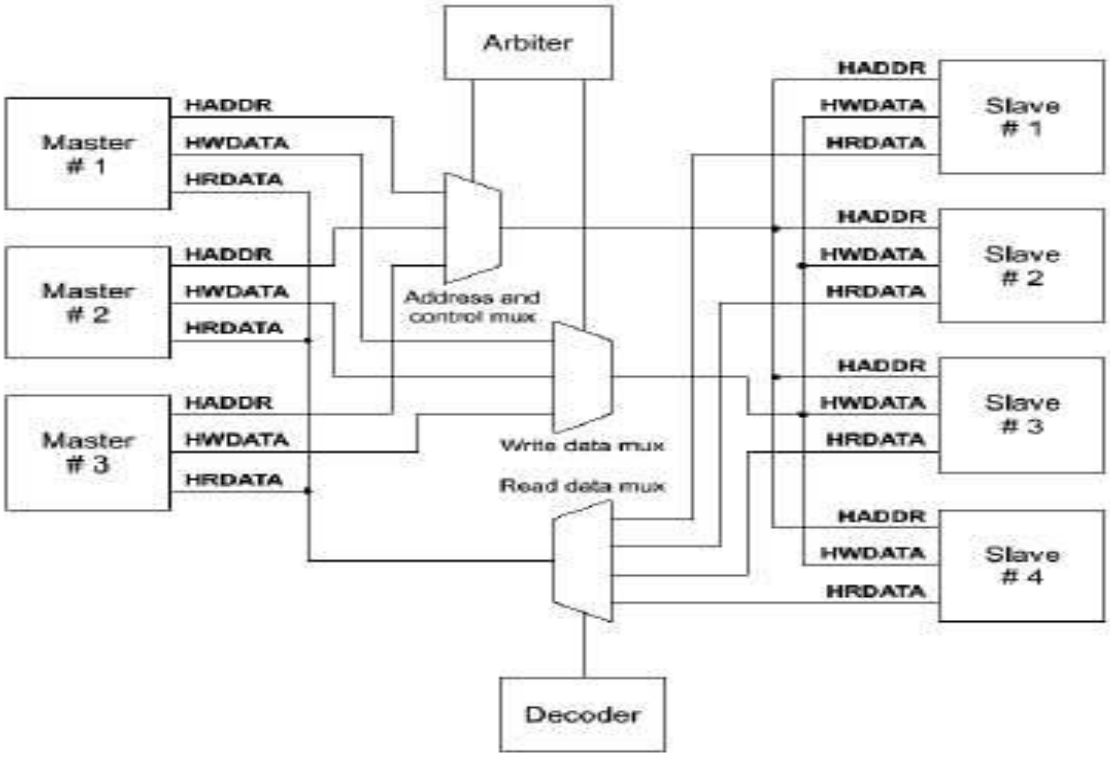


图2-78 AHB 总线信号之间的基本互联示意图

(2) AHB 总线数据传输

在 AHB 总线上，一次完整的传送可以分成两个阶段：地址阶段与数据阶段。地址阶段传送的是地址与控制信号，而数据阶段则是读写数据与响应信号。下图给出了 AHB 上的基本传输示意图：

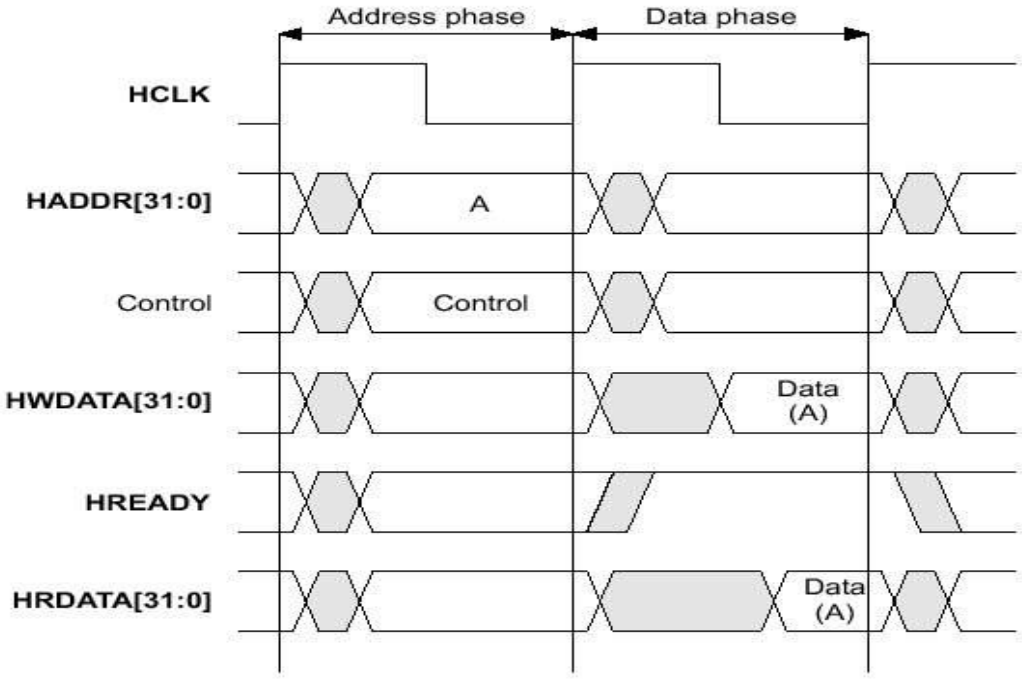


图2-79 AHB 总线基本传送周期示意图

传送过程在数据阶段时若无法在 1 个时钟周期内完成，slave 可用 HREADY 信号去延长(extend) 传送，如下图所示。当 HREADY 为低时，表示传送尚未结束；为高时，则代表目前的传送结束了。但该次传输是否完成则需看 Slave 响应的 HRESP 信号(可能是 OKAY、ERROR 等)。

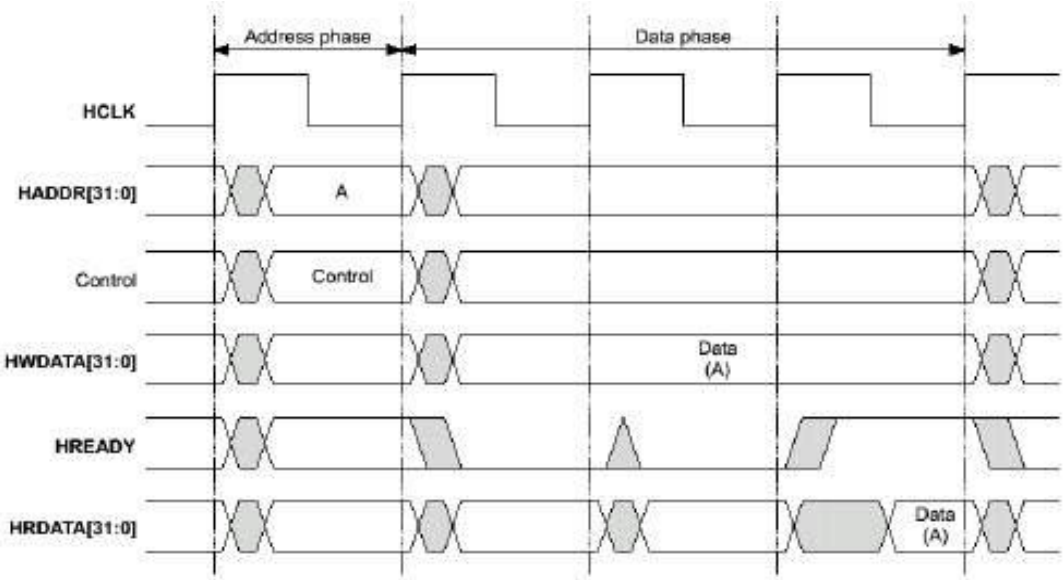


图2-80 AHB 带有等待周期的传输示意图

由于一次传送需要两个阶段才能完成，为了增进总线的性能，AHB 总线将多笔业务的传送给进行水操作，传送过程中地址阶段和数据阶段是交叠在一起的，如下图所示。从图中可以看到由于当前传送的数据阶段与下一次传送的地址阶段是交叠的，所以当前传送的数据阶段被延长时，地址阶段也得跟着延长。

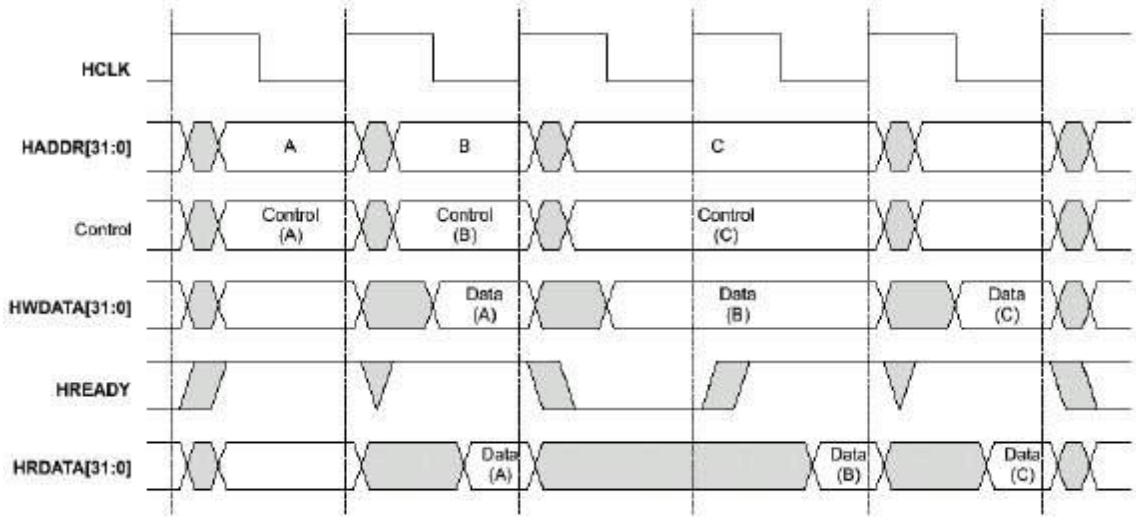


图2-81 AHB 多个传输周期的传送流水示意图

(3) AHB 总线响应

在 AHB 协议中，slave 除了可以使用 HREADY 信号去延长传送(插入几个等待周期)外，在传送结束时(HREADY 在数据相位为高)，Slave 还可以使用 HRESP[1:0]去告诉 master 传送结束时的状态。在 AHB 里传送结束时可以表示的状态共有四种，OKAY、ERROR、RETRY、SPLIT。

表2-24 AHB 总线响应 HRESP 说明表

HRESP[1:0]	Type	Description
00	OKAY	传送成功的完成
01	ERROR	传送失败，失败的可能原因为，譬如说企图写入read-only的memory location，或读写根本不存在的memory location等。2-cycle响应。
10	RETRY	当前传输尚未完成，bus master仍然需要retry传输，直至完成。2-cycle响应。
11	SPLIT	当master被再次授权访问总线时，必须retry传输；当slave能够完成传输时，必须通知master。2-cycle响应。

- RETREY 响应。仲裁器内 master 的优先权不变，当有更高优先权的 master 发出请求时，总线访问的权力会由高优先权的 master 取得；但如果原来得到 RETRY 响应的 master 是当时请求总线的 master 中拥有最高优先权者，则总线还是会继续被占住而法释放给其它有需要的 master。
- SPLIT 响应。当仲裁器观察到 master 收到 SPLIT 响应时，则会将 master 的优先权给屏蔽起来，当屏蔽后，master 将无法再获得总线访问的权力，即使已经没有其它 master 向仲裁器发出请求也一样。如果所有的 master 都收到 SPLIT 响应，则仲裁器会把总线访问的权力给 dummy master (只会发出 IDLE 传送的 master)。当回应 SPLIT 的 slave 处理完传送的要求后，会发出 HSPLIT 信号给仲裁器，则仲裁器会将 master 的优先权取消屏蔽，如此 master 的优先权就回复原状而有机会访问总线了。SPLIT 响应与 RETRY 响应比起来能让低优先权的 master 在合理的情况下(无更高优先权 master 要求总线时)取得总线的拥有权，因此可以让 AHB 总线有更好的设计，但缺点是硬件设计的复杂。首先仲裁器必须要去观察 HRESP 信号，且要有当收到 SPLIT 时变动 master 优先权的设计。再者 Slave 需要纪录 master 的数目以便以后要通知仲裁器恢复那个 master 的优先权，Slave 可以从仲裁器发出的 HMASTER 信号查得。在 AHB 系统里最多可以有 16 个 master，所以 HMASTER 是四个 bit，然而通知仲裁器可以取消屏蔽的信号 HSPLIT 因是独热编码的表示方式，所以需要 16 bit 。

另外要注意的是 AHB 并没有强制规定 Slave 需支持 Retry 或 Split 响应，Slave 可以只用 HREADY 去延迟对传送 的响应(延迟的时间无上限值，但规范中建议不要超过 16 个周期)。然而 master 则需要支持对 Retry 或 Split 响应的处理：重新再发出相同的传送。

其中 3 个非 OKAY 响应需要两个周期，这是因为 AHB 为流水操作，当前传送的数据相位与下一个传送的地址相位是交叠的。而这三种响应皆可能因为目前的传送并未成功而影响了下一个传送的存取，使的 master 需要取消预备要进行的下一个传送(在 AHB 的规定里，因为 Retry/SPLIT 是代表目前传送正在处

理，所以下一个传送必须一定要取消，而 ERROR 响应表示传送 failed，在某些情况下，master 仍然会尝试下一笔传送，因此当 master 收到 ERROR 响应时，继续存取下一笔传送是允许的，不过 ERROR 响应此时还是需要花 2 个周期)。如下图所示：

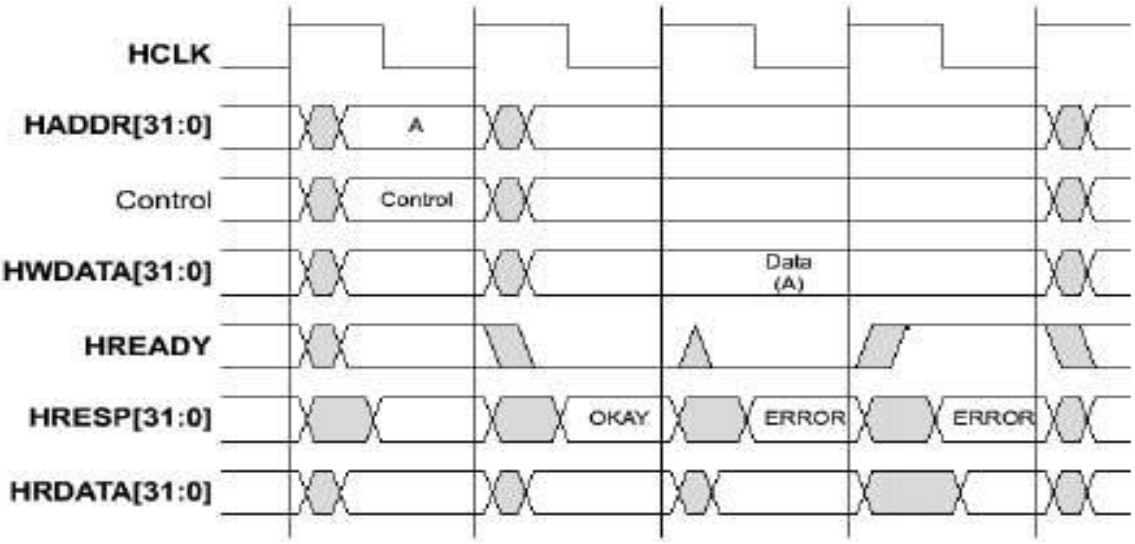


图2-82 AHB 总线中带有错误的响应示意图

(4) AHB 总线系统仲裁

由于 AHB 为 Multi-Master 系统，而同一时间只允许一个 master 去访问总线，因此需要仲裁器去做仲裁，以下先简述 AHB 仲裁机制。

当 master 想要访问总线时，master 将 HBUSREQ 信号给驱动高(每个 master 都有自己的 HBUSREQ 信号)，同一时间可能有多个 master 都想要访问总线，因此仲裁器在 HCLK 的上升延去采样各个 master 的 HBUSREQ 信号后，需决定那个发出请求的 master 有最高的优先权(AHB 并无规定优先权算法，由系统设计者自定)，然后将此 master 的 HGRANT 信号驱动高，表示其可以访问总线了。(若原本已有 master 在访问总线，仲裁器会将原本 master 的 HGRANT 信号给驱动低表示其已丧失访问的权力了)。

当 master 正进行固定长度突发传送时，如果有更高优先权的 master 发出了请求，仲裁器可以等待突发完成后再将总线 grant 给新的 master，也可以在突发进行中，就中断原有 master 的总线拥有权(ownership)，让高优先权的 master 去访问总线。而被中断的 master 就需要重新发出请求，等待下次 Grant 总线时继续完成突发传输。若 master 想要进行的连续传送是不可被中断的(譬如在访问 shared memory 时)，则 master 可以在请求时，同时将 Hlock 给驱动高，告诉仲裁器：目前进行的是不可被中断的传送，则当 master 获得访问总线权力后，仲裁器将不会再把总线释放给其它 master，直到 master 自行将 HLOCK 给驱动低，仲裁器才会再进行仲裁的动作。下图显示出基本的总线切换的情形，

图中可以看到 HMASTER 会显示出总线所有者的 master 号(每个 master 在仲裁器里皆有一个号)，另外因为流水操作的原因，master 获得数据阶段的拥有权时会比地址相位延迟一个周期 ，所以当 master 获得总线时第一个传送的地址阶段是与前一个 master 的数据阶段交叠在一起的。而当前一个 master 的数据阶段被 slave 延迟时，现在拥有地址总线的 master 也跟着被延迟了，这将在后面的时序图里介绍这样的情形。

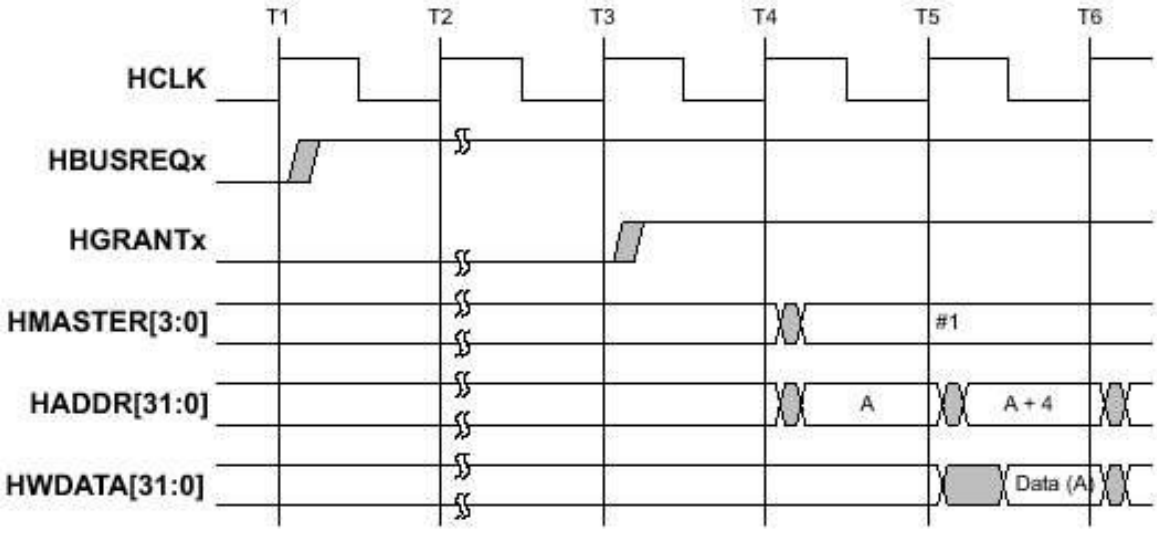


图2-83 AHB 中不带有等待态的 grant 的访问

当 master 的 HGRANT 被确认后，且 HREADY 为高，则代表 master 在下个周期就可以访问总线了。若 HREADY 的信号一直为低，而此时仲裁器接到更高优先权的 master 的请求而更改了 Grant 信号，那么原本被认可的 master 就没有访问的权力了。下图给出因为 HREADY 信号而延迟了 GRANT 的时间。

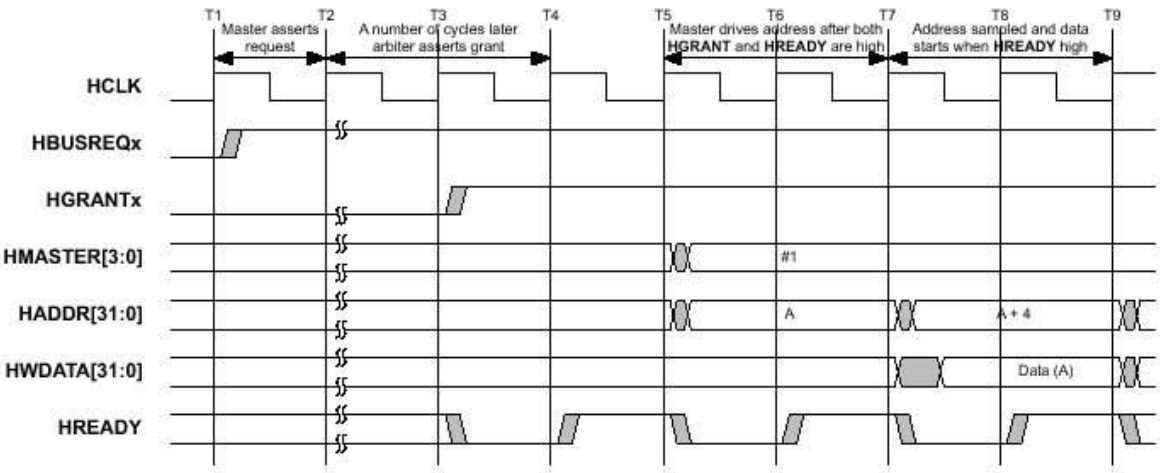


图2-84 AHB 带有等待态的 grant 的访问

下图主要描述数据总线在总线切换时转移的情形，及地址总线因前一个 master 的传送被延迟的情形。

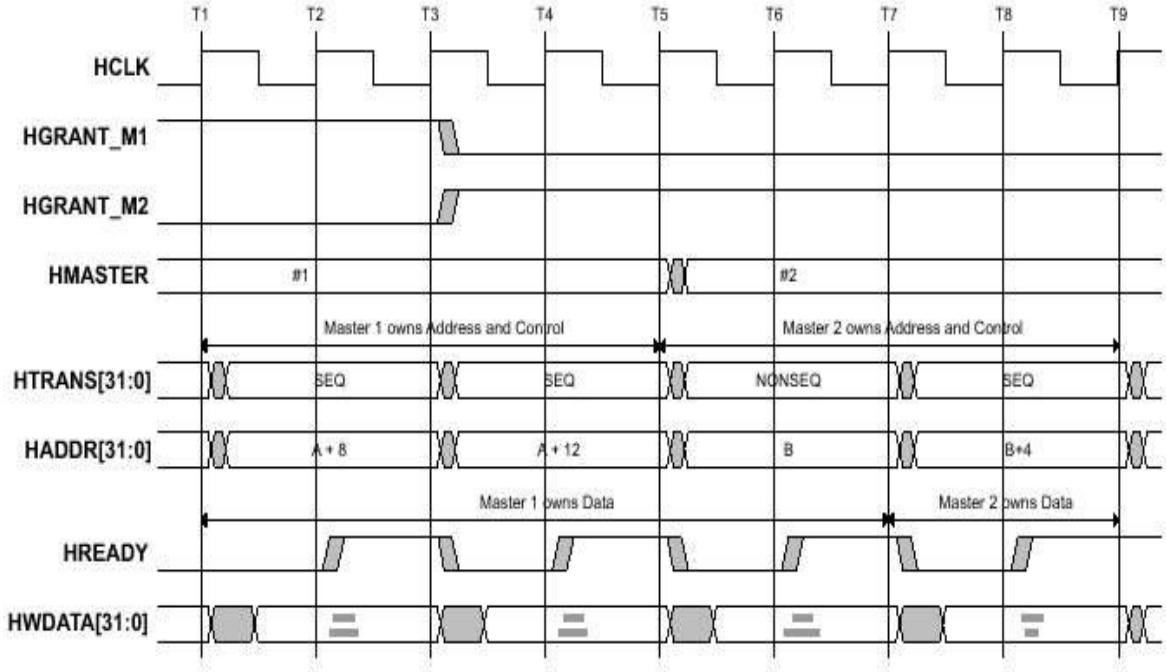


图2-85 AHB arbiter 对数据总线的所有权切换示意图

下图主要在说明 HGRANT 在总线切换时变化的情形。以突发传送的情形来说，当仲裁器决定在突发结束后，转换总线的拥有权时，仲裁器会在倒数第二个传送的地址被采样后，改变 HGRANT 信号，所以新的 HGRANT 信号将跟最后一个传送的地址一起在同一个时间被采样。

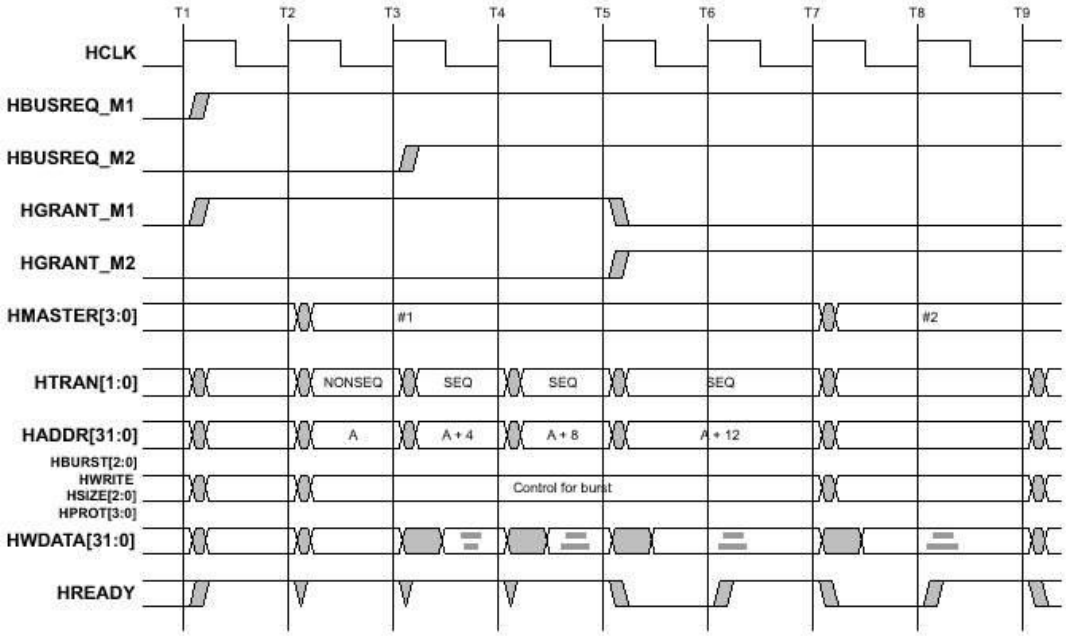


图2-86 AHB arbiter 对数据总线的所有权切换示意图（突发传输之后的切换）

在前一节介绍 RETRY/SPLIT 响应时，曾经介绍其为 2 个周期的响应，且第二个周期必须是 IDLE 传送以便让仲裁器有机会去改变总线所有权，下图就给出了 HGRANT 信号如何利用 IDLE 传送的周期去变化的示意图：

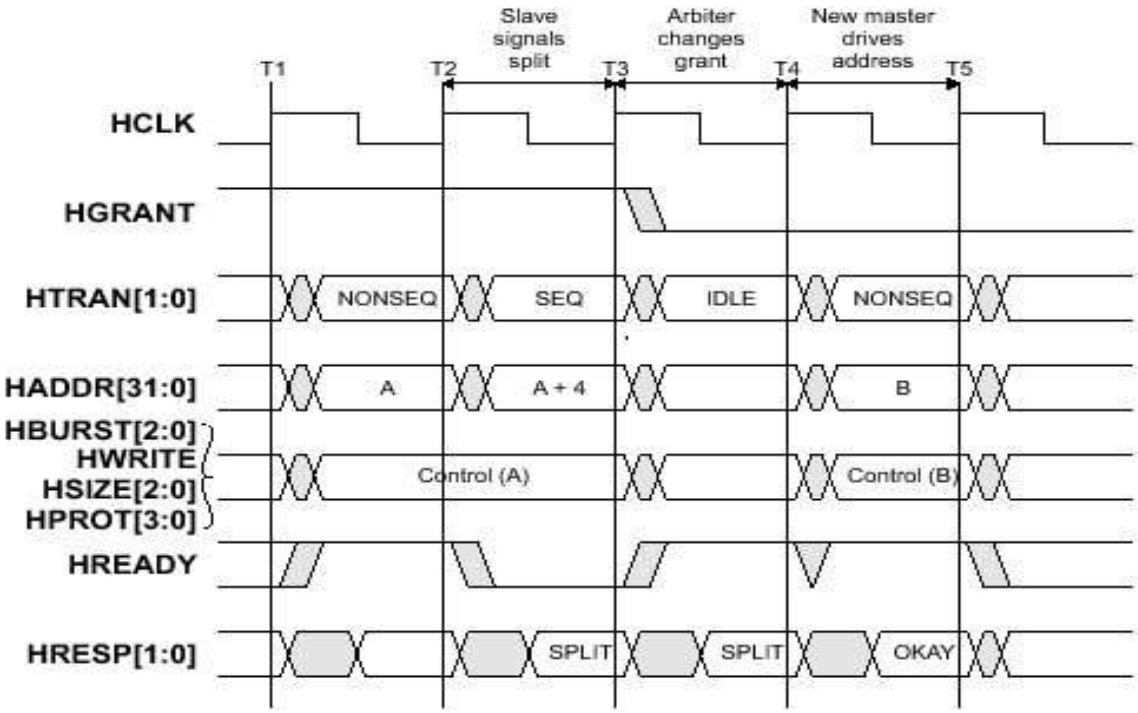


图2-87 AHB arbiter 对 split 传输之后的切换示意图

在当前的 SOC 设计中，设计者往往更倾向于使用没有上述仲裁器的 AHB lite 总线架构。其在一定程度上简化了设计，但另一方面也使得 bus matrix 的总线信号数量增加不少！

3. APB 总线

(1) APB 总线概述

AMBA2.0 APB 总线主要是用在连接低速的周边外设，例如 UART、USB 等。它的总线架构不像 AHB 为 Multi-Master，在 APB 总线里唯一的 master 就是 AHB2APB 桥，因此不需要仲裁器以及一些 request/grant 信号。AMBA2.0 APB 协议十分简单，甚至不是流水操作。下面给出了 APB 总线 2 条主要特性：

- 总是 2 个周期完成传送
- 没有等待周期和响应信号

AMBA2.0 中 APB 总线的信号名称及其相关描述如下表所示：

表2-25 AMBA2.0 中 APB 总线的信号列表

Name	Description
PCLK	APB时钟信号
PRESETn	APB复位信号，低电平有效
PADDR[31:0]	APB地址总线.
PSELx	器件选择信号，低电平有效

PENABLE	使能信号
PWRITE	传送方向指示，高电平写、低电平读
PRDATA	数据总线

AMBA2.0 APB 上的传送可以用下面的 state diagram 来说明：

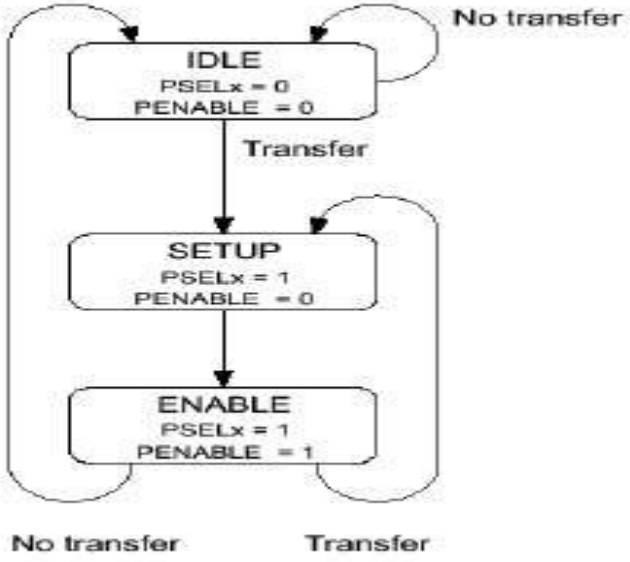


图2-88 APB 传输的状态机

- IDLE 态

此时并没有传送在进行，也没有 slave 被选择到(PSELx=0)。

- SETUP 态

当有传送要进行时，PSELx=1，PENABLE=0，进入 SETUP 态，而且只会在 SETUP 态停留 1 个周期，在 PCLK 下一个 rising edge 进入 ENABLE 态。

- ENABLE 态

在进入 ENABLE 态时，之前在 SETUP 态的 PADDR、PSEL 以及 PWRITE 皆维持不变，只有 PENABLE 被置位（assert）。传送也只会在 ENABLE 态维持 1 个周期，之后若没有传送则又进入 IDLE 态，若有连续的传送则继续进入 SETUP 态

(2) APB 总线数据传输

- 写传送

在下图中，时间 T2->T4 为 APB 完整的一个写传送时序图：T2->T3 为 SETUP 阶段，T3->T4 为 ENABLE 阶段。在整个传送中，地址/控制/数据信号都需维持不变。

在传送结束后，PENABLE 信号一定会被拉低(deasserted)，而 PSELx 信号则视情况而定：若继续对同一个 slave 进行传送，则维持不变(高)；否则，则会被驱动低。

另外为了节省功耗，若接下来没有数据传送，则地址/写信号会一直维持不变，直到又有传送发生。

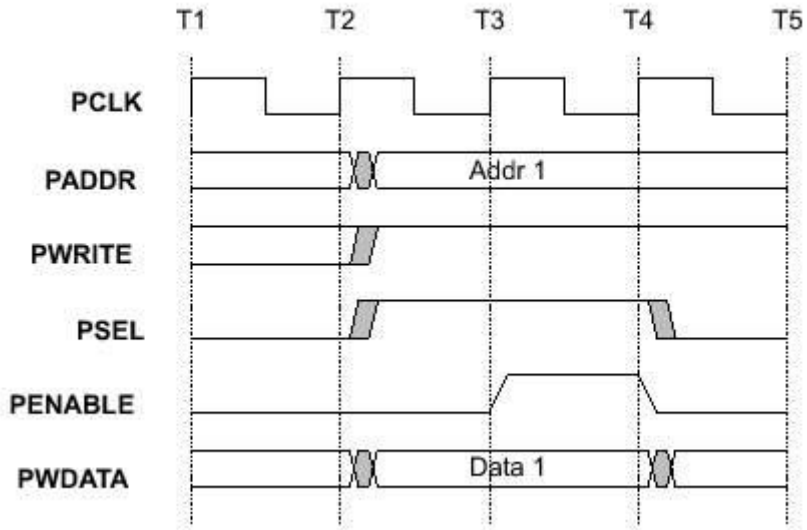


图2-89 APB 的基本写传送

- 读传送

下图为 APB 一个完整的读传送时序图，除了 PWRITE 为低外，其余的地址/PSELx 等控制信号皆与写传送相同。在读传送时，slave 必须在 ENABLE 周期提供数据给 AHB2APB 桥，该桥会在 T4 的时间点进行数据采样。

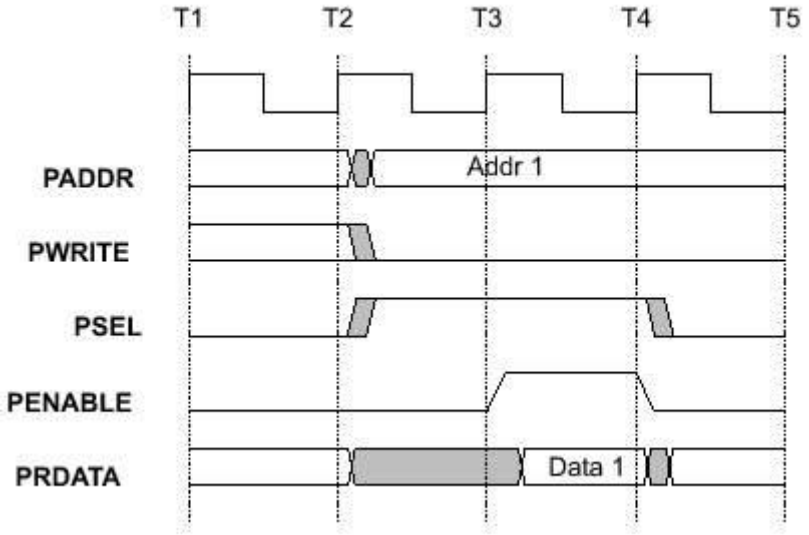


图2-90 APB 的基本读传送

4. AMBA3.0 总线

在 AMBA2.0 的基础上，目前已出现以 AXI 总线协议为代表的 AMBA3.0 系统规范。AXI 总线协议与 AHB 协议相比，在一定程度上提高了总线的利用率以及带宽，其主要特性简单列举如下：

- 地址/控制操作与数据操作分离
- Byte Strobe 信号支持非对齐传输
- 每次突发传输只需要发送传输的首地址
- 分离的读、写通道，可支持低成本 DMA
- 支持 Outstanding 地址发送功能
- 支持乱序传输
- 方便集成寄存器 slice，以满足时序要求

如需进一步了解有关 AMBA3.0 中 AXI 协议，可自行参考 AXI 总线协议的相关文档，此处不对其进行过多的描述。

此外，AMBA3.0 中对 APB 总线协议也作了一定扩展，使得 APB 也不再局限于“always 2 cycles”的操作方式。其增加了与 AHB 总线中类似的 ready 信号，使得基于 AMBA3.0 APB 的接口设计更加灵活，要进一步了解需自行查阅 AMBA3.0 APB 总线协议部分。

2.4 常用工具

概述：这里介绍了 ASIC 设计工程师常用的工具的使用入门。还可以简要介绍其他工具。

2.4.1 Debussy 入门

Debussy 是一套很好的 Verilog\VHDL 调试工具，可以帮助设计者快速理解复杂的设计（设计小组中别人开发的复杂、不熟悉的代码或者 IP），查找和定位设计中存在的 bug，提高效率。

使用范围：

- Debussy 调试系统集成了分析，验证和调试等，针对复杂 SOC、ASIC、FPGA 设计。
- 支持业内主流仿真器，如：Verilog-XL、NC-Verilog、VCS、Modelsim、Leapfrog、Ploaris……
- 支持所有设计及验证语言：
  设计语言：Verilog/Verilog-2001、VHDL/87/93/2000、SystemVerilog
  断言语言：PSL、OVA、SVA
  验证语言：e、Vera

各模块之间均可通过鼠标中键来快速选择拖放定位。

1. nTrace

非常成熟的源代码追踪工具，可以快速地穿越整个设计结构去自动地追踪信号的连接关系。

- 从源代码中提取出详尽的设计结构，在一个视图中自由切换各层次代码，便于解读代码。
- FSDB Active Annotation 动态显示信号的仿真值，以便于更轻松的理解和分析。
- Active Trace 自动地跨层次寻找相关信号值和信号转换的有效激励信号和负载信号。
- 可快速使用习惯的编辑器编辑修改源代码。
- 方便快捷的查找信号之间的连接关系，易于理解设计。

2. nSchema

一个电路原理图萃取工具。它根据相关的 RTL 或者是 Gate-level 的代码描述智能地创建层次化和扁平化的电路原理图，方便的跨越层次结构的信号追踪

- Active Annotation 动态显示信号的仿真值，以便于更轻松的理解和分析
- Trace Two Points 追踪两点间的所有路径
- 快速提取有效电路，方便信号追踪和分析，如：Browser Window、Flatten Window、Fan-in Cone、Fan-out Cone…
- 提取设计中的 Clock tree、Reset tree
- Cross Probing with Laker，与版图工具 Laker 相互定位

3. nWave

一个完整的波形显示工具，提供了一个易于理解和非常直觉化的视窗来帮助工程师分析信号。

- 波形格式为 FSDB，比 VCD 格式小 1/5~1/50，比 SHM 小 1/3，压缩量大，加载速度快。
- 双击错误信号跳变，自动追踪到源代码有效的驱动语句。
- Zero time glitch 显示设计中存在的毛刺，并使用 EventSequence 分析成因。
- 可对两个仿真结果进行信号对比。
- 波形中查找任意数值，字符串，甚至某个跳变。。
- 支持模拟仿真结果，进行 Mixed-Signal 波形调试。
- 创建 complex event。

4. nState

有限状态机提取及分析

- 提取整个设计中所有 FSM，自动追踪分析各 FSM 间存在的转换关系。
- 与 nWave 同步，便于理解状态转移情况。
- 自动生成有限状态转移报告。

2.4.2 nLint 入门

1. 功能、特点概述

nLint 是一个检查 HDL 代码的工具，能够帮助逻辑设计者发现代码的语法以及语义错误。nLint 主要分为图形界面模式（GUI Mode）和批处理模式（Batch Mode）。nLint 是一个独立的 EDA 工具。它支持 User-defined Rules(UDR)检查，基于 TCL 界面，用户能够编写程序调整设计信息和设计规则。nLint 同时也提供一个完整的 user-friendly 环境来检查和修改错误。用户还能自建编辑器来编辑源代码和查看错误。编辑后，用户可以保存编辑后的源代码重新运行 nLint。这些功能都可以在 nLint GUI 模式下得到实现。下面对图形界面模式做一个简单的使用描述。

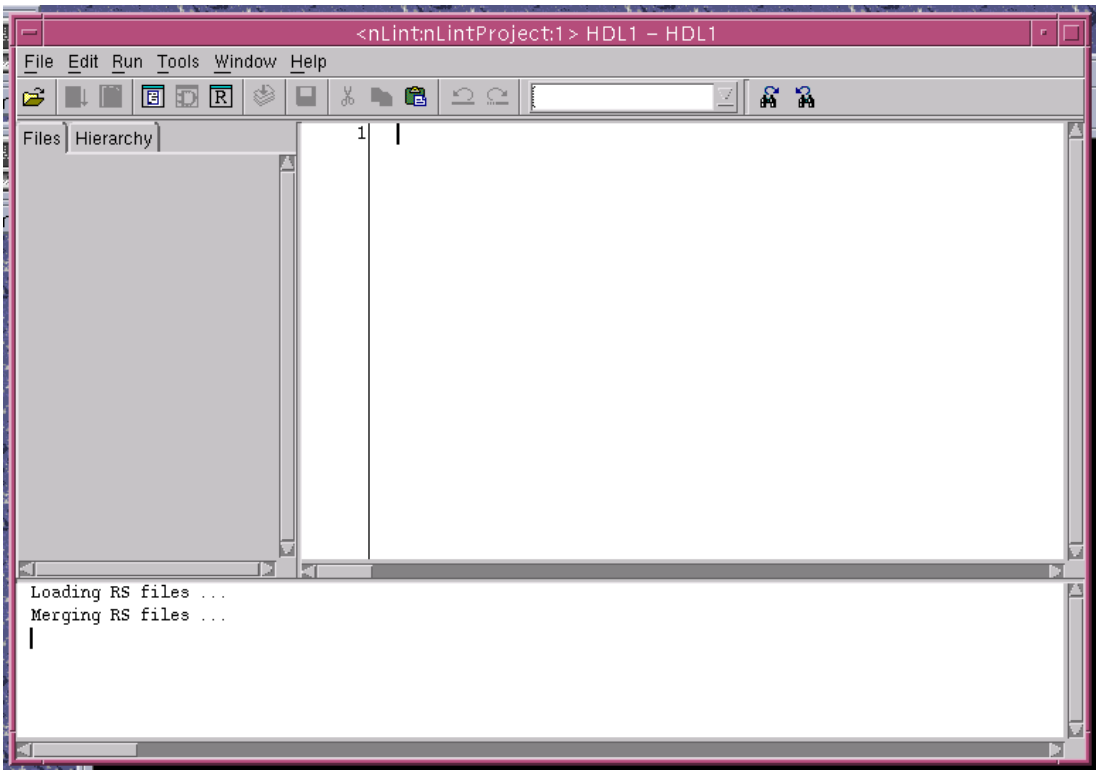
2. 图形界面模式（GUI Mode）快速教程

建立一个用 nLint 检查的目录，在此目录下键入：

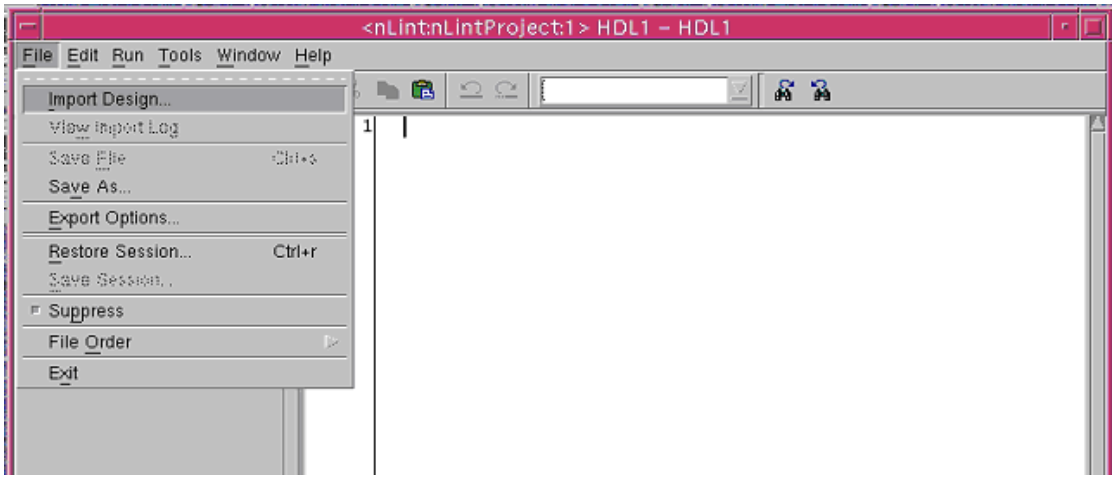
nLint –gui

```
HW188:/home/h43307/huolei_path/spdifv1.0/nLint_check> ls
nLint.rc    nLintLog/
HW188:/home/h43307/huolei_path/spdifv1.0/nLint_check> nLint -gui
```

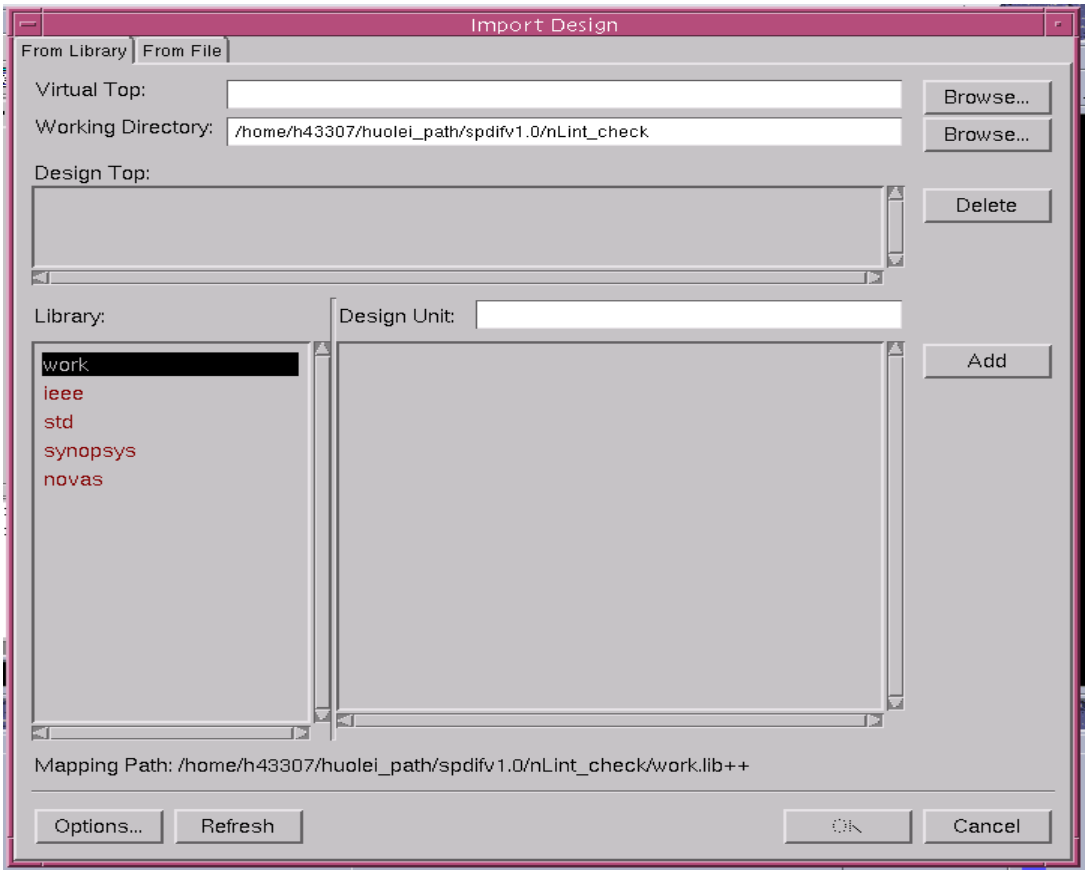
回车后出现下面的界面：



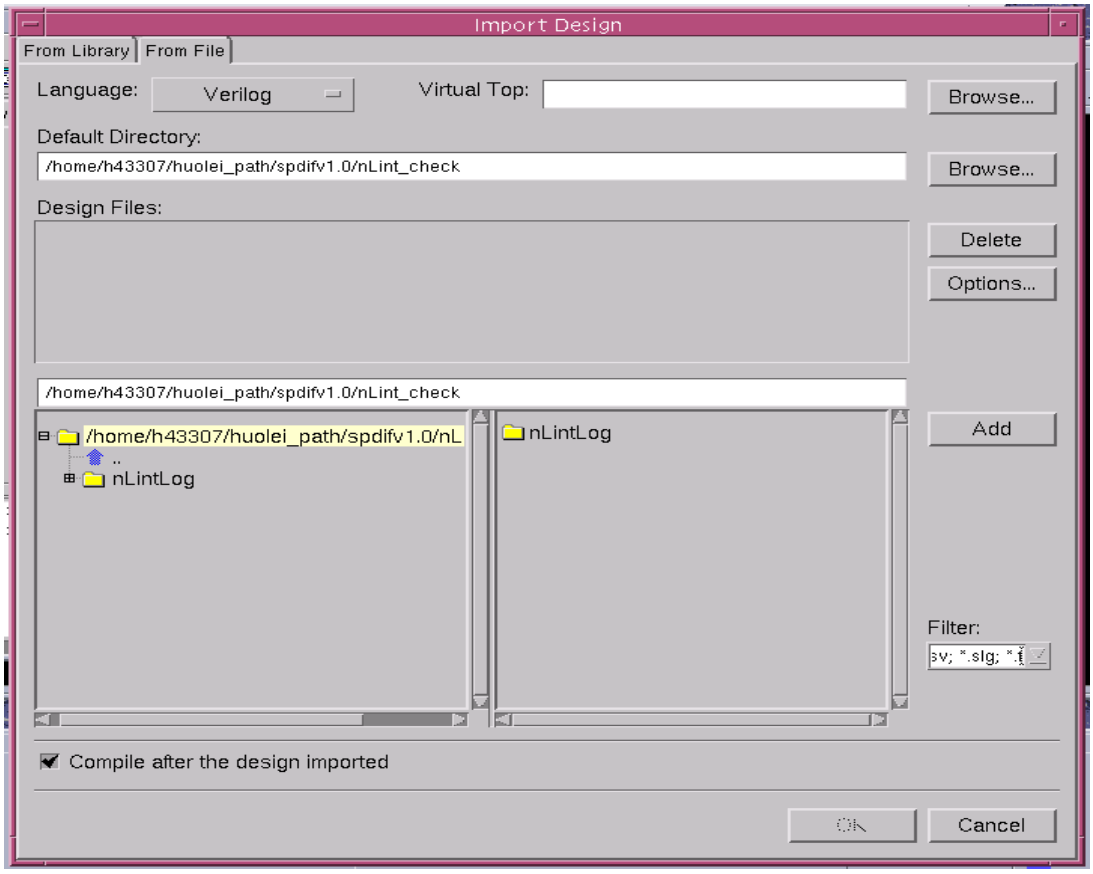
点击 File，出现下拉菜单：



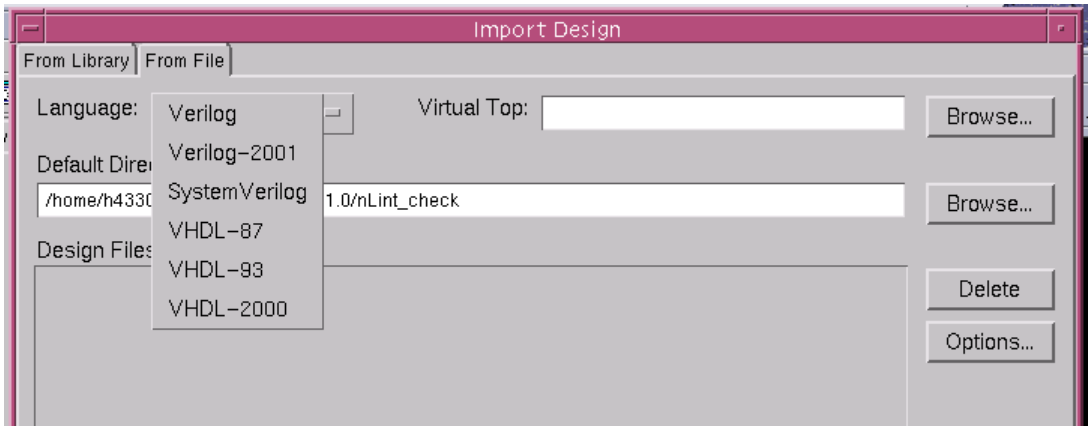
然后点击 Import Design 后会出现下面的窗口（对话框包含两个 TABS：From Library 和 From File）：



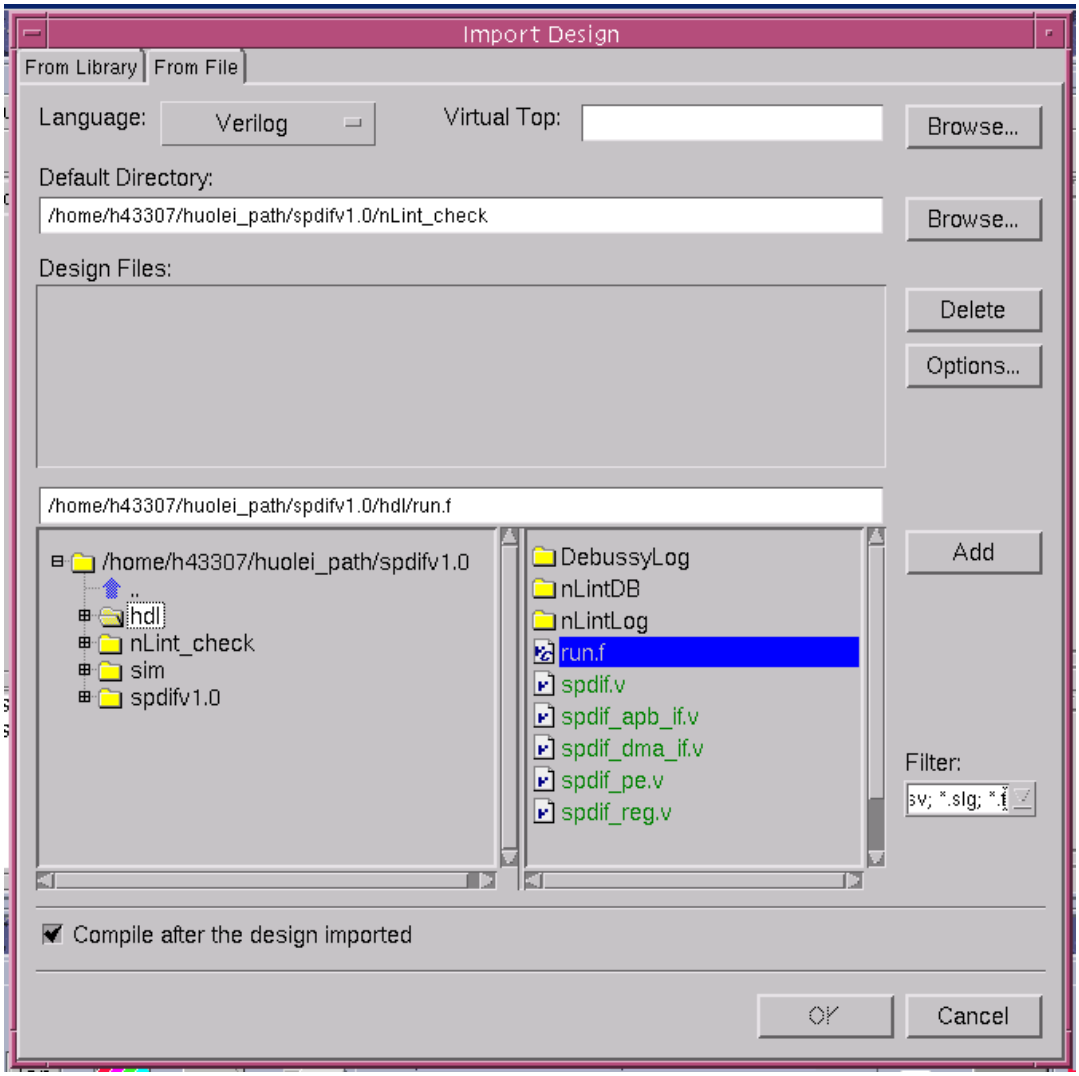
选择 From File，会出现如下的界面：



我们可以通过 Language 选项选择不同类型的 HDL 语言：

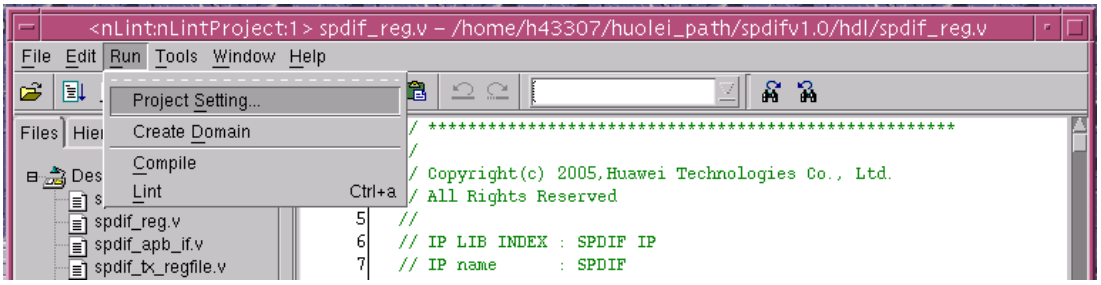


这时可以开始加载 RTL 设计文件了，选择要加载的文件，点击 Add，然后再点击 OK 就可以了：



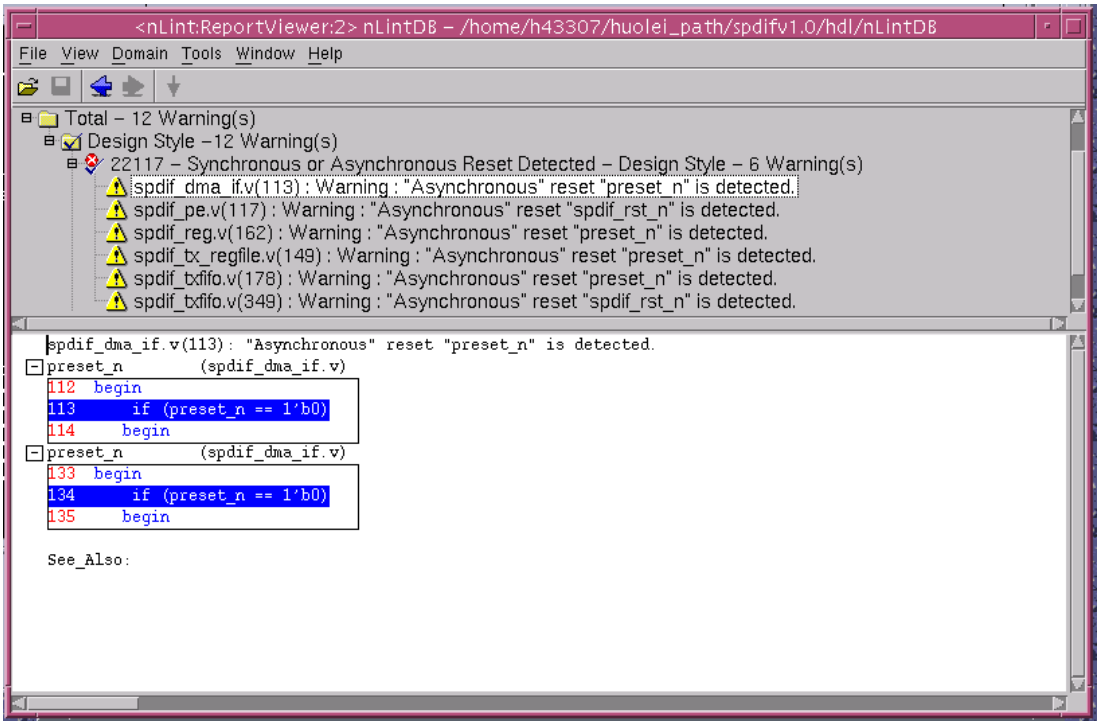
如果你想每次同时编译几个 HDL Files，可以建立一个*.f 的文件，*.f 相当于一个 Filelist，指明路径和文件就 OK 了。

当你要 Compile 的文件加载进去后，然后就可以开始编译了，当然同时你也可以编辑 HDL Files，并且修改，重新编译。



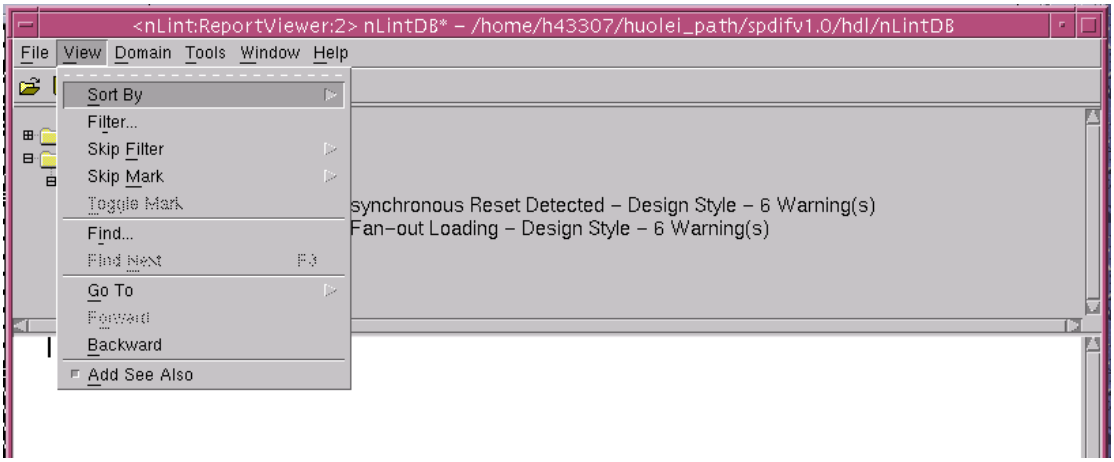
当要编译时，用鼠标点击 Run，然后再点 Lint，这样就可以 Compile 了。

编译结束后会出现 nLint Report 的一个窗口，通过这个窗口可以检查设计代码出现问题的地方：

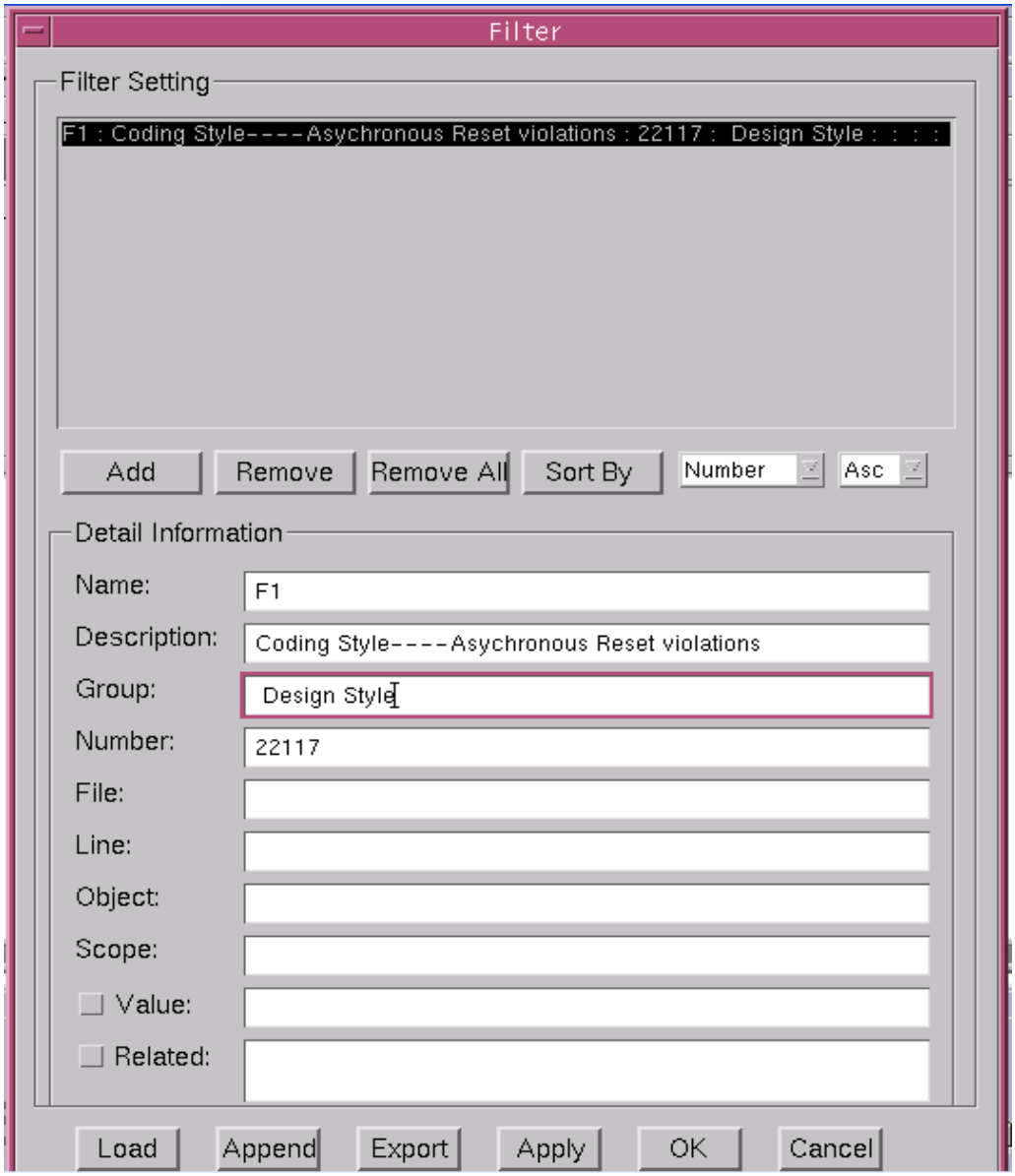


点击窗口“黄色叹号”处将会弹出主窗口，这样就可以对设计代码进行编辑修改了。

如果你想过滤掉一些不想要的 Violations 可以进行下面这些操作：



点击 Report 窗口的 View ——> Filter 后会出现下面的窗口：



如果要过滤掉某一条 Violation，指定这条 Violation 的 Number 就可以了，当然相应的过滤也可以做一些描述；同时还继续可以 Add 过滤的内容，或者指定要过滤的 File（*.v）。

2.4.3 Quartus 入门

Quartus 软件是 Altera 系列 FPGA 的专用集成编译环境。通过简单的操作，quartus 可以完成从综合、布局布线、时序计算到 FPGA 编程、调试的全部工作。本文针对初学者，介绍 quartus 的基本流程，以期减少初学者的摸索时间，带来少许帮助。

1. 预综合

设计者完成 RTL 代码设计和系统仿真后，首先要做一个简单的预综合，以便生成网表，为后面的正式综合做准备。

通过菜单里的 File－>New Project Wizard 可以创建一个新的工程。然后根据提示依次输入新工程的各项设置，设置的内容包括：工作目录、工程名、设计顶层 module 名、加入设计文件、Edatools 选择、选择器件等等。

完成工程创建后，选择 processing－>start compilation 开始综合。也可以直接点击粉红色的箭头图表开始综合。quartus 将自动完成综合、布局布线等一系列工作。为节省时间，预综合可以仅跑综合，不跑后面比较费时间的布局布线等步骤，方法如下

processing－>start －>start analysis & Elaboration

2. quartus 基本设置

点击菜单 assignments－>settings，这里集中了 quartus 的各种基本设置。在创建新 project 时没有设或设错的各种配置，都可以在该界面下重新设置。例如：

- add/remove：添加删除设计文件
- EDA tool Settings：设置 EDA tools 工具。
- device：设置器件型号、封装。
- general：设置待综合层次模块名（顶层模块名）。

除了以上这些基本信息外，还有几个选项应当注意：

(1) default logic option settings：

这里是一些综合策略的选项，可以选择默认值。我们在实践中发现，有些情况下将 remove duplicate logic 和 remove duplicate registers 选为 off 有助于提高性能，也不尽然，需要在实际应用中尝试。

(2) timing analysis reporting 时序报告

这里可以选择报多少条关键路径，在后期做详细分析时会比较有用。

(3) mode

这里可以选择 smart compilation/more disk space。quartus 会保留很多信息，下次综合时对于没有修改的部分可以调用原有信息，提高综合速度。

(4) fitting

其中有一个选项是随机种子（seed），当时序只差一点点，而没有其他什么办法好想时，可以尝试修改一下种子。

(5) netlist optimization

这里的选项和综合策略有关，我们在实践中发现，全部选中有时有助于提高性能，但是综合时间会大大增加。

3. 时钟设定

(1) 设置时钟频率及占空比

点击菜单 Assignments－>settings。

选中 Timing Setting－>Clocks，在右面出现的窗口中选择 Settings for individual clock signals。点击 New 按钮，可以创建一个时钟，设置其频率和占空比。如果需要，可以重复使用上述方法，将设计中所需的时钟逐一建立。

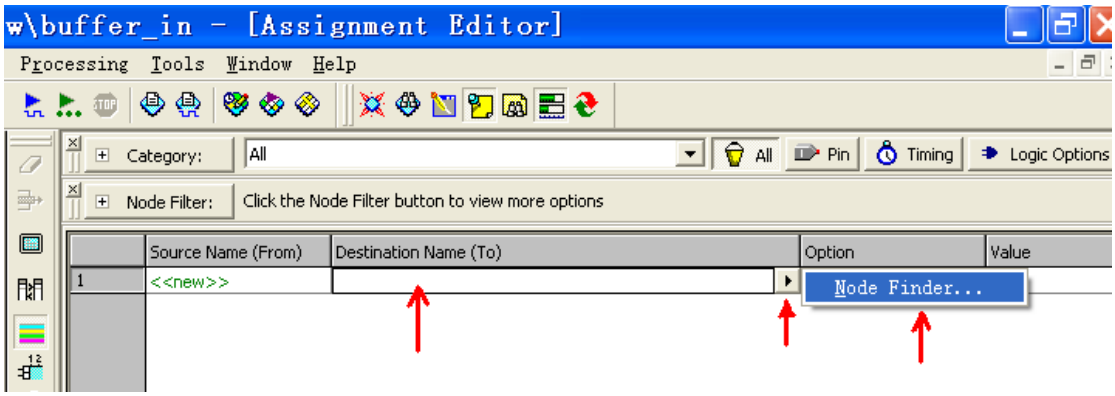
⚠ 注意：

这里所定义的时钟是虚拟的，它的命名可以随意，而且和设计尚未发生任何关系。如果只定义到这一步就进行综合布局布线，会发现在时序报告中，这里所定义的时钟及频率没有被体现。

(2) 映射到网表节点

将上文中定义的时钟映射到设计的具体节点，才能使时钟设定真正发挥作用。

选择 assigments－>assigment Editor。这个界面是 quartus 另一个非常重要的界面，里面包含了 quartus 的各种高级设置，包括时序、路径、管脚、电流、扇出等等。如果用户想设定一些选项而无处下手，不妨到这里来找一找。对于更多复杂的选项，Altera 的工程师给出的建议是：如果看不太懂，就不要做修改。



上图是 assigment editor 的界面，因为设置时钟只涉及一个节点，就不用设置 Source Name 了。首先双击 Destination Name 下面的空白框，右侧出现了一个箭头，点击箭头，出现了 Node Finder，点击 Node Finder 就进入了一个节点选择界面。Quartus 在其他需要节点选择的地方，也用相类似的方法。

在随后出现的界面中，Name 框中可以通过通配符选择所需节点名。filter 框，如果是选择输入管脚，则选择 pins：all。如果选择内部寄存器作为时钟，选择 Registers：pre_synthesis。也可以选择 Signal Tap：pre_synthesis。look 框通过选择子层次来节约搜索所需时间。点击 start，选中所需节点。注意，如果 quartus 正在运行中，或者 project 尚未经过预综合，都无法选取节点。

选完节点后，回到 assign Editor 界面，在 option 一栏中选择 clock Settings，点击 value 一栏的下拉条，可以看到在 assigments－>settings－>clock_setting 中设定的几个时钟，选取一个。至此完成时钟设定。

时钟设定可以将某输入 pin 设为时钟，也可以将逻辑内部某节点设为时钟（例如 PLL 的输出）。时钟设定的可以通过更改其频率来达到过约束的效果，以得到更好的时序。在内部时钟关系比较复杂时（例如通过几个 PLL 进行处理后作为系统时钟），时钟设定还可以帮助 quartus 明确时钟关系，产生合理的时序报告。

4. 管脚约束

管脚约束指将逻辑的输入输出端口约束到 FPGA 的特定管脚上，并设定该管脚的电气特性。

通过菜单：Assignments －>Assign Pins 或者 Assignments －>settings －>device－>assign Pins 可以进入 Assign Pins 界面。

首先选择 FPGA 管脚号，可以在列表框中选取，也可以在 Pin Name 框中输入，quartus 会自动联想，跳到对应的 FPGA 管脚。Quartus 会根据输入立即联想。例如想设置 G24 管脚，当在 Pin Name 框中输入 G2 时，会跳到 G2 管脚，并且 Pin Name 框被清空。需要再次输入 G24，才能跳到 G24 管脚。

选中管脚后，仍然在 Pin Name 框中，通过节点选取选到逻辑的对应输入输出管脚，也可以直接用手写输入管脚名。

在 I/O 框中，选取所需的 IO 类型。

照此逐一设定逻辑的所有输入输出管脚。对于 FPGA 没有用到的管脚，可以设置 reserve_pin(even if it dose not exist in the design)，一般设置为输入三态是比较安全的选择。

5. 时序约束

如果想对设计管脚进行 input delay，output delay，Tco，Tsu 等参数进行设置，需要在 assignment editor 里，对所需管脚进行逐一设置。

另一种比较简单的方法是，通过菜单 assignments－>settings－>clocks，在界面右下角有一个按钮 Default External Delays，这里可以对所有输入输出管脚定义一个统一的 input delay 和 output delay。

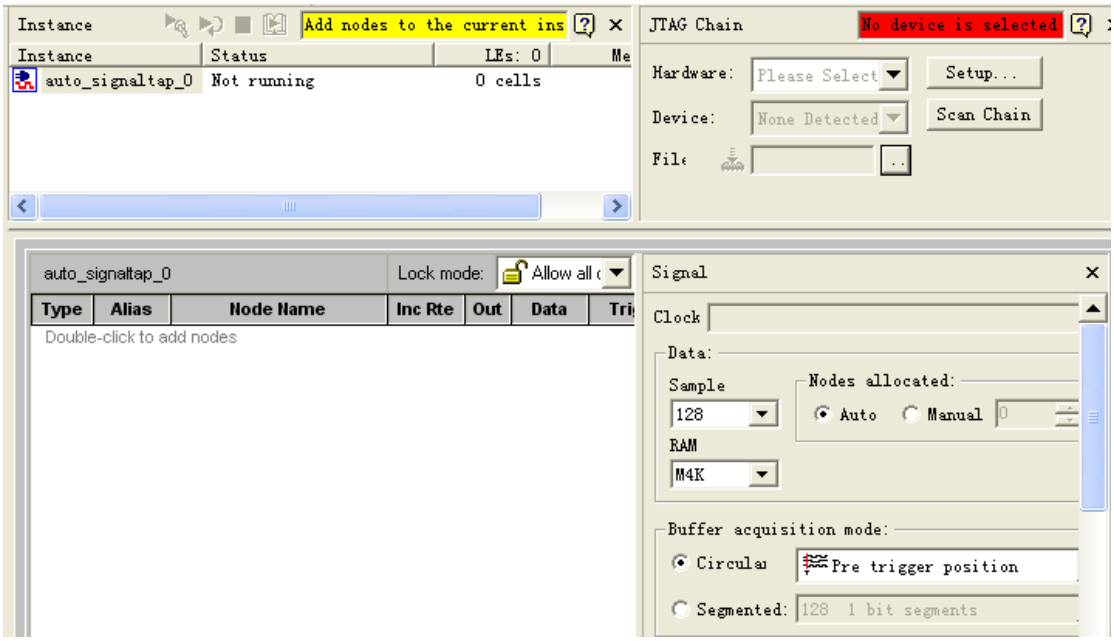
6. 正式综合

完成上述设置后就可以进行正式综合了。用 Processing－>start compilation 或点击粉红色图标，可以完成全部流程。

7. Signal_Tap

Signal_Tap 是 quartus 内部集成的一个逻辑调试工具，通过 JTAG 对 FPGA 进行现场观测。

通过菜单 Tools－>signal Tap Analyzer 可以进入 Signal Tap 界面。



在 Instance 窗口中默认出现一个信号组：auto_signaltap_0。在该窗口内可以用右键对信号组进行添加、删除、重命名等操作。每个信号组设置一个采样时钟，统一触发。

选中某一个信号组后，data/setup 两者间选择 setup，用右键，add nodes 即可添加信号。注意 signaltap 检测的是网表节点，所以必须预综合后才可以添加节点。选择节点应选取寄存器，wire 型信号有些虽然允许选取，但综合时会丢失。选取信号后，默认作为 trigger 使用，为节省资源，可以把一些不必要的信号 trigger 使能取消。一个信号组限制最大采样 bit 为 1000。

在窗口右侧设置信号组采样时钟，否则无法正常工作。sample 一栏表示采样深度，采样越深，占用资源越多，会引起时序恶化和调试时采样窗口迟钝，一般不要超过 512。

以上选项必须在综合前选定，如需修改，则要再次综合才能有效。

buffer acquisition mode 可以选择连续采样（circulal）还是间隔采样（segmented）。如果是间隔采样，还可以选择每次采样点数。采样位置可以选择 pre trigger position、center trigger position 或 post trigger position，表示触发点位于采样窗口的前部，中部，后部。采样点在当前的时间坐标为零，以前为负，以后为正。这些选项可以在调试时现场更改，无需再次综合就可有效。

在完成 signaltap 设定，开始综合时，quartus 会提示存为一个.stp 文件，并询问是否为该工程使能该 signaltap 文件。也可以通过 assignments－>settings－>SignalTap II Analyzer 来选择是否使能 signaltap 和所用的 stp 文件。

有时，打开 quartus 时发现 signaltap 的一些设置全部丢失，即使用 File－>open 打开*.stp 文件仍然显示错误。这是 quaruts 的一个小 bug，解决办法是：打开所需的*.stp 文件（尽管显示错误），关掉整个 quartus，然后再通过双击*.quartus 文件打开工程，signaltap 就会正常显示了。

8. 下载与调试

signaltap 界面就是一个很友好的下载调试界面。在 JTAG 窗口完成下载功能，通过 signaltap 进行调试。

首先在 JTAG 窗口用 setup 选择硬件端口，quaruts 会自动检测端口是否有效。如果硬件端口已经选中，也可以直接按 scan chain 按钮检测端口是否有效。如果有效，quartus 会自动检测板上 FPGA 的型号并显示出来，否则 device 一栏就是空的。

确认硬件端口有效之后，在 FILE 一栏通过浏览方式选中所需的*.sof 文件，点下载按钮即可进行下载。位于屏幕下部的 message 窗口会显示首先找到了一个某某 id，然后开始下载，最后显示下载成功。

如果不小心将 message 窗口关闭了，可以通过 view－>utility window－>message（Alt＋3）来重新打开它。

下载完成后，需要用 signaltap 来进行调试。选择你要观察的信号组，首先在 setup 一栏设定 trigger 条件。右键选择 low、high、rising edge、falling edge 或者 don't care，对于多 bit 的信号，可以将其打开后对特定 bit 进行单独设置。

设置完成后，可以点 instance 窗口的红色箭头进行连续触发或单次触发。如果触发成功，quartus 会自动跳到 data 窗口显示波型。在波型窗口，左键为放大局部波型，右键为缩小显示整体波型。还可以拖动窗口显示某一范围波型。

9. RAM 模型的使用

quartus 可以产生供 RTL 仿真用的 RAM 模型，将生成的仿真模型文件（例如*.verilog）直接作为设计文件读入 quartus，quartus 就可以识别并综合成 RAM，比较方便。要生成 RAM 模型，在 tools－>mega wizard plugin manage，选择 memory compiler，按照其提示，根据设计者要求一步一步往下做就行了。这里需要注意几个问题：

- 有一步提示 RAM 输出是否要加一级寄存器锁存。注意 RAM 的输出实际上已经是同步输出，即在写使能的下一拍输出数据。如果在选择加一级锁存，就会在读使能的下两拍输出数据。请选择前考虑好。
- 有一步提示是否选择“implement with logic cells only，even if the device contains EAB or ESBs”。如果选择，则表示该 RAM 将用寄存器搭成，将获得比较好的时序但比较大的面积，请提前考虑好。
- quartus 产生的 RAM 有设定初始值的功能，但比较麻烦。Fles－>new 的界面里选择 others file－>Memory Initialization files。生成后存为*.mif 文件。该文件只能在 quartus 综合时被使用，如果用于 RTL 仿真，还要做格式转换，生成*.rif 文件。最严重的问题是，Altera 的 RAM 值在 FPGA 芯片被重起后，不能恢复初始值，只有下电才能恢复初始值。所以建议不要使用 RAM 的初始值功能，可以在复位后在有效的保护下对 RAM 所有单元做写操作来实现初始化功能。

10. 生成其他格式加载文件

.sof 是供 JTAG 加载的文件格式，在正式产品中，FPGA 加载往往不通过 JTAG 加载。在我们的设计中，需要生成.rbf 文件。

File－>convert programming files 进入转换界面。在 programming file 窗口选中 rbf 格式，在 mode 中可以选择各种模式，根据实际电路的要求选择。然后在 input file for convert 窗口加入要转换的*.sof 文件。点 ok，完成转换。

如果需要生成其他格式的加载文件，过程类似，在 programming file 窗口选中其他格式即可。

11. 用脚本跑 quartus

在后期，可能需要在无人职守情况下连续跑多个工程，这样图形界面就不能胜任，可以用脚本方式运行 quartus。

(1) 在 unix 下：

首先在 quartus 图形界面下做预综合，做好全部设置，就可以用 quartus 生成一个 tcl 脚本：project－>generate tcl file for project。

然后在 unix 下运行：***quartus_cmd -f *.tcl***

(2) dos 下

首先在 quartus 图形界面下做预综合，做好全部设置，运行：***quartus_cmd *.quartus -c *.csf -s *.ssf -w release.fsf***

2.4.4 Xilinx 工具入门

Xilinx 作为当今世界上最大的 FPGA/CPLD 生产商之一，长期以来一直推动着 FPGA/CPLD 技术的发展。其开发软件也不断升级换代，由 Foundation 系列逐步发展到 ISE 系列。ISE 是集成综合环境的简称，是 Xilinx 提供的一套工具集，其集成的工具可以完成上述整个 FPGA/CPLD 的开发过程：可以完成代码设计输入、综合、实现与布局布线、优化、仿真、配置芯片和在线测试等系列功能。

ISE 的主要特点：

- 它是一个集成环境，可以完成整个开发过程。ISE 几乎集成了所有著名的 FPGA/CPLD 设计工具，根据设计流程合理应用这些工具，会使工程师的设计工作如鱼得水。
- ISE 界面风格简洁流畅，易学易用。ISE 的界面风格秉承了可视化编程技术，界面根据设计流程而组织，整个设计过程只需按照界面组织结构点击下去即可，十分方便。易学易用使设计工程师可以专心于电路的设计工作。
- ISE 的帮助十分方便。ISE 有丰富的在线帮助信息，结合 Xilinx 的技术支持网站，一般设计过程中可能遇到的所有问题都能得到很好的解决。
- 强大的设计辅助功能。ISE 秉承了 Xilinx 设计软件的强大辅助功能。在编写代码时可以使用编写向导生成文件头和模块框架，也可使用语言模板

（Language Templates）帮助编写代码。在图形输入时可以使用 ECS 的辅助项帮助设计原理图。另外，ISE 的 Core Generator 和 LogiBLOX 工具可以方便地生成 IP 核与高效模块为用户所用，大大减少了设计者的工作量，提高了设计效率与质量。

1. ISE 设计流程

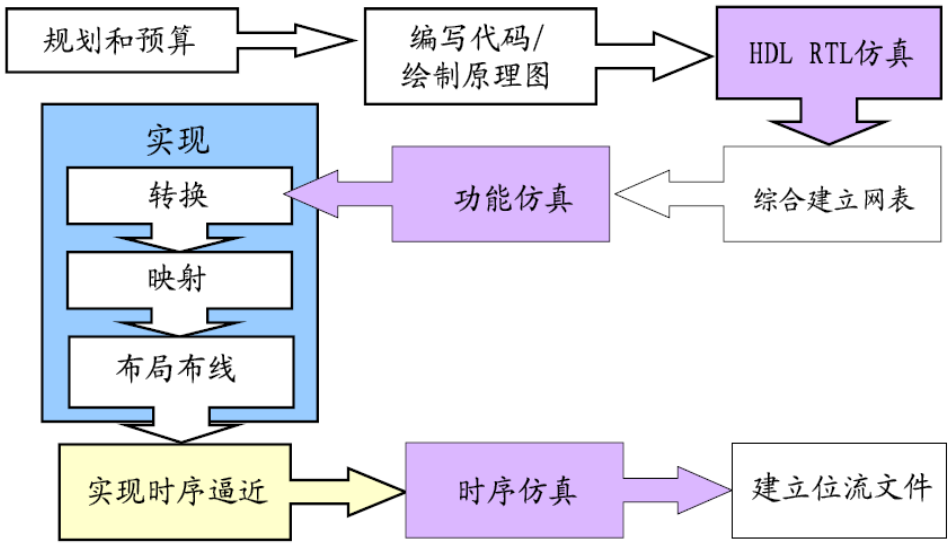


图2-91 Xilinx 设计流程图

(2) 设计输入

ISE 有两种设计输入方法：硬件描述语言 HDL 或原理图；结构向导和核生成器 Core Generator 可以辅助设计输入。不管你采用何种设计方法你都需要一个工具来生成 EDIF 网表以便对 Xilinx 的 FPGA 编程。适用的综合工具有 Synplify、Leonardo 、SpectrumFPGA Compiler II、XST。

(3) 实现

一旦建立了网表，就可以利用它来实现用户的设计了。

实现不仅仅是指“布局布线”，实现包括很多步骤：

- 转换将多个设计文件合并为一个网表;
- 映射将网表中的逻辑符号门组装到物理元件(CLB 和 IOB)中;
- 布局布线将元件放置到器件中并将它们连接起来同时提取出时序数据并生成各种报告。

每个步骤都会生成一些文件,使得可以使用 Xilinx 的其它工具(例如 Floorplanner、FPGA Editor、XPower、Multi-Pass Place & Route)。这些文件有以下内容：

- 报告
- 时序仿真网表
- 平面布局文件
- FPGA 编辑器文件

- 其它更多

(4) 时序逼近

时序逼近流程是一个推荐的设计方法可以帮助设计满足它们的时序目标，下图是 Xilinx 时序逼近流程图：

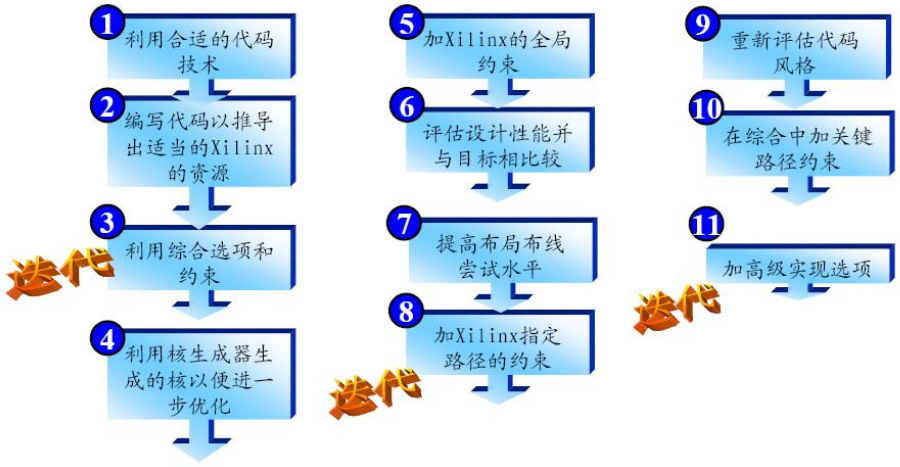


图2-92 时序逼近流程图

(5) 下载

一旦一个设计实现完成后你必须建立一个 FPGA 可以识别的文件，这个文件叫位流文件一个 BIT 文件(以.bit 为扩展名)，它可以被直接下载到 FPGA 中，或者可以被转换为存贮编程信息的 PROM 文件。

2. ISE 的使用

下面本文将采用例子来阐述 ISE 的使用方法：

(1) 建立一个新的工程

登录到 ISE 项目导航器在 C:\training\fund\labs\flow 子目录下建立一个名称为 FlowLab 的新项目。目标器件为 xc2v40-4fg256，设计流程可以是 XST Verilog 或 XST VHDL，你可以根据你的喜好选择相应的语言。

- 打开 Xilinx ISE 5.1 软件从开始菜单里选择程序 Programs → Xilinx ISE 5.1 → 项目导航器 Project Navigator
- 在项目导航器中选择文件 File → New Project，这时新项目窗口就会打开。（下图）

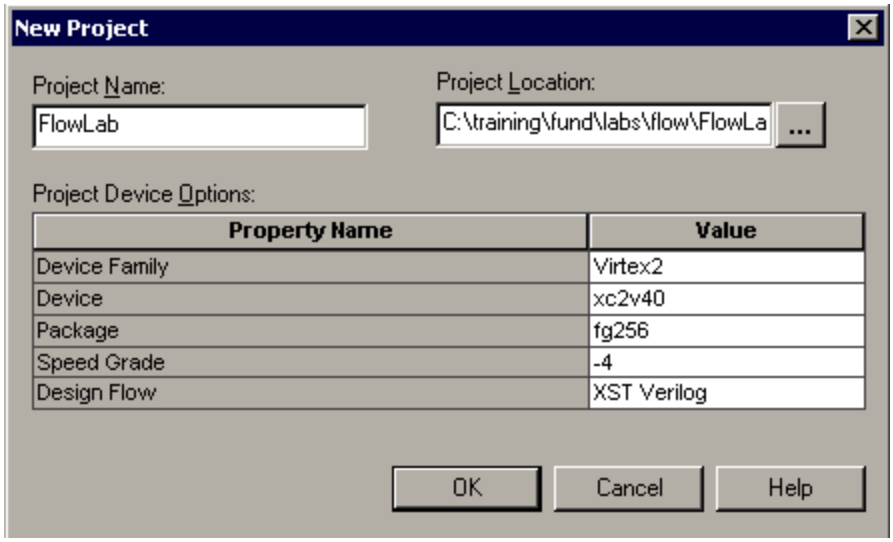


图2-93 新项目窗口

- 对于项目名，键入 FlowLab
- 对于项目位置使用按钮浏览到 C:\training\fund\labs\flow
- 单击 OK
- 选择器件家族： Virtex 2
- 选择器件为 xc2v40，封装为 fg256，速度等级为–4
- 选择设计流程 XST Verilog 或 XST VHDL
- 单击 OK 。

(2) 添加源文件

将子目录 C:\training\fund\labs\flow\verilog 或 C:\training\fund\labs\flow\vhdl 中所有源文件添加到 FlowLab 项目中 ：

- 在项目导航器窗口中选择项目 Project→ Add Source ；
- 在添加已有源文件对话框 （下图）中用下拉菜单浏览到 C:\training\fund\labs\flow 子目录，然后进入 VHDL 或 Verilog 子目录，这取决于你在建立项目时所选择的流程。

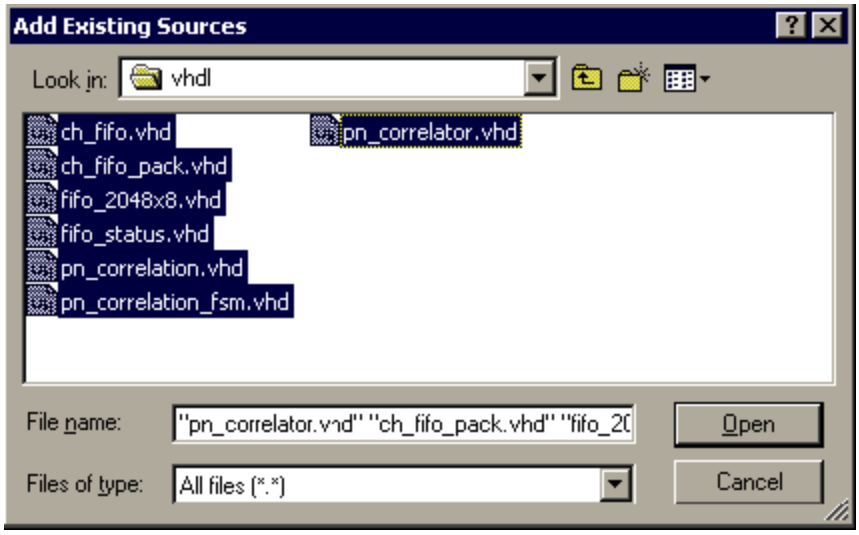


图2-94 添加已有源文件对话框

- 选中所有文件单击第一个文件 ch_fifo，然后在按住 Shift 键的同时，在添加已有源文件对话框中单击文件 pn_correlator 。
- 点击打开 Open
- 对于 VHDL 用户，每个文件将出现选择源文件类型对话框。对于除了 ch_fifo_pack.vhd 之外的的其它所有文件选择 VHDL 模块（VHDL Module）并点击 OK。对于 ch_fifo_pack.vhd ，选择 VHDL 封装 VHDL Package 并点击 OK 。

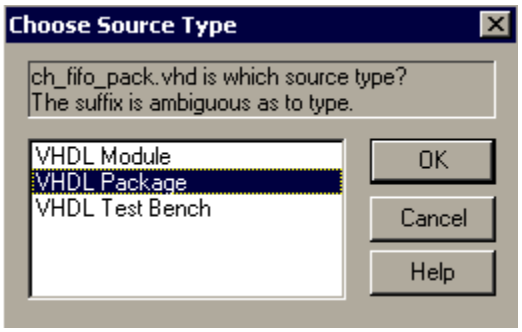


图2-95 选择文件类型

- ISE 将处理所有的文件并确定设计层次。在项目导航器的右下角的进程条显示分析的进程。添加的文件被分析后，设计文件和层次将反映在项目窗口的源文件中。

(3) 用结构向导建立 DCM 例化

用结构向导中的 DCM 向导创建一个名为 MyDCM 的新的源文件使用 CLK0 CLK2X 和 LOCKED 引脚 设定输入时钟频率为 100MHz 而其它选项为缺省设置。

- 在项目导航器中，选择项目 Project → 新源文件 New Source；
- 在新源文件窗口中，选择结构向导(Architecture Wizard)，并输入 MyDCM 作为文件名。

- 点击下一步 Next，然后在下一个窗口中点击完成 Finish 。
- 在结构向导选择对话框中，确定 DCM 向导已被选择并点击 OK（图 6）

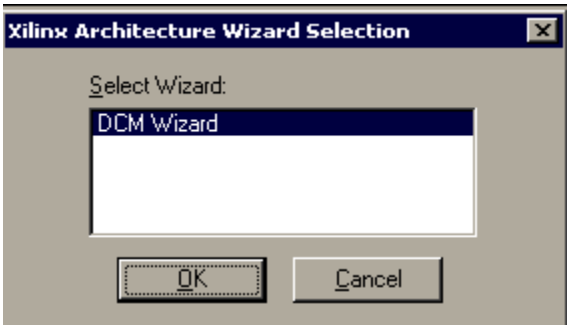


图2-96 结构向导选择对话框

注意：Rocket I/O 向导在此不可选，这是因为我们的目标器件为 Virtex-II，而 Rocket I/O 能力仅在 Virtex-II PRO 器件中才有。

在 Xilinx DCM 向导 – 通用设置窗口，操作如下：

- 输入 DCM 例化名： MyDCM_inst；
- 选中 CLK 2X 输出框；
- 去除 RST 引脚的选择；
- 输入输入时钟频率为 100 MHz，然后点击下一步；

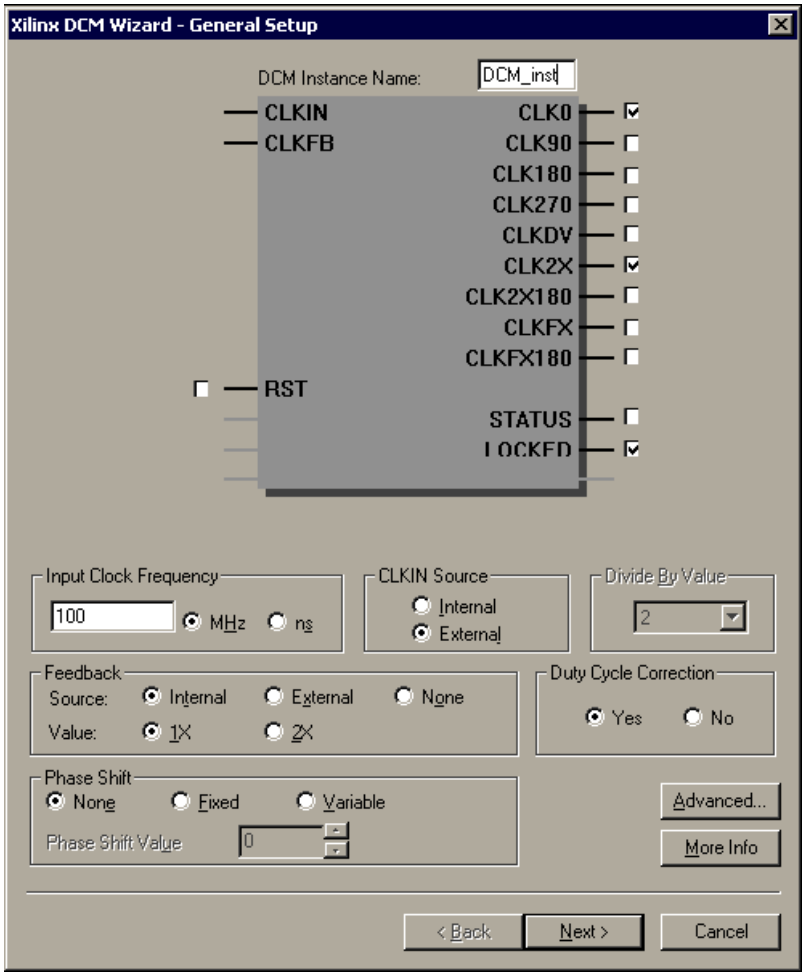


图2-97 DCM 向导—通用设置窗口

注意：在 DCM 向导中指定频率的目的是让向导知道 DCM 是工作在高频模式还是低频模式。基于输入的频率，还可以计算输出的频率以确保是否在规定的频率范围之内。另外，如果选择 CLKFX 输出的话，输入频率还可以被用来计算在 CLKFX 上的输出频率以及计算估计的抖动。

在 Xilinx DCM 向导中 – 时钟缓冲器窗口，保持选项为缺省设置并点击完成。

注意一个新的文件(MyDCM.xaw)被添加到项目源文件窗口。选择 MyDCM.xaw 文件，进入当前源文件进程窗口，双击查看 HDL 源文件并检查结构向导所产生的源代码。

(4) 将 DCM 置于设计（Verilog）中

将部件例化从例化模板中拷贝到 ch_fifo.v 中的合适位置。最后，分配信号和修改代码以完成例化（见本节的子步骤 4 和 5，将 DCM 例化置于设计中)，保存修改好的文件。

- 在项目源文件窗口，双击 ch_fifo.vhd 打开源代码；
- 现在选择文件 MyDCM.xaw

- MyDCM.xaw 选中后，进入当前源文件进程窗口，双击查看 HDL 例化模板。
- 在 HDL 例化模板中，复制部件声明这一部分 (从 MyDCM 开始一直到最后的 “);” 结束) ，并粘贴到 ch_fifo.v 的第十行(在 wire [7:0] wr_data 之后，在 pn_correlator pn_correlator_inst 之前)。
- 按照如下方式修改例化 ：

```
MyDCM  MyDCM_inst (
    .CLKIN_IN(wr_clk_in),
    .LOCKED_OUT(lock),
    .CLK2X_OUT(rd_clk),
    .CLK0_OUT(wr_clk)
);
```

目前，设计中有两个时钟信号没有用到 DCM。既然我们现在已使用了 DCM，我们需要修改和添加一些信号名以便例化能工作。有四个地方需要修改：

- 在 ch_fifo 模块的端口列表中将 wr_clk 改为 wr_clk_in
- 同样在 ch_fifo 模块的端口列表中，删除 rd_clk 端口
- 在 ch_fifo 的端口声明中从输入信号列表中删去 rd_clk 信号因为此信号将由 DCM 产生
- 添加下列 wire 声明到第七行

```
lock, rd_clk, wr_clk
```

这样整个一行将变成

```
wire pn_lock, wr, wr_addr_srst, lock, wr_clk, rd_clk;
```

选择 文件 File → 保存 Save 以保存文件

(5) 实现设计

选择顶层设计文件 ch_fifo.v/vhd，实现设计：

- 确认在项目源文件窗口中选择的是顶层的设计文件（如下图）。顶层设计文件名为 ch_fifo.v/vhd

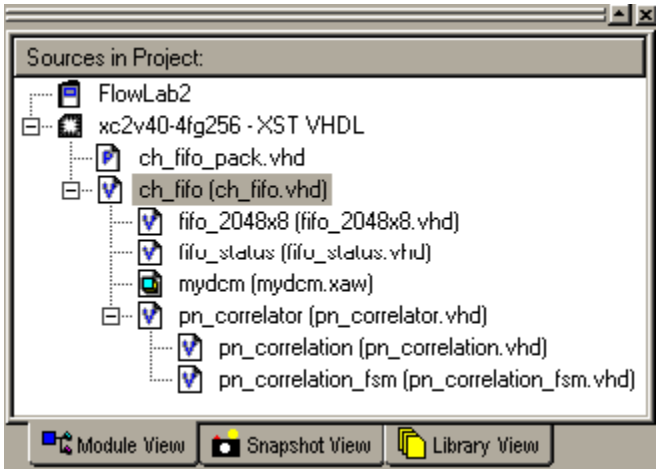


图2-98 项目源文件窗口

- 在当前源文件进程窗口（如下图）中，双击实现设计 Implement Design。注意，工具将会执行实现设计所必需的所有进程。在本实现中，在进入实现之前，工具将先执行综合 。

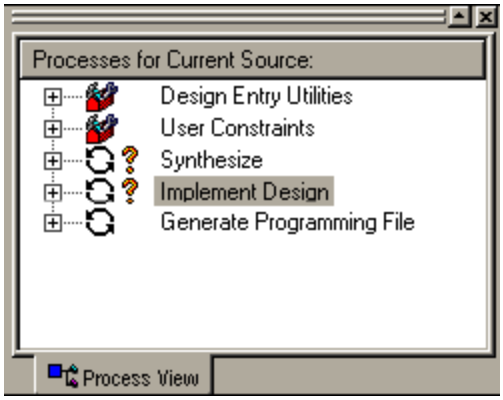


图2-99 当前源文件进程窗口

(6) 实现期间的观测

注意进程条，每一个进程旁边的状态符号和控制信息 。

注意项目导航器窗口的右下角显示的是进程和设计实现的完成百分比（如下图）。



图2-100 进程条

在当前源文件进程窗口，点击实现设计旁边的“+”展开实现步骤并查看进程。我们称此为“展开”一个进程 。每一步完成之后，在每一步旁边将出现一个符号：

- 绿色的对号表示成功
- 黄色的惊叹号表示有警告
- 红色的 X 表示有错误

对于这个特定的设计，在每一步将会有一个黄色的惊叹号（警告），但这些警告无碍，可以忽略。

⚠ 注意：

当实现完成时，项目导航器窗口底部的控制/信息窗口将会显示以下信息：
Done: completed successfully.

(7) 打开 PACE

在项目导航器中，以建立一个新源文件的方式建立一个名为 myucf 的 UCF 文件。然后从当前源文件进程窗口中打开 PACE 工具(用户约束 User Constraints) → 指定封装引脚（ Assign Package Pins） 。

- 在我们能够打开 PACE 之前，我们必须建立一个 UCF 文件（按照警告信息）。要建立一个 UCF 文件，只要选择项目 Project → New Source ；
- 选择实现约束文件，输入 myucf 作为文件名，点击下一步 Next ；
- 在下一个窗口中，确定选择的是 ch_fifo ，点击 下一步，然后在最后一个窗口中点击完成如（如下图) 。

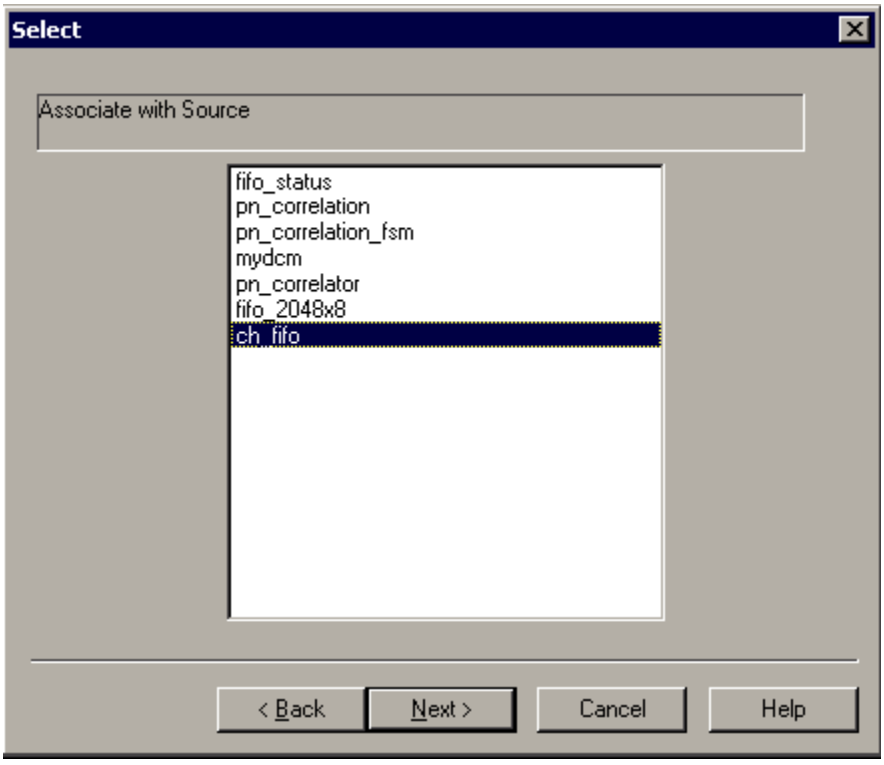


图2-101 关联一个约束文件

- 在当前源文件进程窗口，展开用户约束，然后双击指定封装管脚以打开 PACE。如果你不能看到这些进程，确认在项目源文件窗口中选择的是 ch_fifo 。

(8) 测试 PACE

检查 PACE 中的四个窗口检测这些窗口怎样相互作用

- 在 PACE 窗口中，移动器件结构窗口和封装引脚窗口以使它们相邻 。

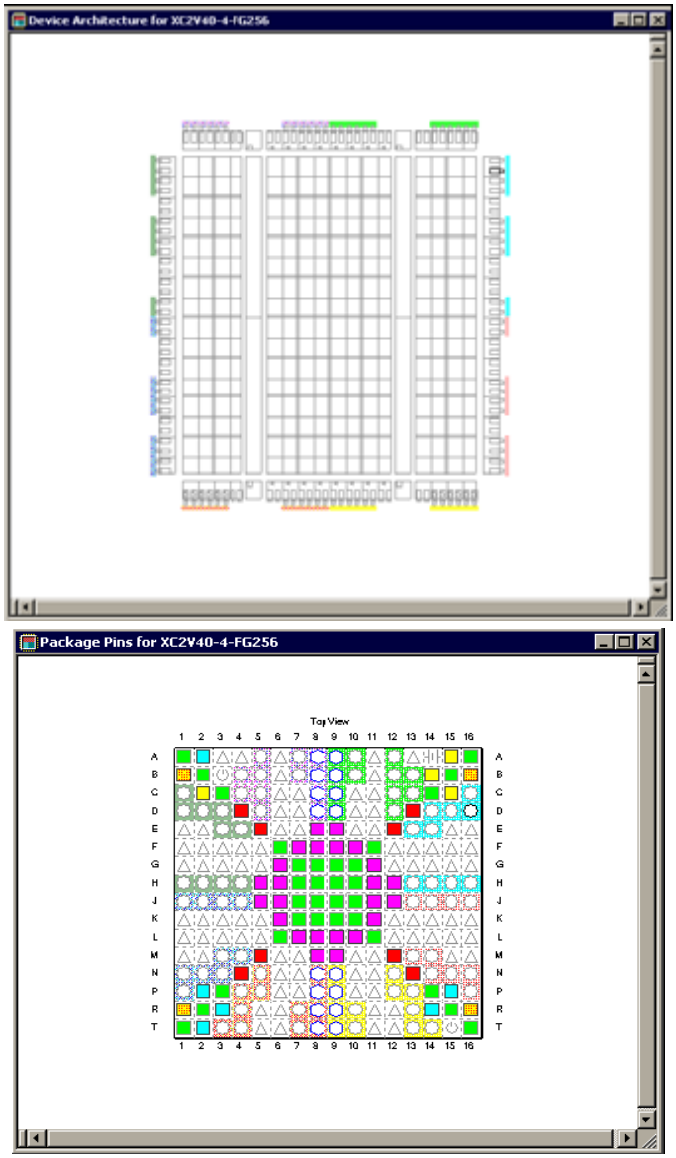


图2-102 器件结构窗口和封装引脚窗口

- 在封装引脚窗口中，点击用户 I/O 符号， 这些符号是中间为圆圈的框。它们可能是不同颜色的方框。圆圈后的不同颜色代表不同的 I/O 组。注意在器件结构窗口中相应的引脚是如何被高亮的 。
- 在设计层次窗口，点击 I/O 引脚旁的的”+”号展开 I/O 引脚子目录。你也可以展开全局逻辑和逻辑子目录来检查其中的内容，这些名称来自于设计 。

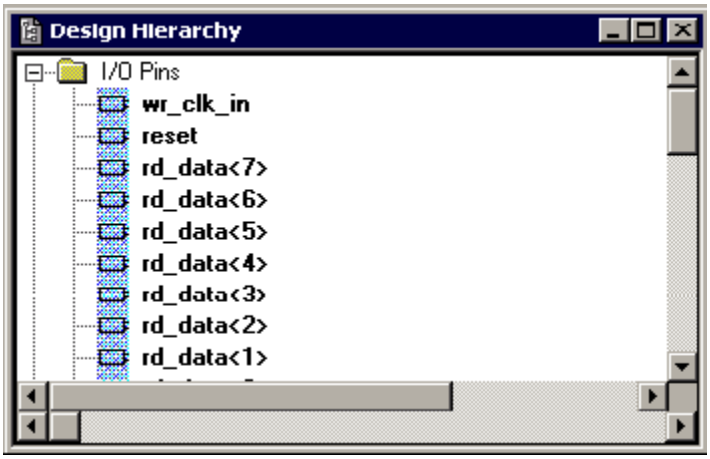


图2-103 设计层次窗口

⚠ 注意：

设计对象列表窗口是如何跟着设计层次窗口中所选的子目录而变化的。

(9) 建立引脚和区域指配

将所有 rd_data<?>信号指定到 BANK2 上。将 DCM 元件放置在位置 DCM_X1Y1 上，将 BRAM 元件放置在块 RAM 位置 X1Y1 上。

- 在设计层次窗口选择 I/O 管脚子目录
- 注意：可以为每一个信号指配一个实际的引脚位置，或为一个信号指定在特定组中的引脚范围。
- 在设计对象列表-I/O 引脚窗口中，将所有 rd_data<?>信号指定到 BANK2 上。可以这样操作，选择一个信号，在位置这一列点击框的右侧使用选择组的下拉菜单，然后选择 BANK 2 。

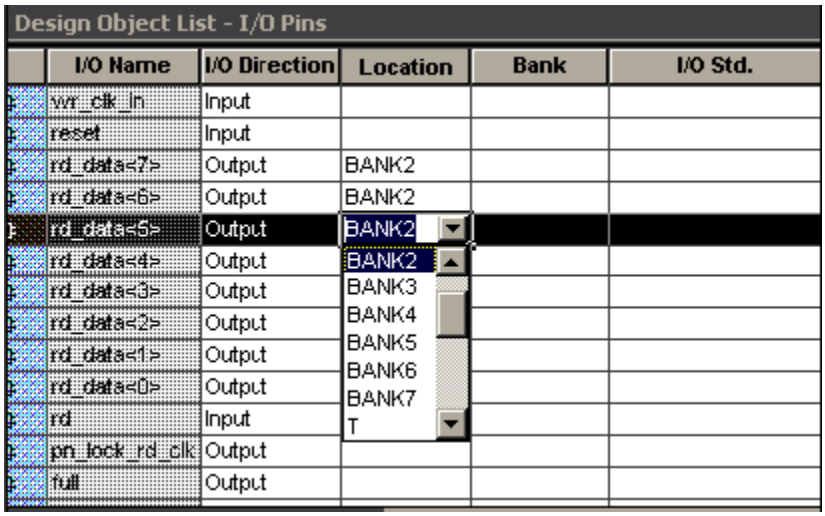


图2-104 设计部件列表－I/O 引脚窗口

⚠ 注意：

你可以在下拉菜单向下滚动时看到其它的位置选择：T, L, R, TL, TR, BL, BR, RT, RB, 等等。这些词语是指组，它们表示：

1）上部 （组）硅片顶部的两组

2）左部 （组）硅片左部的两组

3）左上部 （组）硅片左上部的组，等等。

这在你不了解组号，但你又想将引脚指定到器件的那个区域是非常有帮助的。

- 在设计层次窗口，点击全局逻辑文件夹。在接下来的几步中，我们将要将它们拖放到器件结构窗口中以便为这些部件指定特定的位置 。
- 在设计对象列表窗口中，选择 DCM 元件，并将其拖到器件结构窗口中。将 DCM 放置在右上方的 DCM_X1Y1 位置上，如图所示：

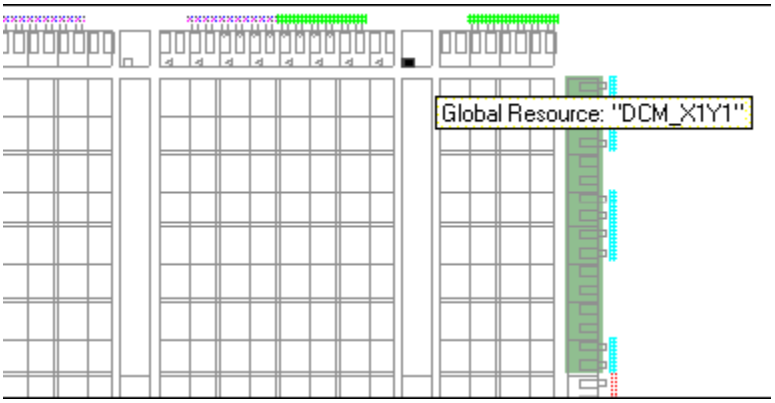


图2-105 指定一个位置

- 在设计对象列表窗口中，选择块 RAMfifo_2048x8_inst_fifo_bram，并将其拖到硅片的右上方的块 RAM 位置 X1Y1 上 。

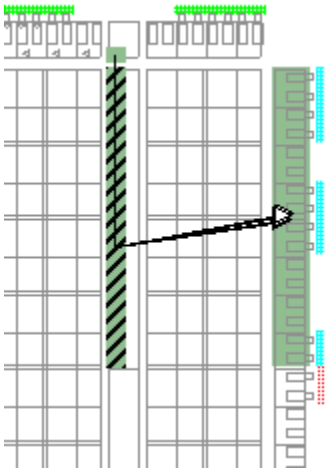


图2-106 指定一个位置

- 选择文件 → 保存 以保存引脚范围指配和 DCM 及块 RAM 布局 。
- 选择 文件 → 退出 以关闭 PACE 工具 。

- 在项目导航器的当前源文件进程窗口中，在用户约束的下方，双击编辑约束文本来查看通过 PACE 工具在文件 myucf.ucf 中所生成的约束。查看文本版本的 UCF 文件检查约束是否被写入文件 。

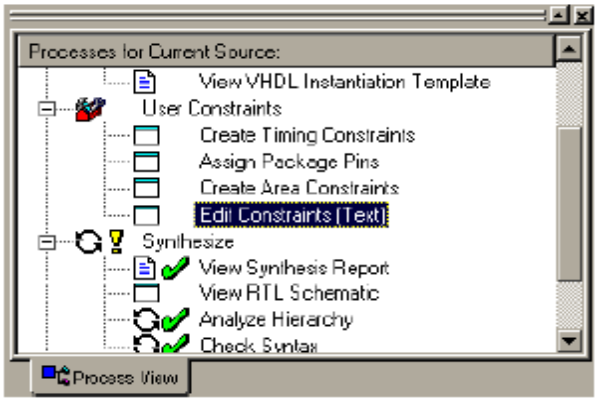


图2-107 当前源文件进程窗口

(10) 实现设计

实现设计确保没有错误或与我们所建约束的冲突

- 确认在项目源文件窗口中选择的是顶层的设计文件 (如图 5b-18) ，顶层设计文件名为 ch_fifo.v/vhd。

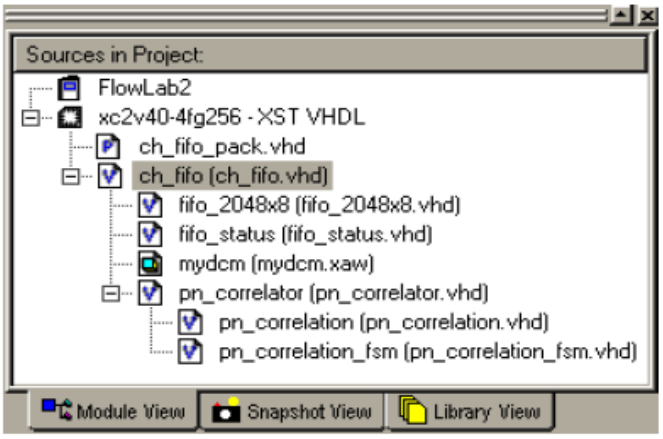


图2-108 项目源文件窗口

- 在当前源文件进程窗口，如图 19 中，双击实现设计 Implement Design，注意：工具将会执行实现设计所必需的所有进程，在本实现中在进入实现之前工具将先执行综合。

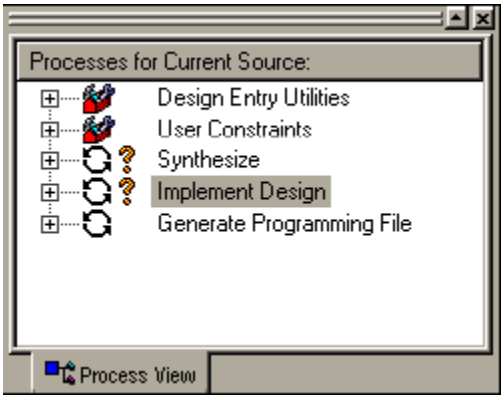


图2-109 当前文件进程窗口

2.5 经典电路

概述：收集相关设计公用电路，提供公用电路的经典设计方法。同事在项目开发中如果遇到相关电路的设计，可以直接使用经典电路，提供设计准确度；并且通过对经典电路的学习，可以提高设计人员的设计能力。

2.5.1 MPI 接口设计

一般来说，ASIC 的规格会兼顾多种工作模式，兼容多种工作环境等。具体工作模式、条件、运行参数等，视具体的应用场合而定。由于 ASIC（SOC 除外）自身不带处理器内核，所以必须和片外 CPU（通常是 MCU）配合使用。

单板上电或复位后，需要根据应用场合，由 CPU 配置 ASIC 的工作模式、条件、运行参数。在芯片运行中，芯片会向 CPU 反馈状态信息、请求中断等。ASIC 内与 CPU 接口的电路，称为 MPI（Micro processor interface）接口。CPU 对 ASIC 的配置，是通过 ASIC 内部的配置寄存器实现；CPU 获取 ASIC 状态，则通过 ASIC 内的状态寄存器、计数器等获得。

单板上 CPU 的输入参考时钟，称为 MPI 时钟（也称为 CPU 时钟），此处标记为 clk_mpi。ASIC 也有输入参考时钟，输入时钟的个数可能不止一个。当芯片内的电路主要工作在一个时钟域时，我们通常命名该时钟为主时钟。当芯片内部有多个时钟域，没有任何一个时钟占主导地位时，主时钟的概念已无意义，此时我们将这些时钟称为芯片的业务时钟，此处标记为 clk_a、clk_b、clk_c……主时钟也可以认为是一业务时钟。

MPI 接口，以 CPU 为主导，接口时序是 clk_mpi 时钟域的时序。clk_mpi 与业务时钟通常是不同频率也不同源的时钟。当配置信号从 CPU 送入 ASIC，状态信号从 ASIC 发往 CPU 时，信号将跨越不同的时钟域。MPI 接口的难点，就在于对异步信号的处理。

1. 寄存器类型

常用的寄存器通常有以下几种：RW 寄存器、RO 寄存器、RC 寄存器、WC 寄存器。

- RW 寄存器，可读可写，一般来说，就是配置寄存器。
- RO 寄存器，只读，指示即时状态信息。
- RC 寄存器，读清，指示历史状态信息。读的同时，寄存器的值以即时值刷新。该类寄存器，通常也包括计数器等。
- WC 寄存器，写清（也是可读的），写入某值时，寄存器的值按写入值和即时值刷新。

2. CPU 与 ASIC 之间的接口时序。

CPU 与 ASIC 之间的接口时序，无非两种：Intel 模式和 Motorola 模式。每种模式分别有读和写两种时序。

Intel 模式的写时序如图 2-110 示，时序参数如表 2-26 所示。

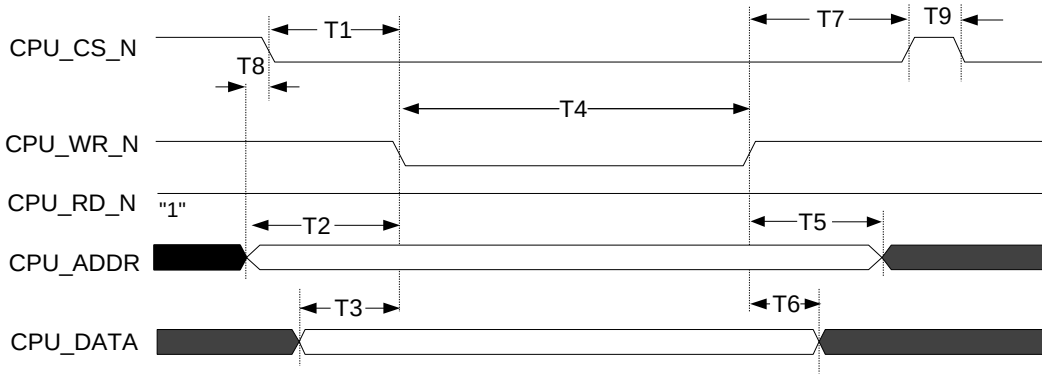


图2-110 Intel 模式写时序图

表2-26 Intel 模式写时序参数表

符号	最小值	典型值	最大值	描述
T1	0	-	-	从 CPU 片选有效到写信号有效的时间。
T2	0	-	-	从 CPU 地址稳定到写信号有效的时间。
T3	0	-	-	从 CPU 数据有效到写信号有效的时间。
T4	-	-	-	CPU 写信号有效的保持时间。
T5	0	-	-	从 CPU 写信号无效到地址无效的时间。
T6	0	-	-	从 CPU 写信号无效到数据无效的时间。
T7	0	-	-	从 CPU 写信号无效到片选信号无效的时间。
T8	0	-	-	从 CPU 地址稳定到片选有效的时间。
T9	-	-	-	CPU 相邻两次写操作间隔时间。

表 2-26 中，T4、T9 的宽带通常是 CPU 可控的，具体 ASIC 会要求具体的最小值。

Intel 模式的读时序如图 2-111 所示，时序参数如表 2-27 所示。

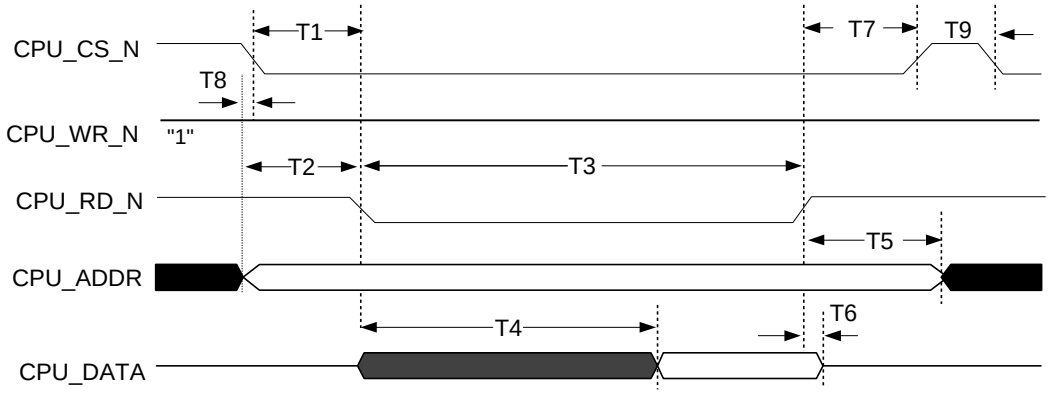


图2-111 Intel 模式读时序图

表2-27 Intel 模式读时序参数表

符号	最小值	典型值	最大值	描述
T1	0	-	-	从 CPU 片选有效到读信号有效的时间。
T2	0	-	-	从 CPU 地址稳定到读信号有效的时间。
T3	-	-	-	读信号有效的保持时间。
T4	-	-	-	从读信号有效到输出数据有效的时间。
T5	0	-	-	从读信号无效到地址无效的时间。
T6	0	-	-	数据在读信号无效后的保持时间。
T7	0	-	-	从 CPU 读信号无效到片选信号无效的时间。
T8	0	-	-	从 CPU 地址稳定到片选有效的时间。
T9	-	-	-	CPU 相邻两次读操作间隔时间。

表 2-27 中，T3、T9 的宽带通常是 CPU 可控的，具体 ASIC 会要求具体的最小值。T4 的值由具体 ASIC 内的设计决定。

Motorola 模式的写时序如图 2-112 所示，时序参数如表 2-28 所示。

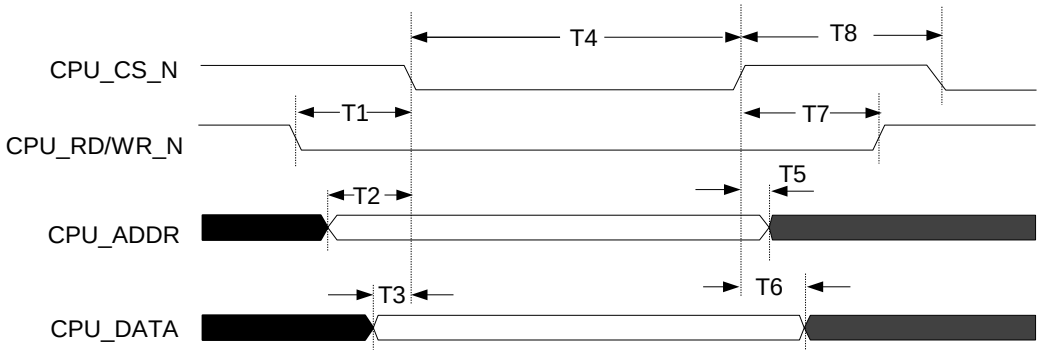


图2-112 Motorola 模式写时序图

表2-28 Motorola 模式写时序参数表

符号	最小值	典型值	最大值	描述
T1	0	-	-	从 CPU 写信号有效到片选信号有效的时间。

符号	最小值	典型值	最大值	描述
T2	0	-	-	从 CPU 地址稳定到片选有效的时间。
T3	0	-	-	从 CPU 数据有效到片选有效的时间。
T4	-	-	-	CPU 片选有效的保持时间。
T5	0	-	-	从 CPU 片选无效到写地址无效的时间。
T6	0	-	-	从 CPU 片选无效到写数据无效的时间。
T7	0	-	-	从 CPU 片选无效到写信号无效的时间。
T8	-	-	-	CPU 相邻两次写操作间隔时间。

表 2-28 中，T4、T8 的宽带通常是 CPU 可控的，具体 ASIC 会要求具体的最小值。

Motorola 模式的读时序如图 2-113 所示，时序参数如表 2-29 所示。

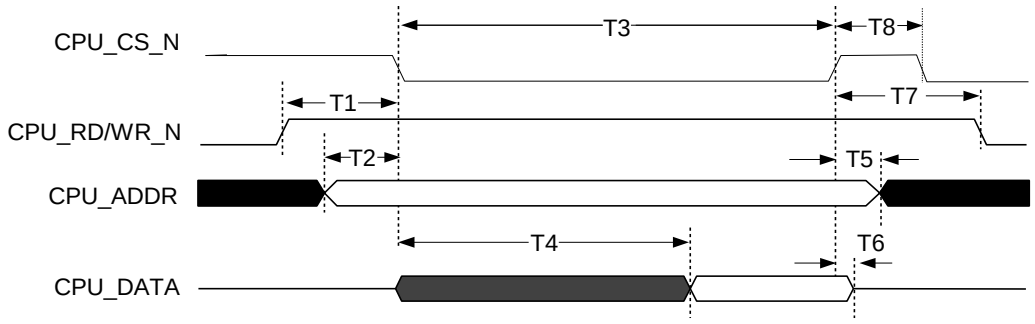


图2-113 Motorola 模式写时序图

表2-29 Motorola 模式读时序参数表

符号	最小值	典型值	最大值	描述
T1	0	-	-	从 CPU 读信号有效到片选有效的时间。
T2	0	-	-	从 CPU 地址稳定到片选信号有效的时间。
T3	-	-	-	片选信号有效的保持时间。
T4	-	-	-	从片选信号有效到输出数据有效的时间。
T5	0	-	-	从片选信号无效到地址无效的时间。
T6	0	-	-	数据在片选信号无效后的保持时间。
T7	0	-	-	从 CPU 片选信号无效到读信号无效的时间。
T8	-	-	-	CPU 相邻两次读操作间隔时间。

表 2-29 中，T3、T8 的宽带通常是 CPU 可控的，具体 ASIC 会要求具体的最小值。T4 的值由具体 ASIC 内的设计决定。

3. MPI 接口的常用结构

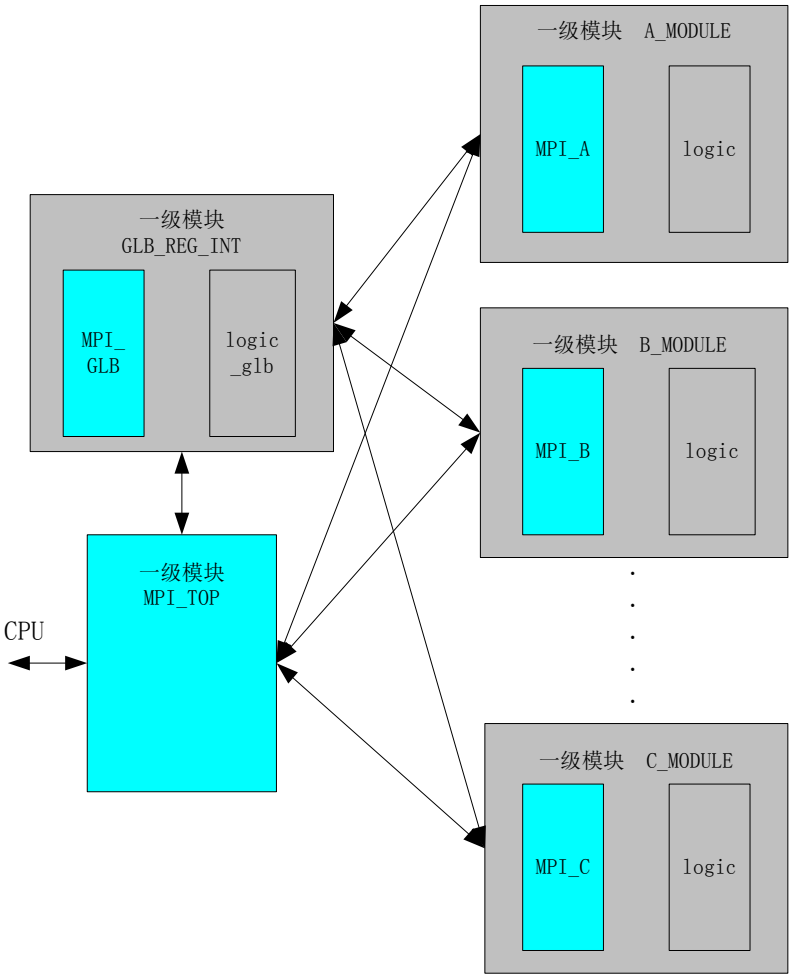


图2-114 MPI 结构图

ASIC 内的 MPI 接口通常分为两级。以图 2-114 为例。MPI_TOP 是第一级 MPI。例化在各个功能逻辑中的 MPI_A、MPI_B、MPI_C，属于第二级 MPI。

第一级 MPI 直接与 CPU 信号相接，主要完成以下功能：

- 检测外部片选、读写信号的有效性；
- 寄存地址、数据；产生内部读写使能信号；
- 完成一级译码；
- 汇集一级模块的输出数据，送往 CPU；
- 产生输出 PAD 的三态控制信号。

功能模块中的二级 MPI 主要完成的功能是：

- 模块内的地址译码；
- 配置控制寄存器，配置由 RAM 实现的控制寄存器；将控制信号送往功能逻辑；
- 收集功能逻辑的即时状态，保存历史状态；进行性能计数；（即构造即时状态寄存器、历史状态寄存器、性能计数器等，包括由 RAM 实现的即时、历史、性能计数寄存器等；）

- 构造中断树；
- 向一级 MPI 输出读数据。
- 上报与全局有关的状态信息和中断信息。
- clk_mpi 时钟域与业务时钟域之间的异步信号处理；

ASIC 中都有与整个芯片全局功能相关的寄存器，该部分的 MPI 一般不与功能逻辑例化在一起，而是通常单立成一模块，如图 2-114 中的 GLB_REG_INT。该部分的功能主要是：

- 进行全局配置，将全局控制信号送往各功能模块；
- 收集各功能模块上报的与全局有关的状态信息；保存历史状态信息；性能计数等；
- 汇集功能模块上报的中断信息，与全局中断合并，产生中断树的高层节点和树根节点；
- 向一级 MPI 输出本模块的读数据。

4. MPI 接口的示例时序

如前所述，MPI 设计的难点和焦点，都在于异步信号处理。

写时序。写操作是把配置数据写入配置寄存器，并将已同步后配置信号送入功能逻辑。

根据同步方式，写操作可分为两种处理方式。

- 配置寄存器分两级，第一级以 clk_mpi 实现，第二级以业务时钟实现。配置数据写入第一级寄存器后，以业务时钟将配置信号同步到业务时钟域。寄存器结构如图 2-115 所示。（注意：该图只是简单地表示了寄存器的结构，省略了所有的地址译码、写使能等信号，并非完整的信号图。以下同。）

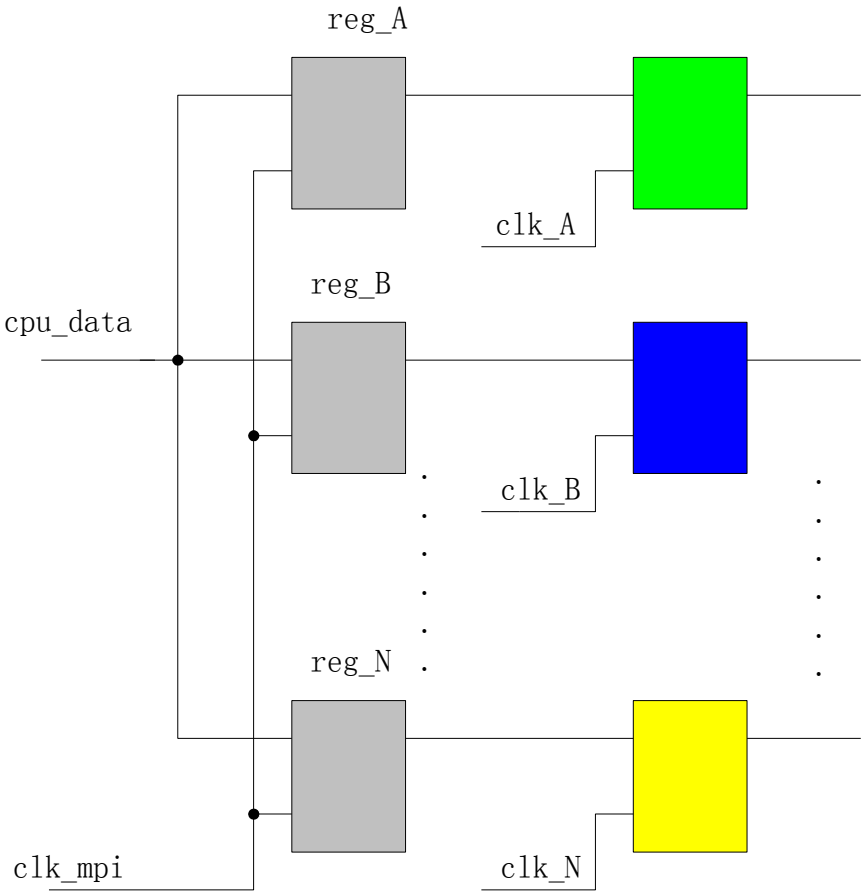


图2-115 两级寄存器的结构

相应的时序如图 2-116 所示。

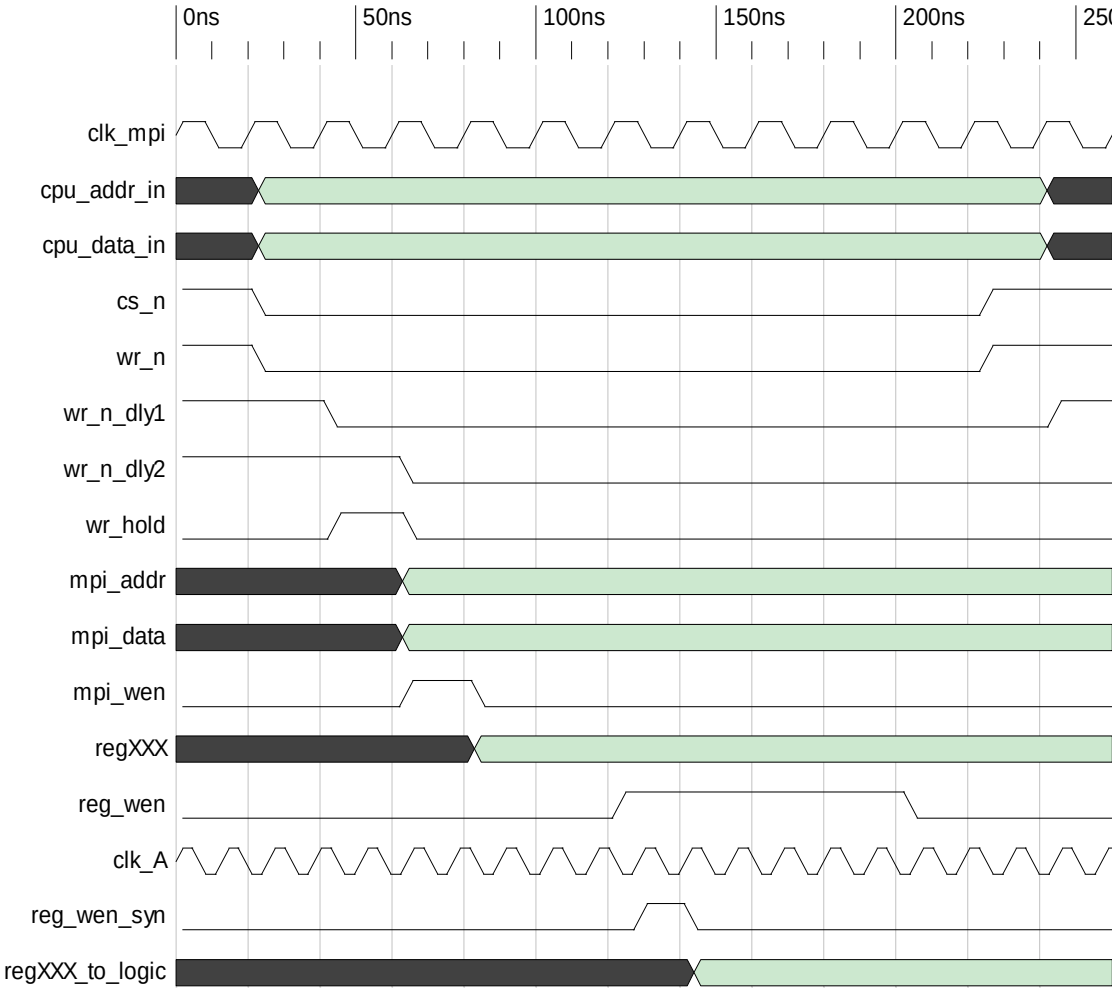


图2-116 写时序 1

mpi_wen 有效时，配置数据被写入第一级寄存器。reg_wen_syn 有效时，配置数据被同步到第二级寄存器，即业务时钟域，而后进入功能逻辑。

- 第二种实现方式是，配置寄存器只有一级，以业务时钟域实现。配置寄存器被写入数据前，地址、输入数据、写使能等已经被同步到业务时钟域。寄存器的信号进入功能逻辑时，无需再做同步。

寄存器结构如图 2-117，相应的时序如图 2-118 所示。

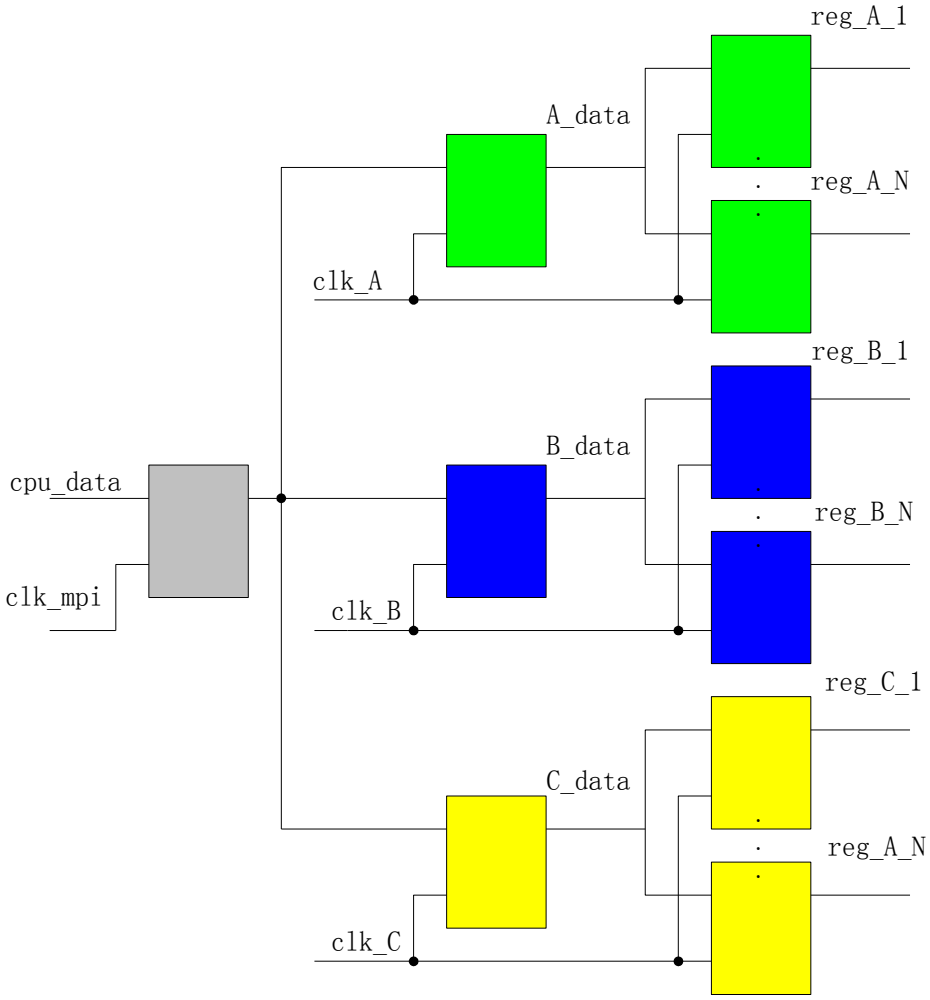


图2-117 单级寄存器的结构

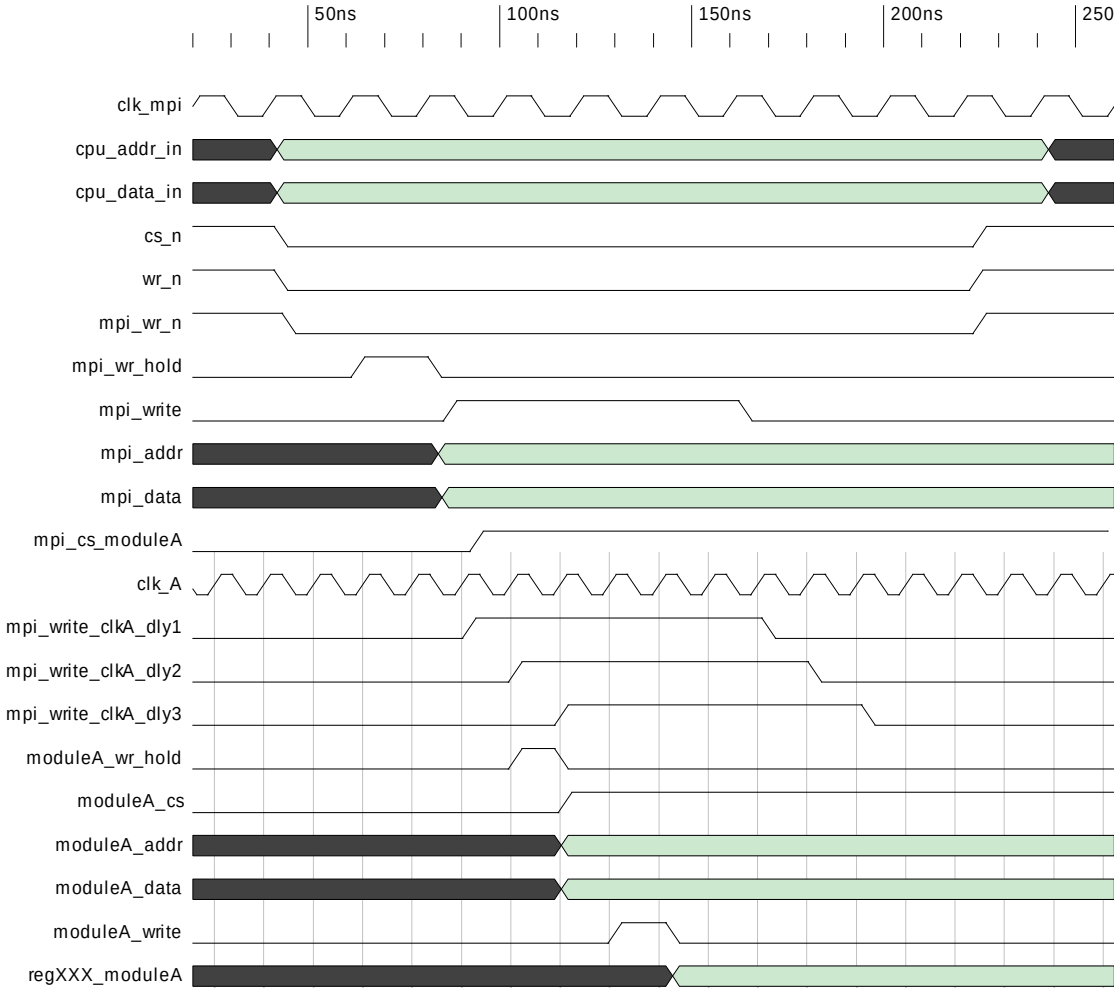


图2-118 写时序 2

moduleA_wr_hold 有效时，moduleA 的片选、地址、输入数据等都被同步到业务时钟域，写入寄存器时后，已无需再次同步。

读（清）时序。RO、RC 寄存器通常在相应的业务时钟域内实现，目的是避免状态信号跨时钟域时可能出现的遗漏。示例读时序如图 2-119。

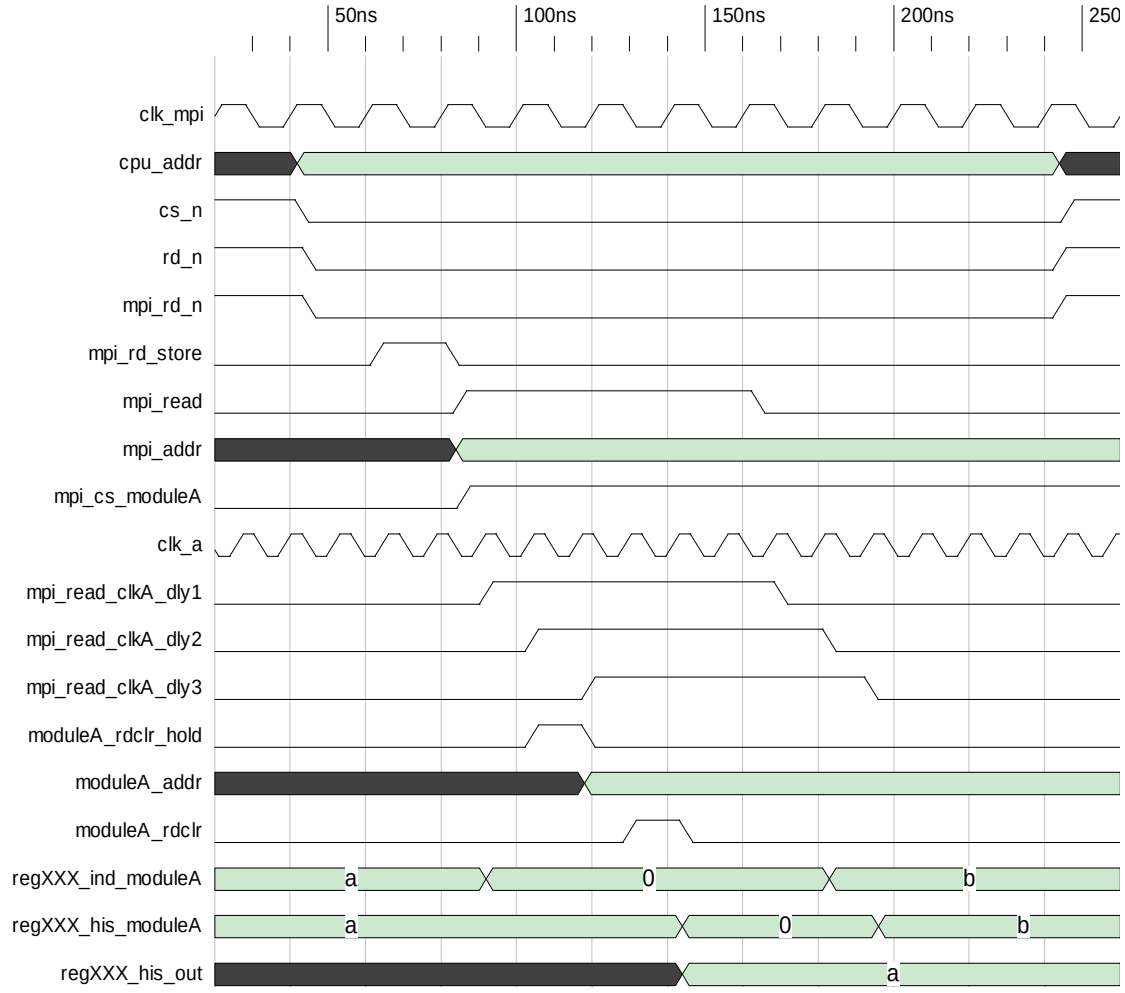


图2-119 读时序

在 moduleA_rdclr_hold 有效时，moduleA 的片选、地址都被同步到业务时钟域。在 moduleA_rdclr 有效时，历史寄存器的值被即时值刷新，历史值被输出。

MPI 接口没有任何协议可遵循，具体芯片的时钟结构各异，实现方式非常灵活。不同的时钟结构，可能要求不同的同步实现方式。

2.5.2 中断电路设计

1. 概述

本节中芯片一词是指完成特定功能的 ASIC 芯片，需要配合外部 CPU 共同构成一个完整的中断系统。对于 SOC 类芯片的中断设计并不在本文的讨论范围之内。

通常在芯片正常工作时会对某些特殊事件或状态进行监视，当某个特殊事件或状态发生时产生告警，并且当该告警的中断屏蔽位未被屏蔽时，芯片可以通过中断的方式向 CPU 发出中断请求，通知 CPU 进入相应的中断服务程序，对发出中断请求的中断源进行处理。

芯片中断信号的产生是由芯片中的中断控制电路完成的，芯片内部的中断控制电路的 Feature List 如下所述：

- 根据内部实时告警信号的状态产生内部中断信号；
- 中断控制逻辑通常采用树形的结构进行组织，中断树结构一般为 3～5 级，树形的中断结构便于中断源的管理和查询；
- 中断源通常按照产生中断的告警类型、优先级，以及所处的功能模块等规则进行划分，以便于 CPU 可以按照功能或者优先级对中断源查询和处理，以及便于芯片的逻辑设计；
- 中断树上的所有中断节点都可以通过设置屏蔽位进行屏蔽，其中包括根节点、支干节点和叶节点；
- 与中断控制逻辑有关的几种寄存器包括：实时告警寄存器、中断屏蔽寄存器、中断指示寄存器、中断源索引寄存器；
- 芯片内部中断信号产生包括电平触发和沿触发两种方式，其中沿触发又包括：单沿触发和双沿触发；
- 中断指示寄存器的清除方式通常包括读清和写清两种方式；
- 从芯片中断引脚送出的中断信号通常为电平触发信号，高低有效电平的选择需要根据具体的应用而定。

2. 芯片中断结构

(1) 中断树结构

中断控制电路通常采用树形的结构进行组织，CPU 按照树形结构从根节点开始逐级查询中断寄存器，从而快速定位中断源。

中断控制逻辑通常包括以下几种寄存器：实时告警寄存器、中断指示寄存器、中断源索引寄存器、中断屏蔽寄存器。

- 实时告警反映了芯片内部告警的实时状态，其值随实时状态变化，为只读类型；
- 中断指示寄存器由实时告警的状态触发，并产生中断信号，触发方式包括电平触发和沿触发两种方式，为读清或者写清类型；
- 中断源索引寄存器用于查询中断源的位置，为只读类型；
- 中断屏蔽寄存器用于屏蔽中断指示寄存器和中断源索引寄存器，为可读写类型。

中断指示寄存器和中断源索引寄存器的输出可以通过设置对应的中断屏蔽寄存器加以屏蔽，这样逐级的可屏蔽控制的中断结构便于 CPU 对芯片内部中断源的管理、控制和定位。

中断指示寄存器对应中断树中的叶节点，中断源索引寄存器对应中断树中的支干节点和根节点。

通常越靠近中断树根节点的中断屏蔽位其屏蔽的级别就越高，当屏蔽某个中断节点，该节点以下的所有中断源都被屏蔽。通常需要在芯片内放置一个芯片级全局中断屏蔽寄存器，用于屏蔽中断树根节点中断，当根节点中断被屏蔽时，芯片内所有的中断源将被屏蔽，此时芯片不会产生外部中断信号。

一个采用电平触发方式的 5 级中断树结构示意图如图 2-120 所示。

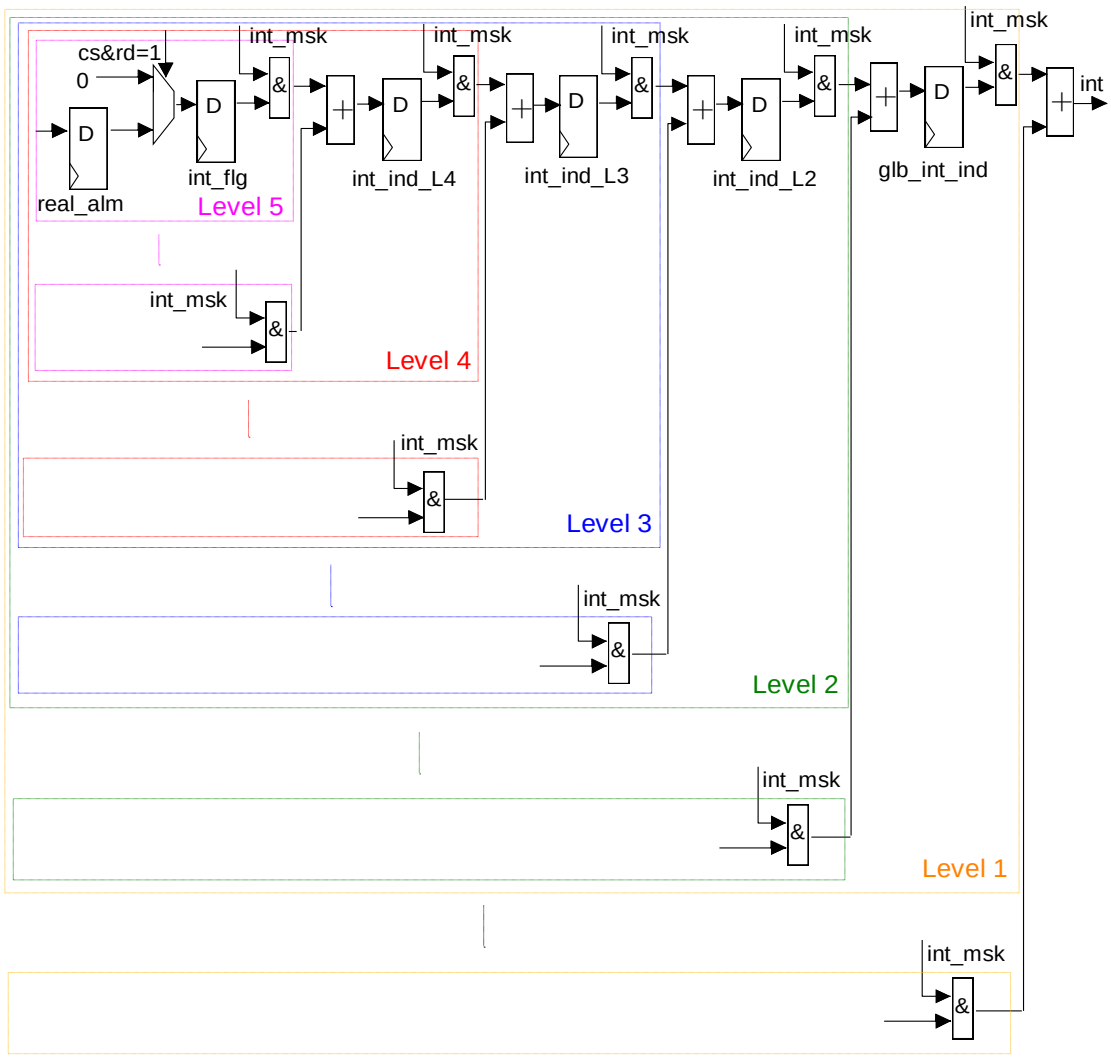


图2-120 中断树结构示意图

对上图中的中断结构作如下说明：

- real_alm：实时告警寄存器
- int_flg：中断指示寄存器，为最后一级中断
- int_ind_Ln：第 n 级中断源索引寄存器
- glb_int_ind：全局中断源索引寄存器，为第一级中断
- int_msk：各级中断屏蔽寄存器

通常除了中断树根节点之外的所有节点都位于各个功能模块，各级中断节点在各个功能模块内逐级组合成一个中断信号输出到顶层，最终组合成芯片级的中断信号。功能模块内部任何一个中断指示的产生，如果没有屏蔽的话，都将引起功能模块内其上一级中断源索引寄存器的置位，而中间层次的中断源索引寄存器同样需要在没有屏蔽的情况才能引起其上一级中断节点的置位。

中断指示寄存器和中断源索引寄存器都可以通过中断屏蔽寄存器进行屏蔽。

中断源查询的顺序与中断产生的顺序相反。当中断产生后，CPU 对中断源进行查询，首先读取全局中断源索引寄存器（glb_int_ind），从中判断中断产生于

哪个或者哪几个一级中断分支，按照由根到叶的次序依次查询下一级的中断源索引寄存器，直至叶节点。如果在同一个层次中同时有多个中断指示，CPU 将按照中断的优先级分别进行查询和处理。

(2) 中断源的划分规则

在中断系统中，当多个中断源同时发出中断请求时，CPU 通常按照中断的优先级的高低顺序依次响应不同级别的中断源，因此需要给每个中断源指定一个优先级。中断优先级的指定由软件完成。从芯片的角度来看，芯片只需要按照特定的规则将所有的中断源划分到不同的中断树分支就可以了，也就是说中断优先级对于硬件来说是透明的。

芯片中断源划分的规则很多，但最终的目的都是要达到便于 CPU 可以高效率快速地对中断源进行查询和处理，以及芯片设计方便的目的。那么怎样才能把一个中断树组织得更合理、更有效？这里列举几点通常需要考虑的方面。

中断源的划分规则如下：

- 按照芯片内部的主要功能模块以及处理方向划分芯片中断源。例如，在 SDH 开销处理芯片中，交叉模块、指针模块等都是功能单独的模块，可以按照功能模块以及接收和发送处理方向来划分中断源。按照功能模块划分中断源的方式主要是从便于芯片内部逻辑设计的角度考虑的。
- 按照外部接口划分中断源，以便于 CPU 能够快速地按照接口查询和屏蔽中断源。例如，对于 SDH 线路板，软件通常按照光口查询和屏蔽中断源，因此在线路板上的开销处理芯片线路侧的告警需要按照光口号组织划分中断源，将同一个光口的中断源划分为同一个分支。
- 将系统软件所关注的具有特殊优先级的中断源单独划分出来，以便于 CPU 快速获取中断源类型和优先级，加快中断处理的响应速度。例如，SDH 开销处理芯片上告的新 K 字节告警会引起业务倒换，并且倒换完成时间严格控制在 50ms 以内，因此在开销芯片中需要将新 K 字节告警中断源单独划分在一个分支上。
- 按照芯片内部的处理通道的序号划分告警中断源。例如，SDH 交叉类芯片以高低阶通道为粒度进行处理，芯片内部结构由多个功能完全相同的通道处理单元组成，因此可以按照通道序号来划分中断源。此外，如果处理的通道较多，可以按照层次化的方法将相邻的多个通道的中断源在更高层次的中断节点上进行合并，以便于中断树的硬件实现和软件查询。
- 中断树结构是按照层次进行组织的，显而易见，如果树形结构的层次越多，CPU 查询中断源所耗费的时间和资源就越多。确定中断树层次的数目需要综合考虑芯片总的中断源的数目和中断树组织的方式。通常芯片中断树层次确定为 3～5 级。比如，如果按照每个节点下最多划分为 16 个子节点计算，一个 4 级的中断树结构可以管理 65,536 个中断源，这个数目基本上就可以满足一般的需求。

- 在中断树结构中，每个节点下划分的子节点的数目通常与 CPU 接口数据宽度有关。例如对于 16 比特数据宽度的 CPU，内部寄存器宽度最大为 16 比特，因此中断树每个节点下也就最多划分为 16 个子节点。

(3) 一个例子

这里举一个 SDH 开销处理芯片中断树的例子，其中断树的基本结构如图 2-121 所示。

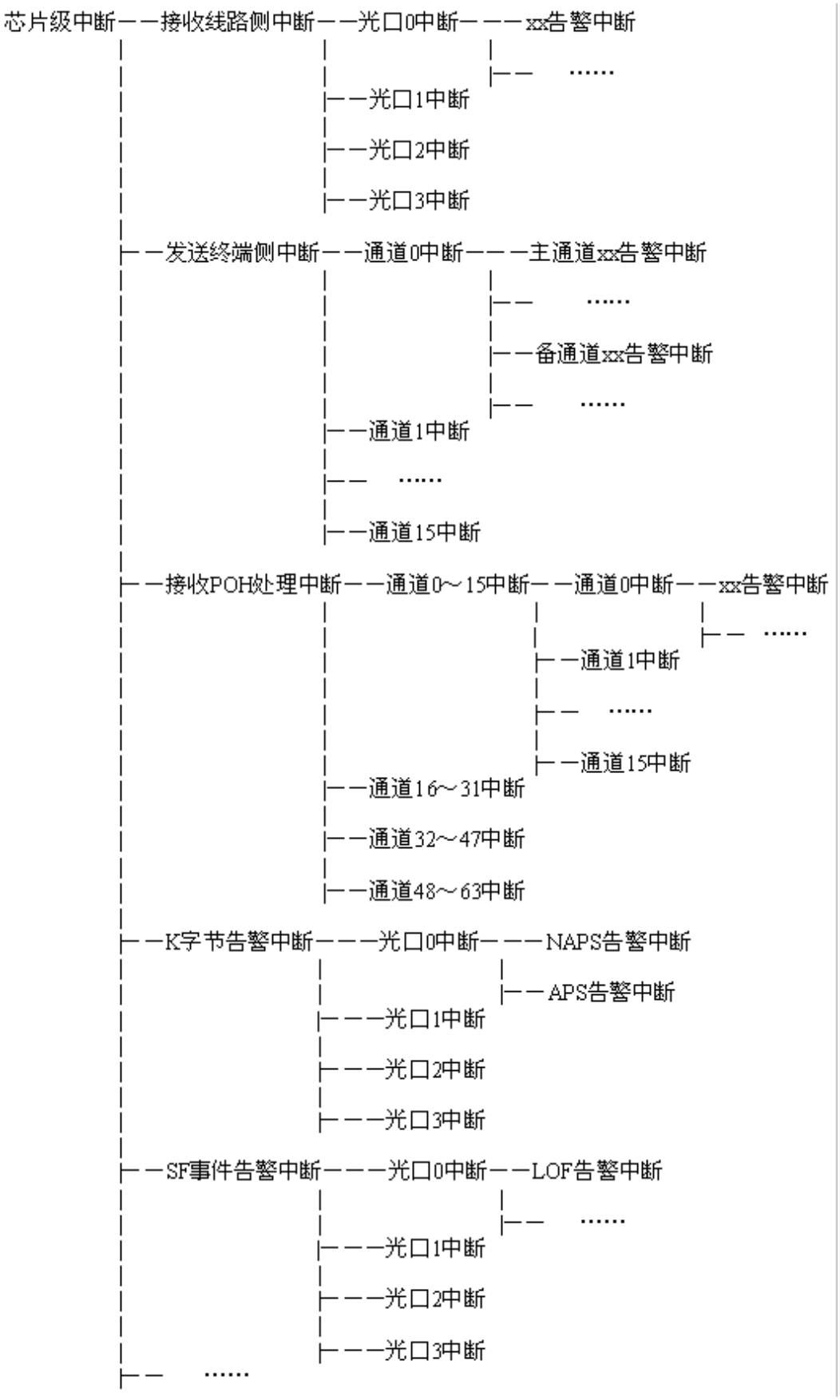


图2-121 中断树的基本结构图

(4) 总结

结合上文提到的中断源划分规则，对该芯片的中断树小结如下：

- 该中断树层次最深可达四级。例如：接收 POH 处理模块中断分支的层次数目（对照规则 5）。
- 每一级最多划分为 16 个子节点，这一点取决于 CPU 数据总线宽度为 16 比特（对照规则 6）。
- 第一级中断按照芯片内部的主要功能模块以及处理方向进行划分。例如：接收线路侧中断、接收 POH 处理中断等（对照规则 1）。
- 将软件关注的具有特殊优先级的中断源单独划分出来，例如：SF 事件告警中断、K 字节告警中断等（对照规则 3）。
- 按照芯片外部接口划分中断源，例如：接收线路侧中断的第二级中断按照光口划分中断源（对照规则 2）。
- 接收 POH 处理中断的第二、三级中断按照内部的处理通道的序号划分中断源，并且当中断源数目大于 16 时，将 64 个通道的中断源按每 16 个通道一组再次进行组合成 4 组中断源（对照规则 4）。

3. 中断触发方式

中断触发方式主要包括电平触发和沿触发两种方式。

(1) 电平触发方式

电平触发方式是指实时告警产生通过电平方式触发中断指示寄存器，产生中断信号。电平触发中断方式的电路描述如下：

```
always @（posedge clk or negedge rst_n）
begin
    if（rst_n == 1’b0）
        int_flg <= 1’b0;
    else
    begin
        if（real_alm == 1’b1）－－中断条件逻辑：实时告警有效
            int_flg <= 1’b1;
        else if（清零有效）－－中断清零逻辑
            int_flg <= 1’b0;
    end
end
assign int = int_flg & int_msk;
```

(2) 沿触发方式

沿触发方式是指通过检测实时告警信号的跳变沿来触发中断指示寄存器，产生中断信号。沿触发又包括两种方式：单沿触发和双沿触发。

单沿触发是指在只在告警开始时产生中断信号，而双沿触发是指在告警开始和结束时都产生中断信号。因此单沿触发通常需要检测实时告警信号的上升沿（这里假设告警信号高电平表示有告警产生），双沿触发通常需要分别检测实时告警信号的上升沿和下降沿。

单沿触发中断方式的电路描述如下：

```
always @（posedge clk or negedge rst_n）
begin
    if（rst_n == 1’b0）
        int_flg <= 1’b0;
    else
    begin
        if（real_alm_rise == 1’b1）——中断触发逻辑：实时告警上升沿有效
            int_flg <= 1’b1;
        else if（清零有效） ——中断清零逻辑
            int_flg <= 1’b0;
    end
end
assign int = int_flg & int_msk;
```

双沿触发中断方式的电路描述如下：

```
always @（posedge clk or negedge rst_n）
begin
    if（rst_n == 1’b0）
        int_flg <= 1’b0;
    else
    begin
        if（（real_alm_rise == 1’b1）||（（int_mode == 1’b1） &&（real_alm_fall == 1’b1））） ——中断触发逻辑：real_alm_rise 表示实时告警上升沿，real_alm_fall 表示实时告警下降沿
            int_flg <= 1’b1;
        else if（清零有效） ——中断清零逻辑
            int_flg <= 1’b0;
    end
```

```
end
assign int = int_flg & int_msk;
```

为了便于灵活应用，通常可以通过一个全局的配置信号来配置中断触发方式，可以将中断逻辑选择配置为单沿触发和双沿触发，如上图中的 int_mode 信号为中断模式选择信号，int_mode＝0 表示单沿触发，int_mode＝1 表示双沿触发。

4. 中断清除方式

中断清除是指 CPU 通过中断树查询中断源后，将中断指示寄存器清零的操作。中断清除的方式有两种，分别是读清和写 1 清。下面以分别介绍：

(1) 读清方式

读清方式是指中断指示寄存器清零逻辑由读信号、片选和地址构成，在 CPU 完成对中断指示寄存器的读操作后将中断指示寄存器清零。

优点：清零操作简单，软件只需要进行一次读操作即可。

缺点：在完成读操作后，将该地址所有的中断指示寄存器全部清零了，如果这个地址的中断指示寄存器属于不同的中断级别，会造成异常清零的后果。

例如下面的应用场景：芯片设置两种类型的中断，类型 A 对应新 K 字节中断事件，类型 B 对应其它告警事件。A 型中断具有更高的优先级。16bit 的寄存器 C 是一个中断指示寄存器，其 bit[15:14]存放 A 型中断的中断指示信号，bit[13:0]存放 B 型中断的中断指示信号。

如果在某个时刻，bit[15]和 bit[7]中断指示信号被同时置位，这样将会同时触发类型 A 和类型 B 的中断。软件首先响应 A 中断，在查询中断源的过程中，对寄存器 C 进行读取操作，会同时将 bit[15]和 bit[7]清零。而后，软件响应 B 型中断，就会面临找不到中断源的情况。

为了规避上面的情况，采用读清方式清除中断的时候，需要首先将中断源进行分组，不要将不同类型的中断源放置到同一个地址下。

(2) 写 1 清方式

写 1 清方式是指清零逻辑由写信号、片选、地址和写数据构成；CPU 通过读操作查询中断源，通过写操作对中断指示寄存器清零。

优点：CPU 查询中断源和清除中断指示寄存器的操作是分离的，很好的从设计上规避了读清方式的缺陷。

仍然以上面的应用场景为例：CPU 在响应 A 型中断的时候，通过查询中断源的操作对寄存器 C 进行读操作，虽然同时读到 bit[15]和 bit[7]为 1，但是只有 bit[15]才是需要处理的中断源；而后 CPU 向寄存器 C 写入 0x8000，将 bit[15]清零。这样在完成 A 型中断的处理后，bit[7]依然维持为有效状态。

缺点：中断查询和中断指示寄存器的清零需要 CPU 通过两步操作实现，稍显麻烦。

2.5.3 复位电路设计

1. 应用场景

复位电路在 ASIC 芯片的设计中是必不可少的，如状态机使用前或死机要复位以进入确定的状态、RAM 使用前要复位、非配置寄存器复位、配置寄存器复位、为降低功耗要对不使用的逻辑复位等。对于绝大多数芯片来说，复位模块的功能也是完全相同的。复位的来源可能是一个芯片管脚引入（硬复位）或者内部配置寄存器产生（软复位）。复位设计是指复位信号的生成，面临的问题主要是去毛刺、时钟域的同步、可否做 STA 检查、以及达到这一目的对时钟的依赖程度有多大等。而对复位信号的使用则一般采用异步复位，这样即使在时钟失效的情况下，有效的复位也能让逻辑进入正常状态。

2. 电路说明

下面给出一些经典的复位设计方法和电路结构，并分析各自的优缺点，以供工程技术人员参考。（这里有效的复位电平定为低）

(1) 寄存器延迟的同步处理复位结构（推荐）

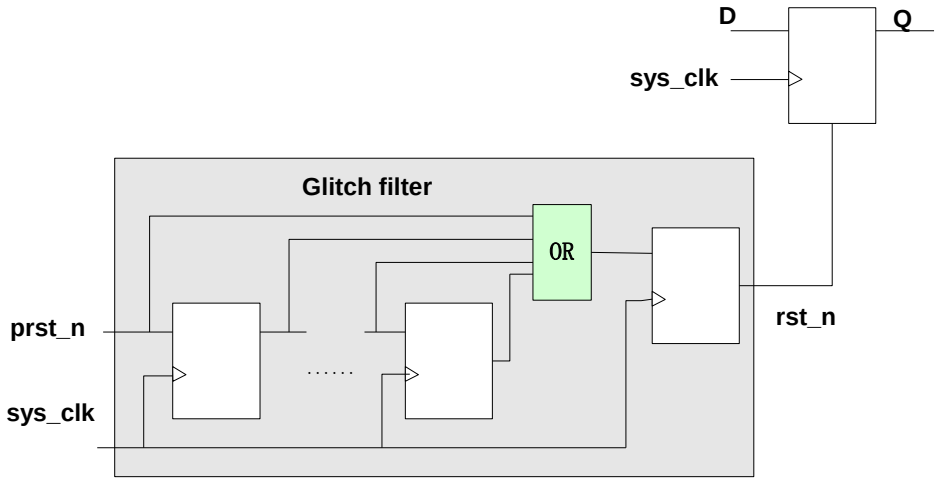


图2-122 寄存器延迟的同步处理复位结构

上面的方案是在我们的设计中常常被采用的。该设计中采用了不带异步复位端的寄存器来实现滤毛刺功能。外部管脚过来的异步复位信号利用系统时钟打几拍作延时，然后将延时后的数据取或，并利用系统时钟同步之后作为全局异步复位信号输出。

该方案的优点：

- 可以稳定的滤掉毛刺，不会再次引入新的窄脉冲；
- 对于 sys_clk 时钟域中的寄存器，可以进行异步复位端 removal/recovery 的 STA 分析

该方案的缺点：

- 系统复位的时候需要 sys_clk 有效，否则复位信号无法正常生成。

- 对于不同于 sys_clk 的时钟域，不能进行 removal/recovery 的 STA 分析。

(2) 专用延迟逻辑的同步处理复位方案（推荐）

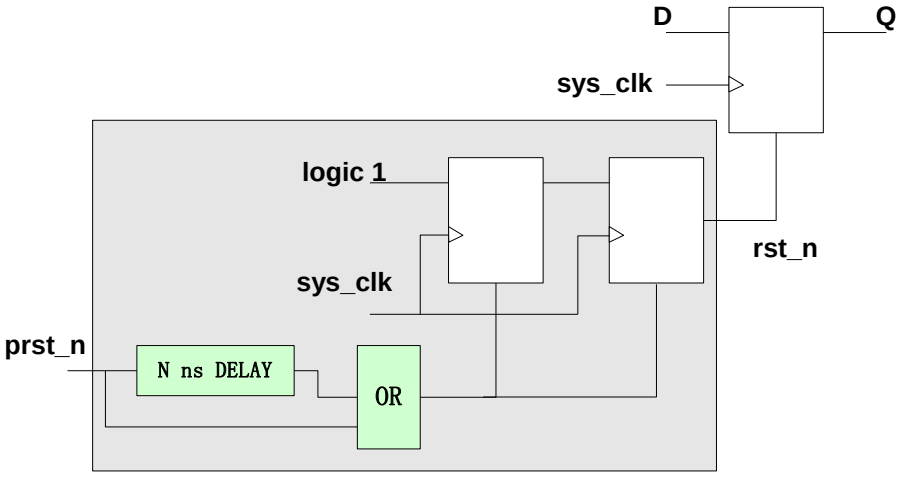


图2-123 异步处理的异步复位结构

图 2-123 是异步复位模块基本结构的设计方案。其中由滤毛刺电路（Glitch Filter）和一个用于同步的两级寄存器组成。来自于外部管脚的异步复位信号（prst_n）经过由组合逻辑（可由 vendor 提供延迟单元）组成的滤毛刺电路后分别接到两级寄存器的异步复位端。

在正常工作时，rst_n 为高电平。在进行复位时，外部管脚的复位信号经过滤毛刺后作用于两级寄存器上，使得该寄存器的输出（rst_n）为低电平，作为芯片内部的全局复位信号。当外部管脚的复位信号（prst_n）撤消时，内部的复位信号（rst_n）并不会立即撤消，其上升沿是通过寄存器同步后输出的。

该方案的优点：

- 可以稳定的滤掉毛刺，滤毛刺时不需要时钟。由于复位信号采用组合逻辑滤毛刺，并且接寄存器的异步复位端，因此即使在时钟丢失的情况下，仍然能够通过复位管脚对芯片内的寄存器进行复位，使其处于确定的状态；此时的逻辑相当于退化为通常的异步复位功能。
- 因为内部的复位信号是经过所在时钟域同步处理后的信号，复位信号与时钟具有确定的相位关系，可以对寄存器的异步复位端进行 removal/recovery 的分析，利用 STA 分析保证时序的正确性，增加 STA 分析的覆盖面。

该方案的缺点：

- 对于不同于 sys_clk 的时钟域，不能进行 removal/recovery 的 STA 分析。

(3) 同步处理复位方案（多时钟域）

大多数的 ASIC 芯片都涉及到多个时钟域。上面的复位方案是针对某个特定的时钟域（sys_clk）给出的，对于多个时钟域的情况，则要针对各个时钟域分别进行相应的复位模块的设计。

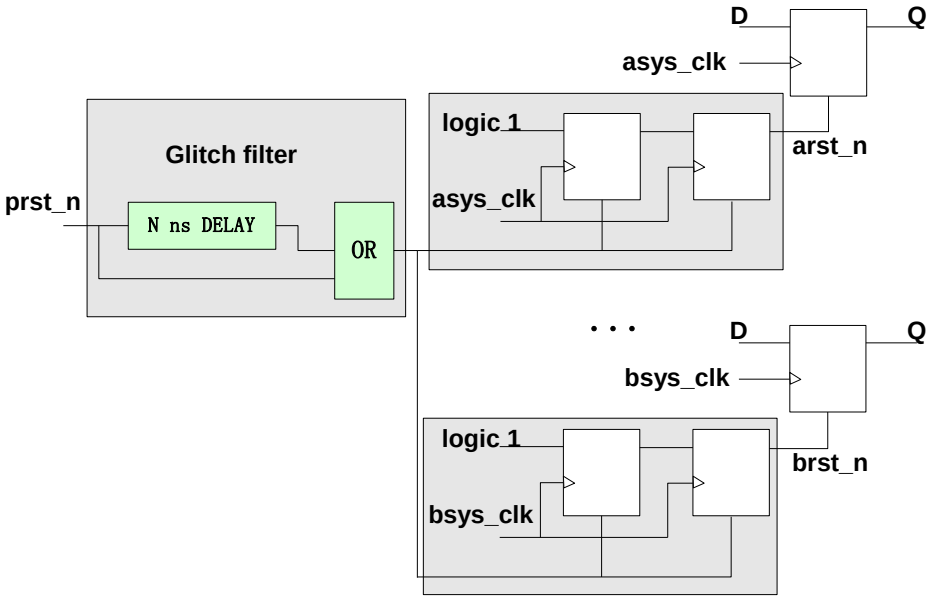


图2-124 多时钟域的复位设计

由图中可以看到，由管脚输入的复位信号经过公共的组合逻辑滤毛刺模块后，利用各个时钟域的时钟分别进行同步，生成各个时钟域内部的复位信号。

(4) 专用延迟单元的纯异步结构（不推荐）

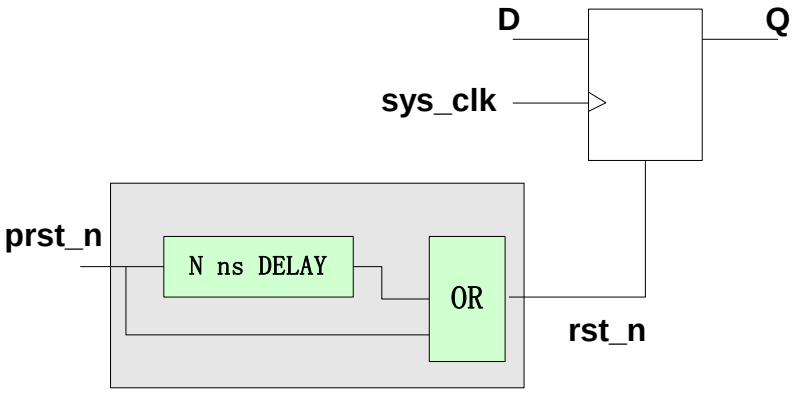


图2-125 专用延迟单元的纯异步结构

复位模块采用了纯粹的组合逻辑实现滤毛刺的功能，经过滤毛刺处理的复位信号直接和内部寄存器的异步复位端相连。

该方案的优点：

- 完全意义的异步复位，复位信号的生成无需时钟。

该方案的缺点：

- 生成的复位信号与触发器工作时钟 sys_clk 异步，时钟无法对内部寄存器的复位端进行 removal/recovery 的分析。
- DFT 需增加特别的设计逻辑。

(5) 寄存器延迟的纯同步结构（不推荐）

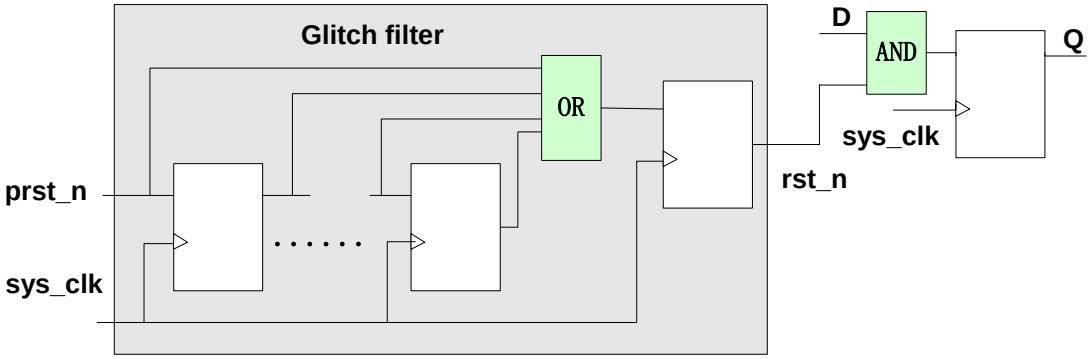


图2-126 寄存器延迟的纯同步结构

该方案的优点：

- 完全意义的同步设计。
- 可以对复位信号进行时序分析，增大了 STA 的覆盖面。
- DFT 无需增加特别的设计逻辑。

该方案的缺点：

- 复位信号的生成和生效均需要相应时钟域的时钟，否则无法复位。（这一点在有些时候是致命的）。
- 可能会增加额外的逻辑（厂家的库可能不提供同步复位寄存器）

(6) 若干注意点

【复位顺序】

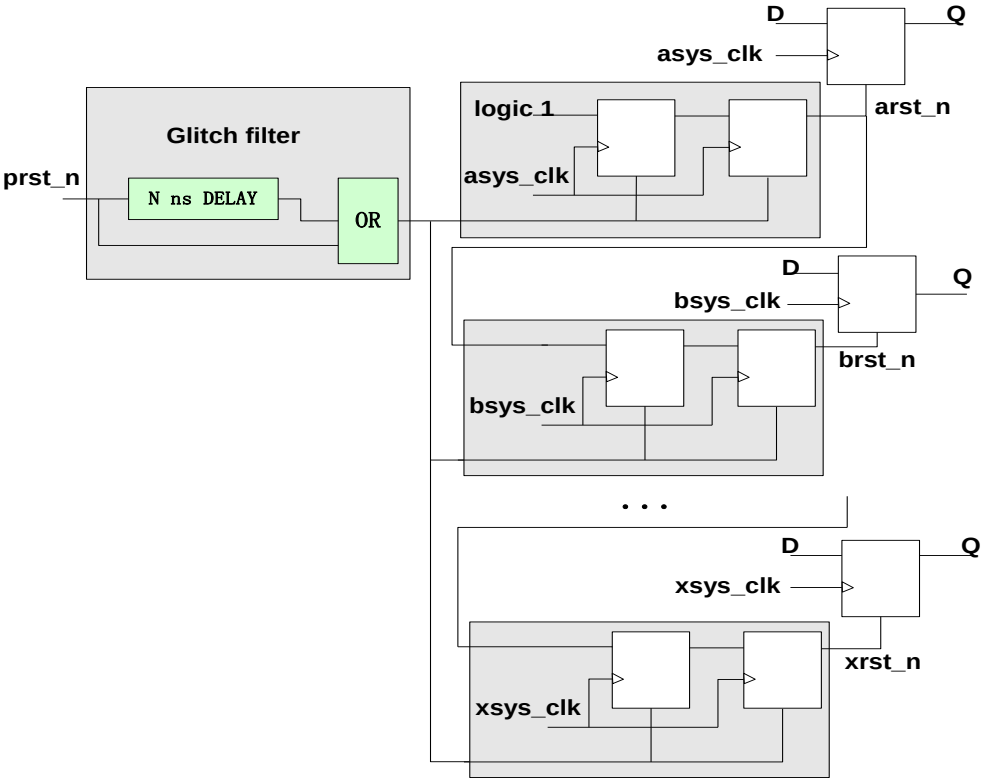
在多时钟域时，各个模块的复位顺序在如下两种情况下可能会比较重要，甚至造成麻烦。

一种是在多个时钟域中存在时钟频率差异比较大的几个时钟时，复位逻辑将会导致某个时钟域先复位完毕，而部分时钟域仍然处于复位状态，由此可能会造成逻辑的错误。

另一种情况是设计本身对复位顺序有严格要求。值得注意的是，复位顺序的问题是以上几种方案都要面对的问题，而并不是某种设计方案本身引入的新问题。

对于复位顺序敏感的设计，若采用前一种方案，可以通过触发器打拍很方便的给出具有先后顺序的复位信号。

若采用后一种方案 2 可以使用下图所示的“串联”的方法，合理的确定复位的先后顺序。



【多时钟的处理】

对于多时钟域的复位模块可以针对具体的情况灵活处理，例如整个芯片以某个单一的时钟域为主，只是在接口部分引入了额外的时钟域，而且该部分逻辑比较简单，则可以考虑整个芯片只使用一个单一的复位信号。但此时一定要分析电路，确保复位的正确性，特别是要分析在复位撤消时可能出现的复位状态不确定对设计造成的影响。

对于内部分频出来的时钟域，可以不用另外增加复位信号 ，而是直接使用其源时钟域的复位信号。因为其源时钟域的复位信号与分频时钟是可以通过 STA 保证其相位关系的。

在芯片存在多时钟域时，选取带触发器最多的时钟，在各时钟域负载都比较均衡的时候，选取最不可能丢失的时钟。

【编码方面的注意点】

由于在多时钟域时复位信号需要区分时钟域，因此在编码的时候也需要对不同时钟域的复位信号区别对待，由此可能在编码中引入一些错误，比如笔误等造成信号错误（可能要到 STA 的时候才能发现），对此要尤为注意。

【对后端流程的影响】

将复位信号根据时钟域的不同进行划分后，使得对复位信号的布线与以前的将它作为一个整体来布线略有不同。在后端处理的过程中需要注意。此变化不会给后端流程造成太大的影响。

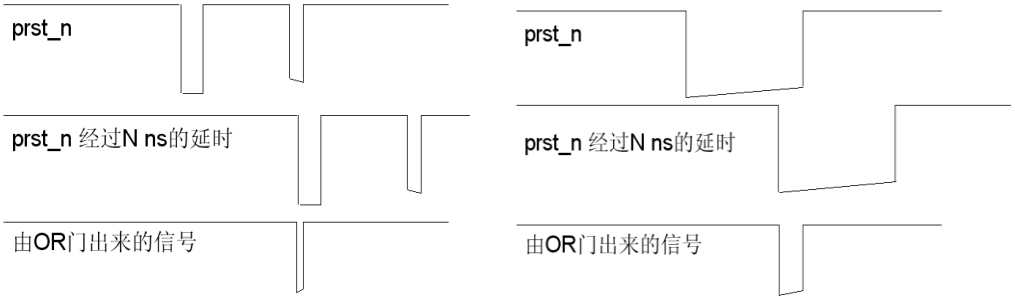
【关于软复位】

来自于 CPU 的软复位信号不需要经过滤毛刺处理，而且一般来说都是具有一定的针对性的（复位某一部分），需要经过另外的逻辑来实现。与硬复位相同的是，软复位信号也需要利用相关时钟域的时钟信号进行同步处理。

【关于滤毛刺电路的设计】

滤毛刺电路是为了减小毛刺对电路造成异常的可能性。对于滤毛刺电路，存在两种不同的实现方案，一种是利用组合逻辑的延时来实现，另外一种是利用寄存器来实现。其具体实现方法可以参考前面的电路逻辑。两种方法各有优劣。

下面我们从性能上分析一下两种方案的异同。在此现给出一个定义：所谓滤毛刺电路的性能是指干扰信号被滤毛刺电路过滤掉的概率，即外部的干扰信号被过滤掉的可能性有多大。本文仅定性的进行简单分析。利用寄存器实现的滤毛刺电路性能较组合逻辑的方案更优。如下图是滤毛刺电路失效的两种情形：



左图中信号线上连续有两个干扰毛刺，间隔为 N ns，干扰信号的第一个脉冲经过延时到达或门的时候正好与干扰信号的第二个脉冲一起到达或门，这样就会有一个毛刺通过该过滤电路。这种情况下，只是要求两个干扰信号之间的时间间隔为 N ns 就会使电路失效。

同样的干扰信号条件下，若采用寄存器滤毛刺的方案，如果要导致滤毛刺失效，除了要求毛刺之间的时间间隔满足一个时钟周期之外，还要求干扰信号同时与系统时钟满足 setup/hold 关系。

右图中当毛刺信号（或者是正常的复位信号）比较小，在这样的一个范围内：大于 N ns 小于 2N ns，则会产生一个小于 N ns 的毛刺信号。

同样的干扰信号条件下，如果是寄存器滤毛刺，生成的毛刺将会满足一个时钟周期（此时应该不是完全意义的毛刺了）由此可见，寄存器滤毛刺电路失效的可能性较组合逻辑的要低的多。

不过可以稍微修改一下滤毛刺电路的设计是可以增强其性能的，方法是多用几个延迟单元。

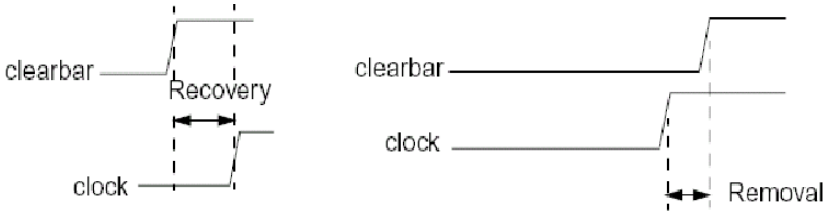
【概念理解】

下面是从 Synopsys 的《PrimeTime Modeling User Guide》摘取出来的有关 Recovery/Removal 的定义部分。

RECOVERY： The RECOVERY timing check specifies limit values for recovery timing checks. A recovery timing check measures the time between the release of an asynchronous control signal from the active state to the next active clock edge—for example, the time between the clearbar and the clock of a flip-flop （shown in Figure B-3）. If the active edge of the clock occurs too soon after the release of the clearbar, the state of the flip-flop becomes uncertain. The state can be the value set by the clearbar or the value clocked into the flip-flop from the data input.

REMOVAL：The REMOVAL timing check specifies limit values for removal timing checks. A removal timing check is a limit of the time between an active clock edge and the release of an asynchronous control signal from the active state, such as between the clock and the clearbar for a flip-flop. If the release of the clearbar occurs too soon after the active edge of the clock, the state of the flip-flop becomes uncertain. The uncertainty can be caused by the value set by the clearbar or the value clocked into the flip-flop from the data input.

As with all port timing checks, the COND construct can be used to specify conditions associated with the recovery timing check.



2.5.4 CRC 算法电路设计

1. 应用场景

循环冗余编码（Cyclic Redundancy Codes，简称 CRC）是一种常用的冗余编码。是在二进制通信系统中常用的差错检测方法，它是通过在原始数据后加冗余校验码来检测差错。冗余位越多，检测出传输错误的机率越大。

2. 基本原理

CRC 校验的基本原理是：CRC 校验码可由一个常数去除该数据流的二进制数值而得，商数被放弃，余数作为 CRC 校验码追加到数据流尾，产生新的数据流进行发送。在接收端，新的数据流被同一常数去除，检查余数是否为一个常数（通常为 0〕。如果余数为一个常数，就认为传输正确，否则就认为传输中已发生差错，该数据流重发。

(1) 多项式除法解释

- 将待传输帧（定长比特串，既有信息码元 k 位）看作一个多项式：$b_{k-1}b_{k-2}b_{k-3}\cdots\cdots b_3b_2b_1b_0$ ，解释成多项式 $P(x)$ ：$P(x)=b_{k-1}x^{k-1}+b_{k-2}x^{k-2}+b_{k-3}x^{k-3}+\cdots+b_3x^3+b_2x^2+b_1x^1+b_0x^0$

比如：我们要传输比特串 10010101110，将其解释成：$x^{10}+x^7+x^5+x^3+x^2+x^1$<—参数为 0 项忽略。

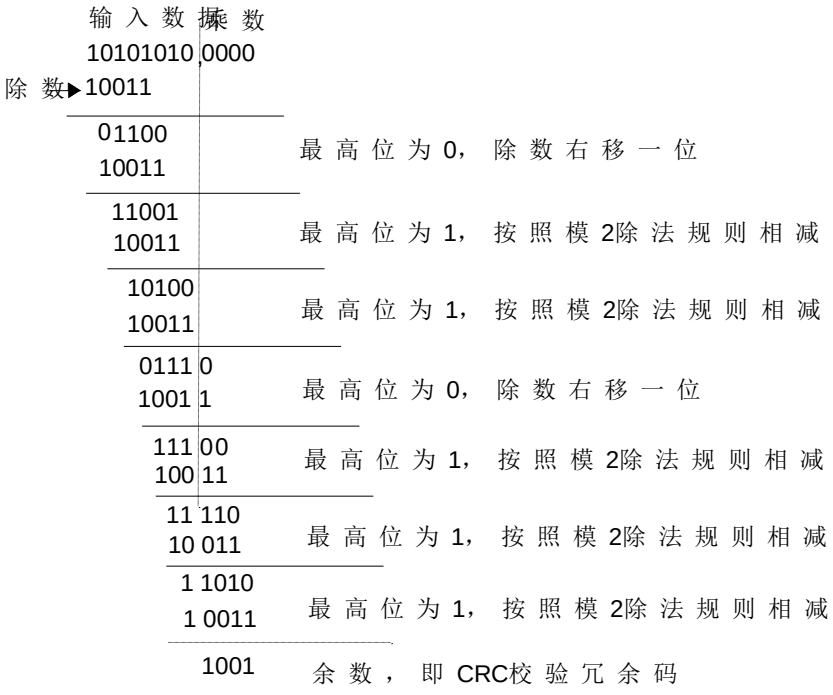
- 事先约定一个生成多项式$G(x)$（Generator Polynomial），G（x）的最高项次数为 r，比如对 CRC-4 而言是 10011，即对应的多项式为$G(x)=x^4+x+1$；r 的值为 4。
- 用$x^{n-k}\cdot P(x)=x^r\cdot P(x)$得到的积来除以生成多项式$G(x)$，最后得到一个$r$位的余式$r(x)$，其最高项的次数为$r-1$。这个余式所代表的比特串就是$k$位信息码的的 CRC 校验码，把校验码接在位信息码的后面，就形成了$n=k+r$位的比特串。对于 CRC-4 而言，其中的$r=4$。$x^4\cdot P(x)$相当于在k位比特串的末尾添加上$r=4$位 0。注意到$x^4\cdot P(x)-r(x)$再除以$G(x)$的话，余数为 0。对于二进制的模 2 运算来说，加法与减法是一样的。这样的话，多项式$T(x)=x^4\cdot P(x)-r(x)=x^4\cdot P(x)+r(x)$对应的比特串就是在信息码元后面接上了 4 位的 CRC 码。
- 上一步中讲的是如何形成校验码。如果接收到一个含有 CRC 校验码的位比特串，它所对应的多项式除以生成多项式的话，余数应该为 0。这样便可以对数据进行校验。

(2) 模 2 除法解释

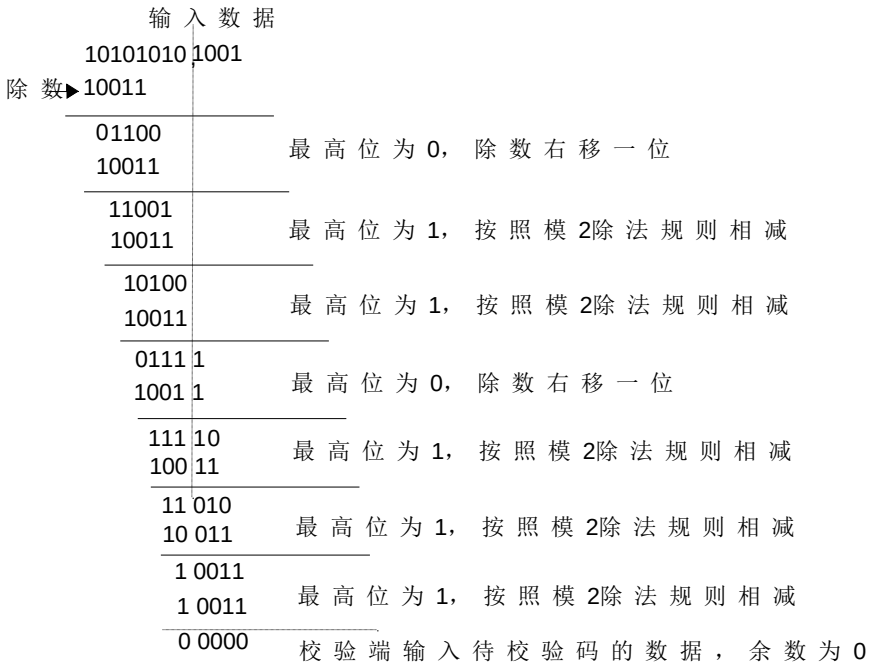
与十进制除法不同的模 2 运算规则如下：

0 + 0 = 0	0 - 0 = 0
1 + 0 = 1	1 - 0 = 1
0 + 1 = 1	0 - 1 = 1
1 + 1 = 0	1 - 1 = 0

具体的操作如下例所示。如果输入的数据 Din 为 10101010，如果生成多项$G(x)=x^4+x+1$，则对应二进制除数为 10011，则模 2 除法的结果如下：



故如果输入 CRC 校验产生器的数据为 10101010，则输出带有 CRC 校验码的比特串为 10101010，1001。在 CRC 校验端，输入数据为 10101010，1001，则校验的结果余数应该为 0，如下所示。



3. 电路说明

CRC 校验码产生器包括串行码流 CRC 校验码产生器和并行码流 CRC 校验码产生器。

(1) 串行 CRC 校验码产生器的硬件实现

串行 CRC 校验码产生器的硬件实现有两种：二进制长除法串行 CRC 校验码产生器和二进制相乘相除法串行 CRC 校验码产生器。

A. 长除法串行码流 CRC 校验产生器

由以上所讲模 2 除法原理，串行码流 CRC 校验的值可以通过线性移位寄存器和异或门求得，线性移位寄存器一次移进一位，完成输入数据右移功能，异或门完成不带进位的减法功能。如果商数为'1'，则从被除数的高阶位减去除数，同时移位寄存器右移一位 ，准备对被除数的较低位进行运算。如果商数为'0'，则移位寄存器直接右移一位。

以 CRC4 校验产生器为例：

其生成多项式：$G(x)=x^4+x+1$，对应的 b4b3b2b1b0 为：10011

建立步骤如下：

- 建立 4 个（与多项式最高次次数一致）寄存器，自左而右，对应 b3 到 b0，（因为生成多项式的最高次数位系数肯定是 1 ）；
- 异或电路位于对应的 bi 值为 1 的寄存器的右边；
- 比特串从最右边进入寄存器电路，一次一位；
- 每下一个比特位进入时，寄存器中的比特位左移一位，经过异或电路的，异或后进入下一位；
- 最左边的寄存器发送比特位到各异或电路，作为第二个操作数；
- 初始时寄存器全为 0，到达的数据串从第一位进入开始到最后一位进入寄存器结束（注意最后一位是 Din（0）后面追加的第四个 0），此时寄存器中的内容即为余数。

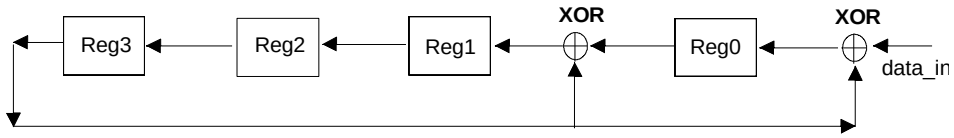


图2-127 长除法的 CRC-4 电路

说明：

利用 4 个余式寄存器来实现除法的话，在开始应该将这四个寄存器清 0。然后，把信息码元作为输入数据，串行的移位输入（每个时钟周期输入一个比特）。经过 4 个时钟周期以后,就可以将信息码元的前 4 比特输入到余式寄存器中。这个过程相当于信息码所代表的多项式$P(x)$乘以多项式$x^4(r=4)$。

在第五个时钟周期，得到下一个输入的信息码元 data_in，这样加上已经保存起来的 4 位比特，构成{CRC_reg[3:0],data_in}这样的 5 位，可以根据模 2 除法规则来进行运算，减去除式 10011。具体的过程如下面的代码描述：

always @（posedge clk） begin //由于生成多项式为x^4+x+1，即 10011

```
If（CRC_reg[3]==1’b1） begin //最高位为 1，减 1 后肯定为 0，丢弃
    CRC_reg[3] <= CRC_reg[2] ^ 1’b0；//第 2 位减 0 后不变，移位保存
    CRC_reg[2] <= CRC_reg[1] ^ 1’b0; //第 1 位减 0 后不变，移位保存
    CRC_reg[1] <= CRC_reg[0] ^ 1’b1; //第 0 位减 1 后变反，移位保存
    CRC_reg[0] <= data_in ^ 1’b1; //减 1 后变反，移位保存
end
else begin //最高位为 0，不进行减法，左移一次，把新数据
    CRC_reg[3] <= CRC_reg[2];  //移入，等待下个周期进行处理
    CRC_reg[2] <= CRC_reg[1];
    CRC_reg[1] <= CRC_reg[0];
    CRC_reg[0] <= data_in;
end
end
```

以上的过程中，会把进行模 2 减法后得到的值（部分余式）放入到寄存器中暂时的存放起来。

下次运行就可以重复的进行了。如果注意到模 2 运算的特点，再结合最高位当前的取值，以上的代码可以简化如下：

```
always @（posedge clk） begin
    CRC_reg[3] <= CRC_reg[2];
    CRC_reg[2] <= CRC_reg[1];
    CRC_reg[1] <= CRC_reg[0] ^ CRC_reg[3];
    CRC_reg[0] <= data_in ^ CRC_reg[3];
end
```

按以上简化代码综合的结果即为图 2-127CRC4 电路。

B．相乘相除法串行码流 CRC 校验产生器

a．多项式相乘电路

假设有两个多项式如下：

$$A(x)=a_n x^n+a_{n-1}x^{n-1}+...+a_1 x+a_0$$

$$B(x)=b_m x^m+b_{m-1}x^{m-1}+...+b_1 x+b_0$$

其中，a_i, b_j的取值在二进制中就是 0 和 1 ；$i=0,1,...,n; j=0,1,...,m$。

那么二者的积为：

$$C(x)=A(x)B(x)=a_n b_m x^{m+n}+(a_n b_{m-1}+a_{n-1}b_m)x^{m+n-1}+...+$$
$$(a_n b_{m-i}+a_{n-1}b_{m-i+1}+...+a_{n-i}b_m)x^{m+n-i}+...+$$
$$(a_1 b_0+a_0 b_1)x+a_0 b_0$$

可以使用下图的电路来实现上述运算：

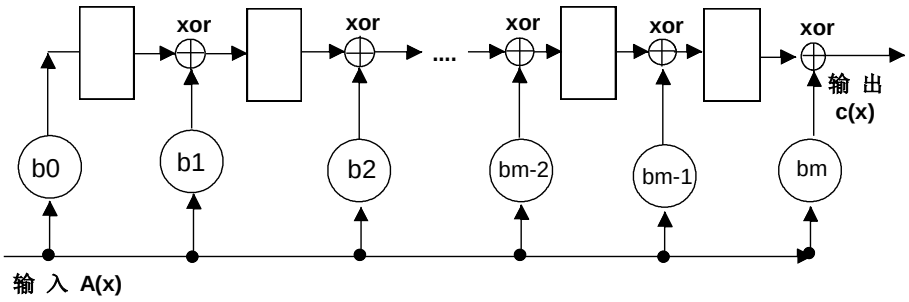


图2-128 多项式乘法电路

开始工作的时候，置移位寄存器的初态全为 0。规定被乘多项式$A(x)$的系数按先高次后低次的顺序送入$B(x)$的乘法电路。第一次移位后，$A(x)$的最高次x^n的系数a_n与b_0相乘后所得之积a_nb_0存入移位寄存器的第 1（最左边）级，与b_1的积a_nb_1存入第 2 级，……，与b_{m-1}的积a_nb_{m-1}存入最后一级，同时输出a_nb_m，它就是$C(x)$的最高次项x^{n+m}的系数。此时移位寄存器组中的存数自左向右是：$(a_nb_0, a_nb_1, ..., a_nb_{m-1})$。第 2 次移位后，$A(x)$的$x^{n-1}$的系数$a_{n-1}$送入电路，此时电路输出$C(x)$的$x^{n+m-1}$的系数$(a_nb_{m-1}+a_{n-1}b_m)$；移位寄存器组中的内容为$(a_{n-1}b_0, a_{n-1}b_1+a_nb_0, ..., a_{n-1}b_{m-1}+a_nb_{m-2})$。重复这种过程$(n+m+1)$次后，完成了乘法，移位寄存器组恢复到全 0 的状态。注意最后输入的 m 个 0 只是把残留在寄存器中的相乘结果推出来而已。

由以上电路的工作过程可以看到，经$(n+m+1)$次移位后，得到：

$$C(x)=A(x)B(x)=a_nb_mx^{m+n}+(a_nb_{m-1}+a_{n-1}b_m)x^{m+n-1}+...+$$
$$(a_nb_{m-i}+a_{n-1}b_{m-i+1}+...+a_{n-i}b_m)x^{m+n-i}+...+$$
$$(a_1b_0+a_0b_1)x+a_0b_0$$

b．多项式相乘相除电路

设有多项式：

$$A(x)=a_kx^k+a_{k-1}x^{k-1}+...+a_1x+a_0$$
$$H(x)=h_rx^r+h_{r-1}x^{r-1}+...+h_1x+h_0$$
$$G(x)=g_rx^r+g_{r-1}x^{r-1}+...+g_1x+g_0$$

若$A(x)$与$H(x)$相乘后再被$G(x)$除，则

$$A(x)H(x)=q(x)G(x)+r(x)$$

其中，多项式的次数（由多项式的最高次数决定）关系为：$\deg r(x)<\deg G(x)$，或者$r(x)=0$

可以利用下图电路来实现相乘相除的运算。显然该电路是前面的除法电路和乘法电路的组合，下半部分是一个乘法电路，上半部分是除法电路，乘法得到的结果马上用于了除法运算。

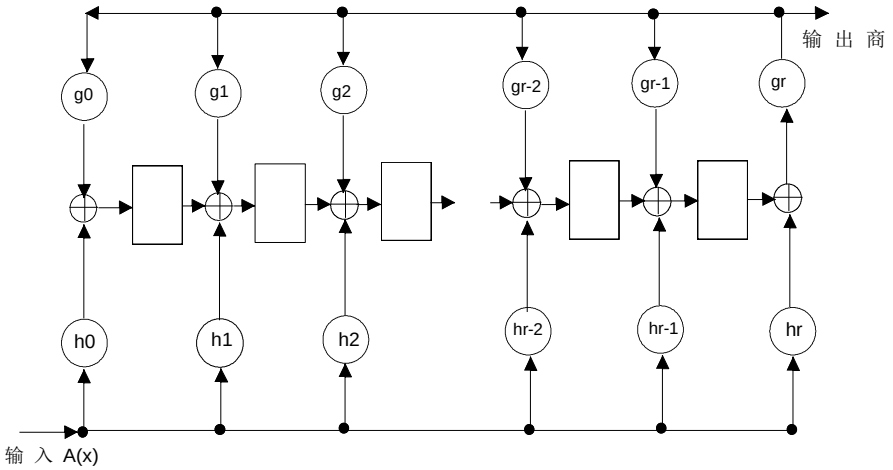


图2-129 多项式相乘相除电路

C．两种实现电路的比较

我们以 CRC-4 的编/解码电路为例，对二进制长除法串行 CRC 校验码产生器和二进制相乘相除法串行 CRC 校验码产生器进行一个简单比较：

如果有一个 4bit 的码流：1010，如果按照长除法实现的 CRC-4 电路,需要 4 个时钟周期才能将 1010 移入到寄存器中，然后再需要 4 个时钟周期才能计算出 CRC 校验码放在寄存器中。如果想并行的输出，还要 1 个时钟周期；如果想串行输出，则还要 4 个时钟周期。总共需要的时钟周期为：4+4+1 或 4+4+4。

而使用相乘相除电路实现的 CRC-4 校验码电路则是：从 4bit 的码流（如 1010）的第一位 1 输入开始，到最后一位 0 输入，需要 4 个时钟周期,这时寄存器中就已经是 CRC 校验码了。如果想把校验码并行移出需要再加 1 个时钟周期，如果想把校验码串行移出需要再加 4 个时钟周期。总共需要的时钟周期是：4+1 或 4+4。

(2) 并行码流 CRC 校验产生器的硬件实现

在设计并行 CRC 校验码产生器的时候，我们可以采用串行 CRC 校验码的思想。与串行 CRC 校验码产生器不同的是，要求并行码流 CRC 校验码产生器的 CRC 校验码在同一拍输出，以 CRC4_D8 校验为例（数据总线为 8 位），要求在一个时钟周期内，移位寄存器一次需要移进 8 位数据。实际上，用移位寄存器不可能在一个时钟周期内不可能算完 8 位，所以这部分电路是用组合逻辑来完成。以下对两种高速并行算法分别讨论：

A．长除法并行码流的 CRC 校验产生器

将串行迭代转变为并行迭代：

r0（n+1）=r9n⊕D_in（n+1）;

r1（n+1）=r9n⊕r0n;

r2（n+1）=r1n;

r3（n+1）=r2n;

在硬件描述语言中一个简单的循环语句即可完成,在速度要求很高时,可采用将迭代式展开成 r[i]对应于输入数据位和上一拍 4 位余数的组合逻辑。

以 CRC4_D8 为例，接口信号接口时序首先描述如下：

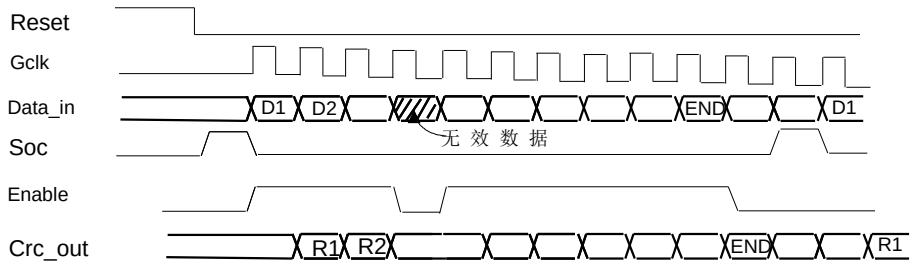


图2-130 CRC4_D8 接口信号接口时序

参考程序如下：

```
module CRC4_GEN1（
                    Reset     ,      //Reset signal
                    Gclk      ,      //Clock signal
                    Soc       ,      //Start of cell
                    Enable,//indicate the data is valid
                    Data_in ,      //input data of cell
                    Crc_out         //output CRC signal
                  ） ;


//----------------------------
// SIGNAL DECLARATIONS
//----------------------------
input             Reset    ;
input             Gclk     ;
input             Soc      ;
input   [7:0]     Data_in ;
output [3:0]      Crc_out ;

//----------------------------
```

```
// SIGNAL DECLARATIONS
//----------------------------
wire              Reset    ;
wire              Gclk     ;
wire              Soc      ;
wire   [ 7 :0]    Data_in ;
reg    [3:0]      Crc_out ;

reg    [3:0]      Crc_tmp ;
reg               Temp     ;

//----------------------------
// PARAMETERS
//----------------------------
parameter   U_DLY=1       ;

//----------------------------
// Crc_out signal
//----------------------------
always @（posedge Reset or posedge Gclk）
begin
        if （Reset）
                Crc_out <= #U_DLY 16'b0 ;
        else if （Soc == 1'b1）
                Crc_out <= #U_DLY 16'b0 ;       //initial value equals to 16'b0;
        else if （Enable = = 1'b1）
                Crc_out <= #U_DLY Crc_tmp ;   //本次 CRC 计算的初始值是上
一拍数据的 CRC 校验码
end

//----------------------------
// Crc_tmp signal
//----------------------------
always @（Crc_out or Data_in）
begin
        Crc_tmp = Crc_out ;
```

```
        for （i=7;i>=0;i=i-1）
        begin
            Temp = Crc_tmp[3]  ;
            Crc_tmp[3]=Crc_tmp[2];
            Crc_tmp[2]=Crc_tmp[1];
            Crc_tmp[1]=Crc_tmp[0]^Temp;
            Crc_tmp[0]=Data_in[i]^Temp;
      end
end
endmodule
```

说明：

注意该程序的 Data_in 信号没有与 2^4 相乘，直接进行除法运算，故每一个数据包的结尾应该加上 4'b0。即在一个数据包结束后还要再输入 4'b0；这样才是真正意义上的 CRC 校验码。所以该程序并不实用。

Crc_tmp[3]是上次计算得到的余数最高位，在长除法运算中也是本次计算被除数的最高位，也是本次运算的商。商为 1，则执行除法运算，即减掉除数 10011 再移位。如果商为 0，则只执行移位操作。

B．相乘相除法并行码流的 CRC 校验产生器

相乘相除法设计思想如下：

{I[4:0]+D0×2^4}/G[4:0]~>{R1[4:0]+D1×2^4}/G[4:0]~>•••~>
{Rt[4:0]+Dt×2^4}/G[4:0]~>•••~>{R7[4:0]+D7×2^4}/G[4:0]
~>R8[3:0]即为 CRC 校验码

该思想用代码描述如下：

```
module CRC4_GEN2（
                    Reset    ,     //Reset signal
                    Gclk     ,     //Clock signal
                    Soc      ,     //Start of cell
                    Enable,    //indicate the data is valid
                    Data_in ,     //input data of cell
                    Crc_out       //output CRC signal
                  ）  ;

//----------------------------
// SIGNAL DECLARATIONS
//----------------------------
```

```
input               Reset     ;
input               Gclk      ;
input               Soc       ;
input    [7:0]      Data_in ;
output [3:0]        Crc_out ;

//----------------------------
// SIGNAL DECLARATIONS
//----------------------------
wire                Reset     ;
wire                Gclk      ;
wire                Soc       ;
wire    [ 7 :0]     Data_in ;
reg       [3:0]     Crc_out ;

reg       [3:0]     Crc_tmp ;
reg                 Temp      ;

//----------------------------
// PARAMETERS
//----------------------------
parameter    U_DLY=1        ;

//----------------------------
// Crc_out signal
//----------------------------
always @（posedge Reset or posedge Gclk）
begin
        if （Reset）
                Crc_out <= #U_DLY 16'b0 ;
        else if （Soc == 1'b1）
                Crc_out <= #U_DLY 16'b0 ;     //initial value equals to 16'b0;
        else if （Enable = = 1'b1）
                Crc_out <= #U_DLY Crc_tmp ; //本次 CRC 计算的初始值是上一拍数据的 CRC 校验码
end
```

```
//----------------------------
// Crc_tmp signal
//----------------------------
always @（Crc_out or Data_in）
begin
        Crc_tmp = Crc_out ;
        for （i=7;i>=0;i=i-1）
        begin
                Temp = Data_in[i] ^ Crc_tmp[3]   ;
                Crc_tmp[3]=Crc_tmp[2];
                Crc_tmp[2]=Crc_tmp[1];
                Crc_tmp[1]=Crc_tmp[0]^Temp;
                Crc_tmp[0]=0^Temp;
      end
end
endmodule
```

说明：

Crc_tmp[3]是上次计算得到的余数最高位，本次 Data_in 乘以 2^4 与 Crc_tmp 不进位相加的结果最高位是 Temp，故 Temp 也是本次计算被除数的最高位，也是本次运算的商。商为 1，则执行除法运算，即减掉除数 10011 再移位。如果商为 0，则只执行移位操作。

【资料来源】

《crc 校验码的电路实现.lwp》，谭　锐

《CRC32 校验模块设计总结》，牛风举

2.5.5 时钟分频电路

1. 应用场景

众所周知，分频器是 FPGA 设计中使用频率非常高的基本设计之一，尽管在目前大部分设计中，广泛使用芯片厂家集成的锁相环资源，如 altera 的 PLL，Xilinx 的 DLL.来进行时钟的分频，倍频以及相移。但是对于时钟要求不高的基本设计，通过语言进行时钟的分频相移仍然非常流行，首先这种方法可以节省芯片内部的锁相环资源，再者，消耗不多的逻辑单元就可以达到对时钟操作的目的。另一方面，通过语言设计进行时钟分频，可以看出设计者对设计语言的理解程度。

因此很多招聘单位在招聘时往往要求应聘者写一个分频器（比如奇数分频）以考核应聘人员的设计水平和理解程度。

【电路说明】

对于各种分频系数进行分频，目前主要存在三种不同的处理方法：

(1) 偶数倍分频

偶数倍分频应该是大家都比较熟悉的分频，通过计数器计数是完全可以实现的。如进行 N 倍偶数分频，那么可以通过由待分频的时钟触发计数器计数，当计数器从 0 计数到 N/2-1 时，输出时钟进行翻转，并给计数器一个复位信号，使得下一个时钟从零开始计数。以此循环下去。这种方法可以实现任意的偶数分频。如图 2-131 所示即为一个二分频/四分频/六分频电路波形图。

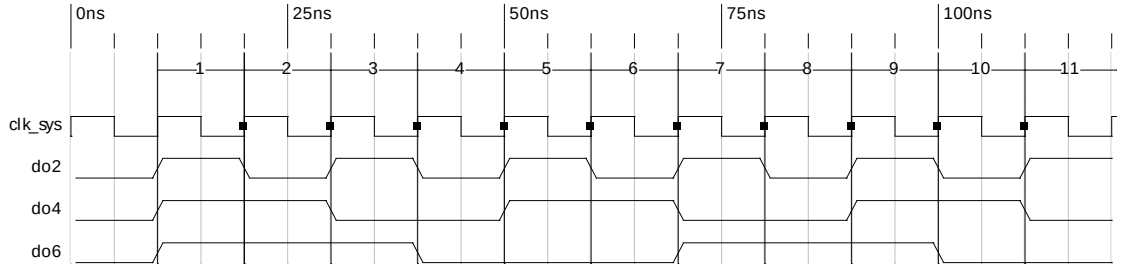


图2-131 偶数分频电路波形图

根据上图所示的波形，可以做出偶数分频电路结构图。本文以 4 分频电路为例，电路结构图如图 2-132 所示。

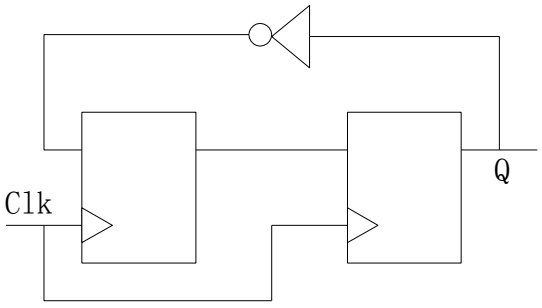


图2-132 四分频电路结构图

(2) 奇数倍分频

奇数倍分频有两种实现方法：首先，完全可以通过计数器来实现，如进行三分频，通过待分频时钟上升沿触发计数器进行模三计数，当计数器计数到邻近值进行两次翻转，比如可以在计数器计数到 1 时，输出时钟进行翻转，计数到 2 时再次进行翻转。即是在计数值在邻近的 1 和 2 进行了两次翻转。这样实现的三分频占空比为 1/3 或者 2/3。如果要实现占空比为 50%的三分频时钟，可以通过待分频时钟下降沿触发计数，和上升沿同样的方法计数进行三分频，然后下降沿产生的三分频时钟和上升沿产生的时钟进行相或运算，即可得到占空比为 50%的三分频时钟。这种方法可以实现任意的奇数分频。归类为一般的方法为：对于实现占空比为 50%的 N 倍奇数分频，首先进行上升沿触发进行模 N 计数，

计数选定到某一个值进行输出时钟翻转，然后经过（N-1）/2 再次进行翻转得到一个占空比非 50%奇数 n 分频时钟。再者同时进行下降沿触发的模 N 计数，到和上升沿触发输出时钟翻转选定值相同值时，进行输出时钟时钟翻转，同样经过（N-1）/2 时，输出时钟再次翻转生成占空比非 50%的奇数 n 分频时钟。两个占空比非 50%的 n 分频时钟相或运算，得到占空比为 50%的奇数 n 分频时钟。另外一种方法：对进行奇数倍 n 分频时钟，首先进行 n/2 分频（带小数，即等于(n-1)/2+0.5），然后再进行二分频得到。得到占空比为 50%的奇数倍分频。下面讲讲进行小数分频的设计方法

图 2-133 所示为比较典型的占空比为 50%的五分频电路波形图。其他奇数分频电路波形图与此类似。

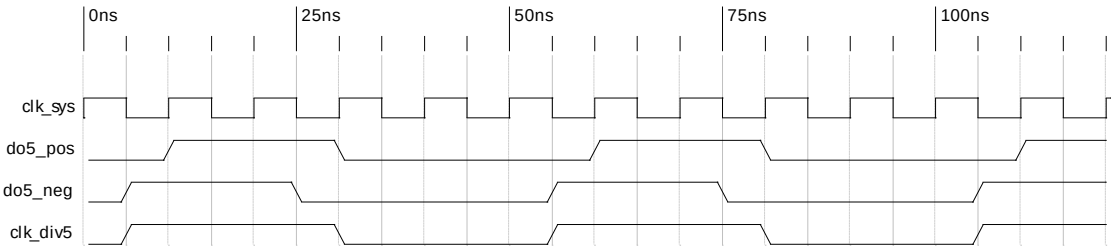


图2-133 占空比 50%的五分频电路波形图

根据上图所示的波形，可以做出占空比 50%的奇数分频电路结构图。本文仍旧以五分频电路为例，电路结构图如图 2-132 所示。

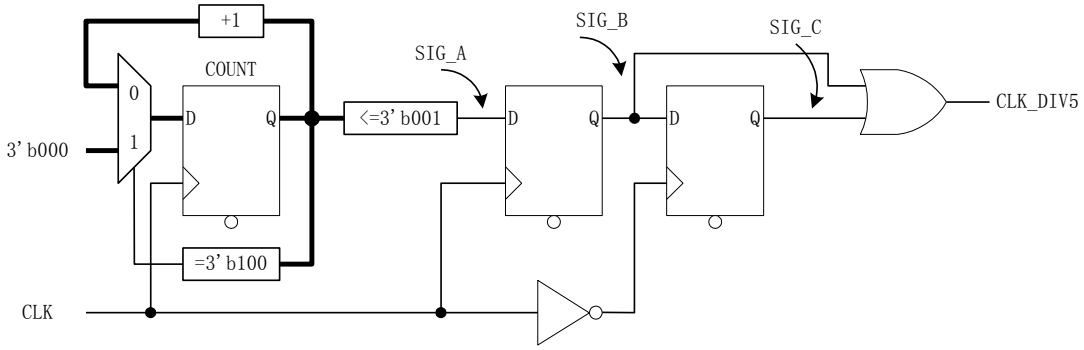


图2-134 占空比 50%的五分频电路结构图

(3) 小数分频

首先讲讲如何进行 n+0.5 分频，这种分频需要对输入时钟进行操作。

基本的设计思想：对于 n+0.5 分频，首先进行模 n 的计数，在计数到 n-1 时，输出时钟赋为‘1’，回到计数 0 时，又赋为 0，因此，可以知道，当计数值为 n-1 时，输出时钟才为 1，因此，只要保持计数值 n-1 为半个输入时钟周期，即实现了 n+0.5 分频时钟，因此保持 n-1 为半个时钟周期即是一个难点。从中可以发现，因为计数器是通过时钟上升沿计数，因此可以在计数为 n-1 时对计数触发时钟进行翻转，那么时钟的下降沿变成了上升沿。即在计数值为 n-1 期间的时钟下降沿变成了上升沿，则计数值 n-1 只保持了半个时钟周期，由于时钟翻转下降沿变成上升沿，因此计数值变为 0。因此，每产生一个 n+0.5 分频时

钟的周期，触发时钟都是要翻转一次。如图 2-135 所示为通用的小数分频电路结构图。

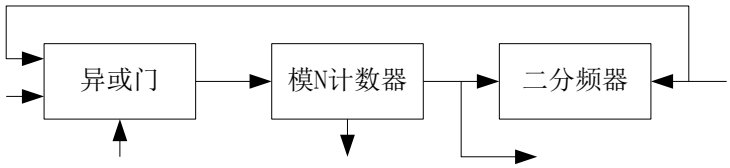


图2-135 小数分频电路结构图

📖 说明

小数分频较前两种分频而言，处理方式较为特殊，本文只对其中的一种处理方式作简单描述，更详细的说明请查阅相关资料。

2.5.6 异步电路

常用的异步电路有时钟切换电路，异步时钟域的数据传输电路等。

1. 异步时钟切换电路

当我们进行数字电路设计的时候，不可避免的会遇到不同时钟的切换。有时是一个时钟和它的衍生时钟之间的切换；有时是异步时钟之间的切换。如果在设计时考虑不周全，就很容易在切换时钟产生毛刺或逻辑上有亚稳态现象。那么，我们想在不同时钟间的切换且不产生毛刺，应该如何设计呢？下面的电路就是一个异步时钟的切换电路，我们详细说明一下，电路描述：

下面是两个相位频率完全没有关系的时钟 C1 和 C2，我们希望进行切换。如果设计要求不能出现毛刺和 spike，我们在开启另一个时钟时先关断一个时钟。这种关断一个并开启另一个的操作可能会引发输出一个“死”周期。例如，在 C1 低电平时被关断，同时 C2 开启。

这个电路中，电路起始时所有的触发器都需要清 0，如果是异步触发器，异步清 0 很容易在上电时实现。当控制信号 ctrl ＝0，在电路上电后，只要进行异步清 0，C˙＝0，当异步清 0 结束后，q11 在 C1 的第一个下降沿跳变成 1，下一个时钟周期 q12 置 1，这样 C˙＝C1；这个状态持续一直到 ctrl 的值改变，当 ctrl＝1 且没有建立/保持时间 violation 时，q11 变低。再过一个时钟周期，q12＝0，这样就关断了 C˙，如果此时 q11 存在亚稳态，就需要更长时间来关断 C˙。C˙被关断后才进行切换。最差的情况就是存在亚稳态需要更长的时间，但是整个切换操作不会引入毛刺或 spikes。

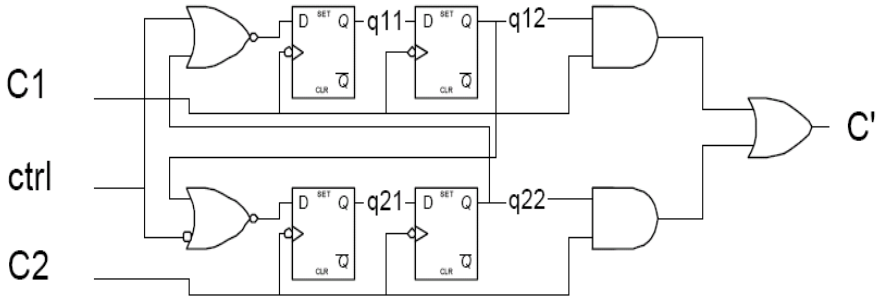


图2-136 无毛刺异步时钟切换电路

2. 异步时钟域数据传输

现行设计的芯片都是存在多个时钟域，不同的时钟域间不可避免的存在数据传输的操作，其中最为常见的就是 CPU 时钟域读写系统时钟域的数据。那么怎么设计才能更为合理呢？

异步时钟域数据传输的设计问题：我们知道同步模块的时序都可以通过内部的时序分析工具来进行检查。但是如果遇到异步时钟域，就会在节点处发生不能检查的情况，这时信号从一个时钟域输出到另一个时钟域采样会给制造带来潜在的问题，因为它的时序关系是不确定的。很可能产生亚稳态的状态。亚稳态是必须避免的，亚稳态现象可以引起功耗损失，降低系统的可靠性，并可能引起芯片的失败。

通常解决做法：通常的方法是使用一系列的 D 触发器，通过打拍的方法减少亚稳态发生的可能性。一般使用目标时钟域的时钟 3 次打拍处理。这种方法是可行的但不是最佳的，因为使用 3 级的打拍处理，需要很多的逻辑资源并且引入了很大的时延。对于多 bit 的异步时钟域数据传输还有以下办法。

(1) 异步时钟域数据传输

下面介绍一种方法完成数据从低速时钟域到高速时钟域传输功能：

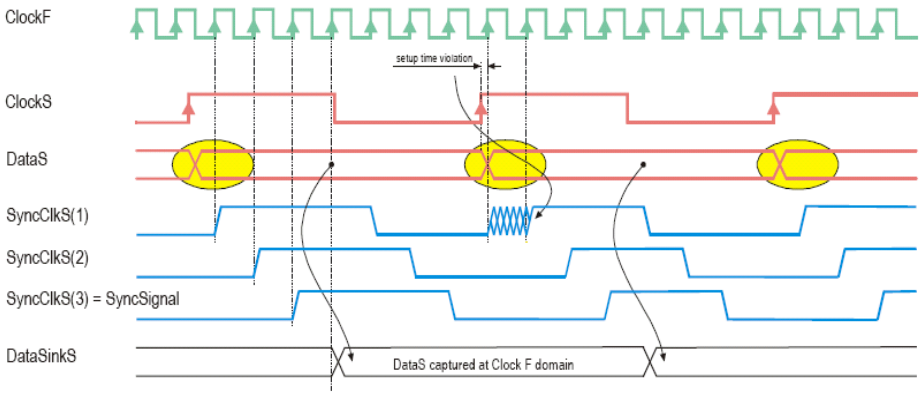


Figure 5. Data Transfer from Slow Clock to Fast Clock Domain.

图2-137 低速到高速数据传输时序图

数据可以在 ClockS 延时为高时进行采样，在 ClockS 为高的周期内，数据没有跳变。

时间窗：在同步信号有效的时间窗内进行异步时钟域的采样是安全的。DataS 在时间窗内是稳定的。同步信号为高的周期内（称为时间窗），在另一个时钟域对数据进行采样很明显是安全的。DataS 确保不会在时间窗内进行跳变。如下图同步信号由标准的打 3 拍的寄存器结构产生的。

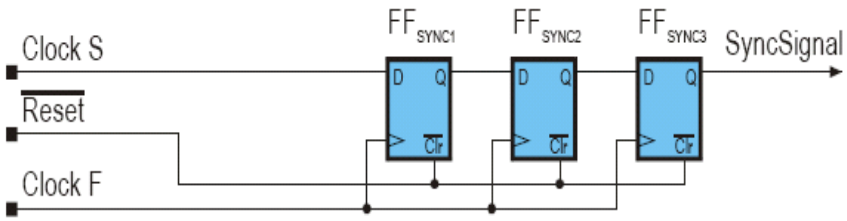


Figure 6. Generation of SyncSignal.

图2-138 同步信号产生电路

从 DataS 到 DataSinkS 传输中的门是打开的，也就是在数据传输的过程中，同步信号 syncsignal ＝1 。两种方法完成这个功能，一种是通过一个简单的 mux 电路来完成这个功能；另一种是通过使能目标寄存器的办法。

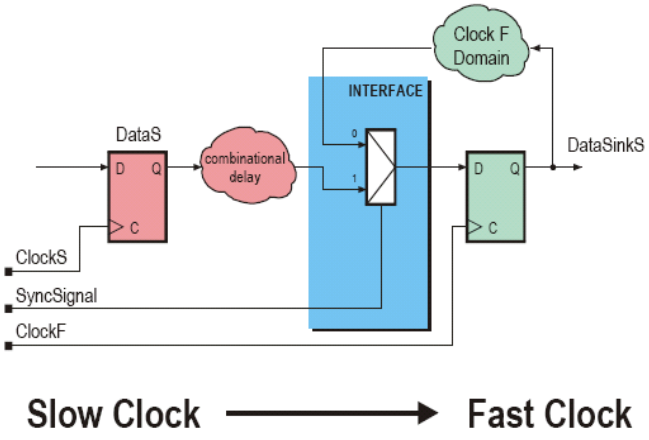


图2-139 切换电路

(2) 异步 FIFO 的方法

异步时钟域之间的数据传输最常用的办法就是异步 FIFO，如映射的过程实现中就经常使用 FIFO 来完成，这需要两个时钟域的带宽相差很小的情况下。

所谓异步是指读、写时钟是完全独立并且不一致的，或者不同频率，或者同频但不同相。读地址和空标志是由读时钟产生的，而写地址和满标志则由写时钟产生，当要产生 FIFO 的空、满标志时，必须进行读写地址的比较时，问题就来临了。如果直接采样地址比较的话，地址线一般有多位，写地址的每一位在写时钟作用下，跳变得不一致，即产生毛刺，要过一小段时间才能稳定。在未稳定期内，刚好读时钟进行采样写地址，这时就出现误判断，逻辑错误。同时采样读写地址相差 N 个来产生空满信号，时间上会多一些，因为涉及加和减操作。

为避免地址跳变的不一致造成读写地址误判断，采样格雷码。该码的优点是相邻两值只有一位跳变，其他不变，这样地址变化的时间较短，极大提高比较精度。

异步 FIFO 实现方式是采用双口 RAM 完成，FIFO 的空信号是在读时钟域产生；FIFO 的满信号是在写时钟域产生的。下面以空信号产生为例说明：

A．产生读时钟域的 Read_addrgray 信号

```
always @（posedge Read_clock or posedge Fifo_rst）
  if （Fifo_rst）
       read_addr <= 'b0;
  else if （read_allow）
       read_addr <= read_addr + 1;
always @（posedge Read_clock or posedge Fifo_rst）
  if （Fifo_rst）
       Read_nextgray <= 9'b100000000;
  else if （read_allow）
       Read_nextgray <= { read_addr[8], （read_addr[8] ^ read_addr[7]），
            （read_addr[7] ^ read_addr[6]），（read_addr[6] ^ read_addr[5]），
            （read_addr[5] ^ read_addr[4]），（read_addr[4] ^ read_addr[3]），
            （read_addr[3] ^ read_addr[2]），（read_addr[2] ^ read_addr[1]），
            （read_addr[1] ^ read_addr[0]） };
always @（posedge Read_clock or posedge Fifo_rst）
  if （Fifo_rst）
       Read_addrgray <= 9'b100000001;
  else if （read_allow）
            Read_addrgray <= Read_nextgray;
```

B．产生写时钟域的 Write_addrgray 信号

```
always @（posedge Write_clock or posedge Fifo_rst）
  if （Fifo_rst）
       write_addr <= 'b0;
  else if （write_allow）
       write_addr <= write_addr + 1;
always @（posedge Write_clock or posedge Fifo_rst）
  if （Fifo_rst）
       Write_nextgray <= 9'b100000000;
```

```
  else if （write_allow）
        Write_nextgray <= { write_addr[8], （write_addr[8] ^ write_addr[7]），
              （write_addr[7] ^ write_addr[6]），（write_addr[6] ^ write_addr[5]），
              （write_addr[5] ^ write_addr[4]），（write_addr[4] ^ write_addr[3]），
              （write_addr[3] ^ write_addr[2]），（write_addr[2] ^ write_addr[1]），
              （write_addr[1] ^ write_addr[0]） };
always @（posedge Write_clock or posedge Fifo_rst）
  if （Fifo_rst）
        Write_addrgray <= 9'b100000001;
  else if （write_allow）
        Write_addrgray <= Write_nextgray;
```

C．产生 Emptyg 信号

```
always @（Write_addrgray or Read_addrgray）
  if（ Write_addrgray == Read_addrgray ）
        Emptyg = 'b1;
  else
              Emptyg = 'b0;
```

D．产生将空信号 Almostemptyg

```
always @（Write_addrgray or Read_nextgray）
  if（ Write_addrgray == Read_nextgray ）
        Almostemptyg = 'b1;
  else
        Almostemptyg = 'b0;
```

E．产生空信号 empty

```
always @（posedge Read_clock or posedge Fifo_rst）
  if （Fifo_rst）
        Empty <= 1'b1;
  else
        Empty <= （Emptyg || （Almostemptyg && Read_enable && ! Empty））;
```

同样满信号 full 类似的处理，在写时钟域进行；

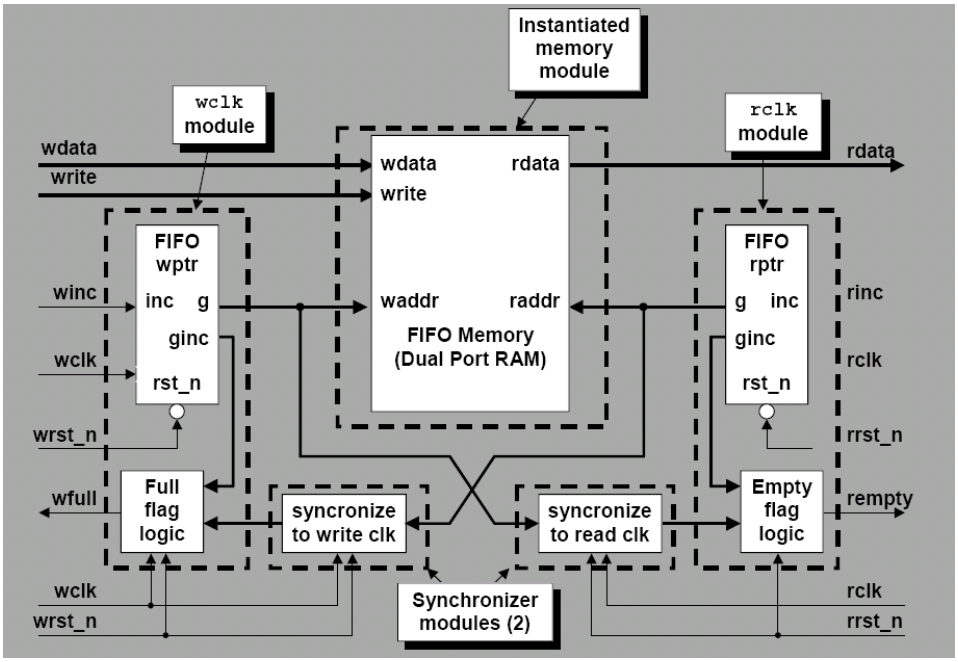


图2-140 异步 FIFO 框图

2.5.7 状态机

1. 应用场景

在数字电路设计中，经常会用到状态机实现复杂逻辑电路的设计，从而使得设计结构清晰，可读性强。

(1) 状态机的概念

状态机（Finite State Machine, FSM）的本质就是对具有逻辑顺序或时序规律事件的一种描述方法。具有逻辑顺序和时序规律的事情都适合用状态机描述。

第一种设计思路，从状态变量入手。如果一个电路具有时序规律或者逻辑顺序，我们就可以自然而然地规划出状态，从这些状态入手，分析每个状态的输入，状态转移和输出，从而完成电路功能。

第二种设计思路是：首先明确电路的输出的关系，这些输出相当于状态的输出，回溯规划每个状态和状态转移条件与状态输入。

(2) 状态机的基本要素

- 状态：也叫状态变量。在逻辑设计中，使用状态划分逻辑顺序和时序规律。
- 输出：指在某一个状态时特定发生的事件。
- 输入：指状态机中进入每个状态的条件，有的状态机没有输入条件，其中的状态转移较为简单，有的状态机有输入条件，当某个输入条件存在时才能转移到相应的状态。

(3) 状态机的分类

- 摩尔（Moore）状态机：摩尔状态机的输出仅仅依赖于当前状态，而与输入条件无关。
- 米勒（Mealy）状态机：米勒状态机的输出不仅依赖于当前状态，而且取决于该状态的输入条件。

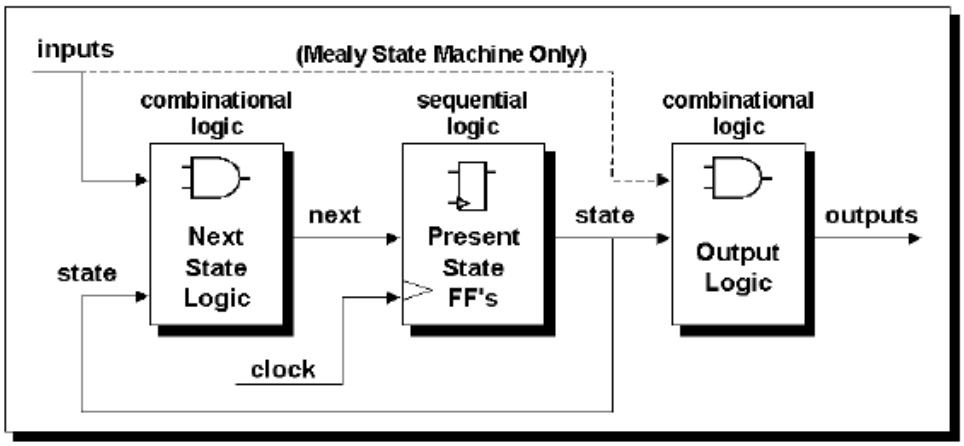


图2-141 状态机的典型结构

(4) 状态机的性能参数

- FSM 要安全，稳定性高。
- FSM 速度快，满足设计的频率要求。
- FSM 面积小，满足设计的面积要求。
- FSM 设计要清晰易懂、易维护。

所谓 FSM 安全是指 FSM 不会进入死循环，特别是不会进入非预知的状态，而且由于某些扰动进入非设计状态，也能很快的恢复到正常的状态循环中来。这里面有两层含义，第一：要求该 FSM 的综合实现结果无毛刺等异常扰动；第二：要求状态机要完备，即使收到异常扰动进入非设计状态，也能很快恢复到正常状态。

不规范的 FSM 写法很难让其他人解读，甚至过一段时间后设计者也发现很难维护。

2. 电路说明

(1) 状态机的描述

状态机一般采用类似于下图的方式描述，结构清晰，易于理解。

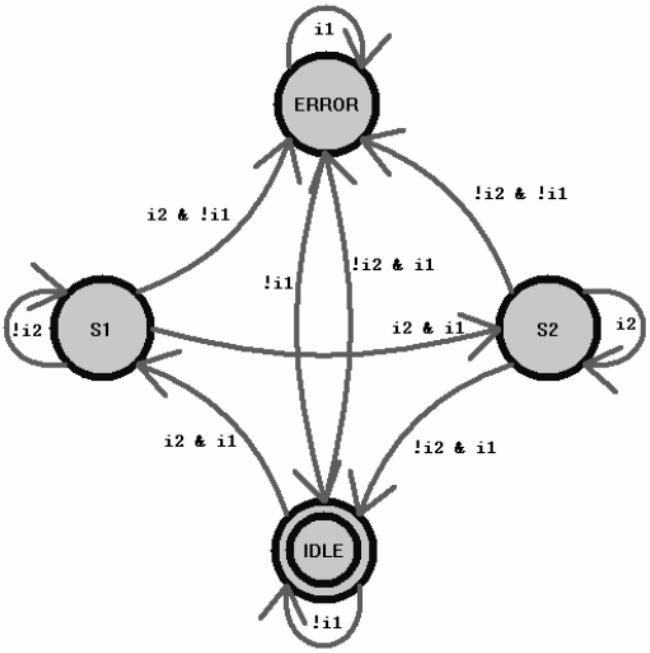


图2-142 状态机的描述图

(2) 状态机设计方式

设计一个状态机，业界存在多种方式，大致可以总结为 3 种：一段式，两段式和三段式。在使用 Verilog 描述时，这三种类型的设计方式分别采用一个、两个和三个 Always 块实现。

一段式写法就是将状态的同步转移，状态输出和状态的输入条件都写在一个 always 块中，如图 2-143 所示，一般不推荐使用。

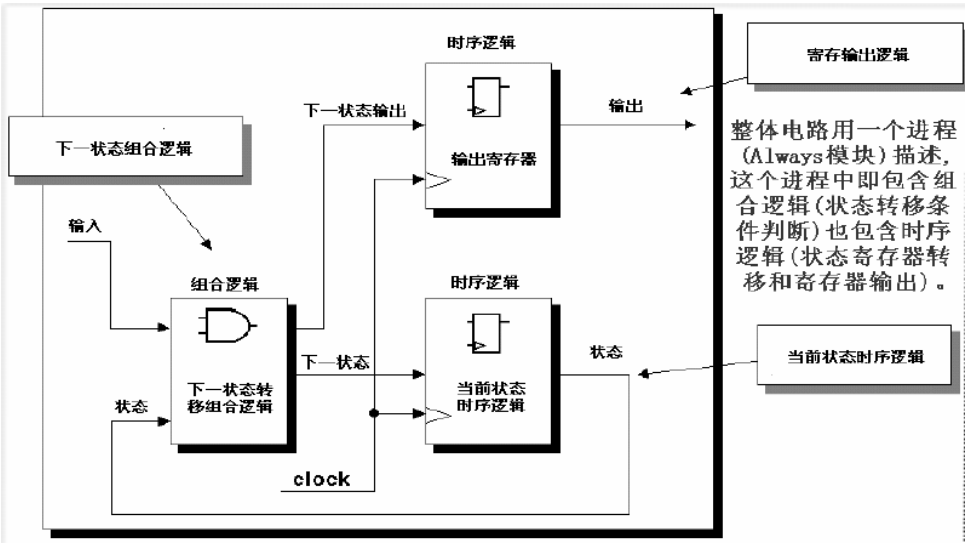


图2-143 一段式状态机

两段式状态机写法是推荐的 FSM 描述方法之一，在此我们仔细讨论一下代码结构。两段式 FSM 的核心就是：一个 always 块采用同步时序描述状态转移；另

一个 always 块采用组合逻辑判断状态转移条件，描述状态转移规律，如图 2-144 所示。

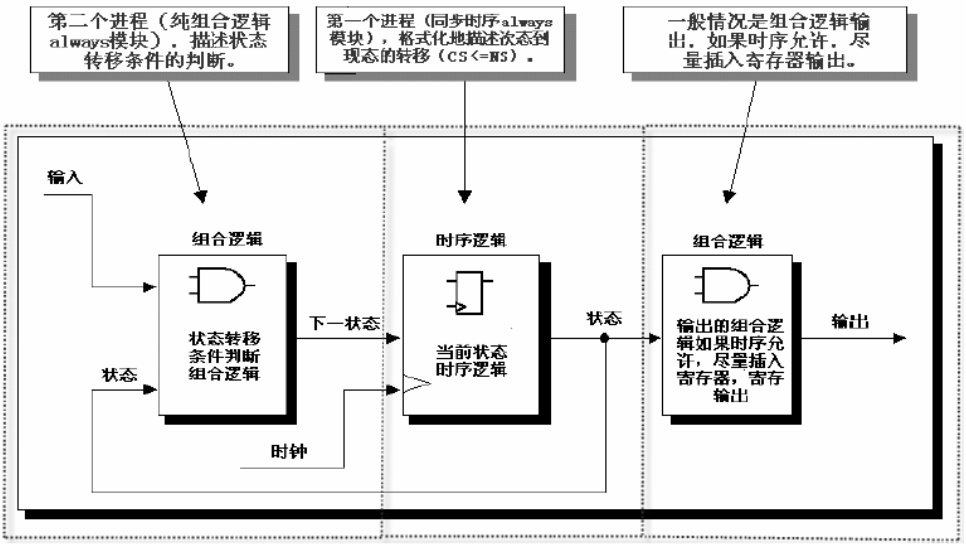


图2-144 两段式状态机

时序 always 块：

- 程式化的描述结构，无论具体到何种 FSM 设计，都可以定义两个状态寄存器“CS”和“NS”，分别代表当前状态和下一状态，然后根据所需的复位方式（同步复位或异步复位），在时钟沿到达时将 NS 赋给 CS。
- 需要注意的是这个同步时序模块的赋值要采用非阻塞赋值“<=”。

组合逻辑 always 块：

- 格式化的书写结构。其中 always 的敏感列表为当前状态“CS”，复位信号和输入条件（如果是米勒状态机，则必须有输入条件；如果是摩尔状态机，一般敏感表和后续逻辑判定没有输入），注意电平敏感表必须列完整。
- 一般来说，在这个组合 always 敏感表下先写一个默认的下一状态“NS”的描述，然后根据实际的状态转移条件由内部的 case 或者 if...else 条件判断确定正确的转移。
- 推荐在敏感表下的默认状态为不定状态 X，这样描述的好处有两个：第一在仿真时可以很好的考察所设计的 FSM 的完备性，如果所设计的 FSM 不完备，则会进入任意状态，仿真很容易发现；第二个好处是综合器对不定态 X 的处理是“Don’t Care”，即任何没有定义的状态寄存器向量都会被忽略。
- 在每个 case 模块的内部的结构也非常相似，都是先描述当前状态的组合逻辑输出，然后根据输入条件（米勒 FSM）判定下一个状态。
- 该组合逻辑模块中所有的赋值推荐采用阻塞赋值“=”。
- 对于每个输出，一般用组合逻辑描述，比较简便的方法是用 task/endtask 将输出封装起来，这样做的好处不仅仅是写法简单，而且利于复用共同的输出。

三段式状态机写法是推荐的 FSM 描述方法之一。三段式状态机与两段式状态机的区别在于，在三段式状态机中，判断状态转移的 always 模块的 case 语句判断的条件是当前状态“CS”而在同步时序 FSM 输出的 always 模块的 case 语句判断的条件是下一状态“NS”。如图 2-145 所示。

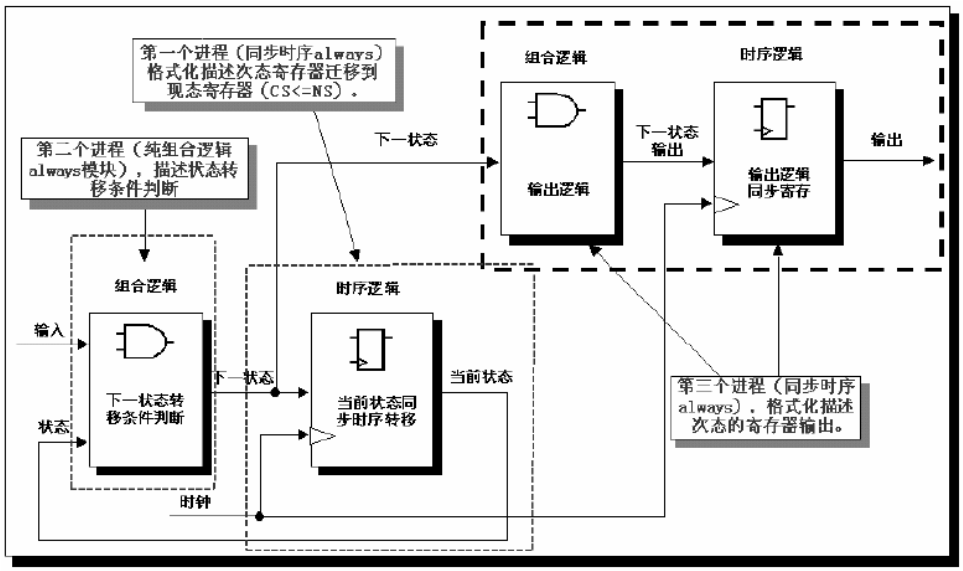


图2-145 三段式状态机

对比一下上节两段式 FSM 的描述，可以清晰发现三段式与两段式 FSM 描述的最大区别在于两段式采用了组合逻辑输出，而三段式巧妙地根据下一状态的判断，用同步时序逻辑寄存 FSM 的输出。

一段式写法也是用寄存器同步了 FSM 的输出，为什么前面介绍一段式的输出代码容易混淆，不利于维护呢？使用一段式建模 FSM 的寄存器输出的时候，必须要综合考虑现态在何种状态转移条件下会进入哪些次态，然后在每个现态的 case 分支下分别描述每个次态的输出，这显然不符合思维习惯。而三段式建模描述 FSM 的状态机输出时，只需指定 case 敏感表为次态寄存器，然后直接在每个次态的 case 分支中描述该状态的输出即可，根本不用考虑状态转移的条件。

三段式描述方法与两段式描述相比，虽然代码结构复杂了一些，但是换来的优势是使 FSM 做到了同步寄存器输出，消除了组合逻辑输出的不稳定与毛刺的隐患，而且更利于时序路径分组，一般来说在 FPGA/CPLD 等可编程逻辑器件上的综合与布局布线效果更佳。

但是电路设计不是一成不变的，在某些情况下，两段式结构比三段式结构更有优势。分析一下两者的结构，可以发现，两段式用状态寄存器分割了两部分组合逻辑（状态转移条件组合逻辑和输出组合逻辑）。而三段式结构中，从输入到寄存器状态输出的路径上，要经过两部分组合逻辑（状态转移条件组合逻辑和输出组合逻辑），从时序上，这两部分组合逻辑完全可以看为一体。这样这条路径的组合逻辑就比较繁杂，该路径的时序相对紧张。也就是说，两段式建模中用状态寄存器分割了组合逻辑，而三段式将寄存器移到组合逻辑的最后端。如果寄存器前的组合逻辑过于复杂，势必会成为整个设计的关键路径，此时就

不宜再使用三段式建模，而要使用两段式建模。解决两段式建模组合逻辑输出产生毛刺的方法是：在 FSM 后级电路额外插入寄存器，从而调整时序，完成功能。

三种设计方式的对比可以参考表 2-30。

表2-30 三种方式的对比

比较项目	一段式方式	两段式方式	三段式方式
推荐等级	不推荐	推荐	推荐
代码简洁程度	冗长	最简洁	简介
always 块个数	1	2	3
是否利于时序约束	不利于	利于	利于
是否组合逻辑输出	否	多数情况是	否
是否利于综合及布局布线	不利于	利于	利于
代码可靠性与可维护性	低	高	最好
代码风格的规范性	低	规范	规范

(3) 状态机状态编码

实现状态机的状态时，可以采用 3 种编码，分别为 Binary（二进制编码）、gray-code（格雷码）和 one-hot（独热码），三种编码各有特点。

Binary（二进制编码）和 gray-code（格雷码）编码使用最少的触发器，较多的组合逻辑，而 one-hot（独热码）编码反之。one-hot 编码的最大优势在于状态比较时仅仅需要比较一个 bit，一定程度上从而简化了比较逻辑，减少了毛刺产生的概率。一般来说，对于小型设计使用 graycode 和 binary 编码更有效，而大型状态机使用 onehot 更高效。

(4) 状态机状态参数化

状态机的定义可以用 parameter 定义，不推荐使用`define 宏定义的方式，因为`define 宏定义在编译时自动替换整个设计中所定义的宏。而 parameter 仅仅定义模块内部的参数，定义的参数不会与模块外的其他状态机混淆。例如一个工程里面有两个 module 各包含一个 FSM，如果设计时都有 IDLE 这一名称的状态，如果使用`define 宏定义就会混淆起来，如果使用 parameter 则不会造成任何不良影响。

(5) 状态机健壮性

一个完备的状态机（健壮性强）应该具备初始化状态和默认状态。

当芯片加电或者复位后，状态机应该能够自动将所有判断条件复位，并进入初始化状态。

当转移条件不满足，或者状态发生了突变时，默认（default）状态能保证逻辑不会陷入“死循环”，即需要具备“自恢复”功能。对应于编码就是对 case 和 if… else 语句要特别注意，尽量使用完备的条件判断语句。Verilog 中，使用 case 语句的时候要用 default 建立默认状态。

3. 资料来源

Lattice 公司《How to design FSM》，Westor Wang ATM。

《State machine design techniques for Verilog and VHDL》，Steve Golson。

《The Fundamentals of Efficient Synthesizable Finite State Machine》，Clifford E. Cummings。

2.5.8 WRR 调度

1. 应用场景

分组调度实现对链路带宽的管理，是按照一定的规则来决定从等待队列中选择哪个分组进行发送，使得所有输入业务能够按照预定的方式共享输出链路带宽。它影响的主要性能参数包括带宽分配、时延和时延抖动等，是实现网络服务质量控制的核心技术之一。

WRR 算法是一种加权轮循分组调度算法，WRR 算法核心思想是通过周期性的访问各个队列达到为队列提供带宽保证的目的，WRR 算法主要适用于分组为定长的应用环境。

2. 算法原理

假定有 X 个队列，每个队列各有一个记录令牌的寄存器 q_token_cnt_X。有一个用于记录当前轮询队列号的计数器 curr_rr_qnum。所有队列的令牌值是 CPU 可配的。

调度之前，如果全部非空队列的 q_token_cnt_X 都为 0，则用队列可配的令牌值刷新所有的 q_token_cnt_X 为对应的令牌。

在某个固定的时刻，以计数器 curr_rr_qnum 表示的队列号对应的队列为起始队列开始依次轮询，直到轮询到第一个非空的 q_token_cnt 非 0 队列 Y，轮询停止，那么 Y 就是当前调度出的队列号。当调度完成后，如果调度出来的队列 Y 的已空，则把对应的 q_token_cnt_Y 清零，否则把对应的 q_token_cnt_Y 减一。然后 curr_rr_qnum 减一指向下一次轮询的开始队列，等待下一个固定时刻的到来。

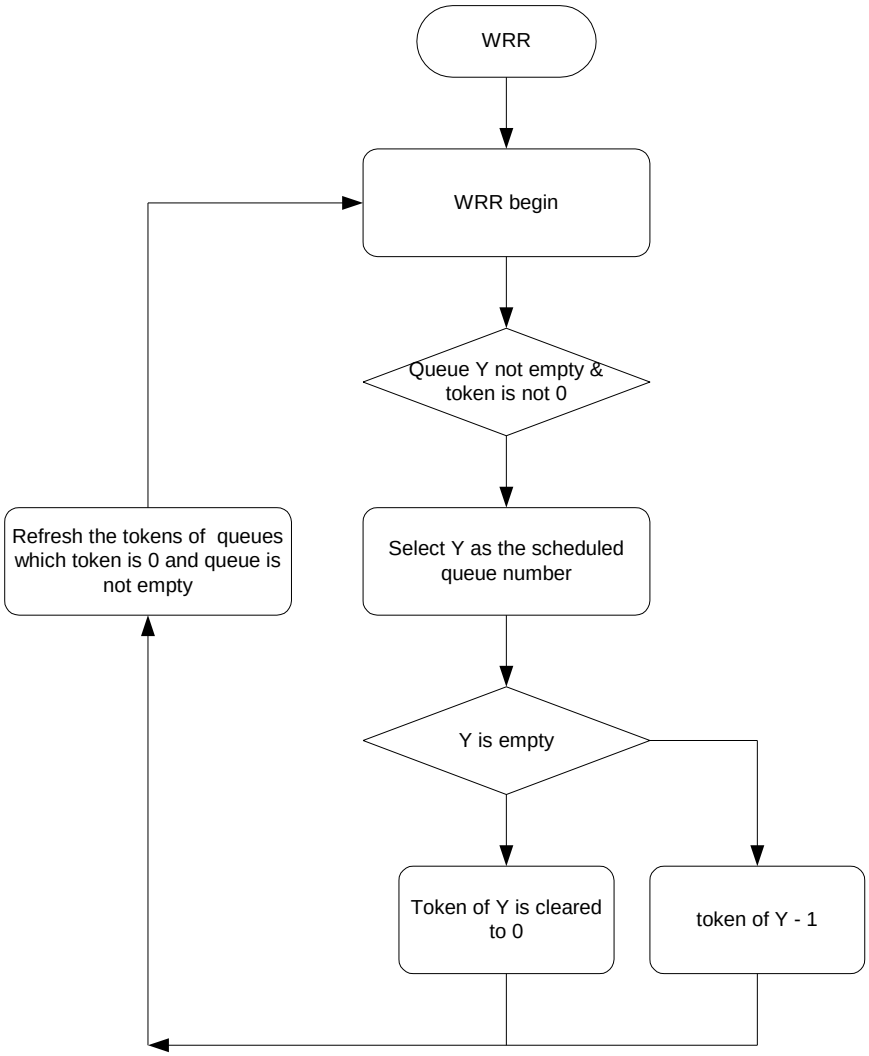


图2-146 WRR 算法流程图

3. 电路实现

下面以 6 个队列、轮询周期为 16 个时钟周期为例，对 WRR 算法的实现进行说明。其中 empty0~5 表示队列的空信号，token0~5 表示队列的令牌。可以设置 vector 从 0～15，one_hot 编码，16 级流水线设计，在 vector[1]~vector[6]之间的 time slot 进行 WRR 轮询，在 vector[8]的 time slot 输出轮询结果，vector[15]的 time slot 对 token 进行刷新，其他流水线时间空闲。

WRR 算法的具体电路设计如下：

轮询进程信号 wrr_prcs，初始值为 1'b0；当 vector[0]时，rr_prcs 置 1；当 vector[6]时，rr_prcs 清零；当前轮询队列号移位寄存器 curr_wrr_cnt[5:0]，初始值为 6'b01，每一个比特对应一个队列。

- 当 wrr_prcs 和 curr_wrr_cnt[0]有效时，如果 empty0=1 或 token0 等于零，curr_wrr_cnt 左移一位。
- 当 wrr_prcs 和 curr_wrr_cnt[1]有效时，如果 empty1=1 或 token1 等于零，curr_wrr_cnt 左移一位。

- 当 wrr_prcs 和 curr_wrr_cnt[2]有效时，如果 empty2=1 或 token2 等于零，curr_wrr_cnt 左移一位。
- 当 wrr_prcs 和 curr_wrr_cnt[3]有效时，如果 empty3=1 或 token3 等于零，curr_wrr_cnt 左移一位。
- 当 wrr_prcs 和 curr_wrr_cnt[4]有效时，如果 empty4=1 或 token4 等于零，curr_wrr_cnt 左移一位。
- 当 wrr_prcs 和 curr_wrr_cnt[5]有效时，如果 empty5=1 或 token5 等于零，curr_wrr_cnt 左移一位。
- 当 vector[8]有效时，判断下面条件是否成立：
  (empty0=0 && token0!=0) || (empty1=0 && token1!=0) || (empty2=0 && token2!=0) || (empty3=0 && token3!=0) || (empty4=0 && token4!=0) || (empty1=0 && token5!=0)
  如果条件成立，这此次队列轮询成功，输出 wrr_queue_vld 和 wrr_queue_id 信号，wrr_queue_id 就是当前的 curr_wrr_cnt 的值。当 wrr_queue_vld 信号有效时，curr_wrr_cnt 左移一位，以便轮询从下一个队列重新开始。

2.5.9 PRBS 电路

1. 应用场景

PRBS 电路主要实现设备的无仪表测试，可以在发端产生测试用伪码序列（一般为 PN 码），对伪码进行误码下插；同时在收端对伪码进行分析，提供各种告警，误码性能计数。

2. 电路说明

常用的伪随机码有 PN15、PN20、PN23、PN31，以串行或并行的方式传送。在这里以并行 16 比特的 PN23 为例，其他码型的实现方式与此相同。

PRBS23 产生模块实现方法：

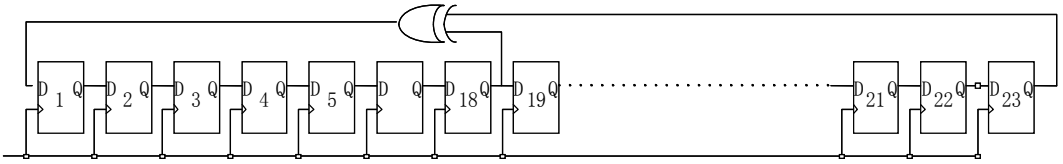


图2-147 PRBS23（$x^{23}+x^{18}+1$）产生电路原理图

PRBS23 并行 16 比特序列的伪代码如下：

prbs[23:1] <= {prbs[7:1], prbs[23]^prbs[18] , prbs[22]^prbs[17] , prbs[21]^prbs[16] , prbs[20]^prbs[15] , prbs[19]^prbs[14] , prbs[18]^prbs[13] , prbs[17]^prbs[12] , prbs[16]^prbs[11] , prbs[15]^prbs[10] , prbs[14]^prbs[9] , prbs[13]^prbs[8] , prbs[12]^prbs[7] , prbs[11]^prbs[6] , prbs[10]^prbs[5] , prbs[9]^prbs[4] , prbs[8]^prbs[3] };

pn16_result = ~prbs[23:8];

在产生 PRBS 序列过程中，PRBS 序列发生器是在外部使能信号有效的情况下发送 PRBS 序列的，其时序关系参见下图。

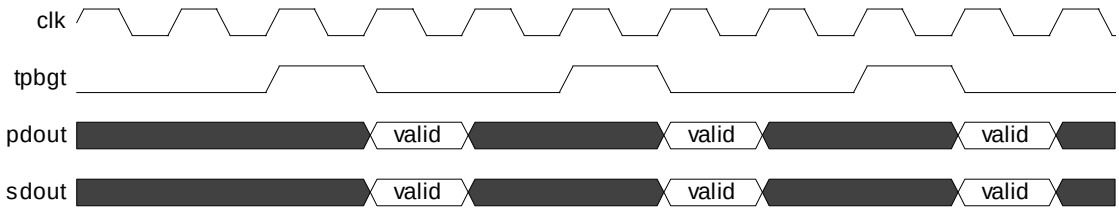


图2-148 PRBS 发送时序

在外部输入使能信号 tpbgt 信号有效后 PRBS 序列发生器延迟一个时钟周期输出有效数据，图中中黑体部分为无效数据。

PRBS23 接收模块实现方法：

在 PRBS 接收机中有个伪随机码发生器与产生器相同，在未同步时首先将输入的数据送入发生器而屏蔽发生器的反馈输入，同时将 PRBS 发生器的反馈值与输入数据进行比较。失步状态下，在连续 2048 个数据（比特）中，PRBS 误码比特高于门限值则重新同步（将输入的数据送入发生器）一次；反之则进入同步状态。需要注意的是，如果发生器的反馈值出现 15 个连 0，则将其置为全 1。同步后则将 PRBS 发生器的反馈值作为发生器的输入而屏蔽输入的数据，比较输入数据和 PRBS 生成器的反馈值，在连续 8192 个数据（比特）中，PRBS 误码比特高于门限值则进入失步状态，并同步一次；反之则保持同步状态。在同步状态下，给出误码指示和误码比特数。

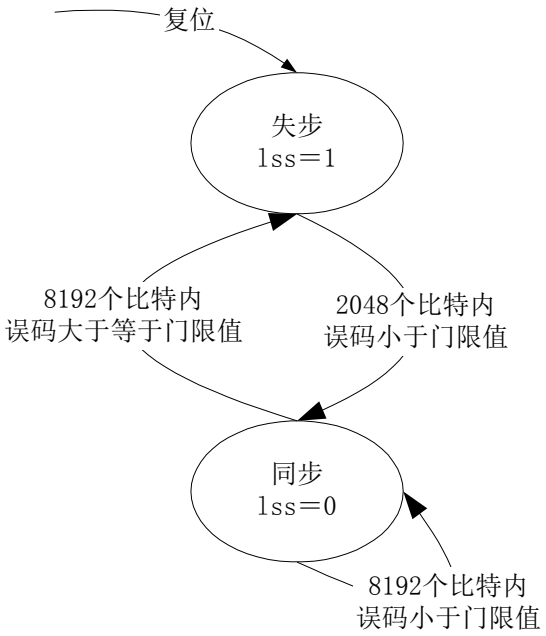


图2-149 PRBS23 接收机状态图

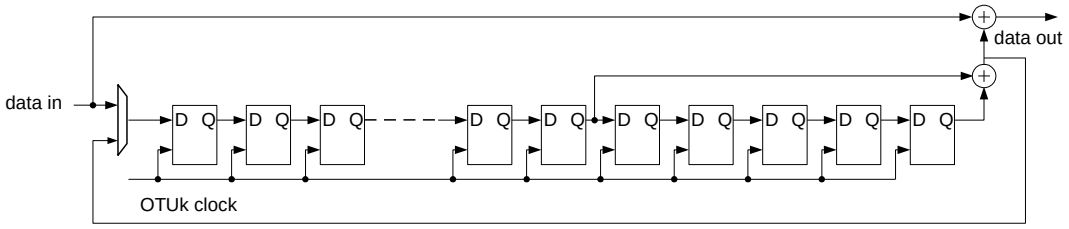


图2-150 PRBS23 测试序列检测图

检测使能有效，并且电路转入测试码流失步检测时，给出同步丢失指示 lss。

检测接收机的代码与发生器相同，请参看产生模块实现说明。

【资料来源】

《SDXXX 详细设计方案》

《PRBS IP CORE 总体方案说明书》

2.5.10 性能计数器

1. 多比特性能计数器

cnt<=cnt+1 是我们常见的，但如果 cnt 的位数很大的时候，时序上就不一定满足了。我们.35 的工艺，跑 200M，8bits 刚好能跑下来。

遇到 20 多比特的计数怎么办？拆！

例如 cnt[23:0] ,拆成 cnt0[7:0], cnt1[7:0], cnt2[7:0]

方法一：

```
always@ (posedge clk)
begin
        cnt0<=cnt0+1;
end
always @(posedge clk)
begin
        if(cnt0==8'hff)
            cnt0<=cnt0+1;
end
always @(posedge clk)
begin
        if(cnt0==8'hff && cnt1==8'hff)
            cnt2<=cnt2+1;
end
```

但是上面的方法是有限制的，加法逻辑变少了，但加法的控制就变多了，二者存在着一个均衡点。

为了解决这个问题，本文推荐方法二，可以解决任何位宽的加法问题。

方法二：

实现 cnt[6:0]

```
reg [2:0]    cnt0;    (cnt0[2]用作进位位)
reg [2:0]    cnt1;
reg [2:0]    cnt2;
always @(posedge clk )
begin
   cnt0<=add_a+1;
end
assign    add_a = cnt0[1:0];       (add_a 的初始值是 0)
assign    c0 = cnt0[2];

always @(posedge clk)
begin
   cnt1<=add_cnt1+c0;
end
assign add_cnt1=cnt1[1:0];
assign c1= cnt1[2];

always @(posedge clk)
begin
   cnt2<=add_cnt2+c1;
end
assign add_cnt2 = cnt2[1:0];

assign cnt= {cnt2[1:0], cnt1_ff1[1:0], cnt0_ff2[1:0]};
```

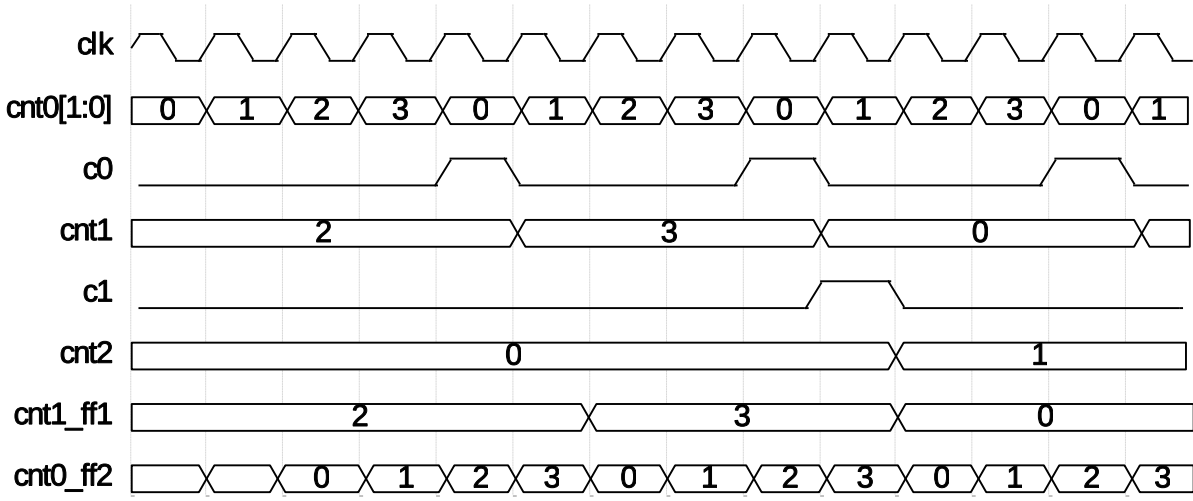


图2-151 流水加法波形图

这种方法会使结果延迟出来，把 cnt 拆为几个，就延迟了几拍。

2. 用 RAM 实现的多通道性能计数器

在网络设备上，对多个不同逻辑通道分别进行超短帧计数，超长帧计数，某些有特殊标识的帧的计数，或所有帧的计数，甚至是字节计数，是实现网络流量统计或网络问题定位的重要方法。

目前的一些多逻辑通道计数，通常采用 RAM 实现。对于多逻辑通道的计数，由于各个逻辑通道的帧到达时间相互独立，有可能在连续的多个时钟周期内，每个时钟周期都要进行计数器的翻转或累加。例如：假设需要在帧尾，进行帧个数的统计，而相继处理的各个逻辑通道在连续多个时钟周期内，到来的正好都是帧尾，则需要在每个时钟周期进行各个逻辑通道的计数器更新，此时如果要进行 mpi 的读访问，则会产生多逻辑通道计数处理和 mpi 的访问冲突。因此，为避免冲突，一般要增加相应的逻辑，进行冲突处理，而这种做法带来的直接问题是：仿真时，难于构造类似的冲突场景，给芯片最终的成功应用带来隐患。另外多逻辑通道计数器还应该具备饱和和读清零的功能，因此在 mpi 访问后，一般还需要进行回写操作，这些地方处理不好，都将会造成计数器多计和漏计等问题。

下面将描述一种简单的实现多逻辑通道计数的方法，该方法接口简单，无需进行冲突处理，能实现某一逻辑通道或不同逻辑通道，连续多个时钟周期均需要进行计数器更新的多逻辑通道计数功能，此外，该计数器具备饱和和读清零功能。

(1) 方案一

采用两块 Dual Port RAM 实现计数功能。2 块 RAM 假定为 RAM_A，RAM_ B，原理如下图所示：

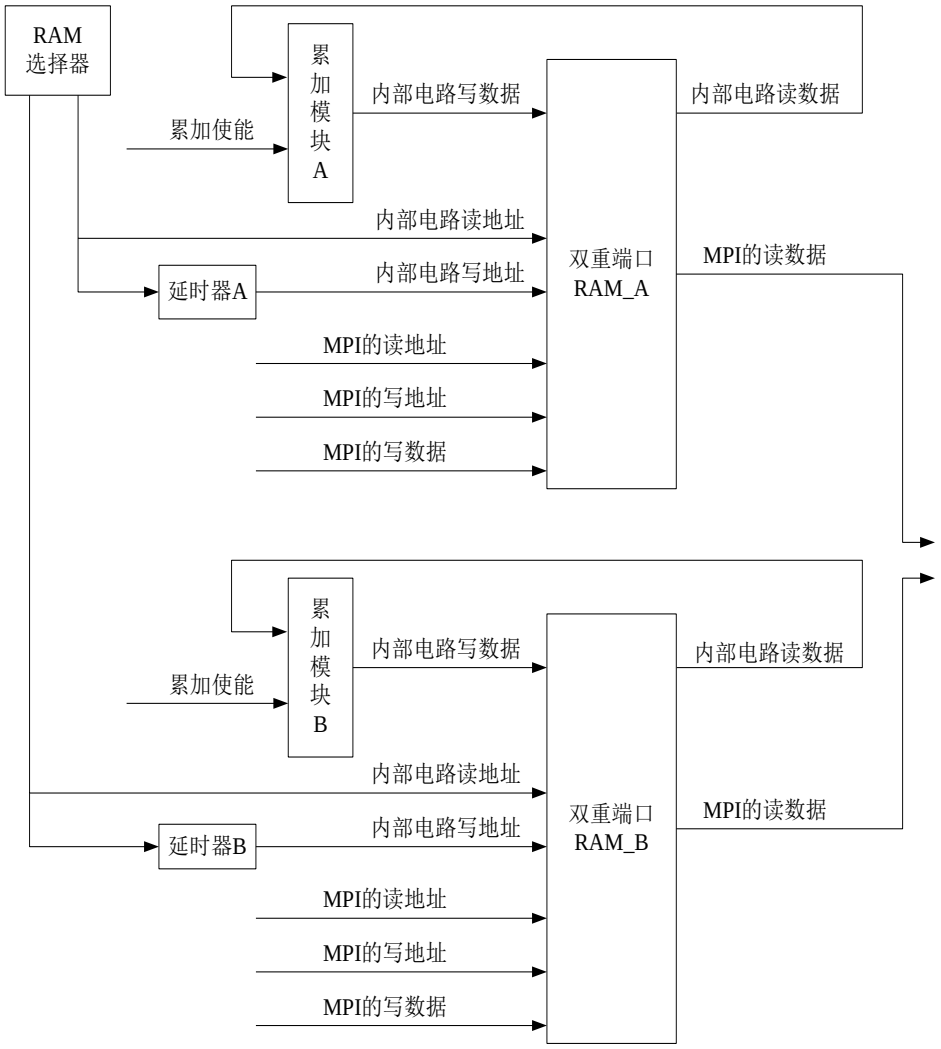


图2-152 采用两块 Dual Port RAM 实现多通道计数示意图

实现原理：

RAM_A 和 RAM_B 为 Dual Port RAM，二者均可以提供两组读写地址，每块 RAM 的两组读写地址，分别供内部逻辑通道和 mpi 读访问使用，但同一组地址中，RAM 的读写地址复用。

逻辑通道计数通常分地址存储在 RAM 中，当某个逻辑通道有计数更新需求时，根据该逻辑通道计数的存储地址，从 RAM 中读出相应的计数值，进行累加，并再次写入该逻辑通道对应的 RAM 地址。从该逻辑通道对应的 RAM 读写地址中读出数据，至少需要一个时钟周期，之后根据读出的值累加后，再将该逻辑通道对应的计数值写入对应的 RAM 地址，至少还需要一个时钟周期，因此至少共需要两个时钟周期才能完成某一个逻辑通道的计数更新处理。考虑到每个时钟周期都可能需要对某些逻辑通道进行计数更新，故采用两块 Dual Port RAM 协同计数的方式实现多逻辑通道的计数，并由 RAM 选择器决定选择 RAM_A 还是 RAM_B 用于当前逻辑通道的计数器处理，如：当前采用 RAM_A 进行当前逻辑通道的计数处理，则下一个时钟周期，采用 RAM_B 进行下一个时钟周期到来的同一逻辑通道或不同逻辑通道的计数处理。

前面提到每块 Dual Port RAM，可以提供两组读写地址，分别供内部逻辑通道和 mpi 读使用。由于采用 RAM_A 和 RAM_B，分时对所有逻辑通道进行计数，则每个逻辑通道的计数分别存储在两块 RAM 中。因此当 mpi 进行某一逻辑通道的读访问时，需要将 RAM_A 和 RAM_B 设置相等的 mpi 读逻辑通道地址，并将二者的读出值相加，才能得到相应逻辑通道的计数值。随后，为进行读清零处理，需要将 mpi 对应的读逻辑通道地址进行回写操作。由于在回写过程中对应的逻辑通道可能有新的计数需求，因此，需要考虑相应的冲突处理：即要根据内部当前处理的逻辑通道号和当前 mpi 读逻辑通道号决定应该在 RAM_A 和 RAM_B 对应地址的回写值，当冲突时，要分别将 RAM_A 和 RAM_B 中对应的逻辑通道回写 1 和 0 或 0 和 1 或 1 和 1，否则将 RAM_A 和 RAM_B 中对应的逻辑通道直接回写 0。

方案一的缺点：

a．需要两块 Dual Port RAM 协同工作，两块 Dual Port RAM 从资源上，约等同于 4 块 Two Port RAM 才能完成相应的计数功能，RAM 资源的消耗较大；

b．要考虑 mpi 和内部逻辑通道的冲突处理，有可能引入未知错误，可靠性低。

(2) 方案二

方案二所要解决的问题有以下两个方面，其一是用较少的 RAM 资源实现多逻辑通道计数，该方案接口简单，无需进行冲突处理，能实现对某一逻辑通道或不同逻辑通道连续多个时钟周期，需要进行计数更新的多逻辑通道计数功能，此外，该计数器还具备饱和和读清零功能。

对于多个逻辑通道的计数需求，主要包括以下 3 个方面：1、可能每个时钟周期，都需要将对应逻辑通道的计数器进行累加；2、mpi 需要随时可读；3、计数器应具有饱和和读清功能。

综合以上规格考虑，本发明采用 3 块 Two Port RAM 实现，3 块 RAM 假定为 RAM_A，RAM_ B，RAM_C。

RAM_A：对当前处理的逻辑通道进行计数更新功能；

RAM_B：主要提供 MPI 的读端口；

RAM_C：和 RAM_B 共同实现计数器的饱和清零功能。

原理如下图所示：

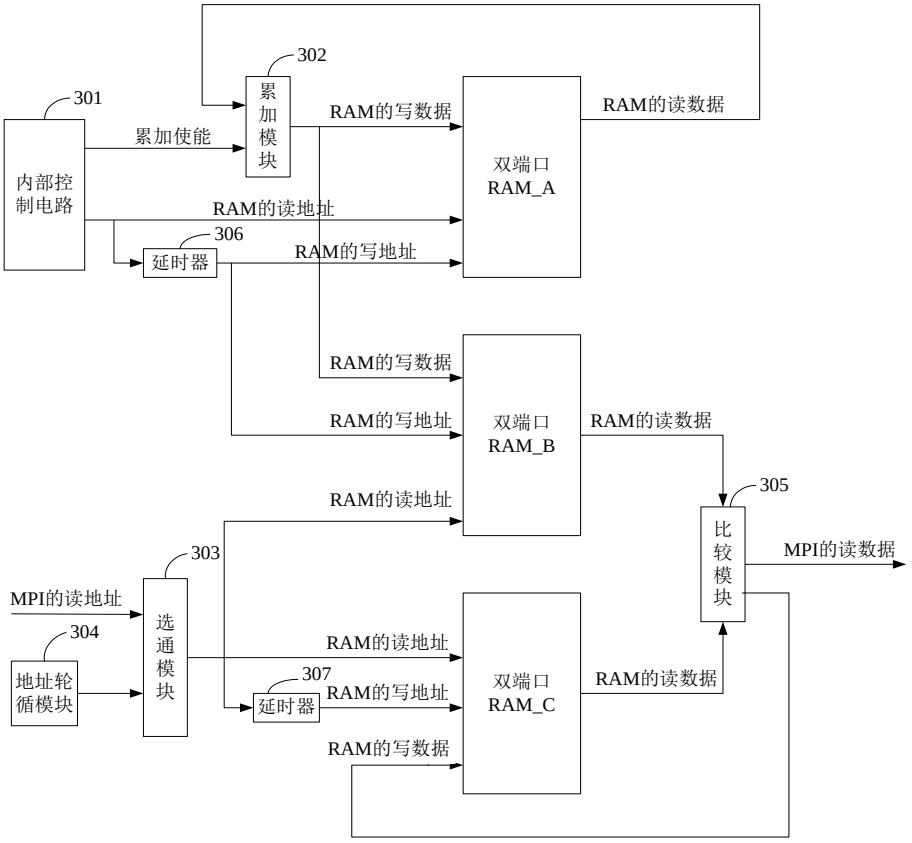


图2-153 采用三块 Two Port RAM 实现多通道计数示意图

实现原理：

当前处理的某一个逻辑通道需要进行计数累加，即内部逻辑给出的计数器累加使能有效时，则在 RAM_A 读出的该逻辑通道对应的计数值基础上累加，并分别重新写入 RAM_A 和 RAM_B；即 RAM_A 和 RAM_B 存储的时钟是最新的计数值。由于读写地址相互独立，应此可以完成每个时钟周期都需要进行计数器更新的需求。

RAM_B 和 RAM_C 配合实现 mpi 读清和饱和计数功能。

当 mpi 不读时，RAM_C 对所有逻辑通道对应的 RAM 地址进行轮循，比较 RAM_B 和 RAM_C 的读数据差值，当二者的差值大于饱和值，则将 RAM_C 中的计数值进行加操作，保证二者差值为饱和值；当 mpi 读操作进行时，二者的差值小于饱和值，则将二者差值送出，否则直接将饱和值送出，从而实现饱和计数功能；

mpi 读操作后，将 RAM_B 中的读出值写入 RAM_C，这样，当没有短帧到来，再次读该通道的计数值，则为二者的差值、即 0，从而实现读清零功能。

从设计的技巧来看，本方案充分利用了 mpi 读操作的慢速性，根据 mpi 的访问特性，不可能出现每个时钟周期都进行不同或同一个逻辑通道的读操作，也就是说，当 mpi 进行某一逻辑通道的读访问后，为满足读清零的功能，有足够的时间将 RAM_B 的读出值写入到 RAM_C 中，而不需要进行任何冲突处理。

由上述的分析可见 3 块 RAM 实现的功能相对独立，进行 mpi 读写时，不需要处理内部逻辑通道和 mpi 的访问冲突。

2.5.11 环回测试电路

环回测试功能作为定位问题的重要手段，开发人员和测试人员通常通过配置相应的环回业务，对芯片相应的数据通路进行问题定位。因此，环回电路是芯片设计中不可缺少的部分。

环回分为内环回和外环回两种方式。

本节以 GE MAC 的环回电路为例，介绍环回电路的基本思想，在此引入两个概念：接收通道和发送通道。

接收通道：数据由 PHY 侧到 SYS 侧所经过的 MAC 模块内部的数据通道；

发送通道：数据由 SYS 侧到 PHY 侧所经过的 MAC 模块内部的数据通道。

GE MAC 提供内环回和外环回两种模式，用于芯片测试（测试时应发送正常 MAC 帧）

1. 内环回

内环回是指在 MAC_CORE 模块和 RGMII 模块相连的 GMII 接口处，将数据由发送通道直接环回到接收通道，环回数据流向如图中红线所示：

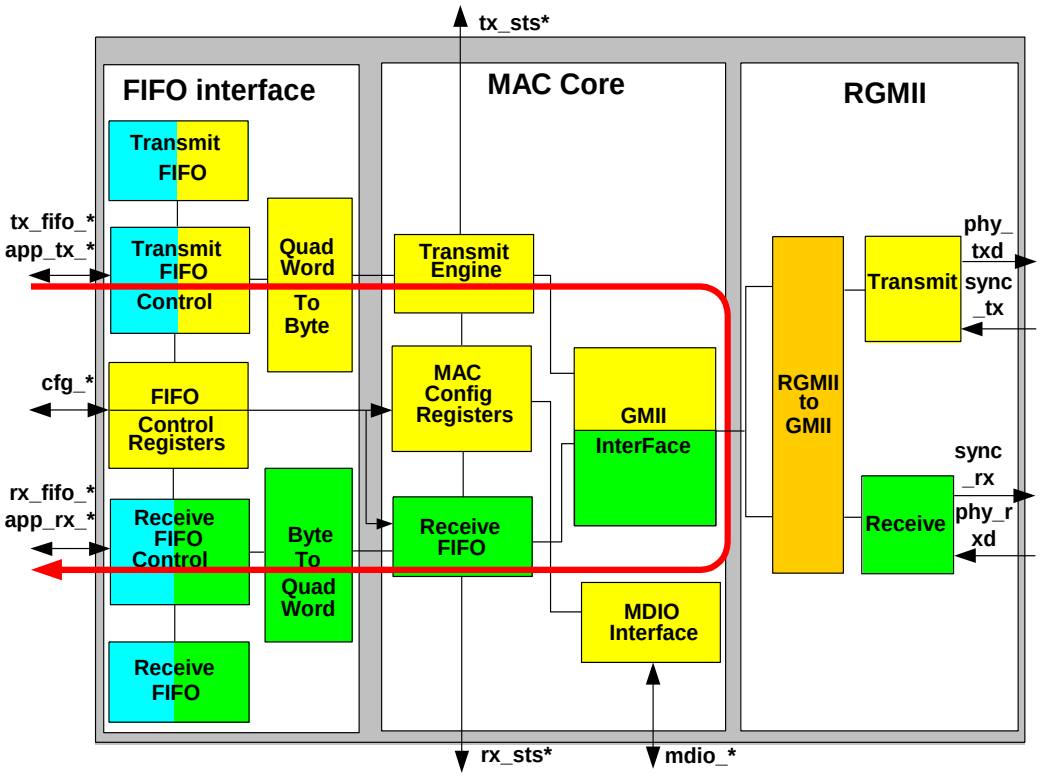


图2-154 GE MAC 内环回示意图

发送通道数据直接环回到接收通道，内环回的实现通过一个异步 FIFO 实现。发送数据通道将数据写入到异步 FIFO 中，同时，接收通道直接从 FIFO 中将数据读出，将数据交给后续逻辑处理。因此，内环回模式能够测试除 RGMII 接口以外的 GE MAC 的子模块，并且包括 MAC 和 SYS 侧接口的各项功能。

如果芯片涉及的 GE MAC 端口较多，为了测试每个 GE MAC 端口和 SYS 侧的各项功能是否符合设计要求，内环回模式能够提供一种简单有效的测试方式（如图所示），完成芯片的测试：GE0—>通道 0—>GE1—>通道 1—>……GEn-1—通道 n-1—GEn。其中，GE1 至 GEn-1 均设置为内环回模式，激励通过 GE0 进入芯片，最后通过 GEn 输出响应。通过这种方式，可以用一个激励同时对所有的 GE MAC 和 SYS 侧的功能进行测试。

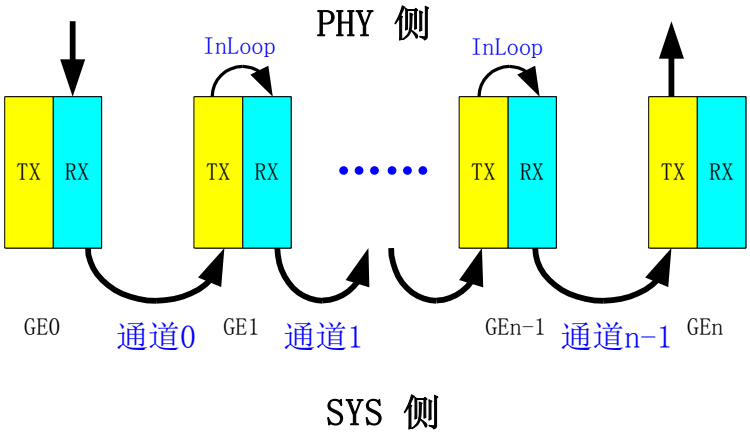


图2-155 内环回测试示意图

2. 外环回

外环回是指在 PHY 侧的异步 FIFO 接口处，将数据由接收通道直接环回到发送通道，环回数据流向如图中红线所示。

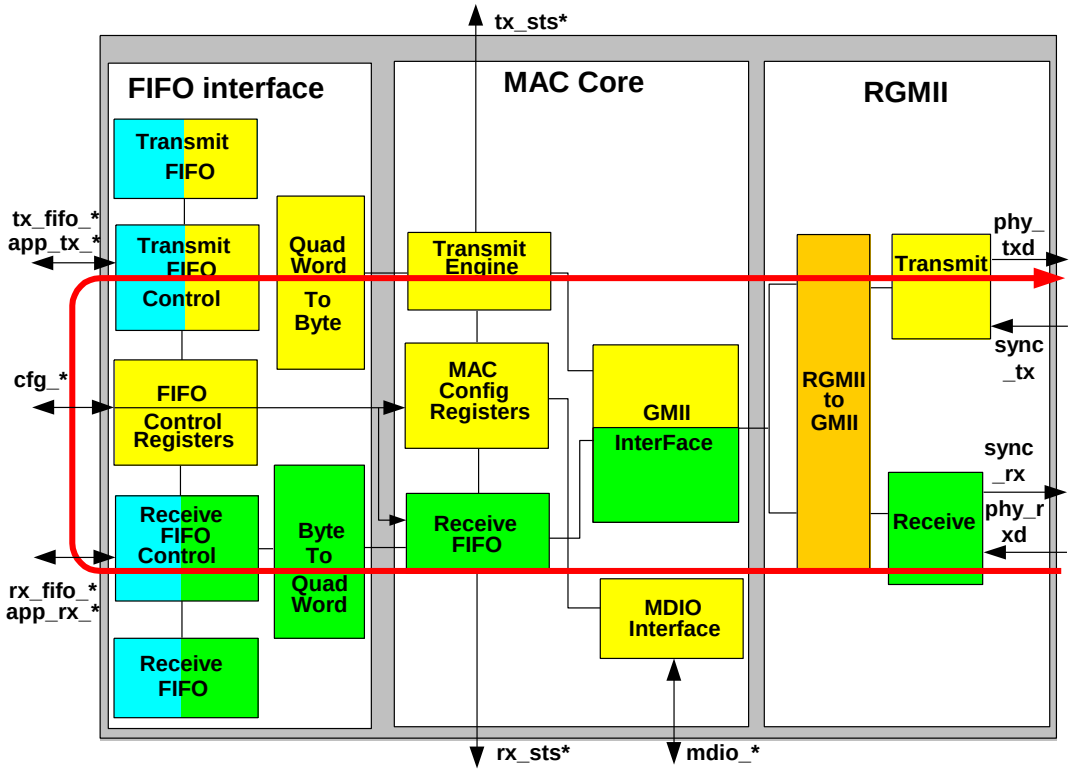


图2-156 GE MAC 外环回示意图

接收通道数据通过与 SYS 侧的异步 FIFO 直接环回到发送通道，数据不需要经过 SYS 侧处理。因此，外环回模式能够最大限度的测试 GE MAC 端口的各项功能。

通过对 GE MAC 的两种环回模式分析可以看出，两种环回模式都是通过 FIFO 将两个数据通道直接连接起来，将其他处理单元的数据通道隔离，以排除其他模块对测试数据通道的影响，极大地提高了定位问题的效率。环回的基本思想如图所示：

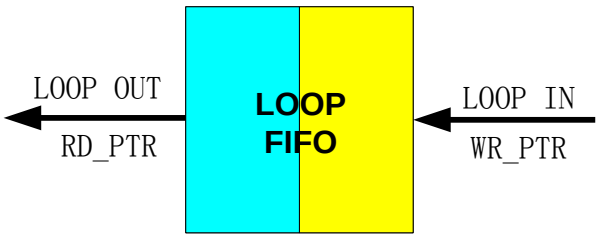


图2-157 环回实现示意图

环回的数据通道，环回输入将数据写入环回 FIFO 中，环回输出将数据从 FIFO 中读出。根据数据报文的特点，保证此环回 FIFO 不丢失数据（FIFO 上溢），也不会上溢或产生错包。此方面的设计和异步 FIFO 设计相类似，可以参考异步 FIFO 的相关章节。

2.5.12 上下沿检测电路

1. 概述

按照应用场景与实现功能区分，时钟沿检测电路可分为上升沿检测电路、下降沿检测电路与上/下沿检测电路。其中上/下沿检测电路支持上升沿与下降沿的检测。

2. 电路实现

- 上升沿检测电路：简单的上升沿检测电路可使用单一的触发器实现，如下图所示：

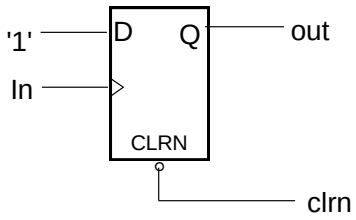


图2-158 简单的上升沿检测电路

图中，被检测信号连接D触发器时钟端，D触发器数据端固定连接高电平，被检测信号发生上升沿跳变则D触发器输出高电平。不建议采用这种电路检测信号的上升沿，因为IN为数据信号，容易引入毛刺，使触发器误动作。

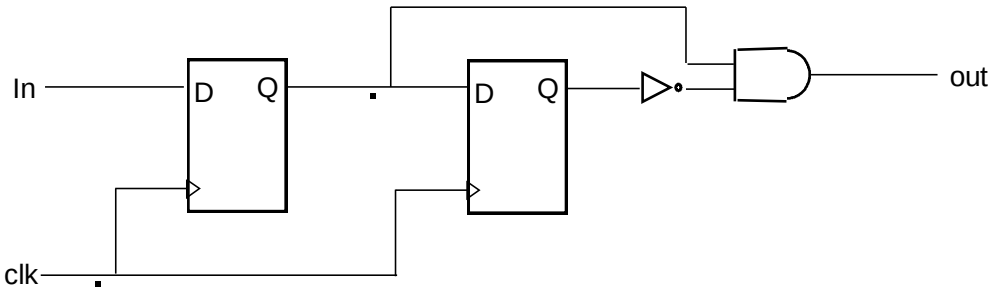


图2-159 上升沿检测推荐电路

上图为推荐使用的上升沿检测电路，该电路采用时钟检测信号，当被检测信号出现0->1的变化，与门输出信号被置为高电平，表示上升沿检测有效。当被测信号为本时钟域的异步信号时，须进行同步化处理后再进行上升沿检测。

本电路的代码描述如下：

```
always @（posedge clk or negedge rst_n）begin
    if（rst_n == 1'b0）begin
```

```
            in_ff1 <= 1’b0;
        end
        else   begin
            in_ff1 <= in;
        end
end

always @（posedge clk or negedge rst_n）begin
        if（rst_n == 1’b0）begin
            in_ff2 <= 1’b0;
        end
        else   begin
            in_ff2 <= in_ff1;
        end
end

always @（in_ff1 or in_ff2）begin
        if（（in_ff1 == 1’b1）&&（in_ff2 == 1’b0））begin
            out = 1’b1;
        end
        else   begin
            out = 1’b0;
        end
end
```

■ 下降沿检测

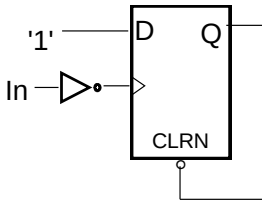


图2-160 简单的下降沿检测电路

上图为一简单的下降沿检测电路，与上文中的简单上升沿检测电路的区别在于，该电路使用被检测信号取反后作为触发器的时钟端输入。同上升沿检测电路，这种电路对毛刺、干扰极为敏感。

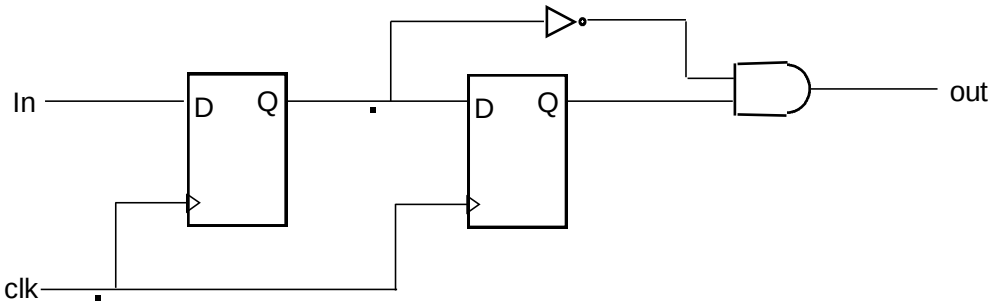


图2-161 下降沿检测推荐电路

上图为推荐使用的下降沿检测电路，该电路采用时钟检测信号，当被检测信号出现1->0的变化，与门输出信号被置为高电平，表示下降沿检测有效。当被测信号为本时钟域的异步信号时，须进行同步化处理后再进行上升沿检测。

本电路的代码描述如下：

```
always @（posedge clk or negedge rst_n）begin
    if（rst_n == 1’b0）begin
        in_ff1 <= 1’b0;
    end
    else   begin
        in_ff1 <= in;
    end
end

always @（posedge clk or negedge rst_n）begin
    if（rst_n == 1’b0）begin
        in_ff2 <= 1’b0;
    end
    else   begin
        in_ff2 <= in_ff1;
    end
end

always @（in_ff1 or in_ff2）begin
    if（（in_ff1 == 1’b0）&&（in_ff2== 1’b1））begin
        out = 1’b1;
    end
```

```
    else   begin
        out = 1'b0;
    end
end
```

10.上升/下降沿检测

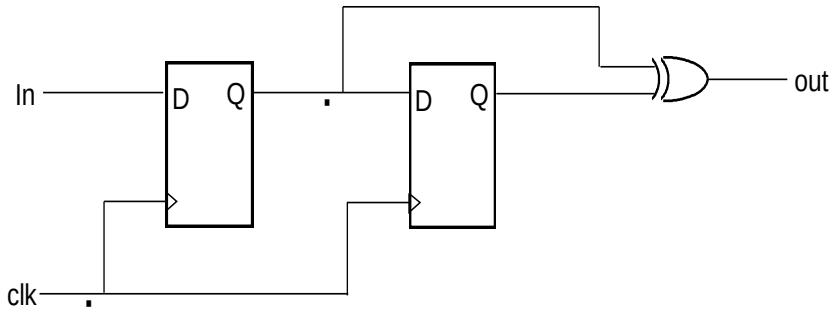


图2-162 上升/下降沿检测推荐电路

上图为推荐使用的上升/下降沿检测电路，与上文的上升沿检测或下降沿检测电路不同，该电路使用一异或门对输入信号进行判断，采用时钟检测信号，当被检测信号出现1->0或0->1的变化时，异或门输出信号被置为高电平，表示检测有效。当被测信号为本时钟域的异步信号时，须进行同步化处理后再进行上升沿检测。

本电路的代码描述如下：

```
always @（posedge clk or negedge rst_n）begin
    if（rst_n == 1'b0）begin
        in_ff1 <= 1'b0;
    end
    else   begin
        in_ff1 <= in;
    end
end

always @（posedge clk or negedge rst_n）begin
    if（rst_n == 1'b0）begin
        in_ff2 <= 1'b0;
    end
```

```
    else   begin
        in_ff2 <= in_ff1;
    end
end

always @（in_ff1 or in_ff2）begin
    if（ in_ff1 != in_ff2 ）begin
        out = 1’b1;
    end
    else   begin
        out = 1’b0;
    end
end
```

2.5.13 FIFO 设计

【应用场景】

先进先出缓存电路 FIFO 在大规模逻辑设计当中，得到普遍地使用。几乎每个芯片都会涉及到它。也就是说每个大规模逻辑设计者都必须掌握该种电路的设计方法。但在实际应用中，FIFO 的类型又不尽相同。有的电路需要的是同步 FIFO（即：读写时钟为同一个时钟），有的需要异步 FIFO（即：读写时钟的频率或相位不一样），有的电路对 FIFO 的速度要求不高，有的则需要高速 FIFO（比如工作频率在 100MHz 以上）。同时具体电路对 FIFO 的空满标志的产生也有一定的要求，有按一个深度进行读写的，也有连续读写一块的（Burst read/write），有读写位宽一致的(256 深×80bit <==> 256 深×80bit)，也有读写位宽是倍数关系的（256 深×80bit<==>1024 深×20bit)。异步 FIFO 的读写地址转变为格雷码再进行比较，产生空满标志，从而保证电路的可靠性。

XILINX 的 Coregen 软件可以生成一些通用的同步和异步 FIFO，如果它不能满足你的特定要求，只好你自己动手了！不管是什么类型的 FIFO，其关键点是产生读、写地址和空、满标志。好的 FIFO 设计的基本要求是： 写满而不溢出； 读空又不多读。

【同步 FIFO 的设计】

同步 FIFO 的读写时钟为相同的一个，所以读写地址是同步的。通过判断 FIFO 中已经存在多少个没有被读走的数据 Cnt_fifo，来判断读空了，还是写满了。Cnt_fifo 的产生：当 FIFO 进行读但不写时，自减 1，当 FIFO 进行写但不读时，自增加 1，如果 FIFO 又读又写或不读也不写，则该计数器不变。

1. 空满标志的产生

满标志的产生：复位时，Full 信号初始化为 0，表示 FIFO 不满；如果 FIFO 不读，并且 Cnt_fifo 等于 FIFO 的深度 Depth_fifo 时，或者 Cnt_fifo 等于(Depth_fifo-1)，并且正在写、没有读（即写使能 Write_enble 有效，且读使能无效）时，满标志应该置 1；其余情况置为 0。

空标志的产生：复位时，让 Empty 信号为 1，表示 FIFO 空；当 FIFO 中的 Cnt_fifo 等于 0，或者等于 1 并且正在读、没有写 FIFO 时，Empty 就应置 1，表示读空了；其余情况则不空。置为 0。

几乎空 Almost_empty 和几乎满 Almost_full 标志的产生：在实际应用中为了便于正确产生读写使能，避免多读和写溢出，经常使用几乎空和几乎满标志，来提前预告 FIFO 的状态。要求 FIFO 还剩下可编程数量 Cnt_empty 的数据没读走时，就报告几乎空，但是仍允许读走几个。或者写入 FIFO 的数据个数超过可编程数量 Cnt_full 的时候，就报告几乎满，但仍允许写入几个数据。也就是说当 Cnt_fifo>Cnt_full 时，FIFO 就要置 Almost_full 为 1，表示几乎满了。当 Cnt_fifo<Cnt_empty 时，FIFO 就要置 Almost_empty 为 1，表示几乎空了。

2. 读写使能的产生

读写 FIFO 的使能的产生：为了确保 FIFO 即不多读又不多写，采用了自我保护方式，即：

真正的写 FIFO 使能为外部的写使能和 FIFO 不满的组合；

（assign Write_allow = Write_enable && ! Full ;）

真正的读 FIFO 使能为外部的读使能和非空的组合；

（assign Read_allow = Read_enable && ! Empty ; ）。

3. 读写地址的产生

FIFO 的读写地址产生比较简单，当读使能有效时，在时钟作用下，读地址加 1；当写使能有效时，写地址加 1。

当 FIFO 深度较大时，同时 FIFO 的速度要求较高时，可以采用线性反馈移位计数器（LFSR）。它的速度非常快，但是要牺牲一个地址。

针对同步的大 FIFO，它们的读写地址完全可以使用线性反馈移位寄存器（LFSR）产生，而不是简单的加 1 操作，极大的提高了速度，如果对 FIFO 的利用率没有很高要求的时候，推荐使用该方法。使用 LFSR 的优点是在 XILINX 的 FPGA 中布线，可以使用 LUT 直接完成。

/***\

* *

* Generation of Read and Write address pointers. They use *

* LFSR counters, which are very fast. Because of the *

* nature of LFSR 扭, one address is sacrificed. *

```
* *
\************************************************************/
wire read_linearfeedback, write_linearfeedback;
assign read_linearfeedback = ! (read_addr[8] ^ read_addr[4]);
assign write_linearfeedback = ! (write_addr[8] ^ write_addr[4]);
always @(posedge clock or posedge fifo_gsr)
if (fifo_gsr) read_addr <= 9'h0;
else if (read_allow)
  read_addr <= { read_addr[7], read_addr[6], read_addr[5],
                 read_addr[4], read_addr[3], read_addr[2],
                 read_addr[1], read_addr[0], read_linearfeedback };
always @(posedge clock or posedge fifo_gsr)
if (fifo_gsr) write_addr <= 9'h0
else if (write_allow)
  write_addr <= { write_addr[7], write_addr[6], write_addr[5],
                 write_addr[4], write_addr[3], write_addr[2],
                 write_addr[1], write_addr[0], write_linearfeedback };
```

Table 3: Taps for Maximum-Length LFSR Counters

n	XNOR from	n	XNOR from	n	XNOR from	n	XNOR from
3	3,2	45	45,44,42,41	87	87,74	129	129,124
4	4,3	46	46,45,26,25	88	88,87,17,16	130	130,127
5	5,3	47	47,42	89	89,51	131	131,130,84,83
6	6,5	48	48,47,21,20	90	90,89,72,71	132	132,103
7	7,6	49	49,40	91	91,90,8,7	133	133,132,82,81
8	8,6,5,4	50	50,49,24,23	92	92,91,80,79	134	134,77
9	9,5	51	51,50,36,35	93	93,91	135	135,124
10	10,7	52	52,49	94	94,73	136	136,135,11,10
11	11,9	53	53,52,38,37	95	95,84	137	137,116
12	12,6,4,1	54	54,53,18,17	96	96,94,49,47	138	138,137,131,130
13	13,4,3,1	55	55,31	97	97,91	139	139,136,134,131
14	14,5,3,1	56	56,55,35,34	98	98,87	140	140,111
15	15,14	57	57,50	99	99,97,54,52	141	141,140,110,109
16	16,15,13,4	58	58,39	100	100,63	142	142,121
17	17,14	59	59,58,38,37	101	101,100,95,94	143	143,142,123,122
18	18,11	60	60,59	102	102,101,36,35	144	144,143,75,74
19	19,6,2,1	61	61,60,46,45	103	103,94	145	145,93
20	20,17	62	62,61,6,5	104	104,103,94,93	146	146,145,87,86
21	21,19	63	63,62	105	105,89	147	147,146,110,109
22	22,21	64	64,63,61,60	106	106,91	148	148,121
23	23,18	65	65,47	107	107,105,44,42	149	149,148,40,39
24	24,23,22,17	66	66,65,57,56	108	108,77	150	150,97
25	25,22	67	67,66,58,57	109	109,108,103,102	151	151,148
26	26,6,2,1	68	68,59	110	110,109,98,97	152	152,151,87,86
27	27,5,2,1	69	69,67,42,40	111	111,101	153	153,152
28	28,25	70	70,69,55,54	112	112,110,69,67	154	154,152,27,25
29	29,27	71	71,65	113	113,104	155	155,154,124,123
30	30,6,4,1	72	72,66,25,19	114	114,113,33,32	156	156,155,41,40
31	31,28	73	73,48	115	115,114,101,100	157	157,156,131,130
32	32,22,2,1	74	74,73,59,58	116	116,115,46,45	158	158,157,132,131
33	33,20	75	75,74,65,64	117	117,115,99,97	159	159,128
34	34,27,2,1	76	76,75,41,40	118	118,85	160	160,159,142,141
35	35,33	77	77,76,47,46	119	119,111	161	161,143
36	36,25	78	78,77,59,58	120	120,113,9,2	162	162,161,75,74
37	37,5,4,3,2,1	79	79,70	121	121,103	163	163,162,104,103
38	38,6,5,1	80	80,79,43,42	122	122,121,63,62	164	164,163,151,150
39	39,35	81	81,77	123	123,121	165	165,164,135,134
40	40,38,21,19	82	82,79,47,44	124	124,87	166	166,165,128,127
41	41,38	83	83,82,38,37	125	125,124,18,17	167	167,161
42	42,41,20,19	84	84,71	126	126,125,90,89	168	168,166,153,151
43	43,42,38,37	85	85,84,58,57	127	127,126		
44	44,43,18,17	86	86,85,74,73	128	128,126,101,99		

图2-163 线性反馈表

【异步 FIFO 的设计】

所谓异步是指读、写时钟是完全独立并且不一致的，或者不同频率，或者同频但不同相。读地址和空标志是由读时钟产生的，而写地址和满标志则由写时钟产生，当要产生 FIFO 的空、满标志时，必须进行读写地址的比较时，问题就来临了。如果直接采样地址比较的话，地址线一般有多位，由于每个地址寄存器的物理空间位置的不一致性，造成写地址的每一位在写时钟作用下，跳变得不一致，即产生毛刺，要过一小段时间才能稳定。在未稳定期内，刚好读时钟进行采样写地址，如果正好读写地址一样，这时就出现误判断，逻辑错误。同时

采样读写地址相差 N 个来产生空满信号，时间上会多一些，因为涉及加和减操作。

为避免地址跳变的不一致造成读写地址误判断，采样格雷码。该码的优点是相邻两值只有一位跳变，其他不变，这样地址变化的时间较短，极大提高比较精度。

格雷码是一种做加一运算时，只变化一位的编码，下表即为一个三位格雷码编码格式：格雷码映射表。

原码	格雷码(Q2)	格雷码(Q1)	格雷码(Q0)
0	0	0	0
1	0	0	1
2	0	1	1
3	0	1	0
4	1	1	0
5	1	1	1
6	1	0	1
7	1	0	0

可见，格雷码每次只在相邻的位发生变化。这种编码带来的好处是它可避免前面介绍的因线延迟不一致而引起的毛刺现象。

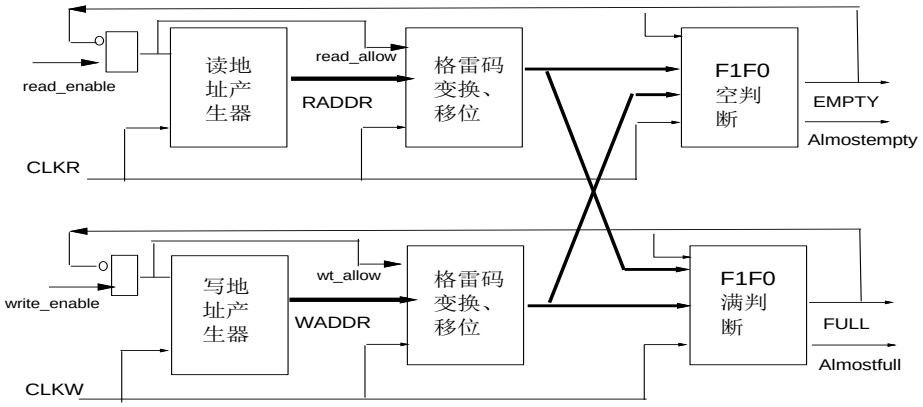


图2-164 格雷码 FIFO 的基本原理图

运用格雷码编写异步 FIFO 的基本思想如上图所示。

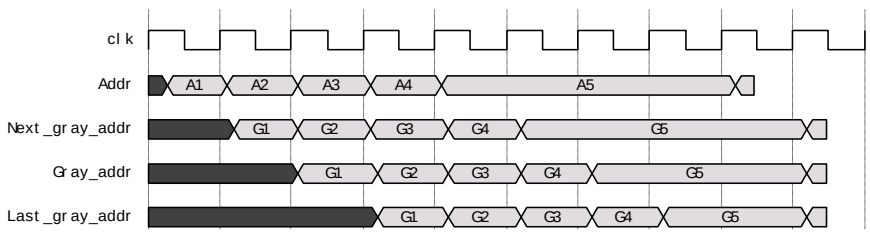


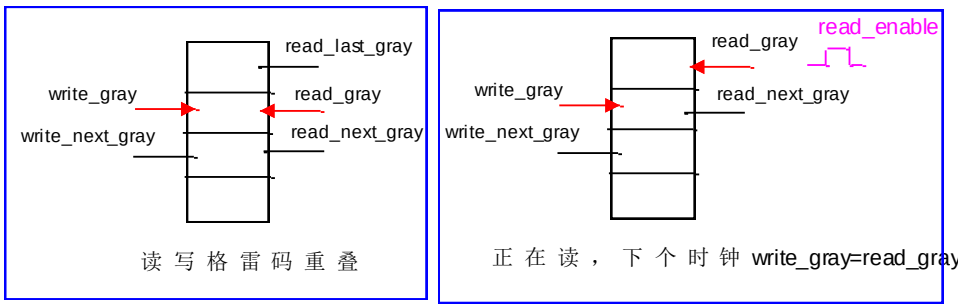
图2-165 格雷码的时序

格雷码是不能进行加减产生空满标志的，那怎么办？采样延时一拍的方法。用读地址 Rd_addr 产生读地址的格雷码 Rd_next_gray_addr，将 Rd_next_gray_addr 延一拍得到 Rd_gray_addr，再将 Rd_gray_addr 延一拍得到 Rd_last_gray_addr。你会发现在绝对时间上，Rd_next_gray_addr、Rd_gray_addr、Rd_last_gray_addr 这个地址有先后关系，从大到小排列，并且相差 1。如图所示。

写地址的格雷码的产生也类似。即： Wt_next_gray_addr、Wt_gray_addr、Wt_last_gray_addr。利用这 6 个格雷码进行比较，同时加上读写使能，就能表示空和满标志。

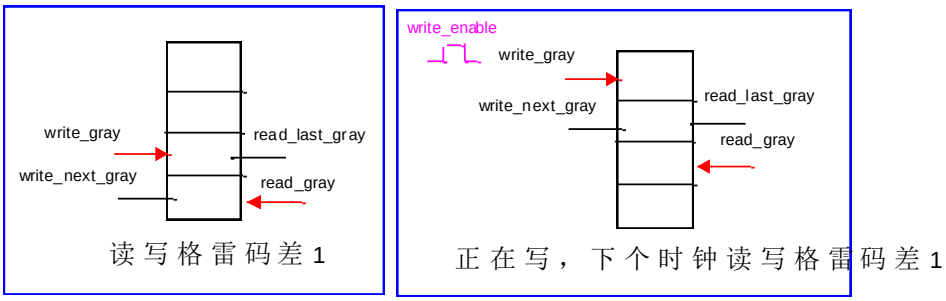
4. 空满标志的产生

先说空标志 Empty 吧！当读写格雷码地址相等（Rd_gray_addr = = Wt_gray_addr）或者 FIFO 内还剩下一个深度的字（Rd_next_gray_addr = = Wt_gray_addr），并且正在不空的情况下执行读操作（Read_enable = = 1），这时 Empty 标志应该置为有效（高电平有效）。如下图所示：



空信号的产生

图2-166 空标志信号产生示意图



满标志的产生

图2-167 满标志信号产生示意图

满标志 Full：当写 FIFO 的格雷码地址等于上次读的格雷码地址时（Wt_gray_addr= = Rd_last_gray_addr），或者下次要写的格雷码地址等于上次读的格雷码地址（Wt_next_gray_addr= = Rd_last_gray_addr），并且正在

执行写操作。此时需要置 Full 标志有效。为了避免复杂的逻辑，提高 FIFO 的整体速度，可使用 FIFO 深度＝实际深度－1。如上图 4 所示。

5. 几乎空、几乎满标志的产生

如果要产生几乎空、几乎满标志时，可以多做几个格雷码的延时地址，利用这些读、写格雷码地址距离远近关系就可以灵活的产生特定读写地址间距的几乎空或几乎满标志。如果需要在大间距内时，产生几乎空满信号（比如读写地址相差 10），那必须采用另外一种方法：以几乎满为例。

一：首先得到 FIFO 已经有多少个未读数据，只要使用写地址减去读地址就可以了。原因是结果和两个读写地址的位宽一样，借位和进位不理会。读者可以自己验算一下。

二：异步电路要产生精确的几乎空和几乎满比较难，都有一定的误差。但是具体情况（已知读、写时钟频率）可以做到误差很小。

下面给出了几乎满的控制逻辑，首先将读地址 read_addr 变成格雷码 Rd_truegray(相对于读时钟)，再用写时钟同步该读地址的格雷码 Rd_truegray 得到 Rag_wt_syn(相对于写时钟)，

然后将同步后的格雷码 Rag_wt_syn 变成二进制的地址 Ra_write_syn，最后用写地址的延时一拍 Wt_addr_p1 减去同步后的地址 Ra_write_syn（为什么写地址延时，是为保证同一时刻的读写地址比较），就得到比较精确的 FIFO 有效数据的个数 Fifo_status。利用该个数与 FIFO 的可用深度（自己决定，高溢出水线）进行比较得到几乎满。这就是范围较大的比较。

下面的参考代码注意时钟！

```
//============================================================//
//---turn the R addr to gray code to become w addr
always @(posedge read_clock or posedge fifo_gsr)
if(fifo_gsr)
  Rd_truegray <=  8'h0;
else
  Rd_truegray <=#1    {
                (read_addr[7]),(read_addr[7]^read_addr[6]),
                (read_addr[6]^read_addr[5]),(read_addr[5]^read_addr[4]),
                (read_addr[4]^read_addr[3]),(read_addr[3]^read_addr[2]),
                (read_addr[2]^read_addr[1]),(read_addr[1]^read_addr[0]) };

//---use w_clk to sync the r_addr----
always @(posedge write_clock or posedge fifo_gsr)
if(fifo_gsr)
  Rag_wt_syn <=  8'h0;
```

```
else
  Rag_wt_syn <=#1     Rd_truegray;
```

//稳妥起见，这里的 Rag_wt_syn 再在写时钟域里打 1 拍，消除亚稳态后，再由格雷码转换为二进制码。

```
//---turn the gray code to bin code----
wire        Ra_7_5  = Rag_wt_syn[7] ^ Rag_wt_syn[6] ^ Rag_wt_syn[5] ;
wire        Ra_7_4  =  Rag_wt_syn[7]  ^  Rag_wt_syn[6]  ^  Rag_wt_syn[5]  ^
Rag_wt_syn[4];
wire        Ra_3_1  = Rag_wt_syn[3] ^ Rag_wt_syn[2] ^ Rag_wt_syn[1]   ;
assign      Ra_write_syn[7]   =    Rag_wt_syn[7];                          //7
assign      Ra_write_syn[6]   =    Rag_wt_syn[7] ^ Rag_wt_syn[6];          //6

assign      Ra_write_syn[5]   =    Ra_7_5;                                 //5
assign      Ra_write_syn[4]   =    Ra_7_4 ;                                //4
assign      Ra_write_syn[3]   =    Ra_7_4 ^ Rag_wt_syn[3] ;                //3
assign      Ra_write_syn[2]   =    Ra_7_4 ^ Rag_wt_syn[3] ^ Rag_wt_syn[2];
                                                                           //2
assign      Ra_write_syn[1]   =    Ra_7_4 ^ Ra_3_1 ;                       //1
assign      Ra_write_syn[0]   =    Ra_7_5  ^  Ra_3_1  ^  Rag_wt_syn[4]  ^
Rag_wt_syn[0];                                                             //0

//--delay wt_addr to one_clk_latency---
always @(posedge write_clock or posedge fifo_gsr)
if(fifo_gsr)
    Wt_addr_p1      <=#1      0;
else
    Wt_addr_p1      <=#1      write_addr ;

//Fifo_status--means the number of valid data   in fifo
always @(posedge write_clock or posedge fifo_gsr)
if(fifo_gsr)
  Fifo_status   <=  8'h0;
else //if(!Full)
  Fifo_status   <=  Wt_addr_p1 - Ra_write_syn;

always @(posedge write_clock or posedge fifo_gsr)
if(fifo_gsr)
```

```
  Almostfull     <=#1     1'b0;
//-- when left 16 depth report almost_full
else if (Fifo_status[7:4] == 4'hF)
  Almostfull     <=#1     1'b1;
else
  Almostfull     <=#1     1'b0;
```

三：几乎空，与空类似。由读时钟产生。

四：几乎满相对于写时钟而言，同理几乎空相对读时钟。这样产生的精确度比较高。

如果几乎满和几乎空用同一个时钟，误差与两个时钟的具体大小相关。比如几乎满和几乎空都是用写时钟得到的，那么几乎满比较精确，如果写时钟比读时钟慢，则几乎空误差较大，如果写时钟比读时钟快，几乎空的误差也比较精确。

读写地址的产生：如果外部写使能（Write_enable）来了，同时 FIFO 的不满的话，写地址自增 1；如果读使能（Read_enable）有效，同时 FIFO 不空的话，读地址自减 1。在进行地址的递增时判断 FIFO 的空满标志是为了自我保护，避免读写地址交错，产生错误逻辑。

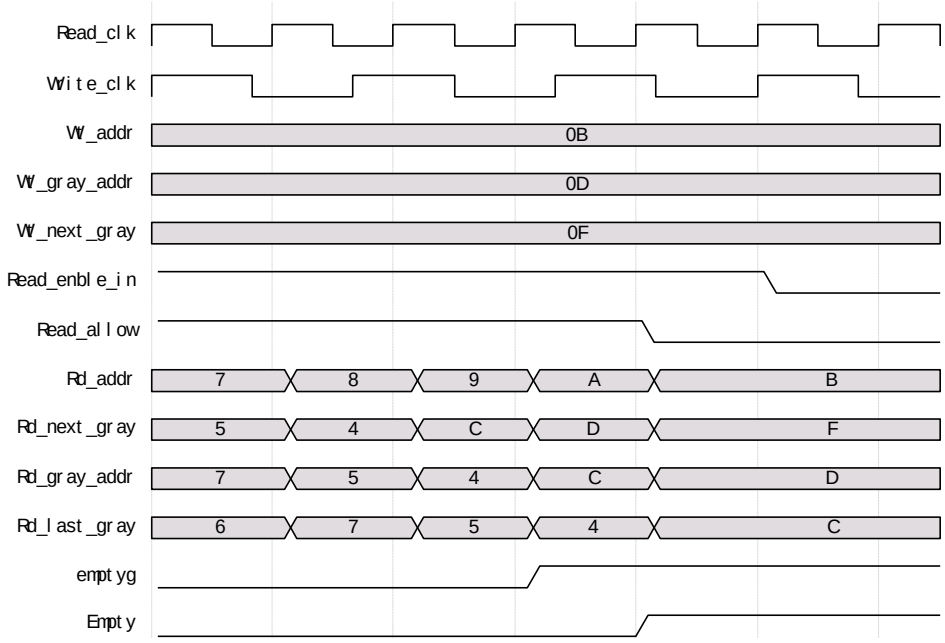


图2-168 FIFO 的自我保护机制

【位宽变换 FIFO】

实际应用中，需要进行数据位宽的变换，比如输入 1024×16bit 的数据，输出 256×64bit 的数据。或者颠倒过来。如图所示。

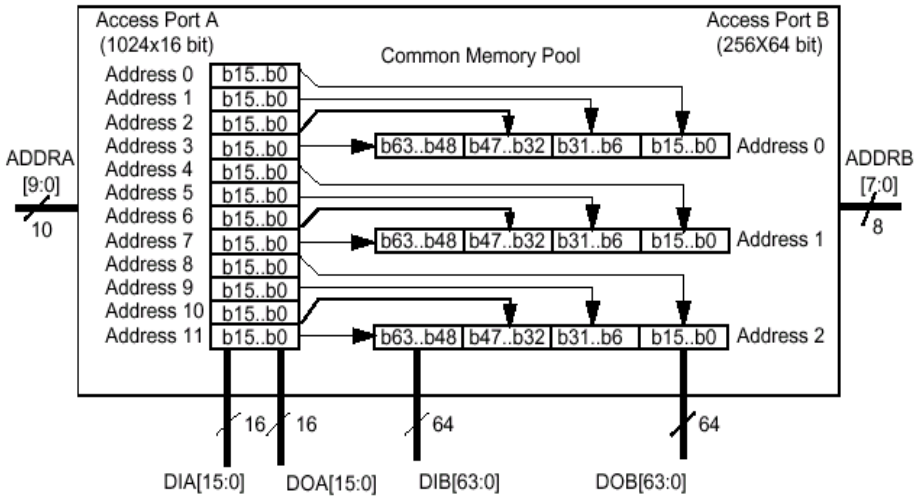


图2-169 FIFO 的位宽的转换

在我们的 FIFO 中使用到了双端口 RAM，它可以同时进行读和写操作。就以输入 1024×16bit 的数据，输出 256×64bit 数据的 FIFO 为例说明，其他情况一样。

显然我们需要一个 256×64bit 的双端口 RAM。读地址（Wt_addr[7:0]）比较好办，该地址就是 FIFO 的深度。但是写地址（Rd_addr[9:0]）就要分为两个部分，一个是与 FIFO 的深度对应的地址（Rd_addr[9:2]），另一个是位宽方向的地址（Rd_addr[1:0]）。因为 4 个写地址才对应一个读地址。比较写地址和读地址的高 8 位就可以产生空、满标志（与前面的 FIFO 类似）。

但在通信领域，经常与包打交道，包有包头，有包尾。当读到包尾时，不仅读地址的低 2 位要变，高 8 位也要变，要跳跃到下一个包的包头。而包尾在 4 个地址中的位置，决定了读地址的跳变方式。

【按块操作的 FIFO】

实际应用中，有执行突发性（burst）的连续读、写 N 个深度的 FIFO。也就是说对 FIFO 要按照块操作。说白了也就是分为高低地址。高地址为块号，低地址为块内的地址。FIFO 的空满标志由读写的高位地址比较产生。

【注意点】

在使用 FPGA 中，经常利用双端口 RAM 作为 FIFO 的存储部件。如果使用 Xilinx 公司的 Virtex、Virtex-e 系列器件的话，双端口 RAM 的具体布线可以使用 BlockRam 或者 Distributed Memory。如果 BlockRam 资源少的话，尽量使用 Distributed Memory（它由 slice 完成），一个 BlockRam 对应 4096（深度）×1（位宽）、2048×2、1024×4、512×8、256×16。注意位宽对 BlockRam 的估算：假如位宽 37bit，那么将要至少使用 3 个 BlockRam（16＋16＋5）。如果使用 Distributed Memory 的话，就可以节约 BlockRam。利用 Distribute Ram 做双端口 RAM 的限制为位宽在区间 1～64bit，深度 16～256。

对于异步大 FIFO 的处理最好先划分为一个低速的大同步 FIFO 和一个异步小 FIFO。其中小 FIFO 使用 Distributed Memory 完成双端口 RAM。

如果仅作为时钟的变换，异步 FIFO 的深度最好小些。

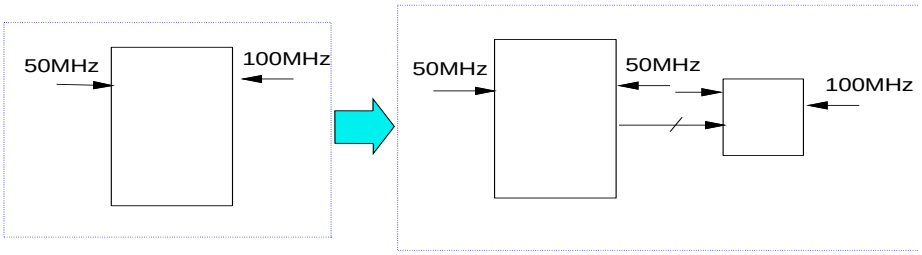


图2-170 FIFO 转换示意图

【FIFO 的稳定性】

FIFO 设计的时候，可以有空和满。但是为了稳定性考虑，最好不用出现 FIFO 满，尤其是异步 FIFO。因为如果时钟不稳定，漂移的情况下，很容易出现错位，造成溢出或覆盖未读的数据。解决办法是提高几乎满的标志，并且留有一定余量（也就是浪费一些 FIFO 空间以提高稳定性）。

空的话，根据实际情况决定。如果是字节级别的，可以设置上、下水线，也就是避免出现空，让 FIFO 内残余一些数据。如果是包级的，除了设置上下水线外，同时对 FIFO 内的包进行计数，如果 FIFO 内数据小于下水线，但 FIFO 内含有一个包尾的话，也必须读走才行。最坏情况是刚读出一个包尾，FIFO 也同时空了。

而且由于设置了溢出、上、下水线，避免了 FIFO 的连续频繁的读写以及 underflow 和 overflow 的现象。提高了 FIFO 的稳定性，此时 FIFO 带有一定的突发性（Burst）。

```
//---Start to read fifo's conditions------
//--rise when fcounter larger than UP_WATER or Cnt_pkt is large than 1-----
//--when fcounter less than UP_WATER ,if has ---
//--one eop in fifo,Drpa should valid ,else invalid----
//--if less than LOW_WATER and has no packet eop ,then drop down
always @(posedge Clk or posedge Reset)
if(Reset)
    Drpa   <=#4      1'b0;
else if(Cnt_pkt > 1 )
  Drpa   <=#4      1'b1;
else if(Fcounter >= `UP_WATER ) begin
    Drpa   <=#4      1'b1;
else if(Cnt_pkt == 1 ) begin
    Drpa   <=#4      1'b1;
```

```
else if(Fcounter <= `LOW_WATER ) begin
    Drpa   <=#4      1'b0;
End
```

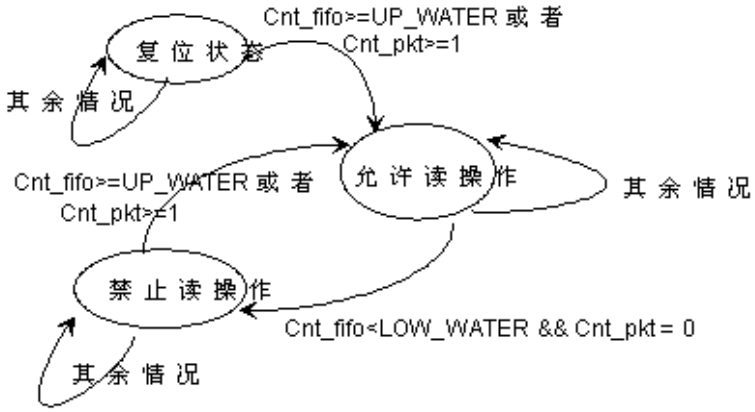


图2-171 有数据包信号 Drpa 的产生状态机

最佳状态是设计过程中不会出现空和满标志，这样控制逻辑简单而且速度可以达到很高。比如 PIMUX 的 1K×37 的异步大 FIFO。原来设计考虑了满和空标志，并且用它们制约读写地址的产生，结果极大的影响了 FPGA 的速度。后来仔细考虑下：该 FIFO 是基于包调度，当 FIFO 内有完整的包后（Pkt_valid 有效），才开始允许读。所以 FIFO 出现空的情况很少（当 FIFO 内只有一个包，并且刚好读完包尾的同时 FIFO 空了）。所以空标志的产生电路是冗余的（完全可由 Pkt_valid 信号取代），节约了大路的电路，并极大的提高的 FPGA 的实现速度（原来 Rd_allow<= Rd_en && !Empty，现在改为 Rd_allow<=Rd_en）。又由于设置几乎满来抑制满状态的出现，这样不仅提高系统的稳定性，又提高了速度，不过要浪费些资源而已。

FIFO 也会出现死机的现象，那就是又空又满！绝对在设计的时候要避免这种情况，宁可浪费一些资源。

如果直接要用 DPRAM（双端口 RAM）读出的数据作为条件（比如包尾标志），必须考虑在门级仿真时，由于 DPRAM 没有写入数据，那么多读出的数据为不定态，将导致与其相关的信号不定态，不定态连锁反应，最终有可能使仿真无法进行下去（尽管现实情况不会这样的）。避免这种情况的方法是加一个有效信号 Rd_valid。例如将原来的 If(Rdat[`EOP])　Sig <= 1　else Sig <=0 改为 if (Rdat[`EOP] && Rd_valid)　Sig<=1 else Sig<= 0 就解决了该问题。

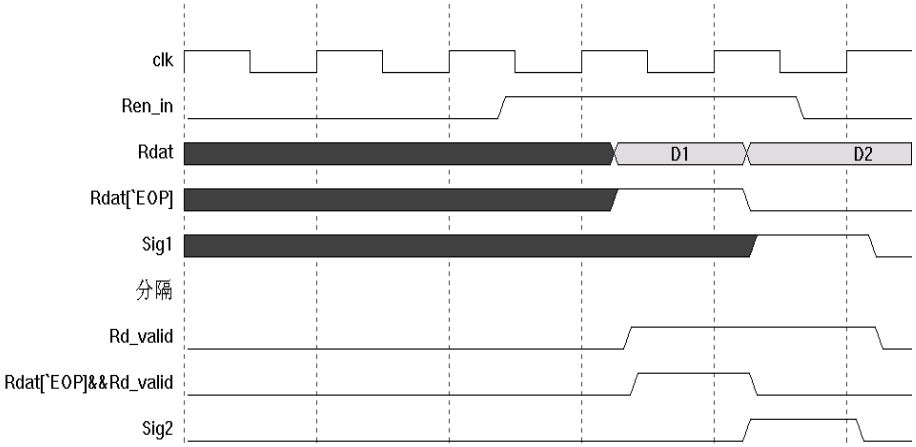


图2-172 不定态的传播和解决办法

【双端口 RAM】

在制作 FIFO 的时候，一定会用到双端口 RAM，针对 Xilinx 产品，它的物理实现有两种方式，一种是基于 BlockRam 的方式，另一种为 Distributed Ram 的方式。最大的区别是读出数据是否寄存。基于 BlockRam 的 DPram 是要寄存的，也就是读地址对应的数据在下个时钟出现。而 Distributed Ram 可以实现不寄存的方式，采用组合逻辑译码，地址直接产生数据，一一对应。可以参考下图。

下面是基于 BlockRam 的例化：

```
//dual RAM 1K * 37 bits
ram1k_37 URAM1K37 (
  .ADDRA(Rd_addr),
  .CLKA(Rd_clk),
  .ADDRB(Wt_addr),
  .CLKB(Wt_clk),
  .DIA({37'b0}),
  .WEA(gnd),
  .DIB(Wt_data),
  .WEB(Wt_allow),
  .ENA(Rd_allow),
  .ENB(pwr),
  .RSTA(gnd),
  .RSTB(gnd),
  .DOA(Rd_data_out),
  .DOB()
  );
```

下面为 Distributed Ram 的例化程序：

```
//---use distribute Ram--
ram_16_20 URAM_SEL_16_20(
    .A          (Wt_addr),
    .CLK        (Rf_clk),
    .D          (Wt_dat),
    .WE         (Wt_allow),
    .DPRA       (Rd_addr),
    .DPO        (R_read_data_in),
    .SPO        ()

    );
```

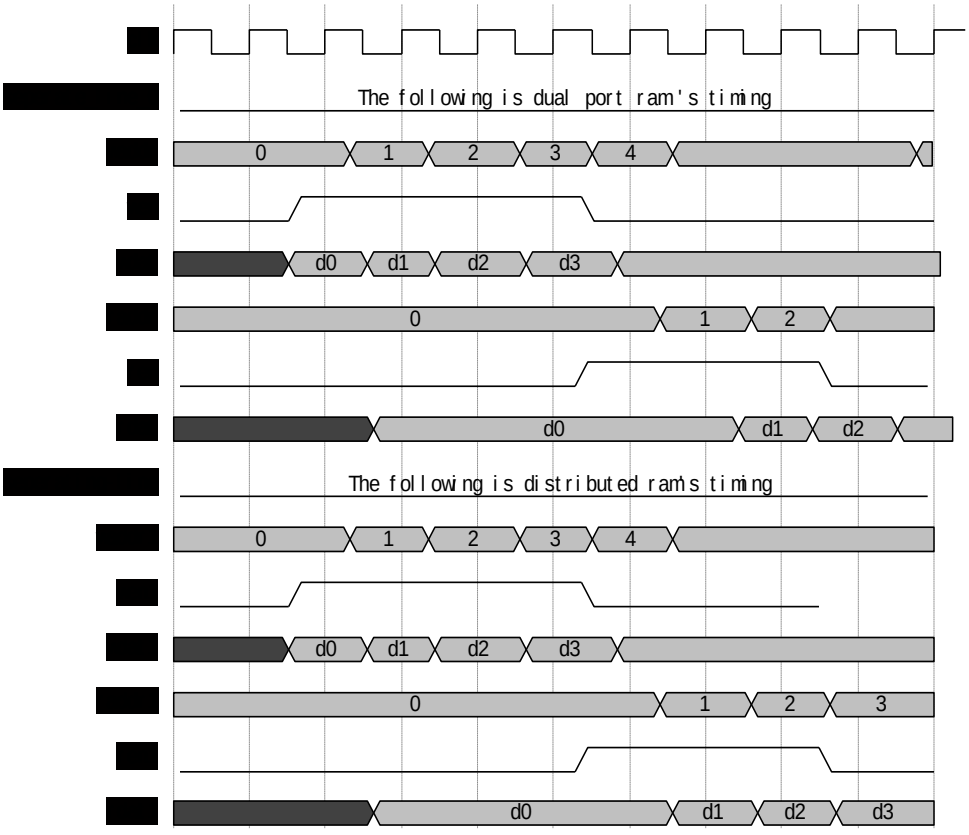


图2-173 双端口 BlockRAM 和分布式 Ram 的读写时序图

2.5.14 缓存管理

1. BM 典型电路简介

(1) BM 电路实现功能

在芯片的设计中，为了缓存大量的帧数据，通常会在芯片外挂 DDR SDRAM，在操作 DDR SDRAM 的过程中，DDR SDRAM 被分成若干个缓存块，用于存放帧数据。每一个缓存块都有一个起始地址，我们把它称作缓存基址。由于帧的处理流程需要反复读写 DDR SDRAM，所以需要有逻辑来管理这些空闲的缓存块基址，当需要写 DDR 时，就分配一个空闲的缓存基址，当 DDR 的这个缓存块使用完之后，其基址就应该被回收再使用。BM 就是完成缓存基址分配与回收的模块。

(2) BM 电路设计方法

由于在帧数据的处理时，可能存在将帧数据从 DDR 中读出若干次的现象，所以 BM 的设计中要支持带权重释放缓存地址。通常 BM 的设计中包括下面两个模块：wgt_ctrl 和 fba_ctrl。

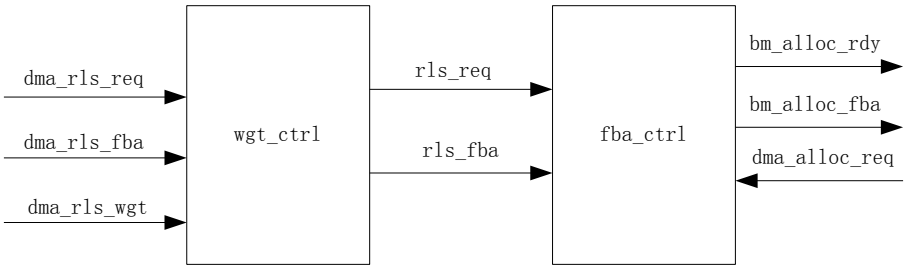


图2-174 BM 电路结构图

wgt_ctrl 模块对每个缓存地址预设一个权重值，每次 DMA 释放一次缓存地址，就将此缓存地址的权重值减去接收到的释放权重值，当权重值减到零时，认为此缓存地址已可以完全释放，发送释放请求到 fba_ctrl 模块。

fba_ctrl 电路设计通常有两种方法，一种方法是直接用 FIFO 来保存缓存地址，此种方法设计简单，但需要较大空间的 RAM 来实现；另外一种方法是用 BITMAP 的方式来实现，此种方法可以用较少的 RAM 资源来实现缓存地址的管理。下面以管理 4K 个缓存地址为例对 fba_ctrl 模块的两种设计方法进行详细的描述。

2. 设计方法一

(1) 设计原理

此种方法中，fba_ctrl 模块就是一个 FIFO，FIFO 宽度为 12bit，深度为 4K，在芯片的初始化时刻，就将 4K 个缓存地址写入到 FIFO 中，即 FIFO 中的内容依次为 0～4095。每次申请一个缓存地址，就从 FIFO 中读出一个地址；每次释放一个地址，就将释放的地址写入到 FIFO 中。图 2-174 中的 rls_req 信号就是 FIFO 的写使能信号，rls_fba 就是 FIFO 的写数据，bm_alloc_rdy 就是 FIFO 的空信号，bm_alloc_fba 就是 FIFO 的读数据，dma_alloc_req 就是 FIFO 的读使能信号。

(2) 设计实现

需要用双端口 RAM 来搭建 FIFO。

(3) 应用场景

此方案设计简单，主要应用在缓存地址不多的场合，如果缓存地址过多，则需要的 RAM 过大，成本过高。

3. 设计方法二

(1) 设计原理

整个缓存的管理主要由 fba_bitmap 和 fba_fifo 配合完成。缓存使用状态表 fba_bitmap 是一个 128X32bit 的 RAM，每个 bit 表示一个 DDR 存储块单元的使用情况：“1”表示尚未使用，“0”表示已经分配出去了。而 fba_fifo 是一个 128X7bit 的 FIFO，所存的数据是 DDR 块地址的高 7 位，同时该 7 比特数据也作为 bitmap 的寻址地址，读出该地址所对应的 32 比特的数据，将这 32bit 的数据由低位到高位的第一个为“1”的比特位进行译码就可以得到 5bit 的地位地址。

所以由 fifo 读出的高 7bit 地址，加上 bitmap 产生的低 5bit 地址，就是分配给 RXDMA 的 12bit 地址了。

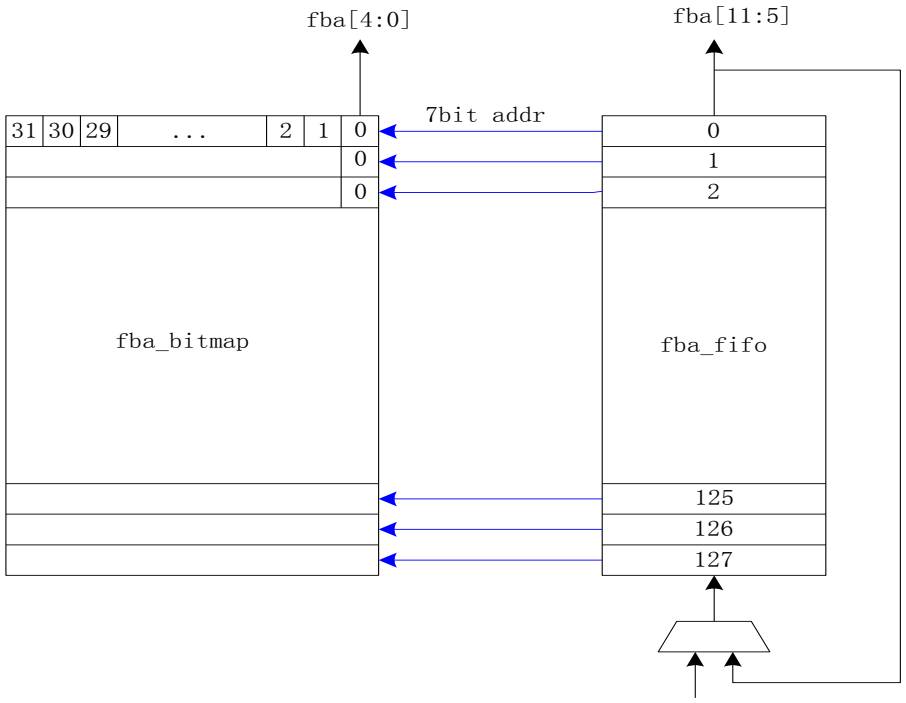


图2-175 fba_bitmap 和 fba_fifo 结构图

如图所示，初始化时，要根据 CPU 的控制来初始化 FIFO 的深度，当为 4K 时，FIFO 的 128 个存储单元依次写入从 0 到 127 的值，bitmap 的值则全为 1。正常情况下，每一次成功分配了地址以后，FIFO 的读出的数据重新返回写入 FIFO

尾部，但是若分配地址时，从 bitmap 读出的数据是第 31bit 为 1 时，在分配了地址的同时，FIFO 读出的数据丢弃，而不再入尾。

当 FBA_CTL 模块接收 WGT_CTL 的释放请求时，根据释放地址的高 7bit 作为地址，读出 bitmap 中相应的 32bit 数据，将第 5bit 对应的 bit 位置 1 后再写入 bitmap，如读出的 32bit 为全 0 的情况下，除了将相应 bit 置 1 写回，还要将释放地址的高 7 比特返回，写入 FIFO 的尾部。

(2) 应用场景

此种设计方案中，使用的 RAM 资源比较小，如方案一中，使用 RAM 资源为 48Kbit，而方案二中只需要 128×32bit＋128×7bit＝4992bit 的 RAM 资源。

2.5.15 8B/10B 编码

8B/10B 编码方案由 IBM 公司于 1983 年提出并开发，IBM 拥有该编码的专利权。这种编码方案被用于高速串行数据传输，IBM 公司在光纤信道和千兆以太网中使用该编码方式。主要功能是将 8bit 数据编码成 10bit 数据，这样产生了一定的数据冗余，可以使在发送数据时可以从 1024 个 10B 码组中挑选出 0、1 分布尽量平衡的码组来发送，以维持链路的直流平衡，同时在发送表示控制信息的特殊码组时可以挑选出数据码之外的码组。

1. 功能特性

很多串行标准（Infiniband、光纤通道、千兆以太网、ATM 等）的需求之一都是针对原始数据流采用 8B/10B 编码和解码。将 8bit 原始数据按照特定的规则编码为 10bit 数据流，然后进行传输，是因为 8B/10B 编码有如下重要特性：

(1) 能确保在接收器恢复时钟时有充足的信号转换。即转换密度能够得到保证，目的是保证在发送数据流中足够的信号转换，以保证 PLL 正常工作。如果没有 8B/10B 编码方法，数据中的一串 1 或 0 有可能引起接收时钟漂移或同步丢失，从而引起数据丢失。

(2) 编码空间允许将控制信号编码到数据流中。即特殊字符能编码到数据流中，8B/10B 编码采用 10 位字符，其数量是 8 位字符空间中字符数的 4 倍。这样，可以将特殊字符编码，放入数据流中，它们不会被解码成数据字符。这些特殊字符可表示分隔符或其他命令，比如在链接、维护、位/字节的校准、多通道抗偏斜、时钟补偿以及数据包限定等。

(3) 能够纠错。8B/10B 编码遵循特定规则，根据这些规则，很多信号传输错误都可以立即检测出来。在 10B 代码出错时，对代码映射指定选择功能，显著地提高了检测出单个和多个比特地概率。

(4) 某些编码（用于控制信号）包含一个唯一的、易识别的代码位模式（类似于句子中的逗号），它有助于快速同步和接收器位对齐。

直流补偿（DC Banlance）。8B/10B 还保证对信号进行 DC 补偿，这意味着，链路中不会随着时间推移而出现 DC 偏移。因为 RD（running disparity，行程偏差），在对 8B 码组编码时，每对一个码组编码后都要生成一个当前 RD（值为 0 或 1 两种情况），判断前面的码组的 0、1 分布情况，如 1 比 0 多或者在 0、1 个数相等的情况下分布不均匀，在下一次编码时会根据这个 RD 的值进行调整，从而达到链路的动态平衡。

2. 8B/10B 的符号和术语

每个传送的字节（8B）有 256 种可能的数据值，而代码字（10B）理论上具有 1024 种可能的数据值。为简化对它们的处理，开发了一种 8B/10B 编码的简写符号，如图 2-176 所示，$D_{X.Y}$ 和 $K_{X.Y}$。

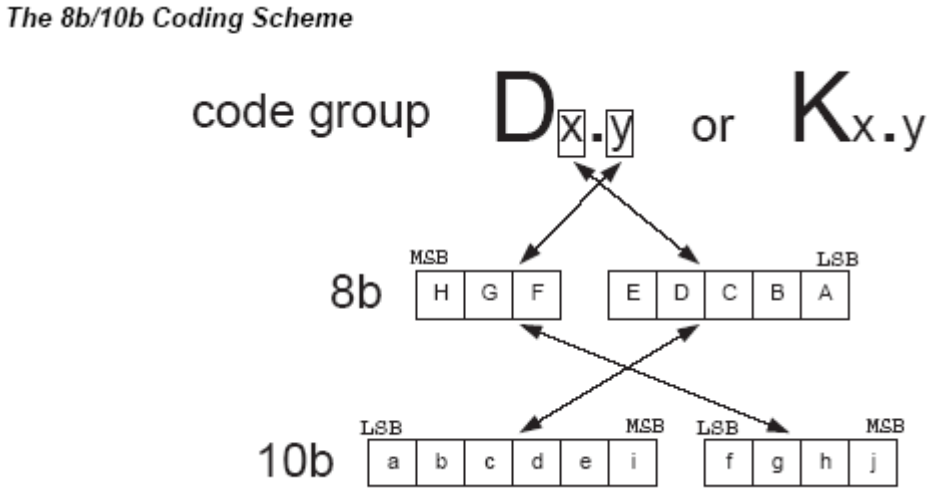


图2-176 8B/10B 编码

(1) D 分组，用于数据字节编码。即图 1 中的 DX.Y，X 为 8b 的低 5 位的值，Y 为 8b 的高 3 位的值，如图 2-177 所示，A 表示最低位，H 表示最高位。

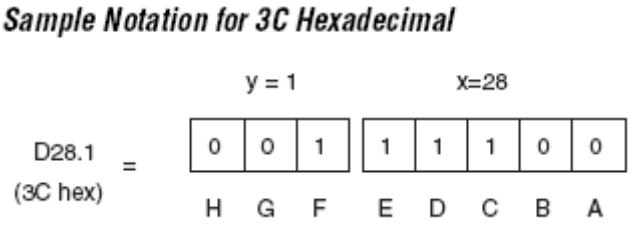


图2-177 D 分组说明

(2) K 分组，用于特殊控制符号编码。即图 1 中的 KX.Y，X 为 8b 的低 5 位的值，Y 为 8b 的高 3 位的值。

(3) RD（running disparity，行程偏差）

(4) DC Banlance（直流平衡）

3. 8B/10B 编码

如图 1 中所示，8 位的数据字节用 A,B,C,D,E,F,G,H 表示（从最低位到最高位），8B 分成了两个子分组，一组包括 3 位，即 3-bit（H,G,F），另一组包括 5 位，即 5-bit（E,D,C,B,A）。编码器将这 8 位数据译成 10 位代码，用 a,b,c,d,e,I,f,g,h,j 表示。10B 代码也分成了两个子分组，一组包括 4 位，即 4-bit（f,g,h,j），另一组包括 6 位，即 6-bit（a,b,c,d,e,i）。并且 8B 中的 3-bit（H,G,F）与 10B 中的 4-bit（j.h.g.f）对应，8B 中的 5-bit（E,D,C,B,A）与 10B 中的 6-bit（I,e,d,c,b,a）对应。编码后的 10B 代码字按照 abcdeifghj 的顺序连续传送。

(1) 3-bit/4-bit 以及 5-bit/6-bit 编码

为了编出 DC 平衡的 10B 码流，则需要尽量平衡 0 和 1 的个数。如果一个 10B 码流中 1 的个数与 0 的个数相等，那么我们称其为均衡的。

如果 4-bit 和 6-bit 两个子分组都是均衡的，那么这两个子分组构成的 10B 代码字也是均衡的。如果 10B 码都是均衡的，那么就是完美的 DC 平衡编码。但是，这是不可能的，因为 4-bit 子分组有 16 种值其中只有 6 种是均衡的，而 4-bit 对应 8B 中的 3-bit，3-bit 子分组有 8 种值，显然，4-bit 子分组中仅仅 6 种均衡的代码不足以表示 3-bit 子分组中的 8 种值。同样，10B 的 6-bit 子分组中所有值种只有 20 种均衡的，也不足以表示 5-bit 子分组中的 32 种值。所以我们只能找 1 的个数与 0 的个数相差最小的值来表示余下来的值，因为 4-bit 和 6-bit 都是偶数位， 1 的个数减去 0 的个数的差不可能是＋1 或者－1，最小是＋2 或者－2，所以在 8B/10B 的编码方案中也需要使用偏差为＋2 或－2 的代码。

表 2-31 列出了 3-bit 转换为 4-bit 的对应关系，表 2 表 2-32 则列出了 5bit 转换为 6-bit 的对应关系，将 4-bit 与 6-bit 串连起来就构成了 10B 的编码值。大家注意到在表 2-31 表 2-32 中有些编码值有两种，一个是偏差为＋2（即 1 的个数减去 0 的个数）的值，另一个是偏差为－2（即 1 的个数减去 0 的个数）的值。8B/10B 的编码规则要求将 4-bit 与 6-bit 结合起来的时候，10B 代码的偏差最大为＋2（即 1 的个数减去 0 的个数）或者－2（即 1 的个数减去 0 的个数）。所以 4-bit 偏差为＋2 的值不能跟 6-bit 偏差为＋2 的值结合，因为这样结合出来的 10B 的偏差就是＋4，不符合要求。

表2-31 3B 到 4B 的编码

Table 1. 3-Bit to 4-Bit Encoding Values

3b Decimal	3b Binary (HGF)	4b Binary (fghi)
0	000	0100 or 1011
1	001	1001
2	010	0101
3	011	0011 or 1100
4	100	0010 or 1101
5	101	1010
6	110	0110
7	111	0001 or 1110 or 1000 or 0111

表2-32 表 2 5B 到 6B 的编码

Table 2. 5-Bit to 6-Bit Encoding Values

5b Decimal	5b Binary (EDCBA)	6b Binary (abcdei)
0	00000	100111 or 011000
1	00001	011101 or 100010
2	00010	101101 or 010010
3	00011	110001
4	00100	110101 or 001010
5	00101	101001
6	00110	011001
7	00111	111000 or 000111
8	01000	111001 or 000110
9	01001	100101
10	01010	010101
11	01011	110100
12	01100	001101
13	01101	101100
14	01110	011100
15	01111	010111 or 101000
16	10000	011011 or 100100
17	10001	100011
18	10010	010011
19	10011	110010
20	10100	001011
21	10101	101010
22	10110	011010
23	10111	111010 or 000101
24	11000	110011 or 001100
25	11001	100110
26	11010	010110
27	11011	110110 or 001001
28	11100	001110
29	11101	101110 or 010001
30	11110	011110 or 100001
31	11111	101011 or 010100

(2) 行程偏差

因为 10B 编码中最大的偏差值是＋2 或者－2，所以发送出去的串行数据流中仍然有可能会有偏差＋2 或者－2 的偏差，这样数据流依然不能保证直流平衡（DC-Balance）。为了维持数据流的直流平衡（DC-Balance），每个 8B 数据都对应着两种可能的值，如表 2-33 所示，两种值分别对应 RD-与 RD+两列。RD-这一列中的值偏差是＋2 或者 0，RD＋这一列中的值偏差则是-2 或者 0，编码器根据当前 RD 的值来决定选择 RD-列中的值还是 RD＋列中的值。

假设开始发送的时候，RD 默认为 RD-，那么当对 8B 数据编码的时候，编译器将用 RD-这一列的值。如果取出来的 10B 代码是均衡的，那么 RD 保持 RD-不

变，下一个字节对应的还是取 RD-这一列的值。否则 RD 变为 RD+，即下一个字节对应的应该取 RD+这一列的值，如果取出来的 10B 代码是均衡的，那么 RD 保持 RD+不变，下一个字节对应的还是取 RD+这一列的值。否则 RD 将从 RD+变回 RD-，即 RD-这一列的值将被使用。以下图 2-178 是行程偏差计算的状态图。

表2-33 8B/10B 编解码映射表

Table 3. Portion of the 8b/10b Encoding/Decoding Mapping Table

Code Group	kin/ kout	8-bit data HGF EDCBA	10-bit data (RD-) abcdei fghj	10-bit data (RD+) abcdei fghj	Code Group	kin/ kout	8-bit data HGF EDCBA	10-bit data (RD-) abcdei fghj	10-bit data (RD+) abcdei fghj
D0.0	0	000 00000	100111 0100	011000 1011	D0.1	0	001 00000	100111 1001	011000 1001
D1.0	0	000 00001	011101 0100	100010 1011	D1.1	0	001 00001	011101 1001	100010 1001
D2.0	0	000 00010	101101 0100	010010 1011	D2.1	0	001 00010	101101 1001	010010 1001
D3.0	0	000 00011	110001 1011	110001 0100	D3.1	0	001 00011	110001 1001	110001 1001
:					:				
D31.0	0	000 11111	101011 0100	010100 1011	D31.1	0	001 11111	101011 1001	010100 1001
D0.2	0	010 00000	100111 0101	011000 0101	D0.3	0	011 00000	100111 0011	011000 1100
D1.2	0	010 00001	011101 0101	100010 0101	D1.3	0	011 00001	011101 0011	100010 1100
D2.2	0	010 00010	101101 0101	010010 0101	D2.3	0	011 00010	101101 0011	010010 1100
D3.2	0	010 00011	110001 0101	110001 0101	D3.3	0	011 00011	110001 1100	110001 0011
:					:				
D31.2	0	010 11111	101011 0101	010100 0101	D31.3	0	011 11111	101011 0011	010100 1100
D0.4	0	100 00000	100111 0010	011000 1101	D0.5	0	101 00000	100111 1010	011000 1010
D1.4	0	100 00001	011101 0010	100010 1101	D1.5	0	101 00001	011101 1010	100010 1010
D2.4	0	100 00010	101101 0010	010010 1101	D2.5	0	101 00010	101101 1010	010010 1010
D3.4	0	100 00011	110001 1101	110001 0010	D3.5	0	101 00011	110001 1010	110001 1010
:					:				
D31.4	0	100 11111	101011 0010	010100 1101	D31.5	0	101 11111	101011 1010	010100 1010
D0.6	0	110 00000	100111 0110	011000 0110	D0.7	0	111 00000	100111 0001	011000 1110
D1.6	0	110 00001	011101 0110	100010 0110	D1.7	0	111 00001	011101 0001	100010 1110
D2.6	0	110 00010	101101 0110	010010 0110	D2.7	0	111 00010	101101 0001	010010 1110
D3.6	0	110 00011	110001 0110	110001 0110	D3.7	0	111 00011	110001 1110	110001 0001
:					:				
D31.6	0	110 11111	101011 0110	010100 0110	D31.7	0	111 11111	101011 0001	010100 1110
K28.0	1	000 11100	001111 0100	110000 1011					
K28.1	1	001 11100	001111 1001	110000 0110					
K28.2	1	010 11100	001111 0101	110000 1010					
K28.3	1	011 11100	001111 0011	110000 1100					
K28.4	1	100 11100	001111 0010	110000 1101					
K28.5	1	101 11100	001111 1010	110000 0101					
K28.6	1	110 11100	001111 0110	110000 1001					
K28.7	1	111 11100	001111 1000	110000 0111					
K23.7	1	111 10111	111010 1000	000101 0111					
K27.7	1	111 11011	110110 1000	001001 0111					
K29.7	1	111 11101	101110 1000	010001 0111					
K30.7	1	111 11110	011110 1000	100001 0111					

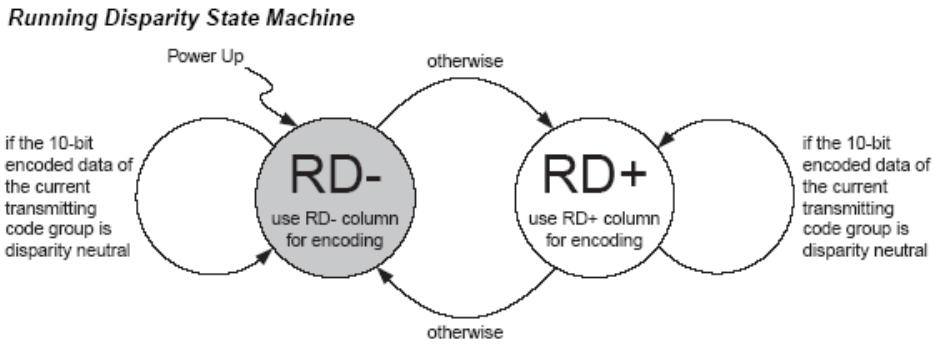


图2-178 行程偏差状态机

(3) 8B/10B 编码

由前面几节的介绍，我们知道为确保 10B 编码信号在不失真、并且不使用大带宽线路编码的情况下，能正确地交流耦合到介质上（例如，通过变压器或电容器），长时间数据传送地代码流中 0 和 1 的数量应该相等。同时，连续的 0 或者 1 的最大数量应尽可能最小（虽然长期的平均数量可能相等）以避免短时间的直流偏差。这称为最小化代码行程长度（run length）。

因此，代码字是从 1024 种可能的编码分组种仔细选出的。用于数据的代码字不能有以下情况：

产生对于 4 个连续的 1 或 0

代码字（一个 10B）种 1 的个数与 0 的个数的差不超过 2（不平衡度不超过 2），即 10B 编码总是由 5 个 1、5 个 0 或者 6 个 1、4 个 0 或者 4 个 1、6 个 0 组成。

那些连续 0 或者 1 过多的代码和那些不平衡度过高的代码一般不用于数据编码（有些用于特殊的编码）。这是使用大代码空间的一个优点：对于 256 种可能的数据值有 1024 种可用的代码，因此你可以把那些不符合要求的代码“扔到一边去”。

除了只选择最平衡的代码外，为每个 8B 数据定义了两个 10B 编码。如果为给定值所选的 10B 编码的 0 和 1 数量相等（各 5 个），那么这两个 10B 编码可能相同。这就是一个完美的平衡代码点，无须补偿。表 3 种的 D3.1 的编码就是平衡编码点的例子。

如果 10B 编码不均衡，即 1 比 0 多或者 0 比 1 多，那么第二种编码在 10B 的子分组 6B（a,b,c,d,e,i）或者 4B（fghj）中使用其反码，或者两个子分组 6B/4B 同时取反码。因此 D23.2 有两个不同的有效编码。

4. 8B/10B 应用

如图 2-179 所示，假设最初 RD 的值为 RD-，所以 D0.0 选择 RD-对应的值，查表 2-33 表 3 得 100111 0100，因为该值 1 的个数等于 0 的个数，即为均衡的，所以 RD 保持 RD-不变；紧接着下一个数据 D31.5 依然取 RD-那一列对应的值，查表 2-33 得 101011 1010，因为 1 的个数不等于 0 的个数，即不均衡，所以 RD 由 RD-变为 RD+；紧接着的数据 D0.0 取 RD+那一列对应的值，查表 2-33 得 01100 1011，因为该值 1 的个数等于 0 的个数，即为均衡的，所以 RD 保持 RD+不变；紧接着的命令 K28.5 取 RD+那一列对应的值，查表 2-33 得 110000 0101，K28.5 为分界符。

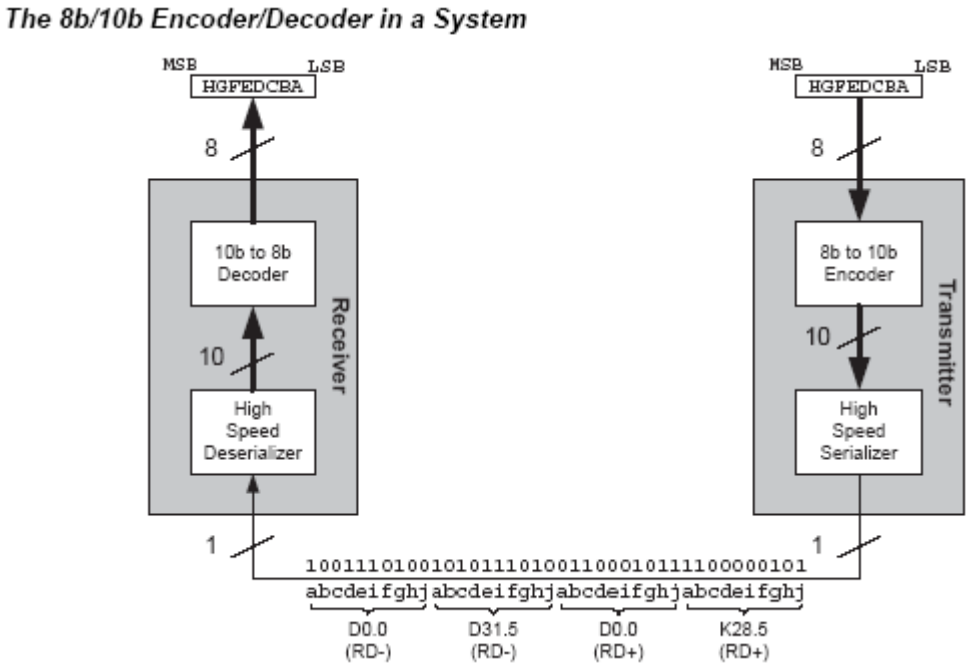


图2-179 8B/10B 编解码在某系统中的应用

2.5.16 64B/66B 编码

1. 编码简介

64B/66B 编码是 IEEE802.3 标准推荐的 10G/40G/100G 通信的标准编码方式。它并不是真正的编码，而是一种基于扰码机制编解码方式。这种编码方式，是通过发送扰码（有点像扩频通信的原理）来使信号的 0 和 1 的数量达到平衡（白噪声）。

2. 编码原理

这里主要介绍 40G、100G 的 64B/66B 编码（10GE 的编码组合略多几个）。

64B/66B 编码中，首先，数据被分成 8 个字节一组（总共 64 比特），每一个字节可以是数据字符或者控制字符。控制字符有/I/、/S/、/T/、/E/、/Q/、/Fsig/。各个控制字符的含义如下表所示：

Control character	Notation	XLGMII/ CGMII control code	40/100GBASE-R O code	40GBASE-R and 100GBASE-R control code
idle	/I/	0x07		0x00
start	/S/	0xFB		Encoded by block type field
terminate	/T/	0xFD		Encoded by block type field
error	/E/	0xFE		0x1E
Sequence ordered_set	/Q/	0x9C	0x0	Encoded by block type 0x4B plus O code, control codes are set to 0x00
Signal ordered_set[a]	/Fsig/	0x5C	0xF	Encoded by block type 0x4B plus O code, control codes are set to 0x00

每 8 个字符编码成 66bit 数据（称之为一个 block），其中前两比特是同步头(sync header)。引入同步头的目的是使接收侧能识别出每一个 66bit 数据的边界。合法的同步头是 2'b01 和 2'b10。

如果 8 个字符全部都是数据字符 ， 那么编码成 66bit 时， 同步头为 2'b01。

如果 8 个字符至少有一个是控制字符 ，那么编码成 66bit 时，同步头为 2'b10。

Input Data	Sync	Block Payload							
Bit Position: / Data Block Format:	0 1	2							65
$D_0\ D_1\ D_2\ D_3\ D_4\ D_5\ D_6\ D_7$	01	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
Control Block Formats:		Block Type Field							
$C_0\ C_1\ C_2\ C_3\ C_4\ C_5\ C_6\ C_7$	10	0x1E	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7
$S_0\ D_1\ D_2\ D_3\ D_4\ D_5\ D_6\ D_7$	10	0x78	D_1	D_2	D_3	D_4	D_5	D_6	D_7
$O_0\ D_1\ D_2\ D_3\ Z_4\ Z_5\ Z_6\ Z_7$	10	0x4B	D_1	D_2	D_3	O_0	0x000_0000		
$T_0\ C_1\ C_2\ C_3\ C_4\ C_5\ C_6\ C_7$	10	0x87		C_1	C_2	C_3	C_4	C_5	C_6	C_7
$D_0\ T_1\ C_2\ C_3\ C_4\ C_5\ C_6\ C_7$	10	0x99	D_0		C_2	C_3	C_4	C_5	C_6	C_7
$D_0\ D_1\ T_2\ C_3\ C_4\ C_5\ C_6\ C_7$	10	0xAA	D_0	D_1		C_3	C_4	C_5	C_6	C_7
$D_0\ D_1\ D_2\ T_3\ C_4\ C_5\ C_6\ C_7$	10	0xB4	D_0	D_1	D_2		C_4	C_5	C_6	C_7
$D_0\ D_1\ D_2\ D_3\ T_4\ C_5\ C_6\ C_7$	10	0xCC	D_0	D_1	D_2	D_3		C_5	C_6	C_7
$D_0\ D_1\ D_2\ D_3\ D_4\ T_5\ C_6\ C_7$	10	0xD2	D_0	D_1	D_2	D_3	D_4		C_6	C_7
$D_0\ D_1\ D_2\ D_3\ D_4\ D_5\ T_6\ C_7$	10	0xE1	D_0	D_1	D_2	D_3	D_4	D_5		C_7
$D_0\ D_1\ D_2\ D_3\ D_4\ D_5\ D_6\ T_7$	10	0xFF	D_0	D_1	D_2	D_3	D_4	D_5	D_6	

上面这幅图就是 64B/66B 编码表，表中 D_0~D_7 表示数据字符；C_0~C_7 表示/I/或者/E/控制字符；S_0 和 S_4 表示/S/控制字符；T_0~T_7 表示/T/控制字符。O_0 表示/Q/控制字符。 每个字母后面跟的下缀数字表示这个字符在 8 个字符中的字节位置。

图的 Input Data 列为被编码的 8 个字符，sync 列为同步头，BlockPayload 列为编码后的净荷。

64B/66B 编码对待编码的 8 个字符有组合限制，比如：

- /S/控制字符必须在 8 个字节的第 0 个位置；
- /O/控制字符必须在 8 个字节的第 0 个位置；
- /T/字符之前的字符必须是数据字符，之后的必须为控制字符；
- /S/控制字符和/T/控制字符不能在一个 8 字节组里。

对于编码的结果里，有一个字节的 block type field，用来区分不同的控制 block。block type field 在数值的选取上，采用了汉明距为 4 的结果，可以识别出最多两比特错误。当然最主要的还是避免一种 block type field 由于出错变为另外一种有效的 block type field。block type field 同时也间接地对/S/和/T/进行了编码。

对于待编码的数据字符是原封不动放进对应的 66 比特 block 里，对于/S/和/T/以外的控制字符，在编码时还需要作进一步转换。如下，/I/被转为 7-bit 0x00，/E/被转为 7-bit 0x1E，/Q/被转为 4-bit 0x0。

Control character	Notation	100GBASE-R PCS control code
idle	/I/	7-bit 0x00
error	/E/	7-bit 0x1E
ordered_set (Sequence)	/Q/	4-bit 0x0

在编码过程中，遇到不满足输入组合的情况，8 个字符统一编为 8 个错误控制字符/E/，block type field 是 0x1E。

编码结果中，条形空白矩形代表的比特在编码时应该添存 0，解码时忽略。

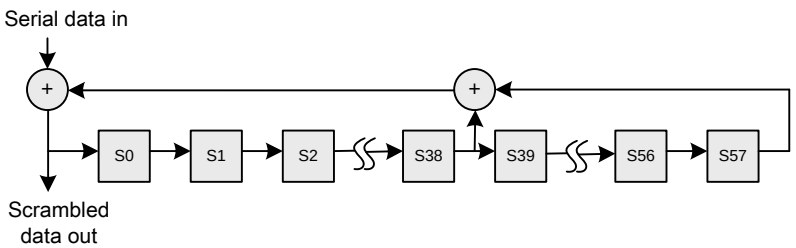
3. 加扰处理

前面讲的编码处理，对数据字符没有任何处理，会把长连 0 或者 1 的数据发送到链路上，这样带来两个问题，一是接收侧 Serdes 时钟恢复会困难，采样不准数据比特，另一方面直流偏置会拉很大。IEEE 对此增加了加扰处理环节。

加扰多项式如下：

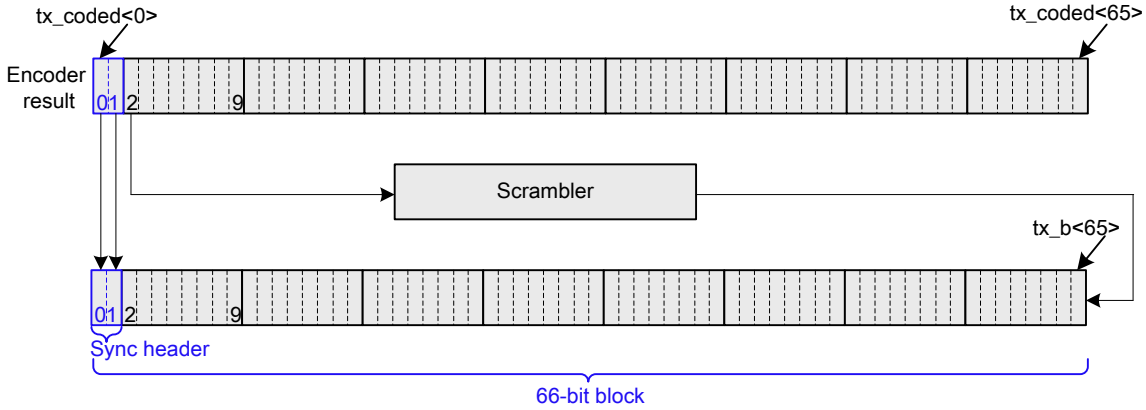
$$G(x) = 1 + x^{39} + x^{58}$$

对应的加扰电路如下（串行加扰）：



加扰时，每个 block 的同步头不参与加扰。此加扰为自加扰，因此加扰电路的初始值是不关心的。

加扰还要关注的一个主题是比特序，对于每一个 block，低比特先进入加扰器。可见下图：



4. 编码特点

优点：

1) 编码开销小：8B/10B 编码的开销约为 20%，而 64B/66B 编码的开销约为 3%；

2) 码字长。

缺点：

1) 对齐时间长：通过数据流中的有效同步位进行数据块对齐，正确匹配后才进入 Lock 状态完成定界，然后通过不间断的识别这两位，维持锁定。

2) DC 不平衡：由于电口传输有一个相关时间常数，在下一位发送之前，高速接口经常不允许全电压摆幅。因此，‘1’或者‘0’数量的连续不均衡会导致差分对眼图中心电位的偏移，会导致接收端电路设计的复杂、增加误码率。因此需要加扰器和解扰器配合。

2.5.17 IBM 的 memory 使用简介

我们常用两种类型的 memory：RA 和 SRAM。下面以 IBM 的 RA 和 SRAM 为例介绍典型的时序。

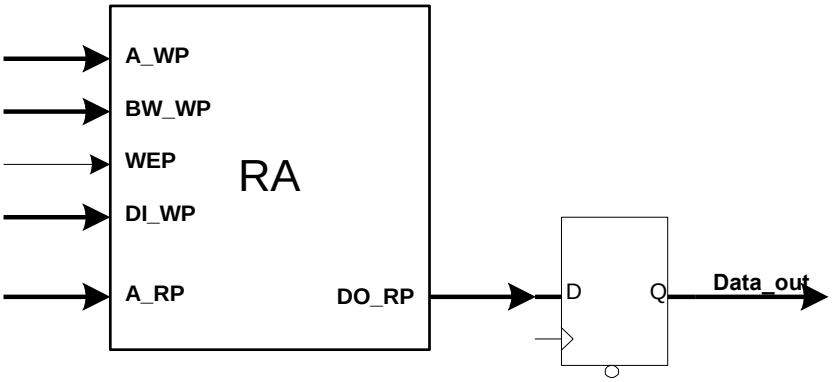
1. RA（Register Array）

(1) RA 的应用范围

- memory 总容量小于 10Kb
- 数据位宽在 2～82bit 之间
- memory 深度在 8～256 之间

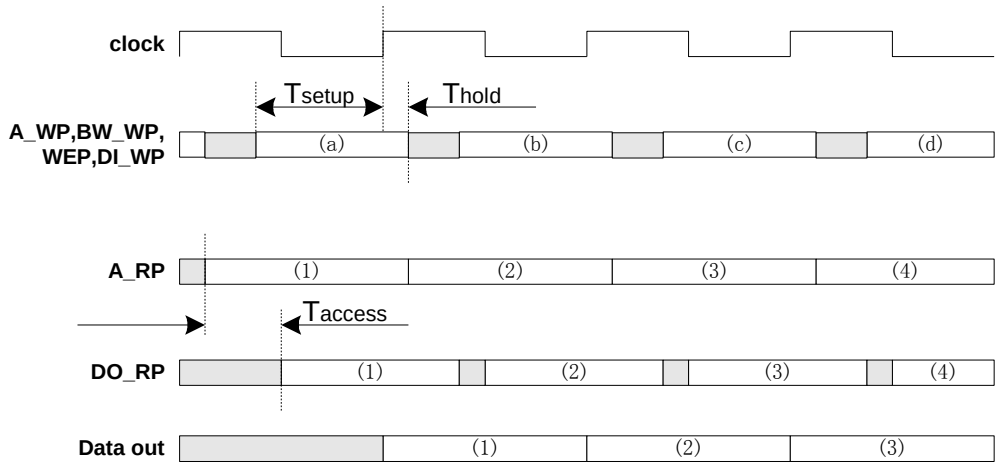
- 支持读时序和写时序之间同步或异步

(2) RA 的接口信号



信号名称	类型	说明
clock	I	写时钟
A_WP	I	写地址总线
BW_WP	I	写数据的位控制。 决定写 1 个 word 的 bit 范围
WEP	I	写使能
DI_WP	I	写数据总线
A_RP	I	读地址总线
DO_RP	O	读数据总线

(3) RA 接口信号的时序



写操作相关的输入信号都属于写时钟域，必须满足时钟的建立时间（Tsetup）和保持时间（Thold）的要求。在写时钟的有效边沿，数据被写入 RA。

读地址信号 A_RP 可以在写时钟域，也可能在另一时钟域，RA 的数据输出和 A_RP 有确定的最大延迟（Taccess）。可以认为 A_RP 在 RA 内部接入一个组合逻辑电路，把 RA 所有 word 选通输出。

(4) 设计建议

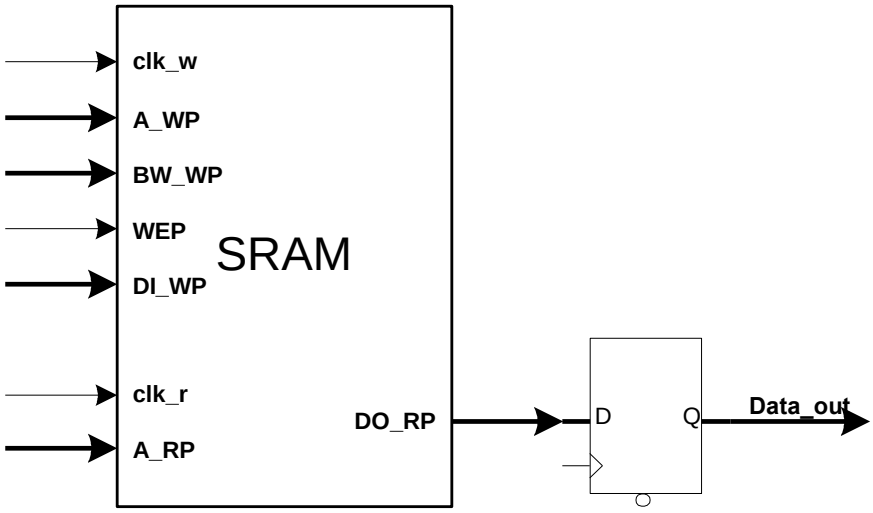
为保证较好的时序，RA 的输出需使用触发器寄存 1 级再引用。

2. SRAM

(1) SRAM 使用范围

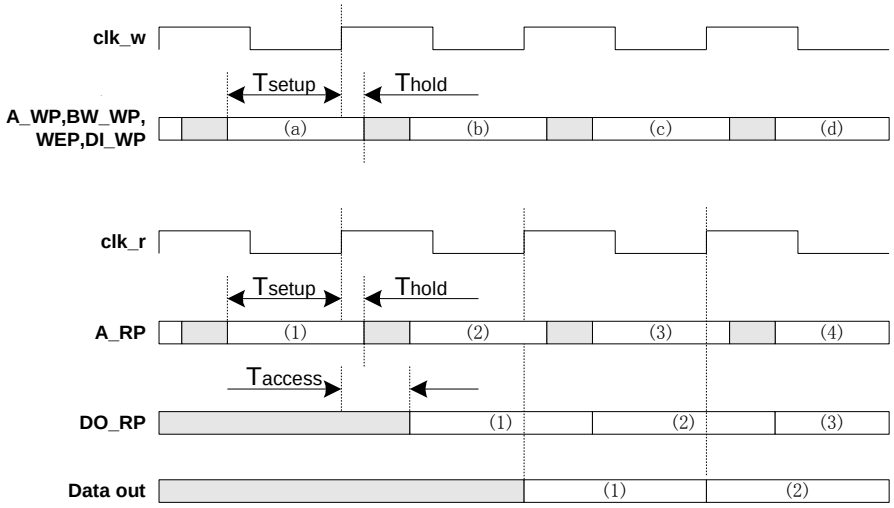
RA 无法有效使用的场合

(2) SRAM 接口信号



信号名称	类型	说明
clk_w	I	写时钟
A_WP	I	写地址总线
BW_WP	I	写数据的位控制。 决定写 1 个 word 的 bit 范围
WEP	I	写使能
DI_WP	I	写数据总线
clk_r	I	读时钟
A_RP	I	读地址总线
DO_RP	O	读数据总线

(3) SRAM 接口时序



SRAM 的写时序和 RA 的写时序基本一致。

SRAM 的读时序和 RA 不同，读地址信号在 SRAM 内部寄存后选通一个 word 输出在 DO_RP。

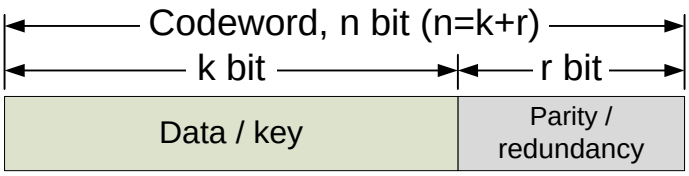
3. 设计建议

为保证较好的时序，RA 的输出需使用触发器寄存 1 级再引用。（注意：一般 Vendor 提供 SRAM 的 Wrapper，具备了输出寄存的功能）。

2.5.18 ECC（Error Correction Code，纠错码）

ECC 是纠错码，有别于检错码只能发现错误，ECC 还能在一定范围内纠正错误，付出的代价就是：ECC 比检错码需要更多的校验位，或者说开销更大。当然 ECC 也能当检错码用，或者既能纠错同时也能检错。

广义的 ECC 包括各种纠错编码，纠错能力也差别很大，我们通常说的 ECC 是狭义的、只纠 1bit 错的(n, k)分组码，其码字（codeword）结构如下图所示：（为方便使用，实际中一般都是把 Data 放在低位，把 Parity 放在高位）



ECC 的基本原理是汉明码或者扩展汉明码（汉明码再加 1bit“覆盖全部有效数据和汉明码校验位”的奇偶校验位），规格和对比如下：（因为扩展汉明码用的更多，也有人就把扩展汉明码叫汉明码）

	最小码距	做纠错码	单纯做检错码

汉明码	3	SEC	DED
扩展汉明码	4	SEC＋DED	能检 3bit 错

（SEC＝Single-bit Error Correct，单比特错纠正；

DED＝Double-bit Error Detect，双比特错检测）

扩展汉明码的纠错能力是 SEC＋DED（或者说 SEC & DED），具体描述如下：

- 如果出现 1bit 错，可以检测并纠正（SEC）；
- 如果出现 2bit 错，也能检测但无法纠正（DED）；
- 如果出现更多 bit 错，无法检测，可能会当作 1bit 错进行错误的纠正；

对一定位宽的有效数据，汉明码/扩展汉明码需要的校验位宽如下表：

数据位宽 k	汉明码（$2^r > n=k+r$）		扩展汉明码（$2^{r-1} \geq n=k+r$）	
	校验位宽 r	分组位宽 n	校验位宽 r	分组位宽 n
2～4	3	5～7	4	6～8
5～11	4	9～15	5	10～16
12～26	5	17～31	6	18～32
27～57	6	33～63	7	34～64
58～120	7	65～127	8	66～128
121～247	8	129～255	9	130～256
248～502	9	257～511	10	258～512

具体电路设计中，为改善 ECC check/correct 的时序和性能，实际采用的是“Optimal Minimal Odd Weight Column Code，最优奇重列码”，其原理类似扩展汉明码，也是 SEC＋DED，所需的校验位宽也相同。以 k＝11、r＝5、n＝16 为例，其 ECC RTL 代码如下：

```
function [4:0] mow_gen_11p5; //Min-Odd-Weight Column Code, data(k) = 5~11bit, parity(r) = 5bit, total(n) = 10~16bit
    input [10:0]  k; //k = n - r
    //     k[10 9 8 7 6 5 4 3 2 1 0]      r[4 3 2 1 0]
    // r[4] = 1 0 1 1 0 1 1 1 0 1 0         1 0 0 0 0
    // r[3] = 1 1 1 0 1 0 1 0 1 1 0         0 1 0 0 0
    // r[2] = 1 1 0 1 0 1 1 0 1 0 1         0 0 1 0 0
    // r[1] = 1 0 1 1 1 0 0 1 1 0 1         0 0 0 1 0
    // r[0] = 1 1 0 0 1 1 0 1 0 1 1         0 0 0 0 1
    //上面 k[10:0]对应的每列都有奇数个 1，r[4:0]对应的每列都是 1 个 1，故名奇重列码
    //上面 k[9:0]对应的 10 列，其序列是精心选择的，以实现对 k 的各种位宽（5～10bit），各校验位的生成矩阵的重量都最为接近、均衡，下面注释中 w 就是 weight 的缩写
    begin  //                              0 9876543210          5 6 7 8 9 10 11
        mow_gen_11p5[4] = ^(k & 11'b1_0110111010);  //w = 3 4 4 5 6   6   7
        mow_gen_11p5[3] = ^(k & 11'b1_1101010110);  //w = 3 3 4 4 5   6   7
        mow_gen_11p5[2] = ^(k & 11'b1_1010110101);  //w = 3 4 4 5 5   6   7
```

```
        mow_gen_11p5[1] = ^(k & 11'b1_0111001101);   //w = 3 3 4 5 6   6   7
        mow_gen_11p5[0] = ^(k & 11'b1_1001101011);   //w = 3 4 5 5 5   6   7
    end
endfunction
```

对读数据进行校验、纠正时，首先计算伴随式（sc＝syndrome calculate）：

```
    //                          0 9876543210          5 6 7 8 9 10 11
    sc[4] = r[4] ^ (^(k & 11'b1_0110111010));   //w = 4 5 5 6 7   7   8
    sc[3] = r[3] ^ (^(k & 11'b1_1101010110));   //w = 4 4 5 5 6   7   8
    sc[2] = r[2] ^ (^(k & 11'b1_1010110101));   //w = 4 5 5 6 6   7   8
    sc[1] = r[1] ^ (^(k & 11'b1_0111001101));   //w = 4 4 5 6 7   7   8
    sc[0] = r[0] ^ (^(k & 11'b1_1001101011));   //w = 4 5 6 6 6   7   8
```

然后使用伴随式进行读数据纠错：

```
    k_cor[10] = k[10] ^ (sc==5'b11111);
    k_cor[ 9] = k[ 9] ^ (sc==5'b01101);
    …… or[ 9] =  对应的 sc 特征值（常数）就是上面生成矩阵中 k[i]对应的列
    k_cor[ 0] = k[ 0] ^ (sc==5'b00111);
```

并输出 SEC/DED 指示：

```
    SEC＝(^sc==10] = k[        //sc[4:0]中有奇数个 1，1 个 1 表示错在 r[4:0]
    DED＝(^sc==10] = k[        //sc[4:0]0 中有偶数个 1 且非 0
```

随着半导体工艺的进步，芯片使用的 RAM（包括片内 RAM 和片外 RAM）出现软失效的风险越来越大，而产品应用对可靠性的要求也是越来越高，这对我们芯片设计的 RAM 采用校验和保护也提出了更多的要求，对片外 RAM（主要是 DRAM 接口）主要是带内 ECC 和带外 ECC，对片内 RAM 主要是 Parity（奇偶校验）和 ECC（SEC＋DED 纠错码），要根据芯片应用和需求的具体情况而定。

片内 RAM 要不要做校验和保护、是做 Parity 还是做 ECC，主要是看功能和用途是否关键/重要、出现软失效的后果和影响是否严重/恶劣，其次看 RAM 类型（RA/RF/SRAM）和容量大小，最后看时序是否允许。

RAM 做校验和保护，也不一定要全部 RAM 位宽和存储数据都做保护，可以只对部分位宽（比如高位的控制信息 而不是低位的报文数据）和部分数据（比如报文/信元头的数据格式 而不是全部报文数据）做保护，以尽量减少校验位开销，降低不必要的成本。

片内做了校验保护的 RAM 如果发现读数据校验错（Parit 发现奇偶校验错或 ECC 发现 DED），则需要有以下动作：

1、 有状态记录（历史状态，需要软件读清零或写清零）或中断上报（中断也是一种历史状态；中断应可屏蔽，且默认为屏蔽）；

2、 对软件访问的表项，要有校验错单元地址记录（记录最后一次出错的 RAM 读地址即可），对间接访问表项的软件读数据要有校验错标识；

3、 可选进行统计（8～16bit 饱和计数器即可，可判断错误发生的频度、累积数目、严重程度等）；

4、对校验错的读数据，要考虑是不是应该区别对待（有别于正确数据；视具体情形而定），比如丢弃、标记错误、或不予执行，防止错误扩大/蔓延，而不是视而不见、照常使用；但另一方面，也应尽量控制错误的范围，不能因为 RAM 软失效而影响、干扰其他功能操作（比如屏蔽对表项的回写）。

ECC RAM，相比 Parity RAM，还需要做定期刷新（读出再写回），以及时发现并纠正软失效比特，避免错误比特累积、超出纠错能力。这个刷新可以软件来做，也可以硬件逻辑做（或者 Memory BIST 来做）：对前者，要考虑软件开销/负荷、软件定时/周期，要提前和软件 SE/软件开发交流清楚，并在 FS 文档中描述清楚；对后者，要考虑 RAM 控制器的复杂度和性能/效率，避免对正常功能的读写访问造成冲击、影响性能和延时。

2.5.19 FEC（Forward Error Correction，前向纠错）

FEC（前向纠错编码）广泛应用于各种数字通信系统，通过在发送数据中添加冗余校验位，使接收方可以自动纠正传输误码，降低接收信号的误码率（BER，Bit Error Ratio）。FEC 使用纠错编码，接收方如果发现错误且在纠错能力范围内，则可以直接纠正，无须通知发送方重发，有别于 ARQ 方式（Auto Repeat reQuest，自动重传请求）。衡量 FEC 纠错能力的指标为“FEC 编码增益”，该增益越大表示纠错性能越强，但通常冗余校验位的开销（R/N）也越大。

FEC 的范畴非常广泛，可以采用多种纠错编码，包括 BCH Code、RS Code、Fire Code、LDPC、Turbo Code、卷积码等，不同编码的纠错能力和适用范围也各不相同，但应用最广泛的还是 RS Code，以下主要以 RS Code 和 IEEE 10G-KR 的 FEC Code 为例进行说明。

（缩写：BCH＝Boss Chaudhuri-Hocquenghem，以“博斯-乔赫里-霍克文黑姆”三个人名首字母命名的一种纠错编码；RS＝Reed-Solomon，以“里德-索罗蒙”两个人名首字母命名的一种纠错编码；Fire Code＝法尔码，一种循环纠错码；LDPC＝Low Density Parity Check Code，低密度奇偶校验码，由 Robert G.Gallager 博士于 1963 年提出的一类具有稀疏校验矩阵的线性分组码）

1. RS Code

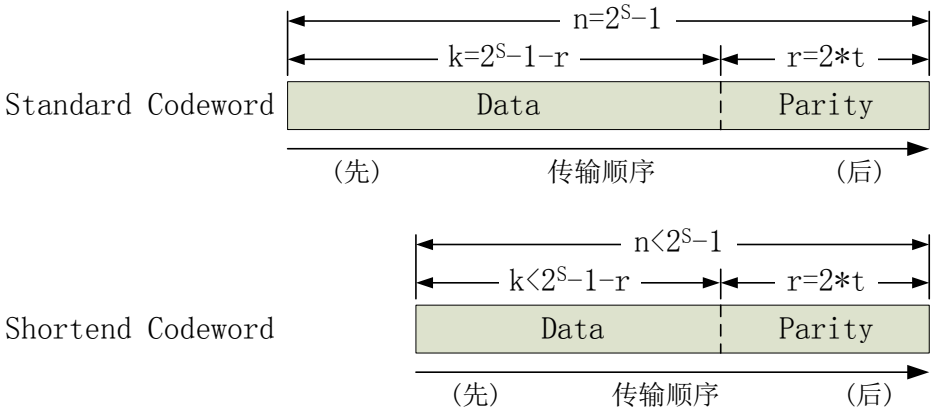
RS Code 是一类纠错能力很强的多进制 BCH 码，因其易于构造、编/解码电路简单，被广泛应用于包括 CD、DVD、蓝光光盘、以太网、WiMAX、SDH、PON 等在内的各种商业用途和通信网络。

RS Code 计算、纠错的单位不是 bit（比特），而是 symbol（符号），1 个 symbol ＝S 个 bit（这里引入了第一个参数：S，symbol 的位宽），S 常见为 8 或 10，如果 S＝8，则 1 个 symbol 就是 1 个 byte（字节），这就是我们常说“RS(255,239) 纠错能力为 8 个字节”的原因。

RS Code 是线性分组系统码，一个分组就是一个 codeword，长度为 n 个 symbol，其中包含 k 个 symbol 的净荷数据和 r 个 symbol 的冗余校验位，也即 n＝k＋r（k

＝key，r＝redundancy），这里 r 直接决定了（或者说取决于）纠错能力：r＝2*t （这里引入了第二个参数：t＝tolerance，纠错能力，一个 codeword 中最多能够纠正 t 个 symbol），所以 RS Code 通常写为 RS(n, n)或 RS(n, k, t)的形式，凸显其结构形式。

另外 n 必须满足：r<n<2^S，或者说 codeword 的最大长度是 2^S-1 个 symbol，短于这个长度的称为缩短 codeword（shortend codeword），缩短 codeword 的解码/纠错计算（Decode）跟标准 codeword（N＝2^S-1）略有不同。



RS Code 的生成多项式为：$G(x)=\prod_{i=I}^{I+2t-1}(x+\alpha^i)=x^{2t}+\sum_{i=0}^{2t-1}g_i x^i$

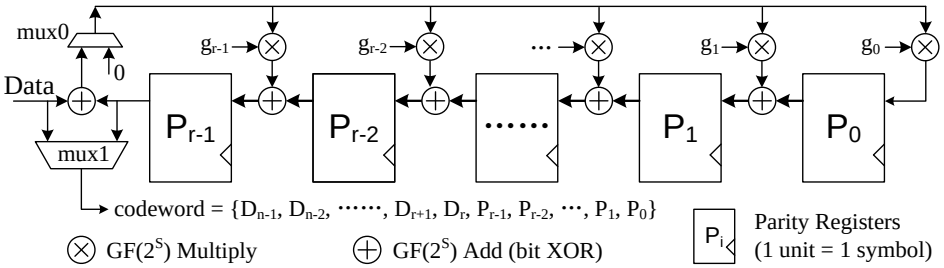
（这里引入了第三个参数：I，α 幂次的起始值；I 绝大多数情形都是 0，但很多资料书籍中 I＝0 和 1 的混淆却比比皆是），其中 α 是有限域 GF(2^S)的本原元，说起来就话长了。以最常见的 RS(255, 239)为例，其生成多项式的 16 个系数 g_i 如下表：

i＝	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Hex	3B	0D	68	BD	44	D1	1E	08	A3	41	29	E5	62	32	24	3B
Dec	59	13	104	189	68	209	30	8	163	65	41	229	98	50	36	59

综上，根据 S、T、I 三个参数（只要不是太变态，I 都是 0），我们可以唯一确定、构造一个 RS Code，然后 RS Decoder 还要区分是标准 codeword 还是缩短 codeword。

RS(n, k) Code	S	T	应用
RS(255, 239)	8	8	G.709、GPON、EPON
RS(252, 220)		16	10G-EPON
RS(248, 216)		16	XG-PON
RS(528, 514)	10	7	IEEE 40G/100G Ethernet
RS(544, 514)		15	

RS Code 编码器（encoder）跟通用的循环码编码器类似，如下图所示。

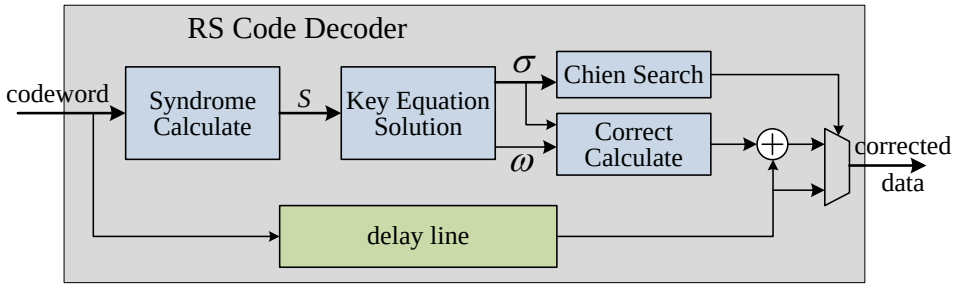


初始化：P_0～P_{r-1} 初始值为全 0；

第一阶段：Data 按从高位到低位的顺序，依次从左边输入，mux0 和 mux1 都选左边；每输入一个 Data symbol，Parity 寄存器就左移一个 symbol，codeword 也同时输出一个 Data symbol；

第二阶段：Data 输入完后，mux0 和 mux1 都选右边；再经历 r 个节拍，Parity 寄存器按从高位到低位的顺序依次从 codeword 输出。

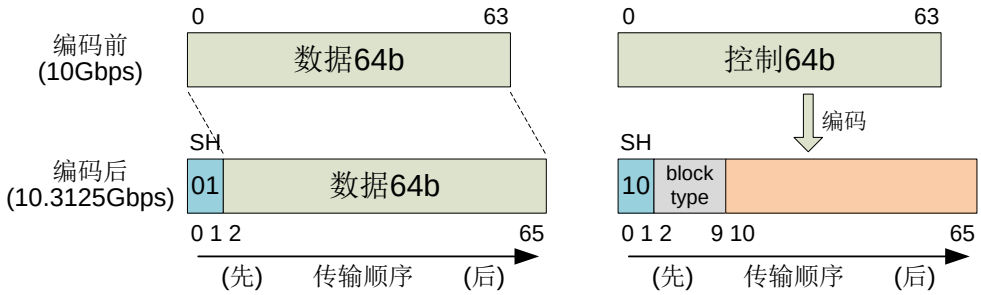
RS Code 解码器（decoder）的内部结构如下图所示。



先根据整个 codeword（包括 key 和 redundancy）计算得到 *S*（伴随多项式），再对 *S* 进行关键方程求解，得到σ（错误位置多项式）和ω（错误幅值多项式），再根据σ进行钱氏搜索，判断错误位置和错误数目（是否可纠），根据σ和ω进行错误幅值计算，跟延迟后的输入数据（key）进行异或即可得到纠正后的有效数据。

2. IEEE 10G-KR FEC

IEEE 10GBASE-KR 采用 64b/66b 编码，每 8 个数据字节或控制字符被编码成一个 66b block，最前面 2bit 是 SH（Sync Header），数据字节编码的 SH＝b01，控制字符编码的 SH＝b10，所以 SH 不可能是 b00 和 b11，接收方正是利用这一点来做接收同步。



64b/66b 编码前的数据速率为 10Gbps，编码后的线路速率为 10.3125Gbps（也即 SerDes 速率）。详细的编码表可以查看 IEEE 802.3 协议第 49 章的“Figure 49–7—64B/66B block formats”。

10G-KR FEC 采用(2112, 2080) Fire Code，N＝2112bit＝32×66b，K＝2080bit＝32×65b，R＝32bit，可在每个 codeword 中纠正一处最长 11bit 的突发错误（所谓突发错误是指从第一个错误比特到最后一个错误比特之间为连续错误，这称为一处突发错误，从第一个错误比特到最后一个错误比特的长度为突发错误长度；之间也可以不是错误比特，但不影响突发错误长度）。

FEC(2112, 2080) codeword 结构为：每连续 32 个 66b block 编为一个 codeword，每个 SH 去掉 bit0，保留 bit1（因为 SH 只可能是 b01 或 b10，bit1 足可表示整个 SH 的信息），正好空余出来 32bit，K＝32×65b＝2080bit；然后再附加 32bit 校验位，整个 codeword 正好又是 32×66b＝2112bit。所以 10G-KR FEC 可以看作是 0 开销，编码前后的数据速率相同，这称为带内 FEC。

2.6 经典案例

2.6.1 概述

经典案例分为两个部分：

- 正面案例，通过实际案例来描述一下好的设计方法能给项目带来很大的好处，引导设计人员多学习；
- 反面案例，通过实际案例来描述一些不恰当的设计给项目代码的危害，让设计人员引以为戒。注意增加一些后端工程方面的案例。

2.6.2 反面案例

1. 芯片多模式共享缓存电路

【发生场景】

SD5×××有两个工作模式，两个工作模式下的时钟来源不同。为节省 Die，有一组信元缓存设计为可以同时在两个模式下时钟。

【案例说明】

由于这一组缓存需要工作在两个时钟域，设计人员决定采用 4 口的 memory 来实现这一组缓存，其中一组读写端口在芯片工作模式 A 中被使用；另一组读写端口在芯片工作模式 B 中被使用，两组读写端口工作在不同的时钟域。

这样两个工作模式共享了一组信元缓存，而且 Vendor 提供的四口 memory 接口可以在 Vendor 提供的 STA 工具中被自动分析，STA 的工作简化了不少。

芯片流程到了后端设计阶段，发现 4 口的 Memory 占用的 Die 几乎等于一个 2 口 memory 的两倍（可能地址总线和测试电路占用了相当部分的面积）。这意味着虽然信元缓存在两个模式下被共享，但 Die 的消耗并没有因此减少。

【案例总结】

通过节省 memory 来减少 Die 的设计必须仔细评估，尤其设计完毕之后要总结真正的 memory 面积数据，避免想当然的设计。

2. 异步信号处理不当问题

【案例描述】

某款芯片在做样片测试的时候发现，CPU 对某些配置寄存器进行操作的时候，出现被异常改写的情况。

【案例分析】

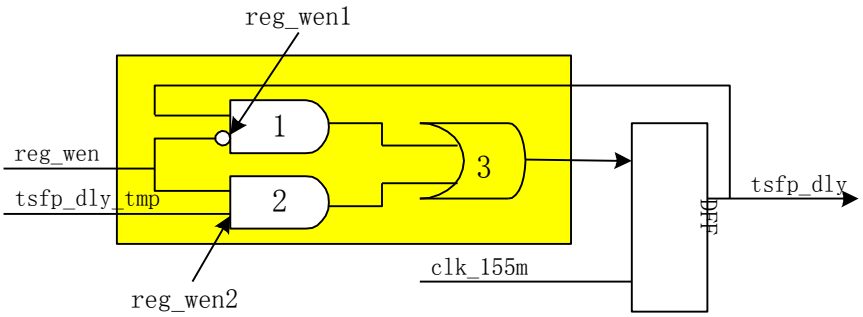
通过对代码的分析发现，在芯片设计中，使用 reg_wen 对模块进行同步操作，仿真确认 cpu 时钟驱动的 reg_wen 在被内部时钟采样的时候存在毛刺，导致上述隔离的不可靠性；测试经过 reg_wen 隔离的多 bit 控制寄存器如 sd/sf 时间窗、门限以及 hp_tim/plm/rdi_flt 门限，在进行反复读写过程中有回读不正确的情况，进一步确认是寄存器被异常改写；

错误代码如下：

```
always @(posedge clk_155m or negedge rst_n)
begin
    if(rst_n == 1'b0)
        tsfp_dly <= 15'b0;
    else if(reg_wen == 1'b1)
        tsfp_dly <= #`U_DLY tsfp_dly_tmp;
end
```

上面为截取的其中一个寄存器的同步配置实现代码，其它的寄存器，在使用 reg_wen 进行同步隔离过程中的代码实现方法与此相同。这里对诸如 tsfp_dly 一类的多比特位宽寄存器，通过使能信号 reg_wen 切换到 clk_155m 时钟域，原意是为了避免如下隐患：在对 tsfp_dly 寄存器进行配置的时候，刚好 clk_155m 时钟来采样这个寄存器，这样由于建立时间的问题，导致采样到的数据不正确。

但是综合后，得到的实际门级电路中，并没有 MUX、或者门控时钟，而是通过与门、或门组合得到，具体的电路形态如下：

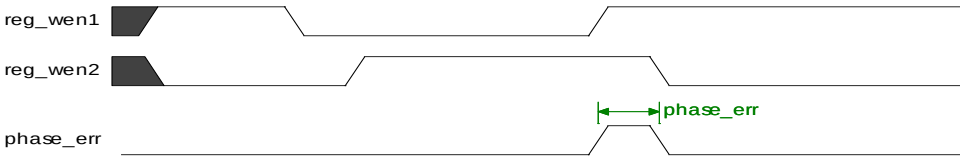


按照实际实现方式，如果 reg_wen 在门 1 和门 2 的入口位置被 clk_155m 采样的时候，相位刚好相反，那么，当 reg_wen 为 1 的时候，门 2 开启，门 1 关闭，此时是寄存器的配置过程，CPU 时钟域的寄存器 tsfp_dly_tmp 被同步到 clk_155m 时钟域，变成 tsfp_dly；当 reg_wen 为 0 的时候，门 1 开启，门 2 关闭，是 tsfp_dly 寄存器的保持过程。但是很可惜，上面提到的“相位刚好相反”，只是理论可能，在实际实现的时候，对于异步接口却是很难保证的。

为了描述方便，我们称 reg_wen 到达门 1 入口处的信号（取反后）为 reg_wen1；我们称 reg_wen 到达门 2 入口处的信号为 reg_wen2；理想情况下，reg_wen1 和 reg_wen2 的相位是严格相反的。

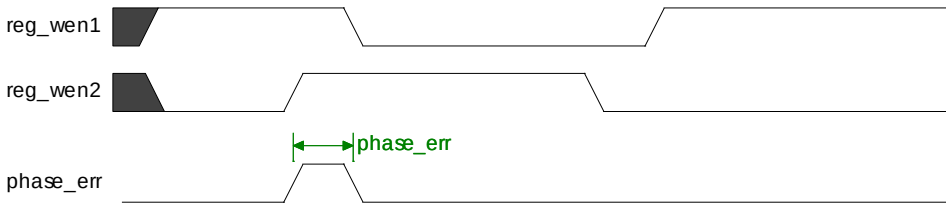
由于 reg_wen 是 CPU 时钟域的信号，由于它到达门 1 和门 2 的入口所经过两条路线的延时不相同，reg_wen1 和 reg_wen2 之间存在一定的相位毛刺。

如果 reg_wen1 对应的路径短，那么在 reg_wen 信号的下降沿位置，存在这样一个时刻：reg_wen1 和 reg_wen2 都是高电平。



如果毛刺在后面，那么在这个毛刺后面，毛刺结束的时刻，本次 reg_wen 的作用也就消失了，如果没有新的 reg_wen 有效时刻到来，clk_155m 就无法采样到 reg_wen1 为 0、reg_wen2 为 1 的情况，从而 tsfp_dly 就会一直保持错误的配置。

如果 reg_wen1 对应的路径长，那么在 reg_wen 信号的上升沿位置，存在这样一个时刻：reg_wen1 和 reg_wen2 都是高电平；



如果毛刺在前面，由于在毛刺后面，clk_155m 还可以继续采样到 reg_wen1 为 0、reg_wen2 为 1 的情况，tsfp_dly 会在 clk_155m 的下一个时钟周期恢复正确 。

【案例总结】

在代码设计的时候，想当然的以为综合工具会将 reg_wen 转换为时钟的门控信号，以达到功能上的信号同步以及性能上的低功耗；但是代码的实际实现形式和我们的预计大不相同。做为逻辑设计人员，我们要时刻提醒自己，我们设计的不是软件，而是电路，在设计的时候需要考虑电路的形态。针对此案例，上述错误代码中，如果之前首先将异步信号 reg_wen 进行同步化处理，就不会出现上述的错误，因此，在设计代码的时候，需要加强异步电路器件具体实现的检视，对于异步电路我们使用的时候要尽量同步化处理，如果不能同步化，一定要加强检视，确保部影响功能。

第3章 ASIC 验证知识

3.1 验证方法学

概述：综合目前的验证方法学方面的著作，在此介绍 candence 和 synopsys 的验证方法学；以帮助员工入门为主，起到掌握思想基本概念的作用。

3.1.1 VMM 验证方法学

1. VMM 验证方法学简介

在 ASIC 设计和验证的发展进程中，经验显示后者往往决定了时间进度。数字电路的规模和复杂度使得功能验证成为一项巨大的挑战。为了满足验证需求，这里介绍了一种有助于减少验证所需时间的 VMM 验证方法学（Verification Methodology Manual for SystmeVerilog）。同时，利用 VMM 中定义的内容，也可以确定产生通用验证环境和组件的标准。

采用可通用的验证环境和组件是减少验证一个完整的产品所需要付出的努力的基本因素。在所有的验证环境中采用可重用的模型——系统级验证环境可以从模块级环境中借用组件、自测结构和覆盖率组件；形式验证工具能够分享仿真中相同的属性；验证 IP 既能够满足模块级验证的需求，也可以满足后期系统级功能验证的需求。

为了随机简化约束机制，方法学中定义了产生事务（transaction）和数据描述符号（data descriptor）的标准和约定。怎样设计 BFM、monitors、transactors 来为模块和系统级验证提供激励和检测功能呢？VMM 提供了使之标准化的方法。而且，VMM 也定义了将不同的验证组件集成到一个验证环境中，以至这些组件可以轻易地被组合、控制和后期剥离出来应用到不同的环境中的标准；定义了开发覆盖率模型和软件验证环境的标准。当将所有这些标准都纳入到一个完整的方法学中时，在验证过程中，切实地应用它，有利于减少验证一个设计所花费的努力。

这里描述的方法学可以用不同的语言或者标准实现。但是当采用相同的语言和标准时，互通性是最好的。

(1) 验证生产率

一个项目的验证过程通常可以通过有多少功能特性被证明是功能正确的来衡量。因此，验证生产率是一种测量方法：在某段时间内，有多少特性被证明是功能正确的，包括需要调试和修改错误的时间。验证生产率越高，一款高性能的产品被制造出来的就越快。这种测量和相同时间内写的代码量没有关系，跟确认功能正确的仿真的实时性能也无关。写少量的代码、运行更多的并发的仿真可能获得更高的验证生产率。众所周知，任何验证环境的瓶颈都是效率问题。

从以前的经验看，验证方法学是随着设计的抽象层次发展的，与设计实现的复杂度密切相关。当设计是在掩模层次被实现时，验证是基于模拟晶体管模型仿真的；当设计发展到采用标准单元实现，验证进化到门级数字仿真；当设计为了引入逻辑综合利用了仿真语言来实现，验证也发展到采用 BFM 的事务级仿真平台。整个发展的过程中，验证的方法没有基本的改变：单条的测试用例验证单条的设计特性。

然而，传统的逐条的测试用例不能完全满足现在大规模的门级设计。一个拥有 1000 个分离特性的项目平均需要 10 个验证工程师超过 1 年的努力，按照每 3 天 1 个测试用例的进度，去编写、调试、维护一个测试用例。这样大的项目需要不同的新的验证方法。

VMM 通过 4 种机制来提高验证的生产率：断言（assertion）、抽象层次（abstraction）、自动化（automation）和重用（reuse）。

当采用断言去判别接口上的错误时，错误会在它发生的那个确切的位置和时刻被准确的报告出来。否则，错误序列或许要在几个时钟周期后，当它传递到输出端口，才能被检测出来。某些类型的错误表现形式可以很容易地在 DUT 的输出被观测到，比如丢包。然而，某些类型的错误表现形式是不清晰的，例如，一个可恢复的仲裁逻辑错误，只是让某种特定类型的业务的流量减少，这种错误通过 DUT 的输出信号是很难检测出来的。断言在设计关键点增加检测而不用产生额外的测试平台，这些关键点实现了外部可见，具备白盒验证的特点。

利用抽象层次的验证方法在过去一段时间里越来越多。但是跟这种抽象层次的提高跟 DUT 的抽象层次提高不完全一样。验证仍然需要测试底层实现和物理层接口信号时序。一旦底层功能被验证充分，验证可以在一个较高层次上继续下去。分层设计的 transactor 使得 transtactor 可以从单独的 BFM 脱离出来，单独的 BFM 引入复杂的、组合的协议处理是比较困难的。

与 DUT 具体实现相关的测试和响应检测机制使得通常意义上的验证自动化过程是不可能的。真正的自动化产生就象完全手动开发的 TC 一样。但是随机激励可以模拟自动化：一个设计合理的随机源将最终产生所需要的激励。随机激励也可以产生我们预测不到的一些条件。当随机激励不能产生需要的激励，或者当所需要的激励不可能被一个无约束的随机激励源产生时，可以通过增加约束条件来提高产生所需要激励的可能性。由于激励的随机特性，用一个覆盖率机制来识别迄今为止，哪些测试用例已经被自动伪随机生成是必要的。覆盖率机制测量验证过程的进行和生产率。通过快速识别哪些验证需求已经被验证到了，允许把精力集中在剩下需要被验证的需求。

重用提高了验证的效率。重用不仅局限在不同的项目中。第一级重用是指在同一个项目中不同的测试用例重用同一个验证环境。通过尽可能重用代码，只需要增加几行额外的代码就可以验证一个特性。最终，测试用例将只是对高度可重用的验证组件组成的跟具体设计相关的验证环境或平台简单的重新配置。

(2) 验证组件

前面提到，第一级重用是指不同的测试用例重用同一个验证环境。第二级重用是指系统级环境重用验证环境中某些组件；第三级重用是指不同的设计之间验证环境的重用。为了实现这些重用，验证组件的开发必须考虑得很全面。

为了实现验证组件的可重用性，这些组件首先必须是功能正确的，同时，也必须是可以配置的以满足调用它们的环境和测试用例的需要。可配置的意思是指验证组件能够表现出来的充分验证 DUT 和系统的能力。一个可配置的验证组件可以用来驱动一个模块级验证环境的接口。然后，同样的组件，不用修改，就可以集成到系统级验证环境中。一个验证组件因此必须满足模块级和系统级验证环境中不同的激励和检测需求。

基于接口的设计

目前，有外部接口和片上总线的的设计多数都是采用标准工业总线协议实现的，例如 AMBA 总线、USB 或者 Utopia。采用标准的外部接口和片上总线的优点是协议容易理解、开发过程中能够很容易获得设计 IP 和可重用的验证组件，工程师也容易理解相关协议。

为了便于对设计细节的管理和实现，项目组在设计阶段会进行功能划分。这个过程引入了很多内部接口。

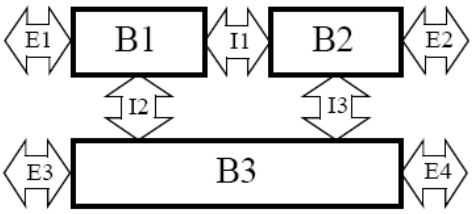


图3-1 分割设计

由于是在内部，设计工程师可以很灵活地实现这些接口。有多少种接口就有多少种不同的实现方式。为了验证分割好的设计，验证组件需要模拟每个内部接口的行为，因此，一个分割好的设计所需要的验证组件和接口数目是成比例，跟分割的块数是成指数增长的。例如，校验上图中的设计，有 3 个内部接口和 4 个外部接口，总共需要 13 个不同的验证组件，如下图所示。

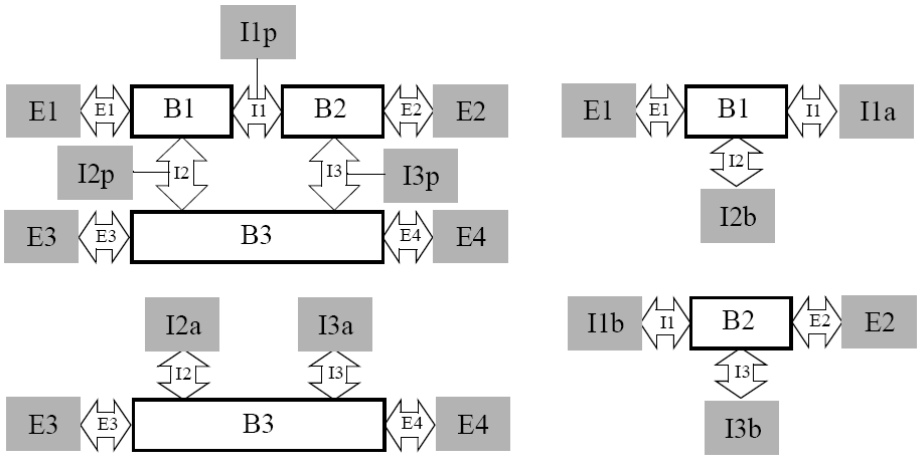


图3-2 分割设计的验证环境

一个基于接口的设计应该在设计阶段的早期定义好内部接口，目的是减少独立的接口数目和尽可能多地利用通用的验证组件。这个方法是让设计者在满足不同接口的性能目标时能够更加关注可重用性设计。

(3) 可验证性设计

Design for verfifacion 是对目前或将来验证复杂微电子设计过程中遇到的问题的一种响应。就如同 Design for synthesis 和 Design for test，需要设计在制订规格，架构分析和具体实现时考虑可验证的需求。

Design for synthesis 是随着基于 RTL 级的规格，特殊的硬件设计语言编码风格和所需的同步设计引入的。这些限制允许设计可以使用一组支持逻辑综合、cycle 级的仿真、静态时序分析和一致性检测的工具，这些工具有利于提高整个设计的生产率。

Design for test 将对结构化缺陷的测试和对器件功能的验证区分开。这些方法增加了对设计的进一步的约束——同步接口、无时钟门控、无锁存器和互斥总线驱动，与支持这些方法的工具一起，有助于提高对内部结构化错误的可控制性和可观测性，在器件可靠性方面提供了极大的好处。

Design for verification 使得设计者尽早地投入到验证过程中，特别是在设计开发的过程中。极少的前期投入可以极大地减少验证一个设计所需要付出的努力和时间。Design for verification 提供了一个方法，可以使设计者简明而自然地表述他的设计目的。最后，SystemVerilog 提供了断言来检测设计和接口信号的行为和假设。同时，提供了特殊的语法结构，例如 always_comb，消除了不确定性和进一步定义了实现代码的意图。

Design for verification 也鼓励设计者在做架构设计时充分考虑具体实现，减少系统验证的困难。例如，一个只写和一个只读的寄存器可以分享同一个地址，使每个可写的寄存器可读有助于验证它是否被正确的写入。同样，使内部独特的、自定义的接口数目尽可能的少和用标准的工业外部总线和内部片上总线，如前面提到的，使得所需要的验证组件的数目最少。对位数较多的计数器增加

预置的能力、旁路掉计算路径或者强置异常状态标志也能极大的简化验证过程。这些需要设计中增加额外的非功能性特性。

断言的好处

我们最终 Taped_out 的设计中，根据统计，主要有 3 类功能缺陷是跟设计错误相关的：规格错误、重用模块和 IP 的错误（可能是不正确的使用和 IP、重用模块的内部错误）。多数这些错误是由于规格的不明确或者修改，没有充分验证到模块的行为导致的。

设计者在 RTL 编码时，经常会对设计周围的一些行为和内部模块间的同步做假使。这些假设通常都是和规格无关的，或者是在仿真中没有被验证到的。任何周边设计的行为的变化或者内部的功能错误可能会与这些假设有冲突，最终导致错误。这些错误的表现形式或许在仿真很晚的时候的才能够在 DUT 的输出端口处被发现。这些没有被描述清楚的假设使得检测错误产生的原因是很困难和很花费时间的。

设计者可以用断言描述这些假设，并把他们插入到 RTL 代码中。如果这些假设与实际情况有冲突，将会在引发错误的具体位置产生一个断言错误。这种方法使得调试设计变得更容易。

当我们重用一个已经写好的 module 或者 IP 时，如果 module 的应用没有被精确的描述和测试，由于重用设计的黑盒特性，错误通常很难发现。断言在验证 IP 的使用方面可以起到很重要的作用。

SystemVerilog 中包含了一些时序结构，可以用明确的可执行的语句来提供有效的系统建模来描述设计规格，当这些描述被断言时，可以精确地描述规格的目的或者需求。这样的断言减少了模糊的概念和错误的理解。相比于 RTL 的设计描述，断言的描述更加抽象。断言提高了仿真中检测设计错误的可能性。

最后，属性可以被用于描述在仿真中需要被覆盖到的我们关心的激励和状态。这些属性不是用来检测错误，只是用来检测某些重要的状态出现过。他们定义了设计实现中不易被功能规格覆盖到的 corner cases。

(4) VMM 实现

VMM 方法学范围相当广阔，主要包括了几个不同的，但是相互关联的方面和元素。

方法学的目的是对一个设计在给定的时间和资源的前提下，得到最大的可信度。为了实现这个目标，方法学采用断言、功能抽象、通过随机化和重用提高自动化。

没有了解整个方法学的具体实现，很难充分意识到这些指导的作用。但是没有详细的指导原则想要有效地实现方法学去构造基本组件也是很困难的。

具体的 VMM 定义中构建验证环境所需要遵循的原则和建议请参考《Verification Methodology Mannual for SystemVerilog》。

VMM 应用

不一定需要采用 VMM 中介绍的所有的方法学中的内容。但是很显然，当所有的方法学中提到的指导原则都能注意到才能得到最高的生产率。可是对于实际的项目，实际的人和项目进度，不可能付出那么多的时间给验证工程师去采用所有的指导原则。可以采用部分的指导原则，提高验证效率。

许多设计团队已经采用了断言去检测设计和接口信号中的错误。也有很多资料描述了断言的好处。采用方法学中的提到一些指导原则，可以加速正确地实现、开发的过程。同样也介绍了一些基于断言的检测器，可以不需要对断言语言了解的很透彻，可以最小地降低在已存在的设计中引入断言所需要的开销。

嵌入在 vmm_log calss 中的消息接口是最容易采用的结构。它可以很容易地替换掉大多数设计团队自己开发的消息进程。

将仿真步骤固化下来的操作，嵌入在 vmm_env 类中，是第二个采用的步骤。为了正确地实现，所有的仿真必须执行相同的顺序的步骤。Vmm_env 有助于将执行的过程固化下来，而不是对于每一个新的验证环境都要产生一个验证环境。如果采用相同的控制流程，不同的模块级环境可以很容易地升级到系统级环境。把仿真步骤固化下来允许在适当的时间调用不同的测试，而不需要违反仿真顺序。

通过扩展 vmm_data base class 用事务描述符建模事务模型和建立所有的数据模型，保证了统一的激励产生机制。无论是是事务还是数据，所有激励的产生的方式都是一样的，都可以采用相同的机制，很容易地被随机和约束。不同的测试能够应用不同的约束，而不需要重写随机产生器。也可以产生全部直接或者部分直接事务激励。

事务和数据以 class 的形式建模，事务产生器之间的接口采用 vmm_channel base class 扩展的 channel 来通信。

一个定义清楚的事务级接口机制可以让 transactors 在较高抽象层次来产生，而不必象传统的 BFM 一样与物理层接口相关。这样的结构使得我们搭建验证环境可以采用自顶向下的的方式：先产生事务级模型，然后应用到 RTL 模型；或者也可以采用自下向上的方式：先验证底层操作，然后验证更复杂和抽象的功能。

采用 factory_patterned 产生器必须使用随机激励和 callback 方式。他们不止提供了在一个文件里用更少的代码产生测试用例的能力，而且可以不用修改或者破坏已经存在的代码，这里 callback 起到了很大的作用。新的测试用例可以在不影响任何已经存在的测试用例的情况下开发（back_forward）。因为可以用很少的代码开发测试用例去测试 DUT 的一个特殊的功能，这也使得实现一个真正的覆盖率驱动的验证方法学更高效。第二个优点，generators 和 transactors 可以满足不同测试的不定的需要，这也满足了不同验证环境或者项目的需要，使他们可以真正得到重用。

如果一个系统的 corner cases 的特性依靠不同接口上并发激励的同步，那么采用 XVC 是一个好办法。对一个接口，一旦我们关心的激励已知并且别实现，他产生一个自然的基于命令的接口来写系统级测试用例而在一个纯随机环境里不

太可能出现的。如果需要更多的激励参数或者 pattern，也可以很容易地加到环境里，而不必对现有的，运行良好的环境做任何修改。也能够产生一个易于学习的测试规格机制，而不用对整个方法学的细节了解或者对他的实现。

Formal 工具在发现在负责的设计模块中那些难于识别或者难于到达的 corner case 错误是很高效的，例如仲裁器、总线协议控制器，指令调度器，流水线控制等等。这些结构的 RTL 实现方式和他们期望的行为的 assertion 的描述相比较。

基本的编码规则，反复强调诸如缩进、注释等编码规范是毫无意义的，这些内容可以在很多资料中获得。所有适用于 Verilog 的编码规范，同样适用于 SystemVerilog。然而，跟验证相关或者一些高级的语法结构需要一些额外的基本编码规范。这里不做过多介绍，有兴趣的可以参照《Verification Methodology Mannual for SystemVerilog》一书。

验证常常涉及到信息交流，设计的详细说明文档（SPEC）必须定义明确清楚，才能够得到正确的实现和充分的验证。验证中经常要澄清同一个项目中不同团队（验证和设计团队）之间的设计理解的偏差。

2. 验证计划

验证一个设计比用 RTL 描述它需要更多的精力。早在 1974 年，Brian Kernighan，C 语言的创造者，曾经说过：Everyone knows debugging is twice as hard as writing a program in the first place”。项目初始阶段，大量的精力投入到设计的需求定义上，但是相比较，但是即便投入更多的精力到定义如何确保设计的正确性上来，验证仍然是更大的任务。

每一个设计团队都希望一次成功，没人计划失败和多次设计反复。但是怎样做到一次成功？在没有定义清楚那些功能是关键的时候，资源如何被合理分配以确保设计的关键功能正确？验证计划是规格说明。验证计划必须基于设计目的，而不是设计实现。当然即使验证计划已经完成，在最初设计目的中不清晰的设计实现过程中产生的 corner cases，也必须被验证到。

本章节内容对于验证经理和项目经理是需要关心的。可以帮助他们定义好项目需求，分配资源，制订工作计划和跟踪项目进展情况。

(1) 制订计划的过程

为了充分利用新的验证技术和方法，传统的制订验证计划的方法需要被修订。如果验证过程中采用了 SystemVerilog 中新的验证结构，那么它可以提供新的验证生产率的保证。传统的验证计划的制订，包括定义以设计的一个特殊功能为目标的测试用例，测试用例可以是带有自测组件的，或者是可以在设计的输出中查找错误。每一个测试用例分配给单独的工程师去实现。除了低层总线功能模型之间连到设计接口上，测试用例之间基本没有重用。这种验证方法和按模块定义设计，然后希望一旦放在一起，能够满足所有需要的设计是类似的。设计实现是分阶段的：首先是需求分析，然后是总体架构设计和最终具体实现。

验证计划的过程遵循类似的步骤。每一个步骤可以作为独立的交叉参考文档，或者在一个文档中定义。

III. 功能验证需求

明确功能验证需求的目的是为了定义哪些为了设计实现预定的功能的验证需求是必须的。验证需求将形成一个基础，后续的验证工作将围绕验证需求进行。这些需求应该在项目周期中尽早地定义清楚，理想情况下，最好在设计总体架构设计阶段前完成。验证需求应该是项目技术总体组 review 的一部分。

建议由大量项目组内外相关人员对验证需求做 review。包括有经验的设计、验证、软件工程师，从硬件和软件的角度去分析、reivew 各种验证需求。这样做的目的是确保验证需求中定义的内容是完整的。设计所能实现的功能应该被详细说明，定义设计做什么——设计能处理那些输入 pattern，能容忍那些错误是验证需求的一个部分。这些需求确保设计实现了想要的功能。这些需求是建立在设计和验证都认可的一个功能描述的文档基础上的。设计不能处理的功能也应该被详细说明，定义哪些情况会使设计出错也是验证需求的一部分。这些需求确保在设计中的功能错误将不会被忽视。

功能详细描述文档（SPEC）关注定义设计的所要实现的行为。验证需求关注需要检测哪些错误。验证只能检测它正在查找的错误。验证需求必须能够列出哪些错误要被查找。有很多种场景可能导致设计功能出错。验证需求只能罗列出跟设计的功能和架构相关的或者可能引起的错误。

验证过程中覆盖不到的功能需要定义清楚，在实际情况下不可能出现的行为，是不需要验证的。为了清楚的描述出设计期望控制或者不期望处理那些行为，在设计应用范围之外的条件也需要列出来。

例如，一个数据通信的逻辑设计可能会希望能够从一个校验错误、一个输入协议的错误或者掉电中自动恢复。但是对于一个处理器，不可能期望它从执行非法指令代码或者程序空间错误中恢复。

IV. 验证环境需求

这一步的目的是定义需要的验证组件的需求，基于前面定义的需求，这里主要定义了实现验证需求所需要的资源。

下面罗列一下我们整理验证环境需要注意的一些基本原则。

A．应该定义清楚需要被单独验证的设计部分。

不是所有的设计模块地位都是等价的，某些关键功能的模块在系统级验证环境中是不可控或者不可观测的；另外一些模块实现的变换可以立刻在 DUT 输出的码流中观测到。设计的物理层次结构划分是为了使独立的模块的设计更合理和更易于集成，而不是为了平衡功能验证环境的相对复杂性。

一些功能验证需求更容易在单元测试中实现。对小的设计，可以获得更大的可控性和可观测性。但是分割越小，也提高了验证环境的数目。对于每一个验证需求的功能的可控性和可观测性必须与产生相应的设计分割的开销相衡量。

B．应该定义清楚可重用的验证组件。

验证环境对每一个单独地被验证的设计的接口必须提供驱动或者监视。这些接口应该有部分会被系统级验证环境所重用。在多个设计之间共享的接口，不管是工业级标准接口还是用户自定义的接口，应该有同样的 transactor 去驱动和监视他们。这种分享将减少独立的需要被开发的在所有验证环境中的验证组件的数目。最后，设计本身需要采用公共的接口以减少功能验证的负担通常是很有益的。

注意分享相同物理接口的不同设计可以有不同的验证需求，为了实现在不同环境中的重用，这些接口的验证组件必须能够满足所有的这些需求。可重用的验证组件通常在较高抽象层次应用的更多。即使物理层接口是不同的，他们可以传输相同的协议信息，这些接口的协议层应该在一个分离的、可重用的验证组件来实现。例如，MII 和 RMII 接口是两类用于传输 MAC 帧的接口。尽管他们有不同的接口信号，他们传输相同的数据内容，都遵循相同的 MAC 层协议规范。因此，他们可以共用相同的 MAC 帧描述符和 MAC 层 transactor。

C．应该定义清楚设计不同层次的抽象建模。

许多功能验证需求不需要对一个具体实现建模，例如一个 RTL 和门级模型。进一步，在较高抽象层次建模能够被满足的需求可以得到更大的性能。某些需求，例如软件有效性，如果只有一个实现级别的模型，想要满足是很困难的。能够选择不同抽象层次上的设计建模能够极大地提高验证进程的效率。

D．设计支持的配置需要定义清楚。

设计经常需要支持多种配置。验证过程首先可能只关注某些配置，然后再扩展开来。类似地，设计也可以首先实现某些可配置的功能，与验证过程相互配合。

配置选择机制应该被定义清楚，只有一组预定义的配置需要被验证吗？或者，配置是随机选择的吗？如果配置是随机选择的，那么配置参数之间的相关组合如何被覆盖到？可以增加约束来限制随机配置的产生。

E．相应检测机制需要被定义清楚。

功能验证需求描述在操作正确的时候，设计被期望的行为。也应该定义设计不应该表现那些错误的状态。自测结构能够容易地判别是否执行了正确的时序。但是，在整个过程中没有产生任何错误是相当困难的。自测机制只能上报与期望相对的错误。能够观测到的错误的越多，设计的响应就越容易得到验证。自动检测机制应该选择那些能够表现出来的基于期望的错误。

某些错误在信号层比较清楚。这类响应用 assertions 验证效率更高。有些错误只要一个输出 transaction 被收到就能判别出来。这类响应用 scoreboarding 机制检测效率会比较高。其他错误，例如性能测试或者对共享资源的公平访问，需要对输出数据进行统计分析。这类响应检测用 offline checking 效率更高。

F．激励需求应该被定义。

为了满足功能验证的需求，对设计输入特定的激励序列，或者把设计放在某种环境中是很必要的。通过输入某些激励序列，把设计置于某种状态下。因此，问题简化成为了满足功能验证需求产生某种特定的激励序列。传统上，用一个直接测试序列来满足这些需求。然而，为了避免写很多的直接测试用例，这里推荐使用随机产生器去自动产生激励。

如果随机激励不能产生所需要的激励序列，可以在稍后的仿真中增加约束来提高产生所需激励的可能性。

约束随机产生的能力能够产生不会偶尔出现的，所需要的激励。Generators 必须设计成能够产生验证需求所需要的激励。他们必须能够提供产生所需要的

如果随机产生一个所需要的激励序列是不可能的，那么会产生一个直接测试激励序列。

Trivial tests（基本功能测试）是那些首先要跑的测试用例，这些基本功能测试用例不是用来测试功能验证需求的，只是在运行更复杂的测试前用来确认设计的基本功能是正确的。例如，执行一个写操作后，执行一个读操作，输入一个包，或者执行一系列空操作都是基本功能测试。

基本功能测试不需要直接测试。只是因为他们用于确定设计的基本功能，他们可以用来确定验证环境的基本正确。一个基本功能测试可以是一个简单的约束随机测试只产生少量的激励序列。例如，一个基本功能测试可以只维持 2 个周期：第一个周期写，第二个周期读，2 个周期约束使用相同的地址。

G．应该定义错误插入机制

设计可以容忍某种类型的错误或者激励异常。这些异常必须定义清楚，和产生这些异常的机制。然后，需要定义对应这些异常那些响应是正确的。基于验证需求，在验证环境中定义错误产生机制是必须的。对于低层设计，在激励产生的同时输入错误是可能的。对于更复杂的、多层次的系统，从顶层协议栈精确描述低层错误是不太可能的。而且，可能会有错误。

异常需要和其他激励同步，例如中断请求要和译码流水线的不同状态同步。同步一个异常码流和一个数据流可能需要一个多码流产生器。

还有很多需要注意的地方，这里限于篇幅不再过多介绍。

V. 验证实现计划

开发功能验证计划的基本目的是确保在给定的项目周期内，能够尽可能地对设计的功能进行尽可能多的测试。开发基于上面提到的验证环境的需求。

开发计划应该在项目周期中尽早地开始。理想情况下，应该在 RTL 编码阶段开始前和验证平台代码开发前完成。

(2) 响应检测

上面提到要求验证需求中所罗列的所有错误都必须被验证环境检测到。这些检测机制要求能够描述期望的响应和能够比较观测到的响应和这些期望响应。这里重点关注这些策略。因为响应一般都是和设计相关的，因此想要描述一些用于检测设计的响应的方法是很困难的。响应检测这里只做一般概述。定义一些一般的自动检测结构的概念。SystemVerilog 中的高级语法结构极大地简化了响应检测的实现。

传统的直接测试用例，因为激励和设计的功能已知。可以预先知道期望的响应。在随机激励条件下，尽管功能已知，采用的激励未知。期望的响应必须基于设计的配置和功能计算得到。然后观测到的响应和计算得到的响应比较以确定其正确性。

清楚在验证环境中响应检测结构只能找到问题是很重要的。为了发现矛盾，正确性是从错误中推断出来的。如果响应检测结构不能明确地检测某种症状的错误，那么错误仍然没有被发现。功能验证需求必须包括所有可能出现的错误的症状的定义。

下面罗列了一些常用的响应检测的办法。

VI. Embedded Monitors

响应通常可以理解为被验证的设计的外部接口输出信号的检测。然而，外部接口信号的受限使得识别某些错误症状是很困难的。如果验证环境对整个设计没有足够的可观测度，就需要对远离外部接口的那些内部设计结构的正确性花费更多的精力去验证。这个问题在那些有内部总线和功能单元，不能被外部直接观测到的系统表现得更明显。

可以用 transactors 替换设计组件，Transactors 不需要被限定在与外部接口的通讯上。内嵌在设计中的 monitors 可以观测或者替换掉内部设计单元，提供功能单元接口的可观测性。这些内嵌的 monitor 的事务级接口保留了外部的可访问性，可以在外部镜像或者替换内部接口信号的逻辑。例如，一个嵌入式 RAM 模块可以被个 reactive transctor(slave)替换。

另外一个办法，Assertion 也可以用来验证内部信号的响应，然而，assertion 不能提供高级的检测响应的能力，也不能提供激励，因此不能替换一个 reactive transactor。

VII. Assertions

Assertion 意味着正确的描述。从一个验证的角度看，一个 assertion 是对期望行为的描述。对于观测到的行为，任何与期望的不一致都会导致一个错误。基于这个定义，整个验证平台就是一个大的 assertion：对整个设计的期望行为的一个描述。这里我们提到的 assertion 指的是用时序表达式表述的设计的属性。

用 assertion 去检测和调试错误发生在具体的功能点处的功能错误已经证明是非常有效的。但是尽管他们是非常有效的，assertion 被限制于那些能用时钟控制的时序表达式的属性上。一些关于设计的期望行为的描述仍然必须被明确地

用行为级代码表达出来，或者易于表达出来。Assertion 和行为级检测可以协同工作，例如，一个协议检测器可以用 assertion 来描述低层信号协议，用行为级编码来描述高级，面向事务的属性。

Assertion 擅长验证局部信号之间的关系，在检测接口间信号，状态跳变和物理层协议方面是非常有效的。他们也可以很容易地确定不被期望的，虚假的条件。

另一方面，Assertion 不适合用来检测数据传输，计算和顺序错误。例如，Assertion 很难检测出所有数据包根据他们各自的优先级都路由到了合适的输出端口。

下面一些指导原则将确定那些响应检测适合用 Assertion 还是在验证环境中实现。

- Assertion 应该局限于验证物理层假设和响应。
  时序表达式最适合用于描述 cycle-based 的物理层关系，尽管时序表达式可以用于描述表示高层协议事件，但是由于没有时钟参考，使得描述变得很困难。而且，这些信息可能已经在 transactor 中已经得到了，在 transactor 中实现一个行为级的检测更简单。
- 在不同的设计抽象层次，都需要实现的检测需求，应该用过程代码在验证环境中实现。
  Assertion 很依赖于 RTL 或者门级模型。他们不能轻易地被事务级模型调用。在不同的设计抽象层次的响应都要检测的需求，最好在验证平台中实现。

VIII. Scoreboarding

一个 Scoreboard 用于动态地预测设计的响应。如图，作用在设计上的激励同时传给一个发送函数，这个发送函数为了产生最终的响应模式，执行所有的对激励的变换操作，然后插入到一个数据结构中。从激励得到的测试的响应被送给比较函数校验这个响应是否是期望的。

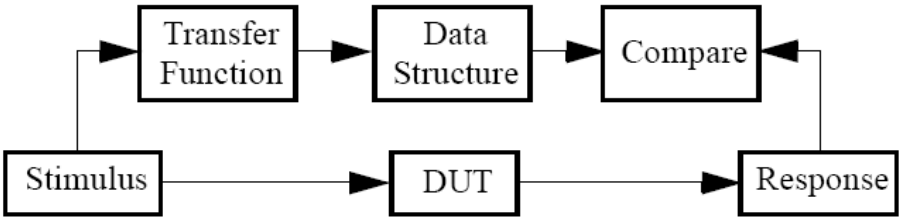


图3-3 采用 Scoreboard 的验证环境

Transfer Function 是一个事务级的参考模型通常是在 0 时刻运行。他也可以用一个参考，或者 Golden reference 来实现。Data structure 存储期望的响应，直到他跟观测到的输出做比较。Compare function 在 Data structure 中查找期望的响应，来确认观测到的响应是否与期望的匹配。Data structure 和 Compare fucntion 处理任何可以接受的不一致，在观测到的响应和期望的输出之间，例如顺序和延时。

Transfer function 和 Data structure 通常是可配置的，和 DUT 的配置相匹配：不同的配置可以产生不同的响应。Transfer function 可以用 C 语言实现。可以用 DPI 接口把他们集成到 SystemVerilog 环境中。

Scoreboard 一词在业界定义的不是很清楚。它有时候指的是只用来存储数据结构，有时候也包括 Transfer function，有时候包括比较功能，在这里，scoreboard 用来指整个的动态响应检测结构。

Scoreboard 适合用于验证端到端设计的响应，和输出数据的完整性。它能有效地检测出数据计算中的错误，转换和分类（顺序）。他也能容易地识别丢失或错误的数据。另一方面，他不善于检测某类错误症状不明显的响应。例如，scoreboard 很难验证内部资源分配的公平性和业务质量仲裁逻辑；也很难用于验证 DUT 的整个性能。

IX. Reference Model

参考模型和 Scoreboard 类似，用于动态预测设计的响应。如图，作用在设计上的激励同时传递给参考模型，参考模型的输出与观测到的响应相比较。

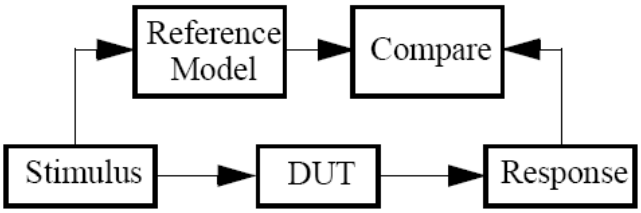


图3-4 采用 RM 的验证环境

参考模型有和 scoreboard 同样的能力和挑战。跟 scoreboard 不一样的是，比较函数直接接收参考模型的输出。参考模型因此必须产生跟设计本身一样顺序的输出。然而，不需要产生同样的延迟，也不要保证 cycle 级精确:比较函数能够处理期望和观测到的响应之间 latency 和 cycle 之间的差异。一个参考模型可以是事务级的，有一个高级事务接口：在事务层比较观测到的响应和参考模型输出的响应，而不是在 cycle-cycle 层。

使用参考模型很严重依赖于其实用性。如果实用，应该用他。如果不实用，Scoreboard 技术更易于实现。参考模型可以用 C 来实现。

X. Offline Checking

Offline Checking 用于在仿真的前后预测设计的响应。如图 3-5，仿真前预测的 Offline Checking，Offline 检测器产生一个期望响应的描述，动态地和仿真中观测到的响应比较。比较函数可以动态地比较预期的响应和观测到的响应。

如图 3-6，仿真后预测的 Offline Checking，offline 响应比较器比较记录的设计的激励和响应与预期的结果。在这两种方式下，响应可以在不同等级的细节和精确度层次被检测，从 cycle-by-cycle 到事务级。

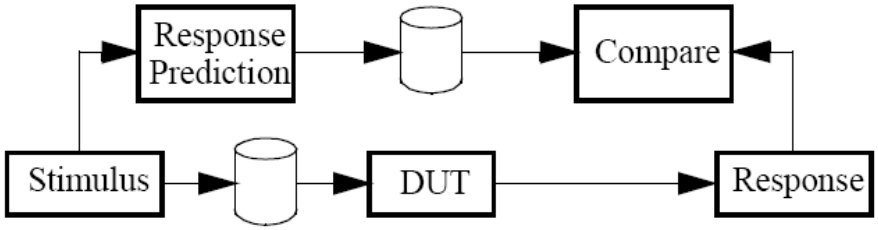


图3-5 Pre_Simulation Offline Checking

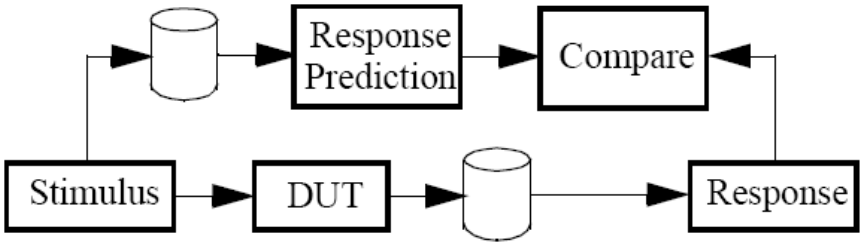


图3-6 ost-Simulation Offline Checking

(3) 总结

首先，验证计划中必须要能够满足对验证需求的覆盖。这些需求产生验证环境中激励，响应检测和功能覆盖等方面的详细说明（SPEC）。

然后，要提供计算或定义 DUT 的期望响应的策略。不同的策略有不同的优缺点和限制，当比较观测到的响应和期望的响应的时候。某种类型的错误可能会用到特殊的策略产生激励，用其他的方法不容易产生。

断言非常适合用于验证基于具体实现和物理层的问题；而验证平台适合用来验证事务级响应。断言不象测试平台，不局限于对 DUT 的输出检测其响应。设计的完整响应可以通过验证环境和断言组合来验证。

3. Assertion

并发的即时的断言是 SystemVerilog 一个组成部分，他们提供了一个有效的方法提高设计错误的可观察性和定位，通过在设计内部和外部接口的关键点上放置功能检测器。当仿真中一个断言报错，错误产生的原因的定位比观测仿真进程要容易的多。

断言是对设计的属性的描述。断言的作用是保证设计目的（属性）和最终的设计实现的一致。

如果一个在模拟中被检查的属性不像我们期望的那样表示，那么这个断言失败。

如果一个被禁止在设计中出现的属性在仿真过程中发生，那么这个断言也失败。

在 DAC2004，Accellera 把 SystemVerilog3.1 定为一个标准。断言语言也被纳入 SystemVerilog 语言并成为标准的一部分。这就是通常说的“SystemVerilog Assertion(SVA)”。这里不详细介绍 SVA 语法本身，那需要太多的篇幅，有兴趣的同事可以看看附录的资料。

这里只简单介绍断言能做什么，和我们怎么应用它。

关于 Assertion，《Assertion-Based Design》一书开篇提了个很有趣的问题：谁应该学习基于断言的设计？设计工程师？还是验证工程师？

答案是都需要。

设计中那些位置需要加入 assertion？

设计工程师可以在具体实现时在代码的一些关键逻辑处，增加 assertion，一旦逻辑行为在某种场景没有按照预期的方式发生，assertion 就会即时报出相关错误信息。

验证工程师可以在 DUT 与环境接口处，在 Monitors 中，增加 assertion，一旦接口时序没有满足相关协议，assertion 就会即时报出相关错误信息。

(1) 断言的作用

断言可以帮助我们及时的发现错误，如果没有它，我们就必须根据 DUT 的输出，去一个一个模块追踪产生的错误的模块，我想，大多数验证人员都有这样的经历，过程是很枯燥的。首先，验证工程师对具体的设计实现是不清楚的，他需要根据各个模块的详细说明书中定义的接口信号去判断错误发生在某个模块中，更多时候需要和设计工程师不断地交流，或者需要设计工程师亲自定位问题。

如果在设计内部增加了断言，一旦断言描述的属性不被满足，它会即时地打印出具体的错误信息，不需要在一个模块一个模块地追溯，极大地节省了仿真的时间。

XI. 提高可观测性

在传统的验证环境中，测试平台产生输入给 DUT 的激励，通过检测 DUT 输出的信号判断是否与期望的输出相一致。用这种方法去找 BUG，环境必须能够做到：

- 产生合适的输入能够触发 BUG。
- 产生的这个合适的输入必须能够使得这个 BUG 影响的结果能够传递到输出端口。

但是，通常我们会遇到这种情况，能够制造出产生设计内部错误的输入激励，但是这个错误不能传递到一个可以观测的输出端口。

嵌入在代码中的断言能够提高设计的可观测性。有了断言，验证环境不再只能依靠观测设计的输出来判别设计的行为。任何不符合期望的行为都可以在错误的发生源处被抓到。

XII. 缩短了调试时间

我们前面提到，断言能够提高可观测性，使得我们可以在 BUG 出现的具体位置，能够即时地发现。相对地，没有加断言的验证环境，不可能直接定位到错误发生的具体逻辑电路那个位置和具体的仿真时刻，一般都是在过了若干个仿真周

期后，在其他的位置（基本上都是在 DUT 模块间的接口，或者 DUT 输出端口处）才能检测到。

这需要验证工程师花费大量的时间去追溯错误发生的具体位置。

XIII. IP中的断言提高了有效集成

随着设计规模越来越大，逻辑越来越复杂，设计中集成的 IP 越来越多，项目组不可能对每一个用到 IP 都了如指掌，相反，通常状况下，项目组只是把 IP 当做黑盒子来用。

但是，我们在使用 IP 时，遇到最多的问题不是 IP 本身有问题，而是不正确的使用，配置不合理，输入给 IP 的信号不符合 IP 输入接口规范。如果 IP 中存在断言代码，尤其是在接口处有检测接口协议的断言，这样的问题就很容易发现。

如果 IP 中没有断言，验证工程师在遇到一个问题时，就要不断地和设计工程师、IP 供应者进行交互，不可避免地影响了验证的效率。

(2) 断言的使用

目前常用的有三种断言标准：Accellera Open Verification Library (OVL)、Accellera Property Specification Langauge (PSL)和 Accellera SystemVerilog。我们关注的是第三种 SVA 和第一种 OVL。

我们可以自己用 SVA 开发自己的断言检测器，这里不做过多介绍，有兴趣的同事请参考 SystemVerilog LRM 中关于 assertion 的介绍。

也可以使用预开发的断言库。SVA 断言检测器主要包括两部分：SVL 和 SVA 高级检测器。SVL 包括的检测器与 OVL 中提供的检测器有类似行为和控制方式。SVA 高级检测器可以产生更复杂的行为。

SVA 断言检测器的具体应用请参考《SystemVerilog Checker Library Reference Manual》。

在 VCS 中可以调用预定义的 SVA 断言检测器。

```
    vcs -sverilog +verilog2001ext+.v -debug_all -ntb_opts rvm +no_tchk_msg
\
          +incdir+../cmd+../th+../bfm+../tt+../rm   +define+ASSERT_ON   -f
Project.f \
        ../tc/TC0000.sv -y /software/synopsys/vcs/2006.06/packages/sva \
                                                          +libext+.sv+.v
+incdir+/software/synopsys/vcs/2006.06/packages/sva \
        -timescale=1ns/1ps
```

其中+define+ASSERT_ON 是必须加的，这是打开所有 Checkers 的开关。/software/synopsys/vcs/2006.06/packages/sva 是 VCS 提供的断言库。

4. 测试平台的基本结构

从一种过程语言切换到一种面向对象的语言，例如从 Verilog 或 VHDL，切换到 SystemVerilog，如何有效地运用面向对象的技术建模是一项挑战。如果运用的

合理，这些特性能极大提高验证平台组件的可重用性。这一章提供了一些指导，可以最大限度地利用 SystemVerilog 语言的面向对象的编程特性去搭建验证组件，并把他们组合成一个高效的验证环境满足所有作用在 DUT 上的 TC 的需要。

(1) 测试平台架构

测试用例的位置一般应用在验证环境的最顶层，如图 3-7。验证环境实现抽象和自动化功能，有助于减少需要开发的测试用例的数目和细节。验证环境需要尽可能地做到不需要进行任何地修改地被不同的测试用例重用，尽可能地减少验证 DUT 所需要的测试用例的代码量。对于一个给定的 DUT，可能需要几个验证环境。方法学的一个重要目的就是减少开发一条测试用例所需要的代码量，使得所需要的验证环境最少，尽量在已存在的环境上的开发测试用例。为了节省在也许是几千条测试用例中的一行代码，在几个或者一个验证环境上的投入是很有价值的。

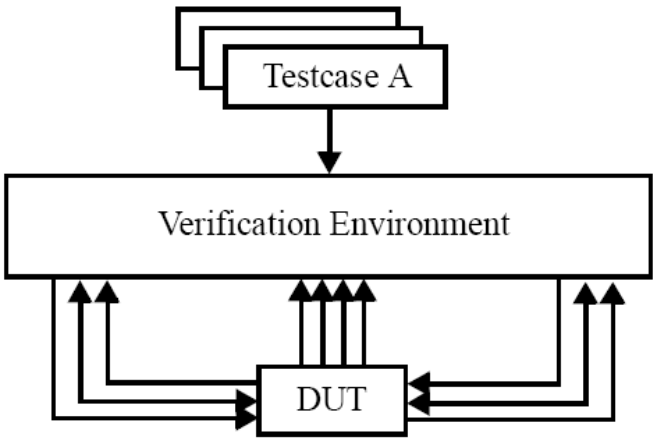


图3-7 验证环境

验证环境由不同的抽象层次组成，为了满足需要在环境之上开发的所有的测试用例的不同验证需求，验证环境的架构对照设计处理的数据的不同的抽象层次，当从低层抽象时，每一层给更高的抽象层次或者测试用例提供大量业务接口，可以供其调用。

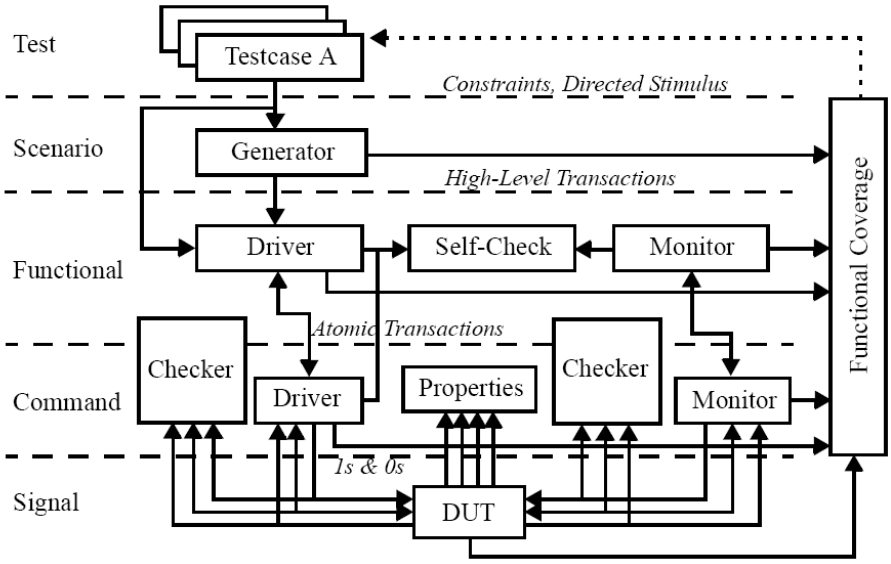


图3-8 分层的验证环境

尽管测试用例只在验证环境较高的层次相互作用，为了实现目的，他们可以旁路掉中间任何层次和环境中的任何组件或者 DUT 直接交互。测试用例就是对 generators 增加额外的约束，定义新的随机场景，在 transactors 之间的同步，插入错误的使能，DUT 的状态监测和直接测试的应用等等操作的组合。为了不用做任何修改地支持所有用于测试 DUT 的测试用例，测试环境必须包括精心设计的，可重用的验证组件。

完整的验证环境从来都不会是一蹴而就的。他们不会是作为一个成品提供给测试用例编写者的，相反，为了满足越来越多的复杂的需求，环境是不断完善的。最初，只需求支持简单的直接测试，或者也可以没有自动校验，逐步地，添加不同的层次。但是必须能够做到反向兼容（back-forward），新增的内容不会破坏现有的已经开发过的测试用例正常运行。

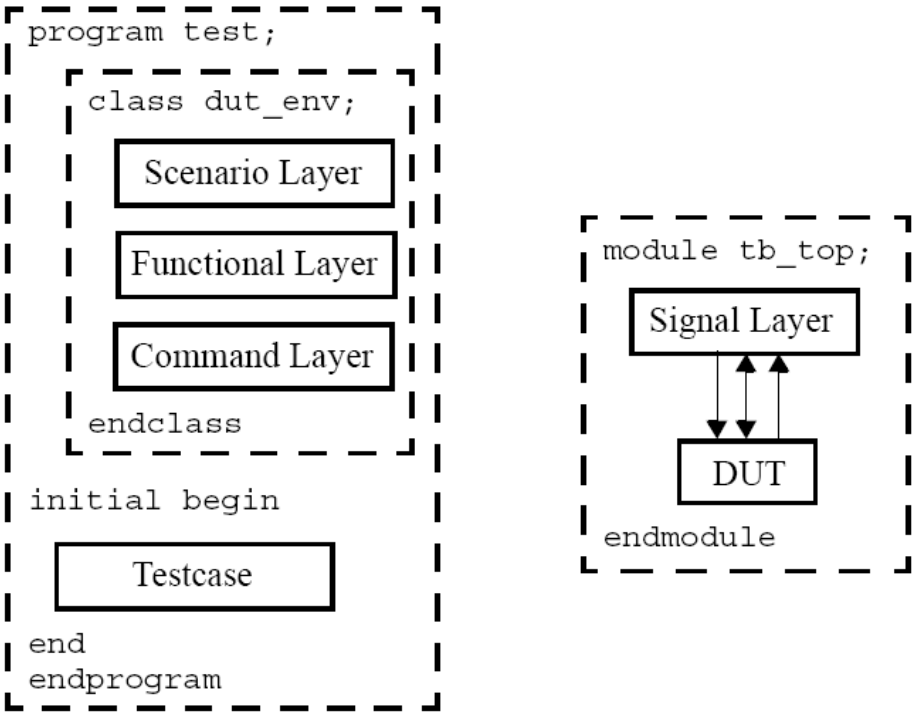


图3-9 测试平台的具体实现

图 3-9 显示了 VMM 验证方法学推荐的分层的测试平台，主要包括了 program block 和 test_top module。program block 包括验证环境类 env class（又包括 Command Layer、Funtion Layer、Scenario_Layer）和具体的 Testcase；test_top 包括 DUT 和与 DUT 相连的 Signal Layer）。

XIV. 信号层

信号层提供与 DUT 之间的物理信号级的连接。信号层提供 PIN 名抽象，为了使得验证组件可以在不同的 DUT，或者相同 DUT 不同实现模型之间，可以不被修改地重用。例如，一个使用 interface 结构的 RTL 描述和相同一个 DUT 的门级网表用独立的 I/O 信号。

信号层也可以用来实现抽象同步机制，或者实现跟一个参考信号同步。

信号层之前的所有更高层和测试用例都可以通过通过层次调用来访问信号层中的信号。然而，验证环境和测试用例如果不是绝对的必要，应该尽量避免之间访问信号。如果过多的更高层次直接访问信号层上的信号，那么分层的验证环境就不是那么特别有优势，采用分层的测试平台，就是希望把对物理信号层信号的操作局限在命令层和信号层之间，更高的抽象层次可以实现基于事务级的更高级功能。

XV. 命令层

采用 Verilog 或 VHDL 写的事务级的测试用例，通常在命令层上实现。

命令层一般包括总线功能模型，物理层驱动器，与 DUT 包括的不同接口和物理层协议相关的检测器和监视器。命令层提供了一个不需要考虑 DUT 是如何具体实现的，稳定的与 DUT 的底层事务接口。命令层的事务定义为一个原子数据或者一个接口上的命令操作，例如一个寄存器写操作，一个 Ethernet 帧发送，或者一条指令的获取。

读写寄存器是一个原子操作的例子。命令层提供访问 DUT 内部寄存器的方法。为了加速设计初始化，这一层需要提供可以旁路掉物理接口直接访问 DUT 内部寄存器的能力。

命令层的 transactor 主要可以分为 3 类：proactive、reactive、passive。

一个 proactive 驱动器能够控制事务产生的初始化和类型，能够主动地产生给 DUT 的激励。无论何时，验证环境中高层产生一个新的事务给 proactive 驱动器，事务立即可以在物理层接口被执行。例如，一个 AMBA AHB 接口的 master 总线功能模型就是一个 proactive 驱动器。

一个 reactive 驱动器不能控制事务产生的初始化和类型，但是可以控制执行的时序，例如引入等待状态。事务被 DUT 发起，reactive 驱动器提供需要的数据完成事务。例如，一个程序存储器的总线功能模型就是一个 reactive 驱动器：DUT 发起读周期获取下一条指令，总线功能模型提供新的数据以译码指令的模式。

一个 monitor 记录观测到的高层事务时序和数据信息。一个 reactive monitor 包括产生所需要的终结一个接口和成功地完成一个事务的低层握手信号。跟 reactive driver 不一样的是，一个 reactive monitor 不产生事务级信息。例如，一个 Utopia Level 1 接收器是一个 reactive monitor：他接收 ATM 信元的时候，只需要产生一个信元使能信号给 DUT 用做流控用途，而不需要产生额外的数据。一个 passive monitor 只是收集事务中的所有的数据。一个 passive monitor 适合用于在一个系统级的验证环境中，监视 2 个 DUT 模块间的接口时序是否满足要求。

命令层的一端与信号层相连，处理物理层接口信号，产生符合设计要求的时序；另外一端提供标准的事务级接口，将底层物理信号与更高抽象层次分割开，使相同的验证环境和测试用例可以用来测试 DUT 的不同模块。

例如，DUT 包括 GMII 和 MII 接口，GMIIBFM 和 MIIBFM 对应命令层 transactors，这两个 transactor 的物理层接口是不同的，但是事务层接口是相同的，都是接受符合 Ethernet 协议的事务包。这样，验证环境中命令层以上的组件在测试 GMII 和 MII 接口功能时可以完全重用，不同的就是命令层的 transactors。

XVI. 功能层

功能层提供处理应用层事务和验证 DUT 正确性的必须的抽象层次。

跟基于接口的物理层事务不同的是，功能层的事务可以不是和一个接口或者物理层事务一一对应的。功能层事务是在物理层接口之上的高层操作的抽象。一

个简单的功能事务可能需要执行很多条在不同的接口命令层的事务。可以依靠某些物理层事务的完成状态来重传某些事务或者延迟他们。

功能层 transactor 可以是 proactive，reactive，passive。一个 proactive transactor 控制事务的初始产生和事务类型，通常典型操作是提供 transaction 需要的所有或者部分数据；一个 reactive transactor 不能控制事务的初始产生和事务类型。它只能用来在适当的时刻终结事务，通过提供响应数据或者握手。reacive transactor 也可以报告他们观测到的数据。Passive transactor 监视接口上的事务和简单的上报观测到的事务。

功能层的自检结构在仿真开始后，就一直基于配置和输入激励的码流根据验证的功能测试 DUT 响应的正确性。正确性的判断可以由不同的抽象层次给出，物理层或者功能层。

根据不同协议结构，功能层应该再细化出子层。例如，一个 TCP/IP over Ethernet 的功能层应该包含一个传输 Ethernet 帧的子层。额外的子层可以用来封装 IP 段到 Ethernet 帧，大段的 IP 帧分割成小段的和封装一个 TCP 包到一个 IP 帧中。

测试用例在合适的抽象层次执行功能事务，而不是总采用低层物理事务或物理信号。必须可以关掉全部或者某些子层。当开始测试的时候，我们最先关注的是验证 DUT 的低层操作。这些低层操作相对功能层的最低的子层。

不同的抽象子层的激励和自测结构可以很方便地产生不同的场景和 coner cases.

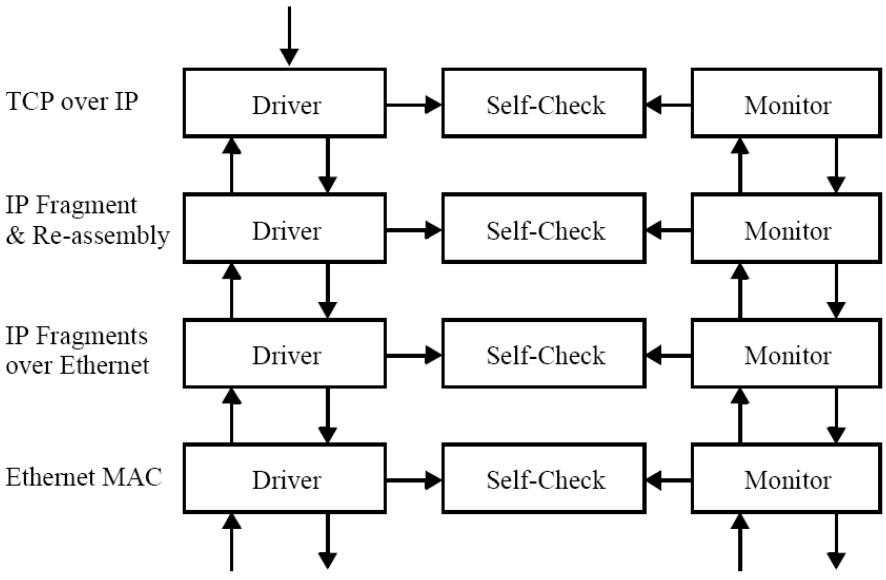


图3-10 功能子层的划分

为了避免影响已有的测试用例，需要较高的功能子层可以关掉。这个需求通常是验证环境开发中自身产生的副产品：环境通常是自下向上实现的，较低层次的测试用例首先被开发。当其他层次的功能需要被验证时，在功能层上增加子层。为了维护反向兼容与已经存在的底层测试用例，这些后加的子层必须默认是关掉的。

功能层也负责配置 DUT 根据配置描述符。配置描述符是一个 DUT 配置的高级的描述，它被“编译”进特定的寄存器读写，或者嵌入到存储器中。

功能层也包括一个功能覆盖模型用来收集高层激励和响应的事务。他记录相关的信息在这一层上执行的所有的事务的。

XVII. 场景层

场景层提供可控的和可同步的数据和事务产生器。默认，他们产生给 DUT 的激励，不同的 generator 和 managers 用于给不同的功能子层提供数据和事务。也包括 DUT 配置产生器。

Atomic generators 产生单独的可约束的事务。Atomic generators 适用于不需要约束事务的顺序的激励。例如，配置描述产生器是一个 Atomic generators。

Scenarios 是一些有着某种关系的随机序列。每个 scenario 代表为了满足特定功能 corner case 的一个有意义的序列。例如，在 Ethernet 网络中的一个场景是有特定密度的一系列帧，

XVIII. 测试用例层

测试用例包括调整产生器的约束，新的随机场景的定义，不同 transactors 之间的同步和直接激励的产生。

测试用例层也可以提供其他的跟测试用例相关的自测，在事务层的功能层不能提供的。例如，依赖于跟同步事件相关的检测。

(2) 仿真控制

通常，一条测试用例的运行需要执行下面几个主要的步骤：

A．产生测试用例配置

包括验证环境的配置和 DUT 的配置，也包括测试用例执行的持续时间的设置。主要用来决定验证环境怎样配置 DUT 和自检结构如何决定合适的响应来期望。

B．根据产生的测试用例的配置建立 DUT 周围的验证环境

为了正确地测试 DUT，在 DUT 周围实例的确定类型的和数目的 transactor 依赖于将要用到的配置。例如，一个 DUT 可以配置成 INTEL 或者 MOTO 的处理器接口。每一种需要不同的命令层 transactor。同样地，16 根 GPIO pin 可以配置成 16 个 1bit 的接口或者 16bit 接口。每一种配置需要不同的命令层和功能层 transactor 和 scoreboard 和自测结构。

C．禁止所有的断言和复位 DUT

D．根据产生测试用例的配置信息，配置 DUT

这里的配置包括写特定的值到 DUT 的寄存器中，或者设置接口信号到特定的电平。

E．使能断言，启动环境中所有的 transactor 和 generator。

transactor 和 generator 不应该一实例就启动，DUT 必须先被配置，准备后正确接收任何激励。Generator 启动的太快可能会使得响应检测变得困难，因为某些初始机理需要被丢弃掉。

F．检测结束条件

一条测试结束可能由几个条件决定。依赖 DUT，一个测试用例在运行固定的时间或周期数或事务数或产生某些错误信息后结束，或者当所有的监视器都 IDLE 的时候。

G．按一定的顺序停掉所有 generator

H．排干 DUT 内部数据，收集状态信息

为了确定仿真的正确，需要排干 DUT 内部缓存的数据，和读取计数或者统计寄存器。任何留在 scoreboard 中的期望数据应该被丢弃。统计计数器的值与期望的值比较。

I．报告仿真运行的成功还是失败

不是所有的 DUT 都需要这些步骤。一些步骤可能对某些 DUT 是无关紧要的。另外一些可能是很复杂的，但是每一个成功的仿真遵循着普遍的步骤。

如图，vmm_env base class 将这些仿真步骤固化到定义好的虚方法中。这些方法必须在验证环境中被扩展，为了实现 DUT 相关的需求。vmm_env base class 支持通过扩展每一个虚方法实现 DUT 所需要独立的仿真步骤。基本类也包括管理仿真步骤的顺序和执行的功能。DUT 相关的环境类扩展也实例和连接了所有的 transactors，generators 和自测结构来构成了一个层次化的验证环境。

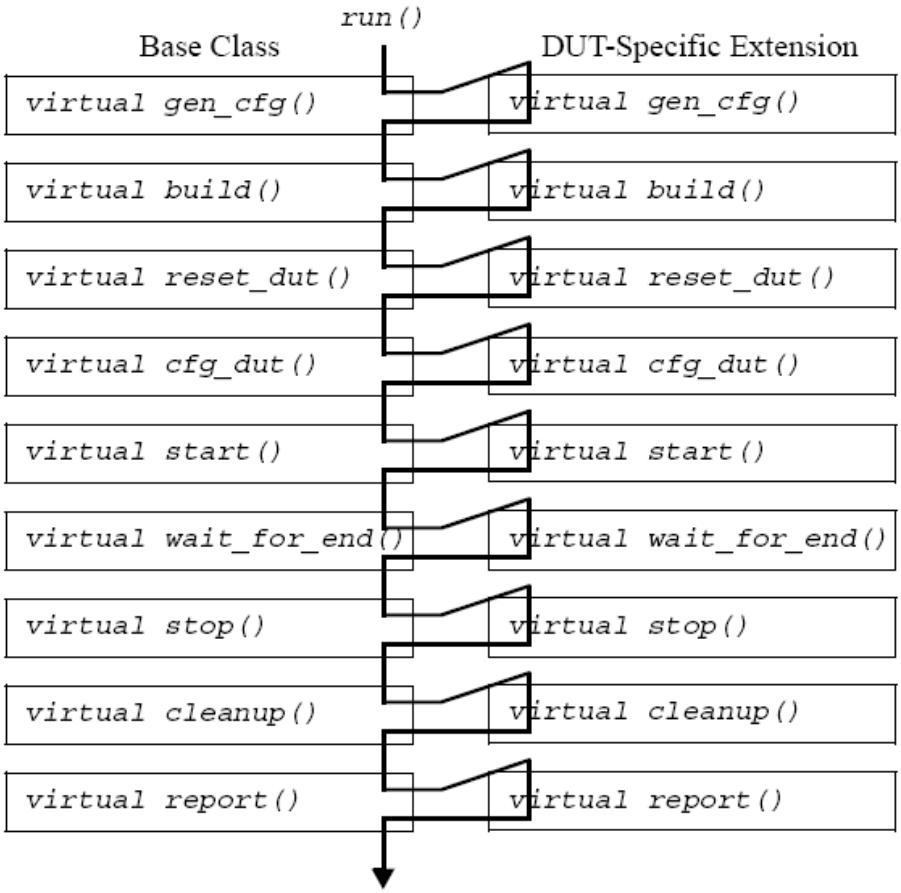


图3-11 vmm_env base class

(3) 数据和事务

从一种过程语言切换到一种面向对象的语言（例如从 Verilog/VHDL 切换到 SystemVerilog），如何有效地利用面向对象的特性建模是一个挑战。VMM 方法学中建议使用 class 对数据建模，如何使用 SystemVerilog 的面向对象的特性请参照 SystemVerilog LRM。

(4) Transactors

Transactors 指的是用来定义实现两个描述特殊的协议或产生协议事务的抽象层次之间接口的验证组件。Transactors 在仿真过程中的生命周期是静态的，在仿真开始时产生，一直持续到整个仿真过程结束。它们在验证环境中的位置类似与 DUT 中的 module。相比较 transaction 就是动态的，generators 产生很多的 transaction，流过 transactors，被处理、上报和比较，然后释放掉。

事务级的总线功能模型叫做命令层 transactors。命令层 transactors 的一侧是事务层的接口，另外一侧是物理层的接口。功能层和场景层的 transactors 只有事务层的接口，没有和物理层的直接接口。

开发 transactors 时，要注意可重用性、可控性和可扩展性，但是这些特性不是目的。这些特性使 transactors 可以被不同的测试用例和不同的验证环境重用；可以使 transactors 为了满足不同测试用例的特殊需求可控；可以试 transactors

为了满足特定的验证环境扩展出需要的特性。可控制性和可扩展性必须保证不能与 transactors 中已经实现的功能有冲突，避免需要修改已经验证过的测试用例。

(5) 事务级接口

事务级接口将抽象层次更高的组件与物理接口信号隔离开，在命令层上的 transactors（drivers、monitors），事务级接口可以允许验证环境中更高层的事务通过定义哪些事务需要传递给哪些 transactors，或者那些 transactors 需要及时采样 DUT 接口的输出。

事务级接口将验证环境中抽象层次比较高的组件同物理接口分离。在功能层的 transactors，一个高层的事务级接口用于接收将要被执行的高层事务的描述。这些高层事务在 transactors 中被转换成一个或多个较低层次的 transactions，一般是不同类型的 transactions。这些较低层次的 transaction 传递给一个较低层次的 transactors，可以是功能子层，或者是命令层，通过一个较低的事务级接口。例如，一个 IP 包 transaction 可以被一个分段 transactor 分割成一个或多个 IP 段。事务级接口用来在 2 个 transactors 或一个直接测试用例和一个 transactor 交换 transaction 的结构。

事务是通过描述符定义的，两个 transactors 之间通过一个管道交互事务描述符。两个 transactors 之间或者测试用例与 transactor 之间通过指向同样一个管道建立连接。如图 3-12 所示。VMM 方法学中通过 vmm_channel base class 实现这个管道，或者也可以直接用 mailbox 实现。

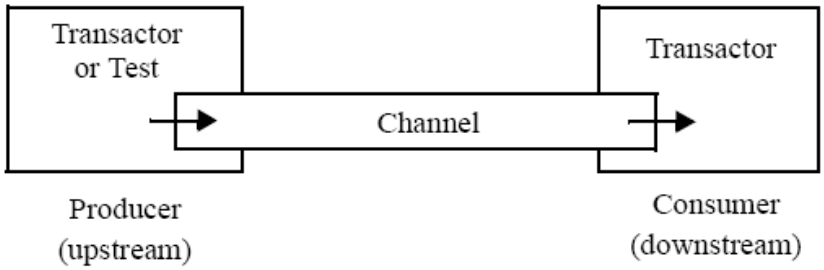


图3-12 事务级接口的连接

(6) 时序接口

在某些协议检测和环境中，高层 transactors 需要等待某个信息或者事件发生后，才能执行下一步操作。这就需要两个 transactors 之间具有跟时序相关的接口机制，VMM 方法学中提供了 vmm_notify base class 用来扩展通知业务接口，来实现不同 transactors 之间的同步功能。如图 3-13 所示。

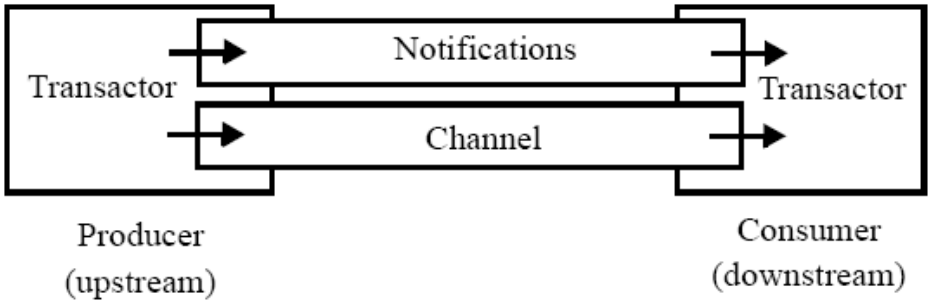


图3-13 通知接口

(7) Callback 方法

Transactor 的行为必须是验证环境和测试用例可控的，而不用对 Transactor 做任何的修改。在 Transactor 被开发的时候，这些需求经常是不明确的。通过在 callback 中允许加入用户定义的代码，Transactor 可以满足验证环境和测试用例的需要。例如，callback 方法可以用来实现监测一个 Transactor 中处理的数据是否正确，或者在数据中加入某种错误，或者收集覆盖率信息等等。

通常 generator 产生的 transaction 也是 transactor 中的 callback 方法传递给 RM 或者 scoreboard 的。

transactor 在接收完数据后应该调用一个 callback 方法，目的是让用户可以记录、修改或者丢弃接收到的数据。

无论是一个 transaction 描述符，还是在物理接口上采样到的一个数据，为了记录，或者给一个 scoreboard 检测，或者修改，插入一个错误，或者收集覆盖率信息，新获得的数据应该通过一个 post-reception callback 方法，上报给用户。

在 transactor 发送数据前，应该调用一个 callback 方法，目的是让用户记录，修改和丢弃数据。

无论是一个 transaction 描述符，还是在物理接口上驱动一个 BYTE 数据，为了记录，或者给一个 scoreboard 检测，或者修改，插入一个错误，或者收集覆盖率信息，新获得的数据应该通过一个 post-reception callback 方法，上报给用户。

(8) AD-HOC 验证平台

前面提到的验证平台基本组件是很灵活、易于升级的和强大的，最大限度地利用了 SystemVerilog 中面向对象的特性去产生可重用的验证组件。在动态仿真环境中仿真步骤也是定义好的。

然而，不是所有的测试平台都需要利用所有的这些灵活性、可升级性和可重用性。

必须能够快速搭建那种只用一次就丢弃的，能立刻验证到某些功能的 ad-hoc 验证平台。通常是由设计工程师，产生一些简单的测试激励验证一个单元的基本功能。响应一般是通过肉眼观测。

这样简单的测试平台，不需要完整地控制整个仿真步骤，动态重配置 transactors，不需要功能层的多个子层，不需要集成一个自测结构，不需要采样功能覆盖信息，也不需要随机产生大量数据。用 VMM-compliant 的 transactors 去产生这样简单的测试平台有些大材小用。

(9) 现有的总线功能模型

对一些继承来的 BFM 的处理过程和 AD-HOC 验证平台的处理正好相反，AD-HOC 验证平台是将 VMM-Compliant 的验证组件简化，而对那些继承过来的，需要应用才 VMM 环境里的 BFM 的处理是将以 modules 形式存在的组件更新到以 class 形式存在的组件。

(10) 总结

本章主要介绍了为了建立验证环境，我们需要关注那些必需的验证组件。层次化的验证结构为高级抽象层次的出现提供了机会，也鼓励为了提高模块级和系统级之间、事务级和实现级验证环境之间的可重用性，分解独立的协议功能。消息业务和仿真步骤管理的基本类使仿真信息和仿真控制标准化。

基于对数据流（transactions）操作的 transactors，采用了定义清楚地的事务级接口。Transactors 的这种结构实现了真正的层次化环境，也使得能够产生相比我们现有传统的总线功能模型抽象层次更高的验证组件。通过打破总线功能模型也物理层接口的紧密联系，transactors 可以在逻辑层开发、操作和被重用。

5. 激励和响应

验证计划阶段，会产生 3 类主要的需求：功能覆盖、激励产生和响应检测。这一章节主要关注后面两类需求。功能覆盖在后面的章节再讨论。

测试用例是通过直接或随机测试用例实现的。直接激励可以认为是符合某种约束的随机激励的子集。随机激励必须能够产生所需的测试计划中提到的所有的测试需求。

(1) 产生激励

数据（packets、frames、instructions）和事务描述符根据他们各自生命周期在验证环境中被实例。在通常的仿真环境中，会产生成千上万的数据和事务，流过各种 transactors，在自测结构中被比较和上报。另一方面，只有少数的数据和事务在仿真开始的时刻就产生，并且一直存在到仿真结束。

(2) 控制随机产生器

随机产生器的目的是能够产生可以完全验证设计的所有需要的激励。某些激励可以不用约束，某些激励需要额外的约束才能测试到某种 corner case 或者插入错误。

产生某种激励的能力跟能够表述约束产生那些激励的能力是直接相关的。如果不能表述清楚两个变量之间的约束，不可能产生这两个值之间的关系。

随机产生器主要分为两类：atomic generators 和 scenario generators。

atomic generators 用于产生无明显顺序关系的 transaction；scenario generators 用于产生具有特定顺序关系的 transaction。

(3) 自检测机制

因为每一个自检测结构对所验证的设计来说都是唯一的，基本上很少有功能可以在不同的自检测模块间重用。

对于自检测结构，我们已经在 2.2 节提到了很多，这里就不再重复。

6. 覆盖率驱动的验证

通常，覆盖率统计是作为一种建立信心的测试，用来确认验证计划已经被贯彻执行和设计已经尽可能地被充分验证到。覆盖率测试一般是在已经开发出了大量测试用例，在验证过程的后半段阶段开始做的，通常希望覆盖率统计开始会比较高，显示之前几个月开发的验证环境是有效的。项目剩下的时间是调整测试激励去满足覆盖率的进一步要求。

但是如果覆盖率测试不象我们预期的那样，刚开始就比较低，那意味着我们花了很多的精力开发的验证环境效率很低。

就算刚开始覆盖率就比较高，验证团队或许可以比较欣慰，但是如果用较少的测试环境也许就可以达到同样等级的覆盖率了，那又说明什么问题呢？说明我们验证环境的低效，验证精力的无效投入。

覆盖率度量能够指导验证精力的投入。让我们把验证的精力投入到没有覆盖到的验证需求上。因此，开发和维护一个高效的覆盖率模型虽然需要花费一定的验证投入，但是还是很值得的。

(1) 覆盖率测量

覆盖率测试是对收集的覆盖率数据的测量，通常以百分比的形式给出。覆盖率数据用仿真或静态分析工具收集在一个或多个数据库中。覆盖率针对每个覆盖的功能点，分析或上报工具比较收集到的数据，通常合计多个仿真或静态分析结果。如果收集的数据完全覆盖了目标，覆盖率测试将会上报 100%。

多个 coverage point 的测量通常提取为一个单一的覆盖测试用不同的权重表示。

覆盖率目标在验证项目中不总是静态的。在验证计划中会提出一组覆盖率目标。但是当测试用例开发出来后，这些目标被新的 corner case 和新出现的条件刷新。

(2) 覆盖率模型

覆盖率模型包括了整个功能验证过程的所有的需求。一个复杂设计的全部激励和响应空间是多维的，几乎是无限的。因此，期望能够穷尽验证到所有可能的组合和激励序列和响应是不现实的。覆盖率模型是在某个可信度量级上的，可以认为设计的功能是正确的，将要被验证的激励和响应空间子集。

覆盖率又可分为结构覆盖率和功能覆盖率，结构覆盖率又可细分为状态机覆盖率、表达式覆盖率、交叉覆盖率和断言覆盖等等。覆盖率测量的收集与覆盖率模型是独立的。覆盖率测量数据来自仿真、形式验证分析和结构分析工具。

(3) 功能覆盖率实现

结构覆盖率统计可以用工具自动的产生和收集，只需要在仿真脚本中增加几个命令行就可以，不需要验证工程师额外的工作量。Synopsys 公司之前结构覆盖率的统计的工具是 CoverMeter，目前推荐使用 URG 和 DVE，可以收集结构覆盖率和功能覆盖率信息。

功能覆盖率需要我们手动定义和添加。功能覆盖率是对验证需求的功能建模，因为设计的目的不可能自动从设计实现中获取，功能覆盖率必须手动添加。

(4) 反馈机制

在 CDV 过程中，对收集到的数据的分析用来确定下一步关注的验证的重点在哪里。因此，需要一个反馈机制来把覆盖率模型中的没有覆盖到的区域，传递给额外的测试激励。

SystemVerilog 中有些结构可以动态地查询不同 coverage points 的覆盖率情况，例如$get_coverage、$get_inst_coverage，这些覆盖信息根据测试输入激励变化，可以在运行时用来改变，规定激励中的约束。这样一个仿真可以自动地关注覆盖空间中的没有覆盖到的区域，减少同样的 coverage points 被重复命中的可能性。

这是理论，事实上，这样操作有一些困难。

首先，对没有覆盖的区域的关注，只对输入的覆盖率有效，这种测量跟特定的 coverge points 和激励的约束直接相关。这种关系使得对内部 coverage points 和输出的覆盖的测试没用。

此外，自动反馈机制必须确保激励的产生是向着更高覆盖的方向，目的不是减少测试平台的数目，而是减少达到期望的覆盖率目标所需要付出的精力。采用更多的较简单的测试平台可能比采用一个复杂的平台更容易达到。具体选用哪种方式，要具体情况，具体分析。

(5) 总结

这一章介绍了覆盖率度量的概念和怎样产生一个覆盖率模型。覆盖率模型是验证需求的具体化。覆盖率模型包括了结构覆盖率度量和功能覆盖率度量。结构覆盖率度量可以自动的收集和测量，而功能覆盖率度量是对验证目的的描述，所以功能覆盖率度量必须手动开发。

功能覆盖率采用 coverage groups 和 coverage properties 实现，收集覆盖率数据时，确保数据就是从 DUT 中观测到的。

覆盖率度量必须设计成能够简化目标的分析。从一个覆盖率报告中，可以很容易的判别哪些测试用例需要被产生。从功能覆盖率度量反馈到测试用例中可以

是自动化的。但是在大多数测试用例中，一个对目标 hole 采用不同的收敛机制智能的反馈过程是更有效的。

7. 总结

VMM 验证方法学部分很大一部分是对《Verification Methodology Mannual for SystemVerilog》一书前半部的翻译，由于译者水平有限和篇幅限制，建议有兴趣的同事可以参考原版。

3.1.2 软硬件联合验证

嵌入式系统是为专门应用而设计的计算机系统，通信系统中的计算机系统大多属于嵌入式系统。典型的嵌入式系统由微处理器、存储控制器、其它功能 IP 以及外设接口等部件构成，这些部件之间通过公共总线通信；除了硬件以外，嵌入式系统还包括运行于硬件之上的软件，典型的为操作系统、驱动程序、应用程序等，其是软硬件的有机结合体。

传统的嵌入式系统的设计周期包括下列几个阶段：

（1）高层系统设计

确定系统需求、子系统划分、软硬件功能划分以及软硬件接口确定等。

（2）详细设计和实现

在这个阶段，硬件和软件依据接口规范分别进行设计。硬件设计人员设计原理图，然后设计 PCB 板，加工电装成硬件样机，对样机进行简单的硬件调测；与此同时，软件人员进行编程，并进行纯软件调试。

（3）软硬件集成

在硬件样机和软件单独调试就绪后，就可以将软件加载到硬件样机上，进行各项系统调测发现并修正硬件、软件的设计缺陷，遇到严重的硬件设计问题可能导致重新进行硬件样机设计。

上述传统设计的主要缺陷是：在硬件样机就绪之前，软件和硬件的设计和调试基本上是相互隔离的；这使得一些设计问题直到系统仿真时才被发现，导致设计反复，设计周期变长。对集成 CPU 的片上系统（SOC）而言，由于建造硬件样机困难、设计流片周期长、较高的 NRE 费用等因素，使得问题更为严重。

软硬件协同验证工具的引入，可以基本解决传统嵌入式系统设计流程的上述问题。通过建造虚拟器件（处理器、RAM 等），既使没有实际的硬件样机，也可以对虚拟硬件加载程序进行软件和硬件（SW/HW）的联合仿真，及早发现软件和硬件的设计缺陷，从而缩短设计周期，降低设计风险和开发成本。据报道，采用协同验证工具可以使 SOC 设计周期缩短 20%以上。协同验证工具同样可以用于板级嵌入式系统开发。

协同验证工具涉及软件和硬件两大环境，要求使用者具有下列预备知识：

- 逻辑仿真器（如 NC-verilog、VCS）

- 软件调试工具（如 Microtec XRAY）及高级编程语言（如 C）；
- 所用 CPU 的体系结构及指令系统等。

1. Seamless CVE 介绍

Seamless CVE 软硬件协同验证环境架构如下图所示。它将协同验证 kernel 与存储一致性 server、软件仿真器（ISS）/主机代码执行器（HCE）、逻辑仿真 kernel 三大部分有机地组合在一起，构成一个统一完整的软硬件协同验证环境，且各部分都有对应的图形用户界面与用户直接交互。

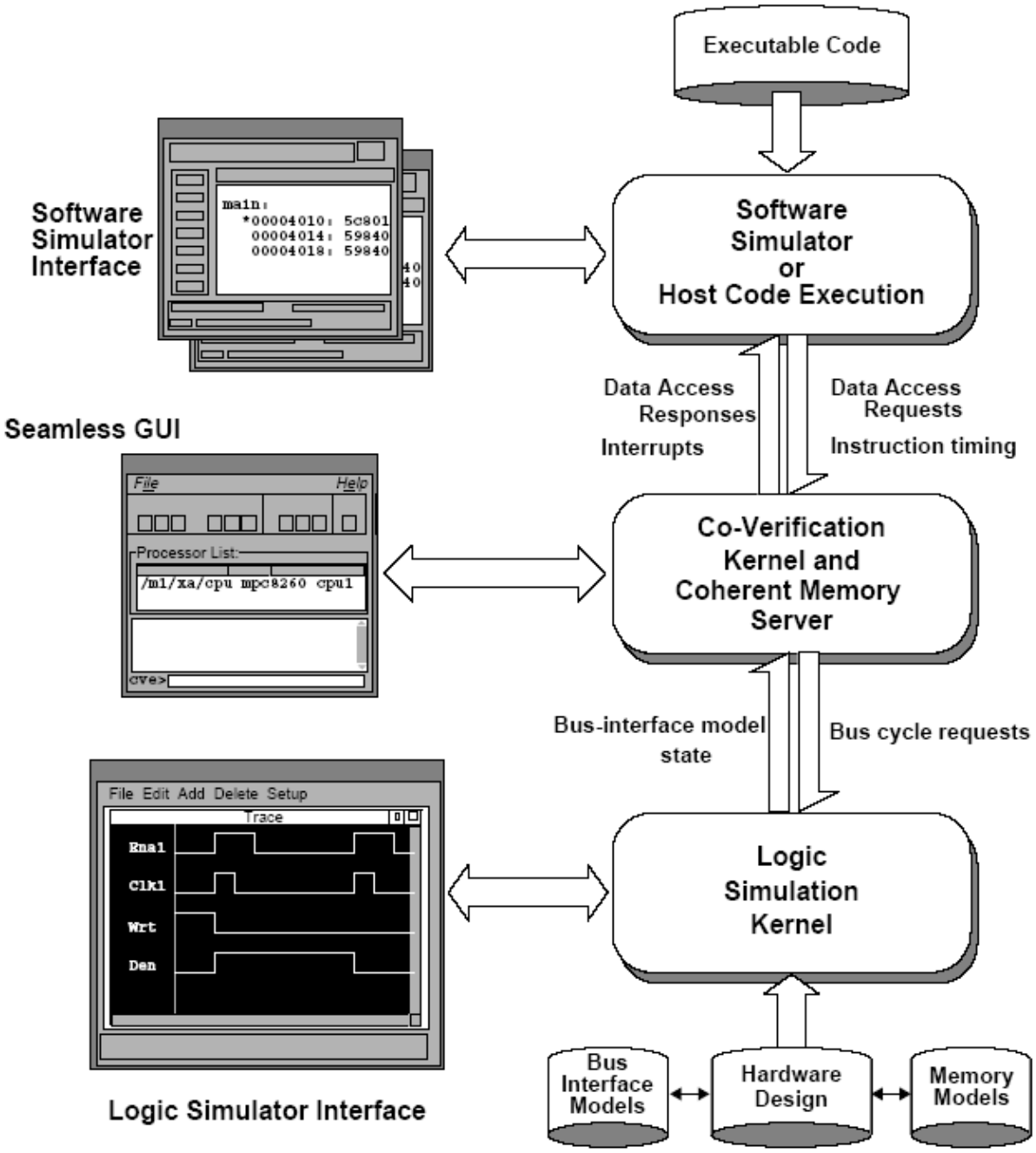


图3-14 Seamless CVE 软硬件协同验证环境的组成

(2) 协同验证 kernel 与 memory 一致性 server

协同验证 kernel 是整个协同验证环境的核心，其在协同验证过程中控制软件和硬件两大部分之间的通信，提供用户设置协同验证参数的配置管理。协同验证参数设置既可以通过配置管理界面交互进行，也可以输入配置管理命令行实现。所有的配置管理接口命令都是基于 Tcl 的脚本语言。

其主要的任务如下：

- 允许用户通过配置管理器设置协同验证环境的各种属性
- 根据软件执行情况适时向逻辑仿真 kernel 发送总线请求
- 通过逻辑仿真 kernel 监视总线接口模型(BIM)的状态
- 可选择通过转发总线请求给总线接口模型(BIM)进行数据访问或者直接通过 memory 访问来响应总线请求而不必完成相应的硬件总线周期
- 接收指令集仿真器的数据访问请求并响应请求
- 监视指令集仿真器执行指令需要的时钟周期的数目
- 接收逻辑仿真 Kernel 的中断信号传递到指令集仿真器

Memory 一致性 server 是实现仿真加速的基础之一，在上面的任务（4）中，当软件仿真器直接访问了 memory 而没有在 BIM 上产生相应的总线周期时，有很可能导致软件、硬件之间看到的 memory 内容不一致，memory 一致性 server 就是自始至终保持两者的 memory 一致。所以这对 memory 的类型也提出了特殊的要求，必须使可优化的 memory 类型。这些在后续部分会进行进一步讨论。

(3) 软件仿真器（ISS）/主机代码执行器（HCE）

ISS 执行协同验证过程中的软件部分功能。

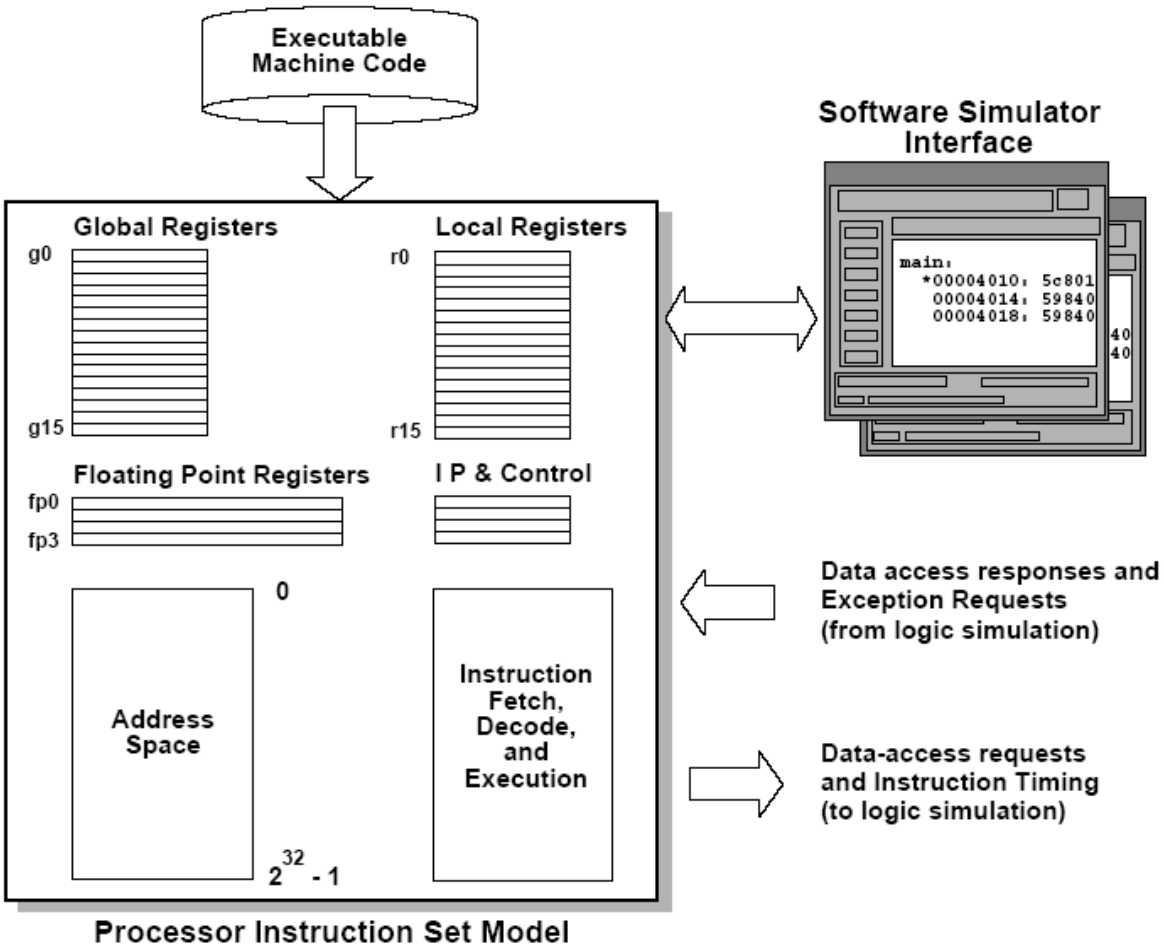


图3-15 ISS 基本功能示意图

指令集仿真器（ISS）是仿真处理器指令集运行的软件，是指令行为的抽象模型，由处理器支持包（PSP）提供。由于 ISS 不计算信号在处理器中的门和寄存器传递，属于纯软件仿真，它的运行速度比逻辑仿真器快得多（高约 5 个数量级）。用软件调试来替代处理器里的大部分硬件仿真来加快仿真也是目前所有协同验证工具的基本策略。ISS 运行对某种处理器进行交叉汇编或编译生成的机器码。

HCE 是用高级语言编写的、直接在工作站上编译运行的软件。HCE 比 ISS 执行速度快，但不如 ISS 具有通用性，部分处理器（如 ARM）尚没有提供 HCE。

软件仿真接口控制 ISS 或 HCE 的运行，如单步执行、设置断点、查看存储器和寄存器的内容等 Debugger 功能等，XRAY 就是一个典型的 Debugger 工具，是由 Microtec 公司（Mentor 下属的独立子公司）提供的，它也是 Seamless CVE 标准的 Debugger 工具。

(4) 逻辑仿真 kernel

逻辑仿真 kernel 和硬件仿真接口执行协同验证过程中的硬件部分功能。Seamless CVE 提供了各种流行的硬件仿真器的接口，如 ModelSim、Verilog-XL、VCS、NC-Verilog 及 Leapfrog 等；而硬件仿真 kernel 主要管理协同仿真验证环境中的硬件。这些硬件包括

了可寻址范围以内的总线接口模型（BIM）、存储器模型，以及待验证的硬件 RTL 设计等。如图 1 所示的逻辑仿真 kernel 部分。

总线接口模型（BIM）定义了一个处理器在逻辑仿真时引脚的行为，不包括处理器的内部逻辑，因此比全功能模型的仿真速度快得多，也由处理器支持包提供。Seamless CVE 存储器模型包括 DRAM、SRAM、FIFO、DPRAM 及寄存器等基本存储模型。这些模型的地址线、数据线宽及延时均是参数化的，用户在引用时可重新设定。与通常 RAM 模型相比，Seamless CVE 提供的存储器模型可受 Seamless CVE 核控制，在协同验证过程中能够进行优化加速仿真。

实际上 CVE 中的硬件仿真器除去和协同仿真 Kernel 通信的接口部分，和一般硬件仿真器无异。

(5) PSP 与 BIM

协同验证环境总是离不开相应的 CPU 子系统的支持。Seamless CVE 有相关的处理器支持包，简称为 PSP，各个 CPU 有其特定的 PSP。Seamless CVE 目前对 ARM、PowerPC、ZSP、TI 的 DSP 等多种 CPU 或者 DSP 都提供了相应的 PSP。

一个总线接口模型(BIM)的功能示意图如下：

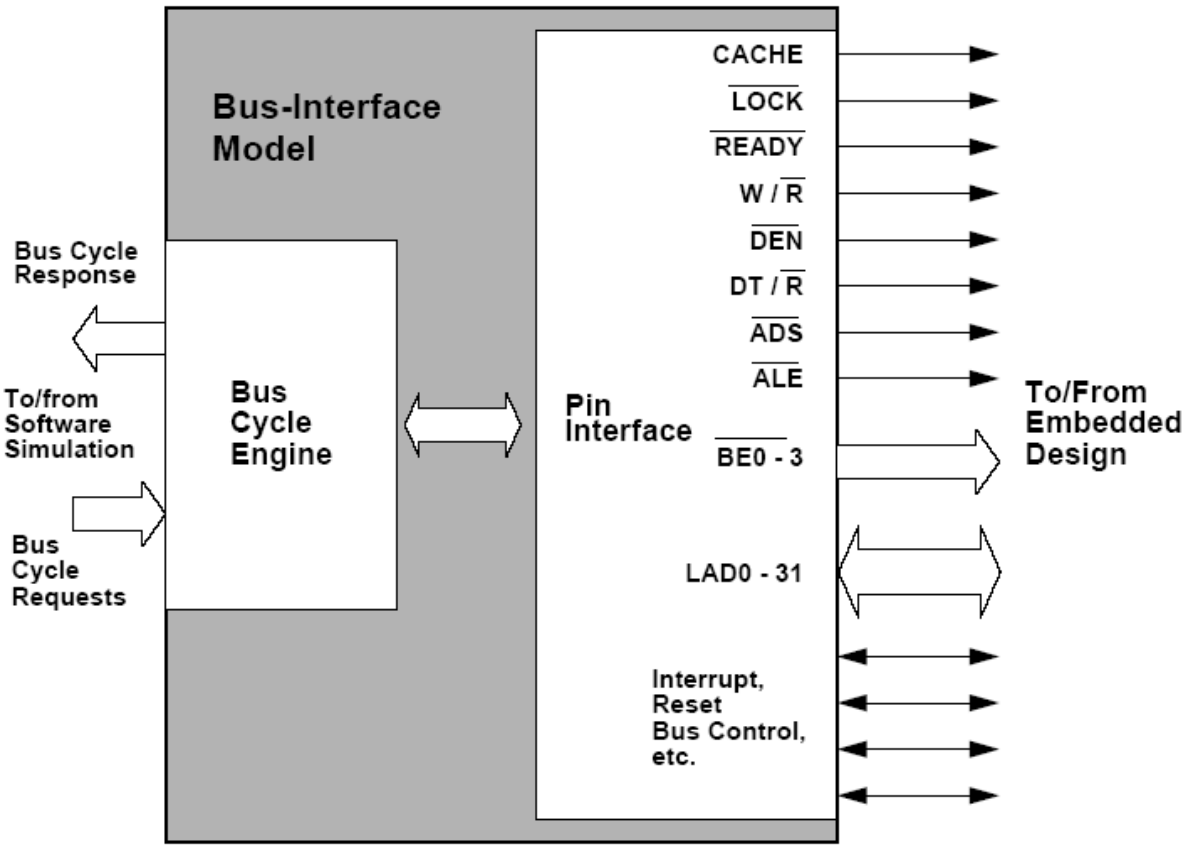


图3-16 BIM 模型示意图

在协同验证过程中，BIM 通过其 Bus Cycle Engine 接受 ISS 的总线周期请求，然后对相应引脚产生相应的逻辑动作。BIM 执行处理器各种总线周期，包括：存储器读写、I/O 口读写、复位和暂停、中断、总线请求和仲裁等。

对于硬件仿真来说，处理器表现出来的主要特性就是通过输入输出引脚和外部设备相互作用。硬件人员完全可将其看成是一个硬件仿真中用到的 CPU 的 BFM，此外由于 BIM 内部只抽象到引脚传输行为建模，所以比那种完整功能模型的仿真速度更快。

总线周期中总线接口模型具体的执行过程如下：硬件仿真 kernel 通过协同仿真 kernel 接收到了 ISS 的总线请求，于是就根据 BIM 预定的逻辑在相关引脚接口上进行对应的引脚信号翻转。如果相关引脚接口上接收到了外部的控制信号以及数据响应或者是中断信号，也根据 BIM 预定的逻辑判断以后通过协同仿真 kernel 向 ISS 传递相关的信息。

(6) 仿真加速与优化

整个协同验证的重点往往并不在 CPU 本身，而是在于与 CPU 构成一个系统的外部硬件电路是否工作正常和软件是否运行正常。然而对于 CPU 而言，软件处理在大多数的处理器总线周期过程中绝大部分都是在 CPU 内部访问或者是访问外部固定的内存空间(如程序存储器)，只有很少的一部分是在和其它硬件进行 I/O 数据交换和复位、中断以及异常处理。

协同仿真过程中影响系统仿真速度的关键路径就是硬件仿真器的仿真速度。如果能够忽略掉和外部硬件仿真并不相关的那些总线周期，就能加快整个系统的协同仿真进程。Seamless 的 CVE 主要采取以下基本策略达到仿真的加速：

- 将处理器的指令处理部分和接口部分分开。CVE 工具软件把处理器分成了指令集仿真器（ISS)和总线接口模型（BIM）两大部分。总线接口模型（BIM）完成处理器与其它硬件的连接。 指令集仿真器（ISS)完成处理器内部的处理。
- 硬件仿真器的仿真时间可控。不和 CPU 外部硬件（也可不包括程序存储器）通信的那些 CPU 总线周期在协同仿真过程中将被硬件仿真器忽略，从而加快硬件仿真器的速度。
- memory 一致性控制器（coherent memory server）。在 Seamless 中能够减少逻辑仿真器的活动从而加速仿真速度的一个重要因素就是可优化的 memory 模型的存在（或者说协同验证环境中特殊的 memory 环境与管理）。在验证过程中可以实例化 Seamless 的可优化存储模型，存储模型内的内容主要由 memory 一致性控制器进行维护，即：软件仿真器与逻辑仿真器都可以独立的访问该存储器，且 memory 访问结果一致。Seamless CVE 的 memory 系统示意图如下：

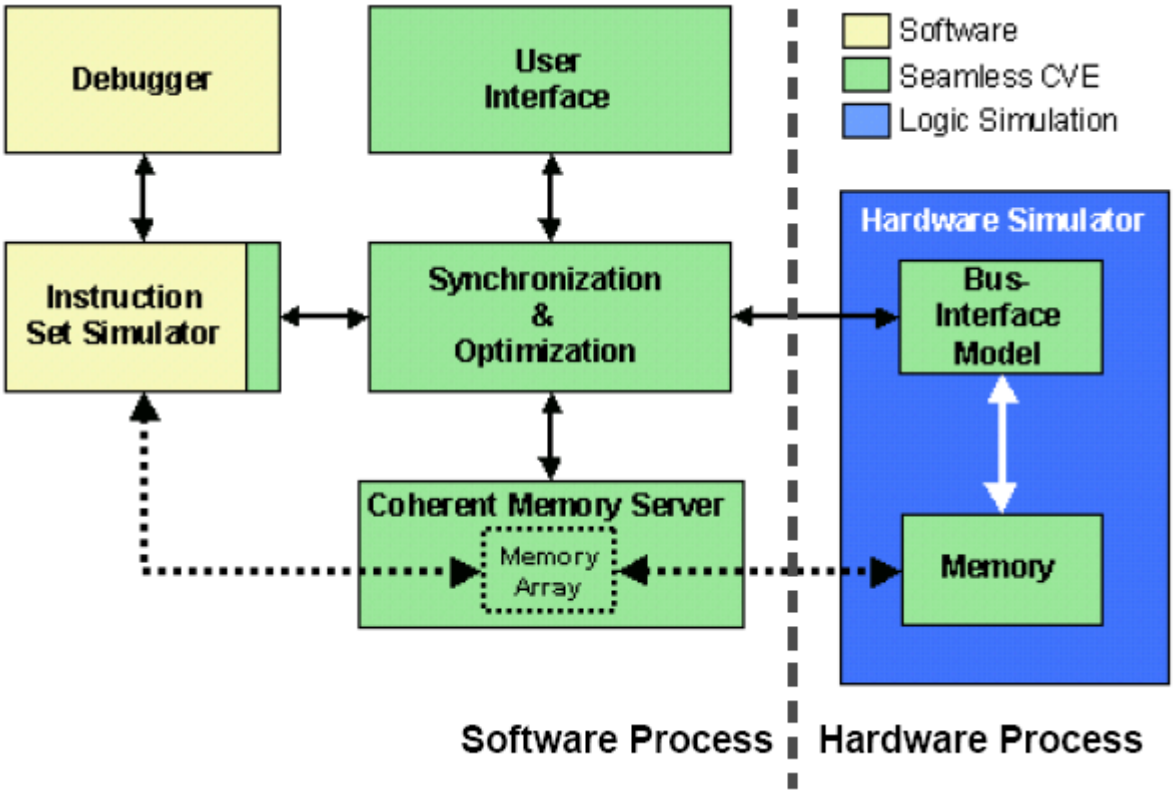


图3-17 Seamless CVE memory 系统

如上图所示，比起通过硬件仿真器来访问 memory 来说（实线箭头），软件仿真器就以软件仿真速度全速的访问 memory（虚线箭头），并且 CVE 的存储一致性控制确保：即使软件仿真脱离（decoupled）开了硬件仿真，两者的 memory view 也始终保持一致。

Seamless CVE 的另外一个重要特性就是对协同验证进行优化。优化也就是忽略了处理器中的一部分总线周期而把仿真时间主要用在 CPU 与外部电路交互中对于特定的地址段访问的总线周期上，达到优化硬件仿真过程的目的。以下面的小段程序为例：

Instructions and Data

Code Segment

```
00FE    0214 D6A3    LD SP, 6030
00FF    0000 6030
0100    0214 41A7    LS R5, @SP
0101    0214 42B8    LD R6, @SP+1
0102    351C 5C7D    JMPS 5C7D
...
5C7D    5260 E355    OUT 55, R3
5C7E    537C 2A05    IN @R5, 2A
```

Data Segment

```
6030    0080 0000
6031    1234 5678
6032    55AA AA55
```

Bus Cycle Sequence

Fetch @ 0100
Memory Read @ 6030
Fetch @ 0101
Memory Read @ 6031
Fetch @ 0102
Fetch @ 5C7D
IO Write @ Port 55
Fetch @ 5C7E
IO Read @ Port 2A
Memory Write @ 0080000

- 数据访问（Data Access）优化

数据访问优化允许在访问特定的地址段的时候禁止产生相应的硬件总线周期。在没有设定数据访问优化的情况下，每次数据访问都在 BIM 上产生一个完整的读写总线周期，也就是硬件仿真器也产生了一个完整的读写周期。但是这些数据访问周期绝大多数都只是对程序本身有用，对于与硬件设计没有太大关系，设计人员对此几乎不关心，完全可以忽略掉，除非是对读写指令执行的时间有特殊的限制。下图为没有经过任何优化时的硬件总线行为。

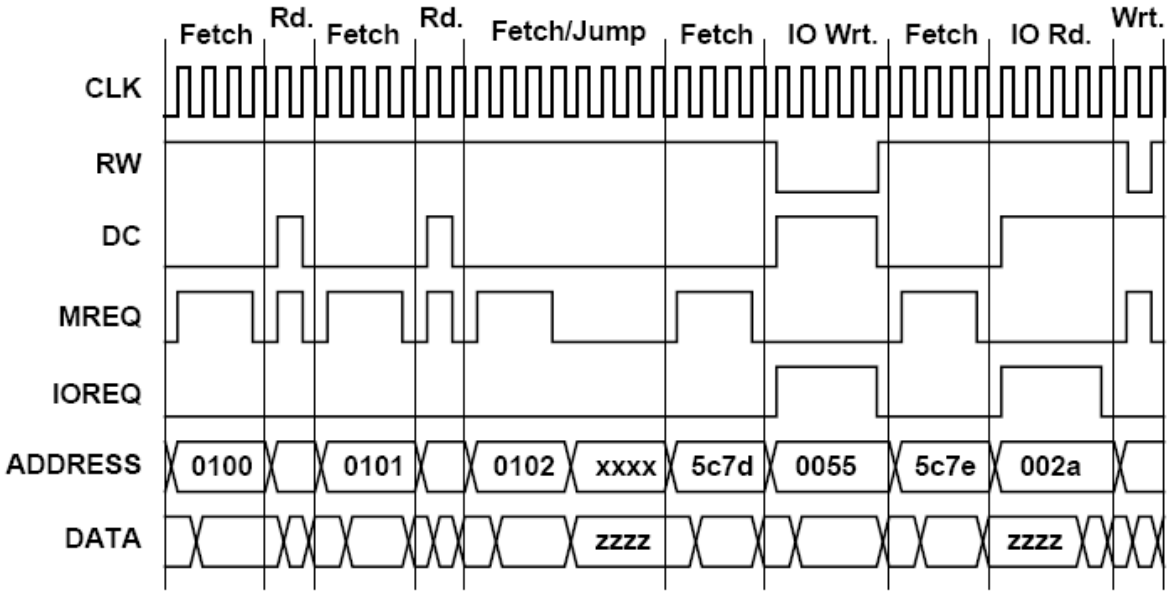


图3-18 未优化时的硬件总线行为示意图

假定仿真前已经设定了某个地址段进行数据访问优化。那么当 ISS 执行某个指令、刚好要对这一地址段的某个地址进行读写操作的时候，协同仿真 Kernel 并不响应 ISS 的请求在 BIM 上产生一个读写总线周期，ISS 直接从内存完成读写。考虑到时间的同步，协同仿真

Kernel 还是产生 CPU 的时钟周期，完成 CPU 的正常的总线传输逻辑，产生 CPU 的时钟周期的数目和没有加入取指优化的时候是一样的。

➢ 取指（Instruction Fetch)优化

这种优化可以在取指令的时候禁止产生相应的硬件总线周期。该优化必须使用 Seamless CVE 提供的特殊的可优化的存储器模型或者其它可优化 memory 模型（如 Denali 模型）。

在不进行取指优化的情况下，进行取指令操作的结果是处理器对外产生一个读操作总线周期，硬件仿真器也对应的产生一个相应的总线周期。取指优化使得硬件仿真器在取指令过程中不产生相应的总线周期。数据直接从内存读取。存放指令的存储器必须是 CVE 自带的可优化的存储器，才能够进行取指优化。取指优化是减少总线周期的有效方法，而且几乎不影响协同仿真的结果的准确度。

➢ 时间（Time）优化

时间优化打破了硬件仿真和软件调试之间的时间同步，可以适时的停止硬件仿真器的仿真，而让软件继续运行。

通常的数据访问优化，硬件仿真器不产生相应的总线周期，但是允许产生时钟以保持软件调试器的同步。时间优化可以将程序中可以不关心硬件时序的部分的硬件仿真忽略掉，设定了时间优化之后，硬件仿真器连时钟都不产生，这部分的程序执行在硬件仿真器里面是看不到任何波形的。该优化的设置以及应用场景相对麻烦得多，这里不作进一步介绍，如有兴趣可自行查阅 Seamless 的时间优化部分。下图为进行取指优化、数据访问优化、时间优化后的硬件总线行为示意图。

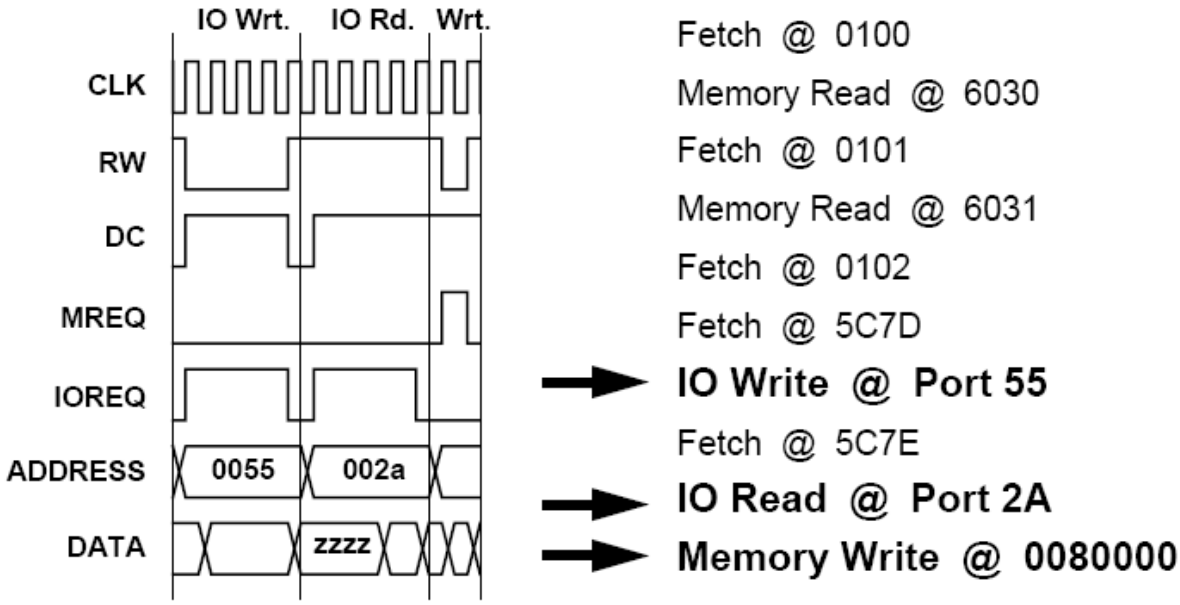


图3-19 优化后的硬件总线行为示意图

(7) memory 映射

存储器映射是 CVE 中的一项重要的配置。存储器映射的配置包含了两项内容：

- 存储器例化（Memory Instance mapping）。这主要是针对Seamless CVE的可优化的存储器模型的配置命令。这一命令通知Seamless CVE存在哪些可优化存储器模型及其数据线宽度和起始地址。
- 存储器访问映射（Memory Access mapping）。定义了存储空间中不同地址空间的访问权限。下表罗列了存储空间不同的访问权限配置。

表3-1 地址空间的访问权限表

访问权限配置类型	描述
Software-Only Access	仅软件访问。
Hardware-Only Access	仅硬件访问。这一地址不能做优化。
Optimizable	可优化。
illegal	非法，禁止访问。

(8) 可优化 memory 模型

出于优化的需要，Seamless CVE 需要可优化的存储器模型。这些模型和其它模型一样可以在一个设计中被例化并被仿真。然而这些模型也有着其特殊性，那就是在一个协同仿真的环境中如果允许优化加速仿真的话，那么软件调试器就都可以直接访问这些模型对应的内存空间取数据，而不仅仅是硬件可直接访问。普通的存储器模型只能被硬件仿真器所访问，软件调试器无法直接进行访问，是无法进行优化的！ Seamless CVE 提供了三种建立可优化的存储器模型的途径，如下表所示：

表3-2 可优化存储器模型

可优化存储器模型的使用途径	描述
Seamless CVE自带存储器模型	包含了一些常用的带参数的存储器模型，动态RAM，静态 RAM，双口RAM，FIFO，寄存器基本单元。
使用Denali Memory Modeler等工具产生一个可以优化的存储器模型	可以指定存储器的类型和制造商，存储器的特性，总线规格，时序参数。可定义的存储器模型比自带模型的种类要多。
HDL存储器接口	对于自定义的特殊的采用HDL编写的存储器模型，需要在HDL中加入调用相应的接口函数，变为可优化的存储器模型。

(9) 总结

以上只是对 Seamless CVE 的基本使用作了一个简单、概要性的介绍。如需要加深对其的掌握，除了 Seamless CVE 本身的用户手册以外，对于逻辑仿真器、XRAY 软件仿真回溯器的基本使用也需要加深了解。此外还应该加强对基本常用的脚本语言的掌握，以提高协同验证环境编写的效率。

3.2 验证语言

概述：本章介绍常用验证语言。包括 Specman E、SystemC；VERA、SystemVerilog。

3.2.1 SystemC

1. SystemC 简介

SystemC 是一种 C++的类库和方法，它由一组 C++的类库和一个仿真内核组成。其本质就是 C++，是一种专门为模拟硬件的 C++，可以用 systemC 建立在 C++中的硬件模型和软件模型来进行仿真。它利用了 C++的类扩展了一般 C++不支持的、但是在硬件仿真中所必须的功能，如：硬件数据类型中的位向量；利用进程来仿真并发性；引入了时钟的概念；采用信号来进行通信。 在 systemC 中，能与 HDL 语言一样使用 modules、ports 和 signals。模块能在其他模块中被实例化，从而建立设计的层次结构。

SystemC ＝ 标准 C++ ＋ SystemC 库 ＋ SystemC 方法。

SystemC 的仿真内核支持系统级、行为级、RTL 级别上的硬件建模。可以用 SystemC 高效地描述软件算法、硬件结构、SOC 或 cycle 级精确模型。用 SystemC 或标准的 C++开发工具创建的系统级模型，可以快速仿真以验证和优化设计，开发不同的算法，为软件和硬件开发团队提供一个可执行规格的系统。

systemC 能在任何支持 ANSI C++的编译环境下编译产生可运行的程序，如 Gnu g++、Microsoft Visual C++ 6.0。可以在 Unix、Linux 下的 gdb、ddd、xgdb、dbx，NT 下的 Visual C++的集成开发环境中，在代码中可以利用 C++的 I/O 语句，也可以采用 systemC 内嵌函数创建文件记录波形，进行仿真调试。

systemC 在 2000 年推出 1.0 版本，2001 年升级为 2.0 版本。SystemC 2.0 版本比老版本增加了接口（interface）和通道（channel）的概念。用 Interfaces、ports、channels 三元素来实现内部模块之间的通讯功能。C++擅长于算法实现，而 SystemC 提供的基本通道（primary channel)则专长于数据通路的控制。SystemC 2.0 版本还增加了事件（event）的概念，并将 1.0 版本中的三种进程的概念更新为两种类型的进程：方法进程（method processes)和线程进程（thread processes）。因 2.0 版本有新增和改动的内容，本文就按版本分别进行介绍。

2. SystemC1.0 的特点

SystemC 支持软硬件协同设计，能够描述复杂的由软硬件组件构成的系统结构。它能够支持在一个 C++环境中对软硬件的描述。下面是 SystemC 作为一个协同设计语言的基本特点：

- Modules：SystemC 有一个容器类的概念称为 Module。这是一个能够包含其他 Module 和 Process 的层次化实体。
- Processes：进程包含于 Module 中，被用于描述模块功能实现。SystemC 能提供给硬件和软件设计者 3 种不同类型的进程。
- Ports：Modules 有 Ports，以使其能和其他的 Modules 相连。SystemC 能够支持单向和双向端口。systemC2.0 版本对端口类型还有扩展。
- Signals：SystemC 支持 Resolved 和 Unresolved 信号，Resolved 信号能够有多余一个的驱动（例如总线），Unresolved 信号只能有一个驱动。

- 丰富的 Port 和 Signal 类型集：为了支持不同抽象级别的模型，从功能行为到 RTL 级，SystemC 支持丰富的 Port 和 Signal 类型。SystemC 既支持两值信号类型，也支持四值信号类型。
- 丰富的数据类型：SystemC 有一套丰富的数据类型以支持多设计领域和抽象级别。固定精度数据类型用于快速仿真，任意精度数据类型用于大数的计算，定点数据类型能用于 DSP 应用。对于任意精度数据类型没有大小的限制。
- 时钟：SystemC 有时钟的概念。时钟是系统仿真时的时间记录者。SystemC 还支持任意相位关系的多时钟。
- 多抽象级别：SystemC 支持在不同抽象层次建模，既支持高级的功能模型，也支持 cycle 级的 RTL 模型。能够支持高级抽象模型和低级抽象模型之间的交互过程。
- 通信协议：SystemC 提供多级别层次通信语法，使你能够描述在不同抽象层次的 SoC 和系统 IO 协议。
- 调试支持：SystemC 类有 Run-time 错误检查功能，能够通过一个编译标志开启。
- 波形跟踪：SystemC 支持 VCD、WIF、ISBD 格式的波形跟踪。

3. SystemC1.0 语法简介

在 systemC 中，系统是由一系列的模块所组成，模块由一系列并发的进程 process 所构成。每个进程完成一定的功能，进程之间通过信号 signal 来通信，clock 可以用来控制事件和同步进程。在 systemC 中模块是分解设计的最基本的功能单元，这样在开发组中能将一个复杂系统分解成模块给不同的设计者来完成。模块化的设计隐藏了内部的数据和算法，通过公共接口与其他模块通信，这样使得整个系统能很容易改变和维护。

(1) systemC 数据类型

C++内部的数据在 systemC 中都能使用，如：long、int、short、char、unsigned long、unsigned int、unsigned short、unsigned char、float、double、long double, and bool。但是用来仿真硬件还是不够的，所以 systemC 还提供了以下的数据类型用来仿真硬件。

- 布尔型：sc_bit－2 值 1 位型
- 标量型：sc_logic－4 值 1 位型
- 整型：
  sc_int<length>－1 到 64 位符号整数型
  sc_uint<length> －1 到 64 位无符号整数型
  sc_bigint<length> － 任意位长度的符号整数型
  sc_biguint<length>－ 任意位长度的无符号整数型
- 逻辑/布尔向量类型：
  sc_bv<length >－任意位的 2 值向量
  sc_lv<length> － 任意位的 4 值向量

- 定点数据类型：
  sc_fixed<wl,iwl,q_mode,o_mode,n_bits> －模板符号定点类型
  sc_ufixed<wl,iwl,q_mode,o_mode,n_bits> －模板无符号定点类型
  sc_fix<list of options> － 无模板符号定点类型
  sc_ufix<list of options> － 无模板无符号定点类型

具体选择哪一种数据类型，需要在仿真速度和代码的结果之间折衷。下面列出的数据类型的仿真速度是从 1）到 6）依次变慢。

1）C++本身的数据类型（int，short，long，char，float，double，struct）（尽可能地使用）；

2）如果在少于 64 位精度情况下，采用 sc_int / sc_uint（常用）（算术运算/逻辑运算）；

3）向量大于 64 位长度但是 2 值时，使用 sc_bv 进行位操作、bool 计算；

4）向量长度大于 64 位但是 4 值时，使用 sc_logic/sc_lv 仿真 'X' 和 'Z'；

5)如果大于 64 位精度，使用 sc_bigint>/sc_biguint 进行任意长度的算术运算；

6）使用 sc_fixed, sc_fix or sc_ufixed, sc_ufix（算术运算）。

C++里的 int 类型能与 sc_int、sc_uint 通用。前面提到 systemC 的整型使用 64 位精度，但是如果需要考虑仿真速度，而且不要求用超过 32 位精度的算法的时候，可以在编译时加上一个编译开关' -D_32BIT_'，这样编译器将后面的算法的精度改为 32 位而不是 64 位。

sc_logic 和 sc_bit 很好地模拟了 1 位的情况；sc_int、sc_uint、sc_bigint 和 sc_biguint 模拟了进行算术运算的情况，但是不能模拟三态总线。当设计需要模拟三态而且位宽大于 1 的数据类型时，systemC 提供了一种叫 sc_lv<length> 的数据类型。这种类型的向量长度是任意的，每位的值可以取值为'0'、'1'、'X' 和'Z'。其可以进行的操作与 sc_bv 的完全一样，唯一不同的是仿真的速度，用 sc_bv 实现的设计的仿真速度要快于用 sc_lv 实现的。同样，sc_lv 也不能直接进行算术运算，它也需要转换成正确的 systemC 整数类型再进行运算。位向量仿真速度总是要比逻辑向量的快。在一个设计中，能使用位向量的尽可能使用位向量。

影响仿真速度的因素主要有：

- 抽象的层次
- 所使用的数据类型
- 所使用的进程类型
- 进程的敏感表
- 代码的质量

如果仿真地越细，速度就越慢。通常使用最高级抽象级别的代码。要充分利用 C++语言的能力，如使用 C++数据类型、结构，正确使用面向对象特性。

(2) 主程序

顶层的主程序称之为 sc_main，主要是创建信号、时钟和实例化模块，并调用仿真函数。在 systemC 中的语法是：

int sc_main （int argc, char* argv[ ] ）;

argc 和 argv 提供了主程序的参数入口如同 C++一样。参数 argc 是命令行参数的个数；argv 是参数值的向量，参数类型为 char* 字符指针向量。注：如运行命令行需要两个参数，参数计数 argc 应该赋值为 3，因程序的名字是占用 argv[0]传递的。

sc_main 文件包含了设计中所有其它的模块。为了例化一个低级的模块，这个模块的接口必须是可见的。例化的模块的.h 文件提供了接口的可见性，要在最先声明。定义信号用于连接实例化的模块的端口。然后产生时钟，实例化模块，创建波形文件，开始仿真。注意在最后要加上 return 0；语句。代码示例如下：

```
// main.c
#include "sd555.h"
#include "filemanage.h"
……
int sc_main(int argc, char* argv[]) {
  sc_signal<bool> hw_reset;
  sc_signal<sc_uint<32>> runcycle;
  ……
  hw_reset = 0;
  sc_clock clk_100M (“clk_100M”, 10, SC_NS);

  CSD555   dut1("dut1");
  dut1.CLK(clk_100M);
  dut1.RSTN(hw_reset);
  dut1.runcycle(runcycle);
  ……
  sc_start(100);
  hw_rst = 1;
  sc_start(100000);
  return   0;
}
```

调用 sc_start（argument）函数表示开始仿真，argument 可以是一个 double 的变量或者常量，用来指定仿真的时间单元数，如果为-1 则表示仿真不会停止，直到 sc_stop（ ）函数。sc_stop（）函数没有参数，用来停止仿真，使 sc_start（）将控制权返回给 sc_main（ ）主程序。对于在主程序中采用了 sc_start（-1）函数，一定要在程序的其他地方加入 sc_stop（）来中止仿真。

注意：

- 信号必须在需要使用的进程的实例化之前声明。
- 时钟必须在需要使用的进程的实例化之前实例化。
- 进程的实例化没有先后顺序。
- 变量名和进程的实例化给的名字最好是一致的，这是一种好的编程习惯。
- 改成可编程的仿真持续时间，可以用如下所示的语句替换 sc_start（200）：

```
if （argc  == 2 ）
    n = atoi（argv[1] ）；
else
    n = 200;
sc_start （n）；
```

- 仿真的开始是从调用 sc_start（）开始，而且直到仿真结束，这个函数返回，才执行这条语句后面的语句。

(3) 模块（Module）

设计的层次是靠 Module 的搭建来实现的。每个 Module 都是一个结构，它是一个 C++类——sc_module。模块中包含一些其他的元素如端口（port）、局部信号（local signal）、局部变量（local data）、其他模块（module）、进程（process）和构造体（constructor）。模块的功能就是由这些元素来实现的。

每个 Module 都含有输入输出端口。Module 可以没有时钟的输入，也可以有多个时钟的输入，信号有效沿不是在成员变量中指定，而是在 Module 的构造体（constructor）中作为一个参数指定。不同有效沿触发的进程可以在同一个 Module 中。如果是异步进程，则不需要指定任何沿。

在 SystemC 中，模块可用关键字 SC_MODULE（宏）来定义，如下所示。

```
SC_MODULE（transmit） {
```

另一种定义方式如下所示。

```
struct transmit : sc_module {
```

这种定义形式和 c++中对 struct 和 class 的定义相似。用宏定义的方式来描述一个模块更加简单而且可读性更强。

(4) 端口和信号（port and signal）

端口是模块与外界传递信息的接口并触发模块内的动作。信号用来联接模块端口，模块间就可以相互通信了。如一个端口直接绑定到另一个端口，就不需要信号来联接。其他情况下，每个端口都是要绑定到一个信号的。Module 中可以定义局部信号，用于联接子模块的端口。

端口的模式有三种：in、out、inout 。端口的数据类型可使用 C＋＋的数据类型、SystemC 的数据类型、自定义的数据类型。

- 输入 sc_in<port type>，作为输入端口，将数据传入模块。如：sc_in<bool> reset；
- 输出 sc_out<port type>，作为输出端口，将数据从模块中输出。如：sc_out<int> dest；
- 输入输出 sc_inout<port type>，作为输入输出端口，依靠模块的操作来决定是输入还是输出数据。如：sc_inout<sc_bit> dnt;

在模块实例化时进行端口与信号绑定。模块实例化是指产生一个对象的实例，这与 C++中声明一个对象是相同的。systemC 在构造体中实例化模块。

每一个模块都有一个构造体 SC_CTOR，宏 SC_CTOR 可以方便的声明构造函数。构造体创建并初始化例化的模块，即构造体创建模块内部数据结构并初始化为给定的值。在构造体中还声明进程、描述端口连接、定义进程的类型和确定敏感列表。

```
SC_MODULE(module_name){
 // 端口，数据成员，成员函数，进程等
 SC_CTOR(module_name){
      //构造函数，进程注册，敏感列表，其他模块例化，端口绑定等。
 }
};
```

构造体的描述在 .h 文件中，进程的具体实现在 .c 文件中。

实例化时有两种方式将信号对应连到端口上：位置绑定（positional mapping）和名称绑定（named mapping）。端口绑定方式如下：

```
SC_MODULE(ex3){
 sc_fifo_in<int>     a;
 sc_fifo_out<int>    b;
 sc_fifo<int>        ch1;
 //模块例化
 ex1 ex1_inst;
 ex2 ex2_inst;
 SC_CTOR(ex3):ex1_inst(“ex1”),ex2_inst(“ex2”){
      //名字绑定
      ex1_inst.m(a);
      ex1_inst.n(ch1);
      //位置绑定
      ex2_inst(ch1,b);
 }
```

(5) 时钟（clock）

时钟是 systemC 中一个特殊的对象。时钟在时间上对事件排序，用来实现在顺序运行的仿真器上模拟并行产生事件的硬件。它提供了同步机制和进程之间通信的顺序。

在一个系统中可以有多于一个的时钟，它们不必是相关的（如 CPU 内核的时钟、总线时钟……），可以在时钟的第一个边缘来建立相位关系。一般时钟都是在设计的顶层－testbench 中产生，并通过模块的层次结构将它传给设计的其他部分。这使整个设计或者设计的一部分同步在同一个时钟下。

时钟是一个 C++类——sc_clock。时钟对象有多个数据成员来约定时钟的特性设置，创建时钟对象的语法为：

sc_clock 变量名 （*name,period, duty_cycle,start_time,positive_first*）;

name: 时钟的名字； 类型: char* ； 缺省值: none

period: 时钟的周期； 类型:double； 缺省值:1

duty_cycle: 时钟的占空比； 类型:double； 缺省值:50%

start_time: 第一个边缘的时间； 类型:double； 缺省值:0

positive_first: 第一个是上升沿； 类型:bool； 缺省值:true

如下例所示。

sc_clock clk（"clk", 20.0, 0.5, 10, 1）;

sc_clock clk_100M (“clk_100M”, 10, SC_NS);

一个好的习惯就是将时钟的名字和变量的名字设成一样，每个进程只有一个时钟。

每个时钟有两个沿，每个时钟沿也都是 C++类：sc_clock_edge。采用沿来触发同步进程，而且也只有同步进程采用时钟的沿，可以用 pos（）和 neg（）指定所使用的时钟的沿。还可以用 name（）得到时钟的名字，period（）得到时钟的周期，duty_cycle（） 得到时钟的占空比。

(6) 进程（process）

模块真正的功能实现部分就是在进程。进程是 systemC 中运行的最小单元，被调用来仿真目标设备或者系统的行为。进程很像普通的 C++函数，一个进程是一个模块的成员函数。用 SystemC 内核(kernal)将成员函数在构造体中注册为进程。

SystemC1.0 版本将进程分为方法进程（Method process）、线程进程（thread process）、时钟线程（clocked thread process）。进程的类型决定了进程怎样被调用和执行。在 systemC 中，就是利用 Methods、Threads 和 Clocked Threads 的概念来模拟系统的并发性的。（2.0 版本中没有 Clocked Threads 的概念了）

进程必须被包含在模块内。宏 SC_METHOD、SC_THREAD 和 SC_CTHREAD 分别可以用来方便的将成员函数注册为进程。

在初始化阶段，所有的进程将被执行。想要阻止进程在初始化阶段执行，可以使用 don't_initialize()函数，将它写在紧跟着进程注册的后面。

进程是一个成员函数，这个函数的返回类型为 void，且函数无参数。

进程在模块的构造体中进行声明。进程没有层次，所以它不能直接调用其它的进程或者被其它进程调用，但是它可以调用方法（method）和函数（function）。

进程的声明和注册示例如下：

```
SC_MODULE(my_module){
  void my_thread_proc();
  void my_method_proc();
  SC_CTOR(my_module){
        //线程进程声明和注册
        SC_THREAD(my_thread_proc);
        //方法进程声明和注册
        SC_METHOD(my_method_proc);
  }
};
```

进程的运行机制为：进程所敏感的信号的值有改变时，执行进程。进程内部的语句是顺序执行的，执行到进程结束退出或执行到 wait 函数调用处进程被挂起。三种进程类型都有自己独特的行为。

方法进程：在刚 run 的时候就会执行一遍。执行完后它会返回控制权给 SystemC kernal。于是方法进程不会自己维护状态，这意味着方法进程内的局部变量在函数返回后将被释放掉。用户如果想维护状态信息，那么必须定义静态变量或在模块内定义成员变量。方法进程不可以使用 wait()函数挂起。

当敏感列表中的信号发生变化，执行 SC_METHOD，执行速度无限快，它运行完了以后将控制权返回给仿真内核。

```
SC_MODULE(adder) {
  // Ports, internal signals
  //process
  void add1();

  // Constructor
  SC_CTOR(adder) {
        // process registration & declarations of sensitivity lists
        SC_METHOD(add1);
        sensitive_pos << clk;
```

```
        sensitive << a << b;
  }
};
```

线程进程：在仿真的初始化阶段被执行，且仅仅执行这一次。当线程进程被挂起后，必须等到 wait()执行才会再次执行下去。当挂起后，线程进程的状态是被保存了的。挂起后的回复执行将从上次挂起点（wait()）处开始执行。如果希望线程函数体的某个部分多次，那么必须使用循环（通常是无限循环）。如果一个线程有一个无限循环然而没有 wait()，那么这个进程将在一个 delta 时间无限地重复地执行。

当敏感列表中的信号发生变化，执行 SC_THREAD，当遇到 wait()，将程序挂起，返回主程序；如敏感列表中信号变化，再次进入 SC_THREAD，从刚才挂起的部分开始执行。

```
//"adder.h"
SC_MODULE(adder) {
  // Ports, internal signals
  //process
  void add1( );
  // Constructor
  SC_CTOR(adder)   {
        // process registration & declarations of sensitivity lists
        SC_THREAD(add1);
        sensitive_pos << clk;
  }
};
```

SC_CTHREAD 是一种特殊的线程，SC_CTHREAD 只在时钟沿触发，时钟必须是 sc_in_clk。在使用 SC_CTHREAD 的时候，不仅要说明 process 的名字，还要有触发这个 process 的时钟。

```
//"adder.h"
SC_MODULE(adder) {
  // Ports, internal signals
  sc_in_clk       clock;
  //process
  void add1( );
  // Constructor
  SC_CTOR(adder)   {
```

```
    // process registration & declarations of sensitivity lists
    SC_CTHREAD(add1, clock.pos());
  }
};
```

3 种进程的区别如下：

- SC_METHOD 不能使用 wait()，进入 process 后只能将这一处理过程执行完后才能退出，常用于初始化和 testbench，速度快。
- SC_THREAD 使用 wait()，将整个 process 分步执行，对时序逻辑非常简单有用，速度慢。
- Wait()、wait_until() 可以在 SC_THREAD 和 SC_CTHREAD 中使用。
- watching 只能在 SC_CTHREAD 中使用。
- SC_CTHREAD 要指定时钟对象，声明进程名和触发进程的时钟沿（例如 SC_CTHREAD（xfer, clock.pos（））； ）。 而模块构造体中的其他类进程仅仅声明进程名即可。

(7) 用户自定义类型

对于自定义数据类型如 packet_type，可以自定义相关操作。如下例中定义了 packet_type 的＝ ＝操作：

```
struct packet_type {
  long info;
  int seq;
  int retry;
  inline bool operator = = （const packet_type& rhs） const {
      return （rhs.info == info && rhs.seq == seq && rhs.retry == retry）;
  }
};
```

函数 inline 的使用是为提高编译效率，编译器不必作函数调用，而是直接进行代码替代。const 表示常数引用，指出函数不得修改引用参数。

4. SystemC 仿真调度

SystemC 调度器（Simulation semantics）控制着进程的执行顺序，并且掌控着事件通告和掌管着通道的更新（updates）。它支持 delta_cycles 的概念。一个 delta_cycle 包括求值（evaluate）和更新（updata）。

调度器的启动是在 sc_start()函数被调用后。当然也可以在一定时间后启动。

调度器的停止条件是：sc_stop()的调用，再无任何事件发生。

SystemC 仿真遵循求值更新方式，在这种方式下，所有就绪态的进程都会被执行，也只有在被执行时，进程的输出值才被更新。

在仿真过程中 SystemC 的调度器按以下步骤执行：

- 在当前时间被赋予新值时所有的时钟信号都改变他们的值；
- 所有 SC_METHOD/SC_THREAD 进程的输入值改变时这些进程被执行；
- 所有被触发的 SC_CTHREAD 进程更新输出，同时进程被保留在队列中等待在步骤五执行；
- 步骤 2 和步骤 3 被反复执行直到没有信号值改变；
- 所有被触发过的在步骤 3 入队的 SC_CTHREAD 进程。被执行这些进程的执行顺序不固定，执行的输出结果在下一个跳变沿被更新。在执行步骤 3 时更新输出结果被暂时保存在内部；
- 仿真时间推进到下一个时钟跳变沿同时调度器返回步骤 1。

如果进程使用信号通信，进程的执行顺序不会影响仿真结果。然而如果使用全局变量和指针进行进程间通信，进程执行顺序会影响仿真结果。注意这些仿真语义与 Verilog/VHDL 仿真语义中的信号赋值方式相同。

只有在完成了实例化工作，同时合理连接所有模块和信号以后，才可以开始仿真。在 SystemC 中，仿真开始于从顶层调用 sc_start()函数，也就是 sc_main()例程。sc_start()函数使用 double 型变量作为参数，并且以此参数值指示的时间单位数量作为系统仿真执行时间。如果你希望能够一直不停仿真，只要把负数值作为参数传递给这个函数。这个例程在合适的时间产生所有的时钟信号并且调用 SystemC 调度器。

通过调用 sc_stop()函数，仿真可以在任何时刻从任何进程停止。这个函数可以不需要任何参数，仿真可以在调度器中的步骤四或步骤五停止。

调用函数 sc_time_stamp() 或 sc_simulation_time() 均可获知当前的仿真时间，其中 sc_simulation_time()返回的是 double 类型值。

SystemC2.0 版本的时间数据类型采用 64 位无符号整数，最小的时间精度可以为 1fs，缺省的时间单位为 ns，可以利用 sc_set_time_resolution（ ）来设定时间精度。注意：在 sc_main（）中最多只能在仿真开始之前调用一次 sc_set_time_resolution（ ），而且第一个参数要是 10 的幂。例如：

```
sc_set_time_resolution（ 10, sc_ps ）;
```

或者：

```
sc_time t1（ 10, sc_ps ）;
sc_set_time_resolution（ t1 ）;
.....
wait（ 3.456, sc_ns ）;      // wait for 3460 ps;
```

另可以调用函数 sc_get_time_resolution（）来获知当前的时间精度。

时间单位的类型有以下几种：

sc_fs femtoseconds

sc_ps picoseconds

sc_ns nanoseconds

sc_us microseconds

sc_ms milliseconds

sc_sec seconds

调用函数 sc_set_default_time_unit（）可以设定缺省的时间单位，例如：

```
sc_set_default_time_unit（ 1, sc_ps ）;
```

注意：缺省时间单位的设置必须大于等于时间精度，必须在仿真开始之前就设定好（即在调用 sc_initialize（）或 sc_start（）之前），在 sc_main（）中可以调用多次 sc_set_default_time_unit（ ），但只有最后的设定才是有效的，而且第一个参数要是 10 的幂。

```
sc_set_default_time_unit（ 100, sc_ps ）;
...
sc_clock clk1（ "clk1", 10 ）;          //定义 clk1 的周期为 1000ps
```

另可以调用函数 sc_get_default_time_unit（）来获知当前的时间单位。

```
sc_clock clk1（ "clk1", 15 ）;
sc_start（ 1000 ）;
```

sc_clock 与 sc_start 的用法可以是上面所示，但是最好还是写成下面的格式：

```
sc_clock clk1（ "clk1", 15, sc_ns ）;
sc_start（ 1000, sc_ns ）;
```

你可以不使用 sc_start()而选择使用其它方式产生时钟，或者选择使用其它方式控制仿真。如果要这样做，必须首先调用 sc_initialize()初始化 SystemC 调度器，然后你可以通过赋值设定信号值，然后调用 sc_cycle()例程仿真信号设置的结果，这个函数使用一个 double 型参数作为变量。这个函数调用 SystemC 调度器，并执行调度器循环中的步骤二到步骤五仿真，直到当前信号赋值的效果在整个系统中传播完毕，然后这个函数在当前仿真时间上加一个函数参数中传入的时间值，例如：

如果仿真时间单位是 ns，则 sc_cycle(10)增加当前仿真时间 10ns。

例如按以下语法定义了一个时钟：

```
sc_clock clock("my clock", 20, 0.5);
```

可以通过调用以下函数仿真 200 时间单位的时钟产生

```
sc_start(200);
```

从另一方面看你可以通过以下语句产生自己的时钟

```
sc_initialize();
for (int i = 0; i <= 200; i++)
  clock = 1;
  sc_cycle(10);
  clock = 0;
```

```
  sc_cycle(10);
}
```

使用这种功能你可以在系统中加入相对时钟异步的事件如图 3-20 所示。

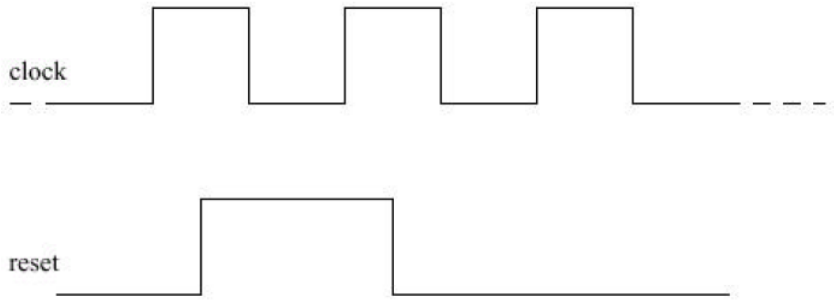


图3-20 相对时钟异步的信号

你可以在 sc_main()中加入以下代码以实现上述功能：

```
sc_initialize();
// 让时钟运行 10 个周期
for (int i = 0; i <= 200; i++)
{
  clock = 1;
  sc_cycle(10);
  clock = 0;
  sc_cycle(10);
}
// 加入异步复位
clock = 1;
sc_cycle(5);
reset = 1;
sc_cycle(5);
clock = 0;
sc_cycle(10);
clock = 1;
sc_cycle(5);
reset = 0;
sc_cycle(5);
clock = 0;
sc_cycle(10);
// 现在让时钟不停止的运行
for (;;)
```

```
{
  clock = 1;
  sc_cycle(10);
  clock = 0;
  sc_cycle(10);
}
```

⚠ 注意

sc_cycle()如 sc_start()一样只可以从顶层调用。

5. 跟踪波形

SystemC 提供函数用于创建 VCD(Value Change Dump),ASCII WIF(Waveform Intermediate Format),或 ISDB(Integrated Signal Data Base)文件这些文件可以记录仿真过程中的变量和信号值变化在这些文件中定义的波形可以用支持 VCD, WIF, 或 ISDB 格式的标准波形查看软件观察。

在产生波形过程中请注意：

- 只有在整个仿真过程中都可见的变量可以被跟踪也就是说所有的信号和模块数据成员都可以被跟踪函数内的局部变量无法跟踪
- 变量和标量信号数组和聚合类型可以被跟踪
- 可以在同一次仿真中创建不同类型的跟踪文件
- 在不同的跟踪格式中单个信号可以被跟踪任意多次

(1) 创建跟踪文件

跟踪波形的第一步是创建跟踪文件一般情况下在所有的模块和信号都被实例化以后在顶层创建跟踪文件通过调用 sc_create_vcd_trace_file() 函数并以文件名作为参数可以创建 VCD 格式的跟踪波形文件这个函数返回一个指向数据结构的指针这个指针可以在整个仿真过程中使用。例如：

```
sc_trace_file * my_trace_file;
my_trace_file = sc_create_vcd_trace_file(“my_trace”);
```

以上代码创建了名为 my_trace.vcd 的 VCD 文件扩展名.vcd 是自动加上的函数返回一个指向数据结构的指针你应该保存这个指针供调用跟踪例程时使用。

通过调用 sc_create_wif_trace_file()函数可以创建 WIF 文件例如：

```
sc_trace_file *trace_file;
my_trace_file = sc_create_wif_file( my_trace );
```

以上代码创建了名为 my_trace.awif 的 WIF 文件扩展名.awif 是自动加上的可以用类似的方法创建 ISDB 跟踪文件。

在仿真结束的时候必须关闭跟踪文件否则会引发错误。可以用以下函数之一关闭跟踪文件：

sc_close_isdb_trace_file(my_trace_file);

sc_close_wif_trace_file(my_trace_file);

sc_close_vcd_trace_file(my_trace_file);

调用以上函数的时间应该是在 sc_main 例程的 return 语句之前。

(2) 跟踪标量和信号

SystemC 提供用于跟踪标量和信号的函数所有的跟踪函数，有以下共同特点：

- 函数名都是 sc_trace()
- 函数的第一个参数是指向 sc_trace_file 文件数据结构的指针
- 函数的第二个参数是指向被跟踪变量的引用或指针
- 函数的第三个参数是指向字符串的引用

模块中的所有变量和信号都可以存储在波形文件中。

//创建一个名为 simplex.vcd 的文件，记录仿真结果

sc_trace_file *tf = sc_create_vcd_trace_file （"simplex"）;

//接下来声明所要观察的模块中的信号和变量

sc_trace（tf, CLOCK.signal（）, "clock"）;

sc_trace（tf, TIMEOUT, "timeout"）;

sc_trace（tf, START, "start"）; //最后的一个参数指明信号存在 simplex.vcd 文件中的名字

要想观察模块中定义的变量，方法类似于 verilog 中层次化调用的语法，使用符号 '.' 。

例如：想观察 sc_module（timer）中定义的局部变量 count，代码如下：

sc_trance（tf, timer.count, "count"）;

跟踪函数注册创建了个列表，被跟踪的信号和变量实际的跟踪发生在仿真过程中由 SystemC 调度器进行实际的处理。注意 sc_trace()只有在过程和信号实例化后以及跟踪文件打开后才可调用。

(3) 跟踪集合类型的变量和信号

SystemC 定义的跟踪函数只可以处理标量类型的信号和变量

为了跟踪集合类型的变量需要用 SystemC 中提供的基本跟踪函数针对特殊数据类型定义特殊的跟踪函数。

例如考虑以下数据结构

struct bus {

```
  unsigned address;
  bool read_write;
  unsigned data;
};
```

你需要为以上结构定义一个跟踪函数

```
void sc_trace(sc_trace_file *tf, const bus& v, const
sc_string& NAME)
{
  sc_trace(tf, v.address, NAME + ".address");
  sc_trace(tf, v.read_write, NAME + ".rw");
  sc_trace(tf, v.data, NAME + ".data");
}
```

调用时这个跟踪函数跟踪数据结构中的每一个单独的数据域注意每一个单独的数据域都通过在结构名后附加数据域名的方式被赋予了独一无二的名字。

(4) 跟踪变量和信号数组

为了跟踪变量或数组需要利用 SystemC 提供的基本数据或信号跟踪函数定义特殊的跟踪函数，例如以下是跟踪 sc_signal<int>类型数组的跟踪函数：

```
void sc_trace(sc_trace_file *tf, sc_signal<int> *v,
const sc_string& NAME, int len)
{
  char stbuf[20];
  for (int i = 0; i< len; i++) {
        sprintf(stbuf,"[%d]",i);
      sc_trace(tf, v[i], NAME + stbuf);
  }
}
```

这个跟踪函数有一个附加的参数这个参数表示被跟踪数组的长度。

SystemC 已经为所有 SystemC 定义的矢量类型预先定义了跟踪函数(例如 sc_int<>,sc_uinit<>, sc_biginit<>, sc_bigunit<>, sc_lv<> 等。

6. SystemC 仿真调试

仿真流程：编译（.cpp, ,v 等）->elaboration（scelab,ncelab）

->初始化（initialization）->仿真（ncsim）

SystemC 仿真器是基于事件的。

Elaboration 被定义为从开始执行 sc_main()到 sc_main()调用 sc_start()之间的部分。Elaboration 包括了一些东西的构造过程：模块例化，通道连接，sc_clock 和 sc_time 变量。

函数 sc_set_default_time_unit()和 sc_set_time_resolution()必须在 elaboration 之前被调用。同时，必须在 sc_time 对象被构造前被调用。

在 elaboration 期间，系统结构的各个元素都被创建好，而且它们的关系（即端口，通道等）都会按照系统的架构层次连接好。（这里利用了 C++构造函数）

初始化（Initialization）是 SystemC 调度器的第一个步骤。每一个方法进程都会在初始化阶段被执行一次，而线程进程还需要等到 wait()声明。

如果想关闭进程的初始化，可以使用 don't_initialize()函数。

需要特别注意的是：进程的执行顺序是不确定的！但是在相同的环境下，执行的顺序是一定的。这个就意味着，使用两个不同版本的仿真器仿真同一个电路可能得到不同的结果。所以在建模的时候需要注意。

因为每一个线程 thread 或基于时钟线程 clocked-thread 的过程都产生了一个新的执行线程，调试仿真过程会比调试线性执行的 C++程序的难度大。仿真中执行线程的概念代表了仿真是非线性执行的，很难确定哪一段代码紧接着将要被执行。

你可能只希望调试自己的代码，而不是调试 SystemC 的类库。在你感兴趣的过程的入口处设置断点，是最简单的调试方法。当仿真在某个断点停住时，仿真过程挂起，这时你可以调试自己感兴趣的过程。

7. SystemC 2.0 版本新增内容

systemC2.0.1 版本增加了通道（Channel）、接口（interface）等概念，有助于我们更方便的描述所要实现的功能。systemC2.0.1 版本语言结构图如下所示。

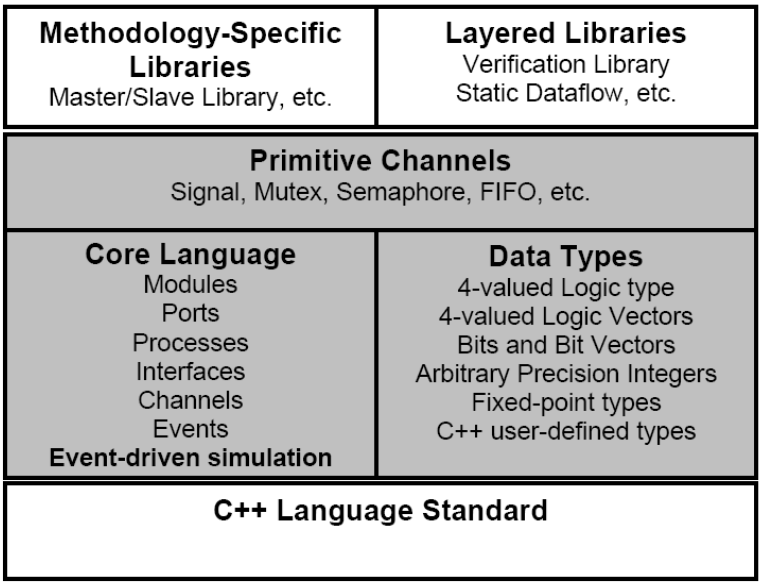


图3-21 systemC 语言结构

SystemC 2.0 版本比老版本增加了接口（interface）和通道（channel）的概念。Interfaces、ports、channels 三元素来实现内部模块之间的通讯功能。

不同模块中的进程直接的通讯使用端口（port），接口（interface）和通道（channel）概念。接口为通道定义声明了一套函数，在通道中描述了这些函数的具体实现，通过端口就可以访问通道了。系统级验证的方法要求将数据通路和运算分开，C++擅长于算法实现，而 SystemC 提供的基本通道（primitive channels ）则专长于数据通路的控制。

systemC 仿真调度器从 sc_start()函数开始执行，运行停止的条件是有进程调用了 sc_stop()，或者是没有要运行的事件了，或者是有例外情况发生。

(1) 事件概念

SystemC 2.0 版本增加了事件（event）的概念，事件（event）用于进程间的同步和通道中阻塞行为的实现。事件的发生通常是由相关的进程或通道的某些状态改变引起的。一旦事件被触发（notify），它就会通知 systemC 调度器哪个进程要被执行。此运行机制示意图如下所示。

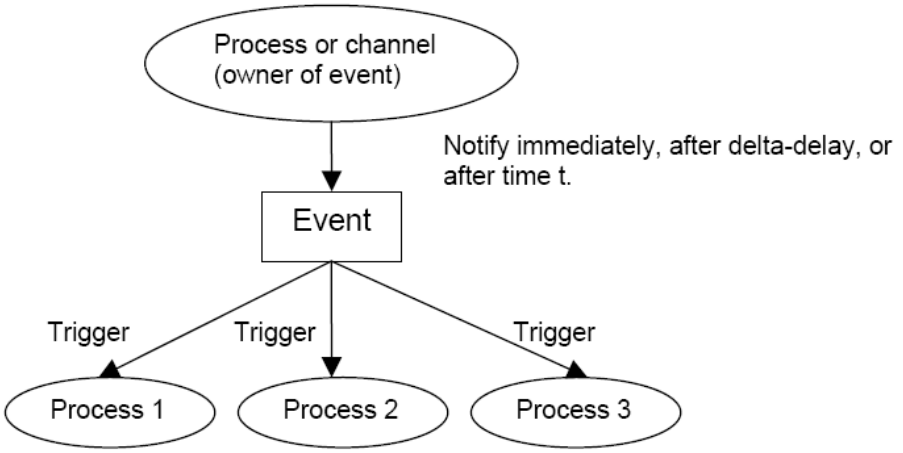


图3-22 事件的发生示意图

事件对象可以调用 notify()方法，有 3 种 notify()：立即触发、等待 delta 延迟后触发、等待一段时间后触发，如下所示。

```
sc_event my_event;
my_event.notify(); // notify immediately
my_event.notify( SC_ZERO_TIME ); // notify next delta cycle
sc_time t( 10, SC_NS );
my_event.notify( t ); // notify in 10 ns
```

取消已经挂起等待的事件 notify 可以使用方法 cancel()，但立即触发的挂起事件不能取消，如下所示。

```
sc_event a, b, c;
sc_time t(10, SC_MS);
a.notify(); // current delta-cycle
notify(SC_ZERO_TIME, b); // next delta-cycle
notify(t, c); // 10 ms delay

//Cancel an event notification
a.cancel(); // Error! Can't cancel immediate notification
b.cancel(); // cancel notification on event b
c.cancel(); // cancel notification on event c
```

事件的触发是基于敏感信息的，敏感又分为动态敏感和静态敏感。动态敏感是指在仿真运行时还能动态改变。伴随动态敏感概念的提出，要求进程可以等待多个事件的触发。事件敏感的概念在后面再详细介绍。

(2) 模块实例化

2.0 版本中描述有两种方法进行模块实例化，可以使用指针或不使用指针来实例化，如下：

不使用指针的方法：先在父模块中作为成员进行声明，再进行实例化。

```
SC_MODULE(ex3) {
  // Ports
  sc_fifo_in<int> a;
  sc_fifo_out<int> b;
  // Internal channel
  sc_fifo<int> ch1;
  // Instances of module types ex1 and ex2
  ex1 ex1_instance;
  ex2 ex2_instance;
  // Module Constructor
  SC_CTOR(ex3):                    ex1_instance("ex1_instance"),
  ex2_instance("ex2_instance") {
        // Rest of constructor body not shown
  }
  // Rest of the module body not shown
};
```

使用指针的方法：先在父模块中声明指向模块类型的指针，再进行实例化。

```
SC_MODULE(ex3) {
  // Ports
  sc_fifo_in<int> a;
  sc_fifo_out<int> b;
  // Internal channel
  sc_fifo<int> ch1;
  // Pointers to instances of module type ex1 and ex2
  ex1 *ex1_instance;
  ex2 *ex2_instance;
  // Module Constructor
  SC_CTOR(ex3){
        // allocate and initialize both instances
        ex1_instance = new ex1(”ex1_in_ex3");
        ex2_instance = new ex2("ex2_in_ex3");
        // Rest of constructor body not shown
```

```
  }
  // Rest of the module body not shown
};
```

使用 new 分配了空间的对象，需要最后在模块中释放占用的空间。

```
SC_MODULE(ex3) {
  // Rest of the module not shown
  ~ex3() {
        delete ex1_instance;
        delete ex2_instance;
  }
};
```

(3) 接口

一个接口为一个通道定义了一套访问函数，但它只是指定了函数名、函数参数和返回值类型，并没提供函数的实现，函数的实现在通道中描述。SystemC 2.0 版本定义的接口类型如下所示。

sc_fifo_in_if

sc_fifo_out_if

sc_mutex_if

sc_semaphore_if

sc_signal_in_if

sc_signal_inout_if

sc_fifo_in_if 代码如下：

```
template <class T>
class sc_fifo_in_if : virtual public sc_interface {
public:
      virtual void read( T& ) = 0; // blocking read
      virtual T read() = 0;
      virtual bool nb_read( T& ) = 0; // non-blocking read
      virtual int num_available() const = 0; // get the number of available samples
      virtual const sc_event& data_written_event() const = 0; // get the data written event
protected:
```

```
    // constructor
    sc_fifo_in_if(){}
private:
    // disabled
    sc_fifo_in_if( const sc_fifo_in_if<T>& );
    sc_fifo_in_if<T>& operator = ( const sc_fifo_in_if<T>& );
};
```

(4) 通道

通道定义了对应接口的函数的实现。通道可以提供模块之间的通讯，也可以提供模块内的进程之间的通讯。一个通道可以实现一个或多个接口，不同的通道可以用不同的方法实现一个接口。通道分两种类型：基本通道和层次通道（primitive channels and hierarchical channels）。

基本通道是最基本的，不能直接访问其他通道，不能包含进程和其他模块。基本通道如下所示。

sc_buffer

sc_fifo

sc_mutex

sc_sempahore

sc_signal

sc_signal_resolved

sc_signal_rv

有 systemC 结构的通道被定义为层次类型通道，这里的结构可以包括端口、模块实例、其他通道和进程。这样的通道更像一个模块。层次通道的概念大大提高了通道定义的灵活性。

(5) 端口

端口为模块提供了与周边进行连接与通讯的手段。模块通过端口可与一个或多个通道进行通讯。一个端口需要一个接口。所有端口肯定都是类 sc_port 的派生类。端口声明示例如下：

```
SC_MODULE(my_module){
  sc_port<IF, N > port_name ; // IF 指定了端口连接的接口；N 指定了端口可以连接的最大接口数，缺省值为 1
  // rest of module not shown
};
```

如 N 为 1，表示是一个简单的单端口；如 N 大于 1，表示是一个连接了多个接口的多端口。

对端口所绑定的接口的函数调用使用操作符 “－>”，其返回值是指向接口的指针。如下所示。

```
sc_port<sc_signal_in_if<int> > a; //port declaration
a->read();//calls the read() interface method of the channel connected to port a
```

要访问多端口中的一个接口，使用“[ ]”操作符，如下所示。

```
// port declaration, a is bound to two channels
sc_port<sc_signal_in_if<int>, 2 > a;
// calls the read() interface method of the 2nd channel connected to port a
a[1]->read();
// calls the read() interface method of the 1st channel connected to port a
a[0]->read(); // or a->read();
```

SystemC 2.0 版本针对某些通道定义了专用端口，以后的版本还会继续增加以方便使用。这些专用端口是为特定场合提供特定接口的端口，它们还是 sc_port 的子类。这些端口提供了对通道的特殊或额外的支持并且好用。如下所示。

对于 sc_buffer 和 sc_signal 类的通道使用端口：sc_in、sc_inout、sc_out。

对于 sc_fifo 通道使用端口： sc_fifo_in、sc_fifo_out。

对于 sc_signal_rv 通道使用端口： sc_in_rv、sc_inout_rv、sc_out_rv 。

对于 sc_signal_resolved 通道使用端口：sc_in_resolved、sc_inout_resolved、sc_out_resolved。

(6) 进程

模块真正的功能实现部分就是在进程。进程是模块的成员函数，进程没有层次，其运行机制为：进程所敏感的信号的值有改变时，执行进程。

systemC2.0 版本中有两种进程类型：SC_METHOD 和 SC_THREAD。

进程作为模块的成员函数声明是没有返回参数的，进程的声明是在模块的构造体中进行的，如下所示。

```
SC_MODULE(my_module) {
  void my_thread_proc(); //member function declaration
  void my_method_proc(); //member function declaration
  SC_CTOR(my_module) {
```

```
        // thread process declaration and registration
        SC_THREAD(my_thread_proc);
        // method process declaration and registration
        SC_METHOD(my_method_proc);
        // rest of constructor not shown
  }
  // rest of module not shown
};
```

(7) 进程的静态敏感

所谓静态敏感，意即进程所敏感的事件在仿真过程中是不变的。静态敏感的进程使用 sensitive 关键词在模块构造体中声明敏感列表，如下所示。

```
SC_MODULE(my_module) {
  sc_event c;
  void my_thread_proc();
  SC_CTOR(my_module) {
        SC_THREAD(my_thread_proc);
        // declare static sensitivity list
        sensitive(c); // sensitive to event c
  }
  // rest of module not shown
};
```

如果进程要敏感于多个事件，描述方式如下所示。

```
SC_MODULE(my_module) {
  sc_event c;
  sc_event d;
  void my_thread_proc();
  SC_CTOR(my_module) {
        SC_THREAD(my_thread_proc);
        // declare static sensitivity list
        sensitive(c); // sensitive to event c
        sensitive(d); //sensitive to event d
  }
  // rest of module not shown
};
```

sensitive 也支持如下描述方式：

```
SC_MODULE(my_module) {
  sc_event c;
  sc_event d;
  void my_thread_proc();
  SC_CTOR(my_module) {
        SC_THREAD(my_thread_proc);
        // declare static sensitivity list
        sensitive << c << d; // sensitive to events c & d
  }
  // rest of module not shown
};
```

如果模块中有多个进程描述，则如下所示。

```
SC_MODULE(my_module) {
  sc_event c, d;
  void proc_1();
  void proc_2();
  void proc_3();
  SC_CTOR(my_module) {
        SC_THREAD(proc_1);
        sensitive << c << d; //proc_1 sensitive to c & d
        SC_THREAD(proc_2); // no static sensitivity
        SC_THREAD(proc_3);
        sensitive << d ; // proc_3 sensitive to d
      }
  // rest of module not shown
};
```

(8) 进程的动态敏感

systemC2.0 版本增加了了动态敏感的概念。对于 Method 进程和 Thread 进程都可以使用静态敏感、动态敏感，也可以同时使用静态敏感和动态敏感。它们动态敏感的语法是有区别的。Method process 的动态敏感是利用 next_trigger（）函数指定事件列表实现的，Thread process 的动态敏感是利用 wait（）方法指定事件列表来实现的。一个进程中可以调用多个 wait（），每个 wait（）又可以有不同的敏感事件列表。

（1）Method Process

Method 进程使用 SC_METHOD 宏在模块构造体中进行声明：

```
SC_MODULE(my_module) {
sc_event c;
// process member function declaration
void my_method_proc();
SC_CTOR(my_module) {
    // method process declaration & registration
    SC_METHOD(my_method_proc);
    // declare static sensitivity list
    sensitive(c); // sensitive to event c
    dont_initialize(); // don't run at initialization
} // Rest of module not shown
};
```

Method 进程动态敏感描述语句 next_trigger(arg)，可以指定事件、事件列表或延时作为下一次触发进程的条件。

如果使用 next_trigger()没有指定参数，那么进程的下一次触发条件就是静态敏感所指定的条件。但是如果进程并没有静态敏感列表描述，那么这个进程就不会再次被触发执行了。

如果多个 next_trigger(arg)被执行，最后一个将起决定作用（last one wins）。

next_trigger()表示等到构造体中声明的敏感信号有事件发生后触发。

单事件触发的描述为：

```
sc_event e1; // event
next_trigger(e1);
```

延时触发的描述为：

```
sc_time t(200, SC_NS); // variable t of type sc_time
next_trigger(t); // trigger 200 ns later
next_trigger(200, SC_NS); // trigger 200 ns later
```

如果延时值为 0，则进程会在 delta 延时之后触发，如下：

```
next_trigger( 0, SC_NS );
next_trigger( SC_ZERO_TIME );
sc_time t(0, SC_NS); // variable t of type sc_time
next_trigger( t ); // trigger the next delta-cycle
```

进程敏感的事件可以是或的关系：

sc_event e1,e2,e3; // events

next_trigger(e1 | e2 | e3); //trigger on e1, e2 or e3

也可以是与的关系：

sc_event e1,e2,e3; // events

next_trigger(e1 & e2 & e3);//trigger on e1, e2 and e3

如果要求某一事件触发就执行进程或者指定的延时到达也可以触发进程，则可以有如下两种写法：

sc_time t(200, SC_NS); // variable t of type sc_time

// trigger on e1, e2, or e3, timeout after 200 ns

next_trigger(t, e1 | e2 | e3);

// trigger on e1, e2, or e3, timeout after 200 ns

next_trigger(200, SC_NS, e1 | e2 | e3);

如果要求几个事件都触发后再执行进程或者指定的延时到达也可以触发进程，则可以有如下两种写法：

sc_time t(200, SC_NS); // variable t of type sc_time

// trigger on e1, e2, and e3, timeout after 200ns

next_trigger(t, e1 & e2 & e3);

// trigger on e1, e2, and e3, timeout after 200ns

next_trigger(200, SC_NS, e1 & e2 & e3);

（2）Thread Processes

Thread 进程使用 SC_THREAD 宏在模块构造体中进行声明：

```
SC_MODULE(my_module) {
sc_event c;
void my_thread_proc();
SC_CTOR(my_module) {
        SC_THREAD(my_thread_proc);
        // declare static sensitivity list
        sensitive(c); // sensitive to event c
}
// rest of module not shown
};
```

Thread 进程执行到进程中的 wait 语句就挂起，直到进程敏感信号列表中的事件触发，进程继续接着执行 wait 后的语句。

Thread 进程动态敏感描述语句 wait(arg)，可以指定事件、事件列表或延时作为触发进程继续执行的条件。

如果使用 wait()没有指定参数，那么进程的触发条件就是静态敏感所指定的条件。但是如果进程并没有静态敏感列表描述，那么这个进程就不会再次被触发执行了。

单事件触发的描述为：

```
sc_event e1;
wait（ e1 ）;
```

延时触发的描述为：

```
sc_time t(200, SC_NS); // variable t of type sc_time
wait(t); // trigger 200 ns later
wait(200, SC_NS); // trigger 200 ns later
```

如果延时值为 0，则进程会在 delta 延时之后触发，如下：

```
sc_time t(0, SC_NS); // variable t of type sc_time
wait( t ); // resume after a delta-cycle delay
wait( 0, SC_NS ); // ditto
wait( SC_ZERO_TIME ); // ditto
```

进程敏感的事件可以是或的关系：

```
// wait on any one or more of the events e1, e2, or e3;
wait（ e1 | e2 | e3 ）;
```

也可以是与的关系：

```
// wait until all events e1, e2, and e3 have occurred;
wait（ e1 & e2 & e3 ）;
```

如果要求某一事件触发就执行进程或者指定的延时到达也可以触发进程，则可以有如下两种写法：

```
sc_time t(200, SC_NS); // variable t of type sc_time
// resume on e1, e2, or e3, timeout after 200 ns
wait(t, e1 | e2 | e3);
// resume on e1, e2, or e3, timeout after 200 ns
wait(200, SC_NS, e1 | e2 | e3);
```

如果要求几个事件都触发后再执行进程或者指定的延时到达也可以触发进程，则可以有如下两种写法：

```
sc_time t(200, SC_NS); // variable t of type sc_time
// trigger on e1, e2, and e3, timeout after 200ns
wait(t, e1 & e2 & e3);
// trigger on e1, e2, and e3, timeout after 200ns
wait(200, SC_NS, e1 & e2 & e3);
```

⚠ 注意

| 操作和& 操作不可以混用。有时间参数时，时间必须做第一个参数。

8. SystemC 与 Verilog 的嵌套设计

systemC 与 verilog 设计是可以相互嵌套的，但是目前只支持 systemC 嵌入 verilog 或者 verilog 嵌入 systenC，不支持“三明治”式的嵌套方式。仿真工具 LDV5.1 的自带工具 nc_systemC 支持混和仿真功能，例化方法如下：

(1) Verilog 例化 systemC

如果你的顶层设计是 Verilog 代码编写的 module，某个子模块是 systemC 编写的 module，这时要将你的设计连在一起的方法如下：

第一步：编写一个 verilog shell，shell 的名字和 systemC 模块名一样，增加语句（* const integer foreign = "SystemC"; *）；。

第二步：在 systemC 模块中增加语句 SC_MODULE_EXPORT（模块名）。

第三步：compile、elaborate、simulate。

下例中顶层是 verilog 模块 model_test.v，嵌入 systemC 模块 model，编写了一个 verilog shell（model.v）。

```
`timescale 1 ns/1 ns
module model_test;      //verilog testbench
  reg in;
  wire out;
  initial begin
        in = 1'b0;
        #10 in = 1'b1;
        #10 in = 1'bx;
        #10 in = 1'bz;
        #10 in = 1'b0;
        #10 in = 1'b1;
```

```
        #10 in = 1'bx;
        #10 in = 1'bz;
        #10 $finish;
  End
  model test （in, out）;            // Instantiate the SystemC HDL Shell
  initial $monitor（$time,,"in = %b, out = %b", in, out）;
endmodule
-------
model.v （Verilog shell file for SystemC）
-------
`timescale 1 ns / 1 ns

module model （ in, out ）// The foreign attribute string value must be a SystemC value.
 （* const integer foreign = "SystemC"; *）;
  input in;
  output out;
endmodule

-------
model.h （SystemC .h file）
-------
#include "systemC.h"
SC_MODULE（model） {
public:
  sc_in<sc_logic> in;
  sc_out<sc_logic> out;

  SC_CTOR（model） : in（"in"）, out（"out"） {
        SC_METHOD（run）;
        sensitive << in;
  }
};
-------
model.cpp （SystemC .cpp file）
-------
```

```
#include "model.h"
Void model::run（）{
  out.write（in.read（））;
}
SC_MODULE_EXPORT（model）
```

编译、仿真步骤如下：

```
ncsc –compiler g++ model.cpp –mess
make COMPILER = gnu SYSTEMC_SOURCES = “model.cpp”
ncshell model -import systemC -into verilog -shell model.v
ncvlog model.v model_test.v -mess
ncelab -loadsc systemC model_test:vlog
ncsim model_test:vlog
```

(2) systemC 例化 verilog

如果你的顶层设计是 systemC 代码编写的 module，某个子模块是 verilog 编写的 module，这时要将你的设计连在一起的方法如下：

第一步：编写一个 systemC shell，shell 的名字和 verilog 模块名一样，关键语句在下文中用红色标示。

第二步：在 systemC 模块中增加语句 SC_MODULE_EXPORT（模块名）。

第三步：compile、elaborate、simulate。

下例中顶层是 systemC 模块 inv_test.v，嵌入 verilog 模块 inv，编写了一个 systemC shell（inv.cpp）。

```
-------
inv.v
-------
module inv            //verilog module
  (output wire z,
  input wire a
  );
  assign z = ~a;
endmodule
-------
inv.h.      /* SystemC shell Header */
-------
```

```
#include "systemC.h"
class inv : public sc_foreign_module {
public:
  sc_inout<sc_logic> z;
  sc_in<sc_logic> a;
  inv（sc_module_name nm） : sc_foreign_module（nm）,
  z（"z"）, a（"a"） { }
  const char* hdl_name（） const { return "inv"; } // NC mode name
};
-------
inv.cpp
-------
#include "inv.h"

-------
inv_test.h （SystemC Testbench Header file）
-------
#include “inv.h“
#include “systemC.h”
SC_MODULE（inv_test） {
  sc_signal<sc_logic> z;
  sc_signal<sc_logic> a;
  inv i1;
  void test_thread();
  SC_CTOR（inv_test）: a（"a"）, z（"z”）,i1 (“i1”) {
        I1（z, a）;
        SC_THREAD（test_thread）;
  }
};
-------
inv_test.cpp （SystemC Testbench file）
-------
#include "inv_test.h"
void inv_test::test_thread() {
  a = SC_LOGIC_0;
```

```
  wait(10, SC_NS);
  a = SC_LOGIC_1;
  wait(10,SC_NS);
  cout<<"TEST DONE!"<<endl;
  sc_stop();
}
SC_MODULE_EXPORT（inv_test）
```

编译、仿真步骤如下：

```
ncvlog  inv.vlog
ncshell  -import  verilog  -into  systemC  inv:vlog
ncsc  inv_test.cpp
make  COMPILER=gnu  SYSTEMC_SOURCES="inv.cpp inv_test.cpp"
ncelab  -loadsc  systemC  -sctop  inv_test  inv:vlog
ncsim  inv:vlog
```

3.2.2 SystemVerilog

1. 概述

本章节没有罗列 SystemVerilog 的全部的语法结构，有需要的可以参考 SystemVerilog 的 LRM，这里只是介绍了 SystemVerilog 的部分常用和基本的语法，目的是使读者能够用 VCS 和 SV 独立地开发 testbench，能够快速地 SVTB 有一定的了解。

这里关注的重点将是：

- 怎样将 DUT 和 SVTB 连接以来？
- 怎么做随机约束测试？
- 怎样实现并发激励？
- 怎样做功能覆盖率分析？

为什么选择 SystemVerilog 开发我们的验证环境？

SystemVerilog 提供了丰富的语法结构来支持实现高级验证方法来验证越来越复杂、规模越来越大的设计。这些方法包括事务级的验证方法 TBV，覆盖率驱动的验证方法 CDV，随机约束测试的验证方法 CRT，基于断言的验证方法 ABV 等等。

SystemVerilog 作为一种硬件描述和验证语言 HDVL，是在 Verilog 基础上扩展来的。在 2004 年，Accellera 委员会把 SystemVerilog3.1 定为一个标准。SystemVerilog 发展的很快，目前大部分 EDA 厂商的工具都已经支持它。也由于它是一个标准，用 SystemVerilog 开发的验证 IP 可以得到最大限度的重用，而不必关心项目组使用的是 Synopys 的工具，还是 Candence 的工具。

2. 仿真环境

我们一般开发仿真环境包括 4 步：

- 创建一个 interface 与 DUT 相连接；
- 创建一个仿真程序的入口（program block）；
- 创建顶层模块；
- 编译和运行；

(1) 接口（Interface）

不同的模块之间需要进行信息传递，那么模块间复杂的信息传递需要创建一个新的实体，例如一个复杂的设计顶层也许有太多的信号连接，有些设计不同的模块间需要传递的信号多达几十根、上百根信号。

Interface 主要封装了这几类信息：

- 信号间的连接关系；
- 方向信息（通过 modport block）；
- 时序信息（通过 clocking block）；
- 功能信息（通过 assertion、initial/aways block）；

Interface 默认是编译时指定连接关系，但是如果定义为 Virtual interface，那么就是在运行时指定连接关系了，下面的章节会讲到。

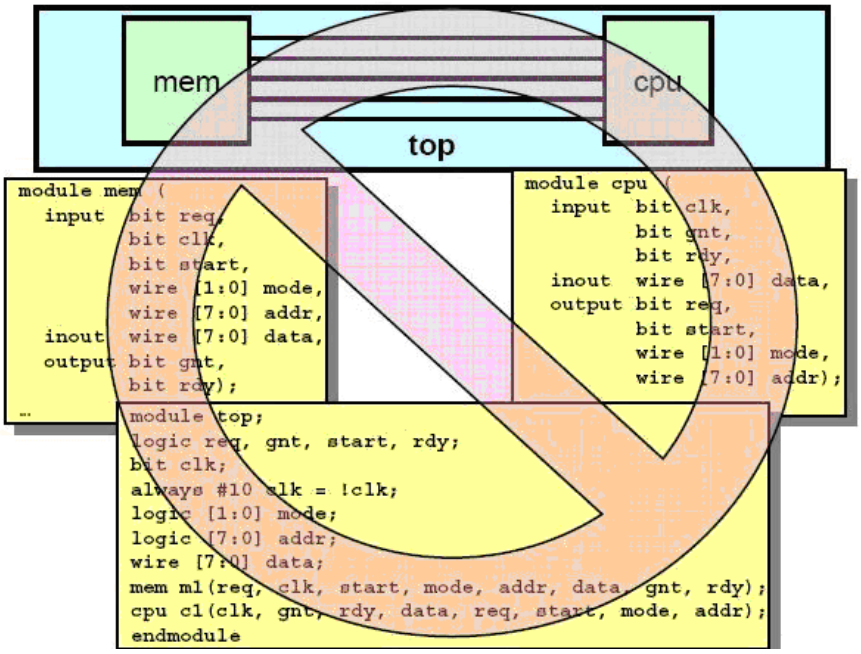


图3-23 直接用 netlist 连接的 top

图 3-23 中，我们在 top 模块中实例了 2 个模块，2 个模块间的连接关系是直接通过 logic、wire 定义的 netlist 连接的。这一般是 verilog 的写法，有过设计经验的朋友会很熟悉。

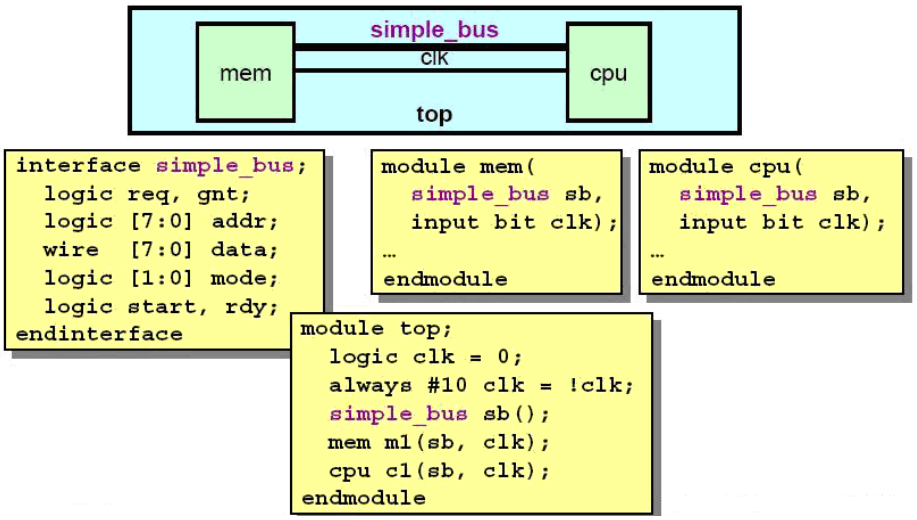


图3-24 采用 interface 连接的 top

在图 3-24 中，我们将 mem 和 cpu 这 2 个模块之间的连接关系封装到了一个 interface 中，我们看到在 top 模块中，我们象对待 mem 和 cpu 一样，把 interface 实例了一个 sb，在 mem 和 cpu 的端口列表中直接引用实例后的 interface，相比较图 3-23，采用 interface 封装后的 top 模块简洁了许多。不过值得注意的是图 3-24 中 mem 和 cpu 模块是用 SystemVerilog 编写的，Verilog 的端口列表是不支持 interface 结构的。

我们以下面一段代码为例说明 interface 中的几个特性：

```
        module cpu(simple_bus sb, input bit clk);
    logic addr_reg;
    always @(posedge clk)
    sb.addr <= addr_reg;
        endmodule : cpu

interface simple_bus;
    logic req, gnt;
    logic [7:0] addr;
    wire [7:0] data;
    logic [1:0] mode;
    logic start, rdy;
endinterface: simple_bus
```

- 多驱动的信号必须定义成 wire，例如双向数据总线等；
- 过程赋值中的信号必须定义成 logic，这里主要指 always block 中，建议除了多驱动的信号都定义成 logic 类型；
- 赋值语句必须用非阻塞性赋值；

```
interface arb_if (input bit clk);
    logic [1:0] grant, request;
    logic reset;

    clocking cb @(posedge clk);
        input grant;  // TB input
        output request; // TB output
```

```
    endclocking

    modport DUT ( input clk,
                  input request, reset, // Design under test
                  output grant);
    modport TB (  clocking cb,    // Synch signals
                  output reset); // Async signals
endinterface: arb_if
```

上面一段代码是 interface 的标准写法，clocking block 定义了 interface 中的同步时序特性，clocking block 中定义的 input、output 是相对于 testbench 而言的。Interface 可以输入多个 clk，对应的也可以有多个 clocking block，clocking block 中的信号是与@(posedge clk)中的 clk 时钟上升沿同步的，当时钟的边沿省略时，例如@(clk)，意思是 clocking block 中的信号与 clk 的上升沿、下降沿都同步，注意对输出信号是上升沿、下降沿都驱动。这里默认 input skew 为 1 个时间单位，output skew 为 0。

modport block 指明端口的方向特性，注意，如果 DUT 采用 Verilog 编码，modport 在 DUT 中我们是用不到的，因为 Verilog 的语法不支持 interface 结构。

(2) 程序块（Program block）

Program block 封装了 testbench 代码，它作为一个仿真程序执行的入口，隔离了仿真代码和 DUT 代码。

Program block 在 top 模块中被实例，program block 里可以包含 initial block，但是不可以有 always block。program block 里面实现的是一个测试用例。

```
        program test(arb_if.TB arbif);
  initial begin
      // Asynch drive reset
      arbif.reset <= 0;
      #15 arbif.reset <= 1;
      #35 arbif.reset <= 0;
      // Synch drive request
      ##1 arbif.cb.request <= 1;
      ##1 arbif.cb.request <= 0;
      wait (arbif.cb.grant == 1);
             end
        endprogram


        interface arb_if (input bit clk);
            logic grant, request, reset;
            clocking cb @(posedge clk);
      input grant;
      output request;
             endclocking

  modport TB (    clocking cb,
                  output reset);
        endinterface: arb_if
```

采用 SystemVerilog 语法编写的文件后缀名都为*.sv。

上面一段 program block 中包含了一个 initial block，#15（#35）指的是延迟 15（35）个时间单位；##1 指的是延迟 1 个时钟沿。

```
##1 arbif.cb.request <= 1;
```
这行代码中中如果 cb 省略，编译会报错：

```
"Error: arbif.request not visible via modport"
```

注意信号调用的层次结构。

如果我们在 testbench 中需要延迟一个 clk 周期，可以用`@arbif.cb` 表示。

(3) 编译和运行

以 Synopsys 公司的仿真工具 VCS 为例，编译命令:

```
vcs -sverilog -debug top.sv test.sv dut.sv
```

其中：

```
-sverilog        Enable SystemVerilog constructs
-debug           Enable debug except line stepping
-debug_all       Enable debug including line stepping
```

运行命令:

```
simv +user_tb_runtime_options
```

其中：

```
-l logfile       Create log file
-gui         Run GUI
-ucli            Run with new command line debugger
-i cmd.key       Execute UCLI commands
```

3. 基本语法

SystemVerilog 即可以开发 testbench，也可以用来做设计。Verilog 支持的语法结构，SystemVerilog 基本都支持，并且为了满足验证的需求，SystemVerilog 提供了丰富面向对象的编程结构。

SystemVerilog 的语法结构主要包括：

- 数据类型
- 数组
- 子程序
- 接口

(1) 数据类型

【基本数据类型】

SystemVerilog 继承了 Verilog 大部分数据类型，并在此基础上进行了扩展。

在 Verilog 中，reg 类型只能用于在过程赋值语句中，在 SystemVerilog 中对 reg 类型进行了扩展，产生了一种新的数据类型 logic，logic 类型除了不可以用来定义有多个驱动源的信号，基本上其他信号都可以用 logic 来表示。

此外，SystemVerilog 还提供下列数据类型：

```
bit [31:0] b;        // 2-state bit 0 or 1
```

```
integer i;           // 4-state, 32-bits, signed Verilog-1995
int i;               // 2-state, 32-bit signed integer
byte b8;          // 2-state, 8-bit signed integer
shortint s;       // 2-state, 16-bit signed integer
longint l;           // 2-state, 64-bit signed integer
```

2-state 的变量效率会比 **4-state** 高，但是 **2-state** 不能用于描述跟 DUT 相关的信号，因为 **2-state** 不支持 X 和 Z。

【用户自定义类型】

SystemVerilog 也支持用户自定义数据类型，例如下面这段代码：

```
typedef bit [31:0] uint;
typedef bit [0:5] bsix_t;  // Define new type
bsix_t my_var;             // Create 6-bit variable
```

这里定义了一种新的数据类型 **bsix_t**，代表 **bit [0:5]**。

```
typedef struct {    bit [7:0] opcode;
                    bit [23:0] addr; }
instruction;               // named structure type
instruction IR;            // define variable
```

不再一一列举。

【枚举类型】

SystemVerilog 是一种 OOP 的语言，它数据类型与 C、C++比较类似，也包括枚举类型（enum type）。

```
// Declare single enum variable
enum {RED, BLUE, GREEN} color;
// declare data type
typedef enum {INIT, DECODE, IDLE} fsmstate_t;
fsmstate_t pstate, nstate; // declare variables
int i = 1;
case (pstate)
    IDLE: nstate = INIT; // data assignment
    INIT: nstate = DECODE;
    default: nstate = IDLE;
endcase

$display("Next state is %0s", nstate.name);
nstate = fsmstate_t'(i); // cast integer to enum
```

上面一段代码包含了枚举类型和用户自定义类型。

(2) 数组

【静态数组】

静态数组指的是长度固定的数组，静态数组支持多维数组。一般常用的是 1 维、2 维、3 维数组。

如果对超出数组长度范围进行写操作，这个动作将会被忽略；而对超出数组长度范围进行读操作时，返回值将是 X，既是是 2-state 变量。

```
int twoD1[0:7][0:23];  // 2D array
```

```
int twoD2[8][24];       // same as above
```

一般数组有上面 2 种表示方法，表达的意思是一样的。

下面是 3 维数组的表示方法：

```
bit [3:0][7:0] bytes [0:2];
```

【动态数组】

在 SystemVerilog 中，动态数组也是我们经常要用到的，例如创建一个 transaction，我们经常会包含一个数据包，那么数据包都是不定长的，这里就要用到动态数组。

以下面一段代码为例，我们介绍一下动态数组的一些特性：

```
int d[], b[];           // Two dynamic arrays
d = new[5];             // Make array with 5 elements
foreach (d[j])          // Initialize
d[j] = j;
b = d;                  // Copy a dynamic array
b[0] = 5;
$display(d[0],b[0]);    // See both values (0 & 5)
d = new[20](d);         // Expand and copy
d = new[20];            // Allocate 20 new integers
                        // Old values are lost
d.delete();             // Delete all elements
```

动态数组和静态数组是类似的，不同的地方是，静态数组在编译前就指定了数组的长度；而动态数组的范围是运行的时候确定的。

动态数组使用前，必须调用构造函数 `new[]`，用到几个就需要分配几个空间，对超出动态数组长度的内容赋值编译会出错的。

初学者经常会遇到这样一个错误：

“Error: null object access。。。”

95%以上都是访问动态数组越界，或者是根本没有对动态数组分配内存空间；另外 5%可能是对一些没有分配内存空间对象访问。

我们还经常需要对动态数组重新指定其长度，这也需要用到 `new[]`，这里需要注意 `d = new[20](d)`与 `d = new[20]` 的区别，前者是保留动态数组中原有的数据，再在此基础上扩展；后者是重新分配 20 个内存空间，动态数组中原有的数据都被清除掉了。

SystemVerilog 中的动态数组的语法跟 class 的语法类似，体现了 OOP 编程的特性。例如使用动态数组前需要调用构造函数 new()，如果不需要这个动态数组了，还可以调用动态数组自带的方法 delete()删掉数组中所有的元素。

【队列】

队列也可归为数组一类。队列和动态数组一样，长度是运行时指定的，区别是队列提供了更多的函数或方法，实现一些更高级的用法，而且队列只支持 1 维的。超出队列范围的操作编译会报错。

```
int q[$] = {0,1,3,6};
int j = 2, b[$] = {4,5};
q.insert(2, j);             // {0,1,2,3,6} Insert before s[2]
q.insert(4, b);             // {0,1,2,3,4,5,6} Insert whole queue
q.delete(1);                // {0,2,3,4,5,6} Delete element #1
q.push_front(7);            // {7,0,2,3,4,5,6} Insert at front
j = q.pop_back();           // {7,0,2,3,4,5} j = 6
q.push_back(8);             // {7,0,2,3,4,5,8} Insert at back
j = q.pop_front();          // {0,2,3,4,5,8} j = 7
$display(q.size);           // "6"
foreach (q[i]) $display(q[i]);
```

我们以上面一段代码为例说明队列的一些基本的语法结构：`int q[$] = {0,1,3,6},$`是表示一个队列的声明，并且赋了初值。

队列提供了 `insert()`、`delete()`、`q.pop_back()`、`q.pop_front()`等方法，具体怎么使用的，参见上面代码中的注释。SystemVerilog 中还提供了一些更高级的方法，例如 find_first_index(exp)、find_last_index(exp)等等，这里就不多赘述了，具体用法可以参考 SystemVerilog 的 LRM。

【相关数组】

上面几类数组结构在内存中的存放是连续，相关数组可以被分配给不连续的内存地址空间，相关数组的语法结构如下：

```
int aa[*], i;
// Print full array
foreach(aa[i])
    $display(i,,aa[i]);
```

`aa[*]`表示相关数组，`i` 表示相关数组的长度。

SystemVerilog 的数组结构也体现了面向对象语言的特性，SV 的提供了许多方法对数组中的元素进行操作：

```
int q[$] = {1,3,5,7}, tq[$];
int d[] = {9,1,8,3,4};
int f[6] = {1,6,2,6,8,6};
$display(q.sum, q.product);         // 16 105
tq = q.min();                       // {1}
tq = q.max();                       // {7}
tq = f.unique;                      // {1,6,2,8}
tq = d.find with (item > 3);        // {9,8,4}
tq = d.find_index with (item > 3); // {0,2,4}
```

这里 tq 需要是个队列。

`q.min()`找出 `q` 里的最小值；

`q.max()`找出 `q` 里的最大值；

f.unique 找出 f 里的唯一值；

d.find with (item > 3)找出 d 里大于 3 的值；

d.find_index with (item > 3)找出 d 里大于 3 的值的序号；

还有很多其他的高级方法，请参考 SystemVerilog 的 LRM。

【字符串】

SystemVerilog 中的字符串的长度可以是任意长，与 C 语言中的类似。可以使用比较运算符（==, !=），可以使用{}来串联两个字符串。

SV 也提供了许多方法对字符串进行操作，例如 comare()和 icompare()等等。

(3) 子程序（任务和函数）

SystemVerilog 的 function 不能包含阻塞语句，也不可以调用 task，void function 没有返回值。

```
task reset();
    reset_l = 1'b0;
    #100
    reset_l = 1'b1;
endtask

function int add2(int n);
    return n + 2;
endfunction

function void print_sum(ref int a[], input int start=0);
    int sum = 0;
    for (int j=start; j<a.size; j++)
        sum += a[j];
        $display("Sum of array is %0d", sum);
endfunction
…
print_sum(my_array);
print_sum(my_array, );
```

上面 3 段代码展示了 SystemVerilog 中 task 和 function 的基本语法。与 C 语言类似，与 OpenVerA 的区别在于 SV 以“；”和“endfunction（task）”表示函数（任务）的范围；而 VERA 是以“{”和“}”表示，其他基本类似。

task 和 function 中传递的参数类型主要有：

```
        input   - copy value in at beginning - default
output  - copy value out at end
inout   - copy in at beginning and out at end
ref     - pass by reference (effects seen right away)
        - Saves time and memory for passing arrays to tasks & functions
const   - argument is not allowed to be modified
```

这里 ref 与 VERA 中的 var 变量类似。

(4) 接口

前面已经介绍过了。

4. 面向对象的编程模式

面向对象的编程模式（OOP）与传统的编程模式是不同，传统的编程模式把数据结构和算法是分开处理的。OOP 可以更好的管理 transactions 和 transactors，这里对象（object）把相关的数据和算法封装在一起，task 和 function 是对 object 里的数据（成员）的操作。

OOP 的编程模式主要下面 3 个特点：

- Encapsulation： OOP 把相关的数据和函数封装在一起；
- inheritance： 可以对已有的对象进行扩展；
- polymorphism： 可以在运行的时候把数据传递给函数；

OOP 编程模式将一个 testbench 划分成不同的模块来实现验证的目的，OOP 易于重用和扩展，允许复杂的数据结构，可以使用 SystemVerilog 某些高级特性。object 中的变量、function 和 task 可以避免被其他代码误操作，易于调试和维护。

图 3-25 是 Verilog 和 SystemVerilog 编程风格的对比。

	HDL	OOP
	Verilog	SystemVerilog
Block definition	module	class
Block instance	instance	object
Block name	instance name	handle
Data Types	registers & wires	Properties: Variables
Executable Code	behavioral blocks (always, initial), tasks, functions	Methods: tasks and functions
Communication between blocks	Ports or cross-module task calls	calls, mailboxes, semaphores, etc.

图3-25 SystemVerilog 与 Verilog 的对比

```
class Transaction;
    // properties (variables)
    logic [31:0] src, dst, data[1024], crc;
    logic [7:0] kind;

    // methods
    function void display;
        $display("Tr: %h, %h", src, dst);
    Endfunction

    function void calc_crc();
        crc = src ^ dst ^ data.xor;
```

```
    endfunction

endclass

program automatic test;
endprogram
```

以上面一段代码为例，**properties** 和 **methods** 默认都是 public 的。SystemVerilog 中要遵循先声明后创建的原则，SystemVerilog 中每个 class 都有个默认的构造函数 new()，但是我们一般会重新定义它。

构造函数 new()是对一个声明的 object 分配内存空间和初始化的，object 使用前必须要调用 new()函数，否则编译的时候会出错，错误信息如下：

"Error: null object access in file xx.sv line xxx"

每次调用构造函数 new()，都会产生一个新的 object，具有一个新的独立的内存地址。当一个 object 不会再用时，可以通过下面的代码回收已经分配的内存地址：

```
    if (tr.done)        // No longer needed?
tr = null;      // clear handle
```

(2) 类成员的访问

可以通过 object handle 去访问类的成员，以下面代码为例：

```
class Transaction;
    bit [31:0] src, dst, data[1024];
    bit [7:0] kind;

    function void display;
        $display("Tr: %h, %h", src, dst);
    Endfunction

Endclass

Transaction tr;

initial begin
    tr = new();
    tr.src = 5;
    tr.dst = 7;
    tr.display();
end
```

class 里可以定义 static 变量，一个 static 变量是和 class 相关的，而不取决于 object。Class 里的 static 变量经常用做一个 id，用来表示当前这个 class 被实例了多少次。

```
class Transaction;
    static int count = 0;
    int id;
    …
    function new();
```

```
        id = count++;
    endfunction
```

这里我们看到在构造函数里我们对 static 变量 count 进行加计数，每次调用构造函数 new()，生成一个 object，id 是 object 生成次数的计数。

(3) 类拷贝

Assignment 不是拷贝，Object 彼此间的赋值操作不能拷贝数据，而只能影响 handle。

```
class Thing;
   int data;
endclass
…
Thing t1, t2;          // Two handles
initial begin
  t1 = new();          // Allocate first thing
  t1.data = 1;
  t2 = new();          // Allocate second
  t2.data = 2;
  t2 = t1;             // Second Thing is lost
  t2.data = 5;         // Modifies first thing
  $display(t1.data); // Displays "5"
end
```

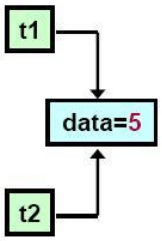


图3-26 Object assignmet

如图 3-26，class Thing 产生 2 个 object，t1，t2，将 t1 赋给 t2，这时改变的是 t2 的 handle，赋值结束后，t2 的 handle 指向了 t1 的地址。

那么怎样拷贝一个对象呢？

与 OpenVera 类似，我们可以用这样的语法：t2 = new t1；这时 t2 会重新被分配给一个地址空间，但是这种拷贝属于 shallow copy，只有 data 能被拷贝。

(4) 继承

继承是 OOP 编程风格的一大特点，我们通常把一些公用的代码封装到一个 base class 中，可以通过扩展 base class 来得到我们需要的各种其他的类。

继承实现了类与类之间代码的共享。

继承允许我们对一个 base class 增加特定的属性（数据），增加特定的方法，或者改变一个已经存在的方法的具体行为。

继承的优点是尽量重用已经实现的 base class，能够减少调试时间和不会与已经调试过的工作冲突。

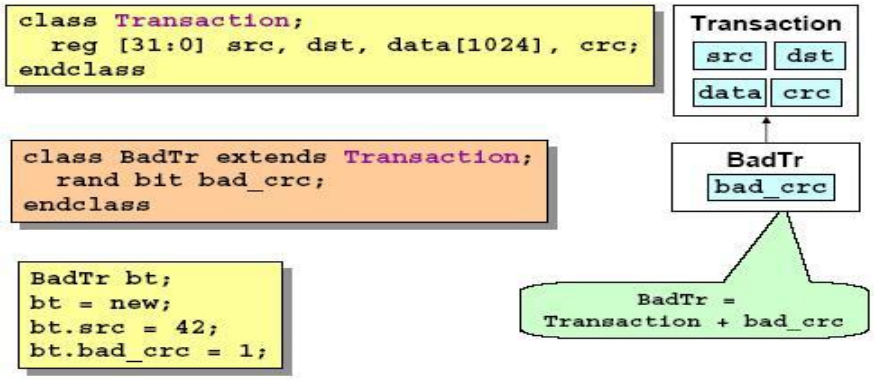


图3-27 Inheritance – extended a class with new fields

上图中，我们产生一个 Transaction 的扩展类 BadTr，在 BadTr 中新增了一个 data field。

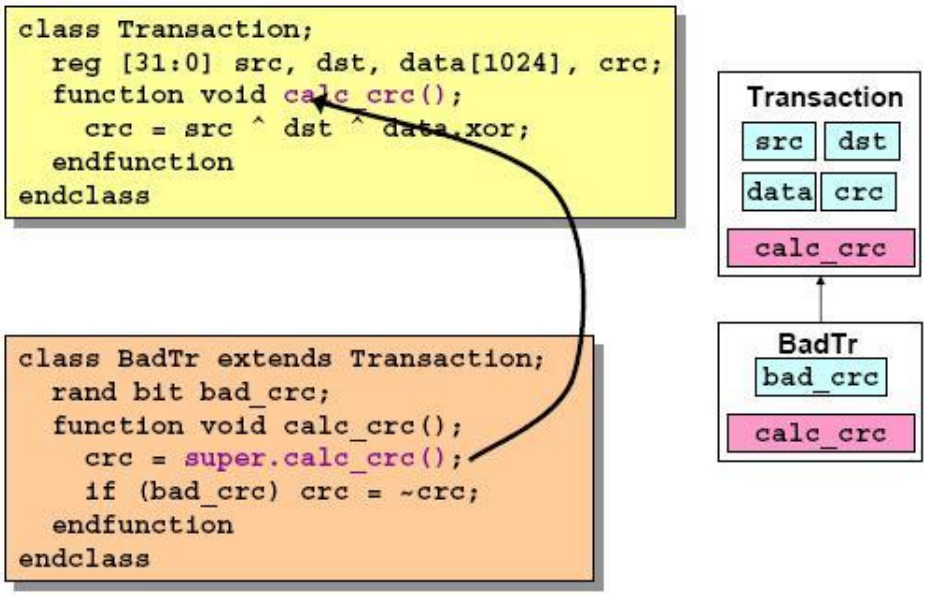


图3-28 Inheritance – override existing fields to a class

上图中，我们在扩展类 BadTr 中，对一个基类 Transaction 中的一个方法 calc_crc()进行了重载。

在系统验证中，我们经常会做一些异常测试，比较在输入激励中增加一些错误的 pattern，检测 DUT 是否能够识别出这些错误的 pattern。对于错误的引入，我们通常采用继承来实现。在扩展类里引入错误。

继承特性使得 driver（BFM）可以不必区分不同的 transactions，例如

Cell.display()

_ Print ATM cell data if I'm at ATM cell

_ Print Ethernet MCA data if I'm an Ethernet packet

_ Print Sonet frame data if I'm a Sonet frame
_ Print USB packet data if I'm a USB packet

这里具体打印什么信息，driver 不必清楚，Testcase 里输入那种 cell，打印的就是那种 cell 的信息，这样的特性提高了 driver 的重用。

(5) 多态性

SystemVerilog 中提供一个关键字：virtual，用来实现扩展类与基类之间方法调用的多态性。

多态性允许用一个对象的名称去调用不同类的方法。

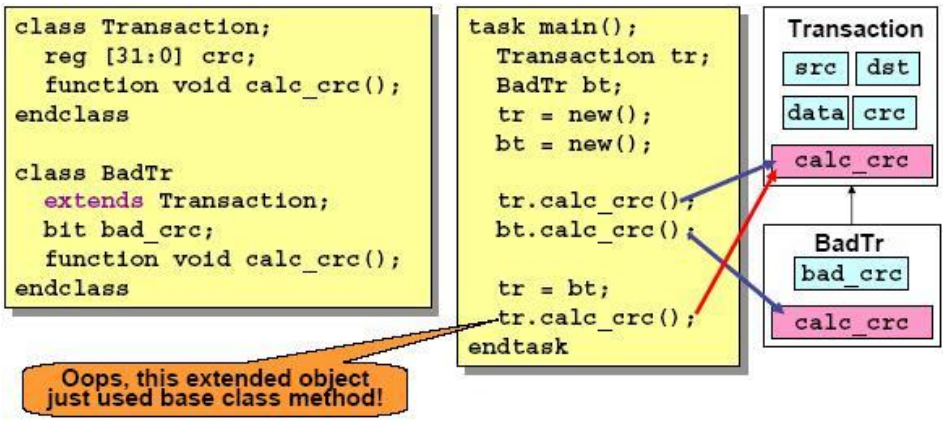


图3-29 Inheritance – overridden

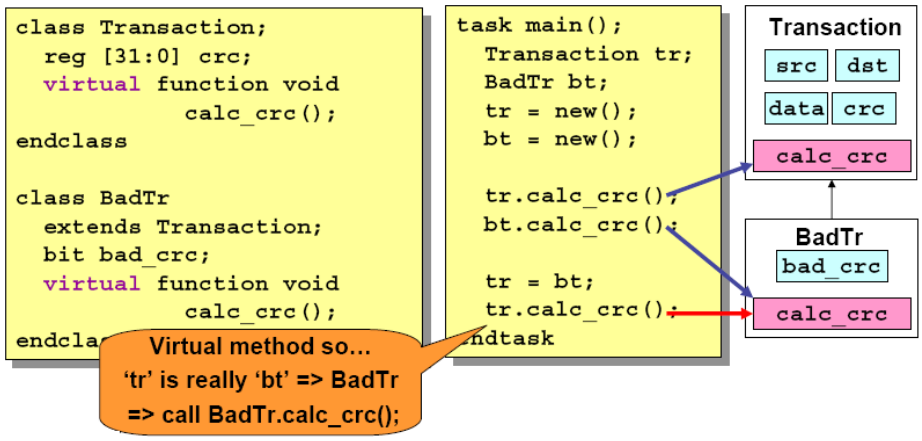


图3-30 polymorphism

从上面 2 个图中，我们可以看到，基类与扩展类中没有用到 virtual 关键字的方法，是重载；基类与扩展类中用到 virtual 关键字的方法，是多态。

带 virtual 关键字的方法是运行的时候（run time）去查找这个方法属于哪个类；而不带 virtual 关键字的方法是编译的时候（compile time）查找这个方法属于哪个类。

这里我们要注意基类和扩展类之间的 handle 赋值。

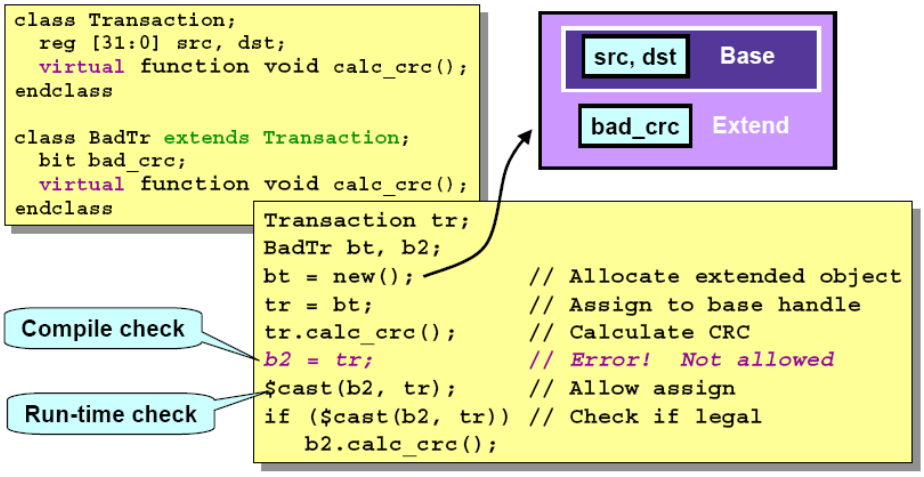


图3-31 handle assignment

在上图中，我们看到基类的 handle 指向扩展类的 handle 是允许的；扩展类的 handle 指向基类的 handle 是要报错的。这里我们用$cast(b2, tr);这个语法，cast 与 virtual 类似，都是在运行的时候检查，这里又有 tr 的 handle 前面已经指向了 bt 的地址，所以这条语句执行后，b2 的 handle 指向了 bt 的地址。

5. 随机

随机测试能够提高功能覆盖率，直接测试通常用于检查可以预见到的 bug，随机测试用于发现我们难以预见的 bug。

我们现在用到的随机发生器都是伪随机发生器，跟随机种子是相关的，运行相同种子的随机测试，得到的结果是一样，这有助于我们回归测试定位问题。

我们以下面一段代码为例，说明随机特性：

```
program automatic test;
    class Transaction;
        rand bit [31:0] src, dst, data[];  // Dynamic array
        randc bit [2:0] kind;              //  Cycle  through  all
    kinds
        constraint c_len{
            data.size inside {[1:1000]}; } // Limit array size
  endclass

  Transaction tr;

  initial begin
    tr = new();
    assert(tr.randomize());
    send(tr);
  end

endprogram
```

关键字 rand 表示随机变量在随机范围内任意取值，可以重复；

关键字 randc 表示随机变量在随机范围内任意取值，但是每次取值不会重复，直到所有的值都覆盖到后，再重新下一个循环。

可以用 **randomize()**对类的成员的随机初始化。

assert(tr.randomize());是对 tr 随机化，如果没有被分配到内存空间，就报错。

SVTB 中提供了两个 void function:pre_randomize()和 post_randomize()用来扩展随机操作。

pre_randomize()一般用来设置随机权重；**post_randomize()**一般用来对 randomize()后的随机数据进一步处理。

完全随机测试随机到我们感兴趣，或者我们需要的激励需要很长的时间，所以验证方法学中提到的随机测试，都是指的约束随机测试。

通过 constraint blocks 约束所有可能的激励中的，我们需要的一个子集。在 constraint blocks 中，可以定义各种随机变量出现的权重。约束和权重可以形成一个基准作用到激励产生器上。

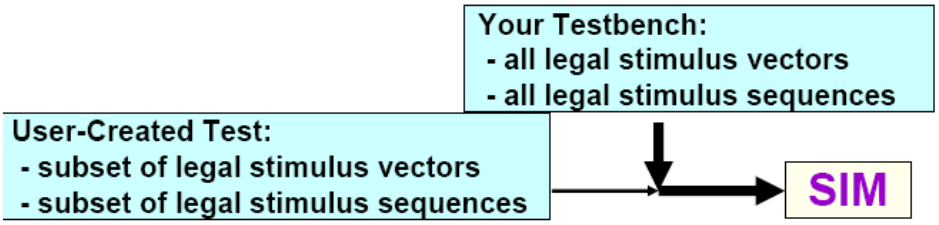


图3-32 约束与权重对仿真的作用

(2) Constraint Blocks

Constraint Blocks 是由关系表达式组成的，而不是赋值，Constraint 可以通过：

handle.[constraint_name.]constraint_mode(1/0)

动态地被使能或者禁止，有冲突的约束会产生一个 run-time 错误。

CC

```
constraint c_default {
   data.size <= 1000;
   data.size > 0;
   kind == 0;  // Equivalent, not assignment
   cntrl inside {[2:10], 20, 40, [100:107]};
   if (test_mode == CONGEST)
      dest inside {[src-100:src+100]};
}

constraint c_long {
   data.size > 5000;
}
```

Disable this with:
handle.c_long.constraint_mode(0)

图3-33 Constraint Block

(3) 分布权重

分布权重主要有两种：

：=表示赋予每个元素的权重；

：/ 表示在一个范围内平均分配权重；

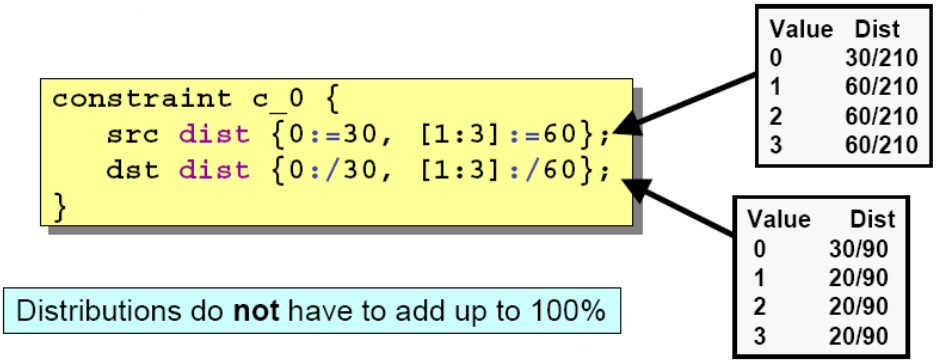


图3-34 分布权重

(4) In-Line 约束和分布

SystemVerilog 支持在随机时再定义约束，可以不因为某个单独的测试用例修改原始 class 的约束，通过 with 实现。

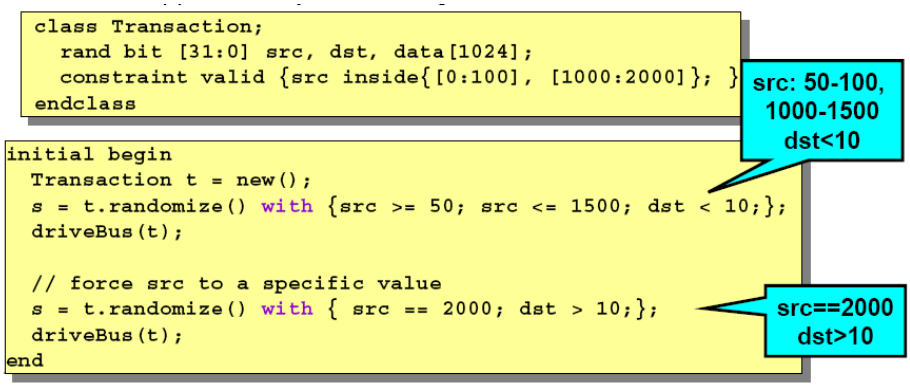


图3-35 In-Line 约束和分布

(5) 随机数的系统函数

如果我只想产生一个简单的随机数，可以调用系统函数$urandom_rage()来实现：

```
$urandom_range(0, 2);
```

每次随机地从 0，1，2 中选择一个数值。

6. 控制进程

这里主要介绍并行进程的能力、并发机制的定义、产生和控制并行进程和进程间的通信。

(1) 并行进程

并发能力是对验证环境的基本要求，每一个我们验证的设计基本上都不止一个接口，需要每个接口都并行地接收发送数据。

SystemVerilog 中同样提供 fork-join 结构允许并行进程的执行。

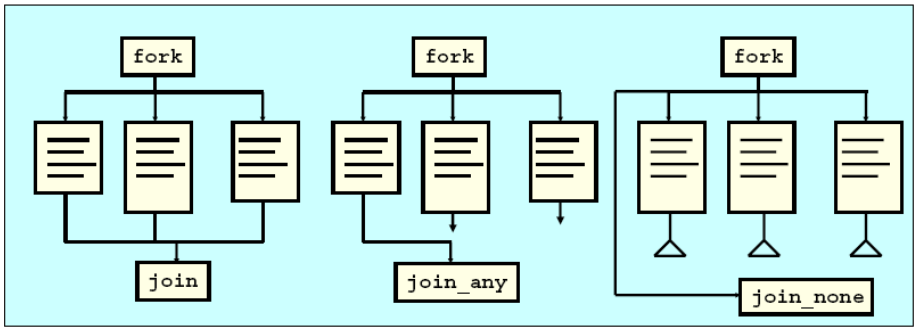


图3-36 Fork-join

这 3 种结构在 OpenVera 里有，SystemVerilog 中提供了更灵活的操作。

disable label ：可以终止 label 定义的块执行。

disable fork ：终止所有子进程的执行。

wait fork ：挂起一个进程，直到所有其他的子进程执行完毕。

wait (expression) ：挂起一个进程，直到 expresstion 为真（level 敏感）。

@(edge-exp) ：挂起一个进程，直到 edge-exp 值变化（edge 敏感）。

例如：

```
        // Timeout example
        fork : check_block
wait (arbif.cb.grant == 1); // May never complete
#1000 $display("@%0d: Error, grant never received", $time);
        join_any
        disable check_block;
```

当等到 **arbif.cb.grant** 为高时，或者经过 1000 个时间单位都没有判断到 **arbif.cb.grant** 为高，跳出 fork-join，并且终止 wait 进程。

(2) 进程间的通信

【Mailbox】

SystemVerilog 中的很多的结构都遵循面向对象的编程风格，mailbox 也一样。下面罗列了 mailbox 使用的基本方法。

```
mailbox mbx;     // Declare a mailbox
mbx = new();     // allocate mailbox
mbx.put(p);  // Put p object into mailbox
mbx.get(p);  // p will get object removed from FIFO
success = mbx.try_get(p);    // Non-blocking version
mbx.peek(p);                 // Look but don't remove, can block
success = mbx.try_peek(p); // Non-blocking version
count = mbx.num();           // Number of elements in mailbox
```

下图是使用 mailbox 的实例。

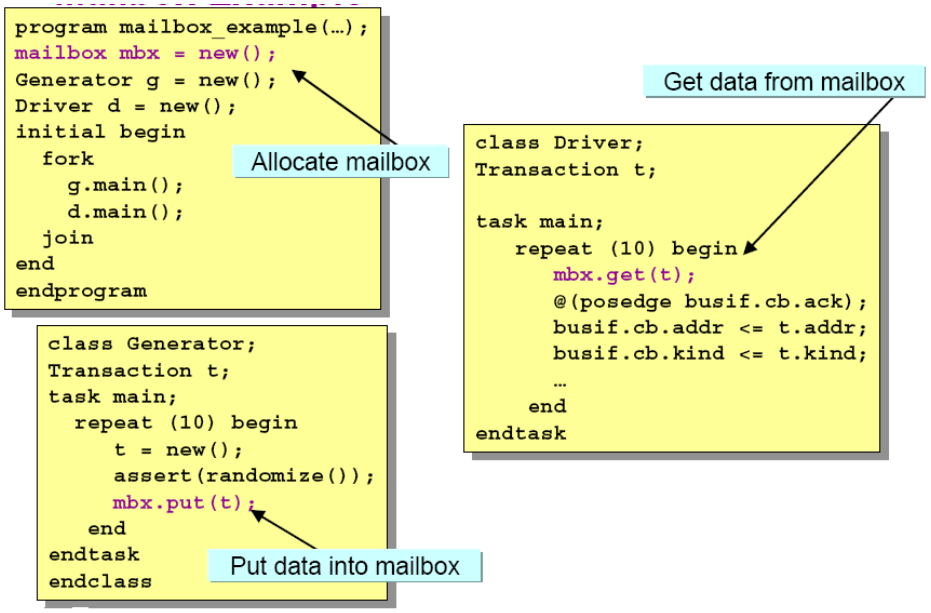


图3-37 Mailbox

【Semaphore】

Semaphore（旗语）用于同步互斥性进程。

具体语法：

```
semaphore sem;
sem = new(optional_initial_keycount = 0);
sem.get(optional_num_keys = 1);
sem.put(optional_num_keys = 1);
```

下图是使用 semaphore 的实例。

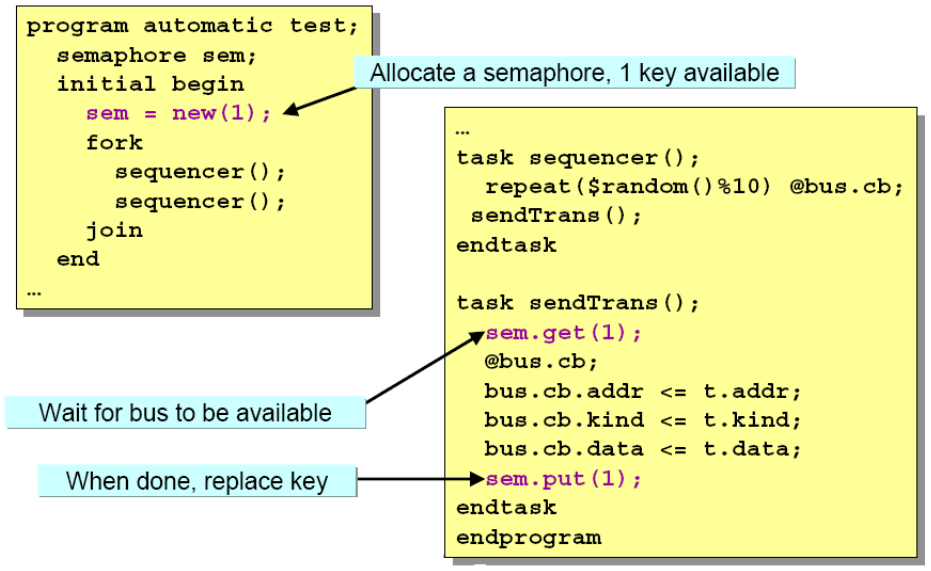


图3-38 Semaphore

【Events】

Events 用于同步并发进程。

语法：

```
event ev;     // Declare event
-> ev;        // Trigger an event
@ev;      // Block process, wait for future event
wait (ev.triggered);     // Block process, wait for event,
              // including this timeslot
              // Reduces race conditions
driver = new(ev); // Pass event into task
```

Main Testbench

```
event gen_done[4];

Generator gen[4];

initial begin
  // Instantiate testbench
  foreach (gen[i])
    gen[i] = new(gen_done[i]);

  // Run transactors
  gen[i].main();
  …

  // Wait for finish
  foreach (gen[i])
    wait(gen_done[i].triggered);
end
```

Generator transactor

```
class Generator;
  event done;
  // Pass event from TB
  function new (event done);
    this.done = done
  endfunction

  task main( );
    fork
      begin
      // Create transactions
      -> done;
      end
    join_none
  endtask

endclass
```

图3-39 Events

7. 虚拟接口

SystemVerilog 中的虚拟接口跟 OpenVera 中的虚拟端口作用是类似的，但是应用不同。

在 Openvera 中，我们在 BFM 中定义虚拟端口，在环境里将虚拟端口帮定到具体的物理借口上； SystemVerilog 定义虚拟接口，在环境里将实际的接口赋给虚拟接口，实现互连。目的是一样的，使 BFM 可以不关心实际的物理接口，提高了 BFM 的重用性。

使用 Virtue interface 的步骤：

(1) 定义物理接口
使用 clocking block 和 modports

(2) 连接接口
一般在 test_top 中

(3) 在 BFM 中定义虚拟接口
Virtural xx

(4) 连接虚拟接口到具体的物理接口上
一般在 testbench 中

(5) 在 transactors（drivers、receivers、monitors）中使用虚拟接口

8. 功能覆盖率

我们验证一个 DUT 时，总是会遇到 2 个问题：**Does it work?**和 **Are we done?**。我们前面介绍的内容是用来搭建一个测试平台（TestBench）是解决第一个问题的。

覆盖率统计是解决第二个问题。

覆盖率度量可以分为 structure coverage 或者 code coverage 和 function coverage 两类，代码覆盖率可以通过 EDA 工具自动统计，我们需要做的就是在仿真脚本中增加几行命令就可以了，不需要太多精力，统计出来的数据给设计工程师确认，对没有达到 100%覆盖的逻辑，设计工程师需要确认是否需要增加测试用例覆盖。

功能覆盖率要复杂的多，功能覆盖率是对验证需求、Feature 的建模，功能覆盖率是对设计目的的检测，因此必须手动开发。需要验证工程师开发覆盖率模型，覆盖率模型是否覆盖到验证计划及 Feature 中所有的特性，直接决定功能覆盖率是否完备。由于覆盖率模型是人开发的，所有不可避免会有遗漏，增加项目组内外相关人员的 REVIEW 是很必要的。

覆盖率度量是个建立信心的过程，达到什么程度，我们可以认为我们的设计的功能是正确的，这个目标需要在验证计划阶段就给出，但是这个目标不是一成不变的。在设计的具体实现过程中，会有基于实现阶段的覆盖率要求以及一些 corner case 的需求。就象整个验证过程一样，我们在给定的进度内只能尽可能地覆盖到所有的验证需求和 Feature，但是想要做到没有疏漏是很难的。

(1) 覆盖率模型 covergroup

我们还是参照一个例子，能够描述的更清楚些：

```
        program automatic test(busifc.TB ifc);
class Transaction;
    rand bit [31:0] src, dst, data;
    rand bit [ 2:0] kind;
endclass

covergroup CovKind;
    coverpoint tr.kind; // Measure coverage
endgroup

Transaction tr = new(); // Instantiate transaction
CovKind ck = new(); // Instantiate group
initial begin
    repeat (32) begin // Run a few cycles
        assert(tr.randomize());
        ifc.cb.kind <= tr.kind; // Transmit transaction
        ifc.cb.data <= tr.data; // onto interface
        ck.sample(); // Gather coverage
        @ifc.cb; // Wait a cycle
    End
```

```
end
        endprogram
    end
    endprogram
```

首先，我们需要定义覆盖率模型 covergroup，封装我们需要检测的一些 coverpoints 和 cross coverage，有时候需要定义 bins，也可以不定义，系统自动产生 bins。在一个 coverage group 中的变量可以是一个变量、一个 class 的成员，也可以是这些变量的 cross coverage。注意 coverage group 跟其他的面向对象的结构一样，定义一次，然后在需要的时候，可以实例化很多次。

Coverage group 也可以放在一个 class 里面，用来覆盖类的成员的一个子集。这样需要在 class 的构造函数 new()中实例 covergroup，而且通常把 coverage group 放在一个“staic” class 中，也就是 transactors 中，放在 transactions 中的 coverage group 是没有意义的，因为 transactions 的生存周期是有限制的。

(2) Bin 产生

如果没有在 coverage group 设置 bins，SystemVerilog 会自动地给每个 cover points 产生 bins，具体数目由每个 cover points 的取值空间决定，例如在下段代码中，x 的 bins 就有 16 个，对应的取值范围（0~15）。

```
        class Driver;
  bit [3:0] x, y;
  covergroup cg;
      coverpoint x;
      coverpoint y;
  endgroup

  function new;
      cg = new;
  endfunction
        endclass
```

也可以根据需要定义 bins：

```
logic [2:0] port_number;
          covergroup CovPorts;
  coverpoint port_number {
      bins lo = {[0:1]};      // 1 bin for 2 values
      bins hi[] = {[4:$]};    // 4 separate bins
      bins misc = default;    // Unspecified values
      bins t1 = (0=>1), (0=>2), (0=>3); // transitions
      }
        endgroup
```

很清楚，这里当 port_number 为 0、1 时是落在 lo 里；4~7 是落在 hi 里；2~3 是落在 misc 中。

(3) option 定义

选项（option）可以控制 covergroup、coverpoint 和 cross 的行为。

```
bit [3:0] x, y;
          covergroup Cov1;
    coverpoint x;
    coverpoint y;
    option.auto_bin_max = 2; // Divide ranges in 1/2
          endgroup
```

这段代码中用 **auto_bin_max** 定义了自动的 bins。

还有其他 name、weight、goal 等等 option，具体用法参照 SystemVerilog 的 LRM。

(4) Cross Coverage

Cross 的对象可以是一个 coverpoint 或者一个变量，cross 的 bins 都是自动产生的。Cross products 指的是 cross 中的对象的组合。

```
bit [3:0] global
class MyClass;
    bit [4:0] y;
    covergroup cg;
        cc1: cross y, global;
    endgroup
。。。
```

这里覆盖率度量会检测当 y 取 0~31，gloal 取 0~15 的任意组组合情况，bins 的总数为 32*16。

(5) 定义采样事件

可以在 coverage group 中定义采样事件，也可以调用 instace.sample()；调用这个方法是即时采样的。

有效的采样表达式包括：

```
@SV_event;
@(SV_variable);
@(interface.xx);
```

可以是事件、变量，也可以是 interface 上的信号。

Covergroup 可以利用 SVA 的行为去触发事件，采样 coverpoint。

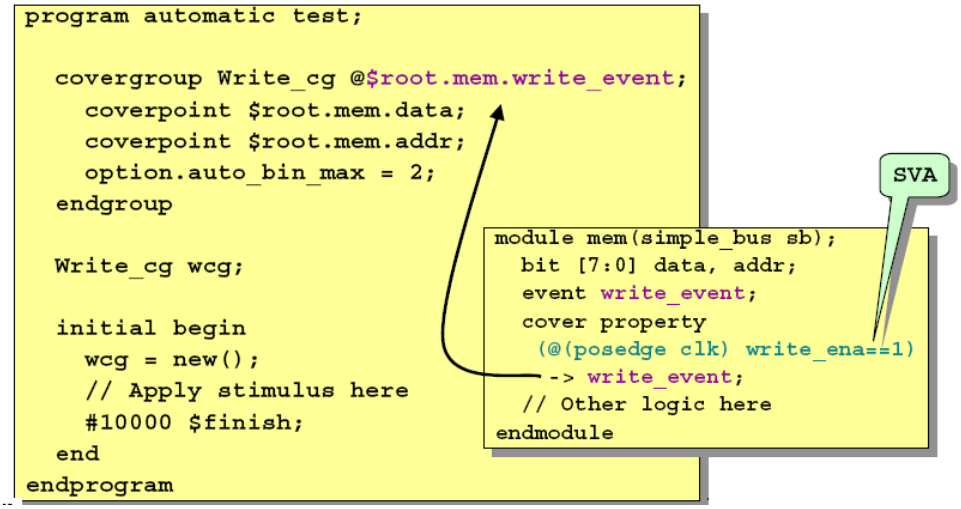


图3-40 SVA 触发覆盖率采样

(6) 覆盖率统计

由于目前 Synopsys 公司对它原有的覆盖率统计工具 CoverMeter 已不再继续 update，Synopsys 公司推荐两种统计分析功能和代码覆盖的方法。

【URG】

URG（Unified Report Generator）可以产生所有覆盖率信息的报告，但是注意 URG 不支持 path 和 branch 覆盖率统计，这两钟我们一般也不做统计，重点关心的代码覆盖率包括 Line、FSM、Condition，也许包括 Assertion 覆盖率统计。

URG 输出的报告支持 HTML 和 TXT 的格式，HTML 可以用 NetScape 打开，进行统计分析覆盖率信息。

使用 URG 时需要注意 VCS 的版本，在 VCS0606 以前的版本，urg 寻找当前目录下的_root.db 文件收集覆盖率信息。而 VCS0606 以后的版本中，urg 是寻找 simv.vdb/中的的文件收集覆盖率信息，覆盖率报告都是默认产生一个/urgReport/的目录。

用法：

VCS0606 以前的版本：

Urg –dir

VCS0606 以后的版本，包括 0606：

Urg –dir simv.vdb simv.cm

其中 simv.vdb 存放的是测试平台和 assertion 的覆盖率信息；simv.cm 存放的是代码覆盖率的信息。

【DVE】

DVE（Discovery Verification Environment）是 synopsys 的图形验证环境，是常用 VCS 仿真调试工具 VirSim 的替代品，能工作在交互模式(interactive mode)和后处理模式(post process mode) 。

可以通过 DVE 来统计分析 VCS 仿真后的功能和代码覆盖率，具体使用步骤：

a．设置环境变量

```
setenv DVE_COV_ENABLE 1
setenv DBG_PLUS_COV 1
setenv DVE_ACTIVEDRIVERS 1
```

b．键入 dve -cov

c．在 DVE 界面上选择 File，然后选中 Open Database

d．选择 simv.cm

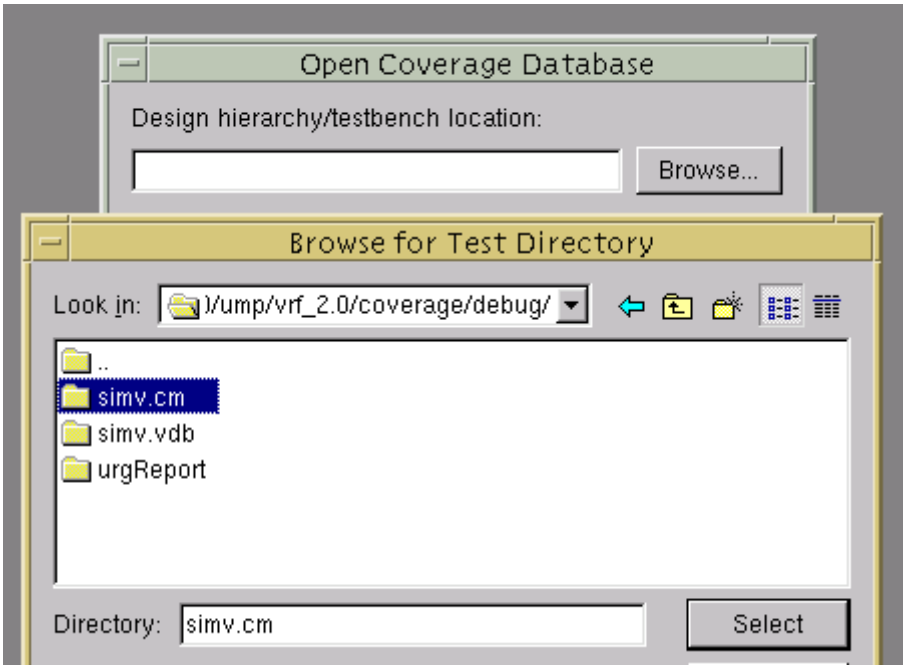


图3-41 DVE 统计分析覆盖率 1

e．选择 simv.vdb

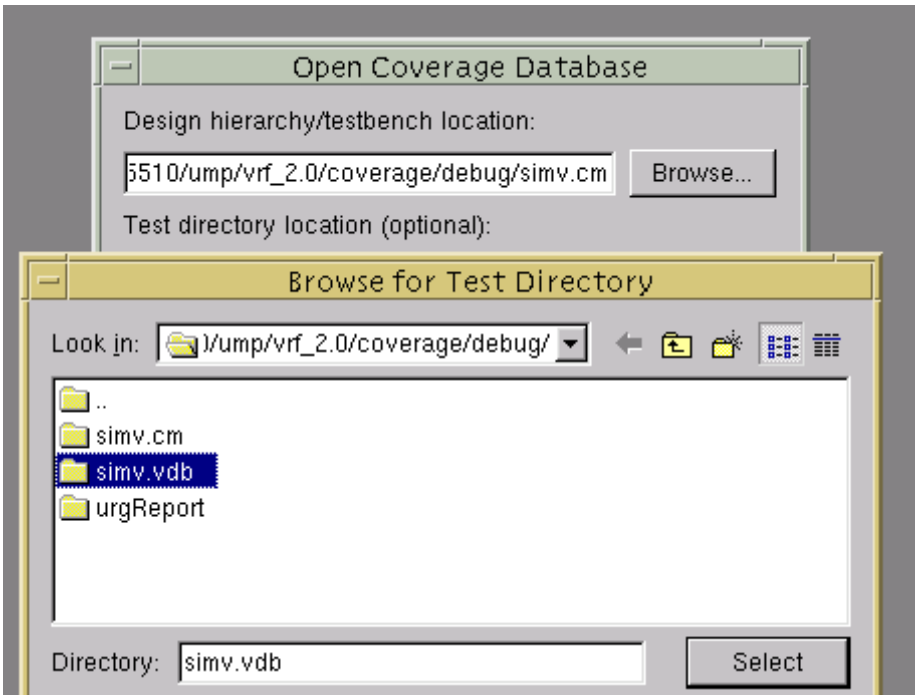


图3-42 DVE 统计分析覆盖率 2

f. 选择你要的分析的代码覆盖率或是功能覆盖率

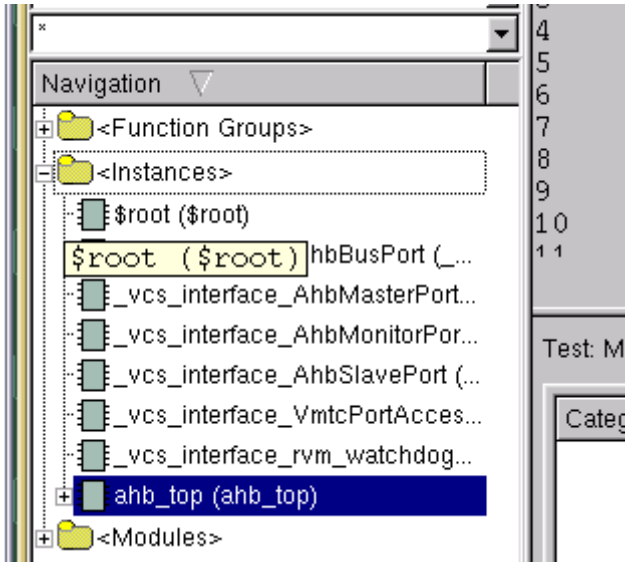


图3-43 DVE 统计分析覆盖率 3

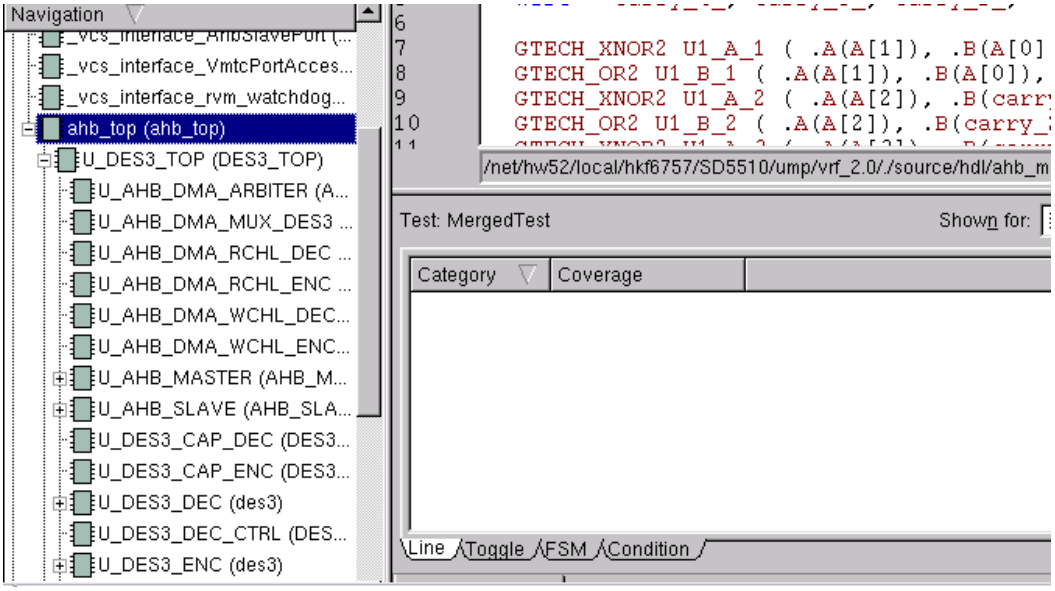


图3-44 DVE 统计分析覆盖率 4

9. 总结

本章节重点是描述如何用 SystemVerilog 搭建一个验证环境，没有对SystemVerilog的语法结构做详细的说明，读者定位是SystemVerilog的初学者，鉴于作者能力和经验，文中可能会有诸多描述不清楚的地方，欢迎来跟作者讨论。

想了解 SystemVerilog 中丰富的面向对象的编程结构和 SVA 的同事，请参考SystemVerilog LRM。

3.3 SVA 简介

3.3.1 断言简介

断言（Assertion）是对设计的属性的描述。断言的作用是保证设计目的（属性）和最终的设计实现的一致。

如果一个在仿真中被检查的属性的表现不像我们期望的那样，那么这个断言失败。

如果一个被禁止在设计中出现的属性在仿真过程中发生，那么这个断言也失败。

并发的即时的断言是 SystemVerilog 一个组成部分，他们提供了一个有效的方法提高设计错误的可观察性和定位，通过在设计内部和外部接口的关键点上放置功能检测器。当仿真中一个断言报错，错误产生的原因的定位比观测仿真进程要容易的多。

在 DAC2004，Accellera 把 SystemVerilog3.1 定为一个标准。断言语言也被纳入 SystemVerilog 语言并成为标准的一部分。这就是通常说的“SystemVerilog Assertion(SVA)”。

关于 Assertion，《Assertion-Based Design》一书开篇提了个很有趣的问题：谁应该学习基于断言的设计？设计工程师？还是验证工程师？

答案是都需要。

设计中那些位置需要加入 assertion？

设计工程师可以在具体实现时在代码的一些关键逻辑处，增加 assertion，一旦逻辑行为在某种场景没有按照预期的方式发生，assertion 就会即时报出相关错误信息。

验证工程师可以在 DUT 与环境接口处，在 Monitors 中，增加 assertion，一旦接口时序没有满足相关协议，assertion 就会即时报出相关错误信息。

Assertion 意味着正确的描述。从一个验证的角度看，一个 assertion 是对期望行为的描述。对于观测到的行为，任何与期望的不一致都会导致一个错误。基于这个定义，整个验证平台就是一个大的 assertion：对整个设计的期望行为的一个描述。这里我们提到的 assertion 指的是用时序表达式表述的设计的属性。

用 assertion 去检测和调试错误发生在具体的功能点处的功能错误已经证明是非常有效的。但是尽管他们是非常有效的，assertion 被限制于那些能用时钟控制的时序表达式的属性上。一些关于设计的期望行为的描述仍然必须被明确地用行为级代码表达出来，或者易于表达出来。Assertion 和行为级检测可以协同工作，例如，一个协议检测器可以用 assertion 来描述低层信号协议，用行为级编码来描述高级，面向事务的属性。

Assertion 擅长验证局部信号之间的关系，在检测接口间信号，状态跳变和物理层协议方面是非常有效的。他们也可以很容易地确定不被期望的，虚假的条件。

另一方面，Assertion 不适合用来检测数据传输，计算和顺序错误。例如，Assertion 很难检测出所有数据包根据他们各自的优先级都路由到了合适的输出端口。

下面一些指导原则将确定那些响应检测适合用 Assertion 还是在验证环境中实现。

1、Assertion 应该局限于验证物理层假设和响应。

时序表达式最适合用于描述 cycle-based 的物理曾关系，尽管时序表达式可以用于描述表示高层协议事件，但是由于没有时钟参考，使得描述变得很困难。而且，这些信息可能已经在 transactor 中已经得到了，在 transactor 中实现一个行为级的检测更简单。

2、不同的设计抽象层次的响应检测需求都必须满足的设计应该用过程代码在验证环境中实现。

Assertion 很依赖于 RTL 或者门级模型。他们不能轻易地被事务级模型调用。在不同的设计抽象层次的响应都要检测的需求，最好在验证平台中实现。

3.3.2 断言的作用

断言可以帮助我们及时的发现错误，如果没有它，我们就必须根据 DUT 的输出，去一个一个模块追踪产生的错误的模块，我想，大多数验证人员都有这样的经历，过程是很枯燥的。首先，验证工程师对具体的设计实现是不清楚的，他需要根据各个模块的详细说明书中定义的接口信号去判断错误发生在某个模块中，更多时候需要和设计工程师不断地交流，或者需要设计工程师亲自定位问题。

而且某些类型的错误表现形式可以很容易地在 DUT 的输出被观测到，比如丢包。然而，某些类型的错误表现形式是不清晰的，例如，一个可恢复的仲裁逻辑错误，只是让某种特定类型的业务的流量减少，这种错误通过 DUT 的输出信号是很难检测出来的。往往会被验证工程师所忽略掉。

断言在设计关键点增加检测而不用产生额外的测试平台，这些关键点实现了外部可见，具备白盒验证的特点。如果在设计内部增加了断言，一旦断言描述的属性不被满足，它会即时地打印出具体的错误信息，不需要在一个模块一个模块地追溯，极大地节省了仿真的时间。

1. 提高可观测性：

在传统的验证环境中，测试平台产生输入给 DUT 的激励，通过检测 DUT 输出的信号判断是否与期望的输出相一致。用这种方法去找 BUG，环境必须能够做到：

- 产生合适的输入能够触发 BUG。
- 产生的这个合适的输入必须能够使得这个 BUG 影响的结果能够传递到输出端口。

但是，通常我们会遇到这种情况，能够制造出产生设计内部错误的输入激励，但是这个错误不能传递草一个可以观测的输出端口。

嵌入在代码中的断言能够提高设计的可观测性。有了断言，验证环境不再只能依靠观测设计的输出来判别设计的行为。任何不符合期望的行为都可以在错误的发生源处被抓到。

2. 缩短了调试时间

我们前面提到，断言能够提高可观测性，使得我们可以在 BUG 出现的具体位置，能够即时地发现。相对地，没有加断言的验证环境，不可能直接定位到错误发生的具体逻辑电路那个位置和具体的仿真时刻，一般都是在过了若干个仿真周期后，在其他的位置（基本上都是在 DUT 模块间的接口，或者 DUT 输出端口处）才能检测到。

这需要验证工程师花费大量的时间去追溯错误发生的具体位置。

3. IP 中的断言提高了有效集成

随着设计规模越来越大，逻辑越来越复杂，设计中集成的 IP 越来越多，项目组不可能对每一个用到 IP 都了如指掌，相反，通常状况下，项目组只是把 IP 当做黑盒子来用。

但是，我们在使用 IP 时，遇到最多的问题不是 IP 本身有问题，而是不正确的使用，配置不合理，输入给 IP 的信号不符合 IP 输入接口规范。如果 IP 中存在断言代码，尤其是在接口处有检测接口协议的断言，这样的问题就很容易发现。

如果 IP 中没有断言，验证工程师在遇到一个问题时，就要不断地和设计工程师、IP 供应者进行交互，不可避免地影响了验证的效率。

3.3.3 断言的使用

目前常用的有三种断言标准：Accellera Open Verification Library (OVL)、Accellera Property Specification Langauge (PSL)和 Accellera SystemVerilog Assertion。本文中我们关注的是 SVA。

SVA 的使用可以有两种选择：

1、使用预开发的断言库（SVL）；

2、用 SVA 自己开发断言检测器；

3.3.4 断言的好处

我们最终 Taped_out 的设计中，据统计，主要有 3 类功能缺陷是跟设计错误相关的：规格错误、重用模块和 IP 的错误（可能是不正确的使用和 IP、重用模块的内部错误）。多数这些错误是由于规格的不明确或者修改，没有充分验证到模块的行为导致的。

设计者在 RTL 编码时，经常会对设计周围的一些行为和内部模块间的同步做假设。这些假设通常都是和规格无关的，或者是在仿真中没有被验证到的。任何周边设计的行为的变化或者内部的功能错误可能会与这些假设有冲突，最终导致错误。这些错误的表现形式或许在仿真很晚的时候的才能够在 DUT 的输出端口处被发现。这些没有被描述清楚的假设使得检测错误产生的原因是很困难和很花费时间的。

设计者可以用断言描述这些假设，并把他们插入到 RTL 代码中。如果这些假设与实际情况有冲突，将会在引发错误的具体位置产生一个断言错误。这种方法使得调试设计变得更容易。

当我们重用一个已经写好的 module 或者 IP 时，如果 module 的应用没有被精确的描述和测试，由于重用设计的黑盒特性，错误通常很难发现。断言在验证 IP 的使用方面可以起到很重要的作用。

SystemVerilog 中包含了一些时序结构，可以用明确的可执行的语句来提供有效的系统建模来描述设计规格，当这些描述被断言时，可以精确地描述规格的意

图或者需求。这样的断言减少了模糊的概念和错误的理解。相比于 RTL 的设计描述，断言的描述更加抽象。他们提高了仿真中检测设计错误的可能性。

最后，属性可以被用于描述在仿真中需要被覆盖到的我们关心的激励和状态。这些属性不是用来检测错误，只是用来检测某些重要的状态出现过。他们定义了设计实现中不易被功能规格覆盖到的 corner cases。

3.3.5 总结

断言（assertion)非常适合用于验证基于具体实现和物理层的问题；而验证平台（testbench）适合用来验证事务级响应。断言不象测试平台，不局限于对 DUT 的输出检测其响应。设计的完整响应可以通过验证环境和断言组合来验证。

这里我们首先介绍 SVL（SystemVerilog Assertions Checker Library），SVL 是 EDA 厂家预开发的、经过验证的断言检测器，设计人员可以作为标准单元直接调用，而不用花精力去开发自己 SVL，不会影响研发进度。

对 SystemVerilog Assertion 语言有兴趣的同事可以参照 SystemVerilog LRM。

3.4 验证工具

3.4.1 VCS 使用入门

1. VCS 的工作流程

以下是 VCS 的工作流程：

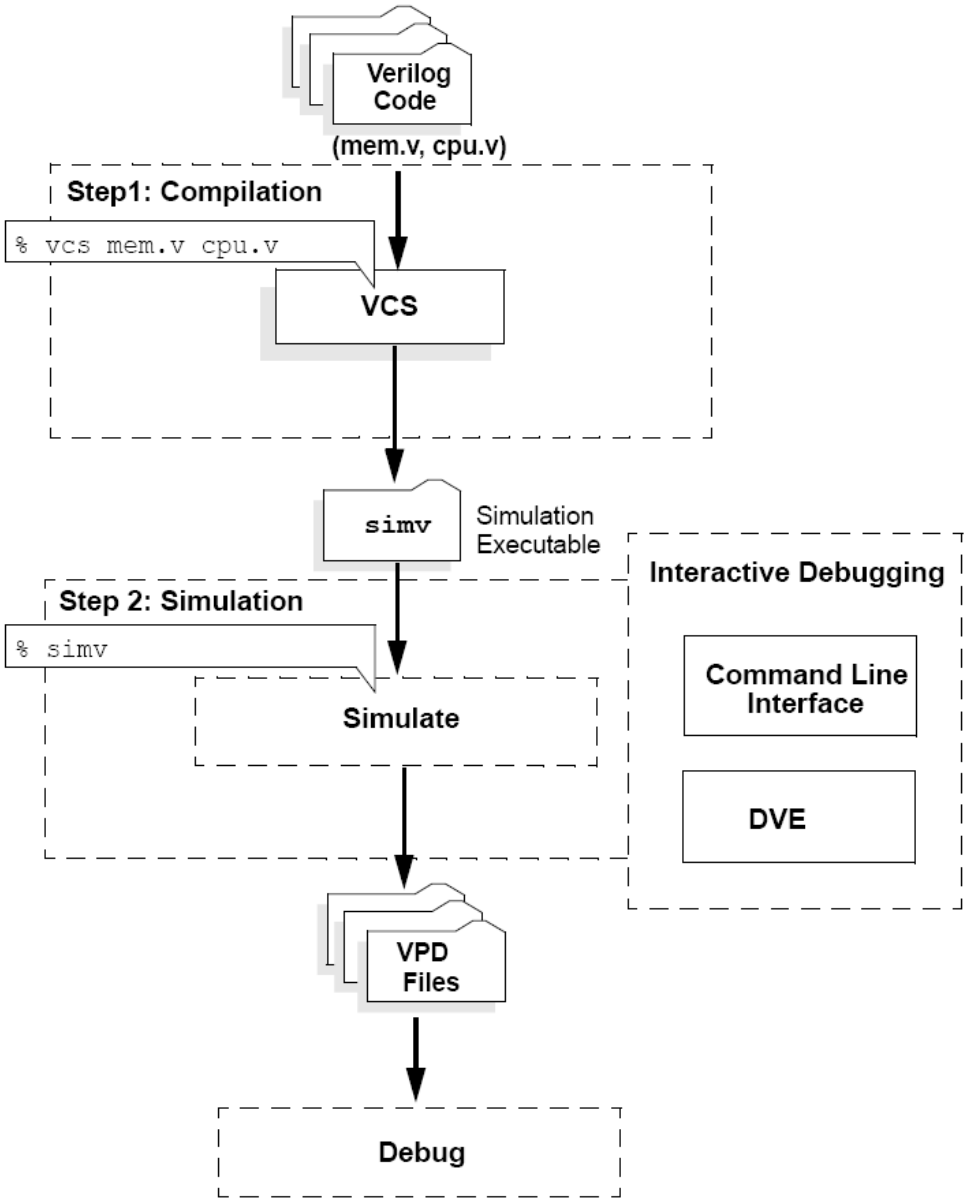


图3-45 VCS 流程图

其工作流程主要分两步：编译生成可执行文件，仿真运行。

(2) 编译

编译命令如下：

vcs source_files options

Source_file 指要编译的源文件，包括 duv 的 verilog 文件，用 NTB 或 SV 编写的验证环境文件等 VCS 能支持的源文件格式。源文件可以直接在 VCS 后面列出文件名，也可以通过文件列表加-f 选项来制定，命令如下：

vcs –f file_lst [options]

①②③④⑤⑥⑦options 指 vcs 的编译选项，基本选项如下：

- +define+MACRO=val 指定编译的宏定义值，如 verilog 文件定义的‘ifdef 定义的宏等。
- ＋incdir＋dir 指定文件中通过`include 调用的文件存放位置。
- －I 使用户能进行互动调试
- －R 指定 VCS 编译后马上启动仿真
- －L out.log 指定编译生成的 log 文件名
- －P 编译 VCS 提供的系统函数，如果用户使用了$vcdpluson 等命令时，需要加上此选项
- －lm 解析 VCS 提供的算法函数，如果用户在仿真环境中使用了 VCS 自带的算法模块，需要将此选项加上
- －line 调试单步执行选项，编译时加上此选项
- －ntb 支持 NTB 的语法
- －RI 打开交互式图形界面
- －v2k 支持 2001 的 verilog 语法
- －v lib.v 指定包含库 module 的文件名
- －y lib/dir 指定包含库文件的目录名
- +libext+v 指定在库目录下查找以.v 结尾的文件作为库文件
- －Mupdate 进行增量编译
- －RPP 启动仿真后处理，不启动编译和仿真，如分析波形等。
- -timescale=1ns/1ps 制定仿真的时间精度，如果源文件中有的文件有 timescale 有的没有，则需要加上此选项。
- -cm [line|cond|fsm|tgl|path]用于进行覆盖率统计时的编译选项，具体含义为：
  - Line 行覆盖率
  - Cond 条件覆盖率
  - Fsm 状态机覆盖率
  - Tgl 翻转覆盖率
  - Path 路径覆盖率
  - 多个覆盖率同时统计时，在覆盖率选项之间用＋进行连接如：vcs –cm line+cond+fsm

VCS 编译后生成几个文件：

- Simv 生成的可执行文件
- 目录 Csrc，编译生成的目标文件
- Simv.dadir 用户接入目录，里面存放了 design 的数据文件和设计的结构

高级选项：

- 如果编译需要的内存太大，可以通过加+memopt 或＋memopt＋2 来进行内存优化，会减少内存的需求，但会导致仿真的速度降低

- 初始化标准的 memory（如：reg 定义的多维数组）值：+vcs+initmem+0|1|x 或+vcs+initreg+0|1|x 等，但注意使用此选项时要与初始赋值语句不能冲突。
- 通过编译命令改变 design 内部定义的参数：-pvalue+hier_name=val.
- 如果要 dump 其中的 memory 的波形，需要同时加上+v2k 和+memcbk
-

(3) 运行

executable_file options

如果编译中没有加－o 选项指定生成的可执行文件名，编译生成的文件名为 simv。

Options 是运行选项。基本的运行选项如下：

1）－l 文件名 指定生成的 logfile 名

2）需要通过命令选项传递参数时，可以通过在文件中添加$value$plusargs 的方式来实现。具体可以见下面的例子：

status=$value$plusargs("r1=%d",r1)；

在命令行里可以输入

simv +r1=10 其中 rl 的值就为 10

3）+vcs+dumpvarsoff 不 dumpvars 波形

(4) 调试

1）使用 UCLI 接口进行互动式调试

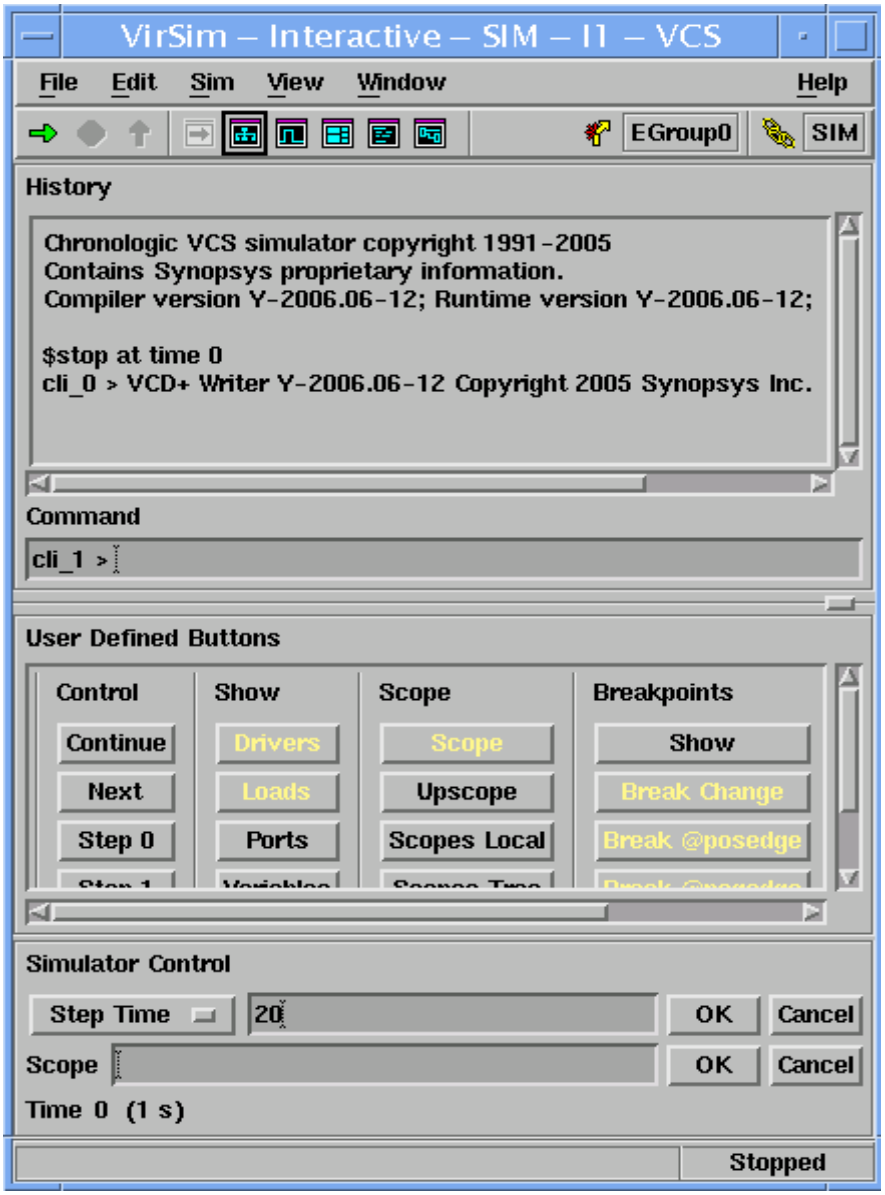
在 VCS 命令后面加-debug 或-debug_all

在运行命令后加-ucli

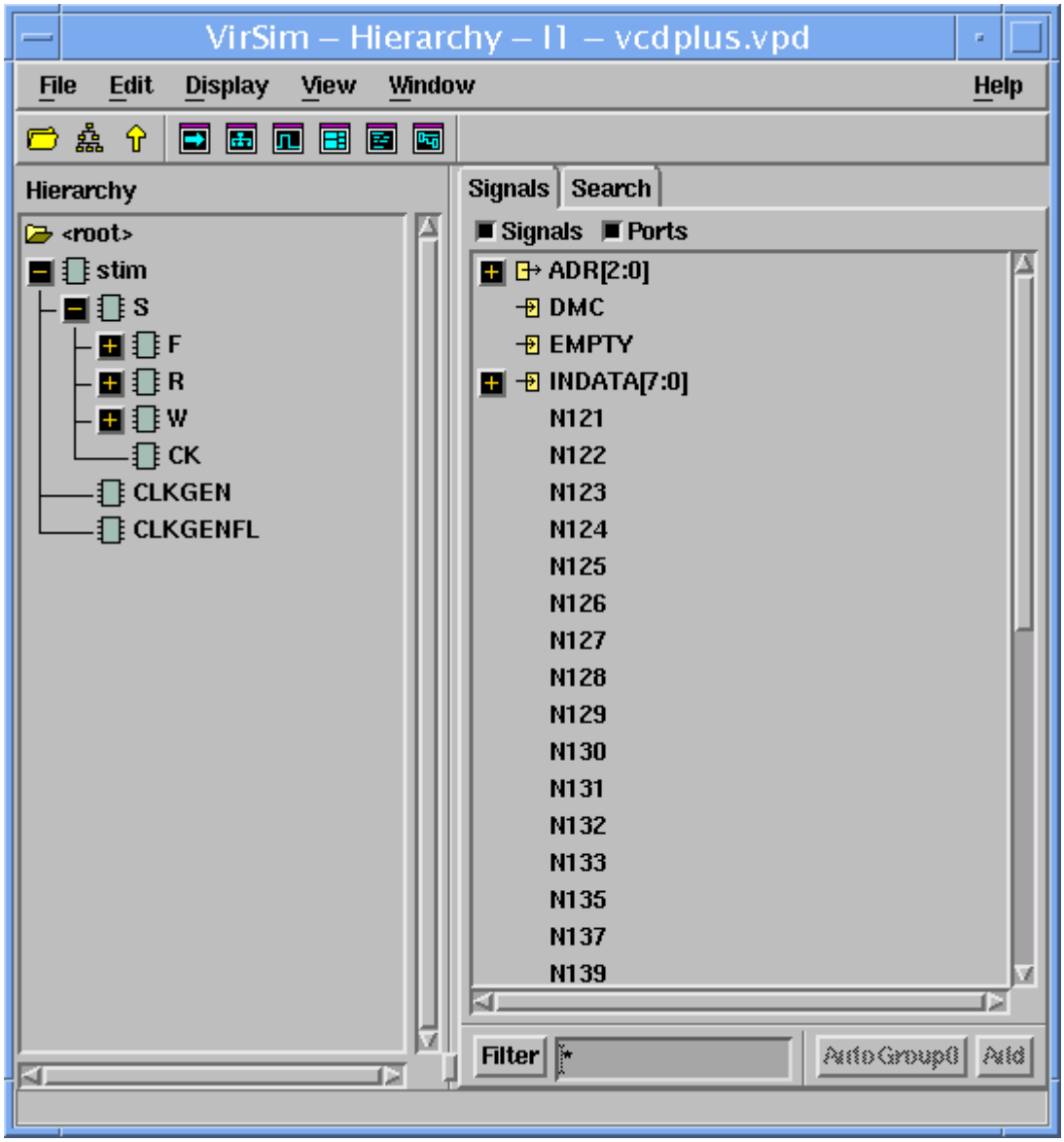
具体的 CLI 的命令参见 VCS User Guide

2）启用 VirSim 图形界面进行调试

在 VCS 编译命令加选项-RI 将启动图形调试界面 VirSim，主界面如下：

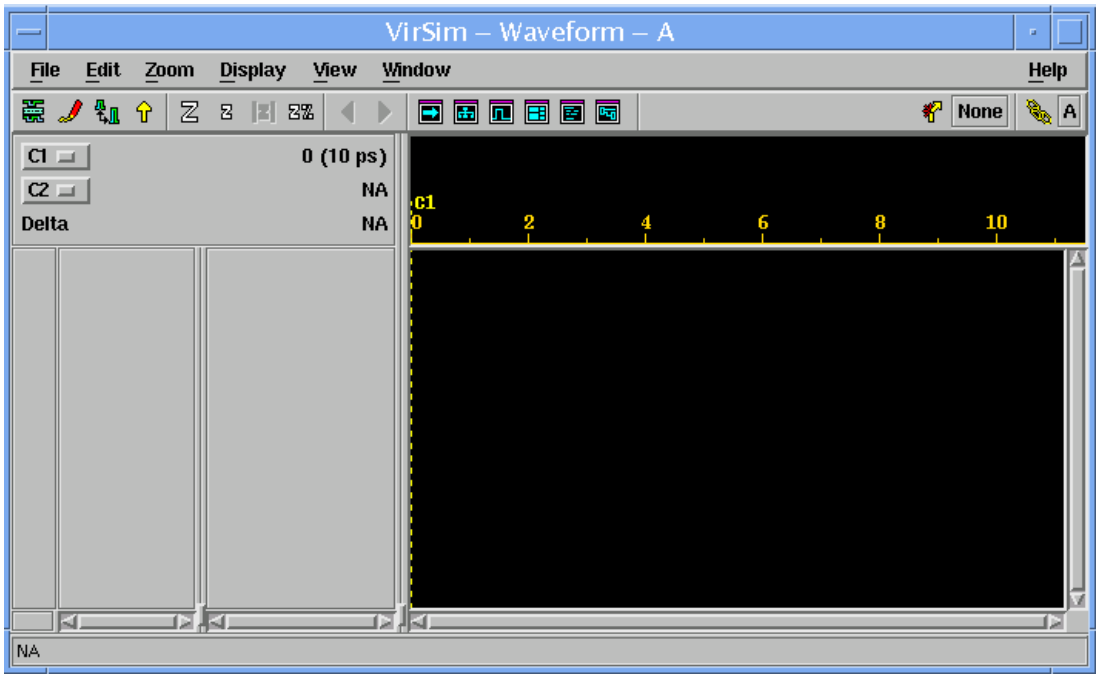


点击  查看层次结构，选择信号。如下窗口：



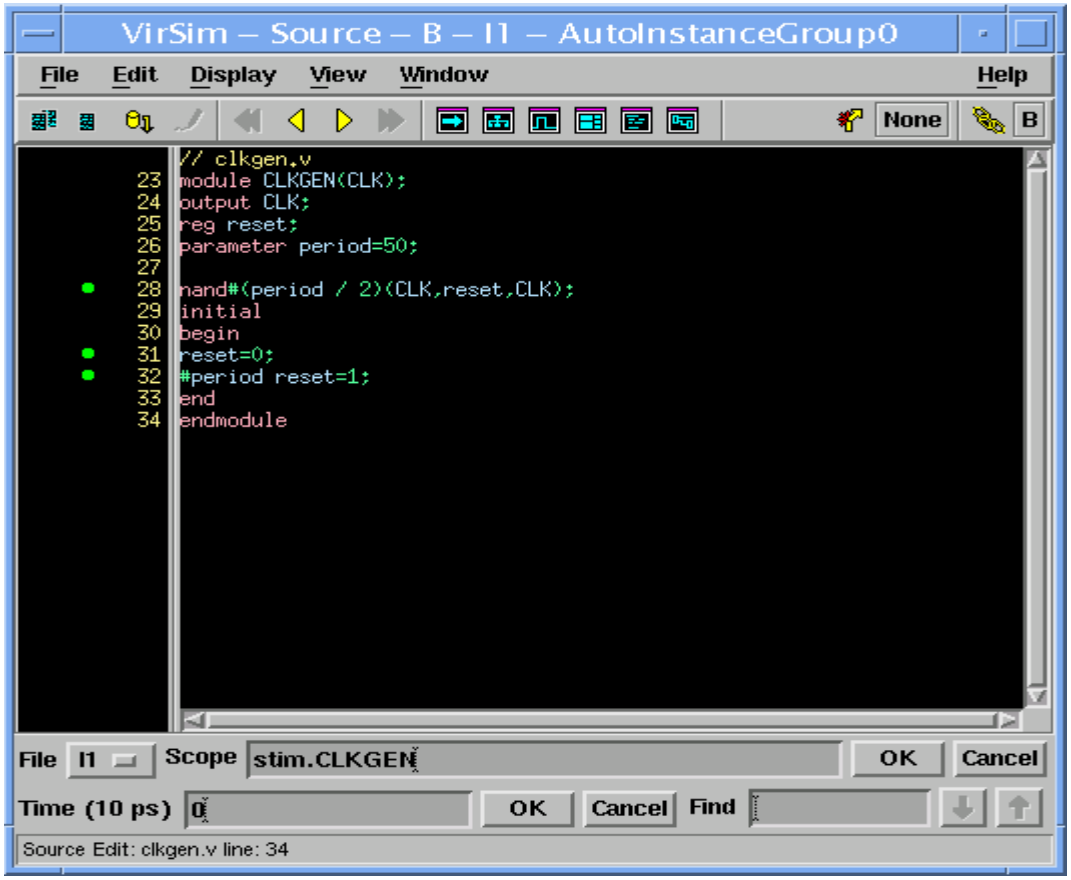
其左边为 design 的 hierarch 结构，右边为当前层次下的信号。

 打开波形窗口，其波形窗口如下：

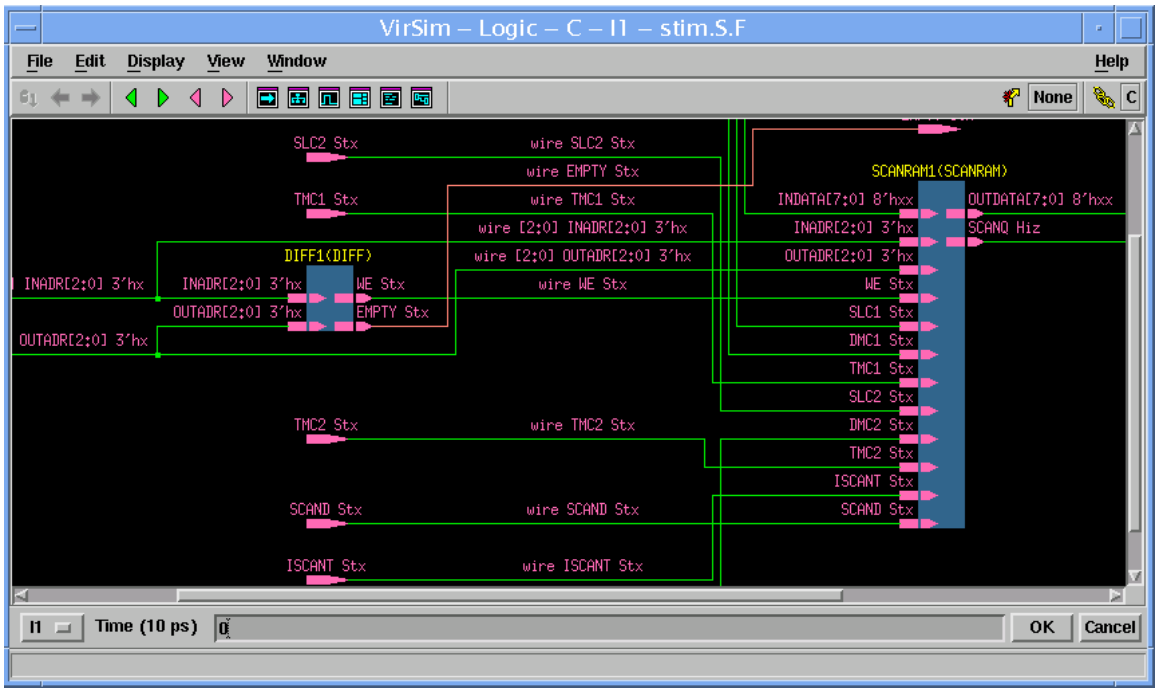


通过按住鼠标中键将在 hierarch 中选择的信号拖到 waveform 窗口中来进行信号的观测。

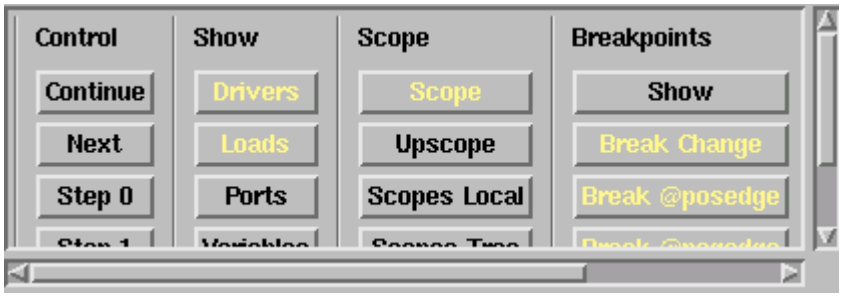
打开代码窗口，可以查看仿真的源代码。其窗口如下：



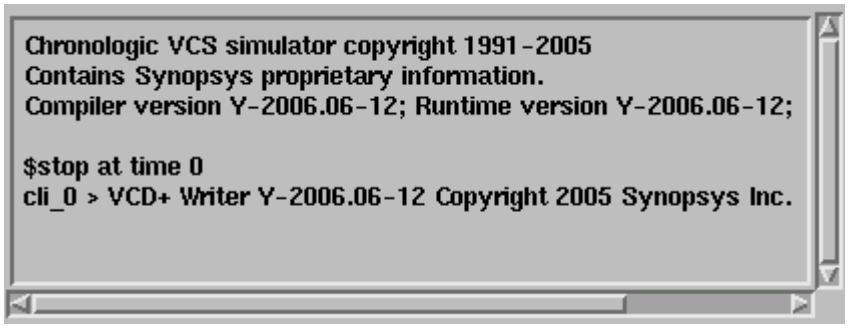
 打开逻辑图形窗口，观察电路结构图。如下：



点击  来启动仿真运行。



上面的窗口是进行调试控制的窗口。



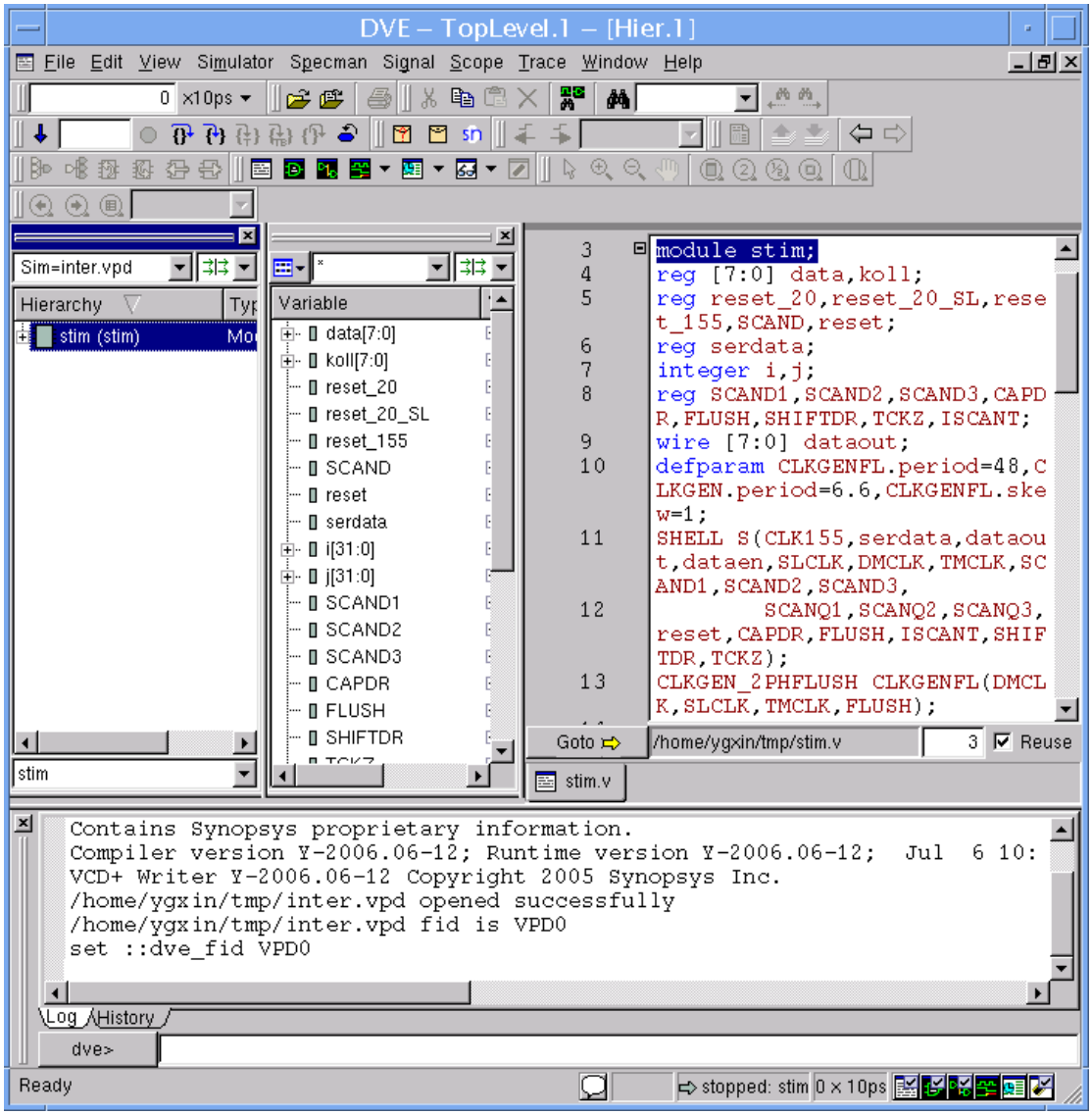
上面是显示告警和 log 信息的窗口。

3）使用 DVE 进行调试

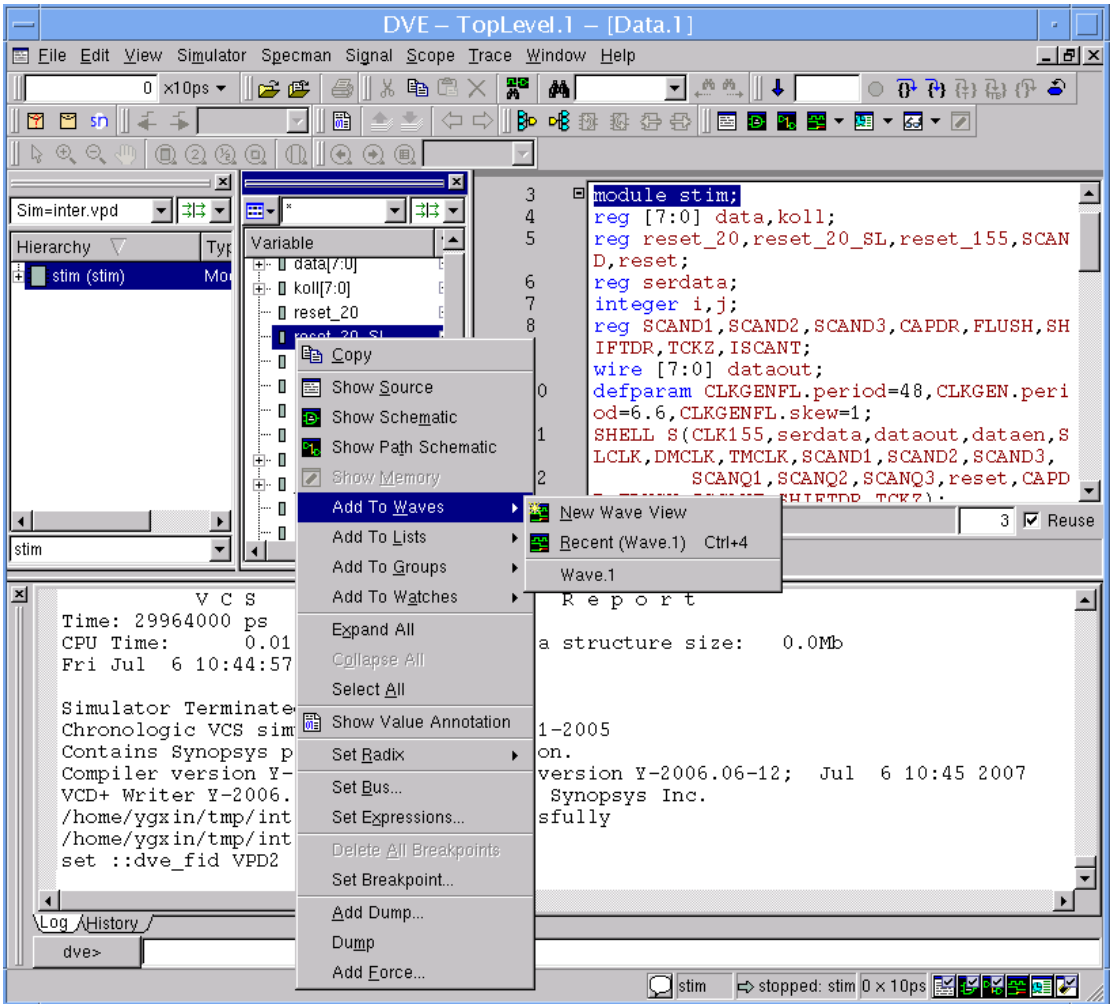
通过如下命令启动 DVE：

vcs source_file –R –gui –debug

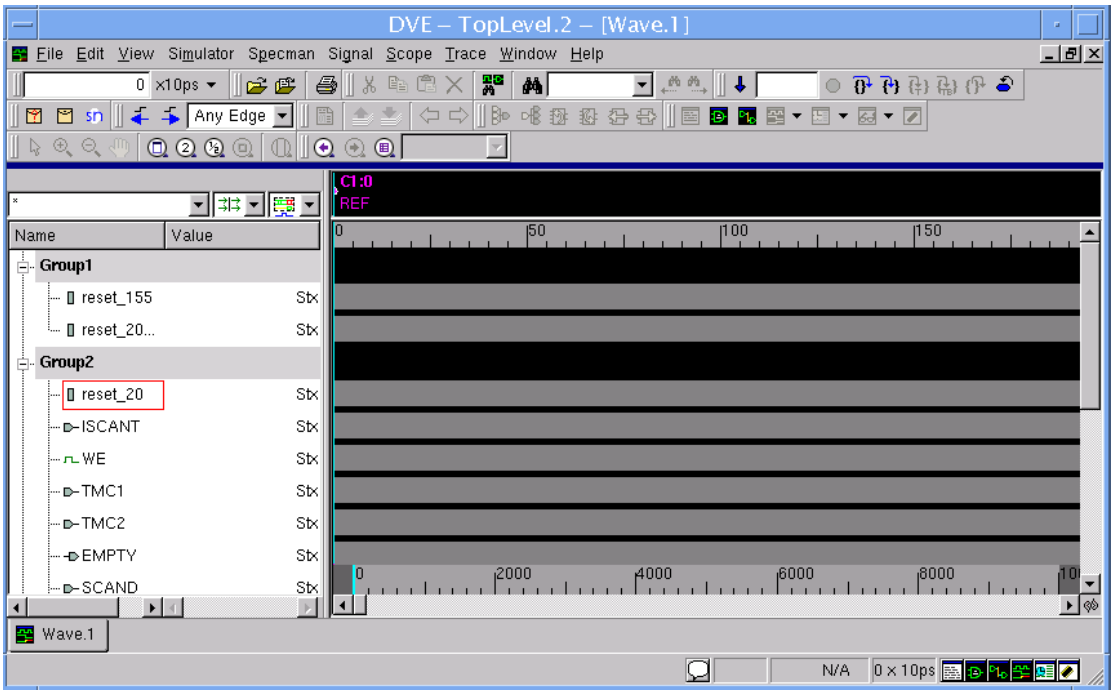
其启动主窗口如下：



在中间信号上点击鼠标右键选择添加到 waves，或者拖动信号到 wave 窗口，如下：



显示的 wave 窗口如下：



其中  为启动仿真按钮。

 为查看电路图的按钮，启动后可以观察到目前仿真的电路结构图。

为信号 path 图，通过点击其中的节点可以观察信号的流向。

具体关于 DVE 更详细的使用参见《VCS UserGuider》。

2. 用 VCS 进行后仿

(1) 反标 SDF 文件

在 testbench 中反标 SDF，通过下面的语句进行：

$sdf_annotate(“sdf_file”[,module_instance][,”sdf_configfile”][,”sdf_logfile”][,”mtm_spec”] [,”scale_factors”];

mem_spec 用来设置 delay 类型，如 MINIMUM, TYPICAL, MAXIMUM 等

scale_factors 用来设置最大，典型，最小延时的系数，缺省情况下为 1.0:1.0:1.0

反标负数的 delay 时需要加选项+old_iopath

(2) 后仿注意事项

如果不进行时序检查，使用+notimingcheck 在编译选项或仿真选项中。

如果 SDF 文件中包括负数的 delay，则需要在 VCS 编译选项中添加+neg_tchk 选项。

如果在后仿中不需要产生 timeing 告警，可以在编译选项中或在运行选项中添加+notimingcheck，将不会进行时序检查，从而能提高仿真速度。

3. 使用 PLI 接口

首先编写好的应用程序。

编写 tab 文件，其格式如下：

$xx call=xx

如果有多个函数，可以分多行进行连接，每行格式一样。

编译运行时命令如下：

vcs -P pli.tab pli.c my_design.v

在应用程序中就可以直接通过$来调用编写的应用程序。

3.4.2 cadence 公司的 NC 系列仿真工具

1. NCSIM 仿真器

(1) 工具介绍

NC_verilog 是 Cadence 公司的编译型的 verilog 仿真器，所以，NC_verilog 无论是 RTL 级还是门级仿真，其速度都比 xl 快很多。

(2) 常用命令介绍

- ncvlog：分析和编译 verilog 原代码
- ncelab：连接编译好的设计，为仿真做准备
- ncsim：仿真器
- ncprep：引导用户从 XL 转向 NC_verilog。可将 XL 的 script 文件转为 NC 的仿真环境
- ncsdfc：编译 SDF 文件，通过对 SDF 进行编译而加快了门级仿真的速度。

编译一个 verilog 设计（ncvlog）

命令格式：ncvlog [options] filname | hdl_design_unit_name

ncelab 工具从顶层设计出发，构造一个设计树。例化所有的模块并产生码流，计算信号的链接关系并写入一个文件（Simulation Snopshot File .sss ）。

命令格式：ncelab -SNAPSHOT worklib.TB_NODE_IPS：snap

ncsim 仿真器输入文件：

A、由 ncelab 产生的 snapshot file (.sss)。同时调入由 snapshot 指定的文件如.vst（Verilog Syntax tree ）和.cod 文件(The machine code file)。

B、在交互式调试过程中输入文件还包括批处理文件，任务调用文件和 textio 文件。

ncsim 仿真器输出文件：

A、SHM 格式的波形文件（Simulation History Manager）

B、仿真报告 log 文件，默认为 ncsim.log，输出仿真过程中相关的 Warning、Error 信息等等。

(3) ncsim 仿真器参考资料

/software/cadence/ldv/ius583s03/doc/ncvlog/ncvlog.pdfncvlog 指导书

/software/cadence/ldv/ius583s03/doc/simvision/simvision.pdf 看波形工具参考资料

2. 代码覆盖率统计工具 ICC

ICC（Incisive Comprehensive Coverage）是集代码覆盖、状态机覆盖、翻转覆盖和功能覆盖为一体的综合覆盖率工具。目前，我们主要使用 ICC 的代码覆盖和状态机覆盖功能。

本文通过解答以下三个问题，来帮助读者熟悉 ICC 工具，以便快速上手。

(1) 如何生成覆盖率数据

```
specman -c "load $TC_PATH/mac_svm_config; save ./mac_svm.esv; write stub -ver"

ncprep +overwrite specman.v -f $HDL_PATH/mac.f

ncvlog -f ncvlog.args -messages -update \
       -work worklib -cdslib cds.lib -hdlvar hdl.var

ncelab TB_MAC specman specman_wave \
       -access +rwc -messages -update -cdslib cds.lib -hdlvar hdl.var \
       -covfile cov.cfg

specview -p "restore ./mac_svm.esv; @pre_specman.ecom" \
ncsim -i pre_nc.cmd -tcl -NBASYNC worklib.TB_MAC:v
```

A．ncprep：无变化

B．ncvlog：无变化

C．ncelab：加入-covfile 选项，其后的覆盖配置文件（名字自便）cov.cfg，用于定义覆盖的范围、覆盖的类型以及覆盖的选项等。

```
select_coverage -be -module *
deselect_coverage -be -module TPSRAM*
deselect_coverage -be -module TPARAM*
deselect_coverage -be -module ftpsram*
deselect_coverage -be -module ftparam*
deselect_coverage -be -module ALTERA*
deselect_coverage -be -module alt*
deselect_coverage -be -module smiiclksel*
deselect_coverage -be -module stratixii*
deselect_coverage -be -module byte_reg
deselect_coverage -be -module edge_det
deselect_coverage -be -module register
deselect_coverage -be -module s2f_sync
select_fsm -module G_MD_INTF MDIO_SEL G_RX_FSM G_TX_FSM
```

select_coverage：定义代码覆盖的范围和类型。

-be：表示代码覆盖的类型为 block（行覆盖率）和 expression（表达式覆盖率）

-module *：表示覆盖的范围为所有模块（*是通配符，代表任意字符串）

deselect_coverage：定义代码覆盖不涉及的范围和类型。

-be：表示代码覆盖不涉及的类型为 block 和 expression

-module alt*：表示代码覆盖不涉及所有以 alt 为前缀的模块。

select_fsm：定义状态机覆盖的范围。

-module MDIO_SEL：表示对 MDIO_SEL 模块做状态机覆盖。

D．ncsim：加入-i(-input)选项，其后的 TCL 命令文件（名字自便）pre_nc.cmd，加入覆盖相关命令，以便建立环境，收集覆盖率。

```
coverage -setup -design     mac
coverage -setup -dut        TB_MAC.U_MAC
coverage -setup -workdir    ../cov
coverage -setup -testname   $env(TC)
coverage -code
coverage -fsm

run

exit
```

coverage -setup：建立收集覆盖率的环境

-design mac：指定生成出来的 design 文件名（本例即为 mac.dgn），不指定该选项，缺省为 top_level_instance.dgn（本例即为 TB_MAC.dgn）

-dut TB_MAC.U_MAC：指定 DUT Hierarchy 参考 instance 点，即只收集该 instance 之下的覆盖率（本例即为 TB_MAC.U_MAC，只收集 U_MAC 下的覆盖率），不指定该选项，缺省为 top_level_instance（本例即为 TB_MAC）

-workdir ../cov：指定存放覆盖数据文件的目录名（本例即为../cov），不指定该选项，缺省为./cov_work

-testname $env(TC)：指定生成出来的覆盖数据文件的文件名，一般是仿真的 TC 名

（本例设定了一个全局环境变量，由上层脚本指定 TC 名，例如仿真 tc_mac_basic，就生成出 tc_mac_basic.cov 和 tc_mac_basic.fsm）

不指定该选项，缺省为 top_level_instance.cov 和 top_level_instance.fsm（本例即为 TB_MAC.cov 和 TB_MAC.fsm）

coverage –code：收集代码覆盖率命令

coverage –fsm：收集状态机覆盖率命令

(2) 如何处理覆盖率数据

```
iccr order.cfg -work ./
```

iccr：ICC Report 工具，用来分析处理、显示报告代码覆盖率和状态机覆盖率。

-work <dirname>：指定存放处理过的覆盖数据文件的目录名（本例即为./）

<command_filename>：ICCR 配置文件（名字自便，本例即为 order.cfg），用于加入分析处理覆盖率数据的命令

```
load_design ./mac.dgn

load_coverage ./*.cov
load_coverage -fsm ./*.fsm

union result.cov
union -fsm result.fsm

quit
```

load_design <design_filename>：载入 design 文件（本例即为./mac.dgn）

load_coverage <cov_filename>：载入代码覆盖率文件

（本例即为./*.cov，表示 cov 目录下所有代码覆盖率文件）

load_coverage -fsm <fsm_filename>：载入状态机覆盖率文件

（本例即为./*.fsm，表示 cov 目录下所有状态机覆盖率文件）

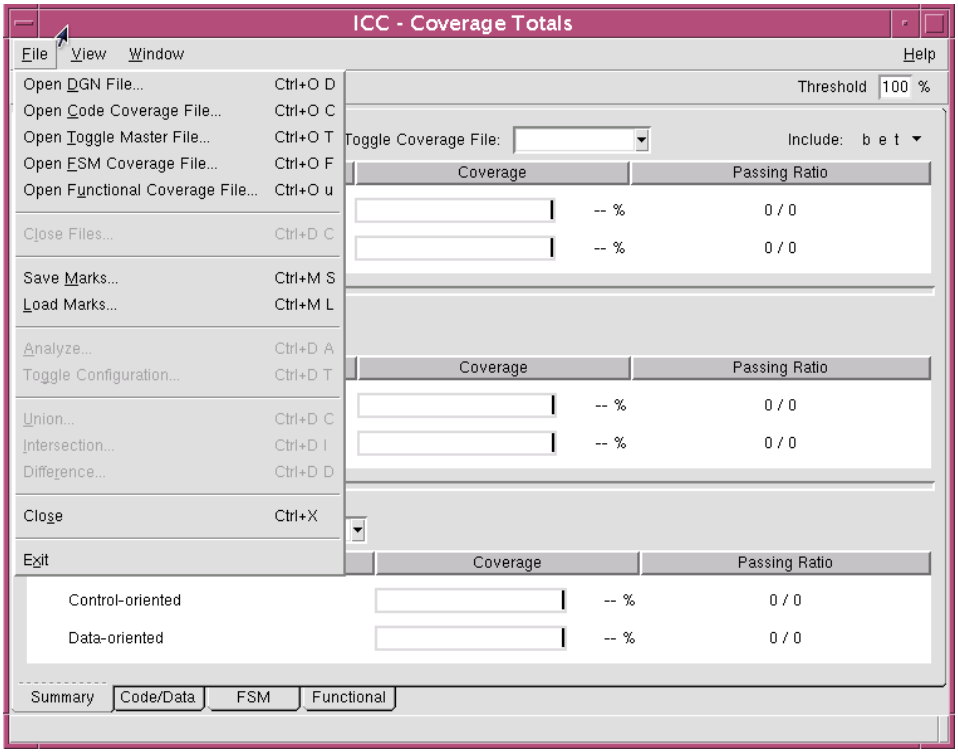
union result.cov：合并所有已载入的代码覆盖率文件为一个文件（本例即为 result.cov）

union -fsm result.fsm：合并所有已载入的状态机覆盖率文件为一个文件

（本例即为 result.fsm）

(3) 如何阅读覆盖率报告

（A）打开 ICC 图形化界面：使用 iccr –g(-gui) 命令

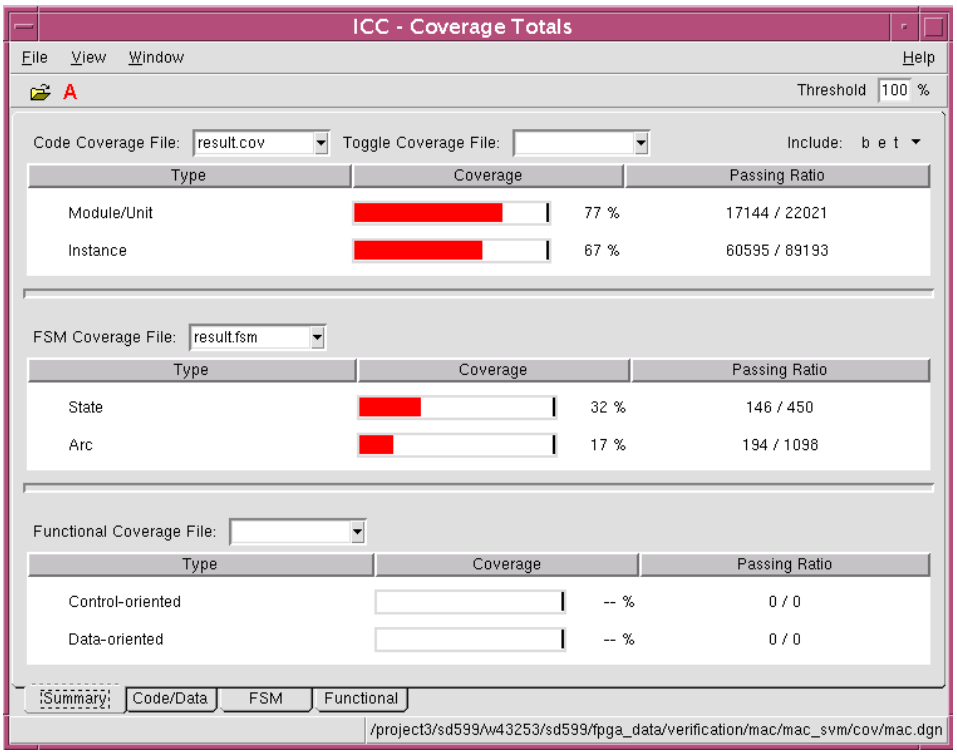


File->Open DGN File，打开后缀为.dgn 的 design 文件（本例即为 mac.dgn）

File->Open Code Coverage File，打开后缀为.cov 的代码覆盖率文件（本例即为 result.cov）

File->Open fsm Coverage File，打开后缀为.fsm 的状态机覆盖率文件（本例即为 result.fsm）

（B）Summary 界面：可以看到代码覆盖和状态机覆盖总的情况。



由上图可以看出，

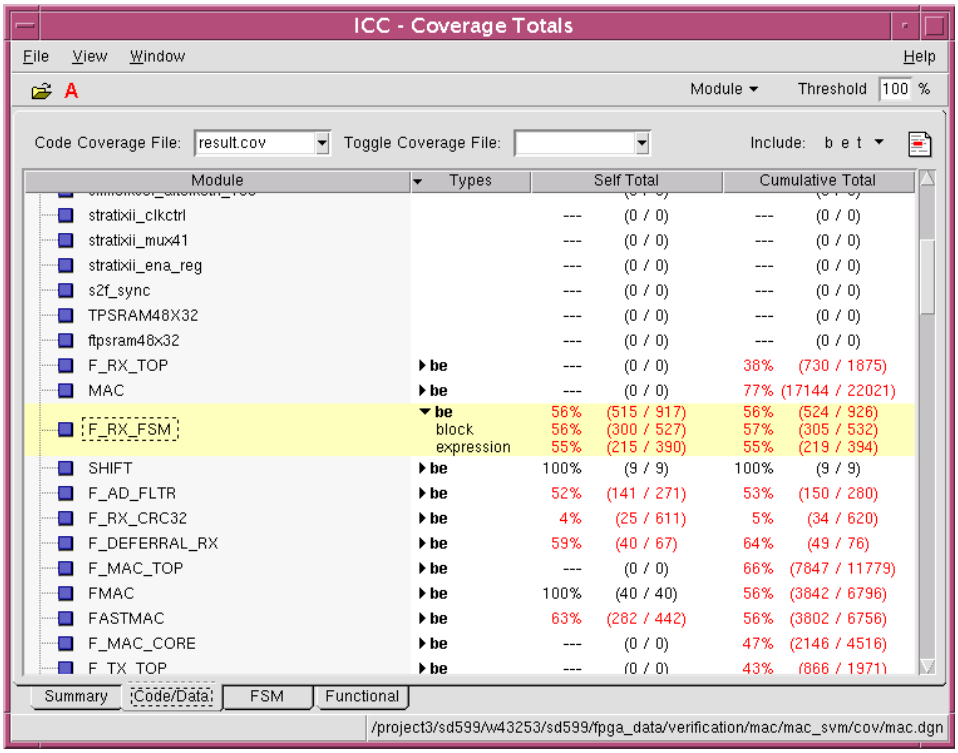
- 代码覆盖率
  对于 Module 而言，总的代码覆盖率为 77%
  对于 Instance 而言，总的代码覆盖率为 67%

⚠ 注意：

由于一个 Module 可能例化成多个 Instance，所以 Instance 的代码覆盖率比 Module 的代码覆盖率更为全面，应该以 Instance 的代码覆盖率达到 100%为目标。

- 状态机覆盖率
  对于 State 而言，其覆盖率为 32%
  对于 Arc（状态间的跳转）而言，其覆盖率为 17%

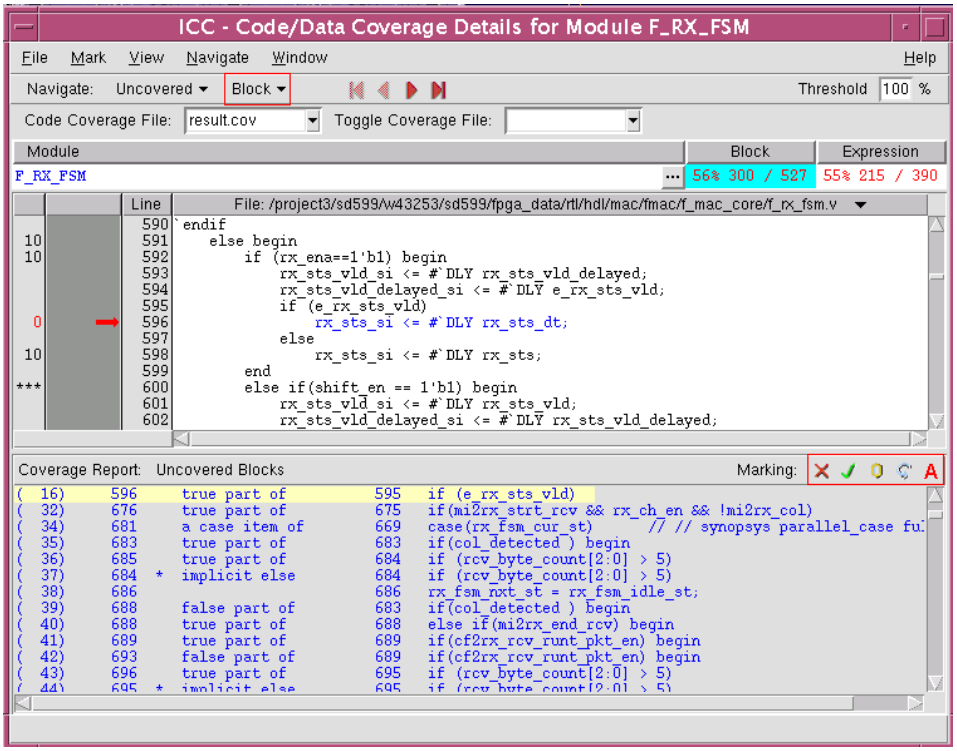
（C）Code/Data 界面：可以看到各 module 或各 Instance 的代码覆盖情况。



由上图可以看出以下三种情况，

- 不涉及的 Module：例如，TPSRAM48x32 和 ftpsram48x32 这些 RAM Module 没有覆盖率显示，这正是由于定义覆盖范围时去除了这些 Module。
- 本身没有覆盖率的 Module：例如，MAC Module 本身的覆盖率为 0，因为该 Module 只是包含了多个 Instance，实现了组装连线功能。
- 覆盖率未达到 100%的 Module：例如，F_RX_FSM Module 的本身的 Block Coverage 为 56%，Expression Module 为 55%，在其上双击，便可进入 Code/Data Coverage Details 界面。

（D）Code/Data Coverage Details 界面：可以看到模块代码覆盖的具体情况。

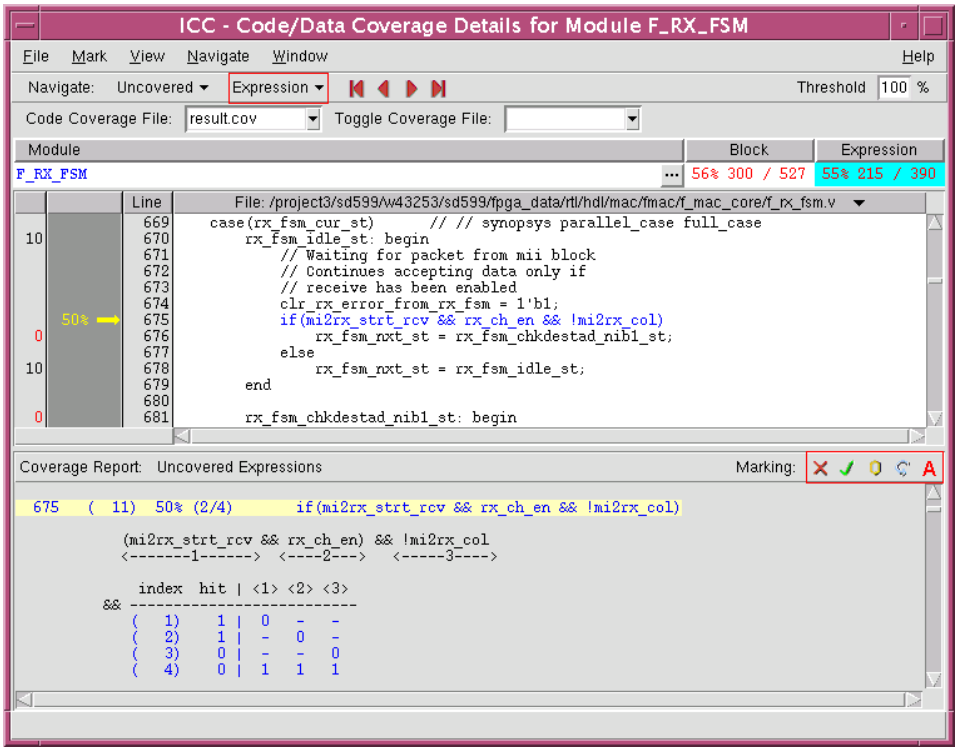


由上图可以看出，自上而下分别为以下三个区域

- Navigate 区域：目前显示的是 Uncovered Block，即该界面下显示的是该模块未覆盖到的代码行情况。点击 Block 这个下拉菜单，可以切换到 Expression，即将显示出该模块未覆盖到的表达式情况。
- Code 区域：显示该模块的具体代码。
- Report 区域：显示代码覆盖报告，点击其中的条目，就能在 Code 区域定位到相应的代码行，结合报告中的信息，我们就可以知道未覆盖的原因。

例如，第一个未覆盖到的条目对应的是第 596 行，信息显示 596 True part of 595，即第 596 行是第 595 行逻辑为真的分支。第 596 行没有覆盖到=>第 595 行逻辑一直为 FALSE=>信号 e_rx_sts_vld 一直为 FALSE，我们便找到了未覆盖的原因。

该条目实际上体现了 Branch Coverage，Branch Coverage 已经包含在 ICC 的 Block Coverage 中了。



I

ICC 会将逻辑表达式转换为逻辑真值表，由逻辑真值表我们可以直观的看出哪些组合情况没有出现过。

以本图为例：

- 表达式行号：675
- 表达式覆盖率：50%，四种组合情况中击中两种情况。
- 逻辑表达式：(mi2rx_strt_rcv && rx_ch_en && !mi2rx_col)
- 逻辑变量：一共三个逻辑变量，mi2rx_strt_rcv，rx_ch_en 和!mi2rx_col
- 逻辑真值表：index 表明一共有四种组合情况，hit 表明前两种组合情况击中过，而后两种组合情况没出现过，即!mi2rx_col==0 的情况和 mi2rx_strt_rcv， rx_ch_en 和!mi2rx_col 都为 1 的情况没有出现过。

（E）Marking：标记的作用

由于代码写法的原因或者设计本身的原因，覆盖率常常达不到 100%。

例如 Case 的所有分支都已列出，但最后还是加上“default;”，那么这一行就将会成为未覆盖到的 BLOCK。

再如经设计人员确认，有些信号受到条件限制而不可能为某些值时，那么包含该信号的表达式也将成为未覆盖到的 Expression。

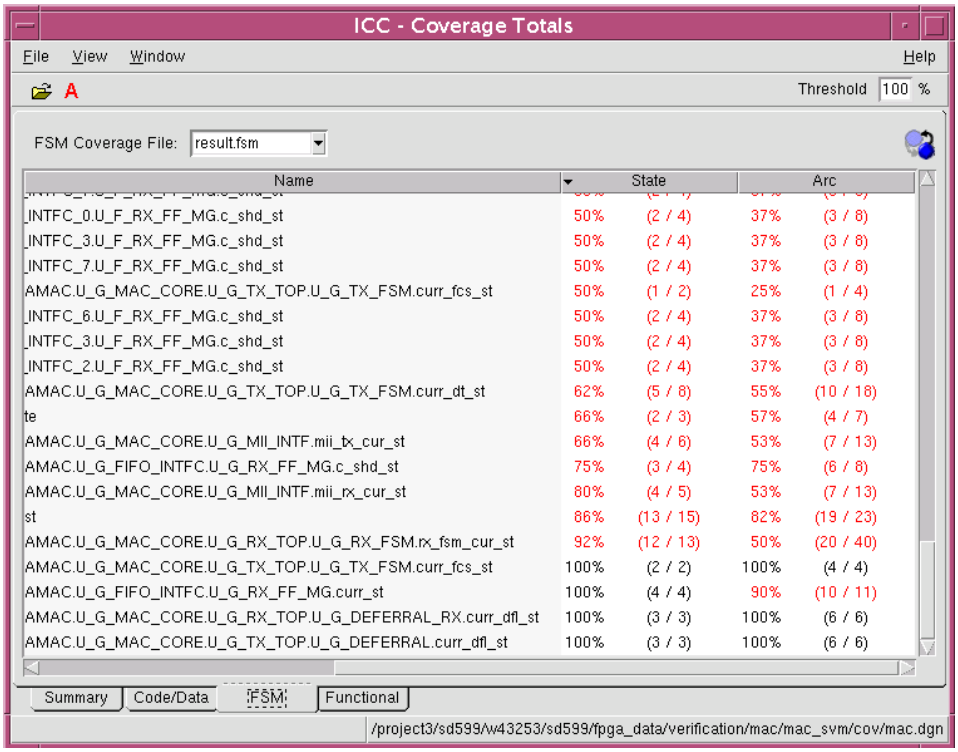
为了消除已确认覆盖不到的地方对覆盖率带来的影响，我们可以用标记来处理这些代码。

注意上两张图的右侧红框区域为 MARKING 区域，一共五个按钮，从左往右依次为

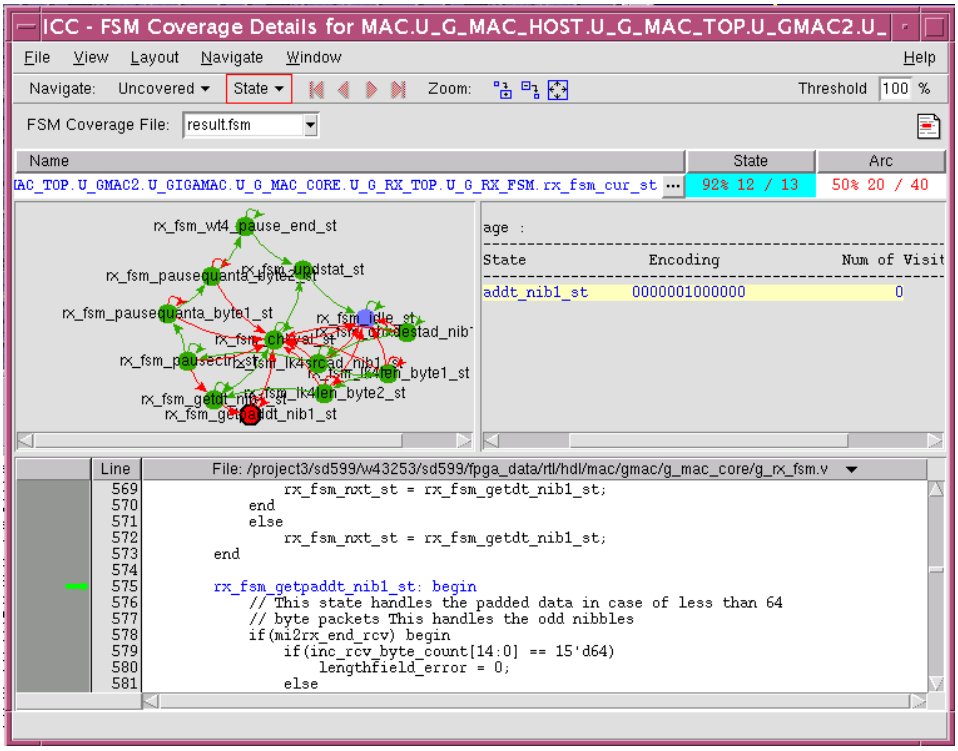
Mark as Ignore，Mark as Covered，Undo Last Mark，Undo Mark 以及 Regenerate Report。

将选中的未覆盖的 Block 或 Expression 标记为 Ignore 或 Covered，然后 Regenerate Report， ICC 就会忽略掉这些标记过的地方，重新计算覆盖率，结果是覆盖率得到了提高，实现 100%的代码覆盖率成为一种可能。

（F）FSM 界面：可以看到各状态机的覆盖情况。

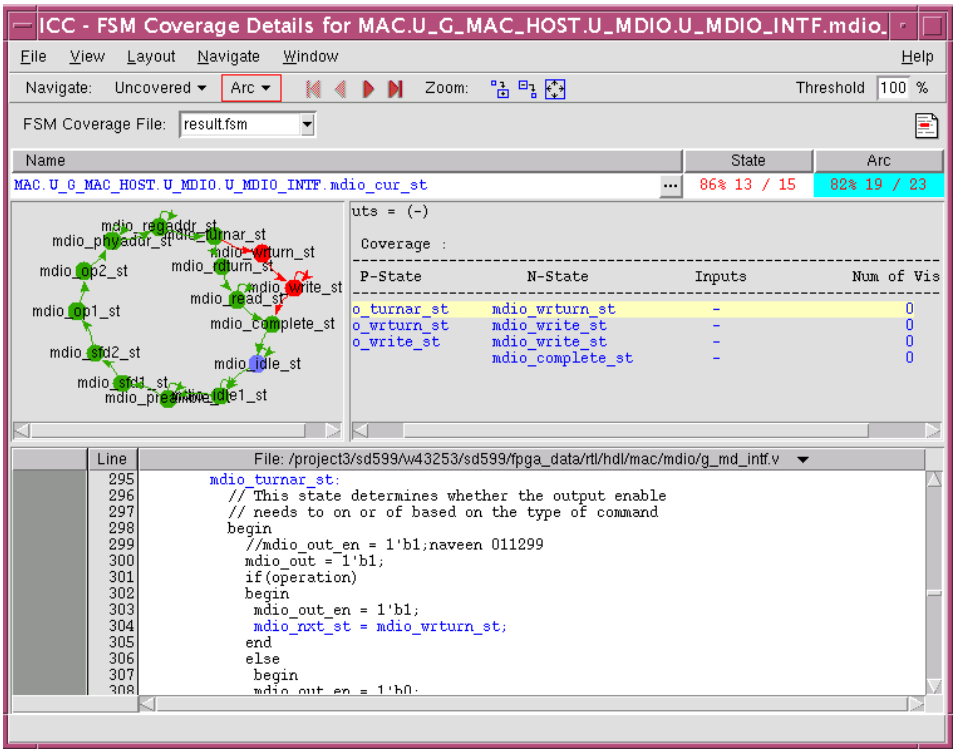


（G）FSM Coverage Details 界面：可以看到模块状态机覆盖的具体情况。



由上图可以看出，自上而下分别为以下四个区域

- Navigate 区域：目前显示的是 Uncovered State，即该界面下显示的是该模块未覆盖到的状态情况。点击 Block 这个下拉菜单，可以切换到 Arc，即将显示出该模块未覆盖到的状态跳转情况。
- State Diagram 区域：以图的形式直观的表示出状态机的结构。
- Report 区域：显示出未覆盖到的状态、编码以及达到的次数。
- Code 区域：显示该状态机的具体代码。



由上图可以看出，自上而下分别为以下四个区域

- Navigate 区域：目前显示的是 Uncovered State，即该界面下显示的是该模块未覆盖到的状态情况。点击 Block 这个下拉菜单，可以切换到 Arc，即将显示出该模块未覆盖到的状态跳转情况。
- State Diagram 区域：以图的形式直观的表示出状态机的结构。
- Report 区域：显示出未覆盖到的状态跳转（目前状态=>下一个状态），输入以及达到的次数。
- Code 区域：显示该状态机的具体代码。

(4) 如何自动生成 html 格式的报告文档

在代码覆盖率统计工具 ICCR 中，可以让其自动生成报告文档，文档中统计了各项的总覆盖率，同时也进行了分类处理，并可将未覆盖到的（或者覆盖到的、被打上 Mark 标志的）代码行数及其代码内容、所在文件（或者模块）的名字都打印到报告中，这非常方便我们的编写仿真报告，同时也有效提高了仿真报告的客观性和可靠性。

工具生成的报告有好几种格式，一般我们将其生成为 html 格式，最后再数传到我们的 PC 机上，对未覆盖到的代码加上注释说明未覆盖的原因即可。

生成该报告的脚本如下：

// 1、加载设计代码所生成的.dgn 文件

Load_design xxxx.dgn

//2、 加载覆盖率报告.cov 文件，可以使用通配符“？”

Load_coverage test_xx0_???.cov

Load_coverage test_xx1_???.cov

Load_coverage test_xx2_???.cov

.

.

.

// 3、将上面所加载的众多覆盖率报告文件合成新的覆盖率报告文件，文件名用户自定义

Union summary_xxx.cov

//4、 生成 html 格式的报告文档

Report_html -instance -uncovered -hidezero -be *

其中 Report_html 指令的使用说明如下：

Report_html [-instance | -module | -entity] [-coverage | -uncovered | -both | -marked] [-hidezero] [-besa] [-sort besa] [raw] <list>

-

3.4.3 仿真工具脚本

1. 前仿脚本介绍

(1) 单个 TC 编译，提交脚本

本脚本主要用在调试阶段，或者需要单个 TC 进行提交的时候，脚本比较简单，较容易理解，run.sh 脚本详细内容如下：

```
#!/bin/csh -f
set is_linux = `/bin/uname -s`
set env_path = "/project6/SD8901/c47206/SD8901/digital_data/verification/tupp/env"
set env_name = "tupp_top.e"
set filelist_path = "/project6/SD8901/c47206/SD8901/digital_data/verification/tupp/dut"
set filelist_name = "filelist.f"
set top_name = "TUPP_TOP"
set tc_path  = "/project6/SD8901/c47206/SD8901/digital_data/verification/tupp/tc"
set tc_name  = "tc_tu12_basic"
set log_path = "./log"
\rm -r INCA_libs
\rm cds.lib
\rm hdl.var
\rm RUN_NC
\rm *.log
\rm *.args
\rm *.elog
\rm *.key
\rm *.esv
\rm *.v
\rm core.*
specman -c "load  $env_path/$env_name; save ./tb_env.esv;write stubs -verilog ./specman.v"
ncprep +overwrite -f $filelist_path/$filelist_name
ncvlog specman.v -f ncvlog.args -messages -update -cdslib cds.lib -hdlvar hdl.var -work wor
klib
ncelab_specman $top_name specman specman_wave  -access +rwc -messages -update -cdslib cds.l
ib -hdlvar hdl.var -timescale 1ns/100ps -override_timescale -notimingchecks
specview -p "res tb_env.esv;set log $log_path/$tc_name.elog;load $tc_path/${tc_name}.e;test
 -seed=random;" \
ncvlog_specman -i -LICQUEUE -gui -NOLOG -nbasync -NOSTDOUT -tcl worklib.${top_name}
```

XIX. run.sh脚本内容

该脚本适合环境有修改，或需要重新编译环境时应用，并且，该脚本只适合在交互模式（qsh）下编译和提交 TC。如果用户在编译完成后，需要利用 qsub 逐一提交 TC，则可以利用下面 run_tc 脚本来实现，详细内容如下：

```
#!/bin/csh -f
#setenv DISPLAY hw230:0.0
set tc_path = "/home/c47206/vrf/tupp/tc"
set log_path = "./log"
set top_name = "TUPP_TOP"
specview -p "res tb_env.esv;set log $log_path/$1.elog ;load $tc_path/$1.e ; test -seed=rand
om;" ncvlog_specman -i wave.cmd -LICQUEUE -gui -NOLOG -NOSTDOUT -neverwarn -nbasync -tcl wo
rklib.$top_name
```

XX. run_tc 脚本内容

本脚本设置说明：

set env_path = “ /../” 是设置验证环境的路径

set env_name = “*.e” 是设置验证环境的顶层文件名

set filelist_path = “/../” 是设置 verilog 文件列表的路径

set filelist_name = “filelist.f” 是设置 verilog 文件列表的名称

set top_name = “TUPP_TOP” 设置 DUT 顶层 TB 的 module 名称

set tc_path = “/../” 是设置 TC 文件路径

set tc_name = “ tc_tu12_basic” 是设置 TC 的名称，注意，本脚本要求 TC 文件名是以.e 为后缀的，在配置该 tc_name 时，去掉.e 后缀。

set log_path = “/../” 是设置 TC 运行 log 文件存放的路径 ，log 文件最后以 ${tc_name}.elog 方式保存。

本脚本的使用方法：

- 用户将 /nas/data2/share/sim_script/presim_script01 目录下 run.sh run_tc wave.cmd 三个文件拷贝到本地目录。
- 按实际需要修改以上配置项（路径和名称）
- 在交互式界面下，运行 run.sh 进行环境编译、提交 TC，使用方式如下：
  run.sh
- 在交互式界面下，如果需要在已经编译的环境上，逐一提交 TC，则可以直接运新 run_tc + TC 的名称，使用方式如下：
  run_tc tc_tu12_basic
- 如果需要在提交机上，利用 qsub 逐一提交 TC，则可以使用如下方式来实现：
  qsub -l arch=lx24-amd64 -l men_free=2G run_tc tu12_basic

(2) 提交单个 TC、并保存波形的自动化脚本

本脚本一次只能提交一个 TC，但是可以根据 TC 名称保存波形，其优点是，不需要每次都要根据 TC 名称修改脚本。makefile 脚本详细内容如下：

```
#!/bin/sh
tcfile_path="/home/c47206/vrf/tupp/script/tupp2"
tcfile_name="tcfile"
exec 4<&0 0<${tcfile_path}/${tcfile_name}
COUNTER=1
waveflag=$1
start=$2
while test $COUNTER -lt $start
  do
    read line
    COUNTER=`expr $COUNTER + 1`
  done
    echo $waveflag > temp_file
    read  line
    echo  $COUNTER
    echo  $line
    echo  $line  >> temp_file
    qsub -l arch=lx24-amd64 -l mem_free=2G make $line
exec 0<&4
```

XXI. makefile脚本文件内容

该文件中，tcfile_path、tcfile_name 分别设置 TC 列表文件的路径和名称，用户在使用时需要将该路径和文件名称修改成本地的名称。本文件中，还调用了一个 make 和 tcfile 文件，其详细内容如下：

```
1 cd  /home/c47206/vrf/tupp/script/tupp2
2 run_tc  $1
```

XXII. make文件内容

```
13 tc_tu12_lop2int_maskoff_check
14 tc_tu12_uneq2int_mask_check
15 tc_tu12_uneq2int_maskoff_check
16 tc_tu12_h4lom_instais_check
17 tc_tu12_h4lomlalm_mask_check
18 tc_tu12_h4lomlalm_maskoff_check
19 tc_tu12_aisinsert_check
20 tc_tu12_lopinsert_check
21 tc_tu12_tuaisinsert_check
22 tc_tu12_tulopinsert_check
```

XXIII. tcfile文件内容

脚本第一行是进入脚本运行目录，第二行是调用 run_tc 文件，run_tc 文件内容如下：

```
#!/bin/csh -f
#setenv DISPLAY hw230:0.0
set tc_path = "/home/c47206/vrf/tupp/tc"
set log_path = "/project2/SD8901/c47206/tupp_log"
set top_name = "TUPP_TOP"
specview -p "res tb_env.esv;set log $log_path/$1.elog ;load $tc_path/$1.e ; test -seed=rand
om" ncvlog_specman -i wave.ecom -LICQUEUE -gui -NOLOG -NOSTDOUT -neverwarn -nbasync -tcl wo
rklib.$top_name
```

XXIV.run_tc文件内容

本文件在使用时，需要将 tc_path 、log_path 和 top_name 按实际需要修改成对应的路径和名称。该文件中，调用了一个 wave.ecom 的文件，该文件详细内容如下：

```
1 #!/apptools/GNU/bin/tclsh
2 set  f1  [open temp_file r+]
3 if { [gets $f1 line1] != -1 } {
4    if { [gets $f1 line2] != -1 } {
5     database -open -shm /project2/SD8901/c47206/tupp_wave/$line2 -default -incsize 2G
6     probe -create -shm TUPP_TOP -all -depth all
7     }
8 }
9 close $f1
10 run
11 quit
```

XXV. Wave.ecom文件内容

该文件主要功能是用来保存波形，用户使用的时候，需要修改波形保存的路径。同时，可以根据实际需要定义更多的 probe 语句来保存波形。

脚本使用方法：

- 将/home/c47206/share/presim_script02 目录下的 makefile 、make、run.sh、run_tc、wave.ecom 复制到本地。
- 根据实际需要修改路径和名称配置
- 建立列表文件，如：tcfile
- 在交互界面下，运行 run.sh 脚本编译环境
- 在提交机上，直接运行 makefile 脚本，提交单个 TC，使用方式如下：
- Makefile xxx 5

📖 说明：

第一个参数是任意字符，第二个参数是需要提交的 TC 在 tcfile 中的行号。

(3) 多个 TC 依次自动提交脚本

在回归测试阶段，如果机器资源和 license 资源充足的情况下，便于提高工作效率，同时减少修改脚本带来不必要的错误，可以使用本脚本依次提交多个 TC 到不同的机器上运行，而不需要修改脚本文件，makefiile 脚本文件详细内容如下：

```
#!/bin/sh
exec 4<&0 0</home/c47206/vrf/tupp/script/tupp2/tcfile
COUNTER=1
start=$1
end=$2
echo  "  " > temp_file
while test $COUNTER -lt $start
  do
    read line
    COUNTER=`expr $COUNTER + 1`
  done

while  test $COUNTER -le $end
 do
    read  line
    echo  $COUNTER
    echo  $line
    qsub -l arch=lx24-amd64 -l mem_free=1G make $line
    COUNTER=`expr $COUNTER + 1`
 done
exec 0<&4
```

XXVI.makefile文件详细内容

用户在使用时，需要将第二行中的路径和 tcfile 修改成本地对应的路径和名称，本文件中调用了一个 make 文件，其详细内容如下：

```
1 cd  /home/c47206/vrf/tupp/script/tupp2
2 run_tc  $1
```

XXVII. make文件详细内容

make 文件中调用了 run_tc 命令文件，详细内容如下：

```
#!/bin/csh -f
#setenv DISPLAY hw230:0.0
set tc_path = "/home/c47206/vrf/tupp/tc"
set log_path = "/project2/SD8901/c47206/tupp_log"
set top_name = "TUPP_TOP"
specview -p "res tb_env.esv;set log $log_path/$1.elog ;load $tc_path/$1.e ; test -seed=rand
om" ncvlog_specman -i wave.ecom -LICQUEUE -gui -NOLOG -NOSTDOUT -neverwarn -nbasync -tcl wo
rklib.$top_name
```

XXVIII. run_tc文件内容

用户在使用时，需要对路径和名称进行对应修改，该文件中，调用了一个 wave.ecom 命令文件，其内容如下：

```
run
quit
```

XXIX.wave.ecom文件内容

脚本使用方法：

- 将/nas/data2/share/sim_script /presim_script03 目录下的 makefile 、make、run.sh、run_tc、wave.ecom 复制到本地。
- 根据实际需要修改路径和名称配置
- 建立列表文件，如：tcfile
- 在交互界面下，运行 run.sh 脚本编译环境
- 在提交机上，直接运行 makefile 脚本，提交多个 TC，使用方式如下：

```
HW174:/home/c47206/vrf/tupp/script/tupp2> makefile 1 3
```

📖 说明：

参数“1”和“3”是 tcfile 中 TC 索引编号，该命令将 1~3 号 TC 依次进行提交。

(4) 在同一处理器上依次运行多个 TC 的自动化脚本

在回归测试阶段，如果机器资源和 license 资源非常紧张，不便于同时提交多个进程并行运行，在这种情况下，为了能够充分利用资源，提高回归测试的效率，又可以避免修改脚本带来的麻烦。可以使用本脚本来完成，makefile 命令脚本详细内容如下：

```
#!/bin/sh
exec  4<tcfile
COUNTER=1
fileflag=$1
start=$2
end=$3
\rm file_temp
while test $COUNTER -lt $start
  do
    read  line <&4
    COUNTER=`expr $COUNTER + 1`
  done
echo $fileflag >> file_temp
while  test $COUNTER -le $end
 do
    read  line <&4
    COUNTER=`expr $COUNTER + 1`
    echo  $line >> file_temp
 done
    exec 5<file_temp
    read line1<&5
    read line1<&5
    if [ $line1 != "" ]; then
    echo " the qsub is ok "
    qsub -l arch=lx24-amd64 -l mem_free=2G make
    fi
```

XXX. makefile脚本内容

本脚本文件中调用了两个文件，一个是 TC 列表文件，与前面的内容和作用一样；一个是 make 文件，其详细内容如下：

```
1 cd  /home/c47206/vrf/tupp/script/tupp3
2 run_tc
3
```

XXXI.make命令文件内容

该命令文件中，第一行是进入到命令文件 run_tc 的目录，第二行是调用 run_tc 命令文件，其详细内容如下：

```
specview -p "res tb_env.esv:" ncvlog_specman -i pre_nc.tcl -LICQUEUE -nbasync -gui -NOLOG -
NOSTDOUT -tcl worklib.TUPP_TOP
```

XXXII. run_tc命令文件内容

此命令文件主要是启动 specview 和仿真器，命令脚本中调用了一个 pre_nc.tcl 命令脚本。其详细内容如下：

```
#!/apptools/GNU/bin/tclsh
    set f1 [open file_temp r+]
    gets $f1 line
    set f2 [open temp_$line w]
    while {[gets $f1 line1] != -1 } {
          puts $f2  $line1
      }
    close $f1
    close $f2
    set f3 [open file_temp w]
    close $f3
    set i 0
    set f4 [open temp_$line r+]
    while { [gets $f4 line3] != -1 } {
        puts "the tc : $line3 is begining"
        catch {database -close *}
        reset
        call sn {"clear screen"}
        call sn {"br on error"}
        call \sn \"res\:\set\ log\ /project2/SD8901/c47206/tupp_log/$line3.elog\:\loa\ /hom
e/c47206/vrf/tupp/tc/$line3.e\:\ test\ -seed=1\"
        reset
        run
        incr i
        puts "No.$i $line3 test has finished\n"
    }
    close $f4
    file delete temp_$line
    unset i
```

XXXIII. pre_nc.tcl命令文件内容

在使用该命令脚本时，需要将“高亮”处的命令行中的路径改成实际需要的本地路径，测试种子参数-seed 可以根据实际需要自由修改。

脚本使用方法：

- 将/nas/data2/share/sim_script /presim_script04 目录下的 makefile 、make、run.sh、run_tc、pre_nc.tcl 复制到本地。
- 根据实际需要修改路径和名称配置
- 建立列表文件，如：tcfile
- 在交互界面下，运行 run.sh 脚本编译环境
- 在提交机上，直接运行 makefile 脚本，提交多个 TC，使用方式如下：

```
HW174:/home/c47206/vrf/tupp/script/tupp3> makefile tc2_6 2 6
```

📖 说明：

第一个参数“tc2_6”可以是任意字符，后两个参数“2”和“6”是需要提交的 TC 在 tcfile 中的行号，该命令脚本将 tcfile 中 2~6 行的 TC 提交到同一处理器上依次进行运行，当第一个 TC 运行完毕后，自动调度第二个 TC 运行，直到所有的 TC 都调度运行完毕为止。注意：在使用该脚本时，验证环境中必须要有 run_stop()语句。

2. 代码覆盖率统计脚本介绍

(1) 多个 TC 依次自动提交脚本

在统计代码覆盖率时，如果有充足的机器和 license 资源，则可以利用本脚本，自动将多个 TC 自动提交到“不同”机器上并行运行，其脚本的内容与前方对应脚本的内容相似，其中 run.sh、pre_nc.tcl 或 wave.ecom 文件内容不相同。makefile 脚本详细内容如下：

```
#!/bin/sh
exec  4<tcfile
COUNTER=1
start=$1
end=$2
while test $COUNTER -lt $start
  do
    read  line <&4
    COUNTER=`expr $COUNTER + 1`
  done
while  test $COUNTER -le $end
 do
    read  line <&4
    echo  $COUNTER
    echo  $line|tee file_temp
    qsub -l arch=lx24-amd64 -l mem_free=2G make $line
    COUNTER=`expr $COUNTER + 1`
    sleep 10
    exec  5<file_temp
    read  line1<&5
    cnt=1
    while test $line1 != "finish"
    do
    sleep 15
    exec  5<&0
    exec  5<file_temp
    read  line1<&5
    if [ $cnt -gt 20 ] ; then
    line1="finish"
```

XXXIV. makefile文件内容

脚本中 tcfile 文件内容与前面的一样，脚本中调用的 make 命令文件，其详细内容如下：

```
1 cd  /home/c47206/vrf/loxcs/cover/
2 run_tc  $1
3
```

XXXV. make文件内容

该命令脚本中，第一行是进入命令脚本 run_tc 所在的目录，第二行是调用 run_tc 命令文件，其详细内容如下：

```
#!/bin/csh -f
#setenv DISPLAY hw230:0.0
set tc_path = "/home/c47206/vrf/tupp/tc"
set log_path = "/project2/SD8901/c47206/tupp_log"
set top_name = "TUPP_TOP"
specview -p "res tb_env.esv;set log $log_path/$1.elog ;load $tc_path/$1.e ; test -seed=rand
om" ncvlog_specman -i wave.ecom -LICQUEUE -gui -NOLOG -NOSTDOUT -neverwarn -nbasync -tcl wo
rklib.$top_name
```

XXXVI. run_tc文件内容

在使用该命令文件时，需要根据实际需要，将对应路径和名称修改成本地的路径和名称，在该脚本中调用了一个 wave.ecom 命令文件，其主要功能是保存覆盖率统计数据，脚本详细内容如下：

```
coverage -setup -workdir  /project2/SD8901/c47206/tupp_cov_work
coverage -setup -vmgrdir  /project2/SD8901/c47206/tupp_vmgr_work
coverage -setup -design tupp
coverage -setup -dut TUPP_TOP.LOPP_TOP
set f1 [open file_temp r+]
gets  $f1 line1
coverage -setup -testname $line1
coverage -code
#coverage -fsm
coverage -toggle -database -local_db $line1.mst
close $f1
set f2 [open file_temp w]
puts  $f2 "finish"
close $f2
run
exit
```

XXXVII. wave.ecom 文件内容

脚本使用方法：

- 将/nas/data2/share/sim_script /cover_script01 目录下的 makefile 、make、run.sh、run_tc、wave.ecom、covfile.cf 复制到本地。
- 根据实际需要修改路径和名称配置
- 建立列表文件，如：tcfile
- 在交互界面下，运行 run.sh 脚本编译环境
- 在提交机上，直接运行 makefile 脚本，提交多个 TC，使用方式如下：

```
HW174:/home/c47206/vrf/tupp/script/tupp2> makefile 1 3
```

📖 **说明：**

参数“1”和“3”是 tcfile 中 TC 索引编号，该命令将 1~3 号 TC 依次进行提交。

(2) 在同一处理器上依次运行多个 TC 的自动化脚本

在资源紧张的情况下，可以利用脚本将多个 TC 提交到同一处理器上，按给定的次序依次运行，这样可以节省资源，又可以避免修改脚本带来的麻烦，makefile 命令文件详细内容如下：

```
#!/bin/sh
exec  4<tcfile
COUNTER=1
fileflag=$1
start=$2
end=$3
\rm file_temp
while test $COUNTER -lt $start
  do
    read  line <&4
    COUNTER=`expr $COUNTER + 1`
  done
echo $fileflag >> file_temp
while  test $COUNTER -le $end
 do
    read  line <&4
    COUNTER=`expr $COUNTER + 1`
    echo  $line >> file_temp
 done
    exec 5<file_temp
    read line1<&5
    read line1<&5
    if [ $line1 != "" ]; then
    echo " the qsub is ok "
    qsub -l arch=lx24-amd64 -l mem_free=2G make
    fi
```

XXXVIII. makefile文件内容

在脚本中，tcfile 文件内容与前面相同，make 文件内容如下：

```
1 cd  /home/c47206/vrf/loxcs/cover/
2 run_tc  $1
3
```

XXXIX. make文件内容

该命令脚本中，第一行是进入命令脚本 run_tc 所在的目录，第二行是调用 run_tc 命令文件，其详细内容如下：

```
#!/bin/csh -f
#setenv DISPLAY hw230:0.0
specview -p "res tupp.esv:" ncvlog_specman -nostdout -NBASYNC -neverwarn -LICQUEUE -i ./wav
e.ecom -gui -tcl worklib.TUPP_TOP
```

XL. run_tc文件内容

在该脚本中调用了一个 wave.ecom 命令文件，其主要功能是保存覆盖率统计数据，脚本详细内容如下：

```
set f1 [open file_temp r+]
gets  $f1 line
set f2 [open tcfile_$line w]
while {[gets $f1 line1] != -1 } {
        puts $f2  $line1
    }
close $f1
close $f2
set  f3 [open file_temp w ]
close $f3
set i 0
set f4 [open tcfile_$line r+]
while { [gets $f4 line3] != -1 } {
        puts "the No.$i $line3 is begining to run "
        catch {database -close *}
        reset
        coverage -setup -workdir  /project2/SD8901/c47206/tupp_cov_work
        coverage -setup -vmgrdir  /project2/SD8901/c47206/tupp_vmgr_work
        coverage -setup -design tupp
        coverage -setup -dut TUPP_TOP.LOPP_TOP
        coverage -setup -testname $line3
        coverage -code
        #coverage -fsm
        coverage -toggle -database -local_db $line3.mst
        call sn {"clear screen"}
        call sn {"br on error"}
        call \sn \"res\:\set\ log\ /project2/SD8901/c47206/tupp_log_cover/$line3.elog\:\loa
\ /home/c47206/vrf/tupp/tc/$line3.e\:\ test\ -seed=random\"
        run
```

XLI. wave.ecom文件内容

用户在使用该脚本时，需要对该文件中对应的路径和名称进行修改，使得其满足本地的需要。

脚本使用方法：

- 将/nas/data2/share/sim_script /cover_script02 目录下的 makefile 、make、run.sh、run_tc、wave.ecom、covfile.cf 复制到本地。
- 根据实际需要修改路径和名称配置
- 建立列表文件，如：tcfile
- 在交互界面下，运行 run.sh 脚本编译环境
- 在提交机上，直接运行 makefile 脚本，提交多个 TC，使用方式如下：

```
HW174:/home/c47206/vrf/tupp/script/tupp3> makefile tc2_6 2 6
```

📖 说明：

第一个参数“tc2_6”可以是任意字符，后两个参数“2”和“6”是需要提交的 TC 在 tcfile 中的行号，该命令脚本将 tcfile 中 2~6 行的 TC 提交到同一处理器上依次进行运行，当第一个 TC 运行完毕后，自动调度第二个 TC 运行，直到所有

的 TC 都调度运行完毕为止。注意：在使用该脚本时，验证环境中必须要有 run_stop()语句。

(3) 代码覆盖率统计分析和报告生成脚本

本脚本是用来对每条 TC 统计结果进行分析，并根据需要生成统计报告，其内容如下：

```
# the coverage report script file
define  rptdir  "/project2/SD8901/c47206/tupp_cover_report"
define  instlist "*..."
define  covdir  "/project2/SD8901/c47206/tupp_cov_work"
sh rm                   $covdir/alltc.cov
set_working_directory    $covdir
load_design              tupp.dgn
load_coverage            *.cov
union                    alltc.cov
load_coverage            alltc.cov
report_summary -instance -hidezero -be $instlist    > $rptdir/tupp_cov_summary.rpt
report_detail  -instance -both  -be   $instlist     > $rptdir/tupp_cov_both_be.dtl
report_detail  -instance -both  -b   $instlist      > $rptdir/tupp_cov_both_b.dtl
report_detail  -instance -both  -e   $instlist      > $rptdir/tupp_cov_both_e.dtl
report_detail  -instance -covered -be $instlist     > $rptdir/tupp_cov_covered_be.dtl
report_detail  -instance -uncovered -be $instlist   > $rptdir/tupp_cov_uncovered_be.dtl
report_detail  -instance -uncovered -b  $instlist   > $rptdir/tupp_cov_uncovered_b.dtl
report_detail  -instance -uncovered -e  $instlist   > $rptdir/tupp_cov_uncovered_e.dtl
report_tabular_summary -instance -be -sort be -raw  -hidezero $instlist > $rptdir/tupp_cov_
tab_summary.rpt
view_graphics_and_wait
exit
```

Coer_report.cmd 文件内容

脚本使用方法：

- 将/nas/data2/share/sim_script /cover_report/cover_report.cmd 脚本复制到本地
- 修改文件内容中的路径和名称
- 在交互式界面下，运行命令文件，使用方法如下：

```
szxcs125-lnx:/home/c47206/vrf/tupp/cover> iccr cover_report.cmd
```

- 最后在图形界面上，分析统计报告。

3. 后仿脚本介绍

后仿过程中，一般向量比较少，而且，后仿向量运行时间长，没有必要像前仿一样采用脚本一次提交多个 TC 来提高效率，但是，可以利用脚本节约编译时间，后仿中，SDF 文件反标过程一般耗时比较长，在实际反标过程中，只要没有修改，只需要反标一次就可以了。makefile 脚本内容如下：

```
1 #!/bin/sh
2 \rm file_temp
3 echo $1
4 echo  $1 >> file_temp
5 qsub -l arch=lx24-amd64 -l mem_free=5G  make  $1
```

XLII. makefile文件内容

脚本中调用了一个 make 命令文件，其详细内容如下：

```
1 cd /home/c47206/vrf/top/script/sd8901_postnetlist
2 run_tc  $1
```

XLIII. make文件内容

该命令脚本中，第一行是进入命令脚本 run_tc 所在的目录，第二行是调用 run_tc 命令文件，其详细内容如下：

```
#!/bin/csh -f
#setenv DISPLAY hw230:0.0
specview -p "res tb_env.esv;set log /project2/SD8901/c47206/top_postnetlist_log/$1.elog;loa
d /home/c47206/vrf/top/tc/$1.e ;break on error ;test -seed=1" ncvlog_specman -i wave.ecom -
LICQUEUE -gui  -LOGFILE /project2/SD8901/c47206/top_postnetlist_log/$1.simlog -SDF_NO_WARNI
NGS -NOSTDOUT -NBASYNC -neverwarn -tcl worklib.TB_SD8901_NETLIST
```

用户在使用过程中，需要将脚本中的路径和名称改成本地需要的路径和名称，在该脚本中调用了一个 wave.ecom 命令文件，其主要功能是保存波形文件和 force 网表中的信号，脚本详细内容如下：

```
set f1 [open file_temp r+]
if {[gets $f1 tc] != -1 } {
database -open -shm /project2/SD8901/c47206/top_postnetlist_log/${tc}_core -incsize 3G
database -open -shm /project2/SD8901/c47206/top_postnetlist_log/${tc}_lopp -incsize 3G
set wave_core_path "/project2/SD8901/c47206/top_postnetlist_log/${tc}_core"
set wave_lopp_path "/project2/SD8901/c47206/top_postnetlist_log/${tc}_lopp"
source wave_core.cmd
source wave_lopp.cmd
source netlist_force.cmd
file delete file_temp
run
} else {
exit
}
```

XLIV. Wave.ecom 文件内容

在 wave.ecom 文件中，主要定义波形保存的路径和 database 名称，source 波形保存命令文件（如：wave_lopp.cmd）和 force 命令文件（如：netlist_force.cmd），其详细内容如下：

```
probe -create -shm TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_CLKRST_GLB/clk_1
55m_reg  } -all -database $wave_lopp_path
probe -create -shm TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_CLKRST_GLB/cfg_r
st_n_reg } -all -database $wave_lopp_path
probe -create -shm TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_HOXC/U_HOXC_DM/U
_XC_DM_CHAN14/xc_data_out_reg[9] } -all -database $wave_lopp_path
probe -create -shm TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_HOXC/U_HOXC_DM/U
_XC_DM_CHAN14/xc_data_out_reg[8] } -all -database $wave_lopp_path
probe -create -shm TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_HOXC/U_HOXC_DM/U
_XC_DM_CHAN14/xc_data_out_reg[7] } -all -database $wave_lopp_path
probe -create -shm TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_HOXC/U_HOXC_DM/U
_XC_DM_CHAN14/xc_data_out_reg[6] } -all -database $wave_lopp_path
probe -create -shm TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_HOXC/U_HOXC_DM/U
_XC_DM_CHAN14/xc_data_out_reg[5] } -all -database $wave_lopp_path
```

XLV. wave_lopp.cmd文件内容

```
#force the core fast mode
force TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_ULR4B8ADEWV2/H2O_ULR4B8ADEWV2_001_00
}.PLLREGOVR = 1'b1
force TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_ULT4B8ADEWV2/H2O_ULT4B8ADEWV2_001_00
}.PLLREGOVR = 1'b1
force TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_ULF8S8ADEWV1/H2O_ULF8S8ADEWV1_001_03
}.PLLREGOVR = 1'b1
force TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_CLKRST_GLB/hss_core_rst } = 1
'b0 -after 3 us
force TB_SD8901_NETLIST.reset_n = 1'b1 -after 100 us
force TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_LOPP/U_US_CHPROC16_0/U_CHPROC
16/U_CHPROC8_0/U_RETIMING_TOP/U_LP_TIMING/ts_id[9] } = 1'b0 -release -keepvalue 100 us
force TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_LOPP/U_US_CHPROC16_0/U_CHPROC
16/U_CHPROC8_1/U_RETIMING_TOP/U_LP_TIMING/ts_id[9] } = 1'b0 -release -keepvalue 100 us
force TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_LOPP/U_US_CHPROC16_1/U_CHPROC
16/U_CHPROC8_0/U_RETIMING_TOP/U_LP_TIMING/ts_id[9] } = 1'b0 -release -keepvalue 100 us
force TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_LOPP/U_US_CHPROC16_1/U_CHPROC
16/U_CHPROC8_1/U_RETIMING_TOP/U_LP_TIMING/ts_id[9] } = 1'b0 -release -keepvalue 100 us
```

XLVI. netlist_force.cmd文件内容

在运行以上脚本提交 TC，以前还必须将环境和代码进行编译，编译脚本 run.sh 详细内容如下：

```
#!/bin/csh -f
set is_linux = `/bin/uname -s`
setenv SPECMAN_PRE_COMMANDS "set_config(memory,max_size,1600M);set_config(memory,absolute_m
ax_size,2800M);set_config(memory, gc_threshold, 600m);set_config(memory, gc_increment, 250m
)"
\rm -r INCA_libs
\rm cds.lib
\rm hdl.var
\rm RUN_NC
\rm *.log
\rm *.args
\rm *.esv
\rm *.v
\rm core.*
\rm -r *.shm
\rm *.sdf.x
specman -c "load  /project6/SD8901/c47206/SD8901/digital_data/verification/top/env_postsim/
sd8901_top.e; save tb_env.esv;write stubs -verilog specman.v"
ncprep +overwrite -f /project6/SD8901/c47206/SD8901/digital_data/verification/top/dut/filel
ist_postnetlist1.f
ncvlog specman.v -f ncvlog.args -messages -update -cdslib cds.lib -hdlvar hdl.var -work wor
klib
ncelab_specman TB_SD8901_NETLIST specman specman_wave -NONOTIFIER -messages -update -cdslib
 cds.lib -hdlvar hdl.var -timescale 1ns/100ps -access +rwc -MAXDELAYS -SDF_NO_WARNINGS -SDF
_CMD_FILE sdf_annotate.cmd
specview -p "res tb_env.esv;" \
ncvlog_specman -i -LICQUEUE -NBASYNC -gui -NOLOG -NOSTDOUT -tcl worklib.TB_SD8901_NETLIST
```

XLVII. run.sh文件内容

此 run.sh 脚本文件与前面的文件作用一样，对环境和代码进行编译，不同点是，在本脚本中，增加了 SDF 反标功能，调用了 sdf_annotate.cmd 命令文件，其内容如下：

```
compiled_sdf_file = "/project2/SD8901/c47206/sdf/netlist_sdf_compiled" ,
scope = TB_SD8901_NETLIST.U_SD8901_CHIP ,
log_file = "/project2/SD8901/c47206/netlist_sdf_log/sdf.log" ,
mtm_control = "TOOL_CONTROL" ,
scale_factors = "1.0:1.0:1.0" ,
scale_type = "FROM_MTM" ;
```

XLVIII. sdf_annotate.cmd 文件内容

在 sdf_annotate.cmd 文件中，第一行参数指定经过 sdfc 命令编译过的 SDF 文件，第二行参数指定需要反标 SDF 的 INSTANCE，第三个参数指定反标过程中 log 文件存放的位置。用户在使用过程需要对这些路径和名称进行修改。

编译 SDF 文件的方法：

```
szxcs128-lnx:/home/c47206> sdfc -messages /project6/SD8901/vd2hw/SD8901V100_100netlist_baseline/netlist/20070202/sdf/SD8901_WC_update.s
output netlist_sdf_compiled
```

本脚本的使用方法：

- 用户将/nas/data2/share/sim_script /postsim_script 目录下 run.sh 、run_tc 、wave.ecom、makefile、make 等文件拷贝到本地目录。
- 按实际需要修改以上配置项（路径和名称）
- 在交互式界面下，运行 run.sh 进行环境编译、提交 TC，使用方式如下：

```
szxcs127-lnx:/home/c47206/vrf/tupp/script/tupp2> run.sh
```

- 在提交机上，则可以使用如下方式来实现：

```
HW174:/home/c47206/vrf/top/script/sd8901_postsim> makefile tc_name
```

📖 说明：

参数 tc_name 表示需要提交的 TC 的名称，tc_name 是 TC 名除去后缀.e 的部分。

---

4. Synopsys VCS 仿真环境下的 Makefile 脚本

(1) 常用指令说明

使用 Makefile 调试时，比较常用的语法是：make svtb TC＝<TC 名称> <SEED=XXXX>

本文中对应的 Makefile 主要包含了以下命令：

- svtb： 用 VCS 编译验证环境，对某条 TC 进行仿真，生成 Log，并 dump VPD 格式的波形；
- svtb_log：用 VCS 编译验证环境，对某条 TC 进行仿真，只生成 Log，不 dump 波形；
- compile_*：用 VCS 编译验证环境和代码；
- run_*：对某条 TC 进行仿真，生成 Log，并 dump VPD 格式的波形；
- run_log_*：对某条 TC 进行仿真，只生成 Log；
- svtb_sdf： 用 VCS 编译验证环境，对某条 TC 进行门级仿真，生成 Log，并 dump VPD 格式的波形；
- svtb_log_sdf：用 VCS 编译验证环境，对某条 TC 进行门级仿真，只生成 Log，不 dump 波形；
- cmp_sdf_*：用 VCS 编译验证环境和网表文件；

- run_sdf_*：对某条 TC 进行门级仿真，生成 Log，并 dump VPD 格式的波形；
- run_log_sdf_*：对某条 TC 进行门级仿真，只生成 Log；
- help：打印指令帮助信息；
- init：清除所有覆盖率文件；
- clean：清除某条 TC 的所有仿真文件；
- clean_TC：清除某条 TC 的所有仿真临时文件，保留 log 和波形文件；
- cleanall：清除所有仿真文件；

还有一些与覆盖率相关的指令：regress_test_*、regress_clean、regress_build_*、regress_run_*、regress_urg、regress_dve_cov 等，由于暂时没有使用 SV 做覆盖率的经验，这部分指令暂时以 synopsys 提供的模板为主，后面根据验证组成员使用的意见反馈再做相应修改。

其余几个指令是使用 dve 时用到的，暂时没有使用经验，后面再做修改。

(2) 常用修改说明

在 Makefile 中，将常用的路径都用了相应的宏定义来标识：

- LIB_HOME：对应项目的 SV 库文件的位置；
- DESIGNWARE_HOME：对应于项目的 VIP 库的位置；
- PRJ_NAME：对应项目的路径；
- DUT_SRC_DIR：对应验证代码的位置；
- NETLIST_PATH：项目网表存放位置;
- NETLIST：网表文件；
- LIB_PATH：项目所用库文件位置；
- LIB：库文件列表；
- OUTPUT_DIR：仿真临时文件存放目录；
- COV_DIR：覆盖率文件存放目录；
- LOG_DIR：log 信息存放目录；
- WAVE_DIR：波形文件存放目录；
- TB_SRC_DIR：仿真对应目录；
- SDF_TYPE：门级仿真时对应的延时类型；

Makefile 中用到的宏定义主要有以上这些，针对不同的项目组，甚至是同一项目组的不同模块做单元验证时，上述的宏定义都要做相应的修改。

在使用 VIP 做仿真验证时，对于 Makefile 文件中第 80～126 行，TB_INC 中所调用的 VIP，只需将要用到的打开即可，将不用的都注释掉。下图对应的 VIP module 列表也要做相应的修改

```
141 # ================ vip filelist ==================
142 TB_SRC         += -f $(PRJ_NAME)/digital_data/verification/ut/wdt/vip/model_packages
143 TB_SRC         += $(TB_SRC_DIR)/tc/$(TC).sv
144
```

下图中对应的是后仿真时所提取出来的异步路径（仿真时无法关注到的路径），将项目代码中所有异步路径提出，组成一个文件——postsimlist（可自己命名，但要注意及时同步这里）。

```
332 #********************* declare asynchronism register ******************
333 POS_CMP_OPTIONS  += +optconfigfile+postsimlist
334 #--------------------------------------------------------------------------
```

下图为不使用 VIP 时，验证环境调入的目录结构，根据实际需要，将所有验证环境中使用到的文件目录都包含进去，对于自己项目中没有的目录结构，及时注释掉。

```
78
79 # ***************************** env file used to save env.sv ***********
   **********
80 TB_INC          += +incdir+$(TB_SRC_DIR)/tt
81 TB_INC          += +incdir+$(TB_SRC_DIR)/tc
82 TB_INC          += +incdir+$(TB_SRC_DIR)/env
83 TB_INC          += +incdir+$(TB_SRC_DIR)/th
84 TB_INC          += +incdir+$(TB_SRC_DIR)/bfm
85 TB_INC          += +incdir+$(TB_SRC_DIR)/rm
86 TB_INC          += +incdir+$(TB_SRC_DIR)/cmd
87
```

此外，Makefile 中还多次用到了文件列表，对于模板中所有 –f 参数后面对应的列表文件，在使用时都应该一一检查，保证所有文件列表都与实际要调用的一致。

如果在仿真过程中需要 DUMP 下来相应的波形文件，需要用到以下几个 VCS 指令：

$dumpvars（）：dump VCD 格式的波形文件；

$vcdpluson（）：dump VPD 格式的波形文件；

$shm_open（）、$shm_probe（）：dump SHM 格式的波形文件；

$fsdbDumpvars（）：dump fsdb 格式的波形文件；

具体使用方式间下图：

```
`ifdef DUMPVCD
    initial begin
        $dumpvars();
    end
`else
`endif

`ifdef DUMPVPD
    initial begin
        $vcdpluson;
    end
`else
`endif

`ifdef DUMPSHM
    initial begin
    $shm_open( "th.shm" );
    $shm_probe( "AS" );
    end
`else
`endif
```

```
170 ********************************
    *******/
171
172 `ifdef DUMPVPD
173     initial begin
174         $vcdpluson;
175     end
176 `else
177 `endif
178
179 `ifdef DUMPFSDB
180     initial begin
181         $fsdbDumpvars(0,DUT);
182     end
183 `else
184 `endif
185
186 endmodule
```

图中定义的 DUMVCD、DUMPVPD 等宏定义，是为了方便大家在仿真中屏蔽相应的 DUMP 波形功能。我们可以在 VCS 指令中定义这个宏，也可以在验证环境中的宏定义文件中定义，不过要注意在编译的中间将宏定义打开，才能正常 DUMP 波形文件。

如果不需要 dump 下来所有模块的波形文件，大家还可以指定模块来 dump 波形：

$vcspluson(0, th.m_DUT.m_USER)；

这样就可以指定只 dump th.m_DUT.m_USER 模块的波形。

📖 **说明：**

本说明中没有 VCS 中各种指令参数的说明，大家可以根据模板中调用的参数，在 VCS 用户手册中查到相应的功能介绍。不明白的地方，欢迎拿出来大家一起讨论。

---

cshrc.project 文件是整个项目的公用验证环境设置 cshrc 文件，在使用时需要将 env 文件也下载下来，并修改其中的 DESIGNWARE_HOME 路径到本项目所用 VIP 路径中。

在使用本 Makefile 脚本时，可以不用在每条指令中都加入 source $(CSHRC)/cshrc.project，只需要在.cshrc 文件中加入如下几个宏定义即可：

```
314  setenv DESIGNWARE_HOME /project8/SD5000V100/LIBRARY/designware
315  setenv PRJ_NAME /project8/SD5000V100/z67234/SD5000V100
```

当这样使用时，注意保证项目组内部所有仿真工具的版本要一致。

3.5 验证案例

3.5.1 后仿真指导

1. 后仿真基本概念和背景知识

后仿真，也就是 Postlayout simulation, 准确的说是在网表已经完成了物理层的设计，cell 本身的延时（cell delays）还有连接延迟（interconnect delays）已经被准确的估算出来之后，再对网表反标（back-annotation) 准确的 SDF 信息，然后进行的仿真。后仿真的主要目的是对网表的时序进行检查。[22]

通常，我们所指的功能和时序的仿真分为以下几类：

- 功能仿真（Functional Simulation）
- 静态时序验证（STA）
- 门级网表仿真（Gate-level simulation or prelayout simulation）
- 后仿真（postlayout simulation or dynamic timing analysis）
- Swith-level simulation（不涉及）
- Transistor-level simulation（不涉及）

功能仿真通常是在进行逻辑设计，投递 100%网表之前的仿真工作。对于物理层的仿真主要是静态时序验证，门级网表前仿真和后仿真。这里重点谈一下三者之间的区别。对于静态时序仿真，不需要使用任何测试向量（test vector），主要是通过分析各条路径上的数据和时钟之间的相位关系，来分析网表的时序是否正确。现在在光网络的流程中，这部分工作由专职的后端人员来做。对于 prelayout simulation 和 postlayout simulation 而言，prelayout simulation 通常是在返回 85%，95%和 100%网表时进行的。这时候进行仿真的网表的 cell 已经有了 delay 信息，但是 interconnect delays 还没有包含在内，同时仿真器的 -notimingcheck 选项被打开，不进行时序检查。此时的仿真仅仅进行了网表功能上的仿真。

只有在 IBM 的 RTC（release to customer）阶段，IBM 返回了最终网表和交付了 SDF 信息之后，才能进行 postlayout simulation，但是前期仍有许多的准备工作需要完成。

本文中将没有反标 SDF 信息的网表仿真都称为前仿真，反标了 SDF 信息的仿真都通称为后仿真。

2. 后仿真流程介绍

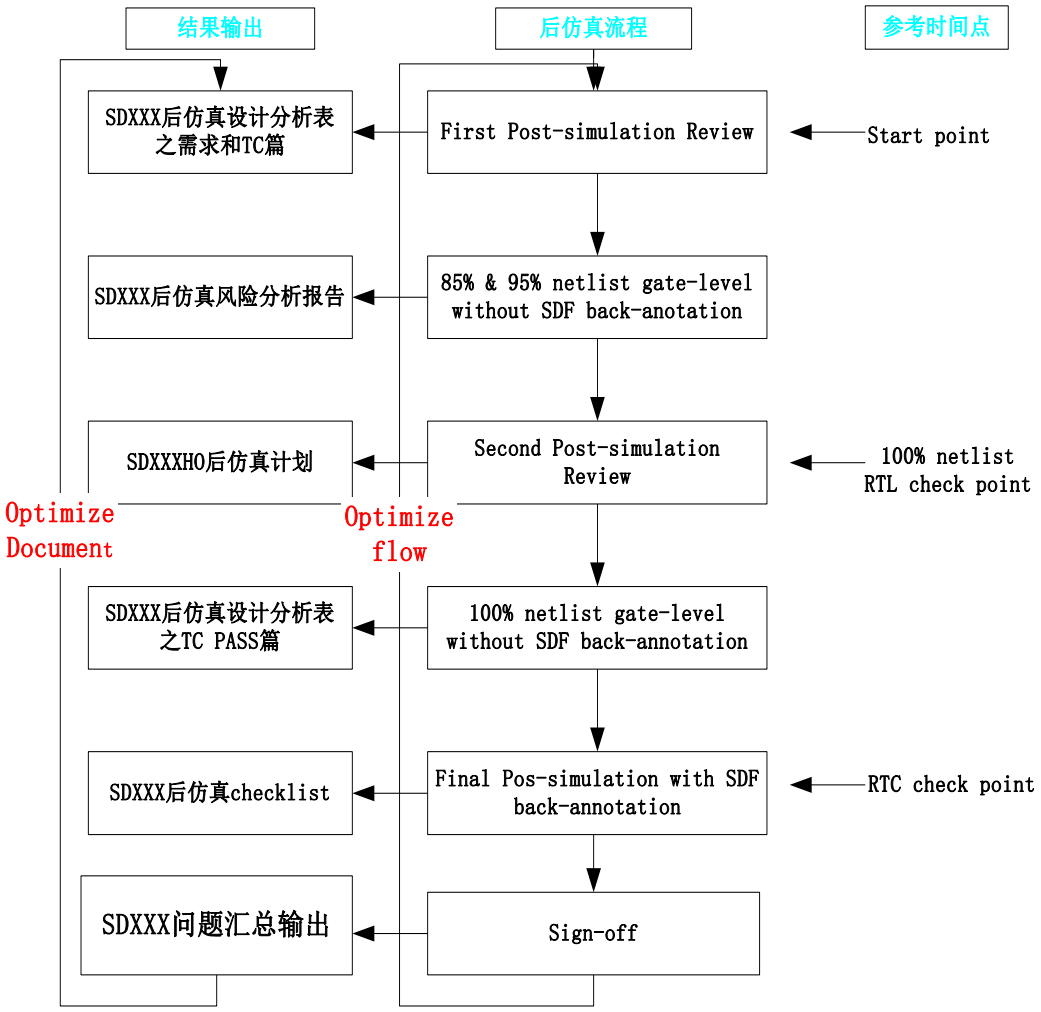


图3-46 后仿真流程

在投出 85%或者 95%网表之后，需要进行后仿真环境的分析，确认需求和仿真用例，此时最好在项目组内部和设计人员和 STA 人员进行一次评审，确认需求是否完整，测试向量是否完整。经过评审要保证后期的仿真验证的需求更改不能够超过 5%。然后开始对网表进行 prelayout simulation，根据仿真的结果和其它因素要确认出后仿真的关键路径所在。这将为后期的项目制定后仿真计划提供可恒量的依据。在 100%提交后，根据前期的仿真向量的运行过程和第一次的评审结果，进行第二次评审。评审后要完全确认后仿真向量，同时制定出可行的后仿真计划。在此之后，应该再没有仿真需求的变化。根据后仿真计划，可以开始对 100%网表进行再次仿真，此时要确认 TC 能够全部仿真通过。RTC 之后，在后仿真计划的指导下，开始进行 postlayout simulation.在后仿真运行的过程中，填写后仿真 check list，为最终的 TR4 评审提供依据。在整个仿真验证结束后，需要输出后仿真总结文档和问题，不断地优化流程和添加新的仿真典型用例，为后来人指路。

3. 后仿真的需求分析

(1) 需求分析的必要性

在后仿真开始时，很多做后仿真的新员工（包括我自己）都是想着先把后仿真运行起来，运行通过一条是一条，但是在真正做到了后期发现非常的痛苦。因为经常是一条向量非常辛苦地运行，定位，回归结束后，发现这条向量不符合要求，或者发现这条向量完全是重复的是没有必要的，但是自己已经辛辛苦苦地跑了好几个迭代来定位问题。我的反面例子告诉了大家在仿真还没有开始的时候，非常有必要做好需求的分析，为后期的仿真打个好基础。这就好比做芯片先确认好规格一样。

(2) 需求分析中需要考虑的因素

在分析后仿真需求的时候，要始终站到一个正确的角度上，后仿真不是验证芯片的功能而是为了验证芯片的时序。尤其从光网络几个失败的项目上看，后仿真在很大程度上需要将接口时序的验证放在首位。

【芯片的时钟域划分】

通常，我们在验证中要首先考虑芯片的时钟域的划分，一个芯片的后仿真工作难度的大小往往取决于芯片有几个时钟域，时钟域怎么分布，时钟域什么关系。因此，在开始后仿真前，验证人员应该对 STA 方案有一个清晰的认识。

通常的时钟之间的关系如表 3-3。

表3-3 时钟关系

Class	$\Delta\phi$	Δf
同步时钟	0	0
倍频时钟	0	n
异步时钟		

如果时钟域是倍频时钟域和同步时钟域，那么芯片的跨时钟域一般都位于 MPIGLB 模块，对于其它模块可以通过调整时钟相位解决。

如果时钟是异步时钟域，那么需要格外关注时钟域的分布状态。比如说我们的项目有 50MHZ,100MHZ,77MHZ, 155MHZ, 200MHZ,311MHZ.。在仿真中不仅仅要关住这些可能出现不定态的部分，还要根据时钟域的划分来设计仿真向量，尽量将可能出现不定态的跨时钟域部分做成一个运行速度快的小向量，减少因为定位跨时钟域导致部分不定态而导致的仿真回归时间较长的问题。

【芯片的内部特性】

芯片的内部特性通常指的是注芯片的工作模式，通道数等工作特性。后仿真应该保证覆盖到每一种模式，每个通道。这样做的目的是为了保证能够覆盖到芯片的边边角角。比如说 598 的项目中，有 622M 和 2.5G 工作模式，有主用和备用电路，所以确认向量的时候，因素都被评估进去。更加详细的描述见文档[14]。

【芯片的接口的种类和特殊需求】

在做后仿真的过程中，芯片的接口一定要重点验证，要尽量做到每一个接口都有一个运行时间能够容忍的向量。对于接口本身，要覆盖到各种模式和各种情况。比如说做 SRAM 接口和 SDRAM 的接口的测试，要将有时序检测的模型挂上，然后添加上单板的延时进行测试。单板的延时要根据 STA 人员提供给产品线的值在一个范围内进行左右抖动。也就是说，要模拟将芯片放在了一个单板上的情况，把它的时序情况测试完整。

芯片的特殊需求主要是指针对设计人员一些担心的问题的测试和仿真，这个需要在后仿真人员的引导下，挖掘设计人员的需求。

(3) 仿真向量的确认

在确认了仿真需求后，根据需求和原有的仿真向量，来确认那些向量需要运行，那些向量需要修改，列出详细的向量的列表。

4. 前仿真和后仿真的环境搭建

在运行后仿真之前，使用前仿真运行通过后仿真的所有向量是十分必要的。如果环境没有调试通过前仿真或者前仿真的时间太长，都将会大大加大后仿真的难度和短时间内完成后仿真的风险。从我们几个过去的项目经验上来看，前仿真已经成为后仿真一个重要的步骤。在前仿真环境搭建的过程中，需要详细考虑的问题在本章节中将会有详细的论述。

(1) 接口延迟添加 [3]

添加接口的延迟主要从两个方面来考虑，因为 cell 的延迟已经在前仿真中被包含进去，如果没有添加接口的延迟会导致数据的采集错误的情况。另一方面，在前仿真和后仿真中，需要对接口的时序进行验证，此时就要将一个芯片模拟在单板的应用环境中，而接口的延迟是我们提供给产品线的一个重要参数。它也需要被验证，所以需要将单板的延时模拟进去。

在海思其它的项目中，出现过没有模拟单板延迟导致芯片的接口时序有错误，而导致 eco 的情况，同时，在 sd598 的项目中，也验证提供给产品线的 SRAM 接口的单板延时不准确的情况。所以接口的单板延迟一定要被添加进去。

(2) 不使用的管脚不要悬空，避免不必要的干扰[14]

按照芯片的实际使用情况，被悬空的管脚应该不需要进行任何处理，而它本身的值也应该不会造成任何的影响。在做前仿真何后仿真时，为了一方面验证悬空的管脚对芯片是否有影响，同时也为了方便分析波形，所以通常要将悬空的管脚进行处理。

处理的方式应该是随心所欲的，可以置为‘1’或者‘0’或者随机都可以。[14]

(3) SRAM 熔丝复位问题[9]

在使用 IBM 库文件中的 SRAM Cell 进行前仿真的时候，发现 SRAM cell 不能够正常的工作，如图 3-47 所示。

```
Release Date 03/03/31 10:48:02

IBM ASIC Cu-11 FUSEDECOMPRESSION

Notes:

RequireReset parameter:

  The FUSEDECOMPRESSION macro requires the user to execute a
  power-on-reset before the memories in the decompression domain can be
  used. However, in simulation it may be desirable to bypass this reset
  requirement in order to expedite the verification process. Please
  note: A post-DFT simulation must be run with the power-on-reset
  enforcement enabled to ensure proper functionality of the
  design. Failure to ensure proper POR will result in broken redundant
  memories!

  To bypass the fuse power-on-reset requirement in simulation:

    defparam <path to cell>.iO.RequireReset = 0;

    Please note that the simulation looks for the inputs to move to a
    valid post-POR state.  For instance ACLK low, POR low and SE low
    should work.  The memories will begin to work 20ns after the inputs
    become valid.

Messages:

  Define the MessageLevel parameter to change the message
  verbosity.  For example:
    defparam <path to instance>.iO.MessageLevel = 1;
  Message levels are:
  0 - no messages      - totally quiet
  1 - normal limited   - same as 2
  2 - normal (default) - severe messages only
  3 - debug limited    - same as 4
  4 - debug            - verbose debugging messages

  You can also set a string that will print with each message by defining
  the 'CellIdent' parameter, perhaps to provide a human readable name, or
  an easily searchable string in the simulation log.
    defparam <path to instance>.iO.CellIdent = "Decompression Macro 1";
```

图3-47 SRAM cell 的文档注释

图 3-47 中 SRAMCELL 的注释文档所示，在使用 SRAM cell 进行仿真的时候，需要将 RequireReset 设置为‘0’，否则 SRAM 需要 10ms 的时间进行复位。所以在进行仿真的时候，我们可以将这个参数设置为‘0’，但是在进行 post-DFT 时，这个参数需要设置为‘1’。

所以在进行前仿真和后仿真的时候，需要将这个值进行置为‘0’。

(4) 将 IBM 的补丁库调入到编译文件列表[11]

在前仿真和后仿真的过程中，需要将 IBM 的补丁库编入到仿真列表中，然后进行编译，否则会遭成部分器件工作不正常的情况。

在我们的前仿真和后仿真过程中，出现过一个寄存器的时钟输入端 Wclkp 端没有正常的时钟脉冲信号，这导致寄存器不能够正常的工作，但是我们在仿真没有反标 sdf 信息的网表时，这个寄存器的时钟输入端工作正常。当我们将 IBM 的补丁库中的 clkchop.v 提出来，手工加入编译文件。这个寄存器的时钟输入端恢复正常。

```
// (C) Copyright International Business Machi
// All rights reserved
// Verilog ModuleGen generated module
// Book:           CLKCHP
// Func:           CLKCHOP
// Date Generated: Tue May 16 15:32:04 2000
// Date Extracted: 3/28/04
// Version:        1.2
//
module CLKCHP (Z,ENI,ENN,GATE,OSC);

  output  Z;
  input   ENI;
  input   ENN;
  input   GATE;
  input   OSC;

  parameter clkch_pw=0.5;
```

(1)

```
 9 //
10 `ifdef VCS
11 `else
12 `delay_mode_path
13 `endif
14 `timescale 1 ns / 10 ps
15
16 module CLKCHP (Z,ENI,ENN,GATE,OSC);
17
18   output  Z;
19   input   ENI;
20   input   ENN;
21   input   GATE;
22   input   OSC;
```

(2)

图3-48 IBM 的库文件（1）与补丁库文件（2）的比较

通过比较分析发现原来的 IBM 库文件中的 clkchp.v 文件和补丁库中的 clkchp.v 文件，发现补丁库中的文件中设置了一个局部的 time scale. 在编译的过程中，如果一个文件没有 time scale, 它会默认调用上一个文件的 time scale。如果上一个文件也没有的的话，它会使用系统的 time scale. 这可能会造成 time scale 的精度不够，脉冲太窄。在没有反标 sdf 的仿真中，这个脉冲的宽度为 1ps。因为没有打开-timing check 的功能，所以仿真不检查时序。只要脉冲有上升和下降沿，寄存器就可以正常的工作。在反标了 sdf 之后，线上有了以 ns 为单位的 interconnect delay 延迟。在这种情况下，这个 1ps 的脉冲会被仿真器忽略掉，认为为 0。

(5) 编译脚本的注意问题[1][14]

前仿真和后仿真的编译脚本都可以在前人哪里找到很多好的样板，这里所要讲的重点是在编译脚本应该注意的一些问题。

在编译脚本中，还要注意-SDF_PRECISION 的参数的使用，根据仿真本身的原理，如果不加入-SDF_PRECISION 参数，反标的时候按照 SDF 中真实延时进行反标，但是如果设 -SDF_PRECISION 100ps，那么仿真器保留 SDF 文件中的 1 位小数，而 IBM 返回的 SDF 文件中的延时都带有 3 位小数，所以如果错误的使用这个参数将会导致 SDF 的反标信息不准去，导致仿真错误。

另一个需要注意的是使用 +no_pulse_msg 和–pulse 这两个参数，这两个参数的使用导致一个项目有问题，但是我本人对于这个参数的使用理解的也不是特别的深刻，大家可以参考林强编写的案例[1]。我本人觉得如果多个项目没有设置这个参数，同时也没有问题出现，根据经验可以考虑不设置这两个参数，但是实际的情况需要实际分析。

5. 后仿真问题定位和注意问题

后仿真过程中，最令人头痛的是问题的定位，因为仿真向量的运行周期非常长，同时，我们会发现我们遇到的问题经常都是一些不是问题的问题。我对 598 项目做过一个问题的统计分析。如图 3-49 所示，仿真的环境问题占到了问题的大多数。这从侧面也反映了前期进行环境调试非常的必要。

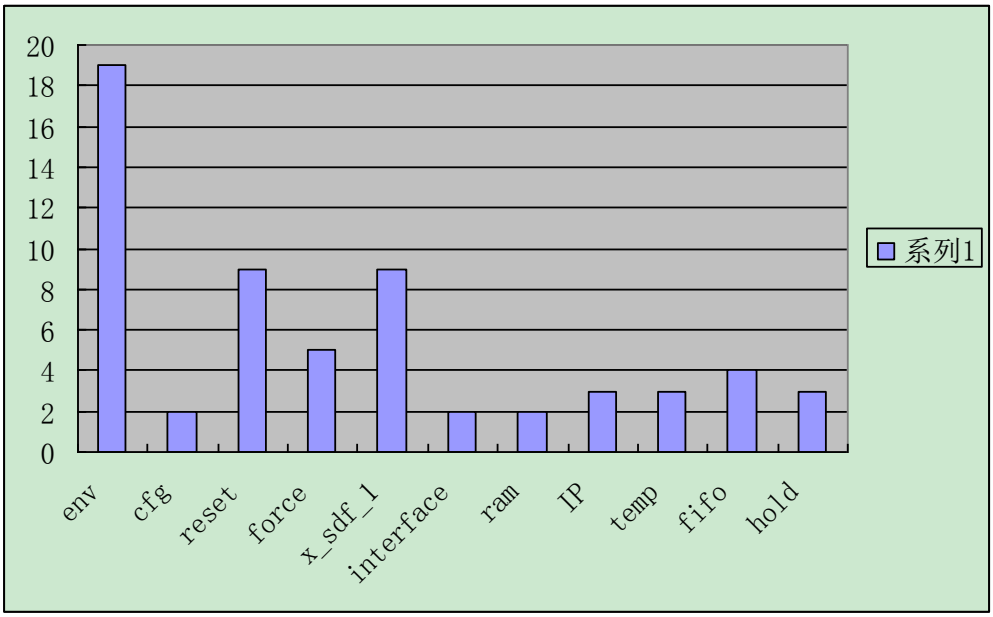


图3-49 SD598 的问题分析图

在问题的定位中，掌握一些定位问题要领，熟练的使用工具和提前熟悉一些常见问题，都会对后仿真工作有极大的帮助。本章节将会从这几个方面来论述一下。

(2) 问题定位方法和工具介绍

【后仿真问题定位的原则】

后仿真定位问题主要是和不定态打交道，初学者喜欢直接追踪不定态，一级一级向上查找，这种方法可以但是是一个事倍功半的方法。一个提高定位问题效率的原则是从宏观到微观。详细地说也就是先确认问题出现在那个模块，然后再确认问题出现早期模块内部的那个小模块，到通过分析代码无法再细分的时候，再在一个相对较小的范围内追踪不定态的来源。

在这个原则的基础上，需要熟练的地掌握 Debussy 和 Estimer 两个工具，它们将会使你如虎添翼，大大缩短问题定位的时间。

【常用不定态的定位工具的介绍】

从经验上说，后仿真人员应该熟练的掌握 DEBUSSY 的 trace 功能和 Estimer 的 trace 功能。整体上说，DEBUSSY 工具提供的 TRACE 功能和图形功能能够帮助你在不熟悉设计人员代码的情况下，很轻松地分析代码和模块的结构。Estimer 工具能够帮助非常微观地具体到每一个 cell 上来分析网表和分析问题。

通常情况下，没有办法分辨出那个工具更加优越，只能说各有千秋，需要在使用中体会。两种工具的使用有很多的培训课程和相关专家，应该很容易掌握，关键是使用中要体会那种工具在什么情况下，更适合当前的情况。

(3) 后仿真中的注意问题

本章节描述了后仿真常遇到的几种问题，每个问题在后仿真典型案例库中都有更加详细的描述。

【反标 sdf 信息是否成功】

在后仿真的开始时，第一件需要注意的重要的事情就是关注 SDF 信息的反标是否正确，影响 SDF 信息正常反标的原因有很多种，现在已经遇到过的项目，遇到的典型情况有以下几种情况：

- SDF 的格式不符合要求。有时候后端提供给仿真人员的 SDF 格式不是按照我们通常使用的 MIN:TYP:MAX）的格式，而是按照（MIN::MIN）(MAX::MAX)的格式，这将会导致我们在反标 typical 格式的时候，全部反标了零[2]。这种情况在 COT 流程遇到过，在光网络的后端中还没有遇到过。
- 另一个方面，我们在通常的反标过程中，worst case 的 SDF 通常对应着 MAXDELAY 的参数进行反标，best case 的 SDF 通常对应着 MINDELAY 的反标。这一点在这里再次强调。
- 在反标编译结束后，需要确认 ncelab.log 文件，将每一个告警文件都过一遍，分析每个告警是否正常，这个步骤是非常必要的，即使在告警信息较多的情况下，也需要通过脚本对告警信息进行分类分析。
- 在仿真启动后，要确认波形，分析一下 cell 的延时和 interconnect 的延时是否都添加正确。
- 在前面设计编译脚本的小节中，已经起到了仿真精度不准确造成的后仿真运行错误的情况，请大家再次关注这个问题。

【RAM 初始化的问题】

在后仿真时，需要将我们通常称为 ASIC RAM 从厂家提供的库中调出，然后进行仿真。此时如果没有对 RAM 进行初始化的话，RAM 中将会读出大量的不定态，这将会导致后仿真中有大量的不定态出现，会严重影响调试后仿真环境的进度。

为了解决这个问题有两类通常常见的方法：第一种方法是做一个批处理文件，然后使用这个批处理文件进行处理，完成 RAM 的初始化操作。详细文档参考[13]

第二种方法是在做芯片的顶层文件时，做一个用于 RAM 初始化的过程的操作信号，包括使能，时钟产生和初始化地址和初始化数据。强烈建议使用第二种方法。

【数据线清零的操作】

Embeded RAM 和 RA 的数据总线的初始值是不定态。在门级网表后仿真的过程中，这些不定态会发生扩散或者导致仿真不能进行。

例如：如图 3-50 所示，状态机中 Embeded RAM 对状态机造成了很大的影响。

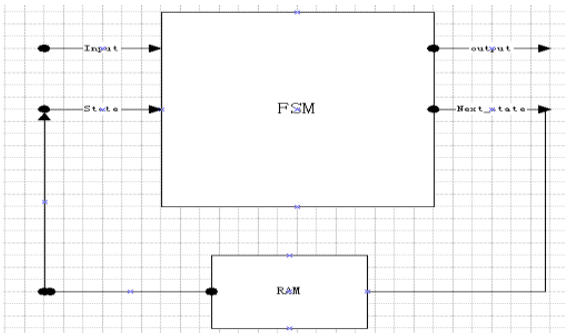


图3-50 状态机典型案例分析

如果 RAM 的初始化状态是'X'状态，那么'X' state 将会在反馈电路中被记忆和循环应用。这将导致状态机不能够恢复，进入到不定态出不来。这种现象在具有反馈逻辑的电路很常见，问题也很严重。

针对这个问题，通常有以下两种处理方法，使用 verilog fore 和使用 ncvlog 的 force 语句进行处理。根据个人经验 建议后仿真人员使用方案 1。

解决方案 1：使用 Verilog's force 语句，这种方法的详细使用说明见 verilog 参考手册。这种方法的优点是：这种方法功能原理简单，逻辑功能可靠，对仿真的影响很小；这种方法的缺点是：不能够进行批处理，需要大量的时间来进行调整。通常在使用这个方法时，有一个要领，不要对数据线进行处理，而应该使用对 RAM 的控制信号进行强制读德操作，将装状态读到数据线上，对数据线进行请零操作。

解决方案 2：使用 ncelab's force 的 force 功能，这种方法的详细使用说明见 nc_verilog 参考手册。这种方法的优点是：NC 仿真器提供了一个-keepvalue 的功能，这种功能可以在上电后，就对数据线进行 force 操作，然后可以保持这个 force 值，直到这个数据线被它的 driver 再次驱动。所以我们可以在上电后对 embedded RAM 和 RA 进行批处理 FORCE 为零，然后 20NS 后进行释放，这种方法具有仿真环境容易维护和调试的优点。

如图 3-51 所示。-ncelab's force 的使用方法：

```
force U_TOP_TOP.U_SD598_WRAP.U_SD598.@{\SD598_CORE/U_ENCP/U_RENCP/U_RENCP_CPU/U_PCNT16_CHEC_ERR/U_TPRAM_0/U_0/DO_RP1 } 0 -after 0ns -release -keepvalue
20ns
force U_TOP_TOP.U_SD598_WRAP.U_SD598.@{\SD598_CORE/U_ENCP/U_RENCP/U_RENCP_CPU/U_PCNT16_CHEC_ERR/U_TPRAM_1/U_0/DO_RP1 } 0 -after 0ns -release -keepvalue
20ns
```

图3-51 ncelab's force 的使用方法

【异步复位信号造成的窄脉冲的问题】

我们的设计中，存在异步复位的问题。IBM 在做物理层的设计的时候，异步复位信号会把时钟也复位掉。图 3-58 描述了寄存器异步复位的典型门级网表设计，寄存器的复位信号，同时控制 clock splitter 和 LPH6001。这样会导致在寄存器的时钟端出现了窄脉冲。波形和告警文件如图 3-52 和图 3-53 所示。

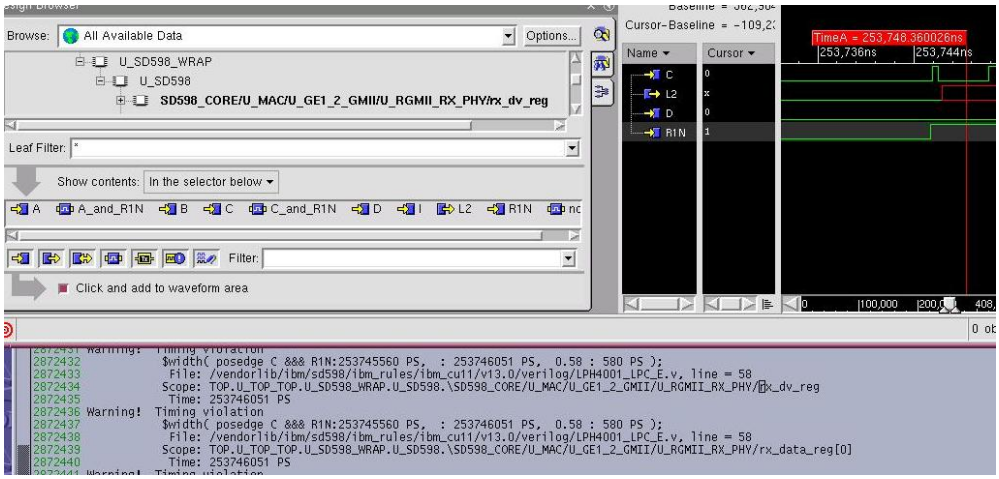


图3-52 窄脉冲现象的波形和告警文件

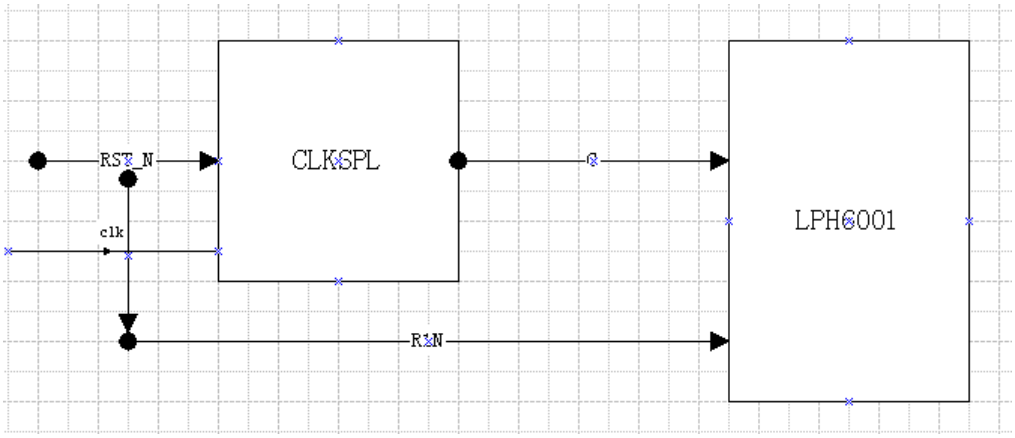


图3-53 典型的寄存器异步复位网表设计

问题的解决方案：

解决方案 1：建议在做 STA 时，将异步复位信号和时钟之间的相位关系做一定的约束，消除时钟上的这个窄脉冲。STA 小组有现成的脚本。

解决方案 2：根据告警信息的提示，我们可以将时钟信号平移 0.2ns，就可以解决这个问题，但是在平移时钟的同时，我们要考虑到这个平移对其他模块或者 TC 可能造成的影响。

【NC verilog 的 force 和 verilog force 问题的比较】

在描述数据线请零操作的时候，已经将两种方法进行了分析，比较和说明。我在这里再次提出来进行比较的原因是：在选择解决数据线上的不定态问题的解决方案时，我们在一开始使用的是 ncelab's force 的 force 功能，但是后来发现一个没有办法解释的问题，导致我们最终放弃了这种方法，采用了 verilog's force

语句方案，这个问题耗费了很长的时间。我将在使用 ncelab's force 的 force 时遇到的问题现象描述一下，同时在后面将 verilog's force 这种方法的使用过程也将进行详细地阐述。

问题的现象：

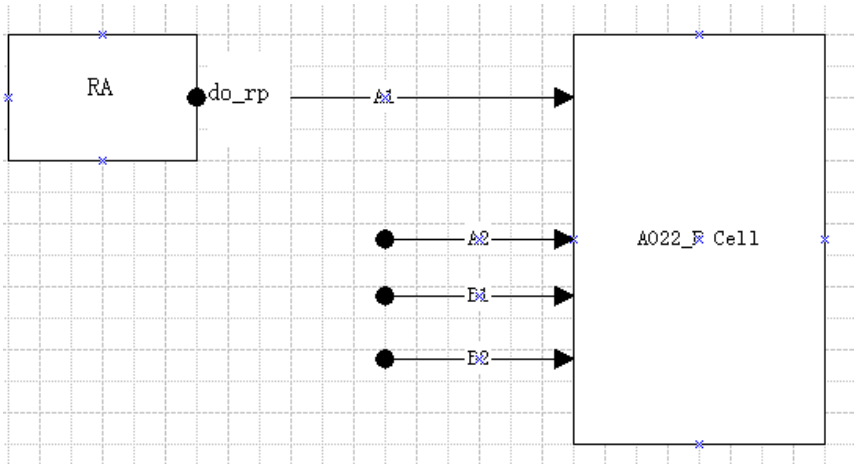


图3-54 ncelab'force 问题的逻辑电路图

我们在仿真的过程中，仿真反标了 SDF 信息的网表，发现 AO22_F CELL 工作不正常。我们将 AO22_F cell 和 register cell 提出来，分析了这两个 cell 的输入和输出的接口。Register 的输出接口 do_rp 有数据的正常变化，但是在 AO22_F cell 的 A1 输入端一直保持零，这导致 AO22_F cell 的输出为常零。我们将这条数据线的 ncelab's force 释放掉之后，AO22_F 的 A1 输入端恢复到正常工作，cell 本身恢复正常工作。根据 Ncelab 的工作原理，这种现象是不应该出现的。我们做了详细的仿真实验过程，仿真过程的纪录见附录 1。

问题现象的原因分析：

这个问题很奇怪，我们也定为不出来问题的原因，我们认为仿真器本身可能有 bug 存在，希望在以后的仿真中能够找到这个问题产生的原因。

问题的解决方案：

使用 verilog's force 语句，来进行数据线上不定态的清除操作。

verilog's force 的使用方法：

方法 1：在上电后对数据线进行 force，当数据线恢复正常后，进行 force 的释放。这种方法有两个问题：一、我们需要找到每个数据线准确的 force 释放的时间点。二、每次网表更换后，将需要重新查找这个时间点。这两个缺点造成环境调试时间长和很难维护。

方法 2 ：在 ram 初始化结束后，使用 force 语句对 RAM 和 RA 进行一次强制的读操作。在读操作完成后，数据线会有正常的数据产生。

进行强制读操作时的注意问题：

IBM's RAM 的工作原理如图 3-55 所示，RAM 不是持续的提供时钟信号。读写控制信号相‘或’，然后控制时钟信号的 MUX，当进行读或者写操作时，向

RAM 提供时钟信号。根据 RAM 的工作原理，可以由两种进行强制读操作的方法：第一种，我们控制写（wr）和读（rd）信号，这样可以在进行强制读得过程中，同时有时钟信号送入。第二种，直接在 cell 的接口上进行 read 的强制读操作。这样的话，我们需要直接向 RAM 输送时钟信号。代码如图 3-55 所示。

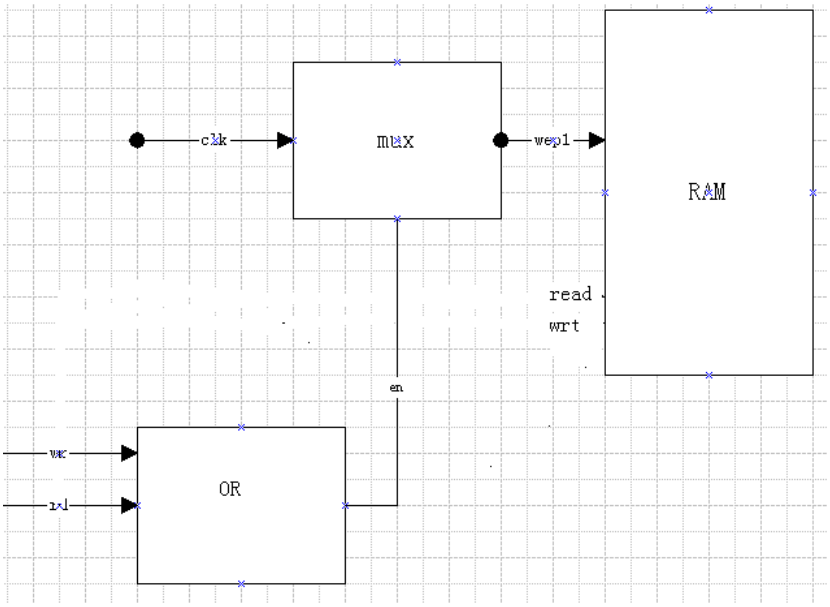


图3-55 IBM's RAM 典型逻辑设计图

```
always @(posedge U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_CLKRST/rst_n ) begin

if (rst_num < 2) begin
        #5000
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_ENCP/U_TENCP/U_TENCP_TDP_MATH/U_TPIRAM_FIFOMEM1/U_TPRAM/U_0/READ_P2_11 = 1;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_A/U_R2W2RAM4096X37/U_0/MUX_CTL_N38910 = 1;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_B/U_R2W2RAM4096X37/U_0/MUX_CTL_N38911 = 1;


        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_ENCP/U_TENCP/U_TENCP_TDP_MATH/U_TPIRAM_FIFOMEM1/U_TPRAM/U_0/CCLK2 = 1;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_A/U_R2W2RAM4096X37/U_0/CCLK = 1;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_B/U_R2W2RAM4096X37/U_0/CCLK = 1;

        #4
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_ENCP/U_TENCP/U_TENCP_TDP_MATH/U_TPIRAM_FIFOMEM1/U_TPRAM/U_0/CCLK2 = 0;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_A/U_R2W2RAM4096X37/U_0/CCLK = 0;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_B/U_R2W2RAM4096X37/U_0/CCLK = 0;

        #4
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_ENCP/U_TENCP/U_TENCP_TDP_MATH/U_TPIRAM_FIFOMEM1/U_TPRAM/U_0/CCLK2 = 1;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_A/U_R2W2RAM4096X37/U_0/CCLK = 1;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_B/U_R2W2RAM4096X37/U_0/CCLK = 1;

        #4
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_ENCP/U_TENCP/U_TENCP_TDP_MATH/U_TPIRAM_FIFOMEM1/U_TPRAM/U_0/CCLK2 = 0;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_A/U_R2W2RAM4096X37/U_0/CCLK = 0;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_B/U_R2W2RAM4096X37/U_0/CCLK = 0;

        #4
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_ENCP/U_TENCP/U_TENCP_TDP_MATH/U_TPIRAM_FIFOMEM1/U_TPRAM/U_0/CCLK2 = 1;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_A/U_R2W2RAM4096X37/U_0/CCLK = 1;
        force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_B/U_R2W2RAM4096X37/U_0/CCLK = 1;

        #4
        release U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_ENCP/U_TENCP/U_TENCP_TDP_MATH/U_TPIRAM_FIFOMEM1/U_TPRAM/U_0/CCLK2 ;
        release U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_A/U_R2W2RAM4096X37/U_0/CCLK ;
        release U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_B/U_R2W2RAM4096X37/U_0/CCLK ;

        #10
        release U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_ENCP/U_TENCP/U_TENCP_TDP_MATH/U_TPIRAM_FIFOMEM1/U_TPRAM/U_0/READ_P2_11 ;
        release U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_A/U_R2W2RAM4096X37/U_0/MUX_CTL_N38910 ;
        release U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_SPI/U_RSPI/U_RSPI_BUF_MNG/U_CELL_BUF_RAM_B/U_R2W2RAM4096X37/U_0/MUX_CTL_N38911 ;
end

rst_num = rst_num + 1;
```

图3-56 RAM 的 verilog 强制读操作代码

- 一定要在 RAM 初始化之后，RAM 使用之前进行读操作，来保证数据线的正常。
- 在进行强制操作的时候，一定要避免在对 ram 进行写操作的同时，进行读操作，这将会造成人为的读写冲突。这种读写冲突将会造成不定态产生和扩散。

【NC verilog 对 force 功能支持不够好的问题】

在 SD598 和 SD8901 的项目中都遇到过 NC verilog 对 force 功能支持不够好的现象，这里将 598 遇到的问题和问题的现象进行了一个详细的记录和评论 。

问题描述：我们在仿真的过程中，在 nc 的 console window 进行 force 一下两个信号的 force，如图 3-57 所示。

```
wire [7:0] \SD598_CORE/U_MPIGLB/U_DDR_RDP/rdata ;
wire [7:0] \SD598_CORE/U_MPIGLB/U_DDR_RDP/rdata_1 ;
```

图3-57 准备进行 force 的信号

但是仿真器上的进程马上变成了死进程。

我们将这两个信号的 force 的操作，改为使用 verilog 语言进行实现，如图 3-58 所示。

```
//———————————— for DDPPHY module ————————————
//initial begin
//force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_MPIGLB/U_DDR_RDP/rdata_1 [4] = 1;
//force U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_MPIGLB/U_DDR_WDP/rdata [4] = 1;
//end
```

图3-58 采用 verilog 语言进行的 force 操作

我们重新使用仿真器运行仿真，仿真进程不能够正常的启动，显示如下图所示的报警信息。

```
ncsim: *internal* (rts_net_fse - bad stopreason (27)).
Observed simulation time : 0 FS + 0
Please contact Cadence Design Systems about this problem
    and provide enough information to help us reproduce it.
***Current stack trace:
 -->[User Code       ] (nil)     <don't know>
 -->[Don't Know      ] 0x86191f7 <don't know>
 -->[Don't Know      ] 0x8619dc5 <don't know>
 -->[Don't Know      ] 0x847ca77 <don't know>
 -->[Don't Know      ] 0x847e637 <don't know>
 -->[Don't Know      ] 0x84f033a <don't know>
 -->[Don't Know      ] 0x84db8f6 <don't know>
 -->[CMD RUN/CMD Overhead] 0x8235ba5 simcmd_run          + 4db
 -->[CMD Overhead    ] 0x8237215 <don't know>
```

图3-59 进行 Force 时 NC 仿真器出现的出错信息

问题的分析和解决过程：我们首先将除了以上所述的 froce 信号之外的所有信号，回归到以前的版本。这些版本和环境已经经过了很长时间的验证。

解决过程：

- 我们仿真不反标 SDF 信息的进程，仿真能够正常的启动。
- 我们仿真反标 SDF 信息的进程，仿真不能够正常的启动。
- 我们仿真反标 SDF 信息的进程，不使用 specman E，仅仅使用 nc 启动进程，仿真不能够正常的启动。
- 我们仿真反标 SDF 信息的进程，分别使用 ncelab_specman 和 ncelab 进行进程的启动，我们将编译中的 timing check 选项关闭，仿真能够正常的工作。

我们对问题的结论：这两个信号的 force 不能够与 timing check 同时使用。这个问题和我们上一次遇到的 force 问题有很大的类似性。上一次不反标 sdf 信息，不打开 timing check 选项的仿真能够正常的运行。反标了 SDF 信息，打开 Timing check x 选项的仿真不能够正常的运行。

【不同频异步电路的解决方法】

问题现象：跨时钟域电路部分出现了不定态。通过将 library cell 提出来分析和分析 ncsim.log 文件，发现因为 re 现因为不满足时序的要求导致不能够正常的工作，有不定态产生。典型报警文件显示如图 3-60：

```
Warning!  Timing violation
          $setuphold(setup)( negedge C:507440 PS, negedge D:507310 PS,  0.15 : 150 PS,  -0.05 : -50 PS );
           File: /vendorlib/ibm/sd598/ibm_rules/ibm_cu11/v13.0/verilog/LPH0101_LPC_E.v, line = 50
          Scope: TOP.U_TOP_TOP.U_SD598_WRAP.U_SD598.\SD598_CORE/U_MAC/U_GMAC_1/U_MAC_CORE/fifo_intfc_1/tx_fifo/U_TPRAM/U_0/DIO2WP1_latch
           Time: 507440 PS
```

图3-60 不同频异步电路出现不定态的典型报警信息

问题现象的分析：

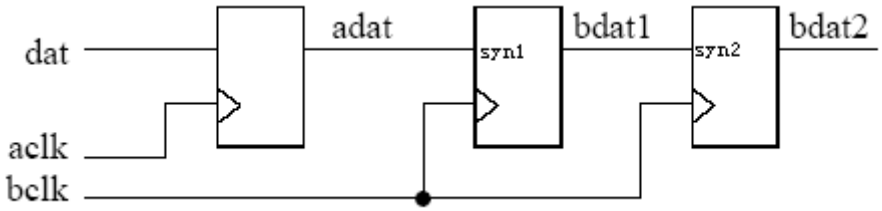


图3-61 SD598 项目中跨时钟域电路的同步器的典型设计

如图 3-61 所示，当 a 时钟域的数据 adat 传送到 b 时钟域时，在 syn1 寄存器将会产生时序不满足的情况，这导致了 bdat1 为'x' 态，导致了不定态会向下一个寄存器传播。Syn1 时序不满足情况的波形如图 3-62 所示。

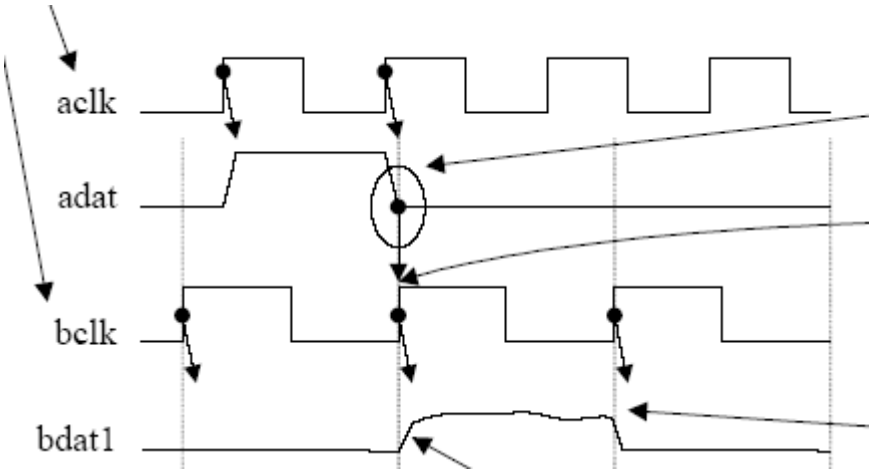


图3-62 syn1 寄存器时序不满足的典型波形

问题的解决方案：

解决方案 1：根据 syn1 的 setup 的时序约束的要求，平移 aclk 或者 bclk 使 adat 和 bclk 之间的时间间隔满足时序要求。这种解决方案的应用前提通常是 aclk 和

bclk 之间成倍频的关系或者最小公约数能够满足 setup 时序约束的要求。在 sd8501 和 SD579 项目组中，都有成功的应用。

解决方案 2：在 sd598 项目中，因为整个 sd598 芯片有 100MHZ,155MHZ,50MHZ 和 125MHZ 四个时钟域,所以使用时钟平移的方法，能够得到的最大时间间隔为 0.4ns。这个值不能够满足约束的要求，所以我们选择使用了仿真器的-tfile 功能来解决以上问题。

问题的影响度：高

-tfile 解决方案详述：

1）-tfile 的方法描述：

-tfile 是 ncelab 提供的屏蔽某个单元的时序检查功能的方法。参考 nc_vlog manual 中的 Elaborating the design with ncelab 这个章节中的 disabling the timing checks 部分。这种方法将会关闭掉整个 cell 的所有接口的 timing checks, 同时不再打印告警信息。但是它不会影响到 cell 的 delay 的计算。

2）-tfile 的使用方法：

首先，将所要关闭掉的 REG 编写成一个单独的*. tfile 文件 例如：aa.tfile，file 内容如下：

PATH XXX_reg1 －tcheck

PATH XXX_reg2 －tcheck

PATH XXX_reg3 －tcheck

其次，在 ncelab 的编译脚本上添加一个参数－tfile aa.tfile

3）-tfile 的使用注意问题：

- 在使用-tfile 关闭了某一个 cell，将会导致这个 cell 上所有管脚的 timing check 都被关闭掉，所以在使用这个功能关闭某一个 cell 之前，一定要确定这个 cell 确实是只有异步路径，没有同步路径的存在。例如图 3-63 所示。

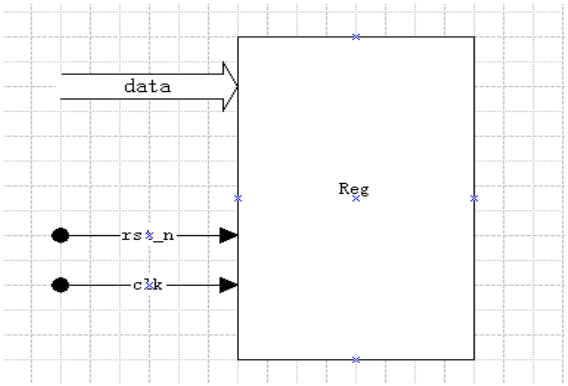


图3-63 同步数据接口，异步复位接口的寄存器

复位信号是一个异步复位信号，导致了 rst_n 的时序不满足。虽然我们可以使用 -tfile 的方法将这个 reg 的时序检查关闭掉，但是会导致同步数据端口 data 也无法进行时序检查，这将会导致一个同步电路无法检测。类似这种情况不能够使用-tfile 方法关闭 timing check 的方法进行仿真。

- 在某些情况下，使用-tfile 功能并不一定是最简单和有效的方法。如果可以使用平移相位的方法将问题解决掉，就尽量的使用平移相位的方法。这种方案影响面比较的小，可靠性较高。-tfile 的方法还需要时间的验证。
- 对于同频不同相位的问题，建议使用平移相位的方法解决。

【Hold violation 的问题分析】

在 IBM 的网表仿真中，存在此类影响比较大的问题。这类问题的特点是在这类问题的出现没有任何的规律，很难说在前仿真解决了之后，就不会再出现，所以需要大家对这类问题的现象以及问题的来龙去脉搞清楚，这会方便你后期的问题的解决速度。

问题的现象：

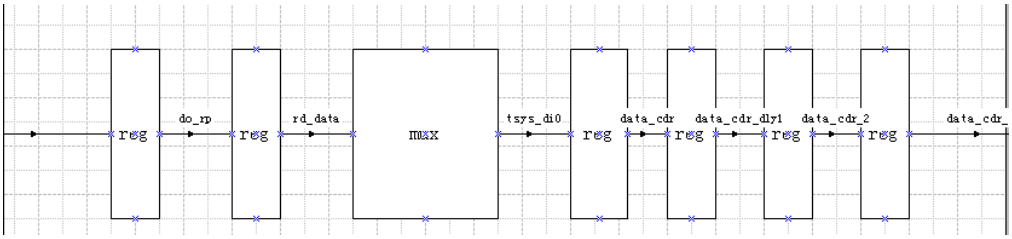


图3-64 逻辑设计图

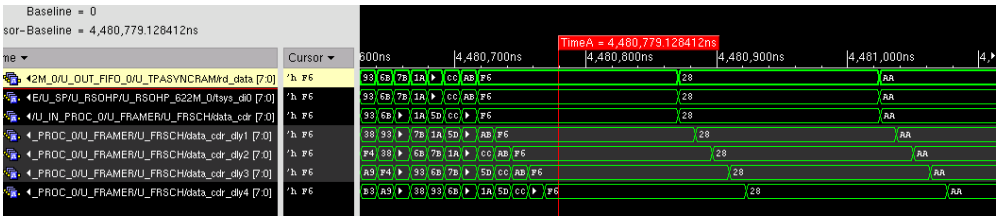


图3-65 对应的波形图

如图 3-65 所示，根据逻辑设计图的要求，数据在经过 DATA_CDR_REG 时，应该进行对应的打拍的操作，但是实际上波形中反映没有这种操作。

问题现象的分析：

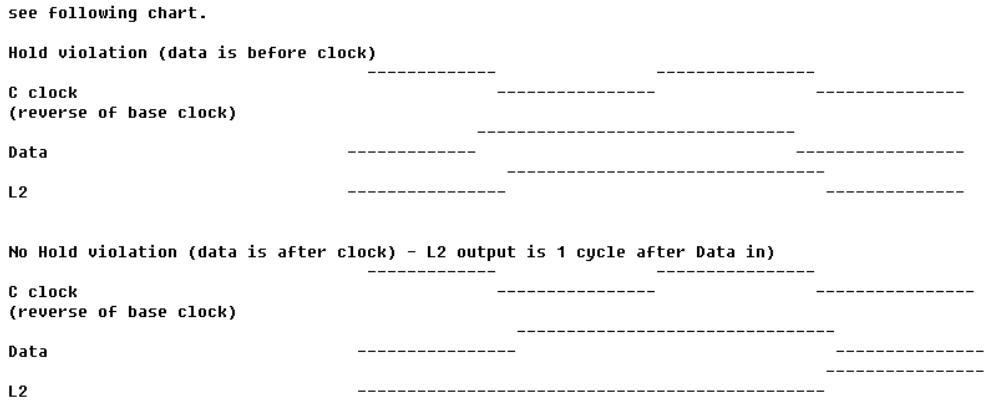


图3-66 LPH_4001_LPC_E 的时序图

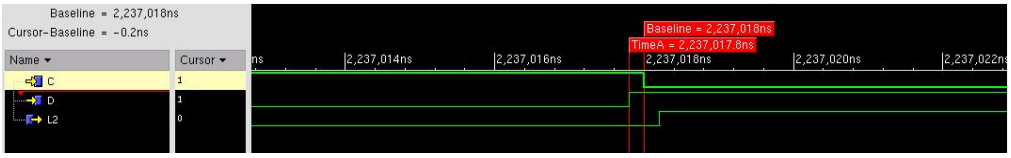


图3-67 LPH_4001_LPC_E 的实际仿真波形

结合 LPH_4001_LPC_E 的时序图和 LPH_4001_LPC 的实际仿真波形，我们可以清晰的看到，因为存在 hold violation－数据变化在时钟的沿之前。所以导致了数据采集时，没有正常的延迟数据的输出。

我们重新对 95%网表中，遇到的相类似的问题进行了分析。如图 3-68 所示，在 95%网表中，出现了部分寄存器没有打拍，如: DATA_CDR_REG[0]；和部分寄存器打拍的现象，如 DATA_CDR_REG[1]。

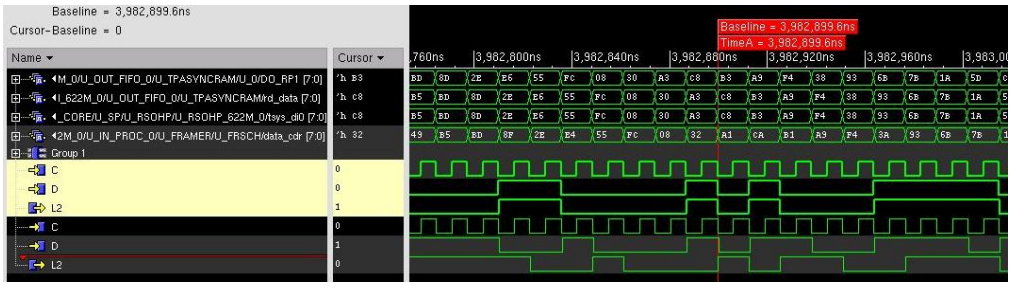


图3-68 95%网表中 hold violation 的波形

这种现象和我们 RTL 代码的实现是不同的。RTL 代码入图 3-69 所示。

```
133
134 //MAIN CODE
135 always  @(posedge clk_cdr or negedge rst_n)
136     begin :GEN_DATA_CDR
137         if(rst_n==1'b0)
138             data_cdr <= #`U_DLY 8'd0;
139         else
140             data_cdr <= #`U_DLY cdr_di;
141     end
142
```

图3-69 DATA_CDR 的 RTL 代码

为了分析产生这种现象的原因，我们使用 ET 工具将两个寄存器的网表显示出来，如图 3-70 所示。

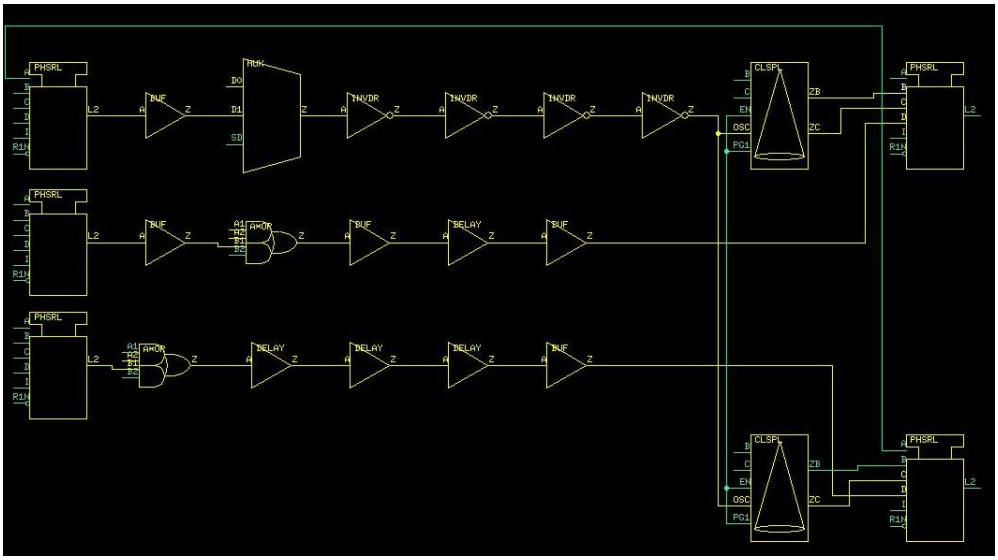


图3-70 DATA_CDR_REG[0]DATA_CDR_REG[1]的网表图

两个寄存器使用相同的时钟树。在数据链路上，data_cdr_reg[0]使用了 3 个 delay cell，但是 data_cdr_reg[1]使用了 1 个 delay cell。因为在没有插入 sdf 信息时，Buffer 没有延迟信息和 Delay 有 0.2 ns 延时信息，所以 data_cdr_reg[0]的数据线上的数据来的比 data_cdr_reg[1]晚 0.4ns，所以它没有 hold violation 现象。

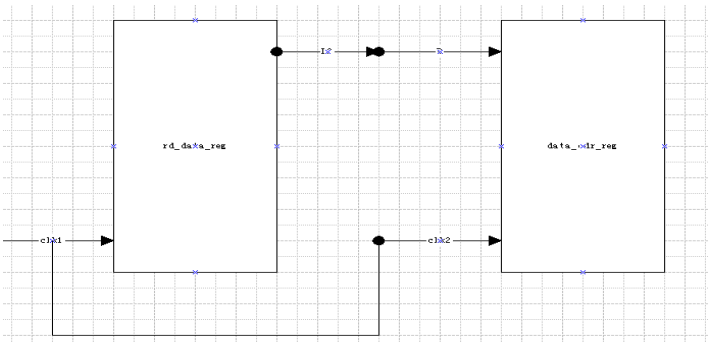


图3-71 两级寄存器之间的逻辑设计图

结合图3-69和图3-70，分析得到两个寄存器之间的时钟相位差值（CLK2-CLK1）为 2 个 buffer，数据链路的差值(D-L2)为 3 个 DELAY cell 和 4 个 buffer。

解决方案：此类问题没有明确的解决方案，需要实际问题实际分析，但是建议在进行后仿真的时候，要求 IBM 提供一份已经处理过 SDF 后的网表，来进行仿真。详细的文档描述参考[13]。

【FIFO 不定态问题的定位技巧】

FIFO 是我们通常处理数据流中异步电路时，普遍使用的方法，所以 FIFO 通常是比较容易不定态的位置。定位异步电路的不定态是在分析不定态工作中的一个难点。这类问题的定位方法在本小节中会有详细的描述。

问题的现象：因为异步 FIFO 经常被应用到跨时钟域的数据链路上，所以在数据链路上经常出现不定态。

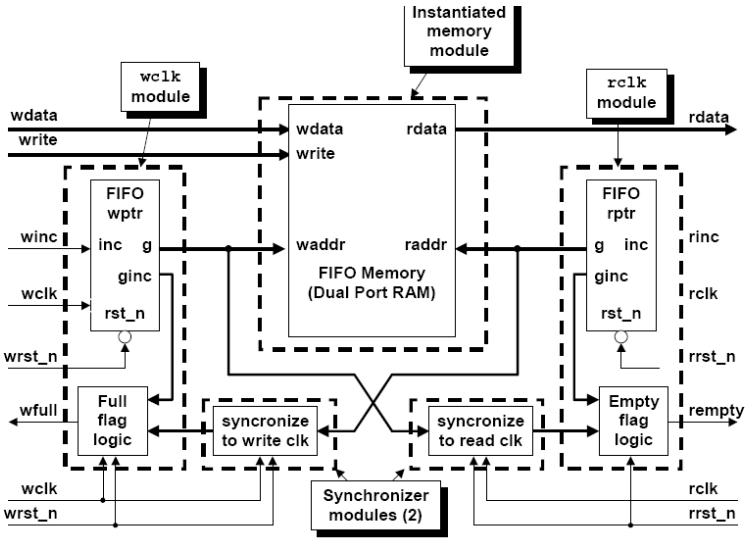


图3-72 FIFO 的典型逻辑设计图

问题现象的分析：如图 3-72 所示，典型 FIFO 的逻辑设计原理图。

通过分析发现，在 FIFO 的设计中，异步的读写地址需要首先经过同步器进行同步处理，然后进行比较处理，产生空满指示信号。空满指示信号和其它的逻辑控制信号进行组合处理，再产生计数器的控制信号，再产生地址信号。很明显，数据路径通常不是不定态产生的源头，而控制电路通常是不定态的产生源头所在。所以在分析异步 FIFO 电路产生不定态的原因时，可以排除掉数据链路对于分析的干扰，而集中分析读写控制电路和空满指示信号。

SDxx 芯片的 MAC 模块的数据链路不定态分析过程描述如下：

如图 3-60 所示，因为模块的数据链路了不定态，所以我直接开始分析模块内部的异步 FIFO 部分。首先，确认 FIFO 是否产生了不定态。如图 3-73 所示，RAM 的输出 DO_RP 上由不定态的存在，根据通常的不定态的分析思路，会继续追踪数据链路的上一级 cell，但是通常这种方法无法高效的找到不定态产生的源头。然后，直接提取读写地址指针和读些控制信号和空满指示信号，然后分析控制信号是否正常。如图 3-73 所示，因为异步 FIFO 的写控制器的同步器在第一拍采数据的时候产生了不定态，这一拍的不定态导致了数据链路上不定态的产生。

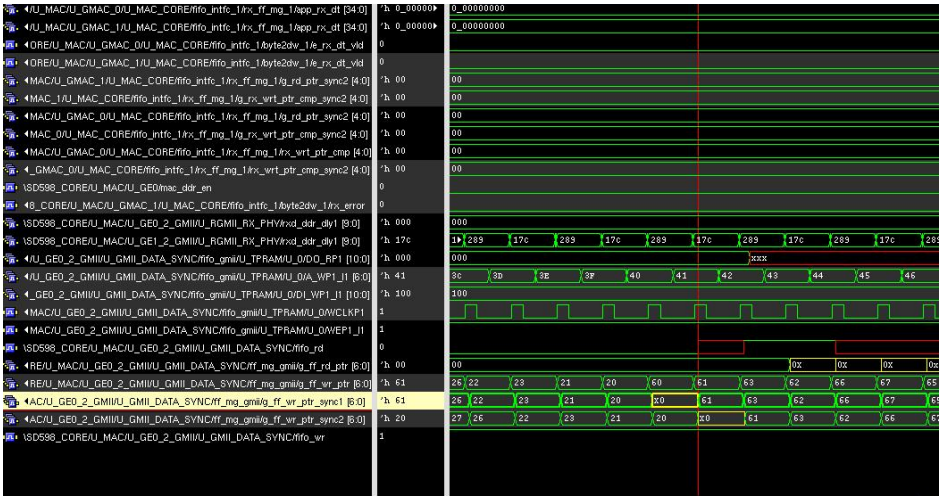


图3-73 MACDDR's FIFO 典型分析

问题的解决方案：为了解决 FIFO 上的不定态，将同步器上的同步寄存器的 timing check 关闭掉。

问题结论延伸：通常情况下，如果一个芯片的内部包含异步 FIFO，那么当这个模块的数据链路产生了不定态时，可以直接集中分析异步 FIFO 是否正常。在分析异步 FIFO 的过程中，可以先排除对数据链路的分析，而集中于对 FIFO 的控制电路的分析。

(4) 后仿真中的小技巧

本文描述了做过后仿真的人员在后仿真中积累的一些小技巧，但是这些技巧常常能够起到四两拨千斤的作用，所以在本文中我将一些心得描述出来。

【创建一个信号提取的通用文件】

在仿真的过程中，建立一个通用的接口信号，模块接口信号，以及其他的一些信号提取文件，然后在每次的仿真向量的运行过程中，都将这个文件读进来，然后在波形运行的过程中，定位出现的问题，以及问题可能出现的模块，这对你定位问题和提前发现问题都有重要的意义。

【创建一个波形分析文件】

在后仿真分析问题的时候，有两个比较常见的现象，一方面是问题总是在重复出现，另一方面是定位问题需要搜集大量的信号，这将大大降低我们的定位问题的速度。针对这两个问题，我们建议后来人可以针对每种常用的波形分析文件，将 simvision 存储下来然后方便后续的问题的分析。

【使用-no_sdf_warning 提高编译效率的技巧】

当大家做后续的后仿真工作时，也会发现一个比较让大家头痛的问题就是每次都要进行重新的编译，并且每次编译的结果都会有大量的 log 文件进行打印，这将大大降低我们的工作效率。所以在 SD8910 的项目中，进行了一次编译的告警信息的确认，然后不再进行编译信息的分析，而是使用-no_sdf_warning 的命令将告警信息全部屏蔽掉。

【使用通用的信号提取脚本】

在 SD598 的项目中，我们开发了一个信号提取的脚本，这个脚本可以使用关键字匹配的方法将所有匹配的信号都存下来，这将会保证你在分析问题的时候，不会出现因为没有足够的信息而导致无法分析波形的问题。脚本的存放位置可以向验证组组长需求帮助。

【IBM HSSCORE 的快速锁定模式[22]】

在仿真过程中，3 个 PLL 的锁定时间都非常的短，很容易看到 PLLLOCK 信号的锁定，但对于 HSSCore 上的 ULPLLLOCK 信号很长时间没有锁定，经过很长时间的仿真，发现在复位后大概 200us 时锁定（手册上描述为不小于 500 的时间才锁定），此时数据能够正常恢复。仔细阅读手册，发现手册中有个 PLLREGOVR 信号，此信号在仿真时设为 1，可以大大缩短 HSSCORE 内部 PLL 的锁定时间（大概 1.6us 左右），但 SD8901 用的是 WRAP，此引脚已经在 WRAP 内部接为常 0（正常功能），所以需要对 WRAP 内部 PLLREGOVR 进行 force 为 1。具体为下面三个信号：

U_ULR4B8ADEWV2.H2O_ULR4B8ADEWV2_001_00.PLLREGOVR

U_ULT4B8ADEWV2.H2O_ULT4B8ADEWV2_001_00.PLLREGOVR

U_ULF8S8ADEW1.H2O_ ULF8S8ADEW1_001_03.PLLREGOVR

在使用的过程中，需要注意的是上面三个信号在 simvision 图形窗口中进行 force 等于 1，不能达到快速锁定的目的，不知道这是不是 nc 的 bug！必须在脚本中或者 tb 文件中进行 force，HSS CORE 才可以快速锁定！

PLLREGOVR	Input - Digital	Tie '0' for normal operation. **Note:** This pin can be set to '1' when simulating the core, in order to shorten the activation time of ULPLLLOCK.

图3-74 手册中对 PLLREGOVR 信号的描述

【后仿真中断点的设置】

后仿时向量仿真时间长，如要存储波形就必需要分段存储，因为波形要是大于 2G 就很难打开了。通过在仿真脚本中设置选项-incsize 的参数就可以很好地控制波形的大小了。如图 3-75 所示，先指定波形文件的大小为 2G 一个，然后在脚本中将要观测的信号 probe 出来。向量跑起来之后，指定的波形目录下会出现 wave_loop.shm 波形目录，如波形大小超过 2G 后，工具自动会在此目录下再另存一个波形文件。

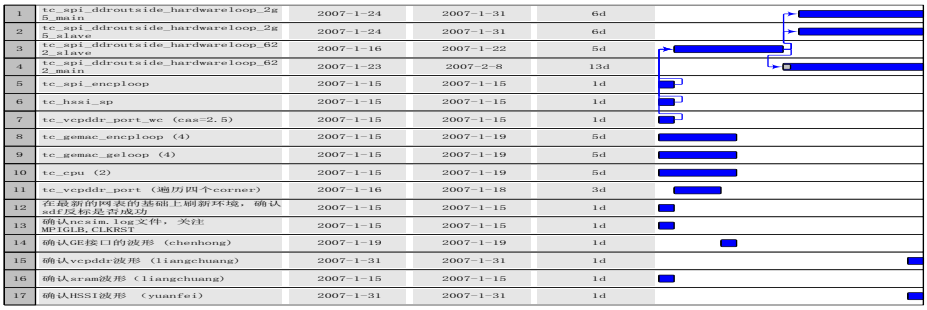
```
database -open -shm /project2/SD8901/gexiang/0126_min_postnetlist_log/wave_lopp
 -incsize 2G -default

probe -create -shm \
TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_PDH/U_OHPBC/U_OHP/U_OHM
PI/U_PRBSBLKREG2/regw[7] } \
TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_PDH/U_OHPBC/U_OHP/U_OHM
PI/U_PRBSBLKREG2/regw[6] } \
TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_PDH/U_OHPBC/U_OHP/U_OHM
PI/U_PRBSBLKREG2/regw[5] } \
TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_PDH/U_OHPBC/U_OHP/U_OHM
PI/U_PRBSBLKREG2/regw[4] } \
TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_PDH/U_OHPBC/U_OHP/U_OHM
PI/U_PRBSBLKREG2/regw[3] } \
TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_PDH/U_OHPBC/U_OHP/U_OHM
PI/U_PRBSBLKREG2/regw[2] } \
TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_PDH/U_OHPBC/U_OHP/U_OHM
PI/U_PRBSBLKREG2/regw[1] } \
TB_SD8901_NETLIST.U_SD8901_CHIP.@{\SD8901_TOP/U_LOGIC/U_PDH/U_OHPBC/U_OHP/U_OHM
PI/U_PRBSBLKREG2/regw[0] } -all
```

图3-75 使用示例

6. 后仿真计划制订

可以根据项目组评估的运行向量的数目和每一个向量的不同的运行情况，制定出符合自己项目的后仿真运行计划，建议使用 GANT 图进行计划的分析，下面将 SD598 的运行计划在这里演示一下，希望能够对后续的项目有帮助。



1	tc_spi_ddroutside_hardwareloop_2g5_main	2007-1-24	2007-1-31	6d	
2	tc_spi_ddroutside_hardwareloop_2g5_slave	2007-1-24	2007-1-31	6d	
3	tc_spi_ddroutside_hardwareloop_622_slave	2007-1-16	2007-1-22	5d	
4	tc_spi_ddroutside_hardwareloop_622_main	2007-1-23	2007-2-8	13d	
5	tc_spi_encploop	2007-1-15	2007-1-15	1d	
6	tc_hssi_sp	2007-1-15	2007-1-15	1d	
7	tc_vcpddr_port_wc (cas=2.5)	2007-1-15	2007-1-15	1d	
8	tc_gemac_encploop (4)	2007-1-15	2007-1-19	5d	
9	tc_gemac_geloop (4)	2007-1-15	2007-1-19	5d	
10	tc_cpu (2)	2007-1-15	2007-1-19	5d	
11	tc_vcpddr_port (遍历四个corner)	2007-1-16	2007-1-18	3d	
12	在最新的网表的基础上刷新环境，确认sdf反标是否成功	2007-1-15	2007-1-15	1d	
13	确认ncsim.log文件，关注MPIGLB,CLKRST	2007-1-15	2007-1-15	1d	
14	确认GE接口的波形 (chenhong)	2007-1-19	2007-1-19	1d	
15	确认vcpddr波形 (liangchuang)	2007-1-31	2007-1-31	1d	
16	确认sram波形 (liangchuang)	2007-1-15	2007-1-15	1d	
17	确认HSSI波形 (yuanfei)	2007-1-31	2007-1-31	1d	

图3-76 SD598 的后仿真计划 GANT 图

在制定后仿真计划的时候，需要注意下列问题：

- 明确后仿真中的关键路径。
- 明确后仿真的出口条件。
- 将运行时间较长的向量和运行时间较短的向量进行交叉运行。

7. Design for Post-layout simulation

建议 1：在设计中，应该对 RAM 和 RA 进行初始化操作。在操作结束后，对 RAM 和 RA 进行一次读操作，保证数据线上的信号没有不定态存在，可以被正常的使用。

建议 2：[5] 在设计中， 使用 if 和 case 设计结构，一定要使用 default 和 else 分支结构。这样在综合的时候，不会产生不需要的 level sensitive latch， 同时保证了电路即使遇到了不定态，也能够恢复成正常的状态。

建议 3：[1]对同步器上的数据信号，进行规范性的命名

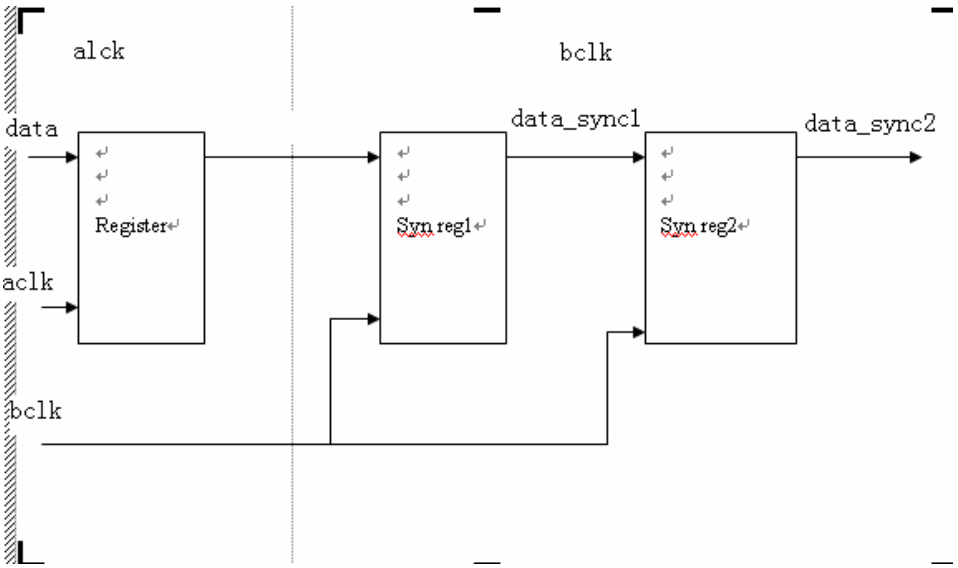


图3-77 同步器的规范命名示意图

在数据线上清楚地表明数据经过了两个寄存器的同步处理。这个命名可以规范为同步器使用。这样对于做后仿真时，判断是不是因为异步处理导致了不定态存在有重要的意义。

建议 4：在进行仿真的过程中，发现 MAC 模块加入了一个测试电路，这个测试电路可以将上行的数据链路的工作状态很好的反映出来，可以方便问题的定位。如图 3-78 所示。

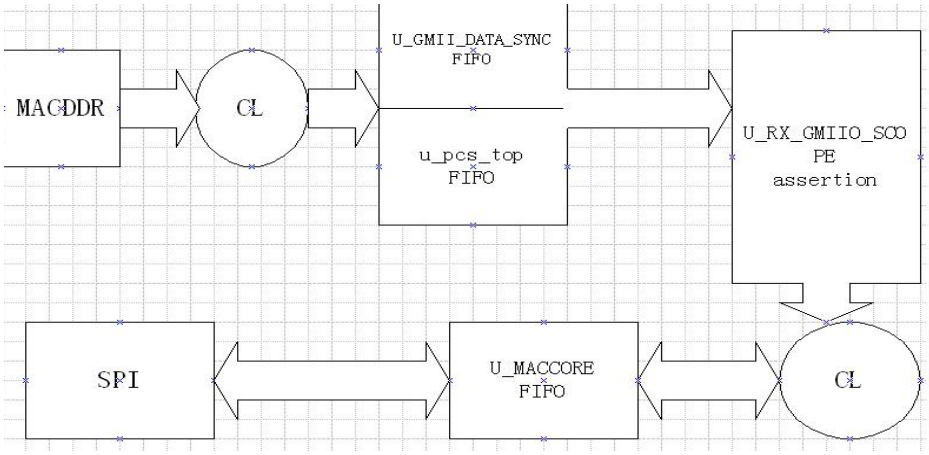


图3-78 MAC 模块的测试电路

8. 参考文献

[1] 杨小飞，反标 SDF 时序精度不够导致仿真错误的案例， 2007-02-05

[2] 林强，后仿真中时序信息反标不上问题， 200-11-23

[3] 林泗喜，在后仿真中如何对接口信号添加线延时， 2006-09-05

[4] 康毅，窄脉冲问题案例 2006-09-07

[5] 康毅，异步 FIFO 数据链路不定态的快速定位 2006-09-07

[6] 康毅，NC 仿真器对于 verilog force 功能支持不够好的典型案例 2006-09-07

[7] 康毅，LPH4001_LPC_E 寄存器的数据的变化在时钟的下降沿之前（hold violation）典型案例 2006-09-07

[8] 康毅，后仿真 RAM 数据线清零问题简析 2006-09-07

[9] 康毅，后仿真中 RAM 熔丝复位问题 2006-09-07

[10] 康毅，后仿真中 verilog force 功能与 NC force 功能的使用和比较 2006-09-07

[11] 康毅，后仿真中设定 time scale 问题 2006-09-07

[12] 康毅，解决异步电路不定态的典型案例 2006-09-07

[13] 袁丽霞，SD579 后仿真经验总结

[14] 徐善峰，SD599 后仿真策略， 2005-01-03

[17] 杨小飞，sdf 告警信息汇总， 2007-02-05

[18]林强，sdf 格式不满足要求，2007-02-05

[19]林强，关于使用-pulse 参数的问题， 2007-02-05

[20]杨小飞，反标 SDF 导致仿真错误的案例， 2007-02-05

[21] 程功宝，杨小飞，葛湘， SD8901 后仿真经验总结， 2007-02-05

[22] Michael, J, S, Smith, Application-Specific Intergrated Circuits, 1997

[23] SDF3.0 参考文档

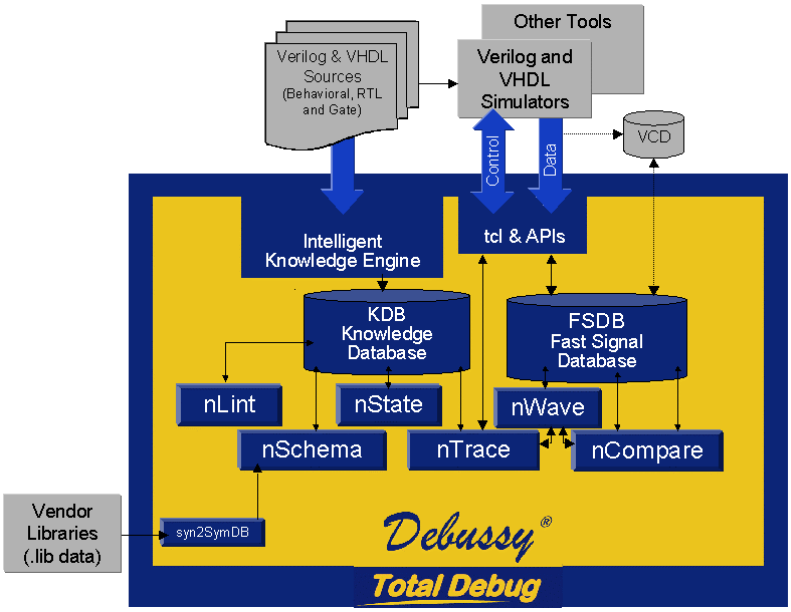
3.5.2 Debussy 在验证中的应用

1. Debussy 的介绍

Debussy 是一套很好的 Verilog\VHDL 调试工具，可以帮助设计者快速理解复杂的设计（设计小组中别人开发的复杂、不熟悉的代码或者 IP），查找和定位设计中存在的 bug。

Debussy 主要有以下几个模块：

- nTrace：超文本连接方式的源代码追踪及分析
- nSchema：原理图显示及分析
- nWave：波形显示及分析
- nState：有限状态机的显示及分析
- nCompare：分析仿真结果，比较其相异处。



2. Debussy 在验证各个阶段中的应用

RTL 模块级代码验证阶段：

- 直接保存 FSDB 仿真结果，FSDB 文件小，占用资源少，仿真速度快
- 从波形中发现问题可快速定位到对应代码。

系统级验证阶段：

- 建议先用 nLint 对顶层连线进行检查，排除基本问题如：输入/输出浮接，Bus 位数不匹配等。。。还可使用 Verdi 中电路图方式直观观察连接情况
- 将整个系统设计载入 Verdi，从中提取设计结构，在同一视窗中自由快速切换源代码，便于对整个设计的把握理解
- 提供电路图方式解读设计及状态机，便于理解不熟悉或继承的设计代码
- 源代码，波形，电路图相互间快速定位；Active trace 可以快速从波形中定位到源代码上并追踪源头

门级网表验证阶段：

- 使用电路图能够快速提取所关注的局部电路进行调试分析；
- 快速定位不定态问题；
- 能够提取复杂的时钟域和复位树，并可以图形方式显示，便于解读不熟悉的时钟树。

3. 如何把 NC 仿真结果保存成为 FSDB 格式文件

Debussy 提供的新的波形文件格式 FSDB。FSDB 文件相比于 VCD，SHM 格式，压缩量大，加载速度快,相同仿真时间的 FSDB 格式文件，可能只有 SHM 文件的 1/5。在 nWave 导入仿真波形文件后，我们可以用 Active Annotation 在源代码、原理图、状态图上动态显示变量的仿真值，跟踪信号的 driver 和 load，加快问题的定位。Debussy 提供了 PLI（for Verilog）和 FLI（for VHDL）接口，

我们可以在仿真时直接导出 FSDB 文件。如何把仿真的波形保存成为 FSDB 文件呢？需要进行下面 3 个地方的设置：

(1) chsrc 中的设置，目前海思这里使用 Debussy 的最新版本的是 Linux 平台下 64 位的 6.1v1 版本。

```
############################################################
# Set up enviroment for debussy
############################################################

source /home/guest/cshrc/novas/cshrc.debussy6.1v1
source /home/guest/cshrc/novas/cshrc.license
setenv LD_LIBRARY_PATH $NOVAS/share/PLI/nc_xl/LINUX/nc_loadpli1:$LD_LIBRARY_PATH
setenv LD_LIBRARY_PATH $SPECMAN_HOME/linux:$LD_LIBRARY_PATH
```

(2) 在 Testbench 中加入下面的代码：

```
`define FSDB_FILE_SAVE
module TB_TOP;
……
`ifdef FSDB_FILE_SAVE
initial begin
$fsdbDumpfile("loc2auais.shm");
 $fsdbDumpvars; //定义Dump波形的层次和信号变量
 $fsdbDumpflush;
end
`endif
```

在仿真脚本设置如下，注意蓝色的部分：

```
#!/bin/csh -f

setenv ENV_DIR  ~y47455/sd8901/verification/rx/filelist
setenv ENV_FILE tb_env
setenv DUT_DIR   ~y47455/sd8901/verification/rx/filelist
setenv DUT_FILELIST rx_filelist
setenv TC_DIR  ~y47455/sd8901/verification/rx/rx_tc
setenv LOG_DIR  ~y47455/sd8901/verification/rx/rx_log

specman -c "load ${ENV_DIR}/${ENV_FILE}.e;
save ./${ENV_FILE}.esv; write stubs -verilog specman"

ncprep +overwrite -f ${DUT_DIR}/${DUT_FILELIST}.f

ncvlog specman.v -f ncvlog.args -messages -updata -csdlib cds.lib
-hdlvar hdl.var -work worklib

ncelab TB_TOP specman specman_wave -access +rwc -messages -update
csdlib cds.lib -hdlvar hdl.var -timescale 1ns/100ps-loadpli1
libncvlog_sn_boot: sn_boot  -loadpli1 debpli: deb_PLIPtr

specview -p "res ${ENV_FILE}.esv; set log  ${LOG_DIR}/$1.elog;
load ${TC_DIR}/$1; test -seed =random " \
ncsim -i pre_cmd -LICQUENE -gui -NOLOG -NOSTDOUT -tcl
worklib.TB_TOP
```

Debussy 提供了一些系统任务来 Dump FSDB 波形文件，我们可以将这些系统任务加入到 tb 中来 Dump 想要得到的波形文件，下面对部分常用系统函数作简单解释：

$fsdbDumpfile： 定义 Dump 波形的文件名，语法：

$fsdbDumpfile(<"FSDB name">)；

$fsdbDumpvars：定义所要 Dump 波形的层次和信号变量，语法：

Dump 所有信号及变量波形： $fsdbDumpvars；

Dump 特定层次，特定子模块，特定信号的波形： $fsdbDumpvars (<level><,<module|var>>)；

$fsdbDumpon/$fsdbDumpoff

这两个系统函数控制 dump 的 on 和 off。这两个函数只对$fsdbDumpvars()起控制作用。比如：

```
initial
begin
  #50000;
  $fsdbDumpvars;
  #50000;
  $fsdbDumpoff;
  #50000;
  $fsdbDumpon;
  #50000;
  $finish;
end
```

上面的代码，保存 5000~10000 ，15000~20000 仿真时间段的数据，中间的 10000～15000 时间段的数据没有 dump。

$fsdbSwitchDumpfile

该系统函数支持仿真过程中生成多个 FSDB 格式的文件，当使用该函数后，前面打开的 FSDB 文件自动关闭，打开新的指定的 FSDB 文件，比如：

```
  $fsdbDumpfile(“test0.fsdb”);
  $fsdbDumpvars(0,system);
  #1000;
  $fsdbSwitchDumpfile(“test1.fsdb”);
  #1000;
  $fsdbSwitchDumpfile(“test2.fsdb”);
```

……

上面的方法，首先建立一个 test0.fsdb 文件保存波形，在仿真 1000 个时间单位后，关闭 test0.fsdb 文件，切换到 test1.fsdb，在经过 1000 个仿真时间单位后，关闭 test1.fsdb 文件，切换到 test2.fsdb。

4. 如何在波形窗口，源代码窗口中追溯信号的驱动源

在波形窗口中我们可以通过双击信号的波形来追溯信号的驱动源，从而定位错误出现的位置。在源代码窗口中可以通过双击信号的名字来追溯该信号的驱动源。在源代码窗口中也可以通过右键菜单来实现这个功能。在窗口中选择 Trace->ResetTraced Signals’s Color 可以取消追溯过程中带来的信号高亮度的问题。一般情况下我们是从信号的波形图中某一时刻的波形异常处双击该波形，追溯到源代码窗口中的对应赋值语句，从而找到该时刻的驱动源。

5. 如何让波形，源代码和原理图三个窗口同步显示

我们可以通过选择 Tool->Options->Sync Signal Selection，来同步所有窗口中的信号。一旦你在 nWave 或者源代码窗口或者原理图窗口选择了任何一个信号，其他两个窗口也会同步显示该信号。

6. 如何反标信号数值到源代码和原理图窗口中？

在很多调试情况下，在源代码窗口显示信号或者总线的数值也有一定的好处。nTrace 提供了这样一个功能，Source->Active Annotation。信号值显示 Cursor 当前时刻的数值。在原理图窗口中也可以反标，Schematic->Active Annotation。

3.5.3 提高验证效率的技巧

1. 后仿真加断点：

在项目的后仿真阶段，仿真时间比较长，如果仿真进程被异常中断，向量需要重新仿真，导致结果不能及时确认，影响 Tapeout 时间。

利用 e 与 nc-verilog 工具的断点存储功能，设置仿真断点，在仿真异常中断后自动保存了中断时刻信息，重新仿真时可以从中断点开始仿真，而不需要从起始点开始，节省大量时间。

在 SD85XX/89XX 中得到应用，避免了因为系统异常中断和不可预知原因导致中断而需要重新仿真的问题，节省了超过一周的时间。仿真设置断点，相当于给后仿真加了一把安全锁，在项目后期时间紧张的情况下不会在白白浪费不必要的时间。

2. RAM 验证的自动化平台

现在的芯片设计中需要大量的使用 RAM，对于第三方提供的 RAM，设计的时候通常需要进行封装，使之接口时序关系适应设计的需要，方便芯片逻辑设计，但是随着芯片的规模越来越大，使用的 RAM 个数和种类也急剧增大，这时，RAM 封装和验证的工作量增大。

编写自动化脚本：编写自动化 RAM 封装脚本，接口时序特性编写自动化脚本对 RAM 进行批量封装；编写自动化 RAM 测试脚本，按照封装后的 RAM 接口时序特性对 RAM 进行批量测试。

SD89XX 项目使用 RAM 自动化封装脚本进行 RAM 封装，批量完成了大量 RAM 的封装工作。使用自动化测试脚本进行 RAM 的专项测试，快速、高效的完成了 RAM 的测试，并发现两个致命问题。

3. E 验证环境的效率分析工具

Specman 自带的性能分析工具－Profile，统计分析验证环境中各部分所消耗的 CPU 和 Memory 资源情况，根据分析结果，指导我们对验证环境进行优化，提高仿真效率。尤其在回归测试时候，经过优化后的环境的工作效率大大高于原环境，节约的仿真时间相当可观。

4. 脚本工具的使用

在验证过程中，部门积累了一些优秀的脚本，这些脚本专门解决某个特定的问题，通过使用脚本可以减少重复劳动，大大提高工作效率。举例如下：

后仿真 SDF 告警提取脚本：后仿真 SDF 反标过程中，会生成大量的告警信息，需要验证人员逐个确认，工作量太大。编写了一个 Perl 脚本，自动对反标的告警信息进行提取，归类。

后仿真从网表提取特定模块信号的脚本：可以根据输入的模块名字，从网表中提取出该模块中所有的寄存器信号，并且在 ncsim 中把这些信号加为 probe。避免了手工逐个在网表中查找信号，手到设置 probe 的繁琐过程，减少了笔误的可能性，大大提高后仿真的效率。

3.5.4 在 VMM 中如何不让 TC 里设定的低打印等级的 components 打印 log？

开发 TC 时经常需要在 TC 中指定某个验证组件的打印等级，打印相关的信息来达到 debug 的目的，而在回归时，我们不需要 TC 打印大量的 debug 信息，当有很多条 TC 需要回归时，修改每条 TC 中的打印等级是很烦琐的事。

可以通过在 run option 中增加 command line 来重新设置所有验证组件的打印等级。

```
./simv +vmm_force_verbosity=NOTE
```

3.5.5 cond coverage 有些条件没有统计？

A：&&和||是逻辑操作符，&和|是按位操作符；

Synopsys 的 CoverMeter 中 condition coverage 默认只检测逻辑操作符描述的条件。

因为 Condition Coverage 默认下只检测“&&”和“||”所构成的条件，因此在 report 中是默认没有“&”所构成的条件的，如果你需要让 VCS 收集相应的覆盖率，你可以在编译时添加“-cm_cond allops”,或者是用”&&”替换”&”。

第4章 ASIC 后端知识

后端流程知识主要以介绍后端工具的使用方法为主，详细程度达到帮助读者入门，可以初步开始使用的程度。

4.1 ASIC 后端基本概念

在 ASIC 设计过程中通常把需求分析、系统设计、模块设计编码、RTL 仿真等过程称为前端设计；这些过程和 Vendor 无关，而把和 Vendor 相关的的综合功能验证、时序验证、Floorplan、电气检查、Layout 等称为后端设计。这里说的 Vendor 是指提供芯片后端设计服务的 IC 厂家，这些厂家包括 IBM、TI、LSI、ST、Faraday 等

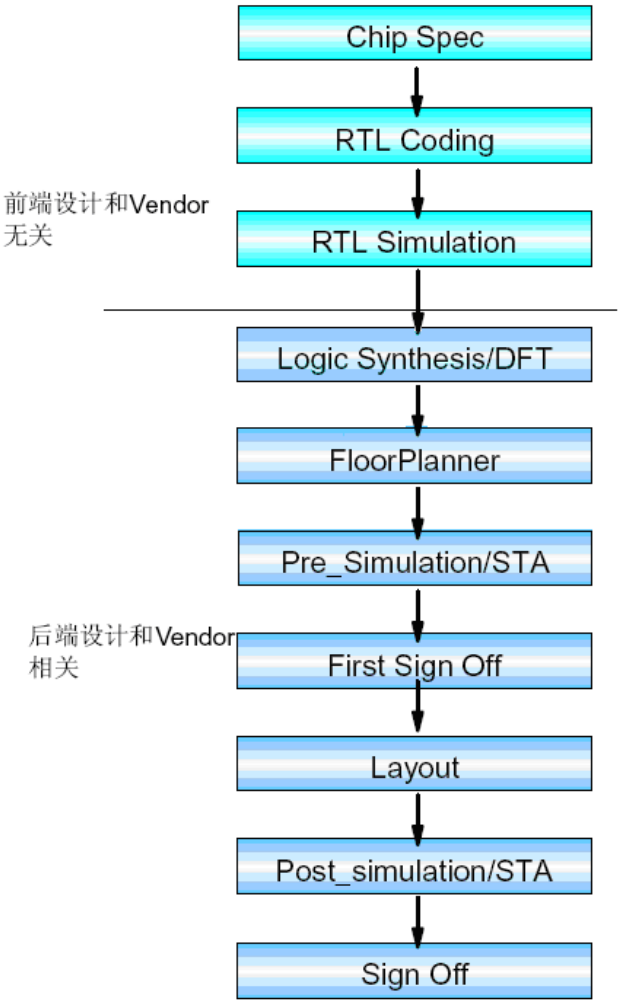


图4-1 设计流程简图

不管是走哪一个 Vendor 的流程，后端设计流程中一般有四个比较关键的点，就是 85%网表阶段、95%网表阶段、100%网表阶段以及 Signoff：

4.1.2 85%网表

本阶段 RTL 代码已经基本完成了，并且已经进行了部分代码的验证，验证还不是很充分。

我们选好 Vendor 后，需要在 RTL 中例化设计中用到的 IP，这些 IP 包括 PLL、RA/SRAM、HSS、SPI4 等，如何正确例化 Vendor 的 IP 是本阶段项目后端设计的最重要工作。由于 Vendor IP 的一些特殊性，往往会对时钟结构产生影响。例化好 Vendor 提供的 IP 后，综合出 85%网表并提交给 Vendor。

Vendor 拿到 85%网表后会对该网表进行一系列的分析，比如网表是否符合后端处理的要求、Die size 是否足够大、布线有没问题、时序是否能跑上去等等，最终是要得到一个能够进行 Floorplan，没有违反后端设计规则的网表。

从以往的经验来看，本阶段将会发现设计中存在的许多问题，我们需要按照 Vendor 的要求进行修改，修改完后最好重新提交网表给 Vendor 确认。

4.1.3 95%网表

本阶段的 RTL 代码应该是比较稳定并进行了充分验证的，在此阶段我们可能还会用 FPGA 对 RTL 代码进行验证，以进一步确保 RTL 的正确/可靠性，在这过程中可能还会有局部的代码需要进行改动，但对整个设计影响应该不大。

而 Vendor 这时候可以拿到一个比较稳定的网表，进行 Floorplan，布局，布时钟树，对时序进行收敛，对功耗进行估计等等。一般来说，本阶段后端的注意力往往集中在时序收敛上，另外管脚排布、封装设计在这个阶段基本上就定下来了，芯片的功耗也可根据设计得出。本阶段 Vendor 的目标是尽量解决后端设计中遇到/将要遇到的所有问题。

总的来说这个阶段比较关键，首先我们在设计/验证上应把所有问题收敛掉，确保网表的稳定性，另外后端需要收敛时序，确保下一步工作畅通无阻

4.1.4 100%网表

本阶段的 RTL 代码完全固定下来了，一般不再允许更改。我们需要把最终的时序约束以及网表提交给 Vendor。

在 Vendor 对网表进行后端处理的同时，我们将对整个设计进行一系列的 review，如果 review 发现问题，则需要进行 ECO，如果 ECO 难道很大，则需要重新提交 100%网表，这将对整个后端进度产生影响，不过问题发现得越早，对后端进度的影响也越小。

这个阶段 Vendor 的工作除了要完成 95%网表阶段的工作外，还进行真正的 Route，并从网表中提取延时参数信息分析时序，得到准确的时序分析结果。

4.1.5 Signoff

Vendor 给我们返回最终完成后端设计的网表，我们对该网表进行后仿真，时序验证，形式验证后如果达到要求，并通过评审，就可以与 Vendor 签署相关文件，对芯片的样片生产进行许可，即所谓的 signoff。一般 Signoff 两个月后样片即可返回。

4.2 ASIC Vendor 后端流程概述

在 RTL 仿真完成后我们需要选择 Sign Off Vendor，确定芯片的制造工艺，进行综合，得到正确的电路(通常所说的网表 Nestlist)。 无论哪一个 Vendor ，其后端流程都是以正确的网表作为其设计流程的输入，借助其特有的设计流程和一系列工具的支持，完成芯片 Floorplan、延时估算、功能验证、时序验证、设计规则检查、测试向量的产生、Layout、后功能和时序的验证等工作。经过可能的反复并修正所有错误后，最后产生合格的网表、测试向量、Pin Assignment 等数据，完成 Sign Off 工作，进行芯片的制造。

4.2.1 IBM 后端流程简介

1. IBM 后端流程概述

鉴于 IBM 后端流程的特殊性、集成的开发工具，以及 SERDES 和 RAM 的灵活性，IBM 的后端流程相对于其它 VENDOR 而言，能够明显的缩短项目开发周期，提高项目整体开发质量。图 4-2 为 IBM 整个后端流程的流程图。

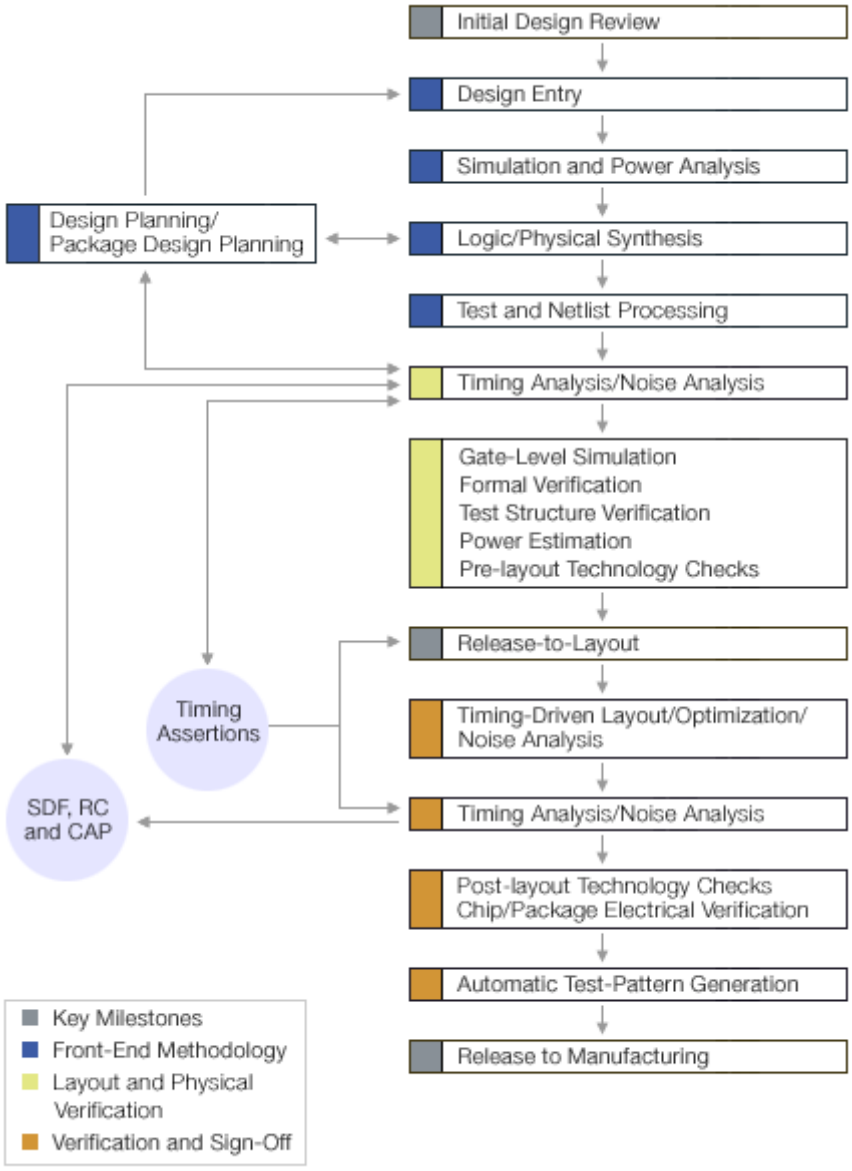


图4-2 IBM 后端流程图

以下各部分分别对 IBM 后端流程的各个环节进行详细描述：

2. IBM 专有名词

- IDR：Initial Design Review
- RTF：Releas To Floorplan，对应于我们通常说的 85%网表阶段
- RTP：Releas To Preliminary，对应于我们通常说的 95%网表阶段
- RTL：Releas To Layout，对应于我们通常说的 100%网表阶段
- RTC：Releas To Check，这个阶段指 IBM 完成了所有的后端工作，返回网表及其它文件给我们进行时序分析和后仿真
- RTM：Releas To Manufacturing，对应于我们通常说的 sigoff/tape out。

3. IBM 后端流程之 IDR（Initial Design Review）阶段

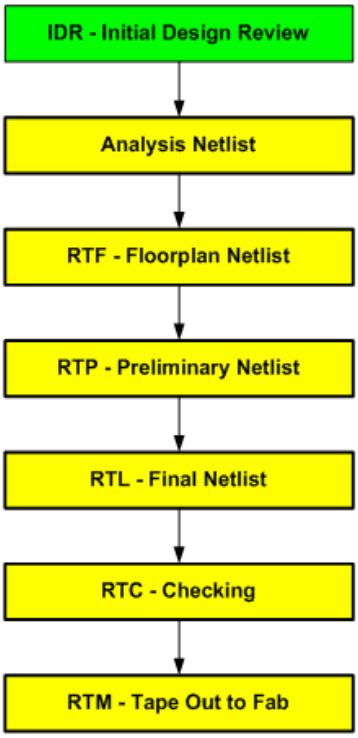


图4-3 IBM 后端流程 Milestones－IDR 阶段

在 RTM 之前要进行三次设计 Review。Initial Design Review（IDR）一般在 IBM 收到 NRE order 后的两周后进行。第二次设计 Review 在 Release to Layout（RTL）之前进行，第三次 Review 则在 RTM 之前进行。IDR 会议作为项目后端流程正式启动的开工会，主要完成以下目标：

- 填写 CDS(Customer Design Spec)文件，CDS 文件内容包括时钟、IO、RAM、PLL、SERDES CORE 等具体内容；
- 同 VENDOR 明确双方（海思和 IBM）各自的开发任务和角色分工；
- 明确芯片采用的工艺、Die Size、封装以及芯片的性能指标；
- 确定芯片的工作环境；
- 简要概括项目的大致情况，并确定整个项目后端的开发周期和开发进度。
- 确定芯片的性能指标，检查 FloorPlanning 的有关问题；

该阶段主要围绕 IDR 和需求分析文档 Requirement Analysis Document(RAD)进行，各个方面的设计人员如 Design Center Designer、Customer 及 Manufacturing Designer 均会参加，以确定 ASIC 的要求，同时讨论决定哪些设计步骤可以省略，哪些需要保留。

4. IBM 后端流程之 Design Entry 阶段（准备 85%网表）

在 IDR 会议结束后，进入 Design Entry 阶段。本阶段持续一周左右，IBM 需要完成的目标如下：

- 定义输入输出的时序规格详细说明；

- 进行初步的 FloorPlan；
- 描述项目的 IO 接口和封装类型；

5. IBM 后端流程之 RTF （85%网表）阶段

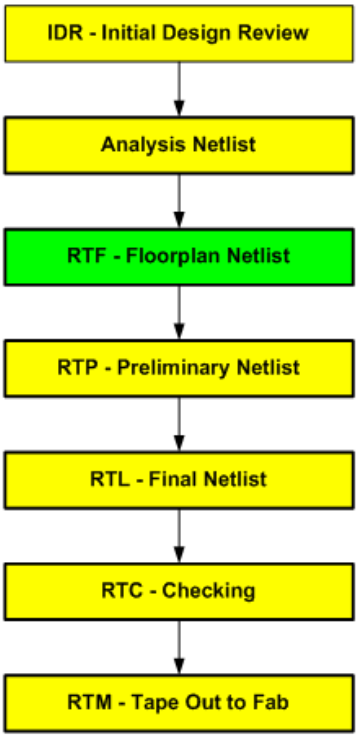


图4-4 IBM 后端流程之 RTF 阶段

本阶段我们需要准备 85%的网表，持续的时间为 1.5 个月左右。

目标：使网表达到 Floorplan Netlist 的要求。

主要任务如下：

- 完成相应的 RA，SSRAM，EDRAM、PLL 和 SERDES CORE 的替换。（也可在 Analysis Netlist 阶段完成），这个工作由海思项目组负责。
- 完成初步的芯片管脚排布。这个工作主要由海思的 SI 工程师进行，分两步进行。

  第一步，项目组向 SI 工程师提供芯片的用户手册，芯片的使用环境（PCB 原理图），以及外部芯片的资料（主要是考虑芯片的驱动能力），供 SI 工程师进行 PAD 选型。同时还要给出我们希望的管脚排布的方位，SI 工程师会尽快给出一个初步的管脚排布结果，这个结果要求芯片四边的输出管脚数目相对平衡（主要是为了满足噪声要求，利用 IBM 提供的 IOSpecify 可以进行检查）。

  第二步，SI 工程师进行具体的计算，以最终确定 PAD 类型，以及管脚位置。
- 进行整个封装的噪声分析。

这个工作由 IBM 的美国公司负责，我们需向 IBM 提供所有管脚的输出翻转情况，主要是有哪些 output 有可能会同时翻转。如果噪声分析有问题的话，可能采取的措施有：

- 调整管脚排布的次序，在芯片的每一边让输入输出管脚均匀分布，不要让很多输出挤在一起。
- 在噪声比较差的部分插入电源，地。
- 实在没有办法，只好换封装了。
- 海思向 IBM 提供更新后的网表。

该网表同样不要求功能的准确性，但是要求完成所有特殊模块的替换。该网表要求同初步的管脚排布一同提供，这样 IBM 可以开始一些前期工作。

- 海思给 IBM 提供完整的电路 Paper Floor Plan。

即画出我们期望的各个模块在芯片中的位置，以及各个模块之间的数据流向，以便 IBM 做 Floor Plan 时参考。越详细越好。

- 完成 IDR 表。

该表由双方共同完成，其中需要我们完成的部分大部分内容同报价表是一样的。

- 海思提交 Initial Timing Assertions

将更新后的 Floorplan Netlist 和 Initial Timing Assertions STA 脚本一起提交给 IBM。这阶段的 STA 脚本中的时钟，复位以及 RA/SRAM 的模式需要正确设置，IBM 会重点评估芯片内部逻辑的时序。STA 分析是用 IBM 的专用工具 Einstimer。

(2) IBM 在 RTF 阶段的网表分析工作

该阶段主要由 IBM 来完成，主要完成以下目标：

- 对 Customer 方提供的网表进行调试；
- 对网表进行早期的可测性、floorplan、layout 分析。

该阶段包括以下过程：

- IBM 根据提供的网表作 trade off 分析；
- IBM 根据 floorplan 分析的结果，向 Customer 返回 floorplan 修改建议；
- IBM 返回噪声分析结果。

(3) 本阶段需要做的优化工作

该阶段要完成以下目标：

- 对芯片的性能、功耗、面积和布局进行优化；
- 设计采用边沿触发寄存器。

6. IBM 后端流程之 RTP （95% 网表）阶段

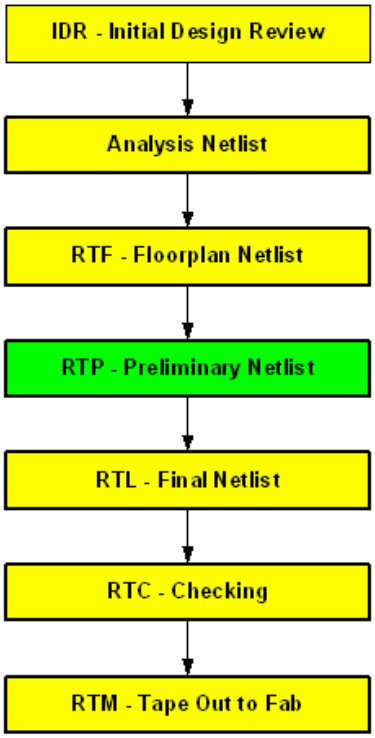


图4-5 IBM 后端流程之 RTP 阶段

本阶段我们的主要任务是要完成芯片的RTL级仿真，完成STA的Assertion file，并且提交 Priliminary Netlist。整个阶段持续 1.5 个月左右，由芯片的规模和复杂程度决定。

目标：网表达到 Priliminary Netlist 的要求。

(2) DFT

由于 IBM 的库使用的是 LSSD，相对其他厂家来说，做 DFT 非常简单，当芯片中有 EDRAM，PLL 等 CORE 时，会稍微麻烦一些，但是由于 IBM 有针对不同 CORE 的 DFT 的模版，因此做起来还是相当快的。DFT 流程一般由 IBM 完成，IBM DFT 流程如下图所示。

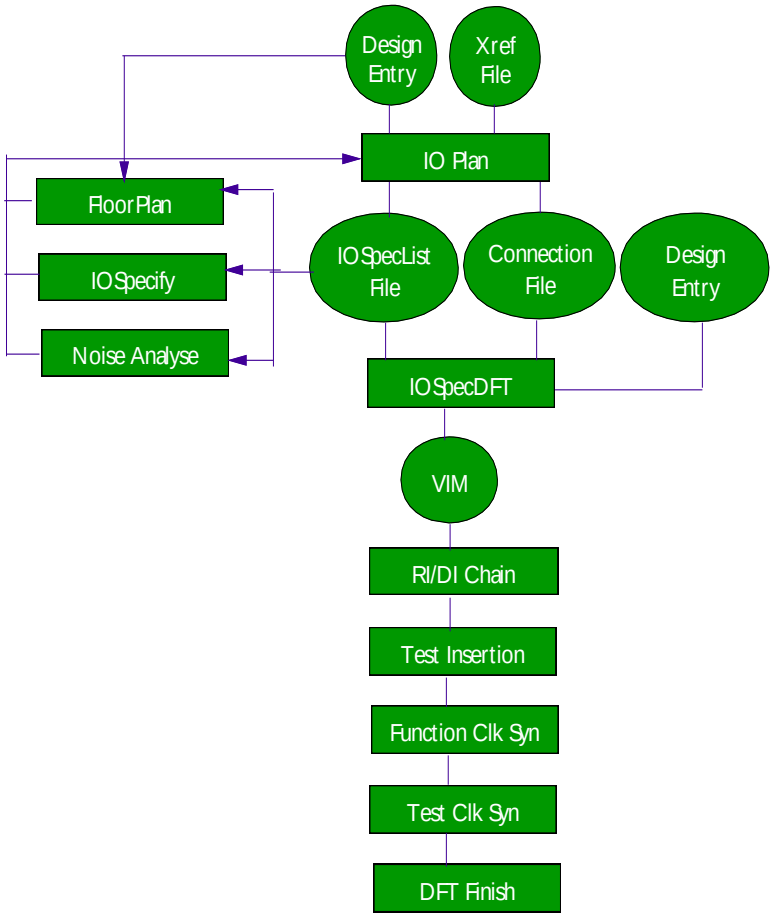


图4-6 IBM DFT 流程图

DFT 过程一般不需要我们干预。在 DFT 过程中，IBM 会返回一个经过 DFT 后的网表，由我们来进行后仿，以确保 DFT 的正确性。

(3) FloorPlan

按照 IBM 的说法，FloorPlan 将解决 80%以上的 timing 问题，因此这个阶段会有多次反复，比如说布局的调整，管脚位置的调整，综合约束的调整。FloorPlan 之后，IBM 会返回一个网表，我们可以利用 Formality（一致性检查），STA（时序检查）对其进行检查，如果没有问题，就可以提交 Priliminary Netlist。

7. IBM 后端流程之 RTL （100%网表）阶段

这个阶段，IBM 主要任务是利用 Priliminary Netlist 进行正式的 DFT 和 FloorPlan。我们的主要任务是利用各种手段检查 IBM 返回的网表，包括形式验证，STA 等。本阶段持续 1.5 月左右。

目标：使网表达到 Final Netlist 的要求。

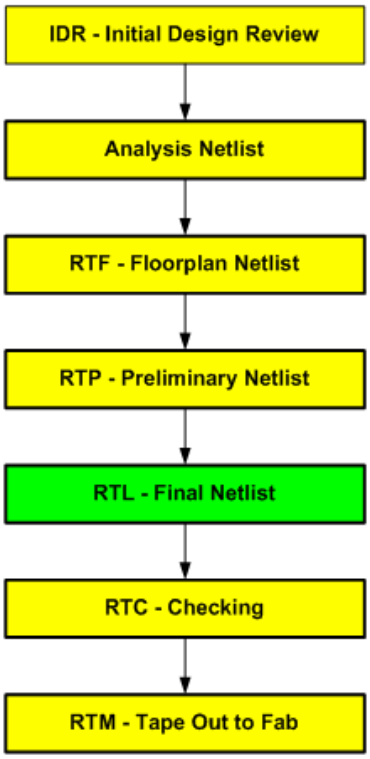


图4-7 IBM 后端流程 Milestones－RTL 阶段

由于 IBM FloorPlan 之后返回的网表的时延信息并不是非常准确，而且这个阶段并没有解决所有的 RC cap 问题（因为这个网表并不是最终的网表，因此解决所有的 rc cap 问题没有实际意义），所以对于 FloorPlan 之后的网表是否需要进行后仿真可根据各项目实际情况来决定，有些项目时序收敛得快一些，有些又比较慢；一般来说，后仿真启动得越早越好；

如果没有发现问题的话，就可以进行 RTL，也就是提交所谓的 Final Netlist 了，同时，我们也要提供最终的 STA 的 assertion file。

8. IBM 后端流程之 RTC & RTM （sigoff）阶段

在进入这个阶段之前，IBM 会利用我们的网表以及 Assertion file 进行最终时序收敛的工作(仍然处于 FloorPlan 的阶段)。在时序收敛之后，就可以开始 Routing 了。Routing 结束以后，进入 RTC。RTC 是三次设计 Review 中的最后一次，也是最重要的一个阶段点，IBM 将会对他们自己所有的后端流程进行检查后，将返回一个最终的网表给我们。本阶段时间为 0.5 个月左右

目标：使网表达到 sigoff 的要求，并使所有的检查通过，最终 sigoff。

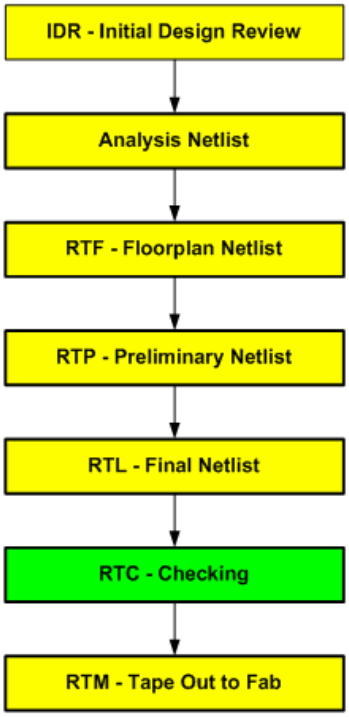


图4-8 IBM 后端流程 Milestones－RTC 阶段

RTC 阶段 IBM 主要工作是对整个芯片整行 REVIEW，需要澄清所有的问题：

- 本阶段的所有文档/checklist 必须确认
- 物理版图/设计规则/时序/check
- 产生 GDS-II 文件
- 返回网表、参数文件及 IBM 自己进行检查的结果给海思

海思收到 IBM 的最终网表及参数文件后，需要做以下工作，对网表进行验证：

- 启动正式的后仿真，并通过后仿真验证
- 进行 STA，对 STA 报告进行 Review，需要通过 STA checkklist
- 进行 Formaltiy，对 Formality 的报告进行 Review，需要通过 Formality checklist
- 对 IBM 返回的 check 结果，进行 Review

海思对网表各项检查都通过后，就可以正式 sigoff 了，也就是 RTM。IBM 在 RTM 两个月后返回样片。

4.2.2 LSI 后端流程简介

LSI 的后端流程如图 4-9 所示。

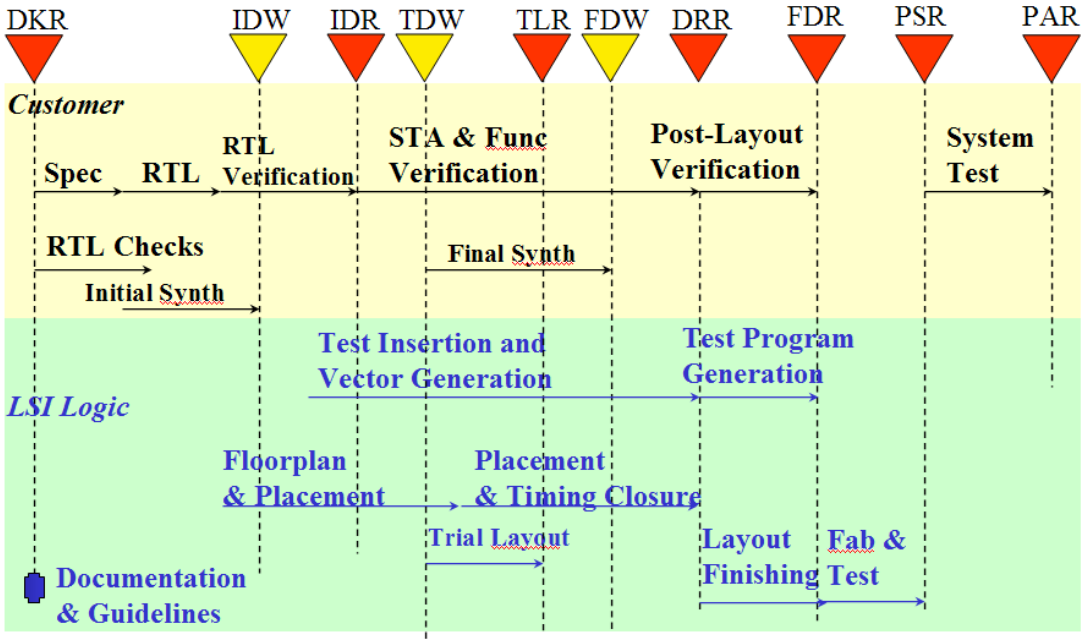


图4-9 LSI 后端流程图

LSI 的后端流程从后端开工会到最终的芯片量产，将经过 7 次 Review，LSI 把这 7 次 Review 称之为“7 setp signoff”

DKR：Design Kickoff Review，为正式的后端开工会，我们要与 LSI 把整个设计 review 一遍，并详细讨论接下来的工作与计划。LSI 将介绍他们的后端流程，以及 LSI 后端工具的培训等等

IDR：Initial Design Review，对 85%网表的后端处理结果进行 Review，以及讨论下一阶段工作的目标

TLR：Trial Layout Review，对 Trial layout 也就是 95%网表的后端处理结果进行 Review，并制定下一阶段工作的目标。

DRR：Detail Route Review，这个 Review 是在初步 detail route 后进行，由于时序分析可以用布线后的延迟信息，时序分析结果也比较准确，所以这个 Reivew 将用布线后的时序结果来评估下一步最终布线的策略，并由此制定一个计划。

FDR：Final Design Review，相当于 Signoff 之前的 Review，这时整个芯片的时序已经收敛，网表达到 Signoff 的要求

PSR：Proto Ship Review，样片生产完成，通过了生产的测试，并且把样片发给客户

PAR：Proto Approval Release，客户对样片进行测试并达到量产的要求，LSI 收到客户量产的许可，对芯片进行量产。

4.2.3 TI 后端流程简介

TI 的后端流程如下图所示，也称之为 Engagement Model。

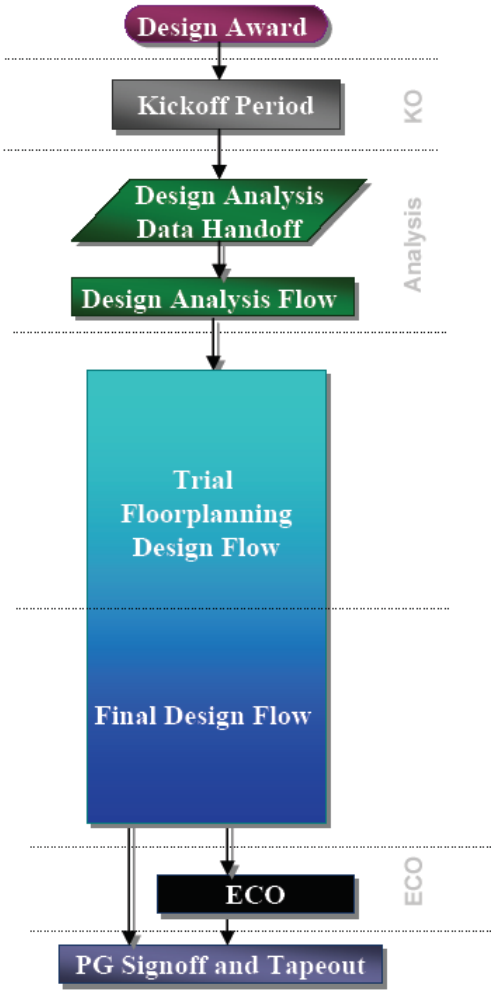


图4-10 TI 后端流程图

主要分为五个阶段：

2. TI 后端流程之 Kickoff Meeting

TI 的后端开工会，与 TI 工程师对整个设计的后端规格做一次 Review，制定后端设计策略以及计划，后端分工以及各自的职责。进一步讨论下一阶段的后端工作，一般关注 IP core 的用法、时钟结构、复位结构、Floorplan 等，我们对 TI

3. TI 后端流程之 Preliminary Handoff

这个阶段的需给 TI 提交综合后的 85%网表，以及 STA 约束。TI 将开展 Preliminary Stage 工作；对网表进行检查，对 I/O 进行初步摆放，对 RAM 以及较大的模块进行初步摆放，对网表进行 Floorplan 后进行初步的时序分析。本阶段 TI 刚开始上手熟悉项目，双方的交流也比较多

4. TI 后端流程之 Trial Handoff

这个阶段的需给 TI 提交综合后的 85%网表，以及全面的 STA 约束。TI 将在完成 Preliminary Stage 工作后还将继续做 Place&Route，并进行时序收敛，此阶段 TI 基本完成 Physical Design 工作

5. TI 后端流程之 Final Handoff

这个阶段的需给 TI 提交综合后的 100%网表，以及最终的 STA 约束。TI 将展开的工作有基于 Trial Stage 的 Floorplan 结果完成 Place&Rout 和最终的时序收敛工作，完成 Tape out 需要的所有数据准备工作。如果在此阶段发现芯片有 bug，刚需要做 ECO，ECO 将会对整个进度产生影响，影响视 ECO 的复杂程度而定。

6. TI 后端流程之 PG Handoff and Tapout

从 Final Handoff 到 PG 前的工作主要由 TI 工程师完成，属于 Final Stage 的工作，经过双方的检查确认后即可 PG 进行 Tapout

4.2.4 Faraday 后端流程简介

Faraday 后端流程如图 4-11 所示。

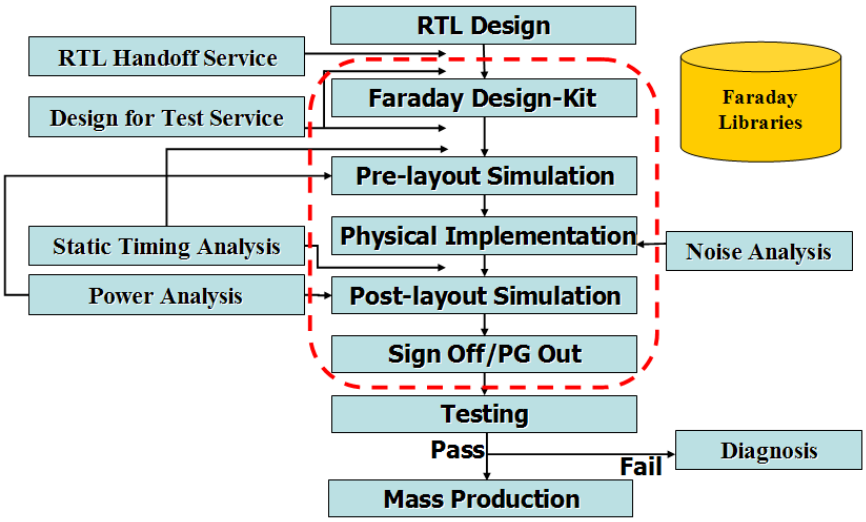


图4-11 Faraday 后端流程图

4.2.5 ST 后端流程简介

ST 后端流程如图 4-12 所示。

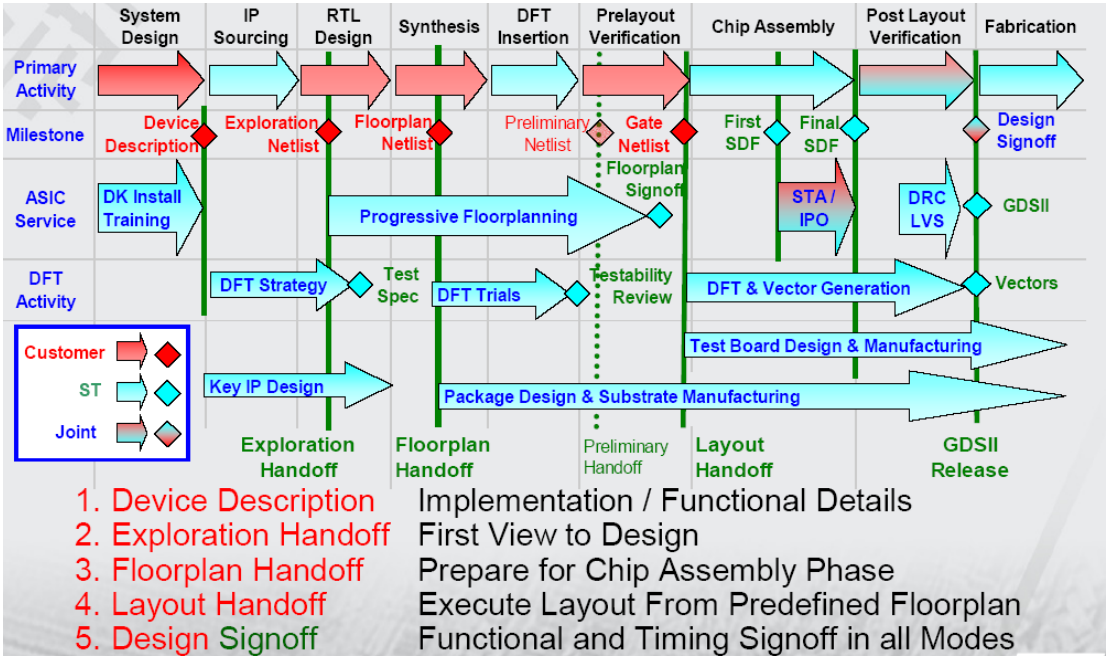


图4-12 ST 后端流程图

ST 的后端流程从客户角度来看，有以下几个里程碑：Device Description，Exploration Netlist，Floorplan Netlist，Preliminary Netlist 与 IBM/LSI/TI 流程类似这里就不介绍了

4.2.6 海思后端（COT）流程介绍

abbreviations缩略语	Description 描述
QA	Quality Assurance Engineer 质量保证工程师
PM	Project Manager 项目经理
BPM	Backend Project Manager 后端项目经理
SE	System Engineer 系统工程师
FE	Frontend Engineer 前端工程师
BES	Backend Support Engineer 后端支撑工程师（综合 STA）
DFTE	Design For Test Engineer DFT 工程师
PDE	Physical Design Engineer 物理设计工程师
PAE	Package Engineer 封装工程师
SIE	SI 工程师

PE	Process Engineer 工艺工程师

1. inputs 输入

技术开发任务书，项目设计数据包、项目设计库包、相关技术文档、过程规范、checklist。（详见《海思 COT 项目前后端交付件表单》

2. outputs 输出

完整的可制造的物理设计数据、完整的可供测试机台使用的测试向量，项目后端设计过程文档。（详见《海思 COT 项目前后端交付件表单》）

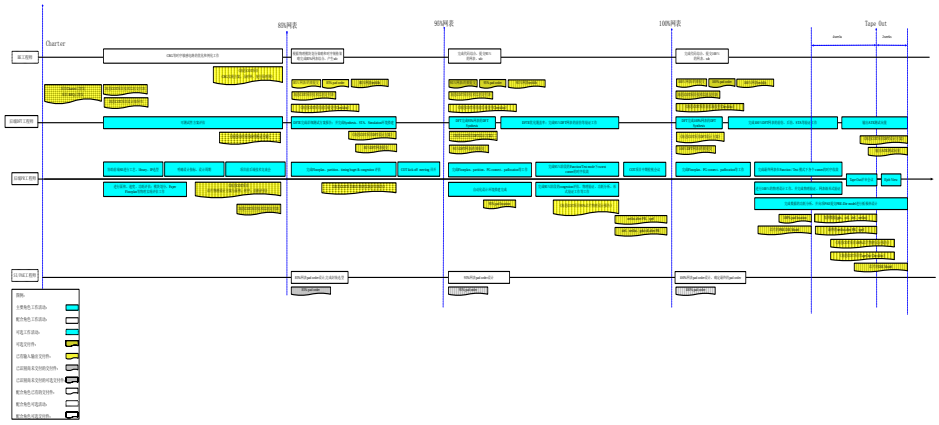
3. 控制机制

后端项目经理和项目 QA 保证遵循本流程；

后端 ST 负责批准所有针对流程的修改，后端设计部部长、质量部部长负责签发更改后的流程；

进行周期性交付件审计，基线审计和内部质量审计来确保符合流程；

4. 开发规程



详见《开发流程总图》

- COT需求分析阶段（Charter到85%）

阶段定义：

COT 需求分析阶段的时间段为项目立项到 85%网表提交。

任务定义：

完成 COT 项目的需求分析和前期评估工作，包括项目的可实现性、可测性评估。

BPM 协助 SE 完成面积、性能、频率评估。

DFTE 评估 IP 的可测性，并确定 IP 的可测性方案；规划芯片的生产测试方案。

PDE 确定物理可实现性方案

- COT规格制定阶段（85%阶段）

阶段定义：

COT 预设计阶段的时间段为：85%网表提交到 COT Kick-off meeting 召开

任务定义：

前端根据需求分析阶段“前后端技术交流会”达成的可测性策略、模块划分策略、综合策略、提交第一版完整的网表，及设计数据；并完成相关质检。

后端按照需求分析阶段制定的方案进行预设计，包括设计环境的搭建和各项设计指标的详细确认；为 Kick-off meeting 提供详尽的数据支撑。

完成 DFT 设计实现、STA、基本仿真环境搭建，为 95%网表的 DFT 设计快速交付进行充分准备。

PDE 完成 85%阶段的物理设计工作，包括 Floorplan、congestion、timing budget 等。

前后端团队一起熟悉彼此交付模式，前端协助后端熟悉项目设计数据与时序、功耗约束，后端协助前端熟悉 COT 的检查、交付流程。

COT Kick-off meeting 顺利召开，双方就 COT 设计指标、开发进度达成一致。

- COT预设计阶段（95%阶段）

阶段定义：

COT 预设计阶段的时间段为：Kick-meeting 召开后 95%网表提交到后端完成该阶段的 COT 设计要求的全部活动，并返回时序已收敛的网表。

任务定义：

前端根据“Kick-off meeting”达成的规格要求、策略进行逻辑优化、完成综合并提交完成自检后的 95%网表和相关数据。

SIE 根据“Kick-off meeting”达成的规格要求，输出 95%阶段的 IO file/Bond def。

DFTE 完成 DFT 设计，并通过相关的形式、时序验证和前仿真，输出 DFT 后的网表和 DFT Mode 下的时序/功耗约束文件(sdc)。

PDE 完成 95%阶段的物理设计工作，并进行相关的物理、功耗、时序、形式验证，输出 cworst 条件下时序收敛的网表。

预设计阶段的目的在于通过 COT 全流程的初步设计，自动化设计环境搭建完成，确认芯片最终规格的可实现性。

- COT交付设计阶段（100%阶段）

阶段定义：

COT 交付设计阶段的时间段为“COT 中期评审会议”召开后 100%网表提交到后端完成项目交付，并通过 TR4 评审。

任务定义：

BES 前端根据“COT 中期评审会议”达成的修改意见、综合策略完成综合并提交完成自检通过后的 100％网表和相关数据。

SIE 输出 100％阶段的 IO file/Bond def。

DFTE 完成 DFT 设计，并通过相关的形式、时序验证和后仿真，输出 DFT 后的网表、DFT 各 Mode 下的时序/功耗约束文件(sdc)及后仿通过的 ATE 测试向量。

PDE 完成 100％阶段的物理设计工作，并进行相关的物理、功耗、时序、形式验证，输出各种模式、Corner 下时序收敛的网表及通过相关验证的图形数据。

交付设计（100％）阶段后端的任务在于完成 DFT、物理设计为芯片制造、测试提交零缺陷数据。

- Tape Out后交付

阶段定义：

COT Tape Out 后交付，主要是完成相关 Tape Out 后数据检视、Metal EC，数据归档、资源释放工作；以及在 Tape Out 前无法完成可进入后期迭代开发模式的周边数据交付工作。

任务定义：

Tape Out 后 PDE 需向 PAE 提供最终的 pad location file。

Tape Out 2 周内，后端 PM 签发最终版《海思 COT 项目后端 Tape Out Checklist》

Tape Out 2 周内，PDE 确认完成最终版 Metal EC 后，完成时序验证、物理验证（drc、lvs、lvl、ant、dfm）后的 metal 层 gds 数据交付。

数据完整 Tape Out 后 PDE 需向项目组交付最终的 backup 数据包。

Tape Out 2 周后 PDE 提供最终的 die model。

Tape Out 2 周后 DFTE 提供 ATE 向量。

4.2.7 各厂家后端对比

1. 各阶段对比

	80%网表	95%网表	100%网表	signoff/tape-out
IBM	RTF	RTP	RTL/RTC	RTM
TI	Preliminary Handoff	Trial Handoff	Final Handoff	PG Handoff
LSI	IDR	TLR	DRR	FDR->Signoff
ST	Exploration Netlist	Floorplan Netlist	Preliminary Netlist	Signoff

2. 后端设计能力对比

	IBM	TI	LSI	ST	Faraday
Technology	Very Good	Good	?	?	?
Management	Very good	Good	?	?	?
DFT	LSSD	DFF	DFF	DFF	DFF
Design kit	TheGuide	Pyramid	LSI toolkit	?	FTC
STA signoff	Einstimer	PrimeTime	PrimeTime	PrimeTime	PrimeTime

4.3 逻辑综合 Logic Synthesis

概述：重点为基本概念、实现流程、脚本编写，注重实用性和可参考性

4.3.1 逻辑综合基本概念

1. 综合的定义

综合（Synthesis）简单地说就是将 HDL 代码转化为门级网表的过程。对电路的综合包括三个步骤，表示如下：**Synthesis = Translation + Mapping + Optimization。**

- **Translation**：是指把 HDL 语言描述的电路转化为用 GTECH 库元件组成的逻辑电路的过程。GTECH 是 Synopsys 的通用工艺库，它仅表示了逻辑函数的功能，并没有映射到具体的厂家工艺库，也就是说是独立于厂家工艺的。对于 HDL 描述输入的设计，这个步骤一般由 HDL compiler 完成。
- **Mapping**：把用 GTECH 库元件构成的电路映射到某一特定厂家的工艺库上，此时的电路包含了厂家的工艺参数 Library cells。
- **Optimization**：是根据设计者对电路设定的延时和面积等约束条件对电路进行优化的过程。它通过各种方法尽量满足设计者对电路的要求。

常用的 ASIC 综合工具是 Synopsys 的 Design Compiler，简称 DC。其输入输出如下图所示：

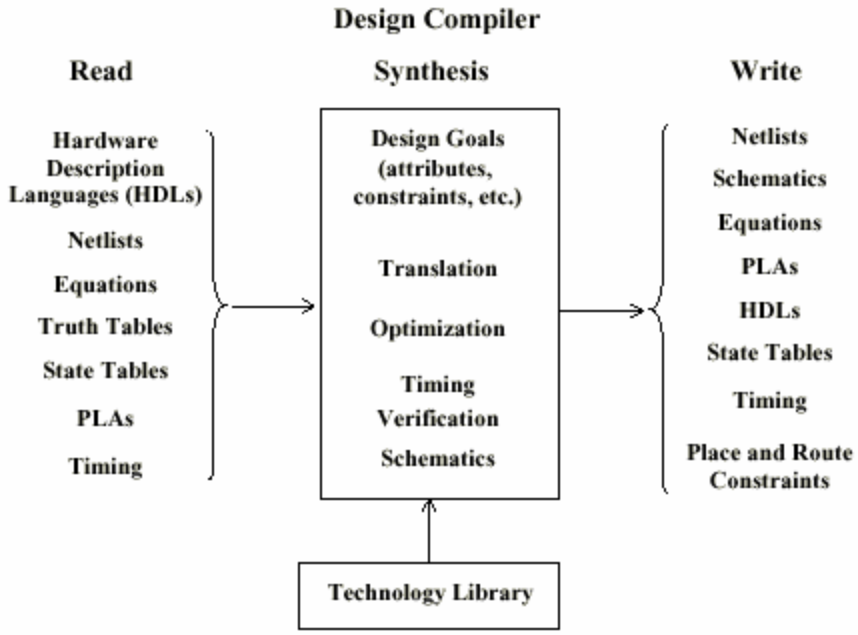


图4-13 Design Compiler 的输入和输出

2. 综合的基本流程

下面我们来看一下综合的基本流程/命令，如下图所示：

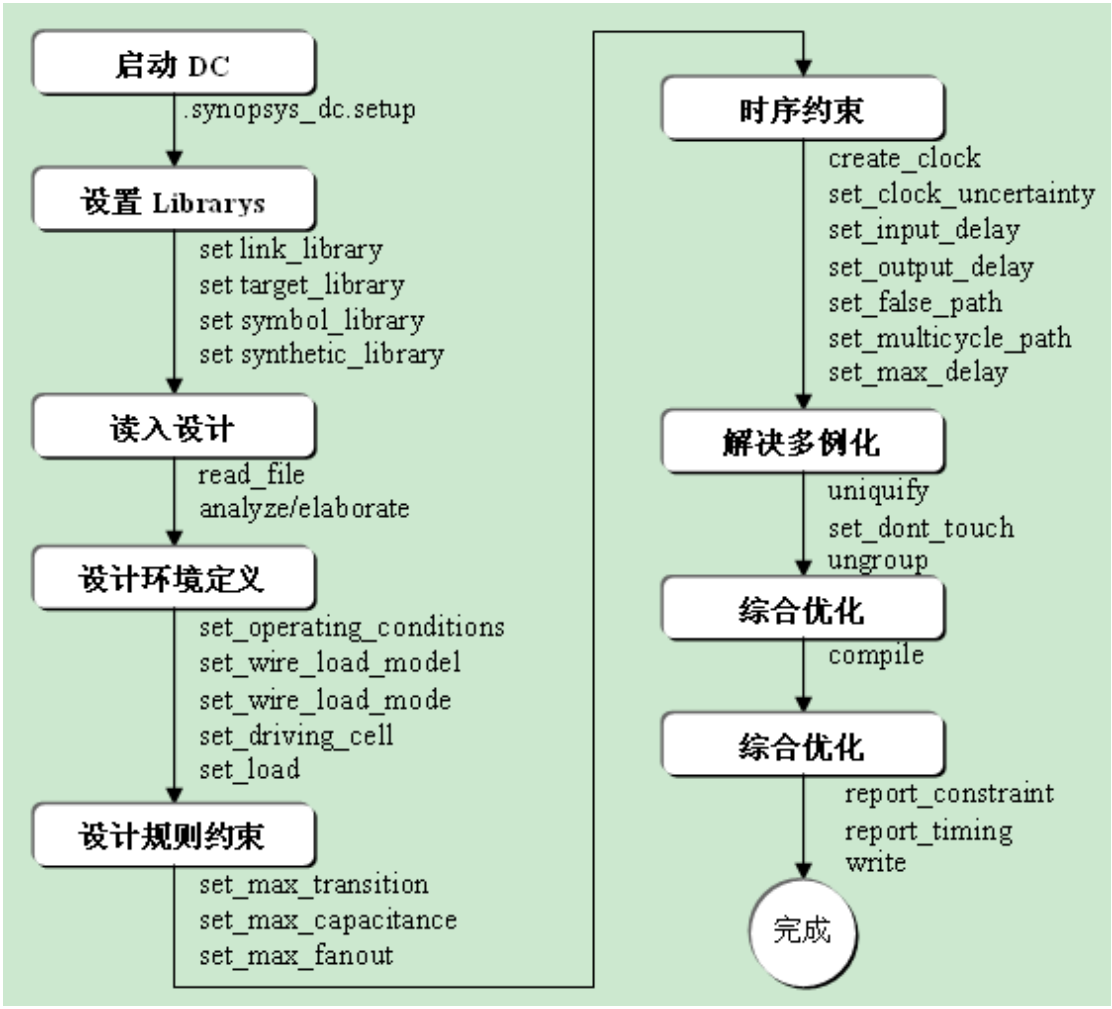


图4-14 综合的基本流程/命令

其中，综合约束（constraint）用来设定电路综合的目标，它包括设计环境的定义，设计规则约束、时序约束和面积约束，这些部分在后续章节进行介绍。

4.3.2 逻辑综合工具介绍

1. DC 的启动和基本操作

DC 提供了两种界面，即 Shell 界面和 GUI 图形界面。

启动 DC 的 Shel 界面，在系统提示符号下键入命令：

%dc_shell

当 DC 启动，完成一些信息后，系统提示符将变成：

dc_shell >

如果要启动 TCL 模式的 Shell 界面，在系统提示符号下键入命令：

%dc_shell –tcl_mode

或者

%dc_shell-t

这时就可以启动 TCL 模式的 DC-shell，并且系统提示符变为：

dc_shell-t>

启动 DC 的 GUI 界面，在系统提示符号下键入命令：

%design_vision

或者

%da

dc_shell 命令还有一些其它的参数，例如，通常我们使用参数"-f file.name"，让 DC 启动后自动运行 file.name 文件。例如：

%dc_shell-t –f path/file.name | tee –i path/file.log

tee 命令将屏幕文本同时记录在文件 path/file.log，-i 允许 CTRL+C 传给 Dc_shell，终止一个命令的运行。

以及使用 –x "命令"，DC 启动后将首先执行参数 －x 后面的命令。例如，我们可以配合 –f 参数，实现一些变量的定义，指引 Script 的运作，达到 Script 的重用。

在 DC 启动后，首先自动执行.synopsys_dc.setup，按照下面的顺序搜索该文件，如果存在则依次执行该文件。

- 安装目录：$SYNOPSYS/admin/setup
- 用户目录：~ / ，即 /home/usename
- 当前目录：./

在.synopsys_dc.setup 文件中，我们通常定义 Library 的设置，环境变量的设置，以及 Naming rules 等。在下一章中将到的库的设置，我们通常放在 .synopsys_dc.setup 文件。附录中，我们给出了一个典型的.synopsys_dc.setup 文件的示例。

如果要退出 DC，键入命令 exit 或者 quit 即可。

在 DC_shell 模式下，使用 help 命令可以列出所用的命令以及简要的说明。help 也支持 Glob 匹配（例如：help get_* ，列出所用以 "get_" 为前缀的命令）。

使用 "命令 -help" 可以得到命令及其参数的简要说明。

使用 "man 命令" 可以得到该命令及其参数的详细说明。

dctcl 环境由很多的系统变量，以及用户定义的变量。使命命令 "printvar " 可以列出所用的变量以及目前的变量值。printvar 也支持 Glob 匹配。使用命令

“man 系统变量” 可以得到该系统变量的详细说明。使用 set 命令可以修改变量的变量值。环境变量通常导引综合的行为，修改环境变量时要谨慎，明确该环境变量的作用。

在使用 man 命令时，由于 man 得到解释在 dcsh / dctcl 模式下是一样的；因此在解释帮助中一些示意命令可能只适用在 dcsh，无法在 dctcl 模式先适用，注意识别，并且应知道在 dctcl 模式下的等效命令。 例如 find ，在 dctcl 模式下用 get_ports、get_nets 、get_pins 等命令取代（也可以使用 find 命令，但语法和 dcsh 不同）。get_ports、get_nets 、get_pins 等命令是获得设计对象的命令，关于设计对象，以及相关的命令操作请参考附录文档。

2. Library 设置

(1) Library 介绍与设置

这一节主要介绍 Design Complier 中的库（Library） 。库包含了 Design Compiler 综合过程中所需要的各种信息。从库的种类说，分为如下的四种类型的库：

- GTECH Library
- DesignWare library
- technology Library （工艺库）
- symbol Library（符号库）

【Gtech Library】

GTECH Library 是 Synopsys 的通用工艺库，它由 Design Compiler 自带，是独立于厂家工艺的。该库中包含的元件仅代表一定的逻辑功能而不带有任何工艺参数，因此是工艺无关的。在综合过程中，DC 首先将 HDL 代码 Translate 到 GTECH 库，然后在综合（compile）时再映射（Map）到工艺库。例如，我们在代码中写 “assgin a = b&c”，那么读入 HDL 代码后，DC 将 HDL translate 到 Gtech cell 表现，即使用一个 GTECH 的与门例化。

在综合过程中，DC 自动调用 GTECH 库，不需要任何设置。

【DesignWare Library】

DesignWare Library 是由 Synopsys（或者第三方）提供的一些可重用的电路宏单元。它们可以完成诸如复杂的算术运算（＋，－，* ，/ 等），数值比较(==,>=,<= 等)，以及 FIFO 、CPU 等功能，DesignWare 库就包含了这些电路模块。例如我们在 HDL 代码中的“A*B”，在 Translate 过程中工具就会调用 Designware 来实现。我们也可以在设计中直接例化这些 Designware 中的电路单元。DC 识别的 DesignWare 是 .sldb 格式的 Library。DesignWare 库一般由 Synopsys 或者第三方提供。

DesignWare library 使用变量 **synthetic_library** 设置，将你想使用的 DesinWare Library 作为一个列表，赋值给变量 synthetic_library ，这样 DC 在优化时就可以调用 量 synthetic_library 中的 DesignWare Library 优化设计（例如：算数运算，如乘法，可以用多种实现方式，DC 根据时序要求，自动选择速度/面积最优的方式实现这个乘法操作。用户也可以指定该乘法的实现方式）。

standard.sldb 是个特殊的 DesignWare 库，在 HDL translate 时，用于构建 HDL 中的操作符，并且 DC 会自动调用 standard.sldb，而无需设置到 synthetic_library 的列表 中去。

⚠ 注意：

DesignWare Library （变量 synthetic_library 的内容）也要设置到 link_library 变量中去，以维持 Cell 的例化关系，下面将介绍 link_library。

关于 DesignWare 的更多内容可参看 Synopsys 在线文档 DesignWare User Guide。

【Technology Library】

Technoligy Library 是由芯片生产厂家提供的，它包含特定的工艺器件以及一些必须的参数信息，如工作环境、逻辑延时、驱动负载以及线负载模型等。工艺库一般由 Vendor 提供。DC 支持的 .db 格式的工艺库。

在 DC 中，使用到的 Technoligy Library 要设置到变量 link_library 中去。用来实现设计的 Technoligy Library 还要设置到变量 target_library 中去。一般来说，变量 link_library 要包含所有的 target_library。例如：

dc_shell-t> set target_library ｛vendor_tech1.db vendor_tech1.db｝

dc_shell-t> set link_library ｛* vendor_tech1.db vendor_tech1.db｝

注意变量 link_library, 还需要加一个星号 (*)，指示 DC 在实例化时也要搜索内存中的设计。如果变量 link_library 不含有这个星号，那么就不会搜索调入内存中的设计。例如对于层次化的设计，如果 link_library 中未加“*”，DC 在 Link 设计时，被实例化的模块就找不到（这些模块已经读入内存，但未指定内存搜索“*”），DC 无法正确完成整个设计的 Link。

【Symbol Library】

符号库包含元件的图形符号，用来 GUI 界面时显示设计的电路图。符号库一般由 Vendor 提供。

在 DC 中，使用变量 symbol_library 来用到的 Symbol Library。如果不使用图形界面显示原理图，该变量不需要设置。

(2) Library 在 DC 中的应用

下面，我们来看一下 DC 的综合过程，可以对 Library 有较好的理解。Library 在流程中的应用如下：

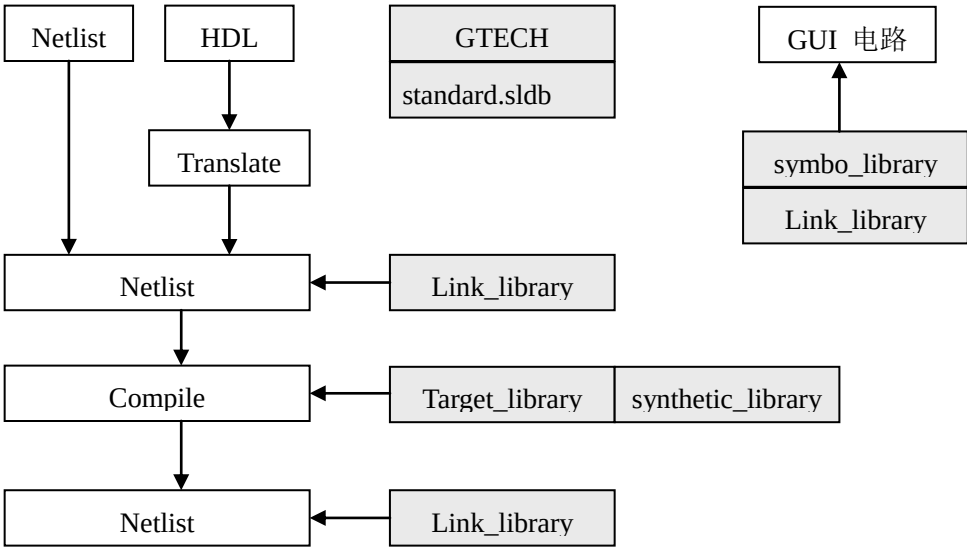


图4-15 DC 中 Library 的调用关系

GTECH、standard.sldb 是基本的 Library，DC 会自动调用，在整个过程中都可能使用。

将 RTL Translate 后，Link，使用 Link Library 构建。由于 GTECH、standard.sldb 是基本的 Library，不需要额外设置，可以自动调用。此时的 Netlist（HDL compile 得到）主要由 GTECH 构建，如果设计中有算数运算、数值比较等，调用 standard.sldb 实现。如果直接例化工艺库元件，那么该工艺库必需在 link_library 指定。如果直接例化 Designware，那么该 DesignWare Library 也必需在 Library 指定。

standard.sldb 中的 DesignWare 用于 Translate 时的调用（相当于一个空壳），因此，在 Compile 过程中，需要选择具体的 DesignWare，DesignWare 在 synthetic_library 中定义，选择了具体的 DesignWare 后，那么维持网表关系就需要 link_library，因此 DesignWare Library 也要设置到 link_path 中 。

Compile 的 Map 过程，是进行工艺库的映射，调用 target_library 中的 Library Cell 来完成映射，映射后也需要维持网表关系，因此 target_library 中的 Library，一般也要在 link_library 定义。

图形显示需要显示元件的形状，管脚位置等信息，因此需要 symbol_library。

下面再举一个例子，可以更好的理解 link_library 和 target_library。

假设我们有 A library 实现的网表，但是我们想用 B library 替换 A library 来实现。那么我们可以将 A library 和 B library 设置到 link_library 中，而 target_library 中只包含 B library。 让 DC 读入 A library 的实现的网表，Compile 后，我们就可以得到用 B library 实现同样功能的设计网表。

(3) Search_path

通常一个设计中可能要调入很多的库，那么在设计库是每一个库都写全路径比较麻烦，因此可以使用“search_path”。当你只指明文件名时，DC 会使用 search_path 中定义的路径进行搜索。search_path 可以按照下面的方法定义：

dc_shell-t> set search_path { . /vendor/ibm/SA27E_STD /vendor/ibm/SA27E_MEM}

因此，DC 在所用上面 Link_library 中的库时，就会依次察看 search_path 定义的路径是否存在该文件，并且调用。

⚠ 注意：

在后面讲到的读入 HDL 等文件时，也是支持 Search_path 的。

(4) 库的察看

在 Design Compiler 中，使用命令 “get_libs *” 可列出当前内存中的库。注意库名（library_name）和库文件名（library_file_name）不是一个概念，两个不同文件名的库文件可以包含同名的库，此时要区分这两个库可以用 library_file_name:library_name 的表示方法。使用 report_lib library_name 可以查看库的内容。在后续的章节中，将会有一些具体的 Library 察看的例子。

3. 读入设计文件

Library 设置完成后，我们就可以读入我们的设计文件了。常有两种方法读入代码，read_file 和 analyze / elaborate 。读入代码的过程就是将 HDL 文件 **Translate** GTECH Library 的描述。当读入了 HDL 代码，此时得到的设计是一个 Unmapped 的设计。此时的设计组合逻辑使用 GTECH 例化，时序逻辑使用 SEQGEN 表现，而算术运算符则使用 DesignWare 表现。你可以使用 **report_cell** 或者 **write** 命令察看。

(1) analyze / elaborate 方法

这种方法读入代码分两个部分：

analyze HDL Code file

elaborate Top Module

其中，analyze 命令执行下面的操作：

- 读入 HDL 代码（verilog 或者 VHDL），并且执行语法检查，Synopsys Rules 的检查。
- 转化 HDL 代码为中间格式。
- 而 elaborate 命令执行下面的操作
- 将设计转化为 GTECH 逻辑描述
- 允许代码中的参数重定义操作
- 执行 VHDL architecture 的选择

- 将 HDL 中的算数操作替换为 DesignWare 元件
- 自动执行 Link 操作

命令示意如下：

dc_shell-t> analyze “Demo1.v Demo2.v”

dc_shell-t> elaborate TOP

(2) Read_file 方法

使用 read_file 读入 HDL code 时，执行下面的操作：

- 执行和 analyze 相似的操作，以及 elaborate 命令的第一步操作：将设计转化为 GTECH 逻辑描述
- 对于 Verilog Code，read_file 不产生中间格式文件，对于 VHDL Code，则产生部分中间格式的文件
- 不自动执行 Link 命令
- 支持更多的文件格式读入

读入设计文件可以使用 read_file 命令。read_file 支持多种文件格式的读入，我们常用的就是 db 和 verilog 格式的文件了。

dc_shell-t> read_file –format db Demo1.db

dc_shell-t> read_file –fromat verilog Demo2.v

dc_shell-t> current_design TOP

dc_shell-t> link

你可以使用 read_db 和 read_verilog 命令来读入 db 和 verilog 文件，其实 read_db 和 read_verilog 是 read_file –format db 以及 read_file –fromat verilog 的化名（alias）。读入多个文件时可以把文件作一个列表。

dc_shell-t> read_verilog “Demo1.v Demo2.v”

(3) analyze / elaborate 和 read_file 的比较

下表我们对这两种方法进行简单的对比

表4-1 analyze / elaborate 和 read_file 的比较

	read_file Command	analyze / elaborate Commands
输入文件格式	支持多种文件格式	VHDL, Verilog
使用场所	综合 VHDL or Verilog 代码，读入 Netlist 等等	Synthesize VHDL or Verilog
Design libraries	不能存储 analyzed 的中间结果	可以 指定存储 analyzed 的中间结果
parameters	不支持传参（支持代码中的传参)	可以在 elaborate 命令指定传参
Architecture	不能指定 architecture elaborated	运行指定 architecture elaborated
Link	需要指定模块进行 Link	自动 Link

(4) Presto

在进行 HDL compiler 时，允许用户是否使能 Presto。Presto 可以加快代码读入的速度，提高容量，以及一些代码新特性的支持，而且也可以得到更好的综合效果。

使用 Presto 将不支持 FSM Complier 以及 implicit state machines，而使用 Presto 则可以支持 Verilog 2000 以及 array instantiation of gates or modules 等。

在早期的版本中（DC 2001.08 之前）, Presto 还不是很稳定，目前较新的版本 Presto 已经比较稳定，推荐使用（DC 默认是允许的），如果不使用 Presto 而使用 HDL complier，则：

dc_shell-t> set hdlin_enable_presto false

4. 设计环境定义

设计环境（Design Enviroment）指的是用来描述设计在工作时的温度、电压、驱动、负载等外部条件的一系列属性，它包括以下三个方面：

- 工作条件（The operating conditions）
- 线负载模型（The wire load models）
- 系统接口（The system interface）

这些属性在综合电路的时候是必须加的，如果用户没有进行显式的说明则 Design compiler 会使用缺省值。Design compiler 提供了相应的命令来设定环境属性。

常用的命令如下表所示：

表4-2 常用的设计环境定义的命令

命令	描述
list_libs	显示当前调入的所有 Library
report_lib	报告指定的 Library 信心
get_libs	得到指定的 Library 集合
set_operating_conditions	设置设计的工作环境
set_wire_load_model	设置设计的线负载模型
set_wire_load_mode	设置设计的线负载模式
set_driving_cell	设置输入的驱动能力，使用 Cell 电阻值
set_drive	直接设置输入的驱动能力
drive_of	得到 Cell 的电阻值
set_load	设置输出负载的电容值
load_of	得到 Cell 的电容值

(2) 设定工作条件

工作条件指的是芯片的工作温度（Operating temperature）、工作电压（Supply voltage）和工艺参数（Process）。这三个因素的变化会对芯片的性能造成很大的影响。Design compiler 在进行时序分析时将综合考虑这些因素的最坏和最好情况。

在大多数工艺库中已经预定义了多种情况下的工作条件，在设定工作条件之前应先用 **report_lib** 命令查看工艺库中提供了哪些预定义的工作条件。注意如果库没有在内存中的话要先把库读进来。注意有些库的文件名和库的名字是不一样的。我们可以使用 list_libs 得到调到内存中的所有库文件，以及库的名字。如下：

```
dc_shell> list_libs
Logical Libraries:
--------------------------------------------------------------------------------
Library                    File                    Path
-------------------------  ----------------------  ----------------------------------------
standard.sldb              standard.sldb           /edatools/synopsys/2003.06/libraries/syn
dw01.sldb                  dw01.sldb               /edatools/synopsys/2003.06/libraries/syn
dw02.sldb                  dw02.sldb               /edatools/synopsys/2003.06/libraries/syn
dw_foundation.sldb         dw_foundation.sldb
/edatools/synopsys/2003.06/libraries/syn
gtech                      gtech.db                /edatools/synopsys/2003.06/libraries/syn
my_lib                     my_lib.db               /vendor/demo/synopsys
```

⚠ 注意：

在上面的 Library 列表中的 my_lib，是我们要使用的工艺库，而且可以发现库的名字和文件名是不一样的。我们使用 report_lib 命令来看看库中关于工作环境的定义。如果有 Library 名字相同但文件名字不同时，可以使用 File:Library 名方式指示。

```
dc_shell> report_lib my_lib
****************************************
Report : library
Library: my_lib
Version: 2003.06
Date : Tue Jul 15 06:22:47 2003
****************************************
```

...

Operating Conditions:

Name Library Process Temp Volt Interconnect Model

WCCOM my_lib 1.50 70.00 4.75 worst_case_tree

WCIND my_lib 1.50 85.00 4.75 worst_case_tree

WCMIL my_lib 1.50 125.00 4.50 worst_case_tree

...

从报告中我们可以看到库中定义了三种工作模式：商用 Worst（WCCOM ：Worst Case Commercial），工业 Worst（WCIND：Worst Case Industrial）以及军用 Worst（WCMIL：Worst Case Military）。每种模式有工艺、温度、电压以及互联模型的区别。

此时要根据设计工作环境，和 Vendor 讨论该选择哪种工作模式。例如设计的最差工作电压只有 4.50v，温度是 125°C，你就可以向 Vendor 求证综合是工作模式是否设置为 WCMIL，并且使用命令 set_operating_conditions 来实现。

dc_shell> set_operating_conditions WCMIL -lib my_lib

然后可以使用命令 report_design，看看约束的设置情况。

dc_shell-t> report_design

****

Report : design

Design : demo

Version: 2003.06

Date : Tue Jul 15 06:48:43 2003

****

. . .

Operating Conditions:

Name Library Calc_mode Process

Temp Volt Interconnect Model

WCMIL my_lib 1.50 125.00 4.50 worst_case_tree

. . .

(3) 设定线负载模型

线负载模型用来估算电路中线的长度和扇出数对电阻、电容和面积的影响。它根据连线的扇出数给出连线的等效长度，根据这个长度值又可进一步估算出线的电容和电阻值。线负载模型是由芯片生产厂家根据特定工艺的统计信息建立的，并且在工艺库中给出了定义。

Design Compiler 根据如下三点确定当前设计所采用的线负载模型（优先级由高到低）：

- 用户指定。
- 根据设计的面积自动选择。
- 工艺库中指定的缺省模型。

可以使用 report_lib 命令可以查看库中定义的线负载模型，命令行和报告示例如下：

```
dc_shell> report_lib my_lib
****************************************
Report : library
Library: my_lib
Version: 2003.06
Date : Thu Jul 17 11:53:48 2003
  ****************************************
...
Wire Loading Model:
Name : 05x05
Location : my_lib
Resistance : 0
Capacitance : 1
Area : 0
Slope : 0.186
Fanout Length Points Average Cap Std Deviation
------------------------------------------------------------
1 0.39
Name : 10x10
Location : my_lib
Resistance : 0
Capacitance : 1
Area : 0
Slope : 0.311
Fanout Length Points Average Cap Std Deviation
------------------------------------------------------------
1 0.53
Name : 20x20
Location : my_lib
```

Resistance : 0

Capacitance : 1

Area : 0

Slope : 0.547

Fanout Length Points Average Cap Std Deviation

---

1 0.86

Wire Loading Model Selection Group:

Name : my_lib

Selection Wire load name

min area max area

0.00 1000.00 05x05

1000.00 2000.00 10x10

2000.00 3000.00 20x20

...

一些工艺库支持线负载模型的自动选择，此时 Design Compiler 根据设计的面积自动选择线负载模型。如果在综合时不是很清楚该设置那种 Wire_load_Model，可以向 Vendor 请教，或者使用自动选择（此时 Library 中必须有 Wire Loading Model Selection Group 的定义）。

但对一个很大的设计，自动选择线负载模型会大大增加综合的时间。用如下命令可以关掉自动选择：

dc_shell> set auto_wire_load_selection false

此时可以用命令 set_wire_load_model 显式地指定线负载模型，例如如果选择“10x10”的线负载模型，可以输入：

dc_shell-t> set_wire_load_model -name "10x10"

Design dff: Using wire_load model '10x10' found in library 'my_lib'.

对于一个层次化的设计，可能顶层和子模块的线负载不同，那么子模块中的 Net 选用那种 线负载呢？使用 set_wire_load_mode 命令来指定子模块选用什么样的线负载。在层次化的设计中，Design Compiler 用 top、enclosed 和 segmented 三种模式来确定子模块的线负载模型，这三种模式示意如下：

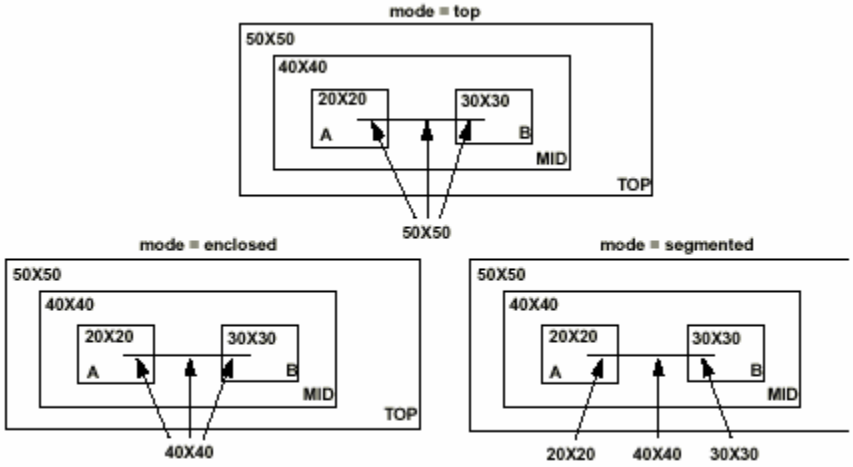


图4-16 三种线负载模型的区别

例如对于下图的设计，我们详细看一下不同的 Wire Load Model，各个 net 的 Wire Load Model 如何选择：

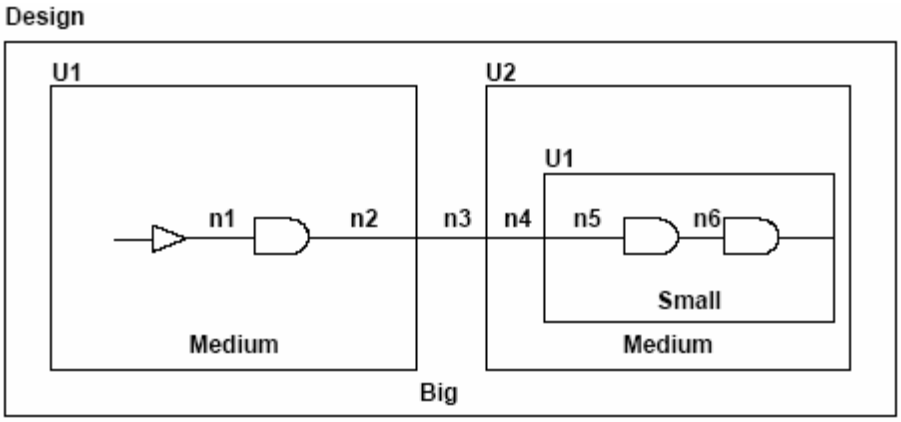


图4-17 Wire Load Mode 设置示意

下表列出了对于上面的设计，不同的 Wire load mode 下，各个 net 的 Wire Load Model 使用。

表4-3 Wire Load Model 选择

Wire Load Mode setting	Wire load model	Applies to these net
top	Big	All nets
enclosed	Big	n3, U1/n2, U2/n4, U2/U1/n5
	Medium	U1/n1
	Small	U2/U1/n6
Segmented	Big	n3
	Medium	U1/n1, U1/n2, U2/n4
	Small	U2/U1/n5, U2/U1/n6

Design Compiler 根据如下三点确定子模连线采用的线负载模型（优先级由高到低）：

- 用户指定。
- 工艺库缺省。
- Design Compiler 缺省（top mode）。

例如我们设置为 enclosed，命令如下：

dc_shell-t> set_wire_load_mode enclosed

这时仍可以使用 report_design 检查一下我们的设置，例如：

dc_shell-t> report_design

****

Report : design

Design : demo

Version: 2003.06

Date : Thu Jul 17 12:13:11 2003

****

Wire Loading Model:

Selected manually by the user.

Name : 10x10

Location : my_lib

Resistance : 0

Capacitance : 1

Area : 0

Slope : 0.311

Fanout Length Points Average Cap Std Deviation

---

1 0.53

. . .

Wire Loading Model Mode: enclosed.

在实际的综合中选择什么样的线负载模型取决于电路是用何种工艺如何实现的，因此应当向 ASIC 生产厂家咨询采用哪种线负载模型。

另外要注意的是对于 Bottom-Up 的综合策略（先综合底层模块，最后合成顶层模块，后面的综合策略中详细讲述），如果设置线负载模式为 Top，并且线负载模型使用自动选择（或者子模块选用小的线负载模型，顶层选用大的线负载模型），那么到了顶层，经常出现已经满足时序的子模块有出现了 Timing Violations。这时如果在顶层优化将花费很长的时间。因此如果线负载模式为 Top，那么顶层和底层应该选择相同的线负载模型。

如果负载模式为其他的模式，这时子模块和顶层可以是不同的线负载模型，要注意子模块输入、输出端口的约束，留出一定的裕量（实际中，通常我们对子模块的端口约束比较严格，因此一般在接口上没有问题）。

(4) 描述系统接口

在 Design compiler 中，电路与外部系统的接口包括如下方面：

- 输入端口的驱动。
- 输出端口的负载。
- 输出端口的扇出负载（Fanout load）

【设定输入端口的驱动】

在 Design Compiler 中，驱动强度用驱动电阻值表示，阻值越小驱动强度越大，对输入端口如果没有设定驱动则缺省值为 0，表示无穷大的驱动强度。Design Compiler 提供了 set_drive 和 set_driving_cell 两条命令来设定端口的驱动。如果同时使用这两条命令，那么后面的设定将覆盖前面的。下面分别解释这两条命令。

A．set_driving_cell 命令

当知道端口的驱动元件时，可以用 set_driving_cell 为该端口设定驱动。命令语法如下：

set_driving_cell -library library_name -cell cell_name -p pin_name [get_ports {port_list}]

当指定的元件有多个输出脚时，应通过 -pin 选项指明是哪个输出脚。库名可使用 list_libs 查询，使用 report_lib library_name 查看库中的元件。

set_driving_cell 命令示意如下：

dc_shell-t> set_driving_cell –lib_cell FD1 –pin Q [get_ports IN1]

取消设置使用-none 选项，即：set_driving_cell -none [get_ports {port_list}]。

用该命令设定端口驱动时，不论当前库用的是何种延时模型，Design Compiler 都可以精确计算出端口的延时。因此在可能的情况下推荐尽量使用这个命令。

B．set_drive 命令

这个命令直接对端口设定驱动电阻值。在顶层设计中，因为输入端口没有实际的库元件驱动，所以应使用该命令。命令语法如下：

dc_shell-t> set_drive drive_value [get_ports {port_list}]

set_drive 可以和 drive_of 以及 tcl 的数学表达式运算（expr）结合使用，如下：

dc_shell-t> set_drive [expr [drive_of lib_name/cell_nema/pin_name] * 5] [get_ports {port_list}]

在上面的示意中，就是以 5 倍的 drive_of 返回 pin 的电阻值，作为 drive_value。这样就避免了直接指定驱动电阻值，显得更直观一些。

使用 set_drive 命令设定驱动电阻值后，Design Compiler 用公式 T = Rdrive (Cpin + Cwire)计算输入端口的延时。因此 set_drive 只有对线性延时模型是准确的，而不适用于其它模型。

【设定输出端口的负载】

使用 set_load 命令设定输出端口的电容负载值，利用这个值，Design Compiler 可以为输出端口选择合适的驱动，以及用来计算输出端口的延时。电容负载的单位在工艺库中定义，注意设定的值的单位要和库中的一致。电容负载的缺省值为“0”，表示没有负载。用 report_lib 命令可以查看库中电容负载的单位，报告示例如下：

```
dc_shell-t> report_lib my_lib
****************************************
Report : library
Library: my_lib
Version: 2003.06
Date : Thu Jul 17 12:13:11 2003
****************************************
Library Type : Technology
Tool Created : 1997.01
Date Created : February 7, 1992
Library Version : 1.800000
Time Unit : 1ns
Capacitive Load Unit : 0.100000ff
Pulling Resistance Unit : 1kilo-ohm
Voltage Unit : 1V
Current Unit : 1uA
...
```

电容的单位为 0.100000ff。

set_load 命令的语法格式如下：

```
set_load    load_value    [get_ports {port_list}]
```

同样如果不想直接指定电容值的话可以用 load_of 以及 TCL 的算数运算命令，用法如下：

```
set_load    [expr [load_ofl ib_name/cell_nema/pin_name] * n]    [get_ports {port_list}]
```

【设定输出端口的扇出负载】

用 set_fanout_load 命令设定输出端口的扇出负载，它描述了输出端口外部的扇出负载总和。Design Compiler 将尽量保证输出端口的扇出负载数加上设计内部

与该端口相连的元件的扇出负载数的总和小于规定的最大扇出负载（max_fanout，参见下节关于 max_fanout 的设定）。 set_fanout_load 的语法格式如下：

set_fanout_load value [get_ports {port_list}]

5. 设计规则约束

设计规则（Design Rule）反映了特定工艺对设计的要求，这些要求是必须满足的。Design Compiler 支持如下的 Design Rule 约束：

- Transition time (最大值和最小值)。
- Fanout load (最大值和最小值)。
- Capacitance (最大值和最小值)。
- Cell degradation。
- Connection class。

在一些工艺库中给出了 Design Rule 约束的缺省值，在这种情况下一般不需要再另外定义。如果需要更严格的约束，可以自己手工定义。对于同一个约束的重复定义，Design Compiler 总是取最严格的一个值。在优化的过程中，Design Rule 约束相对于其他的约束条件具有更高的优先级，即 Design Compiler 总是首先考虑保证满足 Design Rule 的要求。

在一般的设计中，需要人工调整的 Design Rule 约束主要是最大 Transition time（max_transition）和最大 Fanout（max_fanout），其它约束一般采用库中的缺省值就可以了。下面就详细说明一下这两个约束的含义和设定方法，如果想了解其它几个约束可以查阅 Synopsys 的相关资料。

常用的 Design Rules 设置命令如下：

表4-4 常用的 Design Rules 限制命令

命令	描述
set_max_transition	设置最大 Transition 时间限制
set_max_fanout	设置最大 Fanout 数目限制
set_max_capacitance	设置最大电容值限制
set_min_capacitance	设置最小电容值限制

(2) 设定 max_transition

一根连线（net）的 Transition time 指的是驱动它的 pin 的逻辑电平跃变所需要的时间。Design Compiler 计算 Transition time 的方法是用驱动 pin 的驱动强度乘以它驱动的连线上的所有容性负载的总和。用公式表示为：

TT = Rdrive (Cwire + Cpin)

设定 max_transition 用命令 set_max_transition，它的语法格式如下：

set_max_transition value { object_list }

其中 object_list 可以为端口或整个设计。当对一个端口使用该命令时，最大 transition time 的约束只对该端口驱动的 net 有效；如果对整个设计设定 max_transition，那么该设计中的所有 net 都被加上了这个约束。

例如，如果要对设计 adder 的所有 net 加上 3.2 单位的最大 transition time 约束，可以输入：

```
dc_shell> set_max_transition 3.2 [get_design DEMO]
```

如果想取消这一约束，可以用：

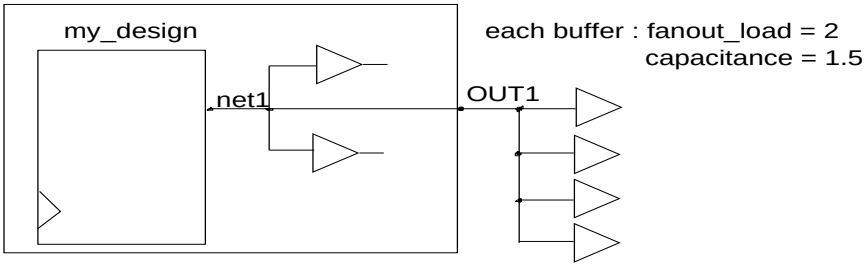
```
dc_shell> remove_attribute [get_design DEMO] max_transition
```

通常对于我们现在常见的设计，Max_transition 一般限定小于 0.5 ~ 1.0 ns，具体的可以向 Vendor 咨询。

(3) 设定 max_fanout

一根 net 的扇出负载（Fanout load）指的是它所驱动的所有输入 pin 的 fanout load 的总和。注意区分 load、fanout load 和 fanout number 的区别。load 指的是 net 上的各种容性负载的总和，单位是电容的单位；fanout load 是一根 net 所驱动的所有输入 pin 的 fanout load 的总和，它没有单位，并且一个输入 pin 的 fanout load 值可以不为 1；fanout number 则指的是一根 net 所驱动的输入 pin 的总个数。请看如下例子：

```
set_max_fanout 12 [get_design my_design]
set_fanout_load 8 [get_ports OUT1]
set_load 6 [get_ports OUT1]
```



在图中每个 BUFFER 的输入 pin 的 Fanout load 值设为 2，端口 OUT1 的 Fanout load 值设为 8。则连线 net1 的 fanout load 为 12 ，其中内部 fanout load 为 4 ，外部 fanout load 为 8；而它的 fanout number 为 6 ，其中端口 OUT1 的 fanout number 为 4 。

在线负载模型中用来计算线的等效长度 fanout_length 的就是 实际的 fanout number 的值。例：

```
wire_load ("10 x 10") {
   resistance : 5.0
   capacitance : 1.1
   area : 0.05
   slop : 0.15
   fanout_length (1, 2.6)
   fanout_length (2, 2.9)
```

fanout_length = (4-2) x 0.15 + 2.9 = 3.2
C_{wire} = length(3.2) x capacitance coeff.(1.1) = 3.52
R_{wire} = length(3.2) x resistance coeff.(5.0) = 16
Area of wire = length(3.2) x area coeff.(0.05) = 0.16

在综合时，Design Compiler 会计算驱动 pin 所驱动的所有输入 pin 的 fanout load 的总和，并设法使它小于 max_fanout 约束。在有些工艺库中对库里的所有元件或某些特定元件已经设定了 max_fanout 约束，用如下命令可以查看工艺库是否设定了缺省的 max_fanout 属性：

dc_shell> get_attribute my_lib default_max_fanout

用如下命令可以查看工艺库中是否为元件的输入 pin 设定了 fanout load 属性：

dc_shell> get_attribute [get_lib_pins my_lib/*/*] fanout_load

如果想自己设定更为严格的 max_fanout 约束，可以用 set_max_fanout 命令，语法如下：

set_max_fanout value { object_list }

object_list 可以为输入端口或整个设计。当对整个设计加 max_fanout 时，该设计中的所有 net 都被约束。

例如，如果要将设计 adder 的 max_fanout 设为 16，可以输入：

dc_shell> set_max_fanout 8 [get_design demo]

取消设定可以用：

dc_shell> remove_attribute [get_design demo] max_fanout

通常对于我们现在常见的设计，Max_fanout 一般限定小于等于 8，具体的可以向 Vendor 咨询。

6. 时序约束

(1) 时钟和时钟网络

表4-5 描述时钟的一些常用命令

功能	命令	描述
创建时钟	create_clock	创建一个时钟，并且指定周期，波形，一般用于 Input Port
	create_generated_clock	生成一个时钟并指示它和源时钟关系，用于内部生成时钟
时钟属性	set_clock_latency	描述时钟的延迟信息，包括时钟源头的 delay（source latency）和时钟树的 delay（insertion latency）
	set_clock_uncertainty	描述时钟的抖动，沿的不确定区间
	set_clock_trainstion	描述时钟沿的 Trainstion 时间，一般用于理想时钟

功能	命令	描述
	set_propgated_clock	指示时钟为传播时钟（相对于理想时钟）

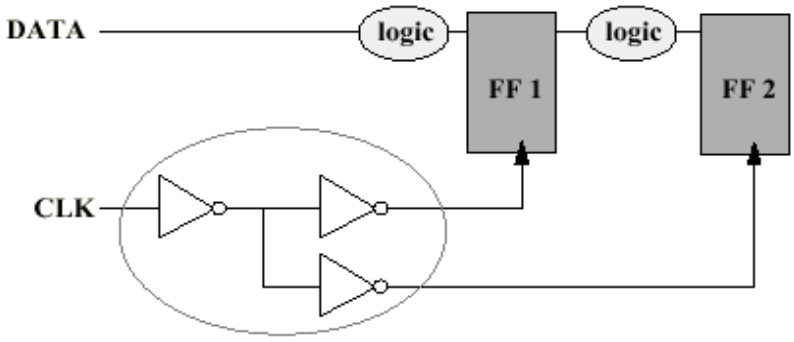
当 design compiler 对一个设计进行 timing 约束和分析时，它不能自动产生时钟，必须对设计中的时钟源进行显式的定义。

design compiler 中时钟分为两种：真实时钟（real clock）和虚拟时钟（virtual clock）。

真实时钟有时钟源，它可以是理想时钟（ideal clock）和传播时钟（propagated clock）。理想时钟在时钟树上不会引起延时，当定义时钟后，缺省就是理想时钟。而传播时钟和理想时钟不同，若一个寄存器的时钟输入是传播时钟，那么就会计算时钟源到寄存器时钟输入端路径的所有延时，得到时钟的到达时间。使用命令 set_propagated_clock 可以把理想时钟转变为传播时钟（在综合时我们一般不使用传播时钟，后面给出原因）。

虚拟时钟（virtual clock）没有时钟源。使用虚拟时钟作参考可以定义 input 和 output 相对的延时。在某些情况下，例如在定义组合逻辑的时序约束时，使用虚拟时钟特别有用。

时钟树（clock tree），又称时钟网络， 是指位于时钟源和它所有扇出的寄存器时钟输入端之间的组合逻辑。



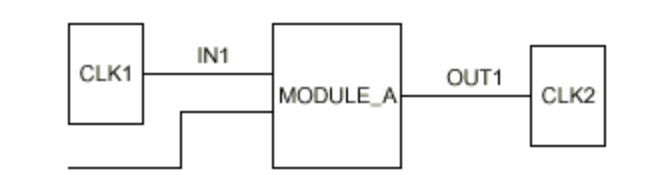
上图圆圈内为一个简单的时钟网络。

时钟树一般由厂家根据物理布局的情况综合产生，一般不使用 design compiler 对时钟树进行综合（CTS）。通常，后端的 P&R 工程师进行时钟网络综合，并给出一个保证的 clock skew（clock skew 指时钟树不同分支之间的最大延时差），以及 Clock 的 Insertion Delay（就是指时钟树的延时）。因此，在我们进行逻辑综合时，可以使用 set_clock_uncertainty、和 set_clock_latency 对时钟网络进行模型化。

理想时钟的 Clock skew / Clock insertion Delay 的值为 0。

【时钟定义】

design compiler 能够处理单相、多相时钟以及多频率时钟方案。它不能在综合时自动地定义时钟，必须使用 create_clock 命令来完成这个功能。另外， 组合逻辑没有馈入的时钟，在定义输入输出的延时前，可以定义一个虚拟时钟，然后设定相对于这个虚拟时钟的延时值。 时序逻辑一般都有时钟，通过设定相对于时钟的输入输出延时来对设计进行优化。



例如上图中，输入延时(IN1)相对于时钟 CLK1 设定，输出延时(OUT1)相对于时钟 CLK2 设定。

对一个时钟来说，使用下列参数来描述其特征：

时钟源（clock source）

设计中加入时钟波型的 pins 或 ports 。一个时钟可以拥有多个源。设定时钟源后，design compile 自动将 时钟信号传递到所有扇出的寄存器的时钟输入端。在一个已 map 的设计中，可以通过命令 derive_clock 识别出时钟源。

时钟周期（clock period / cycle time)

时钟周期就是下面讲述的 waveform 的最小重复时间。对于简单的 waveform，只有一个上升沿和一个下降沿，那么时钟周期就是连续两个上升沿（或者下降沿）的时间差。

waveform

waveform 描述一个周期内的时钟波形。一个简单的波形有两个沿，例如 {0.0 5.0 } ，0.0 时是上升沿，5.0 时是下降沿。复杂的波形，例如 { 3 5 11 13 } ，表示该波形在 3 和 11 时是上升沿，5 和 13 时是下降沿。

rising edge

波形的上升沿，就是从低电平变化为高电平。

falling edge

波形的下降沿，就是从高电平变化为低电平

A. create_clock 命令

使用 create_clock 命令，进行时钟的定义，你可以：

- 定义时钟的信息，如：period、 waveform、source 等；
- 修改已存在的时钟信息，例如：period、waveform 、source 等；
- 约束时钟源驱动的触发器及触发器之间的路径；
- 对于多时钟的设计，定义多个时钟，约束各个钟源驱动的触发器及触发器之间的路径；

create_clock 语法格式为：

create_clock [-period period_value] [-name clock_name] [-waveform edge_list] [-add] [source_list]

参数描述如下：

-period period_value : 设定 period

-name clock_name : 设定 clock name

如果没有设定 source_list，必须设定 clock name 。如果没有设定 clock name，使用 source_list 中的第一个名字作 clock name;

-waveform edge_list : 定义一个周期时钟的 waveform

edge list 包含偶数个 edge timing points。 edge 必须以递增的顺序出现。最后一个时刻与第一个时刻之差不能大于一个时钟周期。 如果省掉该选项，design compiler 会把时钟缺省设为 50 % duty cycle, 上升沿设为 0ns 处 。

-add : 表示在 Source_list 再在定义一个新的时钟。

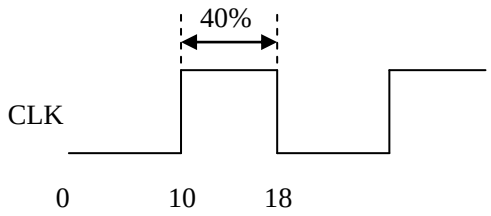
如果没有选项 –add，那么在 source_list 上，新定义的时钟将修改原来定义的时钟。

source_list : 合法的对象是设计中的一个或多个 ports 或 pins 。

如果使用 create_clock 命令将时钟设定在一个 cell 的输出 pin 上，design compiler 自动给该 cell 加上 dont_touch 的属性，防止在优化综合时将该 cell 删除或改为其它类型的 cell。

以下是几个定义时钟的实例。

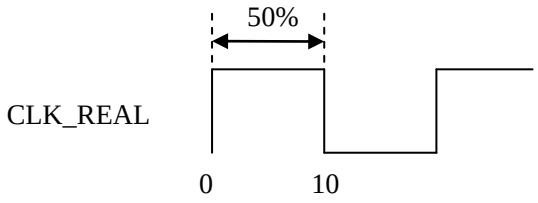
例 1：创建一个周期 20ns, 40% duty cycle 的虚拟时钟。



dc_shell > create_clock -name CLK -period 20 -waveform {10 18 }

例 2：定义一个真实的时钟。

dc_shell > create_clock -name CLK_REAL -period 20 -waveform { 0 10} [get_ports CLK_IN]



在这个示意中，-waveform { 0　10} 可以不写出，因为缺省的 waveform 就是这样的。

例 4：如何对一个设计进行较严的约束。

为了给方便 Vendor 的后端实现（P&R），需要对时钟进行过约束。例如，对一个设计要求在 100Mhz 频率下运行的电路，如果要求过约束 20%，那么，在 create_clock 时，可以设定时钟的周期设定 period 为 8ns。

关于时钟的过约束，需要向 Vendor 确认过约束的比例，对于 0.18 工艺，通常要过约束 20%。

B．create_generated_clock 命令

一个设计中可能会有内部生成的时钟，如分频，倍频等，为了较好的描述这样的生成时钟，我们可以使用 create_generated_clock。create_generated_clock 的语法如下，

create_generated_clock

[-name clock_name] [-add] port_pin_list

-source master_pin [-master_clock clock]

[-divide_by freq_div_factor]

-edges {edge1, edge2, edge3}

[-edge_shift {shift1, shift2, shift3}]

-multiply_by freq_mult_factor]

[-duty_cycle percent] [-invert]

各个选项的简单描述如下，详细的请在 DC 中使用 Man 查看。

-name clock_name 定义时钟的名字，同 create_clock 中的选型。

-add 表示在 port_pin_list 上再定义一个新的时钟，同 create_clock 中的选型。

-source master_pin [-master_clock clock] 定义生成时钟的参考点、参考时钟。

因为 create_generated_clock 不会显示的给出时钟的周期，wavefrom，只是后面介绍的选项，只是指示和参考点时钟的关系。

[-divide_by freq_div_factor] 定义分频时钟的分频系数。

-edges {edge1, edge2, edge3}可以用来定义一些特殊的分频时钟。

-multiply_by freq_mult_factor] 定义倍频时钟的系数

[-duty_cycle percent] 指示生成时钟的 Duty。

-invert ,指示生成时钟和参考时钟的相位关系

下图示意一个二分频时钟，我们看看如何使用 create_generated_clock 定义时钟。

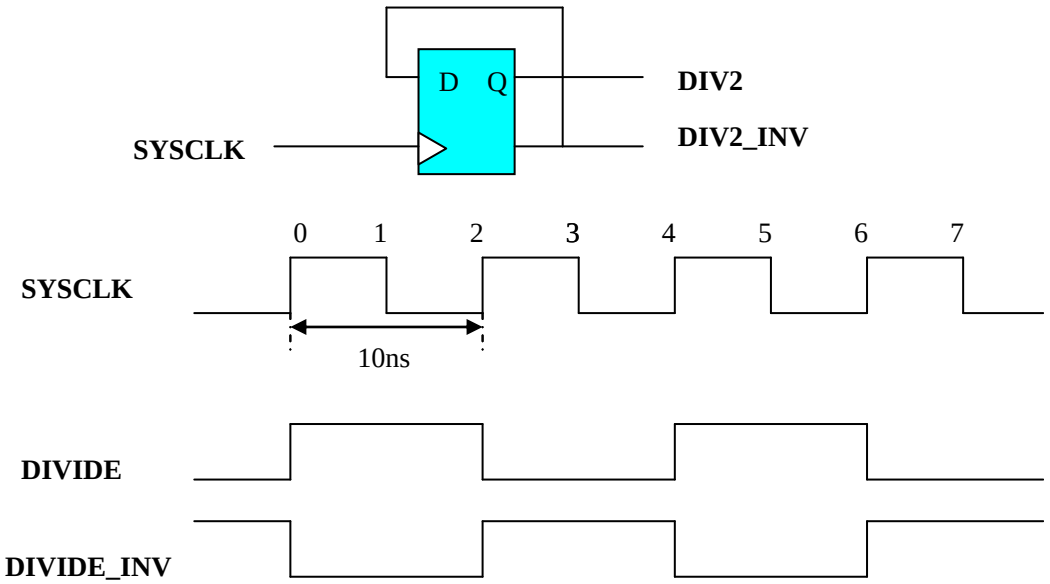


图4-18 创建分频时钟电路、波型示意 1

dc_shell_t> create_clock -period 10 [get_ports SYSCLK]

dc_shell_t> create_generated_clock -name DIV2 -source [get_ports SYSCLK] -divide_by 2 [get_pins DIV_reg/Q]

dc_shell_t> create_generated_clock -name DIV2_INV -source [get_ports SYSCLK] -divide_by 2 –invert [get_pins DIV_reg/QN]

对于生成时钟，我们也可以时钟-edge 选项来描述，例如：

dc_shell_t> create_generated_clock -name DIV2 -source [get_ports SYSCLK] -edge {1 2 3} [get_pins DIV_reg/Q]

dc_shell_t> create_generated_clock -name DIV2_INV -source [get_ports SYSCLK] -edge {2 3 4} –invert [get_pins DIV_reg/QN]

对于一些比较复杂的分频时钟，只能使用-edge 定义了，如下所示：

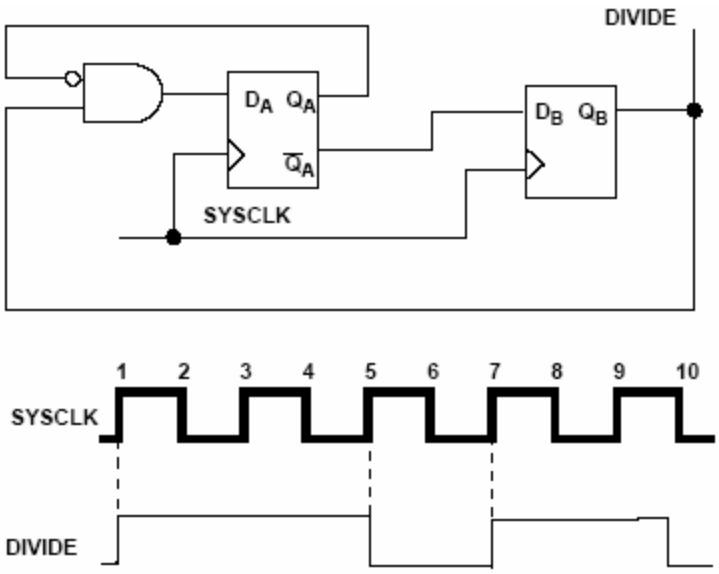


图4-19 创建分频时钟电路、波型示意 2

dc_shell-t> create_generated_clock -edges { 1 5 7 } -source SYSCLK [get_pins DIVIDE]

倍频时钟的定义，可以使用选项-multiply_by 实现，示意如下：

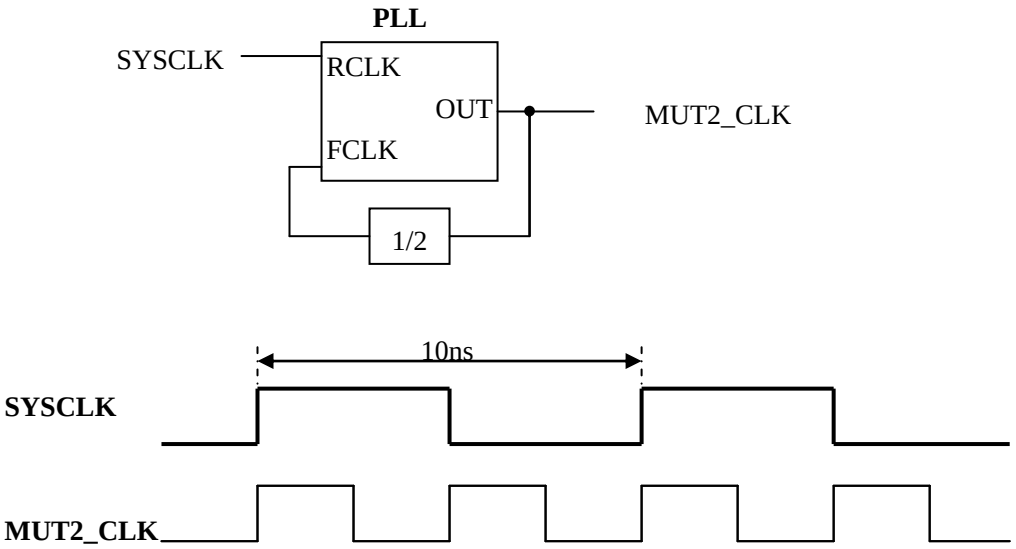


图4-20 创建倍频时钟电路、波型示意

dc_shell_t> create_generated_clock -name MUT2_CLK -source [get_ports SYSCLK] -multiply_by 2 –invert [get_pins MUT2_CLK]

【指定时钟网络的时序】

在目前的流程中，通常时钟网络的由后端的 P&R 完成，我们在综合时只是保证功能的正确。因为在综合时，工具不考虑触发器的摆放位置，而在后端工程师在 CTS （Clock tree synthesis），触发器的摆放已经确定。这样根据触发器的

具体摆放位置，进行时钟树的生成，可以更好的平衡触发器时钟的 Skew，Insertion Delay 等。下图示意中表现了综合以及 CTS。

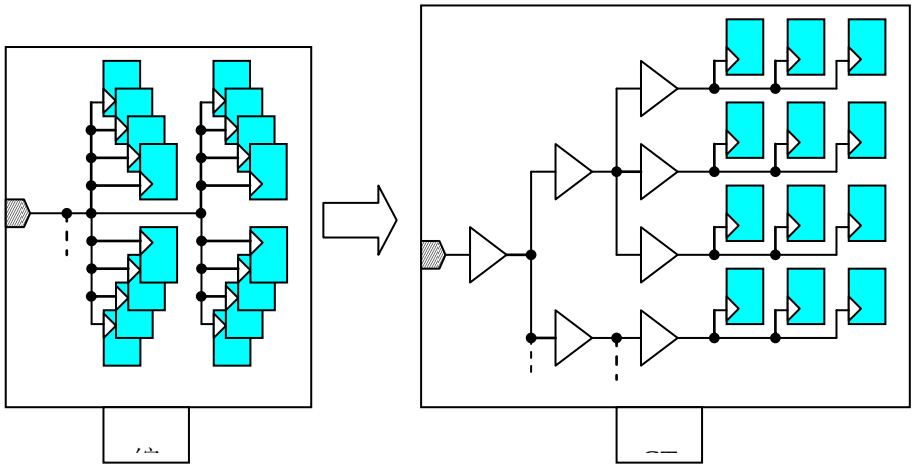


图4-21 综合和 CTS 后的时钟网络示意

在综合时，时钟网络是理想的，没有延时（延时为 0），没有 Skew（Skew 为 0），时钟的沿也是理想的（Transition 时钟为 0）。这样和 CTS 后的时序就有了很大的不一致。因此在综合时，我们尽管没有 CTS，但应该把以后这些 CTS 的特性表示出来，通常我们会使用下面这 3 个命令来表现。

A．set_clock_latency 命令

set_clock_latency：其语法如下：

set_clock_latency value [-rise] [-fall] [-min] [-max] [-source][-early] [-late] [clocks object_list]

使用 set_clock_latency 我们可以表现 CTS 的 Insertion Delay（就是时钟源点到触发器时钟 Pin 的延时）。例如，

dc_shell_t> set_clock_latency 4.0 [get_clocks SYSCLK]

表示我们定义了时钟 SYSCLK 的 Insertion Delay 为 4（ns）。

B．set_clock_uncertainty 命令

set_clock_uncertainty 其语法如下：

set_clock_uncertainty value [-from object_list -to object_list] [-rise] [-fall] [-setup] [-hold] clocks object_list

set_clock_uncertainty 描述了 clock 沿的不确定区间，因此我们可以使用该命令表现 CTS 的 Skew。例如，

dc_shell_t> set_clock_uncertainty 0.5 –setup [get_clocks SYSCLK]

使用 –setup 选项，是由于一般综合不考虑 Hold 的 Timing check，因为此时的物理位置、延时都没有最终确定，因此可能此时修复的 Hold，在 P&R 后就变的多余了。因此综合时只考虑最大延时（即 Setup check）满足时序要求。

表示在 Setup 检查时，Capture Clock 的时钟沿会提前 0.5（ns）。

由于在 CTS 后，CTS Skew 导致 Capture Clock 的沿可能提前，当然也可能滞后，但是不可预知，因此我们按照比较严格的约束。

使用 set_clock_uncertainty 也可以模拟时钟源的抖动，时钟源的抖动也是导致 Capture Clock 的沿可能提前或者滞后。例如，假设，时钟源的抖动是 0.2ns，CTS 的最大 Skew 为 0.5 ns，那么，我们在约束时钟的 Uncertainty 时应该为：

dc_shell_t> set_clock_uncertainty [expr 0.2 + 0.5] –setup [get_clocks SYSCLK]

C．set_clock_transition 命令

set_clock_transition 的语法如下：

set_clock_transition transition_value [-rise | -fall][-min] [-max] clock_list

由于实际的 CTS，时钟的上升、下降沿不可能是理想的，都可能存在一定的 Transition 时间，我们可以使用该命令定义时钟的上升、下降沿的 Transition 时间。例如：

dc_shell_t> set_clock_ transition 0.5 [get_clocks SYSCLK]

表示时钟上升、下降沿的 Transition 时间为 0.5(ns)。时钟的 transition 时间会影响触发器输出数据的 Delay。

下面的示意中表示了理想时钟，经过模型化后的形态。

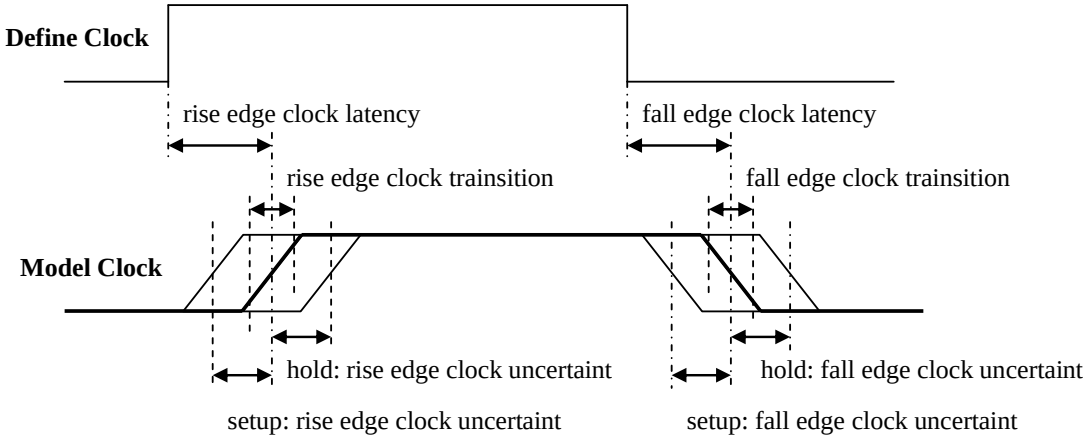


图4-22 综合时 CTS 的模型化

在 design compiler 中，还提供其它一些辅助命令，方便对时钟进行操作。

表4-6 Clock 操作的其它常用命令

命令	描述
all_clocks	列出存在的所有时钟
derive_clock	在时钟网络上回溯到时钟源点。
get_clocks	得到设计中时钟的集合
remove_clock	去掉一个或全部已存在的时钟
report_clock	报告时钟的相关信息

命令	描述
report_transitive_fanin	报告指定源的所有 fanin 逻辑
report_transitive_fanout	报告指定源的所有 fanout 逻辑。
set_clock_gating_check	对门控时钟信号设定 setup 和 hold 检查的裕度
remove_clock_transition	去掉时钟网络的 transition 时间
set_fix_hold	给时钟加上 fix_hold attribute， 使得在综合优化时解决 hold 问题。
set_dont_touch_network	给时钟网络加上 dont_touch attribute,使它不在综合时被改动。

以上几条命令，你可以通过在 design compiler 中查阅在线帮助获得具体用法。

(2) I/O 端口的时序设定

Design Compiler 默认 Input ports 信号的到达时间为 0，并且不对 Output ports 进行任何的约束。design compiler 提供 set_input_delay 和 set_output_delay，分别对对 I/O 端口的 Input Ports 和 Output Ports 进行时序约定。

A. set_input_delay 命令

set_input_delay 是设置设计的输入端口的延时相对于时钟的延时，延时值是外部逻辑的延时，语法如下：

set_input_delay delay_value [-clock clock] [-clock_fall] [-level_sensitive] [-network_latency_included] [-source_latency_included] [-rise] [-fall] [-max] [-min] [-add_delay] port_pin_list

例如下面的电路，

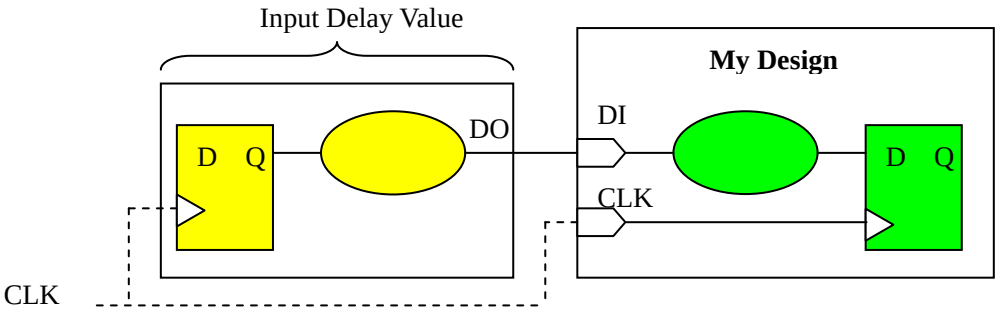


图4-23 Input Delay 设置示意

如果前级电路（芯片）的 DO 相对以时钟 Clk 的延时为 10 ns，那么我们可以设置输入的 Input Delay 为 10ns，在 DC 中设置如下，

dc_shell-t> create_clock –name clk –period 20 [get_ports CLK]

dc_shell-t> set_input_delay 10 –clock clk [get_ports DI]

延时值是在指定的目标（ DI）相对与时钟（–clock clk）的延时。

一般情况下，在不知道设计外面的 input delay 时，例如模块级综合，可设 input delay 为整个时钟周期的 40% ~ 60%。

B．set_output_delay 命令

set_output_delay 是设置设计的输出端口相对于时钟的延时。要特别注意，延时值也是外部逻辑的延时，set_output_delay 的含义如下图所示：

set_output_delay 的语法如下：

set_output_delay delay_value [-clock clock] [-clock_fall][-level_sensitive] [-network_latency_included] [-source_latency_included] [-rise] [-fall] [-max] [-min] [-add_delay] [-group_path group_name] port_pin_list

Output Delay 设置示意如下，延时值是设置片外的 Delay。

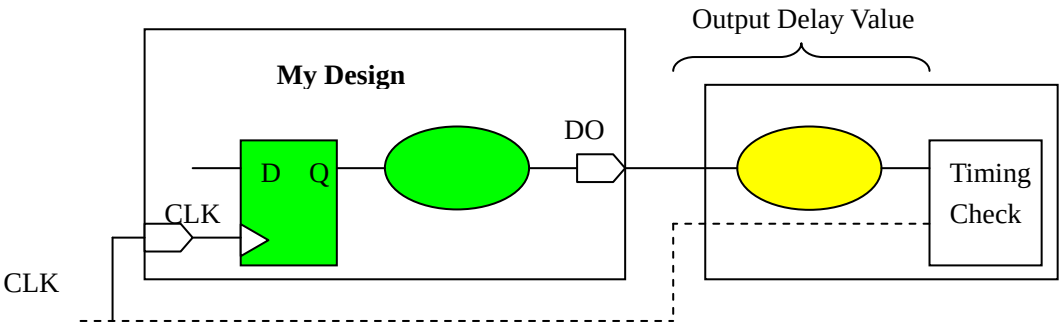


图4-24 output Delay 设置示意

例如：

dc_shell-t> create_clock –name clk –period 20 [get_ports CLK]

dc_shell-t> set_input_delay 10 –clock clk [get_ports DO]

通常大家感觉对 DC 中的 set_output_delay 理解起来很困难，下面详细的阐述一下。

假设片外延时为 tCO（也就是要设置的 Output Delay 数值），在上面示意的设计，如果输出 DO 相对于时钟（–clock clk）的延时设为 tD，时钟周期为 T，那么根据时序检查，应该满足下面的等式（假设时钟没有抖动）：

Setup Check：tD + tCO <= T

Hold Check：0 <= tD + tCO

在设置 Output Delay 时，终点的 Timing check 是没有建立/保持时间的。

那么：

tCO（MAX）= T – tD（MAX），

tCO（MIN）＝ 0 – TD （MIN）。

当 tCO 取值在 tCO（MIN）到 tCO（MAX）之间时，上面的等式恒成立。

假设对于我们的设计，要求时钟到输出的延时（tD），最大为 10ns，最小为 5ns，时钟周期为 20ns，那么：

MAX tCO <= T – tD（MAX） = 20 – 10 = 10ns;

MIN tCO = 0 – tD（MIN）= 0 – 5 = -5ns。

因此可以如下约束：

dc_shell-t> set_input_delay -max 10 –clock clk [get_ports DO]

dc_shell-t> set_input_delay -min -5 –clock clk [get_ports DO]

在模块级综合时，如果不清楚外面逻辑的延时，通常设置 Output Delay 为时钟周期的 80%。如果设计输出都是由寄存器输出，也可以不加 output delay 约束。

(3) 非同步设计的时序约束

design compiler 的时序约束和检查都是基于静态时序分析。在静态时序分析中，默认所有的路径都是基于单周期（Single-Cycle）的检查。单周期检查的含义就是源（source）触发器在一个时钟沿打出数据，那么接收（Sink）寄存器就要使用紧接着源触发器打出数据的有效时钟沿接收数据。

因此，对于一些复杂的逻辑，我们进行了多周期处理；在一些异步时钟域，通常我们要进行一些异步处理，等等。DC 无法侦测到这些，需要我们显式的定义，对于这些路径，不进行检查，或者按照其它的方式检查，我们通常称之 Timing Exception。有如下的命令：

表4-7 常用 Timing Exception 命令

命令	描述
set_false_path	设置指定的路径不进行时序检查
set_multicycle_path	设置 multicycle 检查
set_max_delay	设置路径的最大延时
set_min_delay	设置路径的最小延时

【false path】

使用 set_false_path 将指定的路径设置为 false path，这时 DC 将不对该路径进行时序检查，因此也就不会对该路径进行时序优化。当要注意，对于 DC ，设置了 false path，通常会增加 RunTime。

set_false_path 的语法如下：

set_false_path [-setup | -hold] [-rise | -fall] [-from from_list] [-through through_list] [-to to_list][-reset_path]

-from start_point : false path 的起始点，可以是设计的 input port 或内部寄存器的时钟输入；

-to end_ponit : false path 的终点，可以是设计的 output port 或内部寄存器的数据输入；

-through through_point： 定义 false path 的中间经过点。可以是内部任意 cell 的 pins。

在设定 false path 时，可以只定义 –from / -to / -through 中的任意一个，或者组合，-through 可以多次使用。

例如，所有到触发器 FFB1 的路径为 False Path

dc_shell_t> set_false_path -to [get_pins FFB1/D]

时钟域 clk_100 和 clk_33 之间为 False path，

dc_shell_t> set_false_path –from [get_clocks clk_100] –to [get_clcoks clk_33]

dc_shell_t> set_false_path –from [get_clocks clk_33] –to [get_clocks clk_100]

【multicycle path】

A．multicycle path 的概念

通常，我们的设计都是 single cycle 的。在电路设计中还会遇到 muticycle 时钟设计的时候。特别是在 data path 的设计时，往往寄存器间的组合逻辑不能在一个周期内完成，这时我们在设计上进行 multicycle 的设计，并且使用命令 set_multicycle_path 告诉 DC ，该路径是 multicycle 的。

set_multicycle_path 的语法如下：

set_multicycle_path path_multiplier [-setup | -hold] [-rise | -fall] [-start | -end] [-from from_list] [-through through_list] [-to to_list] [-reset_path]

path_multipler ： 定义 multicycle 时钟的具体周期数。

-setup | - hold ： 定义 multipler 是对 setup 检查还是 hold 检查。

-from start_point : 指定路径的起始点。

-through through_point: 指定路径的经过点。

-to end_point : 指定路径的结束点。

⚠ 注意：

如果不指定 –setup 和 -hold，默认是 -setup。-setup 检查时，工具同时将 Setup、Hold Check 沿向右移动 （path_multipler） -1 Cycles。

例如下面的电路，从 A 、B 到 C_result 是 2 周期的 multicycle 设计。

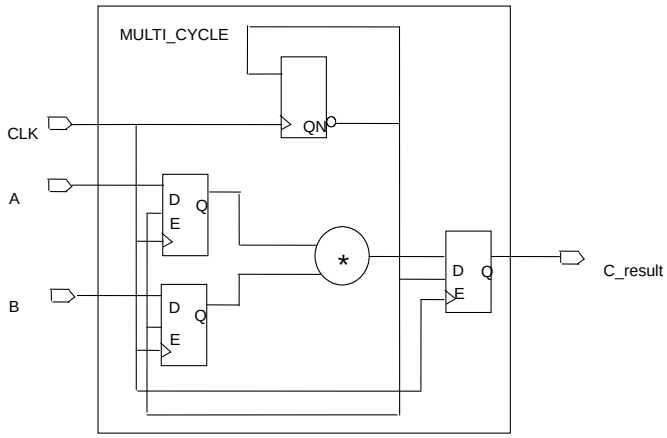


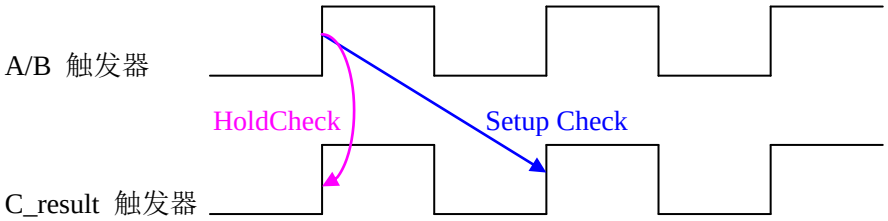
图4-25 multicycle 设计电路示意

对 C_result 寄存器设定 multicycle path

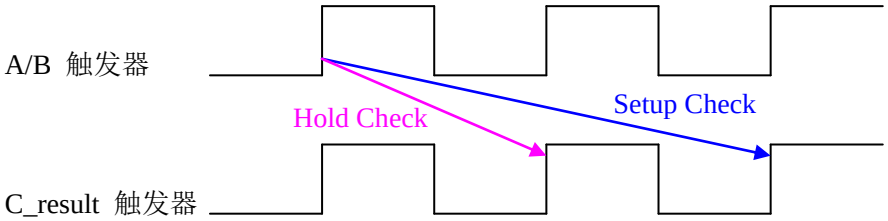
dc_shell-t> current_design MULTI_CYCLE

dc_shell-t> create_clock -period 5 [get_ports port CLK]

时序检查 Clock Edge 的关系如下：



对于到触发器 D Pin 的路径，此时 DC 默认的最大延时为 T- tSetup = 5 – tSetup。



dc_shell-t> set_multicycle_path 2 -setup -to [get_pins C_result/D]

此时，时序检查 Clock Edge 的关系如下：

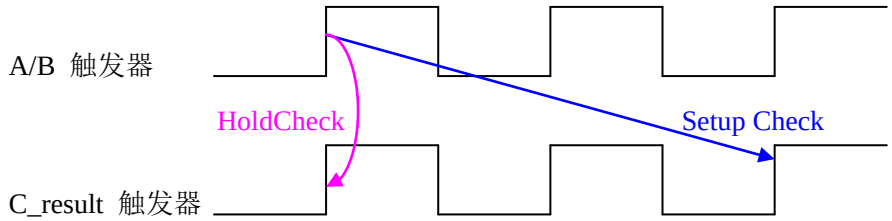
此时，对于到触发器 D Pin 的路径，此时 DC 默认的最大延时为 2T- tSetup = 10 – tSetup。

此时，Hold 检查（Min Delay），是在 Setup check Edge 的前一个沿进行。因此对于到 C_result D Pin 的最小延时为 T + tHold = 5 + tHold。

由于设计中 C_result 每隔一个周期就被 disable 一次，因此没有必要在时刻 5ns 处的时钟上升沿保持 数据的稳定性，Hold 检查应该是 Launch Clock 的同沿检查，即 Hold 检查应该在默认的沿在往前移动一个沿。

dc_shell-t> set_multicycle_path 1 –hold -to [get_pins C_result/D]

此时时序检查 Clock Edge 的关系如下：



到 C_result D Pin 的最小延时为 tHold，和设计要求的时序一致了。

通过下图（单时钟，上升沿有效），我们把 set_multicycle_path 的 path_multipler 在阐述一下，如下图所示：

首先我们我们通过命令，set_multicycle_path path_multipler –setup 确定 Setup Check 的发送（Launch）时钟沿和接收（Capture）时钟沿的关系，然后在通过命令，set_multicycle_path path_multipler –setup 确定 Hold 检查沿和 Setup 检查沿的关系，这样也就间接确定了 Hold Check 时的发送（Launch）时钟沿和接收（Capture）时钟沿的关系。

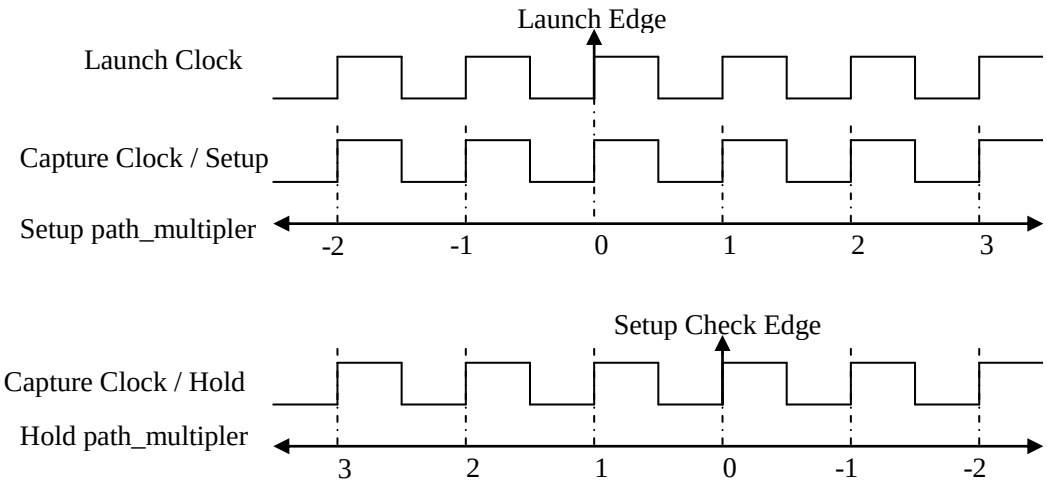


图4-26 set_multicycle_path 的 path_multipler 示意

【最大最小延时设定】

A．最大最小延时设定的用处

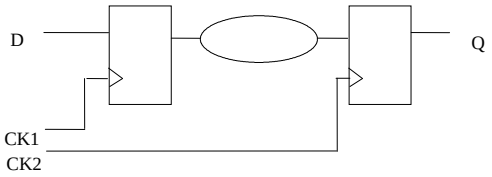
DC 提供两个命令，分别设定最大最小延时：set_max_delay 和 set_min_delay 。设定最大最小延时的用处:

- 对组合逻辑电路进行时序约束；
- 对异步时钟之间的信号进行时序约束。

DC 缺省会对异步时钟之间的信号进行 setup 和 hold 检查，但这种检查往往过严，常常造成大量无用的 violation。此时，可以使用 set_max_delay、set_min_delay 手工约束此路径。DC 会按设定的最大最小延时检查异步时钟之间的接口。

B．最大最小延时设定实例

一异步时序电路如下：



图中 CK1 为 50Mhz 时钟，CK2 为 49Mhz 时钟(假定都为理想时钟)，如果不设最大最小延时约束，design compiler 检查 setup 时，requirement time 会按 最小相位差 1ns (50ns -49ns) 再扣去器件的 setup 要求时间。实际组合逻辑不可能完成，于是最后 design time 会报告 setup violation。 根据设计要求，设定 max delay 为 6 ns，于是 design compiler 会按最大延时 6ns 约束中间组合逻辑 。

dc_shell-t> set_max_delay 6 –from [get_clocks CK1] –to [get_clocks CK2]

(4) 综合约束的确认

在设定了综合约束后，不是马上就进行 compile 了。因为对一个较大的设计来说，compile 一次的时间很长。综合约束的命令输入了，并不一定能正确加到设计上，所以，在运行 compile 命令之前，先确定 constraints 是否正确地加到设计上了是很有必要的，可以大大减少重新 compile 的风险，节省大量时间。

检查 constraints 设置的一些命令：

report_design

report_port

report_clock

report_constraints

report_timing_requirement

具体用法请查阅 design compiler 的在线帮助。

7. 解决 Multiple Instance 的问题

Multiple Instance 是指在上层设计中多次调用同一个子层设计单元，如图所示，U2/U3 和 U2/U4 都是实例化设计单元 C。

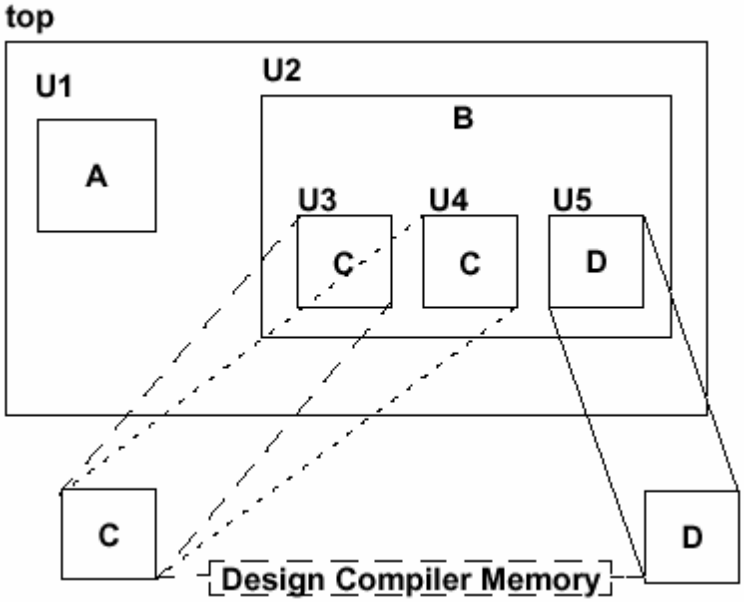


图4-27 Multiple Instance 问题示意

对于这种包含多次调用同一个设计单元的设计，在综合前必需给出处理办法，否则无法直接 Compile。常用的方法有 Uniquify，Don't-Touch 以及 Ungroup，下面分别介绍这几种处理方式：

(2) Uniquify

对于上层设计每次调用的各个子层设计，其外围工作环境各不相同，此时可采用 uniquify 方法，DC 给每次调用的子层设计建立一复本，以及唯一的名字。经过这样的处理，将消除 Multiple Instance 情况。如下图所示。

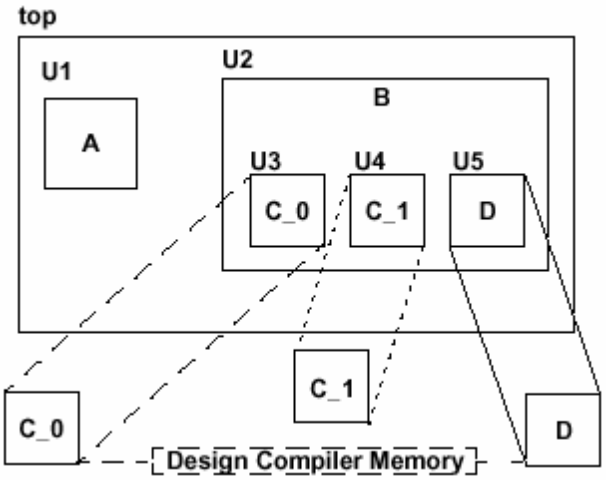


图4-28 uniquify 解决 Multiple Instance 问题示意

命令示意如下：

```
dc_shell-t> current_design  top
dc_shell-t> uniquify
```

dc_shell-t> compile

该种方法有一些缺点：

- 占用较大的内存。
- 花费较长的综合优化时间。

(3) Set_dont_touch

对于上层设计每次调用的各个子层设计，其外围工作环境相似，则可以先对该子层设计进行综合优化，再在对上层设计进行综合优化时，将子层设计设定为 dont_touch 。具体情况如图所示。

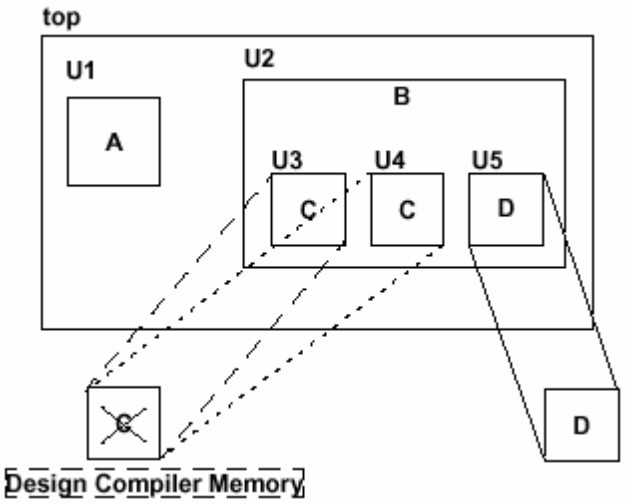


图4-29 Set_dont_touch 解决 Multiple Instance 问题示意

其具体操作方法如下：

- 对上层设计添加约束。
- 用最坏工作条件对子层设计进行 characterize。
- 综合优化该子层设计。
- Set_dont_touch 子层设计。
- 综合优化上层设计。

相应的 Script 文件为：

dc_shell-t>current_design top

dc_shell-t>characterize U2/U3

dc_shell-t>current_design C

dc_shell-t>compile

dc_shell-t>current_design top

dc_shell-t>set_dont_touch {U2/U3 U2/U4}

dc_shell-t>compile

该种方法具有以下优点：

- 对子层设计只需综合优化一次。

- 占用较少的内存空间。
- 占用较短的综合优化时间。

在子设计被引用的外围环境基本相同的情况下，建议使用 set_dont_touch 的方法。

(4) Ungroup

对于上层设计每次调用的各个子层设计，其外围工作环境各不相同的情况，也可采用 ungroup 的方法，这样就去除了设计的层次性。不过这种方法不常用。script 文件为：

```
dc_shell-t>current_design B
dc_shell-t>ungroup {U3 U4}
dc_shell-t>urrent_design top
dc_shell-t>compile
```

该方法将占用较多的内存资源和较长的综合优化时间，但能取得较好的综合结果。

下面我们将这三种方法进行对比，如下：

表4-8 三种解决 Multiple Instance 方法的比较

	Uniquify	Set_dont_touch	Ungroup
脚本复杂度	简单	复杂	简单
RunTime	较长	较短	较长
时序/面积	较好	一般	最好
层次	保留层次	保留层次	没有层次

由于 Ungroup 去除了设计的例化层次，对以后的网表级 Debug 很不方便，因此用的不多，除非是时序特别的紧张。通常在例化 DesignWare 时我们使用这种方法。

Set_dont_touch 脚本比较麻烦，综合的效果取决于例化环境的相似性，通常在底层小模块综合时用的不多。在顶层模块的综合时，由于 RunTime 时间太长，如果例化环境相似，采用这种方法。

Uniquify 用于综合效果好，而且保留了层次，在 RunTime 可以接受时，我们通常采样这种方法。

8. Compile

当环境、约束设定完成后，就可以使用 Compile 命令进行逻辑的映射和优化了。如下：

```
dc_shell-t > compile
```

compile 命令的有很多的选项，用于控制门级映射，也有很多的命令，如 set_flatten 和 set_structure ，可以用于逻辑优化的控制，使用 set_max_area 对面积优化进行控制。使用下面我们将详细介绍综合的逻辑优化的过程，对综合以及相关命令有更深刻的了解。

(1) 逻辑优化

逻辑优化包括下面三个步骤：

【architectural 优化】

architectural 的优化通常也叫做 high-level synthesis。它是基于 DHL 代码的。优化根据约束，代码风格，包括逻辑的共享，以及 DesignWare 的选择，算数运算的 Reorder 等。经过 architectura 优化，除了 DesignWare，整个设计都是用 GTECH 来表现的。

【逻辑优化】

逻辑优化包括两个步骤，structuring and flattening。这两个步骤都是基于 GTECH 的设计。我们可以认为布尔等式的优化。

structuring

Structuring 通过使用一些中间项进行逻辑化简。它是基于约束进行的，对速度和面积优化都很有用。在缺省情况下采用这种方式进行优化。其方式的设定用 set_structure 命令，命令格式如下：

set_structure true -timing true (-boolean true)

-timing ：基于时序约束对局部或全局的 structure 过程进行优化，对关键路径进行修正。适合于面积和速度优化。

-boolean ：基于布尔代数运算规则精简电路规模，仅适合面积优化。

flattening

flattening 使用两级的、最小项之和实现设计，去掉中间逻辑。它具有如下特点：

化简不依赖于约束。

面积较大，但比较适合于速度优化。

不会破坏原设计的层次性。

flattening 需要消耗较多的 RunTime，而且可能增加逻辑面积，在缺省情况下 flattening 是关闭的。可以使用 set_flatten 命令来设定 flattening 优化方式，命令格式如下：

set_flatten true -effort (low/medium/high) -minimize (single/multiple) -phase (true/false)

具体参数意义：

-effort ：是指综合工具用来 flattening 设计的能力等级。

low ：适合于大部分设计。如果设计较难 flatten ，综合工具就终止该过程，为缺省值。

medium ：能在重构设计的程度范围内 flatten 设计。

high ：该选项的使用比较危险，将占用很长的综合时间。

-minimize ：调整逻辑等式中的项目的数目和尺寸。

single ：每个输出信号都是输入信号的最小项之和式，为缺省值。

multiple ： 输出信号间可共享输入信号的最小项，目的可减少面积。

-phase ：可设置成 true 或 false ，缺省值为 false ，为 true 时，能求反化简以减小电路面积。

因为 flattening 的优化时间较长，综合出的电路面积较大，因此使用时要慎重。建议在如下情况下不使用 flattening：

多个 XOR 的电路如加法器、ALU 等

数据通道器件如多路器等

多于 20 个输入的电路。

希望保持其结构的设计。

set_flatten 和 set_structure 是基于模块的，因此，在层次化设计中可对不同的子模块，根据实际需要分别局部使用 set_flatten 或 set_structure 以达到设计的目的，避免全局使用。

如果对同一模块同时使用了 set_flatten 和 set_structure ，则综合工具先进行 flattening 过程，再进行 structuring 过程，在加工工艺的特征尺寸已小于亚微米的今天，面积已不是首要考虑的因素，故只使用 set_structure 。

【门级优化】

门级优化包过四个过程： mapping, delay optimization, design rule fixing, and area optimization.

Mapping

这个优化过程是用指定的 Target_library 中的 Cell ，来实现设计。

Design Rule Fixing

在这个过程中，DC 通过插入 Buffer，以及更换不同驱动能力的 Cell，来解决 Design Rules 的违反。

Delay Optimization

在这个优化过程，DC 解决由于 Mapping 引入的时序违反（violations）。在这个过程中不考虑 Design Rules violations 以及时序约束。在时序优化过程中，不会引入新的 Design Rules 问题。

Area Optimization

这时最后的优化过程，满足面积的要求。在面积优化的过程中，DC 不会引入新的 Design Rules、时序问题。

逻辑优化和门级优化都是通过 Compile 命令完成。当所有的约束满足，Compile 也就结束了。这时你可以写出综合后的网表，以及进行一些约束的分析。

(2) 面积约束

在 ASIC 设计中，面积和速度之间是一个 trade off 的关系。要获得最小面积，必定会牺牲一些速度。design compiler 在综合时，认为 timing constraints 比 area constraints 拥有更高的优先级。如果你关注于综合后的面积，那么可使用 set_max_area 命令设定面积的约束，使得 design compiler 在完成 timing 的优化后，继续进行面积的优化。例如：

dc_shell-t> set_max_area 0

注意，一些优化 timing 的命令可能会大大影响综合后的面积。例如，对 critical path 进行优化的命令 compile -map_effort high，虽然可以在一定程度上改善 timing，但同时也可能会使面积加大。此外，balance_register 作为一个 behavior compile 的命令，也有很大可能性让设计的面积膨胀起来。

(3) Compile 命令

Compile 命令实现逻辑层次以及门级网表层次的综合和优化，其语法、选项如下：

compile

[-no_map]

[-map_effort low | medium | high]

[-area_effort none | low | medium | high]

[-incremental_mapping]

[-exact_map]

[-verify]

[-verify_hierarchically]

[-verify_effort low | medium | high]

[-ungroup_all]

[-boundary_optimization]

[-auto_ungroup area | delay]

[-no_design_rule | -only_design_rule | -only_hold_time]

[-scan]

[-background run_name]

[-host machine_name]

[-arch architecture]

[-xterm]

[-top]

我们可以看到，其选项很多，各个选项的含义大家可以在 DC 环境中使用 man 命令察看，下面我们举一些例子。

【预综合】

为了要快速地估算出设计的面积和其它一些性能，我们可能希望对设计的面积、速度有一个大致的估算，这时在综合时可以使用如下的选项：

dc_shell-t> compile -map_effort low

可以不加任何约束或只加时钟约束，这样可以耗时最少。注意这时综合出来的电路只能作为估计之用，不能作为最终结果。

【缺省综合】

缺省综合是指综合命令后面不带任何选项，采用了缺省的 map_effort 为 medium ，以及 structuring 启用而 flattening 关闭的属性进行电路的优化。对电路的正式综合都是从这一步开始的。采用的命令如下：

dc_shell-t> compile

【SCAN 的综合】

由于触发器替换为 SCAN 触发器后，对时序，逻辑面积都会产生一定的影响。因此对于要进行 SCAN 测试册设计，在综合时，我们希望 DC 选用带 SCAN 的触发器。

dc_shell-t> compile -scan

【二次综合】

如果缺省综合的结果不能满足时序要求，则可在此基础上进行 -map_effort 为 high 并加上-incremental_mapping 选项的二次综合以达到设计目标。采用的命令如下：

dc_shell-t> compile -map_effort high -incremental_mapping

注意-map_effort high 是用来解决关键路径时序不满足的，它耗时比较长，因此一定不要从一开始就采用这一选项，而要先用缺省的 compile 进行优化。如果想节省综合时间，可以将已满足时序要求的模块设置成 dont_touch 。

【顶层综合】

当完成底层模块综合，进入顶层时，通常我们将已经综合好的底层模块直接 Link 起来，如果此时仍不能满足时序要求，有少量的 Violations，可以使用如下的命令进行优化。

dc_shell-t> compile -top -boundary_optimizatio

9. 输出报告，网表

综合完成后，我们需要察看综合结果是否满足我们的时序要求，可以使用如下的命令：

dc_shell-t> report_timing

dc_shell-t> report_constraint

dc_shell-t> report_area

如果要输出综合的网表，可以使用 Write 命令，该命令也支持多种文件格式的网表，通常我们要保存 DB 和 Verilog 格式的网表，如下：

dc_shell-t> write –f db –h –o TOP.db

dc_shell-t> write –f verilog –h –o TOP.db

10. 综合策略的选择

层次化设计的综合优化策略有以下两种：自顶向下（Top-down）和自底向上（Bottom-up）。下面分别介绍这两种综合策略。

(1) 自顶向下

这时比较简单，而且综合效果最好的策略，当通常会受到内存以及综合时间的制约。通常如果综合时间小于 8-10 小时（一个晚上），推荐采用这种方法。这里是这种策略的优点：

- 简单，Script 少
- 综合工具自动分析内部逻辑的关系，满足约束的要求。
- DC 可以自动分析计算穿越层次路径的 Delay，Load，Fanout 等。
- 脚本有较好的通用性，移植方便。
- 优化效果较好。

综合步骤以及脚本示意如下，

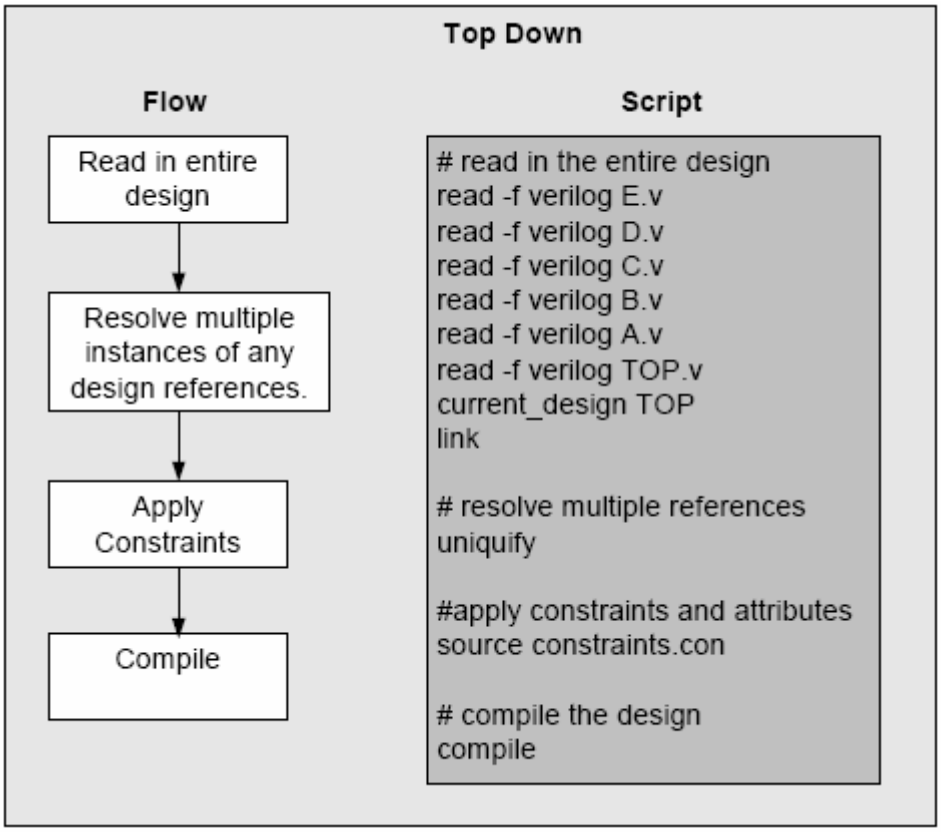


图4-30 Top-down Script

(2) 自底向上

由于运行时间，内存等的考虑，不能采用自顶向下的策略，而使用自底向上策略，这种策略有如下的优点：

- 可以根据不同模块的不同特点和要求单独进行优化。
- 对内存要求较小并且可以在多个机器上并行执行。
- 当某个模块作修改后不必重新综合整个设计，减少了综合的时间。
- 当然这种方法的确定也是显而易见的，
- 需要进行多次迭代直到各模块的接口稳定下来。
- Script 很多，需要手工进行版本的控制和管理，通用性不好。
- 综合的效果一般没有自顶向下好。

自底向上的过程通常如下，

1. 对各子模块使用估计的约束进行综合 ，并保存其 db 文件。
2. 读入所有已经综合后的设计的 db 文件。
3. 将顶层设计设为 current_design ，然后 characterize 某个子模块。
4. 用 write_script 命令生成该子模块的新约束文件。
5. 清除内存。
6. 读入刚被 characterize 的子模块，用新产生的约束文件综合，并保存其 db 文件。

7. 读入整个设计的 db 文件，但不包括旧的子模块，而是重新用上一步新产生的子模块代替。

8. 选择 另一个子模块，重复步骤 3~7 直到所有的子模块都重新综合过。

用自底向上的方法综合设计实例的 Script 文件，这里假设各子模块的约束文件已经存在。

Bottom-Up Script

```
set all_blocks {E,D,C,B,A}
#compile each subblock independently
foreach (block, all_blocks) {

#read in block
set block_source block + ".v"
read_file -format verilog block_source
current_design block
link
uniquify

#apply global attributes and constraints
source constraints.con

#apply block attributes and constraints
set block_script block + ".con"
source block_script

#compile the block
compile
}

#read in entire compiled design
read_file -format verilog TOP.v
current_design TOP
link
write -hierarchy -output first_pass.db

#apply top-level constraints
source constraints.con
source top_level.con

#check for violations
report_constraint
```

```
# characterize all instances in the design
set all_instances {U1,U2,U2/U3,U2/U4,U2/U5}
characterize -constraint all_instances

# save characterize information
foreach (block, all_blocks) {
current_design block
set char_block_script block + ".wscr"
write_script > char_block_script
}

# recompile each block
foreach (block, all_blocks) {

# clear memory
remove_design -all

# read in previously characterized subblock
set block_source block + ".v"
read -format verilog block_source

# recompile subblock
current_design block
link
uniquify

#apply global attributes and constraints
source defaults.con

# apply characterization constraints
ser char_block_script block + ".wscr"
include char_block_script

# apply block attributes and constraints
set block_script block + ".con"
source block_script

# recompile the block
compile
}
```

图4-31 Bottom-up Script

(3) 如何选择综合策略

单纯从综合的效果来说，自顶向下的方法更有优势，因为可以在更大范围内调整、规划时序。但随着现在设计规模越来越大，如果采用纯粹的自顶向下策略进行综合会造成运行时间过长，而且有可能造成机器内存不够。因此对于大规

模的设计应采用自底向上的策略或混合的策略。根据一些资料并结合我们自己的经验，给出如下的选择综合策略的建议：

1. 在设计的规模小于 10 万门，用自顶向下的方法综合耗时不超过 3、4 个小时而且设计对时序要求比较紧的情况下，应采用自顶向下的方法。这样可以充分发挥这种方法优化效果好，Script 文件简单的优点，又不会花费太多的时间。

2. 当设计在 10~30 万门之间，且电路代码已经基本稳定时，也可考虑用自顶向下的方法。这时虽然耗时较长（可能要 10 个小时），但由于不需要频繁地重复综合，可以利用下班或晚上的时间进行，所以也是可以接受的。

3. 当设计在 30 万门以上，尤其是代码还不稳定时，应该采用自底向上的策略或混合的策略。我们不建议使用纯粹的自底向上策略，因为步骤太繁琐且优化效果不好。一般都采用混合的策略，即将设计划分成几个规模适中且要求相似的子模块，对每个模块进行自顶向下的综合，综合时间控制在 1、2 个小时以内。子模块的综合完成后将整个设计在顶层连接起来（link 命令），如果顶层没有逻辑（建议这样做）且约束已经满足则综合就可以完成了。如果还有不满足的地方可以在顶层用 compile -inc 或 compile -top 进行增量的优化调整。这两个选项的主要区别是后者不进行元件的 map，只进行优化，因此耗时很少。在顶层没有逻辑的情况下推荐用 compile -top。采用这种混合的策略进行综合，既保证了优化的效果，又不会花费太多的时间，同时还便于电路的修改和局部的重新综合，并且可以在多台机器上同时运行，进一步减少了运行时间。

4.4 静态时序分析 Static Timing Analysis

概述：重点为基本概念、实现流程、脚本编写，注重实用性和可参考性

对于逻辑系统的开发而言，在芯片送去加工之前都会遇到 2 个基本的时序问题：

- 设计是否能够稳定工作在期望的工作频率？
- 如果设计的工作频率上不去，瓶颈是什么？

STA 是一种在芯片加工之前分析逻辑电路时序是否满足的方法。它可以检查所有时序器件是否存在时序问题，并能够给出存在时序问题的完整路径，可以有效的解决上述 2 个问题。

4.4.1 STA 基本概念

1. 常用术语

- Best Case：也称作“Fast Chip”，即芯片内部各路径延时最小。这种情况在芯片最低温度、最高电压且 fastest process 工艺时出现。
- Worst Case：也称作“Slow Chip”，即芯片内部各路径延时最大。这种情况在芯片最高温度、最低电压且 slowest process 工艺时出现。
- /Arrival Time：到达时间，信号到达逻辑中某一点的时间。设计中每个门的输入输出点的上升沿和下降沿都会计算到达时间，而且每一点都会计算

2 个不同的到达时间，一种是 early 模式，另外一种是 late 模式。在所有情况下，到达时间都是在 50％电压点计算的。

- RAT/ETA：“Required Arrival Time”/“Expected Time of Arrival”。要求到达时间。在时序检查点，信号在 RAT 约束的时间范围之内到达时认为该点时序检查通过。ETA 由用户在基本输出端口指定，STA 工具根据这些指定的值反向计算内部各测试点的 RAT。
- Slack：要求到达时间与到达时间之间的差值，也被称作“timing margin”。Slack 值为负就表示该路径时序有问题，必须修正掉。
- 斜坡时间（Slew Time）：在实际电路中一个信号从逻辑 0 跳变到逻辑 1，或从逻辑 1 跳变到逻辑 0，由于实际电路寄生参数（电阻、电容等），不可能是 90 度，而是肯定有一个斜率。在时序分析时，我们把这个斜率用时间来衡量，也就是 Slope Time。Slope Time 的定义就是信号在翻转时，对于下降沿，就是从上限域值点到下限域值点的时刻差；对于上升沿，就是从下限域值点到上限域值点的时刻差。上限域值点通常定位为高电平的 90%，下限域值点通常定义为高电平的 10%。Slope Timing 也称作 Transition Time 或者 Slew Timing。
- 时钟波型（Waveform）：描述一个理想时钟周期的时钟上升沿（逻辑 0 跳变到逻辑 1）、下降沿（逻辑 1 跳变到逻辑 0）的到达时间（输入延时为 0，相位和占空比由设置的到达时间控制）。
- 时钟抖动（clock jitter）：理想时钟的时钟沿位置确定。但实际上时钟沿的位置是在理想时间沿附近一个区间内抖动，即时钟抖动。在时序分析时需要考虑时钟的抖动。

2. 时序路径

在逻辑设计中包含 4 条时序路径，如图 4-32 所示。

- 输入管脚－>组合逻辑－>寄存器
- 寄存器－>组合逻辑－>寄存器
- 寄存器－>组合逻辑－>输出管脚
- 输入管脚－>组合逻辑－>输出管脚

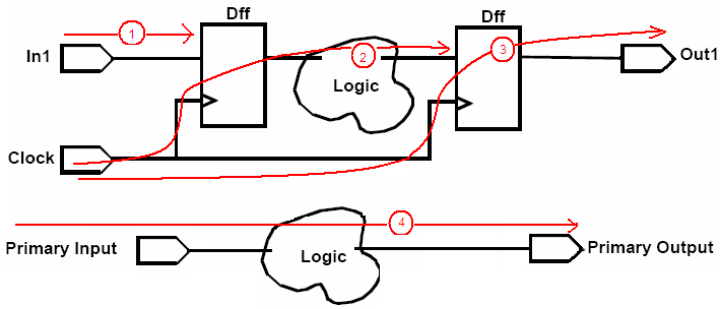


图4-32 基本时序路径

3. 信号传输

(1) Arrival Time（达到时间）

AT 是沿信号传递方向向下传递的。如图 4-33 所示，输入管脚的到达时间为 0 时刻，中间每个门延时 1ns，可以得到 test point 点的最早/最晚到达时间。

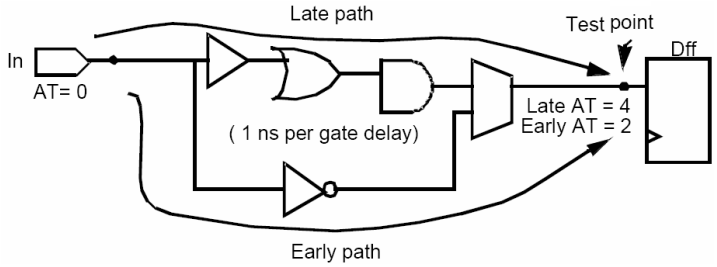


图4-33 到达时间传递

注意：到达时间计算的起点都是芯片的输入管脚，图 4-32 中路径 2 和路径 3 到达时间计算的起点都是 Clock 输入管脚。输入管脚到达时间是由 SI 工程师分析上游芯片的输出延时、板上延时及芯片输入管脚 PAD 延时后给出的输入管脚最早/最晚延时。STA 工程师根据 SI 提供的输入延时约束芯片的输入管脚。

(2) Request Arrival Time（要求到达时间）

RAT 的计算是沿信号传输的反方向向后计算的。如图 4-34 所示，test point 点的要求到达时间为 6，沿信号传递的方向反向计算，可以得到输入管脚的最早/最晚要求到达时间.

STA 工具会自动根据每个引脚的 AT 和 RAT 值计算出该点的 Slack 值，以判断该点时序是否满足。

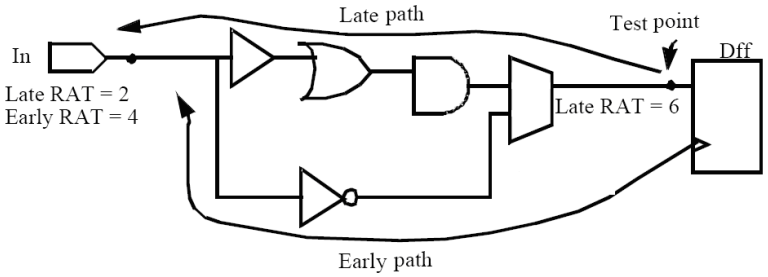


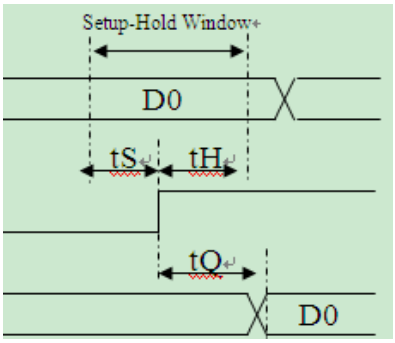
图4-34 要求到达时间传递

注意：要求到达时间计算的起点为存储单元的输入点、时序单元的输入点和芯片的外部输出管脚。其中输出管脚的要求到达时间由 SI 工程师分析下游芯片的输入建立时间要求、板上延时及芯片输出管脚 PAD 延时后给出的输出管脚最早/最晚要求延时得到。存储单元的输入点和时序单元的输入点的要求到达时间由 STA 工具分析该存储单元和时序单元时钟到达时间及器件内在建立保持时间后自动计算。

4. 建立保持时间

触发器是边沿触发是时序元件，理想情况是采样时钟有效点时的数据。由于工艺、寄生参数、触发器结构等原因，因此，要求的被采样数据必需有一个稳定区间，保证数据可以正确的被触发器采样。通常我们把这个要求的稳定区间称

为 Setup-Hold window。我们把 Setup-Hold window 和时钟沿对应起来，把 Setup-Hold window 分解为两部分，建立时间（Setup Time）和保持时间（Hold Time）。触发器的建立时间和保持时间的含义如图 4-35 所示。



建立时间（tS）：

指触发器时钟有效沿前，数据必需保持稳定不变的时间。

保持时间（tH）：

指触发器时钟有效沿后，数据必需保持稳定不变的时间。

图4-35 触发器的建立时间和保持时间

当触发器在接收数据时，满足了建立时间和保持时间，认为触发器在时钟沿 tQ 时间后，可以可靠输出该数据。如果触发器的建立时间不满足，或者保持时间不满足，那么触发器不能稳定的接收该数据，我们认为此时触发器处于亚稳态。

检查建立时间时，数据路径到达时间选择延时最大的数据路径传递（称为“late”模式），时钟路径到达时间选择延时最短的时钟路径传递（称为“early”模式）。这种情况下得到的送给触发器的数据建立时间最恶劣，如果在这种情况下 setup 检查通过，则该检查点不存在 setup 问题。如图 4-36 所示。

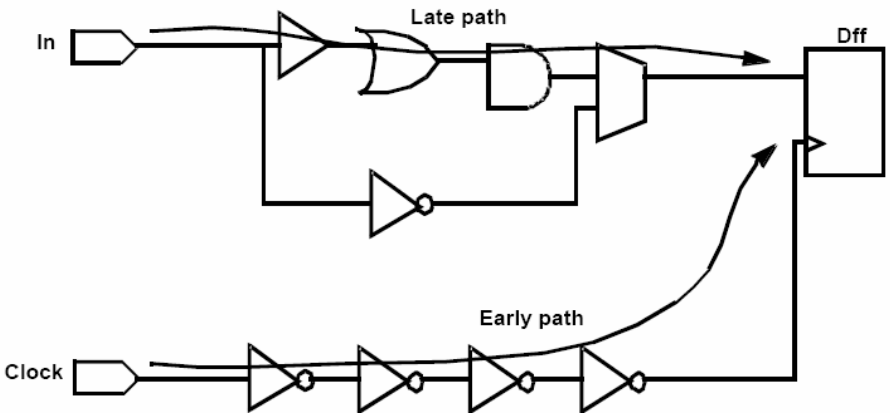


图4-36 STA 分析中建立时间检查

检查保持时间时，数据路径到达时间选择延时最短的数据路径传递（称为“early”模式），时钟路径到达时间选择延时最长的时钟路径传递（称为“late”模式）。这种情况下得到的送给触发器的数据保持时间最恶劣，如果在这种情况下 hold 检查通过，则该检查点不存在 hold 问题。如图 4-37 所示。

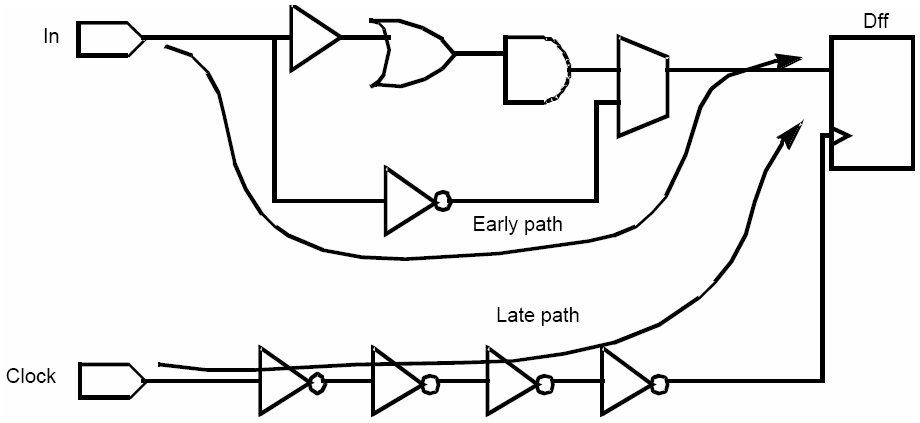


图4-37 STA 分析中建立时间检查

5. 时钟抖动对时序的影响

Cycle-to-cycle jitter（周期抖动）

周期抖动会缩短（或增长）相邻某个时钟周期的长度。时钟周期缩短（或增长）时只需要考虑 setup 检查，不需要 hold 检查。在同一时钟域中，hold 检查在同一个时钟沿发生，所以不会由什么问题。

注意图 4-38，由于周期抖动导致某个时钟周期长度缩短，第二个时钟上升沿发生在 9.9ns 处，而不是 10.0ns 处。此时 setup 检查就会比理想时钟下检查出来的结果更恶劣，但也更切合实际场景。



图4-38 Cycle-to-cycle jitter 抖动波形

4.4.2 STA 工具 Einstimer 与脚本介绍

目前业界流行的 STA 工具有 2 个，IBM 的 EinsTimer 目前主要用于 IBM 流程，其它 Vendor 基本上都使用的是 Synopsys 的 PrimeTime。本节主要介绍 EinsTimer 的使用及时序约束文件编写。

1. Einstimer 工具使用

IBM 的后端工具都集成在 theguide 中。本节只介绍 Theguide 图形界面的启动及参数设置，以及如何从 theguide 图形界面中启动图形化的 EinsTimer。并介绍 EinsTimer 工具中参数的设置和如何利用 EinsTimer 快速准确的分析、定位时序问题。

注意：使用 ET 进行时序分析之前，需要确认已经安装正确的 ToolKit 版本以及 Vendor 库。

(1) Theguide 启动及参数设置

第一步：图形界面 Theguide 启动：

```
/net/szxfs15-lnx/local/w46022/HI001/signoff1> theguide-10.1 -proj HI001.proj -gui&
```

Proj文件所处目录

Theguide版本（目前使用11.1）

工程文件

第二步：Thguide 启动完成后会出现两个界面：theguide 界面（工作界面）和 theguide console（运行的 log）界面。

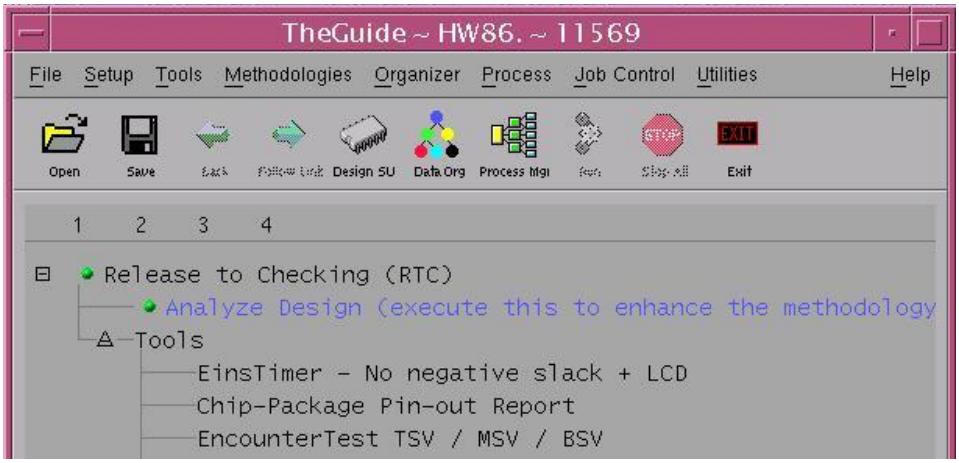


图4-39 Theguide 工作界面

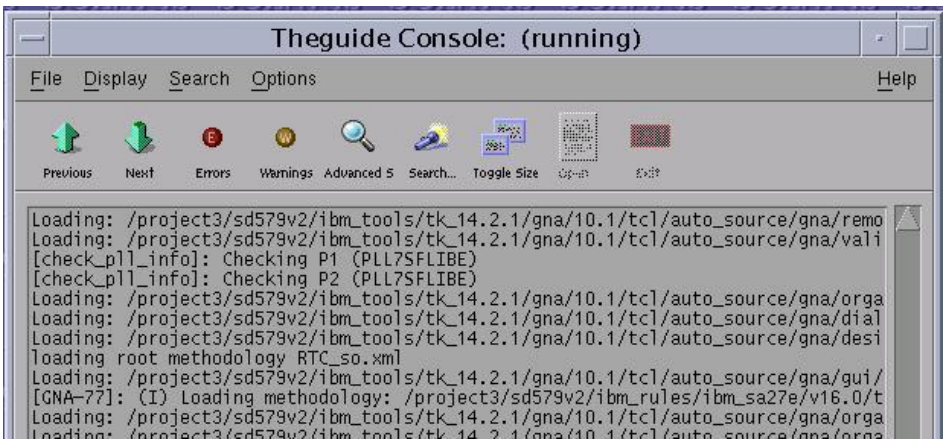


图4-40 Theguide console 界面

第三步：Theguide 中常用的菜单项介绍

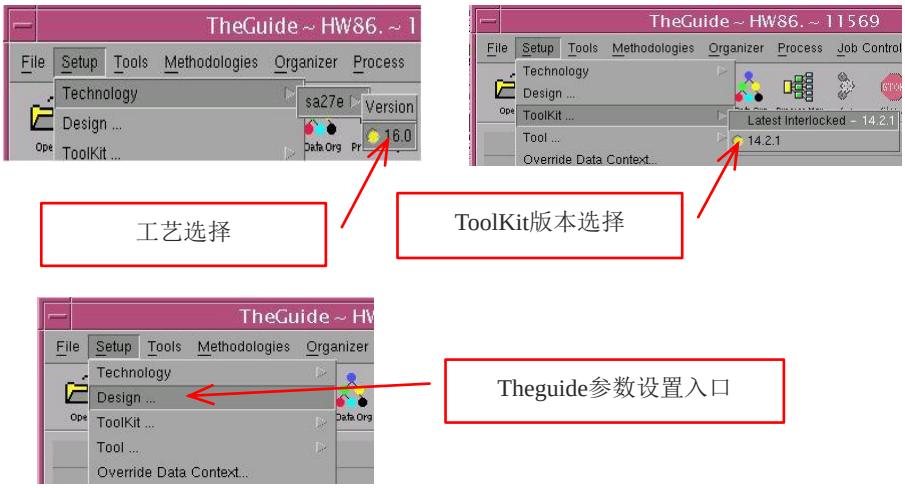


图4-41 Theguide 中常用的菜单项

第四步：Theguide 基本参数设置：

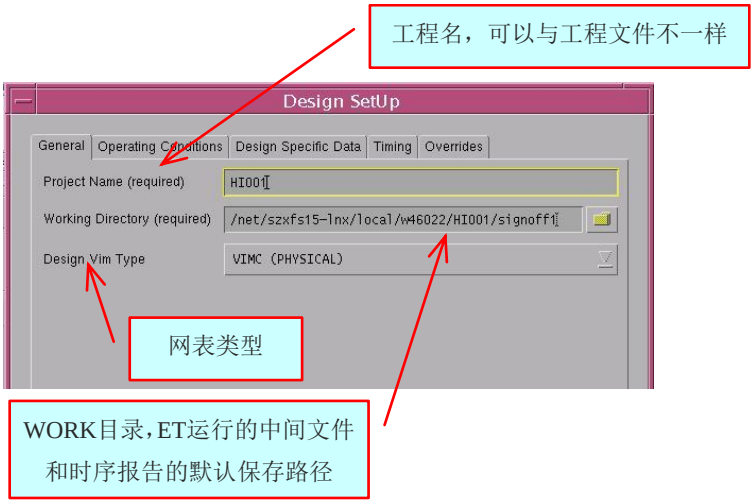


图4-42 Theguide 工作路径设置

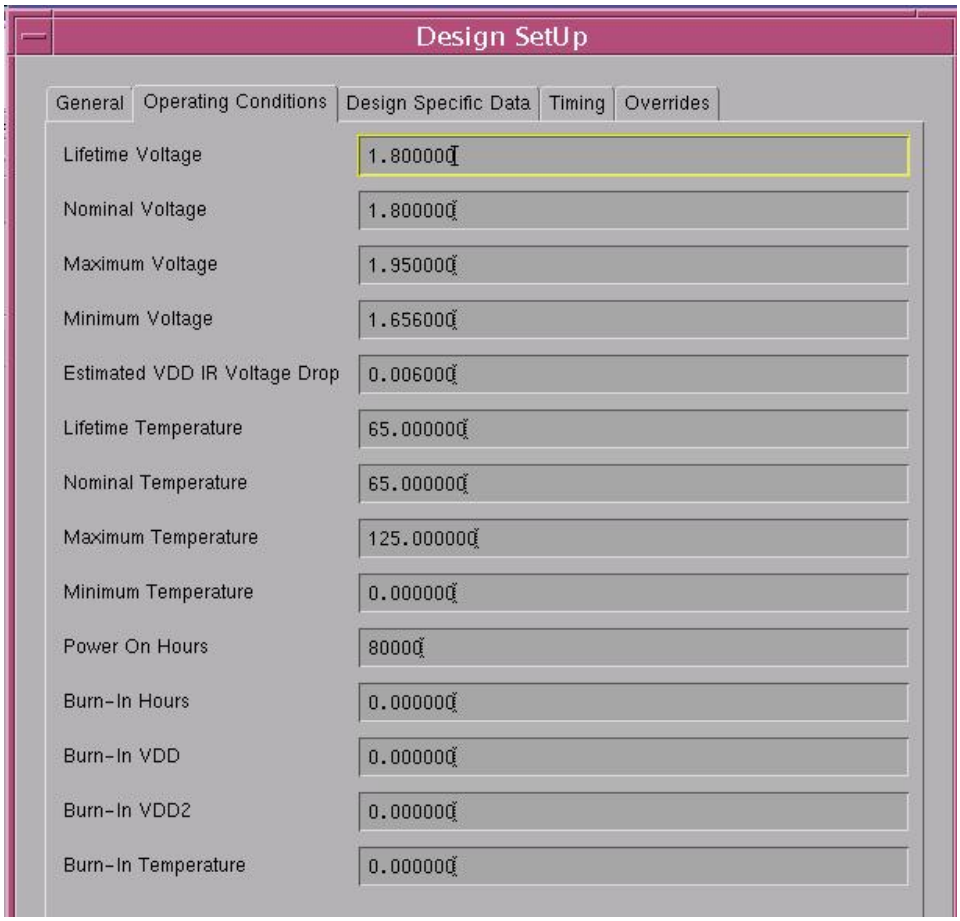

Design SetUp
General
Operating Conditions
Design Specific Data
Timing
Overrides
Lifetime Voltage
1.800000
Nominal Voltage
1.800000
Maximum Voltage
1.950000
Minimum Voltage
1.656000
Estimated VDD IR Voltage Drop
0.006000
Lifetime Temperature
65.000000
Nominal Temperature
65.000000
Maximum Temperature
125.000000
Minimum Temperature
0.000000
Power On Hours
80000
Burn-In Hours
0.000000
Burn-In VDD
0.000000
Burn-In VDD2
0.000000
Burn-In Temperature
0.000000

图4-43 环境参数设置

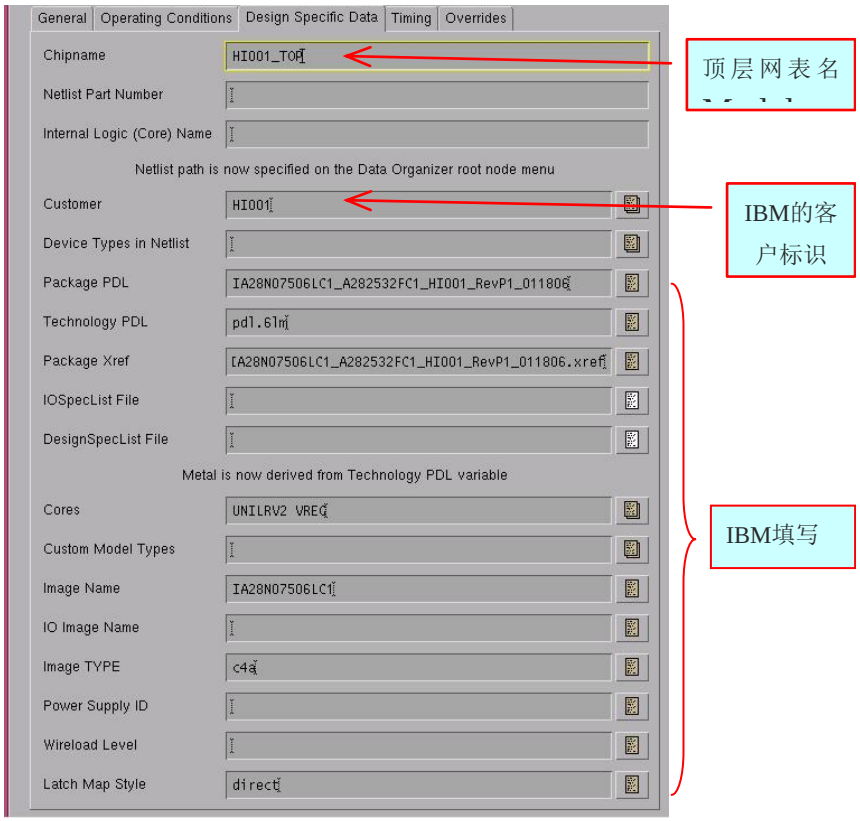


图4-44 设计相关参数

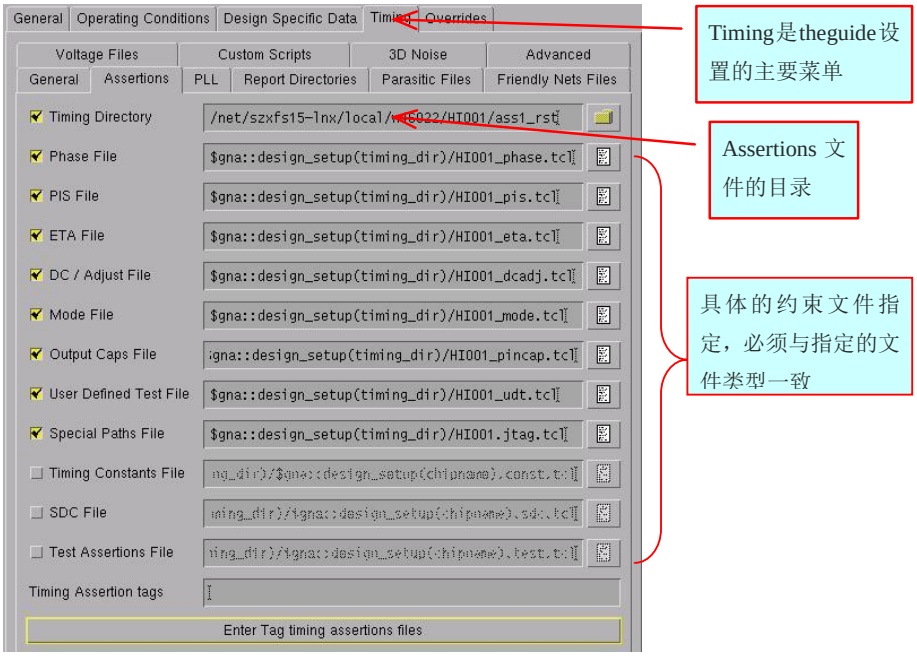


图4-45 时序约束文件路径设置

图 4-46 为 PLL 简要设置信息，PLL 的详细设置需要根据实际选用的 PLL 的型号、工作频率等进入“Enter PLL info”界面进行设置，此处不再介绍。

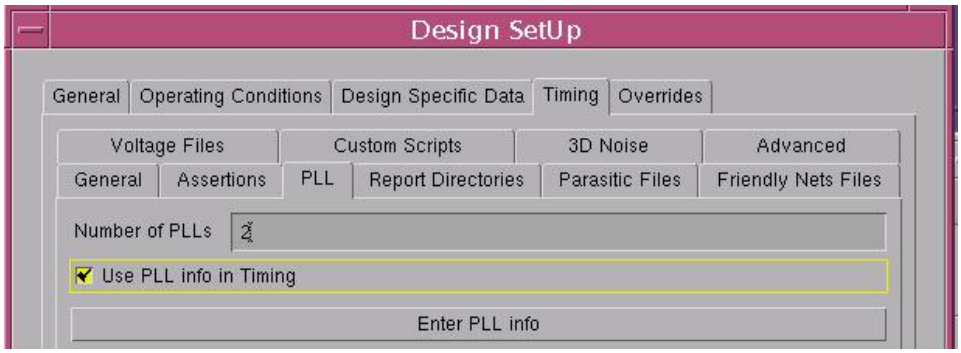


图4-46 PLL 信息设置

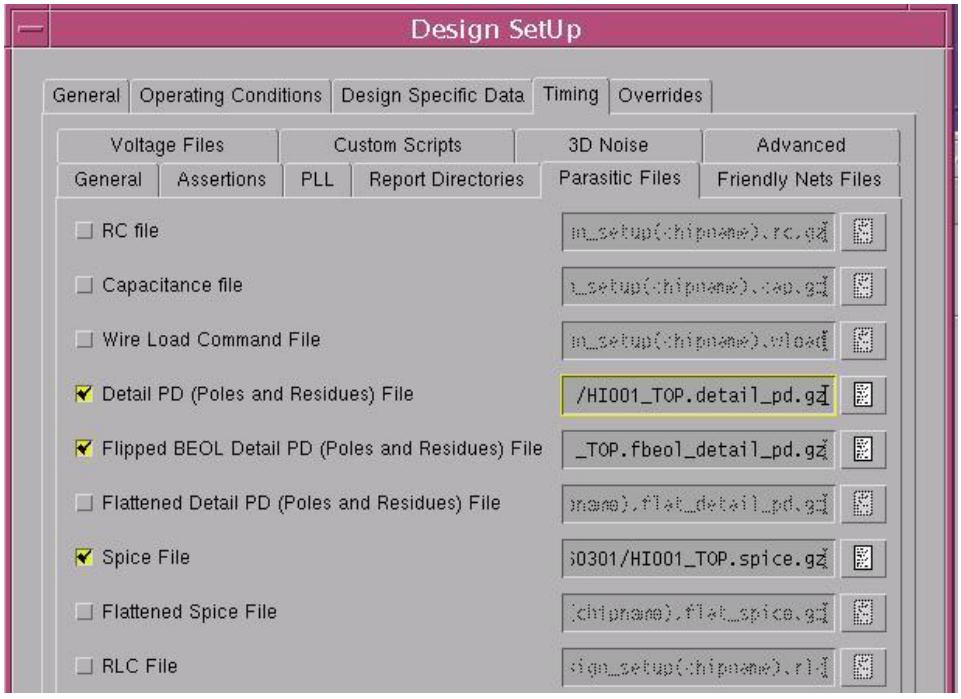


图4-47 寄生参数文件路径设置

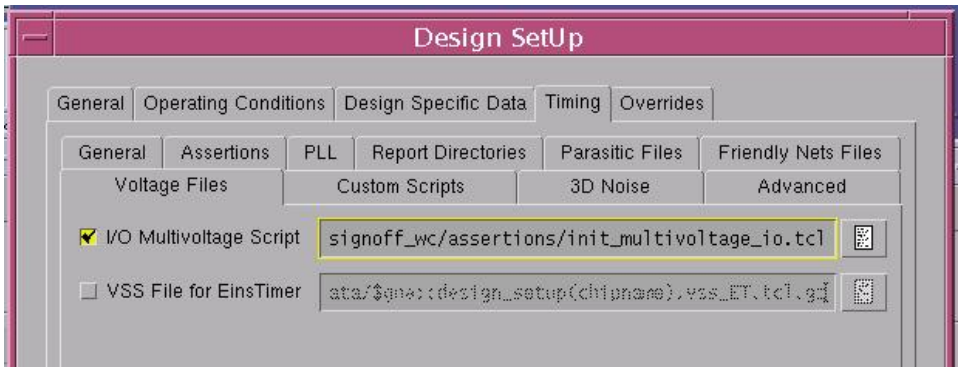


图4-48 多电压文件路径设置

⚠ 注意：

Timing 选项共有 10 个子选项，上面介绍了 4 个常用子选项的设置，在有些工艺下还需要设置 3DNoise 文件的路径，其余选项取默认值。

图 4-49 给出了库文件设置界面，用来设置 rule 的路径，包括通用库路径和定制 RAM 库路径。

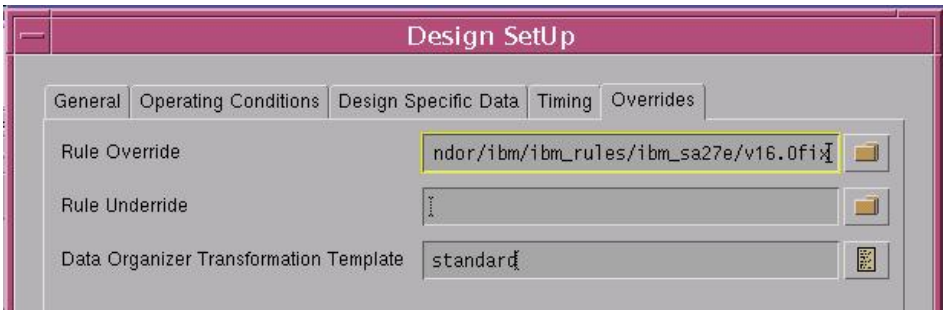


图4-49 库文件路径设置

第五步：Methodologies release 模式选择。

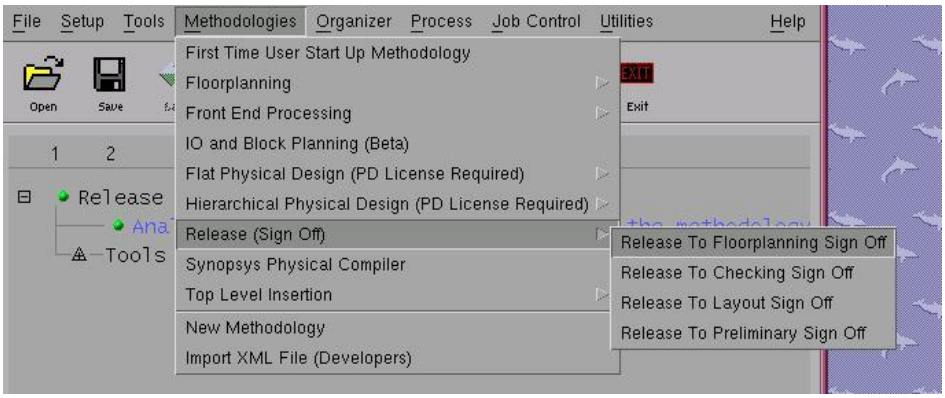


图4-50 RTF、RTL 等 Methodologies 选择

第六步：Data Organizer 设置

Data Organzier 是比较重要的一个设置，它指示了文件存储目录（即 WORK 目录），网表存放位置等必要信息，如果 Data Organzier 有问题，则无法启动 ET，错误信息如图 4-51 所示。

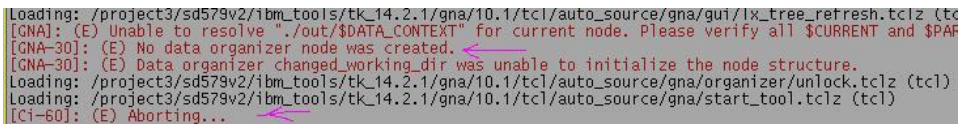


图4-51 Organzier 设置错误时提示信息

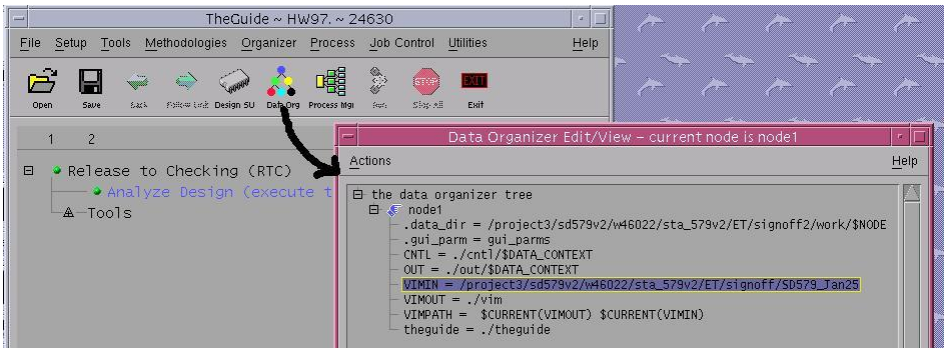


图4-52 正常的 Data Organizer 设置

注意：Data Organizer 中 VIMIN 这一项需要手动指定，它是网表的存放目录。如果 Data Organzier 出现问题，则右键图 4-52 中的“the data organizer tree”，然后选择”delete”删除现有的 Data Organizer。

建立新的 Data Organizer，如图 4-52 所视，右键“the data organizer tree”，选择“create”，这时弹出一个窗口，选择“OK”后，弹出如图 4-53 所示的窗口：

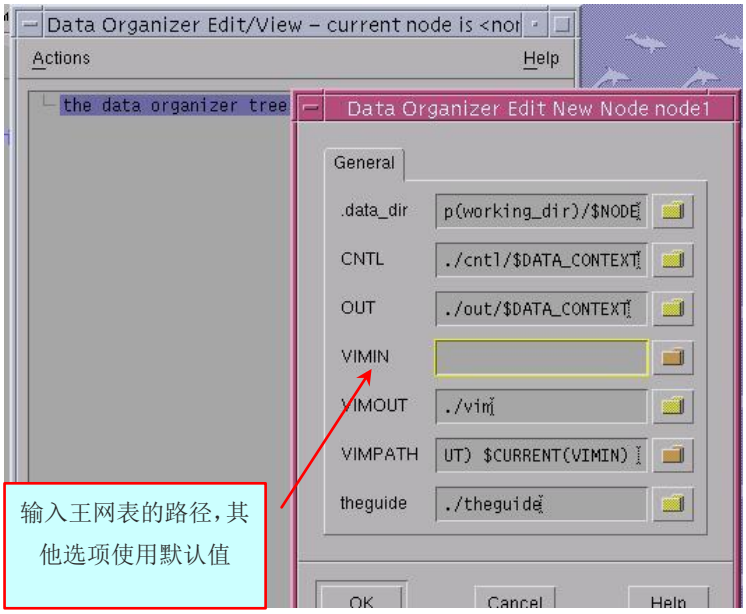


图4-53 建立新的 Data Organizer

(2) ET 启动及参数的设置

第一步：ET 启动

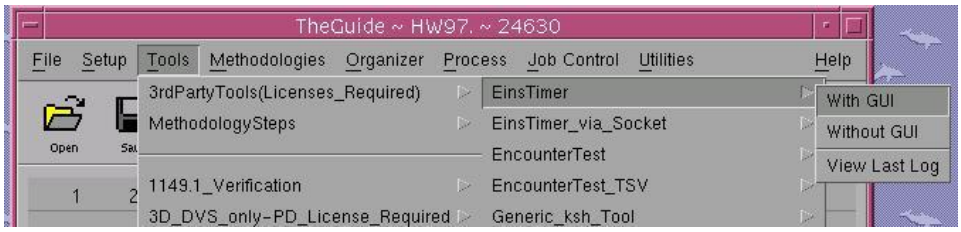


图4-54 Theguide 中 ET 启动选项

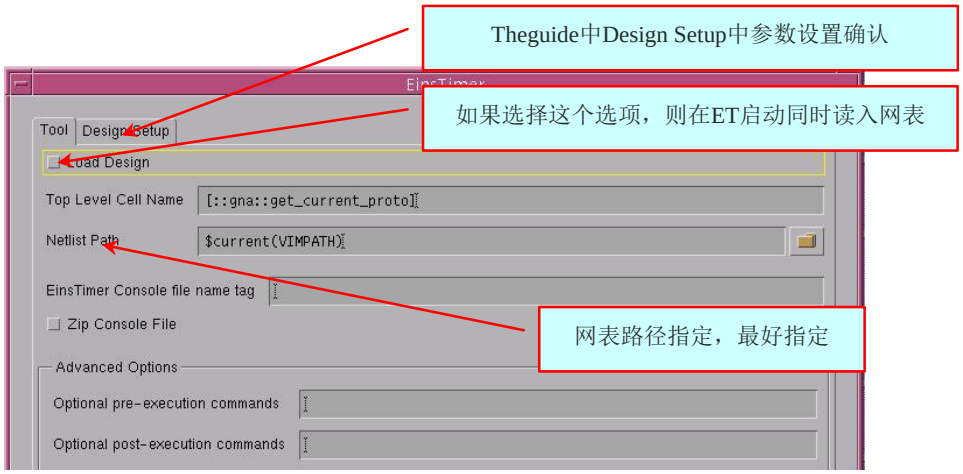


图4-55 ET 启动参数设置

完成上面的设置，点击“OK”启动 ET。

第二步：ET 图形界面介绍

ET 启动后，会弹出两个窗口：ETConsole 窗口与 Einstimer 窗口。其中 ETConsole 窗口完成类似 log 的功能，它显示 ET 执行的所有的命令，告警，错误以及提示信息。

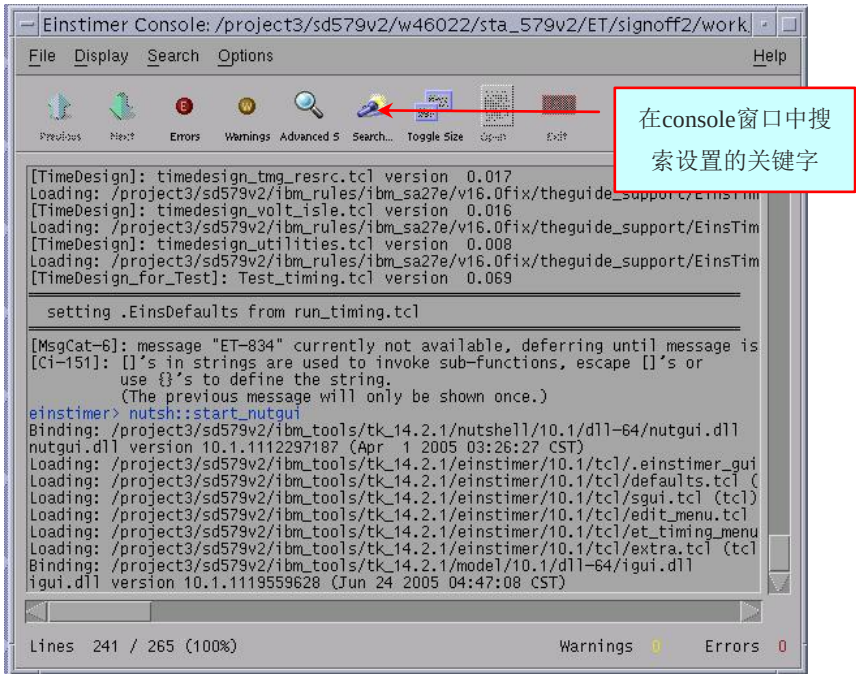


图4-56 ET console 窗口

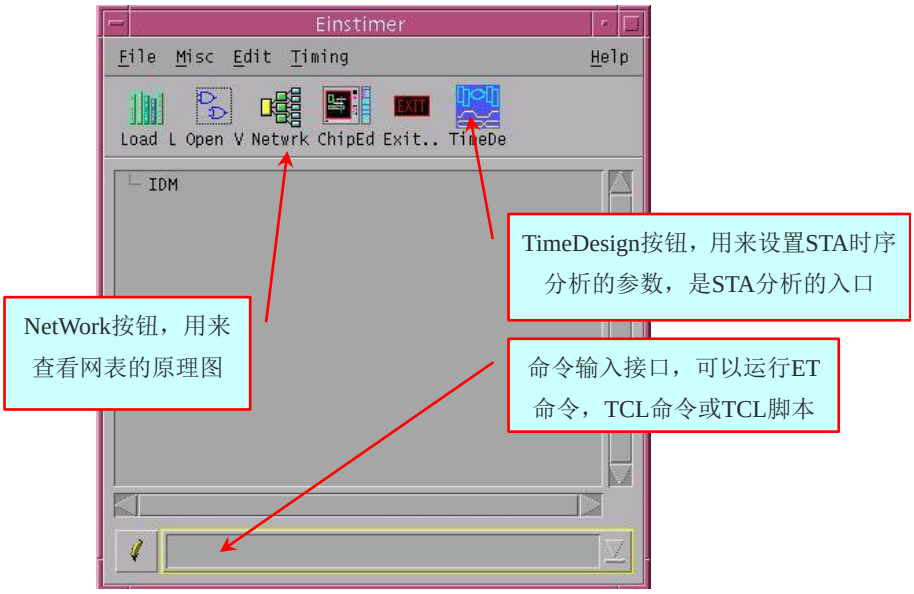


图4-57 ET 工作窗口

第三步：运行 STA

点击 Einstimer 的 TimeDesign 按钮（如图 4-57 所示），就会出现 STA 时序分析参数设置窗口。这些参数设置完成后，就可以进行 STA 分析。在后端的各个阶段，STA 参数设置不一样，主要表现在寄生参数的应用上；在每个阶段，STA 的分析策略也不一样，可以是 BC（Best case），也可以是 WC（Worst case）。TimeDesign 窗口如图 4-58 所示。

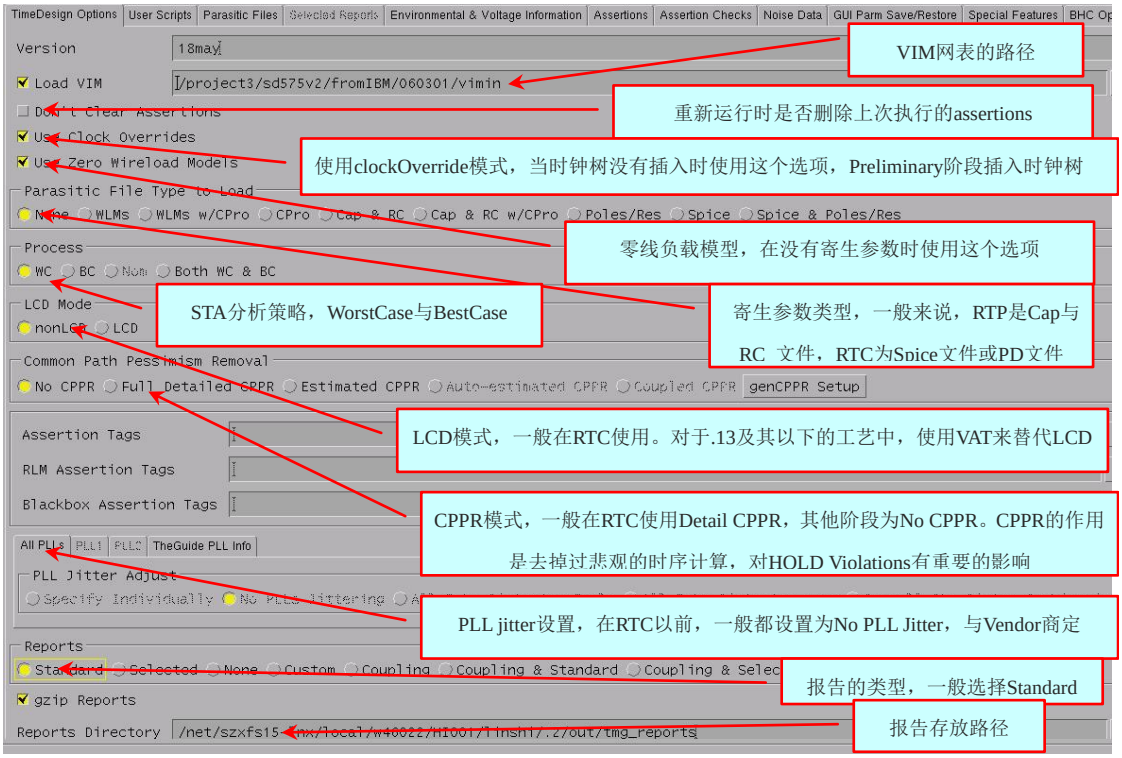


图4-58 时序分析参数设置

TimeDesign 有多个选项，其中 TimeDesign options（图 4-58 所示），是最重要的设置。其他选项含义大致为：

- user script：用户自定义的 assertions，其执行的时间点有严格定义，一般不常用。
- Parasitic Files：寄生参数路径设置，会默认沿用 Theguide 中的设置。
- Selected Reports：图 4-58 中，如果 Reports 选择 Selected 模式，则需要在此手动选择输出的 Reports 类型。Reports 的设置会在图 4-59 中详细描述。
- Environmental & Voltage Infomation：确认温度环境参数，设置多电压 assertions 的路径。会默认沿用 Theguide 中的设置。
- assertions：设置 assertions 文件的路径。会默认沿用 Theguide 中的设置。
- Assertion Checks：这是个非常有用，但是我们现在的项目不使用的选项。它能提供许多有用的信息，比如说时钟进入触发器的 D 端等。
- Noise data：VAT 使用，.13 及其以下工艺使用。
- GUI Parm Save/Restore：可以使用默认选项，一般不关心。
- Special Featurs：特殊的选项，一般在 STA 初期使用它的 TCL Debug 功能，SA27 类工艺下选择“Don’t use any Metal Violation”
- BHC opt：一些优化选项，由 Vedor 来完成

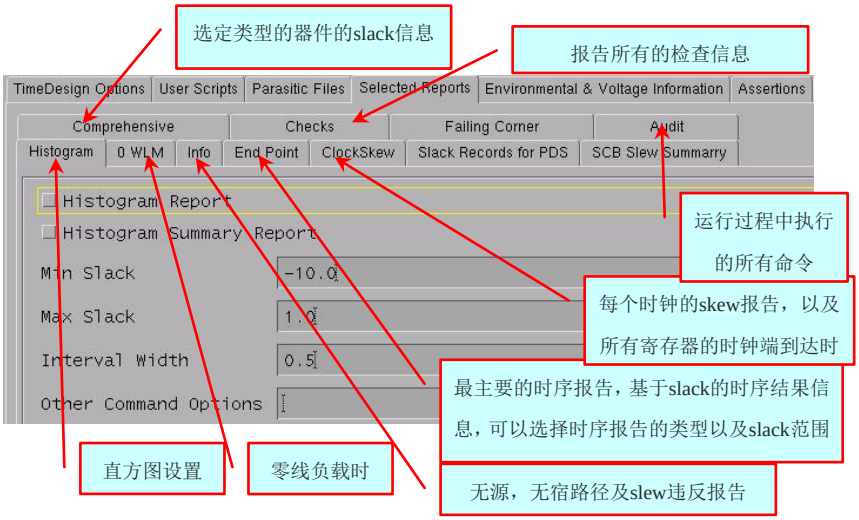


图4-59 输出报告设置

上述选项设置完成后，点击“OK”就可以运行时序分析了。ET 运行的时间可能会比较长，所消耗的内存也可能较大。ET 的运行过程会记录在 ETConsole 上，包括所有的提示，告警或者错误。

(3) NetWork 使用

在 NetWork 中可以方便地看到电路原理图，常用的功能有：查找，追踪，查看时序信息。

点击图 4-60 所示的 Netwrk 按钮启动 NetWork 工作界面。

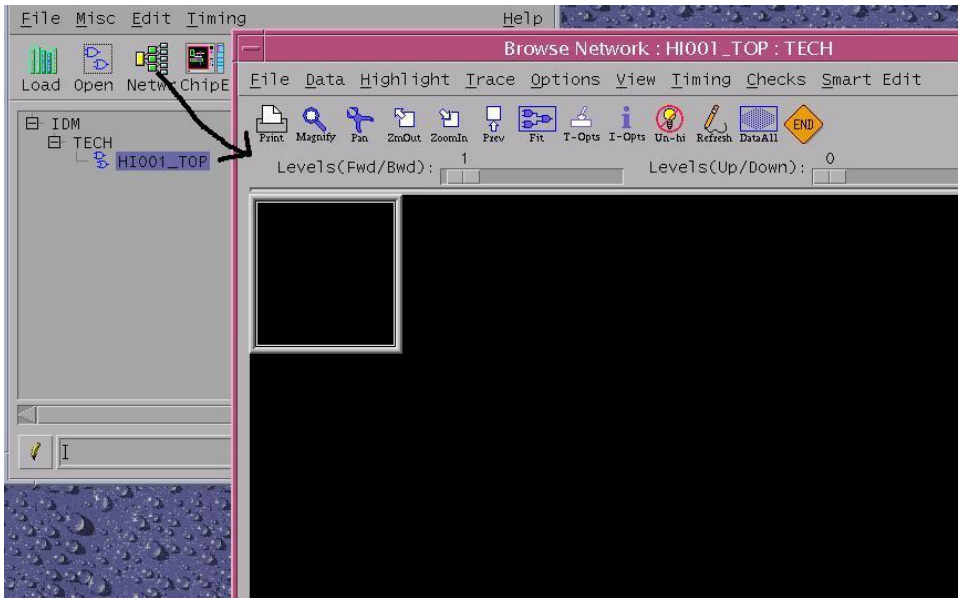


图4-60 NetWork 工作界面

启动搜索某个器件，如图 4-61 所示，进入搜索界面并设置搜索选项。设置完成后点击 Search 按钮，搜索结果就会显示在 Found Objects 所示的框中。选择需要的器件双击，该器件就会显示在 Browse NetWork 窗口中。

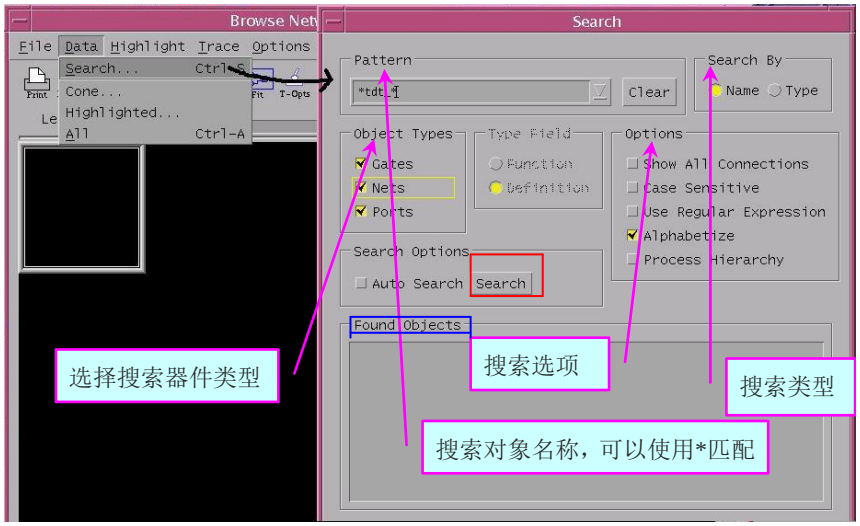


图4-61 搜索界面设置

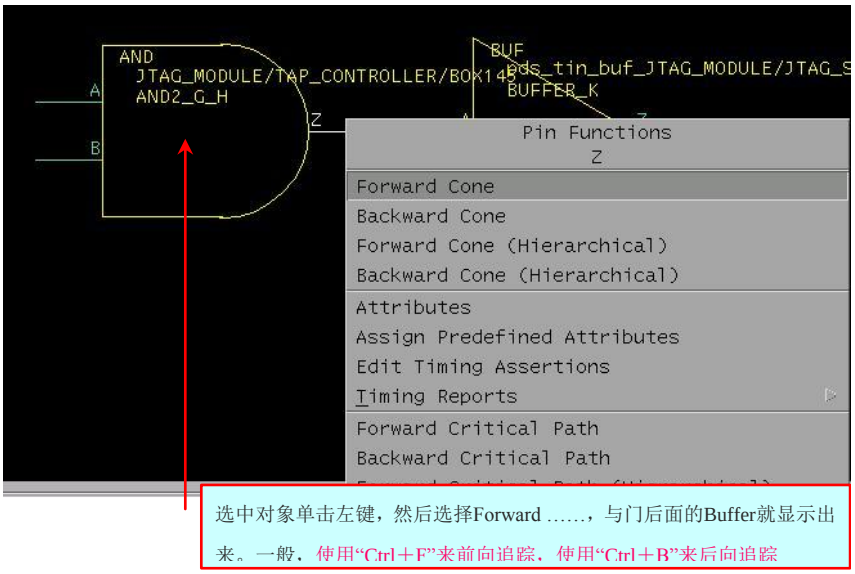


图4-62 电路连接关系追踪

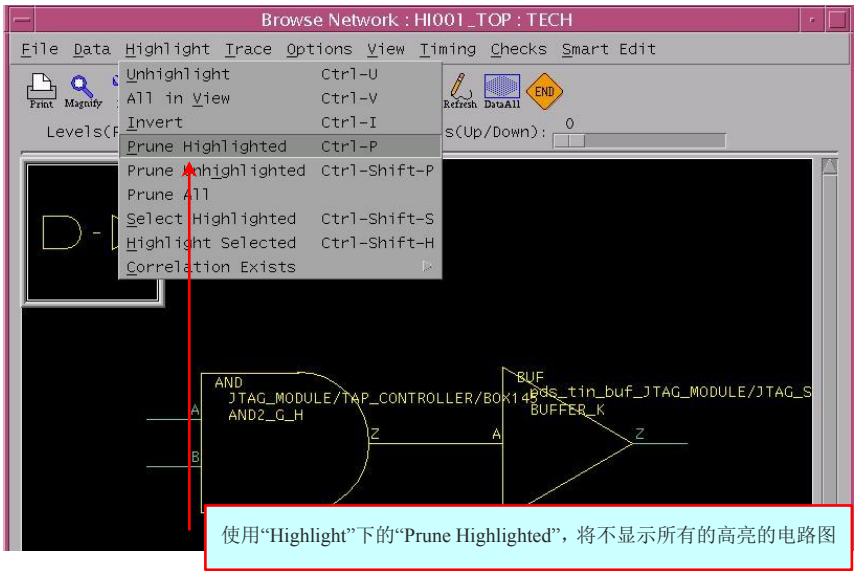


图4-63 去掉窗口显示的某个器件

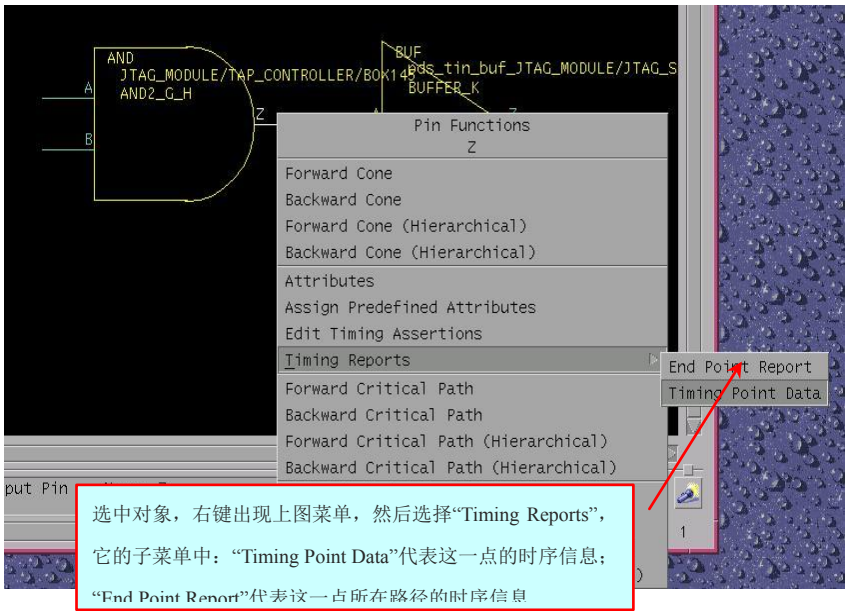


图4-64 查看器件的时序信息

NetWork 在定位时序问题及后期作 ECO 时非常有用，是 ET 提供的较常用功能之一。

2. 时序约束文件介绍

用户需要提供的 ET 时序约束文件有 7 个，分别约束时钟、输入管脚、输出管脚、输出负载、内部时序无关路径和时钟调整、时序模型及自定义的测试。

- Clocks： .phase.tcl
- Primary Inputs： .pis.tcl
- Primary Outputs： .eta.tcl
- Output Loading： .pincap.tcl
- Don’t Care and Adjust： .dcadj.tcl
- Timing Modes： .mode.tcl
- User Defined Tests： .udt.tcl

(1) Assertion: .phase.tcl

在 EinsTimer 中，每个时钟域对会被分配一个相位标签。如图 4-65 所示，图中有 3 个时钟域，则需要有 3 个时钟标签与之对应。

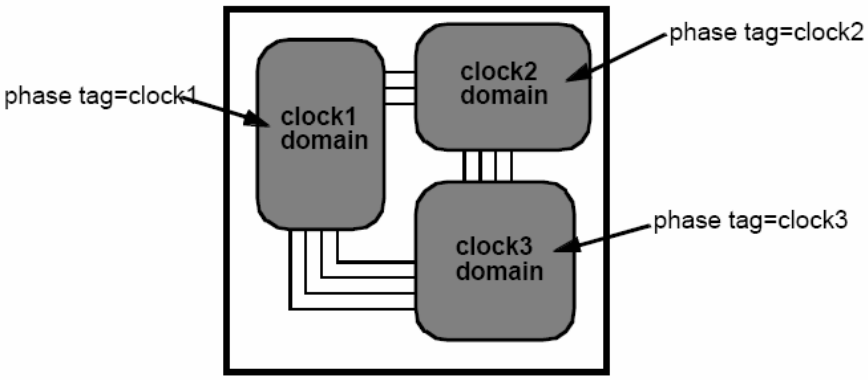


图4-65 时钟域标签划分

.phase.tcl 文件的简单例子

```
#-------------------------------------------------------------------------
#相位标签及理想时钟波形定义
#-------------------------------------------------------------------------
et::create_clock -name "CLK" -period 20.0 -waveform { 0.0 10.0 };
et::create_clock -name "CLK80" -period 12.5 -waveform { 0.0 6.25 };
#-------------------------------------------------------------------------
#为理想的时钟网络设置 override 时钟延时
#-------------------------------------------------------------------------
et::override_clock -latch -name "CLK" -time 0.3 -slew 0.50 -lead;
et::override_clock -latch -name "CLK" -time 10.3 -slew 0.50 -trail;
#-------------------------------------------------------------------------
#禁止没有相位关系的时钟域之间的时序检查
#-------------------------------------------------------------------------
et::set_phase_pair_exclusion -clock1 CLK -clock2 CLK80
```

Phase.tcl 定义每个时钟域的相位标签及特性，主要包括 4 个方面：

- 相位标签和理想时钟波形
- 为理想的时钟网络设置 override 时钟延时
- 禁止异步时钟域和相互排斥的时钟域之间的时序检查
- 加入微小的时钟波形调整模拟晶震的抖动

【相位标签】

EinsTimer 根据每个信号的相位标签计算路径上的时序信息，每个信号有一个或多个相位标签。

- 相位标签的名称部分在.phase 文件中定义
- 相位标签的名称长度没有限制（建议不大于 8 个字符）
- 相位标签的名称应该带有时钟域或信号名称信息

- 相位标签的后缀有特殊含义（@L 表示前沿, @T 表示后沿）

信号类型	相位	含义
Clocks	'+' or '-' suffix	时钟相位，表示时钟是高电平有效还是低电平有效
Data	'@L' or '@T' suffix	表示数据是在时钟有效电平的前沿还是后沿有效
Asynchronous	NONDTA	异步信号，与时钟没有关系

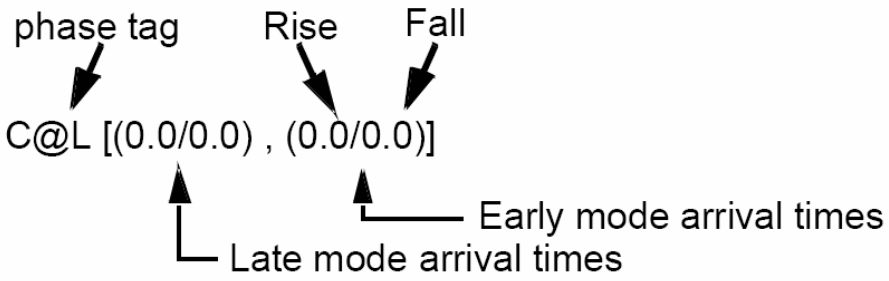


图4-66 数据标签

【相位标签的定义分 2 个步骤】

Phase.tcl：定义相位标签的名称和理想波形。

Pis.tcl：为每个输入管脚绑定适当的相位标签，ET 工具通过相位标签的后缀区分一个输入管脚是时钟信号还是数据信号。。

Sample (.phase.tcl)

et::create_clock -name "CLK1" -period 20.0 -waveform { 0.0 10.0 };

-name 定义了时钟标签名称；-period 定义时钟周期，单位 ns；-waveform 定义时钟跳变沿到达时间（时序检查时时钟调整的依据）。图 4-67 给出了 CLK1 的两种相位波形图和跳变沿的定义：

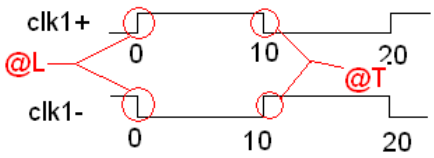


图4-67 相位含义

Sample (.pis.tcl)

set_arrival -ports "clock1" -phase "CLK1+" -rise -time 0.000 ;

set_arrival -ports "clock1" -phase "CLK1+" -fall -time 10.000;

set_slew -ports "clock1" -phase "CLK1+" -time 0.500 ;

#将-phase 指定的相位"CLK1+"绑定到-phase 指定的输入管脚"clock1"，并定义时钟跳变沿到达时间和 slew 值（用来计算路径的 AT 时间）。

#定义 clock1 管脚输入的时钟相位为正相位，上升沿到达时间 0ns，下降沿到达时间 10ns，跳变沿 slew 0.5ns。

```
set Databusports [et::expand_wildcard_list -pins {DataBus[*]}]
set_arrival -ports $Databusports -phase "CLK1@L" -time 17.000  -late ;
set_arrival -ports $Databusports -phase "CLK1@L" -time 5.000    -early;
set_slew   -ports $Databusports -phase "CLK1@L" -time 1.000    ;
```

#为-ports 指定的端口绑定数据相位，并定义该端口信号最早最晚到达时间和 slew 值（用来计算路径的 AT 时间）。

#定义 Databusport 端口输入数据最早在 CLK1 时钟上升沿 5ns 后有效，最晚在 CLK1 时钟上升沿 17ns 后有效，信号跳变 slew 1ns。

【相位标签传递－组合逻辑】

- 单一数据相位：数据标签合并，只向下传递一个数据相位标签。

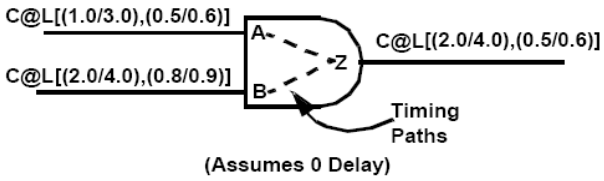


图4-68 单一数据相位标签传递

- 多个数据相位：独立传递，延时信息独立计算，互不影响。

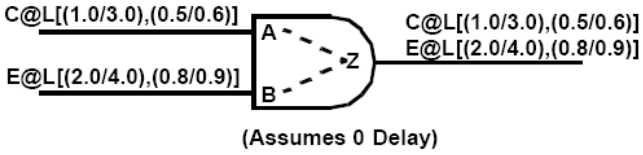


图4-69 多数据相位标签传递

- 多个时钟相位：各时钟相位独立的向下传递。

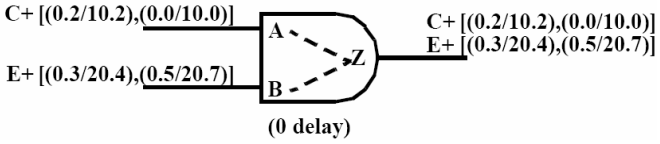


图4-70 多个时钟相位标签传递

- 门控时钟：当数据与时钟相位通过同一个组合逻辑合并后就出现门控时钟，只有时钟相位的信息传递下去。在门控逻辑的输入端作门控时钟检查。

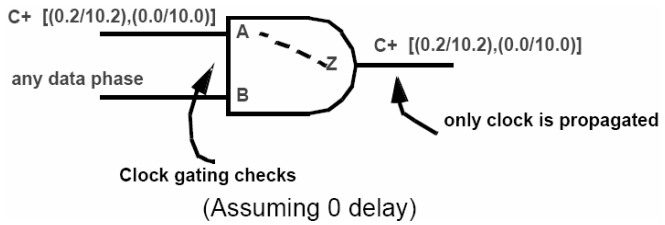


图4-71 门控时钟相位标签传递

【相位标签传递－寄存器（上升沿触发）】

- 寄存器单元默认的时序路径是从时钟的输入端到数据的输出端。
- 寄存器单元的数据输入和时钟输入之间要做 Setup/Hold 检查
- 数据标签的传递只与时钟的上升沿有关
- 如果输入寄存器的时钟是正相位，则寄存器输出的数据相位为时钟相位的名字加上@L 后缀。
- 如果输入寄存器的时钟是负相位，则寄存器输出的数据相位为该时钟相位的名字加上@T 后缀

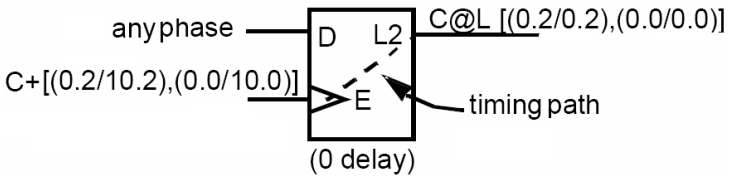


图4-72 单时钟域相位标签传递

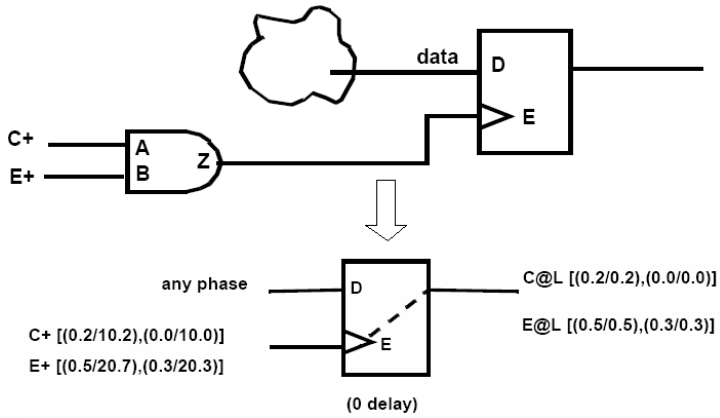


图4-73 多时钟域相位标签传递

【相位标签传递－反相器】

- 时钟标签经过一个反相器后相位标签取反（相位取反，跳变沿到达时间取反）

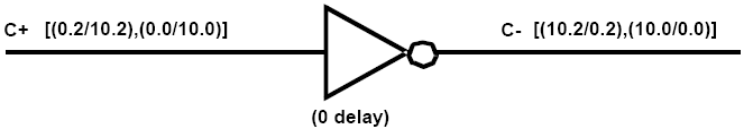


图4-74 时钟相位标签经反相器传递

- 寄存器的输入时钟是一个反相时钟时，该寄存器输出的数据相位后缀为“@T”，因为数据是在定义的时钟的下降沿寄存的。

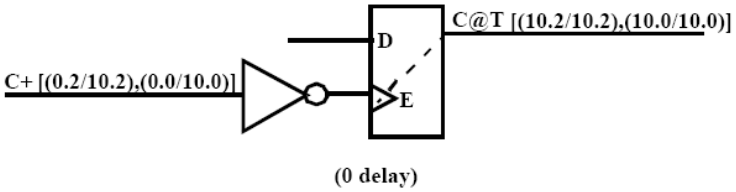


图4-75 寄存器时钟反相影响输出相位标签

【时钟调整－同步时钟】

EinsTimer 根据信号的数据相位标签判断该信号是在当前时钟周期检查还是不在当前时钟周期检查。

建立时间：数据要在时钟沿到来之前有效。因此上游的数据发送与下游的数据捕获不可能同时发生，setup 检查上游的发射时钟沿要早于下游的捕获时钟沿。

保持时间：数据要在时钟沿之后保持有效一段时间。因此 hold 检查上游发送时钟沿不能早于下游的捕获时钟沿。

- 发射时钟和捕获时钟前沿是同相位的

.phase.tcl file

```
#-------------------------------------
et::create_clock -name "CLK1" -period 20.0 -waveform { 0.0 10.0 };
et::create_clock -name "CLK2" -period 20.0 -waveform { 0.0 10.0 };
```

A．建立时间检查调整

下游寄存器的捕获时钟沿要晚于上游发射时钟沿。Clock2 的到达时间向后调整 1 个 CLK2 时钟周期

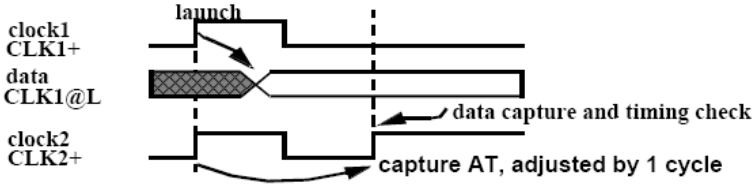


图4-76 发射时钟和捕获时钟前沿同相位建立时间检查调整

B．保持时间检查调整

下游寄存器的捕获时钟沿要早于上游发射时钟沿。Clock2 的到达时间不作调整

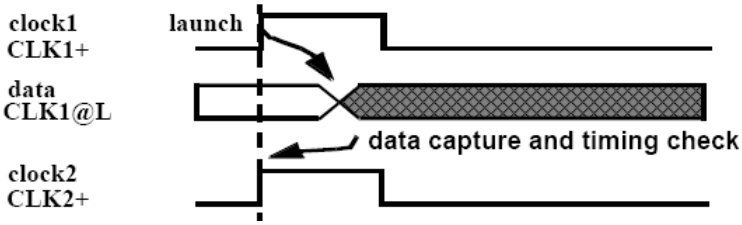


图4-77 发射时钟和捕获时钟前沿同相位保持时间检查调整

- 发射时钟的前沿晚于捕获时钟前沿

.phase.tcl file

#----------

et::create_clock -name "CLK1" -period 20.0 -waveform { 3.0 13.0 };

et::create_clock -name "CLK2" -period 20.0 -waveform { 0.0 10.0 };

A．建立时间检查调整

下游寄存器的捕获时钟沿要晚于上游发射时钟沿。Clock2 的到达时间向后调整 1 个 CLK2 时钟周期

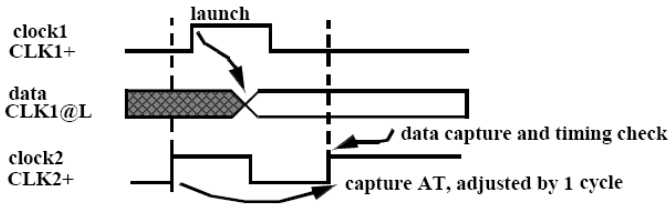


图4-78 发射时钟的前沿晚于捕获时钟前沿建立时间检查调整

B．保持时间检查调整

下游寄存器的捕获时钟沿要早于上游发射时钟沿。Clock2 的到达时间不作调整

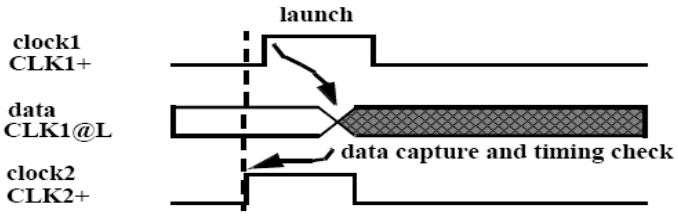


图4-79 发射时钟的前沿晚于捕获时钟前沿保持时间检查调整

- 发射时钟的前沿早于捕获时钟前沿

.phase.tcl file

#----------

et::create_clock -name "CLK1" -period 20.0 -waveform { 0.0 10.0 };

et::create_clock -name "CLK2" -period 20.0 -waveform { 11.0 21.0 };

A．建立时间检查调整

下游寄存器的捕获时钟沿要晚于上游发射时钟沿。 Clock2 的到达时间不作调整

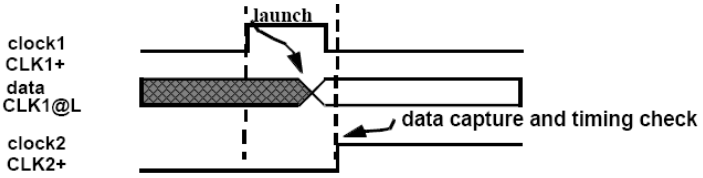


图4-80 发射时钟的前沿早于捕获时钟前沿建立时间检查调整

B．保持时间检查调整

下游寄存器的捕获时钟沿要早于上游发射时钟沿。 Clock2 的到达时间向前调整 1 个 CLK2 时钟周期

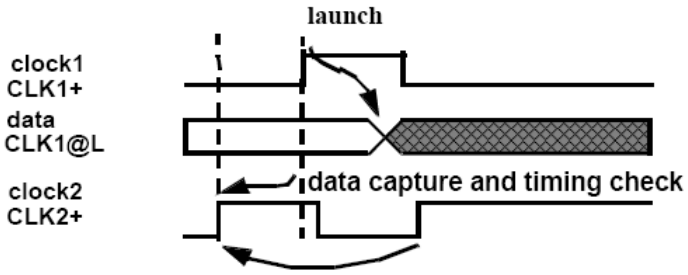


图4-81 发射时钟的前沿早于捕获时钟前沿保持时间检查调整

⚠ 注意：

时钟沿相位关系（早于/等于/晚于）完全是基于.phase.tcl 中的波形定义判断。ET 工具作检查时的时钟自动调整与该检查点的到达时间无关。

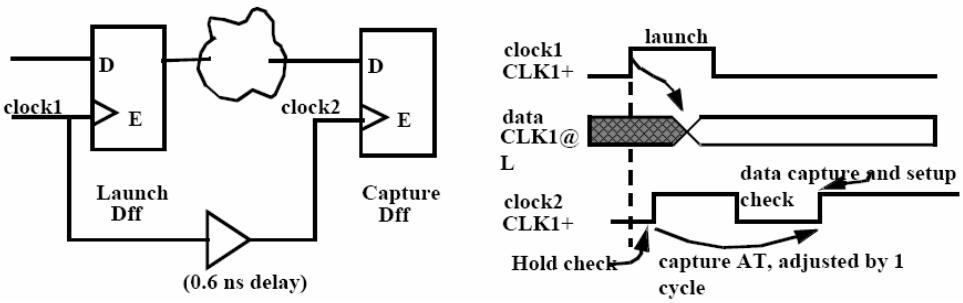


图4-82 时钟路径延时不同时检查调整关系

如图4-82所示，尽管下游捕获时钟沿到达时间晚于上游发送时钟沿的到达时间，setup 检查时 clock2 时钟到达时间仍会自动后移一个 CLK1 时钟周期。

【时钟调整－异步时钟】

异步数据与时钟之间无相位关系，工具检查时不对其作相位调整。但工具默认会对异步信号作时序检查，这样就会上报很多时序不满足。真正的异步输入信号是不能作检查的，异步接口应该由设计人员保证功能和时序的正确性。

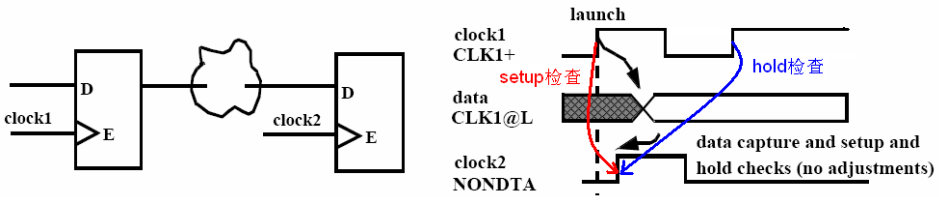


图4-83 异步时钟检查关系

图 4-83 所示例子中，hold 检查能够通过，但是 setup 检查不能通过。此时应该在 phase.tcl 中定义这组异步时钟域之间不作检查。

【Clock override】

时序分析时，如果网表中还是理想时钟网络（FEP 之前），则需要对寄存器的时钟管脚 override 一个 slew 值和到达时间。否则时钟路径会由于负载过大导致延时非常大，无法进行时序分析。

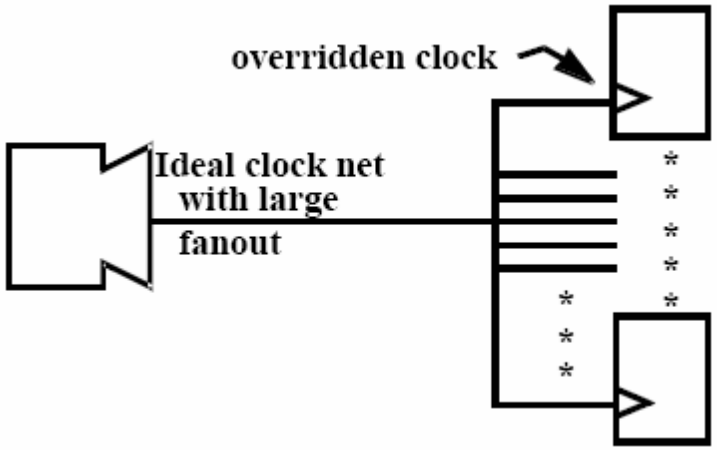


图4-84 FEP 之前网表时钟连接

在计算 override 值时，需要将估算的时钟树的延时计算在内。ET 作时序分析时，可以通过在图形界面选择“Use Clock Overrides”选项使能 Override 时钟。

Clock Override 设置举例：

```
#Sample (.phase.tcl)
et::create_clock -name "CLK1" -period 20.0 -waveform { 0.0 10.0 };
#理想时钟网络的 override 时钟定义，时钟树延时定义为 0.3ns，slew 0.5ns
et::override_clock -latch -name "CLK1" -time 0.3 -slew 0.50 -lead;
et::override_clock -latch -name "CLK1" -time 10.3 -slew 0.50 -trail;
```

【Clock Pair Exclusion】

如果两个时钟域之间是异步或相互排斥关系，就需要禁止这两个时钟域之间的检查。

et::set_phase_pair_exclusion 会禁止定义的一对时钟和它们的数据之间的所有检查（时钟 1 和时钟域 2 的数据，时钟 2 和时钟域 1 的数据）。这条命令要非常小心的使用，使用不当可能会将真正时序不满足的信息屏蔽掉。

假设设计中有 3 个时钟域 CLK1、CLK2 和 CLK3，其中 CLK1 和 CLK3 时钟域有同步关系，CLK2 和 CLK1、CLK2 和 CLK3 是异步关系。下面给出这三个时钟域之间检查关系的约束。

Phase Pair Exclusion (.phase.tcl)

```
#-----------------------------------
et::create_clock -name "CLK1" -period 20.0 -waveform { 0.0 10.0 };
et::create_clock -name "CLK2" -period 30.0 -waveform { 0.0 15.0 };
et::create_clock -name "CLK3" -period 16.66 -waveform { 0.0 8.3 };
#-----------------------------------------------------------------------------
#phase pair exclusions to prevent timing tests between clock domains
#-----------------------------------------------------------------------------
et::set_phase_pair_exclusion -clock1 CLK1 -clock2 CLK2
et::set_phase_pair_exclusion -clock1 CLK2 -clock2 CLK3
```

【时钟抖动设置】

ET 支持用命令控制更改自动时钟调整，这个命令通常被用来模拟时钟的周期抖动。下面给出了一个用户自定义微调模拟 PLL 的输出驱动 CLK1 时钟域　抖动的例子，抖动范围为+/-100ps

User Delta Adjust (.phase.tcl)

```
#-----------------------------------
et::create_clock -name "CLK1" -period 20.0 -waveform { 0.0 10.0 };
#User Delta Adjust
#-----------------------------------------------------------------------------
et::set_user_delta_adjust -data_clock "CLK1" -clock "CLK1" -adjust -0.100 -late -lead_data_edge -lead_clock_edge;
et::set_user_delta_adjust -data_clock "CLK1" -clock "CLK1" -adjust -0.100 -late -trail_data_edge -trail_clock_edge;
```

【.phase.tcl 约束文件编写建议】

- 相位标签的名称最好能够与对应的时钟命名相同。
- 相位标签的名称长度最好小于 8 个字符，这样报告格式才会比较规整。
- .phase.tcl 中定义的相位标签不要带’+’或’-’后缀。
- 相位标签是区分大小写

- 相位标签的命名要有意义，且要加注释，便于其他人检视
- 一般将时钟的前沿定义在时刻点 0.0ns
- 定义一个低电平有效时钟时，.phase.tcl 中定义语句与高电平有效时钟定义相同（前沿定义在 0.0ns，后沿定义在时钟周期的一半处）。在.pis.tcl 中给低有效的相位标签分配一个'-'相位标签（例如 clk-）

(2) Assertion: .pis.tcl

- 给输入管脚绑定相应的相位标签，并指定该管脚上信号的到达时间信息。
- 输入数据的约束信息来源于设计规格，且要与综合约束保持一致。
- 信号的到达时间与该信号的相位标签相关。
- 一个输入管脚可能会有多个相位标签和到达时间。
- 异步输入管脚要使用命令禁止进行时序检查。

Input Arrival Times (.pis.tcl)

```
# 约束与“CLK1”时钟域相关的时钟和数据输入管脚
et::set_arrival -ports "clock1" -phase "CLK1+" -rise -time 0.000;
et::set_arrival -ports "clock1" -phase "CLK1+" -fall -time 10.000;
et::set_slew -ports "clock1" -phase "CLK1+" -time 0.500;
set Databusports [et::expand_wildcard_list -pins {DataBus[*]}]
et::set_arrival -ports $Databusports -phase "CLK1@L" -time 17.000 -late;
et::set_arrival -ports $Databusports -phase "CLK1@L" -time 5.000 -early;
et::set_slew -ports $Databusports -phase "CLK1@L" -time 1.000;
```

上述语句约束的效果如图 4-85 所示。

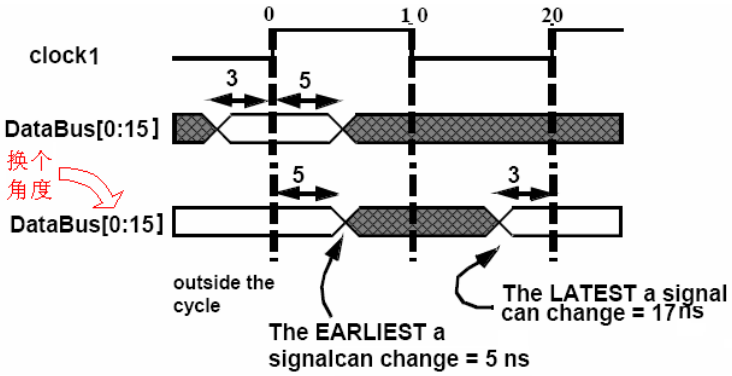


图4-85 输入管脚信号到达时间约束与延时关系

(3) Assertion: .eta.tcl

- 指定输出管脚的输出延时
- 输出延时与相位标签相关
- 如果不指定输出管脚的输出延时，ET 会对其使用默认的输出延时。

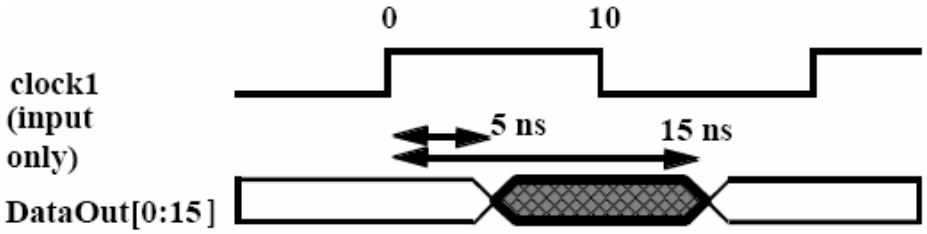


图4-86 输出管脚延时约束

如图 4-86 所示，输出管脚 dataout[0:15]要求最早在时钟有效沿后 5ns 有效，最晚在时钟有效沿后 15ns 有效，其约束语句如下：

```
#-------------------------------------------------------------
# Output Arrival Times (.eta.tcl)
#-------------------------------------------------------------
set Databuspins [et::expand_wildcard_list -pins {Data_out[*]/A}]
et::set_required -phase "CLK1@L" -pins $Databuspins -late -time 15.0
et::set_required -phase "CLK1@L" -pins $Databuspins -early -time 5.0 ;
```

(4) Assertion: .pincap.tcl

- 指定输出管脚的集总容性负载
  电容值是估算的与输出管脚相连的连线的电容，这个值只对输出管脚的时序分析有效。
  要指定最小和最大电容值，扇出项是可选的，只对输出管脚有效。

.pincap.tcl

```
#---------------------------------------
ess::set_pin_capacitance -pins { Output_Net } -wc -value 50.0
ess::set_pin_capacitance -pins { Output_Net } -bc -value 20.0
```

⚠ 注意：

ET 工具对输入输出 PAD 的时序分析不准确，IBM 建议我们约束语句都加在内部逻辑与输入输出 PAD 连接的引脚上，不要约束到外部的管脚上。所以对于输出管脚容性负载的估算就要包括 PAD、封装、芯片输出管脚、板上连线和对接芯片的输入管脚，这些信息由 SI 工程师通过仿真获得。

(5) Assertion: .dcadj.tcl

- Don‘t care 或调整引脚和连线上的到达时间。
- 人工控制测试、false 路径指定、多周期路径指定以及时钟的生成
- 重命名引脚和连线的相位标签

.dcadj.tcl 语法

```
#-------------------------------------------------------------------------------
```

```
et::dont_care_signal   -pins {} -phase {}
et::adjust_signal      -pins {} -phase {}   -time   -fall
et::rename_phase       -pins {} -phase {}   -new_phase {}
#-------------------------------------------------------------------------------
```

dont_care_signal：中止指定测试点相应相位的所有时序信息的传递。

adjust_signal ：将指定测试点的到达时间调整指定的时间单位。

rename_phase ：将指定的给出的相位重新命名为其它的相位。

【Don’t Cares】

Don’t Cares 指出那些点不需要计算到达时间、要求到达时间和 slack。

Don’t Cares 可以约束门单元的输入和输出，约束引脚和连线的名字。

Don’t Cares 语句比较危险，一定要确认不能忽略有效路径的检查。

- 输出端口 Don’t Cares：

输出端 Don’t care 时，沿该输出点向下的所有时序路径都不作检查。该输出点的所有到达时间不会向下传递。该输出点的所有要求到达时间都停止反向传递。对图 4-87 所示电路中"U1/Z"管脚作 Don’t care 约束：

et::dont_care_signal -pins "U1/Z" -phase "*"

约束结果如图 4-87 中红所字体所示。

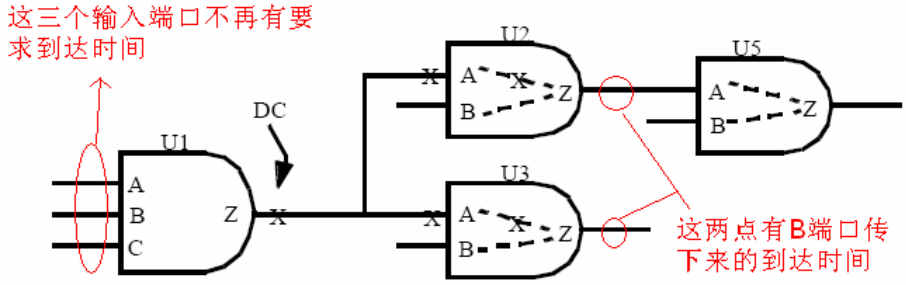


图4-87 输出端口 Don’t Cares 约束

注意：

Don’t care 一条 net 与 Don’t care 一个 cell 的输出引脚结果是等效的。

- 输入端口 Don’t Cares：

输入端 Don’t care 时，该点的到达时间不会传递到输出端。该输入点的要求到达时间被忽略，不会反向向下传递。对图 4-88 所示电路中" U3/A"管脚作 Don’t care 约束：

et::dont_care_signal -pins "U3/A" -phase "*"

约束结果如图 4-88 中红所字体所示。

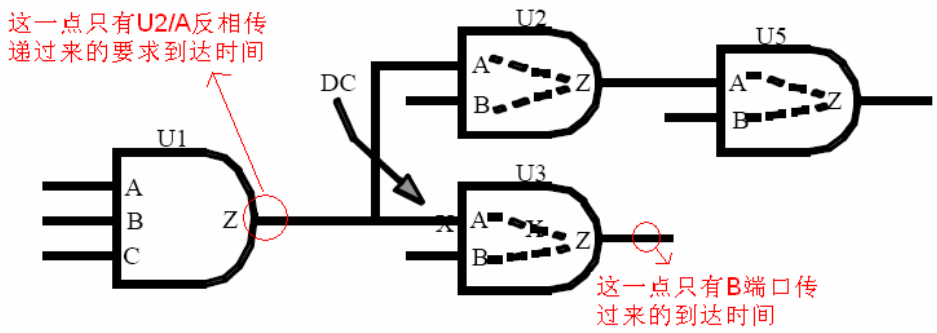


图4-88 输入出端口 Don't Cares 约束

【相位调整】

人为的用命令的方式调整某点的到达时间。常用于多周期路径检查和分频时钟相位调整。

如图 4-89 所示，DFF1 和 DFF2 之间数据路径延迟大于一个时钟周期，小于 2 个时钟周期，此时需要将此路径设置为多周期路径，否则 ET 检查就过不了。设置方法如下，只要将 DFF2/D 端到达时间人为的提前 1 个时钟周期就可以了。

#Multi-Cycle Adjustment (.phase.tcl)

et::create_clock -name "CLK" -period 20.0 -waveform { 0.0 10.0 };

#Multi-cycle Adjustment (.dcadj.tcl)

et::adjust_signal -pins "DFF2/D" -phase "CLK@L" -time -20

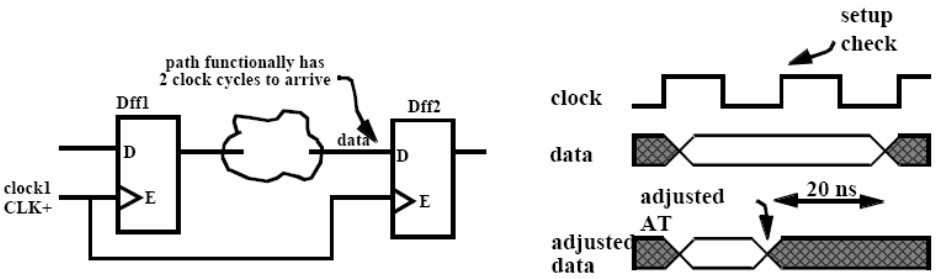


图4-89 多周期路径设置

【相位重命名】

时序分析时在多种情况下需要作 Phase renames，比如生成时钟、多时钟域分析以及独特逻辑接口。

当时钟由内部逻辑生成时，如图 4-90 所示，时钟相位 CLK1+经过 DFF1 寄存器分频后变成一个数据相位 CLK1@L。所以需要用 Phase renames 将 U2 的输出重新定义成一个新的时钟相位。对于本例给出的分频时钟电路来说，只作相位重命名是不够的，从图 4-90 中给出的 clock2 波形可以看出重命名后的时钟波形只是一个毛刺（因为寄存器的输出跳变只会发生在时钟的上升沿）。所以还需要对 U2 的输出下降沿到达时间进行调整。具体设置如下：

分频时钟重命名 (.phase.tcl)

et::create_clock -name "CLK1" -period 20.0 -waveform { 0.0 10.0 };

et::create_clock -name "CLK2" -period 40.0 -waveform { 0.0 20.0 };

分频时钟重命名 (.dcadj.tcl)

et::adjust_signal -pins "U2/A" -phase "CLK1@L" -time 20 -fall

et::rename_phase -pins "U2/A" -phase "CLK1@L" -new_phase "CLK2+“

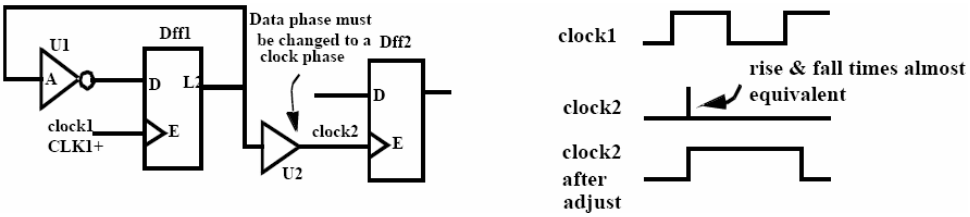


图4-90 分频时钟重冲命名

【.dcadj.tcl 编写指导】

- 要对 adjusts 和 don’t cares 的内容加注释。
- ‘/’是层次分隔符，如 instance/pin。不过这种分隔符是可以由用户自定义的。
- 作 phase rename 时，可以用通配符‘*’代表所有的相位。
- 作 adjust 时，如果指定的相位标签错误，不会有错误和告警上报，且不作任何调整操作。

(6) Assertion: .mode.tcl

- 大多数时序单元具有多种功能配置，不同的功能模式下作时序检查时的路径是不同的。
- EinsTimer 只能在设计中找出时序路径并作检查，不能知道设计的功能。
- Mode 定义了要检查哪些时序路径，作哪些检查。一般会使用默认的时序模式
- 当器件的工作模式不是默认模式时，就需要在.mode.tcl 文件中对该器件指定适当的时序检查模式。
- ET 支持对某种类型的所有器件指定时序检查模式。

.mode.tcl 语法

et:::set_mode -boxes {} -name "mode_name"

- -box：指定要修改工作模式的器件，如果要指定某一类型的所有器件，就要使用参数 –cell_type {}。
- -name：Mode_name 指定期望的时序模式。推荐使用此参数，用工作模式的名字指定，不要用模式的编号指定。
- 在选择适当的时序模式时，可以下列资源获得帮助：
  用户手册
  Theguide 中集成的 Nilplot 工具

et::set_mode -boxes "FIFO/SRAM" -name Array_Clocked

All elements of type LPH0101_A are set to ACTEST mode

set_mode -cell_types "LPH0101_A" -name ACTEST

(7) Assertion: .udt.tcl

【用户自定义任意两个引脚之间的 setup/hold 检查】

如图 4-91 所示，ET 工具默认不会对 U1/A 和 U1/B 之间进行时序检查，如果这两个输入端口之间确实有时序关系，我们可以添加约束语句，强制 ET 工具对这两点之间时序进行检查。

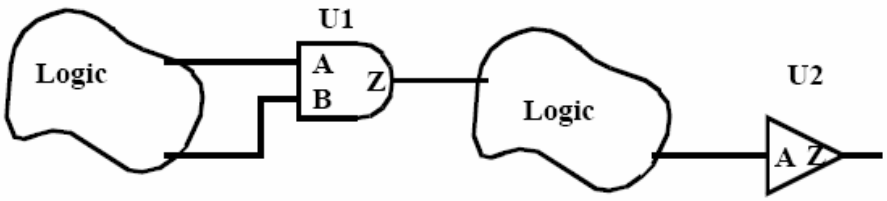


图4-91 任意两个引脚之间的 setup/hold 检查

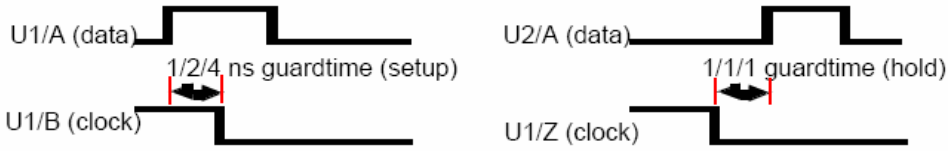


图4-92 时序检查图

如果需要作如图 4-92 所示时序检查，则需要添加如下的约束语句。

et::set_user_defined_test -late -pin1 U1/A -pin2 U1/B -rise1 -fall2 -bc 1.0 -nom 2.0 -wc 4.0

et::set_user_defined_test -early -pin1 U2/A -pin2 U1/Z -rise1 -fall2 -bc 1.0 -nom 1.0 -wc 1.0

【异步路径时序约束】

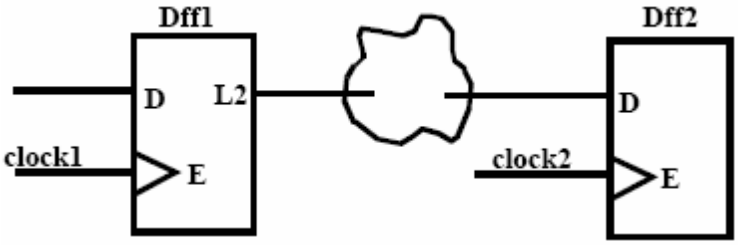


图4-93 异步路径示例图

如图 4-93 所示，假设 clock1 和 clock2 是异步时钟，clock1 周期为 20ns，clock2 周期为 13ns。要求 Dff1/L2 到 Dff2/D 路径延时小于 8ns（wc）、5ns（nom）、3ns（bc）。则需加如下约束：

et::set_user_defined_test -late -pin1 Dff2/D -pin2 Dff1/E -rise1 -rise2 -bc 17.0 -nom 15.0 -wc 12.0;

et::set_user_defined_test -late -pin1 Dff2/D -pin2 Dff1/E -fall1 -rise2 -bc 17.0 -nom 15.0 -wc 12.0;

【输出管脚 skew 约束】

用户自定义测试可以检查输出数据和输出随路时钟之间的相位关系是否满足。

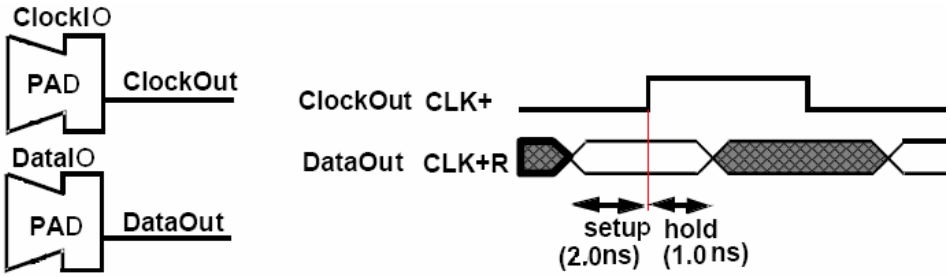


图4-94 输出管脚 skew 约束

为了实现图 4-94 所示的时序关系，需添加如下约束语句：

Primary Output Timing (.udt)

et::set_user_defined_test -late -pin1 DataIO/PAD -pin2 ClockIO/PAD -rise1 -rise2 -bc 2.0 -nom 2.0 -wc 2.0;

et::set_user_defined_test -late -pin1 DataIO/PAD -pin2 ClockIO/PAD -fall1 -rise2 -bc 2.0 -nom 2.0 -wc 2.0;

et::set_user_defined_test -early -pin1 DataIO/PAD -pin2 ClockIO/PAD -rise1 -rise2 -bc 1.0 -nom 1.0 -wc 1.0;

et::set_user_defined_test -early -pin1 DataIO/PAD -pin2 ClockIO/PAD -fall1 -rise2 -bc 1.0 -nom 1.0 -wc 1.0;

4.4.3 STA 工具 Primetime 与脚本介绍

PrimeTime 是 Synopsys 公司的一个独立的芯片级的 Gate_level 静态时序分析工具。静态时序分析是指在不做仿真的情况下来确定电路的时序是否符合要求的一种方法，主要包括以下三个步骤：

- 将电路分为若干条 Timing paths。
- 计算每条路径的延迟。
- 检查每条路径的延迟是否符合时序要求。

它适合分析大规模的同步数字 ASIC。同时与 Synopsys 公司的其它工具，如 DC，有很好的接口。Primetime 适合对系统级（SOC）的设计进行时序分析，而且由于它的 synthesis compatible 特性，在整个设计流程中都可以应用。

除了 Primetime 外，Design Compiler 内也提供了时序分析工具 DesignTime。但 Prime Time 与 Design Compiler 的 DesignTime 不同。PrimeTime 一个独立的工具，不需所有逻辑综合所需的数据结构，因此它的性能比 DesignTime 要好，对内存资源的需求也要少。它有以下特点：

- 适用于同步数字系统
- 是芯片级的 gate_level 静态时序分析工具，不适用于板级、晶体管级分析
- 与 DC 共用同样的延时计算方案
- 既支持图形用户界面又支持 shell 方式

- 自动检测和排除伪路径
- 灵活的时序报告方式，多途径观察分析结果
- 能找出所有时序错误
- 支持 SDF back-annotation
- 分析模式有 Single、min-max 和 On-Chip_Variation

在 Synopsys 的 Design Compiler 和 PrimeTime 中，三种 Operating condition 模式与反标文件 SDF 有如下关系：

表4-9 Operating condition 模式与 SDF 的关系列表

Operating condition	Operating condition based		1 个 SDF file	2 个 SDF files
Single	setup	Max data at operating condition.	(a:b:**C**)	
		Min clock at operating condition.	(a:b:**C**)	
	hold	Min data at operating condition.	(a:b:**C**)	
		Max clock at operating condition.	(a:b:**C**)	
Min_Max	setup	Max data at worst case.	(a:b:**C**)	File1: (a:b:C)
		Min clock at worst case.	(a:b:**C**)	File2: (a:b:C)
	hold	Min data at best case.	(**A**:b:c)	File1: (A:b:c)
		Max clock at best case.	(**A**:b:c)	File2: (A:b:c)
On_chip_ variation	setup	Max data at process, voltage and temperature max.	(a:b:**C**)	File1: (a:b:C)
		Min clock at process, voltage and temperature min.	(**A**:b:c)	File2: (A:b:c)
	hold	Min data at process, voltage and temperature min.	(**A**:b:c)	File1: (A:b:c)
		Max clock at process, voltage and temperature max.	(a:b:**C**)	File2: (a:b:C)

注：表中右两列“(a:b:c)”表示 cell 的（最小延时值：nomal 延时值：最大延时值）。其中，大写黑体为 PrimeTime 做该项 STA 分析时，所采用的 SDF 文件中的延时值。

- PrimeTime 主要做如下时序检查：
- Setup 和 Hold 时序检查
- Recovery 和 removal 检查
- Clock pulse width 检查
- Clock gating check
- PrimeTime 还可以做的设计检查有：
- Un-clocked 寄存器
- 没有约束的 timing endpoints

- Master-slave clock separation
- 多时钟驱动寄存器
- 电平触发时钟
- 组合反馈环逻辑电路
- Design rule checks (如 maximum capacitance， maximum transition time， maximum fan-out)

2. Primetime 基本流程

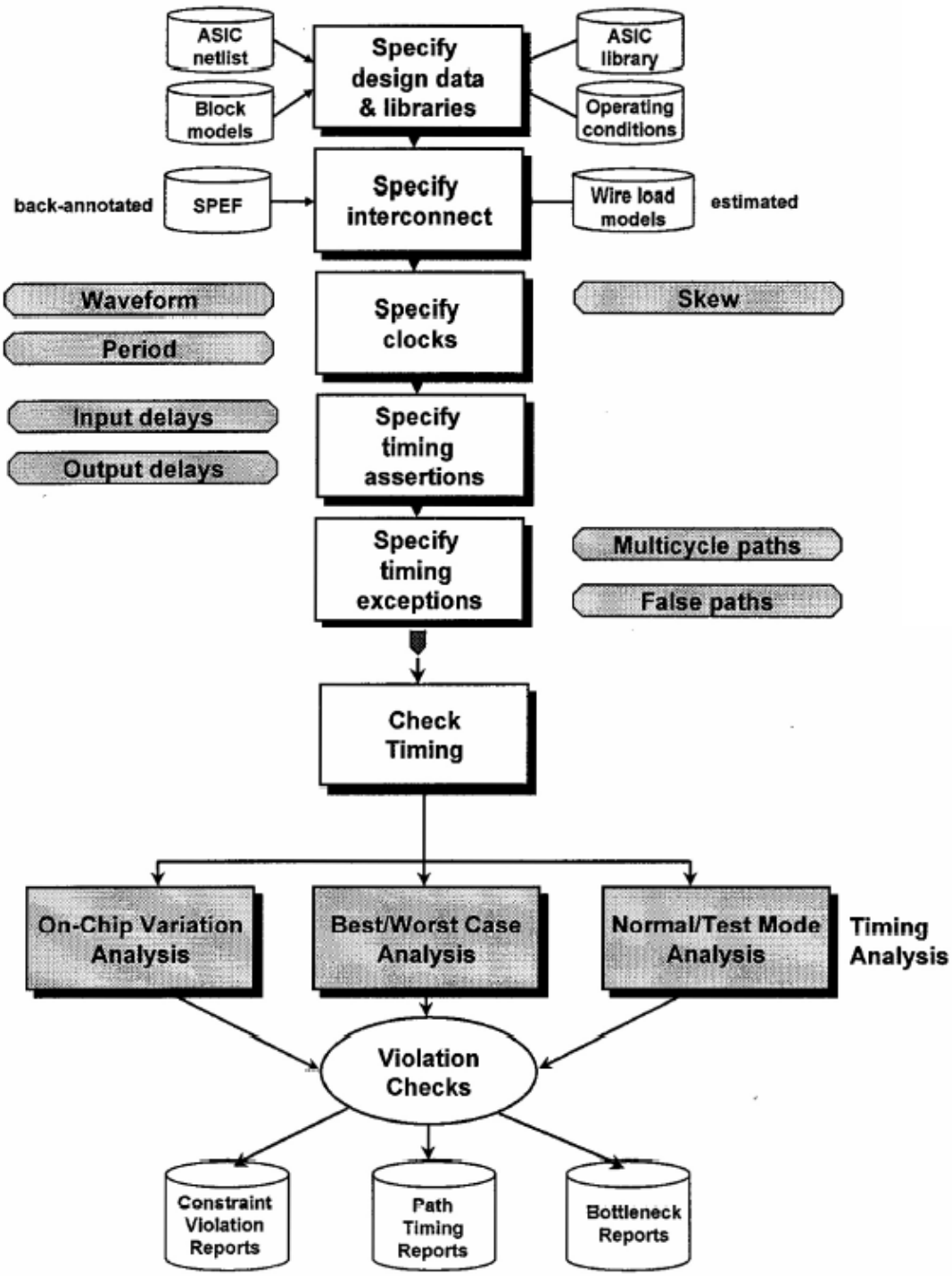


图4-95 PrimeTime 基本流程

3. PrimeTime 的启动和退出

启动：

%> primetime (GUI 方式)

%> pt_shell (Shell 方式)

退出：

pt_shell> exit

4. 建立设计环境

(1) 设置搜索路径

将分析中要用到的设计库、RAM 库、基本单元库、输入输出单元库所在路径设置在搜索路径中。

如：

```
set search_path ". \
./db /usr/systemFAME/cae/synopsys-dc/current/ce66 \
./cae/synopsys-dc "
```

设置连接路径(link_path)：将分析中用到的 RAM 库、基本单元库、输入输出单元库设置在连接路径中，以便在连接(link)时能自动加载这些库。

如：set_link_path " * ce66_core.db ccl.db ce66_io.db"

(2) 读入设计

可以 Verilog、db 等多种格式读入设计,如：

```
read_db CN332720.db
```

(3) 连接顶层模块

```
link_design CN332720
```

(4) 设置工作条件(set operating conditions)

在进行 setup 检查时，将用到 worst operating condition；

在进行 hold 检查时，将用到 Best operating condition。

如: set_operating_conditions -library pt_lib -min BCCOM -max WCCOM

(5) 设置线负载模型(wire load model)

在进行 setup 检查时，将用到 worst wire load model；

在进行 hold 检查时，将用到 Best wire load model。

```
set_wire_load_mode top
set_wire_load_model -library pt_lib -name 05x05 -min
set_wire_load_model -library pt_lib -name 20x20 -max
```

5. 指定时序约束

(1) 定义时钟

需定义的时钟信息一般包括：

- 时钟周期(period)
- 相位(waveform)
- Clock latency
- Clock skew
- Clock transition

```
create_clock -period 30 [get_ports CLOCK]
set clock [get_clock CLOCK]
set_clock_uncertainty 0.5 $clock
set_clock_latency -min 3.5 $clock
set_clock_latency -max 5.5 $clock
set_clock_transition -min 0.25 $clock
set_clock_transition -max 0.3 $clock
```

若时钟网络的时延信息已完全可通过反标得到，则 skew、transition 等均可通过如下命令设置：

```
Set_propagated_clock  clock_object_list
```

(2) 读入 SDF 时延文件

若有准确的时延文件，则读入文件实现时延反标。

```
Read_sdf  CFN332720.sdf
```

(3) 定义输入输出延时

为了使得时序约束更完全，我们需要定义输入输出延时。另外，可对输入端口上设置其驱动单元，对输出端口设置其容性负载。

```
set_input_delay 0.0 [all_inputs] -clock $clock
set_output_delay 2.0 [get_port INTERRUPT_DRIVER_ENABLE] -clock $clock
set_output_delay 1.25 [get_port MAPPING_ROM_ENABLE] -clock $clock
set_output_delay 0.5 [get_port OVERFLOW] -clock $clock
set_output_delay 1.0 [get_port PIPELINE_ENABLE] -clock $clock
set_output_delay 1.0 [get_port Y_OUTPUT] -clock $clock
set_driving_cell -lib_cell IV -library pt_lib [all_inputs]
set_capacitance 0.5 [all_outputs]
```

6. 设置 Timing Exceptions

Timing exceptions 主要包括：

- 伪路径
- 多周期路径
- 用户定义的最小和最大延时约束

- Disabled timing arcs

PrimeTime 支持用 -through 选项设置 timing exceptions。你必须准确地设置 timing exceptions，否则不会被接受。设置伪路径和多周期路径时必须指定完全、有效的路径，其起点和终点可以是：

- Startpoints are primary input ports, clocks, pins, or cells.
- Endpoints are primary output ports, clocks, pins, or cells.

为了保证 timing exceptions 设置正确，最好用 report_exceptions 命令来检查一下。一般可用

report_exceptions -ignored 命令检查是否有 timing exceptions 由于无效而被被忽略。

```
set_false_path -from U3/OUTPUT_reg[*]/CP -to U2/OUTPUT_reg[*]/D
set_multicycle_path -setup 2 -from INSTRUCTION[*] -to U2/OUTPUT_reg[*]
set_multicycle_path -hold 1 -from INSTRUCTION[*] -to U2/OUTPUT_reg[*]
update_timing
report_exceptions
report_exceptions -ignored
```

7. 时序分析及报告产生

在加了必要的时序约束后，就可以进行时序分析。但在分析之前一定要对时序约束和电路的结构做一下检查。 如哪些端点没任何约束，哪个寄存器的时钟端没定义时钟等。

```
pt_shell> check_timing
```

若存在没法解释的问题，将导致不完全或不准确的分析结果。

时序分析可先用 report_constraint 命令从总体上检查一下设计的时序。

report_constraint 命令能检查：

- Setup and hold timing
- Min clock pulse width
- Minimum period
- Recovery and removal timing check
- Clock gating setup and hold
- Minimum and maximum capacitance (design rule check)
- Minimum and maximum transition (design rule check)
- Minimum and maximum fanout (design rule check)

可以用 Report_constraint -all 报出所有时序错误。

我们可用 report_timing 命令报告出基于路径的时序。 例如，命令

Report_timing 杌 delay_type min 杌 max_paths 10 杌 group TCLK1

对路径组 TCLK1 进行 hold time 时序分析，并在文件 tclk1_rpt 中给出 slack 最少的 10 条路径的详细报告。

Primetime 的时序分析非常直观、方便。Reports-Histograms 菜单中可为我们提供很直观的分析结果。如我们想分析电路中的 setup、hold 时序是否满足，可选 reports-histograms-End_point_slack。

8. 如何获得帮助及参考资料

SYNOPSYS 提供 on-line help，可用 help 命令获得帮助。

SYNOPSYS 提供的随机手册有：

- PrimeTime Modeling Guide
- PrimeTime Tutorial
- PrimeTime User Guide

这些手册可用 Acroread 打开$SYNOPSYS/sold/top.pdf 后，选取相关手册打开。

9. Script 文件样例

```
############################################################
# Help and man examples
############################################################
help
help *clock
help -verbose set_input_delay
man read_db
############################################################
# Set search_path and link_path
############################################################
set search_path ??set link_path ? pt_lib.db STACK_lib.db Y_lib.db?
############################################################
# Read and link a Design
############################################################
read_db AM2910.db
link_design AM2910

############################################################
# Setting up the timing environment
############################################################
set link_path ?link_path STACK_lib.db Y_lib.db?link_design AM2910
set_operating_conditions -library pt_lib -min BCCOM -max WCCOM
set_wire_load_mode top
set_wire_load_model -library pt_lib -name 05x05 -min
set_wire_load_model -library pt_lib -name 20x20 -max
```

```
############################################################
# Setting up the timing assertions
############################################################
create_clock -period 30 [get_ports CLOCK]
set clock [get_clocks CLOCK]
set_clock_uncertainty 0.5 $clock
set_clock_latency -min 3.5 $clock
set_clock_latency -max 5.5 $clock
set_clock_transition -min 0.25 $clock
set_clock_transition -max 0.3 $clock
set_clock_gating_check -setup 0.5 -hold 0.1 $clock
set_min_pulse_width 2.0 $clock
report_design
report_reference
Check_timing

############################################################
# Setting up the port delays and checking the timing
############################################################
set_input_delay 0.0 [all_inputs] -clock $clock
set_output_delay 2.0 [get_ports INTERRUPT_DRIVER_ENABLE] -clock $clock
set_output_delay 1.25 [get_ports MAPPING_ROM_ENABLE] -clock $clock
set_output_delay 0.5 [get_ports OVERFLOW] -clock $clock
set_output_delay 1.0 [get_ports PIPELINE_ENABLE] -clock $clock
set_output_delay 1.0 [get_ports Y_OUTPUT] -clock $clock
set_driving_cell -lib_cell IV -library pt_lib [all_inputs]
set_capacitance 0.5 [all_outputs]
Check_timing

############################################################
# Saving the assertions
############################################################
write_script -format dcsh -output AM2910.dcsh
write_script -format ptsh -output AM2910.pt
```

```
############################################################
# Performing basic timing analysis
############################################################
report_constraint
report_constraint -all_violators
Report_timing

############################################################
# Specifing timing exceptions
############################################################
set_false_path -from U3/OUTPUT_reg[*]/CP -to U2/OUTPUT_reg[*]/D
set_multicycle_path -setup 2 -from INSTRUCTION[*] -to U2/OUTPUT_reg[*]
set_multicycle_path -hold 1 -from INSTRUCTION[*] -to U2/OUTPUT_reg[*]
update_timing
report_exceptions
report_exceptions -ignored
report_constraint -all_violators
report_timing
```

4.4.4 STA 案例分析

1. HSS 硬 core 输出两种时钟的问题

问题：SD8901 处理 2.5G/622M/155Mbps 三种速率的业务。在 2.5G/622Mbps 业务下，硬 core 工作在 full-rate 模式，硬 core 输出的随路时钟为 311Mhz；155M/ 业务下，硬 core 工作在 half-rate 模式，硬 core 的输出随路时钟为 155Mhz。

但是硬 core 的参考时钟只有一个，那么输出的相位也就只有一个。如何才能模拟出其中另外一个？

分析与解决：SD8901 利用 ET 的 clone_phase 命令在硬 core 的参考时钟相位“PLL0O622+”克隆了另外一个时钟相位“PLL0O622_CLN+”。于是硬 core 产生了两个数据相位“PLL0O622@L”和“PLL0O622_CLN@L”。利用 adjust_and_rename 命令，分别将“PLL0O622@L”和“PLL0O622_CLN@L”相位变为“LR311E0H+”和“LR155E0H+”。

2. Dont care 时钟相位遇到困惑

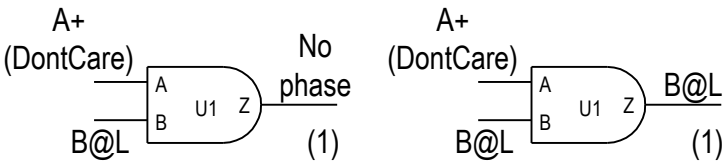


图4-96 相位穿通示意图

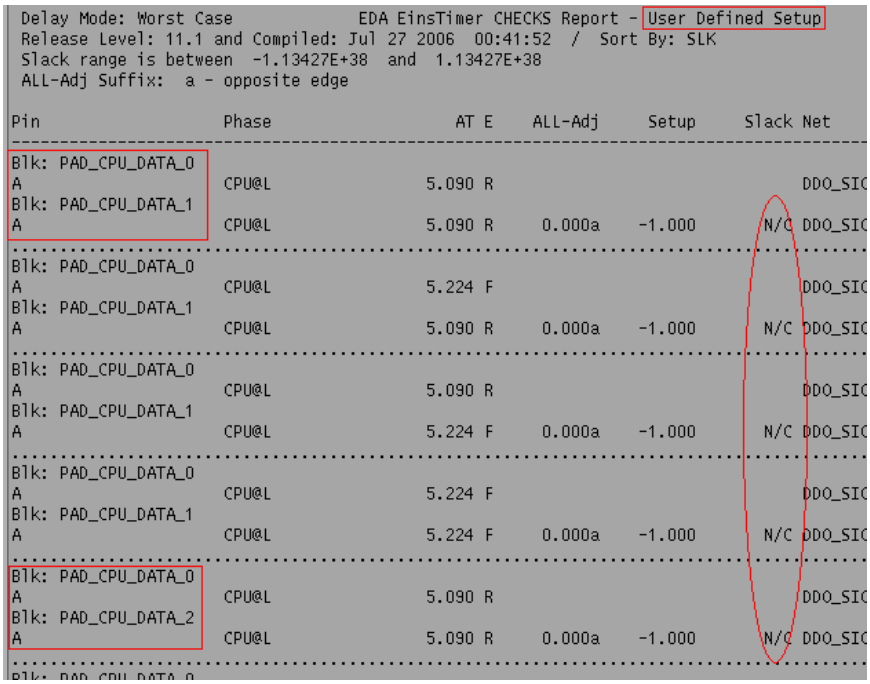
问题：很多时候为了分析的方便或为了减少不必要的时序检查，需要 dont care 一些 gating_clock cell 的时钟相位，如上图 4-96（1）所示。不过在 ET 中，如果将 A+相位 DC，那么 B@L 相位同样无法传递到输出端，所以输出端是没有相位的（No phase）。如何解决上述问题那？

分析与解决：如果 B@L 相位传递不到输出，也就无法对其进行时序分析。ET 中提供了一个 set_clock_flag，用于关闭或开启时钟 flag 的命令。如图 4-96（2）所示。

```
set_clock_flag –off –pins “U1/A”; # ET 认为 A 端不再是个时钟相位。
dont_care_signal –pins “U1/A” –phase “A+”
```

3. ETA 和 UDT 对同一点约束遇到的困惑

问题：如果遇到某点在 eta 中约束了又需要在 udt 中做约束，如何处理？ET 中对某点做约束，要求在 eta 中做了约束就不能在 udt 中做约束，否则 udt 的约束不起作用。如下图：PAD_CPU_DATA_0~15/A 端的相位均为 CPU@L。对点 PAD_CPU_DATA_0/A 在 eta.tcl 中用 set_required 约束了要求到达时间设置了 eta，同时在 udt.tcl 中用 set_user_defined_skew_test 设置了 PAD_CPU_DATA_0/A 与其他 PAD_CPU_DATA_1～15/A 之间的 balance skew。从图 4-97 中可以看出：Apin 的相位为 CPU@L，但是 slack 为 N/C。说明在 udt 中的约束没有起作用。不过，eta.tcl 中约束上了（没有附图）。

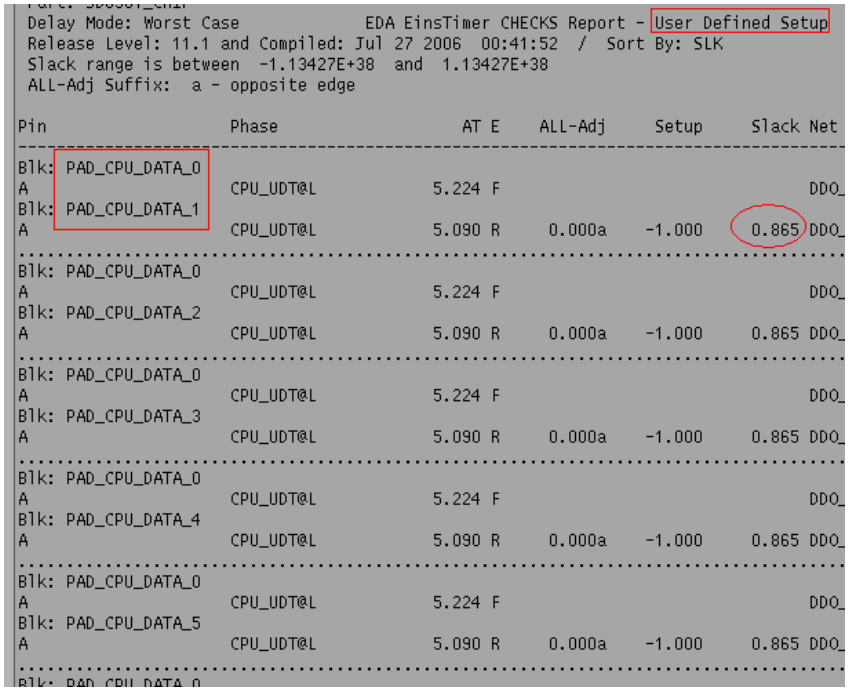


```
Delay Mode: Worst Case            EDA EinsTimer CHECKS Report - User Defined Setup
Release Level: 11.1 and Compiled: Jul 27 2006  00:41:52  /  Sort By: SLK
Slack range is between  -1.13427E+38  and  1.13427E+38
ALL-Adj Suffix:  a - opposite edge

Pin                  Phase                  AT E    ALL-Adj     Setup     Slack Net
-------------------------------------------------------------------------------------
Blk: PAD_CPU_DATA_0
A                    CPU@L                5.090 R                                DDO_SIC
Blk: PAD_CPU_DATA_1
A                    CPU@L                5.090 R     0.000a    -1.000       N/C DDO_SIC
.....................................................................................
Blk: PAD_CPU_DATA_0
A                    CPU@L                5.224 F                                DDO_SIC
Blk: PAD_CPU_DATA_1
A                    CPU@L                5.090 R     0.000a    -1.000       N/C DDO_SIC
.....................................................................................
Blk: PAD_CPU_DATA_0
A                    CPU@L                5.090 R                                DDO_SIC
Blk: PAD_CPU_DATA_1
A                    CPU@L                5.224 F     0.000a    -1.000       N/C DDO_SIC
.....................................................................................
Blk: PAD_CPU_DATA_0
A                    CPU@L                5.224 F                                DDO_SIC
Blk: PAD_CPU_DATA_1
A                    CPU@L                5.224 F     0.000a    -1.000       N/C DDO_SIC
.....................................................................................
Blk: PAD_CPU_DATA_0
A                    CPU@L                5.090 R                                DDO_SIC
Blk: PAD_CPU_DATA_2
A                    CPU@L                5.090 R     0.000a    -1.000       N/C DDO_SIC
.....................................................................................
```

图4-97 udt 与 eta 冲突时，udt 约束无效的时序报告

分析与解决：在 A 端用 clone_phase 克隆一个新的相位，在 udt 中使用用这个新的相位做 balance 检查。A 的相位为 CPU@L，克隆一个相位 CPU_UDT@L 专门用在 udt 中做约束。这样在 eta 中用相位 CPU@L 约束，而在 udt 中用相位 CPU_UDT@L 约束。这样就不存在冲突问题，在 udt 中做的约束也同样有效了。如图 4-98 所示，Apin 的相位为 CPU_UDT@L，但是 slack 为 0.865，说明在 udt 中的约束有效。eta 中也是有效的，图省略。



```
Delay Mode: Worst Case            EDA EinsTimer CHECKS Report - User Defined Setup
Release Level: 11.1 and Compiled: Jul 27 2006  00:41:52  /  Sort By: SLK
Slack range is between  -1.13427E+38  and  1.13427E+38
ALL-Adj Suffix:  a - opposite edge

Pin                  Phase                  AT E    ALL-Adj     Setup     Slack Net
-------------------------------------------------------------------------------------
Blk: PAD_CPU_DATA_0
A                    CPU_UDT@L            5.224 F                                DDO_
Blk: PAD_CPU_DATA_1
A                    CPU_UDT@L            5.090 R     0.000a    -1.000     0.865 DDO_
.....................................................................................
Blk: PAD_CPU_DATA_0
A                    CPU_UDT@L            5.224 F                                DDO_
Blk: PAD_CPU_DATA_2
A                    CPU_UDT@L            5.090 R     0.000a    -1.000     0.865 DDO_
.....................................................................................
Blk: PAD_CPU_DATA_0
A                    CPU_UDT@L            5.224 F                                DDO_
Blk: PAD_CPU_DATA_3
A                    CPU_UDT@L            5.090 R     0.000a    -1.000     0.865 DDO_
.....................................................................................
Blk: PAD_CPU_DATA_0
A                    CPU_UDT@L            5.224 F                                DDO_
Blk: PAD_CPU_DATA_4
A                    CPU_UDT@L            5.090 R     0.000a    -1.000     0.865 DDO_
.....................................................................................
Blk: PAD_CPU_DATA_0
A                    CPU_UDT@L            5.224 F                                DDO_
Blk: PAD_CPU_DATA_5
A                    CPU_UDT@L            5.090 R     0.000a    -1.000     0.865 DDO_
.....................................................................................
```

图4-98 udt 与 eta 冲突时，udt 约束有效的时序报告

4. 后仿应用 SDF 经常遇到的严重问题

问题：通常 IBM 利用其工具 chipedit 来提取寄生参数文件(RC 文件或 spice 文件)；ET 读入 spice 文件，run timing 之后写出 SDF 文件；后仿时反标 SDf 文件。SD8901 对 IBM 返回的 20061231 号的网表和 SDF（WC，已经修改负的 hold 的文件）做仿真的结果如图 4-99 所示；IBM 返回的 20070108 号的网表和 SDF（WC，已经修改负的 hold 的文件）仿真的结果如图 4-100 所示。

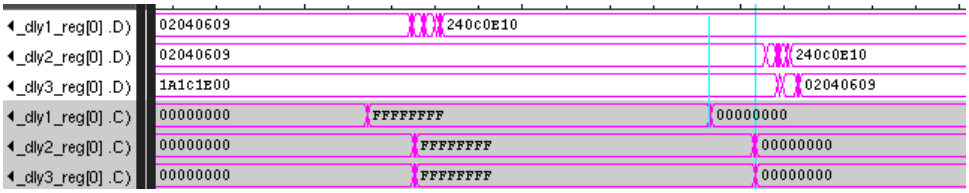


图4-99 反标 SDF 正确后仿的结果

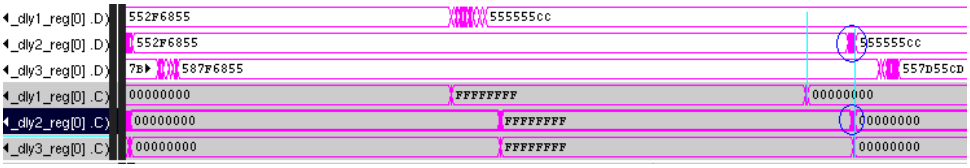


图4-100 反标 SDF 不正确后仿的结果

分析：

- 图 4-99 为 12 月 31 号的结果。可以看出，时钟在数据的前面。第 1、4 行是 dly1[31:0]的数据和时钟；第 2、5 行是 dly2[31:0]的数据和时钟；第 3、6 行是 dly3[31:0]的数据和时钟。时钟有 F—>0 的变化，即下降沿采样。每个寄存器的 D 端对应一个时钟 C。为方便起见，表示成一个时钟 bus 类型。
- 图 4-100 为 1 月 8 号的结果。可以看出，dly2[31:0]的时钟和数据是不定的，有的 bit 的数据在时钟前面有的在后面。导致数据采错。
- 更有个奇怪的现象：由于 1231 和 0108 两次的逻辑没有做任何的改动，只是修改了 DRC。SDF 的文件经过 diff，只有 7 个 cell 的延时信息不同，其他相同。出现数据采错的 cell 的延时信息完全相同。那么应该延时信息应该一致才对？

解决：经过多次的反复定位，问题出现在 verilog 和 SDF 文件中的 cell 或信号名不一致造成的！这样 verilog 在 SDF 反标的时候无法对应这些信号名不一致的延时。图 4-101 是 ncsim 在编译 SDF 时的告警信息：

```
ncelab: *W,SDFINF: Instance hold_fix_N112 not found at scope level <top-le
vel> </project6/SD8901/vd2hw/SD8901V100_100netlist_baseline/netlist/200701
08/sdf/SD8901_WC_update.sdf.gz, line 4617401>.
```

图4-101 ncsim 的告警信息

我们再看看 SDF 和 verilog 中的这两个信号。

Verilog 代码中，Instance 是 “hold_fix_N112_C”；而 SDF 文件中为“hold_fix_N112”没有“_C”。这样反标 SDF 时无法找到相应的 cell，延时无法标上。

问题主要出现在 IBM 的 ET 工具产生 SDF 和 verilog 的流程上。正确的流程如下：

- 首先 run timing （worst case 下，普通模式下 nonVAT 模式，选项如下：WC/BC，nonLCD，Full Cppr， -nbti off，PLL early all 或 PLL late all）。
- 在图形界面生成 verilog：一定要选上“Use Save Names”，如图 4-102 所示。这个选项会允许其他的命令，如 write_sdf 产生的文件可以与 verilog netlist 相对应。这个是一个关键选项。否则后面生成的 SDF 可能又会出现与 verilog netlist 不一致的情况。所以这个步骤不能省却！不管需不需要这个 verilog 文件。

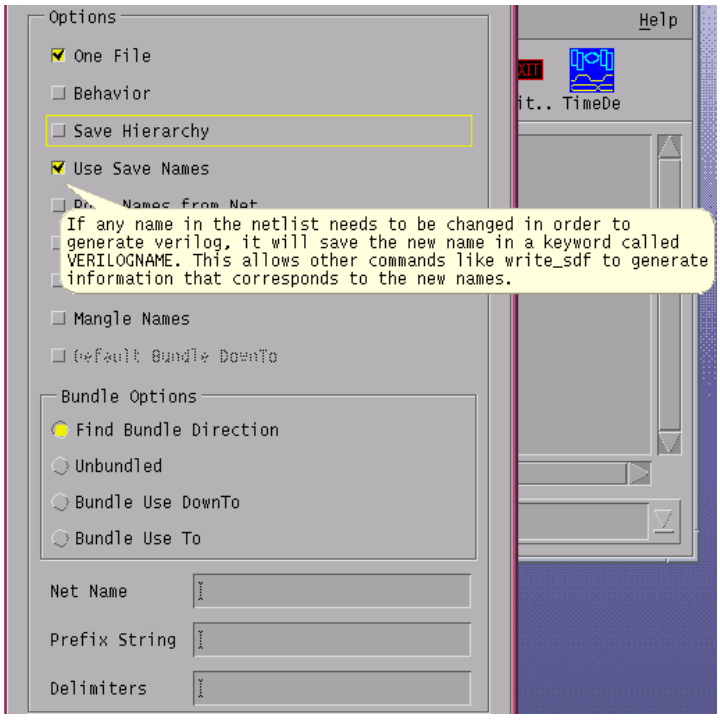


图4-102 ET 中生成 SDF 文件的选项

- 在 ET 中运行下列的脚本可生成 SDF 文件，如图 4-103 所示。

```
write_sdf -file "${out_dir}/${top}_${op}.sdf" \
    -divider "." \
    -version 2.1 \
    -aussim \
    -detail_all \
    -delay_mode_current \
    -current_compressed \
    -no_primary_delay \
    -verilog_names;
```

图4-103 生成 SDF 的脚本

经验：由于很多 IBM 的具体实现过程对我们来说是屏蔽的，所以有时候出了问题并不是很好定位。如果再加上 IBM 的设计人员经验不是很丰富，出现这种问题的几率就会增加。上述问题就是因为我们对 IBM 的流程得不到很好的监控造成的。因此，需要我们的设计人员要完善自己的后端流程知识，获得监控 IBM 流程的能力。

5. Chopper 时钟设置很可能出现的误区

问题：通常情况下，设置 chopper 时钟总是惯性的把以前的脚本中的设置方法“挪用”到我们的脚本中，殊不知存在着较大的陷阱。

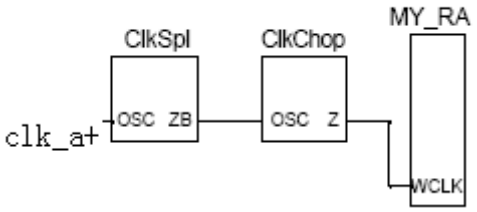


图4-104 chopper 电路正相位输入

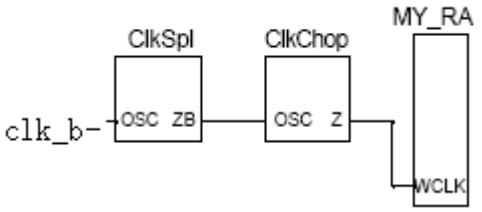


图4-105 chopper 电路负相位输入

SD8901 电路中同时存在两种情况的 chopper 电路，如图 4-104 和图 4-105 所示。这是由于时钟前面有个选择正反两相的 mux，两个相反的相位传递到 OSC 端。

（1）通常的 chopper 时钟电路结构如图 4-104 所示，clkspl 的输入端为正相位，ZB 为 clk_a+。在 phase.tcl 和 dcadj.tcl 中约束如下：

phase	et::create_clock –n clk_a –p 20 –w {0 10}; et::create_clock –n clkchp_a –p 20 –w {0 0.1};
dcadj	et::rename_phase –pins OSC –cell_type CLKCHP_Q –phase “clk_a+” –new_phase “clkchp_a+”; et::rename_phase –pins Z –cell_type CLKCHP_Q –phase “clkchp_a-” –new_phase “clkchp_a+”;

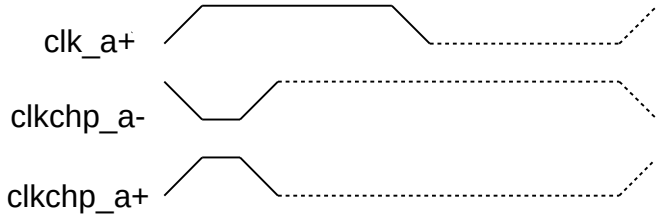


图4-106 时钟定义示意图

phase 定义中 leading edge 都是 0 起点。OSC 的相位从 clk_a+ 变成 clkchp_a+，无需调整。因为实际 chopper 脉冲波形就是如图 4-106 所示。Z 端的相位有两个：clkchp_a+，clkchp_a-。其中 clkchp_a-与实际波形不符，所以直接把 clkchp_a-相位 rename 成 clkchp_a+。报告中会有极性问题的 warning，可以忽略。

（2）如果 clkspl 的输入端为负相位，ZB 为 clk-，如图 4-105 所示。那么约束就会不同，很容易漏掉。如果仍然按照上如定义：

phase	et::create_clock –n clk_b –p 20 –w {0 10}; et::create_clock –n clkchp_b –p 20 –w {0 0.1};
dcadj	et::rename_phase –pins OSC –cell_type CLKCHP_Q –phase “**clk_b-**” –new_phase “**clkchp_b+**”; et::rename_phase –pins Z –cell_type CLKCHP_Q –phase “**clkchp_b-**” –new_phase “**clkchp_b+**”;

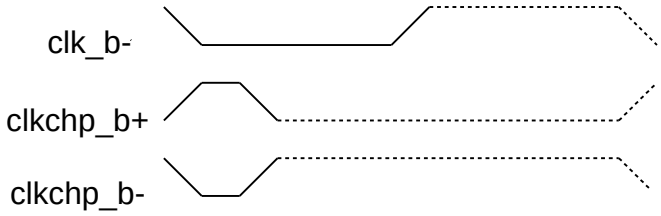


图4-107 时钟定义示意图

那么产生的波形如图 4-107 所示，并非实际电路 chopper 的脉冲波形。同时还会报极性问题的 warning。因为实际电路的脉冲是在 clk_b-的上升沿之后出现。

出现上述情况，应该在 phase 中定义与实际情况相一致的时钟，chopper 脉冲应该与 clk_b-的上升沿是一致的。如图 4-108 所示实线部分。由 clk_b-相位变成 clkchp_b+，如何调整？显然 clk_b-上升沿无需调整，与 clkchp_b+的上升沿一致；而下降沿需要调整成在上升沿之后，所以调整一个时钟周期。

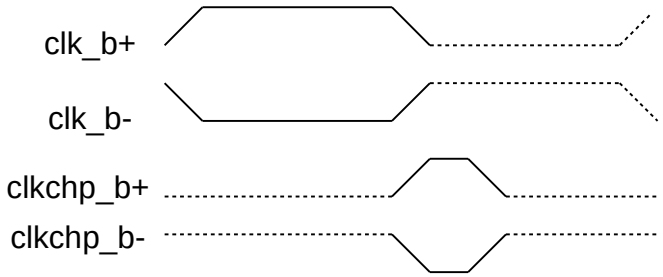


图4-108 时钟定义示意图

phase	et::create_clock –n clk_b –p 20 –w {0 10};＃如图 clk_b+和 clk_b- et::create_clock –n clkchp_b –p 20 –w {10 10.1};＃如图 clkchp_b+和 clkchp_b-
dcadj	et::rename_phase –pins OSC –cell_type CLKCHP_Q –phase “**clk_b-**” –new_phase “**clkchp_b+**” **-rise**; et::adjust_and_rename –pins OSC –cell_type CLKCHP_Q –phase “**clk_b-**” –new_phase “**clkchp_b+**” **–fall –time 20**; et::rename_phase –pins Z –cell_type CLKCHP_Q –phase “**clkchp_b-**” –new_phase “**clkchp_b+**”;

6. IBM 返回插入时钟树的网表，如何进行时序分析

问题：在后端进程中，IBM 返回了插入时钟树，但没有时钟 wire 的网表（上述第 4 步完成）。即只有 Macros 和 Std cells 的布局和时钟树布局的信息，没有布线（clock wire 和 cell wire）信息。为什么这时候的 netlist 不能用 ET 来分析？

分析与解决：先说说 IBM 的后端流程。

IBM 的后端流程主要的几个步骤如下：

- Placement Of Macros and IOs:摆放大的 Macros 和 IO。
- fix_all_placed_cells:放置一些预先保留的 cell，如时钟 mux 和 clock divider register，PDS（Physical Driving Synthesis）不会优化掉这些 cell。
- Full place-PDS placement:放置标准单元 cell。
- clock tree build：时钟树的创建，也就是摆放时钟 buffer。
- IO wiring：IO 布线
- clock wiring：时钟树布线
- detail wiring：整体的详细布线

在后端进程中，IBM 返回了插入时钟树，但没有时钟 wire 的网表（上述第 4 步完成）。由于只有 Macros 和 Std cells 的布局和时钟树布局的信息，没有布线（clock wire 和 cell wire）信息，所以需要用 chipbench 来分析 timing。

chipbench 工具类似与 synopsys 的 astro 自动布局布线工具，IBM 用 chipbench 来做 floorplan、placement 和 routing。上述网表完成了 floorplan 和 placement，虽然没有 routing，但 chipbench 会根据布局后的单元位置信息来计算连线延迟，时序结果和详细布线之后的连线延迟非常接近。

ET 工具则不同，在没有 spice 或 RC 文件之前，ET 计算的 wire 的延时是根据 fanout（换算成线长）来估算线延时，时序分析结果很不准确。

chipbench 和 chipedit 的区别：chipbench 是自动布局布线工具，类似与 synopsys 的 astro 工具；chipedit 是 manual 布线工具，类似于 candence 的 virtuso 工具。一般来说，布局布线大部分有 chipbench 自动布局布线 run 出来；若有小的改动则用 chipedit 来手动更改布线。

此阶段在 chipbench 中应该选择如下的 options：

wc/lcd/estimate_cppr/pll_jitter-late/parasitic_type-none。

7. SRAM2DR 应用时两时钟 CCLKA 和 CCLKB（接同一时钟）如何做 clk seperation 检查

问题：IBM 的 SRAM2DR 存在两个时钟，一般来说一个写时钟一个读时钟。如果两个时钟为同一时钟，可能存在读写地址冲突（读写地址相同）问题。在读写地址冲突时，读出老数据还是新数据由两个时钟之间的相位差决定，如图 4-109 所示。这里有 Tabs 和 Tbas 的要求值，SRAM 的 srule 中会有这个相位差值，在时序分析时会用到此值作为 guardtime 值。如何在脚本中约束并做 timing-driven 的 P&R？

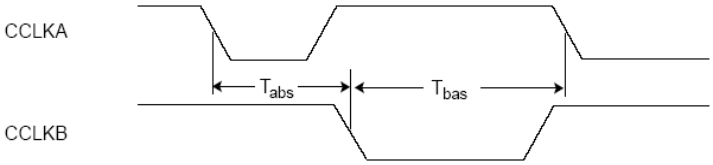


图4-109 CCLKA 和 CCLKB 相位关系

分析与解决：做时序约束前需要设计人员提供两个方面的信息（1）读写时钟 CCLKA 和 CCLKB 要分清，哪个是用作写时钟，哪个用作读时钟。（2）在读写地址冲突时，读出的数据是老数据还是新数据。

以实例说明，原电路图连接如图 4-110 所示。假设设计 CCLKA 用作写时钟，CCLKB 用作读时钟；要求当读写地址冲突时希望读出新数据，需要 CCLKB 的相位要滞后于 CCLKA，所以需要插入一些 buffer 用于相位 seperation，如图 4-111 所示。根据上述信息，需要在 phase.tcl 和 dcadj.tcl 两个文件中做相关的约束。

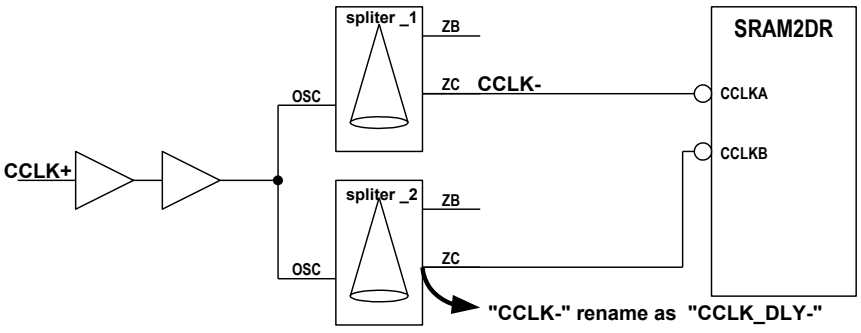


图4-110 SRAM2DR 应用实例：原电路

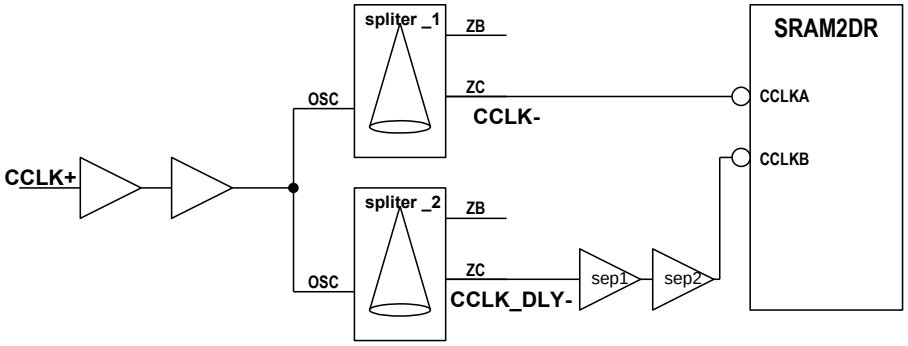


图4-111 SRAM2DR 应用实例：有 seperation 的电路

（1）由于读写地址冲突时要读出新数据， CCLKB 的相位要滞后与 CCLKA。相位如图 4-109 所示。所以在 phase.tcl 中定义要体现这种相位差。CCLKA 和 CCLKB 的初始相位均为“CCLK-”，所以要使 CCLKB 的相位为滞后于 CCLKA 的相位，需要额外创建一个时钟“CCLK_DLY”。

phase.tcl
et::create_clock -name "CCLK" -period $period_clk -waveform {0 10} et::create_clock -name "CCLK_DLY" -period $period_clk -waveform {0+Δx 10+Δx }

Δx 可以为较小的一个正值。

（2）在 dcadj.tcl 文件中需要 rename 定义。

dcadj.tcl
et::rename_phase –pins “” –phase “” –new_phase “”;

没有 rename 之前，CCLKA 和 CCLKB 的相位都是“CCLK-”，所以 ET 对同一个时钟分析当然是认为没有相位差，肯定不会做 seperation 的检查。rename 之后，“CCLK_DLY-”相位滞后“CCLK-”，ET 发现两个不同的相位分别连接到 CCLKA 和 CCLKB 两个 pin 上，根据 srule 描述的 seperation 的值，做 clock seperation 的检查。P&R 工具以 timing 为驱动的布线工具，它会根据时序约束插入 sep1 和 sep2 等 buffer 以满足时序的要求，如图 4-111 所示。

8. 脉宽检查有哪几种，都是如何做检查的

问题：ET report 中有几种 pulse width 检查报告：pulse width，clock tree pulse width 和 clock tree inactive period 三种检查。那么它们分别检查的是什么？又是如何做检查的那？

分析：下面我们一一做分析。

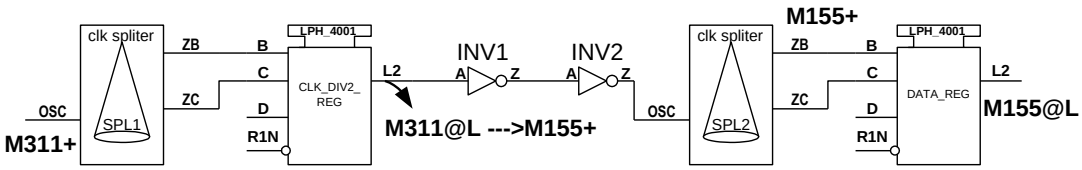


图4-112 时钟树示意图

(2) pulse width

检查寄存器如 LPH4001_LPC_*的输入时钟端 B 和 C 的脉冲宽度。图 4-112 中的两个寄存器 CLK_DIV2_REG 和 DATA_REG 中的 B 和 C 都要做 pulse width 检查，时序检查如图 4-113 和图 4-114 所示。可见，正相位检查高电平的脉冲宽度，负相位检查低电平的脉冲宽度。

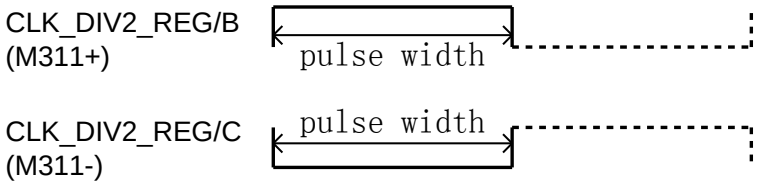


图4-113 CLK_DIV2_REG 的 B 和 C 的 pulse width 检查

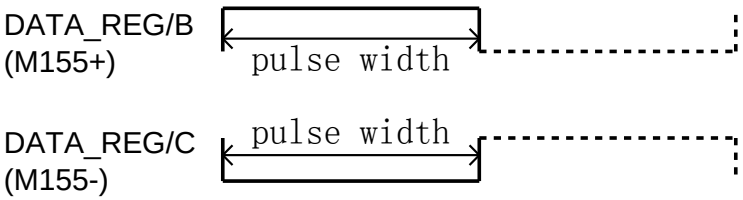


图4-114 DATA_REG 的 B 和 C 的 pulse width 检查

(3) clock tree pulse width

检查时钟树上所有器件的输入输出 pin 上的时钟脉冲宽度。时钟树起点是时钟相位定义的起点。图 4-112 中，M155+时钟树起点的定义在 CLK_DIV2_REG/L2 端，那么下列点将会检查 clock tree pulse width：“CLK_DIV2_REG/L2”，“INV1/A”，“INV1/Z”，“INV2/A”，“INV2/Z”，“SPL2/OSC”，“SPL2/ZB”，“SPL2/ZC”。正相位检查高电平的脉冲宽度，负相位检查低电平的脉冲宽度，请参考图 4-115。

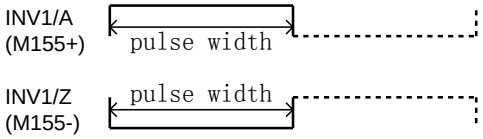


图4-115 INV1 的 pulse width 检查

不过，如果 M155+的时钟定义在 INV1/Z 端定义，那么检查点就不包含“CLK_DIV2_REG/L2”，“INV1/A”。如图 4-116 所示。

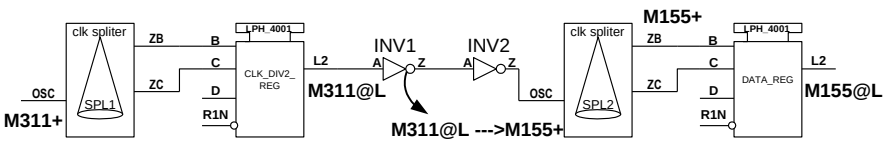


图4-116 M155+时钟定义在 INV1/Z 的情况

(4) clock tree inactive period

检查时钟树上所有器件的输入输出pin上的时钟inactive的脉冲宽度，即inactive period。时钟树起点是时钟相位定义的起点。以图 4-112 为例，检查点同(3)的检查点：“CLK_DIV2_REG/L2”，“INV1/A”，“INV1/Z”，“INV2/A”，“INV2/Z”，“SPL2/OSC”，“SPL2/ZB”，“SPL2/ZC”。不过检查的是“非有效时钟区域”的脉宽。

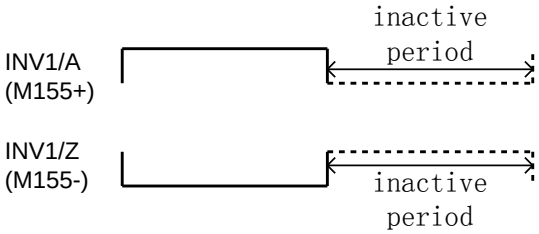


图4-117 INV1 的 inactive period 检查

上述讲了三种 pulse width 的检查。

ET 在检查的过程中总是以最坏的情况做检查。

9. 如何对 MUX 的两个输入时钟或数据做 skew 约束

```
#-------------------------------------------------------------------------------#
# MUX U_HTBCLKSEL two input clocks: skew constraints
#-------------------------------------------------------------------------------#
set   HTBCLKSEL_MUX
"SD8901_TOP/U_LOGIC/U_CLKRST_GLB/U_HTBCLKSEL"
et::set_user_defined_skew_test -pins "${HTBCLKSEL_MUX}/D0
${HTBCLKSEL_MUX}/D1" \
      -bc_skew      0.3       \
      -nom_skew     0.3       \
      -wc_skew      0.3       \
      -rise_fall              \
      -same_mode_late;
et::set_user_defined_skew_test   -pins   "${HTBCLKSEL_MUX}/D0
${HTBCLKSEL_MUX}/D1" \
      -bc_skew      0.3       \
      -nom_skew     0.3       \
      -wc_skew      0.3       \
      -rise_fall              \
      -same_mode_early
```

上述检查的是 U_HTBCLKSEL 选择器件的两个输入 pin：D0 和 D1 之间的 skew 值。第一个约束是最晚（-late）情况下，检查 D0 和 D1 的 rise 和 fall 的所有组合的 skew，即 D0:D1=rise:fall，rise:rise，fall:rise，fall:fall 共四种约束。第二个约束是最早(-early)情况下，检查 D0 和 D1 的 rise 和 fall 的所有组合的 skew，即 D0:D1=rise:fall，rise:rise，fall:rise，fall:fall 共四种约束。

PrimeTime：通常可以利用 set_data_check 命令。

10. 如何做 bus 的 balance 约束

```
#-------------------------------------------------------------------------------#
# CPU_BUS output balance:skew constraints
#-------------------------------------------------------------------------------#
set CPU_DATA_PINA_LIST [list ]
for {set i 0} {$i<=15} { incr i } {
      set CPU_DATA_PINA_LIST [concat $CPU_DATA_PINA_LIST
PAD_CPU_DATA_${i}/A]
}
et::set_user_defined_skew_test -pins ${CPU_DATA_PINA_LIST} -phase
"CPU_UDT@L" \
      -bc_skew       0.5     \
      -nom_skew      0.5     \
      -wc_skew       0.5     \
      -same_mode_late;
et::set_user_defined_skew_test -pins ${CPU_DATA_PINA_LIST} -phase
"CPU_UDT@L" \
      -bc_skew       0.5     \
      -nom_skew      0.5     \
      -wc_skew       0.5     \
      -same_mode_early
```

上述是 PAD_CPU_DATA_0~15/A 之间所有两两组合的的 skew 约束。同 9. 中的做法。

PrimeTime：通常可以利用 set_data_check 命令。set_data_check 有个缺点就是每次只能为两个点设置 skew 检查，如果一个 bus 中存在多个约束点，则设置起来比较麻烦。

11. 如何做随路时钟和随路数据之间的时序约束

```
#-------------------------------------------------------------------------------#
# HTB udt
#-------------------------------------------------------------------------------#
set output_htb [list \
AHTBRFP       \
DHTBFP        \
DHTBD_0       \
DHTBD_1       \
]
foreach p $output_htb {
        # M51N0+ VS. M51@L
       et::set_user_defined_test     -late     -pin1 "PAD_${p}/A" -pin2
"PAD_DHTBCLK/A"    \
                                                -rise2 -b 9.600 -n 9.600 -w 9.600 \
                                                -pin1_phase "M51@L" -pin2_phase
"M51N0+";
       et::set_user_defined_test     -early    -pin1 "PAD_${p}/A" -pin2
"PAD_DHTBCLK/A"    \
                                                -rise2 -b 3.000 -n 3.000 -w 3.000 \
                                                -pin1_phase "M51@L" -pin2_phase
"M51N0+"
}
```

上述约束数据 PAD_AHTBRFP/A，PAD_ DHTBFP/A，PAD_DHTBD_0/A，PAD_DHTBD_1/A 和时钟 PAD_DHTBCLK/A 之间的相位关系，也就是随路数据和随路时钟之间的关系。

不过这个命令要注意下列两点：（1）pin2/rise2/pin2_phase 都是指“时钟”，而 pin1/rise1/pin1_phase 都是指“数据”；（2）“-late”实际上是“setup”值，如上面的 9.600ns 实际上是 setup guardtime 值；“-early”实际上是“hold”值，如上面的 3.000ns 实际上是 hold guardtime 值，不要被表面迷惑了。上述约束的电路如图 4-118 所示，时序关系如图 4-119 所示。

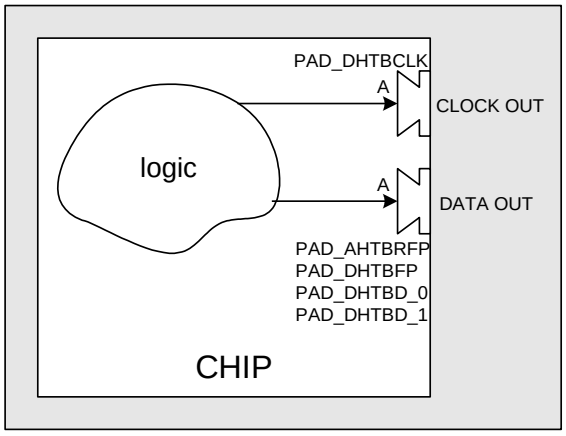


图4-118 随路时钟和随路数据输出约束电路示意图

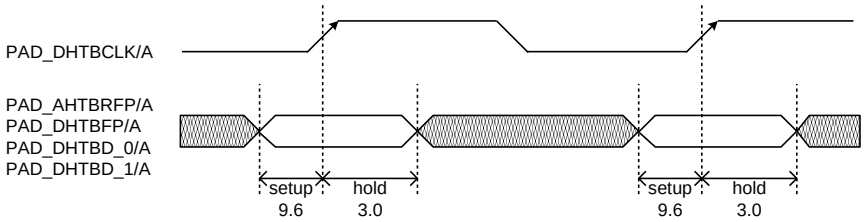


图4-119 随路时钟和随路数据输出约束时序示意图

PrimeTime：在时钟的输出端口创建一个时钟，然后用此时钟的随路数据做 set_output_delay 约束。

12. 如何加 hold 裕量约束

问题：有时候我们希望给 hold 加上一些裕量，即增加了芯片的成功率，又可以给后仿真带来方便。那么在脚本中如何做约束那？

分析：一般来说用 set_user_delta_adjust -data_clock 命令可以给 hold 加上余量。

```
set_user_delta_adjust -data_clock CLK1 -clock CLK1 -adjust +0.010 –hold
set_user_delta_adjust -data_clock CLK2 -clock CLK2 -adjust +0.010 –hold
set_user_delta_adjust -data_clock CLK1 -clock CLK2 -adjust +0.010 –hold
set_user_delta_adjust -data_clock CLK2 -clock CLK1 -adjust +0.010 –hold
```

假设 CLK1，CLK2 为同步时钟。第一行约束了 CLK1 时钟域之间数据交互时 hold 检查加上 0.010ns 的余量。第二行约束了 CLK2 时钟域之间数据交互时 hold 检查加上 0.010ns 的余量。第三行约束了“CLK1 时钟打出的数据”与“CLK2 时钟”之间数据交互 hold 检查加上 0.010ns 的余量。第四行约束了“CLK2 时钟打出的数据”与“CLK1 时钟”之间数据交互 hold 检查加上 0.010ns 的余量。

上述命令用一个函数来写非常方便：

```
#procedure for hold margin
proc UserDeltaAdj_hold {clk1 clk2} {
if { $clk1 != $clk2 } {
        set_user_delta_adjust -data_clock   $clk1 -clock $clk2 -adjust +0.010 -hold;
        set_user_delta_adjust -data_clock   $clk2 -clock $clk1 -adjust +0.010 -hold;
} else {
        set_user_delta_adjust -data_clock   $clk1 -clock $clk1 -adjust +0.010 -hold;
}
}
```

这样只要调用这个函数就可以了：“UserDeltaAdj_hold CLK1 CLK2”

PrimeTime：使用命令 set_clock_uncertainty –hold xx 来加 clock jitter，而且，这个 jitter 值不会被 CRPR（与 ET 中的 CPPR 一样）优化掉。

13. 如何约束异步电路延时

内容包括： 同步器、格雷码的 MAXDELAY、摆放；及 MPI 回读的 multi-cycle 等。

由于异步电路发送时钟和采样时钟相位的不确定关系，导致时序采样的不确定性。异步电路除了在电路设计时候需要采用合适的同步电路，还需要结合在 STA 中进行正确的约束才能保证异步电路正常工作。

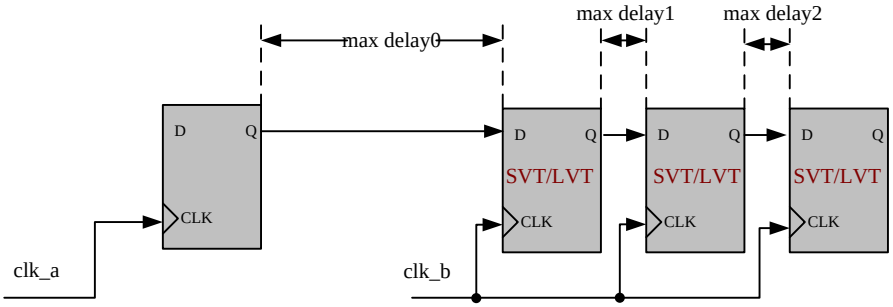
设计中常见的异步电路约束主要分两种，单比特异步路径约束和多比特异步路径约束。

(1) 单比特异步路径同步约束

单比特异步路径约束常见的有两种电路，单比特数据同步和异步复位同步撤离。

- **单比特数据同步**

单比特数据路径同步电路如下图所示：



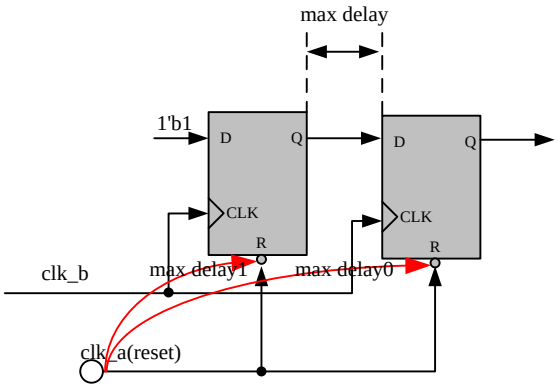
后端约束重点：

1、 max_delay0约束，主要约束异步时钟域之间的最大延时，由电路设计功能决定该延时值大小；

2、 max_delay1和max_delay2约束，约束同步寄存器之间的最大延时，减小亚稳态概率发生；

3、 同步寄存器最好采用SVT或LVT器件，减小亚稳态概率发生；

- **异步复位同步撤离电路：**



后端约束重点：

1、 max_delay约束，约束同步寄存器之间的最大延时，减小亚稳态概率发生；

2、 max_delay0和max_delay1的最大延时约束，主要约束时钟域之间的最大延时，由电路设计功能决定该延时值大小。

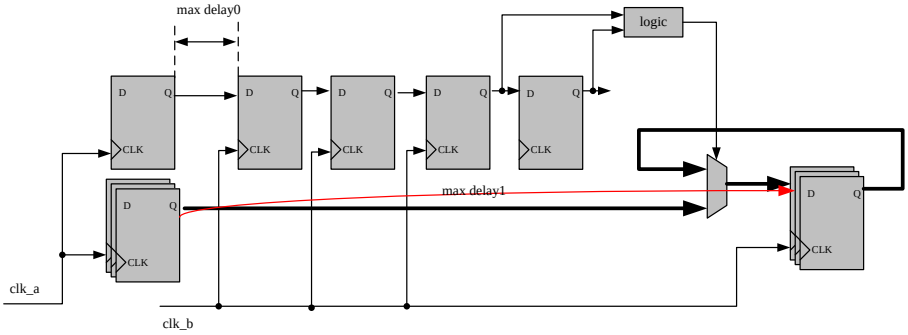
3、 max_delay0和max_delay1的skew约束，如果max_delay0 – max_delay1 > Tclk_b，该复位同步电路将无法达到同步撤离效果。

(2) 多比特异步路径同步约束

多比特异步路径约束常见的有三种电路，带使能的多比特数据同步、格雷码数据同步和异步 fifo 数据同步。

- **带使能的多比特数据同步**

多比特数据路径同步电路如下图所示：



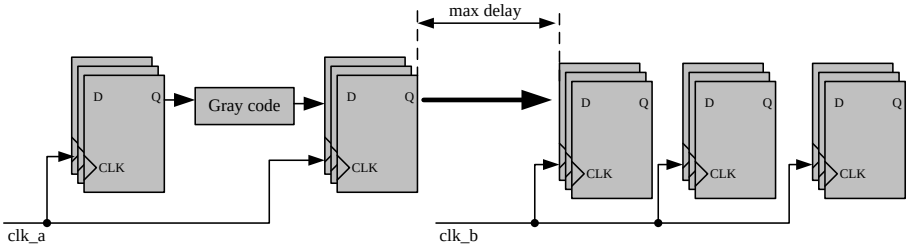
后端约束重点：

- max_delay0约束，约束异步单比特同使能信号最大延时；
- max_delay1约束，约束多比特同步总线信号最大延时，根据设计中单比特信号和多比特信号的时序关系确定max_delay1的最大延时设

置。例如单比特信号和多比特信号在clk_a时钟域对齐时，延时要求max_delay1<max_delay0+3Tclk_b。

- 格雷码数据同步

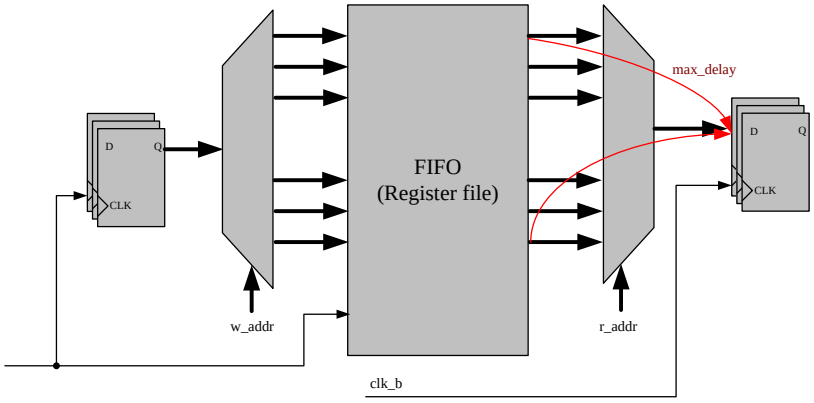
格雷码数据同步电路如下图：



后端重点约束：

- Max_delay约束，约束异步格雷码数据最大延时，该延时尽可能约束小；
- 异步fifo数据同步

设计中异步 fifo 一般采用实用 register file 搭建的 fifo 或者直接使用 Vendor 提供的 ram，使用 vendor 提供的 ram 在 RAM 的输出数据端会有读时钟的 timing arc，因此 ram 读数据端口到读数据寄存器之间可以进行同步检查，而使用 register file 搭建的 fifo，寄存器的时钟使用写时钟，因此，该 fifo 寄存器到读寄存器是异步路径。



后端约束重点：

需要约束所有fifo中寄存器到读寄存器的max delay，最大延时需要小于读写地址的间隔时间，保证正在读取的数据存储单元不被新数据写入。

14. 由于库丢失文件，导致的 3Dnoise 问题

在 RTC 中，STA 会启动 3Dnoise 分析，其中会出现一些告警，比如：

*[3dnoise]: (W) duplicate data in noise abstracts: macro "OLVDS15_A", pin "PADN".*这个告警是说调用了两次 abstracts，可以忽略。

*[3dnoise]: (W) sink sensitivity data not found for pin: XXX/pin, infinite windows will be used.*这个告警是噪声分析更加严格，也可以忽略。

[3dnoise]: (W) failed to locate noise abstract file for block: PCI.abstract. 这个告警出现的原因是因为库中没有相应器件的 3Dnoise 模型，本文中 PCI 缺少 3Dnoise 模型，从而出现这个告警。在 IBM 后端流程中，PCI 是一种物理设计器件，它的噪声可以不用去分析，但是如果我们逻辑中使用的器件缺少 3Dnoise 模型，就是库有问题了。SD8901 出现了这个问题，是许多定制的 RAM 缺少 3。

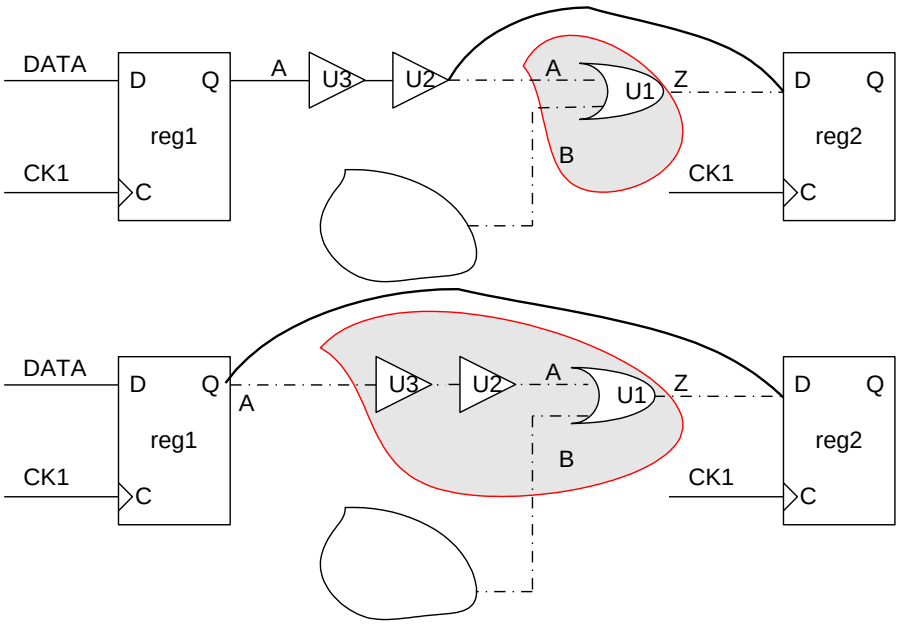
3Dnoise 模型文件在库的位置是：…/ibm_cu11/13.0（fix）/noise/.下，每个器件都应当有它对因的 abstract 文件，如果没有，则需要联系 Vendor 解决。

15. ECO 遇到的问题

问题：布局布线完成且 timing 收敛之后发现芯片需要 ECO：需要将一段中间逻辑去掉，直接连接源寄存器的输出和宿寄存器的输入。然而，EC 之后，再次 run timing 发现有很大的 setup violation 问题。正常来讲，如果去掉了一些 cell，应该引起 hold violation，为什么这里却出现了 setup violation？

分析与解决：placement 和 routing 之后，所有的 cell 的位置都是固定的。做 ECO 只是小范围内的改动，一般不做 cell 位置的改动。

第一次 eco 脚本是将 U1,U2,U3 全部去掉，然后连接 reg1 和 reg2。但是由于 reg1 和 reg2 的 place 距离很远，导致 wire 连接很长，发生非常大的 slew(transition)问题，由此引发了 setup violation。U2 和 U3 两个 buffer 是用于增加驱动的两个 cell，可以有效的减少 slew 值。因此我们第二次 eco 脚本只将 U1 去掉，而保留了 U2 和 U3，这样 IBM 可以无需为了解决 slew 而额外的重新加上 buffer。



4.5 CDC 检查 Clock Domain Crossing Analysis

概述：重点为基本概念、实现流程、脚本编写，注重实用性和可参考性。

随着芯片规模的增加，人工 review 异步设计效率低下，CDC 检查可以工具可以快速检查出大量的包括很隐蔽的异步路径，挖掘电路异步设计中的潜在风险。

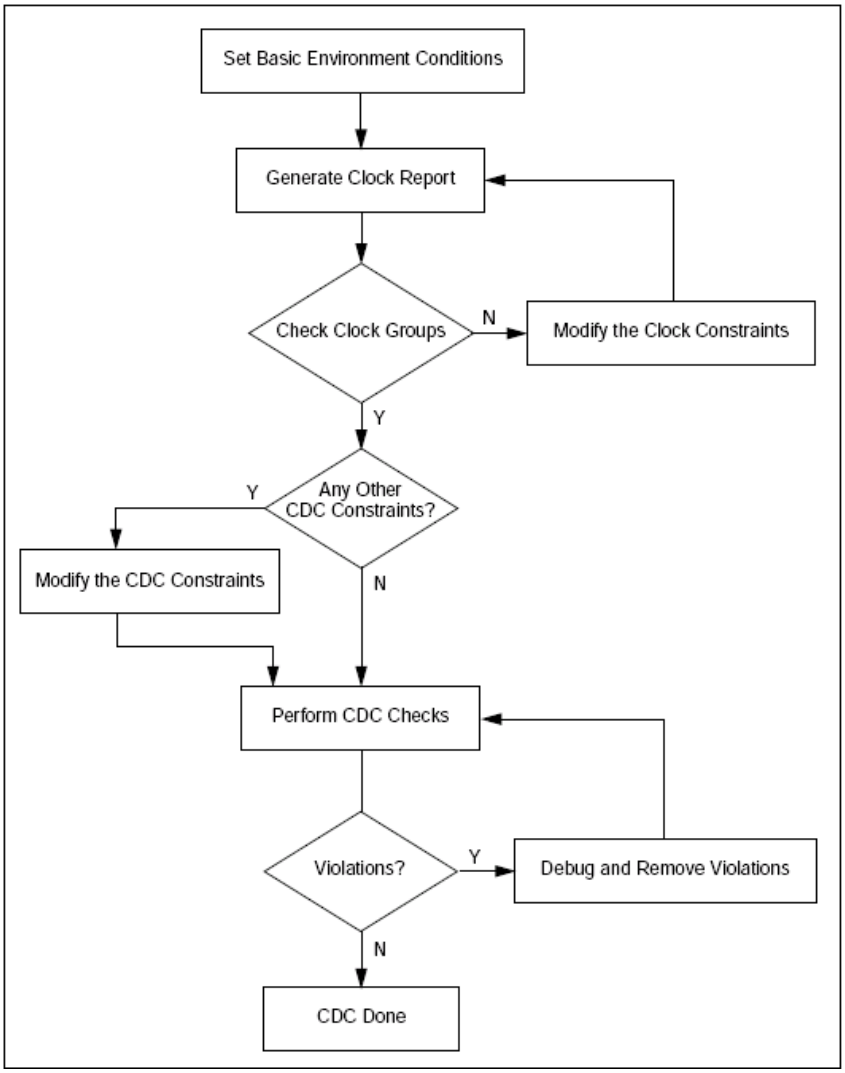
4.5.1 CDC 检查基本概念

CDC 检查用于发现芯片的异步处理问题，属于基于 RTL 代码的分析检查。CDC 识别时钟和时钟域关系的方式有两种：一、根据 Design 中的 register 来自动 Infer 时钟（也包括复位），不同连接的时钟默认为异步即不同 Group；二、根据 Ctrl 约束文件所显式定义的时钟及其 Group 属性来识别，不同 Clock name 可以有相同的 Group 属性即同步关系。异步 Clock 之间的逻辑路径就是 CDC 路径。工具通过分析这些路径上的逻辑处理来判断其有无异步处理，异步处理即同步器的结构（类别）和其 Severity 级别。其报告的同步器类别和 Severity 基于工具自定义的一套 Synchronizer Schemes；

4.5.2 CDC 检查工具与脚本

1. CDC 检查简要流程

CDC 检查流程如图所示，首先建立一个基本的 CDC 运行环境，初步运行产生时钟报告，然后确认时钟分组后，根据要求修改时钟约束，然后进行 CDC 检查，修改时钟约束或代码并反复迭代，直到所有异步路径都确认无误。



2. CDC 工具的运行

CDC 检查使用 Mentor 公司的 CDC compiler 工具，只需要在 C shell 下按如下格式输入命令即可。

```
0in -od ./demo \
-cmd cdc \
-f /home/z190847/demo_filelist.f \
-d DEMO \
-ctrl ./0in_ctrl_demo.v
```

-od：指定输出报告的文件夹。

-f： 指定 RTL 代码的文件列表。

-d：指定顶层模块名。

-ctrl：指定 CDC 约束文件，如果没有约束文件，可以直接按默认约束检查。

CDC 约束文件是一个.v 文件，内部需要定义一个空的 module，中间用注释的形式添加约束，新版 CDC compiler 已经支持专门的脚本格式，不需要使用注释。

3. CDC 常见约束命令

(1) 设置时钟属性

0in set_cdc_clock clk_sys –module DEMO

指定 CCIF 模块的端口 clk_sys 为时钟属性。

0in set_cdc_clock clk_a clk_b –group CLK_SYS –module DEMO

指定 DEMO 模块的端口 clk_a、clk_b 为时钟属性，并且同为 CLK_SYS 时钟域。有的模块有多个时钟输入，并且这些时钟同源，可以将他们设置成一个 group。

(2) 指定端口的时钟域

set_cdc_port_domain data_in –clock clk_sys –module DEMO

设置 DEMO 模块端口 data_in 的时钟域为 clk_sys 时钟域。

set_cdc_port_domain data_in –async –module DEMO

设置 DEMO 模块端口 data_in 为异步信号，不属于任何时钟域。

(3) 设置信号属性

set_cdc_signal U_MPI.chip_mode {–stable|-gray_coded|-control|-data},

-stable 表示该信号是静态信号，将其设置 stable 属性，可以屏蔽这些信号的时序路径，不做检查。合理设置静态信号，可以极大地减少报告数量。

设置 stable 属性要采取保守策略，完全确定是静态信号才可设置，如果不是很明确是否静态，尽量不设置。

-gray_coded 表示该信号是经过格雷码编码的，异步 FIFO 中的常用设置。

-control 表示该信号是控制信号。

-data 表示该信号为数据信号，需要使用异步 FIFO、DMUX 等多比特同步的手段来处理跨时钟域。**默认情况下工具认为多比特的信号如 A[7:0] 为 data 类型，单比特的为 control 类型。通过这两个选项可以改变这种默认规则**。

(4) 指定同步器类型

set_cdc_synchronizer dff –level 3 –tx_clock tck –rx_clock rck

设置 tck 到 rck 所有路径的同步器必须寄存三拍，小于三拍将报告 violation。工艺制程下降到 45nm 以后，传统的寄存两拍的方法可能不能将亚稳态概率降低到可接受的范围，往往需要寄存三拍以上。如果不带-tx_clock 和 -rx_clock 参数，则表示所有同步器都将按三拍检查。

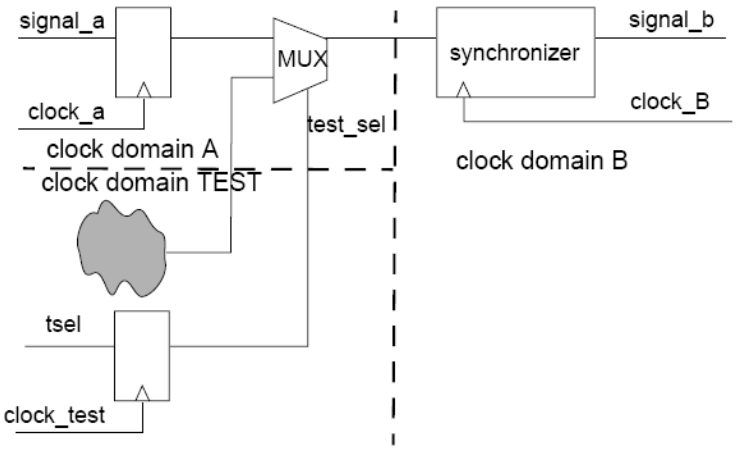
set_cdc_synchronizer custom –module MY_SYNC –clock clk –pins data_in data_out

有些厂家可以提供专用的同步器。如果使用厂家提供的 cell 做同步处理，指定自定义的同步处理单元。

(5) 设置常数信号

set_constant test_mode 1'b0

这条命令的用途在于可以指定电路中的诸如 DFT、FuncSel 等常量 Ports 从而避免由此带来的 CDC 分析异常等假像。如下图所示的电路我们就可以在 Mux 的 S0 端设置 Case 分析，旁路 DFT 逻辑。



4. 如何获得帮助及参考资料

可通过以下几个获得帮助的途径：

C_shell> 0in –help 可以得到启动 0in 命令的 Options 和语法格式

0in_shell> help 得到 0in shell 所支持的所有 Comand

0in_shell> command –help 得到特定 Command 所支持的 Options 和语法格式。

Mentor 提供了以下随机参考文档：

1） cdc_user.pfd

2） tutorials_cdc_verilog_user.pfd

3） tutorials_cdc_vhdl_user.pfd

可在$HOME_0IN/x.x/share/doc 下用 acroread 打开查看 CDC 用户手册。

4.5.3 CDC 报告确认

1. CDC 报告确认方法

CDC 检查运行完后，在输出文件下生成如下文件：

```
0in.log
0in_cache/
0in_cdc.db
0in_cdc.rpt
0in_cdc_ctrl.v
0in_cdc_design.rpt
0in_cdc_param.inc
0in_design.rpt
0in_detail.log
```

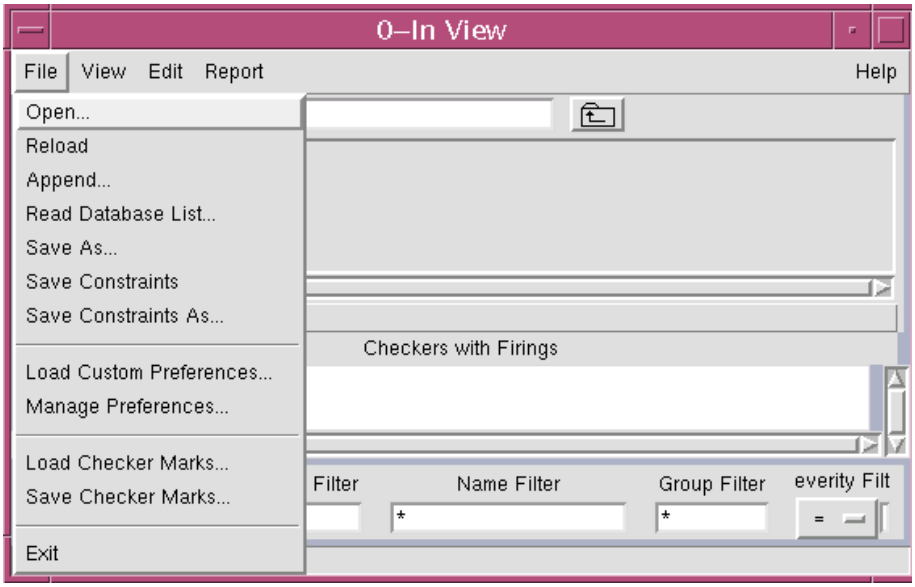
0in.log 和 0in_detail.log 中生成编译日志，可以查看 RTL 代码和约束脚本的问题。

0in_cdc_design.rpt 为工具识别的时钟域分组信息，可以查看时钟的设置是否正确。

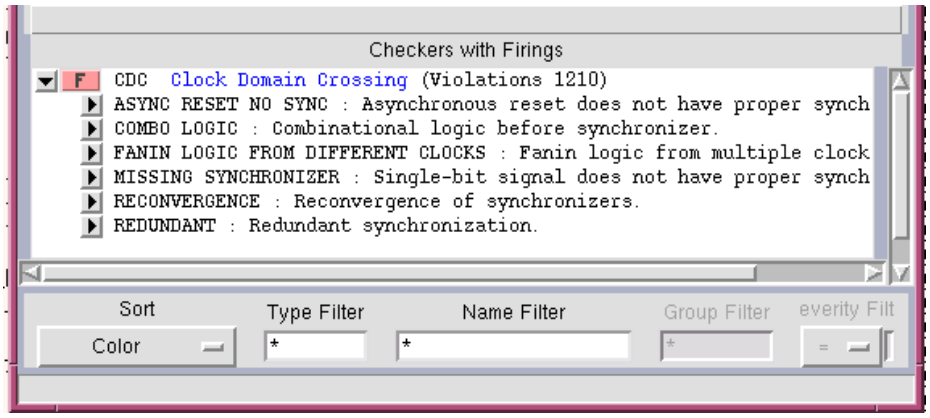
0in_cdc_ctrl.v 和 0in_cdc_param.inc 为工具检查到的同步器信息以及时钟的参数。

0in_cdc.rpt 即所有的异步路径报告。

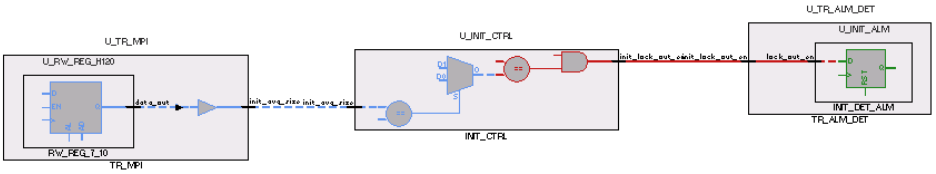
在 C shell 下输入 0in_view 可以打开 0in 的图形界面。



在图形界面下选择打开 0in_cdc.db 文件可以通过图形界面查看所有的异步路径。



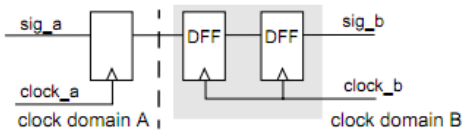
双击这些异步路径可以进一步查看这些路径的逻辑电路图。红色部分即为具体的 violation 点。



2. CDC 常见报告类型

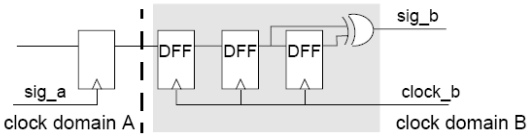
CDC Checker 所定义的同步器安全等级有 Violation、Unkown、Evaluation 三种，Violation 表示异步处理有明显的违规，需要修改设计；Unkown 表示工具无法判断，在不同条件下可能有潜在风险；Evaluation 类型基本是安全的。用户也可以通过脚本调整或自定义各种同步器的安全级别。

(1) Two DFF Synchronizer



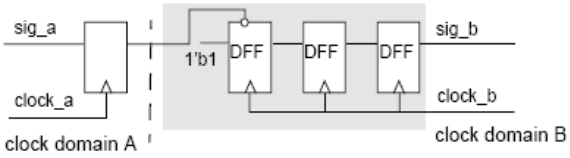
单 bit 信号同步。安全等级：Evaluated。

(2) Pulse Sync



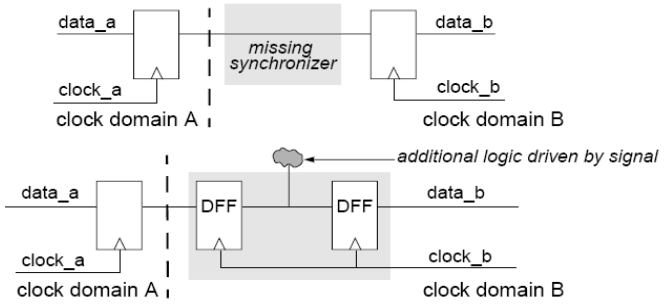
同步后采沿电路。安全等级：Evaluated。

(3) Async Reset



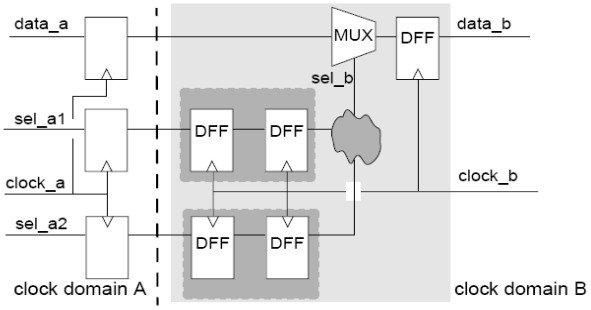
异步复位同步撤离。安全等级：Evaluated。

(4) Multiple Bits



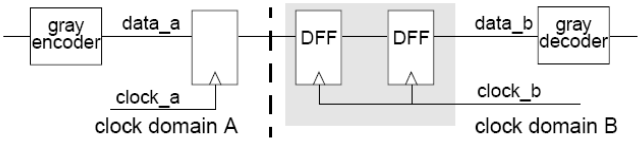
多 bit 总线信号同步。安全等级：Unkown。与总线信号的编码强相关，如果是 GRAY 码，可以认为是安全的。

(5) Multi Synchronizers at MUX select



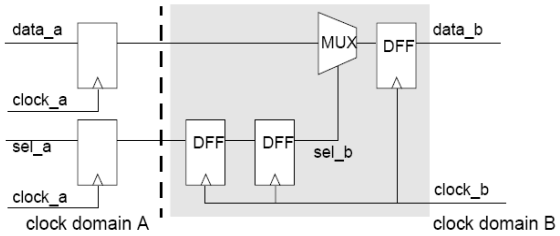
MUX 的选择端存在多个同步器。安全等级：Unkown。A 时钟域的信号同时跳变，在 B 时钟域不一定能同时采样。

(6) Two DFF Bus Sync



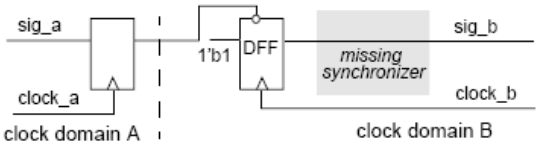
总线同步。安全等级：Unkown。如果总线之间各自是相互独立的，如多个单比特信号拼接而成的总线，则认为安全。

(7) Dmux



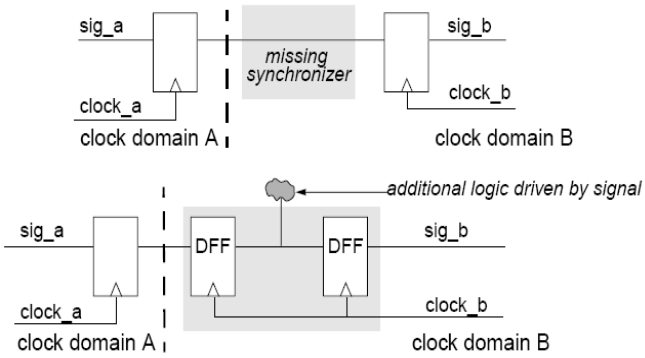
总线 DMUX 同步。安全等级：Unkown。需要保证 sel_b 信号有效时 data_a 必须是稳定的。

(8) Async_reset_no_sync



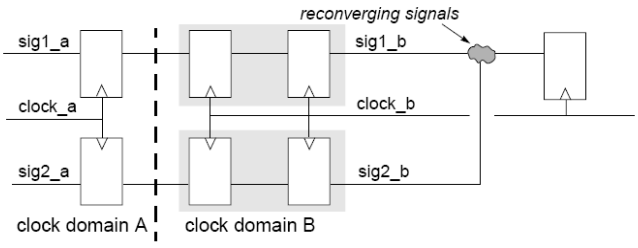
异步复位未作同步撤离。安全等级：Violation。需要修改代码。

(9) Miss Synchronizer



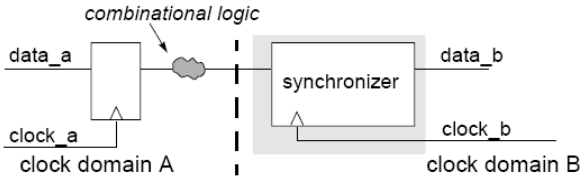
异步信号未做同步处理。安全等级：Violation。需要修改代码。

(10) Recovergence



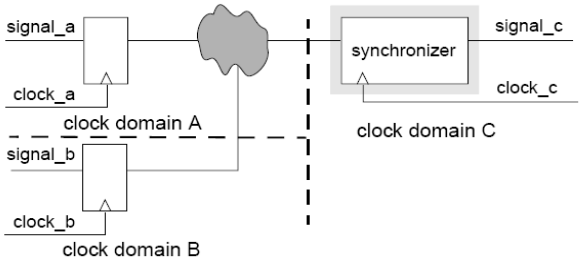
各个异步信号同步后有聚合。安全等级：Violation。聚合点的采用时间不可预期。需要修改代码。

(11) Combo_logic



组合逻辑信号同步。安全等级：Violation。组合逻辑的信号时序不可预期。需要寄存后再做同步。

(12) Fanin_different_clks



与 comb_logic 类似，只是组合逻辑的信号来自于不同的时钟域。安全等级：Violation。

4.5.4 CDC 案例分析

1. 例化厂家库单元的影响

在 CRG 设计中，常常需要手工例化厂家提供的库单元，如时钟 MUX、CG 器件，端口上的时钟经过这些器件，可能无法向下传递，需要将这些单元替换成一个功能等效的行为模型。例如 clk_a 和 clk_b 选择出 clk_c，例化一个二选一时钟 MUX，CDC 会认为 clk_c 是一个独立的时钟，与 clk_a 和 clk_b 属于不同时钟域，误报出很多异步路径。解决方案是将时钟 MUX 替换为一个行为模型。

```
`ifdef USE_EMU
    assign clk_o = (sel == 1'b0) ? clk_in0 : clk_in1;
```

还有一类模块，为了保证后端实现的规整性，会大量重复例化厂家库单元，做 CDC 检查时可能会很慢，如果确认模块是同步的，可以将其设置为 blackbox。

2. 使用 botom-up 方式

全芯片做 CDC 检查往往速度很慢，甚至不可接受。可以先按照子系统做 CDC 检查，顶层检查时，将子系统设置成 blackbox。如果子模块只有一个时钟，也可以将其设置为 blackbox，但是需要确认子模块的端口全部都是同步信号。

3. RAM 相关的设置

厂家提供的 RAM 对于 CDC 来说也是一个 blackbox，CDC 无法识别 RAM 端口的时钟域，导致 report 中有大量 RAM 相关的 violation，解决方法有两种：

1）手动设置 RAM 的时钟，并将对应的端口设置到其时钟域。

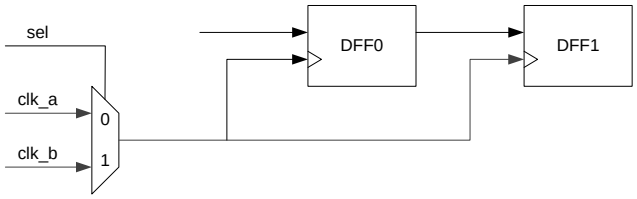
```
//0in set_black_box RAM2PRF256W162BC2B1HD
//0in set_cdc_clock CLKA -module RAM2PRF256W162BC2B1HD
//0in set_cdc_clock CLKB -module RAM2PRF256W162BC2B1HD
//0in set_cdc_port_domain QB        -clock CLKB -module RAM2PRF256W162BC2B1HD
//0in set_cdc_port_domain ADRA      -clock CLKA -module RAM2PRF256W162BC2B1HD
//0in set_cdc_port_domain DA        -clock CLKA -module RAM2PRF256W162BC2B1HD
//0in set_cdc_port_domain WEA       -clock CLKA -module RAM2PRF256W162BC2B1HD
//0in set_cdc_port_domain MEA       -clock CLKA -module RAM2PRF256W162BC2B1HD
//0in set_cdc_port_domain ADRB      -clock CLKB -module RAM2PRF256W162BC2B1HD
//0in set_cdc_port_domain MEB       -clock CLKB -module RAM2PRF256W162BC2B1HD
```

2）使用 RTL 代码写一个 RAM 仿真模型，替换厂家的 RAM 后，再做 CDC 检查。

另外，一些简单的 IP 也可以用类似的方法处理。

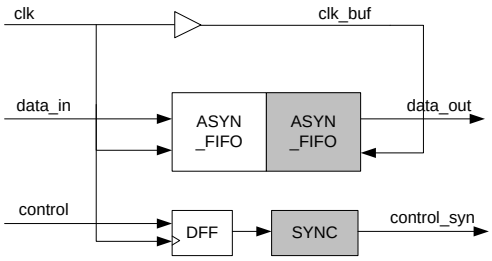
4. 时钟特殊约束

1）对于存在时钟 MUX 的设计，如下图所示，由于 clk_a 和 clk_b 的相位信息会同时传递到 DFF0 和 DFF1，CDC 会交叉检查两个时钟的信息，即 DFF0 工作在 clk_a，DFF1 工作在 clk_b 或者 DFF0 工作在 clk_b，DFF1 工作在 clk_a，而实际上不可能出现时钟域交叉的情况，两个寄存器本质上是同步的。



解决方法是将时钟 MUX 端将 sel 信号设置为常数，使用 set_constant 命令（使用 set_cdc_signal xxx –stable 不能解决此问题）。

2）为了方便时序收敛，有时候我们需要将同步时钟域的信号，在综合、STA 脚本中人为地设置成 false_path。下图为 SDXXXX 中的一个案例，由于 clk 时钟树很短，需要将 data_in 和 control 信号隔离到一个新的时钟树下，新的时钟树是 clk 经过一个 buffer 重新起源的，从功能上看，clk 和 clk_buf 是同步时钟与，但物理实现是 clk 和 clk_buf 是两个不同的时钟域，需要按异步时钟域处理。



这种情形需要单独将 clk_buf 和 clk 设置成不同的时钟分组。

set_cdc_clock clk –group CLK

set_cdc_clock U_CRG.clk_buf –group CLK_BUF

如果不分开设置，CDC 会认为 clk 和 clk_buf 同源，不作检查。

4.6 形式验证 Formal Verification

概述：重点为基本概念、实现流程、脚本编写，注重实用性和可参考性

4.6.1 形式验证基本概念

形式验证是一种与传统的居于事件驱动的动态验证手段完全不同的功能验证技术，是独立于测试向量的静态验证技术。在 ASIC 设计流程中，静态验证技术有两种：STA 和形式验证。

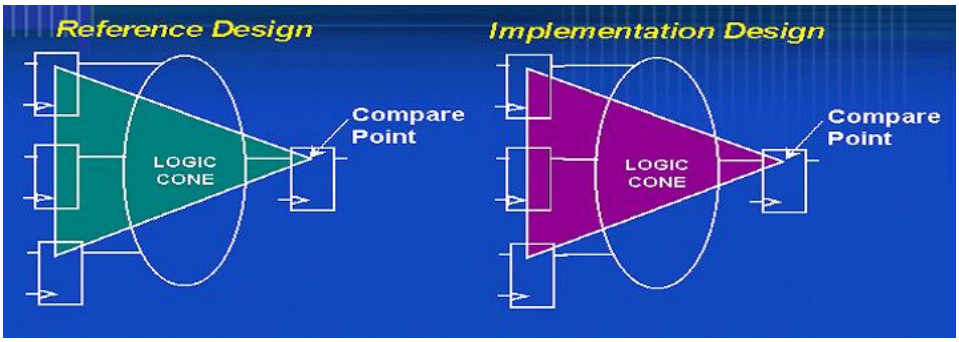
形式验证技术不需要外界提供测试向量和时钟激励，它只是把输入的 Design 的逻辑功能解剖分割成软件所定义的逻辑块，用一种特定算法下的固定运算方式，使用穷极向量对所有小逻辑块进行仿真，且在保证正确性的前提下分割任何仿真数据必须在整个 Design 的运算逻辑中的传播。

软件将忽略 Design 所要实现的逻辑功能而进行 Designs 之间的比较，因此形式验证并不能保证您所要求的功能，它必须要求有一个被认为是功能正确的供形式验证工具参考的 Golden Design。

形式验证有两个优点：1，形式验证不存在验证覆盖率的问题，即形式验证的目的是比较的两个 Design 的功能，并确认它们的功能是否 100%的相等；2，它用一种简单的运算把一个大的 Design 分割成若干个很小的逻辑块，因此可以消除 Design 中某一局部逻辑块对其某一相同输入向量的重复运算，从而大大的缩短了验证所需时间。

形式验证的工具有多种，其中应用最广泛的是 Synopsys 公司开发 Formality，它可以支持 RTL 级对 RTL 级、RTL 级对门级网表、门级网表对门级网表的等价性验证。下面介绍 Formality 中有一些概念：

- 正如前面所述，Formality 把一个大的设计分割成许多小逻辑块，定义为“Logic cone”，即“逻辑锥”。



- Reference design 和 Implementation design。Reference design 是作为向 Formality 输入的参考设计，即前面所述的 Golden design，Implementation design 是被验证的对象。
- Match。在 Reference design 和和 Implementation design 中相对应的两个逻辑锥称为“逻辑锥对”，他们被称为“Matched point”，Formality 中寻找“Matched point”的过程就是 match。
- Verify。完成 match 后，每个逻辑锥对都通过遍历向量来验证，这个过程就是 verify。
- Black Box，即“黑盒子”，他们在 Design 中存在，但没有任何逻辑的模块。
- Container，在 Formality 中，存放 Design 的叫做 container，也就是“容器”。

在 ASIC 设计流程中，每当需要验证设计的一致性时，就会用到形式验证。

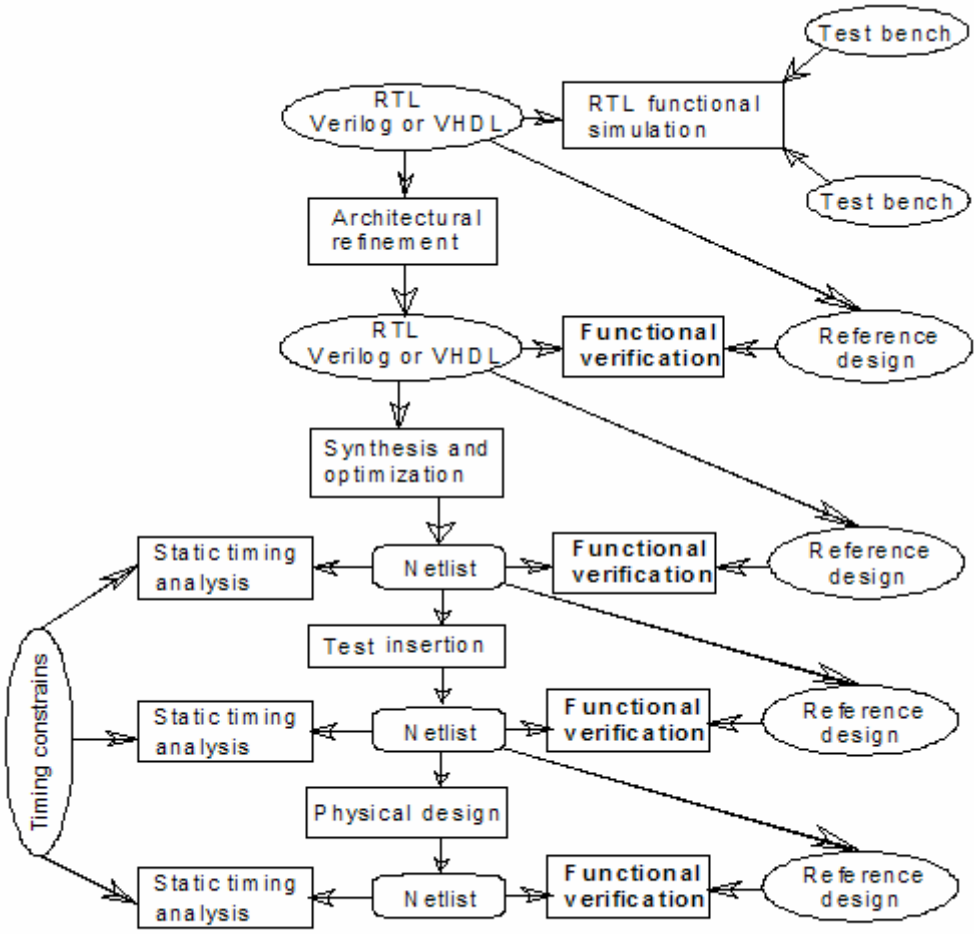


图4-120 形式验证在后端流程中的位置

图中可以看出 Formality 事实上并不保证 Design 的功能正确性。因此在使用 Formality 进行功能验证之前必须确保 RTL simulation 的覆盖率达到一定的要求(至少 90％），在确定 RTL 级功能没有任何问题的情况下将其作为下一步形式验证的参考（将其定义为 reference design ），以后每以通过验证的 Design（implementation design）作为下一次验证开始的参考

4.6.2 Formality 的文件结构以及脚本书写方法

形式验证目录结构包括：run 目录，scr 目录，log 目录以及 rpt 目录。其中 run 目录存放 Formality 的 run 文件以及 setup 文件，run 文件将调用 scr 目录下的脚本文件；scr 目录存放 Formality 脚本；log 目录存放运行的过程 log；rpt 目录存放比较结果报告。

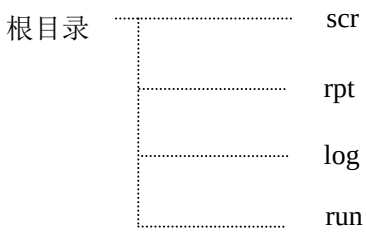


图4-121 Formality 目录结构

Formality 有两种用户操作界面：shell 和 gui（shell 用“fm_shell”启动，gui 用“formality”启动）。各自的特点是：shell 界面下运行速度快，gui 界面下可以 debug。

4.6.3 形式验证实现与脚本介绍

Formality 脚本文件。这个文件中，一般分 8 步执行。第一步是设置环境变量；第二步是创建 Reference container 并读入 golden design；第三步是创建 Implementation container 并读入待比较 design；第四步是设置常数点；第五步是设置不比较点；第六步是 match；第七步是 verify；第八步是写报告。

下面介绍如何写 Formality 脚本：

1. 设置变量和环境

- 告诉 Formality 检测到多少个 fail 点就停止（建议初始设置较小）；
  Set verification_failing_point_limit XX；
- 告诉 Formality,检测到不认识的模块就视为“黑盒”；
  Set hdlin_unresolved_modules black_box;
- 设置 DesignWare 的路径（只有在需要使用 DesignWare 库时才会设置）
  Set hdlin_dwroot “/software/synopsys/synthesis/2004.06sp2” ;
- 设置屏蔽不关心的 Warning 信息
  Set hdlin_warn_on_mismatch_message “RMR_ELAB-149”;
- 设置 Black_Box：
  一般用在”read”之前：Set hdlin_interface_only {mALU};
  一般用在”match”之前：set _black_box i:/WORK/mAIU；

2. 读入 Reference Design

（1）#Formality 开始前，先删除上一次的 container;

```
remove_container –all;
```

（2）＃设置 search_path，读入厂家库；

```
set search_path [list   $path1\
                        $path2   ]
read_db [list std_cell.db]
```

3. 读入 Implementation Design

(1) 读入参考文件

```
read_verilog –r [list  file1 \
                       file2 ]
```

(2) 设置 TOP 模块和参考设计

```
Set_top  r:/WORK/TOP_design_name
```

(3) 读入待比较文件

```
Read_verilog –i [list  file1\
                       file2 ]
```

(4) 设置 TOP 模块和参考设计

```
Set_top  i:/WORK/TOP_design_name
```

4. setup 设置

（1）设置常数，屏蔽逻辑点（通常用作屏蔽 scan 扫描链）

```
Set_constant   i:/WORK/alu/scan_en  0;
```

（2）由于 DC 优化产生大量不匹配点时，需要将 DC 产生的优化信息（.svf）读入 Formality；2004。12 后 DC 工具能自动生成 default.svf 文件；

```
set_svf  default.svf
```

5. match 设置

（1）设置单点匹配

```
Set_user_match r:/WORK/mcntrl/pc_arb_bus[0]  i:/WORK/mcntrl/pcarb0
```

（2）设置比较规则

```
Set_compare_ruler  r:/WORK/chip –from a_reg –to a;
```

（3）开始匹配；

```
Match;
```

6. verify 设置

（1）开始比较

```
Verify;
```

7. report 输出报告

（1）报告不匹配的点

```
Report_unmatched_points>./rpt/unmatched_points.rpt;
```

（2）报告失败的点

```
Report_failing_points>./rpt/failed_points.rpt
```

（3）报告异常的点（可能 Formality 到了用户定义的 failing point 的上限，或 Formality 无能为力的点）

Report_aborted_points>./rpt/aborted_points.rpt

8. Formality 实现小结

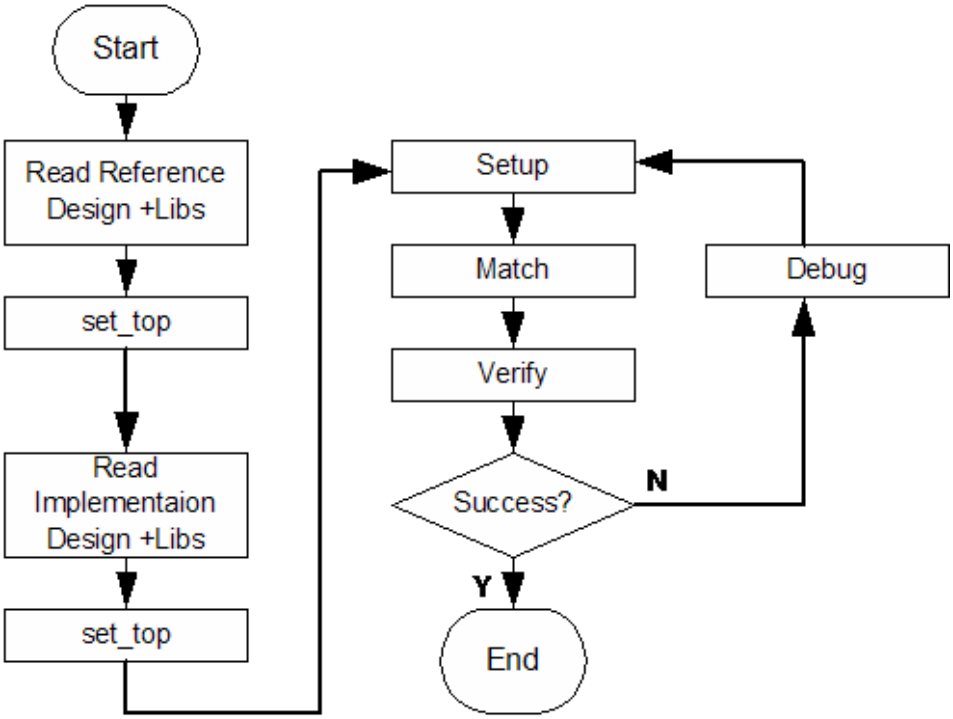


图4-122 Formality 流程

9. Formality 中的常用变量

(1) signature_analysis_matching

这是 Formality 在分析网表时为了找到对应比较点而使用的一种算法。这项功能的最大用处是在两个设计的寄存器变量或者输入输出端口的名字有所区别的时候它可以使用功能匹配的方法将其匹配起来。一般在网表的当中总是会存在有名字被改变的情况，所以在默认条件下这个功能开关是被设为 true 的。除此之外，对于端口的功能匹配 Formality 还有专门的两个变量可供选择是否要将其打开，它们是：signature_analysis_match_primary_input 和 signature_analysis_match_primary_output。一般情况下端口名字是不会改变的，所以这两个变量的默认设置是 false。

(2) verification_remove_passing_subdesigns

这是一个可以提高 Formality 使用缓冲区效率的变量，在 Hierarchy 的比较过程中 Formality 会先从最底层一个模块开始进行比较，当一个模块比较通过的的时候 Formality 会认为不再需要使用这个模块因此当您把它设为 true 的时候，Formality 会将内存中比较通过的模块删除以节省内存的使用。但是，如果你的

设计在综合的时候两个模块之间存在 retiming 的情况，那么设计很可能会由于缺少某个模块（单个比较通过之后被删除的），而导致在 Formality 第一次比较失败后，进行向上 flatten 的比较再次失败。

(3) verification_propagate_const_reg_x

这是一个对于设置了 constant 的分析有重要影响的变量，其中“x”是指 don't care。这个变量的值可设置为 false、ture，默认设置为 false。当该变量被设置为 true 的时候，所有由于设置了 constant 而被 disable 的触发器将会向下游逻辑锥的输入传递一个“x”，而且 Formality 会根据这一情况给下游逻辑锥进行一定的优化，随后分析逻辑锥的功能等价性。

(4) verification_set_undriven_signals

在设计中，如果一个信号的输入没有接驱动，那么 Formality 将依据这个变量来为其设置输入向量。这个变量可以设置为“0”、“1”、“PI”和“X”。其中设置 0 或 1 比较容易理解，设置为“PI”的含义是：告诉 Formality，这个点是 undriven 点，而且应该遍历 0 和 1。如果设置为 x，那么它就表示 don’t care。需要提一句就是：如果代码中存在 undriven 信号，综合后它会被接 0。

(5) hdlin_unresolved_modules

这个变量的作用是，如果设计中存在 Formality 所不认识的 module，工具该如何决定，可选的方案是工具停止运行或设置它为“黑盒子”。一般来说，在物理设计后，会出现许多奇怪的 module 名，比如说“PCI”等。如果他们应该被当成“黑盒子”，则工具可以正常运行，否则，就必须在网表中删除这些 module。

(6) hdlin_warn_on_mismatch_message

这个变量可以屏蔽告警，使得告警不能中断 Formality 的运行。

(7) distributed_64bit_mode

Formality 支持多 CPU 工作。当你的 design 太大时，如果只用一个 CPU 跑，耗时比较长。这时你可以将这个变量设上，然后再增加以下这个命令指定你要用到的 CPU 名列表：Add_distributed_processors “HW95*2 HW96”。这里表示跑 formality 时用到 HW95 的两个 CPU 和 HW96 上的一个 CPU。

4.6.4 形式验证案例分析

1. 综合策略对 Formality 的影响

有两种综合策略对 Formality 有直接影响。

(1) 使用 DW 库

在一些项目中，用到了乘法器或除法器，从电路实现上来说，这样的电路最好使用 DW 库，因为可以节省面积，提高速度。但是，使用 DW 库常导致 Formlity 比较失败。DW 导致 Formality 比较失败的主要原因是 DW 库中使用了某些 IP，

这些 IP 被当成了 BlackBox，很明显，组合电路中增加的 BlackBox 会影响 Formality 的比对。

解决的方法一般是是使用 SVF 文件，或者在 Formality 脚本中对每个 DW 乘法/除法器指明使用的是 DW 库的那个结构，比较麻烦。在 SD8901 中，也使用了乘法器，但综合中不使用 DW 库，所以 SD8901 Formality 中就没有出现上面问题。

(2) 优化策略

综合中，常常会使用一些优化策略，因为这样可以节省面积。在优化策略中，有一种会对 Formality 造成重要的影响：常数寄存器的优化。常数寄存器的优化常常导致 Formality 比较不通过，这个时候，必须手动指定那些寄存器是常数寄存器，实现起来比较麻烦。SD8901 使用不优化常数寄存器的策略，所以 Formality 比较非常顺利。

小结：综合时，除非必要，不要使用 DW 库，不要优化常数寄存器。

2. 参考模型中出现 Unmatch 点的原因

参考模型中出现 unmatch 点主要有两个原因，其中一个是因为综合过程中存在寄存器优化；另一个原因是 Vendor 做后端过程中存在 BIST 优化，即 bist 共享。bist 共享导致的 unmatch 点概括如下：SRAM 的 SCANIN、MI、PF 管脚和 RA 的 SIN 管脚。

参考模型的所有 unmatch 点都必须加以确认，如果是优化的寄存器，则需要与设计人员确认，如果是优化 bist，则需要与 Vendor 确认。

3. 常数寄存器导致 Failing 的问题

在 RTL 代码对完成后端的网表比较中，可能出现常数寄存器比较不通过。

常数寄存器比较是构通过，主要有一个变量决定：verification_assume_reg_init。这个变量的含义是：网表中寄存器的初始值是什么。这个变量可以设置成如下几个值：

- None：任何寄存器的初始值都一视同仁，都是遍历 0 和 1。
- Conservative：如果只有复位 pin，则为 0；只有置位 pin，则为 1；其他情况都为 X。
- Liberal：与 Conservative 基本一致，更多的功能是可以识别常数寄存器（输入为 0，且只有复位的寄存器为常 0 寄存器；输入为 1，且只有置位的寄存器为常 1 寄存器）。
- Liberal0：初始值设置为 0，但是如果这个寄存器只有置位端，会设置为 1。
- Liberal1：初始值设置为 1，但是如果这个寄存器只有复位端，会设置为 0。

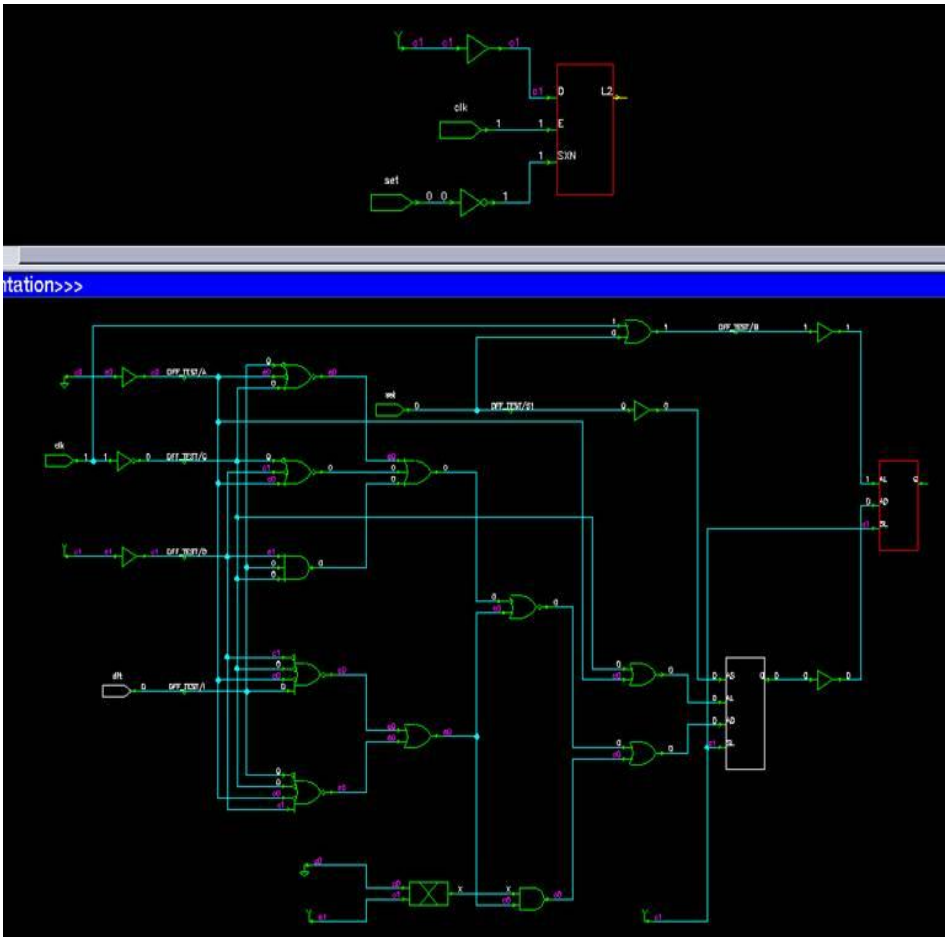
这个变量的默认值为 Liberal。如果采用默认值，那么 ref 模型中所有的常数寄存器会被 Formality 标识出来，那么这个寄存器的输出就是常数。常数寄存器对它负载的逻辑锥有影响，因为无论在何种情况下，它的输出永远保持不变。但

是，如果采用默认值，它对 impl 模型不起左右，这是因为在 IBM 器件单元中，一个寄存器由两个 Latch 和数个组合逻辑 cell 组成，这种结构导致 Formality 会认为 impl 模型中没有任何常数寄存器。

显而易见，ref 模型中有些 寄存器是常数，但是 impl 中同样的寄存器却不是，这种情况下必然后很多 failing 点。这个时候，如果把变量设置为“none”，那么无论 ref 还是 impl 中都被认为不存在常数寄存器，这样就可以比较通过。

需要说明一点，在一些旧的 Formality 版本中，这个变量缺省值就是“none”，这也是为什么使用新的 Formality 版本会出现这样的问题，但是使用旧版本则不会。

两外，如果使用默认设置，常数寄存器本身也会 failing。这里通过手工例化一个寄存器的方法，来看一下是什么原因导致比较失败。设置为 Liberal，failing 点的一个 pattern 如下图所示。



ref 寄存器被认为是常数 1，impl 则会既有初始值 0 又有初始值 1 的情况。上图中，当 impl 寄存器初始值取 0，且时钟为 0 时，就出现了比较不一致：impl 的输出为 0，但 ref 中是 1。（上图中，所有的逻辑都是包含在寄存器内）。

从这一点来说，IBM 流程下，如果要使用高版本的 Formality，就应当把变量 verification_assume_reg_init 设置为“none”。

第一个问题中说到综合策略对 Formality 的影响，这里进行更深入的讨论。假设，综合中优化了常数寄存器，就会引入这样一个问题：由于综合器认为的常数寄存器与 Formality 不一致，导致比较始终不能通过。这一点发生在 SD579 上，Formality 支持工程师也确认有这种问题。这个时候，如果把变量 verification_assume_reg_init 设置为“none”，那么肯定会比较不通过，因为 ref 模型中那些常数寄存器的输出值变成了既有 0 又有 1；如果设置为“Liberal”，也会比较失败，这是因为有的寄存器在 Formality 被认为不是常数，但在综合器中被认为是常数而优化。也就是说，如果采用优化策略，那么代码与最终的网表可能因为优化问题而无法比较通过。

小结：强烈建议，综合时采用不优化的策略。

4. 扫描链路的处理

在插入扫描链后，电路的功能发生了改变，用 set constant 的方法将扫描电路屏蔽，可进行正常功能的比较。

对于边界扫描电路（JTAG），将其 TRST 端口用 set constant 的方法设置为 0，除 Scan 链外，边界扫描功能也被屏蔽了。

在插入扫描链时，Reset 网络可能会增加一些电路，也需要用同样的方法来处理；类似于这些情况的电路均可采用同样的方法来处理。

这些操作在读入设计之后验证之前进行，示例如下。

```
set_constant postgate:/WORK/CN3D2920/TRST 0
set_constant pregate:/WORK/SDX10/TRST 0
set_constant postgate:/WORK/CN3D2920/RST 0
set_constant pregate:/WORK/SDX10/RST 0
```

5. 某项目的代码与综合后网表的比较（SDX07 项目）

- SDX07 中有软 core，软 core 也就是综合后的门级网表，在读入时，应该限制到存放代码的容器，而不应该当作技术库读入。
- SDX07 中对 RAM 用到了条件编译指令，如果定义了 IBM_RA 就用厂家返回的网表，没有就用我们自己采用 verilog 编写的行为级模型。所以在读入 RAM 相关代码时，需要加入参数-vcs “+define+IBM_RA”。
- SDX07 中的除法和求余的代码开始不能比较通过，后来修改代码，用 case 语句实现除法和求余，通过。
- 对于一些信号，虽然名字不同，但是功能应该相同，可以通过 set_user_match 使它们成为一个 matched point。
- 对代码进行综合时，如果用了 set_boundary_optimization，就会有一些寄存器被优化掉，在做形式验证时，reference 有一些 unmatched point 是正常的。

6. 某项目综合后的网表与厂家返回网表的比较（SDX07 项目）

对于 SDX07，厂家返回的网表插入了 IO BUFFER，从而比综合后的网表多了一个层次，为了 match，需要对综合后的网表包一个顶层。

查看 unmatched point 报告时，如果报告中的 unmatched point 都属于 implementation。不需要关心。但如果有属于 reference，必须找到原因。

定位 unmatch 的问题，首先要查看执行过程中生成的 formality.log。如果 log 中出现了 fail to link 就是库中没有找到相应的单元。一般都是库没有读全。

由于厂家返回的网表插入了 scan 链，为了比较正常功能就需要屏蔽该 scan 链的功能，利用命令 set_constant 来实现。

由于 IBM 的综合库中的一些 cell 为 blackbox，为了实现正常的功能，就需要读入仿真库中相应的单元来代替。

SDX07 的 IO BUFFER 都属于三态 buffer，其中一个控制端为 DI，如果 DI 为 0，所有的芯片外的输入数据都不能进入芯片内部。所以正常工作时，该 DI 信号应该为 1，它会从一个 BUFFER 的 DI－>ZDI，ZDI 又进入下一个 BUFFER 的 DI 端，就这样依次传递，由于传递链中出现了 PLL，该 PLL 又是 blackbox，由于它的存在，导致 DI 不能正确的传递到 ZDI，从而导致下面的 IO BUFFER 单元全部比较不能通过，通过 set_constant 来手工设置 PLL 的输出 ZDI 为 1，就可以保证 DI 控制信号的正确传递。

SDX07 的返回网表的末尾经常会多一些 module 语句，需要删除。如果不删除，就会报这些单元找不到。

对于 fail point，先查输入（寄存器，锁存器或 black box 的输出），再判断逻辑输出，再查看 reference 和 implementation 的 matched point 的值。

对于其中一些跟 dft 或 jtag 相关的比较点，由于其并不影响正常的功能，可以通过 set_dont_verify_point 来实现对 matched point 不进行验证。

在用 formality0406sp3 进行形式时，SDX07 中的版本寄存器有一位是常 1 值没有比较通过，但后仿真得到的结果确实是常 1，而 formality0403 也比较通过，后来借鉴李小波向 synopsys 支持工程师咨询的结果，设置 verfication_assume_reg_init 为 none 对于 formality0406sp3 也比较通过。

7. Formality 不能检查出的错误

在 SDX02 的验证中，我们发现 Formality 2002.05 不能检查出如下的一种错误：

假设在 reference_design 中我们希望的是 b 随着 a 的变化而不断变化，采用如下的 assign 语句：

```
module top (a, b);
input     a;
output    b;
assign b=a;
```

endmodule

如果在我们的 implementation_design 中，我们将其改写成

```
module top (a, b);
input      a;
output     b;
assign a=b;
endmodule
```

上述 Assign 表达的含义则是 a 随着 b 的变化而不断变化。两种表达方式在 Verilog 仿真中是不同的两种行为，但是 Formality 中却不能检查出这种错误；在 SDX02 项目的网表仿真中，通过 Formality 检查没有能够报告出这种错误，只有通过网表的门级仿真才能发现这种错误。

4.7 可测性设计 Design For Test

概述：重点为基本概念、实现流程、脚本编写，注重实用性和可参考性

4.7.1 DFT 基本概念

在介绍 DFT 之前我们先了解一下芯片在制造过程，封装过程中可能产生什么缺陷。

第一，在芯片生产过程中灰尘落入 wafer，导致芯片电路功能不正确，如图 4-123 所示。

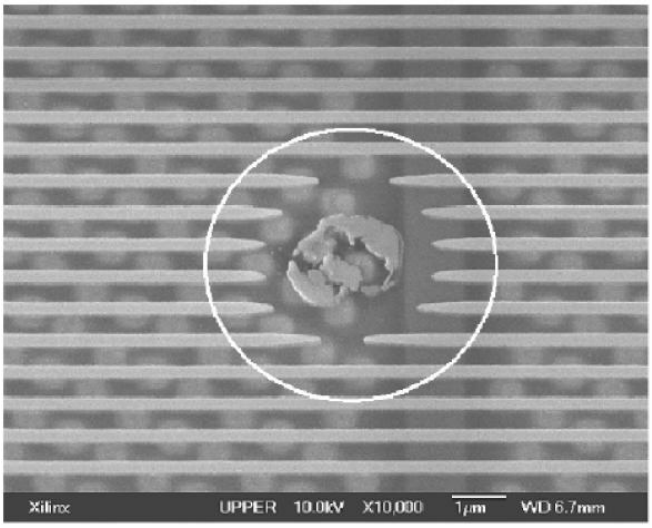


图4-123 灰尘落入 wafer 导致芯片缺陷

第二，制造过程中 Metal 层部分金属被压断，导致芯片缺陷，如所示。

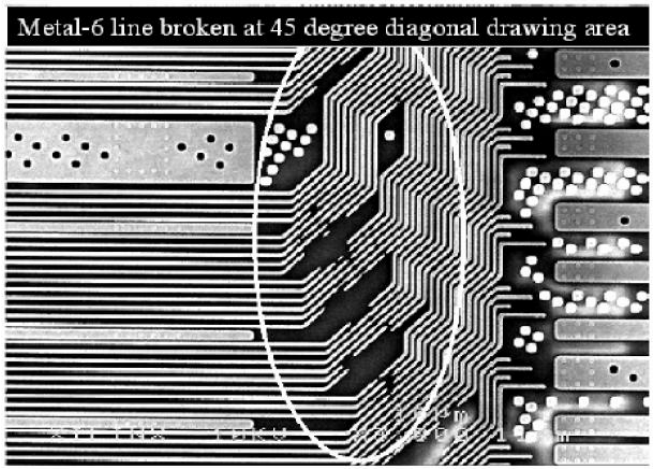


图4-124 金属被压断导致芯片缺陷

第三，制造过程中两条不相连的金属线出现了桥接，如所示。

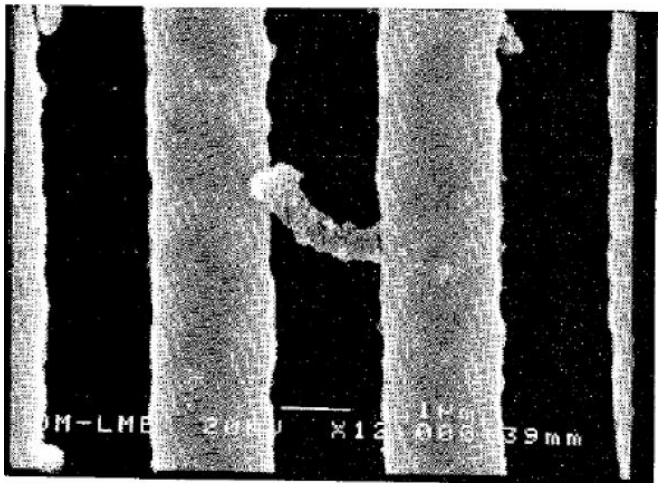


图4-125 桥接导致的芯片缺陷

此外，芯片在封装过程中也可能出现 die 和芯片管脚连线不不正常等问题。

我们再来看看芯片缺陷的代价。如图所示，在 wafer 阶段的代价为 1 个单位，但是在有设备发货后才发现芯片缺陷的代价就变成了 1000 个单位。这是不难理解的，在系统阶段，要将系统其它芯片、软件、单板出错的可能性排除，并最终定位为本芯片的问题相当困难，现加上对客户的影响，损失将会非常大！



图4-126 芯片缺陷的的代价

因此，在芯片交付之前就将有缺陷的芯片筛选出来是很有必要的，而这个工作，就是 DFT 的内容。

4.7.2 DFT 工具介绍与 DFT 实现

4.7.3 DFT 案例分析

4.8 其它

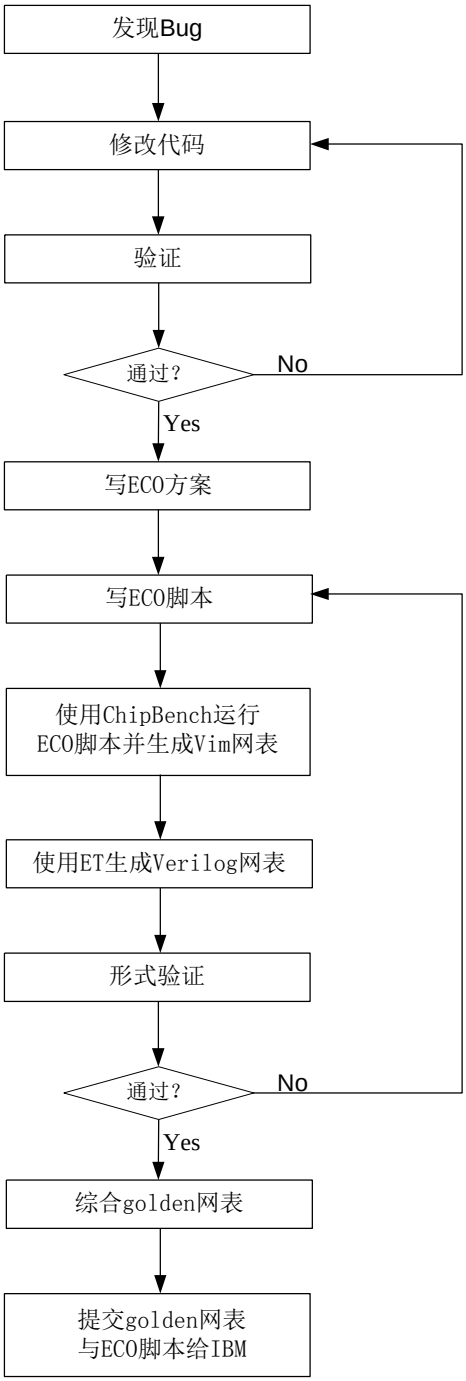
4.8.1 ECO 介绍

1. ECO 的概念

ECO（Engineering Change Order）即工程更改，对于 ASIC 来说，就是直接修改网表。接下来的章节，介绍 IBM 流程下的 ECO 流程以及实现方法。一般来说，只有项目完成了布线，即临近 RTC 流程时，如果设计中的 bug 被发现，才会启动 ECO。与其他设计方法相比，ECO 更繁琐，一个很小的修改点，也可能需要投入大量的精力。

2. ECO 流程简介

在 IBM 工艺下，ECO 分为 Metal ECO 和 Full ECO。Metal ECO 都是使用在只修改“线”的场合下，比如说 die 已经做好，这个时候只修改金属层会减少相应的成本，于是就会使用 Metal ECO。Full ECO 一般是在 routing 完成后进行，这个时候如果重新提交网表，会对进度影响很大，使用 ECO 流程可以在一个星期内完成。Metal ECO 与 Full ECO 的区别在于他们作用的范围不同：Metal ECO 只能修改连线，不能增加任何 Cell 和 Net，但 Full ECO 则可以。不论是那种 ECO 方法，其流程如下图所示。



上图是 IBM 流程下 ECO 流程的简介，需要说明的有以下几点：

- 综合得到的 golden 网表是 Vendor 用来做形式验证的，golden 网表可以是使用综合器综合生成的，也可以是手工修改 verilog 网表得到的。
- 完成 ECO 脚本后，需要使用 ChipBench 来运行，但是 ChipBench 只能生成 Vim 网表，必须使用 ET 把 ChipBench 生成的 Vim 再转化成 Verilog 网表。
- ECO 脚本的书写必须必须使用标准的 ecl 语法，第 3 章会有专门的介绍。
- 运行 ECO 脚本后，生成的 Vim 网表可以用来做 STA 分析，但是被 ECO 过的线延时反标是不准确的，因为这些线会被重新 routing。

- 在 ECO 过程中，非常有用的一个工具就是 NetWork，使用它可以直观地看到网表内的电路连接关系。
- 上图中没有画出 review 以及评审，他们在各个环节都是需要的。

4.8.2 如何写 ECO 脚本

1. ECO 的限制

ECO 过程有两个限制，一个是 ECO 方案的确定，另一个是 ECO 脚本的书写。

ECO 是直接在网表上修改的一种设计方法，所以要想确定 ECO 方案，一定要非常清楚 ECO 点的电路结构以及修改方法。另外，为了不影响 DFT 设计，ECO 方案需要尽可能不增加新的寄存器。

ECO 脚本是修改电路连接关系的直接描述，所以，写 ECO 脚本是一件很费力气的任务，而且非常容易出错导致反复修改。实际上，ECO 脚本的书写难度限制了 ECO 的规模。

2. ECL 脚本介绍

写 ECO 脚本所使用的脚本是 ecl 格式。Ecl 格式脚本只有 4 条命令：

- addNet{(TOP_name, Net_name)}
- addCell{ (TOP_name ,(Cell_type, Cell_name))}
- connectPort{ (TOP_name ,(Net_name, pin_of_Cell))}
- disconnectPort{ (TOP_name ,(Net_name, pin_of_Cell))}

其中，“TOP_name”是芯片顶层 module 名；“Cell_type”是器件类型；“Net_name”是 net 名；“Cell_name”是器件名；“pin_of_Cell”是器件的 pin。

4 条命令各举例如下：

```
addNet{(SD8901_CHIP, ECO_CH0_mod_1)}
addCell{(SD8901_CHIP,(INVERT_D ,ECO_CH0_U40))}
connectPort{(SD8901_CHIP,(ECO_CH0_u_divnPartRemn6nn1n ,ECO_CH0_U29.A ))}
disconnectPort{(SD8901_CHIP,(ECO_CH3_n1 ,ECO_CH3_u_divnu_mx_PartRem_0_0_0.SD ))}
```

3. 一种提升 ECO 脚本书写效率的方法

上文中说到，ECO 脚本的书写是 ECO 设计的一个关键点，甚至限制了 ECO 的规模。本文提出了一种提高 ECO 脚本书写效率的方法，它使用在 ECO 规模较大的场合。这种方法使用 TCL 脚本根据网表来生成 ecl 脚本，大大提升了 ECO 设计的效率，容易一次成功，而且突破了 ECO 规模的限制。由于采用了自动化，使用这种方法可以轻松地完成大规模 ECO，大大提升了 ECO 的使用空间。

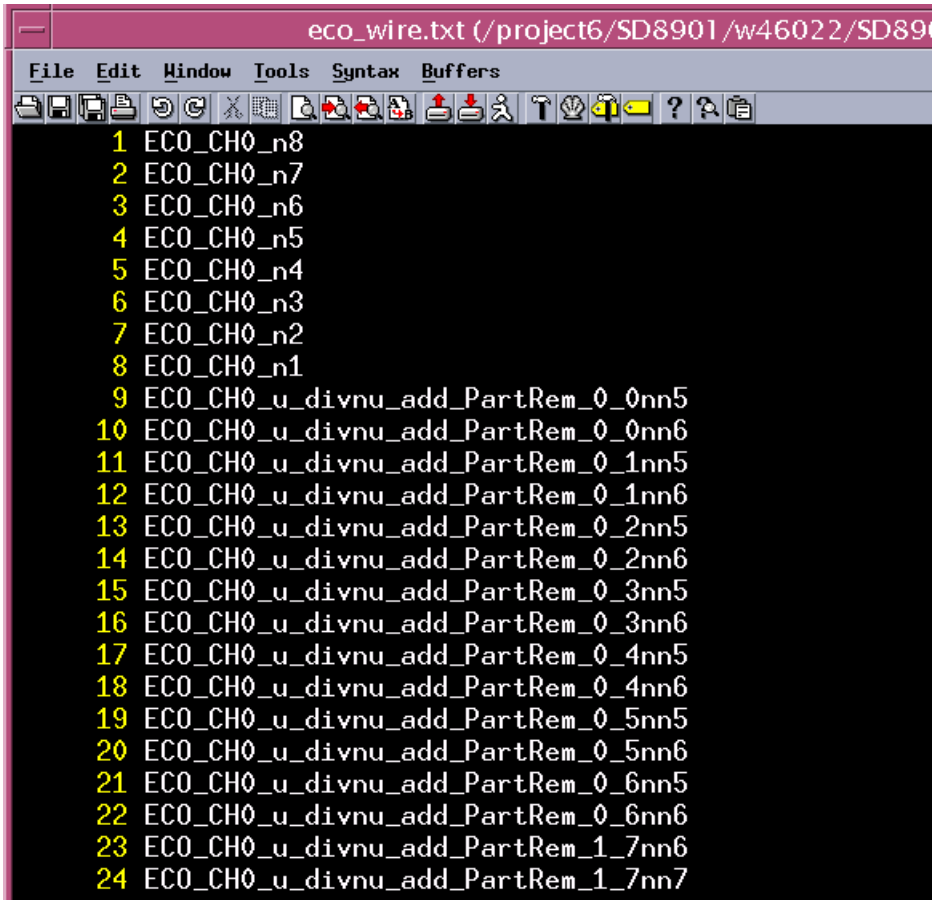
在这种方法，ECO 点被看成是一个“补丁”，我么需要先把这个补丁综合成一个网表，然后使用 TCL 脚本把这个网表转化成 ecl 脚本，最后通过手动把“补丁”与芯片网表连接在一起。

4. 自动化 TCL 脚本的编写方法

把网表转换成 ECO 脚本的基本思路是：ECO 脚本需要得到所有的 net 的名字、cell 名字、cell 类型以及 cell 的某个 pin 与 wire 的连接关系，这些要素网表中都有，所以，只要从网表中提取上诉的这些信息，就可以转换成 ECO 脚本。

使用 TCL 来编写这个脚本，可以有多种方法，这里介绍一种简单的方法，它需要先把网表进行简单排版，然后用非常简洁的 TCL 语句实现转换。这种方法可以分 3 步实现，本文会详细讲解。

第一步是从网表中把所有的 wire 和 output 信号提取出来，放在一个文件中，文件名为：“eco_wire.txt”。如下图所示。



⚠ 注意：有以下几点需要注意：

- 每条 net 都单独分成一行写，不能在一行中包括多条线。
- 不能有空行。
- net 的名字中不要出现“\”字符（与综合器的命名设置有关）
- 不能有标点符号

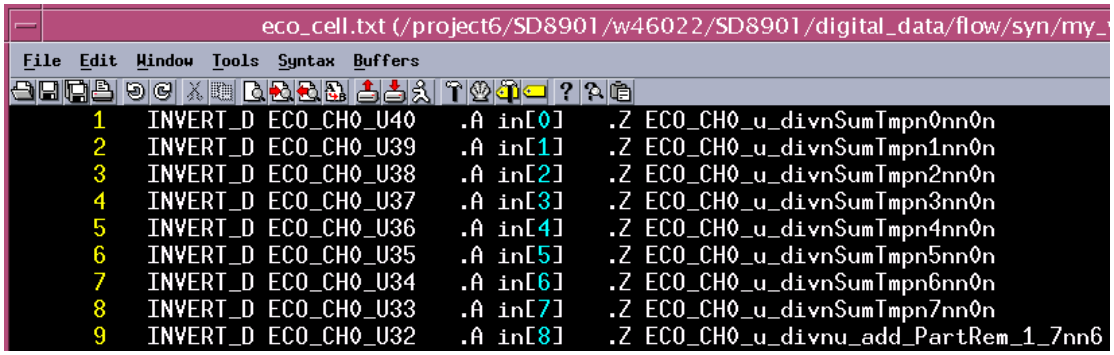
📖 说明：借助 vim 编辑器，可以容易地实现排版：按照顺序执行下列命令：

- :%s/,\ /\r/g （把“,”加空格替换成回车）
- :%s/,//g （把“,”删除）

使用“说明”中的两条命令，可以把所有的 net 单行列出来。

检查文件，如果符合“注意”中提到的 4 个条件，就完成了 eco_wire.txt 文件的提取。

第二步是把网表中的连接关系都提取出来，放到一个名为“eco_cell.txt”的文件中，其格式如下图所示。



如图所示，这个文件需要的格式是：把每个 cell 的例化放到一行，同时把所有的括号、逗号和分号替换成空格（空格的数量不会影响结果）。

⚠ 注意：Cell 的提取需要注意一下几点：

- 每个 cell 的例化只能放在一行中
- 不能有空行，且不能多个 cell 例化在同一行中
- cell 类型、cell 名字、cell 的 pin 以及与 pin 连接的 net 之间，必须以空格隔开
- 空格的数量不会影响结果，不必专门删除空格

📖 说明：Cell 连接格式可以使用如下的 vim 命令处理：

- :%s/(\n/\ /g （把“(”加回车替换成空格）
- :%s/(/\ /g （把“(”替换成空格）
- :%s/)/\ /g （把“）”替换成空格）
- :%s/,/\ /g （把“,”替换成空格）
- :%s/;/\ /g （把“;”替换成空格）

使用“说明”中的命令，可以得到符合格式要求的 cell 连接关系。

检查文件，如果符合“注意”中提到的前 3 个条件，就完成了“eco_cell.txt”文件。

第三步是 TCL 脚本的编写

- 从 eco_wire.txt 文件中读入 net 信息。

Net 信息的读入如下图所示，其含义如标注所示。

```
#!/apptools/GNU/bin/tclsh
set f1 [open eco_wire.txt r]
set f2 [open eco_cell.txt r]
set f3 [open sd8901_0114.ecl w]

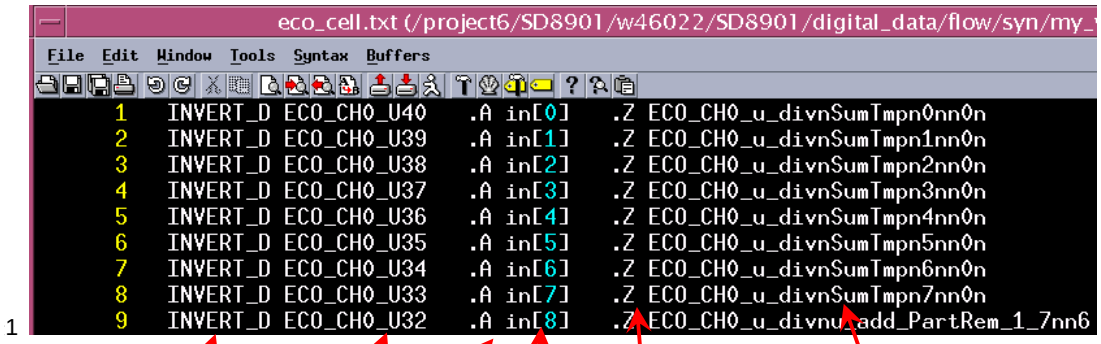
puts $f3 "#All the wires are added here"
while {[gets $f1 str_wire] != -1} {
    set wire [join $str_wire]
    #join $str_wire
    set wire_name [lindex $wire 0]
    #addNet{(SD579_TOP, SD579_CORE/U0/U_SP/U_TXPOHP/U_TXAU/U_TXAU_SLOTGEN/N_ECOPRTGEN_2_AND3)}
    puts $f3 "addNet\{(SD8901_CHIP, ${wire_name})\}"
}
```



在ECO脚本中，增加net的命令格式是：“addNet{module_name, net_name}”。

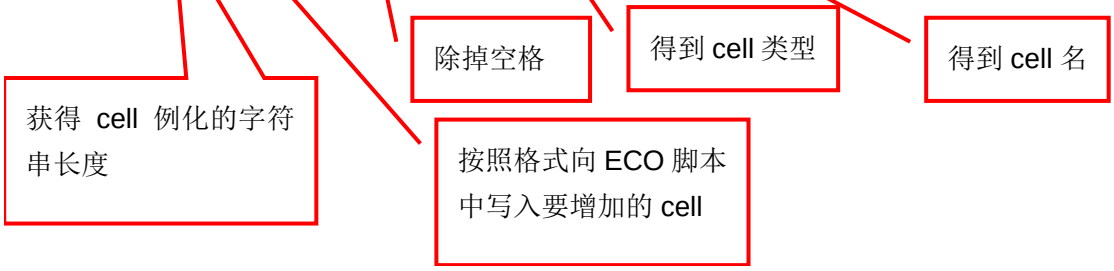
- 从 eco_cell.txt 文件中读取 cell 类型、cell 名以及连接关系

在 eco_cell.txt 文件中，已经包含了所有需要的信息，各信息的位置如下图标注所示。



连接关系信息的读入如下图所示，其含义如标注所示。

```
puts $f3 "#All the cells and all the connects are added here"
while {[gets $f2 str_cell] != -1} {
    set cell_info [join $str_cell]
    set cell_type [lindex $cell_info 0]
    set cell_name [lindex $cell_info 1]
    #addCell{(SD579_TOP,(AND3_H    ,SD579_CORE/U0/U_SP/U_TXPOHP/U_TXA
    puts $f3 "addCell\{(SD8901_CHIP,(${cell_type} ,${cell_name}))\}"
    set size [llength $cell_info]
```

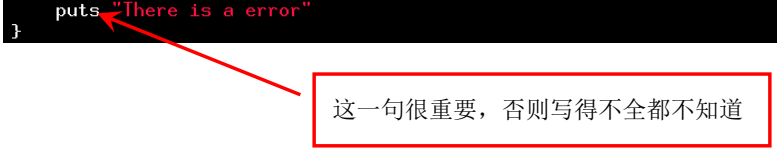


在 ECO 脚本中，增加 cell 的命令格式是：“addCell{module_name, (cell_type, cell_name)}”

上图中出现一个变量 size，它是 cell 例化的一个重要参数：一旦知道 size 的值，也就知道这个 cell 中有几个 pin，也就知道有几个连接关系。从上图中可以看出，Size 的值与 pin 的数目关系是：size_value=pin_num*2+2。

前面讲了增加 cell 的方法，下面讲如何增加 pin 与 net（也就是前文中说到的 wire）的连接关系，如下图所示。

```
#connectPort{(SD579_TOP,(SD579_CORE/U0/U_SP/U_TXPOHP/U_TXAU/U_TXAU_SLOTGEN/N_ECOh11h
if {$size == 6} {
    set con_pin1 [lindex $cell_info 2]
    set con_net1 [lindex $cell_info 3]
    set con_pin2 [lindex $cell_info 4]
    set con_net2 [lindex $cell_info 5]
    puts $f3 "connectPort\{(SD8901_CHIP,(${con_net1} ,${cell_name}${con_pin1} ))\}"
    puts $f3 "connectPort\{(SD8901_CHIP,(${con_net2} ,${cell_name}${con_pin2} ))\}"
} elseif {$size == 8} {
    set con_pin1 [lindex $cell_info 2]
    set con_net1 [lindex $cell_info 3]
    set con_pin2 [lindex $cell_info 4]
    set con_net2 [lindex $cell_info 5]
    set con_pin3 [lindex $cell_info 6]
    set con_net3 [lindex $cell_info 7]
    puts $f3 "connectPort\{(SD8901_CHIP,(${con_net1} ,${cell_name}${con_pin1} ))\}"
    puts $f3 "connectPort\{(SD8901_CHIP,(${con_net2} ,${cell_name}${con_pin2} ))\}"
    puts $f3 "connectPort\{(SD8901_CHIP,(${con_net3} ,${cell_name}${con_pin3} ))\}"
} elseif {$size == 10} {
    set con_pin1 [lindex $cell_info 2]
    set con_net1 [lindex $cell_info 3]
    set con_pin2 [lindex $cell_info 4]
    set con_net2 [lindex $cell_info 5]
    set con_pin3 [lindex $cell_info 6]
    set con_net3 [lindex $cell_info 7]
    set con_pin4 [lindex $cell_info 8]
    set con_net4 [lindex $cell_info 9]
    puts $f3 "connectPort\{(SD8901_CHIP,(${con_net1} ,${cell_name}${con_pin1} ))\}"
    puts $f3 "connectPort\{(SD8901_CHIP,(${con_net2} ,${cell_name}${con_pin2} ))\}"
    puts $f3 "connectPort\{(SD8901_CHIP,(${con_net3} ,${cell_name}${con_pin3} ))\}"
    puts $f3 "connectPort\{(SD8901_CHIP,(${con_net4} ,${cell_name}${con_pin4} ))\}"
} else {
    puts "There is a error"
}
```



在 ECO 脚本中，增加连接关系的命令格式是：

“connectPort{module_name, net_name, pin_of_cell_name)}”

这一不完成后，网表就转化成了 ECO 脚本。

- 手动修改“补丁”输入输出的连接关系

把“补丁”网表转化成 ECO 脚本后，还不能直接使用，必须再经过手工处理才行。手工处理包含两个方面：“补丁”的 input 与 output 连接。

“补丁”的 input 连接修改，是指把工程网表中对应的线找出来，然后把对应的“补丁”的 input 替换掉。

补丁”的 output 处理，是指把“补丁”网表的输出连接到工程网表的对应的 cell 的 pin 上。

以上步骤完成后，就完成了整个 ECO 脚本。

5. TCL 脚本范例

TCL 脚本、eco_net.txt、eco_cell.txt 以及最终生成的 ECO 脚本的范例，都存放在工作目录/nas/data2/share/flow/scripts/eco/下。

第5章 ASIC 文档编写

5.1 ASIC 文档体系

ASIC 文档主要包括研发文档和客户文档：

- 研发文档：指导项目组进行设计实现，重在介绍实现过程。
- 客户文档：指导下游客户进行二次开发，重在介绍如何应用。

5.1.1 研发文档体系

表5-1 研发文档体系

文档类别	文档名称	交付 TR 点	责任人
需求管理类	产品包需求	TR1	SE
	产品设计需求	TR1	SE
设计类	逻辑需求规格说明书	TR2	SE
	软件需求规格说明书	TR2	SE
	逻辑总体设计方案（即逻辑概要设计）	TR3	DEE
	软件总体设计方案（即软件概要设计）	TR3	SWE
	逻辑详细设计方案	TR4	DEE
	软件详细设计方案	TR4	SWE
验证类	验证策略	TR3	DVE
	验证报告	TR4	DVE
样片测试类	测试规格	TR2	PTE
	总体测试策略	TR3	PTE
	总体测试计划	TR3	PTE
	产品测试计划	TR4	PTE
	样片测试计划	TR4	PTE
	样片测试报告	TR5	PTE

5.1.2 客户文档体系

表5-2 客户文档体系

文档类别	文档名称	交付 TR 点	使用场景
描述类	产品简介	TR3	用作市场推广或供产品线选型。
	用户指南	TR4	指导软、硬件二次开发。

文档类别	文档名称	交付 TR 点	使用场景
参考类	应用指导	TR5	指导软、硬件二次开发。
	勘误表	TR5	指导软、硬件二次开发。

📖 说明：

TSR PDT 客户文档体系包括产品简介、用户指南、软件指南、硬件指南、应用指导、勘误表。

5.2 AISC 文档规范

5.2.1 IPR 要求

- 总体原则：不推荐“引用”（形式），推荐“参考”（思想）。
- 不能直接拷贝、参考友商资料的图标、图形和表格，严禁采用截屏方式。
- 不能照抄友商资料的自创性、典型词句。
- 寄存器、管脚名称应符合统一的命名规则，不允许照抄友商的寄存器、管脚名称。
- 如果直接引用标准、协议中的内容，必须在引用处进行版权声明。

5.2.2 中文写作规范

海思中文写作规范主要从华为写作规范出发，结合海思产品领域的特点编写的。目的在于提高研发人员的初稿写作质量，增强研发文档和客户文档的规范性、可读性。

1. 段落

- 一个段落一个主题，尽量不超过 7 行。
- 连续段落不超过 1 页，可以增加图、表进行分割。
- 句子简练，避免使用长句，尽量不超过 3 行。
- 使用陈述和肯定语气，避免使用感叹语气。

2. 词语

- 避免不规范用语
  消除土语、口语，如打一拍”、“挂起”等。
  统一前后用词，包括大小写，如不要同时出现 MSRDI”、“MS-RDI”和“MS_RDI”。
- 避免使用表示程度、强调语气的词
  表示程度的词：较多、较好、一般、完全地、基本地。
  表示转折的词：当然、然而、而、同时。
  表示量的词：有些、非常、大量、一些、少许、部分。

3. 表格

- 同类表格的设计风格相同，如管脚、寄存器描述的表头设计。
- 同类表说明的风格全文保持一致。
- 表说明简单扼要，不超过表格宽度。
- 表格内不能有空单元格，无内容的单元格填“-”。

4. 交叉引用

- 正文中引用表格、图形或本文档的其它章节时，需使用“交叉引用”，引用方法如表 5-3 所示。
- 不允许手动输入表格、图形或段落编号。

表5-3 图表编号及标题交叉引用方法

场景	方法
图表与引用处在一个页面	如（图表编号）所示。只引用编号。 如：如表 5-3 所示。
图表与引用处不在一个页面	请参见“图表编号＋图表描述”。 如：请参见“**错误！未找到引用源。错误！未找到源。**”。
章节的引用	请参见“标题号＋标题文字”。 如：对表格的要求请参见“3. 表格”。

5. 单位

- 单位要完整、正确，词头不能孤立使用。
  例如：速率的单位为 bit/s，不能写成 bps 或 bits/s。速率为 2Mbit/s，不能写成 2M。
- 正确区分词头、单位的大小写。
  例如：小写 k（表示 10^3）和大写 K（表示 2^{10}）；是 dB 而不是 DB 或 db。
- 表示数据范围时，前后都要有单位。例如：–3℃～+40℃
- 表示 bit/byte 范围时，用“bit/byte[数字:数字]”，其中的冒号“:”是英文输入法下。一般地，高位在前，低位在后。

常见错误单位分类：

- 完整性
  错误的：2M 接口；正确的：2MHz 时钟，2Mbit/s 接口。
- 大小写
  错误的：8kB；正确的：8KB
  错误的：8MHZ；正确的：8MHz
  错误的：Db；正确的：dB
  错误的：0X65；正确的：0x65

- 单复数
  错误的：10bits；正确的：10bit
- 范围
  错误的： –3℃到 40℃、 –3℃~+40℃、 –3℃－ +40℃；正确的：–3℃～+40℃
- 其它
  错误的：B/s，字节/秒；正确的：byte/s
  错误的：75 欧，75 欧姆；正确的：75Ω
  错误的：220 VAC；正确的：220V AC

6. 符号

- 中文手册的标点符号字体要求为宋体、全角。
- 一些外来的词汇中常用短划线。插入方法：英文输入法下的减号键。
  例如：E-Mail、CD-ROM、ITU-T、RJ-45 接口、STM-16。

7. 数字

除专有名词外（如三三制），产品文档应使用阿拉伯数字。如“第 1 通道”、“第 3 位”。

5.2.3 绘图规范

海思绘图规范是在华为绘图规范的基础上，根据海思产品的特点而制定的，目的在于提高图形的规范性和文档的可读性。

1. 图形分布要求

一幅完整的插图包括：图形、图说明（Figure description）、标注（可选）、图注（Figure text，可选），如图 5-1 所示。

图形分布要求：同一幅插图的所有内容位于同一页内，不能分开。

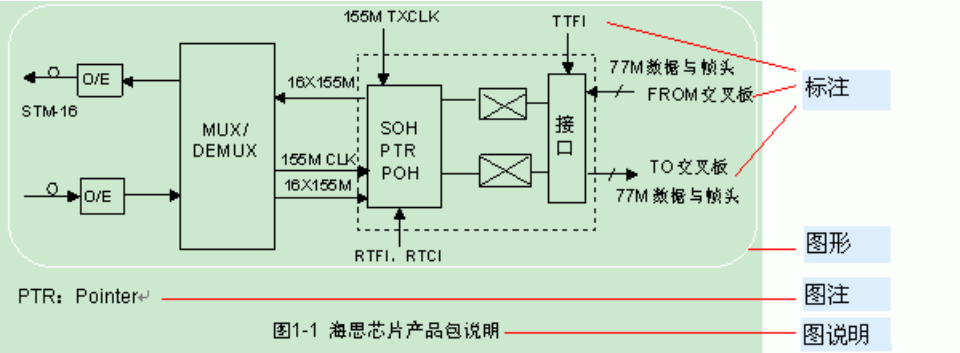


图5-1 图形组成说明

2. 常用工具

在海思领域，常用的绘图工具有：

- Visio（用于处理矢量图）

- TimingDesigner（用于绘制时序图）

3. 图形总体要求

不管使用哪种绘图工具，均需满足以下要求：

- 保留源文件（Visio 的.vsd，Timingdesigner 的.td），用于备份并方便再次编辑。
- 图形大小设计，引入文档时缩放比例为 100%为宜。
- 图形中的模块名、信号名等与正文保持一致。
- 标注线横平竖直，不能交叉。

4. Visio 绘图要求

(1) 线条

- 按住 shift，可使线条横平竖直。
- 全文线型的用法统一。不同线宽的线条表达不同的含义，特殊线型含义应在图形中说明。
- 一般地，线条：0.5pt；箭头：13 号，small。

(2) 文本

- 一般字体大小：9pt；突出内容字体：12pt。
- 框内文字不能溢出边框。
- 文本换行时，各行文字长度尽量接近。阿拉伯数字和英文单词不能跨行。
- 文本方向一般从左到右，从上到下显示。如果文字需要倒置，应逆时钟旋转 90 度，阅读方向为从下到上。如图 5-2 所示。

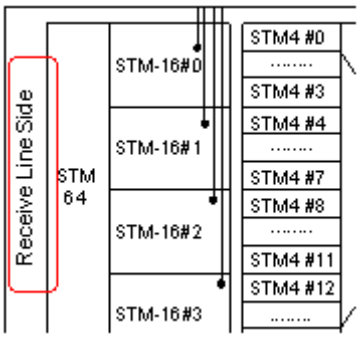


图5-2 文字倒置示意图

(3) 颜色

- 图形颜色尽量不超过 3 种。
- 不允许采用渐变色（转 PDF 时可能变乱）。

(4) 其他

- 图元之间相互连接好（连接线端点必须连到图元上）。
- 图形中相关元素应对齐、均匀分布。
- 同一幅图中相同含义的图元大小要一致。

5. TimingDesigner 绘图要求

- 时序图中的字体统一设置为 9pt，Arial，特殊情况可以缩小一些，最小字体为 7pt。
- 时序图中的线条（除了时钟信号参考线）统一设置为 0.5pt，黑色。
- 时序图中的时钟信号参考线统一设置为 Dot，Pale Gray 色。
- 时序图中信号的命名与文档中的信号名保持一致。
- 时序图统一导出为.emf 格式。

5.3 文档编写工具

5.3.1 Visio

1. 模板和图元

Visio 模板中定义了字体、线型、箭头等，可以提高绘图效率。资料部设计了一套 Visio 模板，满足各种图形的绘制需求，可以找资料人员获取。

一般地，模板、图元文件放在 Visio 的 Solutions 目录下。Visio 的欢迎界面如图 5-3 所示，可以选择选择模板或其他已有 Visio 图形。



图5-3 Visio 欢迎界面

单击  ，可以打开各种图元文件。

2. 常用工具介绍

：格式刷，使用方法与 Word 相同，请参考**错误！未找到引用源。错误！到引用源。**。

：文本 / 文本块工具，用于插入文本框或在图元上增加文字说明。文本块工具用于调整图元上的文字。

：画线工具，打开下拉列表，可选择直线、弧线、曲线和铅笔工具。绘制直线时，按住 shift，可保证直线横平竖直。

：矩形、椭圆工具，用于绘制矩形和椭圆形，按住 shift 时，可分别绘制正方形和圆形。

：颜色设置，分别设置文字、线条和色块的颜色。

：线条设置，分别设置线宽、线型和端点。线条两个端点可分别设置，如图 5-4 所示。

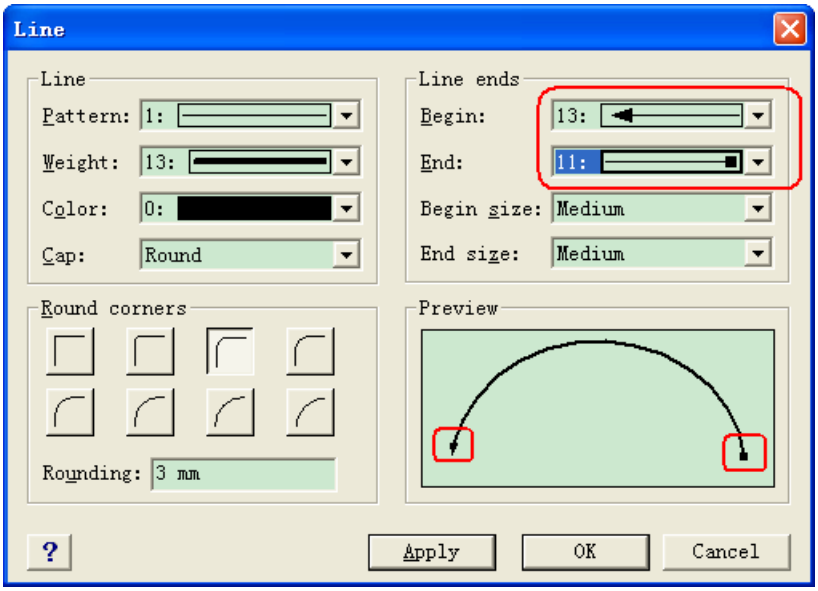


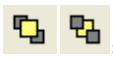
图5-4 线条端点设置

：连接符，用于手动绘制连接线或修改连接线的形状。

：组合、分解工具，非常常用，快捷键分别为 Ctrl+G 和 Ctrl+U。

：对齐工具，可分别在水平/垂直方向设置左对齐、居中对齐、右对齐。选择两个以上图元时有效。

：间距调整，可分别调整水平/垂直方向的图元间距。选择三个以上图元时有效。

 ：图层位置调整，可将选中的图元置于顶层或底层。

 ：连接图元工具，依次选择要连接的图元，再单击该按钮，将使用单箭头连接线连接各图元。

 ：旋转工具，可旋转任意角度。

 ：垂直旋转工具，分别顺时针、逆时针旋转 90°。

3. Snap 和 Glue 设置

Snap 和 Glue 是 Visio 的两个智能功能。当选择 Snap 和 Glue 时，可自动捕捉连接点。从菜单中选择“Tool>snap&glue”可进行设置。

4. 运算工具

从菜单中选择“shape>operations”，可对图形进行合并、拆分等操作，如图 5-5 所示。当选择两个以上图元时有效。

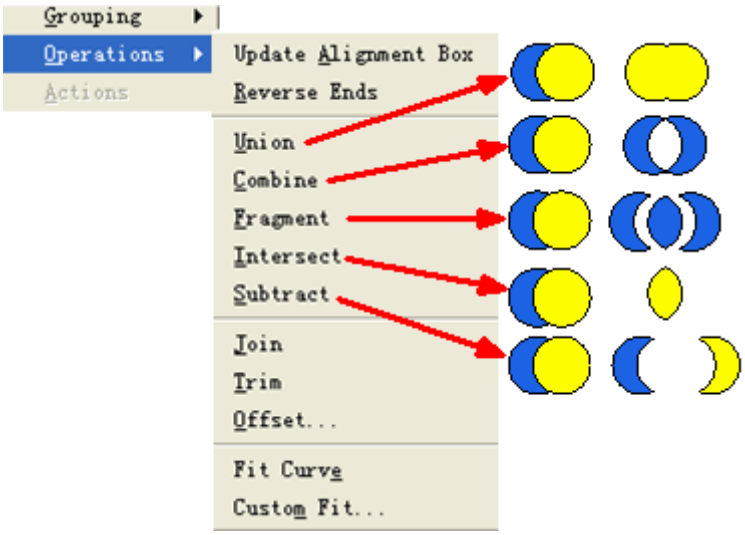


图5-5 运算工具说明

5. 流程图绘制

绘制流程图是 Visio 的特长，也是最常用的图形类型。关键步骤如下：

- 从菜单中选择“File>New>Choose Drawing Type”，打开选择绘图类型窗口选定 Flowchart 方案，如图 5-6 所示。

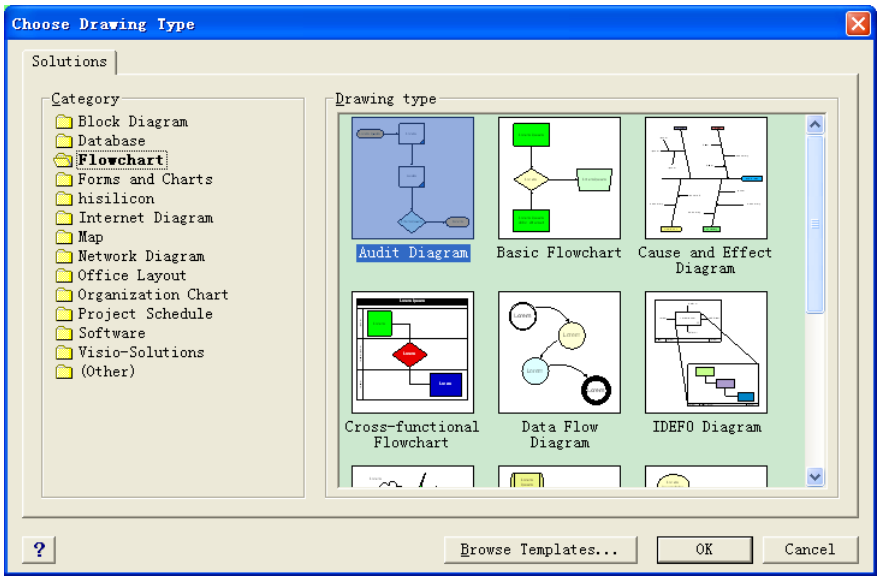


图5-6 流程图方案选择

- 选中连接工具，依次将图元拖至绘图区，完成后取消。
- 调整图元位置，输入文字即可。

5.3.2 TimingDesigner

1. 设置线条和字体属性

在绘制时序图时，先对线条、字体等进行设置。

选择菜单“View>Modify Diagram Style…”，弹出“Diagram Style”对话框，相应的设置线条和字体的样式，如图 5-7 中蓝色线框部分和红色线框部分所示。

- 对于时序图中的线条，统一设置为 0.5pt。
- 对于时序图中的字体，保持默认设置即可。

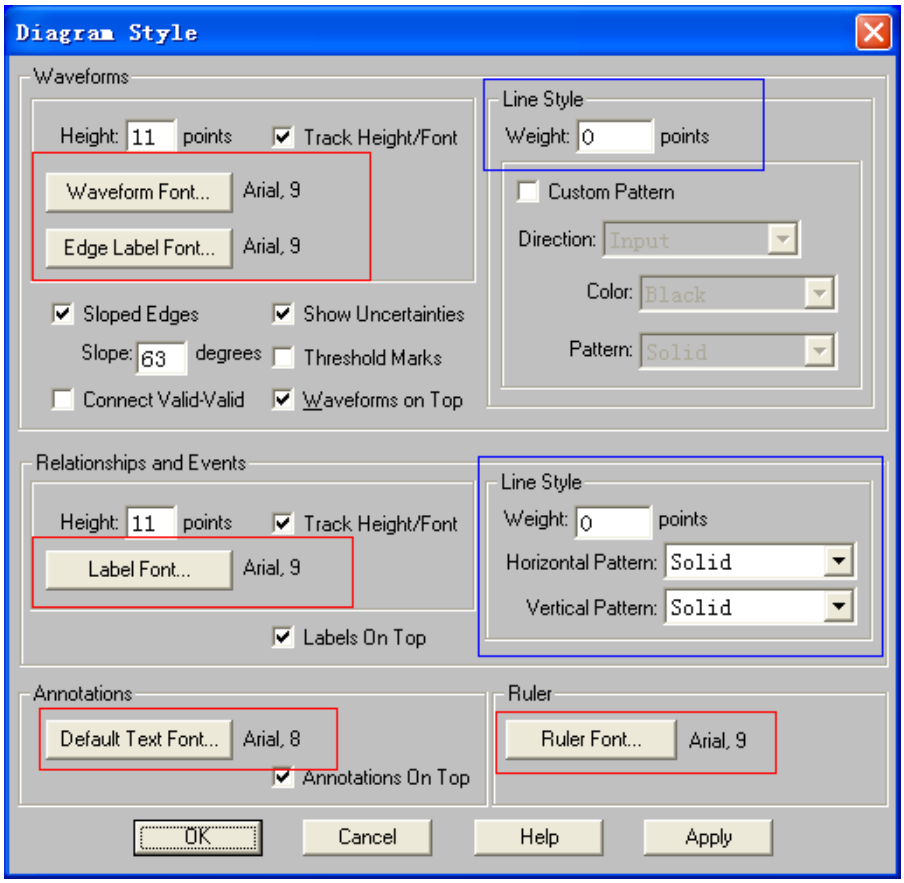


图5-7 TimingDesigner 设置线条和字体示意图

2. 绘制信号

(1) 绘制时钟信号

选择菜单“Tools>Add Clock…”或者单击 

，出现“Clock Attributes”对话框，在对话框中填写相应的参数即可。

(2) 绘制数据信号

选择菜单“Tools>Add Signal…”或者单击 

，“Signal Attributes”对话框，在对话框中填写相应的参数即可。

3. 设置时钟参考线

对于时序图中时钟信号的参考线，一律使用“Dot&Pale Gray”，如图 5-8 所示。时钟信号参考线样例如图 5-9 所示。

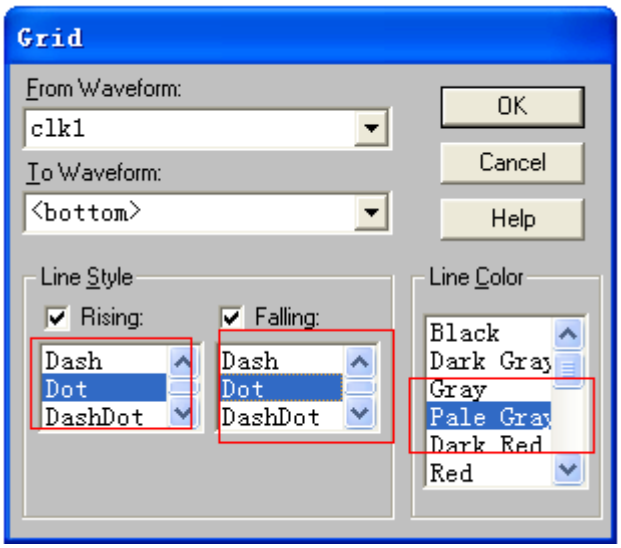


图5-8 时钟信号的参考线设置示意图

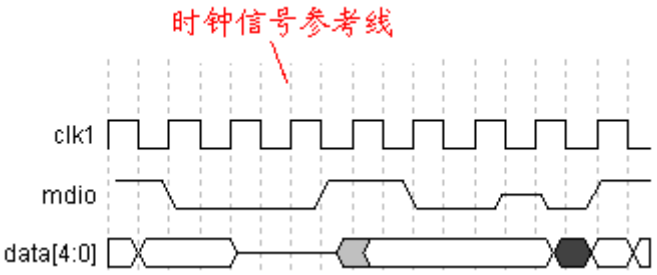


图5-9 时钟信号参考线样例图

4. 标注信号

选择菜单“Tools>Constraint”或单击 

，然后单击欲标注的信号的两个跳变沿，出现如所示的“Constraint Attributes”对话框，在对话框中填写相应参数即可。

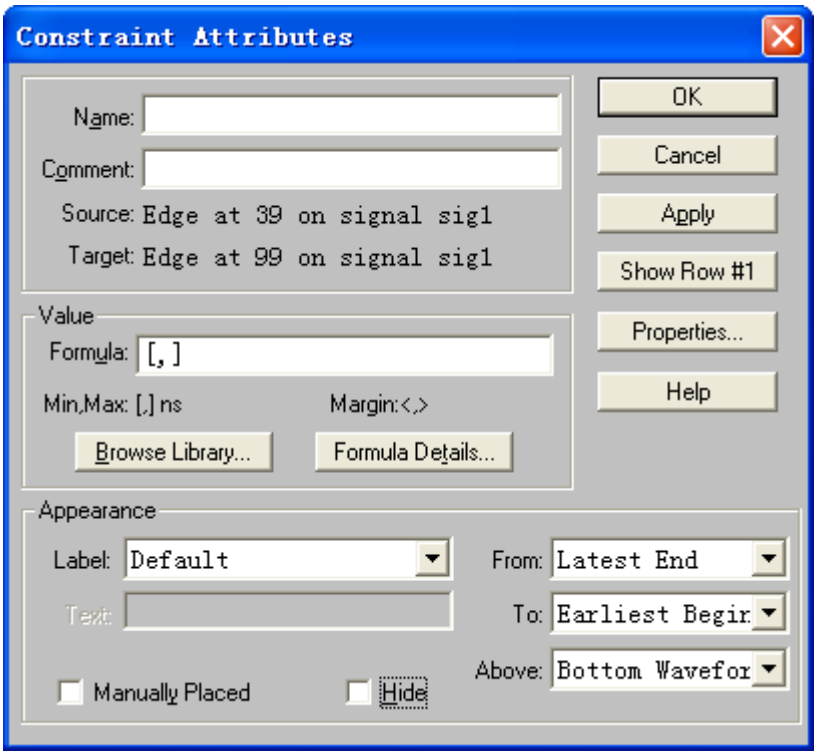


图5-10 约束属性对话框

5. 添加文字

双击欲添加文字的特定信号段，跳出如图 5-11 所示对话框，在“Label”中的下拉菜单选择“text”，即可在“text”中输入您所需要的文字。

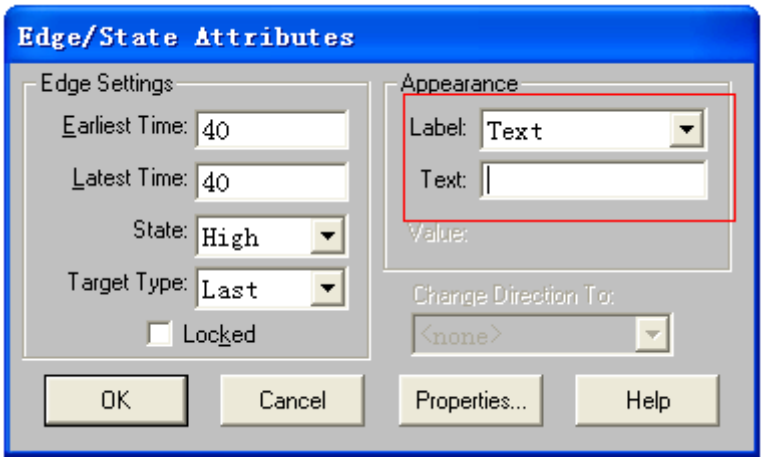


图5-11 输入文本示意图

6. 导出波形

用 Timedesigner 绘制完时序图之后，通过“File>Export..”将时序图以.emf 格式导出，一般以 135mm 和 165mm 为合适长度。导出时必须选中 Monochrone 项（确保字体和线条为黑色），并且不导出标尺，如图 5-12 和图 5-13 所示。

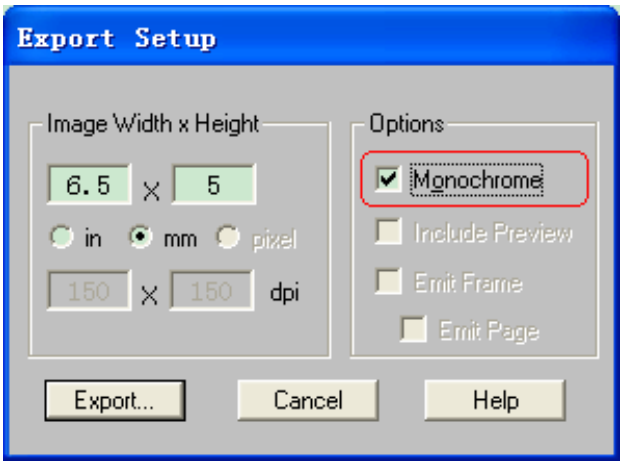


图5-12 导出选项示意图 1

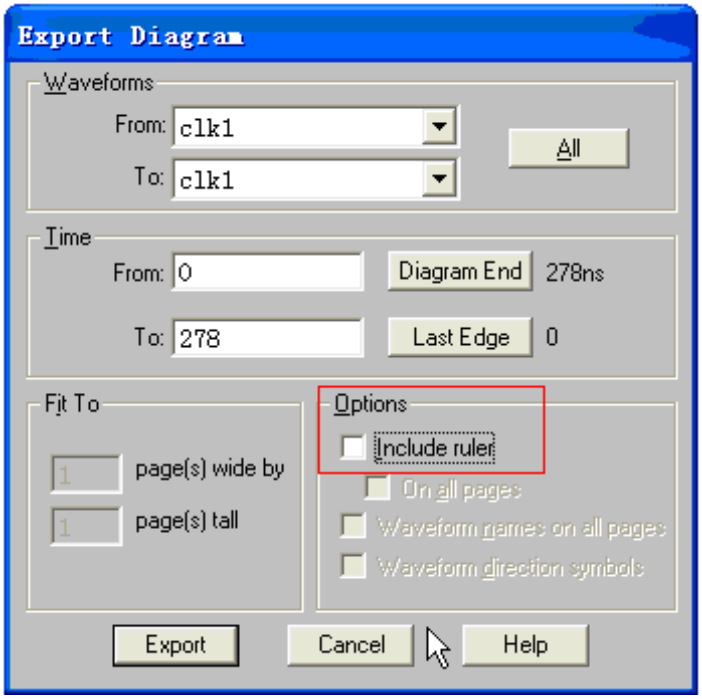


图5-13 导出选项示意图 2

5.4 文档编写技巧

要输出高质量的文档，除了上述的文档规范和工具技巧外，还要掌握一些写作技巧，这些属于“软技巧”，需要大家在实践中体会和总结。以下供有兴趣者参考。

5.4.1 构建文档框架

1. 明确读者对象

- 分析文档读者的对象：下游客户、开发代表、QA……。
- 读者需要哪些信息：描述信息、指导类信息、参考类信息。
- 读者的知识背景如何：专业技术人员、初学者。

- 读者的期望：获得指导操作、学习基础知识。

2. 搭建符合金字塔结构的文档“骨架”

定义文档“写什么”。金字塔式的信息组织方法：自上而下、自下而上。一般地，先用自上而下方法搭建，再用自下而上方法补充完整整个文档“骨架”。

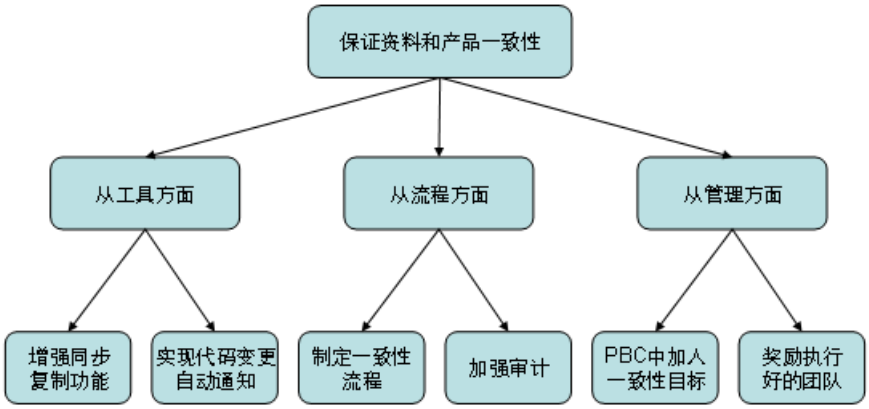


图5-14 自上而下的信息组织方法

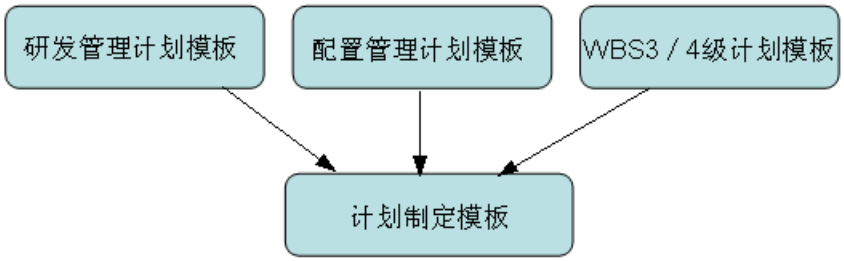


图5-15 自下而上的信息组织方法

信息组织的主要原则：

- 上一层次是下一层次的概括。
- 每层在逻辑上属于同一范畴。
- 层内按照逻辑关系顺序组织。

逻辑关系：

- LATCH 信息组织模式：位置（Location）、字母（Alphabet）、时间（Time）、类别（Category）、层次（Hierarchy）。
- 其他可选逻辑关系：重要性、频率、认知。

3. 定义文档的基本信息模块，疏理文档“脉络”

定义文档“怎么写”。提供写作指导，使团队成员协同合作。

模块是文档的基本组成部分。模块的划分基于信息类型。常见信息类型：

- Concept，提供读者需要的背景知识。
- Task，提供一步步指引的步骤信息。
- Reference，提供快速参考信息。

模块写作要点：一个内容模块只能包含一种信息类型，不能将多种信息类型混合在一个内容模块中。内容模块应相对独立，并涵盖一个具有明确目的的主题。

5.4.2 组织信息结构

1. 确定标题——模块的 label

标题设计要点：

- 同一级标题建议用一种结构表达。
- 下级标题不重复上一级标题的内容。
- 避免孤立的章节和标题存在。

2. 段落写作

段落写作要点：

- 一个段落只能有一个主题，或一个中心句子。
- 段落的中心句子放在段首，对全段内容进行概述。
- 最佳段落长度小于等于 4 行。
- 连续的段落不能超过 1 页，建议不要超过 3 个段落。
- 段落的句子语气要使用陈述和肯定语气，避免使用感叹语气。
- 用项目列表（Item list）、（Item step）呈现并列或序列的信息。

3. 句子写作

句子写作要点：

- 要避免使用长句。一个句子最长不能超过 100 字（正文 3 行）。
- 句子要使用简单句和并列句，避免使用复合句。
- 明确主、谓、宾语，正确地断句。特别是说明操作任务时，要确认动作的发起者。
- 使用陈述和肯定语气，避免使用倒装句和感叹语。
- 谨慎使用表达假设、转折等复杂关系的主从类复合句。

4. 词语写作

- 避免不规范用语
  消除土语、晦涩用语、行话、俚语、错别字。
  统一前后用词。
- 避免使用表示程度、强调语气的词
  - 表示程度的词：较多、较好、一般、完全地、基本地。
  - 表示转折的词：当然、然而、而、同时。
  - 表示量的词：有些、非常、大量、一些、少许、部分。

5.4.3 设计图、表

1. 图形

图形特点：

- 字不如表，表不如图。
- 把复杂逻辑关系的信息单元用图元和线条简单地表达出来。
- 减少文字块给读者带来的歧义和误导。
- 具有国际通用性。

图形设计原则：遵循统一的规范，提高规范性、一致性和标准化。

2. 表格

表格特点：

- 表达并简化了文本信息（块）中的复杂逻辑关系。
- 可以减少读者对文本信息内在逻辑关系的推理。
- 使信息查阅方便、快捷。
- 使写作变得简单化（填充式的写作）。
- 减少了文档的字数。

表格的内容设计要求：

- 设计表头内容。
- 设计首列内容，尽量做到全面。
- 表格语言风格要定义，要简练。

5.4.4 其他需要关注的元素

1. 单位、符号、数字

相关要求请参考“5.2.2 ”的“5. 单位、6. 符号、7. 数字”。

2. 符合读者认知习惯

(1) 符合最简认知的需求

- 组织形式简单。
  如：限制段落、句子的长度，对信息分类。任务介绍中不穿插描述、参考类信息。
- 内容安排顺序：“简单→复杂，已知→未知”。
- 推荐高效的入口，如索引。
- 控制文档层次，纵深结构→扁平结构。
- 突出使用频率高的项目。
  如：任务设计中对频率的考虑、重要场景/任务提升到 1、2 级标题。

(2) 符合可学习、可记忆的需求

- 统一信息组织模式。
  如：采用对等的标题/主题的排列、LATCH 信息组织模式。
- 加入记忆加固元素。
  如：增加章节引子、每章小结、自测试题、案例。
- 大量引入图形，强化记忆。

(3) 符合交互的需求

- 使用用户熟悉的语言，锁定目标客户。
  如：搜索用的关键词是 R&D 还是 RnD？
- 传递尊重、友善的信息。
- 鼓励并提供有针对性的提醒。
- 模拟交互情景。
  如采用第二人称、FAQ。

3. 避免语言歧义

- 文档应该对重复性的关键句式进行定义。
  如：对于建议、注意等措辞和句式的统一。
- 不要使用口语化的写作风格。
- 多使用简单句，以及结构简单的并列句。

4. 常见的容易产生歧义的句式

(1) 动词＋名词＋介词＋名词

- （动词＋名词）＋介词＋名词
- 动词＋（名词＋介词＋名词）

例：看见一个人在窗前

- 看见（一个人在窗前）——被看到的人在窗前
- 看见一个人（在窗前）——看的人在窗前

(2) 动词＋的＋是＋名词

- 主动的语气
- 被动的语气

例：控制的是主控单元

- 主控单元控制着（其他单元）
- （被控制的）是主控单元

(3) 形容词＋名词＋名词

- （形容词＋名词）＋名词
- 形容词＋（名词＋名词）

例：新主控系统数据

- （新主控系统）数据－－主控系统是新的，升级的
- （新）主控系统数据－－数据是新的

(4) 动词＋形容词＋的＋名词

- （动词＋形容词＋的）＋名词
- 动词＋（形容词＋的＋名词）

例：检测故障的电路

- （检测故障的）电路
- 检测（故障的电路）

(5) 名词＋的＋名词＋和＋名词

- 名词＋的＋（名词＋和＋名词）
- （名词＋的＋名词）＋和＋名词

例：中继系统的时钟和主控模块

- （中继系统的时钟）和主控模块
- （中继系统的）时钟和主控模块

(6) 量词＋名词＋的＋名词

- 量词＋（名词＋的＋名词）
- （量词＋名词）＋的＋名词

例：三个模块的数据

- 三个（模块的数据）
- （三个模块）的数据

5.4.5 了解高质量文档的标准

1. 高质量文档的八要素



图5-16 高质量文档的八要素

2. Easy to use

- 面向任务
  内容和描述针对目标用户，确实反映用户的需要。
  引言处应有针对性说明。
  设计以用户的实际工作为主线。
- 准确
  内容一定是验证并确认可行的。
  对时效性强、关键的信息进行突出标识。
  描述正确，没有歧义。
- 完整
  内容覆盖了目标用户实际工作所需的所有主题。
  内容在必要处重复，以减少用户四处查找信息的不方便性。

3. Easy to understand

- 清晰
  内容保持一致。如：用词前后一致；同类主题的描述方式、风格一致。
  段落简洁，一个段落只表达一个逻辑主题，段落之间的衔接自然流畅。
  句子符合标准语序，表达简洁、清晰，无语法错误。
  用词无拼写错误、无土语，没有模糊或口语化的表达。

- 举例恰当
  复杂的、典型的任务要有举例说明。
  考虑实际需要，避免使用抽象、无实际意义的数据。

4. Easy to find

- 结构合理
  目录、标题、章节引言、章节间过渡性文字、图的组织等内容，能够帮助用户迅速定位需要的信息。
  同级别信息的组织和描述，其模块大小及数量要均衡。任务类信息采用 step by step 方式。
  步骤、列表、主题的划分符合 7!2 原则。
  重点内容突出，方便用户快速掌握要点。
- 检索方便
  提供合适的尽可能多的检索入口，满足用户的查找需求。
  标题和段落的首句体现目的和主题。
  序言应给出详细的阅读指导。
- 视觉效果
  使用不同视觉元素（文字、图、表、项目列表等），增加可读性和检索性。
  图、表等视觉元素及其内容设置合理，逻辑清晰。